

PyGame ile Oyun Geliřtirme

Sıfırdan Oyun Programlamaya

Tolga BERBER

1. Baskı | 2025

PyGame ile Oyun Geliřtirme

Sıfırdan Oyun Programlamaya

© 2025 Tolga BERBER

1. Baskı

Bu kitap eğitim amaçlıdır.
İzinsiz çoğaltılması ve dağıtılması yasaktır.

Yazar Hakkında

Dr. Tolga BERBER, Karadeniz Teknik Üniversitesi Fen Fakültesi Bilgisayar Bilimleri Bölümü'nde Dr. Öğretim Üyesi olarak görev yapmaktadır.

Akademik Geçmiş

Lisans eğitimini 2002 yılında Karadeniz Teknik Üniversitesi Bilgisayar Bilimleri Bölümü'nde, yüksek lisans (2007) ve doktora (2013) eğitimini Dokuz Eylül Üniversitesi Bilgisayar Mühendisliği Bölümü'nde tamamlamıştır. 2018 yılından bu yana Dr. Öğretim Üyesi olarak görev yapmakta olup, 2013–2023 yılları arasında bölüm başkan yardımcılığı görevini yürütmüştür.

Araştırma Alanları

- Yapay Zeka ve Makine Öğrenmesi
- Bilgisayarlı Görü
- Metin Madenciliği ve Veri Analizi
- Optimizasyon Algoritmaları
- Yazılım Mühendisliği ve Dağıtık Sistemler

Yayınlar ve Projeler

Akademik kariyerinde 48 bilimsel çalışma yayımlamış, 26 projede görev almış ve 4 patent başvurusunda bulunmuştur. Web of Science h-indeksi 6, Google Scholar h-indeksi 9'dur. Ayrıca 11 yüksek lisans tezi yönetmiştir.

İletişim

- E-posta: tberber@ktu.edu.tr
- Web: avesis.ktu.edu.tr/tberber

İçindekiler

Yazar Hakkında	v
1 Geliştirme Ortamının Kurulumu	1
1.1 Python Nedir ve Neden Python?	1
1.2 Python Kurulumu	2
1.3 IDE Seçimi ve Kurulumu	4
1.4 İlk Python Programı	6
1.5 Ünite Özeti	10
2 Değişkenler ve Veri Tipleri	12
2.1 Değişken Nedir?	12
2.2 Sayısal Veri Tipleri	15
2.3 Metin ve Boolean Veri Tipleri	18
2.4 Tip Dönüşümü	21
2.5 Ünite Özeti	24
3 Operatörler ve İfadeler	26
3.1 Aritmetik Operatörler	26
3.2 Karşılaştırma Operatörleri	29
3.3 Mantıksal Operatörler	32
3.4 Operatör Önceliği	35
3.5 Ünite Özeti	39
4 Temel Giriş/Çıkış	42
4.1 print() Fonksiyonu	42
4.2 input() Fonksiyonu	45
4.3 f-string ile Formatlama	48
4.4 Mini Proje: Sayı Tahmin Oyunu	51
4.5 Ünite Özeti	54
5 Koşullu İfadeler	57
5.1 If İfadesi: Programın Karar Vermesi	57
5.2 Elif ve Else: Çoklu Seçenekler	59
5.3 İç İç Koşullar: Karmaşık Kararlar	63
5.4 Mantıksal Operatörlerle Koşullar	65
5.5 Sonuç: Koşullu İfadeler	68

6	Döngüler	71
6.1	While Döngüsü: Koşul Doğruyken Tekrarla	71
6.2	For Döngüsü: Koleksiyon Üzerinde Gezinme	75
6.3	Range Fonksiyonu: Sayı Dizileri Oluşturma	79
6.4	Break ve Continue: Döngü Kontrolü	82
6.5	Sonuç: Döngüler	85
7	Fonksiyonlar	89
7.1	Fonksiyon Tanımlama: Kodun LEGO Parçaları	89
7.2	Parametreler ve Argümanlar: Fonksiyona Bilgi Aktarma	92
7.3	Return: Değer Döndürme	95
7.4	Değişken Kapsamı: Değişkenler Nerede Geçerli?	98
7.5	Sonuç: Fonksiyonlar	101
8	Veri Yapıları Temelleri	106
8.1	Listeler: Oyuncunun Envanteri	106
8.2	Sözlükler: Anahtar-Değer Eşleştirme	110
8.3	Demetler: Değişmez Koleksiyonlar	114
8.4	Mini Proje: Metin Tabanlı Macera Oyunu	117
8.5	Sonuç: Veri Yapıları ve Bölüm Özeti	123
9	PyGame Kurulumu ve Yapısı	127
9.1	PyGame Nedir?	127
9.2	PyGame Modül Yapısı	130
9.3	PyGame'i Başlatma ve Kapatma	133
9.4	Ünite Özeti	137
10	Oyun Penceresi Oluşturma	139
10.1	Oyun Penceresi Oluşturma	139
10.2	Pencere Başlığı ve İkonu	144
10.3	Tam Ekran ve Ekran Modları	147
10.4	Ünite Özeti	152
11	Ana Oyun Döngüsü	154
11.1	Oyun Döngüsü (Game Loop)	154
11.2	Olay İşleme Temelleri (Event Handling)	159
11.3	FPS Kontrolü ve Zamanlama	163
11.4	Sonuç	168
12	Temel Çizim İşlemleri	171
12.1	Renkler ve RGB Sistemi	171
12.2	Temel Şekil Çizimi	175
12.3	Ünite Özeti	182
13	Event Sistemi	185
13.1	Event Kuyruğu ve Olay İşleme	185
13.2	Olay Türleri ve Özellikleri	189

13.3 Olay Filtreleme ve Özel Olaylar	192
13.4 Sonuç	196
14 Klavye Girdileri	199
14.1 KEYDOWN ve KEYUP Olayları	199
14.2 Sürekli Girdi: <code>key.get_pressed()</code>	204
14.3 Klavye ile Nesne Hareketi	206
14.4 Sonuç	209
15 Fare Girdileri	211
15.1 Fare Olayları	211
15.2 Fare Hareketi ve Takip	216
15.3 Sonuç	220
16 Hareket Mekaniği	223
16.1 Hız ve Yön	223
16.2 Sınır Kontrolü	227
16.3 Mini Proje: Yılan Oyunu	230
16.4 Sonuç	235
17 Görsel Dosyaları	239
17.1 Görsel Dosya Türleri	239
17.2 Görsel Yükleme	242
17.3 Convert ve Alpha Kanalı	244
17.4 Görselleri Ekrana Çizme	246
17.5 Sonuç	250
18 Görsel Manipülasyonu	253
18.1 Boyut Değiştirme	253
18.2 Döndürme	257
18.3 Çevirmeler	261
18.4 Uygulama: Uzay Gemisi Hareket	263
18.5 Sonuç	266
19 Sprite Sınıfı	268
19.1 Sprite Nedir?	268
19.2 Sprite Oluşturma	270
19.3 Update Metodu	274
19.4 Oyuncu Sprite	277
19.5 Sonuç	280
20 Sprite Grupları	282
20.1 Sprite Grubu Nedir?	282
20.2 Grup İşlemleri	284
20.3 Toplu Çizim ve Güncelleme	288
20.4 Mini Proje: Uzay Kaçışı v1	292
20.5 Sonuç	298

21 Rect Sınıfı ve Çarpışma	301
21.1 Rect Sınıfı	301
21.2 Rect Çarpışması	306
21.3 Nokta Çarpışması	309
21.4 Pratik Uygulama: Çarpışma Demosu	311
21.5 Sonuç	313
22 Sprite Çarpışma Fonksiyonları	315
22.1 <code>spritecollide()</code> – Tek Sprite ile Grup Çarpışması	315
22.2 <code>groupcollide()</code> – İki Grup Arası Çarpışma	319
22.3 Çarpışma Yöntemleri: Rect, Circle ve Mask	322
22.4 Mini Proje: Uzay Kaçışı v2 – Düşmanlar ve Mermiler	326
22.5 Sonuç	331
23 Temel Fizik Simülasyonu	333
23.1 Yerçekimi Simülasyonu	333
23.2 Hız ve Momentum	337
23.3 Sürtünme	341
23.4 Basit Bir Fizik Motoru Tasarımı	345
23.5 Sonuç	351
24 Çarpışma Tepkileri	353
24.1 Geri Sekme ve Yansıma Mekaniği	353
24.2 Hasar ve Can Sistemi	357
24.3 Sprite Yok Etme ve Skor Sistemi	362
24.4 Yan Proje: Breakout (Tuğla Kırma)	366
24.5 Sonuç	376
25 Ses Sistemi Temelleri	379
25.1 Mixer Modülü	379
25.2 Ses Dosya Formatları	382
25.3 Sound Sınıfı	385
25.4 Sonuç	391
26 Ses Efektleri	394
26.1 <code>Sound.play()</code> – Ses Çalmanın Ayrıntıları	394
26.2 Ses Kontrolü – Seviye, Durdurma ve Geçişler	397
26.3 Kanal Yönetimi	400
26.4 Uzay Kaçışı – Ses Efektleri Entegrasyonu	404
26.5 Sonuç	408
27 Arka Plan Müziği	410
27.1 Müzik Yükleme ve Çalma	410
27.2 Müzik Kontrolleri	415
27.3 Fade Efektleri ve Müzik Kuyruğu	422
27.4 Sahne Bazlı Müzik Yönetimi	431
27.5 Sonuç	439

28 Ses Tasarımı Prensipleri	441
28.1 Ses Miksajı ve Katmanlar	441
28.2 Ücretsiz Ses Kaynakları ve Lisanslama	444
28.3 Ses Optimizasyonu	446
28.4 Uzay Kaçışı: Tam Ses Entegrasyonu	449
28.5 Sonuç	453
29 Frame Animasyonu	457
29.1 Sprite Sheet Kavramı	457
29.2 Frame Kesme	460
29.3 Animasyon Döngüsü	462
29.4 Pratik Uygulama: 4 Yönlü Yürüyen Karakter	465
29.5 Sonuç	470
30 Animasyon Sınıfı Tasarımı	471
30.1 AnimatedSprite Sınıfı	471
30.2 Animasyon Durumları (State Machine)	475
30.3 Hız Kontrolü ve Gelişmiş Modlar	479
30.4 Uzay Kaçışı v4 – Temel Animasyonlar	482
30.5 Sonuç	485
31 Zamanlayıcılar	486
31.1 Zaman Fonksiyonları	486
31.2 Kullanıcı Olayları ve Zamanlayıcılar	488
31.3 Cooldown Sistemi	491
31.4 Dalga Sistemi ve Spawner	494
31.5 Sonuç	499
32 Görsel Efektler	500
32.1 Parçacık Sistemi	500
32.2 Patlama Efektleri	504
32.3 Görsel Geçişler ve Efektler	508
32.4 Uzay Kaçışı v4 FINAL	514
32.5 Sonuç	518
33 Metin Render Etme	521
33.1 Font Sistemi	521
33.2 Metin Oluşturma ve Ekrana Çizme	523
33.3 Metin Hizalama ve Çok Satırlı Metin	526
33.4 Dinamik Metin Güncelleme	527
33.5 Sonuç	530
34 UI Elemanları	532
34.1 Skor Göstergesi	532
34.2 Sağlık Barı	534
34.3 HUD Tasarımı	537
34.4 UI Uygulaması	539

34.5 Sonuç	541
35 Buton Sınıfı	543
35.1 Temel Buton Tasarımı	543
35.2 Hover Efektı	545
35.3 Click ve Callback Mekanizması	547
35.4 Buton Uygulaması	550
35.5 Sonuç	552
36 Menü Sistemleri	554
36.1 Ana Menü	554
36.2 Pause Menüsü	557
36.3 Game Over Ekranı	560
36.4 Mini Proje: Tam Menü Sistemi	563
36.5 Sonuç	566
37 Tile Sistemi Temelleri	569
37.1 Tile Kavramı ve Grid Tabanlı Dünyalar	569
37.2 Tileset ve subsurface() ile Frame Kesme	572
37.3 Piksel ve Grid Koordinat Dönüşümleri	575
37.4 İlk Haritanı Çiz: 2B Listedeki Ekrana	577
37.5 Sonuç	580
38 Harita Oluşturma	582
38.1 Python 2B Listeleriyle Harita Temsil Etme	582
38.2 CSV Dosyasından Harita Yükleme	585
38.3 Tiled Editor ile Görsel Seviye Tasarımı	588
38.4 pytmx ile .tmx Harita Yükleme	590
38.5 Sonuç	593
39 Kamera ve Scrolling	596
39.1 World-Space ve Screen-Space	596
39.2 Kamera Sınıfı Tasarımı	599
39.3 Oyuncu Takibi ve Lerp	601
39.4 Kamerayı Harita Sınırlarıyla Kısıtla: Clamp	603
39.5 Sonuç	606
40 Seviye Tasarımı	608
40.1 Platformlar ve Solid Tile Çarpışması	608
40.2 Tehlike Tile'ları: Lava ve Diken	612
40.3 Toplanabilirler: Altın ve Anahtar	615
40.4 Mini Proje: Zıplayan Macera – Seviye 1	617
40.5 Sonuç	621
41 Gelişmiş Karakter Hareketi	623
41.1 Platform Hissiyatı: İvme, Sürtünme ve Maksimum Hız	623
41.2 Coyote Time: Hatanın Affedilmesi	626

41.3 Jump Buffer: Erken Basılan Tuşu Kaydetmek	629
41.4 Variable Jump ve Double Jump	632
41.5 Sonuç	635
42 Platform Çarpışmaları	638
42.1 Çarpışma Yönü Tespiti	638
42.2 Tek Yönlü Platform ve Drop-Through	641
42.3 Hareketli Platform ve Delta Transferi	644
42.4 Dağılan ve Zıplatan Platformlar	648
42.5 Sonuç	652
43 Düşman Davranışları	655
43.1 Düşman Temel Sınıfı ve FSM Deseni	655
43.2 Patrol AI ve Uçurum Tespiti	659
43.3 Chase AI: Oyuncuyu Takip Eden Düşman	662
43.4 Ranged AI: Mermi Fırlatan Düşman	666
43.5 Sonuç	670
44 Oyun Sistemleri	672
44.1 Power-Up: Zaman Sınırlı Etkiler	672
44.2 Can Sistemi: Yenileme, Ölçekli Hasar, Knockback	676
44.3 I-Frames ve Flicker Efektleri	680
44.4 Mini Proje: Zıplayan Macera v3 – Mekanik Entegrasyonu	683
44.5 Sonuç	688
45 State Machine Tasarımı	691
45.1 State Machine Nedir? Neden Lazım?	691
45.2 Soyut State Sınıfı ve StateManager	694
45.3 Somut Oyun Durumları: Menu, Play, Pause, GameOver	698
45.4 State Geçişleri: Push, Pop, Replace ve Transition Efektleri	704
45.5 Sonuç	707
46 Dosya İşlemleri	710
46.1 Python'da Dosya Açma, Okuma, Yazma	710
46.2 JSON Formatı ve Diskten Okuma	715
46.3 JSON ile Diske Yazma	719
46.4 pickle ile Karmaşık Nesneleri Serileştirme	723
46.5 Sonuç	728
47 Save/Load Sistemi	730
47.1 Bir Kayıt Formatı Tasarlamak	730
47.2 SaveManager: Kaydetme Tarafı	735
47.3 SaveManager: Yükleme Tarafı	739
47.4 Çoklu Slot Sistemi ve Yükleme Menüsü	744
47.5 Sonuç	750

48 Ayarlar ve Konfigürasyon	752
48.1 Bir Oyunun Ayar İhtiyaçları	752
48.2 Settings Sınıfı: Tek Noktadan Erişim	756
48.3 Kalıcı Ayarlar: Konum, Sürüm, Hata Yönetimi	762
48.4 Oyun İçi Ayar Menüsü: SettingsState	766
48.5 Sonuç	772
49 Performans Optimizasyonu	775
49.1 Profiling: Yavaşlığın Kaynağını Ölçmek	775
49.2 Culling: Görmediğini Çizme, Hesaplama	779
49.3 Object Pooling: Mermiyi Yok Etme, Yeniden Kullan	782
49.4 Uygulama: Uzay Kaçışı'nın Frame Profili	786
49.5 Sonuç	791
50 Gelişmiş Çarpışma	794
50.1 Geniş Faz ve Dar Faz	794
50.2 Spatial Hashing — Hücre Tabanlı Geniş Faz	798
50.3 Quad Tree — Yoğunluğa Uyarlanan Geniş Faz	802
50.4 Karşılaştırma — Hangisi, Ne Zaman?	807
50.5 Sonuç	811
51 Temel AI Teknikleri	813
51.1 Sonlu Durum Makinesi (FSM) Temeli	813
51.2 A* Yol Bulma Algoritması	817
51.3 Steering Behaviors: Yumuşak Hareket	821
51.4 AI Entegrasyon: Zıplayan Macera'da Akıllı Düşman	826
51.5 Sonuç	831
52 Oyun Dağıtımı	833
52.1 PyInstaller ile İlk Paketleme	833
52.2 Paketlenmiş Ortamda Varlık Yönetimi	837
52.3 Dağıtıma Hazır Paket: Simge, Boyut ve Platform	841
52.4 itch.io ile Oyununu Yayınlamak	846
52.5 Sonuç	851

1

Geliştirme Ortamının Kurulumu

“Her büyük yolculuk tek bir adımla başlar. Programlama yolculuğun da bu kurulumla başlıyor!”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Python programlama dilinin kurulumu
- Kod editörü (IDE) seçimi ve kurulumu
- İlk Python programının yazılması ve çalıştırılması
- Terminal ve komut satırı kullanımı

1.1 Python Nedir ve Neden Python?

Bir oyun hayal et: Ekranda bir uzay gemisi var, ok tuşlarıyla hareket ettiriyorsun, düşmanlar saldırıyor ve sen onları lazerlerle yok ediyorsun. Peki bu oyun nasıl çalışıyor? İşte bu kitapta tam da bunu öğreneceksin! Ama önce, oyunumuzu yazacağımız programlama dilini tanıyalım: **Python**.

■ Tanım: Programlama Dili

Programlama dili, insanların bilgisayarlara ne yapması gerektiğini söylemek için kullandığı özel bir dildir. Tıpkı İngilizce veya Türkçe gibi, programlama dillerinin de kendine özgü kuralları ve kelimeleri vardır.

1.1.1 Python’un Hikayesi

Python, 1991 yılında Hollandalı programcı Guido van Rossum tarafından yaratıldı. İsmi ni, sevdiği bir komedi grubu olan "Monty Python"dan aldı. Evet, programcılar da eğlenmeyi sever!

Python’un temel felsefesi şudur: “Okunabilirlik önemlidir.” Bu yüzden Python kodu, diğer programlama dillerine göre çok daha kolay okunur ve anlaşılır.

1.1.2 Neden Python ile Oyun Geliştirme?

Python, oyun geliştirme için mükemmel bir başlangıç noktasıdır. İşte nedenleri:

1. **Kolay Öğrenilir:** Sözdizimi (syntax) İngilizce'ye çok yakındır
2. **Hızlı Prototipleme:** Fikirlerini hızla deneyebilirsiniz
3. **PyGame Kütüphanesi:** Oyun geliştirme için hazır araçlar sunar
4. **Geniş Topluluk:** Sorularına cevap bulman kolaydır

● Bilgi: Biliyor muydun?

Instagram, Spotify, Netflix ve hatta NASA bile Python kullanıyor! Python sadece oyun geliştirme değil, web geliştirme, yapay zeka ve bilimsel hesaplamalar için de kullanılır.

1.1.3 Python vs Diğer Diller

Farklı programlama dillerinde “Merhaba Dünya” yazmak nasıl görünür, karşılaştıralım:

Tablo 1.1: Farklı dillerde “Merhaba Dünya” programı

Dil	Kod
Python	<code>print("Merhaba Dünya")</code>
Java	<code>System.out.println("Merhaba Dünya");</code>
C++	<code>std::cout << "Merhaba Dünya";</code>

Gördüğün gibi Python, en kısa ve en anlaşılır olanı! Bu basitlik, öğrenme sürecini çok daha keyifli hale getiriyor.

● İpucu: Profesyonel İpucu

Programlama öğrenirken sabırlı ol. Hatalar yapmak öğrenmenin bir parçasıdır. Her hata, bir şeyleri daha iyi anlamana yardımcı olur!

1.2 Python Kurulumu

Oyun yapmaya başlamadan önce, araçlarımızı hazırlamamız gerekiyor. Bir ressam nasıl boya ve fırça olmadan resim yapamıyorsa, biz de Python olmadan program yazamayız. Haydi Python'u bilgisayarına kuralım!

1.2.1 İşletim Sistemine Göre Kurulum

Python kurulumu, kullandığın işletim sistemine göre biraz farklılık gösterir. Aşağıda her sistem için adımları bulabilirsin.

Windows için Kurulum

1. **Python.org'u Ziyaret Et:** Tarayıcında python.org/downloads adresine git
2. **İndirme Butonuna Tıkla:** "Download Python 3.x" butonuna tıkla
3. **Kurulumu Başlat:** İndirilen dosyaya çift tıkla
4. **ÖNEMLİ:** "Add Python to PATH" seçeneğini mutlaka işaretle!
5. **Kurulumu Tamamla:** "Install Now" butonuna tıkla ve bekle

⚠ Dikkat: Dikkat!

"Add Python to PATH" seçeneğini işaretlemezsen, Python komutlarını terminalde kullanamazsın. Bu seçeneği kaçırdıysan, Python'u kaldırıp tekrar kurman gerekebilir.

macOS için Kurulum

macOS'ta Python kurulumu için iki yöntem var:

Yöntem 1 - Resmi Yükleyici:

1. python.org/downloads/macos adresine git
2. macOS yükleyicisini indir
3. **.pkg** dosyasını aç ve yönergeleri takip et

Yöntem 2 - Homebrew ile:

```
brew install python
```

Linux için Kurulum

Çoğu Linux dağıtımında Python zaten yüklü gelir. Yüklü değilse veya güncellemek istersen:

Ubuntu/Debian:

```
sudo apt update  
sudo apt install python3
```

Fedora:

```
sudo dnf install python3
```

1.2.2 Kurulumu Doğrulama

Python'un doğru kurulup kurulmadığını kontrol etmek için terminal veya komut satırını aç ve şu komutu yaz:

```
python --version
```

Eğer “Python 3.x.x” gibi bir çıktı görüyorsan, tebrikler! Python başarıyla kurulmuş demektir.

● Bilgi: Windows Kullanıcıları İçin

Windows'ta `python` komutu çalışmazsa, `python3` veya `py` komutlarını dene. Bazen Windows farklı komut isimleri kullanır.

1.2.3 uv Nedir?

Python projelerini yönetmek ve ek kütüphaneler (mesela PyGame!) kurmak için `uv` adında modern bir araç kullanacağız.

■ Tanım: uv

`uv`, Astral tarafından geliştirilen modern bir Python proje ve paket yöneticisidir. Proje oluşturma, bağımlılık yönetimi ve kod çalıştırma işlemlerini hızlı ve kolay bir şekilde yapmamızı sağlar.

uv Kurulumu

macOS / Linux:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

`uv`'nin kurulu olduğunu doğrulamak için:

```
uv --version
```

💡 İpucu: Profesyonel İpucu

İlerleyen bölümlerde PyGame projemizi oluştururken `uv init`, paket eklemek için `uv add pygame` ve programımızı çalıştırmak için `uv run` komutlarını kullanacağız. `uv` sayesinde projelerini kolayca yönetebilirsin!

1.3 IDE Seçimi ve Kurulumu

Python'u kurduk, harika! Ama kod yazmak için bir araç daha lazım: Kod editörü. Teknik olarak Windows'un Notepad'i ile bile kod yazabilirsin, ama bu bir kalemle oyun oynamaya çalışmak gibi olur - işe yarar ama çok zor! İşte bu yüzden **IDE** kullanıyoruz.

■ Tanım: IDE

IDE (Integrated Development Environment - Bütünleşik Geliştirme Ortamı), kod yazmak, çalıştırmak ve hata ayıklamak için gereken tüm araçları bir arada sunan programdır. Kod editörü, terminal ve hata ayıklayıcı gibi araçları tek bir yerde toplar.

1.3.1 Popüler IDE ve Editör Seçenekleri

Tablo 1.2: Popüler Python IDE ve editörleri

Editör	Ücretsiz	Zorluk	Özellik
VS Code	✓	Kolay	Hafif, eklenti desteği
PyCharm	✓*	Orta	Tam IDE, Python'a özel
Thonny	✓	Çok Kolay	Başlangıç için ideal
IDLE	✓	Kolay	Python ile gelir

*Community Edition ücretsiz

1.3.2 VS Code Kurulumu (Önerilen)

Bu kitapta **Visual Studio Code** (VS Code) kullanacağız. Hafif, hızlı ve güçlü bir editördür.

Kurulum Adımları

1. code.visualstudio.com adresine git
2. İşletim sistemine uygun sürümü indir
3. Kurulum dosyasını çalıştır ve yönergeleri takip et
4. Kurulum tamamlandıktan sonra VS Code'u aç

Python Eklentisi Kurulumu

VS Code'u Python için hazır hale getirmek için:

1. VS Code'u aç
2. Sol taraftaki *Extensions* (Eklentiler) simgesine tıkla (veya **Ctrl+Shift+X**)
3. Arama kutusuna "Python" yaz
4. Microsoft tarafından yayınlanan "Python" eklentisini bul ve "Install" butonuna tıkla

● Bilgi: Faydalı Eklentiler

Python eklentisinin yanı sıra şu eklentiler de işine yarayabilir:

- **Pylance:** Daha iyi kod tamamlama
- **Python Indent:** Otomatik girinti düzenleme
- **Code Runner:** Tek tuşla kod çalıştırma

1.3.3 Thonny - Başlangıç Alternatifi

Eğer VS Code sana karmaşık geldiyse, Thonny mükemmel bir alternatiftir. Özellikle programlamaya yeni başlayanlar için tasarlanmıştır.

1. thonny.org adresine git
2. İşletim sistemine uygun sürümü indir ve kur
3. Thonny'yi aç - Python zaten içinde yapılandırılmış olarak gelir!

İpucu: Profesyonel İpucu

Hangi editörü seçersen seç, önemli olan düzenli pratik yapmak. Zamanla editörünün kısayollarını öğrenecek ve çok daha hızlı kod yazacaksın!

1.3.4 IDLE - Python'un Kendi Editörü

Python kurulumu ile birlikte IDLE adında basit bir editör de gelir. Başlangıç için yeterlidir ancak büyük projelerde yetersiz kalabilir.

IDLE'ı açmak için:

- **Windows:** Başlat menüsünde "IDLE" ara
- **macOS:** Applications > Python > IDLE
- **Linux:** Terminal'de `idle` veya `idle3` yaz

1.4 İlk Python Programı

Araçlarımız hazır, artık ilk programımızı yazma zamanı! Her programcının ilk programı geleneksel olarak "Merhaba Dünya" yazdırmaktır. Bu gelenek 1978'den beri sürmektedir. Sen de bu geleneğe katılmaya hazır mısın?

1.4.1 Merhaba Dünya!

VS Code'u (veya tercih ettiğin editörü) aç ve yeni bir dosya oluştur. Dosyayı `merhaba.py` olarak kaydet. `.py` uzantısı, dosyanın bir Python dosyası olduğunu belirtir.

Şimdi şu kodu yaz:

Listing 1.1: İlk Python programımız

```
1 # Bu benim ilk Python programım!
2 print("Merhaba Dünya!")
```

■ Tanım: print() Fonksiyonu

`print()` fonksiyonu, parantez içine yazılan metni ekrana yazdırır. Python'un en temel ve en çok kullanılan fonksiyonlarından biridir.

1.4.2 Programı Çalıştırma

Programı çalıştırmak için birkaç yöntem var:

Yöntem 1 - VS Code İçinden:

1. Sağ üst köşedeki yeşil “Run” (Çalıştır) butonuna tıkla
2. Veya **F5** tuşuna bas

Yöntem 2 - Terminal ile:

```
python merhaba.py
```

Her iki yöntemde de ekranda şunu görmelisin:

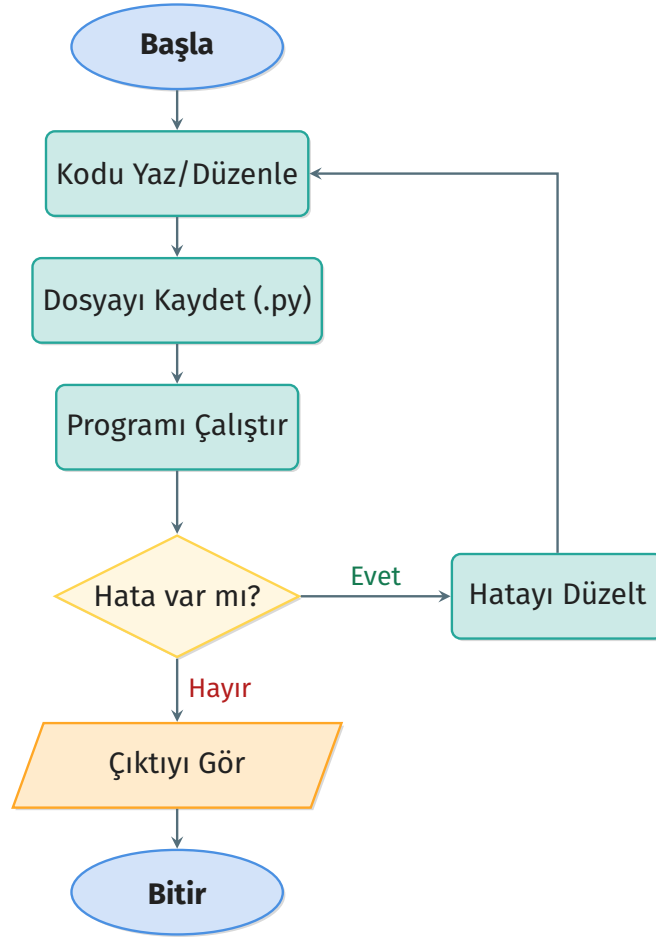
```
>_ Çıktı
```

```
Merhaba Dünya!
```

Tebrikler! Az önce ilk programını yazdın ve çalıştırdın!

1.4.3 Kodu Anlayalım

Bir Python programının nasıl çalıştığını anlamak için aşağıdaki akış şemasına bakalım:



Şekil 1.1: Python programının çalışma akışı

Yazdığımız iki satırı inceleyelim:

1. `# Bu benim ilk Python programım!` - Bu bir **yorum** satırıdır. `#` işaretiyle başlayan satırlar Python tarafından görmezden gelir. Yorumlar, kodun ne yaptığını açıklamak için kullanılır.
2. `print("Merhaba Dünya!")` - Bu satır ekrana metin yazdırır. `print` bir fonksiyon, parantez içindeki `"Merhaba Dünya!"` ise yazdırılacak metindir.

● Bilgi: Tırnak İşaretleri

Python'da metin (string) yazmak için çift tırnak `"..."` veya tek tırnak `'...'` kullanabilirsin. Her ikisi de geçerlidir:

- `print("Merhaba")`
- `print('Merhaba')`

1.4.4 Farklı Şeyler Yazdıralım

`print()` fonksiyonu ile çok daha fazlasını yapabilirsin:

Listing 1.2: Farklı print örnekleri

```
1 # Birden fazla satır yazdırma
2 print("Merhaba!")
3 print("Ben Python öğreniyorum.")
4 print("Oyun yapmak istiyorum!")
5
6 # Sayılar da yazdırılabilir
7 print(42)
8 print(3.14)
9
10 # Matematiksel işlemler
11 print(2 + 2)
12 print(10 * 5)
```

⚠ Dikkat: Dikkat!

Python, büyük/küçük harf duyarlıdır. `print` doğru, `Print` veya `PRINT` yanlıştır ve hata verir!

1.4.5 İlk Hatan

Programlamada hata yapmak çok normaldir. Aşağıdaki kodu dene:

```
1 print("Merhaba)
```

Bu kod çalışmaz çünkü kapanan tırnak işareti eksik. Python sana şöyle bir hata mesajı gösterir:

>_ Çıktı

SyntaxError: EOL while scanning string literal

💡 İpucu: Profesyonel İpucu

Hata mesajlarını okumayı öğren! Python sana neyin yanlış olduğunu söylemeye çalışıyor. “SyntaxError” genellikle yazım hatası demektir. Tırnak işaretlerini, parantezleri ve noktalama işaretlerini kontrol et.

1.4.6 Egzersiz: Kendi Programın

Şimdi sıra sende! Aşağıdaki görevleri yap:

1. Yeni bir dosya oluştur: `tanitim.py`
2. Adını yazdır
3. Yaşını yazdır
4. En sevdiğin oyunun adını yazdır
5. Programı çalıştır ve çıktıyı gör

Örnek çıktı:

```
>_ Çıktı
```

```
Benim adm Ali  
18 yandaym  
En sevdiğim oyun Minecraft
```

1.5 Ünite Özeti

Bu ünite de programlama yolculuğuna ilk adımı attın! İşte öğrendiklerin:

1.5.1 Bu Ünite de Öğrendiklerin

- **Python Nedir:** Python, okunması kolay ve güçlü bir programlama dilidir
- **Python Kurulumu:** python.org'dan indirip kurabilirsin
- **IDE Kullanımı:** VS Code veya Thonny ile kod yazabilirsin
- **İlk Program:** `print()` fonksiyonu ile ekrana metin yazdırabilirsin
- **Yorumlar:** `#` işareti ile açıklama satırları ekleyebilirsin

1.5.2 Anahtar Kavramlar

Python Öğrenmesi kolay, güçlü bir programlama dili

IDE Kod yazmak için kullanılan geliştirme ortamı

uv Python proje ve paket yöneticisi

print() Ekrana çıktı veren fonksiyon

Yorum `#` ile başlayan açıklama satırları

String Tırnak içinde yazılan metin

1.5.3 Önemli Komutlar

Tablo 1.3: Bu ünite deki önemli komutlar

Komut	Açıklama
<code>python --version</code>	Python versiyonunu gösterir
<code>uv --version</code>	uv versiyonunu gösterir
<code>python dosya.py</code>	Python dosyasını çalıştırır
<code>print("metin")</code>	Ekrana metin yazdırır

1.5.4 Sıradaki Adım

Bir sonraki ünite de **değişkenler ve veri tipleri** öğreneceksin. Değişkenler, oyunlarda çok önemlidir - oyuncunun canı, puanı, pozisyonu... hepsi değişkenlerde saklanır!

 İpucu: Pratik Yap!

Programlama öğrenmenin en iyi yolu pratik yapmaktır. Bu üniteyi bitirmeden önce:

1. En az 5 farklı `print()` komutu dene
2. Bilerek hatalı kod yaz ve hata mesajlarını incele
3. Arkadaşlarına veya ailene öğrendiklerini anlat

2

Değişkenler ve Veri Tipleri

*“Değişkenler, programlamanın hafızasıdır.
Onlar olmadan hiçbir şey hatırlayamayız.”*

— ANONİM PROGRAMCI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Değişken kavramı ve kullanımı
- Python’daki temel veri tipleri (int, float, str, bool)
- Değişken isimlendirme kuralları
- Veri tipleri arasında dönüşüm

2.1 Değişken Nedir?

Bir oyun düşün: Ekranda “Can: 3” yazıyor. Düşmana çarpıyorsun ve “Can: 2” oluyor. Sonra bir kalp topluyorsun ve “Can: 3” oluyor. Peki bu “can” değeri nerede saklanıyor ve nasıl değişiyor? İşte cevap: **Değişkenler!**

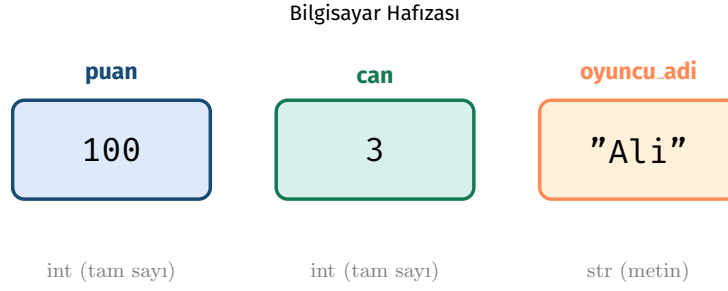
■ Tanım: Değişken

Değişken, bilgisayarın hafızasında bir değer saklamak için kullanılan isimlendirilmiş bir kutudur. Bu kutuya farklı değerler koyabilir ve istediğin zaman değiştirebilirsin.

2.1.1 Gerçek Hayat Analojisi

Değişkenleri etiketli kutular gibi düşün:

- “oyuncu_adi” etiketli kutuya “Ali” yazısını koy
- “puan” etiketli kutuya 100 sayısını koy
- “can” etiketli kutuya 3 sayısını koy



Şekil 2.1: Değişkenler etiketli kutular gibi düşünülebilir

Bu kutulardaki değerleri istediğin zaman alabilir, değiştirebilir veya kullanabilirsin.

2.1.2 Python’da Değişken Oluşturma

Python’da değişken oluşturmak çok basit:

Listing 2.1: Değişken tanımlama örnekleri

```

1 # Değişken oluşturma
2 oyuncu_adi = "Ali"
3 puan = 100
4 can = 3
5
6 # Değişkenleri kullanma
7 print(oyuncu_adi)
8 print(puan)
9 print(can)

```

Bu kod çalıştığında:

>_ Çıktı

```

Ali
100
3

```

● Bilgi: Atama Operatörü

= işareti “eşittir” demek değil, “ata” demektir. `puan = 100` ifadesi “puan değişkenine 100 değerini ata” anlamına gelir.

2.1.3 Değişken İsimlendirme Kuralları

Değişken isimlerini seçerken bazı kurallara uymalısın:

Zorunlu Kurallar:

- Harf veya alt çizgi (`_`) ile başlamalı
- Sadece harf, rakam ve alt çizgi içerebilir

- Boşluk kullanılamaz
- Python'un ayrılmış kelimeleri kullanılamaz (`if`, `for`, `while`, vb.)

Tablo 2.1: Geçerli ve geçersiz değişken isimleri

İsim	Geçerli?	Açıklama
<code>puan</code>	✓	Doğru kullanım
<code>oyuncu_adi</code>	✓	Alt çizgi ile birleştirme
<code>_gizli</code>	✓	Alt çizgi ile başlayabilir
<code>sayi1</code>	✓	Rakam içerebilir
<code>1sayi</code>	×	Rakamla başlayamaz
<code>oyuncu adi</code>	×	Boşluk olamaz
<code>for</code>	×	Ayrılmış kelime

Önerilen Pratikler:

- Anlamlı isimler kullan: `x` yerine `oyuncu_x_pozisyon`
- Küçük harf kullan: `Puan` yerine `puan`
- Alt çizgi ile ayır: `oyuncuadi` yerine `oyuncu_adi`

⚠ Dikkat: Dikkat!

Python büyük/küçük harf duyarlıdır. `puan`, `Puan` ve `PUAN` üç farklı değişkendir!

2.1.4 Değişken Değerini Değiştirme

Değişkenlerin en güzel özelliği: değerleri değişebilir!

Listing 2.2: Değişken değerini güncelleme

```

1 # Başlangıç değeri
2 can = 3
3 print("Başlangıç:", can)
4
5 # Hasar aldık
6 can = can - 1
7 print("Hasar sonrası:", can)
8
9 # Kalp topladık
10 can = can + 1
11 print("İyileşme sonrası:", can)

```

>_ Çıktı

```

Balangç: 3
Hasar sonrası: 2
yileme sonrası: 3

```

💡 İpucu: Profesyonel İpucu

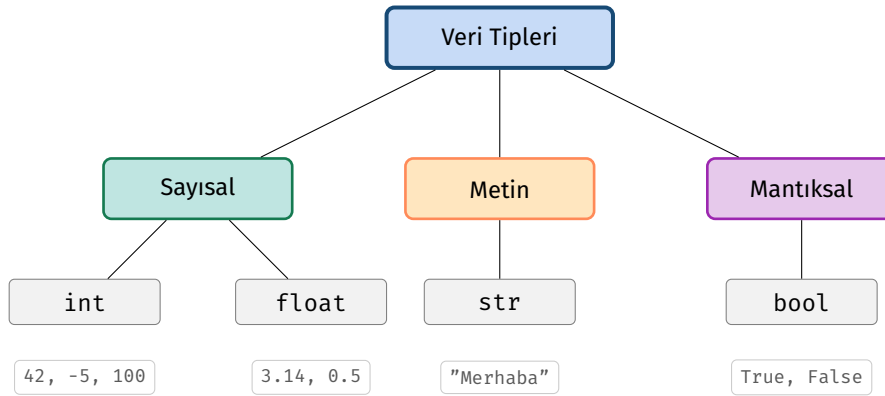
`can = can - 1` yerine kısayol olarak `can -= 1` yazabilirsin. Aynı şekilde `puan += 10` da `puan = puan + 10` demektir.

2.2 Sayısal Veri Tipleri

Oyunlarda sayılar her yerde: puan, can, hız, pozisyon, aç... Python'da iki temel sayısal veri tipi var: **int** (tam sayı) ve **float** (ondalıklı sayı).

■ Tanım: Veri Tipi

Veri tipi, bir değişkenin ne tür veri sakladığını belirtir. Sayı mı, metin mi, doğru/yanlış mı? Python bunu otomatik olarak anlar, ama sen de bilmelisin!



Şekil 2.2: Python'da temel veri tipleri

2.2.1 int - Tam Sayılar

int (integer), tam sayıları temsil eder. Kesirli kısmı yoktur.

Listing 2.3: int veri tipi örnekleri

```

1 # Oyun değişkenleri
2 puan = 0
3 can = 3
4 dusmus_dusmanlar = 42
5 seviye = 1
6
7 # Negatif sayılar da int
8 sicaklik = -10
9 derinlik = -100
10
11 # Büyük sayılar
12 yuksek_skor = 1000000
  
```

```

13
14 # Tip kontrolü
15 print(type(puan)) # <class 'int'>

```

● Bilgi: type() Fonksiyonu

`type()` fonksiyonu, bir değişkenin veya değerin tipini gösterir. Hata ayıklarken çok işine yarar!

2.2.2 float - Ondalıklı Sayılar

`float` (floating-point), ondalıklı sayıları temsil eder.

Listing 2.4: float veri tipi örnekleri

```

1 # Oyun değişkenleri
2 hiz = 5.5
3 x_pozisyon = 100.0
4 y_pozisyon = 250.75
5 aci = 45.0
6
7 # Pi sayısı
8 pi = 3.14159
9
10 # Çok küçük sayılar
11 zaman_dilimi = 0.016 # 60 FPS için saniye başına geçen süre
12
13 # Tip kontrolü
14 print(type(hiz)) # <class 'float'>

```

2.2.3 int ve float Farkı

Tablo 2.2: int ve float karşılaştırması

Özellik	int	float
Kesirli kısım	Yok	Var
Bellek kullanımı	Az	Fazla
Örnek	5, -3, 100	5.0, -3.14, 0.5
Oyun kullanımı	Puan, can, sayaç	Pozisyon, hız, açı

2.2.4 Sayılarla Matematik

Python'u güçlü bir hesap makinesi olarak kullanabilirsin:

Listing 2.5: Matematiksel işlemler

```
1 # Temel işlemler
2 toplama = 10 + 5      # 15
3 cikarma = 10 - 3      # 7
4 carpma = 4 * 5        # 20
5 bolme = 20 / 4         # 5.0 (her zaman float döner)
6
7 # Özel işlemler
8 tam_bolme = 17 // 5    # 3 (tam kısım)
9 kalan = 17 % 5         # 2 (mod - kalan)
10 us = 2 ** 8            # 256 (2 üzeri 8)
11
12 # Oyun örneği
13 puan = 0
14 bonus = 50
15 carpan = 2
16 puan = (puan + bonus) * carpan
17 print(puan) # 100
```

⚠ Dikkat: Sıfıra Bölme

Bir sayıyı sıfıra bölmeye çalışırsan Python hata verir: `ZeroDivisionError`. Oyunlarda bölme yaparken bölenin sıfır olmadığından emin ol!

2.2.5 Oyun Örneği: Hasar Hesaplama

Listing 2.6: Basit hasar hesaplama sistemi

```

1 # Oyuncu istatistikleri
2 oyuncu_can = 100
3 oyuncu_zirh = 20
4
5 # Düşman saldırısı
6 dusmas_hasar = 30
7
8 # Zırh hasarı azaltır
9 gercek_hasar = dusmas_hasar - oyuncu_zirh
10 if gercek_hasar < 0:
11     gercek_hasar = 0
12
13 # Can güncelle
14 oyuncu_can = oyuncu_can - gercek_hasar
15
16 print("Kalan can:", oyuncu_can) # 90

```

💡 İpucu: Profesyonel İpucu

Oyunlarda float kullanırken dikkatli ol! `0.1 + 0.2` tam olarak `0.3` değildir (0.30000000000000004 gibi bir değer olur). Tam sayı kullanabildiğin yerlerde int tercih et.

2.3 Metin ve Boolean Veri Tipleri

Sayılar önemli, ama oyunlarda sadece sayılar yok! Oyuncu isimleri, diyaloglar, menü metinleri... Bunların hepsi **string** (metin) veri tipinde. Ayrıca “Oyun bitti mi?” veya “Oyuncu zıplıyor mu?” gibi evet/hayır sorularının cevapları için **boolean** kullanıyoruz.

2.3.1 str - Metin (String)

str (string), metin verilerini saklar. Tırnak işaretleri içinde yazılır.

Listing 2.7: String veri tipi örnekleri

```

1 # Tek tırnak
2 oyuncu_adi = 'Ali'
3
4 # Çift tırnak
5 oyun_adi = "Uzay Kaçış1"
6
7 # Uzun metin için üç tırnak
8 hikaye = """Uzak bir galakside,
9 cesur bir pilot
10 maceraya atılıyor..."""
11
12 # Boş string

```



```
13 mesaj = ""
14
15 print(type(oyuncu_adi)) # <class 'str'>
```

String Birleştirme

Stringleri `+` operatörü ile birleştirebilirsin:

Listing 2.8: String birleştirme

```
1 ad = "Ali"
2 soyad = "Yılmaz"
3
4 # Birleştirme
5 tam_ad = ad + " " + soyad
6 print(tam_ad) # Ali Yılmaz
7
8 # Oyun mesajı oluşturma
9 mesaj = "Hoş geldin, " + ad + "!"
10 print(mesaj) # Hoş geldin, Ali!
```

String Çoğaltma

Stringleri `*` operatörü ile çoğaltabilirsin:

```
1 çizgi = "-" * 20
2 print(chizgi) # -----
3
4 puan_gosterge = "*" * 5
5 print("Yıldız:", puan_gosterge) # Yıldız: *****
```

⚠ Dikkat: String ve Sayı

String ile sayıyı doğrudan birleştiremezsin! `"Puan: " + 100` hata verir.
Önce sayıyı stringe çevirmelisin: `"Puan: " + str(100)`

2.3.2 bool - Boolean (Mantıksal)

`bool` (boolean), sadece iki değer alabilir: `True` (doğru) veya `False` (yanlış).

Listing 2.9: Boolean veri tipi örnekleri

```
1 # Oyun durumları
2 oyun_aktif = True
3 oyuncu_oldu = False
4 ses_acik = True
5 muzik_acik = False
6
7 # Karşılaştırma sonuçları
8 puan = 100
9 yuksek_puan = 150
```

```

10
11 yeni_rekor = puan > yuksek_puan
12 print(yeni_rekor) # False
13
14 # Tip kontrolü
15 print(type(oyun_aktif)) # <class 'bool'>

```

■ Tanım: Boolean

Boolean, matematikçi George Boole'un adından gelir. Programlamada evet/hayır, doğru/yanlış, açık/kapalı gibi iki durumlu değerleri temsil eder.

Karşılaştırma Operatörleri

Karşılaştırma operatörleri boolean değer döndürür:

Tablo 2.3: Karşılaştırma operatörleri

Operatör	Anlam	Örnek
==	Eşit mi?	5 == 5 → True
!=	Eşit değil mi?	5 != 3 → True
>	Büyük mü?	5 > 3 → True
<	Küçük mü?	5 < 3 → False
>=	Büyük veya eşit mi?	5 >= 5 → True
<=	Küçük veya eşit mi?	5 <= 3 → False

Listing 2.10: Oyun mantığı örneği

```

1 # Oyuncu durumu
2 can = 0
3 puan = 1500
4 hedef_puan = 1000
5
6 # Kontroller
7 oyuncu_oldu = can <= 0
8 print("Oyuncu öldü mü?", oyuncu_oldu) # True
9
10 seviye_gecti = puan >= hedef_puan
11 print("Seviye geçti mi?", seviye_gecti) # True

```

● Bilgi: == ve = Farkı

= atama operatörü (değer ata), == karşılaştırma operatörüdür (eşit mi?). Bu ikisini karıştırmak çok yaygın bir hatadır!

2.3.3 Mantıksal Operatörler

Boolean değerleri **and**, **or** ve **not** ile birleştirebilirsin:

Listing 2.11: Mantıksal operatörler

```

1 # Oyun kontrolleri
2 can = 3
3 mermi = 10
4 boss_yenildi = False
5
6 # and - her ikisi de doğru olmalı
7 saldirabilir = can > 0 and mermi > 0
8 print("Saldırabilir mi?", saldirabilir) # True
9
10 # or - en az biri doğru olmalı
11 oyun_bitti = can <= 0 or boss_yenildi
12 print("Oyun bitti mi?", oyun_bitti) # False
13
14 # not - tersine çevirir
15 hayatta = not (can <= 0)
16 print("Hayatta mı?", hayatta) # True

```

💡 İpucu: Profesyonel İpucu

Oyun geliştirmede boolean değişkenler çok önemlidir. `is_` veya `has_` öneki kullanmak kodu daha okunabilir yapar: `is_jumping`, `has_key`, `is_game_over`

2.4 Tip Dönüşümü

Bazen bir veri tipini başka bir tipe çevirmen gerekir. Kullanıcıdan aldığın sayı aslında string'dir, ama hesaplama yapmak için int'e çevirmelisin. Ya da puanı ekranda göstermek için sayıyı string'e çevirmelisin. Bu işleme **tip dönüşümü** (type casting) denir.

2.4.1 Dönüşüm Fonksiyonları

Python'da üç temel dönüşüm fonksiyonu var:

Tablo 2.4: Tip dönüşüm fonksiyonları

Fonksiyon	Dönüştürür	Örnek
<code>int()</code>	Tam sayıya	<code>int("42") → 42</code>
<code>float()</code>	Ondalıkli sayıya	<code>float("3.14") → 3.14</code>
<code>str()</code>	Metne	<code>str(100) → "100"</code>
<code>bool()</code>	Boolean'a	<code>bool(1) → True</code>

2.4.2 String'den Sayıya

Listing 2.12: String'den sayıya dönüşüm

```
1 # String'den int'e
2 puan_str = "100"
3 puan = int(puan_str)
4 print(puan + 50) # 150
5
6 # String'den float'a
7 hiz_str = "5.5"
8 hiz = float(hiz_str)
9 print(hiz * 2) # 11.0
10
11 # Dikkat: Geçersiz dönüşüm hata verir!
12 # int("merhaba") # ValueError!
13 # int("3.14") # ValueError! (önce float'a çevir)
```

⚠ Dikkat: Geçersiz Dönüşüm

`int("merhaba")` veya `int("3.14")` hata verir! Dönüştürülecek string geçerli bir format olmalı. Ondalıklı bir string'i int'e çevirmek için önce float'a, sonra int'e çevir: `int(float("3.14"))`

2.4.3 Sayıdan String'e

Sayıları ekranda göstermek veya birleştirmek için string'e çevirmen gerekir:

Listing 2.13: Sayıdan string'e dönüşüm

```
1 puan = 1500
2 can = 3
3
4 # String birleştirme için dönüşüm
5 mesaj = "Puan: " + str(puan) + " - Can: " + str(can)
6 print(mesaj) # Puan: 1500 - Can: 3
7
8 # Daha iyi yöntem: f-string (bir sonraki üniteye detaylı)
9 mesaj = f"Puan: {puan} - Can: {can}"
10 print(mesaj) # Puan: 1500 - Can: 3
```

2.4.4 Float ve Int Dönüşümleri

Listing 2.14: Float ve int dönüşümleri

```
1 # Float'tan int'e (kesirli kısım atılır, yuvarlamaz!)
2 hiz = 5.9
3 hiz_tam = int(hiz)
4 print(hiz_tam) # 5 (5.9 değil, 6 değil!)
5
6 # Int'ten float'a
7 puan = 100
8 puan_float = float(puan)
9 print(puan_float) # 100.0
10
11 # Yuvarlama için round() kullan
12 hiz = 5.9
13 hiz_yuvarla = round(hiz)
14 print(hiz_yuvarla) # 6
```

● Bilgi: round() Fonksiyonu

`round()` fonksiyonu sayıları yuvarlar. `round(5.9) → 6`, `round(5.4) → 5`. İkinci parametre ile ondalık basamak sayısını belirleyebilirsiniz: `round(3.14159, 2) → 3.14`

2.4.5 Boolean Dönüşümleri

Her veri tipi boolean'a dönüştürülebilir:

Listing 2.15: Boolean dönüşümleri

```
1 # Sayılar: 0 = False, diğer = True
2 print(bool(0)) # False
3 print(bool(1)) # True
4 print(bool(-5)) # True
5 print(bool(100)) # True
6
7 # Stringler: Boş = False, dolu = True
8 print(bool("")) # False
9 print(bool("Merhaba")) # True
10
11 # Oyun örneği
12 mermi = 0
13 atesle = bool(mermi) # False - mermi yok, ateşleyemez!
```

2.4.6 Pratik Örnek: Oyuncu Girişi

Listing 2.16: Kullanıcı girişi ve dönüşüm

```

1 # input() her zaman string döndürür!
2 yas_str = input("Yaşını gir: ")
3 print(type(yas_str)) # <class 'str'>
4
5 # Sayısal işlem için dönüştür
6 yas = int(yas_str)
7 dogum_yili = 2024 - yas
8
9 print(f"Doğum yılın: {dogum_yili}")

```

💡 İpucu: Profesyonel İpucu

Kullanıcıdan aldığın her girdiyi doğrula! Sayı bekliyorsan ve kullanıcı metin girerse program çöker. İleriki bölümlerde `try/except` ile hata yakalamayı öğreneceksin.

2.5 Ünite Özeti

Bu ünite de Python'un temel yapı taşlarından birini öğrendin: değişkenler ve veri tipleri. Artık oyun verilerini saklayabilir ve yönetebilirsin!

2.5.1 Bu Ünite de Öğrendiklerin

- **Değişkenler:** Verileri saklamak için isimlendirilmiş kutular
- **int:** Tam sayılar (puan, can, seviye)
- **float:** Ondalıklı sayılar (pozisyon, hız, açı)
- **str:** Metin verileri (oyuncu adı, diyalog)
- **bool:** Mantıksal değerler (True/False)
- **Tip Dönüşümü:** `int()`, `float()`, `str()`, `bool()`

2.5.2 Veri Tipleri Özet Tablosu

Tablo 2.5: Python veri tipleri özeti

Tip	Örnek	Oyun Kullanımı	Kontrol
int	42, -5	Puan, can, sayaç	<code>type(x)</code>
float	3.14, -0.5	Pozisyon, hız	<code>type(x)</code>
str	"Ali", 'Merhaba'	İsim, diyalog	<code>type(x)</code>
bool	True, False	Durum kontrolü	<code>type(x)</code>

2.5.3 Önemli Operatörler

Tablo 2.6: Bu ünitedeki operatörler

Operatör	Anlam	Örnek
=	Atama	<code>x = 5</code>
+, -, *, /	Aritmetik	<code>10 + 5</code>
//	Tam bölme	<code>17 // 5 → 3</code>
%	Mod (kalan)	<code>17 % 5 → 2</code>
**	Üs alma	<code>2 ** 3 → 8</code>
==, !=, <, >	Karşılaştırma	<code>5 > 3 → True</code>
and, or, not	Mantıksal	<code>True and False</code>

2.5.4 Sıradaki Adım

Bir sonraki ünite de **operatörler ve ifadeler** konusunu daha detaylı öğreneceksin. Matematiksel işlemler, karşılaştırmalar ve mantıksal operatörleri birleştirerek karmaşık ifadeler oluşturabileceksin.

2.5.5 Mini Görev: Oyuncu Profili

Öğrendiklerini pekiştirmek için şu görevi yap:

```

1 # oyuncu_profili.py
2 # Bir oyuncu profili oluştur
3
4 oyuncu_adi = "KahramanOyuncu"
5 seviye = 15
6 deneyim = 2500.5
7 can = 100
8 altın = 750
9 premium_uye = True
10
11 # Bu bilgileri ekrana yazdır
12 # Her bilgi ayrı satırda olsun

```

💡 İpucu: Pratik Yap!

Bu görevi tamamladıktan sonra:

1. Değerleri değiştir ve tekrar çalıştır
2. Yeni değişkenler ekle (saldırı gücü, savunma)
3. `type()` ile her değişkenin tipini kontrol et

3

Operatörler ve İfadeler

“Programlama, matematikten çok daha fazlasıdır. Ama matematiği iyi bilmek seni güçlü kılar!”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Aritmetik operatörler ve matematiksel işlemler
- Karşılaştırma operatörleri ve koşullar
- Mantıksal operatörler (and, or, not)
- Operatör önceliği ve parantez kullanımı

3.1 Aritmetik Operatörler

Bir oyunda puanını hesaplarken, düşmana verdiğin hasarı belirlerken veya karakterinin hızını ayarlarken sürekli matematik yaparsın. Python’u güçlü bir hesap makinesi olarak kullanabilirsin. Haydi aritmetik operatörleri öğrenelim!

3.1.1 Temel Aritmetik Operatörler

Tablo 3.1: Python aritmetik operatörleri

Operatör	İsim	Örnek	Sonuç
+	Toplama	10 + 3	13
-	Çıkarma	10 - 3	7
*	Çarpma	10 * 3	30
/	Bölme	10 / 3	3.333...
//	Tam Bölme	10 // 3	3
%	Mod (Kalan)	10 % 3	1
**	Üs Alma	2 ** 3	8

3.1.2 Toplama ve Çıkarma

Listing 3.1: Toplama ve çıkarma işlemleri

```
1 # Puan hesaplama
2 baz_puan = 100
3 bonus = 50
4 ceza = 20
5
6 toplam_puan = baz_puan + bonus - ceza
7 print(f"Toplam puan: {toplam_puan}") # 130
8
9 # Can yönetimi
10 can = 100
11 hasar = 30
12 iyilesme = 15
13
14 can = can - hasar + iyilesme
15 print(f"Kalan can: {can}") # 85
```

3.1.3 Çarpma ve Bölme

Listing 3.2: Çarpma ve bölme işlemleri

```
1 # Hasar hesaplama
2 baz_hasar = 25
3 kritik_carpan = 2
4 zirh_azaltma = 0.3 # %30 azaltma
5
6 hasar = baz_hasar * kritik_carpan * (1 - zirh_azaltma)
7 print(f"Final hasar: {hasar}") # 35.0
8
9 # Hız hesaplama
10 mesafe = 100
11 sure = 4
12 hiz = mesafe / sure
13 print(f"Hız: {hiz}") # 25.0
```

⚠ Dikkat: Bölme Sonucu

Normal bölme (/) her zaman float döndürür, tam sayıları bölsen bile! `10 / 2` sonucu `5.0`'dır, `5` değil.

3.1.4 Tam Bölme ve Mod

Tam bölme (//) ve mod (%) operatörleri oyunlarda sıkça kullanılır:

Listing 3.3: Tam bölme ve mod kullanımı

```

1 # Dakika ve saniye hesaplama
2 toplam_saniye = 185
3
4 dakika = toplam_saniye // 60 # Tam dakika
5 saniye = toplam_saniye % 60 # Kalan saniye
6
7 print(f"Süre: {dakika}:{saniye}") # 3:5
8
9 # Satır ve sütun hesaplama (grid sistemleri için)
10 indeks = 17
11 sutun_sayisi = 5
12
13 satir = indeks // sutun_sayisi
14 sutun = indeks % sutun_sayisi
15
16 print(f"Pozisyon: Satır {satir}, Sütun {sutun}") # Satır 3, Sütun 2

```

● Bilgi: Mod Operatörü

Mod (%) operatörü bölümden kalanı verir. Çok kullanışlıdır:

- Çift/tek kontrolü: `x % 2 == 0` ise çift
- Döngüsel değerler: `aci % 360` açığı 0-359 arasında tutar
- Grid pozisyonu: satır ve sütun hesaplama

3.1.5 Üs Alma

Listing 3.4: Üs alma örnekleri

```

1 # Kare ve küp
2 kenar = 5
3 alan = kenar ** 2
4 hacim = kenar ** 3
5
6 print(f"Alan: {alan}") # 25
7 print(f"Hacim: {hacim}") # 125
8
9 # Üstel büyüme (level sistemi)
10 baz_xp = 100
11 seviye = 3
12 gereken_xp = baz_xp * (2 ** (seviye - 1))
13
14 print(f"Seviye {seviye} için gereken XP: {gereken_xp}") # 400

```

3.1.6 Bileşik Atama Operatörleri

Değişkeni güncellerken kısayol kullanabilirsin:

Tablo 3.2: Bileşik atama operatörleri

Kısayol	Uzun Hali	Örnek
x += 5	x = x + 5	Puan arttırma
x -= 3	x = x - 3	Can azaltma
x *= 2	x = x * 2	Çarpan uygulama
x /= 4	x = x / 4	Bölme
x //= 2	x = x // 2	Yarıya indirme
x %= 10	x = x % 10	Mod alma

Listing 3.5: Bileşik atama örnekleri

```

1 puan = 100
2
3 # Bonus kazandık
4 puan += 50
5 print(puan) # 150
6
7 # Çarpan aktif
8 puan *= 2
9 print(puan) # 300
10
11 # Ceza aldık
12 puan -= 75
13 print(puan) # 225

```

💡 İpucu: Profesyonel İpucu

Bileşik atama operatörleri kodu daha kısa ve okunabilir yapar. Oyun geliştirilmede sürekli `puan += 10` veya `can -= hasar` gibi kodlar yazarsın.

3.2 Karşılaştırma Operatörleri

Oyunlarda sürekli kararlar alman gerekir: “Oyuncu öldü mü?”, “Yeni rekor mu?”, “Seviye geçildi mi?” Bu soruların cevapları için değerleri karşılaştırırız. Karşılaştırma operatörleri her zaman `True` veya `False` döndürür.

3.2.1 Karşılaştırma Operatörleri Tablosu

Tablo 3.3: Python karşılaştırma operatörleri

Operatör	Anlam	Örnek	Sonuç
==	Eşit mi?	5 == 5	True
!=	Eşit değil mi?	5 != 3	True
>	Büyük mü?	5 > 3	True
<	Küçük mü?	5 < 3	False
>=	Büyük veya eşit?	5 >= 5	True
<=	Küçük veya eşit?	5 <= 3	False

3.2.2 Eşitlik Kontrolü

Listing 3.6: Eşitlik karşılaştırmaları

```

1 # Sayı karşılaştırma
2 puan = 100
3 hedef = 100
4
5 basarili = puan == hedef
6 print(f"Başarılı mı? {basarili}") # True
7
8 # String karşılaştırma
9 sifre = "gizli123"
10 girilen = "gizli123"
11
12 dogru_sifre = sifre == girilen
13 print(f"Şifre doğru mu? {dogru_sifre}") # True
14
15 # Eşit değil kontrolü
16 oyuncu_renk = "mavi"
17 dusmas_renk = "kirmizi"
18
19 farkli_takim = oyuncu_renk != dusmas_renk
20 print(f"Farklı takım mı? {farkli_takim}") # True

```

⚠ Dikkat: == ve = Karıştırma!

= atama yapar, == karşılaştırma yapar. Bu çok yaygın bir hatadır:

- `x = 5` → x'e 5 ata
- `x == 5` → x, 5'e eşit mi?

3.2.3 Büyüklük Karşılaştırmaları

Listing 3.7: Büyüklük karşılaştırmaları

```
1 # Oyuncu durumu kontrolü
2 can = 30
3 max_can = 100
4
5 dusuk_can = can < max_can * 0.25 # %25'in altında mı?
6 print(f"Düşük can mı? {dusuk_can}") # True
7
8 # Yüksek skor kontrolü
9 puan = 1500
10 yuksek_skor = 1200
11
12 yeni_rekor = puan > yuksek_skor
13 print(f"Yeni rekor mu? {yeni_rekor}") # True
14
15 # Seviye kontrolü
16 oyuncu_seviye = 10
17 gereken_seviye = 10
18
19 girebilir = oyuncu_seviye >= gereken_seviye
20 print(f"Bölgeye girebilir mi? {girebilir}") # True
```

3.2.4 Zincirleme Karşılaştırma

Python'da karşılaştırmaları zincirleyebilirsiniz:

Listing 3.8: Zincirleme karşılaştırma

```
1 # Aralık kontrolü (tek satırda)
2 x = 50
3
4 # x, 0 ile 100 arasında mı?
5 aralikta = 0 <= x <= 100
6 print(f"Aralıkta mı? {aralikta}") # True
7
8 # Oyuncu ekran sınırları içinde mi?
9 oyuncu_x = 350
10 ekran_genislik = 800
11
12 sinirda = 0 <= oyuncu_x <= ekran_genislik
13 print(f"Ekranı mı? {sinirda}") # True
14
15 # Yaş kontrolü
16 yas = 15
17 genc_oyuncu = 13 <= yas <= 19
18 print(f"Genç oyuncu mu? {genc_oyuncu}") # True
```

● Bilgi: Zincirleme Avantajı

`0 <= x <= 100` yazmak, `x >= 0 and x <= 100` yazmaktan daha kısa ve okunaklıdır. Python bu özelliği destekleyen nadir dillerden biridir!

3.2.5 Oyun Örneği: Durum Kontrolü

Listing 3.9: Oyun durum kontrolü örneği

```

1 # Oyuncu değişkenleri
2 can = 0
3 puan = 1500
4 hedef_puan = 1000
5 boss_yenildi = True
6
7 # Çeşitli kontroller
8 oyuncu_oldu = can <= 0
9 print(f"Oyuncu öldü mü? {oyuncu_oldu}") # True
10
11 hedef_tamamlandi = puan >= hedef_puan
12 print(f"Hedef tamamlandı mı? {hedef_tamamlandi}") # True
13
14 seviye_bitti = hedef_tamamlandi and boss_yenildi
15 print(f"Seviye bitti mi? {seviye_bitti}") # True

```

💡 İpucu: Profesyonel İpucu

Karşılaştırma sonuçlarını doğrudan boolean değişkenlerde saklamak kodu daha okunabilir yapar. `if can <= 0:` yerine `oyuncu_oldu = can <= 0` sonra `if oyuncu_oldu:` yazabilirsin.

3.3 Mantıksal Operatörler

Bazen tek bir koşul yetmez. “Oyuncu hem canlı HEM DE mermisi var mı?” veya “Oyun bitti mi VEYA oyuncu çıktı mı?” gibi sorular sormak istersin. İşte mantıksal operatörler tam da bunun için var!

3.3.1 Üç Mantıksal Operatör

Tablo 3.4: Python mantıksal operatörleri

Operatör	Anlam	Sonuç True olur eğer...
<code>and</code>	VE	Her iki koşul da True ise
<code>or</code>	VEYA	En az bir koşul True ise
<code>not</code>	DEĞİL	Koşul False ise

3.3.2 and Operatörü

and operatörü, **tüm** koşulların doğru olmasını gerektirir:

Listing 3.10: and operatörü kullanımı

```

1 # Saldırı kontrolü
2 can = 50
3 mermi = 10
4
5 # Hem can > 0 HEM DE mermi > 0 olmalı
6 saldirabilir = can > 0 and mermi > 0
7 print(f"Saldırabilir mi? {saldirabilir}") # True
8
9 # Can 0 olursa?
10 can = 0
11 saldirabilir = can > 0 and mermi > 0
12 print(f"Saldırabilir mi? {saldirabilir}") # False
13
14 # Çoklu koşul
15 level = 5
16 altın = 500
17 item_fiyat = 300
18
19 satin_alabilir = level >= 5 and altın >= item_fiyat
20 print(f"Satin alabilir mi? {satin_alabilir}") # True

```

● Bilgi: and Doğruluk Tablosu

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

3.3.3 or Operatörü

or operatörü, **en az bir** koşulun doğru olmasını gerektirir:

Listing 3.11: or operatörü kullanımı

```

1 # Oyun bitiş kontrolü
2 can = 0
3 boss_yenildi = False
4
5 # Can <= 0 VEYA boss yenildi ise oyun biter
6 oyun_bitti = can <= 0 or boss_yenildi
7 print(f"Oyun bitti mi? {oyun_bitti}") # True
8
9 # Güç artışı kontrolü
10 hiz_gucu = False

```

```

11 kalkan_gucu = True
12 hasar_gucu = False
13
14 guc_aktif = hiz_gucu or kalkan_gucu or hasar_gucu
15 print(f"Güç aktif mi? {guc_aktif}") # True

```

● Bilgi: or Doğruluk Tablosu

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

3.3.4 not Operatörü

not operatörü, boolean değerini tersine çevirir:

Listing 3.12: not operatörü kullanımı

```

1 # Basit tersine çevirme
2 oyun_duraklatildi = False
3
4 oyun_devam_ediyor = not oyun_duraklatildi
5 print(f"Oyun devam ediyor mu? {oyun_devam_ediyor}") # True
6
7 # Hayatta kontrolü
8 can = 50
9 oldu = can <= 0
10 hayatta = not oldu
11 print(f"Hayatta mı? {hayatta}") # True
12
13 # Envanter dolu kontrolü
14 envanter_bos = False
15 envanter_dolu = not envanter_bos
16 print(f"Envanter dolu mu? {envanter_dolu}") # True

```

3.3.5 Operatörleri Birleştirme

Mantıksal operatörleri karşılaştırma operatörleriyle birleştirebilirsin:

Listing 3.13: Karmaşık koşullar

```

1 # RPG karakteri kontrolleri
2 can = 80
3 mana = 30
4 seviye = 10
5 buyulu_saldiri_mana = 25
6
7 # Büyü yapabilir mi?

```



```

8 # Can > 0 VE mana yeterli VE seviye >= 5
9 buyu_yapabilir = can > 0 and mana >= buyulu_saldiri_mana and
   seviye >= 5
10 print(f"Büyü yapabilir mi? {buyu_yapabilir}") # True
11
12 # Ölüm veya kaçış kontrolü
13 can = 10
14 dusmas_sayisi = 5
15
16 tehlikede = can < 20 or dusmas_sayisi > 3
17 kac = tehlikede and not (can > 50)
18 print(f"Kaçmalı mı? {kac}") # True

```

⚠ Dikkat: Parantez Kullan

Karmaşık koşullarda parantez kullanmak hem hatayı önler hem de kodu okunabilir yapar. `a and b or c` yerine `(a and b) or c` yaz.

3.3.6 Kısa Devre Değerlendirme

Python, gereksiz hesaplama yapmaz:

Listing 3.14: Kısa devre değerlendirme

```

1 # and: İlk False'da durur
2 # or: İlk True'da durur
3
4 x = 0
5
6 # x != 0 False olduğu için ikinci kısım kontrol edilmez
7 # Bu sayede sıfıra bölme hatası olmaz!
8 sonuc = x != 0 and 100 / x > 5
9 print(sonuc) # False (hata yok!)
10
11 # Varsayılan değer atama
12 kullanıcı_adi = ""
13 goster = kullanıcı_adi or "Misafir"
14 print(goster) # "Misafir"

```

💡 İpucu: Profesyonel İpucu

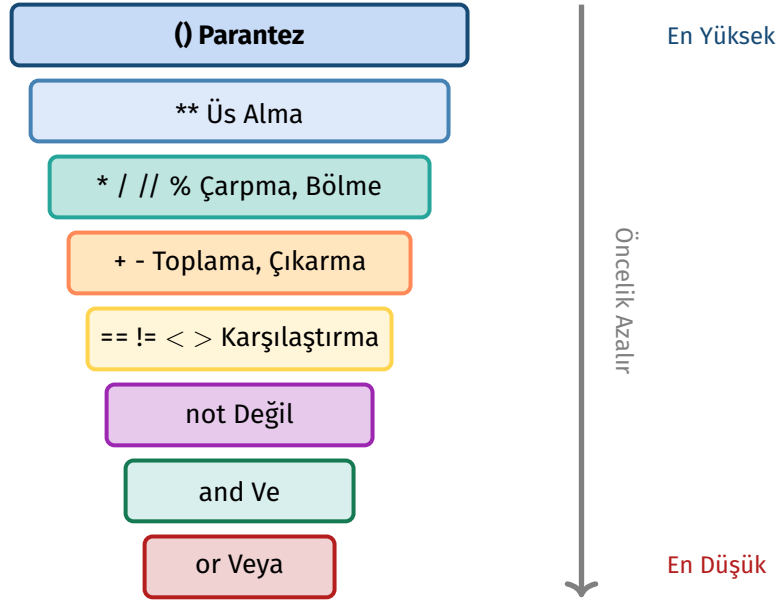
Kısa devre değerlendirme, güvenli kod yazmak için çok kullanışlıdır. Bölme yapmadan önce “bölen sıfır değil” kontrolü yaparak hatalardan kaçınabilirsin.

3.4 Operatör Önceliği

Matematik dersinde çarpmanın toplamadan önce yapıldığını bilirsin: $2 + 3 * 4 = 14$ (20 değil!). Python’da da benzer kurallar var. Bu kurallara **operatör önceliği**

denir.

3.4.1 Öncelik Sırası



Şekil 3.1: Python'da operatör öncelik sırası

Tablo 3.5: Python operatör önceliği (yüksekten düşüğe)

Öncelik	Operatör
1 (En yüksek)	() Parantez
2	** Üs alma
3	+x, -x Tekli artı/eksi
4	*, /, //, % Çarpma, bölme
5	+, - Toplama, çıkarma
6	<, <=, >, >=, ==, != Karşılaştırma
7	not Mantıksal değil
8	and Mantıksal ve
9 (En düşük)	or Mantıksal veya

3.4.2 Matematiksel Öncelik

Listing 3.15: Matematiksel operatör önceliği

```
1 # Çarpma toplamadan önce
2 sonuc = 2 + 3 * 4
3 print(sonuc) # 14 (2 + 12)
4
5 # Üs alma çarpmadan önce
6 sonuc = 2 * 3 ** 2
7 print(sonuc) # 18 (2 * 9)
8
9 # Parantez her şeyden önce
10 sonuc = (2 + 3) * 4
11 print(sonuc) # 20
12
13 # Karmaşık hesaplama
14 sonuc = 10 + 20 * 3 / 2 - 5
15 # Adımlar: 20 * 3 = 60, 60 / 2 = 30, 10 + 30 = 40, 40 - 5 = 35
16 print(sonuc) # 35.0
```

3.4.3 Karşılaştırma ve Mantıksal Öncelik

Listing 3.16: Mantıksal operatör önceliği

```
1 # Karşılaştırma, mantıksallardan önce gelir
2 x = 5
3 y = 10
4
5 # Önce karşılaştırmalar yapılır, sonra and
6 sonuc = x > 3 and y < 15
7 # Adımlar: (5 > 3) = True, (10 < 15) = True, True and True =
  True
8 print(sonuc) # True
9
10 # not, and'den önce gelir
11 sonuc = not True and False
12 # Adımlar: not True = False, False and False = False
13 print(sonuc) # False
14
15 # and, or'dan önce gelir
16 sonuc = True or False and False
17 # Adımlar: False and False = False, True or False = True
18 print(sonuc) # True
```

▲ Dikkat: Dikkat!

and, **or**'dan önce gelir! Bu sık yapılan bir hatadır. **a or b and c** ifadesi **a or (b and c)** olarak değerlendirilir.

3.4.4 Parantez Kullanımı

Karmaşık ifadelerde parantez kullanmak hem doğru sonuç almanı hem de kodu okunaklı yapmanı sağlar:

Listing 3.17: Parantez ile netlik

```
1 # Oyun hasar hesaplaması
2 baz_hasar = 50
3 kritik = True
4 kritik_carpan = 2.0
5 zirh = 20
6
7 # Parantez olmadan (kafa karıştırıcı)
8 hasar = baz_hasar * kritik_carpan if kritik else baz_hasar -
   zirh
9
10 # Parantezle (net ve anlaşılır)
11 hasar = (baz_hasar * kritik_carpan) - zirh if kritik else (
   baz_hasar - zirh)
12
13 # Daha da iyi: adımlara böl
14 if kritik:
15     ham_hasar = baz_hasar * kritik_carpan
16 else:
17     ham_hasar = baz_hasar
18 final_hasar = ham_hasar - zirh
```

3.4.5 Oyun Örneği: Puan Hesaplama

Listing 3.18: Karmaşık puan hesaplama

```
1 # Oyun sonu puan hesaplama
2 dusmus_dusmanlar = 15
3 toplanan_altın = 200
4 bonus_cani = 3
5 zaman bonusu = 50
6 zorluk_carpani = 1.5
7
8 # Adım adım hesaplama (önerilen)
9 dusmas_puani = dusmus_dusmanlar * 100
10 altın_puani = toplanan_altın * 2
11 can_bonusu = bonus_cani * 500
12 ham_puan = dusmas_puani + altın_puani + can_bonusu +
   zaman bonusu
13
14 final_puan = ham_puan * zorluk_carpani
15
16 print(f"Düşman puanı: {dusmas_puani}")
17 print(f"Altın puanı: {altın_puani}")
18 print(f"Can bonusu: {can_bonusu}")
19 print(f"Ham puan: {ham_puan}")
```

```

20 print(f"Final puan: {final_puan}")
21
22 # Tek satırda (karmaşık ama çalışır)
23 tek_satir = ((dusmus_dusmanlar * 100) + (toplanan_altın * 2) +
24              (bonus_cani * 500) + zaman_bonusu) * zorluk_carpani

```

💡 İpucu: Profesyonel İpucu

Karmaşık hesaplamaları adımlara bölmek:

1. Kodu daha okunaklı yapar
2. Hata ayıklamayı kolaylaştırır
3. Ara değerleri görmeni sağlar

Performans farkı yok sayılabilecek kadar küçüktür!

3.5 Ünite Özeti

Bu ünite de Python'un güçlü operatör sistemini öğrendin. Artık karmaşık hesaplamalar yapabilir ve mantıksal kararlar alabilirsin!

3.5.1 Bu Ünite de Öğrendiklerin

- **Aritmetik Operatörler:** +, -, *, /, //, %, **
- **Bileşik Atama:** +=, -=, *=, /=
- **Karşılaştırma:** ==, !=, <, >, <=, >=
- **Mantıksal:** and, or, not
- **Operatör Önceliği:** Parantez en önce!

3.5.2 Operatör Özet Tablosu

Tablo 3.6: Tüm operatörler özeti

Kategori	Operatörler	Örnek Kullanım
Aritmetik	+ - * / // % **	Puan hesaplama
Atama	= += -= *= /=	Değer güncelleme
Karşılaştırma	== != < > <= >=	Koşul kontrolü
Mantıksal	and or not	Çoklu koşul

3.5.3 Oyun Geliştirmede Kullanım

Öğrendiğin operatörlerin bir oyunda nasıl kullanıldığını görelim:

```

1 # --- Puan ve Seviye Sistemi ---
2 puan = 750
3 seviye = puan // 100          # 7 (her 100 puan = 1 seviye)
4 kalan = puan % 100           # 50 (sonraki seviyeye kalan)
5
6 # --- Hasar Hesaplama ---
7 saldiri = 45
8 zirh = 12
9 hasar = saldiri - zirh        # 33
10 kritik_hasar = hasar * 2     # 66
11
12 # --- Can Kontrolu ---
13 can = 30
14 can -= hasar                  # can = 30 - 33 = -3
15 oyuncu_oldu = can <= 0      # True
16
17 # --- Çoklu Kosul Kontrolu ---
18 mermi = 5
19 silah_hazir = True
20 ates_edebilir = mermi > 0 and silah_hazir # True
21
22 # --- Sinir Kontrolu ---
23 x = 820
24 ekran_genislik = 800
25 sinir_disi = x < 0 or x > ekran_genislik # True

```

💡 İpucu: Operatör Seçimi

Hangi durumda hangi operatörleri kullanacağını bilmek, oyun mantığını doğru kurmanın temelidir. Özellikle **and**, **or** ve karşılaştırma operatörleri oyunlarda sıkça bir arada kullanılır.

3.5.4 Sıradaki Adım

Bir sonraki ünite de **temel giriş/çıkış** işlemlerini öğreneceksin. `print()` fonksiyonunu zaten biliyorsun, şimdi `input()` ile kullanıcıdan veri almayı ve f-string ile güzel çıktılar oluşturmayı öğreneceksin.

3.5.5 Mini Görev: Hasar Hesaplayıcı

Öğrendiklerini pekiştirmek için bir hasar hesaplayıcı yaz:

```

1 # hasar_hesaplayici.py
2 # Bir RPG oyunu için hasar hesaplayıcı
3
4 # Oyuncu istatistikleri
5 saldiri_gucu = 50
6 kritik_sans = True # %100 kritik şansı için True
7 kritik_carpan = 2.5

```

```
8
9 # Düşman istatistikleri
10 dusmas_zirh = 20
11 dusmas_savunma_aktif = False
12
13 # Hasar hesapla
14 # 1. Kritik varsa saldırı gücünü kritik çarpanla çarp
15 # 2. Düşman savunma aktifse zırhı 2 katına çıkar
16 # 3. Final hasar = ham hasar - zırh (minimum 1)
17
18 # Kodunu buraya yaz...
```

💡 İpucu: Pratik Yap!

Bu görevi tamamladıktan sonra:

1. Farklı değerlerle test et
2. Ekstra özellikler ekle (zehir hasarı, elementel bonus)
3. Hesaplama adımlarını ekrana yazdır

4

Temel Giriş/Çıkış

“Programlar kullanıcılarla konuşmalı. Ses-siz bir program, kapalı bir kapı gibidir.”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `print()` fonksiyonunun gelişmiş kullanımı
- `input()` ile kullanıcıdan veri alma
- f-string ile formatlı metin oluşturma
- Mini Proje: Sayı Tahmin Oyunu

4.1 `print()` Fonksiyonu

İlk programımızda `print()` fonksiyonuyla tanıştık. Şimdi bu fonksiyonun tüm yeteneklerini keşfedelim! `print()` sadece metin yazdırmakla kalmaz, çok daha fazlasını yapabilir.

4.1.1 Temel Kullanım

Listing 4.1: `print()` temel kullanım

```
1 # Tek değer yazdırma
2 print("Merhaba Dünya!")
3 print(42)
4 print(3.14)
5 print(True)
6
7 # Boş satır
8 print()
```


4.1.2 Birden Fazla Değer Yazdırma

`print()` fonksiyonuna birden fazla değer verebilirsin:

Listing 4.2: Birden fazla değer yazdırma

```
1 # Virgülle ayırarak
2 ad = "Ali"
3 yas = 18
4 puan = 1500
5
6 print("Ad:", ad)
7 print("Yaş:", yas, "Puan:", puan)
8
9 # Çıktı:
10 # Ad: Ali
11 # Yaş: 18 Puan: 1500
```

Bilgi: Otomatik Boşluk

`print()` virgülle ayrılan değerler arasına otomatik olarak boşluk koyar. Bu özelliği `sep` parametresiyle değiştirebilirsin.

4.1.3 sep Parametresi

Değerler arasındaki ayırıcıyı belirler:

Listing 4.3: sep parametresi kullanımı

```
1 # Varsayılan (boşluk)
2 print("Elma", "Armut", "Muz")
3 # Elma Armut Muz
4
5 # Tire ile ayırma
6 print("2024", "12", "25", sep="-")
7 # 2024-12-25
8
9 # İki nokta ile ayırma (saat formatı)
10 saat = 14
11 dakika = 30
12 saniye = 45
13 print(saat, dakika, saniye, sep=":")
14 # 14:30:45
15
16 # Ayırıcı olmadan
17 print("P", "y", "t", "h", "o", "n", sep="")
18 # Python
```

4.1.4 end Parametresi

Satır sonundaki karakteri belirler (varsayılan: yeni satır):

Listing 4.4: end parametresi kullanımı

```

1 # Varsayılan davranış
2 print("Satır 1")
3 print("Satır 2")
4 # Satır 1
5 # Satır 2
6
7 # Aynı satırda devam
8 print("Yükleniyor", end="")
9 print("...")
10 # Yükleniyor...
11
12 # Özel son karakter
13 print("Puan: 100", end=" | ")
14 print("Can: 3", end=" | ")
15 print("Seviye: 5")
16 # Puan: 100 | Can: 3 | Seviye: 5
17
18 # İlerleme çubuğu simülasyonu
19 import time
20 print("Yükleniyor: ", end="")
21 for i in range(5):
22     print("#", end="", flush=True)
23     time.sleep(0.5)
24 print(" Tamamlandı!")

```

⚠ Dikkat: flush Parametresi

Bazı durumlarda (özellikle `end=""` kullanırken) çıktı hemen görünmeyebilir. `flush=True` ekleyerek çıktının hemen ekrana yazılmasını sağlayabilirsiniz.

4.1.5 Özel Karakterler

Metinlerde özel karakterler kullanabilirsiniz:

Tablo 4.1: Python kaçış karakterleri

Karakter	Anlam	Örnek Çıktı
<code>\n</code>	Yeni satır	Satır 1
	Satır 2	
<code>\t</code>	Tab (sekme)	Sütun1 Sütun2
<code>\\</code>	Ters eğik çizgi	C:\Users
<code>\"</code>	Çift tırnak	"metin"
<code>\'</code>	Tek tırnak	'metin'

Listing 4.5: Özel karakterler kullanımı

```

1 # Yeni satır
2 print("Satır 1\nSatır 2\nSatır 3")
3
4 # Tab ile hizalama
5 print("İsim\tPuan\tSeviye")
6 print("Ali\t1500\t10")
7 print("Ayşe\t2300\t15")
8
9 # Tırnak içinde tırnak
10 print("0 dedi ki: \"Oyun başlasın!\"")
11 print('It\'s a game!')
12
13 # Dosya yolu
14 print("C:\\Users\\Oyuncu\\Saves")

```

💡 İpucu: Profesyonel İpucu

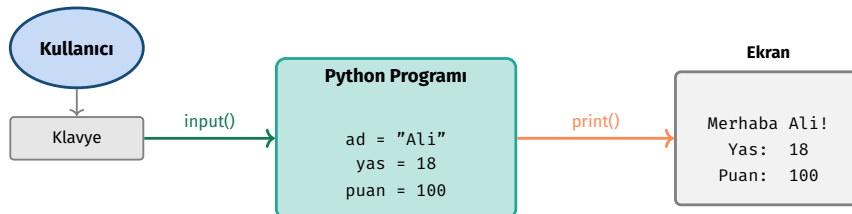
Oyun geliştirmede `print()` genellikle hata ayıklama (debug) için kullanılır. Gerçek oyunlarda ekrana metin çizmek için PyGame'in `font.render()` fonksiyonunu kullanacaksınız.

4.2 input() Fonksiyonu

Şimdiye kadar programlarımız tek yönlü çalıştı: biz kod yazdık, program çalıştı. Ama gerçek oyunlar kullanıcıyla etkileşim kurar. `input()` fonksiyonu, kullanıcıdan veri almamızı sağlar.

■ Tanım: input()

`input()` fonksiyonu, programı duraklatır ve kullanıcının klavyeden bir şey yazmasını bekler. Kullanıcı Enter tuşuna basınca, yazılan metin programa döner.



Şekil 4.1: Giriş ve çıkış işlemlerinin akışı

4.2.1 Temel Kullanım

Listing 4.6: input() temel kullanım

```
1 # Kullanıcıdan isim al
2 isim = input("Adını gir: ")
3 print("Merhaba, " + isim + "!")
4
5 # Çalışma örneği:
6 # Adını gir: Ali
7 # Merhaba, Ali!
```

4.2.2 input() Her Zaman String Döndürür!

Bu çok önemli bir nokta:

Listing 4.7: input() ve veri tipi

```
1 # Sayı girilse bile string döner!
2 yas = input("Yaşını gir: ")
3 print(type(yas)) # <class 'str'>
4
5 # Bu yüzden matematiksel işlem hata verir:
6 # yas + 5 # TypeError!
7
8 # Doğru kullanım: önce dönüştür
9 yas = int(input("Yaşını gir: "))
10 print(type(yas)) # <class 'int'>
11 print("5 yıl sonra", yas + 5, "yaşında olacaksın")
```

⚠ Dikkat: Dönüşüm Hatası

Kullanıcı sayı yerine metin girerse (örneğin “on beş”) `int()` fonksiyonu hata verir. İleriki bölümlerde `try/except` ile bu hataları yakalamayı öğreneceksin.

4.2.3 Farklı Veri Tipleri Alma

Listing 4.8: Farklı tipte veri alma

```
1 # Tam sayı alma
2 yas = int(input("Yaşın: "))
3
4 # Ondalıklı sayı alma
5 boy = float(input("Boyun (m): "))
6
7 # Boolean simülasyonu (kullanıcı e/h girer)
8 cevap = input("Devam etmek istiyor musun? (e/h): ")
9 devam = cevap.lower() == "e"
```

4.2.4 Oyun Örneği: Karakter Oluşturma

Listing 4.9: Karakter oluşturma örneği

```

1 print("=== KARAKTER OLUŞTURMA ===")
2 print()
3
4 # Karakter bilgileri al
5 karakter_adi = input("Karakter adı: ")
6 sinif = input("Sınıf (Savaşçı/Büyücü/Okçu): ")
7 yas = int(input("Yaş: "))
8
9 # Sınıfa göre başlangıç değerleri
10 if sinif.lower() == "savaşçı":
11     can = 150
12     mana = 30
13     saldiri = 25
14 elif sinif.lower() == "büyücü":
15     can = 80
16     mana = 150
17     saldiri = 40
18 else: # Okçu
19     can = 100
20     mana = 60
21     saldiri = 35
22
23 # Karakter özeti
24 print()
25 print("=== KARAKTER ÖZETİ ===")
26 print(f"Ad: {karakter_adi}")
27 print(f"Sınıf: {sinif}")
28 print(f"Yaş: {yas}")
29 print(f"Can: {can} | Mana: {mana} | Saldırı: {saldiri}")

```

4.2.5 Tek Satırda Birden Fazla Değer

Bazen kullanıcıdan birden fazla değer almak istersin:

Listing 4.10: Çoklu değer alma

```

1 # split() ile ayırma
2 koordinat = input("Koordinat gir (x y): ")
3 parcalar = koordinat.split() # Boşluktan ayır
4 x = int(parcalar[0])
5 y = int(parcalar[1])
6 print(f"X: {x}, Y: {y}")
7
8 # Tek satırda (daha kısa)
9 x, y = input("Koordinat (x y): ").split()
10 x, y = int(x), int(y)
11
12 # Sayıları virgülle ayırarak
13 degerler = input("3 sayı gir (virgülle): ") # Örn: 10,20,30
14 a, b, c = degerler.split(",")

```

● Bilgi: split() Metodu

`split()` metodu bir stringi parçalara ayırır. Varsayılan olarak boşluktan ayırır, ama istediğin karakteri parametre olarak verebilirsin: `split(",")` virgülden ayırır.

💡 İpucu: Profesyonel İpucu

Terminal tabanlı oyunlarda `input()` kullanılır, ama grafik tabanlı PyGame oyunlarında klavye girişi farklı çalışır. PyGame'de `pygame.event` ve `pygame.key` modüllerini kullanacaksın.

4.3 f-string ile Formatlama

String birleştirme bazen karmaşık olabilir. f-string (formatted string literals), Python 3.6 ile gelen ve string oluşturmayı çok kolaylaştıran bir özelliktir.

■ Tanım: f-string

f-string, `f` veya `F` harfi ile başlayan ve süslü parantez `{}` içinde Python ifadeleri yazabildiğin özel bir string formatıdır.

4.3.1 Temel Kullanım

Listing 4.11: f-string temel kullanım

```
1 # Eski yöntem (string birleştirme)
2 ad = "Ali"
3 puan = 1500
4 mesaj = "Oyuncu: " + ad + ", Puan: " + str(puan)
5
6 # f-string ile (çok daha temiz!)
7 mesaj = f"Oyuncu: {ad}, Puan: {puan}"
8 print(mesaj) # Oyuncu: Ali, Puan: 1500
```

4.3.2 İfade Yazma

f-string içinde sadece değişkenler değil, ifadeler de yazabilirsiniz:

Listing 4.12: f-string içinde ifadeler

```
1 # Matematiksel işlemler
2 x = 10
3 y = 5
4 print(f"{x} + {y} = {x + y}")
5 # 10 + 5 = 15
6
```

```

7 # Fonksiyon çağrılarları
8 isim = "ali"
9 print(f"Merhaba {isim.upper()}!")
10 # Merhaba ALI!
11
12 # Koşullu ifade
13 puan = 85
14 print(f"Sonuç: {'Geçti' if puan >= 50 else 'Kaldı'}")
15 # Sonuç: Geçti

```

4.3.3 Sayı Formatlama

f-string ile sayıları güzelce formatlayabilirsiniz:

Listing 4.13: Sayı formatlama

```

1 # Ondalık basamak sınırlama
2 pi = 3.14159265359
3 print(f"Pi: {pi:.2f}") # Pi: 3.14
4 print(f"Pi: {pi:.4f}") # Pi: 3.1416
5
6 # Yüzde gösterimi
7 oran = 0.756
8 print(f"Başarı: {oran:.1%}") # Başarı: 75.6%
9
10 # Binlik ayırıcı
11 buyuk_sayi = 1234567890
12 print(f"Puan: {buyuk_sayi:,}") # Puan: 1,234,567,890
13
14 # Sıfır ile doldurma
15 seviye = 5
16 print(f"Seviye: {seviye:03d}") # Seviye: 005
17
18 # Sağa/sola hizalama
19 isim = "Ali"
20 print(f"|{isim:<10}|") # |Ali          | (sola hizalı)
21 print(f"|{isim:>10}|") # |          Ali| (sağa hizalı)
22 print(f"|{isim:^10}|") # |    Ali    | (ortala)

```

● Bilgi: Format Sözdizimi

{değer:format} şeklinde kullanılır:

- `.2f` → 2 ondalık basamak (float)
- `.1%` → yüzde formatı
- `,` → binlik ayırıcı
- `03d` → 3 haneli, sıfırla doldur
- `<10` → 10 karakter, sola hizala

4.3.4 Oyun HUD Örneği

Listing 4.14: Oyun HUD formatlaması

```

1 # Oyuncu durumu
2 isim = "KahramanAli"
3 can = 78
4 max_can = 100
5 puan = 15750
6 seviye = 12
7 zaman = 125.7
8
9 # HUD oluştur
10 can_yuzdesi = can / max_can
11
12 print("=" * 40)
13 print(f" Oyuncu: {isim:^15}")
14 print(f" Seviye: {seviye:>3} {'*' * min(seviye // 5, 5)}")
15 print(f" Can: {can:>3}/{max_can} [{can_yuzdesi:>6.1%}]")
16 print(f" Puan: {puan:>10,}")
17 print(f" Süre: {zaman:>6.1f} sn")
18 print("=" * 40)

```

>_ Çıktı

```

=====
Oyuncu: KahramanAli
Seviye: 12 **
Can: 78/100 [ 78.0%]
Puan: 15,750
Süre: 125.7 sn
=====

```

4.3.5 Çok Satırlı f-string

Listing 4.15: Çok satırlı f-string

```

1 oyuncu = "Ali"
2 puan = 1500
3 seviye = 10
4 can = 85
5
6 ozet = f"""
7 +=====+
8 | OYUN DURUMU |
9 +=====+
10 | Oyuncu: {oyuncu:<14} |
11 | Puan: {puan:>14,} |
12 | Seviye: {seviye:>14} |
13 | Can: {can:>13}% |
14 +=====+

```



```

15 """
16 print(ozet)

```

💡 İpucu: Profesyonel İpucu

f-string, Python'da string formatlama için en modern ve önerilen yöntemdir. Hem okunabilir hem de hızlıdır. Her zaman f-string kullanmayı tercih et!

4.4 Mini Proje: Sayı Tahmin Oyunu

Bu bölümde öğrendiklerini bir araya getirerek ilk oyununu yapacaksın: Sayı Tahmin Oyunu! Bu proje, henüz döngü ve koşulları tam öğrenmemiş olsan da neler yapabileceğini gösterecek.

4.4.1 Oyun Konsepti

- Bilgisayar 1-100 arasında rastgele bir sayı seçer
- Oyuncu sayıyı tahmin etmeye çalışır
- Her tahminde “Çok büyük” veya “Çok küçük” ipucu verilir
- Doğru tahmin edildiğinde kaç denemede bulunduğu gösterilir

4.4.2 Basit Versiyon (Tek Tahmin)

İlk olarak, tek tahminlik basit bir versiyon yapalım:

Listing 4.16: Sayı tahmin - basit versiyon

```

1 # sayi_tahmin_basit.py
2 import random
3
4 print("=" * 30)
5 print("    SAYI TAHMİN OYUNU")
6 print("=" * 30)
7 print()
8
9 # Rastgele sayı seç (1-100 arası)
10 gizli_sayi = random.randint(1, 100)
11
12 # Kullanıcıdan tahmin al
13 tahmin = int(input("1-100 arası bir sayı tahmin et: "))
14
15 # Sonucu kontrol et
16 if tahmin == gizli_sayi:
17     print("*** TEBRIKLER! İlk denemede bildin! ***")
18 elif tahmin < gizli_sayi:
19     print(f"X Yanlis! Sayi {tahmin}'den BUYUK.")
20 else:
21     print(f"X Yanlis! Sayi {tahmin}'den KUCUK.")
22

```

```
23 print(f"Gizli sayı: {gizli_sayi}")
```

● Bilgi: random Modülü

`import random` satırı, Python'un rastgele sayı üretme modülünü programa dahil eder. `random.randint(a, b)` fonksiyonu `a` ile `b` arasında (dahil) rastgele bir tam sayı döndürür.

4.4.3 Gelişmiş Versiyon (Çok Deneme)

Şimdi döngü kullanarak çok denemeli versiyonu yapalım:

Listing 4.17: Sayı tahmin - gelişmiş versiyon

```
1 # sayi_tahmin_oyunu.py
2 import random
3
4 def oyun_baslat():
5     """Ana oyun fonksiyonu"""
6     print()
7     print("=====+")
8     print("|   SAYI TAHMIN OYUNU   |")
9     print("|   1-100 arasi bir sayi   |")
10    print("=====+")
11    print()
12
13    # Oyun ayarları
14    gizli_sayi = random.randint(1, 100)
15    max_deneme = 7
16    deneme = 0
17    kazandi = False
18
19    print(f"[!] {max_deneme} deneme hakkın var!")
20    print()
21
22    # Oyun dongusu
23    while deneme < max_deneme and not kazandi:
24        deneme += 1
25        kalan = max_deneme - deneme
26
27        # Tahmin al
28        try:
29            tahmin = int(input(f"[{deneme}/{max_deneme}]
30                               Tahminin: "))
31        except ValueError:
32            print("[!] Lütfen geçerli bir sayı gir!")
33            deneme -= 1 # Hatalı giriş hak goturmesin
34            continue
35
36    # Kontrol et
```

```

36     if tahmin == gizli_sayi:
37         kazandi = True
38         print()
39         print("*" * 30)
40         print(f"    TEBRIKLER! {deneme} denemede buldun!")
41         print("*" * 30)
42     elif tahmin < gizli_sayi:
43         print(f"    [^] Daha BUYUK! (Kalan: {kalan})")
44     else:
45         print(f"    [v] Daha KUCUK! (Kalan: {kalan})")
46
47     # Kaybettiye
48     if not kazandi:
49         print()
50         print("X" * 30)
51         print(f"    Kaybettin! Sayi: {gizli_sayi}")
52         print("X" * 30)
53
54     return kazandi, deneme
55
56 # Oyunu başlat
57 if __name__ == "__main__":
58     oyun_baslat()

```

4.4.4 Kodu Anlayalım

Bu kodda birçok yeni kavram var. Hepsini henüz bilmesen de ana mantığı anlayabilirsin:

1. **import random:** Rastgele sayı için modül dahil etme
2. **while döngüsü:** Deneme hakkı bitene kadar tekrarla
3. **try/except:** Hatalı girişleri yakala
4. **if/elif/else:** Koşullara göre farklı çıktılar

⚠ Dikkat: Henüz Öğrenmedik!

Bu kodda **while**, **if**, **try/except** ve **def** (fonksiyon) kullanıldı. Bunları bir sonraki bölümde detaylı öğreneceksin. Şimdilik kodu çalıştır ve tadını çıkar!

4.4.5 Puan Sistemi Ekleme

Oyunu daha heyecanlı yapmak için puan sistemi ekleyebilirsiniz:

Listing 4.18: Puan sistemi

```

1 # Puan hesaplama fonksiyonu
2 def puan_hesapla(deneme_sayisi, max_deneme):
3     """Deneme sayısına göre puan hesapla"""
4     if deneme_sayisi == 1:
5         return 1000 # İlk denemede bilmek harika!
6     elif deneme_sayisi == 2:
7         return 750
8     elif deneme_sayisi <= 4:
9         return 500
10    elif deneme_sayisi <= 6:
11        return 250
12    else:
13        return 100
14
15 # Kullanım
16 deneme = 3
17 puan = puan_hesapla(deneme, 7)
18 print(f"Kazanılan puan: {puan}")

```

4.4.6 Ekstra Özellik Fikirleri

Oyununu geliştirmek için şu özellikleri ekleyebilirsin:

- **Zorluk Seviyeleri:** Kolay (1-50), Normal (1-100), Zor (1-500)
- **Yüksek Skor:** En iyi sonucu kaydet
- **Tekrar Oynama:** Oyun bitince tekrar sorma
- **İstatistikler:** Toplam oyun, kazanma oranı
- **İpucu Sistemi:** “Çift sayı”, “30’dan büyük” gibi

💡 İpucu: Profesyonel İpucu

İlk projen basit olsa da, onu geliştirmek için zaman ayır. Her yeni özellik eklediğinde programlama becerin gelişir. Bir sonraki bölümde döngüleri ve koşulları öğrendiğinde bu oyunu çok daha iyi yapabileceksin!

4.5 Ünite Özeti

Bu ünite de programların kullanıcıyla nasıl iletişim kurduğunu öğrendin. Artık kullanıcıdan veri alabilir ve güzel formatlanmış çıktılar oluşturabilirsin!

4.5.1 Bu Ünite de Öğrendiklerin

- **print():** Ekran a çıktı verme
- **sep ve end:** Çıktı formatını kontrol etme
- **input():** Kullanıcıdan veri alma
- **f-string:** Formatlı metin oluşturma
- **Mini Proje:** Sayı Tahmin Oyunu

4.5.2 Fonksiyon Özeti

Tablo 4.2: Giriş/çıkış fonksiyonları

Fonksiyon	Görevi	Örnek
<code>print()</code>	Ekrana yazdır	<code>print("Merhaba")</code>
<code>input()</code>	Kullanıcıdan al	<code>ad = input("Adın: ")</code>
<code>int()</code>	Sayıya çevir	<code>int(input("Yaş: "))</code>
<code>float()</code>	Ondalığa çevir	<code>float("3.14")</code>
<code>str()</code>	Metne çevir	<code>str(100)</code>

4.5.3 f-string Format Özeti

Tablo 4.3: f-string format kodları

Format	Anlam	Örnek
<code>:.2f</code>	2 ondalık	<code>f"{3.14159:.2f}"</code> → 3.14
<code>:.1%</code>	Yüzde	<code>f"{0.75:.1%}"</code> → 75.0%
<code>:,</code>	Binlik ayırıcı	<code>f"{1000000:,}"</code> → 1,000,000
<code>>10</code>	Sağa hizala	<code>f"{5:>10}"</code> → 5
<code><10</code>	Sola hizala	<code>f"{5:<10}"</code> → 5
<code>03d</code>	Sıfır doldur	<code>f"{5:03d}"</code> → 005

4.5.4 Bölüm 1 Tamamlandı!

Tebrikler! Python'a Giriş ve Programlama Temelleri bölümünü tamamladın. Bu bölümde öğrendiklerin:

1. Python kurulumu ve geliştirme ortamı
2. Değişkenler ve temel veri tipleri
3. Operatörler ve ifadeler
4. Giriş/çıkış işlemleri ve formatlama
5. İlk mini projen: Sayı Tahmin Oyunu

4.5.5 Sıradaki Bölüm

Bir sonraki bölümde **kontrol yapıları ve fonksiyonlar** öğreneceksin:

- **if/elif/else:** Koşullara göre farklı kodlar çalıştırma
- **while:** Koşul sağlandığı sürece tekrarlama
- **for:** Belirli sayıda tekrarlama
- **def:** Kendi fonksiyonlarını oluşturma
- **Mini Proje:** Metin Tabanlı Macera Oyunu

💡 İpucu: Son Tavsiye

Programlama öğrenmenin en iyi yolu pratik yapmaktır. Bu bölümdeki örnekleri sadece okuma, bilgisayarında yaz ve çalıştır! Kodu değiştir, hatalarla karşılaş, çözümler bul. Her hata, bir öğrenme fırsatıdır!

4.5.6 Bölüm 1 Kapanış Projesi

Bölümü tamamlamadan önce şu projeyi yap:

Listing 4.19: Kapanış projesi şablonu

```
1 # kisisel_profil.py
2 # Tüm Bölüm 1 konularını kullanan interaktif program
3
4 print("=" * 40)
5 print("      KİŞİSEL PROFİL OLUŞTURUCU")
6 print("=" * 40)
7
8 # Kullanıcıdan bilgi al
9 # ... (sen tamamla)
10
11 # f-string ile güzel çıktı oluştur
12 # ... (sen tamamla)
13
14 # Basit hesaplamalar yap
15 # ... (sen tamamla)
```

5

Koşullu İfadeler

“Hayat seçimlerden oluşur. Programlar da öyle!”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `if` ifadesi ile basit koşul kontrolü
- `elif` ile çoklu koşul kontrolü
- `else` ile varsayılan durum belirleme
- İç içe koşulların kullanımı
- Karşılaştırma ve mantıksal operatörlerin koşullarda kullanımı

5.1 If İfadesi: Programın Karar Vermesi

Bir oyun düşün: Karakterin bir kapıya yaklaşıyor. Elinde anahtar varsa kapı açılır, yoksa “Anahtarı bul!” mesajı çıkar. İşte bu tür kararları programlamak için **koşullu ifadeler** kullanıyoruz. Python’da en temel koşullu ifade `if` ifadesidir.

■ Tanım: Koşullu İfade

Koşullu ifade, bir koşulun doğru veya yanlış olmasına göre farklı kod bloklarının çalıştırılmasını sağlayan yapıdır. Oyunlarda sürekli kullanılır: düşmana çarptın mı, can sıfır mı, yeterli altın var mı...

5.1.1 If İfadesinin Yapısı

Python’da `if` ifadesi şu şekilde yazılır:

Python – If İfadesinin Temel Yapısı

```
1 if koşul:
2     # Koşul doğruysa çalışacak kod
3     print("Koşul doğru!")
```

Dikkat edilmesi gereken noktalar:

- Koşuldan sonra iki nokta üst üste (:) konur
- Koşul doğruysa çalışacak kod **girintili** yazılır (4 boşluk)
- Girinti çok önemli! Python, hangi kodun koşula ait olduğunu girintiden anlar

⚠ Dikkat: Girinti Hatası

Python’da girinti zorunludur. Yanlış girinti `IndentationError` hatasına neden olur. Her zaman 4 boşluk veya 1 tab kullan, ama ikisini karıştıрма!

5.1.2 Basit Örnekler

Oyun geliştirmeden örneklerle if ifadesini anlayalım:

Python – Oyuncu Canı Kontrolü

```
1 oyuncu_can = 100
2
3 if oyuncu_can > 0:
4     print("Oyuncu hayatta!")
```

Bu kodda `oyuncu_can > 0` koşulu kontrol edilir. Eğer can 0’dan büyükse (yani $100 > 0$ doğru), mesaj yazdırılır.

Python – Anahtar Kontrolü

```
1 anahtar_var = True
2
3 if anahtar_var:
4     print("Kapı açıldı!")
```

`True` ve `False` değerleri doğrudan koşul olarak kullanılabilir. `if anahtar_var:` ifadesi “anahtar_var True ise” anlamına gelir.

5.1.3 Karşılaştırma Operatörleri

Koşullarda sıklıkla karşılaştırma operatörleri kullanılır:

Tablo 5.1: Karşılaştırma Operatörleri

Operatör	Anlam	Örnek
==	Eşit mi?	<code>skor == 100</code>
!=	Eşit değil mi?	<code>can != 0</code>
>	Büyük mü?	<code>altın > 50</code>
<	Küçük mü?	<code>zaman < 60</code>
>=	Büyük veya eşit mi?	<code>seviye >= 5</code>
<=	Küçük veya eşit mi?	<code>hiz <= 10</code>

⚠ Dikkat: == vs = Farkı

Tek eşittir (=) atama yapar: `x = 5`

Çift eşittir (==) karşılaştırma yapar: `x == 5`

Bu ikisini karıştırmak çok yaygın bir hatadır!

Python – Skor Kontrolü Örneği

```

1 skor = 150
2
3 if skor >= 100:
4     print("Yeni rekor!")
5
6 if skor == 150:
7     print("Tam 150 puan!")

```

💡 İpucu: Profesyonel İpucu

Koşulları yazarken önce değişkenin değerini düşün, sonra karşılaştırmayı yap. “Skor 100’den büyük mü?” sorusu `skor > 100` şeklinde yazılır.

5.2 Elif ve Else: Çoklu Seçenekler

Gerçek oyunlarda sadece tek bir koşul yetmez. Düşman saldırısında farklı hasar türleri olabilir, oyuncunun seviyesine göre farklı yetenekler açılabilir. Bu durumlar için `elif` ve `else` ifadelerini kullanırız.

5.2.1 Else: Koşul Yanlışsa

`else` ifadesi, `if` koşulu yanlış olduğunda çalışacak kodu belirtir:

Python – Hayatta mı Öldü mü?

```

1 oyuncu_can = 0
2
3 if oyuncu_can > 0:

```

```

4     print("Oyuncu hayatta!")
5 else:
6     print("Game Over!")

```

Bu örnekte `oyuncu_can = 0` olduğu için koşul yanlıştır ve `else` bloğu çalışır.

■ Tanım: Else Bloğu

`else` bloğu, `if` koşulu sağlanmadığında çalışan “varsayılan durum” kodudur. Bir if-else yapısında mutlaka biri çalışır: ya if ya else.

5.2.2 Elif: Birden Fazla Koşul

Bazen ikiden fazla seçenek vardır. Örneğin bir oyunda oyuncunun sağlık durumunu göstermek istiyoruz:

Python – Sağlık Durumu

```

1 can = 35
2
3 if can >= 70:
4     print("Sağlık: İyi")
5 elif can >= 30:
6     print("Sağlık: Orta")
7 else:
8     print("Sağlık: Kritik!")

```

`elif`, “else if” ifadesinin kısaltmasıdır. Birden fazla `elif` kullanabilirsiniz:

Python – Seviye Sistemi

```

1 puan = 2500
2
3 if puan >= 5000:
4     seviye = "Efsane"
5 elif puan >= 2000:
6     seviye = "Usta"
7 elif puan >= 1000:
8     seviye = "Deneyimli"
9 elif puan >= 500:
10    seviye = "Acemi"
11 else:
12    seviye = "Çaylak"
13
14 print(f"Seviyen: {seviye}")

```

● Bilgi: Koşul Sırası Önemli!

Koşullar yukarıdan aşağıya kontrol edilir. İlk doğru koşul bulununca diğerleri atlanır. Bu yüzden en kısıtlayıcı koşulu (en büyük değeri) en üste koy!

5.2.3 Oyun Senaryosu: Kapı Kontrol Sistemi

Bir zindanda kapı açma sistemi yapalım:

Python – Kapı Kontrol Sistemi

```

1 anahtar_sayisi = 2
2 altin = 50
3 buyucu_mu = False
4
5 if anahtar_sayisi >= 1:
6     print("Anahtar ile kapıyı açtın!")
7     anahtar_sayisi -= 1
8 elif altin >= 100:
9     print("Altın ile geçiş hakkı aldın!")
10    altin -= 100
11 elif buyucu_mu:
12    print("Büyü ile kapıyı açtın!")
13 else:
14    print("Kapıyı açamazsın. Anahtar bul veya altın
    ↪ topla!")

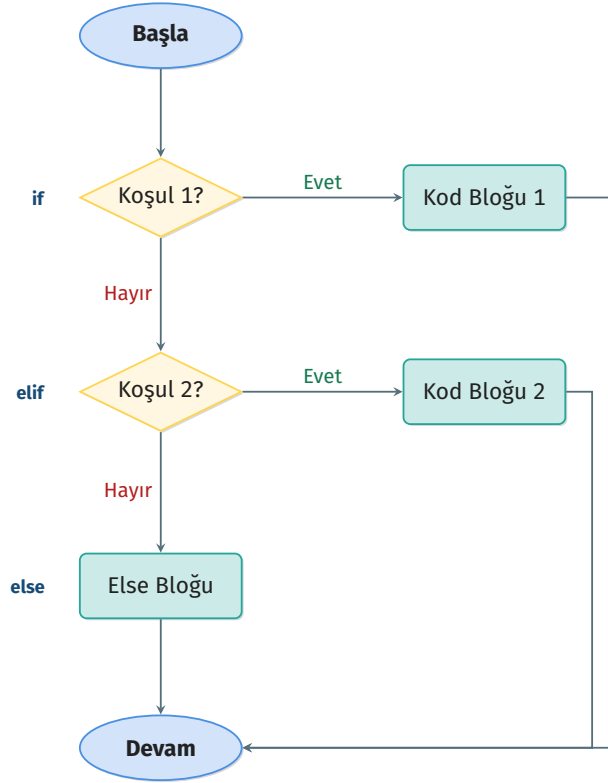
```

Bu örnekte sistem şu sırayla kontrol eder:

1. Önce anahtar var mı? (Varsa aç)
2. Anahtar yoksa, yeterli altın var mı? (Varsa satın al)
3. Altın da yoksa, büyücü mü? (Evet ise büyü yap)
4. Hiçbiri değilse, kapı açılmaz

5.2.4 If-Elif-Else Akış Diyagramı

Koşullu ifadelerin akışını görselleştirelim. Aşağıdaki diyagram, bir if-elif-else yapısının adım adım nasıl işlediğini göstermektedir:



Şekil 5.1: If-Elif-Else Yapısının Akış Diyagramı

Diyagramdaki akışı adım adım inceleyelim:

1. Program önce **if** koşulunu kontrol eder. Eğer bu koşul doğruysa, yalnızca ilk blok çalışır ve diğer tüm koşullar atlanır.
2. İlk koşul yanlışsa, sıradaki **elif** koşuluna geçilir. Burada da aynı mantık işler: doğruysa kendi bloğu çalışır, yanlışsa bir sonraki koşula geçilir.
3. Birden fazla **elif** varsa, her biri sırayla kontrol edilir. İlk doğru bulunan koşulun bloğu çalıştırılır.
4. Hiçbir koşul doğru değilse **else** bloğu devreye girer. Bu, “geri kalan tüm durumlar” anlamına gelir.

Bu yapı oyunlarda sürekli karşımıza çıkar. Örneğin bir düşmanın yapay zekasını düşün: Canı azsa kaçsın, menzildeyse saldırсын, değilse yaklaşsın. Bu üç davranış tam olarak if-elif-else ile modellenir. Önemli olan nokta, koşulların **sırasıdır**—Python ilk doğru koşulu bulduğunda gerisini kontrol etmez, bu yüzden en kritik kontrolü en üste koymalısın.

💡 İpucu: Profesyonel İpucu

Birden fazla koşul yazarken, “karşılıklı dışlayıcı” olmasına dikkat et. Yani koşullar birbirini kapsamamalı. Örneğin “puan > 50” ve “puan > 100” yerine “puan >= 100” ve “elif puan >= 50” kullan.

5.3 İç İç Koşullar: Karmaşık Kararlar

Oyunlarda bazen bir koşulun içinde başka koşullar kontrol etmemiz gerekir. Örneğin: “Oyuncu hayattaysa VE düşmana yakınsa VE silahı varsa ateş et.” Bu tür karmaşık kararlar için iç içe koşullar kullanırız.

5.3.1 İç İç If Yapısı

Bir `if` bloğunun içine başka bir `if` yazabilirsin:

Python – İç İç Koşul Örneği

```

1 oyuncu_can = 50
2 dusman_mesafe = 10
3
4 if oyuncu_can > 0:
5     print("Oyuncu hayatta!")
6
7     if dusman_mesafe < 20:
8         print("Düşman yakında, dikkatli ol!")

```

Bu kodda önce oyuncunun hayatta olup olmadığı kontrol edilir. Hayattaysa, içerideki koşul (düşman mesafesi) kontrol edilir.

⚠ Dikkat: Girinti Seviyeleri

İç içe koşullarda her seviye için 4 boşluk daha eklenir. İlk `if` 0 girinti, içindeki `if` 4 girinti, onun içindeki 8 girinti... Girintiler düzgün olmazsa kod çalışmaz!

5.3.2 Oyun Senaryosu: Saldırı Sistemi

Bir RPG oyununda saldırı mekaniği düşünelim:

Python – Saldırı Sistemi

```

1 oyuncu_can = 80
2 mermi = 5
3 dusman_gorunur = True
4 dusman_mesafe = 15
5
6 if oyuncu_can > 0:
7     # Oyuncu hayatta
8     if dusman_gorunur:
9         # Düşman görünürde
10        if dusman_mesafe <= 20:
11            # Menzilde
12            if mermi > 0:
13                print("Ateş!")
14                mermi -= 1
15            else:

```

```

16         print("Mermi yok! Şarjör değiştir!")
17     else:
18         print("Düşman çok uzakta!")
19     else:
20         print("Hedef yok!")
21 else:
22     print("Game Over!")

```

5.3.3 Daha Temiz Bir Yol: Birleşik Koşullar

İç içe koşullar bazen çok karmaşık hale gelebilir. Mantıksal operatörler kullanarak daha okunabilir kod yazabilirsin:

Python – Birleşik Koşul

```

1  # İç içe yerine
2  if oyuncu_can > 0:
3      if dusman_gorunur:
4          if dusman_mesafe <= 20:
5              print("Ateş edebilirsiniz!")
6
7  # Birleşik koşul
8  if oyuncu_can > 0 and dusman_gorunur and dusman_mesafe <=
   ↪ 20:
9      print("Ateş edebilirsiniz!")

```

Her iki yaklaşım da aynı işi yapar, ama birleşik koşul daha kısa ve okunaklıdır.

● Bilgi: Ne Zaman Hangisi?

- **İç içe if:** Her koşul için farklı mesaj/işlem gerekiyorsa
- **Birleşik koşul:** Sadece tüm koşullar doğruysa işlem yapılacaksa

5.3.4 Örnek: Hazine Sandığı

Bir hazine sandığı açma sistemi tasarlayalım:

Python – Hazine Sandığı

```

1  anahtar_var = True
2  tuzak_var = True
3  tuzak_cozmek = "sihir" # veya "hiz" veya "guc"
4  oyuncu_yetenek = "sihir"
5
6  print("Hazine sandığına yaklaştın...")
7
8  if anahtar_var:
9      print("Anahtarı kullandın!")
10

```

```

11     if tuzak_var:
12         print("Dikkat! Sandıkta tuzak var!")
13
14         if oyuncu_yetenek == tuzak_cozmek:
15             print("Tuzağı etkisiz hale getirdin!")
16             print("Hazineyi aldın: 500 altın!")
17         else:
18             print("Tuzak patladı! 30 hasar aldın!")
19     else:
20         print("Hazineyi aldın: 500 altın!")
21 else:
22     print("Sandığı açmak için anahtar gerekli!")

```

İpucu: Profesyonel İpucu

3-4 seviyeden daha derin iç içe koşullar okunması zor hale gelir. Bu durumda kodu fonksiyonlara böl (Ünite 3'te göreceğiz) veya mantıksal operatörler kullan.

5.4 Mantıksal Operatörlerle Koşullar

Birden fazla koşulu tek bir `if` ifadesinde birleştirmek için mantıksal operatörler kullanırız: `and`, `or` ve `not`. Bu operatörler, oyun mekaniğinde çok sık kullanılır.

5.4.1 and Operatörü: Hepsi Doğruysa

`and` operatörü, tüm koşulların doğru olmasını gerektirir:

Python – and Operatörü

```

1 can = 100
2 mermi = 10
3 dusman_menzilde = True
4
5 # Tüm koşullar doğru olmalı
6 if can > 0 and mermi > 0 and dusman_menzilde:
7     print("Ateş et!")

```

Tablo 5.2: and Operatörü Doğruluk Tablosu

Koşul A	Koşul B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

5.4.2 or Operatörü: Biri Doğruysa

or operatörü, koşullardan en az birinin doğru olmasını gerektirir:

Python – or Operatörü

```

1 anahtar = False
2 maymuncuk = True
3 patlayici = False
4
5 # Kapıyı açmanın birden fazla yolu var
6 if anahtar or maymuncuk or patlayici:
7     print("Kapı açıldı!")
8 else:
9     print("Kapıyı açacak bir şey bul!")

```

Tablo 5.3: or Operatörü Doğruluk Tablosu

Koşul A	Koşul B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

5.4.3 not Operatörü: Tersine Çevirme

not operatörü, koşulun tersini alır:

Python – not Operatörü

```

1 oyun_bitti = False
2
3 # Oyun bitmediyse devam et
4 if not oyun_bitti:
5     print("Oyun devam ediyor...")
6
7 # Şununla aynı:
8 if oyun_bitti == False:
9     print("Oyun devam ediyor...")

```

Bilgi: not Kullanımı

not operatörü kodu daha okunabilir yapar. “if not oyun_bitti” ifadesi “oyun bitmediyse” şeklinde okunur ve çok doğal gelir.

5.4.4 Operatörleri Birleştirme

Mantıksal operatörleri birlikte kullanabilirsin. Parantezler önceliği belirler:

Python – Karmaşık Koşul

```
1 can = 50
2 kalkan_aktif = True
3 zirh = 20
4 dusman_gucu = 40
5
6 # Hasar almaz mı?
7 if kalkan_aktif or (can > 0 and zirh >= dusman_gucu):
8     print("Güvendesin!")
9 else:
10    print("Dikkat, hasar alabilirsin!")
```

⚠ Dikkat: Operatör Önceliği

Karmaşık koşullarda **parantez kullan!** Python önce **not**, sonra **and**, en son **or** işler. Parantezler hem önceliği netleştirir hem okunabilirliği artırır.

5.4.5 Oyun Senaryosu: Yetenek Kontrolü

Bir büyü kullanma sistemi yapalım:

Python – Büyü Sistemi

```
1 mana = 30
2 buyucu_seviyesi = 5
3 hedef_secili = True
4 buz_buyusu_maliyeti = 25
5 gereken_seviye = 3
6
7 # Büyü kullanılabilir mi?
8 if mana >= buz_buyusu_maliyeti and buyucu_seviyesi >=
   ↳ gereken_seviye:
9     if hedef_secili:
10        print("Buz Fırtınası büyüsü kullanıldı!")
11        mana -= buz_buyusu_maliyeti
12    else:
13        print("Önce bir hedef seç!")
14 elif mana < buz_buyusu_maliyeti:
15    print(f"Yetersiz mana! Gereken: {buz_buyusu_maliyeti},
   ↳ Mevcut: {mana}")
16 else:
17    print(f"Bu büyü için seviye {gereken_seviye}
   ↳ gerekli!")
```

5.4.6 Pratik Örnekler

Oyunlarda sık kullanılan koşul kalıpları:

Python – Yaygın Koşul Kalıpları

```

1  # Aralık kontrolü (x, 0-100 arasında mı?)
2  deger = 75
3  if 0 <= deger <= 100:
4      print("Değer geçerli aralıkta")
5
6  # Çoklu değer kontrolü (düşman türü bunlardan biri mi?)
7  dusman_turu = "goblin"
8  if dusman_turu in ["goblin", "ork", "trol"]:
9      print("Yeşil düşman!")
10
11 # Boşluk kontrolü (liste boş mu?)
12 envanter = ["kılıç", "kalkan"]
13 if envanter: # Boş değilse True döner
14     print(f"Envanterde {len(envanter)} eşya var")
15
16 if not envanter: # Boşsa True döner
17     print("Envanter boş!")

```

💡 İpucu: Profesyonel İpucu

`in` operatörü çok kullanışlıdır. Bir değer listede, stringde veya sözlükte olup olmadığını kontrol eder. Çoklu karşılaştırma yerine `in` kullan!

5.5 Sonuç: Koşullu İfadeler

Bu ünite Python'da karar yapılarını öğrendik. Koşullu ifadeler, oyunların temel yapı taşlarından biridir—oyuncunun her eylemi, düşmanların davranışları ve oyun mekaniğinin tamamı koşullara dayanır.

Bir adım geriye çekilip düşünelim: Koşullu ifadeler olmadan oyun yazabilir miydik? Kesinlikle hayır. Oyuncunun bir düğmeye bastığında zıplamasını, canı bittiğinde “Game Over” ekranının çıkmasını, yeterli altını varsa dükkândan eşya alabilmesini—tüm bunları koşullu ifadeler sağlar. Aslında programlamadaki en temel soru “Eğer ... ise ne olacak?” sorusudur ve bu soruyu `if` ile yanıtlandırırız.

5.5.1 Öğrendiklerimiz

Bu ünite beş temel kavram öğrendik. Her birinin oyun geliştirmede özel bir rolü var:

if ifadesi, tek bir koşulu kontrol eder ve kodun belirli bölümlerinin yalnızca belirli durumlarda çalışmasını sağlar. Koşul doğruysa içindeki kod çalışır, değilse tamamen atlanır. Python'da bu yapının çalışması için girintinin (4 boşluk) doğru kullanılması zorunludur.

elif ifadesi, birden fazla seçenek arasından seçim yapmamızı sağlar. Örneğin bir oyuncunun seviyesine göre “Çaylak”, “Deneyimli” veya “Usta” gibi farklı rütbelere atamak istiyorsan, her seviye aralığı için bir `elif` yazarsın. Önemli nokta şu: Python koşulları yukarıdan aşağıya kontrol eder ve ilk doğru koşulu

bulduğunda diğerlerini atlar.

else ifadesi, hiçbir koşul sağlanmadığında devreye giren varsayılan durumdur. Bir “güvenlik ağı” olarak düşünebilirsin: beklenmedik değerler geldiğinde programın ne yapacağını belirler.

İç içe koşullar, bir koşulun içinde başka koşullar tanımlamamızı sağlar. Örneğin “Oyuncu düşmana çarptı mı?” sorusunun cevabı evetse, “Kalkanı var mı?” diye ikinci bir kontrol yaparsın. Her seviye ek girinti gerektirir; bu da kodu karmaşıklaştırabilir, bu yüzden iç içe koşullar yerine mümkünse mantıksal operatörleri tercih etmelisin.

Mantıksal operatörler (**and**, **or**, **not**), birden fazla koşulu tek bir satırda birleştirmeni sağlar. **and** tüm koşulların doğru olmasını ister, **or** en az birinin doğru olmasını yeterli görür, **not** ise koşulun tersini alır.

5.5.2 Hızlı Referans

Aşağıdaki kod bloğu, bu ünite de öğrendiğin tüm koşullu ifade kalıplarını özetlemektedir. Bir sonraki ünite de döngülerle çalışırken bu yapıları sıkça kullanacaksın, bu yüzden bu referansa geri dönmekten çekinme:

Python – Koşullu İfadeler Özeti

```

1  # Basit if
2  if koşul:
3      # kod
4
5  # if-else
6  if koşul:
7      # doğruysa
8  else:
9      # yanlışsa
10
11 # if-elif-else
12 if koşul1:
13     # koşul1 doğruysa
14 elif koşul2:
15     # koşul2 doğruysa
16 else:
17     # hiçbirisi değilse
18
19 # Mantıksal operatörler
20 if koşul1 and koşul2: # ikisi de doğru
21 if koşul1 or koşul2:  # biri doğru
22 if not koşul:        # koşul yanlışsa

```

5.5.3 Sık Yapılan Hatalar

Koşullu ifadeler yazarken en sık karşılaşılan hatalar ve bunlardan nasıl kaçınılacağı:

⚠ Dikkat: Dikkat!

1. **= yerine ==:** Tek eşittir (=) atama, çift eşittir (==) karşılaştırma yapar. `if skor = 100:` yazmak hata verir; doğrusu `if skor == 100:` şeklindedir.
2. **Girinti unutmama:** Python'da girinti sözdiziminin bir parçasıdır. Girintiyi unutmak veya yanlış yapmak `IndentationError` hatasına yol açar. Her zaman 4 boşluk kullan.
3. **: unutmama:** `if`, `elif` ve `else` satırlarının sonuna iki nokta (:) koymayı unutmak `SyntaxError` üretir.
4. **Koşul sırası:** `elif` kullanırken en kısıtlayıcı koşulu üste koy. Örneğin “puan >= 50” kontrolü “puan >= 90” kontrolünden önce gelirse, 95 puanlık bir oyuncu da “50+” grubuna düşer.

5.5.4 Alıştırma Fikirleri

Öğrendiklerini pekiştirmek için aşağıdaki egzersizleri dene. Her biri bu ünitenin farklı bir kavramını pratiğe dökcektir:

1. **Not sistemi:** Girilen puana göre harf notu hesapla (90+: A, 80+: B, ...). Bu egzersiz `if-elif-else` yapısını ve koşul sıralamasını pekiştirir.
2. **Yaş kontrolü:** Yaşa göre “çocuk”, “genç”, “yetişkin” veya “kıdemli” yazdır. Sınır değerlerini (>=, <) doğru kullanmaya dikkat et.
3. **Hava durumu sistemi:** Sıcaklığa göre “çok soğuk”, “soğuk”, “ılık”, “sıcak” veya “çok sıcak” mesajı ver. Beşten fazla koşul olduğunda `elif` zincirinin nasıl uzadığını gözlemle.
4. **Login sistemi:** Kullanıcı adı ve şifre kontrolü yap. Bu egzersiz `and` operatörü ve iç içe koşulları birlikte kullanmak için iyi bir pratiktir.

● Bilgi: Bir Sonraki Ünite

Bir sonraki ünite **döngüler** öğreneceğiz. Döngüler, aynı kodu tekrar tekrar çalıştırmamızı sağlar—oyun döngüsünün (game loop) temelini oluşturur!

6

Döngüler

“Oyunlar tekrarlardan oluşur: her kare yeniden çizilir, her düşman yeniden kontrol edilir.”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `while` döngüsü ile koşula bağlı tekrar
- `for` döngüsü ile koleksiyon üzerinde gezinme
- `range()` fonksiyonu ile sayı dizileri oluşturma
- `break` ile döngüden çıkma
- `continue` ile iterasyonu atlama

6.1 While Döngüsü: Koşul Doğruyken Tekrarla

Oyunların kalbi, sürekli tekrar eden bir döngüdür: Ekranı temizle, girdileri oku, oyun durumunu güncelle, ekrana çiz, tekrarla... Bu “oyun döngüsü” (game loop) aslında bir `while` döngüsüdür. Bu ünite de döngülerin gücünü keşfedeceğiz.

■ Tanım: Döngü

Döngü, belirli bir kod bloğunu birden fazla kez çalıştıran yapıdır. Oyunlarda her frame yeniden çizilir, her düşman yeniden kontrol edilir—hepsi döngüler sayesinde.

6.1.1 While Döngüsünün Yapısı

`while` döngüsü, koşul doğru olduğu sürece tekrar eder:

Python – While Temel Yapısı

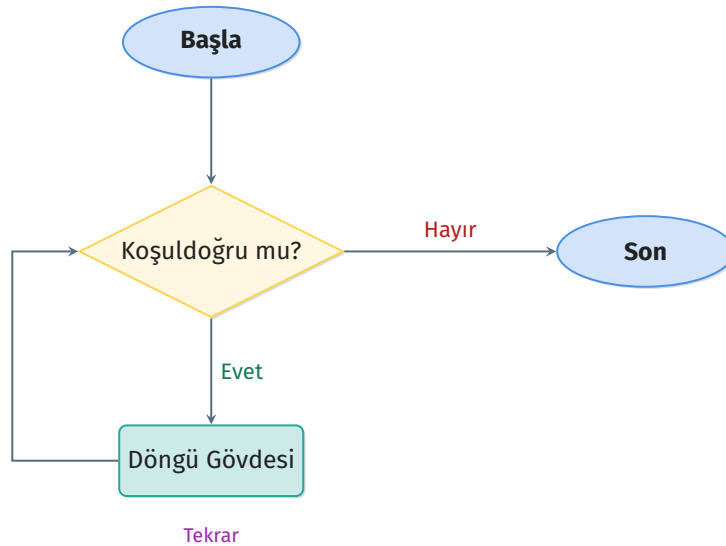
```
1 while koşul:
2     # Koşul doğru olduğu sürece çalışır
3     print("Döngü devam ediyor...")
```

⚠ Dikkat: Sonsuz Döngü Tehlikesi!

Koşul hiç yanlış olmazsa döngü sonsuza kadar devam eder ve program takılır! Her zaman döngüyü sonlandıracak bir mekanizma olmalı.

6.1.2 While Döngüsü Akış Diyagramı

While döngüsünün nasıl çalıştığını Şekil 6.1 şeklinde görebilirsin:



Şekil 6.1: While döngüsü akış diyagramı: Koşul doğru olduğu sürece döngü devam eder

Diyagramı adım adım okuyalım:

1. Program **while** satırına geldiğinde önce koşulu kontrol eder (elmas şekli).
2. Koşul **doğruysa**, döngü gövdesindeki kod bloğu (dikdörtgen) çalışır.
3. Kod bloğu bittikten sonra program tekrar koşul kontrolüne döner—bu “geri ok” döngünün tekrarlayan doğasını gösterir.
4. Koşul **yanlış** olduğunda döngüden çıkılır ve program sonraki satırdan devam eder.

Bir oyun senaryosuyla düşünelim: “Oyuncu hayattayken oyun devam etsin.” Bu cümle doğrudan **while oyuncu_can > 0:** şeklinde koda çevrilir. Her turda oyuncu hasar alır, can azalır ve koşul tekrar kontrol edilir. Can sıfıra düştüğü anda koşul yanlış olur ve döngü sona erer—yani oyun biter.

Dikkat edersen, koşul döngünün **başında** kontrol edilir. Bu, koşul en baştan yanlışsa döngü gövdesinin **hiç çalışmayacağı** anlamına gelir. Örneğin **can = 0** ile başlarsak, **while can > 0:** döngüsüne hiç girilmez.

6.1.3 Basit Örnekler

Geri sayım yapalım:

Python – Geri Sayım

```
1 sayac = 5
2
3 while sayac > 0:
4     print(f"Kalkışa {sayac} saniye...")
5     sayac -= 1 # sayac = sayac - 1
6
7 print("Kalkış!")
```

Bu kod şu çıktıyı verir:

>_ Çıktı

```
Kalka 5 saniye...
Kalka 4 saniye...
Kalka 3 saniye...
Kalka 2 saniye...
Kalka 1 saniye...
Kalk!
```

6.1.4 Oyun Senaryosu: Basit Oyun Döngüsü

Bir oyun döngüsünün iskeletini yazalım:

Python – Basit Oyun Döngüsü

```
1 oyun_calistir = True
2 tur_sayisi = 0
3
4 while oyun_calistir:
5     tur_sayisi += 1
6     print(f"\n--- Tur {tur_sayisi} ---")
7
8     # Oyuncu aksiyonu
9     aksiyon = input("Ne yapmak istersin? (devam/cik): ")
10
11     if aksiyon == "cik":
12         oyun_calistir = False
13         print("Oyundan çıkılıyor...")
14
15 print(f"Toplam {tur_sayisi} tur oynadın!")
```

● Bilgi: Oyun Döngüsü Mantığı

Gerçek oyunlarda döngü şu adımları tekrar eder:

1. Girdileri al (klavye, fare)
2. Oyun durumunu güncelle (hareket, çarpışma)
3. Ekrana çiz
4. Tekrarla (saniyede 60 kez!)

6.1.5 Sayaç Kullanımı

Döngülerde sıklıkla sayaç kullanılır:

Python – Can Azaltma

```

1 can = 100
2 darbe_sayisi = 0
3
4 while can > 0:
5     hasar = 15
6     can -= hasar
7     darbe_sayisi += 1
8     print(f"Darbe {darbe_sayisi}: Can = {can}")
9
10 print(f"Oyuncu {darbe_sayisi} darbede yenildi!")

```

6.1.6 Kullanıcı Girişi Doğrulama

Geçerli giriş alana kadar döngü:

Python – Giriş Doğrulama

```

1 gecerli_secenekler = ["1", "2", "3"]
2 secim = ""
3
4 while secim not in gecerli_secenekler:
5     print("Menü:")
6     print("1. Yeni Oyun")
7     print("2. Devam Et")
8     print("3. Çıkış")
9     secim = input("Seçimin (1-3): ")
10
11     if secim not in gecerli_secenekler:
12         print("Geçersiz seçim! Tekrar dene.\n")
13
14 print(f"{secim} seçildi!")

```

6.1.7 While-Else Yapısı

Python'da **while** döngüsüne **else** eklenebilir. Döngü normal şekilde biterse (break olmadan) **else** çalışır:

Python – While-Else

```

1 deneme = 3
2
3 while deneme > 0:
4     sifre = input("Şifre: ")
5     if sifre == "gizli123":
6         print("Giriş başarılı!")
7         break
8     deneme -= 1
9     print(f"Yanlış! {deneme} hak kaldı.")
10 else:
11     # Döngü break olmadan bittiyse
12     print("Çok fazla yanlış deneme. Hesap kilitlendi!")

```

💡 İpucu: Profesyonel İpucu

`while True:` şeklinde sonsuz döngü oluşturup içeride `break` ile çıkmak yaygın bir kalıptır. Oyun döngüleri genellikle bu şekilde yazılır.

6.2 For Döngüsü: Koleksiyon Üzerinde Gezinme

Oyuncu envanterindeki tüm eşyaları göstermek, ekrandaki tüm düşmanları güncellemek, haritadaki tüm tile'ları çizmek... Bunların hepsinde `for` döngüsü kullanılır. `for` döngüsü, bir koleksiyonun (liste, string, vb.) her elemanı üzerinde işlem yapar.

■ Tanım: For Döngüsü

`for` döngüsü, bir koleksiyonun her elemanını sırayla alır ve belirtilen işlemi uygular. `while`'dan farkı: kaç kez döneceği koleksiyonun boyutuna göre otomatik belirlenir.

6.2.1 For Döngüsünün Yapısı

Python – For Temel Yapısı

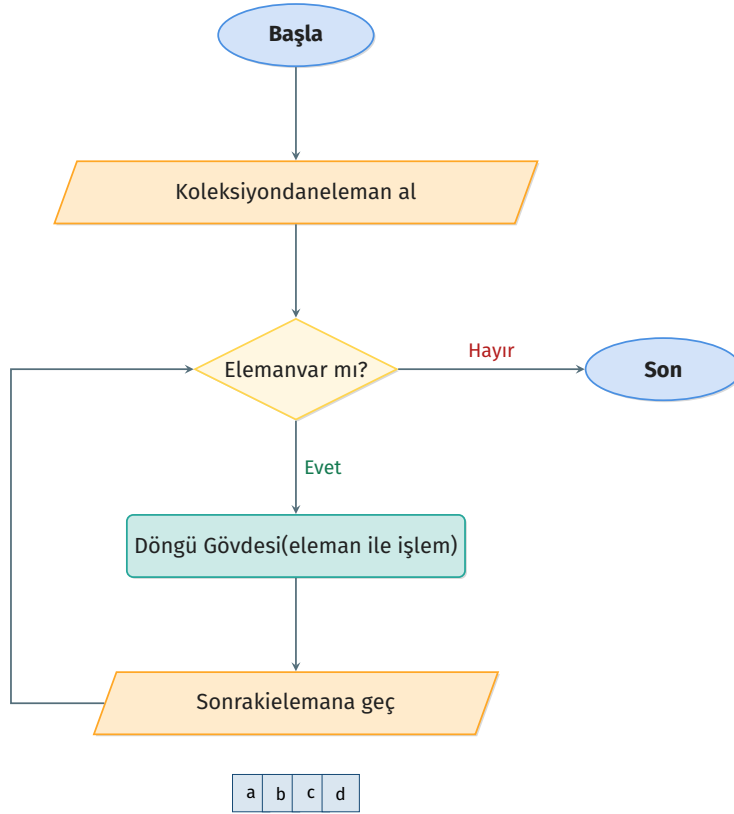
```

1 for eleman in koleksiyon:
2     # Her eleman için çalışır
3     print(eleman)

```

6.2.2 For Döngüsü Akış Diyagramı

For döngüsünün çalışma mantığını Şekil 6.2 şeklinde görebilirsin. Koleksiyondaki her eleman için döngü gövdesi çalıştırılır.



Şekil 6.2: For döngüsü akış diyagramı: Koleksiyondaki her eleman için işlem yapılır

6.2.3 Liste Üzerinde Döngü

En yaygın kullanım, liste elemanları üzerinde gezinmektir:

Python – Envanter Listesi

```

1 envanter = ["kılıç", "kalkan", "iksir", "anahtar"]
2
3 print("Envanterin:")
4 for esya in envanter:
5     print(f" - {esya}")

```

>_ Çıktı

```

Envanterin:
- klç
- kalkan
- iksir
- anahtar

```

6.2.4 String Üzerinde Döngü

String de bir koleksiyondur—karakterler üzerinde gezinebilirsin:

Python – Harfleri Yazdırma

```

1 oyuncu_adi = "HERO"
2
3 for harf in oyuncu_adi:
4     print(f"[{harf}]", end=" ")
5 # Çıktı: [H] [E] [R] [O]

```

6.2.5 Oyun Senaryosu: Düşman Güncellemesi

Tüm düşmanları güncelleme örneği:

Python – Düşman Güncelleme

```

1 dusmanlar = [
2     {"isim": "Goblin", "can": 30},
3     {"isim": "Ork", "can": 50},
4     {"isim": "Trol", "can": 100}
5 ]
6
7 print("Düşman Durumu:")
8 for dusman in dusmanlar:
9     print(f"{dusman['isim']}: {dusman['can']} HP")

```

6.2.6 enumerate() ile İndeks Alma

Bazen elemanın indeksini de bilmek isteriz:

Python – enumerate Kullanımı

```

1 silahlar = ["Kılıç", "Balta", "Yay", "Asa"]
2
3 print("Silah Seç:")
4 for indeks, silah in enumerate(silahlar):
5     print(f"{indeks + 1}. {silah}")

```

>_ Çıktı

Silah Seç:

1. Kılıç
2. Balta
3. Yay
4. Asa

 Bilgi: enumerate Neden Önemli?

`enumerate()` hem elemanı hem indeksi verir. Menü sistemleri, envanter yönetimi ve liste işlemlerinde çok kullanışlıdır.

6.2.7 İç İçe For Döngüsü

2D oyun haritası gibi durumlar için iç içe döngüler kullanırız:

Python – 2D Grid Çizimi

```

1  satir_sayisi = 3
2  sutun_sayisi = 5
3
4  for satir in range(satir_sayisi):
5      for sutun in range(sutun_sayisi):
6          print(".", end=" ")
7      print() # Satır sonu

```

>_ Çıktı

```

. . . . .
. . . . .
. . . . .

```

6.2.8 Oyun Senaryosu: Hasar Hesaplama

Tüm düşmanlara alan hasarı verme:

Python – Alan Hasarı

```

1  dusmanlar = [
2      {"isim": "Goblin 1", "can": 30},
3      {"isim": "Goblin 2", "can": 25},
4      {"isim": "Ork", "can": 50}
5  ]
6
7  alan_hasari = 20
8
9  print("Ateş Topu büyü kullanıldı!")
10 for dusman in dusmanlar:
11     dusman["can"] -= alan_hasari
12     if dusman["can"] <= 0:
13         print(f" {dusman['isim']} yok edildi!")
14     else:
15         print(f" {dusman['isim']}: {dusman['can']} HP
16             ↪ kaldı")

```

6.2.9 For vs While Karşılaştırması

Tablo 6.1: For ve While Karşılaştırması

Özellik	for	while
Kullanım	Bilinen tekrar sayısı	Belirsiz tekrar sayısı
Koşul	Koleksiyon bitene kadar	Koşul yanlış olana kadar
Sayaç	Otomatik	Manuel yönetim
Örnek	Listeyi gez	Kullanıcı “çık” yazana kadar

İpucu: Profesyonel İpucu

“Tüm X’ler için Y yap” diyorsan **for** kullan. “Z olana kadar devam et” diyorsan **while** kullan.

6.3 Range Fonksiyonu: Sayı Dizileri Oluşturma

Bazen bir listeye ihtiyacın yok, sadece belirli sayıda döngü yapmak istiyorsun. İşte **range()** fonksiyonu tam da bunun için var. Oyunlarda seviye döngüleri, mermi spawnu, animasyon frame’leri gibi sayısal işlemler için çok kullanılır.

Tanım: range() Fonksiyonu

range() fonksiyonu, belirtilen aralıkta sayılar üreten bir nesnedir. Bellekte tüm sayıları tutmaz, sadece gerektiğinde üretir—bu yüzden çok verimlidir.

6.3.1 Temel Kullanım

range(n) sıfırdan n-1’e kadar sayılar üretir:

Python – range() Temel

```
1 # 0'dan 4'e kadar (5 dahil değil)
2 for i in range(5):
3     print(i)
4 # Çıktı: 0, 1, 2, 3, 4
```

6.3.2 Başlangıç ve Bitiş Belirtme

range(baslangic, bitis) şeklinde aralık belirleyebilirsiniz:

Python – Aralık Belirtme

```
1 # 1'den 5'e kadar (5 dahil değil)
2 for i in range(1, 6):
3     print(i)
```

```
4 # Çıktı: 1, 2, 3, 4, 5
```

● Bilgi: Neden Bitiş Dahil Değil?

Python'da aralıklar “yarı açık” (half-open) olarak tanımlanır. Bu tasarım `len()` ile uyumludur: `range(len(liste))` liste boyutu kadar döner.

6.3.3 Adım Belirleme

`range(baslangic, bitis, adım)` ile atlama miktarı belirlersin:

Python – Adım Kullanımı

```
1 # 2'şer 2'şer say
2 for i in range(0, 10, 2):
3     print(i)
4 # Çıktı: 0, 2, 4, 6, 8
5
6 # Geriye say
7 for i in range(10, 0, -1):
8     print(i)
9 # Çıktı: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

6.3.4 Oyun Senaryosu: Seviye Spawn

Her 10 puanda bir seviye atlama:

Python – Seviye Sistemi

```
1 max_puan = 100
2
3 print("Seviye Eşikleri:")
4 for puan in range(0, max_puan + 1, 10):
5     seviye = puan // 10 + 1
6     print(f" {puan} puan = Seviye {seviye}")
```

6.3.5 Oyun Senaryosu: Mermi Oluşturma

Birden fazla mermi spawn etme:

Python – Çoklu Mermi

```
1 mermi_sayisi = 5
2
3 print(f"{mermi_sayisi} mermi ateşleniyor:")
4 for i in range(mermi_sayisi):
5     aci = i * 30 # Her mermi 30 derece farklı açıda
6     print(f" Mermi {i+1}: {aci}° açıyla ateşlendi")
```

6.3.6 Range ile Liste Oluşturma

`range()`'i listeye çevirebilirsiniz:

Python – Liste Oluşturma

```
1 # 1'den 10'a kadar sayılar listesi
2 sayilar = list(range(1, 11))
3 print(sayilar) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4
5 # Çift sayılar
6 cift_sayilar = list(range(2, 21, 2))
7 print(cift_sayilar) # [2, 4, 6, 8, 10, 12, 14, 16, 18,
  ↪ 20]
```

6.3.7 Oyun Senaryosu: 2D Grid Koordinatları

Harita hücrelerinin koordinatlarını oluşturma:

Python – Grid Koordinatları

```
1 harita_genislik = 4
2 harita_yukseklik = 3
3
4 print("Harita Koordinatları:")
5 for y in range(harita_yukseklik):
6     for x in range(harita_genislik):
7         print(f"({x},{y})", end=" ")
8     print()
```

> Çıktı

```
Harita Koordinatlar:
(0,0) (1,0) (2,0) (3,0)
(0,1) (1,1) (2,1) (3,1)
(0,2) (1,2) (2,2) (3,2)
```

6.3.8 Range Özet Tablosu

Tablo 6.2: `range()` Kullanım Örnekleri

Kullanım	Üretilen Sayılar
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(1, 6)</code>	1, 2, 3, 4, 5
<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1
<code>range(10, 0, -2)</code>	10, 8, 6, 4, 2

💡 İpucu: Profesyonel İpucu

Oyunlarda frame numarası, tile indeksi, seviye sayısı gibi durumlar için `range()` vazgeçilmezdir. PyGame'de animasyon frame'leri genellikle `for i in range(frame_sayisi)` şeklinde işlenir.

6.4 Break ve Continue: Döngü Kontrolü

Bazen döngüyü erken sonlandırmak veya bir iterasyonu atlamak isteriz. Örneğin: aranan düşman bulununca aramayı durdur, veya ölü düşmanları atla. İşte `break` ve `continue` komutları bu kontrolü sağlar.

■ Tanım: Döngü Kontrol Komutları

- `break`: Döngüyü tamamen sonlandırır
- `continue`: Mevcut iterasyonu atlar, sonraki iterasyona geçer

6.4.1 Break: Döngüyü Sonlandır

`break` komutu döngüden hemen çıkar:

Python – Break Örneği

```

1  # İlk düşmanı bul ve dur
2  dusmanlar = ["goblin", "ork", "trol", "ejderha"]
3  aranan = "trol"
4
5  for dusman in dusmanlar:
6      print(f"Kontrol ediliyor: {dusman}")
7      if dusman == aranan:
8          print(f"{aranan} bulundu!")
9          break
10
11 print("Arama bitti.")

```

>_ Çıktı

```

Kontrol ediliyor: goblin
Kontrol ediliyor: ork
Kontrol ediliyor: trol
trol bulundu!
Arama bitti.

```


6.4.2 Oyun Senaryosu: İlk Çarpışma Tespiti

Python – Çarpışma Tespiti

```

1 oyuncu_x = 100
2 engeller = [50, 75, 100, 150, 200]
3
4 for engel in engeller:
5     if oyuncu_x == engel:
6         print(f"Çarpışma! Engel pozisyonu: {engel}")
7         break
8 else:
9     print("Çarpışma yok, yol açık!")

```

● Bilgi: for-else Yapısı

`for` döngüsüne de `else` eklenebilir. Döngü `break` olmadan tamamlanırsa `else` çalışır. “Bulamadım” durumlarını yakalamak için idealdir.

6.4.3 Continue: İterasyonu Atla

`continue` komutu mevcut iterasyonu atlar:

Python – Continue Örneği

```

1 # Sadece canlı düşmanları işle
2 dusmanlar = [
3     {"isim": "Goblin", "can": 0},      # Ölü
4     {"isim": "Ork", "can": 30},       # Canlı
5     {"isim": "Trol", "can": 0},       # Ölü
6     {"isim": "Ejderha", "can": 100}   # Canlı
7 ]
8
9 print("Canlı düşmanlar:")
10 for dusman in dusmanlar:
11     if dusman["can"] <= 0:
12         continue # Ölü düşmanı atla
13
14     print(f" {dusman['isim']}: {dusman['can']} HP")

```

> Çıktı

```

Canlı dümanlar:
Ork: 30 HP
Ejderha: 100 HP

```

6.4.4 Oyun Senaryosu: Envanter Filtreleme

Python – Kullanılabilir Eşyalar

```

1 envanter = [
2     {"isim": "Kırık Kılıç", "kullanilabilir": False},
3     {"isim": "Sağlık İksiri", "kullanilabilir": True},
4     {"isim": "Paslanmış Zırh", "kullanilabilir": False},
5     {"isim": "Ateş Topu Parşömeni", "kullanilabilir":
6         True}
7 ]
8 print("Kullanılabilir Eşyalar:")
9 for esya in envanter:
10     if not esya["kullanilabilir"]:
11         continue
12
13     print(f" - {esya['isim']}")

```

6.4.5 While ile Break Kullanımı

`break`, `while True` ile birlikte çok yaygındır:

Python – Oyun Ana Döngüsü

```

1 import random
2
3 print("Sayı Tahmin Oyunu!")
4 gizli_sayi = random.randint(1, 10)
5
6 while True:
7     tahmin = int(input("Tahminin (1-10): "))
8
9     if tahmin == gizli_sayi:
10         print("Tebrikler, bildin!")
11         break
12     elif tahmin < gizli_sayi:
13         print("Daha büyük!")
14     else:
15         print("Daha küçük!")

```

6.4.6 İç İç Döngülerde Break

`break` sadece en içteki döngüden çıkar:

Python – İç İç Break

```

1 # 2D haritada hazine ara
2 harita = [
3     [".", ".", "."],

```

```

4     [".", "X", "."], # X = Hazine
5     [".", ".", "."]
6 ]
7
8 hazine_bulundu = False
9 for y, satir in enumerate(harita):
10     for x, hucre in enumerate(satir):
11         if hucre == "X":
12             print(f"Hazine bulundu: ({x}, {y})")
13             hazine_bulundu = True
14             break
15     if hazine_bulundu:
16         break

```

⚠ Dikkat: İç İç Break

İç içe döngülerde `break` sadece bir döngüyü kırar. Tüm döngülerden çıkmak için bayrak değişkeni kullan veya fonksiyona al.

6.4.7 Özet: Ne Zaman Hangisi?

Tablo 6.3: Break vs Continue

Komut	Ne Yapar?	Kullanım Örneği
<code>break</code>	Döngüyü sonlandırır	Aranan bulundu, dur
<code>continue</code>	İterasyonu atlar	Geçersiz ögeyi atla

💡 İpucu: Profesyonel İpucu

Çok fazla `break` ve `continue` kodu karmaşık hale getirebilir. Mümkünse döngü koşulunu veya if yapısını düzenleyerek bunları azalt.

6.5 Sonuç: Döngüler

Bu ünite de döngülerini öğrendik. Döngüler, oyun geliştirmenin temel yapı taşıdır—her oyun, saniyede 60 kez tekrar eden devasa bir döngüdür! Frame çizimi, düşman güncellemesi, çarpışma kontrolü... hepsi döngüler sayesinde gerçekleşir.

Önceki ünite de öğrendiğimiz koşullu ifadeler “ne yapılacağına” karar veriyordu. Döngüler ise “ne kadar süre yapılacağını” belirler. Bu iki kavram birlikte çalıştığında ortaya güçlü programlar çıkar: Bir döngü içinde koşul kontrolü yaparak “düşman hayatta olduğu sürece saldırmaya devam et” gibi karmaşık davranışları kolayca modelleyebilirsiniz.

6.5.1 Öğrendiklerimiz

Bu ünite de dört temel kavram öğrendik. Hangi durumda hangisini kullanacağını bilmek, yazdığın kodun hem doğru hem de okunabilir olmasını sağlar.

while döngüsü, koşul doğru olduğu sürece devam eden bir tekrarlama yapısıdır. Ne zaman duracağını önceden bilmediğin durumlarda—örneğin “oyuncu çıkış yapana kadar” veya “can sıfıra düşene kadar”—while tercih edilir. Ancak dikkatli ol: koşulu hiç güncellemezsen döngü sonsuza kadar devam eder ve programın kilitlenir.

for döngüsü, bir koleksiyon (liste, string, range) üzerinde baştan sona gezinmek için tasarlanmıştır. Eleman sayısı belli olduğunda for kullanılır: “tüm düşmanları güncelle”, “envanterdeki her eşyayı göster” gibi. `enumerate()` ile hem elemanı hem indeksini alabilir, iç içe for döngüleri ile 2D grid gibi yapıları işleyebilirsin.

range() fonksiyonu, belirli bir aralıkta sayı dizisi üretir. `range(5)` 0’dan 4’e kadar, `range(2, 8)` 2’den 7’ye kadar, `range(0, 20, 3)` ise 0’dan başlayıp 3’er 3’er artarak sayı üretir. Bitiş değerinin diziye dahil olmadığını unutma!

break ve continue, döngü akışını kontrol eden iki güçlü araçtır. **break** döngüyü tamamen sonlandırır—örneğin aranan eşya bulunduğunda aramayı durdurur. **continue** ise mevcut iterasyonu atlayıp bir sonrakine geçer—örneğin ölü düşmanları güncelleme adımından atlamak için kullanılır.

6.5.2 While mı, For mu?

Bu iki döngü türü arasında doğru seçim yapmak önemlidir:

- Kaç kez tekrar edeceğini biliyorsan → **for** kullan
- Bir koşula bağlıysa (ne zaman biteceği belirsiz) → **while** kullan
- Bir koleksiyon üzerinde geziniyorsan → **for** kullan
- Kullanıcı girişi beklerken → **while** kullan

Oyun geliştirmede ana oyun döngüsü genellikle **while** ile yazılır (çünkü oyuncunun ne zaman çıkacağı belirsizdir), ancak döngü içindeki işlemler—tüm düşmanları güncelleme, tüm nesneleri çizme—**for** ile yapılır.

6.5.3 Hızlı Referans

Python – Döngüler Özeti

```

1  # While döngüsü
2  while koşul:
3      # kod
4
5  # For döngüsü
6  for eleman in liste:
7      # kod
8
9  # Range ile for
10 for i in range(10):      # 0-9
11 for i in range(1, 11):  # 1-10

```

```

12 for i in range(0, 10, 2): # 0, 2, 4, 6, 8
13
14 # Enumerate
15 for i, eleman in enumerate(liste):
16     print(i, eleman)
17
18 # Döngü kontrolü
19 break # Döngüden çık
20 continue # Sonraki iterasyona geç

```

6.5.4 Oyun Döngüsü Kalıbı

Tüm oyunların temelindeki yapı aşağıdaki gibidir. Bu kalıp, ileride PyGame ile çalışırken de birebir aynı mantıkta kullanılacak:

Python – Temel Oyun Döngüsü

```

1 oyun_aktif = True
2
3 while oyun_aktif:
4     # 1. Girdileri işle
5     for olay in olaylar:
6         if olay == "cikis":
7             oyun_aktif = False
8
9     # 2. Oyun durumunu güncelle
10    for dusman in dusmanlar:
11        dusman.hareket_et()
12
13    # 3. Ekrana çiz
14    for nesne in tum_nesneler:
15        nesne.ciz()

```

Bu kalıpta dikkat edilmesi gereken üç aşama var. **Birincisi**, girdilerin işlenmesi: Oyuncunun bastığı tuşlar, fare hareketleri ve “Pencereyi kapat” gibi sistem olayları bir **for** döngüsüyle tek tek kontrol edilir. **İkincisi**, oyun durumunun güncellenmesi: Düşmanlar hareket eder, mermiler ilerler, çarpışmalar kontrol edilir—yine **for** döngüleriyle her nesne üzerinden geçilir. **Üçüncüsü**, çizim aşaması: Güncellenen duruma göre tüm nesneler ekrana çizilir. Bu üç adım, dıştaki **while** döngüsü sayesinde saniyede onlarca kez tekrar eder ve böylece akıcı bir oyun deneyimi oluşur.

6.5.5 Sık Yapılan Hatalar

Döngülerle çalışırken en sık karşılaşılan hatalar ve bunlardan kaçınma yolları:

⚠ Dikkat: Dikkat!

1. **Sonsuz döngü:** `while` koşulunu döngü içinde güncellemeyi unutursan program kilitlenir. Her `while` döngüsünde “bu koşul ne zaman yanlış olacak?” sorusunu kendine sor.
2. **range bitiş değeri:** `range(5)` 0’dan 4’e kadar üretir, 5 dahil değildir! 1’den 5’e kadar istiyorsan `range(1, 6)` yazmalısın.
3. **Döngü içinde liste değiştirme:** Bir liste üzerinde `for` ile gezerken o listeden eleman silmek beklenmedik sonuçlara yol açar. Bunun yerine listenin bir kopyası üzerinde gez veya silinecekleri ayrı bir listede topla.
4. **İç içe döngülerde break:** `break` yalnızca bulunduğu en içteki döngüyü kırar, dıştaki döngüyü etkilemez. Dıştaki döngüden de çıkmak istiyorsan bir bayrak değişkeni (flag) kullan.

6.5.6 Alıştırma Fikirleri

Aşağıdaki egzersizlerin her biri döngülerin farklı bir yönünü pratiğe dökecektir:

1. **Çarpım tablosu:** 1’den 10’a kadar çarpım tablosu yazdır. İç içe `for` döngüsü ve `range()` kullanımını pekiştirir.
2. **FizzBuzz:** 1-100 arası sayıları yazdır; 3’e bölünenler “Fizz”, 5’e bölünenler “Buzz”, ikisine de bölünenler “FizzBuzz” olsun. Döngü + koşul kombinasyonunu pratiğe döker.
3. **Asal sayı kontrolü:** Girilen sayının asal olup olmadığını kontrol et. `for` döngüsü ve `break` kullanımını pekiştirir.
4. **Zar oyunu:** 2 zar at, toplamı 7 gelene kadar tekrarla ve kaç deneme gerektiğini yazdır. “Ne zaman biteceği belirsiz” senaryosunda `while` kullanımını pekiştirir.

● Bilgi: Bir Sonraki Ünite

Bir sonraki ünite **fonksiyonlar** öğreneceğiz. Fonksiyonlar, kodu tekrar kullanılabilir parçalara böler—oyunlarda `ciz()`, `hareket_et()`, `carpisti_mi()` gibi fonksiyonlar sürekli kullanılır!

7

Fonksiyonlar

“Fonksiyonlar, kodun LEGO parçalarıdır. Bir kez yap, istediğin yerde kullan!”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `def` anahtar kelimesi ile fonksiyon tanımlama
- Parametreler ve argümanlar
- `return` ile değer döndürme
- Varsayılan parametre değerleri
- Değişken kapsamı (scope) kavramı

7.1 Fonksiyon Tanımlama: Kodun LEGO Parçaları

Bir oyunda karakterin zıplaması, düşmanın saldırması, mermi atılması... Bu işlemler oyun boyunca defalarca yapılır. Her seferinde aynı kodu yazmak yerine, bu işlemleri **fonksiyonlara** paketleriz. Bir kez yaz, istediğin yerde kullan!

■ Tanım: Fonksiyon

Fonksiyon, belirli bir görevi yerine getiren, isimlendirilmiş kod bloğudur. Çağrıldığında içindeki kod çalışır. LEGO parçası gibi düşün: bir kez yap, projenin her yerinde kullan.

7.1.1 Fonksiyon Tanımlama: `def`

Python’da fonksiyon `def` anahtar kelimesiyle tanımlanır:

Python – Fonksiyon Tanımlama

```
1 def selamla():
2     print("Merhaba, Oyuncu!")
3     print("Maceraya hoş geldin!")
```

Dikkat edilmesi gerekenler:

- **def** anahtar kelimesiyle başlar
- Fonksiyon adı küçük harfle ve snake_case ile yazılır
- Addan sonra parantez () ve iki nokta :
- Fonksiyon gövdesi girintili yazılır

7.1.2 Fonksiyon Çağırma

Tanımlamak yetmez, fonksiyonu **çağır**malısın:

Python – Fonksiyon Çağırma

```
1 def selamla():
2     print("Merhaba, Oyuncu!")
3
4     # Fonksiyonu çağır
5     selamla() # Çıktı: Merhaba, Oyuncu!
6
7     # İstedikğin kadar çağır
8     selamla()
9     selamla()
```

Bilgi: Tanımlama vs Çağırma

Fonksiyon tanımlamak (def) onu çalıştırmaz, sadece hafızaya kaydeder. Çalıştırmak için fonksiyonu çağırmalısın (isim + parantez).

7.1.3 Neden Fonksiyon Kullanırız?

1. **Tekrar kullanım:** Aynı kodu birden fazla yerde kullan
2. **Organizasyon:** Kodu mantıksal parçalara böl
3. **Okunabilirlik:** `dusman_oldu()` yazmak, 10 satır kod yazmaktan iyidir
4. **Bakım kolaylığı:** Bir yerde değiştir, her yerde etkili olsun

7.1.4 Oyun Senaryosu: Oyun Fonksiyonları

Tipik bir oyunda kullanılan fonksiyonlar:

Python – Oyun Fonksiyonları

```
1 def oyunu_baslat():
2     print("=" * 40)
3     print("    UZAY MACERASI")
```



```

4     print("    Yıldızlara Yolculuk")
5     print("=" * 40)
6     print()
7
8     def menu_goster():
9         print("1. Yeni Oyun")
10        print("2. Devam Et")
11        print("3. Ayarlar")
12        print("4. Çıkış")
13
14    def oyun_bitti():
15        print()
16        print("GAME OVER")
17        print("Tekrar denemek ister misin?")
18
19    # Fonksiyonları kullan
20    oyunu_baslat()
21    menu_goster()

```

7.1.5 Fonksiyon İsimlendirme Kuralları

Tablo 7.1: Fonksiyon İsimlendirme

Kural	İyi	Kötü
snake_case kullan	hasar_hesapla()	HasarHesapla()
Fiil ile başla	ciz_oyuncu()	oyuncu_cizim()
Anlamli isim	dusman_oldu()	d()
Kısa ama açık	yukle()	y()

⚠ Dikkat: İsimlendirme Önemli!

Fonksiyon adı ne yaptığını anlatmalı. 6 ay sonra koda baktığında `abc()` ne yapıyor anlamazsın, ama `oyuncu_hareket_et()` çok açık!

7.1.6 Çoklu Fonksiyon Örneği

Python – Savaş Sistemi

```

1     def savas_baslat():
2         print("\n*** Savas basliyor! ***")
3         print("-" * 30)
4
5     def saldir():
6         print("Kılıcını salladın!")
7
8     def savun():
9         print("Kalkanını kaldırdın!")

```

```

10
11 def savas_bitti():
12     print("-" * 30)
13     print("Savaş sona erdi!\n")
14
15 # Savaş simülasyonu
16 savas_baslat()
17 saldır()
18 savun()
19 saldır()
20 savas_bitti()

```

💡 İpucu: Profesyonel İpucu

Her fonksiyon **tek bir iş** yapmalı. “Çiz ve hareket ettir” yerine `ciz()` ve `hareket_et()` olarak ayır. Bu prensibe “Single Responsibility” denir.

7.2 Parametreler ve Argümanlar: Fonksiyona Bilgi Aktarma

Şu ana kadar yazdığımız fonksiyonlar hep aynı şeyi yapıyordu. Peki ya farklı düşmanlara farklı hasar vermek istersen? Farklı oyunculara farklı mesaj göstermek istersen? İşte **parametreler** bu esnekliği sağlar.

■ Tanım: Parametre ve Argüman

- **Parametre:** Fonksiyon tanımında belirtilen değişken (placeholder)
- **Argüman:** Fonksiyon çağrılırken gönderilen gerçek değer

7.2.1 Tek Parametrelili Fonksiyon

Python – Tek Parametre

```

1 def selamla(isim):
2     print(f"Merhaba, {isim}!")
3
4 # Farklı argümanlarla çağır
5 selamla("Ahmet")      # Çıktı: Merhaba, Ahmet!
6 selamla("Zeynep")     # Çıktı: Merhaba, Zeynep!
7 selamla("Oyuncu1")    # Çıktı: Merhaba, Oyuncu1!

```

`isim` bir parametredir—fonksiyon her çağrıldığında farklı değer alabilir.

7.2.2 Birden Fazla Parametre

Virgülle ayırarak birden fazla parametre tanımlayabilirsiniz:

Python – Çoklu Parametre

```

1 def hasar_ver(hedef, miktar):
2     print(f"{hedef} {miktar} hasar aldı!")
3
4 hasar_ver("Goblin", 25)      # Goblin 25 hasar aldı!
5 hasar_ver("Ejderha", 100)   # Ejderha 100 hasar aldı!

```

⚠ Dikkat: Parametre Sırası

Argümanlar sırayla parametrelere atanır. `hasar_ver("Goblin", 25)` çağrısında “Goblin” `hedef`’e, 25 `miktar`’a gider.

7.2.3 İsimli Argümanlar (Keyword Arguments)

Parametre adını belirterek sıra önemsiz hale getirilebilir:

Python – İsimli Argüman

```

1 def karakter_olustur(isim, sinif, seviye):
2     print(f"{isim} - {sinif} (Seviye {seviye})")
3
4 # Sıralı çağrı
5 karakter_olustur("Kara Şövalye", "Savaşçı", 10)
6
7 # İsimli çağrı - sıra önemli değil
8 karakter_olustur(seviye=5, isim="Mavi Büyücü",
9                 ↪ sinif="Büyücü")

```

7.2.4 Varsayılan Parametre Değerleri

Parametrelere varsayılan değer atayabilirsin:

Python – Varsayılan Değer

```

1 def saldır(silah="yumruk", hasar=5):
2     print(f"{silah} ile saldırdın! {hasar} hasar!")
3
4 saldır()                                # yumruk ile saldırdın! 5
5 ↪ hasar!
6 saldır("kılıç")                         # kılıç ile saldırdın! 5
7 ↪ hasar!
8 saldır("balta", 15)                     # balta ile saldırdın! 15
9 ↪ hasar!
10 saldır(hasar=100)                       # yumruk ile saldırdın! 100
11 ↪ hasar!

```

● Bilgi: Varsayılan Değer Kuralı

Varsayılan değeri olan parametreler, varsayılan değeri olmayan parametrelerden sonra gelmelidir. `def fonk(a, b=5)` doğru, `def fonk(a=5, b)` hatalı!

7.2.5 Oyun Senaryosu: Esnek Karakter Sistemi

Python – Karakter Sistemi

```

1 def karakter_olustur(isim, sinif="Savaşçı", can=100,
   ↪   guc=10):
2     print(f"\n=== {isim} ===")
3     print(f"Sınıf: {sinif}")
4     print(f"Can: {can}")
5     print(f"Güç: {guc}")
6
7     # Farklı karakterler
8     karakter_olustur("Thorin")
9     karakter_olustur("Gandalf", sinif="Büyücü", guc=25)
10    karakter_olustur("Legolas", "Okçu", can=80, guc=15)

```

7.2.6 Oyun Senaryosu: Saldırı Fonksiyonu

Python – Gelişmiş Saldırı

```

1 def saldır(saldıran, hedef, hasar, kritik=False):
2     if kritik:
3         hasar *= 2
4         print(f"[KRITİK] KRİTİK VURUŞ!")
5
6     print(f"{saldıran} -> {hedef}: {hasar} hasar!")
7
8     # Kullanım örnekleri
9     saldır("Oyuncu", "Goblin", 20)
10    saldır("Oyuncu", "Ork", 20, kritik=True)
11    saldır(saldıran="Ejderha", hedef="Oyuncu", hasar=50)

```

7.2.7 Parametre Özeti

Tablo 7.2: Parametre Türleri

Tür	Örnek
Zorunlu parametre	<code>def f(x):</code>
Varsayılan değerli	<code>def f(x=10):</code>
Sıralı argüman	<code>f(5)</code>
İsimli argüman	<code>f(x=5)</code>

💡 İpucu: Profesyonel İpucu

Fonksiyon çok fazla parametre alıyorsa (5+), muhtemelen çok fazla iş yapıyordur. Fonksiyonu daha küçük parçalara bölmeyi düşün.

7.3 Return: Değer Döndürme

Şu ana kadar fonksiyonlarımız sadece ekrana yazdırıyordu. Peki hasar hesapladıktan sonra sonucu kullanmak istersen? Örneğin: “Düşmana verilen hasar 25 ise ve düşmanın canı 20 ise, düşman öldü.” Bu hesaplamalar için `return` ifadesini kullanırız.

■ Tanım: Return İfadesi

`return` ifadesi, fonksiyondan bir değer döndürür ve fonksiyonun çalışmasını sonlandırır. Dönen değer, fonksiyonun çağrıldığı yerde kullanılabilir.

7.3.1 Temel Return Kullanımı

Python – Return Örneği

```
1 def topla(a, b):
2     sonuc = a + b
3     return sonuc
4
5 # Dönen değeri kullan
6 cevap = topla(5, 3)
7 print(cevap) # Çıktı: 8
8
9 # Doğrudan kullan
10 print(topla(10, 20)) # Çıktı: 30
```

7.3.2 Return vs Print

Python – Print vs Return Farkı

```
1 def print_ile(x):
2     print(x * 2) # Sadece yazdırır
3
4 def return_ile(x):
5     return x * 2 # Değer döndürür
6
7 # Print ile
8 sonuc1 = print_ile(5) # Ekrana 10 yazar
9 print(sonuc1)         # None (çünkü return yok)
10
11 # Return ile
```

```

12 sonuc2 = return_ile(5) # Sessiz, değer döner
13 print(sonuc2)          # 10

```

⚠ Dikkat: Return Olmadan None

Return ifadesi olmayan fonksiyonlar **None** döndürür. Hesaplama yapan fonksiyonlarda mutlaka return kullan!

7.3.3 Oyun Senaryosu: Hasar Hesaplama

Python – Hasar Sistemi

```

1 def hasar_hesapla(guc, silah_bonusu, zirh):
2     ham_hasar = guc + silah_bonusu
3     gercek_hasar = ham_hasar - zirh
4
5     if gercek_hasar < 0:
6         gercek_hasar = 0
7
8     return gercek_hasar
9
10 # Hesapla ve kullan
11 oyuncu_gucu = 20
12 kilit_bonusu = 10
13 dusman_zirhi = 15
14
15 hasar = hasar_hesapla(oyuncu_gucu, kilit_bonusu,
16     ↪ dusman_zirhi)
17 print(f"Verilen hasar: {hasar}") # 15
18
19 dusman_can = 50
20 dusman_can -= hasar
21 print(f"Düşman canı: {dusman_can}") # 35

```

7.3.4 Boolean Döndürme

Kontrol fonksiyonları genellikle True/False döndürür:

Python – Boolean Return

```

1 def hayatta_mi(can):
2     return can > 0
3
4 def yeterli_mana_var_mi(mana, maliyet):
5     return mana >= maliyet
6
7 # Koşullarda kullan
8 oyuncu_can = 50
9 if hayatta_mi(oyuncu_can):

```

```
10     print("Oyuncu hâlâ savaşılabılır!")
11
12     mana = 30
13     buyu_maliyeti = 25
14     if yeterli_mana_var_mi(mana, buyu_maliyeti):
15         print("Büyü kullanılabilir!")
```

7.3.5 Birden Fazla Değer Döndürme

Python'da tuple ile birden fazla değer döndürebilirsin:

Python – Çoklu Return

```
1 def oyuncu_durumu(isim, seviye, deneyim):
2     sonraki_seviye_xp = seviye * 100
3     kalan_xp = sonraki_seviye_xp - deneyim
4
5     return seviye, kalan_xp, sonraki_seviye_xp
6
7     # Çoklu değer al
8     mevcut, kalan, gereken = oyuncu_durumu("Kahraman", 5, 420)
9     print(f"Seviye: {mevcut}")
10    print(f"Sonraki seviyeye: {kalan} XP kaldı")
```

7.3.6 Erken Return

Return, fonksiyonu hemen sonlandırır:

Python – Erken Return

```
1 def buyu_kullan(mana, maliyet, sessiz_mi):
2     if mana < maliyet:
3         return "Yetersiz mana!"
4
5     if sessiz_mi:
6         return "Büyü sessizce kullanıldı..."
7
8     return "Büyü kullanıldı!"
9
10    print(buyu_kullan(10, 50, False)) # Yetersiz mana!
11    print(buyu_kullan(100, 50, True)) # Büyü sessizce
12    ↪ kullanıldı...
13    print(buyu_kullan(100, 50, False)) # Büyü kullanıldı!
```

● Bilgi: Erken Return Avantajı

Erken return, “guard clause” olarak bilinir ve kodu daha okunabilir yapar. Geçersiz durumları en başta ele alırsın.

7.3.7 Oyun Senaryosu: Seviye Atlama

Python – Seviye Sistemi

```

1 def xp_kazandi(mevcut_xp, mevcut_seviye, kazanilan_xp):
2     yeni_xp = mevcut_xp + kazanilan_xp
3     gereken_xp = mevcut_seviye * 100
4
5     if yeni_xp >= gereken_xp:
6         yeni_seviye = mevcut_seviye + 1
7         kalan_xp = yeni_xp - gereken_xp
8         return yeni_seviye, kalan_xp, True # Seviye
9         ↪ atladi
10    else:
11        return mevcut_seviye, yeni_xp, False # Atlamadi
12
13 # Test
14 seviye = 3
15 xp = 250
16
17 seviye, xp, atladi = xp_kazandi(xp, seviye, 100)
18 if atladi:
19     print(f"[TEBRIK] Seviye atladin! Yeni seviye:
20         ↪ {seviye}")
21 else:
22     print(f"XP: {xp}/{seviye * 100}")

```

💡 İpucu: Profesyonel İpucu

Hesaplama yapan fonksiyonlar return kullanmalı, ekrana yazdırma işini çağıran kod yapmalı. Bu “separation of concerns” prensibine uyar.

7.4 Değişken Kapsamı: Değişkenler Nerede Geçerli?

Bir fonksiyon içinde tanımladığınız değişkene dışarıdan erişebilir misin? Dışarıda tanımladığınız değişkene fonksiyon içinden erişebilir misin? Bu soruların cevabı **kapsam** (scope) kavramında gizli.

■ Tanım: Değişken Kapsamı (Scope)

Kapsam, bir değişkenin geçerli olduğu kod bölgesidir. Python’da iki ana kapsam vardır:

- **Global kapsam:** Fonksiyon dışında, her yerden erişilebilir
- **Yerel kapsam:** Fonksiyon içinde, sadece o fonksiyonda geçerli

7.4.1 Yerel Değişkenler

Fonksiyon içinde tanımlanan değişkenler **yerel** (local) değişkendir:

Python – Yerel Değişken

```
1 def fonksiyon():
2     x = 10 # Yerel değişken
3     print(f"Fonksiyon içi: {x}")
4
5 fonksiyon()
6 print(x) # HATA! x burada tanımlı değil
```

⚠ Dikkat: Yerel Değişken Hatası

Yerel değişkenlere fonksiyon dışından erişmeye çalışmak `NameError` hatasına neden olur.

7.4.2 Global Değişkenler

Fonksiyon dışında tanımlanan değişkenler **global** değişkendir:

Python – Global Değişken

```
1 oyuncu_can = 100 # Global değişken
2
3 def durum_goster():
4     # Global değişkeni okuyabilirsin
5     print(f'Oyuncu canı: {oyuncu_can}')
6
7 durum_goster() # Çıktı: Oyuncu canı: 100
```

7.4.3 Global Değişkeni Değiştirme

Global değişkeni fonksiyon içinden değiştirmek için `global` anahtar kelimesi gerekir:

Python – Global Anahtar Kelimesi

```
1 skor = 0 # Global
2
3 def puan_ekle(miktar):
4     global skor # Global değişkeni kullanacağımızı belirt
5     skor += miktar
6     print(f'Skor: {skor}')
7
8 puan_ekle(10) # Skor: 10
9 puan_ekle(25) # Skor: 35
10 print(skor) # 35
```

⚠ Dikkat: Global Kullanımına Dikkat!

`global` kullanımı genellikle **kötü pratik** sayılır çünkü kodu takip etmeyi zorlaştırır. Mümkünse parametreler ve return kullan.

7.4.4 İyi Pratik: Parametre ve Return

Global yerine parametre/return kullanımı:

Python – İyi Pratik

```

1  # KÖTÜ - Global kullanımı
2  skor = 0
3  def puan_ekle_kotu(miktar):
4      global skor
5      skor += miktar
6
7  # İYİ - Parametre ve return
8  def puan_ekle_iyi(mevcut_skor, miktar):
9      return mevcut_skor + miktar
10
11 # Kullanım
12 skor = 0
13 skor = puan_ekle_iyi(skor, 10) # Skor: 10
14 skor = puan_ekle_iyi(skor, 25) # Skor: 35

```

7.4.5 Aynı İsimli Değişkenler

Global ve yerel değişken aynı isimde olabilir—farklı değişkenlerdir:

Python – İsim Gölgeleme

```

1  x = "global"
2
3  def fonksiyon():
4      x = "yerel" # Yeni yerel değişken, global'i gölgeler
5      print(f"Fonksiyon içi: {x}")
6
7  fonksiyon() # Fonksiyon içi: yerel
8  print(f"Dışarı: {x}") # Dışarı: global

```

● Bilgi: İsim Gölgeleme

Yerel değişken global değişkeni “gölgeler” (shadow). Fonksiyon içinde aynı isim kullanılırsa yerel olan tercih edilir.

7.4.6 Oyun Senaryosu: Savaş Sistemi

İyi tasarlanmış bir savaş sistemi:

Python – Savaş Sistemi

```

1 def saldır(oyuncu_guc, dusman_can, dusman_zirh):
2     """Saldırı yap, yeni düşman canını döndür."""
3     hasar = oyuncu_guc - dusman_zirh
4     if hasar < 0:
5         hasar = 0
6     return dusman_can - hasar
7
8 def savasi_baslat(oyuncu_guc, dusman_can, dusman_zirh):
9     """Savaş simülasyonu."""
10    tur = 0
11    while dusman_can > 0:
12        tur += 1
13        dusman_can = saldır(oyuncu_guc, dusman_can,
14                             ↪ dusman_zirh)
15        print(f"Tur {tur}: Düşman can = {dusman_can}")
16
17    return tur
18
19 # Savaş başlat
20 toplam_tur = savasi_baslat(
21     oyuncu_guc=25,
22     dusman_can=100,
23     dusman_zirh=10
24 )
25 print(f"Düşman {toplam_tur} turda yenildi!")

```

7.4.7 Kapsam Özeti

Tablo 7.3: Kapsam Kuralları

Durum	Erişim	Değiştirme
Yerel → Yerel	✓ Evet	✓ Evet
Yerel → Global	× Hayır	× Hayır
Global → Yerel	✓ Okuma	× Direkt değil
Global → Global (değiştir)	✓ Okuma	! global gerekli

💡 İpucu: Profesyonel İpucu

Değişkenleri mümkün olan en küçük kapsamda tut. Her şeyi global yapmak yerine, fonksiyonlara parametre geçir ve return kullan. Bu, kodun anlaşılmasını ve hata ayıklamasını kolaylaştırır.

7.5 Sonuç: Fonksiyonlar

Bu ünite de fonksiyonları öğrendik. Fonksiyonlar, profesyonel oyun geliştirmenin temel taşıdır. PyGame’de `update()`, `draw()`, `handle_events()` gibi fonksi-

yonlar sürekli kullanılır. Kodunu fonksiyonlarla organize etmek, oyununu büyütmeyi ve bakımını kolaylaştırır.

Neden fonksiyonlar bu kadar önemli? Şimdiye kadar yazdığın kodlar doğrusal—yukarıdan aşağıya çalışıyordu. Ancak bir oyun büyüdükçe aynı işlemleri defalarca yapmak gerekir: her turda düşmanı güncelle, her çarpışmada hasar hesapla, her frame’de ekranı çiz. Bu işlemleri her seferinde tekrar yazmak yerine bir kez fonksiyon olarak tanımlar ve ihtiyaç duyduğun her yerde çağırırsın. Böylece kod hem kısılır hem de düzenlenmesi kolaylaşır.

7.5.1 Öğrendiklerimiz

Bu ünite de dört temel kavram öğrendik. Bu kavramlar birlikte kullanıldığında, karmaşık programları anlaşılır parçalara bölebilirsin.

Fonksiyon tanımlama: `def` anahtar kelimesiyle yeni fonksiyonlar oluşturduk. İyi bir fonksiyon **tek bir iş** yapar ve adı o işi açıkça ifade eder. Python’da fonksiyon isimleri `snake_case` ile yazılır (örneğin `hasar_hesapla`, `can_guncelle`). Fonksiyonun gövdesi girintili yazılır—tıpkı `if` ve `while` bloklarında olduğu gibi.

Parametreler ve argümanlar: Fonksiyonlara dışarıdan veri göndermek için parametreler kullandık. Zorunlu parametreler çağrılırken mutlaka verilmelidir; varsayılan değerli parametreler ise verilmezse önceden belirlenmiş değeri kullanılır. Örneğin `saldir(silah="yumruk")` tanımında, fonksiyonu `saldir()` şeklinde çağırırsan silah “yumruk” olur, ama `saldir(silah="kılıç")` dersen “kılıç” olur.

Return ifadesi: Fonksiyonların hesapladığı sonuçları `return` ile geri döndürmeyi öğrendik. Bir fonksiyon hesaplama yapıyorsa sonucu `print()` ile ekrana yazmak yerine `return` ile döndürmeli—böylece sonucu bir değişkene atayabilir veya başka bir hesaplamada kullanabilirsin. Boolean döndüren fonksiyonlar (`hayatta_mi()`, `carpisti_mi()`) koşul kontrollerinde son derece kullanışlıdır.

Değişken kapsamı: Fonksiyon içinde tanımlanan değişkenlerin yalnızca o fonksiyona ait olduğunu (yerel kapsam), dışarıdan erişilemeyeceğini öğrendik. `global` anahtar kelimesiyle dışarıdaki değişkenlere erişmek mümkün olsa da, bu yaklaşım kodu karmaşıktır ve hatalara davetiye çıkarır. Her zaman parametre ve `return` kullanarak veri aktarmayı tercih et.

7.5.2 Hızlı Referans

Aşağıdaki kod bloğu, fonksiyonların farklı kullanım kalıplarını özetler. Her kalıp, oyun geliştirmede sıkça karşılaşılabilecek bir senaryoya karşılık gelir:

Python – Fonksiyonlar Özeti

```

1  # Basit fonksiyon
2  def merhaba():
3      print("Merhaba!")
4
5  # Parametrelili fonksiyon
6  def selamla(isim):
7      print(f"Merhaba, {isim}!")

```

```
8
9 # Varsayılan parametre
10 def saldır(silah="yumruk", hasar=5):
11     return hasar
12
13 # Return ile değer döndürme
14 def topla(a, b):
15     return a + b
16
17 # Çoklu return
18 def konum_al():
19     return x, y
20
21 # Boolean döndürme
22 def hayatta_mi(can):
23     return can > 0
```

7.5.3 Oyun Fonksiyonları Kalıbı

Tipik bir oyun, aşağıdaki fonksiyon yapısıyla organize edilir. Bu kalıp, önceki ünite de öğrendiğimiz oyun döngüsünün fonksiyonlarla modüler hale getirilmiş halidir:

Python – Oyun Yapısı

```
1 # Başlatma
2 def oyun_baslat():
3     pass
4
5 # Girdi işleme
6 def olaylari_isle():
7     pass
8
9 # Güncelleme
10 def guncelle():
11     pass
12
13 # Çizim
14 def ciz():
15     pass
16
17 # Ana döngü
18 def ana_dongu():
19     oyun_baslat()
20     while oyun_devam:
21         olaylari_isle()
22         guncelle()
23         ciz()
```

Bu yapıda her fonksiyonun tek bir sorumluluğu var: `oyun_baslat()` oyun başladığında bir kez çalışır ve değişkenleri hazırlar. `oyaylari_isle()` klavye ve

fare girdilerini okur. `guncelle()` oyun mantığını çalıştırır—hareket, çarpışma, puan hesaplama gibi. `ciz()` ise güncellenmiş durumu ekrana yansıtır. Bu dört fonksiyon, `ana_dongu()` içindeki `while` döngüsü tarafından her frame’de tekrar tekrar çağrılır.

Bu yaklaşımın en büyük avantajı şu: bir hata olduğunda nereye bakacağını hemen bilirsin. Çizimde sorun mu var? `ciz()` fonksiyonunu kontrol et. Oyuncu hareket etmiyor mu? `olaylari_isle()` ve `guncelle()` fonksiyonlarını incele. Fonksiyonlar, büyük bir problemi küçük, yönetilebilir parçalara böler.

7.5.4 Sık Yapılan Hatalar

Fonksiyonlarla çalışırken dikkat edilmesi gereken yaygın hatalar:

▲ Dikkat: Dikkat!

1. **Return unutmama:** Bir fonksiyon hesaplama yapıyorsa sonucu `return` ile döndürmelidir. Return olmadan fonksiyon `None` döndürür ve `sonuc = hesapla()` yaptığında `sonuc` değişkeni `None` olur.
2. **Parantez unutmama:** `fonksiyon` yazmak fonksiyonu *çağırma*z, sadece referans verir. Çağırma için `fonksiyon()` yazmalısın. Bu hata sessizce geçer ve bulması zordur.
3. **Çok fazla global:** Global değişkenleri değiştirmek kodun takibini zorlaştırır. Veriyi parametre ile gönder, sonucu return ile al—bu “temiz” yaklaşım hata ayıklamayı kolaylaştırır.
4. **Çok fazla parametre:** 5’ten fazla parametre alan bir fonksiyon muhtemelen birden fazla iş yapıyordur. Böl! Daha küçük, odaklanmış fonksiyonlar yaz.

7.5.5 Alıştırma Fikirleri

Her egzersiz, fonksiyonların farklı bir yönünü pratiğe dökcektir:

1. **Hesap makinesi:** `topla()`, `cikar()`, `carp()`, `bol()` fonksiyonları yaz. Parametreler ve return kullanımını pekiştirir.
2. **Dönüştürücü:** Celsius’u Fahrenheit’a çeviren fonksiyon. Matematiksel hesaplama ve return pratiği.
3. **Faktöriyel:** Bir sayının faktöriyelini hesaplayan fonksiyon. Döngü ve fonksiyonu birlikte kullanma pratiği.
4. **Zar atma:** 1-6 arası rastgele sayı döndüren fonksiyon. `import random` ve return pratiği.
5. **Envanter:** `esya_ekle()`, `esya_cikar()`, `envanter_goster()` fonksiyonları. Parametrelili fonksiyon ve liste kullanımını birleştirir.

● Bilgi: Bir Sonraki Ünite

Bir sonraki ünite de **veri yapıları** öğreneceğiz: listeler, sözlükler ve demetler. Bu yapılar, oyuncu envanteri, düşman listesi, oyun ayarları gibi verileri organize etmek için kullanılır. Ayrıca bölümün mini projesi olan **Metin Tabanlı Macera Oyunu**'nu yazacağız!

8

Veri Yapıları Temelleri

“Bir oyuncu envanteri olmadan nasıl maceraya çıkabilir? Listeler senin envanterin!”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Liste (`list`) oluşturma ve temel işlemler
- Sözlük (`dict`) ile anahtar-değer eşleştirme
- Demet (`tuple`) ve değiştirilemez koleksiyonlar
- Veri yapıları ile döngü kullanımı
- Mini Proje: Metin Tabanlı Macera Oyunu

8.1 Listeler: Oyuncunun Envanteri

Bir RPG oyununda oyuncunun çantasını düşün: kılıç, kalkan, 3 iksir, 2 anahtar... Tüm bu eşyaları tek tek değişkenlerde tutmak pratik olmaz. İşte **listeler**, birden fazla değeri tek bir değişkende saklamamızı sağlar.

■ Tanım: Liste

Liste, sıralı ve değiştirilebilir bir veri koleksiyonudur. Farklı türdeki verileri bir arada tutabilir. Python’da köşeli parantez `[]` ile tanımlanır.

8.1.1 Liste Oluşturma

Python – Liste Tanımlama

```
1 # Boş liste
2 envanter = []
3
4 # Başlangıç değerleri ile
5 silahlar = ["kılıç", "balta", "yay"]
6
7 # Karışık tipler
8 oyuncu = ["Kahraman", 100, 50, True] # isim, can, altın,
   ↪ aktif
9
10 # Sayı listesi
11 hasarlar = [10, 25, 15, 30, 20]
```

8.1.2 Elemanlara Erişim

Listeler sıfırdan başlayan indekslerle erişilir:

Python – İndeks Erişimi

```
1 meyveler = ["elma", "armut", "kiraz", "muz"]
2 #           [0]      [1]      [2]      [3]
3
4 print(meyveler[0]) # elma (ilk eleman)
5 print(meyveler[2]) # kiraz
6 print(meyveler[-1]) # muz (son eleman)
7 print(meyveler[-2]) # kiraz (sondan ikinci)
```

● Bilgi: Negatif İndeks

Negatif indeksler sondan sayar: [-1] son, [-2] sondan ikinci elemandır. Listenin sonuna erişmek için çok kullanışlı!

8.1.3 Liste Değiştirme

Listeler **mutable** (değiştirilebilir) yapılardır:

Python – Liste Değiştirme

```
1 envanter = ["kılıç", "kalkan", "iksir"]
2
3 # Eleman değiştir
4 envanter[0] = "ateş kılıcı"
5 print(envanter) # ["ateş kılıcı", "kalkan", "iksir"]
6
7 # Eleman ekle (sona)
```

```
8 envanter.append("anahtar")
9 print(envanter) # ["ateş kılıcı", "kalkan", "iksir",
  ↪ "anahtar"]
10
11 # Belirli konuma ekle
12 envanter.insert(1, "zırh")
13 print(envanter) # ["ateş kılıcı", "zırh", "kalkan",
  ↪ "iksir", "anahtar"]
```

8.1.4 Eleman Silme

Python – Eleman Silme

```
1 envanter = ["kılıç", "iksir", "kalkan", "iksir"]
2
3 # Değere göre sil (ilk bulunan)
4 envanter.remove("iksir")
5 print(envanter) # ["kılıç", "kalkan", "iksir"]
6
7 # İndekse göre sil
8 silinen = envanter.pop(0) # İlk elemanı sil ve döndür
9 print(silinen) # kılıç
10 print(envanter) # ["kalkan", "iksir"]
11
12 # Son elemanı sil
13 envanter.pop()
14 print(envanter) # ["kalkan"]
```

8.1.5 Liste İşlemleri

Python – Liste İşlemleri

```
1 sayilar = [3, 1, 4, 1, 5, 9, 2, 6]
2
3 # Uzunluk
4 print(len(sayilar)) # 8
5
6 # Eleman var mı?
7 print(4 in sayilar) # True
8 print(7 in sayilar) # False
9
10 # Sıralama
11 sayilar.sort()
12 print(sayilar) # [1, 1, 2, 3, 4, 5, 6, 9]
13
14 # Ters çevir
15 sayilar.reverse()
16 print(sayilar) # [9, 6, 5, 4, 3, 2, 1, 1]
17
```

```

18 # Min, max, toplam
19 print(min(sayilar)) # 1
20 print(max(sayilar)) # 9
21 print(sum(sayilar)) # 31

```

8.1.6 Oyun Senaryosu: Envanter Sistemi

Python – Envanter Yönetimi

```

1 envanter = []
2 MAX_ESYA = 5
3
4 def esya_ekle(esya):
5     if len(envanter) >= MAX_ESYA:
6         print(f"Envanter dolu! {esya} eklenemedi.")
7         return False
8
9     envanter.append(esya)
10    print(f"{esya} envantere eklendi.")
11    return True
12
13 def esya_kullan(esya):
14     if esya in envanter:
15         envanter.remove(esya)
16         print(f"{esya} kullanıldı!")
17         return True
18     else:
19         print(f"{esya} envanterde yok!")
20         return False
21
22 def envanter_goster():
23     print(f"\n=== Envanter ({len(envanter)}/{MAX_ESYA})")
24     print(f"    ===")
25     for i, esya in enumerate(envanter, 1):
26         print(f"    {i}. {esya}")
27
28 # Test
29 esya_ekle("Kılıç")
30 esya_ekle("Sağlık İksiri")
31 esya_ekle("Anahtar")
32 envanter_goster()
33 esya_kullan("Sağlık İksiri")
34 envanter_goster()

```

8.1.7 Liste Dilimleme (Slicing)

Listenin bir bölümünü almak için:

Python – Dilimleme

```

1 harfler = ['a', 'b', 'c', 'd', 'e', 'f']
2
3 print(harfler[1:4])    # ['b', 'c', 'd'] (1-3 arası)
4 print(harfler[:3])    # ['a', 'b', 'c'] (baştan 3 eleman)
5 print(harfler[3:])    # ['d', 'e', 'f'] (3'ten sona)
6 print(harfler[:2])    # ['a', 'c', 'e'] (2'şer atla)
7 print(harfler[::-1])  # ['f', 'e', 'd', 'c', 'b', 'a']
   ↪ (ters)

```

💡 İpucu: Profesyonel İpucu

Liste kopyalamak için `kopya = liste[:]` kullan. `kopya = liste` yazmak referans kopyalar, aynı listeyi gösterir!

8.2 Sözlükler: Anahtar-Değer Eşleştirme

Bir oyun karakterinin tüm özelliklerini tutmak istiyorsun: isim, can, güç, zırh, seviye... Listede tutsan hangi indeks neyi temsil ediyor hatırlamak zor. İşte **sözlükler**, her değere anlamlı bir isim (anahtar) vererek organize etmemizi sağlar.

■ Tanım: Sözlük (Dictionary)

Sözlük, anahtar-değer çiftlerinden oluşan bir veri yapısıdır. Her anahtar benzersizdir ve bir değere karşılık gelir. Python'da süslü parantez `{}` ile tanımlanır.

8.2.1 Sözlük Oluşturma

Python – Sözlük Tanımlama

```

1 # Boş sözlük
2 oyuncu = {}
3
4 # Değerlerle tanımlama
5 oyuncu = {
6     "isim": "Kara Şövalye",
7     "can": 100,
8     "guc": 25,
9     "seviye": 5,
10    "aktif": True
11 }
12
13 # Düşman sözlüğü
14 goblin = {"isim": "Goblin", "can": 30, "hasar": 10}

```

8.2.2 Değerlere Erişim

Anahtarı kullanarak değere eriş:

Python – Değer Erişimi

```
1 oyuncu = {"isim": "Kahraman", "can": 100, "altın": 50}
2
3 # Köşeli parantez ile
4 print(oyuncu["isim"])    # Kahraman
5 print(oyuncu["can"])     # 100
6
7 # get() metodu ile (güvenli)
8 print(oyuncu.get("can")) # 100
9 print(oyuncu.get("mana")) # None (hata vermez)
10 print(oyuncu.get("mana", 0)) # 0 (varsayılan değer)
```

⚠ Dikkat: KeyError

Olmayan bir anahtara `[]` ile erişmek `KeyError` hatasına neden olur. `get()` kullanmak daha güvenlidir.

8.2.3 Sözlük Değiştirme

Python – Sözlük Güncelleme

```
1 oyuncu = {"isim": "Kahraman", "can": 100}
2
3 # Değer güncelle
4 oyuncu["can"] = 80
5 print(oyuncu)    # {"isim": "Kahraman", "can": 80}
6
7 # Yeni anahtar ekle
8 oyuncu["mana"] = 50
9 oyuncu["seviye"] = 1
10 print(oyuncu)   # {"isim": "Kahraman", "can": 80, "mana":
    ↪ 50, "seviye": 1}
11
12 # Anahtar sil
13 del oyuncu["mana"]
14 print(oyuncu)   # {"isim": "Kahraman", "can": 80, "seviye":
    ↪ 1}
```

8.2.4 Sözlük Üzerinde Döngü

Python – Sözlük Döngüsü

```

1 oyuncu = {"isim": "Kahraman", "can": 100, "guc": 25}
2
3 # Sadece anahtarlar
4 for anahtar in oyuncu:
5     print(anahtar)
6
7 # Sadece değerler
8 for deger in oyuncu.values():
9     print(deger)
10
11 # Hem anahtar hem değer
12 for anahtar, deger in oyuncu.items():
13     print(f"{anahtar}: {deger}")

```

8.2.5 Oyun Senaryosu: Karakter Sistemi

Python – Karakter Sistemi

```

1 def karakter_olustur(isim, sinif):
2     """Yeni karakter oluşturun."""
3     temel_ozellikler = {
4         "Savaşçı": {"can": 150, "mana": 30, "guc": 20,
5             ↪ "zeka": 5},
6         "Büyücü": {"can": 80, "mana": 150, "guc": 5,
7             ↪ "zeka": 25},
8         "Okçu": {"can": 100, "mana": 50, "guc": 15,
9             ↪ "zeka": 10}
10    }
11
12    ozellikler = temel_ozellikler.get(sinif,
13        ↪ temel_ozellikler["Savaşçı"])
14
15    karakter = {
16        "isim": isim,
17        "sinif": sinif,
18        "seviye": 1,
19        "xp": 0,
20        **ozellikler # Sınıf özelliklerini ekle
21    }
22    return karakter
23
24 def karakter_goster(k):
25     """Karakter bilgilerini göster."""
26     print(f"\n=== {k['isim']} ({k['sinif']}) ===")
27     print(f"Seviye: {k['seviye']} | XP: {k['xp']}")
28     print(f"Can: {k['can']} | Mana: {k['mana']}")
29     print(f"Güç: {k['guc']} | Zeka: {k['zeka']}")

```

```

26
27 # Test
28 kahraman = karakter_olustur("Thorin", "Savaşçı")
29 karakter_goster(kahraman)
30
31 buyucu = karakter_olustur("Gandalf", "Büyücü")
32 karakter_goster(buyucu)

```

8.2.6 İç İç Sözlükler

Sözlükler iç içe olabilir:

Python – İç İç Sözlük

```

1 oyun_verisi = {
2     "oyuncu": {
3         "isim": "Kahraman",
4         "can": 100,
5         "konum": {"x": 50, "y": 100}
6     },
7     "dusmanlar": {
8         "goblin_1": {"can": 30, "x": 200},
9         "ork_1": {"can": 50, "x": 400}
10    }
11 }
12
13 # Erişim
14 print(oyun_verisi["oyuncu"]["isim"]) # Kahraman
15 print(oyun_verisi["oyuncu"]["konum"]["x"]) # 50
16 print(oyun_verisi["dusmanlar"]["ork_1"]["can"]) # 50

```

8.2.7 Sözlük Metodları

Tablo 8.1: Yaygın Sözlük Metodları

Metod	Açıklama
<code>keys()</code>	Tüm anahtarları döndürür
<code>values()</code>	Tüm değerleri döndürür
<code>items()</code>	Anahtar-değer çiftlerini döndürür
<code>get(key, default)</code>	Güvenli değer erişimi
<code>pop(key)</code>	Anahtarı sil ve değeri döndür
<code>update(dict2)</code>	Başka sözlükle güncelle
<code>clear()</code>	Tüm içeriği sil

💡 İpucu: Profesyonel İpucu

Oyun nesnelerini sözlüklerle modelle. “Düşman” bir sözlük olabilir: `{"isim": "Goblin", "can": 30, "x": 100, "y": 200}`. Py-Game’de sprite’lar için temel veri yapısı olarak kullanılır.

8.3 Demetler: Değişmez Koleksiyonlar

Bazen verilerin değiştirilmemesini isteriz. Örneğin bir koordinat (x, y) veya RGB renk değeri (255, 128, 0) gibi. **Demetler** (tuple), listelere benzer ama **değiştirilemez** (immutable) yapılardır.

■ Tanım: Demet (Tuple)

Demet, sıralı ve değiştirilemeyen bir veri koleksiyonudur. Bir kez oluşturulduktan sonra elemanları değiştirilemez. Python’da normal parantez () ile tanımlanır.

8.3.1 Demet Oluşturma

Python – Demet Tanımlama

```

1  # Parantez ile
2  koordinat = (100, 200)
3  renk = (255, 128, 0)
4
5  # Parantez olmadan da olur
6  boyut = 800, 600
7
8  # Tek elemanlı demet (virgül gerekli!)
9  tekli = (42,)
10
11 # Boş demet
12 bos = ()

```

⚠ Dikkat: Tek Elemanlı Demet

Tek elemanlı demette virgül zorunlu: `(42,)`. Virgül olmazsa `(42)` sadece parantez içinde sayıdır, demet değil!

8.3.2 Elemanlara Erişim

Listeler gibi indeksle erişilir:

Python – Demet Erişimi

```
1 konum = (150, 300, 0) # x, y, z
2
3 print(konum[0]) # 150 (x)
4 print(konum[1]) # 300 (y)
5 print(konum[-1]) # 0 (z, son eleman)
6
7 # Çoklu atama (unpacking)
8 x, y, z = konum
9 print(f"X: {x}, Y: {y}, Z: {z}")
```

8.3.3 Neden Demet Kullanırız?

1. **Güvenlik:** Yanlılıkla değiştirilemez
2. **Performans:** Listelerden biraz daha hızlı
3. **Sözlük anahtarı:** Demetler sözlük anahtarı olabilir, listeler olamaz
4. **Anlam:** “Bu değişmemeli” mesajı verir

Python – Demet vs Liste

```
1 # Liste - değiştirilebilir
2 liste = [1, 2, 3]
3 liste[0] = 100 # Çalışır
4
5 # Demet - değiştirilemez
6 demet = (1, 2, 3)
7 demet[0] = 100 # TypeError!
```

8.3.4 Oyun Senaryosu: Koordinatlar ve Renkler

Python – Oyun Sabitleri

```
1 # Ekran boyutu (değişmemeli)
2 EKRAN_BOYUTU = (800, 600)
3 GENISLIK, YUKSEKLIK = EKRAN_BOYUTU
4
5 # Renkler (RGB)
6 SIYAH = (0, 0, 0)
7 BEYAZ = (255, 255, 255)
8 KIRMIZI = (255, 0, 0)
9 YESIL = (0, 255, 0)
10 MAVI = (0, 0, 255)
11
12 # Oyuncu başlangıç konumu
13 BASLANGIC_KONUM = (GENISLIK // 2, YUKSEKLIK - 50)
14
15 print(f"Ekran: {GENISLIK}x{YUKSEKLIK}")
16 print(f"Oyuncu başlangıcı: {BASLANGIC_KONUM}")
```

8.3.5 Fonksiyonlardan Çoklu Değer Döndürme

Return ile birden fazla değer döndürürken aslında demet döndürüyoruz:

Python – Çoklu Return

```

1 def fare_konumu_al():
2     x = 150
3     y = 300
4     return x, y # Aslında (x, y) demeti
5
6 # Unpacking ile al
7 fare_x, fare_y = fare_konumu_al()
8 print(f"Fare: ({fare_x}, {fare_y})")
9
10 # Veya demet olarak al
11 konum = fare_konumu_al()
12 print(f"Fare: {konum}")

```

8.3.6 Demet Metodları

Demetlerin sadece iki metodu vardır:

Python – Demet Metodları

```

1 sayilar = (1, 2, 3, 2, 4, 2, 5)
2
3 # Eleman sayısı
4 print(sayilar.count(2)) # 3 (2 üç kez geçiyor)
5
6 # İndeks bulma
7 print(sayilar.index(4)) # 4 (4 değeri 4. indekste)

```

8.3.7 Liste vs Demet Karşılaştırma

Tablo 8.2: Liste ve Demet Karşılaştırması

Özellik	Liste	Demet
Sözdizimi	[1, 2, 3]	(1, 2, 3)
Değiştirilebilir	Evet	Hayır
Hız	Normal	Biraz daha hızlı
Sözlük anahtarı	Olamaz	Olabilir
Kullanım	Değişen veriler	Sabit veriler

8.3.8 Oyun Senaryosu: Hareket Vektörleri

Python – Hareket Sistemi

```

1  # Yön vektörleri (sabit)
2  YUKARI = (0, -1)
3  ASAGI = (0, 1)
4  SOL = (-1, 0)
5  SAG = (1, 0)
6  DURGUN = (0, 0)
7
8  def hareket_et(konum, yon, hiz):
9      """Karakteri hareket ettir."""
10     x, y = konum
11     dx, dy = yon
12
13     yeni_x = x + dx * hiz
14     yeni_y = y + dy * hiz
15
16     return (yeni_x, yeni_y)
17
18 # Test
19 oyuncu_konum = (100, 100)
20 oyuncu_konum = hareket_et(oyuncu_konum, SAG, 5)
21 print(f"Yeni konum: {oyuncu_konum}") # (105, 100)

```

İpucu: Profesyonel İpucu

PyGame’de koordinatlar ve renkler genellikle demet olarak kullanılır: `screen.fill((0, 0, 0))` veya `rect.topleft = (100, 200)`. Bu kullanımı şimdiden öğrenmek faydalı!

8.4 Mini Proje: Metin Tabanlı Macera Oyunu

Bu bölümde öğrendiğimiz her şeyi bir araya getirerek basit ama eğlenceli bir metin tabanlı macera oyunu yapacağız! Koşullar, döngüler, fonksiyonlar ve veri yapıları—hepsi bu projede yer alacak.

8.4.1 Oyun Konsepti

Oyunumuzun adı “Zindanın Derinlikleri”. Oyuncu 10 odalık bir zindanda ilerleyecek. Her odada rastgele bir olay yaşanacak: düşmanla karşılaşma, hazine bulma, tuzağa düşme veya boş oda. Oyuncunun amacı canını kaybetmeden zindanın sonuna ulaşmak.

- Oyuncu bir zindanda ilerliyor
- Her odada farklı olaylar yaşanıyor
- Düşmanlarla savaş, hazine bulma, tuzaklar
- Amaç: Zindanın sonuna ulaşmak!

Bu proje, bu bölümdeki dört ünitenin tümünü bir arada kullanır: **Koşullar** ile savaşta seçim yapılır, **döngüler** ile oyun ve savaş akışı sağlanır, **fonksiyonlar** ile kod modüler hale getirilir, **veri yapıları** ile oyuncu bilgileri ve düşman verileri saklanır.

Projeyi adım adım inşa edeceğiz. Önce verileri tanımlayacağız, sonra yardımcı fonksiyonları yazacağız ve en sonda hepsini ana döngüde birleştireceğiz.

8.4.2 Temel Yapı

İlk adım olarak oyunun ihtiyaç duyduğu verileri tanımlayalım. Oyuncu bilgilerini bir **sözlükte**, düşman türlerini ise iç içe bir **sözlükte** tutacağız:

Python – Oyun Verileri

```

1 import random
2
3 # Oyuncu başlangıç değerleri
4 oyuncu = {
5     "isim": "",
6     "can": 100,
7     "max_can": 100,
8     "guc": 15,
9     "altin": 0,
10    "envanter": []
11 }
12
13 # Düşman türleri
14 dusmanlar = {
15     "Goblin": {"can": 30, "hasar": 10, "altin": 10},
16     "Ork": {"can": 50, "hasar": 15, "altin": 25},
17     "İskelet": {"can": 40, "hasar": 12, "altin": 15},
18     "Trol": {"can": 80, "hasar": 20, "altin": 50}
19 }
```

Burada **oyuncu** sözlüğü oyuncunun tüm özelliklerini bir arada tutar. **envanter** anahtarı bir **liste** içerir—çünkü eşyalar sürekli eklenir ve çıkarılır. **dusmanlar** sözlüğü ise her düşman türü için ayrı bir sözlük barındırır. Bu “sözlük içinde sözlük” yapısı, farklı düşman türlerinin farklı özelliklere sahip olmasını sağlar.

8.4.3 Yardımcı Fonksiyonlar

Şimdi oyunda sıkça kullanacağımız iki yardımcı fonksiyon yazalım. **durum_goster()** oyuncunun mevcut durumunu ekrana yazdırır, **iyiles()** ise oyuncunun canını belirli bir miktar artırır (ancak maksimum canı aşmaz):

Python – Yardımcı Fonksiyonlar

```

1 def durum_goster():
2     """Oyuncu durumunu göster."""
3     print(f"\n{'='*40}")
```

```

4     print(f"{oyuncu['isim']} | Can:
      ↪ {oyuncu['can']}/{oyuncu['max_can']}")
5     print(f"Güç: {oyuncu['guc']} | Altın:
      ↪ {oyuncu['altin']}")
6     if oyuncu['envanter']:
7         print(f"Envanter: {'',
      ↪ '.join(oyuncu['envanter'])}")
8     print(f"{'='*40}\n")
9
10    def iyiles(miktar):
11        """Oyuncuyu iyileştir."""
12        oyuncu['can'] += miktar
13        if oyuncu['can'] > oyuncu['max_can']:
14            oyuncu['can'] = oyuncu['max_can']
15        print(f"[+] {miktar} can kazandın! Şu anki can:
      ↪ {oyuncu['can']}")

```

`durum_goster()` fonksiyonu f-string ile oyuncunun can, güç ve altın bilgilerini formatlar. Envanter boş değilse, eşyaları virgülle ayırarak gösterir. `iyiles()` fonksiyonu ise canı artırdıktan sonra bir `if` kontrolüyle maksimum canı aşmış olduğunu kontrol eder—bu, oyunlarda çok yaygın bir kalıptır: “iyileşme hiçbir zaman maksimum canı geçemez.”

8.4.4 Savaş Sistemi

Savaş sistemi, projenin en karmaşık fonksiyonudur. Oyuncu bir düşmanla karşılaştığında, düşman yenilene, oyuncu ölene veya oyuncu kaçana kadar devam eden bir `while` döngüsü çalışır. Her turda oyuncu saldırmak, iksir kullanmak veya kaçmak arasında seçim yapar:

Python – Savaş Fonksiyonu

```

1     def savas(dusman_adi):
2         """Düşmanla savaş."""
3         dusman = dusmanlar[dusman_adi].copy()
4         print(f"\n*** {dusman_adi} ile karşılaştın!")
5         print(f"Düşman Can: {dusman['can']} | Hasar:
      ↪ {dusman['hasar']}")
6
7         while dusman['can'] > 0 and oyuncu['can'] > 0:
8             print("\n1. Saldır")
9             print("2. İksir Kullan")
10            print("3. Kaç")
11
12            secim = input("Seçimin: ")
13
14            if secim == "1":
15                # Oyuncu saldırır
16                hasar = random.randint(oyuncu['guc'] - 5,
      ↪ oyuncu['guc'] + 5)

```

```

17     dusman['can'] -= hasar
18     print(f"[SALDIRI] {hasar} hasar verdin!")
19
20     if dusman['can'] > 0:
21         # Düşman saldırır
22         hasar = random.randint(dusman['hasar'] -
23     ↪ 3, dusman['hasar'] + 3)
24         oyuncu['can'] -= hasar
25         print(f"[HASAR] {dusman_adi} sana {hasar}
26     ↪ hasar verdi!")
27
28     elif secim == "2":
29         if "Sağlık İksiri" in oyuncu['envanter']:
30             oyuncu['envanter'].remove("Sağlık İksiri")
31             iyiles(30)
32         else:
33             print("İksirin yok!")
34
35     elif secim == "3":
36         if random.random() < 0.5:
37             print("[KACIS] Kaçmayı başardın!")
38             return True
39         else:
40             print("Kaçamadın!")
41             hasar = dusman['hasar']
42             oyuncu['can'] -= hasar
43             print(f"[HASAR] Kaçarken {hasar} hasar
44     ↪ aldın!")
45
46     if oyuncu['can'] <= 0:
47         print("\n[OLUM] Öldün!")
48         return False
49     else:
50         print(f"\n[ZAFER] {dusman_adi}'i yendin!")
51         kazanc = dusmanlar[dusman_adi]['altin']
52         oyuncu['altin'] += kazanc
53         print(f"[ALTIN] {kazanc} altin kazandın!")
54         return True

```

Bu fonksiyonda öğrendiğimiz birçok kavram bir arada çalışıyor:

- `dusmanlar[dusman_adi].copy()` ile düşmanın bir **kopyası** oluşturuluyor—böylece orijinal düşman verisi değişmiyor ve aynı düşman türü tekrar kullanılabilir.
- `while dusman['can'] > 0 and oyuncu['can'] > 0`: döngüsü iki taraftan biri ölene kadar devam ediyor (`and` operatörü).
- `random.randint()` ile hasar miktarına rastgelelik ekleniyor—bu sayede her savaş farklı sonuçlanıyor.
- Kaçış denemesinde `random.random() < 0.5` ile %50 başarı şansı tanınıyor.
- Fonksiyon, savaşın sonucuna göre `True` (oyuncu hayatta) veya `False` (oyuncu

öldü) döndürüyor. Bu boolean dönüş değeri, ana döngüde oyunun devam edip etmeyeceğine karar vermek için kullanılacak.

8.4.5 Oda Olayları

Her oda türü için ayrı bir fonksiyon tanımlayacağız. Bu yaklaşım, yeni oda türleri eklemek istediğimizde kodu değiştirmeyi kolaylaştırır—sadece yeni bir fonksiyon yazıp onu ana döngüye eklememiz yeterlidir:

Python – Oda Sistemi

```

1 def bos_oda():
2     """Boş oda - dinlenme fırsatı."""
3     print("\n[ODA] Bos bir odaya girdin.")
4     print("Biraz dinlenebilirsin.")
5     if oyuncu['can'] < oyuncu['max_can']:
6         iyiles(10)
7
8 def hazine_odasi():
9     """Hazine odası."""
10    print("\n[HAZINE] Bir hazine sandığı buldun!")
11    altin = random.randint(20, 50)
12    oyuncu['altin'] += altin
13    print(f"[ALTIN] {altin} altın buldun!")
14
15    if random.random() < 0.3:
16        oyuncu['envanter'].append("Sağlık İksiri")
17        print("[IKSIR] Bir Sağlık İksiri de buldun!")
18
19 def tuzak_odasi():
20     """Tuzak odası."""
21     print("\n[UYARI] Tuzaga dustun!")
22     hasar = random.randint(10, 20)
23     oyuncu['can'] -= hasar
24     print(f"[HASAR] {hasar} hasar aldın!")
25
26 def dusman_odasi():
27     """Düşman odası."""
28     dusman = random.choice(list(dusmanlar.keys()))
29     return savas(dusman)

```

Her fonksiyon tek bir oda türünü yönetiyor: `bos_oda()` oyuncuyu biraz iyileştirir, `hazine_odasi()` rastgele altın ve bazen iksir verir (`random.random() < 0.3` ile %30 ihtimal), `tuzak_odasi()` rastgele hasar verir. `dusman_odasi()` ise `random.choice()` ile düşman listesinden rastgele birini seçip `savas()` fonksiyonunu çağırır ve sonucunu döndürür.

8.4.6 Ana Oyun Döngüsü

Son olarak tüm parçaları birleştiren ana fonksiyonu yazalım. Bu fonksiyon oyunu başlatır, odalar arasında ilerlemeyi yönetir ve oyun sonucunu belirler:

Python – Ana Döngü

```

1 def oyunu_baslat():
2     """Oyunu başlat."""
3     print("\n" + "="*50)
4     print("          ZINDANIN DERİNLİKLERİ")
5     print("="*50)
6
7     oyuncu['isim'] = input("\nAdın ne, cesur kahraman? ")
8     print(f"\nHoş geldin {oyuncu['isim']}! Zindan seni
9         ↪ bekliyor...")
10
11     oda_sayisi = 10
12     mevcut_oda = 0
13
14     while mevcut_oda < oda_sayisi and oyuncu['can'] > 0:
15         mevcut_oda += 1
16         print(f"\n{'-'*40}")
17         print(f"ODA {mevcut_oda}/{oda_sayisi}")
18
19         durum_goster()
20
21         # Rastgele oda olayı
22         olay = random.choices(
23             ['bos', 'hazine', 'tuzak', 'dusman'],
24             weights=[20, 20, 20, 40]
25         )[0]
26
27         if olay == 'bos':
28             bos_oda()
29         elif olay == 'hazine':
30             hazine_odasi()
31         elif olay == 'tuzak':
32             tuzak_odasi()
33         elif olay == 'dusman':
34             if not dusman_odasi():
35                 break
36
37         input("\nDevam etmek için Enter'a bas...")
38
39     # Oyun sonu
40     print("\n" + "="*50)
41     if oyuncu['can'] > 0 and mevcut_oda >= oda_sayisi:
42         print("[TEBRIKLER] TEBRİKLER! Zindanı
43             ↪ tamamladın!")
44     else:
45         print("[OLUM] GAME OVER")
46
47     print(f"Toplanan altın: {oyuncu['altin']}")
48     print("="*50)

```



```

48 # Oyunu başlat
49 if __name__ == "__main__":
50     oyunu_baslat()

```

Ana döngüde dikkat edilecek birkaç tasarım kararı var:

- `random.choices()` fonksiyonu, `weights` parametresiyle her olayın olasılığını belirler. Düşman odası %40, diğerleri %20 olasılıkla çıkar—bu sayede oyun yeterince zorlu olur.
- `while mevcut_oda < oda_sayisi and oyuncu['can'] > 0:` koşulu iki durumu kontrol eder: ya tüm odalar tamamlanır ya da oyuncu ölür. Her ikisi de döngüyü sonlandırır.
- `dusman_odasi()` fonksiyonu `False` döndürürse (oyuncu öldü), `break` ile döngüden çıkılır.
- Oyun sonunda hem kazanma hem kaybetme durumları kontrol edilerek uygun mesaj gösterilir.

8.4.7 Projeyi Geliştirme Fikirleri

Bu oyun temel bir iskelet. Aşağıdaki fikirlerle geliştirebilirsin—her biri yeni programlama pratikleri kazandıracaktır:

1. **Dükkân sistemi:** Her 3 odada bir dükkân çıksın, altınla iksir veya silah satın alınabilsin. Yeni bir fonksiyon ve sözlük yapısı gerektirir.
2. **Farklı silahlar:** Her silahın farklı hasarı olsun. Envanterdeki silahı seçerek saldırma mekanizması ekle.
3. **Seviye atlama:** Düşman yenince XP kazan, belirli XP’de seviye atla. Seviye attıkça güç ve maksimum can artsın.
4. **Farklı sınıflar:** Savaşçı (yüksek güç), Büyücü (büyü saldırısı), Hırsız (yüksek kaçış şansı) gibi sınıflar ekle.
5. **Boss savaşı:** Son odada özel bir güçlü düşman çıksın. Normal düşmanlardan farklı saldırı kalıplarına sahip olabilir.

● Bilgi: Öğrendiklerinin Özeti

Bu projede kullandıklarımız:

- **Koşullar:** `if/elif/else` ile karar yapıları
- **Döngüler:** `while` ile oyun döngüsü, `for` ile liste gezinme
- **Fonksiyonlar:** Kod organizasyonu ve tekrar kullanım
- **Sözlükler:** Oyuncu ve düşman verileri
- **Listeler:** Envanter sistemi

8.5 Sonuç: Veri Yapıları ve Bölüm Özeti

Bu ünite Python’un temel veri yapılarını öğrendik: listeler, sözlükler ve demetler. Bu yapılar, oyun verilerini organize etmenin temel taşlarıdır. Ayrıca bölümün mini projesiyle öğrendiklerimizi pratiğe döktük.

Bir oyun düşündüğünde, aslında her şeyin “veri” olduğunu fark edeceksin: oyuncunun canı bir sayı, envanteri bir liste, konumu bir koordinat çifti, tüm özellikleri bir sözlük. İşte bu ünite de öğrendiğin veri yapıları, bu bilgileri bellekte tutmanın ve yönetmenin yollarıdır.

8.5.1 Veri Yapıları Özeti

Aşağıdaki tablo, üç veri yapısının temel özelliklerini karşılaştırmaktadır. Hangi durumda hangisini kullanacağına karar verirken bu tabloyu referans olarak kullanabilirsin:

Tablo 8.3: Veri Yapıları Karşılaştırması

Özellik	Liste	Sözlük	Demet
Sözdizimi	<code>[a, b, c]</code>	<code>{k: v}</code>	<code>(a, b, c)</code>
Erişim	İndeks	Anahtar	İndeks
Değiştirilebilir	Evet	Evet	Hayır
Sıralı	Evet	Evet*	Evet
Kullanım	Koleksiyon	Özellikler	Sabit değerler

*Python 3.7+ sürümlerinde sözlükler ekleme sırasını korur.

Tablodaki en kritik fark “Değiştirilebilir” satırıdır. Listeler ve sözlükler oluşturulduktan sonra içerikleri değiştirilebilirken, demetler sabittir. Bu yüzden oyuncunun envanteri (sürekli değişen) bir liste, ekran çözünürlüğü (sabit kalan) ise bir demet olarak tutulur.

8.5.2 Hızlı Referans

Her veri yapısının en sık kullanılan işlemleri:

Python – Veri Yapıları Özeti

```

1  # LİSTE - değiştirilebilir, sıralı koleksiyon
2  liste = [1, 2, 3]
3  liste.append(4)      # Ekle
4  liste.remove(2)     # Sil
5  liste[0] = 10       # Değiştir
6
7  # SÖZLÜK - anahtar-değer çiftleri
8  sozluk = {"can": 100, "guc": 20}
9  sozluk["mana"] = 50  # Ekle/güncelle
10 del sozluk["mana"]   # Sil
11 deger = sozluk.get("can", 0) # Güvenli erişim
12
13 # DEMET - değiştirilemez koleksiyon
14 demet = (800, 600)
15 x, y = demet          # Unpacking

```

8.5.3 Oyun Geliştirmede Kullanım

Aşağıdaki örnek, üç veri yapısının bir oyunda birlikte nasıl kullanıldığını göstermektedir:

Python – Oyun Örneği

```

1  # Oyuncu - sözlük
2  oyuncu = {
3      "konum": (100, 200), # demet
4      "can": 100,
5      "envanter": ["kılıç", "kalkan"] # liste
6  }
7
8  # Düşman listesi
9  dusmanlar = [
10     {"isim": "Goblin", "can": 30},
11     {"isim": "Ork", "can": 50}
12 ]
13
14 # Sabit değerler - demet
15 EKRAN = (800, 600)
16 BEYAZ = (255, 255, 255)

```

Bu kodda dikkat edilmesi gereken yapı şu: Oyuncu bir **sözlük**tür, çünkü birden fazla farklı özelliği (can, güç, konum) isimle erişilebilir şekilde tutmak istiyoruz. Oyuncunun konumu bir **demettir**, çünkü x ve y koordinatı birlikte anlam taşır ve genellikle tek seferde atanır. Envanteri bir **listedir**, çünkü eşyalar sürekli eklenir ve çıkarılır. Düşmanlar ise bir **sözlük listesi**dir—her düşman sözlükle tanımlanır, tüm düşmanlar bir listede tutulur. Bu iç içe yapı, gerçek oyunlarda çok yaygındır.

8.5.4 Bölüm 2 Genel Özeti

Bölüm 2'yi tamamladın! Geriye dönüp baktığında, bu bölümde öğrendiğin dört ünite birlikte güçlü bir temel oluşturuyor. Her biri bir sonrakine zemin hazırladı:

Ünite 1'de koşullu ifadeler ile programlarının karar vermesini sağladın. **if**, **elif**, **else** yapıları ve mantıksal operatörler (**and**, **or**, **not**) sayesinde artık “Eğer şu koşul sağlanırsa şunu yap” gibi senaryoları kodlayabilirsin.

Ünite 2'de döngüler ile tekrarlayan işlemleri otomatikleştirdin. **while** ile koşula bağlı döngüler, **for** ile koleksiyon üzerinde gezinme, **range()** ile sayı dizileri ve **break/continue** ile döngü kontrolü öğrendin. Oyun döngüsünün temel mantığını kavradın.

Ünite 3'te fonksiyonlar ile kodunu organize etmeyi öğrendin. **def** ile fonksiyon tanımlama, parametreler, **return** ile değer döndürme ve değişken kapsamı kavramlarıyla artık büyük programları yönetilebilir parçalara bölebilirsin.

Ünite 4'te veri yapıları ile oyun verilerini depolamayı ve düzenlemeyi kavradın. Listeler, sözlükler ve demetler ile oyuncu bilgileri, düşman listeleri ve sabit değerleri organize etmeyi öğrendin.

Mini projemiz olan **Metin Tabanlı Macera Oyunu**, bu dört ünitenin hepsini bir arada kullanıyordu: Koşullarla karar verme, döngülerle oyun akışı, fonksiyonlarla

kod organizasyonu ve veri yapılarıyla oyun verilerini yönetme.

8.5.5 Sık Yapılan Hatalar

Veri yapılarıyla çalışırken dikkat edilmesi gereken yaygın hatalar:

▲ Dikkat: Dikkat!

1. **Liste indeksi:** Listeler 0'dan başlar. 5 elemanlı bir listede `liste[5]` hata verir, çünkü son geçerli indeks 4'tür. Negatif indeksler de geçerlidir: `liste[-1]` son elemanı verir.
2. **KeyError:** Sözlükte olmayan bir anahtara `sozluk["yok"]` ile erişmek `KeyError` hatasına neden olur. Bunun yerine `sozluk.get("yok", varsayilan)` kullanarak güvenli erişim sağla.
3. **Demet değiştirme:** `demet[0] = 5` yazmak `TypeError` verir. Demetler oluşturulduktan sonra değiştirilemez; ihtiyaç varsa yeni bir demet oluştur.
4. **Liste kopyalama:** `kopya = liste` yazmak yeni bir liste *oluşturmaz*, aynı listeye ikinci bir isim verir. Bağımsız kopya için `kopya = liste[:]` veya `kopya = liste.copy()` kullan.

8.5.6 Alıştırma Fikirleri

Aşağıdaki egzersizler, veri yapılarını farklı senaryolarda kullanarak pratiğini güçlendirecektir:

1. **Gelişmiş envanter:** Eşya miktarlarını sözlükle tutan bir sistem. Örneğin `{"oks": 15, "iksir": 3}` gibi. Ekleme, çıkarma ve listeleme fonksiyonları yaz.
2. **Kart oyunu:** Liste ile kart destesi oluştur, karıştır (`random.shuffle`), dağıt. Her kart bir sözlükle temsil edilsin.
3. **Adres defteri:** İsim-telefon eşleştirmesini sözlükle tut. Ekleme, arama ve silme fonksiyonları yaz.
4. **Metin macera v2:** Mini projeyi geliştir! Dükkân sistemi, yeni düşman türleri veya seviye atlama mekanizması ekle.

● Bilgi: Bir Sonraki Bölüm

Bir sonraki bölümde **PyGame'e giriş** yapacağız! Python temellerini öğrendiğine göre artık gerçek grafik oyunlar yapmaya hazırsın. Oyun penceresi açacak, şekiller çizecek ve ilk oyun döngünü yazacaksın!

9

PyGame Kurulumu ve Yapısı

“Her harika oyun, tek bir satır kodla başlar. Senin oyunun da burada başlıyor!”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- PyGame kütüphanesinin ne olduğu ve neler yapabildiği
- `uv` ile yeni bir PyGame projesi oluşturma
- PyGame’in modül yapısı ve temel bileşenleri
- `pygame.init()` ve `pygame.quit()` fonksiyonlarının kullanımı

9.1 PyGame Nedir?

İlk iki bölümde Python’un temellerini öğrendin: değişkenler, koşullar, döngüler, fonksiyonlar ve veri yapıları. Artık “Merhaba Dünya” yazdırmaktan çok daha heyecan verici bir adıma hazırsın. Düşünsene: ekranda hareket eden karakterler, renkli arka planlar, tıkladığında tepki veren butonlar... Tüm bunları Python ile yapmak mümkün mü? Evet, **PyGame** sayesinde mümkün!

■ Tanım: PyGame

PyGame, Python programlama dilinde 2D oyunlar ve multimedya uygulamaları geliştirmek için kullanılan açık kaynaklı bir kütüphanedir. SDL (Simple DirectMedia Layer) kütüphanesi üzerine inşa edilmiştir ve pencere oluşturma, görsel çizim, ses çalma ve kullanıcı girdilerini işleme gibi temel işlevleri sağlar.

Peki SDL ne demek? SDL, farklı işletim sistemlerinde grafik, ses ve girdi donanımlarına erişim sağlayan düşük seviyeli bir C kütüphanesidir. PyGame, bu

kütüphaneyi Python'a uygun hale getirerek senin karmaşık C kodlarıyla uğraşmadan doğrudan oyun geliştirmene olanak tanır.

Kısacası PyGame, bilgisayarın ekranına piksel çizmeni, hoparlörlerinden ses çıkarmanı ve klavye ile fareyi dinlemeni sağlayan bir “köprü” gibi düşünebilirsin. Sen Python'da komut veriyorsun, PyGame ise bu komutları işletim sistemine iletip sonucu ekrana yansıtıyor.

9.1.1 PyGame ile Neler Yapabilirsin?

PyGame'in yetenekleri oldukça geniştir. İşte bazı örnekler:

1. **2D Oyunlar:** Platform oyunları, uzay savaşları, bulmaca oyunları, kart oyunları
2. **Grafiksel Uygulamalar:** Çizim programları, veri görselleştirme araçları
3. **Simülasyonlar:** Fizik simülasyonları, parçacık sistemleri
4. **Multimedya:** Görüntüleyiciler, basit müzik çalarlar
5. **Prototipler:** Hızlıca fikir denemek için interaktif prototipler

Bu kitap boyunca iki büyük proje geliştireceğiz: bir uzay savaşı oyunu ve bir platform macera oyunu. Ama önce temelleri sağlam atmalıyız. Bu ünite, PyGame ile tanışma üniten olacak.

● Bilgi: Biliyor muydun?

PyGame, 2000 yılından beri aktif olarak geliştirilmektedir. Dünya genelinde binlerce oyun PyGame ile yapılmıştır. PyGame ile başlayıp daha sonra profesyonel oyun motorlarına geçen birçok oyun geliştiricisi vardır. Yani burada öğrendiklerin seni ileride de destekleyecek!

9.1.2 PyGame Projesini Oluşturma

Konu 1.2'da [uv](#) aracını kurmuştun. Şimdi bu aracı kullanarak ilk PyGame projesi oluşturacağız. Terminali aç ve aşağıdaki adımları takip et.

Proje Dizinini Oluşturma

Önce projemiz için yeni bir dizin oluşturalım ve içine girelim:

```
mkdir oyun_projesi
cd oyun_projesi
```

Ardından [uv](#) ile projeyi başlatalım:

```
uv init
```

Bu komut, proje dizininde birkaç dosya oluşturur. En önemlisi [pyproject.toml](#) dosyasıdır. Bu dosya, projenin ayarlarını ve bağımlılıklarını tutar. Bir nevi projenin “kimlik kartı” gibi düşünebilirsin.

PyGame'i Projeye Ekleme

Şimdi PyGame kütüphanesini projemize ekleyelim:

```
uv add pygame-ce
```

● Bilgi: pygame-ce Nedir?

pygame-ce, PyGame'in topluluk tarafından aktif olarak geliştirilen sürümüdür (Community Edition). Orijinal PyGame ile tamamen uyumlu olup daha hızlı güncellemeler ve performans iyileştirmeleri sunar. Kodlarında `import pygame` yazarken her iki sürüm de aynı şekilde çalışır.

Bu komut çalıştığında uv otomatik olarak PyGame'i indirip projeye ekler. `pyproject.toml` dosyasına bakacak olursan, bağımlılıklar kısmında `pygame-ce` satırını göreceksin.

Kurulumu Doğrulama

Her şeyin doğru kurulup kurulmadığını test etmek için kısa bir Python komutu çalıştıralım:

```
uv run python -c "import pygame; print(pygame.ver)"
```

Eğer ekranda PyGame'in versiyon numarasını görüyorsan (örneğin `2.5.2` gibi), tebrikler! PyGame başarıyla kuruldu ve çalışmaya hazır.

▲ Dikkat: Dikkat!

`uv run` komutu, projenin sanal ortamı içinde Python'u çalıştırır. Eğer `uv run` olmadan doğrudan `python` yazarsan, PyGame bulunamayabilir. Projende her zaman `uv run python dosya.py` şeklinde çalıştırmayı alışkanlık edin.

9.1.3 Proje Yapısı

Oluşturduğumuz projenin dizin yapısı şu şekilde görünür:

```
oyun_projesi/
  pyproject.toml
  .python-version
  hello.py
  .venv/
  uv.lock
```

Bu dosyaların ne işe yaradığına kısaca bakalım. `pyproject.toml` dosyası proje ayarlarını ve bağımlılıkları tutar. `.python-version` dosyası hangi Python sürümünü kullanacağını belirtir. `hello.py` dosyası uv'nin oluşturduğu örnek Python dosyasıdır, bunu kendi kodlarınla değiştireceksin. `.venv/` dizini projenin sanal ortamıdır, tüm kütüphaneler burada saklanır. Son olarak `uv.lock` dosyası bağımlılıkların tam sürüm bilgilerini kilitler.

💡 İpucu: Profesyonel İpucu

Her yeni oyun projen için ayrı bir dizin ve ayrı bir `uv init` kullan. Bu sayede projeler birbirini etkilemez ve her projenin kendi bağımlılıkları olur. Profesyonel yazılımcılar da projelerini bu şekilde izole eder.

9.2 PyGame Modül Yapısı

Bir oyun geliştirirken birçok farklı görevle uğraşman gerekir: ekrana bir şeyler çizmek, ses çalmak, klavye girdilerini okumak, zamanlama yapmak... Eğer bunların hepsi tek bir dev dosyada olsaydı, istediğin şeyi bulmak samanlıkta iğne aramak gibi olurdu. İşte bu yüzden PyGame, görevleri **modüller** halinde organize eder. Her modül belirli bir işten sorumludur.

■ Tanım: Modül

Modül, belirli bir konuyla ilgili fonksiyonları, sınıfları ve sabitleri bir arada tutan Python dosyasıdır. PyGame'de her modül, oyun geliştirmenin farklı bir yönünü ele alır. Modüller sayesinde sadece ihtiyacın olan araçlara kolayca ulaşabilirsin.

PyGame'i bir alet çantası gibi düşün. Bu çantanın içinde farklı bölmeler var: birinde çekiçler (grafik araçları), birinde tornavidalar (ses araçları), birinde de ölçü aletleri (zamanlama araçları) bulunuyor. Her bölme bir modüle karşılık gelir.

9.2.1 Temel Modüller

PyGame birçok modülden oluşur. Şimdi en sık kullanacağın modülleri tanıyalım. Aşağıdaki tablo, bu kitap boyunca karşılaşacağın temel modülleri özetliyor:

Tablo 9.1: PyGame'in temel modülleri

Modül	Görevi
<code>pygame.display</code> <code>pygame.event</code>	Oyun penceresini oluşturma ve yönetme Kullanıcı olaylarını (klavye, fare, pencere kapatma) yakalama
<code>pygame.draw</code>	Temel geometrik şekiller çizme (dikdörtgen, daire, çizgi)
<code>pygame.image</code>	Görsel dosyalarını yükleme ve kaydetme
<code>pygame.mixer</code>	Ses efektleri ve müzik çalma
<code>pygame.font</code>	Ekrana metin yazdırma
<code>pygame.time</code>	Zaman kontrolü ve FPS (kare hızı) ayarlama
<code>pygame.key</code>	Klavye durumunu sorgulama
<code>pygame.mouse</code>	Fare pozisyonu ve buton durumunu sorgulama
<code>pygame.sprite</code>	Oyun nesnelerini (sprite) organize etme ve yönetme
<code>pygame.transform</code> <code>pygame.Surface</code>	Görselleri döndürme, boyutlandırma ve çevirme Piksel tabanlı görsel yüzeyler oluşturma ve düzenleme

Tüm bu modülleri ezberlemen gerekmiyor. Kitap boyunca her birini sırasıyla ve detaylıca öğreneceksin. Şimdilik hangi modülün hangi işten sorumlu olduğuna dair genel bir fikrin olması yeterli.

9.2.2 Modüllere Erişim

PyGame'i kullanmak için önce onu programına dahil etmen (*import* etmen) gerekir. Bunun birkaç yolu var.

En yaygın kullanım şekli, tüm PyGame kütüphanesini dahil etmektir:

Python – PyGame'i Dahil Etme

```
1 import pygame
```

Bu satırdan sonra tüm modüllere `pygame.modül_adı` şeklinde erişebilirsin. Örneğin pencere oluşturmak için `pygame.display`, ses çalmak için `pygame.mixer` kullanırsın.

Bazen sadece belirli bir modülü dahil etmek isteyebilirsin:

Python – Tek Modül Dahil Etme

```
1 from pygame import display
2 from pygame import mixer
```

Bu durumda `pygame.display` yerine doğrudan `display` yazabilirsin. Ancak bu kullanımı bu kitapta tercih etmeyeceğiz. Neden mi? Çünkü `pygame.display.set_mode()` yazdığında kodun "PyGame'in display modülünün set_mode fonksiyonunu çağır" diye okunur ve hangi kütüphanenin fonksiyonunu kullandığın her zaman açıkça belli olur.

⚠ Dikkat: Dikkat!

`from pygame import *` şeklinde her şeyi doğrudan dahil etmekten kaçın. Bu kullanım, programdaki isimlerin çakışmasına neden olabilir ve kodun okunabilirliğini azaltır. Örneğin PyGame'in kendi `time` modülü ile Python'un standart `time` modülü karışabilir.

9.2.3 Modüllerin Bağımlılıkları

PyGame modülleri birbirinden bağımsız değildir. Bazı modüller başka modüllere ihtiyaç duyar. Örneğin `pygame.sprite` modülü arka planda `pygame.image` ve `pygame.display` modüllerini kullanır. Ama sen bunun ayrıntısıyla uğraşmak zorunda değilsin. PyGame'i başlattığında (`pygame.init()`) gerekli tüm modüller otomatik olarak hazırlanır.

Modüllerin bazılarının ayrıca kendi başlatma fonksiyonları vardır. Örneğin ses sistemi için `pygame.mixer.init()` veya yazı tipi sistemi için `pygame.font.init()` çağrılabilir. Ancak `pygame.init()` zaten bunların hepsini toplu olarak başlatır; bu konuyu bir sonraki bölümde detaylandıracağız.

9.2.4 Sabitler ve Olay Türleri

PyGame modüllerinin yanı sıra bir dizi **sabit** de tanımlar. Bu sabitler, olay türlerini, tuş kodlarını ve yapılandırma bayraklarını temsil eder.

Python – PyGame Sabitleri

```

1  # Olay turleri
2  pygame.QUIT           # Pencere kapatma olayı
3  pygame.KEYDOWN        # Tus basildi olayı
4  pygame.MOUSEBUTTONDOWN # Fare butonu basildi
5
6  # Tus kodlari
7  pygame.K_SPACE        # Bosluk tusu
8  pygame.K_LEFT         # Sol ok tusu
9  pygame.K_ESCAPE       # Escape tusu

```

Bu sabitlerin hepsi büyük harfle yazılır. Bu, Python dünyasında “bu değer değiştirilmemeli” anlamına gelen bir gelenektir. İlerleyen ünitelerde olay işleme ve klavye girdileri konularında bu sabitleri bolca kullanacaksın.

💡 İpucu: Profesyonel İpucu

PyGame'in tüm modüllerini ve sabitlerini ezberlemeye çalışma. Bunun yerine, ihtiyacın olduğunda PyGame'in resmi dokümantasyonuna (pygame.org/docs) başvur. Zamanla en sık kullandıkların doğal olarak aklında kalacaktır.

9.3 PyGame'i Başlatma ve Kapatma

Bir araba kullanmaya başlamadan önce kontağı çevirirsin, işin bittiğinde de motoru kapatırsın. PyGame de aynı mantıkla çalışır: programın başında PyGame'i **başlatman**, programın sonunda da düzgünce **kapatman** gerekir. Bu iki işlem, her PyGame programının temel iskeletini oluşturur.

9.3.1 pygame.init() Fonksiyonu

`pygame.init()` fonksiyonu, PyGame'in tüm alt sistemlerini (modüllerini) başlatır. Pencere yönetimi, ses sistemi, yazı tipi motoru, zamanlayıcılar... hepsi bu tek fonksiyon çağrısıyla hazır hale gelir.

■ Tanım: `pygame.init()`

`pygame.init()`, PyGame'in iç modüllerini başlatan fonksiyondur. Bu fonksiyon çağrılmadan PyGame'in diğer fonksiyonları düzgün çalışmaz. Her PyGame programının ilk adımı bu fonksiyonu çağırarak olmalıdır.

Kullanımı son derece basittir:

Python – PyGame'i Baslatma

```
1 import pygame
2
3 pygame.init()
4 print("PyGame basariyla baslatildi!")
```

Peki arka planda ne oluyor? `pygame.init()` çağrıldığında PyGame sırasıyla ses sistemini hazırlar (`mixer`), yazı tipi motorunu başlatır (`font`), ekran modülünü hazırlar (`display`), zamanlayıcıları kurar (`time`), girdi sistemlerini etkinleştirir (`event`, `key`, `mouse`) ve diğer alt sistemleri başlatır.

Başlatma Sonucunu Kontrol Etme

`pygame.init()` fonksiyonu aslında bir değer döndürür: başarıyla başlatılan ve başlatılamayan modül sayılarından oluşan bir çift (tuple).

Python – Baslatma Sonucunu Kontrol Etme

```
1 import pygame
2
3 basarili, basarisiz = pygame.init()
4 print(f"Basarili: {basarili}, Basarisiz: {basarisiz}")
```

Normal şartlarda `basarisiz` değeri 0 olmalıdır. Eğer 0'dan büyükse, bazı modüller başlatılamamış demektir. Bu durum genellikle ses kartı veya ekran sürücüsü sorunlarından kaynaklanır.

● Bilgi: Ne Zaman Kontrol Etmeli?

Basit projelerde bu kontrolü yapmana genellikle gerek yoktur. Ancak profesyonel bir oyun geliştirirken veya farklı bilgisayarlarda çalışacak bir program yazarken, başlatma sonucunu kontrol etmek iyi bir alışkanlıktır. Eğer ses modülü başlatılamazsa oyunu sessiz modda çalıştırabilirsin mesela.

Tek Modül Başlatma

Bazen tüm modülleri başlatmak yerine sadece ihtiyacın olan modülü başlatmak isteyebilirsin. Bu durumda ilgili modülün kendi `init()` fonksiyonunu kullanırsın:

Python – Tek Modül Baslatma

```
1 import pygame
2
3 # Sadece ekran modülünü baslat
4 pygame.display.init()
5
6 # Sadece ses modülünü baslat
7 pygame.mixer.init()
```

Bu yaklaşım, programın başlangıç süresini kısaltabilir çünkü kullanmadığın modüller başlatılmaz. Ancak bu kitapta genellikle `pygame.init()` kullanacağız çünkü tüm modüllere ihtiyacımız olacak ve başlatma süresi zaten çok kısadır.

9.3.2 pygame.quit() Fonksiyonu

`pygame.quit()` fonksiyonu, `pygame.init()` fonksiyonunun tam tersidir. PyGame'in kullandığı tüm kaynakları serbest bırakır ve alt sistemleri düzgünce kapatır.

■ Tanım: pygame.quit()

`pygame.quit()`, PyGame'in tüm alt sistemlerini kapatır ve kullandığı sistem kaynaklarını (bellek, ses aygıtı, ekran bağlantısı) serbest bırakır. Her PyGame programının son adımı bu fonksiyonu çağırarak olmalıdır.

Python – PyGame'i Kapatma

```
1 import pygame
2
3 pygame.init()
4 # ... oyun kodlari burada ...
5 pygame.quit()
6 print("PyGame düzgünce kapatıldı!")
```

⚠ Dikkat: Dikkat!

`pygame.quit()` çağırmadan programı sonlandırırırsan, bazı kaynaklar (özellikle ses aygıtı ve ekran bağlantısı) düzgün serbest bırakılmayabilir. Bu durum nadiren sorun yaratsa da, iyi bir programcı her zaman kaynakları temiz bir şekilde kapatır.

9.3.3 PyGame Programının İskeleti

Her PyGame programı aynı temel yapıyı takip eder. Bu yapıyı bir şablon olarak aklında tut:

Python – PyGame Program Iskeleti

```
1 import pygame
2
3 # 1. PyGame'i baslat
4 pygame.init()
5
6 # 2. Oyun penceresini olustur
7 # (bir sonraki unitede ogrenecegiz)
8
9 # 3. Ana oyun dongusu
10 # (Bolum 3, Unite 3'te ogrenecegiz)
11
12 # 4. PyGame'i kapat
13 pygame.quit()
```

Bu dört adım, bundan sonra yazacağın her PyGame programının temelini oluşturacak. İlk ve son adımları az önce öğrendin. Ortadaki adımları ise ilerleyen ünitelerde dolduracağız.

Şimdi bu iskeleti kullanarak gerçek bir program yazalım. Henüz pencere oluşturmayı öğrenmediğimiz için basit bir test programı yapacağız:

Python – İlk PyGame Test Programı

```
1 import pygame
2
3 # PyGame'i baslat ve sonucu kontrol et
4 basarili, basarisiz = pygame.init()
5
6 # Baslama bilgisini yazdir
7 print(f"PyGame surumu: {pygame.ver}")
8 print(f"Baslatilan modul sayisi: {basarili}")
9 print(f"Basarisiz modul sayisi: {basarisiz}")
10
11 # Bazi modullerin durumunu kontrol et
12 print(f"\nDisplay modulu aktif mi:
13     ↳ {pygame.display.get_init()}")
```

```

13 print(f"Mixer modulu aktif mi: {pygame.mixer.get_init()
    ↳ is not None}")
14 print(f"Font modulu aktif mi: {pygame.font.get_init()}")
15
16 # PyGame'i kapat
17 pygame.quit()
18
19 # Kapandıktan sonra kontrol
20 print(f"\nDisplay modulu aktif mi:
    ↳ {pygame.display.get_init()}")
21 print("Program başarıyla tamamlandı!")

```

Bu programı `uv run python test_pygame.py` komutuyla çalıştırdığında, PyGame'in hangi sürümde olduğunu, kaç modülün başarıyla başlatıldığını ve `quit()` sonrasında modüllerin kapandığını göreceksin.

💡 İpucu: Profesyonel İpucu

`get_init()` fonksiyonu, bir modülün başlatılıp başlatılmadığını kontrol eder. Bu fonksiyonu kullanarak programın herhangi bir noktasında modüllerin durumunu sorgulayabilirsin. Hata ayıklama (debug) sırasında çok işe yarar.

9.3.4 sys.exit() ile Temiz Çıkış

Profesyonel PyGame programlarında genellikle Python'un `sys` modülü de kullanılır. `sys.exit()` fonksiyonu, programı tamamen sonlandırır ve işletim sistemine "program başarıyla bitti" sinyali gönderir:

Python – sys.exit ile Temiz Çıkış

```

1 import pygame
2 import sys
3
4 pygame.init()
5
6 # ... oyun kodlari ...
7
8 pygame.quit()
9 sys.exit()

```

Bu üçlü kombinasyon (`import`, `init`, `quit` + `sys.exit`) profesyonel PyGame programlarının standart kapanış rutinidir. İlerleyen ünitelerde ana oyun döngüsünü öğrendiğinde, bu yapının neden önemli olduğunu daha iyi anlayacaksın.

9.3.5 Sık Yapılan Hatalar

PyGame'i başlatma ve kapatma sırasında en sık yapılan hataları bilmek, ileride zaman kazandırır.

Birincisi, `pygame.init()` çağırmaı unutmaktır. Eğer `pygame.init()` çağırmadan bir modül kullanmaya çalışırsan, "pygame.error: video system not initialized" gibi bir hata alırsın. Çözüm basittir: programın başına `pygame.init()` ekle.

İkincisi, `init()` ile `__init__()` karıştırmaktır. `pygame.init()` bir fonksiyon çağırısıdır ve PyGame'in kendisini başlatır. `__init__` ise Python sınıflarının yapıcı metodudur; bunları ilerleyen bölümlerde Sprite sınıflarıyla öğreneceksin.

Üçüncüsü, `quit()` sonrasında PyGame fonksiyonları kullanmaya çalışmaktır. `pygame.quit()` çağırıldıktan sonra PyGame fonksiyonları artık çalışmaz. Eğer tekrar kullanman gerekirse `pygame.init()` ile yeniden başlatman gerekir.

● Bilgi: Hata Mesajlarını Oku

PyGame'in hata mesajları genellikle sorunun ne olduğunu açıkça belirtir. "not initialized" görüyorsan `init()` eksiktir, "display surface quit" görüyorsan pencere kapanmış ama hala ona erişmeye çalışıyorsundur. Hata mesajlarını okumayı alışkanlık haline getir!

9.4 Ünite Özeti

Bu ünite de oyun geliştirme yolculuğunda büyük bir adım attın! Python temellerini öğrendikten sonra artık PyGame kütüphanesiyle tanıştın ve ilk PyGame projesini oluşturdu.

9.4.1 Bu Ünite de Öğrendiklerin

- **PyGame Nedir:** Python ile 2D oyunlar ve multimedia uygulamaları geliştirmek için kullanılan açık kaynaklı bir kütüphanedir
- **Proje Oluşturma:** `uv init` ile proje başlatıp `uv add pygame-ce` ile PyGame'i ekleyebilirsin
- **Modül Yapısı:** PyGame, `display`, `event`, `draw`, `mixer` gibi modüllerden oluşur ve her modül farklı bir görevi üstlenir
- **Başlatma:** `pygame.init()` tüm alt sistemleri başlatır ve her programın ilk adımı olmalıdır
- **Kapatma:** `pygame.quit()` kaynakları serbest bırakır ve her programın son adımı olmalıdır

9.4.2 Anahtar Kavramlar

PyGame Python ile 2D oyun geliştirmek için kullanılan kütüphane

SDL PyGame'in altyapısını oluşturan düşük seviyeli C kütüphanesi

pygame-ce PyGame'in aktif olarak geliştirilen topluluk sürümü

Modül Belirli bir görev için fonksiyon ve sınıfları bir arada tutan yapı

uv Modern Python proje ve paket yöneticisi

pygame.init() PyGame alt sistemlerini başlatan fonksiyon

pygame.quit() PyGame kaynaklarını serbest bırakan fonksiyon

pyproject.toml Proje ayarlarını ve bağımlılıkları tutan yapılandırma dosyası

9.4.3 Önemli Komutlar

Tablo 9.2: Bu ünitedeki önemli komutlar

Komut	Açıklama
<code>uv init</code>	Yeni bir Python projesi oluşturur
<code>uv add pygame-ce</code>	PyGame'i projeye bağımlılık olarak ekler
<code>uv run python dosya.py</code>	Proje ortamında Python dosyasını çalıştırır
<code>import pygame</code>	PyGame kütüphanesini programa dahil eder
<code>pygame.init()</code>	Tüm PyGame alt sistemlerini başlatır
<code>pygame.quit()</code>	Tüm PyGame alt sistemlerini kapatır
<code>pygame.display.get_init()</code>	Display modülünün aktif olup olmadığını döndürür

9.4.4 Sıradaki Adım

Bir sonraki ünite **oyun penceresi oluşturmaya** öğreneceksin! `pygame.display.set_mode()` fonksiyonuyla ekranda bir pencere açacak, boyutunu ayarlayacak ve başlığını belirleyeceksin. Sonunda ekranda kendi oyun pencereni göreceksin; bu, oyun geliştirmenin en heyecan verici anlarından biri!

💡 İpucu: Pratik Yap!

Bu üniteyi bitirmeden önce şunları dene:

1. Kendi bilgisayarında `uv init` ile yeni bir proje oluştur ve `uv add pygame-ce` ile PyGame'i ekle
2. `pygame.init()` sonucunu kontrol eden bir test programı yaz
3. PyGame'in hangi modüllerden oluştuğunu kendi cümlelerinle bir arkadaşına anlat
4. PyGame dokümantasyonunu (pygame.org/docs) ziyaret edip en az iki modülün sayfasına göz at

Oyun Penceresi Oluřturma

“Bir oyunun ilk sihri, ekranda kendi pencerenin açıldığı andır. O an artık sadece izleyici değil, yaratıcısın.”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de řu konuları öğreneceksin:

- `pygame.display.set_mode()` ile oyun penceresi oluřturma
- Pencere boyutlarını ve çözünürlüğü ayarlama
- PyGame koordinat sistemini anlama
- Pencere başlığı ve ikonu belirleme
- Tam ekran modu ve farklı ekran modları arasında geçiř yapma

10.1 Oyun Penceresi Oluřturma

Bir önceki ünite de PyGame’i kurdun, modüllerini tanıdın ve `pygame.init()` ile sistemi başlattın. Artık en heyecan verici adıma geçme zamanı: kendi oyun penceresinin ekranda belirmesini izlemek! Düşünsene, o siyah dikdörtgen aslında senin oyun dünyanın kapısı. İçine düşmanlar yerleřtireceksin, karakterler kořturacaksın ve patlamalar yaratacaksın. Ama önce o kapıyı açalım.

■ Tanım: Oyun Penceresi (Display Surface)

Oyun penceresi, PyGame’in ekranda oluřturduđu ve üzerine çizim yapabileceğin grafik alanıdır. Bu alana *Surface* (yüzey) denir. `pygame.display.set_mode()` fonksiyonu, bu yüzeyi oluřturur ve sana bir *Surface* nesnesi döndürür.

10.1.1 İlk Pencereyi Açmak

Bir oyun penceresi oluşturmak için tek bir satır yeterlidir. `pygame.display` modülündeki `set_mode()` fonksiyonunu çağırıyorsun ve pencere boyutlarını bir demet (tuple) olarak veriyorsun:

Python – En Basit Pencere

```
1 import pygame
2
3 pygame.init()
4
5 # 800 piksel genişlik, 600 piksel yükseklik
6 ekran = pygame.display.set_mode((800, 600))
```

Bu üç satırda neler oluyor? Birincisi, `pygame.init()` tüm PyGame alt sistemlerini başlatıyor – bunu bir önceki ünite de öğrenmiştin. İkincisi, `set_mode()` fonksiyonu çift paranteze dikkat et: dıştaki parantez fonksiyon çağırısı, içteki parantez ise bir Python demet (tuple) oluşturuyor. Bu demet, pencerenin genişlik ve yükseklik değerlerini piksel cinsinden belirtiyor.

⚠ Dikkat: Çift Parantez Tuzağı

`set_mode(800, 600)` yazmak **hata** verir! Fonksiyon tek bir parametre bekler: bir demet. Doğru kullanım `set_mode((800, 600))` şeklindedir. İç parantezleri unutma.

10.1.2 PyGame Koordinat Sistemi

Penceremizi oluşturduk ama içine bir şey çizmeden önce PyGame'in koordinat sistemini anlamak gerekiyor. Matematik derslerinden alışkın olduğun koordinat sistemiyle PyGame'inki arasında önemli bir fark var.

Matematik dersinde y eksenini yukarı doğru artarken, PyGame'de (ve neredeyse tüm grafik kütüphanelerinde) y eksenini **aşağı doğru** artar. Bunun tarihsel bir nedeni var: eski bilgisayar monitörleri ekranı satır satır, yukarıdan aşağıya doğru çizerdi. Bu gelenek günümüze kadar geldi.

Tablo 10.1: PyGame koordinat sistemi – temel noktalar

Konum	Koordinat	Açıklama
Sol üst köşe	(0, 0)	Başlangıç noktası
Sağ üst köşe	(799, 0)	x maksimum, y minimum
Sol alt köşe	(0, 599)	x minimum, y maksimum
Sağ alt köşe	(799, 599)	Her iki eksen de maksimum
Tam ortası	(400, 300)	genişlik/2, yükseklik/2

Dikkat edersen koordinatlar 800 ve 600 değil, 799 ve 599'da bitiyor. Bunun nedeni saymanın 0'dan başlamasıdır. 800 piksellik bir genişlikte pikseller 0'dan 799'a kadar numaralanır.

● Bilgi: Neden Sol Üst Köőe (0,0)?

Eski CRT monitörler görüntüyü bir elektron ışınlıyyla sol üst köőeden başlayarak, satır satır aőağı doğru çizerdi. Bu “tarama” yöntemi, grafik programlamada koordinat sisteminin sol üst köőeden başlamasına neden oldu. Bugün LCD ekranlar farklı çalışsa da bu gelenek devam ediyor.

10.1.3 Pencere Boyutunu Akıllıca Seçmek

Peki 800x600 nereden çıktı? Aslında pencerenin boyutunu istediğın gibi ayarlayabilirsin. Ancak bazı boyutlar oyunlar için daha yaygındır:

Tablo 10.2: Yaygın oyun penceresi çözünürlükleri

Çözünürlük	En-Boy Oranı	Kullanım	Uygun Olduğı Oyunlar
640 × 480	4:3	Retro oyunlar	Piksel sanat tarzı
800 × 600	4:3	Küçük projeler	Eğitim, prototip
1024 × 768	4:3	Orta ölçek	Strateji, bulmaca
1280 × 720	16:9	HD	Modern 2D oyunlar
1920 × 1080	16:9	Full HD	Büyük projeler

Bu kitap boyunca çoğı örnekte **800 × 600** kullanacağız. Bu boyut hem yeterince büyük hem de ekranda rahatça yer kaplıyor.

💡 İpucu: Profesyonel İpucu

Pencere boyutlarını doğrudan sayı olarak yazmak yerine, sabitlere atamak daha temiz bir kod sağlar. Böylece boyutu değıştirmek istediğinde tek bir yeri güncellemen yeterlidir:

Python – Sabitlerle Pencere Boyutu

```
1 GENISLIK = 800
2 YUKSEKLIK = 600
3
4 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
```

10.1.4 set_mode() Parametreleri

set_mode() fonksiyonu aslında sadece boyut almaz. Tam sözdizimi şöyledir:

Python – set_mode Tam Sozdizimi

```
1 pygame.display.set_mode(
2     size=(0, 0),          # (genislik, yukseklik) demeti
3     flags=0,              # Pencere bayraklari (modlar)
4     depth=0,              # Renk derinligi (bit)
```

```

5     display=0,           # Hangi monitorde acilacak
6     vsync=0              # Dikey senkronizasyon
7 )

```

Şimdilik `size` parametresini kullanman yeterli. Diğer parametreleri bu ünitenin ilerleyen kısımlarında ve sonraki ünitelerde öğreneceksin. Yalnız `flags` parametresine kısaca değinelim çünkü pencere davranışını belirleyen çok önemli bir parametre:

Tablo 10.3: `set_mode()` bayrak sabitleri

Bayrak	Açıklama
<code>pygame.FULLSCREEN</code>	Tam ekran modu
<code>pygame.RESIZABLE</code>	Kullanıcı pencereyi boyutlandırabilir
<code>pygame.NOFRAME</code>	Çerçevesiz pencere (başlık çubuğu yok)
<code>pygame.SCALED</code>	Pencere otomatik ölçeklenir
<code>pygame.SHOWN</code>	Pencere görünür (varsayılan)
<code>pygame.HIDDEN</code>	Pencere gizli başlar

Bayrakları birleştirmek için Python'un bitor (`|`) operatörünü kullanabilirsin. Örneğin hem boyutlandırılabilir hem de çerçevesiz bir pencere istersen `pygame.RESIZABLE | pygame.NOFRAME` yazarsın.

10.1.5 Boyutlandırılabilir Pencere

Oyuncuların pencere boyutunu fare ile değiştirebilmesini istiyorsan `RESIZABLE` bayrağını kullanırsın:

Python – Boyutlandırılabilir Pencere

```

1 ekran = pygame.display.set_mode((800, 600),
  ↪   pygame.RESIZABLE)

```

Bu bayrak aktif olduğunda kullanıcı pencerenin kenarlarını sürükleyerek boyutu değiştirebilir. Ancak dikkatli ol: pencere boyutu değiştiğinde oyun içeriğinin de buna uyum sağlaması gerekir. Bu konuyu ilerideki ünitelerde ele alacağız.

10.1.6 Surface Nesnesi

`set_mode()` fonksiyonunun döndürdüğü `Surface` nesnesi, oyun geliştirmede en çok kullanacağın nesnelerden biridir. Bu nesne üzerinden ekrana şekiller çizebilir, görseller yapıştırabilir ve arka plan rengini değiştirebilirsin.

Python – Surface ile Çalışma

```

1 # Pencere olustur ve Surface nesnesini al
2 ekran = pygame.display.set_mode((800, 600))
3
4 # Ekrani siyah renkle doldur

```

```

5 ekran.fill((0, 0, 0))
6
7 # Ekranı beyaz renkle doldur
8 ekran.fill((255, 255, 255))
9
10 # Koyu mavi arka plan
11 ekran.fill((30, 30, 60))

```

`fill()` metodu bir RGB renk demeti alır. RGB, kırmızı (Red), yeşil (Green) ve mavi (Blue) değerlerinin 0–255 arasında belirtildiği bir renk sistemidir. Her üç değer 0 olduğunda siyah, her üçü 255 olduğunda beyaz elde edersin. Renkleri detaylı olarak Ünite 4'te (Temel Çizim İşlemleri) öğreneceksin.

10.1.7 Tam Bir Örnek

Şimdi öğrendiklerini bir araya getirelim. Aşağıdaki parçacıklar bir oyun penceresinin temel yapısını gösterir. Önce sabitleri ve pencereyi oluşturalım:

Python – Pencere Oluşturma

```

1 GENISLIK = 800
2 YUKSEKLIK = 600
3 ARKAPLAN_RENGI = (30, 30, 60) # Koyu mavi
4
5 pygame.init()
6 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
7 pygame.display.set_caption("İlk Oyun Penceresi")

```

Sonra ana döngüde olayları işleyip ekranı güncelle:

Python – Ana Döngü

```

1 calistir = True
2 while calistir:
3     for olay in pygame.event.get():
4         if olay.type == pygame.QUIT:
5             calistir = False
6
7     ekran.fill(ARKAPLAN_RENGI)
8     pygame.display.flip()
9
10 pygame.quit()

```

● Bilgi: Tam Kod

Bu programın çalıştırılabilir tam halini `kod\ornekleri/bolum03/unite2/01\_temel\_pencere.py` dosyasında bulabilirsin.

Bu kodda dikkat edilmesi gereken noktalar:

1. `GENISLIK` ve `YUKSEKLIK` sabitleri pencere boyutunu tanımlıyor
2. `set_mode()` pencereyi oluşturup bir Surface nesnesi döndürüyor
3. `while` `calistir` döngüsü pencerenin açık kalmasını sağlıyor
4. `pygame.QUIT` olayı, kullanıcı pencereyi kapattığında tetikleniyor
5. `fill()` her karede arka planı boyuyor
6. `flip()` çizilen içeriği ekrana yansıtıyor

Peki `flip()` neden gerekli? PyGame çift tamponlama (*double buffering*) kullanır. Sen arka planda bir yüzeye çizersin, sonra `flip()` ile o yüzeyi ekrana aktarırsın. Bu sayede kullanıcı yarım kalmış çizimleri görmez ve görüntü titrememesi oluşmaz. Bunu bir sahne arkası gibi düşünebilirsin: dekorları sahne arkasında hazırlarsın, sonra perdeden bir anda açarsın.

⚠ Dikkat: Pencere Anında Kapanıyor mu?

Eğer programı çalıştırdığında pencere bir anlığına görünüp kapanıyorsa, muhtemelen oyun döngüsünü yazmadın veya döngüde bir hata var. Pencereyi açık tutmak için `while` döngüsü ve olay işleme şarttır. Oyun döngüsünü Ünite 3'te detaylı öğreneceksin.

10.2 Pencere Başlığı ve İkonu

Bir önceki konuda oyun penceresini oluşturdu ve ekranda koyu mavi bir dikdörtgen belirdi. Güzel bir başlangıç ama dikkat ettiysen pencerenin başlık çubuğunda "pygame window" gibi sıradan bir metin yazıyor. Oyuncularının senin oyununu açtığında gördüğü ilk şey bu başlık çubuğudur. Bir oyunun profesyonel görünmesi, bu küçük detaylarla başlar.

Düşün: Steam kütüphanesindeki oyunlara bak. Her birinin başlık çubuğunda oyunun adı ve görev çubuğunda kendine özel bir ikon var. Bu detaylar basit görünse de oyuncuya "Bu ciddi bir proje" mesajı verir.

10.2.1 Pencere Başlığını Ayarlama

Pencerenin başlık çubuğundaki metni değiştirmek için `pygame.display.set_caption()` fonksiyonunu kullanırsın:

Python – Pencere Basligi

```
1 pygame.display.set_caption("Uzay Kacisi - v1.0")
```

Bu kadar basit! Fonksiyon bir string alır ve onu pencerenin başlık çubuğuna yazar. Başlığı istediğin zaman değiştirebilirsin, hatta oyun sırasında bile:

Python – Dinamik Baslik

```
1 # Oyun basliginda
2 pygame.display.set_caption("Uzay Kacisi")
3
4 # Oyun duraklatildiginda
```

```

5 pygame.display.set_caption("Uzay Kacisi - DURAKLATILDI")
6
7 # Skor gostermek icin
8 pygame.display.set_caption(f"Uzay Kacisi - Skor: {skor}")

```

Üçüncü örnekteki **f-string** yapısını hatırlıyor musun? Bölüm 1’de (Ünite 4) öğrendiğin bu yapı sayesinde değişken değerlerini doğrudan başlığa ekleyebilirsin. Bu teknik, oyun geliştirmede sıkça kullanılır: FPS değerini, skor’u veya oyunun durumunu başlık çubuğunda göstermek hata ayıklama (debug) sırasında çok işe yarar.

■ Tanım: Caption (Başlık)

Bir pencerenin başlık çubuğunda görünen metin. İşletim sistemi bu metni görev çubuğunda ve pencere listesinde de kullanır.

10.2.2 Mevcut Başlığı Okumak

Eğer pencerenin o anki başlığını öğrenmek istersen `pygame.display.get_caption()` fonksiyonunu kullanabilirsin:

Python – Basligi Oku

```

1 baslik = pygame.display.get_caption()
2 print(baslik) # ('Uzay Kacisi', 'Uzay Kacisi')

```

Bu fonksiyon iki elemanlı bir demet döndürür. İlk eleman başlık metnini, ikinci eleman ise bazı sistemlerde simge durumuna küçültülmüş pencere için kullanılan kısa başlığı içerir. Çoğu durumda ikisi aynıdır.

10.2.3 Pencere İkonu Ayarlama

Pencerenin sol üst köşesinde ve görev çubuğunda görünen küçük görsele *ikon* denir. PyGame varsayılan olarak kendi logosunu kullanır ama sen bunu değiştirebilirsin:

Python – Pencere İkonu

```

1 # İkon gorselini yukle
2 ikon = pygame.image.load("ikon.png")
3
4 # İkonu ayarla
5 pygame.display.set_icon(ikon)

```

⚠ Dikkat: Sıralama Önemli

`set_icon()` fonksiyonunu `set_mode()`'dan **önce** çağırman gerekir. Bazı işletim sistemlerinde pencere oluşturulduktan sonra ikonu deęiřtirmek etkisiz kalabilir. Güvenli sıralama řöyledir:

1. `pygame.init()`
2. `pygame.display.set_icon(ikon)`
3. `pygame.display.set_mode(...)`
4. `pygame.display.set_caption(...)`

10.2.4 İkon İçin Görsel Hazırlama

İkon görseli hazırlarken dikkat etmen gereken birkaç nokta var.

Boyut konusunda, ikonlar genellikle 32×32 veya 64×64 piksel boyutundadır. Daha büyük görseller otomatik olarak küçültülür ama kalite kaybı yaşanabilir. En temiz sonuç için görseli tam ikon boyutunda hazırlamak en iyisidir.

Format konusunda, PNG formatı saydam (transparan) arka plan desteęi sunduęu için en iyi seçimdir. ICO formatı da kullanılabilir ama PNG daha esnek ve yaygındır.

Şeffaflık konusunda, ikon arka planının saydam olmasını istiyorsan PNG formatında alfa kanalı olan bir görsel kullan. Böylece ikonun kare bir kutu yerine řekline uygun görünür.

💡 İpucu: Profesyonel İpucu

Oyun geliřtirmenin ilk aşamalarında ikon için vakit harcamana gerek yok. Basit bir renkli kare bile işini görür. İkonu oyunun son aşamalarında, görsel kimlik belirlendięinde tasarla.

10.2.5 Tam Bir Örnek

Başlık ve ikon ayarlama işlemlerini bir arada görelim. Önce ikonun güvenli řekilde yüklenmesi:

Python – İkon Yükleme

```
1  # İkon ayarla (set_mode'dan önce!)
2  try:
3      ikon = pygame.image.load("ikon.png")
4      pygame.display.set_icon(ikon)
5  except FileNotFoundError:
6      print("[!] İkon dosyası bulunamadı, varsayılan
      ↪ kullanılacak.")
7
8  ekran = pygame.display.set_mode((800, 600))
9  pygame.display.set_caption("Uzay Kacisi")
```

Döngü içinde başlığı dinamik olarak güncellemek de mümkün:

Python – Dinamik Başlık Güncelleme

```

1 elif olay.type == pygame.KEYDOWN:
2     if olay.key == pygame.K_SPACE:
3         skor += 10
4         pygame.display.set_caption(
5             f"Uzay Kacisi - Skor: {skor}"
6         )

```

● Bilgi: Tam Kod

Bu programın çalıştırılabilir tam halini kod_ornekleri/bolum03/unite2/02_baslik_ikon.py dosyasında bulabilirsiniz.

Bu programda dikkat çekilecek önemli bir detay var: `try-except` yapısı. `image.load()` fonksiyonu, dosya bulunamazsa hata fırlatır. Henüz bir ikon dosyası yoksa program çökmek yerine uyarı vererek devam eder. Bu tarz savunmacı programlama, oyun geliştirmede sıkça başvuracağın bir tekniktir.

● Bilgi: Platformlar Arası Farklılıklar

Windows, macOS ve Linux'ta pencere başlığı ve ikonu biraz farklı görünebilir. Her işletim sistemi kendi pencere yöneticisini kullanır. Oyununu test ederken bunu aklında tut. Başlık ve ikon davranışında tutarsızlıklar görürsen, bu genellikle PyGame'in değil işletim sisteminin bir özelliğidir.

10.3 Tam Ekran ve Ekran Modları

Bir oyunu ilk başlattığında genellikle karşına iki seçenek çıkar: pencereli mi oynayacaksın, tam ekran mı? Bu karar, oyunun görsel deneyimini doğrudan etkiler. Tam ekran modu oyuncuyu oyunun dünyasına çeker, pencereli mod ise hızlıca diğer uygulamalara geçmeyi kolaylaştırır. Bu konuda PyGame'in sunduğu ekran modlarını ve aralarında nasıl geçiş yapacağını öğreneceksin.

■ Tanım: Tam Ekran Modu (Fullscreen)

Tam ekran modunda oyun penceresi monitörün tamamını kaplar. İşletim sisteminin görev çubuğu, başlık çubuğu ve diğer pencereler gizlenir. Oyuncu yalnızca oyunu görür.

10.3.1 Tam Ekran Pencere Oluşturma

Tam ekran modu için `set_mode()` fonksiyonuna `pygame.FULLSCREEN` bayrağını geçersin:

Python – Tam Ekran Modu

```
1 # Tam ekran pencere olustur
2 ekran = pygame.display.set_mode((1280, 720),
  ↳ pygame.FULLSCREEN)
```

Tam ekran modunda belirttiğin çözünürlük, monitörün desteklediği çözünürlüklerden biri olmalıdır. Eğer monitörün desteklemediği bir çözünürlük verersen, PyGame en yakın desteklenen çözünürlüğe geçer veya hata fırlatır.

⚠ Dikkat: Tam Ekrandan Çıkış

Tam ekran modunda **Alt+F4** veya oyun içi bir çıkış mekanizması olmadan sıkışabilirsin! Geliştirme aşamasında tam ekranı test ederken mutlaka bir çıkış tuşu tanımla. Örneğin **Esc** tuşuna basıldığında programı kapat. Aksi halde bilgisayarını yeniden başlatmak zorunda kalabilirsin.

10.3.2 Monitör Çözünürlüğünü Algılama

Tam ekran modunu kullanırken monitörün gerçek çözünürlüğünü bilmek önemlidir. PyGame bu bilgiyi sana sağlıyor:

Python – Cozunurluk Algilama

```
1 import pygame
2 pygame.init()
3
4 # Ekran bilgisi nesnesi
5 bilgi = pygame.display.Info()
6 print(f"Ekran genisligi: {bilgi.current_w}")
7 print(f"Ekran yuksekligi: {bilgi.current_h}")
```

`display.Info()` nesnesi, monitör hakkında birçok bilgi sağlar. En sık kullanılanları `current_w` (mevcut genişlik) ve `current_h` (mevcut yükseklik) özellikleridir.

Bu bilgiyi akıllıca kullanabilirsin. Örneğin, pencerenin monitör boyutuna otomatik uyum sağlamasını istiyorsan:

Python – Otomatik Boyut

```
1 pygame.init()
2
3 bilgi = pygame.display.Info()
4 GENISLIK = bilgi.current_w
5 YUKSEKLIK = bilgi.current_h
6
7 # Monitogun tam cozunurlugunde tam ekran
8 ekran = pygame.display.set_mode(
9     (GENISLIK, YUKSEKLIK), pygame.FULLSCREEN
10 )
```

İpucu: Profesyonel İpucu

Geliştirme sırasında tam ekran yerine pencereci modda çalışmak çok daha pratiktir. Hata ayıklama, konsol çıktılarını okuma ve hızlı geçiş yapma pencereci modda kolaydır. Tam ekranı sadece son testlerde kullan.

10.3.3 Ekran Modları Karşılaştırması

PyGame’de kullanabileceğin üç temel ekran modu var. Her birinin avantajlarını ve dezavantajlarını bilmek, oyunun için doğru seçimi yapmanı sağlar.

Pencereci Mod en basit ve varsayılan moddur. Hiçbir bayrak vermeden `set_mode()` çağırıldığında bu modu kullanırsın. Pencere, işletim sisteminin diğer pencereleriyle birlikte görünür. Kullanıcı pencereyi taşıyabilir, küçültebilir ve diğer uygulamalarla kolayca geçiş yapabilir. Geliştirme sırasında en pratik moddur.

Tam Ekran Modu oyunun monitörün tamamını kaplamasını sağlar. Daha sürükleyici bir deneyim sunar çünkü oyuncu dikkatini dağıtan başka pencereler görmez. Ancak çözünürlük uyumsuzluğu sorunları yaşanabilir ve geliştirme sırasında kullanımı zahmetlidir.

Çerçevesiz Tam Ekran (*borderless fullscreen* veya *windowed fullscreen*), pencereci mod ile tam ekran modun birleşimidir. Pencere çerçevesi ve başlık çubuğu olmadan monitörü tamamen kaplar ama aslında hala bir penceredir. Bu sayede **Alt+Tab** ile hızlıca diğer uygulamalara geçebilirsiniz.

Python – Uc Ekran Modu

```

1  # Pencereci mod (varsayılan)
2  ekran = pygame.display.set_mode((800, 600))
3
4  # Tam ekran modu
5  ekran = pygame.display.set_mode((1280, 720),
6      ↪ pygame.FULLSCREEN)
7
8  # Çerçevesiz tam ekran
9  bilgi = pygame.display.Info()
10 ekran = pygame.display.set_mode(
11     (bilgi.current_w, bilgi.current_h), pygame.NOFRAME

```

Tablo 10.4: Ekran modları karşılaştırması

Özellik	Pencereci	Tam Ekran	Çerçevesiz
Başlık çubuğu	Var	Yok	Yok
Alt+Tab kolaylığı	Kolay	Yavaş	Kolay
Performans	Normal	En iyi	Normal
Çözünürlük esnekliği	Yüksek	Sınırlı	Yüksek
Geliştirme kolaylığı	En iyi	Zor	Orta
Oyuncu deneyimi	Normal	En iyi	İyi

10.3.4 Modlar Arası Geçiş

Birçok oyun, oyuncuya tam ekran ve pencereli mod arasında geçiş yapma olanağı sunar. Bunu PyGame'de `pygame.display.toggle_fullscreen()` fonksiyonuyla yapabilirsin:

Python – Mod Gecisi

```
1  # F11 tusuna basildiginda tam ekran gecisi
2  for olay in pygame.event.get():
3      if olay.type == pygame.KEYDOWN:
4          if olay.key == pygame.K_F11:
5              pygame.display.toggle_fullscreen()
```

Bu fonksiyon mevcut modun tersine geçer: penceredeysen tam ekrana, tam ekrandaysan pencereye döner. Ancak her platformda aynı şekilde çalışmayabilir. Daha güvenilir bir yöntem, modu kendin takip edip pencereyi yeniden oluşturmaktır:

Python – Güvenilir Mod Gecisi

```
1  tam_ekran = False
2  GENISLIK, YUKSEKLIK = 800, 600
3
4  def ekran_modunu_degistir():
5      global ekran, tam_ekran
6      tam_ekran = not tam_ekran
7      if tam_ekran:
8          ekran = pygame.display.set_mode(
9              (GENISLIK, YUKSEKLIK), pygame.FULLSCREEN
10         )
11     else:
12         ekran = pygame.display.set_mode(
13             (GENISLIK, YUKSEKLIK)
14         )
```

Bilgi: SCALED Bayrağı

PyGame 2.0 ile gelen `pygame.SCALED` bayrağı, düşük çözünürlüklü oyunları otomatik olarak monitöre uygun şekilde büyütür. Piksel sanat tarzı bir oyun yapıyorsan, örneğin 320×240 çözünürlükte tasarlayıp büyük ekranlarda da güzel görünmesini istiyorsan bu bayrak çok işe yarar:

Python – Otomatik Ölçekleme

```
1  # 320x240 piksel oyun, ekrana otomatik ölçeklenir
2  ekran = pygame.display.set_mode(
3      (320, 240), pygame.SCALED
4  )
```

10.3.5 Dikey Senkronizasyon (VSync)

`set_mode()` fonksiyonunun son parametrelerinden biri olan `vsync`, dikey senkronizasyonu kontrol eder:

Python – VSync Acma

```
1 ekran = pygame.display.set_mode((800, 600), vsync=1)
```

■ Tanım: VSync (Dikey Senkronizasyon)

VSync, oyunun kare hızını monitörün yenileme hızıyla eşitler. Bu sayede ekran yırtılması (*screen tearing*) – ekranın üst yarısında bir kare, alt yarısında farklı bir kare görünmesi – engellenir.

VSync'i açmak görüntü kalitesini artırır ama kare hızını monitörün yenileme hızıyla (genellikle 60 Hz) sınırlar. Basit PyGame oyunlarında bu bir sorun değildir ama ileride performans optimizasyonu yaparken bu bilgi işine yarayacak.

10.3.6 Tam Bir Örnek

Bu konuda öğrendiğin ekran modlarını bir arada gösteren kapsamlı bir örnek yazalım. Programın kilit kısmı, `F11` ile tam ekran geçişini yöneten olay işleme bloğudur:

Python – F11 ile Mod Geçiş

```
1 elif olay.key == pygame.K_F11:
2     tam_ekran = not tam_ekran
3     if tam_ekran:
4         ekran = pygame.display.set_mode(
5             (bilgi.current_w, bilgi.current_h),
6             pygame.FULLSCREEN
7         )
8     else:
9         ekran = pygame.display.set_mode(
10            (GENISLIK, YUKSEKLIK)
11        )
```

Burada `bilgi = pygame.display.Info()` ile alınan monitör çözünürlüğü tam ekran modunda, sabit `GENISLIK` ve `YUKSEKLIK` ise pencere modda kullanılıyor.

● Bilgi: Tam Kod

`F11` geçişi, `Esc` çıkışı ve ekran bilgisi gösteren çalıştırılabilir tam programı `kod\ornekleribolum03\unite2\03\_ekran\_modlari.py` dosyasında bulabilirsin.

Bu örnek, gerçek bir oyunun ekran yönetiminin temelini oluşturuyor. Oyun geliştirme sırasında bu yapıyı genişletip kullanacaksın.

10.4 Ünite Özeti

Bu üniteye oyun geliştirmenin en temel yapı taşlarından birini öğrendin: kendi oyun pencerenin oluşturmak! Bir önceki üniteye PyGame'i kurmuş ve başlatmıştın, şimdi ise ekranda kendi pencerenin var. İşte öğrendiklerin:

10.4.1 Bu Üniteye Öğrendiklerin

- **Pencere Oluşturma:** `pygame.display.set_mode()` fonksiyonu ile istediğin boyutta bir oyun penceresi açabilirsin
- **Koordinat Sistemi:** PyGame'de (0,0) noktası sol üst köşededir ve y eksenini aşağı doğru artırır
- **Surface Nesnesi:** `set_mode()` bir Surface nesnesi döndürür; bu nesne üzerinden ekrana çizim yaparsın
- **Pencere Başlığı:** `pygame.display.set_caption()` ile pencerenin başlık çubuğunu özelleştirebilirsin
- **Pencere İkonu:** `pygame.display.set_icon()` ile oyununa özel bir ikon ekleyebilirsin
- **Ekrana Modları:** Pencere, tam ekran ve çerçevesiz modlar arasında seçim yapabilir ve geçiş sağlayabilirsin

10.4.2 Anahtar Kavramlar

Surface PyGame'de üzerine çizim yapılan yüzey nesnesi

set_mode() Oyun penceresini oluşturan fonksiyon

Çözünürlük Pencerenin piksel cinsinden genişlik ve yüksekliği

Koordinat Sistemi Sol üst köşenin (0,0) olduğu, y'nin aşağı doğru arttığı sistem

Caption Pencerenin başlık çubuğundaki metin

Fullscreen Oyunun monitörün tamamını kapladığı mod

RESIZABLE Pencere boyutunun kullanıcı tarafından değiştirilebilmesini sağlayan bayrak

VSync Kare hızını monitörün yenileme hızıyla eşitleyen senkronizasyon

Double Buffering Çizimi arka planda yapıp tek seferde ekrana aktarma tekniği

10.4.3 Önemli Komutlar

Tablo 10.5: Bu ünitedeki önemli PyGame komutları

Komut	Açıklama
<code>display.set_mode((g, y))</code>	Pencere oluşturur
<code>display.set_caption("...")</code>	Başlık metnini ayarlar
<code>display.get_caption()</code>	Mevcut başlığı döndürür
<code>display.set_icon(surface)</code>	Pencere ikonunu ayarlar
<code>display.Info()</code>	Monitör bilgilerini döndürür
<code>display.flip()</code>	Ekranı günceller
<code>display.toggle_fullscreen()</code>	Tam ekran geçişi yapar
<code>surface.fill((r, g, b))</code>	Yüzeyi renkle doldurur

10.4.4 Sıradaki Adım

Bir sonraki ünite **ana oyun döngüsünü (Game Loop)** öğreneceksin. Oyun döngüsü, bir oyunun kalp atışıdır: sürekli çalışır, olayları dinler, oyun durumunu günceller ve ekranı yeniden çizer. Penceremiz hazır, şimdi onu canlandırma zamanı!

💡 İpucu: Pratik Yap!

Bu üniteyi bitirmeden önce şu denemeleri yap:

1. Pencere boyutunu farklı değerlerle dene: 640×480, 1024×768, 1280×720
2. Başlık metnini kendi oyun fikrinle değiştir
3. `RESIZABLE` bayrağını ekle ve pencereyi fareyle boyutlandır
4. `NOFRAME` bayrağını dene ve çerçevesiz pencereyi gör
5. Farklı arka plan renkleriyle `fill()` fonksiyonunu dene

Ana Oyun Döngüsü

“Her oyun, görünmez bir kalp atışının ritmiyle yaşar. Bu kalp durunca, oyun da durur.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Oyun döngüsünün (game loop) ne olduğunu ve neden her oyunun temelini oluşturduğunu
- `while` döngüsü ile sürekli çalışan bir oyun yapısı kurmayı
- Oyun döngüsünün üç temel aşamasını: olayları işle, güncelle, çiz
- PyGame olay sisteminin temellerini ve `pygame.QUIT` olayını yakalamayı
- FPS (kare hızı) kontrolünü `pygame.time.Clock()` ile sağlamayı
- Delta time kavramına giriş yaparak zamana bağlı güncellemeyi anlamayı

11.1 Oyun Döngüsü (Game Loop)

Bir an için favori oyununu düşün. Karakter ekranda koşuyor, düşmanlar saldırıyor, arka plan kayıyor, skor artıyor... Tüm bunlar aynı anda ve akıcı bir şekilde gerçekleşiyor. Peki bu nasıl mümkün oluyor? Ekrandaki her şey sürekli güncelleniyor gibi görünse de aslında bilgisayar her seferinde sadece tek bir “an görüntüsü” çiziyor. Bu an görüntülerini saniyede onlarca kez tekrarlayınca gözümüz bunu akıcı bir hareket olarak algılıyor — tıpkı bir sinema filminin hızla akan karelerinden oluşması gibi.

İşte tüm bu sihri mümkün kılan yapı, *oyun döngüsü*dür.

■ Tanım: Oyun Döngüsü – Game Loop

Oyun döngüsü, bir oyunun başlangıcından kapanışına kadar sürekli tekrarlanan ana döngüdür. Her tekrarda üç temel işlem sırasıyla gerçekleştirilir: kullanıcı girdileri ve olaylar işlenir, oyun durumu güncellenir ve ekrana yeni kare çizilir. Bu döngü saniyede onlarca hatta yüzlerce kez tekrarlanır.

Oyun döngüsünü bir kalp atışına benzetebiliriz. Kalbin her atışında vücuda kan pompalanır, organlar çalışır ve vücut işlevlerini sürdürür. Kalp durursa vücut da durur. Aynı şekilde oyun döngüsü de oyunun kalbidir: döngü her döndüğünde girdiler alınır, oyun dünyası güncellenir ve yeni bir kare ekrana çizilir. Döngü durursa oyun da donar.

11.1.1 Döngünün Üç Temel Aşaması

Her oyun döngüsü tekrarı, hangi oyun motoru veya kütüphane kullanılırsa kullanılsın, temelde aynı üç aşamadan geçer. Bu üç aşamayı sırasıyla inceleyelim.

Aşama 1 – Olayları İşle (Process Input): Oyun döngüsünün ilk adımında kullanıcıdan gelen girdiler ve sistem olayları işlenir. Kullanıcı bir tuşa bastı mı? Fare hareket etti mi? Pencere kapatılmak isteniyor mu? Bu aşamada tüm bu sorulara cevap aranır. PyGame’de bu işlem `pygame.event.get()` fonksiyonu ile gerçekleştirilir. Olay işleme konusunu bu ünitenin ikinci konusunda detaylı inceleyeceğiz.

Aşama 2 – Güncelle (Update): Olaylar işlendikten sonra oyun dünyasının mevcut durumu güncellenir. Karakterin konumu değişir, düşmanlar hareket eder, mermiler ilerler, fizik hesaplamaları yapılır, çarpışmalar kontrol edilir. Bu aşamada hiçbir şey ekrana çizilmez; sadece veriler hesaplanır. Örneğin oyuncunun sağ ok tuşuna bastığını birinci aşamada öğrendin, şimdi bu bilgiyi kullanarak karakterin x koordinatını artırırsın.

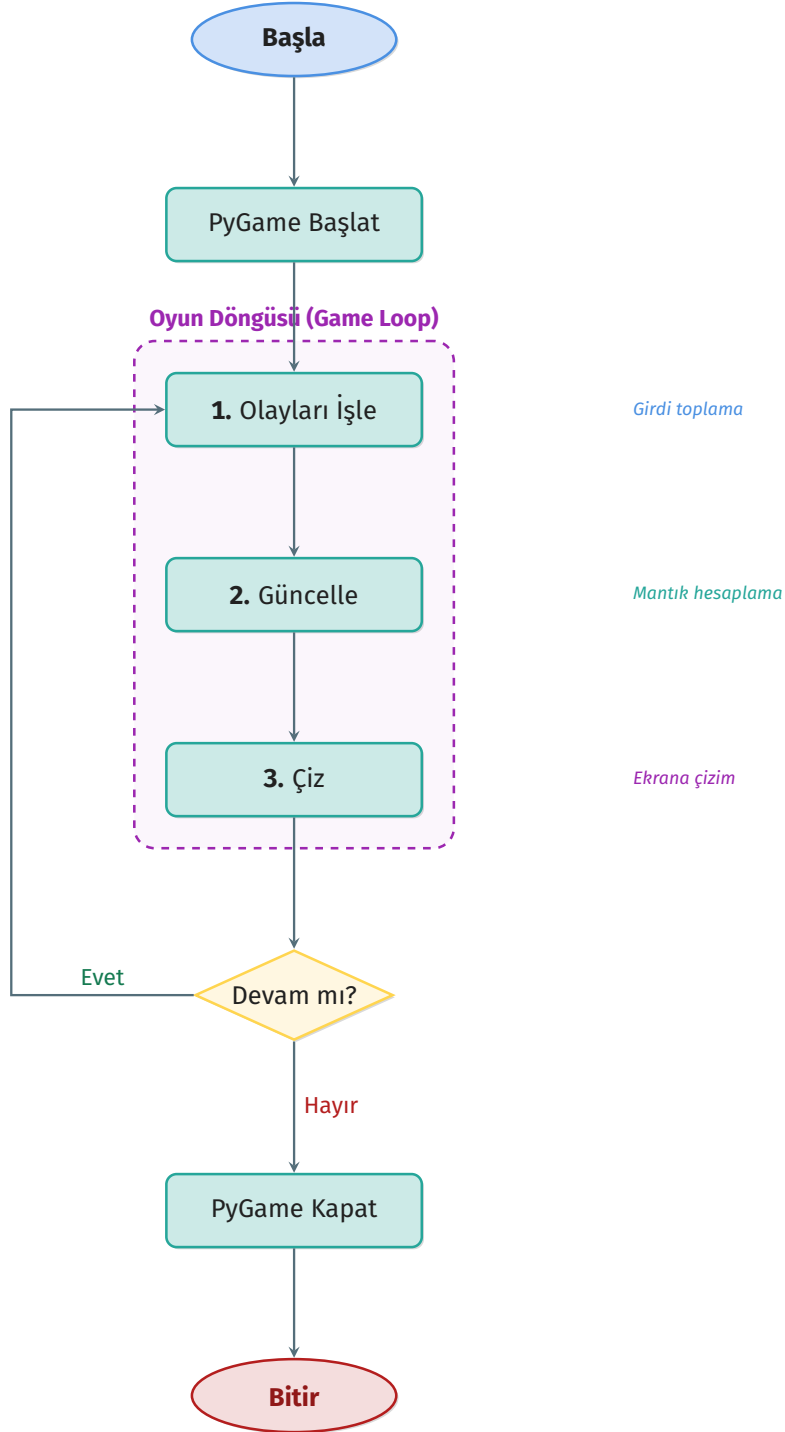
Aşama 3 – Çiz (Render): Son aşamada güncellenen oyun durumu ekrana yansıtılır. Önce ekran temizlenir (genellikle tek bir renkle doldurularak), sonra tüm oyun nesneleri yeni konumlarına çizilir ve son olarak `pygame.display.flip()` veya `pygame.display.update()` çağrılarak ekrandaki görüntü güncellenir. Bu üç adımın sırası çok önemlidir: önce temizle, sonra çiz, en son ekranı güncelle.

Bu üç aşamalı yapıyı Şekil 11.1 ile görselleştirdiğimizde döngünün nasıl çalıştığını daha net anlayacağız.

Şekil 11.1’de görüldüğü gibi döngü “Başla” ile başlıyor, üç aşamayı sırasıyla gerçekleştiriyor ve “Devam mı?” sorusuna verilen yanıtla göre ya döngüyü tekrarlıyor ya da oyunu sonlandırıyor. Dikkat edersen döngünün her tekrarına *kare* (frame) denir. Bir oyunun saniyede 60 kare çalıştığını söylediğimizde, bu döngünün saniyede 60 kez döndüğünü kastediyoruz.

11.1.2 PyGame’de Oyun Döngüsü Yapısı

Şimdi bu teorik bilgiyi PyGame koduna dönüştürelim. Önceki üniteye `pygame.init()` ile PyGame’i başlatmayı ve `pygame.display.set_mode()` ile pencere oluşturmayı öğrendin. Artık bu pencereyi “canlı” tutacak döngüyü ekleme zamanı geldi.



Şekil 11.1: Oyun döngüsünün üç temel aşaması: olayları işle, güncelle ve çiz. Bu döngü, oyun çalıştığı sürece tekrarlanır.

Python – Temel Oyun Döngüsü Yapısı

```

1  import pygame
2
3  # PyGame'i baslat
4  pygame.init()
5
6  # Pencere olustur
7  GENISLIK = 800
8  YUKSEKLIK = 600
9  ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Oyun Dongusu Ornegi")
11
12 # Ana oyun dongusu
13 calistir = True
14 while calistir:
15
16     # ASAMA 1: Olaylari isle
17     for olay in pygame.event.get():
18         if olay.type == pygame.QUIT:
19             calistir = False
20
21     # ASAMA 2: Guncelle
22     # (Simdilik bos - ileride buraya oyun mantigi gelecek)
23
24     # ASAMA 3: Ciz
25     ekran.fill((0, 0, 0))          # Ekrani siyahla temizle
26     pygame.display.flip()          # Ekrani guncelle
27
28 # PyGame'i kapat
29 pygame.quit()

```

Bu kodu satır satır inceleyelim. İlk satırlarda PyGame'i başlatıyor ve bir pencere oluşturuyoruz — bu kısmı önceki ünitelerden zaten tanıyorsun. Asıl yeni olan kısım `while calistir:` satırıyla başlayan döngüdür.

`calistir` adında bir bayrak değişkeni (*flag*) `True` değeriyle tanımlanıyor. Bu değişken `True` olduğu sürece `while` döngüsü tekrarlanmaya devam eder. Kullanıcı pencereyi kapattığında bu değişken `False` yapılır ve döngü sona erer.

Döngünün içinde üç aşamayı açıkça görebilirsin. Birinci aşamada `pygame.event.get()` ile olaylar alınıyor ve `pygame.QUIT` olayı kontrol ediliyor. İkinci aşama şimdilik boş çünkü henüz güncellenecek bir oyun nesnesi yok. Üçüncü aşamada `ekran.fill((0, 0, 0))` ile ekran siyah renkle temizleniyor ve `pygame.display.flip()` ile ekrandaki değişiklikler görünür hale getiriliyor.

⚠ Dikkat: Döngüsüz Pencere Donma Sorunu

Oyun döngüsü olmadan sadece bir pencere oluşturursan, pencere birkaç saniye içinde “yanıt vermiyor” durumuna düşer. Bunun nedeni işletim sisteminin pencereden düzenli olarak olay yanıtı beklemesidir. Oyun döngüsü bu olayları sürekli işleyerek pencerenin canlı kalmasını sağlar.

11.1.3 while True mu, while calistir mı?

İnternette bazı PyGame örneklerinde döngünün `while True:` şeklinde yazıldığını ve döngüden çıkmak için `sys.exit()` veya `break` kullanıldığını görebilirsin. Her iki yaklaşım da çalışır ancak bu kitapta `while calistir:` yaklaşımını tercih ediyoruz.

Bunun birkaç önemli nedeni var. Birincisi, bayrak değişkeni kullanmak programın akışını daha okunabilir kılar: “oyun çalışıyor mu?” sorusu tek bir değişkende saklanır. İkincisi, döngü doğal yoldan sona erdiğinde `pygame.quit()` satırına ulaşılır ve PyGame kaynakları düzgün biçimde serbest bırakılır. Üçüncüsü, ileride oyuna duraklatma (pause) veya birden fazla çıkış koşulu eklediğinde bayrak yaklaşımı çok daha esnek bir yapı sunar.

İpucu: Temiz Çıkış

Oyun döngüsünden çıktıktan sonra mutlaka `pygame.quit()` fonksiyonunu çağır. Bu fonksiyon PyGame’in kullandığı ses, grafik ve diğer sistem kaynaklarını serbest bırakır. Aksi halde bazı işletim sistemlerinde ses cihazı kilitli kalabilir veya pencere düzgün kapanmayabilir.

11.1.4 flip() ve update() Farkı

Ekranı güncellemek için iki fonksiyon kullanabilirsin: `pygame.display.flip()` ve `pygame.display.update()`. Peki hangisini ne zaman kullanmalısın?

`pygame.display.flip()` ekranın tamamını günceller. Tüm pikselleri yeni çizilen içerikle değiştirir. Bu en basit ve en yaygın kullanılan yöntemdir. `pygame.display.update()` ise ya tüm ekranı günceller (parametresiz çağrıldığında `flip()` ile aynı işi yapar) ya da sadece belirttiğin dikdörtgen alanları günceller. Sadece belirli alanları güncellemek performans açısından avantajlı olabilir ancak şu aşamada bu ayrıntıyla ilgilenmene gerek yok.

Bu kitap boyunca basitlik ve okunabilirlik adına `pygame.display.flip()` kullanacağız. İleride oyunun büyüüp performans optimizasyonuna ihtiyaç duyduğunda `pygame.display.update()` ile bölgesel güncellemeyi öğrenebilirsin.

11.1.5 Oyun Döngüsünü Anlamak İçin Bir Deney

Oyun döngüsünün gerçekten sürekli tekrarladığını gözlemlemek için basit bir deney yapalım. Aşağıdaki kodda döngünün her tekrarında bir sayacı artırıyoruz ve belirli aralıklarla konsola yazdırıyoruz.

Python – Döngü Sayacı Deneyi

```
1 import pygame
2
3 pygame.init()
4 ekran = pygame.display.set_mode((400, 300))
5 pygame.display.set_caption("Döngü Sayacı")
6
```

```

7  calistir = True
8  sayac = 0
9
10 while calistir:
11     for olay in pygame.event.get():
12         if olay.type == pygame.QUIT:
13             calistir = False
14
15     sayac += 1
16     if sayac % 1000 == 0:
17         print(f"Dongu {sayac} kez dondu!")
18
19     ekran.fill((30, 30, 50))
20     pygame.display.flip()
21
22 pygame.quit()
23 print(f"Toplam dongu: {sayac}")

```

Bu programı çalıştırdığında konsolda hızla artan sayıları göreceksin. Birkaç saniye içinde sayacın onbinlerce hatta yüzbinlerce değere ulaştığını fark edeceksin. Bu da döngünün ne kadar hızlı çalıştığını gözler önüne seriyor. Peki bu kadar hızlı çalışması iyi bir şey mi? Aslında hayır — işlemciyi gereksiz yere yorar ve farklı bilgisayarlarda farklı hızlarda çalışmasına neden olur. Bu sorunu çözmek için FPS kontrolüne ihtiyacımız var; bunu üçüncü konuda detaylı ele alacağız.

● Bilgi: Oyun Döngüsü Her Yerde

Oyun döngüsü sadece PyGame'e özgü bir kavram değildir. Unity, Unreal Engine, Godot gibi profesyonel oyun motorlarının tümü aynı temel yapıyı kullanır. Unity'de `Update()` fonksiyonu, Godot'da `_process()` fonksiyonu aslında oyun döngüsünün “güncelle” aşamasıdır. Burada öğrendiğin mantık, ileride hangi motora geçersen geç sana rehberlik edecek.

11.2 Olay İşleme Temelleri (Event Handling)

Önceki konuda oyun döngüsünün ilk aşamasının “olayları işle” olduğunu öğrendin. Peki “olay” tam olarak ne demek? Bilgisayarında bir tuşa bastığında, fareyi hareket ettirdiğinde veya pencereyi kapatmaya çalıştığında işletim sistemi bunların her birini bir *olay* (event) olarak kaydeder. PyGame bu olayları bir kuyruқта (queue) biriktirir ve sen bu kuyruğu okuyarak olaylara tepki verirsin.

Bunu bir posta kutusuna benzetebilirsin. Gün içinde postacı farklı zamanlarda mektuplar bırakır; sen de belirli aralıklarla posta kutunu kontrol edersin. Oyun döngüsünde de aynısı olur: kullanıcı tuşlara basar, fare ile tıklar, pencere boyutunu değiştirir ve bunların hepsi olay kuyruğunda birikir. Döngünün her tekrarında bu kuyruğu boşaltarak her olaya uygun tepkiyi verirsin.

■ Tanım: Olay – Event

Bir olay, kullanıcının veya sistemin gerçekleştirdiği bir eylemin programlanabilir temsilidir. PyGame’de her olay bir `pygame.event.Event` nesnesidir ve mutlaka bir `type` özelliği taşır. Bu özellik olayın türünü belirler: pencere kapatma (`pygame.QUIT`), tuşa basma (`pygame.KEYDOWN`), fare tıklaması (`pygame.MOUSEBUTTONDOWN`) gibi.

11.2.1 Olay Kuyruğu ve `event.get()`

PyGame’in olay sistemi bir kuyruk yapısına dayanır. Olaylar oluşukça kuyruğun sonuna eklenir ve sen `pygame.event.get()` fonksiyonunu çağırdığında kuyruktaki tüm olaylar bir liste olarak sana döndürülür. Döndürülen bu olaylar kuyruktan silinir; yani her çağrıda kuyruğu “boşaltmış” olursun.

Bu yapı çok önemli bir sonuç doğurur: eğer döngüde `pygame.event.get()` çağırmazsan olay kuyruğu dolmaya devam eder. Bir süre sonra işletim sistemi programın olaylara yanıt vermediğini düşünerek “yanıt vermiyor” uyarısı gösterir. Bu yüzden oyun döngüsünde her turda mutlaka `pygame.event.get()` çağrılmalıdır — kullanmayacağın olaylar olsa bile.

Aşağıdaki basit örnekte olay kuyruğunu okuyup her olayın türünü konsola yazdırıyoruz.

Python – Olay Türlerini İnceleme

```

1  import pygame
2
3  pygame.init()
4  ekran = pygame.display.set_mode((400, 300))
5  pygame.display.set_caption("Olay Gözlemcisi")
6
7  calistir = True
8  while calistir:
9      for olay in pygame.event.get():
10         print(f"Olay turu: {olay.type}")
11         if olay.type == pygame.QUIT:
12             calistir = False
13
14         ekran.fill((20, 20, 40))
15         pygame.display.flip()
16
17  pygame.quit()

```

Bu programı çalıştırıp fareyi pencerenin üzerinde gezdirdiğinde, bir tuşa bastığında veya pencereyi hareket ettirdiğinde konsolda sürekli olay numaralarının aktığını göreceksin. Her numara farklı bir olay türünü temsil eder. PyGame bu numaraları daha okunabilir sabitlerle eşler: örneğin `pygame.QUIT` aslında 256 sayısına karşılık gelir ama `pygame.QUIT` yazmak hem daha anlaşılır hem de hata yapma olasılığını azaltır.

11.2.2 QUIT Olayı ve Pencere Kapatma

Bir oyunda en temel olay, kullanıcının pencereyi kapatmak istemesidir. Kullanıcı pencerenin sağ üst köşesindeki “X” düğmesine tıkladığında veya **Alt**+**F4** kısayolunu kullandığında PyGame bir `pygame.QUIT` olayı oluşturur. Bu olayı ya-kalamazsan pencere kapanmaz ve kullanıcı programı zorla sonlandırmak zorunda kalır.

Önceki konudaki oyun döngüsünde bu olayı zaten işliyorduk. Yapıyı tekrar hatırlayalım:

Python – QUIT Olayı İşleme

```
1 for olay in pygame.event.get():
2     if olay.type == pygame.QUIT:
3         calistir = False
```

Bu üç satırlık yapı her PyGame programında bulunması gereken minimum olay işleme kodudur. `for` döngüsü kuyruktaki tüm olayları tek tek dolaşır. Her olay için `olay.type` değeri kontrol edilir. Eğer bu değer `pygame.QUIT` ile eşleşirse `calistir` bayrağı `False` yapılır ve ana `while` döngüsünün bir sonraki koşul kontrolünde döngü sona erer.

⚠ Dikkat: Olayları Her Turda İşle

Olay işleme kodunu döngünün dışına veya koşullu bir bloğun içine yerleştirme. Her döngü turunda `pygame.event.get()` mutlaka çağrılmalıdır. Aksi halde olay kuyruğu taşar ve işletim sistemi programının yanıt vermediğini düşünerek uyarı gösterir.

11.2.3 Temel Olay Türleri

PyGame çok sayıda olay türü tanımlar. Bu ünite de sadece en temel olanını (`pygame.QUIT`) kullandık. Ancak ileride karşılaşacağın diğer olay türlerini şimdiden tanıman faydalı olacaktır.

Tablo 11.1: PyGame’de sık kullanılan olay türleri

Olay Türü	Ne Zaman Oluşur	Bölüm
<code>pygame.QUIT</code>	Pencere kapatılmak istendiğinde	Bu ünite
<code>pygame.KEYDOWN</code>	Bir tuşa basıldığında	Bölüm 4
<code>pygame.KEYUP</code>	Basılı tuş bırakıldığında	Bölüm 4
<code>pygame.MOUSEBUTTONDOWN</code>	Fare düğmesine basıldığında	Bölüm 4
<code>pygame.MOUSEBUTTONUP</code>	Fare düğmesi bırakıldığında	Bölüm 4
<code>pygame.MOUSEMOTION</code>	Fare hareket ettiğinde	Bölüm 4
<code>pygame.USEREVENT</code>	Özel kullanıcı olayları	Bölüm 8

Tablo 11.1’de gördüğün gibi klavye ve fare olaylarını Bölüm 4’te detaylı öğreneceksin. Şimdilik `pygame.QUIT` olayını anlamak ve doğru bir şekilde işlemek

yeterli. Önemli olan olay sisteminin genel mantığını kavramandır: olaylar kuyruğa eklenir, sen kuyruğu okursun, her olaya göre uygun tepkiyi verirsin.

11.2.4 Olayları Filtreleme

Bazı durumlarda kuyruktaki tüm olaylarla değil, sadece belirli türlerle ilgilenmek isteyebilirsin. PyGame bunun için birkaç yöntem sunar.

Birincisi, `pygame.event.get()` fonksiyonuna parametre olarak olay türü verebilirsin. Örneğin `pygame.event.get(pygame.QUIT)` çağrısı sadece `pygame.QUIT` olaylarını döndürür. Ancak dikkat: bu durumda diğer olaylar kuyrukta birikmeye devam eder ve kuyruğu taşıyabilir.

İkincisi, `pygame.event.set_blocked()` ile belirli olay türlerinin kuyruğa eklenmesini engelleyebilirsin. Üçüncüsü, `pygame.event.set_allowed()` ile sadece izin verdiğin olay türlerinin kuyruğa eklenmesini sağlayabilirsin. Bu optimizasyon teknikleri ileride performansa duyarlı oyunlarda işine yarayacak; şimdilik standart `for` döngüsü ile tüm olayları dolaşmak en güvenli yoldur.

● Bilgi: Olay Kuyruğunun Boyutu

PyGame'in olay kuyruğu varsayılan olarak 128 olay kapasitesine sahiptir. Bu sınıra ulaşırsa yeni olaylar sessizce göz ardı edilir. Normal şartlarda her döngü turunda kuyruğu okuduğun için bu sınıra ulaşman pek olası değildir. Ancak döngüde ağır bir işlem yaparak olayları okumayı geciktirirsen bu sorunla karşılaşabilirsin.

11.2.5 Tüm Parçaları Birleştirme

Şimdi öğrendiğin olay işleme bilgilerini önceki konudaki oyun döngüsüyle birleştiririm. Aşağıdaki parçacık, farklı olay türlerinin nasıl yakalanıp konsola yazdırıldığını gösterir:

Python – Birden Fazla Olay Türünü İşleme

```

1 while calistir:
2     for olay in pygame.event.get():
3         if olay.type == pygame.QUIT:
4             print("[QUIT] Pencere kapatma istegi alindi.")
5             calistir = False
6         elif olay.type == pygame.KEYDOWN:
7             print(f"[KEYDOWN] Tusa basildi:
8                 ↳ {pygame.key.name(olay.key)}")
9         elif olay.type == pygame.MOUSEBUTTONDOWN:
10            print(f"[MOUSEBUTTONDOWN] Fare tiklandi:
11                ↳ konum={olay.pos}")
12
13    ekran.fill(ARKA_PLAN)
14    pygame.display.flip()
```


● Bilgi: Tam Kod

Bu programın çalıştırılabilir tam halini `kod\_ornekleri/bolum03/unite3/01\_olay\_dongusu.py` dosyasında bulabilirsin.

Bu programı çalıştırdığında pencerenin düzgün açıldığını ve “X” düğmesine bastığında temiz bir şekilde kapandığını göreceksin. Konsoldaki olay mesajlarını inceleyerek farenin, klavyenin ve pencere olaylarının nasıl raporlandığını gözlemleyebilirsin.

Bir sonraki konuda oyun döngüsünün hızını kontrol etmeyi, yani FPS kavramını öğreneceğiz. Bu sayede döngümüz hem düzenli aralıklarla hem de her bilgisayarda aynı hızda çalışacak.

11.3 FPS Kontrolü ve Zamanlama

Birinci konuda oyun döngüsünün saniyede binlerce kez dönebileceğini gözlemlediğimiz deneyi hatırlıyor musun? Döngüyü serbest bıraktığında bilgisayarın işlemcisi olabildiğince hızlı çalışır: güçlü bir bilgisayarda saniyede 10 000 tur, eski bir bilgisayarda belki sadece 500 tur. Bu durum iki önemli sorun yaratır.

Birincisi, oyun her bilgisayarda farklı hızda çalışır. Hızlı bir bilgisayarda karakter fırlayarak ilerlerken yavaş bir bilgisayarda ağır çekim gibi hareket eder. İkincisi, işlemci gereksiz yere sonuna kadar çalışarak enerji harcar ve bilgisayarın ısınmasına neden olur. Zaten ekranın tazeleme hızı genellikle saniyede 60 karedir; bundan fazla kare üretmenin görsel bir faydası yoktur.

İşte bu sorunları çözmek için *FPS kontrolü* kullanırız.

■ Tanım: FPS – Frames Per Second

FPS (saniye başına kare sayısı), oyun döngüsünün saniyede kaç kez tekrarlandığını gösteren bir ölçüdür. 60 FPS, döngünün saniyede 60 kez döndüğünü ve ekrana saniyede 60 kare çizildiğini ifade eder. Bu değer aynı zamanda her karenin yaklaşık 16.67 milisaniye sürdüğü anlamına gelir.

11.3.1 pygame.time.Clock Sınıfı

PyGame, FPS kontrolü için `pygame.time.Clock` adında kullanışlı bir sınıf sunar. Bu sınıf bir kronometre gibi çalışır: her döngü turunda ne kadar zaman geçtiğini ölçer ve hedef FPS değerine ulaşmak için gerekiyorsa bekler.

Kullanımı oldukça basittir. Önce döngüye girmeden bir `Clock` nesnesi oluşturursun, sonra döngünün her turunda `tick()` metodunu çağırırsın. İşte hepsi bu kadar.

Python – Clock Kullanımı

```
1 import pygame
2
```

```

3 pygame.init()
4 ekran = pygame.display.set_mode((800, 600))
5 pygame.display.set_caption("FPS Kontrolü")
6
7 # Saat nesnesi olustur
8 saat = pygame.time.Clock()
9
10 calistir = True
11 while calistir:
12     for olay in pygame.event.get():
13         if olay.type == pygame.QUIT:
14             calistir = False
15
16     # Guncelle (simdilik bos)
17
18     # Ciz
19     ekran.fill((10, 10, 30))
20     pygame.display.flip()
21
22     # FPS'i 60 ile sinirla
23     saat.tick(60)
24
25 pygame.quit()

```

Bu kodda `saat = pygame.time.Clock()` satırı döngü başlamadan önce bir `Clock` nesnesi oluşturuyor. Döngünün en sonunda `saat.tick(60)` çağrısı yapılıyor. Bu çağrı iki iş yapar. Birincisi, eğer döngü 16.67 milisaniyeden ($1000/60$) kısa sürdüyse kalan süre kadar bekleyerek hızı 60 FPS ile sınırlar. İkincisi, son çağrıdan bu yana geçen süreyi milisaniye cinsinden döndürür.

● Bilgi: `tick()` Konumu

`saat.tick(60)` çağrısını döngünün en sonuna koy. Böylece tüm işlemler (olay işleme, güncelleme, çizim) tamamlandıktan sonra bekleme gerçekleşir ve zamanlama en doğru şekilde yapılır.

11.3.2 FPS Değerini Gözlemleme

Gerçek FPS değerini görmek istersen `Clock` nesnesinin `get_fps()` metodunu kullanabilirsin. Bu metod, son birkaç karenin ortalamasına dayalı olarak anlık FPS değerini döndürür.

Python – FPS Göstergesi

```

1 import pygame
2
3 pygame.init()
4 ekran = pygame.display.set_mode((600, 400))
5 saat = pygame.time.Clock()

```

```

6
7 calistir = True
8 while calistir:
9     for olay in pygame.event.get():
10         if olay.type == pygame.QUIT:
11             calistir = False
12
13     ekran.fill((10, 10, 30))
14     pygame.display.flip()
15
16     saat.tick(60)
17
18     # FPS degerini pencere basliginda goster
19     fps = saat.get_fps()
20     pygame.display.set_caption(f"FPS: {fps:.1f}")
21
22 pygame.quit()

```

Bu programı çalıştırdığında pencere başlığında FPS değerinin yaklaşık 60 civarında dalgalandığını göreceksin. Tam olarak 60.0 olmayabilir çünkü işletim sistemi zamanlama konusunda mükemmel hassasiyete sahip değildir; 59.8–60.2 arasındaki dalgalanmalar normaldir.

Deney olarak `saat.tick(60)` satırındaki 60 değerini 30, 120 veya 10 ile değiştirmeyi dene. Düşük değerlerde pencerenin daha “yavaş” tepki verdiğini, yüksek değerlerde ise farkın zor algılandığını göreceksin.

11.3.3 Delta Time: Zamana Bağlı Güncelleme

FPS kontrolü ile döngünün hızını sınırlamayı öğrendin. Ancak burada ince bir sorun daha var: ya bilgisayar hedef FPS’i tutturamıyorsa? Örneğin oyunun 60 FPS hedefle çalışıyor ama ekranda çok fazla nesne olduğu bir anda 40 FPS’e düşüyor. Bu durumda karakterin hareket hızı da düşer çünkü güncelleme daha seyrek yapılır.

İşte *delta time* bu sorunu çözer. Delta time, bir önceki kare ile mevcut kare arasında geçen süredir. Bu süreyi hareket hesaplamalarında çarpan olarak kullanırsan, FPS ne olursa olsun karakter aynı hızda hareket eder.

■ Tanım: Delta Time

Delta time (dt veya Δt), ardışık iki kare arasında geçen süredir. Genellikle saniye cinsinden ifade edilir. 60 FPS’de delta time yaklaşık 0.0167 saniye, 30 FPS’de ise yaklaşık 0.0333 saniyedir. Hareket hesaplamalarında hız değeri delta time ile çarpılarak frame-bağımsız hareket elde edilir.

`saat.tick()` metodunun döndürdüğü değer, son çağrıdan bu yana geçen süreyi **milisaniye** cinsinden verir. Bunu saniyeye çevirmek için 1000’e böleriz. Aşağıdaki örnekte bu kavramı basit bir hareket senaryosuyla gösterelim.

Python – Delta Time ile Hareket

```

1  import pygame
2
3  pygame.init()
4  ekran = pygame.display.set_mode((800, 400))
5  pygame.display.set_caption("Delta Time Ornegi")
6  saat = pygame.time.Clock()
7
8  # Dikdortgenin baslangic konumu ve hizi
9  x = 50.0
10 hiz = 200 # piksel/saniye
11
12 calistir = True
13 while calistir:
14     # Delta time'i hesapla (milisaniye -> saniye)
15     dt = saat.tick(60) / 1000.0
16
17     for olay in pygame.event.get():
18         if olay.type == pygame.QUIT:
19             calistir = False
20
21     # Guncelle: hiz * dt = frame-bagimsiz hareket
22     x += hiz * dt
23     if x > 800:
24         x = -50
25
26     # Ciz
27     ekran.fill((10, 10, 30))
28     pygame.draw.rect(ekran, (0, 200, 100), (int(x), 175,
29         ↪ 50, 50))
30     pygame.display.flip()
31
32 pygame.quit()

```

Bu kodun kilit satırı `x += hiz * dt` satırıdır. Burada `hiz` değişkeni saniye başına piksel cinsinden tanımlanmış (200 piksel/saniye). Delta time saniye cinsinden olduğuna göre çarpım bize bu karede kaç piksel ilerlememiz gerektiğini verir.

60 FPS'de: $200 \times 0.0167 \approx 3.33$ piksel/kare. 30 FPS'de ise: $200 \times 0.0333 \approx 6.67$ piksel/kare. Her iki durumda da saniyede 200 piksel ilerlenmiş olur. FPS düşse bile karakter aynı hızda hareket eder çünkü daha az kare üretilir ama her karede daha fazla piksel ilerlenir.

⚠ Dikkat: Delta Time ve `tick()` Sırası

Delta time hesabını döngünün başına koy ve `saat.tick(60)` çağrısının döndürdüğü değeri kullan. Bazı örneklerde `tick()` döngünün sonunda çağrılır ve delta time ayrıca hesaplanır; bu da işe yarar ancak tutarlılık açısından döngünün başında almak daha temiz bir yapı sağlar.

11.3.4 Hangi FPS Değerini Seçmeliyim?

Hedef FPS değeri oyunun türüne ve hedef platforma göre değişir. Genel bir kılavuz olarak şu tabloyu kullanabilirsin.

Tablo 11.2: Yaygın FPS değerleri ve kullanım alanları

FPS	Kare Süresi	Kullanım Alanı
30	~33.3 ms	Düşük performanslı cihazlar, strateji oyunları
60	~16.7 ms	Standart masaüstü oyunları (en yaygın)
120	~8.3 ms	Yüksek tazeleme hızlı monitörler

Bu kitaptaki tüm örneklerde 60 FPS kullanacağız. Bu değer çoğu bilgisayar için yeterli, pürüzsüz bir görüntü sağlar ve işlemciyi gereksiz yere yormaz. İleride oyununun ihtiyacına göre bu değeri ayarlayabilirsin.

İpucu: FPS Sabiti Tanımla

FPS değerini kodun içine doğrudan yazmak yerine bir sabit olarak tanımla. Böylece değiştirmek istediğinde tek bir yeri güncelleme yeterli olur:

FPS = 60

ve döngüde `saat.tick(FPS)` olarak kullan.

11.3.5 Tam Örnek: FPS Kontrollü Oyun Döngüsü

Şimdi bu ünite de öğrendiğin her şeyi bir araya getirelim: oyun döngüsü, olay işleme ve FPS kontrolü. Aşağıdaki parçacıklar ekranda yatay hareket eden bir karenin kilit noktalarını gösterir.

Önce sabitler ve başlangıç değerleri:

Python – Sabitler ve Başlangıç

```
1 FPS = 60
2 GENISLIK = 800
3
4 saat = pygame.time.Clock()
5 kare_x = 50.0
6 hiz = 200 # piksel/saniye
```

Döngünün başında delta time hesaplanır, güncelleme adımında frame-bağımsız hareket uygulanır:

Python – Delta Time ve Hareket

```
1 # Delta time hesapla (milisaniye -> saniye)
2 dt = saat.tick(FPS) / 1000.0
3
4 # Frame-bagimsiz hareket: hiz * dt
5 kare_x += hiz * dt
```

```

6 if kare_x > GENISLIK:
7     kare_x = -KARE_BOYUT

```

Çizim ve FPS gösterimi:

Python – Çizim ve FPS

```

1 ekran.fill(ARKA_PLAN)
2 pygame.draw.rect(ekran, KARE_RENK,
3     (int(kare_x), int(kare_y), KARE_BOYUT, KARE_BOYUT))
4 pygame.display.flip()
5
6 fps_gercek = saat.get_fps()
7 pygame.display.set_caption(f"FPS: {fps_gercek:.1f} | dt:
   ↳ {dt:.4f}s")

```

● Bilgi: Tam Kod

Bu programın çalıştırılabilir tam halini kod_ornekleri/bolum03/unite3/02_fps_kontrol.py dosyasında bulabilirsiniz.

Bu program, bu ünitenin tüm konularını bir arada kullanıyor. Oyun döngüsü `while calistir:` ile çalışıyor, `pygame.QUIT` olayı işlenerek düzgün kapanma sağlanıyor, `saat.tick(FPS)` ile hız sınırlanıyor ve delta time ile frame-bağımsız hareket elde ediliyor. Pencere başlığında anlık FPS değeri görüntüleniyor.

Bir sonraki ünite de temel çizim işlemlerine geçeceğiz ve ekrana dikdörtgen, daire, çizgi gibi şekiller çizmeyi öğreneceğiz. Ancak önce bu ünite de öğrendiklerini özetleyelim.

11.4 Sonuç

11.4.1 Ünite Özeti

Bu ünite de oyun geliştirmenin en temel yapı taşını, *oyun döngüsünü* (game loop) öğrendin. Bu döngü, bir oyunun kalp atışıdır: çalıştığı sürece oyun yaşar, durduğunda oyun da durur.

Oyun döngüsünün her tekrarında üç aşama sırasıyla gerçekleşir. Önce kullanıcı girdileri ve sistem olayları işlenir (**Olayları İşle**). Sonra oyun dünyasının mevcut durumu güncellenir (**Güncelle**). En son güncellenmiş durum ekrana çizilir (**Çiz**). Bu üç aşamalı yapı sadece PyGame'e değil, tüm oyun motorlarına özgü evrensel bir tasarım kalıbıdır.

Olay sistemiyle tanıştın ve PyGame'in olayları bir kuyrukta biriktirdiğini öğrendin. `pygame.event.get()` ile bu kuyruğu okuyarak `pygame.QUIT` olayını yakalayıp programın düzgün kapanmasını sağladın. Klavye ve fare olaylarını Bölüm 4'te detaylı öğreneceksin.

Son olarak FPS kontrolünü ele aldın. `pygame.time.Clock()` nesnesi ile döngü hızını sınırlamayı ve `tick()` metodunun döndürdüğü delta time değeri

ile frame-bağımsız hareket hesaplamayı öğrendin.

11.4.2 Anahtar Kavramlar

Tablo 11.3: Bu ünite de öğrenilen anahtar kavramlar

Kavram	Açıklama
Oyun Döngüsü (Game Loop)	Oyunun sürekli tekrarlanan ana döngüsü; olayları işle, güncelle ve çiz aşamalarından oluşur
Kare (Frame)	Döngünün tek bir tekrarı; ekrana çizilen tek bir görüntü
Olay (Event)	Kullanıcının veya sistemin tetiklediği eylem; kuyrukta birikir
Olay Kuyruğu	Olayların biriktiği FIFO yapısı; <code>event.get()</code> ile okunur
FPS	Saniyede üretilen kare sayısı; 60 FPS standart değerdir
Delta Time (Δt)	İki kare arası geçen süre; frame-bağımsız hareket için kullanılır
Bayrak Değişkeni (Flag)	Döngünün devam edip etmeyeceğini kontrol eden mantıksal değişken

11.4.3 Önemli Komutlar

Tablo 11.4: Bu ünite de kullanılan PyGame komutları

Komut	İşlev
<code>pygame.event.get()</code>	Olay kuyruğundaki tüm olayları liste olarak döndürür ve kuyruğu boşaltır
<code>olay.type</code>	Bir olay nesnesinin türünü döndürür (QUIT, KEYDOWN vb.)
<code>pygame.QUIT</code>	Pencere kapatma olayının sabit değeri
<code>ekran.fill((r, g, b))</code>	Ekranı belirtilen RGB rengiyle doldurur (temizler)
<code>pygame.display.flip()</code>	Ekrandaki tüm değişiklikleri görünür hale getirir
<code>pygame.time.Clock()</code>	FPS kontrolü için saat nesnesi oluşturur
<code>saat.tick(fps)</code>	Döngü hızını belirtilen FPS ile sınırlar; geçen süreyi (ms) döndürür
<code>saat.get_fps()</code>	Anlık FPS değerini döndürür
<code>pygame.quit()</code>	PyGame'in kullandığı tüm kaynakları serbest bırakır

11.4.4 Sıradaki Adım

Bu ünite de oyun döngüsünü, olay işlemeyi ve FPS kontrolünü öğrendin. Artık ekranda sürekli çalışan, olaylara tepki veren ve sabit hızda dönen bir yapıya sahipsın. Ancak şu an ekranımız sadece düz bir renkle dolu — pek heyecan verici değil!

Bir sonraki ünite de (Ünite 4) **Temel Çizim İşlemlerine** geçeceğiz. `pygame.draw.rect()`, `pygame.draw.circle()`, `pygame.draw.line()` gibi fonksiyonlarla ekrana şekiller çizmeyi öğreneceksin. Renk sistemi (RGB), koordinat düzlemi ve `fill()` fonksiyonunun detaylarını keşfedeceksin. Böylece oyun döngüsünün “Çiz” aşamasını gerçek içerikle doldurmaya başlayacaksın.

Ayrıca Bölüm 4’te olay sistemini derinlemesine inceleyerek klavye ve fare girdileriyle nesneleri kontrol etmeyi öğreneceksin. Oyun döngüsünün “Olayları İşle” ve “Güncelle” aşamalarının gerçek anlamda canlanmasını izleyeceksin.

Temel Çizim İşlemleri

“Bir piksel yerleştirmek, bir fırça darbesi vurmak gibidir. Binlerce piksel bir araya geldiğinde bir dünya yaratırsın.”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- RGB renk sisteminin nasıl çalıştığı ve renkleri nasıl tanımlayacağını
- `pygame.draw` modülündeki temel çizim fonksiyonları
- Dikdörtgen, daire, çizgi, elips ve çokgen çizme
- `fill()` metodu ile ekranı temizleme
- Surface kavramı ve çizim sıralaması (painter's algorithm)

12.1 Renkler ve RGB Sistemi

Önceki ünite de oyun döngüsünü öğrendin ve ekranda siyah bir pencere açmayı başardın. Ama bir oyun sadece siyah bir ekrandan ibaret olamaz, değil mi? Düşünsene: gökyüzünün mavisi, çimenliğin yeşili, düşman gemisinin kırmızı lazer ışınları... Bir oyundaki her piksel bir renge sahiptir. Peki bilgisayar bu renkleri nasıl anlıyor? İşte burada **RGB renk sistemi** devreye giriyor.

■ Tanım: RGB Renk Sistemi

RGB, **R**ed (Kırmızı), **G**reen (Yeşil) ve **B**lue (Mavi) kelimelerinin baş harflerinden oluşan bir renk modelidir. Bu üç temel rengin farklı yoğunluklarda karıştırılmasıyla milyonlarca farklı renk elde edilir. Her bir renk kanalı 0 ile 255 arasında bir değer alır.

Bu sistemi bir ressam paleti gibi düşünebilirsin. Ressam paletinde üç tüp boya var: kırmızı, yeşil ve mavi. Her tüpten ne kadar boya sıkarsan, o kadar yoğun bir

renk elde edersin. Hiç boya sıkılmazsan siyah kalır, üç boyayı da sonuna kadar sıkarsan beyaz olur.

12.1.1 Renk Değer Aralığı: 0–255

Her bir renk kanalı neden tam olarak 0 ile 255 arasında bir değer alır? Bunun sebebi bilgisayarların veriyi *bit* ve *bayt* cinsinden saklamasıdır. Bir bayt 8 bitten oluşur ve $2^8 = 256$ farklı değer tutabilir. Bu da 0'dan 255'e kadar toplam 256 seviye demektir.

- **0:** O renk kanalı tamamen kapalı (hiç yok)
- **255:** O renk kanalı en yüksek yoğunlukta (tam güçte)
- **Ara değerler:** Farklı tonlar elde edilir (örneğin $128 = \text{yarı güçte}$)

Toplam kaç farklı renk elde edebilirsin? Her kanal 256 farklı değer alabildiğine göre: $256 \times 256 \times 256 = 16,777,216$ renk. Yani yaklaşık 16.7 milyon farklı renk! İnsan gözünün ayırt edebildiğinden çok daha fazla.

12.1.2 PyGame'de Renk Tanımlama

PyGame'de renkler **üç elemanlı tuple** olarak tanımlanır. Sıralama her zaman (*Kırmızı*, *Yeşil*, *Mavi*) şeklindedir:

Listing 12.1: Temel renk tanımları

```

1 # RGB renk tanımları - (Kırmızı, Yeşil, Mavi)
2 SIYAH = (0, 0, 0) # Tüm kanallar kapalı
3 BEYAZ = (255, 255, 255) # Tüm kanallar tam güçte
4 KIRMIZI = (255, 0, 0) # Sadece kırmızı açık
5 YESIL = (0, 255, 0) # Sadece yeşil açık
6 MAVI = (0, 0, 255) # Sadece mavi açık

```

Renk değişkenlerini büyük harfle tanımladığımıza dikkat et. Bunun sebebi, renklerin programda değişmeyen *sabit* (constant) değerler olmasıdır. Python'da sabitler büyük harfle yazılır; bu bir kural değil, bir gelenektir, ama kodun okunabilirliğini büyük ölçüde artırır.

12.1.3 Yaygın Renkler Tablosu

Oyun geliştirirken sıkça kullanacağın temel renkleri bir tabloda toplayalım:

Tablo 12.1: PyGame'de yaygın olarak kullanılan renkler

Renk Adı	Değişken	R	G	B
Siyah	SIYAH	0	0	0
Beyaz	BEYAZ	255	255	255
Kırmızı	KIRMIZI	255	0	0
Yeşil	YESIL	0	255	0
Mavi	MAVI	0	0	255
Sarı	SARI	255	255	0
Camgöbeği	CAMGOBEGI	0	255	255
Macenta	MACENTA	255	0	255
Turuncu	TURUNCU	255	165	0
Mor	MOR	128	0	128
Gri	GRI	128	128	128
Koyu Gri	KOYU_GRI	64	64	64
Açık Gri	ACIK_GRI	192	192	192
Kahverengi	KAHVERENGI	139	69	19
Gökyüzü Mavisi	GOKYUZU	135	206	235

Bu tabloyu bir referans olarak kullanabilirsin. Ama aslında bu 15 renk sadece başlangıç; istediğin herhangi bir rengi kendi RGB değerlerini yazarak oluşturabilirsin.

İpucu: Profesyonel İpucu

Renklerini bir Python dosyasının en başında sabit olarak tanımla ve tüm kodda bu sabitleri kullan. Böylece bir rengi değiştirmek istediğinde sadece tanımlı güncellermen yeterli olur. Profesyonel oyun projelerinde renkler genellikle ayrı bir `renkler.py` dosyasında tutulur.

12.1.4 Renk Karıştırma Mantığı

RGB sistemi *toplamsallık ilkesi* (additive color model) ile çalışır. Yani renkler birbirine eklendikçe sonuç **aydınlanır**. Bu, boyalarla karıştırmadan farklıdır; boyada kırmızı ile yeşili karıştırırsan kahverengimsi bir şey elde edersin, ama ışıktaki kırmızı ile yeşili karıştırırsan **sarı** elde edersin.

Temel karışımları şöyle özetleyebiliriz:

- Kırmızı + Yeşil = Sarı: (255, 255, 0)
- Kırmızı + Mavi = Macenta: (255, 0, 255)
- Yeşil + Mavi = Camgöbeği: (0, 255, 255)
- Kırmızı + Yeşil + Mavi = Beyaz: (255, 255, 255)
- Hiçbiri = Siyah: (0, 0, 0)

12.1.5 Ara Tonlar Oluşturma

Her kanalda 0 ile 255 arası değerler kullanarak sonsuz çeşitlilikte ton oluşturabilirsin. İşte bazı pratik örnekler:

Listing 12.2: Ara ton renk örnekleri

```

1 # Kırmızının farklı tonları
2 ACIK_KIRMIZI = (255, 102, 102) # Kırmızıya yeşil ve mavi
   katarak açıldı
3 KOYU_KIRMIZI = (139, 0, 0) # Kırmızı kanalı düşürüldü
4
5 # Yeşilin farklı tonları
6 LIME_YESIL = (50, 205, 50) # Parlak yeşil
7 ORMAN_YESILI = (34, 139, 34) # Koyu doğal yeşil
8
9 # Gri tonları - üç kanal eşit olursa gri elde edersin
10 ACIK_GRI = (200, 200, 200)
11 ORTA_GRI = (128, 128, 128)
12 KOYU_GRI = (50, 50, 50)
13
14 # Oyun için özel renkler
15 ALTIN = (255, 215, 0) # Altın/coin rengi
16 CANAVAR_YESIL = (0, 200, 0) # Düşman canavar rengi
17 LAZER_KIRMIZI = (255, 50, 50) # Lazer ışını rengi

```

Gri tonlarında ilginç bir kural var: **üç kanalın değeri eşit olduğunda** her zaman bir gri tonu elde edersin. (0, 0, 0) en koyu gri yani siyah, (255, 255, 255) en açık gri yani beyaz, (128, 128, 128) ise tam orta gri olur.

● Bilgi: Alfa Kanalı

PyGame ayrıca **RGBA** formatını da destekler. Buradaki **A** (Alpha) saydamlık değerini temsil eder ve 0 (tamamen saydam) ile 255 (tamamen opak) arasında bir değer alır. Örneğin (255, 0, 0, 128) yarı saydam kırmızı demektir. Alfa kanalını kullanmak için Surface'in `convert_alpha()` ile oluşturulmuş olması gerekir. Bu konuya ileriki bölümlerde değineceğiz.

12.1.6 pygame.Color Sınıfı

Renkleri tuple olarak tanımlamanın yanı sıra, PyGame'in `Color` sınıfını da kullanabilirsin:

Listing 12.3: pygame.Color kullanımı

```
1 import pygame
2
3 # İsimle renk oluşturma
4 kırmızı = pygame.Color("red")
5 mavi = pygame.Color("blue")
6 beyaz = pygame.Color("white")
7
8 # RGB değeriyle oluşturma
9 özel_renk = pygame.Color(135, 206, 235)
10
11 # Renk bileşenlerine erişim
12 print(kırmızı.r) # 255
13 print(kırmızı.g) # 0
14 print(kırmızı.b) # 0
```

`pygame.Color` sınıfı İngilizce renk isimlerini tanır: "red", "green", "blue", "yellow", "white", "black" ve daha birçoğu. Ancak bu kitapta genellikle tuple formatını kullanacağız, çünkü oyun geliştirmede doğrudan sayısal kontrol daha yaygındır ve daha hızlı çalışır.

⚠ Dikkat: Dikkat!

Renk değerleri her zaman 0–255 aralığında olmalıdır. Bu aralığın dışında bir değer verersen, örneğin (300, 0, 0) yazarsan, PyGame bir `ValueError` hatası fırlatır. Özellikle renkleri matematiksel işlemlerle hesaplarırken (kararma/aydınlanma efekti gibi) bu sınırı kontrol etmeyi unutma.

12.2 Temel Şekil Çizimi

Renkler konusunu öğrendiğine göre artık ekrana bir şeyler çizmeye hazırsın! Bir oyundaki her görsel eleman, aslında ekrana çizilmiş piksellerden oluşur. Karakter bir dikdörtgen olabilir, mermi bir daire, zemin bir çizgi, patlama efekti bir elips... PyGame'in `pygame.draw` modülü, bu temel şekilleri çizmek için ihtiyacın olan her şeyi sağlar.

■ Tanım: Surface (Yüzey)

PyGame'de çizim yapılan her alan bir `Surface` (yüzey) nesnesidir. Oyun penceresi de aslında özel bir `Surface`'tir. `pygame.display.set_mode()` ile oluşturduğun pencere, "ana Surface" olarak adlandırılır ve tüm çizim işlemleri bu yüzey üzerine yapılır.

Surface kavramını bir tuval gibi düşünebilirsin. Nasıl bir ressam boş bir tuvale

boyayla şekiller çiziyorsa, sen de Surface üzerine PyGame fonksiyonlarıyla şekiller çizeceksin. Aradaki fark, senin tuvalin dijital ve saniyede onlarca kez yeniden çizilebilir olması!

12.2.1 Ekranı Temizleme: fill() Metodu

Herhangi bir şekil çizmeden önce bilmen gereken en temel işlem ekranı temizlemektir. Bir önceki üniteye oyun döngüsü içinde `ekran.fill((0, 0, 0))` satırını görmüştün. Bu satır, ekrandaki tüm pikselleri belirtilen renkle doldurur, yani bir önceki karenin üzerini boyar.

Listing 12.4: fill() ile ekran temizleme

```
1 # Ekranı siyahla temizle
2 ekran.fill(SIYAH)
3
4 # Ekranı gökyüzü mavisiyle temizle (arka plan rengi olarak)
5 GOKYUZU = (135, 206, 235)
6 ekran.fill(GOKYUZU)
7
8 # Ekranı beyazla temizle
9 ekran.fill(BEYAZ)
```

Neden her karede ekranı temizliyoruz? Çünkü oyun döngüsünde nesneler hareket eder. Eğer ekranı temizlemezsen, bir önceki karedeki çizimler yerinde kalır ve hareket eden nesneler arkalarında iz bırakır. `fill()` her kareye temiz bir başlangıç sağlar.

● Bilgi: Arka Plan Rengi

`fill()` metodunu akıllıca kullanarak oyunun atmosferini belirleyebilirsin. Bir uzay oyunu için `(0, 0, 0)` siyah, bir deniz oyunu için `(0, 105, 148)` koyu mavi, bir çöl oyunu için `(237, 201, 175)` kum rengi kullanabilirsin.

12.2.2 Çizim Sıralaması: Painter's Algorithm

Ekranı çizim yaparken sıralama çok önemlidir. PyGame, *Painter's Algorithm* (Resam Algoritması) ilkesini kullanır: **son çizilen şekil en üstte görünür**. Tıpkı bir ressamın önce arka planı, sonra orta planı, en son da ön plandaki nesneleri boyaması gibi.

Listing 12.5: Çizim sıralaması örneği

```
1 # 1. Önce arka plan (en altta)
2 ekran.fill(GOKYUZU)
3
4 # 2. Sonra zemin (arka planın üstünde)
5 pygame.draw.rect(ekran, YESIL, (0, 500, 800, 100))
6
7 # 3. Sonra karakter (zeminin üstünde)
8 pygame.draw.rect(ekran, KIRMIZI, (100, 450, 50, 50))
```

```

9
10 # 4. En son UI elemanları (en üstte)
11 pygame.draw.rect(ekran, BEYAZ, (10, 10, 200, 30))

```

Bu sıralamayı tersine çevirirsen karakter arka planın altında kalır ve görünmez olur. Aklında tutman gereken kural basit: **arka plan** → **uzak nesneler** → **yakın nesneler** → **UI elemanları**.

12.2.3 pygame.draw.rect() – Dikdörtgen Çizme

Dikdörtgen, oyun geliştirmede en çok kullanılan şekildir. Karakterler, duvarlar, butonlar, sağlık barları ve daha birçoğu dikdörtgenlerle temsil edilir.

Listing 12.6: Dikdörtgen çizme

```

1 # Dolu dikdörtgen çiz
2 # pygame.draw.rect(surface, renk, (x, y, genislik, yukseklik))
3 pygame.draw.rect(ekran, KIRMIZI, (100, 50, 200, 100))
4
5 # Çerçeve dikdörtgen çiz (width parametresiyle)
6 # Son parametre çizgi kalınlığı (piksel)
7 pygame.draw.rect(ekran, MAVI, (100, 50, 200, 100), 3)
8
9 # Yuvarlatılmış köşeli dikdörtgen
10 pygame.draw.rect(ekran, YESIL, (100, 200, 200, 100), 0, 15)
11 #
12 #                                     ^      ^
13                                     dolgu(0) köşe yarı
14                                     çapı

```

Fonksiyonun parametrelerini inceleyelim:

1. **surface**: Çizim yapılacak yüzey (genellikle **ekran**)
2. **renk**: RGB tuple formatında renk, örneğin **(255, 0, 0)**
3. **rect**: Dikdörtgenin konumu ve boyutu **(x, y, genislik, yukseklik)**
4. **width** (opsiyonel): Çizgi kalınlığı. 0 verilirse veya belirtilmezse dolu çizilir
5. **border_radius** (opsiyonel): Köşe yuvarlaklığı piksel cinsinden

▲ Dikkat: Koordinat Sistemi

PyGame'de koordinat sistemi sol üst köşeden başlar. **x** değeri sağa doğru artar, **y** değeri ise **aşağıya** doğru artar. Bu, matematikte alışık olduğun sistemin **y** eksenini için tersidir. Yani **(0, 0)** ekranın sol üst köşesi, **(800, 600)** ise sağ alt köşesidir. Bu konuyu Konu 10.1'da detaylı öğrenmiştin.

width parametresi önemli bir rol oynar. Bu parametre 0 olduğunda (veya belirtilmediğinde) dikdörtgen tamamen dolu çizilir. Sıfırdan büyük bir değer verildiğinde ise sadece çerçevesi çizilir ve verilen değer çerçevenin kalınlığını belirler.

12.2.4 pygame.draw.circle() – Daire Çizme

Daireler, mermiler, toplar, paralar, düşman nesneleri ve daha birçok oyun elemanı için kullanılır.

Listing 12.7: Daire çizme

```

1 # Dolu daire çiz
2 # pygame.draw.circle(surface, renk, (merkez_x, merkez_y),
   yarıçap)
3 pygame.draw.circle(ekran, SARI, (400, 300), 50)
4
5 # Çerçeve daire çiz
6 pygame.draw.circle(ekran, BEYAZ, (400, 300), 50, 2)
7
8 # Küçük bir mermi (dolu, 5 piksel yarıçap)
9 pygame.draw.circle(ekran, KIRMIZI, (mermi_x, mermi_y), 5)

```

Fonksiyonun parametreleri:

1. **surface**: Çizim yapılacak yüzey
2. **renk**: RGB renk değeri
3. **center**: Dairenin merkez noktası (**x**, **y**)
4. **radius**: Yarıçap (piksel cinsinden)
5. **width** (opsiyonel): Çizgi kalınlığı, 0 = dolu

Dikdörtgenden farklı olarak daire, **merkez noktası ve yarıçap** ile tanımlanır. Bu, özellikle çarpışma kontrolünde önemli bir avantaj sağlar; iki dairenin çarpışıp çarpışmadığını kontrol etmek için merkezler arası mesafeyi yarıçaplar toplamıyla karşılaştırmak yeterlidir.

12.2.5 pygame.draw.line() – Çizgi Çizme

Çizgiler, lazer ışınları, yol işaretleri, ızgara çizimleri ve görsel efektler için kullanılır.

Listing 12.8: Çizgi çizme

```

1 # Düz çizgi çiz
2 # pygame.draw.line(surface, renk, baslangic, bitis, kalinlik)
3 pygame.draw.line(ekran, BEYAZ, (100, 100), (500, 300), 2)
4 #                               başlangıç   bitiş   kalınlık
5
6 # Yatay çizgi (zemin)
7 pygame.draw.line(ekran, YESIL, (0, 500), (800, 500), 3)
8
9 # Dikey çizgi
10 pygame.draw.line(ekran, KIRMIZI, (400, 0), (400, 600), 1)
11
12 # Lazer ışını efekti (kalın kırmızı çizgi)
13 pygame.draw.line(ekran, (255, 50, 50), (oyuncu_x, oyuncu_y),
14                  (hedef_x, hedef_y), 4)

```

Fonksiyonun parametreleri:

1. **surface**: Çizim yapılacak yüzey
2. **renk**: RGB renk değeri
3. **start_pos**: Başlangıç noktası (x, y)
4. **end_pos**: Bitiş noktası (x, y)
5. **width** (opsiyonel): Çizgi kalınlığı, varsayılan 1

● Bilgi: Yumuşatılmış Çizgiler

`pygame.draw.line()` fonksiyonu düz piksel çizgileri çizer ve bu çizgiler yakından bakıldığında “basamaklı” (aliased) görünebilir. Daha pürüzsüz çizgiler istersen `pygame.draw.aaline()` fonksiyonunu kullanabilirsin. “aa” harfleri *anti-aliasing* anlamına gelir ve kenarları yumuşatarak daha doğal bir görünüm sağlar. Ancak `aaline()` yalnızca 1 piksel kalınlığında çizgi çizebilir.

12.2.6 `pygame.draw.ellipse()` – Elips Çizme

Elips, yassılaştırılmış bir dairedir. Gölgeler, gezegenler, yumurta şekilleri veya özel efektler için kullanışlıdır.

Listing 12.9: Elips çizme

```

1 # Dolu elips çiz
2 # pygame.draw.ellipse(surface, renk, sınırlayıcı_dikdörtgen)
3 pygame.draw.ellipse(ekran, TURUNCU, (150, 200, 300, 100))
4 #           x      y      genişlik
5           yükseklik
6
7 # Çerçeve elips
8 pygame.draw.ellipse(ekran, BEYAZ, (150, 200, 300, 100), 2)
9
10 # Karakter gölgesi (yere yatay elips)
11 golge_renk = (0, 0, 0) # Siyah gölge
12 pygame.draw.ellipse(ekran, golge_renk, (karakter_x - 20, yer_y,
13                                     40, 10))

```

Elips, bir dikdörtgenin içine sığdırılarak çizilir. Bu *sınırlayıcı dikdörtgen* (bounding rectangle) parametresi (x, y, genişlik, yükseklik) formatındadır. Genişlik ve yükseklik eşit olursa elde ettiğin şekil bir daire olur.

12.2.7 `pygame.draw.polygon()` – Çokgen Çizme

Çokgen, üçgen, beşgen, yıldız gibi özel şekiller çizmek için kullanılır. Köşe noktalarının bir listesini vererek istediğin şekli oluşturabilirsin.

Listing 12.10: Çokgen çizme

```

1 # Üçgen çiz (3 köşe noktası)
2 ucgen_noktaları = [(400, 100), (300, 300), (500, 300)]
3 pygame.draw.polygon(ekran, SARI, ucgen_noktaları)
4
5 # Beşgen çiz (5 köşe noktası)
6 besgen = [(400, 100), (475, 175), (450, 275), (350, 275), (325,
7           175)]
8 pygame.draw.polygon(ekran, MAVI, besgen)
9
10 # Çerçeve çokgen
11 pygame.draw.polygon(ekran, BEYAZ, ucgen_noktaları, 2)

```

Fonksiyonun parametreleri:

1. **surface**: Çizim yapılacak yüzey
2. **renk**: RGB renk değeri
3. **points**: Köşe noktalarının listesi [(x1, y1), (x2, y2), ...]
4. **width** (opsiyonel): Çizgi kalınlığı, 0 = dolu

PyGame otomatik olarak son noktayı ilk noktaya bağlar, yani şekli kapatmak için son noktayı birinci noktaya aynı yapman gerekmez.

12.2.8 pygame.draw.lines() – Birden Fazla Çizgi

Birbirine bağlı çizgiler çizmek istediğinde `pygame.draw.lines()` fonksiyonu tek tek çizgi çizmekten çok daha pratiktir.

Listing 12.11: Çoklu çizgi çizme

```

1 # Bağlı çizgiler (kapalı değil)
2 noktalar = [(100, 100), (200, 50), (300, 100), (400, 50), (500,
3           100)]
4 pygame.draw.lines(ekran, BEYAZ, False, noktalar, 2)
5 #
6 #           kapali=False (son nokta ile ilk nokta
7           birleşmez)
8
9 # Kapalı çizgiler (son nokta ilk noktaya bağlanır)
10 pygame.draw.lines(ekran, KIRMIZI, True, noktalar, 2)

```

Üçüncü parametre olan **closed**, çizginin kapalı bir şekil oluşturup oluşturmayacağını belirler. **True** ise son nokta ilk noktaya bağlanır, **False** ise açık kalır.

12.2.9 Çizim Fonksiyonları Özet Tablosu

Tüm çizim fonksiyonlarını bir arada görelim:

Tablo 12.2: pygame.draw modülü fonksiyon özeti

Fonksiyon	Parametreler
<code>rect()</code>	<code>(surface, renk, (x, y, w, h), width=0)</code>
<code>circle()</code>	<code>(surface, renk, (cx, cy), r, width=0)</code>
<code>line()</code>	<code>(surface, renk, start, end, width=1)</code>
<code>ellipse()</code>	<code>(surface, renk, (x, y, w, h), width=0)</code>
<code>polygon()</code>	<code>(surface, renk, noktalar, width=0)</code>
<code>lines()</code>	<code>(surface, renk, closed, noktalar, width=1)</code>
<code>aaline()</code>	<code>(surface, renk, start, end)</code>

Tüm fonksiyonlarda ortak olan bir kural var: `width` parametresi 0 olduğunda (veya belirtilmediğinde) şekil **dolu** çizilir, sıfırdan büyük olduğunda ise sadece **çerçevesi** çizilir ve verilen değer çerçevenin piksel kalınlığını belirler.

12.2.10 Tüm Parçaları Bir Araya Getirmek

Şimdi öğrendiğimiz her şeyi birleştirerek basit bir sahne çizelim. Bu sahne, oyun döngüsü içinde her karede yeniden çizilecek:

Listing 12.12: Basit bir sahne çizimi

```

1 import pygame
2
3 # Sabitler
4 GENISLIK = 800
5 YUKSEKLIK = 600
6 GOKYUZU = (135, 206, 235)
7 YESIL = (34, 139, 34)
8 KAHVERENGİ = (139, 69, 19)
9 KOYU_YESIL = (0, 100, 0)
10 SARI = (255, 255, 0)
11 BEYAZ = (255, 255, 255)
12 KIRMIZI = (255, 0, 0)
13
14 pygame.init()
15 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
16 pygame.display.set_caption("Basit Sahne")
17 saat = pygame.time.Clock()
18
19 calistir = True
20 while calistir:
21     for olay in pygame.event.get():

```

```

22     if olay.type == pygame.QUIT:
23         calistir = False
24
25     # 1. Arka plan: Gökyüzü
26     ekran.fill(GOKYUZU)
27
28     # 2. Güneş (sarı daire)
29     pygame.draw.circle(ekran, SARI, (650, 80), 50)
30
31     # 3. Bulut (üç beyaz daire)
32     pygame.draw.circle(ekran, BEYAZ, (200, 100), 30)
33     pygame.draw.circle(ekran, BEYAZ, (230, 85), 40)
34     pygame.draw.circle(ekran, BEYAZ, (265, 100), 35)
35
36     # 4. Zemin (yeşil dikdörtgen)
37     pygame.draw.rect(ekran, YESIL, (0, 450, GENISLIK, 150))
38
39     # 5. Ev gövdesi (kahverengi dikdörtgen)
40     pygame.draw.rect(ekran, KAHVERENGI, (300, 350, 200, 100))
41
42     # 6. Çatı (koyu yeşil üçgen)
43     pygame.draw.polygon(ekran, KOYU_YESIL,
44                         [(280, 350), (400, 250), (520, 350)])
45
46     # 7. Kapı (kırmızı dikdörtgen)
47     pygame.draw.rect(ekran, KIRMIZI, (370, 400, 60, 50))
48
49     # Ekranı güncelle
50     pygame.display.flip()
51     saat.tick(60)
52
53 pygame.quit()

```

Bu kodu çalıştırdığında gökyüzü arka planlı, güneşli, bulutlu, çimenli bir zemin üzerinde çatılı ve kapılı bir ev göreceksin. Dikkat edersen çizim sıralaması tam olarak painter's algorithm ilkesine uyuyor: önce gökyüzü (en arka), sonra güneş ve bulutlar, ardından zemin, ev gövdesi, çatı ve en son kapı (en ön).

💡 İpucu: Profesyonel İpucu

Gerçek oyunlarda basit şekiller yerine genellikle *sprite* denilen görsel dosyalar (PNG, BMP) kullanılır. Ancak şekil çizme yeteneği prototip oluştururken, debug bilgisi gösterirken ve UI elemanları çizerken vazgeçilmezdir. Profesyonel oyun geliştiricileri ilk önce “programmer art” adı verilen basit şekillerle oyun mekaniğini test eder, görsel dosyaları en son ekler.

12.3 Ünite Özeti

Bu ünite de PyGame'in görsel dünyasına ilk adımı attın. Artık ekrana renkli şekiller çizebilir, arka plan oluşturabilir ve basit sahneler tasarlayabilirsin. Bunlar,

ileride sprite'lar, animasyonlar ve karmaşık oyun grafikleri oluşturmanın temelini oluşturuyor.

12.3.1 Bu Ünite Öğrendiklerin

- **RGB Renk Sistemi:** Kırmızı, Yeşil ve Mavi kanallarının 0–255 aralığında değerler alarak 16.7 milyon renk oluşturma
- **Renk Tanımlama:** Tuple formatında (R, G, B) sabitler oluşturma
- **Surface ve fill():** Ekranı temizleme ve arka plan rengi belirleme
- **Painter's Algorithm:** Son çizilenin en üstte görünmesi ilkesi
- **Dikdörtgen:** `pygame.draw.rect()` ile dolu ve çerçeve dikdörtgenler
- **Daire:** `pygame.draw.circle()` ile merkez ve yarıçap belirterek çizim
- **Çizgi:** `pygame.draw.line()` ile iki nokta arasında çizgi çekme
- **Elips:** `pygame.draw.ellipse()` ile sınırlayıcı dikdörtgen içinde elips
- **Çokgen:** `pygame.draw.polygon()` ile köşe noktalarından şekil oluşturma

12.3.2 Çizim Fonksiyonları Hızlı Başvuru

Tablo 12.3: pygame.draw modülü hızlı başvuru tablosu

Fonksiyon	Şekil	Kullanım Alanı
<code>fill()</code>	Tüm ekran	Arka plan temizleme
<code>draw.rect()</code>	Dikdörtgen	Duvarlar, butonlar, sağlık barı
<code>draw.circle()</code>	Daire	Mermiler, toplar, paralar
<code>draw.line()</code>	Çizgi	Lazerler, ızgaralar, sınır çizgileri
<code>draw.ellipse()</code>	Elips	Gölgeler, gezegenler
<code>draw.polygon()</code>	Çokgen	Üçgenler, yıldızlar, özel şekiller
<code>draw.aaline()</code>	Yumuşak çizgi	Pürüzsüz çizgiler

12.3.3 Önemli Kurallar

1. Renk değerleri her zaman 0–255 aralığında olmalıdır
2. Renk sabitleri **büyük harfle** tanımlanır: `KIRMIZI = (255, 0, 0)`
3. Her karede `fill()` ile ekranı **temizle**, sonra çiz
4. Çizim sıralaması: **arka plan** → **uzak** → **yakın** → **UI**
5. `width=0` dolu çizer, `width>0` çerçeve çizer

12.3.4 Sıradaki Adım

Bir sonraki bölümde **kullanıcı girdileri ve hareket** konusunu öğreneceksin. Klavye tuşlarına basarak ekrandaki şekilleri hareket ettirebilecek, fareyle tıklayarak tepki verebileceksin. Bu ünite çizdiğin statik şekiller, bir sonraki bölümde hayat bulacak!

12.3.5 Mini Görev: Kendi Sahneni Tasarla

Öğrendiklerini pekiştirmek için aşağıdaki görevi yap:

```
1 # sahne.py
2 # Hedef: En az 5 farklı şekil türü kullanarak bir sahne çiz
3 # Gereksinimler:
4 # - Bir arka plan rengi (fill)
5 # - En az 1 dikdörtgen
6 # - En az 1 daire
7 # - En az 1 çizgi
8 # - En az 1 çokgen (üçgen veya başka)
9 # - En az 5 farklı renk
10 #
11 # Fikir önerileri:
12 # - Bir gece manzarası (koyu mavi arka plan, sarı yıldızlar, ay)
13 # - Bir robot yüzü (dikdörtgen kafa, daire gözler)
14 # - Bir trafik lambası (dikdörtgen gövde, daire ışıklar)
```

💡 İpucu: Pratik Yap!

Bu görevi tamamladıktan sonra şunu dene:

1. Çizim sıralamasını değiştir ve sonucu gözlemle
2. Bir şeklin `width` parametresini değiştirerek dolu ve çerçeve arasında geçiş yap
3. Yeni renkler oluşturarak tablodaki 15 rengin dışına çık
4. Ekran boyutunu değiştirdiğinde şekillerin konumunu da güncelle

“Bir oyun, oyuncunun her hareketini dinleyen canlı bir varlıktır. Kulak ver, tepki ver, unutma.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- PyGame olay kuyruğunun yapısı ve `pygame.event.get()` ile olayları okuma
- `Event` nesnesinin özellikleri: `type`, `key`, `pos` ve diğerleri
- Farklı olay türleri ve her birinin taşıdığı bilgiler
- Olay filtreleme teknikleri: `pygame.event.set_blocked()` ve `pygame.event.set_allowed()`
- Kendi özel olaylarını tanımlama (`pygame.USEREVENT`)

13.1 Event Kuyruğu ve Olay İşleme

Bölüm 11’de oyun döngüsünün ilk aşamasının olayları işlemek olduğunu öğrendin. `pygame.event.get()` ile `pygame.QUIT` olayını yakaladın ve olay kuyruğu kavramıyla tanıştın. O ünite de olay sistemi bir “giriş” niteliğindeydi; amacımız oyun döngüsünü ayağa kaldırmaktı. Şimdi ise olay sisteminin derinlerine dalma zamanı geldi.

Bir konser salonunu düşün. Kapıda bir güvenlik görevlisi duruyor ve gelen her izleyiciyi sıraya koyuyor. Biletleri kontrol edilmeden kimse salona giremez. PyGame’in olay sistemi de tam olarak böyle çalışır: kullanıcı bir tuşa bastığında, fareyi hareket ettirdiğinde veya pencere boyutunu değiştirdiğinde bu eylemler birer *olay* (event) olarak bir kuyruğa eklenir. Sen bu kuyruğu okuyana kadar olaylar orada bekler.

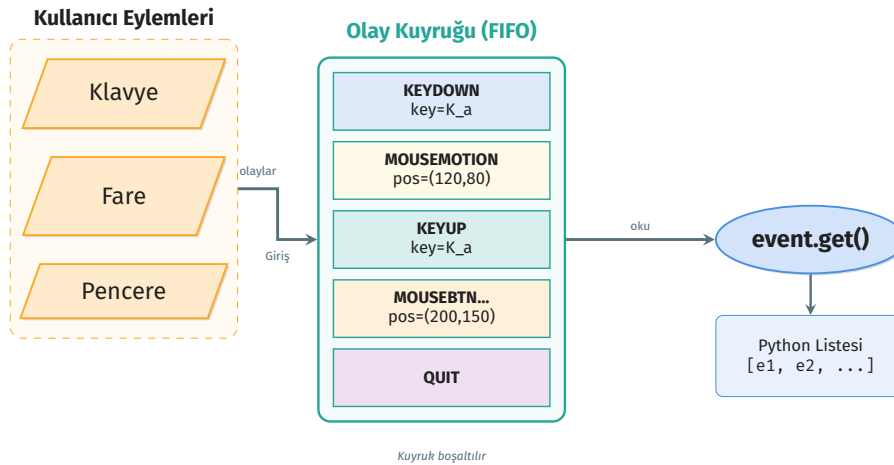
13.1.1 FIFO Kuyruk Yapısı

PyGame'in olay kuyruğu bir *FIFO* (First In, First Out – İlk Giren, İlk Çıkar) yapısıdır. Bu ne demek? Market kasasındaki sıraya benzetebilirsin: kasaya ilk gelen müşteri ilk hizmet alır. Aynı şekilde olay kuyruğuna ilk eklenen olay, `pygame.event.get()` çağrıldığında ilk sırada döndürülür.

■ Tanım: Olay Kuyruğu – Event Queue

PyGame'in olay kuyruğu, kullanıcı ve sistem tarafından üretilen olayları kronolojik sırayla saklayan bir FIFO veri yapısıdır. Kuyruk varsayılan olarak 128 olaya kadar tutabilir. `pygame.event.get()` çağrılana dek olaylar kuyrukta birikir; çağrıldığında tüm olaylar bir Python listesi olarak döndürülür ve kuyruk boşaltılır.

Bu yapıyı görsel olarak incelersek daha net anlayacağız.



Şekil 13.1: PyGame olay kuyruğunun FIFO yapısı

Şekil 13.1'da görüldüğü gibi solda kullanıcı eylemleri (klavye, fare, pencere) olayları kuyruğa ekler. Kuyrukta olaylar oluşum sırasıyla dizilir. Sağda ise `pygame.event.get()` bu olayları aynı sırayla bir Python listesine aktarır ve kuyruğu temizler. Böylece bir sonraki döngü turunda kuyruk yeni olayları toplamaya hazır olur.

13.1.2 Event Nesnesi ve Özellikleri

Olay kuyruğundan okuduğun her eleman bir `pygame.event.Event` nesnesidir. Bu nesnenin en temel özelliği `type` alanıdır: olayın türünü belirten bir tam sayı değeridir. Ancak farklı olay türleri ek özellikler de taşır.

Örneğin bir `pygame.KEYDOWN` olayı şu özelliklere sahiptir: `key` hangi tuşa basıldığını, `mod` Shift veya Ctrl gibi değiştirici tuşların durumunu, `unicode` ise basılan tuşun karakter karşılığını verir. Bir `pygame.MOUSEBUTTONDOWN` olayı

ise `pos` ile farenin ekrandaki konumunu, `button` ile hangi düğmeye tıklandığını bildirir.

● Bilgi: Event Nesnesinin Yapısı

Her `Event` nesnesinin `type` özelliği zorunludur. Diğer özellikler olay türüne göre değişir. Bir olayın hangi özelliklere sahip olduğunu öğrenmek için `print(olay)` veya `olay.__dict__` kullanabilirsin.

13.1.3 `event.get()` Derinlemesine

Konu 11.2’de `pygame.event.get()` fonksiyonunu parametresiz kullanmıştın. Bu durumda kuyruktan tüm olaylar alınır. Ancak bu fonksiyon daha esnek bir kullanım da sunar.

Parametresiz çağrı tüm olayları döndürür:

Python – Tüm Olayları Okuma

```
1 olaylar = pygame.event.get()
2 for olay in olaylar:
3     if olay.type == pygame.QUIT:
4         calistir = False
```

Parametrelili çağrı ile sadece belirli türleri okuyabilirsin:

Python – Filtrelenmiş Okuma

```
1 # Sadece QUIT olaylarini al
2 quit_olaylari = pygame.event.get(pygame.QUIT)
3
4 # Birden fazla tur belirt
5 klavye_olaylari = pygame.event.get([pygame.KEYDOWN,
6     ↪ pygame.KEYUP])
```

⚠ Dikkat: Parametrelili `event.get()` Tuzağı

Parametrelili `event.get()` kullandığında belirtmediğin olay türleri kuyrukta kalır. Eğer bu olayları hiçbir zaman okumazsan kuyruk taşar ve programın yanıt vermez hale gelir. Güvenli yol, döngünün bir yerinde mutlaka parametresiz `event.get()` çağırmaktır.

13.1.4 Diğer Kuyruk Okuma Yöntemleri

`pygame.event.get()` dışında iki yardımcı fonksiyon daha vardır. `pygame.event.peek()` kuyruğu boşaltmadan belirli bir olay türünün kuyrukta olup olmadığını kontrol eder; sadece `True` veya `False` döndürür. `pygame.event.poll()` ise kuyruktan yalnızca bir olay alır ve döndürür; kuyruk boşsa `type` değeri `pygame.NOEVENT` olan bir olay döner.

Bu iki fonksiyon performansa duyarlı durumlarda faydalıdır ama standart kullanımda `pygame.event.get()` ile `for` döngüsü en güvenli ve okunabilir yaklaşımdır.

Şimdi olay kuyruğuyla etkileşimi daha iyi görmek için kapsamlı bir gözlemci programı yazalım. Bu programın kalbi, her olay türüne göre farklı bilgileri konsola yazdıran bir fonksiyondur:

Python – Event Gözlemcisi – Olay Bilgisi Fonksiyonu

```

1 def olay_bilgisi_yazdir(olay, sayac):
2     """Olay turune gore detayli bilgi yazdirir."""
3     if olay.type == pygame.KEYDOWN:
4         tus_adi = pygame.key.name(olay.key)
5         karakter = olay.unicode if olay.unicode else
6             ↪ "(yok)"
7         mod_bilgi = ""
8         if olay.mod & pygame.KMOD_SHIFT:
9             mod_bilgi += " +Shift"
10        if olay.mod & pygame.KMOD_CTRL:
11            mod_bilgi += " +Ctrl"
12        print(f"[{sayac}] KEYDOWN - Tus: {tus_adi}, "
13              f"Karakter: {karakter}{mod_bilgi}")
14
15    elif olay.type == pygame.MOUSEBUTTONDOWN:
16        dugme_adi = {1: "Sol", 2: "Orta", 3: "Sag",
17                    4: "Tekerlek yukari", 5: "Tekerlek
18                    ↪ asagi"}
19        adi = dugme_adi.get(olay.button, f"Dugme
20        ↪ {olay.button}")
21        print(f"[{sayac}] MOUSEBUTTONDOWN - "
22              f"Konum: {olay.pos}, Dugme: {adi}")
23
24    elif olay.type == pygame.MOUSEMOTION:
25        print(f"[{sayac}] MOUSEMOTION - "
26              f"Konum: {olay.pos}, Fark: {olay.rel}")
27    # ... diger olay turleri benzer sekilde ...

```

Dikkat edersen her olay türü farklı özelliklere sahip: `pygame.KEYDOWN` için `key`, `mod` ve `unicode`; `pygame.MOUSEBUTTONDOWN` için `pos` ve `button`; `pygame.MOUSEMOTION` için `pos` ve `rel` kullanılıyor.

Ana döngü ise bu fonksiyonu her olay için çağırır:

Python – Event Gözlemcisi – Ana Döngü

```

1 olay_sayaci = 0
2 calistir = True
3 while calistir:
4     for olay in pygame.event.get():
5         olay_sayaci += 1
6         olay_bilgisi_yazdir(olay, olay_sayaci)
7

```

```

8         if olay.type == pygame.QUIT:
9             calistir = False
10        elif olay.type == pygame.KEYDOWN:
11            if olay.key == pygame.K_ESCAPE:
12                calistir = False
13
14        ekran.fill(ARKA_PLAN)
15        pygame.display.flip()
16        saat.tick(FPS)

```

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite1/01\_event\_gozlemci.py` dosyasında bulabilirsin. Tüm olay türlerini (KEYUP, MOUSEBUTTONDOWN vb.) ayrı ayrı ele alan genişletilmiş versiyonu içerir.

Bu programı çalıştırdığında pencere açılacak ve gerçekleştirdiğin her eylem konsola detaylı olarak yazdırılacak. Fareyi pencerenin üzerinde gezdirdiğinde `pygame.MOUSEMOTION` olaylarının ne kadar sık üretildiğini fark edeceksin — her piksel hareketinde yeni bir olay oluşur. Bir tuşa bastığında ise hem tuşun adını hem de tuş kodunu göreceksin. Bu program, olay sistemini keşfetmenin en iyi yollarından biridir.

💡 İpucu: Olay Kodlarını Keşfet

`01_event_gozlemci.py` programını çalıştırıp farklı tuşlara bas, fareyi hareket ettir, pencereyi yeniden boyutlandır. Konsoldaki çıktıyı inceleyerek hangi eylemin hangi olay türünü ürettiğini ve her olay türünün hangi ek bilgileri taşıdığını keşfedeceksin. Bu deneyim, sonraki konularda olay türlerini ezberlemek yerine *anlamana* yardımcı olacak.

13.2 Olay Türleri ve Özellikleri

Önceki konuda olay kuyruğunun nasıl çalıştığını ve `Event` nesnesinin `type` özelliğiyle olay türünü belirlediğini öğrendin. Peki PyGame kaç farklı olay türü tanımlıyor ve her biri hangi bilgileri taşıyor? Bu konuda tüm ana olay türlerini sistematik olarak inceleyeceğiz.

Bir olay türünü bir zarfın üzerindeki etikete benzetebilirsin. Etiket “Fatura” yazıyorsa zarfın içinde tutar bilgisi vardır; “Davetiye” yazıyorsa tarih ve yer bilgisi vardır. Aynı şekilde PyGame’de bir olayın `type` değeri `pygame.KEYDOWN` ise o olay nesnesinde `key`, `mod` ve `unicode` özellikleri bulunur. `pygame.MOUSEBUTTONDOWN` ise `pos` ve `button` taşır.

13.2.1 Olay Türü Sabitleri

PyGame her olay türünü bir tam sayı sabiti ile temsil eder. Örneğin `pygame.QUIT` aslında 256, `pygame.KEYDOWN` ise 768 değerine karşılık gelir. Ancak kod yazar-ken bu sayıları kullanmak yerine sabit isimlerini kullanmak hem daha okunabilir hem de hata olasılığını azaltır.

Bu sabitler `pygame` modülü altında tanımlıdır ve `pygame.event.EventType` numaralandırmasıyla erişilebilir. Aşağıdaki tabloda en yaygın olay türlerini ve taşıdıkları özellikleri bulacaksınız.

13.2.2 Ana Olay Türleri Tablosu

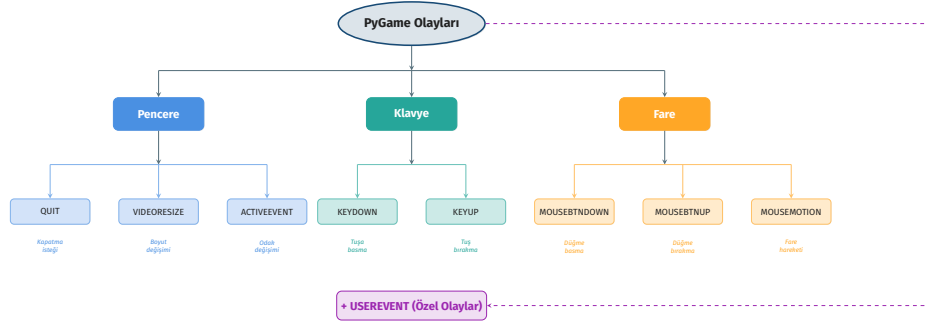
Tablo 13.1: PyGame ana olay türleri ve özellikleri

Olay Türü	Ek Özellikler	Açıklama
QUIT	–	Pencere kapatma isteği (X düğmesi veya Alt+F4)
KEYDOWN	<code>key</code> , <code>mod</code> , <code>unicode</code>	Bir tuşa basıldığında. <code>key</code> : tuş kodu, <code>mod</code> : değiştirici tuşlar (Shift, Ctrl), <code>unicode</code> : karakter karşılığı
KEYUP	<code>key</code> , <code>mod</code>	Basılı tuş bırakıldığında
MOUSEBUTTONDOWN	<code>pos</code> , <code>button</code>	Fare düğmesine basıldığında. <code>pos</code> : ko-num (x,y), <code>button</code> : düğme numarası (1=sol, 2=orta, 3=sağ, 4=tekerlek yukarı, 5=tekerlek aşağı)
MOUSEBUTTONUP	<code>pos</code> , <code>button</code>	Fare düğmesi bırakıldığında
MOUSEMOTION	<code>pos</code> , <code>rel</code> , <code>buttons</code>	Fare hareket ettiğinde. <code>rel</code> : son ko-numdan fark, <code>buttons</code> : basılı düğmeler
VIDEORESIZE	<code>size</code> , <code>w</code> , <code>h</code>	Yeniden boyutlandırılabilir pencere boyutu değiştiğinde
ACTIVEEVENT	<code>gain</code> , <code>state</code>	Pencere odak kazandığında veya kaybettiğinde
USEREVENT	(kullanıcı tanımlı)	Programcının tanımladığı özel olaylar için ayrılmış başlangıç değeri

13.2.3 Olay Türlerinin Sınıflandırılması

Bu kadar olay türünü aklında tutmak zor görünebilir ama aslında mantıklı bir sınıflandırma yaparsak oldukça basitleşir. PyGame olaylarını üç ana kategoriye ayırabiliriz: pencere olayları, klavye olayları ve fare olayları.

Şekil 13.2’de görüldüğü gibi olaylar üç ana dalda toplanır. **Pencere olayları** pencerenin kendisiyle ilgili değişiklikleri bildirir: kapatma (`pygame.QUIT`), boyut değişikliği (`pygame.VIDEORESIZE`) ve odak değişimi (`pygame.ACTIVEEVENT`). **Klavye olayları** tuşa basma (`pygame.KEYDOWN`) ve tuş bırakma (`pygame.KEYUP`)



Şekil 13.2: PyGame olay türlerinin ağaç yapısı

olmak üzere iki tanedir. **Fare olayları** ise düğme basma (`pygame.MOUSEBUTTONDOWN`), düğme bırakma (`pygame.MOUSEBUTTONUP`) ve fare hareketi (`pygame.MOUSEMOTION`) olarak üçe ayrılır.

13.2.4 Klavye Olaylarının Detayı

`pygame.KEYDOWN` olayı klavye girdilerinin temelini oluşturur. Bu olay üç önemli bilgi taşır. Birincisi, **key** özelliği hangi fiziksel tuşa basıldığını belirten bir tam sayıdır. PyGame bu değerler için **K_** ile başlayan sabitler tanımlar: `K_SPACE` boşluk tuşu, `K_a` “a” harfi, `K_ESCAPE` Escape tuşudur.

İkincisi, **mod** özelliği basma anında Shift, Ctrl, Alt gibi değiştirici tuşlardan hangilerinin basılı olduğunu bit maskesi olarak tutar. Örneğin Shift basılıyken bir harfe basıldığında **mod** değerinde `KMOD_SHIFT` bayrağı aktif olur.

Üçüncüsü, **unicode** özelliği basılan tuşun karakter karşılığını verir. “a” tuşuna basıldığında **unicode** değeri “a”, Shift ile birlikte basıldığında ise “A” olur. Bu özellik metin girişi uygulamalarında çok faydalıdır.

● Bilgi: KEYDOWN ve KEYUP Farkı

`pygame.KEYDOWN` tuşa basıldığı anda bir kez tetiklenir. Tuş basılı tutulursa işletim sistemi ayarlarına bağlı olarak belirli bir gecikmeden sonra tekrarlanmaya başlar. `pygame.KEYUP` ise tuş bırakıldığında bir kez tetiklenir. Sürekli hareket gibi işlemlerde `pygame.key.get_pressed()` fonksiyonu daha uygundur; bunu Ünite 2’de detaylı öğreneceksin.

13.2.5 Fare Olaylarının Detayı

Fare olayları üç türde gelir. `pygame.MOUSEBUTTONDOWN` fare düğmesine basıldığında, `pygame.MOUSEBUTTONUP` bırakıldığında tetiklenir. Her ikisi de **pos** (farenin pencere koordinatlarındaki konumu) ve **button** (hangi düğme: 1=sol, 2=orta, 3=sağ, 4=tekerlek yukarı, 5=tekerlek aşağı) özelliklerini taşır.

`pygame.MOUSEMOTION` ise fare her hareket ettiğinde tetiklenir ve üç özellik taşır: **pos** (güncel konum), **rel** (bir önceki konumdan piksel farkı) ve **buttons** (o an basılı olan düğmelerin tuple’ı, örneğin `(1, 0, 0)` sol düğme basılı demektir).

⚠ Dikkat: MOUSEMOTION Sıklığı

`pygame.MOUSEMOTION` olayı fare her piksel hareket ettiğinde üretilir. Fare hızlı hareket ediyorsa saniyede yüzlerce olay oluşabilir. Bu nedenle `pygame.MOUSEMOTION` olayı içinde ağır hesaplamalar yapmaktan kaçın. İhtiyaç duymuyorsan bu olay türünü engelleyebilirsin — bir sonraki konuda bunu öğreneceksin.

13.2.6 Pencere Olayları

`pygame.QUIT` olayını zaten iyi tanıyorsun. `pygame.VIDEORESIZE` olayı ise yalnızca `RESIZABLE` bayrağıyla oluşturulmuş pencerelerde tetiklenir. Kullanıcı pencereyi yeniden boyutlandığında bu olay `size`, `w` (genişlik) ve `h` (yükseklik) özellikleriyle yeni boyutu bildirir.

`pygame.ACTIVEEVENT` pencere odak kazandığında veya kaybettiğinde tetiklenir. `gain` özelliği 1 ise odak kazanıldı, 0 ise kaybedildi demektir. `state` özelliği ise odak türünü belirtir (fare odağı veya klavye odağı). Bu olayı kullanarak oyunu “duraklatma” özelliği ekleyebilirsin — kullanıcı başka bir pencereye geçtiğinde oyun otomatik olarak durur.

💡 İpucu: Olay Türlerini Deneyerek Öğren

Hangi olay türünün hangi bilgileri taşıdığını ezberlemek yerine, önceki konudaki `01\_event\_gozlemci.py` programını çalıştır ve farklı eylemler dene. Her eylemi yaptıktan sonra konsola düşen çıktıyı incele. Bu deneysel yaklaşım, tablodaki bilgileri çok daha kalıcı şekilde öğrenmeni sağlar.

13.3 Olay Filtreleme ve Özel Olaylar

Önceki iki konuda olay kuyruğunun yapısını ve PyGame’in sunduğu olay türlerini öğrendin. Ancak her oyun tüm olaylarla ilgilenmez. Bir puzzle oyununda fare hareketi olayları gereksiz olabilir; bir platformer’da pencere boyut değişikliği önemsiz olabilir. PyGame bu durumlar için güçlü bir filtreleme sistemi sunar. Ayrıca kendi özel olaylarını tanımlayarak olay sistemini genişletebilirsin.

13.3.1 Olay Engelleme: `set_blocked()`

`pygame.event.set_blocked()` fonksiyonu belirli olay türlerinin kuyruğa eklenmesini engeller. Engellenen olaylar hiçbir zaman kuyruğa girmez, dolayısıyla `pygame.event.get()` çağrıldığında da görünmez.

Python – Olay Engelleme

```
1 # Fare hareketi olaylarini engelle
2 pygame.event.set_blocked(pygame.MOUSEMOTION)
3
4 # Birden fazla olay turunu engelle
```

```

5 pygame.event.set_blocked([pygame.MOUSEMOTION,
  ↳ pygame.ACTIVEEVENT])
6
7 # Tum olaylari engelle (None parametresi)
8 pygame.event.set_blocked(None)

```

Bu fonksiyon özellikle performans optimizasyonu için faydalıdır. `pygame.MOUSEMOTION` olayı saniyede yüzlerce kez üretilebildiği için, fare hareketi bilgisine ihtiyacın yoksa bu olay türünü engellemek kuyruğun gereksiz yere dolmasını önler.

13.3.2 Olay İzin Listesi: `set_allowed()`

`pygame.event.set_allowed()` fonksiyonu `set_blocked()` işlevinin tersidir. Daha önce engellenmiş bir olay türünün tekrar kuyruğa eklenmesine izin verir.

Python – Olay İzin Verme

```

1 # Once tum olaylari engelle
2 pygame.event.set_blocked(None)
3
4 # Sadece QUIT ve KEYDOWN olaylarına izin ver
5 pygame.event.set_allowed([pygame.QUIT, pygame.KEYDOWN])

```

Bu yaklaşım bir “beyaz liste” (whitelist) mantığıyla çalışır: önce her şeyi engelle, sonra yalnızca ihtiyacın olanları serbest bırak. Bu sayede kuyrukta sadece senin ilgilendiğin olaylar birikir.

● Bilgi: Engelleme ve İzin Dengesini Kur

Oyununuzun geliştirme aşamasında tüm olaylara izin vermek hata ayıklamayı kolaylaştırır. Oyun tamamlandıktan sonra performans optimizasyonuna geçtiğinde kullanılmayan olay türlerini engelleyebilirsiniz.

13.3.3 Kuyruğu Temizleme: `event.clear()`

Bazen kuyrukta birikmiş tüm olayları okumadan silmek isteyebilirsiniz. Örneğin bir yükleme ekranından oyun ekranına geçerken, yükleme sırasında biriken eski olayların oyunu etkilemesini istemezsiniz.

Python – Kuyruğu Temizleme

```

1 # Tum olaylari sil
2 pygame.event.clear()
3
4 # Sadece belirli turleri sil
5 pygame.event.clear(pygame.MOUSEMOTION)

```


`pygame.event.clear()` fonksiyonu `pygame.event.get()` ile benzer çalışır; fark olarak olayları bir liste olarak döndürmez, doğrudan siler. Parametresiz çağrıda tüm olaylar silinir; parametre verilirse sadece o türdeki olaylar silinir.

13.3.4 Özel Olaylar: **USEREVENT**

PyGame'in olay sistemi sadece klavye, fare ve pencere olaylarıyla sınırlı değildir. `pygame.USEREVENT` sabiti ile kendi özel olay türlerini tanımlayabilirsin. Bu, oyun mantığında zamanlanmış olaylar, skor güncellemeleri veya özel tetikleyiciler oluşturmak için son derece güçlü bir araçtır.

■ Tanım: **USEREVENT**

`pygame.USEREVENT` PyGame'in kullanıcı tanımlı olaylar için ayırdığı başlangıç sayısal değeridir. Her özel olay türü için **USEREVENT + 1**, **USEREVENT + 2** gibi ardışık değerler kullanılır. PyGame 2.x'te `pygame.NUMEVENTS` sabitine kadar (varsayılan 32768) özel olay tanımlayabilirsin.

13.3.5 Olay Gönderme: **event.post()**

Özel olay tanımladıktan sonra bu olayı kuyruğa eklemek için `pygame.event.post()` fonksiyonunu kullanırsın. Kuyruğa eklenen özel olay, standart olaylarla aynı şekilde `pygame.event.get()` ile okunur.

Python – Özel Olay Oluşturma ve Gönderme

```

1  # Özel olay türleri tanımla
2  DUSMAN_OLUSTUR = pygame.USEREVENT + 1
3  BONUS_ZAMANI = pygame.USEREVENT + 2
4
5  # Özel olay olustur ve kuyruğa ekle
6  yeni_olay = pygame.event.Event(DUSMAN_OLUSTUR)
7  pygame.event.post(yeni_olay)
8
9  # Ek bilgi taşıyan özel olay
10 bonus_olay = pygame.event.Event(BONUS_ZAMANI, puan=50,
11     ↪ tur="altın")
12 pygame.event.post(bonus_olay)

```

Dikkat edersen ikinci örnekte `puan=50` ve `tur="altın"` gibi ek özellikler ekledik. Böylece olay yakalandığında `olay.puan` ve `olay.tur` ile bu bilgilere erişebilirsin. Bu esneklik, olay sistemini oyun mantığında bir mesajlaşma altyapısı olarak kullanmanı sağlar.

13.3.6 Zamanlayıcı Olaylar: **set_timer()**

PyGame'in en kullanışlı özelliklerinden biri zamanlayıcı olaylardır. `pygame.time.set_timer()` fonksiyonu belirli bir olay türünü belirli aralıklarla otomatik olarak kuyruğa ekler.

Bu, düşman üretimi, güç yenileme veya geri sayım gibi zamana bağlı mekaniklerde çok işe yarar.

Python – Zamanlayıcı Olay

```

1  # Her 2000 milisaniyede (2 saniye) DUSMAN_OLUSTUR olayini
   ↳ gönder
2  DUSMAN_OLUSTUR = pygame.USEREVENT + 1
3  pygame.time.set_timer(DUSMAN_OLUSTUR, 2000)
4
5  # Zamanlayiciyi durdurmak için süre olarak 0 ver
6  pygame.time.set_timer(DUSMAN_OLUSTUR, 0)

```

⚠ Dikkat: `set_timer()` Dikkat Noktası

`pygame.time.set_timer()` sadece olay kuyruğuna olay ekler; olayı işlemek senin sorumluluğundadır. Olay döngüsünde bu özel olay türünü yakalamayı unutursan zamanlayıcı çalışır ama hiçbir şey olmaz. Ayrıca aynı olay türü için ikinci kez `set_timer()` çağırırsan önceki zamanlayıcı iptal edilir ve yenisi başlar.

Şimdi tüm bu kavramları bir arada görmek için kapsamlı bir örnek inceleyelim. İlk olarak programın başında filtreleme ve zamanlayıcı ayarlarını yaparız:

Python – Filtreleme ve Zamanlayıcı Kurulumu

```

1  # Özel olay türleri tanımla
2  DUSMAN_OLUSTUR = pygame.USEREVENT + 1
3  BONUS_ZAMANI = pygame.USEREVENT + 2
4  SKOR_GUNCELLE = pygame.USEREVENT + 3
5
6  # MOUSEMOTION olaylarını engelle (performans için)
7  pygame.event.set_blocked(pygame.MOUSEMOTION)
8
9  # Zamanlayıcı olayları başlat
10 pygame.time.set_timer(DUSMAN_OLUSTUR, 3000) # Her 3
   ↳ saniye
11 pygame.time.set_timer(BONUS_ZAMANI, 5000)   # Her 5
   ↳ saniye

```

Ardından olay döngüsünde hem standart olayları hem de özel olayları birlikte işleriz:

Python – Özel Olayları İşleme

```

1  for olay in pygame.event.get():
2      if olay.type == pygame.QUIT:
3          calistir = False
4
5      elif olay.type == pygame.KEYDOWN:

```

```

6         if olay.key == pygame.K_SPACE:
7             # Özel skor olayını oluştur ve gönder
8             skor_olayi = pygame.event.Event(
9                 SKOR_GUNCELLE, puan=10, kaynak="oyuncu"
10            )
11            pygame.event.post(skor_olayi)
12
13        # Zamanlayıcı olayları
14        elif olay.type == DUSMAN_OLUSTUR:
15            dusman_sayisi += 1
16            print(f"Yeni dusman! Toplam: {dusman_sayisi}")
17
18        elif olay.type == BONUS_ZAMANI:
19            skor += 25
20
21        # event.post() ile gönderilen özel olay
22        elif olay.type == SKOR_GUNCELLE:
23            skor += olay.puan
24            print(f"+{olay.puan} puan (kaynak:
25                ↪ {olay.kaynak})")

```

Bu programda üç önemli kavramı bir arada görüyorsun. Birincisi, `set_blocked()` ile `pygame.MOUSEMOTION` olaylarını engelleyerek kuyruğun gereksiz olaylarla dolmasını önliyoruz. İkincisi, `pygame.USEREVENT` tabanlı özel olay türleri tanımlayıp `set_timer()` ile belirli aralıklarla kuyruğa eklenmesini sağlıyoruz. Üçüncüsü, `event.post()` ile kullanıcı etkileşimine bağlı özel olaylar gönderiyoruz. Dikkat edersen `SKOR_GUNCELLE` olayına eklediğimiz `puan` ve `kaynak` özelliklerine `olay.puan` ve `olay.kaynak` ile doğrudan erişebiliyoruz.

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite1/02\_event\_fil` dosyasında bulabilirsin. Ekranda mesajları ve skor bilgisini görüntüleyen genişletilmiş versiyonu içerir.

💡 İpucu: Özel Olaylarla Temiz Kod

Özel olaylar sayesinde oyun mantığını modüler hale getirebilirsin. Örneğin düşman oluşturma kodunu doğrudan zamanlayıcıda yazmak yerine bir olay gönder ve olay işleme bölümünde bu olayı yakala. Bu yaklaşım kodun okunabilirliğini ve bakımını kolaylaştırır.

13.4 Sonuç

13.4.1 Ünite Özeti

Bu ünite de PyGame'in olay sistemini derinlemesine inceledik. Bölüm 11'de olay kuyruğuyla tanışmış ve `pygame.QUIT` olayını yakalamayı öğrenmiştin. Şimdi ise

olay sisteminin tüm katmanlarını keşfettin.

Olay kuyruğunun FIFO yapısıyla çalıştığını, olayların kronolojik sırayla biriktiğini ve `pygame.event.get()` çağrıldığında bir Python listesi olarak döndürülen kuyruğun boşaltıldığını öğrendin. Her `Event` nesnesinin zorunlu bir `type` özelliği taşıdığını ve olay türüne göre ek özellikler (`key`, `pos`, `button`, `mod`, `unicode` gibi) içerdiğini gördün.

PyGame'in sunduğu ana olay türlerini üç kategoride (pencere, klavye, fare) sınıflandırdın. Her olay türünün hangi ek bilgileri taşıdığını öğrendin. Son olarak olay filtreleme teknikleri ile `set_blocked()`, `set_allowed()`, `event.clear()` fonksiyonlarını ve `pygame.USEREVENT` ile özel olay tanımlamayı, `event.post()` ile olay göndermeyi, `set_timer()` ile zamanlayıcı olaylar oluşturmayı öğrendin.

13.4.2 Anahtar Kavramlar

Tablo 13.2: Ünite 1'de öğrenilen anahtar kavramlar

Kavram	Açıklama
Olay Kuyruğu (Event Queue)	Kullanıcı ve sistem olaylarının FIFO sırasıyla biriktiği veri yapısı
Event Nesnesi	Her olayı temsil eden Python nesnesi; <code>type</code> zorunlu özelliği taşır
Olay Türü (Event Type)	Olayın ne tür bir eylemden kaynaklandığını belirten tam sayı sabiti
<code>pygame.QUIT</code>	Pencere kapatma isteğini bildiren olay
<code>pygame.KEYDOWN</code> / <code>pygame.KEYUP</code>	Tuşa basma ve bırakma olayları
<code>pygame.MOUSEBUTTONDOWN</code>	Fare düğmesine basma olayı; <code>pos</code> ve <code>button</code> taşır
<code>pygame.MOUSEMOTION</code>	Fare hareketi olayı; her piksel hareketinde tetiklenir
<code>pygame.USEREVENT</code>	Kullanıcı tanımlı özel olaylar için başlangıç sabiti
Olay Filtreleme	<code>set_blocked()</code> ve <code>set_allowed()</code> ile kuyruğu yönetme
Zamanlayıcı Olay	<code>set_timer()</code> ile belirli aralıklarla olay gönderme

13.4.3 Önemli Komutlar

Tablo 13.3: Ünite 1’de kullanılan PyGame komutları

Komut	İşlev
<code>pygame.event.get()</code>	Kuyruktaki tüm olayları döndürür ve kuyruğu boşaltır
<code>pygame.event.get(type)</code> <code>pygame.event.peek(type)</code>	Sadece belirtilen türdeki olayları döndürür Belirtilen türde olay olup olmadığını kontrol eder
<code>pygame.event.poll()</code> <code>pygame.event.clear()</code> <code>pygame.event.set_blocked(type)</code>	Kuyruktan tek bir olay alır Kuyruktaki tüm olayları siler Belirtilen olay türünü kuyruğa eklenmekten engeller
<code>pygame.event.set_allowed(type)</code> <code>pygame.event.Event(type, **attr)</code> <code>pygame.event.post(event)</code> <code>pygame.time.set_timer(type, ms)</code>	Engellenmiş olay türüne tekrar izin verir Yeni bir olay nesnesi oluşturur Oluşturulan olayı kuyruğa ekler Belirtilen sürede olayı tekrarlı gönderir

13.4.4 Sıradaki Adım

Olay sisteminin genel yapısını artık biliyorsun. Bir sonraki ünite (Ünite 2) **Klavye Girdilerine** odaklanacağız. `pygame.KEYDOWN` ve `pygame.KEYUP` olaylarını derinlemesine inceleyecek, `pygame.key.get_pressed()` fonksiyonuyla sürekli tuş kontrolü yapmayı öğrenecek ve ilk kez ekrandaki bir nesneyi klavyeyle hareket ettireceğiz. Olay sistemi bilgin, bu pratik uygulamalarda sağlam bir temel oluşturacak.

💡 İpucu: Pratik Yap

Olay sistemini gerçekten öğrenmenin en iyi yolu deney yapmaktır. Bu ünitedeki kod örneklerini çalıştır, değiştir ve farklı senaryolar dene. Örneğin zamanlayıcı süresini değiştir, farklı olay türlerini engelle veya yeni özel olaylar tanımla. Her deneme, olay sistemini daha iyi kavramanı sağlayacak.

“Klavye, oyuncunun oyun dünyasına uzanan parmaklarıdır. Her tuş, bir eylem kapısı açar.”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.KEYDOWN` ve `pygame.KEYUP` olaylarıyla tuş basma ve bırakmayı yakalama
- `pygame.key` özelliği ve PyGame tuş sabitleri (`K_UP`, `K_SPACE` vb.)
- `pygame.key.get_pressed()` ile sürekli tuş durumu sorgulama
- Ayrık girdi (discrete) ile sürekli girdi (continuous) arasındaki fark
- Ok tuşlarıyla ekrandaki bir nesneyi hareket ettirme

14.1 KEYDOWN ve KEYUP Olayları

Bir oyun düşün: karakterin düşmanın üzerine zıplamak için boşluk tuşuna basması gerekiyor. Tam o anı yakalamak, yani “tuşa basma eylemi”ni algılamak nasıl mümkün oluyor? Önceki ünite de PyGame’in olay kuyruğunu nasıl çalıştığını öğrendin (Konu 11.2). Şimdi bu kuyruktaki en temel girdilerden ikisini tanıyalım: `pygame.KEYDOWN` ve `pygame.KEYUP`.

■ Tanım: KEYDOWN ve KEYUP

`pygame.KEYDOWN` olayı, klavyedeki herhangi bir tuşa **basıldığı anda** tetiklenir. `pygame.KEYUP` ise o tuş **bırakıldığında** tetiklenir. Her iki olay da birer `pygame.event.Event` nesnesidir ve hangi tuşun etkileşime girdiğini gösteren `key`, `mod` ve `unicode` özelliklerini taşır.

Bunu bir kapı ziline benzetebilirsin. Zile bastığın anda “ding” sesi duyulur — bu `pygame.KEYDOWN`. Parmağını zilden çektiğinde ses durur — bu da

`pygame.KEYUP`. Her iki an da oyun için önemli bilgi taşır. Peki bu olaylardan hangi bilgileri çıkarabiliriz?

14.1.1 olay.key Özelliği ve Tuş Sabitleri

Bir `pygame.KEYDOWN` veya `pygame.KEYUP` olayı oluştuğunda, bu olay nesnesinin `key` özelliği basılan tuşun sayısal kodunu içerir. Ancak sayısal kodları ezberlemek yerine PyGame'in sunduğu sabitleri kullanmak çok daha pratiktir. Aşağıdaki tabloda en sık kullanılan tuş sabitlerini görebilirsin.

Tablo 14.1: PyGame'de sık kullanılan tuş sabitleri

Tuş	PyGame Sabiti	Açıklama
Yukarı ok	K_UP	Yön tuşu: yukarı
Aşağı ok	K_DOWN	Yön tuşu: aşağı
Sol ok	K_LEFT	Yön tuşu: sol
Sağ ok	K_RIGHT	Yön tuşu: sağ
Boşluk	K_SPACE	Boşluk tuşu
Enter	K_RETURN	Enter / Return
Escape	K_ESCAPE	Escape tuşu
A-Z	K_a ... K_z	Harf tuşları (küçük)
0-9	K_0 ... K_9	Sayı tuşları

Tablo 14.1'de gördüğün gibi her fiziksel tuşun PyGame'de karşılık gelen bir sabiti var. Bu sabitleri olay işleme kodunda şöyle kullanırsın:

Python – Tuş Kontrolü Örneği

```

1 for olay in pygame.event.get():
2     if olay.type == pygame.KEYDOWN:
3         if olay.key == pygame.K_SPACE:
4             print("Boşluk tusuna basıldı!")
5         elif olay.key == pygame.K_ESCAPE:
6             calistir = False

```

Dikkat edersen burada şu mantık işliyor: önce olayın türünün `pygame.KEYDOWN` olup olmadığını kontrol ediyoruz, sonra hangi tuş olduğunu `olay.key` ile sorguluyoruz. Bu iki aşamalı kontrol şart: çünkü `pygame.QUIT` gibi başka olay türlerinin `key` özelliği yoktur ve doğrudan `olay.key` sorgulamak hata verir.

14.1.2 Modifier Tuşlar: olay.mod

Bazen sadece bir tuşa basmak yetmez; Shift, Ctrl veya Alt gibi *modifier* (değiştirici) tuşlarla birlikte kullanmak isteyebilirsin. Örneğin Shift+A ile büyük harf, Ctrl+S ile kaydetme kısayolu oluşturabilirsin. PyGame'de `pygame.KEYDOWN` olayının `mod` özelliği o anda hangi modifier tuşların basılı olduğunu gösterir.

En sık kullanılan modifier sabitleri `KMOD_SHIFT`, `KMOD_CTRL` ve `KMOD_ALT` olarak sıralanabilir. Bu değerleri bit düzeyinde `&` (AND) operatörüyle kontrol edersin:

Python – Modifier Kontrolü

```
1 if olay.type == pygame.KEYDOWN:
2     if olay.key == pygame.K_s:
3         if olay.mod & pygame.KMOD_CTRL:
4             print("Ctrl+S: Oyun kaydediliyor!")
5         else:
6             print("S tusuna basildi.")
```

● Bilgi: Bit Düzeyinde AND

`olay.mod` değeri birden fazla modifier'ın aynı anda basılı olabileceği için bir bit maskesidir. `olay.mod & pygame.KMOD_CTRL` ifadesi, Ctrl tuşunun basılı olup olmadığını kontrol eder. Eğer sonuç sıfır değilse (truthy) Ctrl basılı demektir.

14.1.3 olay.unicode ve Tuş Adı

`pygame.KEYDOWN` olayının bir diğer kullanışlı özelliği `unicode`'dir. Bu özellik, basılan tuşun ürettiği karakteri metin olarak verir. Örneğin Shift basılıyken "a" tuşuna basarsan `olay.unicode` değeri "A" olur; Shift olmadan ise "a" olur. Ok tuşları veya Escape gibi karakter üretmeyen tuşlarda bu değer boş bir metin ("") döner.

Ayrıca `pygame.key.name()` fonksiyonu herhangi bir tuş kodunu okunabilir bir isme dönüştürür:

Python – Tuş Adını Alma

```
1 if olay.type == pygame.KEYDOWN:
2     tus_adi = pygame.key.name(olay.key)
3     print(f"Basildi: {tus_adi} Karakter:
      ↳ '{olay.unicode}'")
```

Bu fonksiyon hata ayıklama (debug) sırasında çok işine yarar. Hangi tuşa basıldığını konsola yazdırarak olay akışını takip edebilirsin.

14.1.4 KEYUP: Tuşu Bırakma Olayı

`pygame.KEYUP` olayı `pygame.KEYDOWN` ile aynı özelliklere sahiptir: `key`, `mod` ve `unicode`. Fark sadece tetiklenme anıdır — tuş bırakıldığında oluşur.

Peki neden tuşu bırakma anıyla ilgilenelim? Çünkü bazı oyun mekaniklerinde "tuşa ne kadar süre basılı tutuldu" bilgisi önemlidir. Örneğin bir platform oyununda zıplama tuşunu kısa basarsan küçük zıplamak, uzun basarsan yüksek zıplamak isteyebilirsin. Bu mekaniği `pygame.KEYDOWN` ile başlatıp `pygame.KEYUP` ile sonlandırabilirsin.

Python – KEYDOWN ve KEYUP Birlikte

```

1 for olay in pygame.event.get():
2     if olay.type == pygame.KEYDOWN:
3         if olay.key == pygame.K_SPACE:
4             print("Bosluk basildi -> Ziplama basladi!")
5     elif olay.type == pygame.KEYUP:
6         if olay.key == pygame.K_SPACE:
7             print("Bosluk birakildi -> Ziplama bitti!")

```

14.1.5 Ayırık Girdi Kavramı

pygame.KEYDOWN ve pygame.KEYUP olaylarının önemli bir özelliği vardır: her tuş basışında **yalnızca bir kez** tetiklenirler. Tuşu basılı tutsan bile (işletim sisteminin tuş tekrarı haricinde) pygame.KEYDOWN sadece ilk bastığın anda gönderilir. Bu davranışa *ayırık girdi* (discrete input) denir.

Ayrık girdi “bir kez yap” türündeki eylemler için mükemmeldir: menüde seçenek değiştirme, ateş etme, zıplama başlatma gibi. Peki tuşu basılı tuttuğun sürece sürekli hareket etmek istersen ne yaparsın? Bir sonraki konuda tam olarak bunu öğreneceğiz.

⚠ Dikkat: Tuş Tekrarı Yanılgısı

İşletim sistemi, basılı tutulan tuşlar için belirli bir gecikmeden sonra otomatik tekrar gönderir (metin editöründe bir harfi basılı tutunca tekrarlanması gibi). Ancak bu davranış oyunlarda güvenilir değildir ve her platformda farklı çalışır. Karakter hareketi için pygame.KEYDOWN tekrarına güvenme; bunun yerine pygame.key.get_pressed() kullan. Bunu bir sonraki konuda öğreneceksin.

Şimdi tüm bu kavramları bir araya getiren bir örneğe bakalım. Aşağıdaki program bir pencere açar, her tuş basışını ve bırakışını konsola yazdırarak olay sistemini görsel olarak takip etmeni sağlar. Önce modifier bilgisini okunabilir metne dönüştüren bir yardımcı fonksiyon yazalım:

Python – Modifier Metni Yardımcı Fonksiyonu

```

1 def modifier_metni(mod):
2     """Modifier bit maskesini okunabilir metne
3     ↪ donusturur."""
4     parcalar = []
5     if mod & pygame.KMOD_SHIFT:
6         parcalar.append("Shift")
7     if mod & pygame.KMOD_CTRL:
8         parcalar.append("Ctrl")
9     if mod & pygame.KMOD_ALT:
10        parcalar.append("Alt")
11    if not parcalar:
12        return "Yok"

```



```
12 | return "+".join(parcalar)
```

Bu fonksiyon, `olay.mod` bit maskesini insan tarafından okunabilir bir metin haline getirir. Ana döngüde bu fonksiyonu kullanarak her tuş olayını detaylıca yazdırırız:

Python – Tuş Yakalama – Olay Döngüsü

```
1  for olay in pygame.event.get():
2      if olay.type == pygame.QUIT:
3          calistir = False
4
5      elif olay.type == pygame.KEYDOWN:
6          tus_adi = pygame.key.name(olay.key)
7          mod_str = modifier_metni(olay.mod)
8          unicode_str = olay.unicode if olay.unicode else
9              ↳ "(yok)"
10         print(f"[KEYDOWN] Tus: {tus_adi} "
11             f"Karakter: {unicode_str} Modifier:
12             ↳ {mod_str}")
13         # Pencere başlığını guncelle
14         pygame.display.set_caption(f"Son tus: {tus_adi}")
15         if olay.key == pygame.K_ESCAPE:
16             calistir = False
17
18         elif olay.type == pygame.KEYUP:
19             tus_adi = pygame.key.name(olay.key)
20             print(f"[KEYUP] Tus: {tus_adi} (birakildi)")
```

Dikkat edilmesi gereken noktalar:

1. Her `pygame.KEYDOWN` olayında tuşun adı, karakter karşılığı ve modifier bilgisi konsola yazdırılır.
2. `pygame.KEYUP` ile tuşun bırakılma anı da izlenir.
3. `K_ESCAPE` tuşu programı düzgün şekilde kapatmak için kullanılır.
4. Pencere başlığına son basılan tuşun adı yazılır; böylece konsola bakmadan da hangi tuşun algılandığını görebilirsin.

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite2/01\_tus\_yaka` dosyasında bulabilirsin.

💡 İpucu: Kendi Dene

Programı çalıştırıp farklı tuş kombinasyonlarını dene: Shift+A, Ctrl+S, ok tuşları, sayı tuşları. Konsoldaki çıktıyı inceleyerek `key`, `mod` ve `unicode` değerlerinin nasıl değiştiğini gözlemle.

14.2 Sürekli Girdi: `key.get_pressed()`

Önceki konuda `pygame.KEYDOWN` ile tuşa basma anını yakalamayı öğrendin. Bu yöntem “bir kez yap” türündeki eylemler için mükemmeldi: menüde seçim yapma, ateş etme, zıplama başlatma gibi. Peki bir karakteri ok tuşlarıyla ekranda sürüklemek istersen ne olacak? Tuşu basılı tuttuğun sürece karakter hareket etmeli — ama `pygame.KEYDOWN` sadece ilk bastığın anda tetikleniyor. İşte bu ihtiyaç için PyGame’in farklı bir aracı var: `pygame.key.get_pressed()`.

■ Tanım: `key.get_pressed()`

`pygame.key.get_pressed()` fonksiyonu, çağrıldığı anda klavyedeki **tüm tuşların durumunu** bir boolean dizisi olarak döndürür. Her tuş sabiti bu diziye indeks olarak kullanılır: `True` tuşun o anda basılı olduğunu, `False` basılı olmadığını gösterir.

Bunu şöyle düşünebilirsin: `pygame.KEYDOWN` bir “zil çaldı!” bildirimi, `pygame.key.get_pressed()` ise “şu an kapı açık mı?” sorusudur. Birincisi olaya tepki verir, ikincisi durumu sorgular.

14.2.1 Kullanımı

`pygame.key.get_pressed()` fonksiyonu genellikle olay döngüsünün **dışında**, güncelleme aşamasında çağrılır. Çünkü bu fonksiyon olay kuyruğuyla ilgilenmez; doğrudan klavyenin anlık durumunu okur.

Python – `get_pressed()` Temel Kullanım

```
1 # Güncelleme aşamasında (olay döngüsünden sonra)
2 tuslar = pygame.key.get_pressed()
3
4 if tuslar[pygame.K_UP]:
5     print("Yukarı ok sürekli basılı!")
6 if tuslar[pygame.K_LEFT]:
7     print("Sol ok sürekli basılı!")
```

Dikkat edersen burada `elif` değil `if` kullandık. Bunun nedeni birden fazla tuşun aynı anda basılı olabilmesidir. `elif` kullanırsan aynı anda yukarı ve sola basarak çapraz hareket edemezsin — bu da oyuncu deneyimini ciddi şekilde bozar.

▲ Dikkat: `event.get()` Çağrısını Unutma

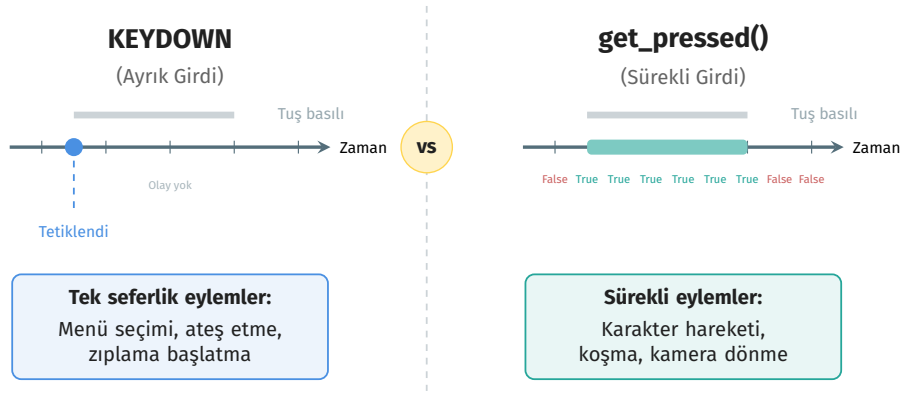
`pygame.key.get_pressed()` fonksiyonunun doğru çalışabilmesi için her döngü turunda `pygame.event.get()` veya `pygame.event.pump()` çağrılmalıdır. Bu çağrılar PyGame’in dahili olay kuyruğunu güncelleyerek tuş durumlarının güncel kalmasını sağlar. Aksi halde tuş durumları donuk kalır.

14.2.2 KEYDOWN ile `get_pressed()` Karşılaştırması

Bu iki yöntem birbirinin alternatifi değil, **tamamlayıcıdır**. Her birinin uygun olduğu senaryolar vardır.

`pygame.KEYDOWN` olay tabanlıdır: tuşa basıldığı anda bir kez tetiklenir. Menüde “bir sonraki seçenek” gibi tekil eylemler için idealdir. `pygame.key.get_pressed()` ise durum tabanlıdır: tuşun o anda basılı olup olmadığını sürekli sorgular. Karakter hareketi gibi her kare tekrarlanan eylemler için idealdir.

Bu farkı görsel olarak aşağıdaki şekilde inceleyebilirsiniz.



Şekil 14.1: Ayrık girdi (KEYDOWN) ile sürekli girdi (`get_pressed`) karşılaştırması

Şekil 14.1’de gördüğün gibi sol tarafta `pygame.KEYDOWN` yalnızca tuşa ilk basıldığı anda (nokta ile gösterilen) bir sinyal üretir. Sağ tarafta ise `pygame.key.get_pressed()` tuş basılı olduğu süre boyunca (kalın çubukla gösterilen) sürekli olarak `True` döndürür.

Tablo 14.2: KEYDOWN ve `get_pressed()` kullanım alanları

Eylem	KEYDOWN	<code>get_pressed()</code>
Menü seçimi	✓	
Ateş etme	✓	
Zıplama başlatma	✓	
Karakter hareketi		✓
Sürekli koşma		✓
Kamerayı döndürme		✓

Tablo 14.2’deki tabloya bakarak hangi işlem için hangi yöntemi seçeceğini kolayca belirleyebilirsin. Genel kural şudur: “bir kez yap” → `pygame.KEYDOWN`, “basılı tuttuğun sürece yap” → `pygame.key.get_pressed()`.

İpucu: Profesyonel İpucu

Çoğu profesyonel oyun her iki yöntemi de birlikte kullanır. Örneğin `pygame.KEYDOWN` ile zıplama **başlatılır**, `pygame.key.get_pressed()` ile yatay hareket **sürdürülür**. Bir sonraki konuda tam da bu yaklaşımı uygulayacağız.

14.3 Klavye ile Nesne Hareketi

Buraya kadar tuşa basma anını yakalamayı (`pygame.KEYDOWN`) ve tuşun basılı olup olmadığını sorgulamayı (`pygame.key.get_pressed()`) öğrendin. Şimdi bu bilgileri birleştirerek her oyunun temel taşını inşa edeceğiz: ekranda bir nesneyi klavyeyle hareket ettirmek. Bu basit görünen işlem aslında oyun geliştirmenin en önemli yapı taşlarından biridir — karakter kontrolünün temeli burada başlar.

14.3.1 Hız Sabiti ve Delta Time

Bir nesneyi hareket ettirmek için her karede konumunu değiştirmemiz gerekiyor. Peki ne kadar değiştireceğiz? Sabit bir piksel değeri (örneğin her karede 5 piksel) kullanmak cazip görünebilir, ancak bu yaklaşımda ciddi bir sorun var: oyun farklı bilgisayarlarda farklı hızlarda çalışır.

60 FPS’de çalışan bir bilgisayarda nesne saniyede $5 \times 60 = 300$ piksel hareket eder. Ama 120 FPS’de çalışan bir bilgisayarda aynı nesne saniyede $5 \times 120 = 600$ piksel hareket eder — iki kat hızlı! Bu sorunu çözmek için hızı “piksel/saniye” cinsinden tanımlayıp *delta time* ile çarpabiliriz.

Tanım: Delta Time

Delta time (*dt*), bir önceki kare ile şimdiki kare arasında geçen süredir (saniye cinsinden). Hızı bu değerle çarparak kare hızından bağımsız hareket elde ederiz. Formül: $\text{hareket} = \text{hız} \times dt$

PyGame’de *dt* değerini şöyle hesaplıyorsun:

Python – Delta Time Hesabı

```
1 saat = pygame.time.Clock()
2
3 while calistir:
4     # tick() hem FPS sinirlama yapar hem de
5     # gecen sureyi milisaniye cinsinden dondurur
6     dt = saat.tick(60) / 1000.0 # saniyeye cevir
```

Burada `saat.tick(60)` fonksiyonu iki iş yapar: oyun hızını 60 FPS ile sınırlar ve son çağrıdan bu yana geçen süreyi milisaniye olarak döndürür. 1000’e bölerek saniyeye çeviririz.

14.3.2 Dört Yöne Hareket

Şimdi `dt` ve `pygame.key.get_pressed()` bilgisini birleştirerek dört yöne pürüzsüz hareket oluşturalım. Mantık basittir: her karede hangi tuşlar basılıysa o yöne $HIZ \times dt$ kadar kaydır.

Python – Dört Yön Hareket Mantığı

```

1  HIZ = 300 # piksel/saniye
2
3  tuslar = pygame.key.get_pressed()
4
5  if tuslar[pygame.K_UP]:
6      y -= HIZ * dt
7  if tuslar[pygame.K_DOWN]:
8      y += HIZ * dt
9  if tuslar[pygame.K_LEFT]:
10     x -= HIZ * dt
11 if tuslar[pygame.K_RIGHT]:
12     x += HIZ * dt

```

Dikkat: yukarı yönde `y` değeri **azalır** çünkü PyGame'in koordinat sisteminde $y = 0$ ekranın üst kenarıdır (Konu 10.1). Aşağı gidildikçe `y` artar.

● Bilgi: Çapraz Hareket

Kodda `if` kullandığımıza dikkat et (`elif` değil). Bu sayede iki tuş aynı anda basılıyken çapraz hareket mümkün olur. Örneğin yukarı ve sağ ok tuşlarına aynı anda basarsan nesne sağ üst köşeye doğru kayar.

14.3.3 Ekran Sınır Kontrolü

Hareket kodunu ekledikten sonra bir sorunla karşılaşsın: nesne ekran dışına çıkabilir! Bunu engellemek için nesnenin konumunu ekran sınırları içinde tutmalıyız. Python'ın `max()` ve `min()` fonksiyonları bu iş için biçilmiş kaftandır:

Python – Sınır Kontrolü

```

1  # Nesnenin ekran disindan cikmasini engelle
2  x = max(0, min(x, GENISLIK - boyut))
3  y = max(0, min(y, YUKSEKLIK - boyut))

```

Bu tek satırlık ifade şunu yapar: `x` değeri 0'dan küçük olamaz (sol kenar) ve `GENISLIK - boyut`'tan büyük olamaz (sağ kenar). Aynı mantık `y` için de geçerlidir. `boyut` değişkeni nesnenin genişliğini/yüksekliğini temsil eder; çıkarılmazsa nesnenin sağ/alt kenarı ekran dışında kalır.

14.3.4 Tam Örnek: Sürekli Hareket

Aşağıdaki program tüm bu kavramları bir araya getirir. Ekranda mavi bir kare görüntülenir ve ok tuşlarıyla dört yöne hareket ettirilebilir. Hareket `dt` tabanlıdır, yani her bilgisayarda aynı hızda çalışır. Kare ekran dışına çıkamaz.

Programın ana döngüsü yukarıda öğrendiğin üç kavramı birleştirir:

Python – Sürekli Hareket – Ana Döngü

```

1  while calistir:
2      dt = saat.tick(FPS) / 1000.0
3
4      for olay in pygame.event.get():
5          if olay.type == pygame.QUIT:
6              calistir = False
7          elif olay.type == pygame.KEYDOWN:
8              if olay.key == pygame.K_ESCAPE:
9                  calistir = False
10
11         # Surekli tus durumunu sorgula
12         tuslar = pygame.key.get_pressed()
13         if tuslar[pygame.K_UP]: y -= HIZ * dt
14         if tuslar[pygame.K_DOWN]: y += HIZ * dt
15         if tuslar[pygame.K_LEFT]: x -= HIZ * dt
16         if tuslar[pygame.K_RIGHT]: x += HIZ * dt
17
18         # Ekrani sinir kontrolu (clamp)
19         x = max(0.0, min(x, float(GENISLIK - BOYUT)))
20         y = max(0.0, min(y, float(YUKSEKLIK - BOYUT)))
21
22         # Ciz
23         ekran.fill(ARKA_PLAN)
24         pygame.draw.rect(ekran, RENK, (int(x), int(y), BOYUT,
25                                     BOYUT))
26         pygame.display.flip()

```

Kodda dikkat edilmesi gereken noktalar:

1. `HIZ` sabiti piksel/saniye cinsinden tanımlanır (300 piksel/saniye).
2. `dt` değeri `saat.tick(60) / 1000.0` ile hesaplanır.
3. Dört `if` bloğu bağımsız çalışarak çapraz harekete izin verir.
4. `max/min` ile sınır kontrolü yapılır.
5. `pygame.KEYDOWN` ile `K_ESCAPE` kontrolü de eklenerek programı kapatma kolayca sağlanır — iki yöntem bir arada.

İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite2/02\_surekli\_h` dosyasında bulabilirsin. Çalıştırıp ok tuşlarıyla dene.

İpucu: Değişiklik Önerileri

Bu programı küçük değişikliklerle geliştirebilirsin: **HIZ** değerini 500'e çıkararak hızlı hareket dene, karenin rengini her yön değiştiğinde farklı yap veya WASD tuşlarını da ekleyerek alternatif kontrol şeması oluştur.

14.4 Sonuç

Bu ünite de klavye girdilerinin oyun geliştirmede nasıl kullanıldığını öğrendin. `pygame.KEYDOWN` ve `pygame.KEYUP` olaylarıyla tuşa basma ve bırakma anlarını yakaladın, `pygame.key.get_pressed()` ile tuşların anlık durumunu sorgulamayı keşfettin ve tüm bu bilgileri birleştirerek ekranda bir nesneyi ok tuşlarıyla hareket ettirdin. Ayrık girdi ile sürekli girdi arasındaki farkı anlamak, ilerideki ünitelerde oluşturacağın oyun mekaniklerinin temelini oluşturuyor.

Anahtar Kavramlar

Tablo 14.3: Ünite 2 — Anahtar kavramlar

Kavram	Açıklama
<code>pygame.KEYDOWN</code>	Tuşa basıldığı anda tetiklenen olay
<code>pygame.KEYUP</code>	Tuş bırakıldığında tetiklenen olay
<code>pygame.key</code>	Basılan tuşun sayısal kodu
<code>pygame.mod</code>	Modifier tuşların bit maskesi
<code>pygame.unicode</code>	Basılan tuşun karakter karşılığı
Ayrık girdi	Her basışta bir kez tetiklenen girdi
Sürekli girdi	Basılı olduğu sürece aktif kalan girdi
Delta time (dt)	Kareler arası geçen süre (saniye)

Önemli Komutlar

Tablo 14.4: Ünite 2 — Önemli PyGame komutları

Komut	İşlev
<code>pygame.key == pygame.K_UP</code>	Tuş sabiti kontrolü
<code>pygame.mod & pygame.KMOD_CTRL</code>	Modifier kontrolü
<code>pygame.key.name(pygame.key)</code>	Tuş adını metin olarak alma
<code>pygame.key.get_pressed()</code>	Tüm tuş durumlarını sorgulama
<code>pygame.time.tick(60) / 1000.0</code>	Delta time hesaplama

Sıradaki Adım

Klavye girdilerini öğrendiğine göre sıra fare girdilerinde! Bir sonraki ünite de `pygame.MOUSEBUTTONDOWN`, `pygame.mouse.get_pos()` ve fare hareketini

takip etmeyi öğreneceksin. Fare ve klavye girdilerini birleştirerek daha zengin kontrol şemaları oluşturabileceksin.

Pratik Yap

💡 İpucu: Öğrendiğini Pekiştir

Bu ünitenin kodlarını kendi başına dene ve şu değişiklikleri yap: birincisi, `K_UP` yerine `K_w`, `K_a`, `K_s`, `K_d` tuşlarını ekleyerek WASD kontrol şeması oluştur. İkincisi, Shift tuşuna basılıyken hızı 2 katına çıkaran bir “koşma” mekaniği ekle. Üçüncüsü, karenin rengi hareket yönüne göre değişsin (yukarı: mavi, aşağı: kırmızı, sol: yeşil, sağ: turuncu).

“Fare, oyuncunun ekrana dokunan eli gibidir. Nereye tıklarsa, orada bir şey olur.”

— ANONİM

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.MOUSEBUTTONDOWN` ve `pygame.MOUSEBUTTONUP` olaylarıyla tıklama yakalama
- `pygame.mouse.get_pos()` ile fare konumunu sorgulama
- `pygame.mouse.button` özelliği ve fare düğme numaraları
- `pygame.MOUSEMOTION` olayıyla fare hareketini izleme
- Fare takibi ve özel imleç oluşturma

15.1 Fare Olayları

Klavye girdilerini Konu 14.1’de detaylıca inceledik. Ancak bir oyunda yalnızca tuşlar yeterli değildir. Düşün: bir strateji oyununda birlikleri seçmek, menülerde butonlara tıklamak, haritayı kaydırmak — bütün bunlar fare ile yapılır. Fare, oyuncunun ekrandaki dünyaya “parmak ucuyla dokunma” aracıdır. Bu konuda fare tıklama olaylarını, fare konumunu sorgulamayı ve düğme numaralarını öğreneceksin.

■ Tanım: Fare Olayı (Mouse Event)

PyGame’de fare ile ilgili üç temel olay türü vardır: `pygame.MOUSEBUTTONDOWN` (düğmeye basıldığında), `pygame.MOUSEBUTTONUP` (düğme bırakıldığında) ve `pygame.MOUSEMOTION` (fare hareket ettiğinde). Her olay, konum ve düğme bilgisi taşır.

15.1.1 pygame.MOUSEBUTTONDOWN ve pygame.MOUSEBUTTONUP

Bir oyunda kullanıcı farenin herhangi bir düğmesine bastığında PyGame bir `pygame.MOUSEBUTTONDOWN` olayı oluşturur. Düğme bırakıldığında ise `pygame.MOUSEBUTTONUP` olayı tetiklenir. Bu yapı, klavyedeki `pygame.KEYDOWN`/`pygame.KEYUP` çiftine oldukça benzer — Konu 14.1’de öğrendiğin mantık burada da geçerli.

Her iki olayın da taşıdığı iki önemli özellik var. Birincisi `olay.pos`: tıklamanın gerçekleştiği piksel konumunu `(x, y)` tuple olarak verir. İkincisi `olay.button`: hangi fare düğmesinin kullanıldığını bir tamsayı ile belirtir. Bu iki bilgiyi birleştirerek “kullanıcı nereye, hangi düğmeyle tıkladı” sorusuna yanıt verebilirsin.

Temel kullanım şöyle görünür:

Python – MOUSEBUTTONDOWN Olayını Yakalama

```
1 for olay in pygame.event.get():
2     if olay.type == pygame.QUIT:
3         calistir = False
4     elif olay.type == pygame.MOUSEBUTTONDOWN:
5         print(f"Tıklama konumu: {olay.pos}")
6         print(f"Dugme numarasi: {olay.button}")
```

Bu kod parçasında `pygame.MOUSEBUTTONDOWN` olayı yakalandığında tıklama konumu ve düğme numarası konsola yazdırılır. `olay.pos` değeri her zaman pencere koordinat sistemine göredir: sol üst köşe `(0, 0)`, sağ alt köşe ise `(pencere_genislik-1, pencere_yukseklik-1)` noktasıdır.

15.1.2 Fare Düğme Numaraları

PyGame’de fare düğmeleri tamsayı değerleriyle temsil edilir. Hangi düğmenin ne anlama geldiğini bilmek, özellikle farklı düğmelere farklı işlevler atamak istediğinde çok önemlidir.

Tablo 15.1: PyGame fare düğme numaraları

Numara	Düğme	Yaygın Kullanım
1	Sol düğme	Seçme, ateş etme, etkileşim
2	Orta düğme (tekerlek bas)	Kamerayı döndürme, özel eylem
3	Sağ düğme	Bağlam menüsü, alternatif eylem
4	Scroll yukarı	Yakınlaştırma, envanter kaydırma
5	Scroll aşağı	Uzaklaştırma, envanter kaydırma

Dikkat edersen scroll tekerleği de bir “düğme” olarak algılanır. Yukarı kaydırma 4, aşağı kaydırma 5 numarasıyla gelir. Bu bilgi, örneğin bir strateji oyununda yakınlaştırma-uzaklaştırma (zoom) özelliği eklerken işine yarayacak.

15.1.3 pygame.mouse.get_pos() ile Anlık Konum Sorgulama

pygame.MOUSEBUTTONDOWN olayı yalnızca tıklama anında tetiklenir. Peki ya her karede farenin o anki konumunu öğrenmek istersen? İşte pygame.mouse.get_pos() fonksiyonu tam olarak bunu yapar. Herhangi bir olay beklemeden, istediğin anda farenin pencere üzerindeki (x, y) konumunu döndürür.

Python – Anlık Fare Konumu

```
1 # Oyun dongusu icinde, her karede:
2 fare_x, fare_y = pygame.mouse.get_pos()
3 print(f"Fare konumu: ({fare_x}, {fare_y})")
```

Bu fonksiyon ile pygame.MOUSEBUTTONDOWN olayındaki olay.pos arasındaki fark şudur: olay.pos sadece tıklama anındaki konumu verir, mouse.get_pos() ise herhangi bir anda — tıklama olmasa bile — farenin güncel konumunu döndürür. Bir nesnenin fareyi sürekli takip etmesini istediğinde mouse.get_pos() daha uygun bir seçimdir.

İpucu: Olay mı, Sorgulama mı?

Tek sefer tetiklenmesi gereken eylemler (silah ateşleme, menü seçimi) için pygame.MOUSEBUTTONDOWN olayını kullan. Sürekli güncellenen bilgiler (imleç takibi, hedefleme) için ise pygame.mouse.get_pos() fonksiyonunu tercih et. Bu ayrım, klavyedeki pygame.KEYDOWN ile pygame.key.get_pressed() arasındaki farkla aynıdır.

15.1.4 pygame.mouse.get_pressed() ile Düğme Durumu

Klavyedeki pygame.key.get_pressed() fonksiyonunun fare karşılığı pygame.mouse.get_pressed() fonksiyonudur. Bu fonksiyon, farenin sol, orta ve sağ düğmelerinin o anki basılı olma durumunu üç elemanlı bir tuple olarak döndürür.

Python – Fare Düğme Durumu Sorgulama

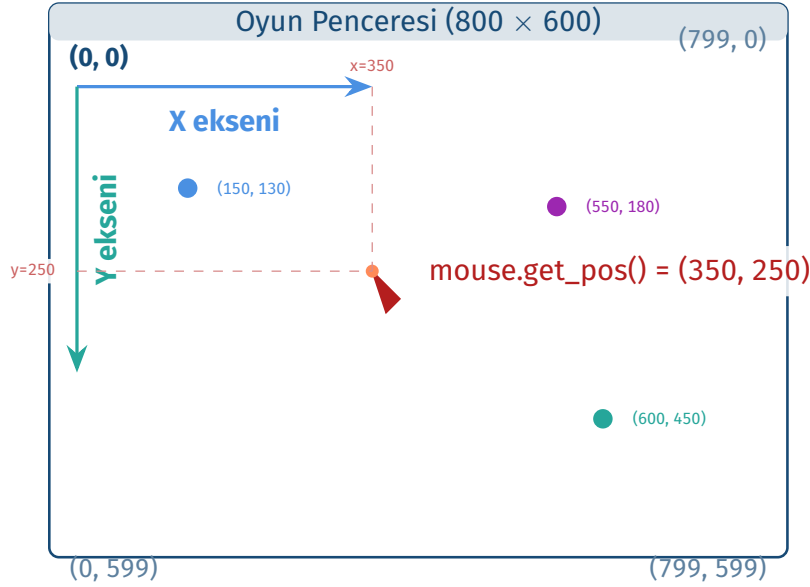
```
1 sol, orta, sag = pygame.mouse.get_pressed()
2 if sol:
3     fare_x, fare_y = pygame.mouse.get_pos()
4     print(f"Sol dugme basili - konum: ({fare_x}, {fare_y})")
```

Bu yöntem özellikle sürükleme işlemlerinde kullanışlıdır: kullanıcı sol düğmeyi basılı tutarken farenin konumunu her karede takip edebilirsin.

15.1.5 Fare Koordinat Sistemi

PyGame'in koordinat sistemi şimdiye kadar çizim işlemlerinde de kullandığımız standart ekran koordinat sistemidir. Sol üst köşe (0, 0) noktasıdır. X eksenini sağa doğru, Y eksenini ise aşağıya doğru artar. Bu, matematik derslerinden alıştığın

koordinat sisteminden farklıdır; ekran koordinatlarında “yukarı” demek daha küçük Y değeri demektir.



Şekil 15.1: PyGame fare koordinat sistemi: sol üst köşe (0,0), X sağa, Y aşağıya artar

Şekil 15.1’da görüldüğü gibi, fare pencere üzerinde herhangi bir konumdayken `pygame.mouse.get_pos()` fonksiyonu o noktanın (x, y) değerini döndürür. Örneğin 800 piksel genişliğinde ve 600 piksel yüksekliğinde bir pencerede farenin ekranın tam ortasında olması $(400, 300)$ değerini verir.

⚠ Dikkat: Pencere Dışında Fare Konumu

`pygame.mouse.get_pos()` farenin **pencere içindeki** konumunu döndürür. Fare pencere dışına çıktığında konum değeri güncellenmez ve en son pencere içindeki konumu korur. Oyun mantığında buna dikkat et — özellikle fare ile hedefleme yapan oyunlarda bu durum beklenmeyen davranışlara yol açabilir.

15.1.6 Tıklama ile Daire Oluşturma Örneği

Şimdi öğrendiklerini birleştiren bir program yazalım. Bu programda sol tıklama ile ekrana rastgele renkli daireler ekleyecek, sağ tıklama ile son eklenen daireyi sileceksin. Her bir tıklama hem `event.pos` ile konumu hem de `event.button` ile düğme numarasını kullanarak farklı işlemler gerçekleştirecek.

Programın olay işleme kısmı, düğme numarasına göre dallanır:

Python – Fare Tıklama ile Daire Oluşturma

```
1 # Daireleri saklayan liste
2 daireler = []
3
```

```

4  # ... oyun dongusu icinde ...
5  for olay in pygame.event.get():
6      if olay.type == pygame.MOUSEBUTTONDOWN:
7          if olay.button == 1:
8              # Sol tiklama: yeni daire ekle
9              yeni_daire = {
10                 "konum": olay.pos,
11                 "renk": rastgele_renk(),
12                 "yaricap": random.randint(10, 30),
13             }
14             daireler.append(yeni_daire)
15         elif olay.button == 3:
16             # Sag tiklama: son daireyi sil
17             if daireler:
18                 daireler.pop()

```

Daireleri çizmek için listeyi dolaşarak her elemanın bilgilerini kullanırız:

Python – Daireleri Çizme

```

1  for daire in daireler:
2      pygame.draw.circle(
3          ekran,
4          daire["renk"],
5          daire["konum"],
6          daire["yaricap"],
7      )

```

Bu programda dikkat edilmesi gereken birkaç önemli nokta var:

1. `daireler` listesi, her dairenin konumunu, rengini ve yarıçapını bir sözlük olarak saklar. Bu, Bölüm 2’de öğrendiğin liste ve sözlük yapılarının gerçek bir kullanımıdır.
2. `olay.button == 1` kontrolü sol tıklamayı, `olay.button == 3` ise sağ tıklamayı yakalar.
3. `random.randint()` ile rastgele renk üretmek oyunu daha renkli ve eğlenceli kılar.
4. `daireler.pop()` son eklenen elemanı listeden çıkarır — bu, listelerin “son giren ilk çıkar” (LIFO) davranışındır.

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite3/01\_fare\_tikl` dosyasında bulabilirsin.

● Bilgi: Önemli Not

Bu örnekte `daireler` listesi boşken `pop()` çağrılırsa hata oluşur. Bu yüzden `if daireler:` kontrolünü eklemeyi unutma. Listenin boş olup olmadığını kontrol etmek, savunmacı programlamanın temel ilkelerinden biridir.

15.2 Fare Hareketi ve Takip

Önceki konuda fare tıklama olaylarını öğrendin. Ancak fare yalnızca tıklamak için değil, hareket ettirmek için de kullanılır. Bir RTS oyununda birlikleri seçmek için fare kutusunu sürüklersin, bir boyama oyununda fareyi hareket ettirerek çizgi çekersin, bir FPS oyununda fareyle nişan alırsın. Tüm bu etkileşimler farenin **hareketini** takip etmeye dayanır.

15.2.1 pygame.MOUSEMOTION Olayı

PyGame, fare pencere üzerinde her hareket ettiğinde bir `pygame.MOUSEMOTION` olayı üretir. Bu olay üç önemli bilgi taşır:

Tablo 15.2: `pygame.MOUSEMOTION` olayının özellikleri

Özellik	Açıklama
<code>olay.pos</code>	Farenin yeni konumu (<code>x, y</code>) tuple olarak
<code>olay.rel</code>	Son konumdan fark (<code>dx, dy</code>) — fare ne kadar hareket etti
<code>olay.buttons</code>	Hangi düğmelerin basılı olduğu (<code>sol, orta, sag</code>) tuple

`olay.rel` özelliği özellikle ilginçtir. Bu değer, farenin son konumuna göre ne kadar hareket ettiğini verir. Örneğin fare sağa 5 piksel, aşağıya 3 piksel hareket ettiyse `olay.rel` değeri `(5, 3)` olur. Bu bilgi, farenin hızını veya yönünü hesaplamak istediğinde işe yarar.

Python – pygame.MOUSEMOTION Olayını Yakalama

```

1 for olay in pygame.event.get():
2     if olay.type == pygame.MOUSEMOTION:
3         print(f"Yeni konum: {olay.pos}")
4         print(f"Hareket farkı: {olay.rel}")
5         print(f"Basili dugmeler: {olay.buttons}")

```

`olay.buttons` tuple değerinin her elemanı boolean bir değerdir. Örneğin kullanıcı sol düğmeyi basılı tutarak fareyi sürüklüyorsa `olay.buttons` değeri `(True, False, False)` olur. Bu, basit bir sürükle-bırak (drag-and-drop) mekanizması kurmak için yeterli bilgidir.

15.2.2 Fare Takibi: Nesnenin Fareyi İzlemesi

Fare konumunu okumayı bildiğine göre, şimdi bir nesnenin fareyi takip etmesini sağlayabilirsin. En basit yöntem nesnenin konumunu doğrudan farenin konumuna eşitlemektir:

Python – Doğrudan Takip

```
1 nesne_x, nesne_y = pygame.mouse.get_pos()
```

Bu yöntem işe yarar ama nesne farenin tam üstüne “yapışır” — hiçbir geçiş efekti olmaz. Oyunlarda daha doğal görünen bir yaklaşım, nesnenin fareye **kademeli olarak yaklaşmasıdır**. Buna *lerp* (linear interpolation — doğrusal enterpolasyon) tekniği denir.

■ Tanım: Lerp (Doğrusal Enterpolasyon)

İki değer arasında yumuşak geçiş yapma tekniğidir. Nesnenin mevcut konumu ile hedef konum arasındaki farkın belirli bir oranı kadar ilerlemesini sağlar. $\text{yeni_konum} = \text{mevcut} + (\text{hedef} - \text{mevcut}) * \text{oran}$ formülü ile hesaplanır. Oran 0 ile 1 arasında bir değerdir: küçük değerler yavaş ve yumuşak takip, büyük değerler hızlı ve sert takip sağlar.

Bu teknikle nesne fareye yakinken yavaşlar, uzakken hızlanır — çünkü fark büyüdükçe adım da büyür. Sonuçta nesne fareye “yuvarlanarak” ulaşır ve görsel olarak çok daha hoş bir etki yaratır.

15.2.3 Fare İmlecini Gizleme ve Özel İmleç

Bazı oyunlarda varsayılan fare imlecini gizleyip yerine kendi çiziminizi koymak istersin. Örneğin bir nişancı oyununda artamlı nişangah, bir boyama oyununda fırça simgesi göstermek isteyebilirsin.

`pygame.mouse.set_visible()` fonksiyonu ile varsayılan imleç gizleyip gösterebilirsin:

Python – İmleci Gizleme

```
1 # Varsayılan fare imlecini gizle
2 pygame.mouse.set_visible(False)
3
4 # Tekrar göster
5 pygame.mouse.set_visible(True)
```

İmleci gizledikten sonra kendi “imleçini” çizmek için her karede farenin konumuna bir şekil çizmen yeterlidir. Örneğin farenin konumuna küçük bir artı işareti çizerek basit bir nişangah oluşturabilirsin:

Python – Özel İmleç Çizimi

```

1 fare_x, fare_y = pygame.mouse.get_pos()
2
3 # Yatay çizgi
4 pygame.draw.line(ekran, BEYAZ, (fare_x - 10, fare_y),
5                   (fare_x + 10, fare_y), 2)
6 # Dikey çizgi
7 pygame.draw.line(ekran, BEYAZ, (fare_x, fare_y - 10),
8                   (fare_x, fare_y + 10), 2)

```

Bu teknik Bölüm 3’te öğrendiğin çizim fonksiyonlarını fare konumuyla birleştirir. Ekran her karede temizlenip tekrar çizildiği için “imleç” farenin hareketini takip eder.

15.2.4 Sürükle-Bırak Temelleri

Sürükle-bırak mekaniği, fare tıklama ve hareket olaylarının birleşimidir. Temel mantık şöyledir: kullanıcı bir nesnenin üzerine tıklayıp (`pygame.MOUSEBUTTONDOWN`) düğmeyi basılı tutarak fareyi hareket ettirirse (`pygame.MOUSEMOTION` ile `buttons[0] == True`), nesne farenin hareketiyle birlikte taşınır. Düğme bırakıldığında (`pygame.MOUSEBUTTONUP`) nesne yeni konumunda kalır.

Python – Basit Sürükle-Bırak Mantığı

```

1 surukleniyor = False
2
3 for olay in pygame.event.get():
4     if olay.type == pygame.MOUSEBUTTONDOWN:
5         if olay.button == 1: # Sol tıklama
6             surukleniyor = True
7     elif olay.type == pygame.MOUSEBUTTONUP:
8         if olay.button == 1:
9             surukleniyor = False
10    elif olay.type == pygame.MOUSEMOTION:
11        if surukleniyor:
12            nesne_x, nesne_y = olay.pos

```

Dikkat edersen burada bir `surukleniyor` bayrak değişkeni kullanılıyor. Bu, `pygame.MOUSEBUTTONDOWN` ile başlayıp `pygame.MOUSEBUTTONUP` ile biten bir “durum” tanımlar. Bu durum aktifken `pygame.MOUSEMOTION` olayları nesneyi taşır. Bu bayrak tekniği, Konu 14.1’de öğrendiğin `pygame.KEYDOWN`/`pygame.KEYUP` bayrak mantığıyla aynıdır.

15.2.5 Fare Takip Örneği: İz Bırakan Daire

Şimdi bu konuda öğrendiklerini birleştiren kapsamlı bir örnek yazalım. Bu programda bir daire fareyi kademeli olarak takip edecek (lerp tekniğiyle), fare imleci gizlenecek ve farenin arkasında giderek küçülen dairelerden oluşan bir iz efekti oluşacak.

Programın güncelleme kısmında lerp ile yumuşak takip ve iz listesi yönetimi bir arada çalışır:

Python – Fare Takip – Lerp ve İz Güncelleme

```

1  TAKIP_HIZI = 0.06      # Kucuk deger = yavas takip
2  IZ_UZUNLUGU = 25      # Saklanacak konum sayisi
3  iz = []
4
5  # ... her karede ...
6  # Farenin anlik konumunu al
7  hedef_x, hedef_y = pygame.mouse.get_pos()
8
9  # Lerp ile nesneyi hedefe yaklastir
10 nesne_x += (hedef_x - nesne_x) * TAKIP_HIZI
11 nesne_y += (hedef_y - nesne_y) * TAKIP_HIZI
12
13 # Konumu iz listesine ekle
14 iz.append((int(nesne_x), int(nesne_y)))
15 if len(iz) > IZ_UZUNLUGU:
16     iz.pop(0)

```

İz efektini çizerken her dairenin boyutu ve renk parlaklığı listedeki sırasına göre hesaplanır. Eski konumlar küçük ve soluk, yeni konumlar büyük ve parlak görünür:

Python – Fare Takip – İz Efektini Çizimi

```

1  for i, konum in enumerate(iz):
2      oran = (i + 1) / len(iz) # 0.0 -> 1.0
3      yaricap = max(2, int(NESNE_YARICAP * oran * 0.7))
4      r = int(IZ_RENGI[0] * oran)
5      g = int(IZ_RENGI[1] * oran)
6      b = int(IZ_RENGI[2] * oran)
7      pygame.draw.circle(ekran, (r, g, b), konum, yaricap)

```

Dikkat edilmesi gereken noktalar:

1. Lerp formülü: `nesne_x += (hedef_x - nesne_x) * TAKIP_HIZI`. `TAKIP_HIZI` değeri küçük olduğunda nesne fareye yavaşça yaklaşır; büyük olduğunda hızlı yapışır.
2. `iz` listesi son N konumu saklar; uzunluk aşıldınca `pop(0)` ile en eski konum silinir.
3. `oran` değişkeni 0'dan 1'e doğru artar, bu sayede eski konumlar küçük/soluk, yeni konumlar büyük/parlak çizilir.
4. `pygame.mouse.set_visible(False)` ile varsayılan imleç gizlenir ve yerine çizilen daire "özel imleç" görevi görür.

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite3/02\_fare\_taki` dosyasında bulabilirsin.

💡 İpucu: Lerp Oranını Dene

`TAKIP_HIZI` değerini 0.02 ile 0.2 arasında değiştirerek farkı gözlemle. Düşük değerler ağır ve rüya gibi bir hareket, yüksek değerler hızlı ve tepkisel bir hareket üretir. Oyun türüne göre doğru oranı seçmek, oyuncu deneyimini önemli ölçüde etkiler.

15.3 Sonuç

15.3.1 Ünite Özeti

Bu ünite de farenin oyunlardaki rolünü ve PyGame'in fare olaylarını nasıl işlediğini öğrendin. Fare tıklama olaylarıyla başladın: `pygame.MOUSEBUTTONDOWN` ve `pygame.MOUSEBUTTONUP` olaylarının `olay.pos` ve `olay.button` özelliklerini kullanarak nereye ve hangi düğmeyle tıklandığını belirlemeyi öğrendin. Beş farklı fare düğmesini (sol, orta, sağ, scroll yukarı, scroll aşağı) tanıdın.

Ardından fare hareketini `pygame.MOUSEMOTION` olayıyla izlemeyi, `olay.rel` ile hareket farkını hesaplamayı ve lerp tekniğiyle bir nesnenin fareyi yumuşak bir şekilde takip etmesini sağlamayı keşfettin. Fare imlecini gizleyip yerine kendi çizimini koymayı ve basit bir sürükle-bırak mekaniği kurmayı deneyimledin.

Bu bilgiler, Konu 14.1'de öğrendiğin klavye girdileriyle birlikte sana eksiksiz bir girdi sistemi sağlıyor. Artık hem klavye hem fare ile oyuncuyla etkileşim kurabilirsin.

15.3.2 Anahtar Kavramlar

Tablo 15.3: Bu ünite de öğrenilen anahtar kavramlar

Kavram	Açıklama
<code>pygame.MOUSEBUTTONDOWN</code>	Fare düğmesine basıldığında tetiklenen olay; <code>pos</code> ve <code>button</code> özellikleri taşır
<code>pygame.MOUSEBUTTONUP</code>	Fare düğmesi bırakıldığında tetiklenen olay
<code>pygame.MOUSEMOTION</code>	Fare hareket ettiğinde tetiklenen olay; <code>pos</code> , <code>rel</code> ve <code>buttons</code> özellikleri taşır
<code>olay.pos</code>	Tıklama veya hareket anındaki fare konumu (<code>x</code> , <code>y</code>)
<code>olay.button</code>	Basılan fare düğmesinin numarası (1=sol, 2=orta, 3=sağ)
<code>olay.rel</code>	Son konumdan fark (<code>dx</code> , <code>dy</code>) — hareket miktarı ve yönü
Lerp	Doğrusal enterpolasyon; iki değer arasında yumuşak geçiş tekniği

15.3.3 Önemli Komutlar

Tablo 15.4: Bu ünite de kullanılan PyGame komutları

Komut	İşlev
<code>pygame.mouse.get_pos()</code>	Farenin o anki pencere koordinatını (<code>x</code> , <code>y</code>) olarak döndürür
<code>pygame.mouse.get_pressed()</code>	Fare düğmelerinin basılı durumunu (<code>sol</code> , <code>orta</code> , <code>sag</code>) tuple olarak döndürür
<code>pygame.mouse.set_visible(bool)</code>	Fare imlecini göster veya gizle
<code>pygame.MOUSEBUTTONDOWN</code>	Fare düğmesine basılma olayının sabit değeri
<code>pygame.MOUSEBUTTONUP</code>	Fare düğmesinin bırakılma olayının sabit değeri
<code>pygame.MOUSEMOTION</code>	Fare hareket olayının sabit değeri

15.3.4 Sıradaki Adım

Bu ünite de fare girdilerini öğrendin. Bir önceki ünite de klavye girdilerini, ondan önce de olay sisteminin temellerini keşfettin. Artık kullanıcıyla etkileşim kurmanın tüm temel yollarını biliyorsun.

Bir sonraki ünite de (Ünite 4) **Hareket Mekaniği**'ne geçeceğiz. Hız vektörü kavramını, delta time ile frame-bağımsız hareketi ve sınır kontrol stratejilerini (clamp, wrap, bounce) öğreneceksin. En önemlisi, tüm bu bilgileri birleştirerek ilk gerçek oyununu — bir **Yılan Oyunu** (Snake) — geliştireceksin!

💡 İpucu: Pratik Yap

Bu ünitenin kodlarını kendi bilgisayarında çalıştır ve üzerinde değişiklikler yap. Örneğin tıklama örneğine scroll ile daire boyutunu değiştirme özelliği ekle. Veya fare takip örneğinde birden fazla takipçi nesne oluşturmayı dene. En iyi öğrenme yöntemi, kodu değiştirip sonucu gözlemlemektir.

“Hareket, bir oyunun ruhudur. Durağan bir ekran tablodur; hareket eden bir ekran oyundur.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Hız vektörü kavramı ve delta time ile konum güncelleme
- Piksel/saniye cinsinden hız tanımlama ve frame-bağımsız hareket
- Sınır kontrolü stratejileri: clamp, wrap ve bounce
- Tüm öğrendiklerini birleştirerek tam bir **Yılan Oyunu** (Snake) geliştirme

16.1 Hız ve Yön

Konu 15.2’de fareyi takip eden nesneler oluşturdu ve Konu 14.3’te ok tuşlarıyla bir kareyi piksel piksel hareket ettirdin. Ancak dikkat ettiysen hareket kodlarında doğrudan `x += 5` gibi sabit değerler kullandık. Peki bu “5” nereden geldi? Farklı bilgisayarlarda oyun farklı hızlarda çalışırsa ne olur? İşte bu sorulara cevap vermek için *hız vektörü* kavramıyla tanışma zamanı.

Bir araba düşün: hız göstergesi sana saniyede kaç kilometre gittiğini söyler. Oyunlarda da benzer bir mantık var ama birim piksel ve zaman birimi saniye. Bir nesnenin hızını **piksel/saniye** (px/s) cinsinden tanımlarsak, FPS ne olursa olsun nesne her saniye aynı mesafeyi kat eder.

16.1.1 Hız Kavramı: Piksel/Saniye

Önceki ünitelerde hızı “kare başına piksel” olarak düşünüyordun: her döngü turunda nesneyi 5 piksel sağa kaydırıyordun. Sorun şu ki oyun döngüsü saniyede 30 kez çalışırsa nesne $5 \times 30 = 150$ piksel ilerler, ama 60 kez çalışırsa $5 \times 60 = 300$ piksel ilerler. Aynı kod, farklı bilgisayarlarda farklı davranır.

Çözüm basit: hızı **piksel/saniye** cinsinden tanımla ve her karede geçen süreyle çarp. Bu süreye *delta time* (**dt**) diyoruz ve Bölüm 11’de detaylıca incelemiştik. Hattırlayalım: **dt = saat.tick(FPS) / 1000.0** ifadesi bize son kare ile şimdiki kare arasındaki süreyi saniye cinsinden verir (Konu 11.3).

■ Tanım: Hız (Velocity)

Hız, bir nesnenin birim zamanda kat ettiği mesafedir. Oyun programlamada genellikle **piksel/saniye** (px/s) birimiyle ifade edilir. Konum güncelleme formülü:

$$\text{yeni_konum} = \text{eski_konum} + \text{hiz} \times \text{dt}$$

Burada **dt**, son iki kare arasında geçen süredir (saniye cinsinden).

16.1.2 Hız Vektörü: (**vx**, **vy**) Bileşenleri

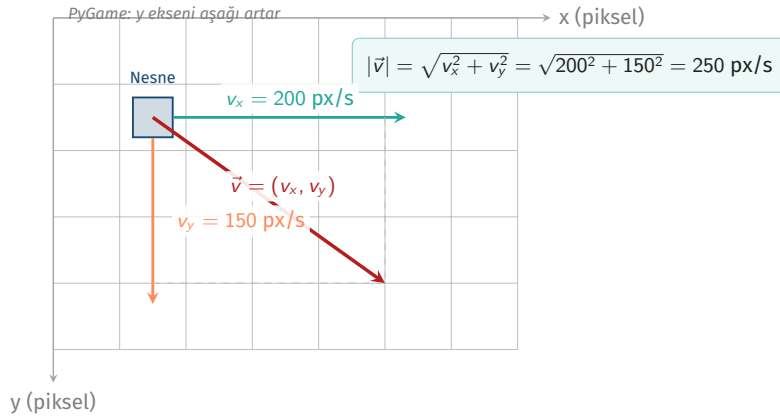
Ekrandaki bir nesne hem yatay hem dikey ekseninde hareket edebilir. Bu yüzden hızı tek bir sayı yerine iki bileşenle ifade ederiz: **vx** yatay hızı (sağa pozitif, sola negatif), **vy** ise dikey hızı (aşağı pozitif, yukarı negatif) belirtir. PyGame’in koordinat sisteminde y eksenini aşağı doğru artar, bunu unutma.

Bu iki bileşen bir araya geldiğinde *hız vektörü* (**vx**, **vy**) oluşur. Konum güncellemesi şöyle yapılır:

Python – Hız Vektörüyle Konum Güncelleme

```
1 # Hız bileşenleri (piksel/saniye)
2 vx = 200 # sağa doğru 200 px/s
3 vy = 150 # aşağıya doğru 150 px/s
4
5 # Her karede konum güncelle
6 x += vx * dt
7 y += vy * dt
```

Bu kavramı görsel olarak incelersek daha net anlayacağız.

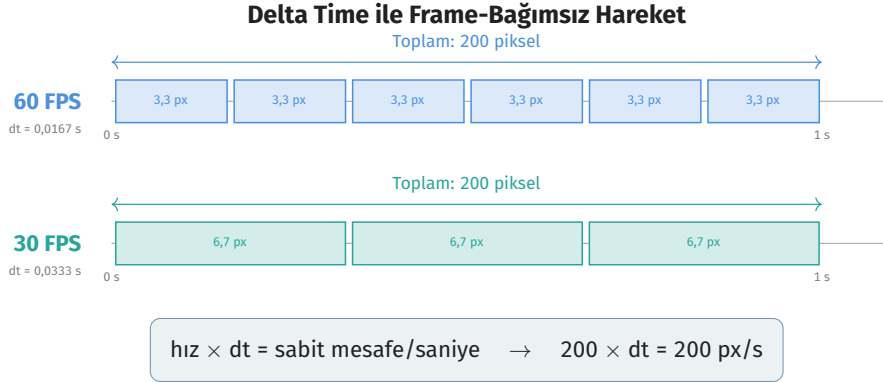


Şekil 16.1: Hız vektörü bileşenleri ve sonuç vektörü

Şekil 16.1’da görüldüğü gibi **vx** bileşeni nesneyi yatayda, **vy** bileşeni ise dikeyde hareket ettirir. Bu iki bileşenin birleşimi olan sonuç vektörü, nesnenin gerçek hareket yönünü ve büyüklüğünü gösterir.

16.1.3 Delta Time Hatırlatma

Konu 11.3’te delta time kavramını öğrenmiştik. Şimdi neden bu kadar önemli olduğunu hız vektörü bağlamında tekrar görelim.



Şekil 16.2: Farklı FPS değerlerinde delta time ile sabit mesafe

Şekil 16.2’da iki farklı FPS durumu karşılaştırılıyor. 60 FPS’te her kare yaklaşık 0,0167 saniye sürer ve nesne her karede $200 \times 0,0167 \approx 3,3$ piksel ilerler. 30 FPS’te ise her kare 0,0333 saniye sürer ve nesne her karede $200 \times 0,0333 \approx 6,7$ piksel ilerler. Her iki durumda da **1 saniyede toplam 200 piksel** kat edilir. İşte delta time’ın büyüğü budur.

16.1.4 Çapraz Hareket ve Normalizasyon

Ok tuşlarıyla hareket ettiğini düşün. Sadece sağ tuşa basarsan nesne yatayda **hız** kadar ilerler. Sadece aşağı tuşa basarsan dikeyde **hız** kadar ilerler. Peki ikisine aynı anda basarsan?

Eğer her iki yönde de aynı hızı uygularsan, çapraz hareketin toplam büyüklüğü $\sqrt{hız^2 + hız^2} = hız \times \sqrt{2} \approx hız \times 1,414$ olur. Yani çapraz hareket eden oyuncu, düz hareket edene göre yaklaşık **yüzde kırk bir daha hızlı** gider. Bu klasik “1.41 sorunu” olarak bilinir ve çoğu oyunda sorun yaratır.

⚠ Dikkat: Çapraz Hız Sorunu

Çapraz harekette **vx** ve **vy** birlikte uygulandığında toplam hız $\sqrt{2}$ katına çıkar. Bunu önlemek için hız vektörünü *normalize* etmen gerekir: vektörü uzunluğuna bölersin.

Çözüm olarak hareket vektörünü normalize ederiz:

Python – Çapraz Hareket Normalizasyonu

```

1 import math
2
3 dx = 0
4 dy = 0
5 if tuslar[pygame.K_LEFT]:
6     dx = -1
7 if tuslar[pygame.K_RIGHT]:
8     dx = 1
9 if tuslar[pygame.K_UP]:
10    dy = -1
11 if tuslar[pygame.K_DOWN]:
12    dy = 1
13
14 # Normalize et (cikartilan yonleri olmayan durumlari ele
15 ↪ al)
16 uzunluk = math.sqrt(dx * dx + dy * dy)
17 if uzunluk > 0:
18     dx /= uzunluk
19     dy /= uzunluk
20
21 # Hizi uygula
22 x += dx * hiz * dt
23 y += dy * hiz * dt

```

Bu kodda önce hareket yönünü (-1,0,+1) olarak belirliyoruz, sonra vektörün uzunluğunu hesaplayıp her bileşeni bu uzunluğa bölüyoruz. Böylece çapraz harekette bile toplam hız sabit kalır.

İpucu: pygame.math.Vector2

İleri seviyede PyGame'in `pygame.math.Vector2` sınıfını kullanarak vektör işlemlerini çok daha kolay yapabilirsin. `v.normalize()` metodu vektörü otomatik normalize eder. Şimdilik `math.sqrt()` ile mantığı anlayan yeterli; ileriki bölümlerde `Vector2` sınıfını tanıyacaksın.

Şimdi tüm bu kavramları bir araya getiren tam bir örnek yazalım. Ana döngünün güncelleme kısmı, yukarıda öğrendiğin normalizasyon mantığını delta time ile birleştirir:

Python – Hız Tabanlı Hareket – Güncelleme Döngüsü

```

1 while calistir:
2     dt = saat.tick(FPS) / 1000.0
3
4     # ... (olay isleme) ...
5
6     # Yon vektorunu belirle
7     tuslar = pygame.key.get_pressed()

```



```

8     dx, dy = 0.0, 0.0
9     if tuslar[pygame.K_LEFT]: dx = -1.0
10    if tuslar[pygame.K_RIGHT]: dx = 1.0
11    if tuslar[pygame.K_UP]: dy = -1.0
12    if tuslar[pygame.K_DOWN]: dy = 1.0
13
14    # Capraz hareket normalizasyonu
15    uzunluk = math.sqrt(dx * dx + dy * dy)
16    if uzunluk > 0:
17        dx /= uzunluk
18        dy /= uzunluk
19
20    # Konum guncelle ve sinir kontrolu
21    x += dx * HIZ * dt
22    y += dy * HIZ * dt
23    x = max(TOP_YARICAP, min(x, GENISLIK - TOP_YARICAP))
24    y = max(TOP_YARICAP, min(y, YUKSEKLIK - TOP_YARICAP))

```

Bu döngüde sırasıyla: delta time hesaplanır, `pygame.key.get_pressed()` ile yön vektörü belirlenir, vektör normalize edilir ve konum güncellenir. Son olarak `clamp` ile nesne ekran içinde tutulur. Pencere başlığında hız bileşenlerini ve konumu görebilirsin — böylece çapraz harekette bile toplam hızın değişmediğini doğrulayabilirsin.

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite4/01\_hiz\_harek` dosyasında bulabilirsin. Ok tuşları ve WASD ile dene.

16.2 Sınır Kontrolü

Bir önceki konuda hız vektörüyle nesneleri hareket ettirmeyi öğrendin. Peki nesne ekranın kenarına ulaştığında ne olacak? Eğer hiçbir önlem almazsan nesne çerçevenin dışına çıkar ve bir daha geri gelmez. Bu, neredeyse hiçbir oyunda istenen bir davranış değildir. İşte bu yüzden *sınır kontrolü* (boundary checking) mekanizmalarına ihtiyacımız var.

Gerçek hayattan bir örnek verelim: bir bilardo topunu düşün. Top masanın kenarına çarptığında ne olacağını masanın kuralları belirler. Bazı oyunlarda top durur (`clamp`), bazılarında diğer taraftan çıkar (`wrap`), bazılarında ise geri seker (`bounce`). Şimdi bu üç stratejiyi tek tek inceleyelim.

16.2.1 Clamp: Sınırdaki Durdurma

En basit ve en yaygın strateji *clamp* (sıkıştırma) yöntemidir. Nesnenin konumunu ekran sınırları içinde tutarsın; sınırı aşmaya çalıştığında konumu sınır değerine sabitlersin.

■ Tanım: Clamp (Sıkıştırma)

Clamp, bir değeri belirli bir aralığın içinde tutan işlemdir. Değer alt sınırın altına düşerse alt sınıra, üst sınırın üstüne çıkarsa üst sınıra eşitlenir.

Formül olarak şöyle ifade edebiliriz:

$$x = \max(0, \min(x, \text{genislik} - \text{boyut}))$$

Python kodunda bu işlem oldukça kısadır:

Python – Clamp Stratejisi

```
1 # Yatay sinir
2 x = max(0, min(x, GENISLIK - BOYUT))
3 # Dikey sinir
4 y = max(0, min(y, YUKSEKLIK - BOYUT))
```

Bu yöntem özellikle oyuncu karakteri için idealdir. Çoğu oyunda oyuncu ekranın dışına çıkamamalıdır ve sınıra geldiğinde durmalıdır. Platform oyunlarında, yarış oyunlarında ve menü sistemlerinde clamp sıkça kullanılır.

16.2.2 Wrap: Sarmalama

Wrap (sarmalama) stratejisinde nesne bir kenardan çıktığında karşı kenardan geri girer. Pac-Man oyununu hatırla: Pac-Man ekranın solundan çıktığında sağdan tekrar belirir. Bu efekt tünel hissi verir ve oyun alanını sonsuz gibi gösterir.

Python – Wrap Stratejisi

```
1 # Sağdan çıktığında soldan gir
2 if x > GENISLIK:
3     x = -BOYUT
4 # Soldan çıktığında sağdan gir
5 elif x < -BOYUT:
6     x = GENISLIK
7
8 # Asagidan çıktığında yukaridan gir
9 if y > YUKSEKLIK:
10    y = -BOYUT
11 # Yukaridan çıktığında asagidan gir
12 elif y < -BOYUT:
13    y = YUKSEKLIK
```

Dikkat edersen nesne tamamen ekran dışına çıktıktan sonra ($x > \text{GENISLIK}$ veya $x < -\text{BOYUT}$) karşı tarafa konumlandırılır. Böylece nesne ekranın kenarında aniden kaybolup belirmez, doğal bir geçiş sağlanır.

16.2.3 Bounce: Sekme

Bounce (sekme) stratejisinde nesne sınıra çarptığında hız yönünü tersine çevirir ve geri döner. Pong oyunundaki top buna klasik bir örnektir. Bu strateji fiziksel

gerçekçilik hissi yaratır.

Python – Bounce Stratejisi

```

1  # Yatay sınır kontrolü
2  if x <= 0:
3      x = 0
4      vx = -vx          # Hız yönünü tersine çevir
5  elif x >= GENISLIK - BOYUT:
6      x = GENISLIK - BOYUT
7      vx = -vx
8
9  # Dikey sınır kontrolü
10 if y <= 0:
11     y = 0
12     vy = -vy
13 elif y >= YUKSEKLİK - BOYUT:
14     y = YUKSEKLİK - BOYUT
15     vy = -vy

```

Burada iki önemli nokta var. Birincisi, nesne sınırı aştığında konumunu sınır değerine geri çekiyoruz; aksi halde nesne sınırın ötesinde kalıp sürekli yön değiştirerek titreyebilir. İkincisi, sadece çarpan eksenindeki hız bileşenini tersine çeviriyoruz: yatay sınıra çarpıyorsa **vx**, dikey sınıra çarpıyorsa **vy** ters döner.

16.2.4 Hangi Strateji Ne Zaman?

Doğru stratejiyi seçmek oyunun türüne ve tasarımına bağlıdır. Aşağıdaki tablo sana rehberlik edecektir:

Tablo 16.1: Sınır kontrol stratejilerinin karşılaştırması

Strateji	Davranış	Kullanım Alanı
Clamp	Sınırdaki durur	Oyuncu karakteri, menü imleci
Wrap	Karşı taraftan çıkar	Uzay oyunları, Pac-Man tarzı
Bounce	Geri seker	Top oyunları, Pong, bilardo

● Bilgi: Stratejileri Birleştirme

Aynı oyunda farklı nesneler için farklı stratejiler kullanabilirsin. Örneğin oyuncu için clamp, düşman mermileri için wrap, top için bounce uygulayabilirsin. Hatta tek bir nesnede eksenlere göre farklı stratejiler bile mümkündür: yatayda wrap, dikeyde clamp gibi.

Şimdi üç stratejiyi aynı anda görebileceğin bir demo programı yazalım. Program üç topu farklı stratejilerle hareket ettirir. Her topun güncelleme kodu yukarıda anlattığımız kalıbı birebir takip eder:

Python – Üç Stratejiyi Bir Arada Güncelleme

```

1  # --- CLAMP guncelle ---
2  clamp_x += clamp_vx * dt
3  clamp_y += clamp_vy * dt
4  clamp_x = max(TOP_YARICAP, min(clamp_x, GENISLIK -
   ↳ TOP_YARICAP))
5  clamp_y = max(TOP_YARICAP, min(clamp_y, YUKSEKLIK -
   ↳ TOP_YARICAP))
6
7  # --- WRAP guncelle ---
8  wrap_x += wrap_vx * dt
9  wrap_y += wrap_vy * dt
10 if wrap_x > GENISLIK + TOP_YARICAP:
11     wrap_x = -TOP_YARICAP
12 elif wrap_x < -TOP_YARICAP:
13     wrap_x = GENISLIK + TOP_YARICAP
14
15 # --- BOUNCE guncelle ---
16 bounce_x += bounce_vx * dt
17 bounce_y += bounce_vy * dt
18 if bounce_x <= TOP_YARICAP:
19     bounce_x = TOP_YARICAP
20     bounce_vx = -bounce_vx
21 elif bounce_x >= GENISLIK - TOP_YARICAP:
22     bounce_x = GENISLIK - TOP_YARICAP
23     bounce_vx = -bounce_vx

```

Her üç strateji aynı `hiz * dt` formülüyle konum güncellemesi yapar. Fark, sınıra ulaşıldığında ne yapıldığındadır: clamp konumu sabitler, wrap karşı tarafa taşır, bounce hız yönünü tersine çevirir.

İpucu: Tam Kaynak Kodu

Üç stratejiyi aynı anda çalışır halde görmek için `kod\ornekleri\bolum04\unite4\02\_sinir\_kontrol.py` dosyasını çalıştır.

Bu programı çalıştırdığında üç farklı renkte top göreceksin. Mavi top sınırdan durur (clamp), yeşil top diğer taraftan çıkar (wrap) ve kırmızı top sınırlardan geri seker (bounce). Hiçbir kullanıcı girdisi gerekmez; toplar otomatik hareket eder ve her stratejinin farkını gözlemleyebilirsin.

16.3 Mini Proje: Yılan Oyunu

Bu bölüm boyunca olay sistemini, klavye ve fare girdilerini, hız vektörlerini ve sınır kontrolünü öğrendin. Şimdi tüm bu bilgileri bir araya getirerek klasik **Yılan Oyunu** (Snake) projesini geliştirme zamanı. Bu proje, öğrendiklerinin gerçek bir oyunda nasıl kullanıldığını gösterecek.

16.3.1 Oyun Mekaniği

Yılan oyununun kuralları basit ama zarif bir tasarıma sahiptir. Bir ızgara (grid) üzerinde hareket eden bir yılanı ok tuşlarıyla yönlendirirsin. Ekranda rastgele bir konumda yem belirir. Yılan yemi yediğinde bir hücre büyür ve skorun artar. Yılan duvara veya kendi gövdesine çarparsa oyun biter.

Bu oyunu ızgara tabanlı hareket sistemiyle tasarlayacağız. Ekranı **20x20** piksellik hücrelere böleceğiz. Yılan her adımda bir hücre ilerler — bu, sürekli piksel hareketi yerine ayrık (discrete) bir hareket modelidir. Konu 14.1’te öğrendiğin `pygame`.`KEYDOWN` olayı tam da bu tür hareketler için idealdir: her tuş basışında bir adım at.

16.3.2 Adım 1: Temel Yapı ve Sabitler

Her oyun projesine sabitlerle başlamak iyi bir alışkanlıktır. Izzaranın boyutunu, hücre boyutunu ve renkleri en başta tanımlayalım:

Python – Yılan Oyunu Sabitleri

```
1 import pygame
2 import random
3
4 # Izgara sabitleri
5 HUCRE_BOYUTU = 20
6 IZGARA_GENISLIK = 30 # 30 hucre = 600 piksel
7 IZGARA_YUKSEKLIK = 20 # 20 hucre = 400 piksel
8 GENISLIK = IZGARA_GENISLIK * HUCRE_BOYUTU # 600
9 YUKSEKLIK = IZGARA_YUKSEKLIK * HUCRE_BOYUTU # 400
10
11 # Renkler
12 SIYAH = (0, 0, 0)
13 BEYAZ = (255, 255, 255)
14 YESIL = (0, 200, 0)
15 KOYU_YESIL = (0, 150, 0)
16 KIRMIZI = (220, 50, 50)
17 GRI = (40, 40, 40)
```

Dikkat edersen pencere boyutlarını ızgara boyutundan hesaplıyoruz. Böylece her hücre tam olarak 20 piksel olur ve hücreler arasında boşluk kalmaz.

16.3.3 Adım 2: Yılanın Hareketi

Yılanın gövdesini bir **liste** olarak temsil edeceğiz. Listenin her elemanı bir **(x, y)** koordinat çiftidir (ızgara koordinatları, piksel değil). Listenin ilk elemanı yılanın başı, son elemanı kuyruğudur.

Hareket mantığı şu şekildedir: her adımda yılanın başının önüne yeni bir hücre eklenir ve kuyruktan bir hücre silinir. Böylece yılan ilerlemiş olur ama uzunluğu aynı kalır. Yem yediğinde ise kuyruktan silme işlemi yapılmaz ve yılan bir hücre uzar.

Python – Yılan Hareketi - Liste Mantığı

```

1  # Yılanın govdesi (izgara koordinatlari)
2  yilan = [(5, 10), (4, 10), (3, 10)] # Bas: (5,10),
   ↳ Kuyruk: (3,10)
3  yon = (1, 0) # Saga dogru hareket
4
5  # Her adimda:
6  bas_x, bas_y = yilan[0]
7  yeni_bas = (bas_x + yon[0], bas_y + yon[1])
8  yilan.insert(0, yeni_bas) # Basa yeni hucre ekle
9  yilan.pop()               # Kuyruktan sil (buyumuyorsa)

```

Bu yaklaşım çok verimlidir: her adımda sadece bir ekleme ve bir silme işlemi yapılır. Yılanın uzunluğu ne olursa olsun hareket maliyeti sabittir.

16.3.4 Adım 3: Yem Oluşturma

Yem, ızgara üzerinde rastgele bir konumda belirir. Python'un `random` modülünü kullanarak ızgara sınırları içinde rastgele bir koordinat seçeriz. Önemli bir detay: yemin yılanın gövdesi üzerinde belirmemesini sağlamalıyız.

Python – Yem Oluşturma

```

1  def yem_olustur(yilan):
2      """Yılanın üzerinde olmayan rastgele bir konum sec."""
3      while True:
4          konum = (
5              random.randint(0, IZGARA_GENISLIK - 1),
6              random.randint(0, IZGARA_YUKSEKLIK - 1)
7          )
8          if konum not in yilan:
9              return konum

```

`while True` döngüsü, rastgele seçilen konumun yılanın gövdesinde olup olmadığını kontrol eder. Eğer yılanın üzerindeyse yeni bir konum dener. Yılan çok büyümekçe bu döngü hemen ilk denemede doğru konumu bulur.

16.3.5 Adım 4: Çarpışma Kontrolü

İki tür çarpışma kontrol edeceğiz. Birincisi **duvar çarpışması**: yılanın başı ızgaranın dışına çıkarsa oyun biter. İkincisi **kendine çarpışma**: yılanın başı kendi gövdesindeki herhangi bir hücreyle aynı konuma gelirse oyun biter.

Python – Çarpışma Kontrolü

```

1  def carpismaya_bak(bas, yilan_govde):
2      """Duvara veya kendine carpma kontrolu."""
3      bx, by = bas
4      # Duvar kontrolu
5      if bx < 0 or bx >= IZGARA_GENISLIK:

```

```

6         return True
7     if by < 0 or by >= IZGARA_YUKSEKLIK:
8         return True
9     # Kendine carpma (bas haric govde)
10    if bas in yilan_govde[1:]:
11        return True
12    return False

```

Kendine çarpışma kontrolünde `yilan_govde[1:]` kullanarak başı hariç tutuyoruz; çünkü baş zaten listenin ilk elemanı olarak kendisiyle eşleşir.

16.3.6 Adım 5: Skor Gösterimi

Skoru ekranda göstermek için `pygame.font` modülünü kullanacağız. Bu modülü **Bölüm 9**'da detaylı olarak inceleyeceğiz; şimdilik sadece temel kullanımını görelim:

Python – Skor Gösterimi (Kısa)

```

1 font = pygame.font.Font(None, 28)
2 skor_yazi = font.render(f"Skor: {skor}", True, BEYAZ)
3 ekran.blit(skor_yazi, (10, 10))

```

Burada `Font(None, 28)` sistem varsayılan yazı tipini 28 punto boyutunda yükler. `render()` metodu metni bir yüzeye çevirir ve `blit()` ile ekrana yerleştirir. Yazı tipi özelleştirme, anti-aliasing ve çok satırlı metin gibi ileri konuları **Bölüm 9**'da detaylı göreceğiz.

16.3.7 Adım 6: Game Over ve Yeniden Başlatma

Oyun bittiğinde ekrana bir “Game Over” mesajı göstereceğiz ve oyuncuya `K_r` tuşuyla yeniden başlama, `K_ESCAPE` ile çıkma seçeneği sunacağız. Oyun durumunu bir `oyun_bitti` değişkeniyle takip edeceğiz:

Python – Game Over Mantığı

```

1 if oyun_bitti:
2     # Game Over yazisi goster
3     go_font = pygame.font.Font(None, 48)
4     go_yazi = go_font.render("GAME OVER", True, KIRMIZI)
5     go_rect = go_yazi.get_rect(center=(GENISLIK // 2,
6     ↪ YUKSEKLIK // 2))
7     ekran.blit(go_yazi, go_rect)
8
9     bilgi = font.render("R: Tekrar | ESC: Cikis", True,
10    ↪ BEYAZ)
11    bilgi_rect = bilgi.get_rect(center=(GENISLIK // 2,
12    ↪ YUKSEKLIK // 2 + 40))
13    ekran.blit(bilgi, bilgi_rect)

```

16.3.8 Tam Oyun Kodu

Yukarıdaki adımlarda oyunun her parçasını ayrı ayrı inceledik. Şimdi bu parçaların `main()` fonksiyonundaki oyun döngüsünde nasıl bir araya geldiğine bakalım. Bu döngü, Bölüm 4'te öğrendiğin `pygame.KEYDOWN` olayları (Konu 14.1), olay kuyruğu (Konu 13.1) ve sınır kontrolü (Konu 16.2) kavramlarını tek bir akışta birleştiriyor.

Python – Yılan Oyunu – Ana Döngü

```

1  while calistir:
2      saat.tick(FPS)
3
4      # --- Olaylari isle ---
5      for olay in pygame.event.get():
6          if olay.type == pygame.QUIT:
7              calistir = False
8          elif olay.type == pygame.KEYDOWN:
9              if olay.key == pygame.K_ESCAPE:
10                 calistir = False
11                 if oyun_bitti:
12                     if olay.key == pygame.K_r:
13                         yilan, yon, skor, yem, oyun_bitti =
14                             ↪ oyunu_sifirla()
15                         continue
16                 # Yon degistir (ters yone donmeyi engelle)
17                 if olay.key == pygame.K_UP and yon != ASAGI:
18                     yon = YUKARI
19                 elif olay.key == pygame.K_DOWN and yon !=
20                     ↪ YUKARI:
21                     yon = ASAGI
22                 elif olay.key == pygame.K_LEFT and yon != SAG:
23                     yon = SOL
24                 elif olay.key == pygame.K_RIGHT and yon !=
25                     ↪ SOL:
26                     yon = SAG
27
28     # --- Guncelle ---
29     if not oyun_bitti:
30         bas_x, bas_y = yilan[0]
31         yeni_bas = (bas_x + yon[0], bas_y + yon[1])
32
33         # Carpismaya bak
34         if carpismaya_bak(yeni_bas, yilan):
35             oyun_bitti = True
36         else:
37             yilan.insert(0, yeni_bas)
38             if yeni_bas == yem:
39                 skor += 1
40                 yem = yem_olustur(yilan)
41             else:
42                 yilan.pop()

```



```

40
41     # --- Ciz ---
42     ekran.fill(SIYAH)
43     # ... (izgara, yilan, yem, skor cizimi) ...
44     pygame.display.flip()

```

Dikkat edersen döngü üç klasik aşamadan oluşuyor: **olay işleme** (yön değiştirme ve yeniden başlatma), **güncelleme** (hareket, çarpışma, yem kontrolü) ve **çizim**. Yön değiştirmede `yon != ASAGI` kontrolü yılanın 180 derece ters dönmesini engeller — aksi halde yılan hemen kendine çarpar.

💡 İpucu: Tam Kaynak Kodu

Bu programın tam kaynak kodunu `kod\_ornekleri/bolum04/unite4/03\_yilan\_oyu` dosyasında bulabilirsin. Çalıştırarak oynat ve kodu incele.

Bu programı çalıştırdığında 600×400 piksellik bir pencere açılacak. Yılanı ok tuşlarıyla yönlendir, kırmızı yemi topla ve büyümeye çalış. Duvara veya kendi gövdeneye çarpmamaya dikkat et. Game Over olduktan sonra `K_r` ile yeniden başlayabilirsin.

💡 İpucu: Oyunu Geliştir

Yılan Oyunu'nu daha da eğlenceli hale getirmek için şu fikirleri deneyebilirsin: yılanın hızını skor arttıkça kademeli olarak artır, ekrana engeller (duvarlar) ekle veya farklı renkte özel yemler koy (örneğin mavi yem 3 puan versin). Bu değişiklikler, öğrendiğin kavramları pekiştirmen için harika birer egzersizdir.

16.4 Sonuç

16.4.1 Ünite 4 Özeti

Bu ünite de hareket mekaniğinin temellerini öğrendin. Hız vektörü kavramıyla tanışıp `(vx, vy)` bileşenleri üzerinden nesneleri piksel/saniye cinsinden hareket ettirdin. Delta time (`dt`) kullanarak frame-bağımsız hareket sağladın ve çapraz harekette ortaya çıkan 1.41 sorununu normalizasyon ile çözdün.

Sınır kontrolü konusunda üç temel strateji öğrendin: nesneleri ekran içinde tutan `clamp`, karşı taraftan çıkaran `wrap` ve geri sektiren `bounce`. Son olarak tüm bu bilgileri bir araya getirerek tam bir Yılan Oyunu geliştirdin.

16.4.2 Bölüm 4 Genel Özeti

Bölüm 4'ü tamamladın, tebrikler! Bu bölüm PyGame'de kullanıcı etkileşiminin tüm boyutlarını kapsıyordu. Şimdi geriye dönüp neler öğrendiğine bir bakalım.

Bölüm 13'de PyGame'in olay kuyruğunun FIFO yapısını, `Event` nesnesinin özelliklerini ve olay filtreleme tekniklerini inceledik. Bölüm 14'de `pygame.KEYDOWN`/`pygame.KEY`

olayları ile ayırık girdi ve `pygame.key.get_pressed()` ile sürekli girdi arasındaki farkı kavradık. Bölüm 15’de fare olaylarını, `pygame.mouse.get_pos()` ile konum sorgulama ve fare takip mekanizmalarını öğrendik. Son olarak bu ünite de hız vektörleri, delta time uygulaması ve sınır kontrolü stratejileriyle hareket mekaniğini tamamladık.

16.4.3 Anahtar Kavramlar

Tablo 16.2: Bölüm 4 – Anahtar kavramlar

Kavram	Açıklama
Olay Kuyruğu	Kullanıcı eylemlerini FIFO sırasıyla saklayan yapı
<code>pygame.KEYDOWN</code>	Tuşa basıldığında tetiklenen olay (ayırık girdi)
<code>key.get_pressed()</code>	Tüm tuşların anlık durumunu sorgular (sürekli)
<code>pygame.MOUSEBUTTONDOWN</code>	Fare düğmesine tıklandığında tetiklenen olay
<code>mouse.get_pos()</code>	Farenin anlık ekran konumunu döndürür
Hız Vektörü	(v_x, v_y) bileşenleri, piksel/saniye cinsinden
Delta Time	İki kare arasındaki süre (saniye), frame-bağımsız hareket
Normalizasyon	Çapraz harekette hız büyüklüğünü sabit tutma
Clamp	Nesneyi sınır içinde tutan strateji
Wrap	Nesneyi karşı kenardan çıkaran strateji
Bounce	Nesneyi sınırdan geri sektiren strateji
Izgara Hareketi	Piksel yerine hücre bazlı ayırık hareket sistemi

16.4.4 Önemli Komutlar

Tablo 16.3: Bölüm 4 – Önemli PyGame komutları

Komut	İşlev
<code>pygame.event.get()</code>	Olay kuyruğundaki tüm olayları liste olarak al
<code>pygame.event.peek()</code>	Belirli olay türünün kuyrukta olup olmadığını kontrol et
<code>pygame.event.post()</code>	Kuyruğa özel olay ekle
<code>pygame.key.get_pressed()</code>	Tüm tuşların basılı durumunu tutan sözlük
<code>pygame.key.get_mods()</code>	Değiştirici tuşların (Shift, Ctrl) durumu
<code>pygame.mouse.get_pos()</code>	Farenin ekrandaki (x, y) konumu
<code>pygame.mouse.get_pressed()</code>	Fare düğmelerinin basılı durumu
<code>saat.tick(FPS)</code>	FPS sınırla ve geçen süreyi (ms) döndür

16.4.5 Sıradaki Adım

Bölüm 5’te **Görseller ve Sprite Temelleri** konusuna geçeceğiz. Artık ekranda basit şekiller çizmek yerine `pygame.image.load()` ile gerçek görsel dosyaları yükleyecek, `pygame.sprite.Sprite` sınıfıyla oyun nesnelerini nesne yönelimli olarak tasarlayacak ve Sprite gruplarıyla birden fazla nesneyi verimli şekilde yöneteceğiz. Uzay Kaçışı projemiz de Bölüm 5’te başlayacak.

● Bilgi: Pratik Yap

Yılan Oyunu'nu temel aldığın ek özellikler dene:

- Skor arttıkça yılanın hızını kademeli olarak artır
- Ekranı sabit engeller (duvar blokları) yerleştir
- Farklı renkte özel yemler ekle (mavi yem = 3 puan)
- Yılanın gövdesine gradyan renk efekti uygula
- En yüksek skoru dosyaya kaydedip bir sonraki oyunda göster

Bu egzersizler hem Python becerilerini hem de oyun tasarımı düşünceni geliştirecektir.

17

Görsel Dosyaları

“Bir oyun, görselleri olmadan sadece mantıktan ibarettir. Görseller, oyuna ruh katar.”

— JOHN CARMACK

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- PNG, JPG ve BMP gibi görsel dosya türlerinin farkları ve alpha kanalı kavramı
- `pygame.image.load()` ile görsel dosyaları yükleme ve dosya yolu yönetimi
- `convert()` ve `convert_alpha()` ile performans optimizasyonu
- `blit()` ile görselleri ekrana çizme ve katman sırası (z-order)

17.1 Görsel Dosya Türleri

Önceki bölümlerde geometrik şekillerle ekrana çizim yaptın: Bölüm 12’de dik-dörtgenler, daireler ve çizgiler çizerek renkli sahneler oluşturdu n. Ancak gerçek bir oyunda karakterler basit şekillerden ibaret değildir. Bir uzay gemisi, bir düşman robot veya bir orman arka planı düşün – bunların hepsi görsel dosyalarıdır. Tıpkı bir ressamın tuvale fırçayla resim yapması gibi, sen de hazır görselleri oyun penceresine yerleştireceksin.

Bu konu boyunca farklı görsel formatlarının özelliklerini, hangisinin ne zaman tercih edilmesi gerektiğini ve oyun geliştirmede neden belirli formatların standart haline geldiğini öğreneceksin.

17.1.1 Temel Görsel Formatları

Bilgisayarda görseller farklı formatlarda saklanır. Her formatın kendine özgü avantajları ve dezavantajları vardır. Oyun geliştirirken doğru formatı seçmek hem görsel kaliteni hem de performansını doğrudan etkiler.

■ Tanım: Raster Görsel

Raster (bitmap) görseller, renk bilgisi taşıyan piksellerden oluşur. Her piksel kırmızı (R), yeşil (G) ve mavi (B) kanallarının birleşiminden bir renk üretir. Bazı formatlarda dördüncü bir kanal olan *alpha* (A) kanalı da bulunur ve bu kanal pikselin şeffaflık derecesini belirler.

Oyunlarda karşılaşılabilecek üç temel görsel formatını inceleyelim. Aşağıdaki tabloda bu formatların önemli özelliklerini yan yana görebilirsin:

Tablo 17.1: Oyunlarda kullanılan temel görsel formatları

Özellik	PNG	JPG (JPEG)	BMP
Sıkıştırma	Kayıpsız	Kayıplı	Yok
Alpha Kanalı	Var	Yok	Sınırlı
Dosya Boyutu	Orta	Küçük	Çok Büyük
Kalite Kaybı	Yok	Var	Yok
Oyun Kullanımı	Sprite, UI	Arka plan fotoğrafları	Nadiren

PNG (Portable Network Graphics), kayıpsız sıkıştırma kullanır. Bu ne demek? Bir görseli PNG olarak kaydettiğinde ve tekrar açtığında tek bir piksel bile değişmez. Ayrıca alpha kanalı desteği sayesinde piksellerin şeffaflık derecesini ayarlayabilirsin. Bu iki özellik PNG'yi oyun geliştirmede en çok tercih edilen format yapar.

JPG (JPEG), kayıplı sıkıştırma kullanır. Fotoğraflar için idealdir çünkü insan gözünün fark edemeyeceği detayları atarak dosya boyutunu küçültür. Ancak her kaydedişte kalite biraz daha düşer ve alpha kanalı desteklemez. Bu nedenle sprite görselleri için uygun değildir. Oyunlarda genellikle sadece fotoğrafik arka planlar için kullanılır.

BMP (Bitmap), sıkıştırma yapmaz. Her pikselin renk bilgisini olduğu gibi saklar. Bu yüzden dosya boyutu çok büyüktür: 800×600 piksellik bir BMP dosyası yaklaşık 1.4 MB yer kaplarken, aynı görsel PNG olarak yalnızca birkaç yüz kilobyte olabilir. Günümüzde oyun geliştirmede BMP tercih edilmez.

17.1.2 Alpha Kanalı ve Şeffaflık

Alpha kanalı, bir görselin en kritik özelliklerinden biridir. Düşün ki bir uzay gemisi çizdin ve bu gemiyi oyun ekranına yerleştireceksin. Geminin etrafındaki boşluk ne olacak? Eğer alpha kanalı yoksa geminin etrafında kare şeklinde beyaz veya siyah bir kutu görürsün – bu da oyunun görsel kalitesini ciddi şekilde düşürür.

■ Tanım: Alpha Kanalı

Alpha kanalı, bir pikselin şeffaflık derecesini 0 ile 255 arasında belirleyen ek bir renk kanalıdır. 0 tamamen şeffaf (görünmez), 255 tamamen opak (katı) anlamına gelir. Ara değerler yarı saydam efektler oluşturur. RGBA formatında her piksel dört değerle temsil edilir: (R, G, B, A).

Alpha kanalının gücünü bir örnekle anlatalım. Diyelim ki bir patlama efekti görselin var. Patlamanın merkezinde parlak sarı tonlar var (alpha = 255, tamamen opak). Kenarlarına doğru ise alpha değeri kademeli olarak azalır ve en dışta 0 olur. Bu sayede patlama efekti arka planla doğal bir şekilde kaynaşır, keskin kenarlar oluşmaz.

● Bilgi: Neden Oyunlarda PNG?

Oyunlarda PNG tercih edilmesinin iki temel nedeni vardır. Birincisi, kayıpsız sıkıştırma sayesinde piksel sanatı (pixel art) ve sprite görselleri bozulmadan saklanır. İkincisi, alpha kanalı desteği sayesinde karakter spritelerinde arka plan şeffaflığı sağlanır. Modern oyun motorlarının neredeyse tamamı PNG formatını birincil görsel formatı olarak kullanır.

17.1.3 Kenney.nl: Ücretsiz Oyun Varlıkları

Profesyonel görünümlü bir oyun yapmak için çizim uzmanı olman gerekmez. İnternette özgürce kullanabileceğin binlerce oyun görseli mevcut. Bu kaynaklardan en popüler ve kalitelisi **Kenney.nl** sitesidir.

Kenney.nl, [CC0](https://creativecommons.org/licenses/by/4.0/) (Creative Commons Zero) lisansı ile yayınlanmış binlerce oyun varlığı sunar. CC0 lisansı ne demek? Bu görselleri ticari veya kişisel projelerinde özgürce kullanabilirsin, kaynak göstermek bile zorunlu değildir (yine de göstermek nezaket kuralıdır). Sitede 295'ten fazla varlık paketi bulunur ve bunlar sprite, arka plan, UI elemanları, ses efektleri gibi kategorilere ayrılmıştır.

💡 İpucu: Kenney.nl'den Görsel İndirme

Kitap boyunca kullanacağımız görselleri indirmek için kenney.nl/assets adresini ziyaret et. "Space Shooter Redux" paketini indir ve [assets/images/](https://kenney.nl/assets/images/) klasörüne yerleştir. Bu pakette uzay gemisi, meteor, lazer ve arka plan görselleri bulunur. Tüm görseller PNG formatında ve alpha kanalıdır.

Bu ünitenin devamında ve ilerleyen bölümlerde Kenney.nl'den indirdiğin görselleri kullanarak oyununa görsel zenginlik katacaksın. İlk adım olarak bir sonraki konuda bu görselleri PyGame ile nasıl yükleyeceğini öğreneceksin.

17.2 Görsel Yükleme

Görsel dosya türlerini ve alpha kanalı kavramını öğrendin. Şimdi bu görselleri PyGame ile nasıl yükleyeceğini keşfetme zamanı. PyGame'in `pygame.image` modülü, disk üzerindeki görsel dosyalarını belleğe yükleyerek oyununda kullanmanı sağlar.

17.2.1 `pygame.image.load()` Fonksiyonu

PyGame'de bir görsel dosyasını yüklemek için `pygame.image.load()` fonksiyonunu kullanırsın. Bu fonksiyon parametre olarak dosya yolunu alır ve geriye bir *Surface* nesnesi döndürür. Surface, PyGame'in temel çizim yüzeyidir – bir boyama tuvali gibi düşünebilirsin.

Python – Temel Görsel Yükleme

```
1 # Dosyadan görsel yükle
2 gemi_gorseli = pygame.image.load("gemi.png")
```

Bu tek satır, `gemi.png` dosyasını okur ve bellekte bir Surface nesnesi oluşturur. Ancak burada dikkat edilmesi gereken önemli bir nokta var: dosya yolu. Eğer Python dosyanı farklı bir dizinden çalıştırırsan `gemi.png` bulunamayabilir.

⚠ Dikkat: Dosya Yolu Tehlikesi

`pygame.image.load("gemi.png")` komutu görseli **çalışma dizinine** (current working directory) göre arar. Eğer programını farklı bir dizinden çalıştırırsan dosya bulunamaz ve `FileNotFoundError` hatası alırsın. Bu sorunu çözmek için mutlaka `os.path` kullanarak göreceli yol oluştur.

17.2.2 Platform Bağımsız Dosya Yolu

Windows'ta dizin ayracı ters eğik çizgi (`\`), macOS ve Linux'ta ise düz eğik çizgidir (`/`). Kodunu her platformda çalışır hale getirmek için Python'un `os.path` modülünü kullanmalısın. `os.path.join()` fonksiyonu, işletim sistemine uygun şekilde dizin yollarını birleştirir.

Daha da önemlisi, `os.path.dirname(__file__)` ifadesi Python dosyanın bulunduğu dizinin yolunu verir. Bu sayede programını hangi dizinden çalıştırırsan çalıştır, görsel dosyaları her zaman doğru yerde aranır.

Python – Güvenli Dosya Yolu Oluşturma

```
1 import os
2
3 # Bu Python dosyasının bulunduğu dizin
4 DOSYA_DIZINI = os.path.dirname(os.path.abspath(__file__))
5
6 # Görsel klasörünün yolu
```



```

7  GORSEL_DIZINI = os.path.join(DOSYA_DIZINI, "assets",
   ↪  "images")
8
9  # Gorseli yukle
10 gemi_yolu = os.path.join(GORSEL_DIZINI, "gemi.png")
11 gemi_gorseli = pygame.image.load(gemi_yolu)

```

Bu yaklaşımla dosya yolu sorunlarından tamamen kurtulursun. Kodun Windows, macOS ve Linux'ta aynı şekilde çalışır.

17.2.3 Hata Yakalama ve Fallback Stratejisi

Peki ya görsel dosyası bulunamazsa? Belki kullanıcı assets klasörünü silmiştir, belki dosya adını yanlış yazmışsındır. Profesyonel bir oyun geliştiricisi olarak bu durumlara hazırlıklı olmalısın. Hata yakalama ile programın çökmesini önleyebilir, yerine geçici bir görsel (placeholder) gösterebilirsin.

■ Tanım: Fallback (Yedek) Stratejisi

Fallback, bir işlem başarısız olduğunda devreye giren yedek plan demektir. Görsel yüklemede fallback, dosya bulunamadığında renkli bir dikdörtgen oluşturarak oyunun çalışmaya devam etmesini sağlar. Bu strateji geliştirme sürecinde eksik görselleri hızlıca fark etmeni de kolaylaştırır.

Kitap boyunca kullanacağımız `gorsel_yukle()` yardımcı fonksiyonunu tanıyalım:

Python – gorsel_yukle() Yardımcı Fonksiyonu

```

1  def gorsel_yukle(dosya_adi, boyut=None):
2      """Gorseli yukle, bulunamazsa placeholder olustur."""
3      dosya_yolu = os.path.join(GORSEL_DIZINI, dosya_adi)
4      try:
5          gorsel =
   ↪  pygame.image.load(dosya_yolu).convert_alpha()
6          if boyut:
7              gorsel = pygame.transform.scale(gorsel, boyut)
8          return gorsel
9      except (FileNotFoundError, pygame.error) as hata:
10         print(f"[UYARI] Gorsel yuklenemedi: {dosya_adi}")
11         print(f"  Hata: {hata}")
12         # Fallback: magenta placeholder
13         yer_tutucu = pygame.Surface(boyut or (64, 64),
   ↪  pygame.SRCALPHA)
14         yer_tutucu.fill((255, 0, 255, 180))
15         return yer_tutucu
16

```

Bu fonksiyon iki kritik şey yapar. Birincisi, `try-except` bloğu ile hem `FileNotFoundError` (dosya yok) hem de `pygame.error` (bozuk dosya) hatalarını yakalar. İkincisi, hata durumunda magenta renkli (parlak pembe) bir dikdörtgen oluşturur. Ma-

genta rengi özellikle seçilmiştir çünkü oyun sahnesinde hemen göze çarpar ve sana “burada bir görsel eksik” mesajını verir.

● Bilgi: Desteklenen Görsel Formatları

PyGame şu görsel formatlarını destekler: PNG, JPG (JPEG), BMP, GIF (animasyonsuz), TGA, TIFF, PCX, XPM, PBM/PGM/PPM ve WebP. SDL_image kütüphanesi kuruluysa SVG desteği de eklenebilir. Ancak oyun geliştirmede pratikte PNG ve JPG dışında bir formata ihtiyaç duymazsın.

Kod örneklerinin tamamını [kod_ornekleri/bolum05/unite1/](#) dizininde bulabilirsin. Bir sonraki konuda yüklenen görsellerin performansını artıran `convert()` ve `convert\_alpha()` fonksiyonlarını inceleyeceğiz.

17.3 Convert ve Alpha Kanalı

Önceki konuda görselleri `pygame.image.load()` ile yüklemeyi öğrendin. Ancak bir görseli yüklemek tek başına yeterli değil – performans için bir adım daha atman gerekiyor. Bu konuda `convert()` ve `convert\_alpha()` fonksiyonlarının ne işe yaradığını, neden bu kadar önemli olduklarını ve ne zaman hangisini kullanacağını öğreneceksin.

17.3.1 Piksel Formatı Dönüşümü

Her görsel dosyası kendi piksel formatında saklanır. PNG dosyasının piksel formatı, JPG’ninkinden farklıdır. Oyun penceren ise `pygame.display.set_mode()` ile oluşturulduğunda kendi piksel formatına sahip olur. Peki bu neden önemli?

Bir görseli ekrana çizdiğinde (blit işlemi), PyGame her pikselin rengini hesaplar. Eğer görselin piksel formatı ile ekranın piksel formatı farklıysa, PyGame her çizim işleminde her piksel için format dönüşümü yapmak zorunda kalır. Bu da ciddi performans kaybına neden olur.

■ Tanım: Fonksiyonu

`convert()` fonksiyonu, bir Surface nesnesinin piksel formatını ekranın piksel formatına dönüştürür. Bu dönüşüm bir kez yapılır ve sonrasında her blit işlemi çok daha hızlı gerçekleşir. Alpha kanalı olmayan görseller için kullanılır.

Python – `convert()` Kullanımı

```
1 # Alpha kanali olmayan gorsel (ornegin JPG arka plan)
2 arkaplan = pygame.image.load("arkaplan.jpg").convert()
```

`convert()` çağrısı görselin piksel formatını ekranın formatına eşitler. Bu sayede blit işlemi sırasında ek dönüşüm gerekmez ve çizim hızlanır.

17.3.2 `convert\_alpha()` Farkı

Peki alpha kanalı olan görsellerde ne yapacaksınız? `convert()` kullandığında alpha kanalı kaybolur. İşte burada `convert\_alpha()` devreye girer.

■ Tanım:

Fonksiyonu

`convert\_alpha()` fonksiyonu, `convert()` ile aynı piksel formatı dönüşümünü yapar ancak alpha kanalını korur. Şeffaflık bilgisi içeren tüm görseller (sprite, UI elemanları, efektler) için bu fonksiyon kullanılmalıdır.

Python – `convert\_alpha()` Kullanımı

```
1 # Alpha kanallı gorsel (PNG sprite)
2 gemi = pygame.image.load("gemi.png").convert_alpha()
3
4 # Karsilastirma:
5 # convert()      -> alpha kanali kaybolur, seffaflik
   ↳ bozulur
6 # convert_alpha() -> alpha kanali korunur, seffaflik
   ↳ calisir
```

⚠ Dikkat: `convert()` Olmadan Yavaşlık

Eğer ne `convert()` ne de `convert\_alpha()` çağırırsan, PyGame her blit işleminde piksel formatı dönüşümü yapar. Onlarca sprite içeren bir sahnede bu, kare hızını (FPS) ciddi şekilde düşürebilir. Her zaman yükleme sonrasında bu fonksiyonlardan birini çağır.

17.3.3 Performans Farkı

Performans farkını somut bir örnekle görelim. Ekranda 100 adet sprite çizdiğini düşün. Her biri 64×64 piksel boyutunda ve her karede çiziliyor. 60 FPS hedefiyle saniyede $100 \times 60 = 6000$ blit işlemi yapılır.

Format dönüşümü yapılmamış bir görselde her blit işlemi yaklaşık 2–3 kat daha uzun sürer. 6000 blit işleminin her biri fazladan mikrosaniyeler alırsa, toplam gecikme milisaniyelere ulaşır ve kare hızın düşer. `convert\_alpha()` ile bu ek maliyet sıfıra iner.

17.3.4 `set\_colorkey()` ile Sahte Şeffaflık

Bazı eski görsel dosyaları alpha kanalı içermez ama yine de şeffaflık gerektirir. Bu durumda `set\_colorkey()` fonksiyonu devreye girer. Bu fonksiyon belirli bir rengi “şeffaf” olarak işaretler. O renkteki tüm pikseller çizilmez.

Python – set_colorkey() Kullanımı

```

1  # BMP dosyasında alpha kanali yok
2  eski_sprite = pygame.image.load("eski_gemi.bmp").convert()
3
4  # Siyah rengi seffaf yap
5  eski_sprite.set_colorkey((0, 0, 0))
6
7  # Artık siyah pikseller çizilmez

```

Bu yöntem basit ama sınırlıdır: sadece tek bir renk şeffaf yapılabilir ve yarı saydamlık (semi-transparency) desteklenmez. Modern oyunlarda alpha kanallı PNG kullanmak her zaman daha iyi bir tercihtir.

17.3.5 Ne Zaman Hangisini Kullan?

Hangi dönüşüm fonksiyonunu kullanacağına karar vermek için aşağıdaki tabloyu referans alabilirsin:

Tablo 17.2: Görsel dönüşüm fonksiyonları karşılaştırması

Durum	Fonksiyon	Örnek
JPG arka plan	<code>convert()</code>	<code>arkaplan.jpg</code>
PNG sprite (alpha)	<code>convert_alpha()</code>	<code>gemi.png</code>
PNG UI elemanı	<code>convert_alpha()</code>	<code>buton.png</code>
BMP (eski format)	<code>convert() + set_colorkey()</code>	<code>eski.bmp</code>
Efekt (yarı saydam)	<code>convert_alpha()</code>	<code>patlama.png</code>

Bu akışı görsel olarak incelediğimizde daha net anlayacağız.

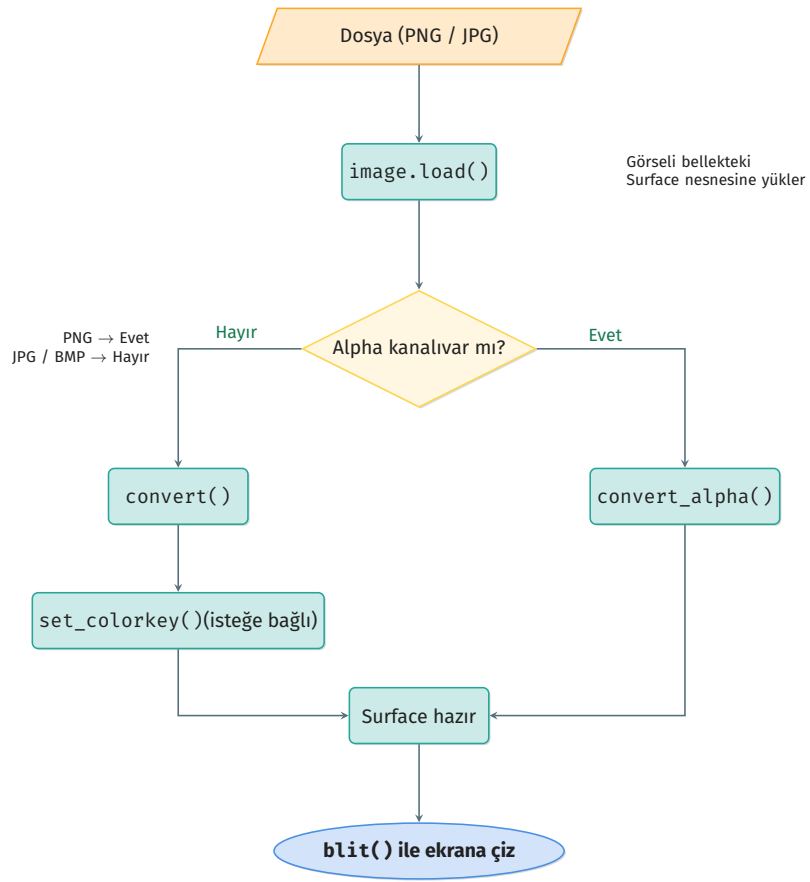
Şekil 17.1’de görüldüğü gibi süreç dosyadan görseli yüklemekle başlar, ardından formatına göre uygun dönüşüm fonksiyonu çağrılır ve son olarak görsel ekrana çizilmeye hazır hale gelir. Bu üç adımlı süreç – yükle, dönüştür, çiz – oyun geliştirmede standart bir kalıptır.

İpucu: Altın Kural

Görseli yüklediğin anda `convert()` veya `convert\_alpha()` çağır. Bunu oyun döngüsü başlamadan önce, başlangıç aşamasında yap. Oyun döngüsü içinde görsel yükleme ve dönüştürme yapma – bu her karede gereksiz iş demektir.

17.4 Görselleri Ekrana Çizme

Görselleri yüklemeyi ve performans için dönüştürmeyi öğrendin. Şimdi en heyecanlı kısma geçiyoruz: bu görselleri oyun penceresine çizme. Tıpkı bir sahne tasarımcısının dekorları sahneye yerleştirmesi gibi, sen de görselleri ekranın istediğin yerine yerleştireceksin.



Şekil 17.1: Görsel yükleme ve dönüştürme akış şeması

17.4.1 blit() Fonksiyonu

PyGame’de bir Surface’i başka bir Surface üzerine çizmek için `blit()` fonksiyonunu kullanırsın. “Blit” kelimesi “Block Image Transfer” (blok görsel aktarımı) ifadesinin kısaltmasıdır ve piksel verilerini bir yüzeyden diğerine kopyalama işlemini tanımlar.

Python – blit() Temel Kullanımı

```
1 # Gorseli ekranin (100, 50) konumuna ciz
2 ekran.blit(gemi_gorseli, (100, 50))
```

`blit()` fonksiyonu üç parametre alabilir. Birincisi çizilecek Surface (kaynak görsel), ikincisi çizim konumu (sol üst köşenin x, y koordinatları) ve isteğe bağlı üçüncü parametre ise kaynak görselin hangi bölgesinin çizileceğini belirleyen bir `Rect` nesnesidir.

Python – blit() ile Bölgesel Çizim

```
1 # Kaynak gorselin sadece bir parcasini ciz
2 # area parametresi: (x, y, genislik, yukseklik)
3 ekran.blit(sprite_atlas, (200, 100), area=(0, 0, 64, 64))
```

Bu üçüncü parametre özellikle *sprite atlas* (tek büyük görselde birden fazla sprite) kullanırken işe yarar. İlerleyen bölümlerde sprite sheet kavramını detaylı olarak inceleyeceğiz.

17.4.2 Katman Sırası: Z-Order

Bir oyun sahnesinde birden fazla görsel üst üste çizilir. Bölüm 12’de öğrendiğin gibi, ekrana son çizilen görsel en üstte görünür. Bu kavrama *z-order* (derinlik sırası) denir.

Bir tiyatro sahnesini düşün: en arkada gökyüzü dekoru var, onun önünde dağlar, daha önde ağaçlar ve en önde oyuncular. Oyun sahnesinde de aynı mantık geçerli. Önce arka planı çiz, sonra uzak nesneleri, daha sonra yakın nesneleri ve en son oyuncu karakterini çiz.

Python – Z-Order: Katmanlı Çizim

```
1 # 1. Arka plan (en arkada)
2 ekran.blit(arkaplan, (0, 0))
3
4 # 2. Uzak nesneler
5 ekran.blit(yildizlar, (0, 0))
6
7 # 3. Dusman gemileri
8 for dusman in dusmanlar:
9     ekran.blit(dusman_gorseli, dusman.konum)
10
11 # 4. Oyuncu gemisi (en onde)
12 ekran.blit(gemi_gorseli, (oyuncu_x, oyuncu_y))
```

```

13
14 # 5. UI elemanlari (her sey in ustunde)
15 ekran.blit(skor_paneli, (10, 10))

```

Dikkat edersen çizim sırası önemlidir. Eğer oyuncu gemisini arka plandan önce çizersen, arka plan geminin üzerine kapatır ve gemiyi göremezsin.

● Bilgi: Çizim Sırası Kuralı

Ekrana çizim yaparken şu sırayı takip et: arka plan → uzak nesneler → yakın nesneler → oyuncu → UI elemanları. Bu sırayı bozarsan görseller birbirini kapatır ve sahne beklediğin gibi görünmez.

17.4.3 Arka Plan Görseli Yerleştirme

Arka plan görseli genellikle tüm ekranı kaplar. Eğer görselin boyutu pencere boyutundan farklıysa, görseli ölçeklendirmen gerekir. Konu 17.2’de tanıttığımız `gorsel_yukle()` fonksiyonunun `boyut` parametresi tam da bu iş içindir.

Python – Tam Ekran Arka Plan

```

1 # Arka planı pencere boyutunda yukle
2 arkaplan = gorsel_yukle("arkaplan.png",
3                       boyut=(GENISLIK, YUKSEKLIK))
4
5 # Oyun dongusunda:
6 ekran.blit(arkaplan, (0, 0))

```

Arka plan görseli (0, 0) konumuna çizildiğinde sol üst köşeden başlayarak tüm ekranı kaplar. Bu yaklaşım `pygame.ekran.fill()` ile düz renk doldurmaktan çok daha etkileyici bir görünüm sağlar.

17.4.4 Birden Fazla Görsel: Tam Bir Sahne

Şimdi öğrendiğin her şeyi bir araya getirerek tam bir uzay sahnesi oluşturalım. Arka plan, oyuncu gemisi ve düşman gemileri ekrana çizilecek. Fare ile oyuncu gemisini hareket ettirebileceksin (Bölüm 15’de öğrendiğin bilgiler burada devreye giriyor).

Listing 17.1: Uzay Sahnesi – Çizim Sırası (Z-Order)

```

1 # =====
2 # CIZIM SIRASI (Z-ORDER)
3 # =====
4
5 # Katman 1: Arka plan (en arkada)
6 ekran.blit(arkaplan_gorseli, (0, 0))
7
8 # Katman 2: Dusman gemileri
9 for dusman in dusmanlar:

```

```

10         ekran.blit(dusman_gorseli,
11                     (int(dusman["x"]), int(dusman["y"])))
12
13     # Katman 3: Oyuncu gemisi (en onde)
14     ekran.blit(gemi_gorseli, (oyuncu_x, oyuncu_y))
15
16     # Katman 4: UI elemanlari (her sey in ustunde)
17     katman_yazi = font.render(
18         "Z-Order: Arkaplan -> Dusmanlar -> Oyuncu -> UI",
19         True, SARI
20     )
21     ekran.blit(katman_yazi, (10, YUKSEKLIK - 30))
22
23     konum_yazi = font.render(
24         f"Fare: ({fare_x}, {fare_y}) FPS: {saat.get_fps()}
25         :.0f)",
26         True, BEYAZ
27     )
28     ekran.blit(konum_yazi, (10, 10))
29
30     # Ekrani guncelle
31     pygame.display.flip()

```

Yukarıdaki kodda dikkat edilmesi gereken noktalar:

1. Çizim sırası: arka plan → düşmanlar → oyuncu gemisi → UI
2. En arkaya çizilen katman altta, en son çizilen üstte görünür
3. Fare konumu `pygame.mouse.get_pos()` ile alınır ve geminin merkezi fareye hizalanır

💡 İpucu: Tam Kaynak Kodu

Bu örneğin tam halini `kod\_ornekleri/bolum05/unite1/03\_uzay\_arkaplan.py` dosyasında bulabilirsin. Orada `gorsel_yukle()` fallback fonksiyonu, yıldız oluşturma ve düşman salınımı da yer alır.

💡 İpucu: Performans: Dirty Rect vs Full Screen Update

Her karede tüm ekranı yeniden çizmek basit ve güvenli bir yöntemdir. Ancak çok sayıda sprite içeren karmaşık sahnelerde sadece değişen bölgeleri güncellemek (*dirty rect* yöntemi) performansı artırır. `pygame.display.flip()` tüm ekranı günceller, `pygame.display.update(rect_listesi)` ise sadece belirtilen dikdörtgen bölgeleri günceller. Başlangıçta `flip()` kullanmak yeterlidir; optimizasyona ilerleyen bölümlerde değineceğiz.

17.5 Sonuç

Bu ünite de oyun geliştirmenin en temel yapı taşlarından birini öğrendin: görsel dosyaları. PNG, JPG ve BMP formatlarının farklarını, alpha kanalının şeffaflık sağ-

lamadaki rolünü, `pygame.image.load()` ile görselleri yüklemeyi, `convert()` ve `convert\alpha()` ile performansı artırmayı ve `blit()` ile görselleri ekrana çizmeyi gördün. Artık geometrik şekillerden gerçek oyun görsellerine geçiş yaptın.

17.5.1 Anahtar Kavramlar

Tablo 17.3: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
PNG	Kayıpsız sıkıştırma ve alpha kanalı destekleyen görsel formatı
Alpha Kanalı	Pikselin şeffaflık derecesini belirleyen kanal (0–255)
Surface	PyGame’in temel çizim yüzeyi; her görsel bir Surface’tir
<code>convert()</code>	Piksel formatını ekranla eşitler; alpha kanalı olmayan görseller için
<code>convert_alpha()</code>	Piksel formatını eşitler ve alpha kanalını korur
<code>blit()</code>	Bir Surface’i başka bir Surface üzerine çizen fonksiyon
Z-Order	Görsellerin çizim sırası; son çizilen en üstte görünür
Fallback	Dosya bulunamadığında devreye giren yedek plan
CC0	Creative Commons Zero; özgürce kullanılabilir lisans

17.5.2 Önemli Komutlar

Tablo 17.4: Ünite 1 – Temel PyGame komutları

Komut	Açıklama
<code>pygame.image.load(yol)</code>	Görsel dosyasını yükler ve Surface döndürür
<code>surface.convert()</code>	Piksel formatını ekranla eşitler
<code>surface.convert_alpha()</code>	Piksel formatını eşitler, alpha kanalını korur
<code>ekran.blit(surface, pos)</code>	Surface’i ekrana çizer
<code>surface.set_colorkey(renk)</code>	Belirtilen rengi şeffaf yapar
<code>os.path.join()</code>	Platform bağımsız dosya yolu oluşturur
<code>os.path.dirname(__file__)</code>	Dosyanın bulunduğu dizini verir

17.5.3 Sıradaki Adım

Bir sonraki ünite de görselleri manipüle etmeyi öğreneceksin. `pygame.transform.scale()` ile boyut değiştirme, `pygame.transform.rotate()` ile döndürme ve `pygame.transform.flip()` ile yansıtma işlemleri seni bekliyor. Bu araçlarla oyun karakterlerini farklı yön lere çevirebilecek, patlama efektleri için görselleri büyütebilecek ve daha dinamik sahneler oluşturabileceksin.

💡 İpucu: Pratik Yap

Bu ünite de öğrendiklerini pekiştirmek için şunu dene: Kenney.nl'den bir sprite paketi indir, birden fazla görseli yükle ve ekranda farklı konumlara yerleştir. `gorssel_yukle()` fonksiyonunu kendi projende kullan ve fallback mekanizmasını test etmek için bilerek yanlış bir dosya adı ver. Hata mesajını gözlemle ve placeholder görselin nasıl görüldüğünü incele.

*“Hayal gücün sınırın, dönüş-
türme araçların fırçan olsun.”*

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.transform.scale()` ve `pygame.transform.smoothscale()` ile boyut değiştirme
- `pygame.transform.rotate()` ile döndürme ve `rect` güncelleme sorunu
- `pygame.transform.flip()` ile yatay ve dikey çevirme
- Kenney uzay gemisi görseli ile hareket ve döndürme uygulaması

18.1 Boyut Değiştirme

Bir uzay oyunu geliştirdiğini düşün: ana gemi büyük, düşmanlar küçük, güç taşları daha da küçük. Hepsi için ayrı ayrı boyutlarda görsel mi hazırlaman gerekiyor? Elbette hayır! PyGame’in `pygame.transform` modülü sayesinde tek bir görseli istediğin boyuta getirebilirsin. Bu ünite de görselleri ölçeklendirme, döndürme ve çevirme işlemlerini öğreneceğiz. İlk durağımız boyut değiştirme.

18.1.1 `pygame.transform.scale()`

PyGame’de bir görselin boyutunu değiştirmenin en doğrudan yolu `pygame.transform.scale()` fonksiyonudur. Bu fonksiyon bir `Surface` ve hedef boyutu alır, geriye yeni boyutlarda bir `Surface` döndürür.

■ Tanım: transform.scale()

`pygame.transform.scale(surface, (genislik, yukseklik))` fonksiyonu, verilen **Surface** nesnesini belirtilen genişlik ve yüksekliğe ölçeklendirir. Orijinal **Surface** değişmez; fonksiyon yeni bir **Surface** döndürür.

Kullanımı son derece basit:

Python – scale() Temel Kullanım

```
1  # Orijinal görsel 128x128 piksel
2  orijinal = pygame.image.load("gemi.png").convert_alpha()
3
4  # Yarisina kucult: 64x64
5  kucuk = pygame.transform.scale(orijinal, (64, 64))
6
7  # İki katina buyut: 256x256
8  buyuk = pygame.transform.scale(orijinal, (256, 256))
```

Dikkat edersen `pygame.transform.scale()` her zaman yeni bir **Surface** oluşturur. Orijinal görsel hiç değişmez — bu önemli bir detay, birazdan neden önemli olduğunu göreceksin.

18.1.2 pygame.transform.smoothscale()

`pygame.transform.scale()` hızlı çalışır ama piksel piksel kopyalama yaptığı için özellikle küçültmelerde görsel kalitesi düşebilir. Kenarlar tırtıklı görünür, ince çizgiler kaybolabilir. Daha pürüzsüz bir sonuç istiyorsan `pygame.transform.smoothscale()` kullanabilirsin.

Python – smoothscale() Kullanımı

```
1  # Pürüzsüz ölçeklendirme (daha kaliteli ama daha yavaş)
2  kaliteli = pygame.transform.smoothscale(orijinal, (64,
3  ↪ 64))
```

Peki ne zaman hangisini kullanmalısın? `scale()` piksel art tarzı oyunlar için idealdir: keskin kenarlar korunur ve performans yüksektir. `smoothscale()` ise fotoğrafik veya yüksek çözünürlüklü görseller için daha uygun bir tercih. Ancak her karede `smoothscale()` çağırmak performansı olumsuz etkileyebilir; bu yüzden sonucu bir değişkende saklayıp tekrar tekrar kullanmak akıllıca olur.

● Bilgi: Performans Karşılaştırması

`smoothscale()` fonksiyonu, `scale()` fonksiyonuna göre yaklaşık 2–3 kat daha yavaş çalışır. Bu fark tek bir çağrıda fark edilmez; ancak her karede onlarca görseli ölçeklendiriyorsan `scale()` tercih etmelisin. En iyi strateji: görselleri oyun başlarken bir kez ölçeklendir ve sonucu sakla.

18.1.3 En-Boy Oranını (Aspect Ratio) Koruma

Bir görseli rastgele bir boyuta ölçeklendirdiğinde en-boy oranı bozulabilir. Örneğin 128×64 piksellik bir gemiyi 100×100 yaparsan gemi kareye dönüşür ve gerçekçi görünmez. Bunu önlemek için hedef boyutlardan birini (genişlik veya yükseklik) belirleyip diğerini oranla hesaplamalısın.

Python – En-Boy Oranı Koruma

```

1 def boyut_oranli(surface, hedef_genislik):
2     """Gorseli en-boy oranini koruyarak olceklendirir."""
3     orijinal_gen = surface.get_width()
4     orijinal_yuk = surface.get_height()
5     oran = hedef_genislik / orijinal_gen
6     yeni_yukseklk = int(orijinal_yuk * oran)
7     return pygame.transform.scale(surface,
8         (hedef_genislik, yeni_yukseklk))
9
10 # Kullanım: genisligi 200 yap, yukseklik otomatik
11     ↪ hesaplansin
12 olcekli_gemi = boyut_oranli(orijinal, 200)

```

Bu fonksiyon görselin genişliğini verdiğin değere ayarlar, yüksekliği ise aynı oranla hesaplar. Böylece gemi, yıldız veya karakter görsellerin hiçbir zaman ezilik görünmez.

18.1.4 Orijinali Saklamanın Önemi

Boyut değiştirme işleminde altın kural şudur: **orijinal görseli her zaman sakla**. Neden mi? Çünkü bir görseli küçültüp sonra tekrar büyüttüğünde piksel bilgisi kaybolur ve görsel bulanıklaşır. Her ölçeklendirme işlemini orijinalden yaparsan kalite kaybı olmaz.

Python – Dogru ve Yanlis Yaklasim

```

1 # YANLIS: Her seferinde onceki sonucu kullanmak
2 gorsel = pygame.image.load("gemi.png").convert_alpha()
3 gorsel = pygame.transform.scale(gorsel, (64, 64)) #
4     ↪ kucult
5 gorsel = pygame.transform.scale(gorsel, (256, 256)) #
6     ↪ buyut -> bulanik!
7
8 # DOGRU: Orijinalden olceklendirmek
9 orijinal = pygame.image.load("gemi.png").convert_alpha()
10 kucuk = pygame.transform.scale(orijinal, (64, 64))
11 buyuk = pygame.transform.scale(orijinal, (256, 256)) #
12     ↪ temiz!

```

⚠ Dikkat: Kalite Kaybı

Bir görseli küçültüp sonra büyütmek geri dönüşü olmayan kalite kaybına neden olur. Bu, fotoğraf düzenleme programlarındaki “pikselleşme” sorunuyla aynıdır. Her ölçeklendirmeyi **orijinal** görsel üzerinden yap.

18.1.5 Scale Sonrası **rect** Güncelleme

Bir görseli ölçeklendirdiğinde görselin boyutu değişir; ancak eski **rect** nesnesi hala orijinal boyutları taşır. Bu durumda çarpışma algılama ve konumlandırma hatalı çalışır. Çözüm basit: yeni görselden yeni bir **rect** oluştur ve konumu koru.

Python – rect Güncelleme

```
1 orijinal = pygame.image.load("gemi.png").convert_alpha()
2 orijinal_rect = orijinal.get_rect(center=(400, 300))
3
4 # Boyut degistir
5 buyuk = pygame.transform.scale(orijinal, (256, 256))
6
7 # rect'i guncelle, merkezi koru
8 buyuk_rect = buyuk.get_rect(center=orijinal_rect.center)
```

Burada `center=orijinal_rect.center` ifadesiyle yeni **rect**’in merkez noktasını eski konumda tutuyoruz. Böylece görsel aynı noktada kalır, sadece boyutu değişir.

Kod örneğimizde en-boy oranını koruyan ölçekleme fonksiyonunu ve `scale()` ile `smoothscale()` karşılaştırmasını görelim:

Listing 18.1: En-Boy Oranı Koruma

```
1 def boyut_oranli(surface, hedef_genislik):
2     """Gorseli en-boy oranini koruyarak olceklendirir.
3
4     Args:
5         surface: Olceklendirilecek Surface.
6         hedef_genislik: Istenen genislik (piksel).
7
8     Returns:
9         Oranli olceklenmis yeni Surface.
10    """
11    orijinal_gen = surface.get_width()
12    orijinal_yuk = surface.get_height()
13    oran = hedef_genislik / orijinal_gen
14    yeni_yukseklk = int(orijinal_yuk * oran)
15    return pygame.transform.scale(surface, (hedef_genislik,
16                                         yeni_yukseklk))
```

Bu fonksiyon orijinal genişlik/yükseklik oranını hesaplayarak yeni boyutu belirler. Böylece görsel bozulmaz. Uygulamanın `main()` fonksiyonunda `scale()` ve `smoothscale()` sonuçları yan yana karşılaştırılır:

Listing 18.2: scale() ve smoothscale() Karşılaştırması

```

1  # Farkli boyutlarda kopyalar olustur (orijinalden!)
2  kucuk = pygame.transform.scale(orijinal, (48, 48))
3  orta = pygame.transform.scale(orijinal, (96, 96))
4  buyuk = pygame.transform.scale(orijinal, (192, 192))
5
6  # smoothscale ile ayni boyutlar
7  kucuk_smooth = pygame.transform.smoothscale(orijinal, (48,
8  48))
9  orta_smooth = pygame.transform.smoothscale(orijinal, (96,
10 96))
11 buyuk_smooth = pygame.transform.smoothscale(orijinal, (192,
12 192))

```

💡 İpucu: Tam Kaynak Kodu

Bu örneğin tam halini `kod\_ornekleri/bolum05/unite2/01\_boyut\_degistirme.py` dosyasında bulabilirsin.

💡 İpucu: Profesyonel İpucu

Oyun başlangıcında tüm görselleri gerekli boyutlara ölçeklendir ve bir sözlükte sakla. Oyun döngüsü sırasında hiç `scale()` çağırmak zorunda kalmazsın ve performansın yüksek kalır.

18.2 Döndürme

Bir uzay gemisi düşün: ekrandaki fare imlecine doğru burnunu çevirmesi gerekiyor. Ya da dönen bir göktaşı, pervane gibi kendi eksenini etrafında sürekli dönüyor. Bu tür efektler için `pygame.transform.rotate()` fonksiyonunu kullanacağız. Ancak döndürme işlemi, boyut değiştirmeden farklı olarak beklenmedik bir sorun yaratır — ve bu sorunu bilmezsen saatlerce hata arayabilirsin.

18.2.1 `pygame.transform.rotate()` Kullanımı

`pygame.transform.rotate(surface, angle)` fonksiyonu bir `Surface` nesnesini belirtilen açı kadar döndürür. Açı **derece** cinsindendir ve **saat yönünün tersi** pozitif kabul edilir (matematik kurallarıyla aynı).

■ Tanım: transform.rotate()

`pygame.transform.rotate(surface, angle)` fonksiyonu, verilen `Surface` nesnesini `angle` derece döndürerek yeni bir `Surface` döndürür. Pozitif açılar saat yönünün tersine, negatif açılar saat yönünde döndürme yapar.

Python – rotate() Temel Kullanım

```

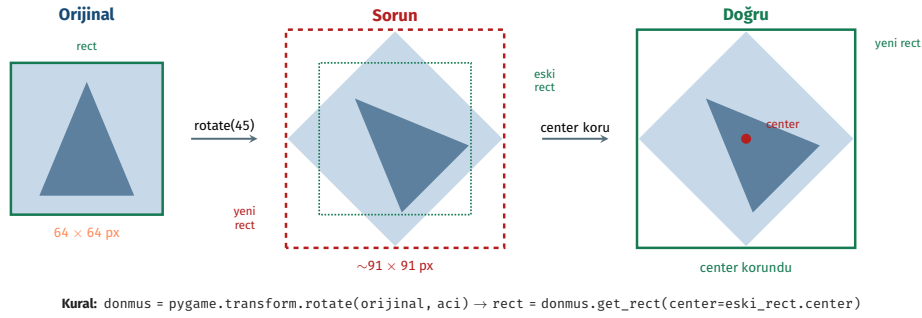
1 orijinal = pygame.image.load("gemi.png").convert_alpha()
2
3 # 45 derece saat yönünün tersine döndür
4 donmus = pygame.transform.rotate(orijinal, 45)
5
6 # 90 derece saat yönünde döndür (negatif açı)
7 donmus_saat = pygame.transform.rotate(orijinal, -90)

```

18.2.2 Döndürmede rect Boyut Büyümesi Sorunu

İşte döndürmenin en kritik noktası: bir **Surface** döndürüldüğünde PyGame yeni bir **Surface** oluşturur ve bu yeni yüzey, orijinalden **daha büyük** olur. Neden? Çünkü döndürülmüş görselin tüm pikselleri yeni bir dikdörtgene sığmalıdır ve bu dikdörtgen, döndürülmüş görselin köşelerini kapsayacak kadar geniştir.

Bu kavramı görsel olarak incelediğimizde daha net anlayacağız.



Şekil 18.1: Döndürmede rect boyut büyümesi ve doğru çözüm

Şekil 18.1’de görüldüğü gibi, soldaki 64×64 orijinal görseli 45 derece döndürdüğünde ortadaki gibi daha büyük bir **Surface** elde edersin. Kırmızı kesikli çizgi eski **rect** sınırını, yeşil düz çizgi ise döndürülmüş görselin gerçek **rect** sınırını gösterir. Eğer eski **rect**’i kullanmaya devam edersen, görsel merkeze göre kayar ve çarpışma algılama hatalı çalışır. Sağdaki doğru çözümde ise yeni **rect**’in merkezi orijinal merkezde tutulmuş.

⚠ Dikkat: Her Frame Orijinalden Döndür

Bir görseli her karede birkaç derece daha döndürmek (birikimli döndürme) kalite kaybına ve boyut şişmesine neden olur. Bunun yerine orijinal görseli sakla ve her karede orijinalden istenen toplam açı kadar döndür.

18.2.3 Doğru Döndürme Yöntemi

Doğru yaklaşım üç adımdan oluşur: orijinali sakla, her karede orijinalden döndür ve **rect.center**’ı koru.

Python – Doğru Döndürme Şablonu

```

1  # Baslarken
2  orijinal = pygame.image.load("gemi.png").convert_alpha()
3  aci = 0
4  konum = (400, 300)
5
6  # Her karede
7  aci += 2 # Her karede 2 derece don
8  donmus = pygame.transform.rotate(orijinal, aci)
9  donmus_rect = donmus.get_rect(center=konum)
10
11 # Ciz
12 ekran.blit(donmus, donmus_rect)

```

Bu kodda her karede `aci` değişkenini artırıyoruz ama döndürme işlemini her zaman `orijinal` görsel üzerinden yapıyoruz. Sonra yeni `rect`'in merkezini eski konuma sabitliyoruz. Böylece görsel yerinde döner, büyümeyi ve kalite kaybı yaşamaz.

18.2.4 Fareye Doğru Döndürme

Oyunlarda en yaygın döndürme senaryolarından biri, bir nesneyi fareye doğru yönlendirmektir. Uzak gemisi farenin bulunduğu noktaya burnunu çevirsin istiyorsan, Python'un `math.atan2()` fonksiyonuyla açıyı hesaplayabilirsin.

Python – Fareye Doğru Açı Hesaplama

```

1  import math
2
3  # Geminin konumu
4  gemi_x, gemi_y = 400, 300
5
6  # Fare konumu
7  fare_x, fare_y = pygame.mouse.get_pos()
8
9  # Radyan cinsinden açıyı hesapla
10 dx = fare_x - gemi_x
11 dy = fare_y - gemi_y
12 radyan = math.atan2(-dy, dx) # -dy cunku y eksenini ters
13
14 # Dereceye çevir
15 aci = math.degrees(radyan)

```

Burada `-dy` kullanmamızın nedeni PyGame'in koordinat sisteminin matematiksel koordinat sisteminden farklı olmasıdır. PyGame'de y eksenini aşağı doğru artar; matematik kurallarında ise yukarı doğru artar. Bu ters çevirmeyi yapmazsak gemi fareye ters yönde bakar.

● Bilgi: `math.atan2()` Nedir?

`math.atan2(y, x)` fonksiyonu, (0,0) noktasından (x,y) noktasına olan vektörün x eksenine yaptığı açığı **radyan** cinsinden döndürür. Standart `math.atan(y/x)` fonksiyonundan farklı olarak kadran bilgisini de doğru hesaplar ve sıfıra bölme hatası vermez.

Döndürme işlemlerini uygulamalı olarak gösteren kod örneğimize göz atalım. Aşağıda doğru döndürme yöntemini (orijinalden, merkez koruyarak) ve fareye doğru açı hesaplamasını görebilirsin:

Listing 18.3: Doğru Döndürme – Orijinalden, Center Kori

```

1  # Surekli dondurme: Her karede aci artir, orijinalden
   dondur
2  goktas_aci += 1.5 # Her karede 1.5 derece
3  if goktas_aci >= 360:
4      goktas_aci -= 360
5
6  # [DOGRU] Orijinalden dondur
7  donmus_goktas = pygame.transform.rotate(goktas_orijinal,
8                                           goktas_aci)
9  goktas_rect = donmus_goktas.get_rect(center=goktas_konum
10 )
11 ekran.blit(donmus_goktas, goktas_rect)
12
13 # Aci bilgisi
14 metin_ciz(ekran, bilgi_font,
            f"Aci: {goktas_aci:.0f} derece", 200, 330, GRI
            )

```

Bu kodda `goktas_orijinal`'den döndürülüyor ve `center` korunuyor. Yanlış yöntemde ise önceki sonuç üzerinden birikimli döndürme yapılır; birkaç saniye içinde görsel bozulur ve büyür.

Fareye doğru döndürme kısmı ise `atan2()` ile açı hesaplar:

Listing 18.4: Fareye Doğru Döndürme

```

1  # Fare konumu
2  fare_x, fare_y = pygame.mouse.get_pos()
3
4  # Fareye dogru aciyi hesapla
5  dx = fare_x - gemi_konum[0]
6  dy = fare_y - gemi_konum[1]
7  radyan = math.atan2(-dy, dx)
8  # -90 duzeltmesi: gorsel yukari bakiyor, atan2 saga
   bakar
9  gemi_aci = math.degrees(radyan) - 90
10
11 # Orijinalden dondur, center koru
12 donmus_gemi = pygame.transform.rotate(gemi_orijinal,
    gemi_aci)

```

```

13     gemi_rect = donmus_gemi.get_rect(center=gemi_konum)
14     ekran.blit(donmus_gemi, gemi_rect)

```

Her iki senaryonun da tam kaynak kodunu `kod\_ornekleri/bolum05/unite2/02\_dond` dosyasında bulabilirsin.

18.3 Çevirmeler

Bir platform oyunundaki karakteri düşün: sağa doğru yürürken sağa bakıyor, sola dönünce de sola bakması gerekiyor. Her yön için ayrı bir görsel mi hazırlaman lazım? Tabii ki hayır! PyGame'in `pygame.transform.flip()` fonksiyonu tek bir görselden ayna görüntüsünü oluşturmanı sağlar.

18.3.1 `pygame.transform.flip()` Kullanımı

■ Tanım: `transform.flip()`

`pygame.transform.flip(surface, xbool, ybool)` fonksiyonu, verilen `Surface` nesnesini yatay (`xbool=True`) veya dikey (`ybool=True`) ekseninde çevirir. Her iki parametre de `True` yapılabilir.

Python – `flip()` Kullanım Örnekleri

```

1  orijinal = pygame.image.load("gemi.png").convert_alpha()
2
3  # Yatay çevirme (ayna görüntüsü: sola/saga bakan)
4  yatay = pygame.transform.flip(orijinal, True, False)
5
6  # Dikey çevirme (yansima efekti: ust/alt)
7  dikey = pygame.transform.flip(orijinal, False, True)
8
9  # Her iki ekseninde çevirme (180 derece donmus gibi)
10 her_ikisi = pygame.transform.flip(orijinal, True, True)

```

Fonksiyonun ilk `bool` parametresi yatay (x eksenini boyunca) çevirmeyi, ikinci `bool` parametresi dikey (y eksenini boyunca) çevirmeyi kontrol eder.

18.3.2 Yatay Çevirme: Karakter Yön Değiştirme

Yatay çevirmenin en yaygın kullanım alanı karakter yön değiştirmedir. Bir karakter veya gemi sağa doğru bakan bir görselle tasarlanmışsa, sola hareket ettiğinde görseli yatay çevirmek yeterlidir.

Python – Yön Değiştirme Örneği

```

1  gemi_saga = pygame.image.load("gemi.png").convert_alpha()
2  gemi_sola = pygame.transform.flip(gemi_saga, True, False)
3

```

```

4  # Oyun dongusunda
5  yonu_saga = True
6
7  if tuslar[pygame.K_LEFT]:
8      yonu_saga = False
9  elif tuslar[pygame.K_RIGHT]:
10     yonu_saga = True
11
12  # Dogru gorseli sec
13  aktif_gorsel = gemi_saga if yonu_saga else gemi_sola
14  ekran.blit(aktif_gorsel, gemi_rect)

```

Burada `flip()` işlemini oyun başlarken bir kez yapıyoruz ve sonucu `gemi_sola` değişkeninde saklıyoruz. Oyun döngüsü sırasında sadece hangi görselin kullanılacağına karar veriyoruz — her karede yeniden çevirme yapmak gereksiz bir işlem yükü olur.

18.3.3 Dikey Çevirme: Yansıma Efekti

Dikey çevirme su yüzeyinde yansıma efekti oluşturmak için kullanılabilir. Orijinal görselin alt kısmına yarı saydam bir dikey çevrilmiş kopyasını yerleştirerek basit bir yansıma efekti elde edebilirsiniz.

Python – Yansıma Efekti

```

1  orijinal = pygame.image.load("gemi.png").convert_alpha()
2  yansima = pygame.transform.flip(orijinal, False, True)
3
4  # Yansimaya seffaflık uygula
5  yansima.set_alpha(100) # 0-255 arasi, 100 yari saydam
6
7  # Orijinali çiz
8  ekran.blit(orijinal, (x, y))
9
10 # Yansimayi hemen altina çiz
11 ekran.blit(yansima, (x, y + orijinal.get_height()))

```

18.3.4 Döndürme ile Çevirme Karşılaştırması

Döndürme ve çevirme benzer görünse de farklı işlemlerdir. Döndürme açısal bir dönüşümdür; çevirme ise ayna yansımasıdır. Bu farkı şöyle düşünebilirsiniz: bir kitabı masada çevirdiğinde yazılar ters görünür (ayna etkisi), ama kitabı döndürdüğünde yazılar hala aynı yönde okunur, sadece eğik durur.

Pratik açıdan bakarsak `flip()` fonksiyonu `rotate()` fonksiyonundan çok daha hızlıdır çünkü sadece piksel sıralarını tersine çevirir, yeni boyut hesaplaması yapmaz. Bir karakterin yönünü değiştirmek için `rotate(180)` yerine `flip(True, True)` kullanmak hem daha doğru sonuç verir hem de daha performanslıdır.


```

9     if boyut:
10         gorsel = pygame.transform.scale(gorsel, boyut)
11     return gorsel

```

18.4.3 Adım 2: Fareye Doğru Döndürme

Geminin fareye bakması için `math.atan2()` ile açıyı hesaplıyoruz. Bu hesaplamayı her karede yapıyoruz çünkü fare sürekli hareket edebilir. Döndürme işlemini orijinal görsel üzerinden yapıp `rect.center`'ı koruyoruz — önceki konuda öğrendiğimiz doğru döndürme yöntemi.

Python – Fareye Dönme Hesaplaması

```

1 import math
2
3 fare_x, fare_y = pygame.mouse.get_pos()
4 dx = fare_x - gemi_x
5 dy = fare_y - gemi_y
6 aci = math.degrees(math.atan2(-dy, dx)) - 90
7
8 donmus_gemi = pygame.transform.rotate(orijinal_gemi, aci)
9 gemi_rect = donmus_gemi.get_rect(center=(gemi_x, gemi_y))

```

Burada `- 90` düzeltmesi, gemimizin orijinal görselinin yukarı baktığını varsayar. Eğer görsel sağa bakıyorsa bu düzeltmeye gerek yoktur. Kendi görselinin yönüne göre bu değeri ayarlamalısın.

18.4.4 Adım 3: Klavye ile Hareket

Bölüm 4'te öğrendiğimiz `pygame.key.get_pressed()` fonksiyonuyla sürekli hareket sağlıyoruz. Her karede basılı tuşları kontrol edip geminin konumunu güncelliyoruz.

Python – Klavye Hareketi

```

1 HIZ = 5
2
3 tuslar = pygame.key.get_pressed()
4 if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
5     gemi_x -= HIZ
6 if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
7     gemi_x += HIZ
8 if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
9     gemi_y -= HIZ
10 if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
11     gemi_y += HIZ

```

Hem ok tuşlarını hem de WASD tuşlarını destekliyoruz. Bu, oyun geliştirmede yaygın bir pratiktir çünkü oyuncuların tercihleri farklı olabilir.

18.4.5 Adım 4: Ekran Sınırı Kontrolü

Geminin ekran dışına çıkmasını engellemek için `clamp()` benzeri bir kontrol yapıyoruz. Python'un `max()` ve `min()` fonksiyonlarıyla konumu sınırlar içinde tutabiliriz.

Python – Sınır Kontrolü

```
1 # Gemiye ekran sinirlarinda tut
2 gemi_x = max(0, min(gemi_x, GENISLIK))
3 gemi_y = max(0, min(gemi_y, YUKSEKLIK))
```

18.4.6 Tam Uygulama

Tüm bu adımları bir araya getiren uygulamanın ana döngüsüne bakalım. Klavye hareketi, fareye dönme ve sınır kontrolü bir arada çalışıyor:

Listing 18.5: Uzay Gemisi – Hareket ve Döndürme

```
1 # --- Klavye ile hareket ---
2 tuslar = pygame.key.get_pressed()
3
4 if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
5     gemi_x -= HIZ
6 if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
7     gemi_x += HIZ
8 if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
9     gemi_y -= HIZ
10 if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
11     gemi_y += HIZ
12
13 # --- Ekran siniri kontrolu ---
14 gemi_x = max(0, min(gemi_x, GENISLIK))
15 gemi_y = max(0, min(gemi_y, YUKSEKLIK))
16
17 # --- Fareye dogru dondurme ---
18 fare_x, fare_y = pygame.mouse.get_pos()
19 dx = fare_x - gemi_x
20 dy = fare_y - gemi_y
21 radyan = math.atan2(-dy, dx)
22 gemi_aci = math.degrees(radyan) - 90
23
24 # Orijinalden dondur, center koru
25 donmus_gemi = pygame.transform.rotate(gemi_orijinal,
26     gemi_aci)
27 gemi_rect = donmus_gemi.get_rect(
28     center=(int(gemi_x), int(gemi_y))
29 )
```

Bu programı çalıştırdığında bir uzay gemisi göreceksin. Ok tuşları veya WASD ile gemini hareket ettir, fareyi hareket ettirerek geminin yönünü değiştir. Gemi her zaman fare imlecine doğru bakar ve ekran sınırlarından çıkamaz. Bu mekanik,

ileride Bölüm 6'da düşman ve mermi sistemi ekleyeceğimiz Uzak Kaçış projesinin temelini oluşturacak.

💡 İpucu: Tam Kaynak Kodu

Bu uygulamanın tamamını (`gorssel_yukle()`, yıldız arka planı ve HUD dahil) `kod\_ornekleri/bolum05/unite2/03\_gemi\_hareket.py` dosyasında bulabilirsin.

● Bilgi: Kenney Görselleri

Bu uygulamada Kenney'nin (kenney.nl) ücretsiz uzay gemisi görsellerini kullanabilirsin. `gorssel_yukle()` fonksiyonu sayesinde görsel dosyası olmasa bile program çalışır; üçgen şeklinde bir yedek gemi gösterilir.

18.5 Sonuç

Bu üniteye PyGame'in `pygame.transform` modülünün üç temel aracını öğrendin: boyut değiştirme, döndürme ve çevirme. Bu araçlar tek bir görsel dosyasından birçok farklı görünüm elde etmeni sağlar.

18.5.1 Anahtar Kavramlar

Tablo 18.2: Ünite 2 – Görsel Manipülasyonu: Anahtar kavramlar

Kavram	Açıklama
<code>scale()</code>	Görseli belirtilen boyuta ölçeklendirir (hızlı, piksel kopyalama)
<code>smoothscale()</code>	Görseli pürüzsüz ölçeklendirir (kaliteli ama yavaş)
<code>rotate()</code>	Görseli derece cinsinden döndürür (saat yönü tersi pozitif)
<code>flip()</code>	Görseli yatay veya dikey ekseninde çevirir (ayna görüntüsü)
Aspect ratio	En-boy oranı; bozulmaması için orantılı ölçeklendirme gerekir
rect güncelleme	Dönüşüm sonrası <code>rect</code> 'in merkezini koruyarak yeniden oluşturma
Orijinal saklama	Her dönüşümü orijinal görsel üzerinden yaparak kalite kaybını önleme
<code>math.atan2()</code>	İki nokta arasındaki açıyı hesaplama (fareye dönme için)

18.5.2 Önemli Komutlar

Tablo 18.3: Ünite 2 – Görsel Manipülasyonu: PyGame komutları

Komut	Kullanım
<code>pygame.transform.scale(s, (w, h))</code>	Görseli (w, h) boyutuna ölçeklendir
<code>pygame.transform.smoothscale(s, (w, h))</code>	Pürüzsüz ölçeklendirme
<code>pygame.transform.rotate(s, angle)</code>	Görseli angle derece döndür
<code>pygame.transform.flip(s, x, y)</code>	Görseli x/y ekseninde çevir
<code>surface.get_rect(center=...)</code>	Belirtilen merkezle rect oluştur
<code>math.atan2(y, x)</code>	Vektör açısını radyan olarak hesapla
<code>math.degrees(rad)</code>	Radyanı dereceye çevir

18.5.3 Sıradaki Adım

Bir sonraki ünite de PyGame'in **Sprite** sınıfını tanıyacağız. **Sprite** sınıfı, bu ünite de yaptığımız görsel yükleme, konum takibi ve dönüşüm işlemlerini tek bir nesne altında toplar. Böylece onlarca oyun nesnesini kolayca yönetebileceksin.

💡 İpucu: Pratik Yap

Bu ünite deki kod örneklerini çalıştır ve şu denemeleri yap: geminin hızını değiştir, fareye dönme yerine ok tuşlarıyla dönmeyi dene, birden fazla gemiyi aynı anda ekrana çiz. Pratik yaparak bu kavramları pekiştireceksin.

“Sprite, oyun dünyasındaki her şeyin temelidir – karakterin, düşmanın, merminin, hatta bulutun bile.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Sprite kavramı ve oyun geliştirmede neden vazgeçilmez olduğu
- `pygame.sprite.Sprite` sınıfından kalıtım ile kendi Sprite'ını oluşturma
- `update()` metodunu override ederek hareket mantığı yazma
- Klavye girdileriyle kontrol edilen bir Oyuncu Sprite'ı geliştirme

19.1 Sprite Nedir?

Bir oyun sahnesini gözünde canlandır: uzay gemisi lazer ateşliyor, göktaşları sağa sola savrulurak yaklaşıyor, arka planda yıldızlar kayıyor. Bu sahnedeki her bir nesne — gemi, lazer, göktaşı, yıldız — aslında ekranda belirli bir konumda duran küçük bir görseldir. 1980'lerden bu yana oyun geliştiricileri bu küçük, hareketli görsellere *sprite* (“canlı parça”) adını veriyor.

■ Tanım: Sprite

Sprite, bir oyundaki görsel nesneyi temsil eden yapıdır. Her sprite üç temel bilgiyi bir arada taşır: **görsel** (nasıl görünür), **konum** (ekranın neresinde) ve **davranış** (nasıl hareket eder veya etkileşir). Bu üçlü birliği sayesinde bir sprite, oyun dünyasında bağımsız bir varlık gibi davranır.

Sprite kavramını bir sahne oyunundaki aktörlere benzetebilirsin. Her aktörün bir kostümü (görsel), sahnedeki bir konumu (koordinat) ve bir rolü (davranış) vardır. Yönetmen “Sahne!” dediğinde her aktör kendi rolüne göre hareket eder.

Oyun döngüsü de aynen böyle çalışır: her karede tüm sprite'lar güncellenir ve ekrana çizilir.

19.1.1 Sprite'sız Yaklaşım: Neden Yetmiyor?

Şimdiye kadar öğrendiğin bilgilerle bir uzay gemisini hareket ettirmek için şöyle bir yol izleyebilirdin:

Python – Sprite Kullanmadan Nesne Yönetimi

```

1  # Her nesne için ayrı degiskenler
2  gemi_gorsel =
   ↳ pygame.image.load("gemi.png").convert_alpha()
3  gemi_x = 400
4  gemi_y = 500
5  gemi_hiz = 5
6
7  # Dusmanlar
8  dusman1_gorsel =
   ↳ pygame.image.load("dusman.png").convert_alpha()
9  dusman1_x = 100
10 dusman1_y = 50
11
12 dusman2_gorsel =
   ↳ pygame.image.load("dusman.png").convert_alpha()
13 dusman2_x = 300
14 dusman2_y = 80

```

Dikkat edersen her nesne için ayrı değişkenler tanımlıyorsun: görsel, x koordinatı, y koordinatı, hız... İki düşman için bile kod karmaşıklıkmaya başladı. Peki 50 düşman olsa? 200 mermi olsa? Her biri için ayrı değişken mi oluşturacaksın?

Üstelik güncelleme kodunda da benzer bir tekrar var:

Python – Tekrarlayan Güncelleme Kodu

```

1  # Guncelleme
2  gemi_x += gemi_hiz
3  dusman1_y += 2
4  dusman2_y += 2
5
6  # Cizim
7  ekran.blit(gemi_gorsel, (gemi_x, gemi_y))
8  ekran.blit(dusman1_gorsel, (dusman1_x, dusman1_y))
9  ekran.blit(dusman2_gorsel, (dusman2_x, dusman2_y))

```

Bu yaklaşım küçük programlarda çalışır, ama gerçek bir oyun geliştirmede sürdürülebilir değildir. İşte tam bu noktada Bölüm 7'te öğrendiğin **sınıflar ve kalıtım** devreye giriyor.

19.1.2 Sprite Yaklaşımı: Görsel + Konum + Davranış

Bölüm 7’te sınıf kavramını öğrenirken, birbiriyle ilişkili verileri ve davranışları tek bir yapı altında toplamanın ne kadar güçlü olduğunu görmüştün. Sprite tam da bu prensibi oyun nesnelere uygular: görseli, konumu ve davranışı tek bir nesne içinde birleştirir.

PyGame, bu iş için hazır bir temel sınıf sunar: `pygame.sprite.Sprite`. Kendi oyun nesnelerini bu sınıftan **kalıtım** yoluyla türeterek oluşturursun. Böylece her nesne kendi verisini ve davranışını taşır, kod tekrarı ortadan kalkar ve yönetim kolaylaşır.

Sprite yaklaşımını kullandığında aynı senaryo şöyle görünür:

Python – Sprite ile Nesne Yönetimi

```

1 class Dusman(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         self.image = pygame.image.load("dusman.png").convert_alpha()
5         self.rect = self.image.get_rect(topleft=(x, y))
6         self.hiz = 2
7
8     def update(self):
9         self.rect.y += self.hiz
10
11 # 50 dusman olusturmak artık tek satirlik is
12 dusmanlar = [Dusman(x * 80, 50) for x in range(50)]

```

Farkı görüyor musun? Artık 50 düşman oluşturmak tek satırlık bir iş. Her düşman kendi görselini, konumunu ve hızını biliyor. Güncelleme mantığı `update()` metodunun içinde kapsüllenmiş durumda. Bu, Bölüm 7’te öğrendiğin *kapsülleme* (encapsulation) ilkesinin doğrudan oyun geliştirmeye uygulanmasıdır.

● Bilgi: Her Karakter Bir Sprite

Bir oyundaki her bağımsız nesne — oyuncu gemisi, düşman, mermi, güç artırıcı, patlama efekti — bir Sprite nesnesidir. Sprite yaklaşımı sayesinde her nesne kendi sorumluluğunu taşır ve oyun kodun düzenli kalır.

Sonraki konuda `pygame.sprite.Sprite` sınıfından kendi Sprite’ını nasıl oluşturacağını adım adım öğreneceksin.

19.2 Sprite Oluşturma

Bir önceki konuda Sprite kavramının ne olduğunu ve neden gerekli olduğunu öğrendin. Şimdi kolları sıvayıp kendi ilk Sprite’ını oluşturma zamanı. Tıpkı bir oyun karakterini tasarlamak gibi: önce görünüşünü belirleyecek, sonra onu sahneye koyacaksın.

PyGame’de Sprite oluşturmanın temel adımı, `pygame.sprite.Sprite` sınıfından **kalıtım** almaktır. Bölüm 7’te öğrendiğin **class** ve `super()` kavramlarını burada doğrudan kullanacaksın.

19.2.1 Temel Yapı: Kalıtım ve `super()`

Her özel Sprite sınıfı şu iskeletle başlar:

Python – Sprite Sınıfı İskeleti

```
1 class Nesnem(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__() # Üst sinifi baslatmayı
4         ↪ UNUTMA!
5         self.image = ... # Surface nesnesi (görünüm)
6         self.rect = ... # Rect nesnesi (konum + boyut)
```

Bu üç satır, bir Sprite oluşturmanın mutlak minimum gereksinimidir. Her birinin ne işe yaradığını tek tek inceleyelim.

■ Tanım: Sprite’ın Zorunlu Bileşenleri

Bir `pygame.sprite.Sprite` alt sınıfının düzgün çalışması için `__init__()` metodu içinde üç şey yapılmalıdır:

1. `super().__init__()` ile üst sınıf başlatılmalı
2. `self.image` özelliğine bir `Surface` atanmalı
3. `self.rect` özelliğine bir `Rect` atanmalı

Bu üçünden herhangi biri eksik olursa Sprite düzgün çalışmaz.

Birincisi: `super().__init__()` çağrısı. Bölüm 7’te öğrendiğin gibi, kalıtım aldığın üst sınıfın `__init__()` metodunu çağırmak **zorunludur**. Bu çağrı, PyGame’in Sprite yönetim sisteminin (grup ekleme, çıkarma, canlılık kontrolü gibi) düzgün çalışmasını sağlar.

İkincisi: `self.image` özelliği. Bu, Sprite’ın ekranda nasıl görüneceğini belirleyen `Surface` nesnesidir. Bölüm 17’te öğrendiğin `pygame.image.load()` ile bir dosyadan yükleyebileceğin gibi, `pygame.Surface()` ile boş bir yüzey oluşturup renkle de doldurabilirsin.

Üçüncüsü: `self.rect` özelliği. Bu, Sprite’ın ekrandaki konumunu ve boyutunu belirleyen `Rect` (dikdörtgen) nesnesidir. `self.image.get_rect()` çağrısı, görselin boyutlarına uygun bir `Rect` döndürür.

▲ Dikkat: image ve rect Olmadan Sprite Çalışmaz

`self.image` veya `self.rect` tanımlamadan bir Sprite’ı gruba ekleyip çizmeye çalışırsan `AttributeError` hatası alırsın. PyGame’in çizim sistemi bu iki özelliğe doğrudan erişir.

19.2.2 Minimal Sprite: Renkli Kare

Şimdi en basit Sprite'ı oluşturalım: ekranda duran kırmızı bir kare. Görsel dosya bile yüklemeye gerek yok; boş bir `Surface` oluşturup renkle dolduracağız.

Python – Minimal Sprite – Renkli Kare

```

1 class KirmiziKare(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         # 40x40 piksel boyutunda bos bir Surface olustur
5         self.image = pygame.Surface((40, 40))
6         self.image.fill((220, 30, 30)) # Kirmizi ile
7         ↪ doldur
8         # Rect'i belirtilen konuma yerlestir
9         self.rect = self.image.get_rect(center=(x, y))

```

Bu kodu satır satır inceleyelim:

1. `class KirmiziKare(pygame.sprite.Sprite)` — Kendi sınıfımız, `Sprite`'tan kalıtım alıyor.
2. `super().__init__()` — Üst sınıfın yapıcısını çağırıyoruz.
3. `pygame.Surface((40, 40))` — 40 x 40 boyutunda boş bir yüzey oluşturuyoruz.
4. `self.image.fill((220, 30, 30))` — Yüzeyi kırmızı renkle dolduruyoruz.
5. `self.image.get_rect(center=(x, y))` — Görselin boyutlarına uygun bir `Rect` oluşturup merkezini (x,y) noktasına yerleştiriyoruz.

● Bilgi: `get_rect()` Konum Parametreleri

`get_rect()` fonksiyonuna anahtar kelime argümanları geçirerek `Rect`'in konumunu doğrudan ayarlayabilirsin: `center`, `opleft`, `midbottom`, `midtop` gibi. Bu sayede `Rect`'i oluşturduktan sonra ayrıca konumlandırma yapman gerekmez.

19.2.3 Görsel Dosyası ile Sprite

Gerçek oyunlarda Sprite'lar genellikle dosyadan yüklenen görsellerle oluşturulur. Bölüm 17'te öğrendiğin `pygame.image.load()` ve `convert\_\alpha()` fonksiyonlarını burada doğrudan kullanacaksın.

Python – Görsel Dosyası ile Sprite

```

1 class UzayGemisi(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         # Gorseli yukle (alpha kanali destekli)
5         self.image = pygame.image.load(
6             "assets/images/playerShip1_blue.png"

```

```

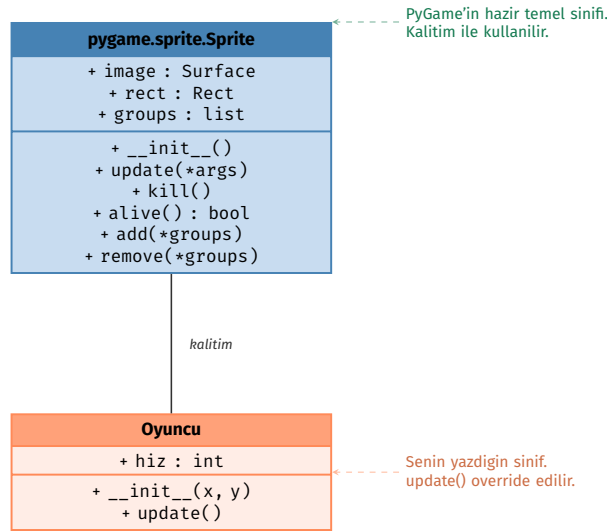
7         ).convert_alpha()
8         # Rect'i görselin boyutlarına göre oluştur
9         self.rect = self.image.get_rect(center=(x, y))

```

Bu örnekte Kenney asset paketinden bir uzay gemisi görseli kullanıyoruz. `convert\alpha()` çağırısı, görselin şeffaf arka planını koruyarak performanslı çizim sağlar — bunu Bölüm 17’te detaylıca öğrenmiştin.

19.2.4 Sprite Sınıfı Yapısını Görselleştirelim

Sprite sınıfının yapısını bir UML diyagramı ile daha net görebilirsin.



Şekil 19.1: pygame.sprite.Sprite sınıfı ve Oyuncu alt sınıfının UML diyagramı

Şekil 19.1’da görüldüğü gibi, üst kısımda PyGame’in `Sprite` sınıfı yer alıyor: `image`, `rect` ve `groups` özellikleri ile `__init__()`, `update()`, `kill()` gibi metotlara sahip. Altında ise kendi yazdığın `Oyuncu` sınıfı, kalıtım oku ile `Sprite`’a bağlı. `Oyuncu` kendi ek özelliklerini (`hiz`) ve override edilmiş `update()` metodunu tanımlıyor.

19.2.5 Sprite’ı Ekrana Çizme

Sprite’ını oluşturmak yetmez; onu ekrana da çizmen gerekir. Henüz Sprite gruplarını öğrenmedik (bir sonraki ünite de göreceksin), bu yüzden şimdilik `blit()` ile doğrudan çizeceğiz:

Python – Sprite’ı blit ile Çizme

```

1     # Sprite oluştur
2     gemi = UzayGemisi(400, 500)
3
4     # Oyun dongusu icinde çiz
5     ekran.blit(gemi.image, gemi.rect)

```

`blit()` fonksiyonuna `gemi.image` (ne çizilecek) ve `gemi.rect` (nereye çizilecek) geçiriyoruz. `Rect` nesnesini konum olarak geçirebilirsin; PyGame otomatik olarak `rect.topleft` koordinatlarını kullanır.

💡 İpucu: Tam Kaynak Kodu

Bu konudaki kavramları bir arada gösteren çalışan örneği kod\`_ornekleri/bolum05/unite3/01/_ilk/_sprite.py` dosyasında bulabilirsin.

19.3 Update Metodu

Bir Sprite oluşturdun, ekrana çizdin — ama o sadece orada öylece duruyor. Bir oyun karakteri durağan olamaz; hareket etmeli, tepki vermeli, belki de yok olmalı. İşte Sprite'lara hayat veren mekanizma `update()` metodudur.

■ Tanım: `update()` Metodu

`update()`, `pygame.sprite.Sprite` sınıfının varsayılan olarak boş bırakılmış bir metodudur. Alt sınıfta bu metodu **override** (ezme) ederek Sprite'ın her karede nasıl davranacağını tanımlarsın. Hareket, animasyon, sınır kontrolü, yok olma gibi tüm kare-bazlı mantık bu metodun içine yazılır.

Bunu bir tiyatro provasına benzetebilirsin: `__init__()` aktörün kostümünü giymesi ve sahneye çıkmasıdır; `update()` ise her perdenin açılışında aktörün rolünü oynamasıdır. Yönetmen (oyun döngüsü) her karede “`update()`!” der ve tüm aktörler (Sprite'lar) bir adım ilerler.

19.3.1 Basit Hareket: `rect` Güncelleme

Bir Sprite'ı hareket ettirmek, aslında `self.rect` özelliğinin koordinatlarını değiştirmekten ibarettir. Hatırlarsan, `Rect` nesnesi Sprite'ın ekrandaki konumunu tutar. Bu koordinatları `update()` içinde her karede değiştirdiğinde Sprite hareket eder.

Python – Basit Hareket – Sağa Kayma

```
1 class HareketliKare(pygame.sprite.Sprite):
2     def __init__(self, x, y, renk):
3         super().__init__()
4         self.image = pygame.Surface((30, 30))
5         self.image.fill(renk)
6         self.rect = self.image.get_rect(center=(x, y))
7         self.hiz = 3
8
9     def update(self):
10        self.rect.x += self.hiz
```


Dikkat edersen `update()` metodu oldukça basit: her çağrıldığında `self.rect.x` değerini `self.hiz` kadar artırıyor. Bu, Sprite'ın her karede 3 sağa kaymasını sağlar.

19.3.2 Sınır Kontrolü: Ekranın Dışına Çıkma

Yukarıdaki Sprite sonsuza kadar sağa gidecek ve bir süre sonra ekranın dışına çıkacak. Gerçek bir oyunda bunu kontrol etmen gerekir. İki temel strateji var:

Birinci strateji, Sprite'ı ekranın diğer tarafından tekrar başlatmaktır — bu *sarmalama* (wrapping) olarak adlandırılır. Aşağıdaki örnekte Sprite ekranın sağ kenarından çıkınca sol kenardan geri giriyor:

Python – Sarmalama – Ekranı Sar

```
1 def update(self):
2     self.rect.x += self.hiz
3     # Ekranın sagindan ciktiysa soldan geri gir
4     if self.rect.left > 800:
5         self.rect.right = 0
```

İkinci strateji ise `kill()` metodu ile Sprite'ı tamamen yok etmektir. Bu, mermiler veya geçici efektler için idealdir.

19.3.3 `kill()` ile Sprite'ı Yok Etme

`kill()` metodu, Sprite'ı ait olduğu tüm gruplardan çıkarır. Grup kavramını bir sonraki ünite de detaylıca göreceksin, ama şimdilik şunu bil: `kill()` çağrıldığında Sprite artık güncellenmez ve çizilmez. Oyundan “silinmiş” olur.

Python – Ekran Dışına Çıkınca Yok Et

```
1 def update(self):
2     self.rect.y += self.hiz
3     # Ekranın altindan ciktiysa yok et
4     if self.rect.top > 600:
5         self.kill()
```

Bu örnek, aşağı doğru hareket eden bir Sprite'ı (düşman veya göktaşı gibi) ekranın alt kenarını geçtiğinde yok ediyor. Böylece artık gereksiz nesneler bellekte birikmez.

● Bilgi: `alive()` ile Kontrol

Bir Sprite'ın hala aktif olup olmadığını `self.alive()` metodu ile kontrol edebilirsin. `kill()` çağrıldıktan sonra `alive()` değeri `False` döner.

19.3.4 Delta Time ile Kare Bağımsız Hareket

Bölüm 16'te öğrendiğin *delta time* kavramını Sprite'ların `update()` metoduna da uygulayabilirsin. Bunun için `update()` metoduna bir parametre geçmen yeterli-

dir:

Python – Delta Time ile Hareket

```

1  class HareketliDusman(pygame.sprite.Sprite):
2      def __init__(self, x, y):
3          super().__init__()
4          self.image = pygame.Surface((30, 30))
5          self.image.fill((200, 50, 50))
6          self.rect = self.image.get_rect(center=(x, y))
7          self.hiz = 150 # Piksel/saniye
8
9      def update(self, dt):
10         self.rect.y += self.hiz * dt
11         if self.rect.top > 600:
12             self.kill()

```

Burada `self.hiz` artık “kare başına piksel” değil, “saniye başına piksel” birimindedir. `dt` değeri (`saat.tick(60) / 1000.0` ile elde edilir) kare süresini saniye cinsinden verir. Böylece oyun 30 FPS’de de 120 FPS’de de aynı hızda çalışır.

⚠ Dikkat: Delta Time Uyumluluğu

`update()` metoduna `dt` parametresi geçiriyorsan, bu Sprite’ı kullanan her yerde `update(dt)` şeklinde çağırmayı unutma. Parametresiz çağırırsan `TypeError` hatası alırsın.

19.3.5 Otomatik Hareket Eden Düşman Örneği

Tüm kavramları bir araya getiren bir örnek yazalım: ekranın üstünden rastgele konumda beliren ve aşağı doğru hareket eden bir düşman Sprite’ı.

Python – Otomatik Düşman Sprite’ı

```

1  import random
2
3  class Dusman(pygame.sprite.Sprite):
4      def __init__(self):
5          super().__init__()
6          self.image = pygame.Surface((40, 40))
7          self.image.fill((200, 50, 50))
8          # Rastgele yatay konum, ekranın üstünde baslat
9          self.rect = self.image.get_rect(
10              center=(random.randint(40, 760), -20)
11          )
12          self.hiz = random.uniform(2, 5)
13
14      def update(self):
15          self.rect.y += self.hiz
16          if self.rect.top > 600:

```

17

`self.kill()`

Bu düşman Sprite'ı her oluşturulduğunda ekranın üst kısmında rastgele bir yatay konumda başlar ve rastgele bir hızla aşağı iner. Ekranın altından çıktığında `kill()` ile kendini yok eder. Farklı hızlar sayesinde her düşman biraz farklı davranır — oyuna çeşitlilik katmanın en kolay yollarından biri.

💡 İpucu: Tam Kaynak Kodu

Birden fazla hareketli Sprite'ı bir arada gösteren çalışan örneği `kod\_ornekleri/bolum05/unite3/02\_hareketli\_sprite.py` dosyasında bulabilirsin.

19.4 Oyuncu Sprite

Şimdiye kadar oluşturduğun Sprite'lar hep kendi başlarına hareket ediyordu: sağa kayan kareler, aşağı inen düşmanlar. Peki ya oyuncunun kontrol ettiği bir Sprite? Uzay Kaçışı projemizin başrolü olan oyuncu gemisini bir Sprite sınıfı olarak oluşturmanın zamanı geldi.

Bir oyuncu Sprite'ı, diğer Sprite'lardan iki temel farkla ayrılır. Birincisi, hareket mantığı kullanıcı girdisine bağlıdır — otomatik değil, klavye komutlarıyla yönetilir. İkincisi, ekran sınırları içinde kalması gerekir; ekranın dışına çıkmamalıdır.

19.4.1 Oyuncu Sınıfı Yapısı

Bölüm 14'de öğrendiğin `pygame.key.get_pressed()` fonksiyonunu hatırla: bu fonksiyon her çağrıldığında tüm tuşların anlık durumunu bir liste olarak döndürüyordu. Sürekli hareket için tam da bize gereken araç buydu. Şimdi bunu bir Sprite'ın `update()` metodunda kullanalım.

Python – Oyuncu Sprite Sınıfı

```
1 class Oyuncu(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         # Gorseli yukle (yoksa fallback: mavi kare)
5         self.image = gorsel_yukle(
6             "assets/images/playerShip1_blue.png", (50, 40)
7         )
8         self.rect = self.image.get_rect(center=(x, y))
9         self.hiz = 5
10
11     def update(self):
12         tuslar = pygame.key.get_pressed()
13         if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
14             self.rect.x -= self.hiz
15         if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
```

```

16         self.rect.x += self.hiz
17     if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
18         self.rect.y -= self.hiz
19     if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
20         self.rect.y += self.hiz

```

Dikkat edilmesi gereken noktalar:

1. `\_\_init\_\_()` içinde görseli yüklüyor ve başlangıç konumunu belirliyoruz.
2. `self.hiz = 5` sabit hız değeri, her karede 5 hareket sağlar.
3. `update()` içinde `pygame.key.get_pressed()` ile anlık tuş durumunu okuyoruz.
4. Hem ok tuşları hem WASD tuşları destekleniyor — `or` ile iki alternatifi birleştiriyoruz.
5. `if` (elif değil!) kullandığımıza dikkat et: böylece çapraz hareket mümkün oluyor (aynı anda sağ ve yukarı).

● Bilgi: `if` vs `elif` Farkı

Hareket kodunda `elif` yerine `if` kullanıyoruz. Neden? Çünkü `elif` ile sadece ilk sağlanan koşul çalışır ve çapraz hareket engellenir. `if` ile her yön bağımsız kontrol edilir, böylece oyuncu aynı anda hem sağa hem yukarı gidebilir.

19.4.2 Ekran Sınırı: `clamp_ip()`

Yukarıdaki kodda bir sorun var: oyuncu tuşlara bastıkça ekranın dışına çıkabilir. Bunu engellemenin en zarif yolu `clamp_ip()` metodudur.

■ Tanım: `clamp_ip()` Metodu

`Rect.clamp_ip(baska_rect)`, bir `Rect` nesnesini verilen dikdörtgenin sınırları içinde tutar. Eğer `Rect` sınırların dışına çıkmışsa, en yakın geçerli konuma geri çekilir. `_ip` soneki “in place” (yerinde) anlamına gelir; yeni bir `Rect` döndürmek yerine mevcut olanı değiştirir.

Bunu bir futbol sahasındaki oyuncuya benzetebilirsin: ne kadar koşarsa koşsun, çizgileri aşamaz. `clamp_ip()` tam da bu işi yapar: Sprite'ı belirtilen alanın içinde tutar.

Python – Ekran Sınırı ile Oyuncu

```

1 class Oyuncu(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         self.image = gorsel_yukle(
5             "assets/images/playerShip1_blue.png", (50, 40)

```

```

6         )
7         self.rect = self.image.get_rect(center=(x, y))
8         self.hiz = 5
9
10        def update(self):
11            tuslar = pygame.key.get_pressed()
12            if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
13                self.rect.x -= self.hiz
14            if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
15                self.rect.x += self.hiz
16            if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
17                self.rect.y -= self.hiz
18            if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
19                self.rect.y += self.hiz
20
21            # Ekran sinirlarini asma
22            ekran_rect =
23                ↪ pygame.display.get_surface().get_rect()
24            self.rect.clamp_ip(ekran_rect)

```

Son iki satıra dikkat et: önce `pygame.display.get_surface()` ile ekran yüzeyini, ardından `get_rect()` ile ekranın boyutlarını temsil eden bir `Rect` elde ediyoruz. `clamp_ip()` çağrısı, oyuncunun `rect`'ini bu sınırların dışına çıkmaktan alıkoyuyor.

19.4.3 Hız Sabiti ve Akıcı Hareket

`self.hiz = 5` değeri oyuncunun her karede kaç piksel hareket edeceğini belirler. Bu değeri değiştirerek hız ayarlaması yapabilirsin. Daha yavaş gemi istersen `3`, daha hızlı istersen `8` dene.

Ancak çapraz hareket sırasında bir sorun var: hem sağa hem yukarı gidiyorsan, her karede yatay ve dikey ekseninde ayrı ayrı 5 hareket edersin. Pisagor teoremine göre gerçek hareket mesafesi $\sqrt{5^2 + 5^2} \approx 7.07$ piksel olur — düz hareketin yaklaşık 1.41 katı. Bu fark özellikle yüksek hızlarda hissedilir.

Bu sorunu çözmek için vektör normalizasyonu kullanılabilir; ancak bu ileri bir konu olduğundan şimdilik mevcut yaklaşım yeterlidir. İleriki bölümlerde `pygame.math.Vector2` ile bu sorunu ele alacağız.

💡 İpucu: Tam Kaynak Kodu

Oyuncu gemisi Sprite'ının tam çalışan halini `kod\ornekler/bolum05/unite3/03\oyuncu\_gemisi.py` dosyasında bulabilirsin. Kodu çalıştırıp ok tuşları veya WASD ile gemiyi kontrol et.

19.5 Sonuç

Bu ünite de oyun geliştirmenin en temel yapı taşlarından biri olan `pygame.sprite.Sprite` sınıfını öğrendin. Sprite kavramının ne olduğundan başlayarak kendi Sprite'ını oluşturmayı, `update()` metodu ile hareket mantığı yazmayı ve klavye girdileriyle kontrol edilen bir oyuncu Sprite'ı geliştirmeyi adım adım gerçekleştirdin.

19.5.1 Anahtar Kavramlar

Tablo 19.1: Ünite 3 – Sprite Sınıfı: Anahtar Kavramlar

Kavram	Açıklama
Sprite	Görsel, konum ve davranışı bir arada tutan oyun nesnesi
<code>self.image</code>	Sprite'ın görünümünü belirleyen <code>Surface</code> nesnesi
<code>self.rect</code>	Sprite'ın konumunu ve boyutunu belirleyen <code>Rect</code> nesnesi
<code>update()</code>	Her karede çağrılan, Sprite'ın davranışını tanımlayan metot
<code>kill()</code>	Sprite'ı tüm gruplardan çıkararak yok eden metot
<code>clamp_ip()</code>	<code>Rect</code> 'i belirtilen sınırlar içinde tutan metot
Kalıtım	<code>pygame.sprite.Sprite</code> 'tan türeterek özel Sprite oluşturma

19.5.2 Önemli Komutlar

Tablo 19.2: Ünite 3 – Sprite Sınıfı: Önemli Komutlar

Komut	Açıklama
<code>class</code>	Sprite alt sınıfı tanımlama
<code>X(pygame.sprite.Sprite)</code>	
<code>super().__init__()</code>	Üst sınıf yapıcısını çağırma
<code>self.image = Surface((w, h))</code>	Boş görsel oluşturma
<code>self.rect =</code>	Rect oluşturma
<code>self.image.get_rect()</code>	
<code>self.rect.x += hiz</code>	Yatay hareket
<code>self.rect.clamp_ip(rect)</code>	Sınır kontrolü
<code>self.kill()</code>	Sprite'ı yok etme
<code>self.alive()</code>	Sprite aktif mi kontrolü

19.5.3 Sıradaki Adım

Şu anda her Sprite'ı ayrı ayrı güncelliyorsun ve `blit()` ile tek tek çiziyorsun. Düşmanlar, mermiler, güç artırıcılar derken onlarca Sprite'ı yönetmek zorlaşmaya başlayacak. Bir sonraki üniteye `pygame.sprite.Group` sınıfını öğrenecek ve tüm Sprite'larını gruplar halinde yönetmeyi keşfedeceksin: tek bir `update()` çağrısıyla tüm Sprite'ları güncellemek, tek bir `draw()` çağrısıyla hepsini çizmek. Uzak Kaçış projesinin temeli orada şekillenecek.

💡 İpucu: Pratik Yap

Bu üniteye kavramları pekiştirmek için şu denemeleri yap: KırmızıKare Sprite'ının rengini ve boyutunu değiştir, farklı hızlarda düşmanlar oluştur, oyuncu gemisine hız artırma (Shift ile turbo) özelliği ekle. Kod örneklerini `kod\_ornekleri/bolum05/unite3/` dizininde bulabilirsin.

Sprite Grupları

“Tek bir Sprite güçlüdür, ama bir grup Sprite bir ordu gibidir – birlikte yönetildiklerinde gerçek güç ortaya çıkar.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.sprite.Group` sınıfı ve Sprite’ları toplu yönetmenin avantajları
- `add()`, `remove()`, `has()`, `kill()` ile grup işlemleri
- `group.draw()` ve `group.update()` ile ana döngüyü sadeleştirme
- **Uzay Kaçışı v1**: Oyuncu gemisi ve kayan yıldız arka planı mini projesi

20.1 Sprite Grubu Nedir?

Bölüm 19’de tek tek Sprite’lar oluşturdu: oyuncu gemisini, hareketli bir kutuyu, hatta ok tuşlarıyla kontrol edilen bir karakteri bile kodladın. Peki bir düşün: ekranda 50 tane düşman olsa, 100 tane yıldız kaysa, 20 tane mermi uça... Her birini ayrı ayrı `blit()` ile çizip, ayrı ayrı `update()` ile güncellemek ne kadar zahmetli olurdu? İşte tam bu noktada *Sprite Grubu* devreye giriyor.

Bir ordu komutanı düşün. Komutan savaşta her bir askere tek tek “ileri yürü, şimdi ateş et, şimdi sağa dön” demez. Bunun yerine birliğe tek bir komut verir ve o birlik içindeki herkes komutu uygular. PyGame’deki `pygame.sprite.Group` sınıfı da tam olarak bu işi yapar: Sprite’larını bir gruba toplar ve tek bir komutla hepsini çizdirir veya güncelletsin.

■ Tanım: Sprite Group

`pygame.sprite.Group`, birden fazla Sprite nesnesini bir arada tutan koleksiyon sınıfıdır. Gruba eklenen tüm Sprite'lar tek bir `draw()` çağrısıyla ekrana çizilebilir ve tek bir `update()` çağrısıyla güncellenebilir. İç yapısı Python'un `set` (küme) veri yapısına benzer: aynı Sprite iki kez eklenemez ve sıralama garantisi yoktur.

20.1.1 Neden Gruplara İhtiyacımız Var?

Gruplama olmadan her Sprite'ı ayrı değişkende tutman ve her birini ayrı ayrı yönetmen gerekir. Diyelim ki ekranda 30 adet düşman var. Grupsuz yaklaşımda `dusman1`, `dusman2`, ..., `dusman30` gibi değişkenler tanımlaman veya bir Python listesi kullanıp her birini tek tek döngüyle çizmen gerekir. Üstelik bir düşman yok edildiğinde listeden çıkarman, bellek yönetimi yapman ve referansları temizlemen lazım.

`Group` tüm bu işleri otomatikleştirir. Ekleme, çıkarma, toplu çizim ve toplu güncelleme işlemlerini tek satırda halledersin. Dahası, bir Sprite'ın `kill()` metodunu çağırdığında o Sprite kendini ait olduğu **tüm gruplardan** otomatik olarak çıkarır. Bu sayede bellek yönetimi de büyük ölçüde kolaylaşır.

● Bilgi: Bir Sprite, Birden Fazla Gruba Ait Olabilir

Aynı Sprite aynı anda birden fazla grubun üyesi olabilir. Örneğin bir düşman Sprite'ı hem `tum_spritelar` grubunda (çizim için) hem de `dusmanlar` grubunda (çarpışma kontrolü için) bulunabilir. `kill()` çağrıldığında her iki gruptan da otomatik silinir.

20.1.2 Group'un İç Yapısı

Teknik açıdan `Group`, Python'daki küme (`set`) benzeri bir koleksiyondur. Bu yapı sayesinde aynı Sprite iki kez eklenmeye çalışılsa bile grup içinde yalnızca bir kez bulunur. Ekleme ve çıkarma işlemleri oldukça hızlıdır çünkü küme yapısı ortalama $O(1)$ karmaşıklıkta çalışır.

Bir `Group` oluşturmak çok basit:

Python – Group Oluşturma

```
1 import pygame
2
3 # Bos bir grup olustur
4 tum_spritelar = pygame.sprite.Group()
5
6 # Olusturma aninda Sprite'larla doldur
7 yildizlar = pygame.sprite.Group(yildiz1, yildiz2, yildiz3)
```

Gördüğün gibi `Group()` yapıcısına hiçbir argüman vermezsen boş bir grup elde edersin. İstersen oluşturma anında Sprite nesnelerini doğrudan parametre

olarak geçebilirsiniz. Bir sonraki konuda gruba eleman ekleme, çıkarma ve sorgulama işlemlerinin tamamını detaylıca inceleyeceğiz.

⚠ Dikkat: Group'a Sadece Sprite Eklenebilir

Group'a yalnızca `pygame.sprite.Sprite` sınıfından türetilmiş nesneler eklenebilir. Sıradan bir Python nesnesi veya sözlük eklemeye çalışırsan hata alırsın. Bu yüzden oyun nesnelerini her zaman Sprite alt sınıfı olarak tasarla.

20.2 Grup İşlemleri

Artık **Group**'un ne olduğunu ve neden ihtiyacımız olduğunu biliyorsun. Sıra geldi bu gruba Sprite eklemeye, çıkarmaya ve sorgulamaya. Tıpkı bir futbol takımının kadrosunu yönetmek gibi düşün: yeni oyuncu transferi (ekleme), oyuncu gönderme (çıkarma), kadroda olup olmadığını kontrol etme (sorgulama) ve tüm kadroyu görmek (listeleme) gibi işlemlerin hepsini **Group** tek bir arayüzle sunar.

20.2.1 Gruba Sprite Ekleme: `add()`

Bir veya birden fazla Sprite'ı gruba eklemek için `add()` metodunu kullanırsın:

Python – Gruba Sprite Ekleme

```
1 # Tek bir Sprite ekle
2 tum_spritelar.add(oyuncu)
3
4 # Birden fazla Sprite ekle
5 tum_spritelar.add(dusman1, dusman2, dusman3)
6
7 # Liste olarak da ekleyebilirsin
8 yeni_dusmanlar = [Dusman() for _ in range(5)]
9 tum_spritelar.add(*yeni_dusmanlar)
```

Zaten grupta olan bir Sprite'ı tekrar eklemeye çalışırsan hata almaz, çünkü grubun iç yapısı küme (`set`) mantığıyla çalışır ve tekrarları otomatik yoksayar.

20.2.2 Gruptan Sprite Çıkarma: `remove()`

Belirli bir Sprite'ı gruptan çıkarmak için `remove()` metodunu kullanırsın:

Python – Gruptan Sprite Çıkarma

```
1 # Tek bir Sprite çıkar
2 tum_spritelar.remove(dusman1)
3
4 # Birden fazla Sprite çıkar
5 tum_spritelar.remove(dusman2, dusman3)
```

Grupta olmayan bir Sprite'ı çıkarmaya çalışırsan hata almaz, sessizce geçer. Ancak dikkat et: `remove()` yalnızca belirtilen gruptan çıkarır. Sprite başka gruplara da aitse o gruplarda kalmaya devam eder.

20.2.3 Üyelik Kontrolü: `has()`

Bir Sprite'ın grubun üyesi olup olmadığını kontrol etmek için `has()` metodunu kullanırsın:

Python – Üyelik Kontrolü

```
1 if tum_spritelar.has(oyuncu):
2     print("Oyuncu hala grupta!")
3 else:
4     print("Oyuncu gruptan cikarilmis.")
```

Bu kontrol, oyuncunun hayatta olup olmadığını veya bir Sprite'ın hala aktif olup olmadığını sorgulamak için oldukça kullanışlıdır. Alternatif olarak Python'un `in` operatörünü de kullanabilirsin: `if oyuncu in tum_spritelar:` ifadesi aynı işi yapar.

20.2.4 Grubu Boşaltma: `empty()`

Gruptaki tüm Sprite'ları tek seferde çıkarmak için `empty()` metodunu kullanırsın:

Python – Grubu Bosaltma

```
1 # Tum Sprite'lari gruptan cikar
2 dusmanlar.empty()
3 print(len(dusmanlar)) # 0
```

`empty()` grubu temizler ama Sprite nesnelerini yok etmez. Sprite'lar başka gruplara aitse o gruplarda kalmaya devam eder. Oyunda “seviye sıfırlama” gibi durumlarda `empty()` oldukça kullanışlıdır.

20.2.5 Sprite Listesini Alma: `sprites()`

Gruptaki tüm Sprite'ların listesini almak için `sprites()` metodunu kullanırsın:

Python – Sprite Listesi

```
1 # Gruptaki tum Sprite'lari listele
2 for sprite in dusmanlar.sprites():
3     print(sprite.rect.topleft)
```

Aslında `Group` nesnesi doğrudan iterasyona izin verir, yani `for sprite in dusmanlar:` yazmak da aynı sonucu verir. Ancak `sprites()` metodu grubun o anki anlık kopyasını döndürdüğü için döngü içinde güvenli değişiklik yapmana olanak tanır.

20.2.6 Eleman Sayısı: `len()`

Python'un yerleşik `len()` fonksiyonu `Group` ile çalışır:

Python – Eleman Sayısı

```
1 print(f"Toplam sprite: {len(tum_spritelar)}")
2 print(f"Dusman sayisi: {len(dusmanlar)}")
```

20.2.7 Sprite'ı Tüm Gruplardan Çıkarma: `kill()`

En güçlü temizleme aracı `kill()` metodudur. Bu metod `Sprite` sınıfına ait olup çağrıldığında Sprite kendisini ait olduğu **tüm gruplardan** otomatik olarak çıkarır:

Python – `kill()` ile Otomatik Temizleme

```
1 # Dusman hem tum_spritelar hem dusmanlar grubunda
2 dusman = Dusman()
3 tum_spritelar.add(dusman)
4 dusmanlar.add(dusman)
5
6 # kill() her iki gruptan da siler
7 dusman.kill()
8 print(tum_spritelar.has(dusman)) # False
9 print(dusmanlar.has(dusman))    # False
```

⚠ Dikkat: `kill()` Sprite'ı Silmez, Gruplardan Çıkarır

`kill()` metodu Sprite nesnesini bellekten silmez, yalnızca tüm grup üyeliklerini sonlandırır. Eğer başka bir değişken hala o Sprite'a referans tutuyorsa nesne bellekte kalmaya devam eder. Python'un çöp toplayıcısı (*garbage collector*) referans kalmadığında nesneyi otomatik temizler.

Şimdi tüm bu işlemleri bir arada gösteren bir örneğe bakalım. Aşağıdaki `RenkliKare` sınıfı `Sprite` alt sınıfıdır ve gruba eklenerek toplu yönetilir:

Listing 20.1: RenkliKare Sprite Sınıfı

```
1 class RenkliKare(pygame.sprite.Sprite):
2     """Rastgele renkli, yatay hareket eden kare Sprite."""
3
4     def __init__(self, x, y, boyut=40):
5         super().__init__()
6         self.renk = random.choice(RENKLER)
7         self.image = pygame.Surface((boyut, boyut), pygame.
8             SRCALPHA)
9         pygame.draw.rect(self.image, self.renk, (0, 0, boyut,
10             boyut),
11             border_radius=5)
12         pygame.draw.rect(self.image, BEYAZ, (0, 0, boyut, boyut))
```

```

11         width=2, border_radius=5)
12     self.rect = self.image.get_rect(center=(x, y))
13     self.hiz_x = random.choice([-2, -1, 1, 2])
14     self.hiz_y = random.choice([-1, 0, 1])
15
16     def update(self):
17         """Her karede konumu guncelle ve sinir kontrolu yap."""
18         self.rect.x += self.hiz_x
19         self.rect.y += self.hiz_y
20
21         # Ekran sinirlarina carparsa yon degistir
22         if self.rect.left < 0 or self.rect.right > GENISLIK:
23             self.hiz_x *= -1
24         if self.rect.top < 0 or self.rect.bottom > YUKSEKLIK:
25             self.hiz_y *= -1
26
27         # Ekran icinde tut
28         self.rect.clamp_ip(pygame.Rect(0, 0, GENISLIK, YUKSEKLIK
29             ))

```

Olay döngüsünde Space ile yeni Sprite ekleniyor, R ile rastgele bir tanesi `kill()` ile siliniyor:

Listing 20.2: Dinamik Ekleme ve Silme

```

1     for olay in pygame.event.get():
2         if olay.type == pygame.QUIT:
3             calistir = False
4         elif olay.type == pygame.KEYDOWN:
5             if olay.key == pygame.K_ESCAPE:
6                 calistir = False
7
8         elif olay.key == pygame.K_SPACE:
9             # Yeni Sprite ekle
10            x = random.randint(50, GENISLIK - 50)
11            y = random.randint(50, YUKSEKLIK - 50)
12            yeni = RenkliKare(x, y)
13            tum_spritelar.add(yeni)
14            if random.random() > 0.5:
15                ozel_grup.add(yeni)
16
17        elif olay.key == pygame.K_r:
18            # Rastgele bir Sprite'i kill() ile sil
19            sprite_listesi = tum_spritelar.sprites()
20            if sprite_listesi:
21                secilen = random.choice(sprite_listesi)
22                secilen.kill() # Tum gruplardan cikar
23
24        elif olay.key == pygame.K_c:
25            # Ozel grubu bosalt
26            ozel_grup.empty()

```

Tam kaynak kodu `kod\_ornekleri/bolum05/unite4/01\_sprite\_grubu.py`

dosyasındadır. Bir sonraki konuda `draw()` ve `update()` metodlarının detaylarına ineceğiz.

20.3 Toplu Çizim ve Güncelleme

Önceki konuda `add()`, `remove()` ve `kill()` ile grup üyeliklerini yönetmeyi öğrendin. Şimdi grubun asıl gücünü keşfetme zamanı: tek bir çağrıyla gruptaki **tüm** Sprite'ları çizdirmek ve güncellemek. Bu iki metod – `draw()` ve `update()` – ana oyun döngünü inanılmaz ölçüde sadeleştirir.

20.3.1 `group.draw(surface)` – Toplu Çizim

`draw()` metodu, gruptaki her Sprite'ın `image` özelliğini `rect` konumuna göre belirtilen yüzeye çizer:

Python – Toplu Çizim

```
1 # Tek satırda tüm Sprite'ları çiz
2 tüm_spritelar.draw(ekran)
```

Bu tek satır, gruba kayıtlı 10, 50 veya 500 Sprite olsa bile hepsini ekrana çizer. Perde arkasında `draw()` şuna benzer bir işlem yapar:

Python – `draw()` Perde Arkası

```
1 # PyGame bunu senin için yapar:
2 for sprite in self.sprites():
3     surface.blit(sprite.image, sprite.rect)
```

Dikkat edersen `draw()` metodunun çalışması için her Sprite'ın `image` ve `rect` özelliklerine sahip olması şart. Bölüm 19'de Sprite alt sınıfı oluştururken bu iki özelliği her zaman tanımladığını hatırlayacaksın. İşte bu tasarım kararının sebebi buydu: Group ile uyumlu çalışmak.

● Bilgi: `draw()` Performans Avantajı

`draw()` tek tek `blit()` çağırmaktan daha verimli olabilir çünkü PyGame bu işlemi C seviyesinde optimize eder. Özellikle çok sayıda Sprite içeren oyunlarda bu fark hissedilir. Ayrıca `draw()` metodu arka planda değişen bölgeleri (*dirty rects*) takip edebilir, bu da ekranın yalnızca değişen kısımlarının güncellenmesini sağlar.

20.3.2 `group.update()` – Toplu Güncelleme

`update()` metodu, gruptaki her Sprite'ın kendi `update()` metodunu çağırır:

Python – Toplu Güncelleme

```

1 # Tek satırda tüm Sprite'ları güncelle
2 tum_spritelar.update()

```

Gruba kayıtlı her Sprite'ın `update()` metodu sırayla çalışır. Bu sayede her Sprite kendi hareket mantığını, animasyonunu veya durum kontrolünü bağımsız olarak gerçekleştirir. Ekstra argümanlar da geçebilirsiniz:

Python – `update()` ile Arguman Gecme

```

1 # dt değerini tüm Sprite'lara ilet
2 tum_spritelar.update(dt)

```

Bu durumda gruptaki her Sprite'ın `update(dt)` metodu çağrılır. Böylece delta time değerini her Sprite'a tek tek iletme yerine gruba bir kez verirsin, o da herkese dağıtır.

20.3.3 Ana Döngü Sadeleşmesi

Sprite gruplarının en büyük faydası ana döngüyü ne kadar sadeleştirdiğidir. Grupsuz ve gruplu yaklaşımı karşılaştıralım:

Python – Grupsuz Yaklaşım (Karmasik)

```

1 # Her Sprite'ı tek tek yönet
2 oyuncu.update(dt)
3 for dusman in dusman_listesi:
4     dusman.update(dt)
5 for yildiz in yildiz_listesi:
6     yildiz.update(dt)
7 for mermi in mermi_listesi:
8     mermi.update(dt)
9
10 ekran.fill(SIYAH)
11 ekran.blit(oyuncu.image, oyuncu.rect)
12 for dusman in dusman_listesi:
13     ekran.blit(dusman.image, dusman.rect)
14 for yildiz in yildiz_listesi:
15     ekran.blit(yildiz.image, yildiz.rect)
16 for mermi in mermi_listesi:
17     ekran.blit(mermi.image, mermi.rect)

```

Python – Gruplu Yaklaşım (Sade)

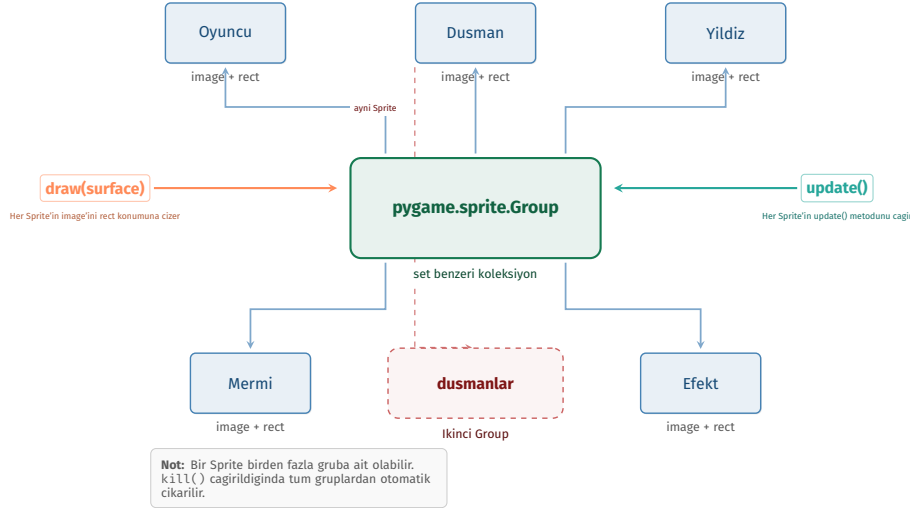
```

1 # Tek satırda hepsini güncelle
2 tum_spritelar.update(dt)
3
4 # Tek satırda hepsini çiz
5 ekran.fill(SIYAH)
6 tum_spritelar.draw(ekran)

```

Fark ortada: grupsuz yaklaşımda her nesne türü için ayrı döngü yazman gerekiyor. Gruplu yaklaşımda ise tüm güncelleme ve çizim işi **iki satıra** sığıyor. Oyunun büyüdükçe düşmanlar, mermiler, güç-artırımlar (*power-up*) ve efektler eklendikçe bu sadeleşmenin değeri katlanarak artar.

Bu kavramı görsel olarak incelersek daha net anlayacağız.



Şekil 20.1: Group ile Sprite ilişkisi: draw() ve update() akışı

Şekil 20.1’da görüldüğü gibi **Group** merkezi bir yönetim noktası olarak çalışır. **draw()** çağırıldığında gruptaki her Sprite’ın **image** özelliğini **rect** konumuna göre çizer. **update()** çağırıldığında ise her Sprite’ın kendi **update()** metodunu tetikler. Bir Sprite birden fazla gruba ait olabilir; bu şekilde farklı amaçlar (çizim, çarpışma kontrolü, güncelleme) için farklı gruplar oluşturabilirsin.

İpucu: Çizim Sırası

draw() Sprite’ları gruba eklenme sırasına göre çizer. İlk eklenen altta, son eklenen üstte görünür. Katmanlı çizim istiyorsan birden fazla grup kullan: önce arka plan grubunu, sonra oyuncu grubunu, en son ön plan grubunu çiz.

Şimdi bu kavramları pekiştirmek için çok sayıda Sprite’ı grupla yöneten bir örneğe bakalım. Aşağıdaki **Yildiz** sınıfı farklı boyut ve hızlarda kayan bir yıldız arka planı oluşturur:

Listing 20.3: Yıldız Sprite Sınıfı

```

1 class Yıldiz(pygame.sprite.Sprite):
2     """Farklı boyut ve hızda kayan yıldız Sprite."""
3
4     def __init__(self, baslangic_y=None):
5         """Yıldızı oluşturun.
6
7         Args:
8             baslangic_y: Başlangıç y konumu. None ise rastgele.
9         """

```



```

10     super().__init__()
11
12     # Rastgele boyut (1-3 piksel yarıcap)
13     self.boyut = random.randint(1, 3)
14
15     # Boyuta göre parlaklık ayarla (buyuk = parlak)
16     parlaklik = 100 + self.boyut * 50
17     self.renk = (parlaklik, parlaklik, min(255, parlaklik +
18         20))
19
20     # Gorsel olustur (kucuk daire)
21     cap = self.boyut * 2
22     self.image = pygame.Surface((cap, cap), pygame.SRCALPHA)
23     pygame.draw.circle(self.image, self.renk,
24         (self.boyut, self.boyut), self.boyut)
25
26     # Konum ayarla
27     self.rect = self.image.get_rect()
28     self.rect.x = random.randint(0, GENISLIK)
29     if baslangic_y is not None:
30         self.rect.y = baslangic_y
31     else:
32         self.rect.y = random.randint(-YUKSEKLIK, YUKSEKLIK)
33
34     # Hiz: buyuk yildizlar daha hizli (yakın görünür)
35     self.hiz = self.boyut * random.uniform(
36         YILDIZ_MIN_HIZ, YILDIZ_MAX_HIZ / 3
37     ) + YILDIZ_MIN_HIZ
38
39     def update(self):
40         """Yıldızı aşağı kaydır, ekran dışından tekrar sok."""
41         self.rect.y += self.hiz
42
43         # Ekranın altından çıktıysa üstten tekrar gir
44         if self.rect.top > YUKSEKLIK:
45             self.rect.x = random.randint(0, GENISLIK)
46             self.rect.bottom = 0
47             # Her dongude hizi biraz degistir (dogallik)
48             self.hiz = self.boyut * random.uniform(
49                 YILDIZ_MIN_HIZ, YILDIZ_MAX_HIZ / 3
50             ) + YILDIZ_MIN_HIZ

```

Ana döngüde 60 yıldız ve bir hareketli nesne tek bir grupta yönetilir:

Listing 20.4: Grup Oluşturma ve Toplu Yönetim

```

1     # --- Gruplari olustur ---
2     tum_spritelar = pygame.sprite.Group()
3     yildizlar = pygame.sprite.Group()
4
5     # Yildizlari olustur
6     for _ in range(YILDIZ_SAYISI):
7         yildiz = Yildiz()

```

```

8         tum_spritelar.add(yildiz)
9         yildizlar.add(yildiz)
10
11         # Hareketli nesneyi olustur (yildizlari ustunde cizdirmek
12         # yildizlardan sonra ekliyoruz)
13         nesne = HareketliNesne()
14         tum_spritelar.add(nesne)

```

Tüm bu işlem sadece `tum_spritelar.update()` ve `tum_spritelar.draw(ekran)` çağrılıyla gerçekleşiyor. Tam kaynak kodunu `kod\_orneklere/bolum05/unite4/02\_cok` dosyasında bulabilirsin.

20.4 Mini Proje: Uzay Kaçışı v1

Bu bölüm boyunca görselleri yüklemeyi, manipüle etmeyi, Sprite sınıfı oluşturmayı ve Sprite gruplarıyla toplu yönetim yapmayı öğrendin. Şimdi tüm bu bilgileri bir araya getirerek kitabın ana projelerinden biri olan **Uzay Kaçışı**'nın ilk versiyonunu geliştirme zamanı.

Bu ilk versiyonda basit ama etkileyici bir uzay sahnesini oluşturacağız: bir oyuncu gemisi ve kayan yıldızlardan oluşan bir arka plan. Düşmanlar, mermiler ve skor sistemi ilerleyen bölümlerde adım adım eklenecek. Şimdilik odağımız Sprite gruplarını gerçek bir projede kullanmak.

20.4.1 Proje Tanıtımı

Uzay Kaçışı, dikey kaydırmalı bir uzay oyunudur. Oyuncu bir uzay gemisini kontrol eder ve engellerden kaçarak hayatta kalmaya çalışır. Projenin gelişim planı şu şekilde:

Tablo 20.1: Uzay Kaçışı proje yol haritası

Versiyon	Bölüm	Eklenen Özellikler
v1	Bölüm 5	Oyuncu gemisi + kayan yıldız arka planı
v2	Bölüm 6	Düşmanlar + mermiler + çarpışma algılama
v3	Bölüm 7	Ses efektleri + arka plan müziği
v4	Bölüm 8	Animasyonlar + patlama efekti + zamanlayıcılar

20.4.2 Oyuncu Gemisi: Oyuncu Sınıfı

Oyuncu gemisi, Bölüm 19'de öğrendiğin Sprite alt sınıfı kalıbını kullanır. WASD veya ok tuşlarıyla dört yönde hareket eder ve `clamp\_ip()` ile ekran sınırları içinde kalır.

Python – Oyuncu Sınıfı

```

1 class Oyuncu(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()

```

```

4         self.image = gorsel_yukle("gemi.png", (50, 40))
5         self.rect = self.image.get_rect()
6         self.rect.centerx = GENISLIK // 2
7         self.rect.bottom = YUKSEKLIK - 20
8         self.hiz = 5
9
10        def update(self):
11            tuslar = pygame.key.get_pressed()
12            if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
13                self.rect.x -= self.hiz
14            if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
15                self.rect.x += self.hiz
16            if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
17                self.rect.y -= self.hiz
18            if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
19                self.rect.y += self.hiz
20            self.rect.clamp_ip(ekran_sinir)

```

Burada birkaç önemli noktaya dikkat edelim. Birincisi, `gorsel_yukle()` fonksiyonu görseli yüklemeye çalışır ve bulamazsa renkli bir yedek yüzey döndürür (Bölüm 19'deki fallback mantığı). İkincisi, `clamp_ip()` metodu Sprite'ın `rect` alanını verilen dikdörtgen içinde tutar – böylece gemi asla ekran dışına çıkamaz. Üçüncüsü, hareket hızı `self.hiz` ile kontrol edilir ve ileride güç-artırım (*power-up*) sistemiyle dinamik olarak değiştirilebilir.

20.4.3 Kayan Yıldız Arka Planı: Yildiz Sınıfı

Uzay temasını güçlendirmek için arka planda kayan yıldızlar oluşturacağız. Her yıldız küçük beyaz bir daire olacak ve farklı hızlarda aşağı doğru kayacak. Bu, basit ama etkili bir paralaks hissi yaratır: yakın yıldızlar hızlı, uzak yıldızlar yavaş hareket eder.

Python – Yildiz Sinifi

```

1 class Yildiz(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.boyut = random.randint(1, 3)
5         self.image = pygame.Surface((self.boyut * 2,
6                                     ↪ self.boyut * 2),
7                                     pygame.SRCALPHA)
8         pygame.draw.circle(self.image, BEYAZ,
9                             (self.boyut, self.boyut),
10                            ↪ self.boyut)
11        self.rect = self.image.get_rect()
12        self.rect.x = random.randint(0, GENISLIK)
13        self.rect.y = random.randint(-YUKSEKLIK,
14        ↪ YUKSEKLIK)
15        self.hiz = random.uniform(1, 4)

```

```

14     def update(self):
15         self.rect.y += self.hiz
16         if self.rect.top > YUKSEKLIK:
17             self.rect.x = random.randint(0, GENISLIK)
18             self.rect.bottom = 0

```

Her yıldız rastgele bir boyut (1-3 piksel yarıçap), rastgele bir başlangıç konumu ve rastgele bir hız alır. `update()` metodunda yıldız aşağı kayar ve ekranın altından çıkınca üstten tekrar girer. Bu döngüsel hareket sayesinde sonsuz bir uzay hissi oluşur.

20.4.4 Sprite Grupları ile Yönetim

Projedeki tüm nesneleri iki grup altında yönetiyoruz:

Python – Grup Yapısı

```

1  # Ana grup: tum Sprite'leri cizdirmek icin
2  tum_spritelar = pygame.sprite.Group()
3
4  # Yildiz grubu: sadece yildizlari takip etmek icin
5  yildizlar = pygame.sprite.Group()
6
7  # Oyuncuyu ekle
8  oyuncu = Oyuncu()
9  tum_spritelar.add(oyuncu)
10
11 # Yildizlari olustur ve gruplara ekle
12 for _ in range(80):
13     yildiz = Yildiz()
14     tum_spritelar.add(yildiz)
15     yildizlar.add(yildiz)

```

`tum_spritelar` grubu çizim için kullanılır: ana döngüde `tum_spritelar.draw(ekran)` tek çağrıyla hem oyuncuyu hem yıldızları çizer. `yildizlar` grubu ise ilerideki bölümlerde çarpışma kontrolü ve yıldız yönetimi için ayrı tutulur.

20.4.5 Ana Döngü ve Skor Gösterimi

Ana döngü, önceki ünitelerde öğrendiğin yapının aynısı. Fark şu: tüm güncelleme ve çizim işi iki satırda halloluyor. Ayrıca basit bir hayatta kalma süresi skoru da ekliyoruz:

Python – Ana Dongu

```

1  skor = 0
2  baslangic = pygame.time.get_ticks()
3
4  while calistir:
5      saat.tick(FPS)

```

```

6     for olay in pygame.event.get():
7         if olay.type == pygame.QUIT:
8             calistir = False
9
10        # Toplu guncelleme
11        tum_spritelar.update()
12
13        # Skor: hayatta kalma suresi (saniye)
14        gecen = (pygame.time.get_ticks() - baslangic) // 1000
15        skor = gecen
16
17        # Cizim
18        ekran.fill(KOYU_MAVI)
19        tum_spritelar.draw(ekran)
20
21        # Skor gosterimi
22        skor_yazi = font.render(f"Sure: {skor}s", True, BEYAZ)
23        ekran.blit(skor_yazi, (10, 10))
24
25        pygame.display.flip()

```

Dikkat et: `tum_spritelar.update()` hem oyuncuyu hem 80 yıldızı günceller. `tum_spritelar.draw(ekran)` ise hepsini çizer. Gruplar olmasaydı 80 yıldız için ayrı döngü yazmamız gerekecekti.

20.4.6 Tam Kaynak Kodu

Uzay Kaçışı v1 projesinde `Oyuncu` sınıfı klavye ile kontrol edilir ve `clamp_ip()` ile ekran sınırları içinde tutulur:

Listing 20.5: Oyuncu Sprite Sınıfı

```

1 class Oyuncu(pygame.sprite.Sprite):
2     """Oyuncu uzay gemisi.
3
4     WASD veya ok tuslari ile dort yonde hareket eder.
5     clamp_ip ile ekran sinirlarini asmaz.
6     """
7
8     def __init__(self):
9         super().__init__()
10        self.image = gorsel_yukle("gemi.png", OYUNCU_BOYUT)
11        self.rect = self.image.get_rect()
12        # Baslangic konumu: ekranin alt ortasi
13        self.rect.centerx = GENISLIK // 2
14        self.rect.bottom = YUKSEKLIK - 20
15        self.hiz = OYUNCU_HIZ
16        # Ekran sinir dikdortgeni (clamp_ip icin)
17        self.sinir = pygame.Rect(0, 0, GENISLIK, YUKSEKLIK)
18
19    def update(self):
20        """Klavye girdilerine gore hareket et."""

```

```

21     tuslar = pygame.key.get_pressed()
22
23     if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
24         self.rect.x -= self.hiz
25     if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
26         self.rect.x += self.hiz
27     if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
28         self.rect.y -= self.hiz
29     if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
30         self.rect.y += self.hiz
31
32     # Ekran sinirlari icinde tut
33     self.rect.clamp_ip(self.sinir)

```

Ana döngüde tüm Sprite'lar tek bir **tum_spritelar** grubu ile yönetilir. Çizim ve güncelleme tek satırda yapılır:

Listing 20.6: Ana Döngü – Grup Yönetimi

```

1  # --- Sprite Gruplari ---
2  tum_spritelar = pygame.sprite.Group()
3  yildizlar = pygame.sprite.Group()
4
5  # --- Yildizlari olustur ---
6  for _ in range(YILDIZ_SAYISI):
7      yildiz = Yildiz(ustten_baslat=False)
8      tum_spritelar.add(yildiz)
9      yildizlar.add(yildiz)
10
11  # --- Oyuncuyu olustur ---
12  # Oyuncu en son eklenir ki yildizlari ustunde cizilsin
13  oyuncu = Oyuncu()
14  tum_spritelar.add(oyuncu)
15
16  # --- Skor ve zamanlama ---
17  skor = 0
18  baslangic_zamani = pygame.time.get_ticks()
19  son_yildiz_zamani = pygame.time.get_ticks()
20
21  # --- Ana Oyun Dongusu ---
22  calistir = True
23  while calistir:
24      saat.tick(FPS)
25
26      # --- Olay isleme ---
27      for olay in pygame.event.get():
28          if olay.type == pygame.QUIT:
29              calistir = False
30          elif olay.type == pygame.KEYDOWN:
31              if olay.key == pygame.K_ESCAPE:
32                  calistir = False
33
34      # --- Yeni yildiz ekleme (spawn) ---

```

```

35     simdi = pygame.time.get_ticks()
36     if simdi - son_yildiz_zamani > YILDIZ_SPAWN_ARASI:
37         yeni_yildiz = Yildiz(ustten_baslat=True)
38         tum_spritelar.add(yeni_yildiz)
39         yildizlar.add(yeni_yildiz)
40         son_yildiz_zamani = simdi
41
42         # Çok fazla yildiz birikmesini engelle
43         if len(yildizlar) > 150:
44             en_eski = yildizlar.sprites()[0]
45             en_eski.kill()
46
47         # --- Toplu guncelleme ---
48         tum_spritelar.update()
49
50         # --- Skor hesaplama ---
51         gecen_ms = simdi - baslangic_zamani
52         skor = gecen_ms // 1000
53
54         # --- Cizim ---
55         ekran.fill(KOYU_MAVI)
56
57         # Tum Sprite'lari tek satirda ciz
58         tum_spritelar.draw(ekran)
59
60         # HUD (skor ve bilgi)
61         skor_ciz(ekran, font, skor, len(yildizlar))
62
63         # Ekrani guncelle
64         pygame.display.flip()

```

Bu programı çalıştırdığında koyu mavi bir arka plan üzerinde farklı hızlarda kayan yıldızlar ve ok tuşları (veya WASD) ile kontrol edebileceğin bir uzay gemisi göreceksin. Sol üst köşede hayatta kalma süresi sayacı çalışacak.

💡 İpucu: Tam Kaynak Kodu

Projenin tamamını (Yildiz sınıfı, gorsel_yukle() fallback fonksiyonu ve skor sistemi dahil) kod\ornekleri\bolum05\unite4\03_uzay_kacisi_v1.py dosyasında bulabilirsin.

💡 İpucu: Projeyi Geliştir

Bu temel yapı üzerine kendin de eklentiler yapabilirsin:

- Yıldızların boyutuna göre farklı parlaklıkta renk ver (büyük yıldızlar beyaz, küçükler gri)
- Geminin arkasından iz bırakan bir egzoz efekti ekle (küçük turuncu daireler)
- Zaman geçtikçe yıldız hızını kademeli olarak artır
- Sağ üst köşeye FPS sayacı ekle

● Bilgi: Sıradaki Versiyon

Bölüm 6'da Uzay Kaçışı v2 ile çarpışma algılama öğrenecek, düşman gemileri ve mermi sistemi ekleyeceğiz. Burada oluşturduğumuz **dusmanlar** ve **mermiler** grupları tam da o zaman devreye girecek. Sprite gruplarının gerçek gücünü çarpışma algılamada (`pygame.spritecollide` ve `pygame.groupcollide`) keşfedeceksin.

20.5 Sonuç

20.5.1 Ünite 4 Özeti

Bu üniteye Sprite'ları toplu olarak yönetmenin yolunu öğrendin. `pygame.sprite.Group` sınıfıyla tanışarak `add()`, `remove()`, `has()` ve `kill()` gibi temel grup işlemlerini uyguladın. Ardından `group.draw()` ve `group.update()` ile ana döngüyü iki satıra indirmenin rahatlığını deneyimledin. Son olarak tüm bu bilgileri Uzay Kaçışı v1 projesinde bir araya getirerek oyuncu gemisi ve kayan yıldız arka planından oluşan ilk gerçek oyun sahnemizi oluşturduğun.

20.5.2 Bölüm 5 Genel Özeti

Bölüm 5'i tamamladın, tebrikler! Bu bölüm boyunca PyGame'in görsel ve Sprite sisteminin tüm temellerini kavradın. Şimdi geriye dönüp neler öğrendiğine bir bakalım.

Bölüm 17'de `pygame.image.load()` ile görsel dosyalarını yüklemeyi, `convert()` ve `convert_alpha()` ile performans optimizasyonu yapmayı ve alfa kanalı sayesinde şeffaf arka planlı görseller kullanmayı öğrendin.

Bölüm 18'nde görselleri `pygame.transform.scale()` ile boyutlandırmayı, `pygame.transform.rotate()` ile döndürmeyi ve `pygame.transform.flip()` ile yansıtmayı kavradın. Bu işlemlerin orijinal görseli değiştirmedikçe ve her zaman yeni bir yüzey döndürdüğünü fark ettin.

Bölüm 19'de `pygame.sprite.Sprite` sınıfından kalıtım alarak kendi Sprite alt sınıflarını oluşturmayı, `image` ve `rect` özelliklerini tanımlamayı ve `update()` metodunu override ederek bağımsız hareket mantığı yazmayı öğrendin. Klavye girdileriyle kontrol edilen bir Oyuncu Sprite'ı geliştirdin.

Son olarak bu ünite de **Group** sınıfı ile Sprite'ları gruplara ayırmayı, toplu çizim ve güncelleme yapmayı ve Uzak Kaçışı v1 ile ilk gerçek projeyi hayata geçirdin.

20.5.3 Anahtar Kavramlar

Tablo 20.2: Bölüm 5 – Anahtar kavramlar

Kavram	Açıklama
<code>image.load()</code>	Dosyadan görsel yükleme
<code>convert()</code>	Görseli ekran formatına çevirme (performans)
<code>convert_alpha()</code>	Alfa kanallı görseli optimize etme
<code>transform.scale()</code>	Görseli yeniden boyutlandırma
<code>transform.rotate()</code>	Görseli döndürme
<code>Sprite</code>	Oyun nesnelerinin temel sınıfı (image + rect)
<code>update()</code>	Sprite'ın her karede çağrılan güncelleme metodu
<code>Group</code>	Sprite'ları toplu yöneten koleksiyon sınıfı
<code>add()</code>	Gruba Sprite ekleme
<code>remove()</code>	Gruptan Sprite çıkarma
<code>kill()</code>	Sprite'ı tüm gruplarından çıkarma
<code>group.draw()</code>	Gruptaki tüm Sprite'ları tek çağrıyla çizme
<code>group.update()</code>	Gruptaki tüm Sprite'ları tek çağrıyla güncelleme

20.5.4 Önemli Komutlar

Tablo 20.3: Bölüm 5 – Önemli PyGame komutları

Komut	İşlev
<code>pygame.image.load(dosya)</code>	Görsel dosyasını yükler
<code>gorsel.convert()</code>	Piksel formatını ekranla eşleştirir
<code>gorsel.convert_alpha()</code>	Alfa kanallı pikselleri optimize eder
<code>pygame.transform.scale(g, boyut)</code>	Görseli boyutlandırır
<code>pygame.transform.rotate(g, aci)</code>	Görseli döndürür
<code>pygame.transform.flip(g, x, y)</code>	Görseli yansıtır
<code>pygame.sprite.Sprite</code>	Sprite temel sınıfı
<code>pygame.sprite.Group()</code>	Yeni grup oluşturur
<code>grup.add(sprite)</code>	Gruba Sprite ekler
<code>grup.remove(sprite)</code>	Gruptan Sprite çıkarır
<code>sprite.kill()</code>	Sprite'ı tüm gruplardan çıkarır
<code>grup.draw(yuzey)</code>	Tüm Sprite'ları çizer
<code>grup.update()</code>	Tüm Sprite'ları günceller
<code>rect.clamp_ip(alan)</code>	Rect'i alan içinde tutar

20.5.5 Sıradaki Adım

Bölüm 6'da **Çarpışma Algılama ve Fizik Temelleri** konusuna geçeceğiz. Uzak Kaçışı v2 ile düşman gemileri, mermi sistemi ve çarpışma algılama ekleyeceğiz. `pygame.sprite.spritecollide()` ve `pygame.sprite.groupcollide()` fonksiyonlarıyla gruplar arası çarpışma tespiti yapacağız. Sprite gruplarının asıl gücü tam da burada ortaya çıkacak: tek bir fonksiyon çağrısıyla hangi merminin hangi düşmana isabet ettiğini tespit edebileceksin.

● Bilgi: Pratik Yap

Uzak Kaçışı v1'i temel alarak şu eklentileri dene:

- Yıldızların boyutuna göre farklı renkler ata (büyük = beyaz, küçük = gri)
- Geminin hareket yönüne göre hafifçe eğilmesini sağla (`transform.rotate()`)
- Ekranın altında yavaşça kayan bir nebula (renkli bulut) Sprite'ı ekle
- Space tuşuyla basit bir mermi ateşleme mekaniği ekle (henüz çarpışmasız)
- Yıldız yoğunluğunu zaman geçtikçe artır (her 10 saniyede 5 yeni yıldız)

Bu egzersizler hem Sprite grubu becerilerini pekiştirecek hem de Bölüm 6'ya hazırlanmanı sağlayacak.

Rect Sınıfı ve Çarpışma

“Oyun geliştirmede en önemli soru şudur: Bu iki nesne birbirine değdi mi?”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.Rect` sınıfının yapısı ve temel özellikleri (x, y, width, height, center, topleft)
- `colliderect()` ile iki dikdörtgenin çarpışma kontrolü
- `collidepoint()` ile fare tıklama algılama ve UI buton etkileşimi
- Çarpışma algılamanın pratik bir oyun demosunda uygulanması

21.1 Rect Sınıfı

Bölüm 19’te ilk Sprite’ını oluştururken `self.rect = self.image.get_rect()` satırını yazmıştın. O zaman bu `rect` nesnesinin konumu temsil ettiğini kısaca görmüştün. Şimdi bu küçük ama güçlü yapıyı derinlemesine keşfetme zamanı geldi.

Bir oyun sahnesini düşün: uzay gemisi meteorlara çarpıyor mu? Mermi düşmana isabet etti mi? Oyuncu güç artırıcıyı topladı mı? Tüm bu soruların cevabı aynı kavramda gizli: *çarpışma algılama* (collision detection). PyGame’de çarpışma algılamanın temel taşı ise `pygame.Rect` sınıfıdır.

■ Tanım: Rect (Dikdörtgen)

`pygame.Rect`, ekrandaki dikdörtgen bir alanı temsil eden sınıftır. Dört temel bilgi taşır: **x** ve **y** koordinatları (sol üst köşe), **width** (genişlik) ve **height** (yükseklik). Bu dört değer, ekrandaki herhangi bir dikdörtgen bölgeyi tam olarak tanımlamaya yeter.

Rect'i bir oyun karakterinin etrafına çizilmiş görünmez bir kutu olarak hayal edebilirsin. Karakter ekranda hareket ederken bu kutu da onunla birlikte hareket eder. İki karakterin çarpışıp çarpışmadığını anlamak için bu görünmez kutuların örtüşüp örtüşmediğine bakarız.

21.1.1 Rect Oluşturma

Bir Rect oluşturmanın birden fazla yolu vardır. En temel yol, dört değeri doğrudan vermektir:

Python – Rect Oluşturma Yolları

```
1 import pygame
2
3 # Yol 1: Doğrudan değer vererek (x, y, genişlik,
4   ↪ yükseklik)
5 kutu = pygame.Rect(100, 200, 50, 30)
6
7 # Yol 2: Bir gorselden otomatik oluşturma
8 gorsel = pygame.image.load("gemi.png").convert_alpha()
9 gemi_rect = gorsel.get_rect() # (0, 0) konumunda, gorsel
10   ↪ boyutunda
11
12 # Yol 3: Konum belirterek gorsel Rect'i
13 gemi_rect = gorsel.get_rect(center=(400, 300)) # Merkezi
14   ↪ (400,300)
15
16 gemi_rect = gorsel.get_rect(topleft=(100, 50)) # Sol üst
17   ↪ (100,50)
```

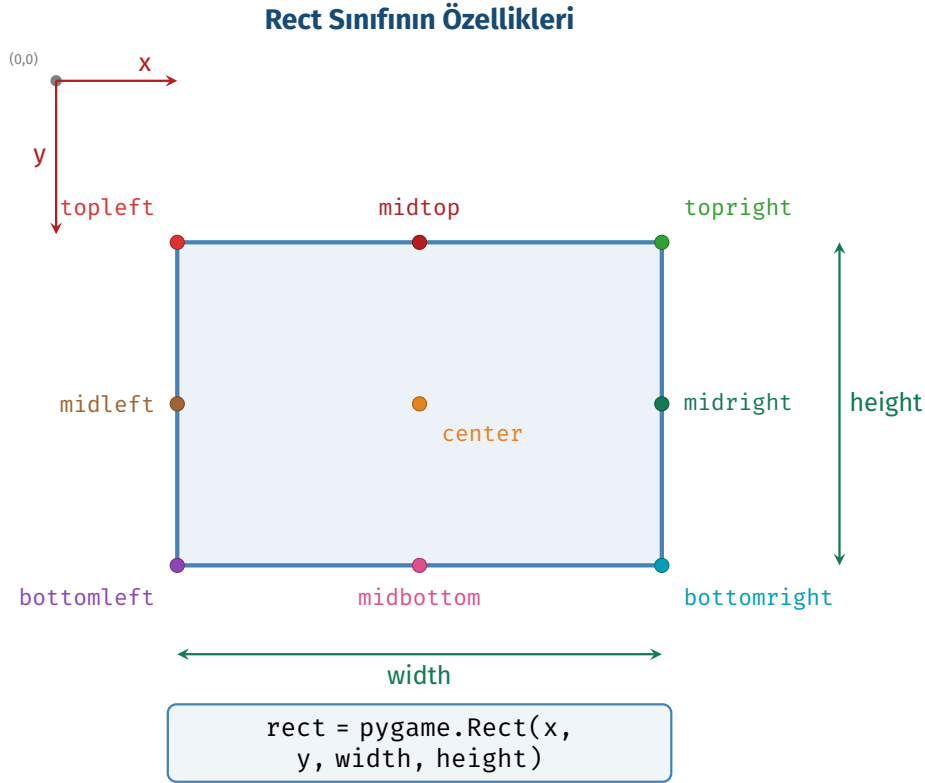
Birinci yolda `pygame.Rect(100, 200, 50, 30)` çağrısı, sol üst köşesi (100, 200) noktasında, 50 genişliğinde ve 30 yüksekliğinde bir dikdörtgen oluşturur. İkinci yolda ise bir görselin boyutlarından otomatik olarak Rect üretilir — Bölüm 19'te bu yöntemi zaten kullanmıştın. Üçüncü yolda `get_rect()` fonksiyonuna konum bilgisi vererek Rect'in ekranın neresinde başlayacağını belirleyebilirsin.

● Bilgi: Sprite ve Rect İlişkisi

Bölüm 19'te öğrendiğin gibi, her Sprite'ın iki zorunlu özelliği vardır: `self.image` ve `self.rect`. Sprite'ın ekrandaki konumu tamamen `self.rect` tarafından belirlenir. Rect'i değiştirdiğinde Sprite'ın konumu da değişir.

21.1.2 Rect'in Konum Özellikleri

Rect sınıfının asıl gücü, dikdörtgenin farklı noktalarına kolayca erişebilmende yatar. Sadece `x` ve `y` değil, dikdörtgenin kenarlarına, köşelerine ve merkezine tek bir özellik ile ulaşabilirsin.



Şekil 21.1: Rect sınıfının konum ve boyut özellikleri

Tablo 21.1: Rect sınıfının temel konum özellikleri

Özellik	Tür	Açıklama
<code>x, y</code>	int	Sol üst köşenin koordinatları
<code>width, height</code>	int	Genişlik ve yükseklik
<code>top</code>	int	Üst kenarın y koordinatı (<code>y</code> ile aynı)
<code>bottom</code>	int	Alt kenarın y koordinatı (<code>y + height</code>)
<code>left</code>	int	Sol kenarın x koordinatı (<code>x</code> ile aynı)
<code>right</code>	int	Sağ kenarın x koordinatı (<code>x + width</code>)
<code>center</code>	(int, int)	Merkez noktasının koordinatları
<code>centerx, centery</code>	int	Merkez noktasının x ve y değerleri
<code>topleft</code>	(int, int)	Sol üst köşe
<code>topright</code>	(int, int)	Sağ üst köşe
<code>bottomleft</code>	(int, int)	Sol alt köşe
<code>bottomright</code>	(int, int)	Sağ alt köşe
<code>size</code>	(int, int)	Genişlik ve yükseklik çifti

Bu özelliklerin en güzel tarafı, hepsinin hem okunabilir hem de yazılabilir olmasıdır. Yani `kutu.center = (400, 300)` dediğinde PyGame otomatik olarak `x` ve `y` değerlerini hesaplayarak dikdörtgeni merkezi o noktaya gelecek şekilde konumlandırır.

Python – Rect Özelliklerini Kullanma

```

1 import pygame
2
3 kutu = pygame.Rect(100, 200, 80, 60)
4
5 # Konum bilgilerini oku
6 print(f"Sol ust: {kutu.topleft}")      # (100, 200)
7 print(f"Sag alt: {kutu.bottomright}")  # (180, 260)
8 print(f"Merkez: {kutu.center}")        # (140, 230)
9 print(f"Boyut: {kutu.size}")           # (80, 60)
10
11 # Konum degistir - merkezi ekranin ortasina tasi
12 kutu.center = (400, 300)
13 print(f"Yeni sol ust: {kutu.topleft}") # (360, 270)
14 print(f"Yeni merkez: {kutu.center}")   # (400, 300)
15
16 # Kenarlari kullanarak sinir kontrolu yap
17 kutu.right = 800 # Sag kenari ekranin sagina yasla
18 kutu.bottom = 600 # Alt kenari ekranin altina yasla

```

Dikkat edersen `kutu.center = (400, 300)` dediğinde sol üst köşe otomatik olarak (360, 270) olarak güncellendi. PyGame bu hesaplamayı senin yerine yaptı: $400 - 80/2 = 360$ ve $300 - 60/2 = 270$. Bu özellik, özellikle nesneleri ekranın ortasına yerleştirirken veya sınır kontrolü yaparken büyük kolaylık sağlar.

💡 İpucu: Sprite Konumlandırma İpucu

Sprite'ını ekranın altına ortalamak istediğinde şunu yaz: `self.rect.centerx = EKRAN_GENISLIK // 2` ve `self.rect.bottom = EKRAN_YUKSEKLIK - 10`. Böylece karakter ekranın alt ortasında, 10 piksel yukarıda konumlanır. Bu tekniği Bölüm 19'teki oyuncu sprite'ında zaten kullanmıştın.

21.1.3 Rect ile Hareket

Rect'in konum özelliklerini değiştirmek, nesneyi ekranda hareket ettirmenin en doğrudan yoludur. Ancak PyGame daha pratik bir yol da sunar: `move()` ve `move_ip()` metotları.

Python – Rect Hareket Metotları

```

1 import pygame
2
3 kutu = pygame.Rect(100, 200, 50, 50)
4
5 # move() - Yeni bir Rect dondurur, orijinali degismez
6 yeni_kutu = kutu.move(10, -5) # x +10, y -5
7 print(kutu.topleft)           # (100, 200) - degismedi
8 print(yeni_kutu.topleft)      # (110, 195) - yeni konum

```

```

9
10 # move_ip() - Rect'i yerinde (in-place) değiştirir
11 kutu.move_ip(10, -5)
12 print(kutu.topleft)                # (110, 195) - degisti!

```

`move()` ve `move_ip()` arasındaki fark kritiktir. `move()` orijinal Rect'e dokunmaz ve yeni bir Rect döndürür. `move_ip()` ise Rect'i doğrudan değiştirir ("ip" ifadesi "in-place", yani "yerinde" anlamına gelir). Sprite'ların `update()` metodunda genellikle `move_ip()` veya doğrudan `self.rect.x += hiz` şeklinde kullanım tercih edilir.

⚠ Dikkat: Dikkat: `move()` Orijinali Değiştirmez

`kutu.move(10, 0)` yazdığında kutunun hareket ettiğini düşünebilirsin, ama aslında hareket eden yeni bir Rect döner ve orijinal kutu yerinde kalır. Eğer orijinali değiştirmek istiyorsan `kutu.move_ip(10, 0)` veya `kutu = kutu.move(10, 0)` kullan.

21.1.4 Rect ile Sınır Kontrolü

Bölüm 16'te nesnelerin ekran dışına çıkmasını engellemek için `if` koşulları kullanmıştın. Rect'in konum özellikleri bu kontrolü çok daha zarif hale getirir:

Python – Rect ile Ekran Sınır Kontrolü

```

1 EKRAN_GENISLIK = 800
2 EKRAN_YUKSEKLIK = 600
3
4 def sinir_kontrolu(rect):
5     """Rect'i ekran sinirlari icinde tutar."""
6     if rect.left < 0:
7         rect.left = 0
8     if rect.right > EKRAN_GENISLIK:
9         rect.right = EKRAN_GENISLIK
10    if rect.top < 0:
11        rect.top = 0
12    if rect.bottom > EKRAN_YUKSEKLIK:
13        rect.bottom = EKRAN_YUKSEKLIK

```

Burada `rect.left`, `rect.right`, `rect.top` ve `rect.bottom` özelliklerini kullanarak nesnenin her kenarını kontrol ediyorsun. Daha da pratik bir yol ise `clamp_ip()` metodunu kullanmaktır:

Python – `clamp_ip` ile Sınır Kontrolü

```

1 ekran_rect = pygame.Rect(0, 0, EKRAN_GENISLIK,
2   ↪ EKRAN_YUKSEKLIK)
3 karakter_rect.clamp_ip(ekran_rect)

```

`clamp\_ip()`, Rect'i verilen dikdörtgenin sınırları içinde tutar. Tek satırla dört kenarın kontrolünü yapmış olursun. Bu metot özellikle oyuncu karakterinin ekran dışına çıkmasını engellemek için idealdir.

21.2 Rect Çarpışması

Rect sınıfının konum özelliklerini öğrendin. Şimdi asıl heyecan verici kısma geçiyoruz: iki dikdörtgenin birbirine değip değmediğini kontrol etmek. Bir platformer oyununda karakterin platforma basıp basmadığını, bir shooter oyununda merminin düşmana isabet edip etmediğini veya bir yarış oyununda arabanın duvara çarpıp çarpmadığını belirlemek — tüm bunlar dikdörtgen çarpışma kontrolüyle başlar.

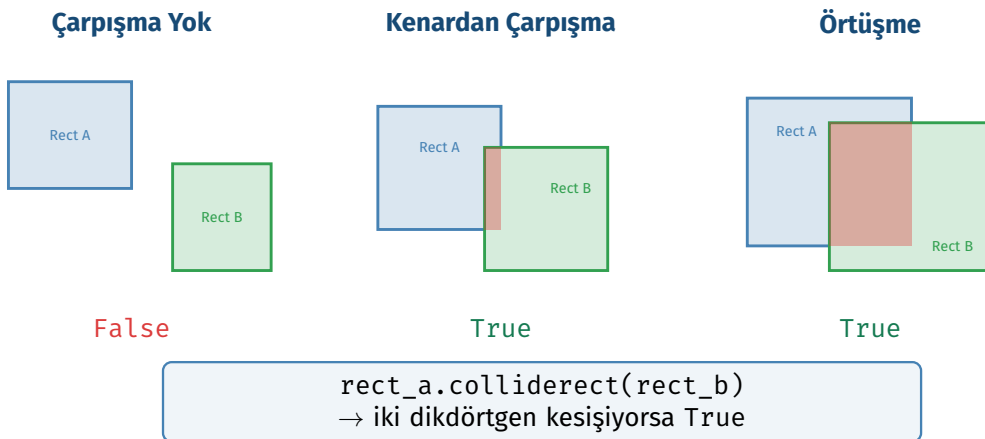
21.2.1 `collidect()`: İki Dikdörtgen Çarpışıyor mu?

`collidect()`, iki Rect nesnesinin örtüşüp örtüşmediğini kontrol eden temel çarpışma metodudur. Eğer iki dikdörtgen en az bir piksel bile örtüşüyorsa `True`, aksi halde `False` döndürür.

■ Tanım: `collidect()`

`rect1.collidect(rect2)` ifadesi, `rect1` ve `rect2` dikdörtgenlerinin herhangi bir noktada örtüşüp örtüşmediğini kontrol eder. Örtüşme varsa `True`, yoksa `False` döndürür. Bu kontrol *AABB* (Axis-Aligned Bounding Box) yöntemiyle yapılır: dikdörtgenler eksenlere paralel olduğu sürece çalışır.

`collidect()` Çarpışma Durumları



Şekil 21.2: `collidect()` çarpışma durumları: ayırık, kenardan temas ve örtüşme

Peki PyGame iki dikdörtgenin örtüştüğünü nasıl anlıyor? Mantık aslında basit: iki dikdörtgen örtüşmüyorsa, birinin sağ kenarı diğerkinin sol kenarının solunda

kalır veya birinin alt kenarı diğerinin üst kenarının üstünde kalır. Bunun tersi doğruysa çarpışma var demektir.

Python – collidirect() Temel Kullanım

```

1 import pygame
2
3 # İki dikdörtgen olustur
4 oyuncu = pygame.Rect(100, 100, 50, 50)
5 dusman = pygame.Rect(130, 120, 40, 40)
6
7 # Carpisma kontrolu
8 if oyuncu.collidirect(dusman):
9     print("Carpisma var!")
10 else:
11     print("Carpisma yok.")

```

Bu örnekte oyuncu dikdörtgeni (100, 100) ile (150, 150) arasında, düşman ise (130, 120) ile (170, 160) arasında yer alıyor. İki alan örtüştüğü için `collidirect()` `True` döndürür.

⚠ Dikkat: Kenar Çarpışması

`collidirect()`, iki dikdörtgenin kenarları tam olarak aynı çizgideyken (örneğin birinin `right` değeri diğerinin `left` değerine eşitse) `False` döndürür. Yani kenarlar temas ediyorsa ama örtüşme yoksa çarpışma algılanmaz.

21.2.2 Oyun Döngüsünde Çarpışma Kontrolü

Gerçek bir oyunda çarpışma kontrolü her karede yapılmalıdır. Bölüm 11’te öğrendiğin oyun döngüsünün güncelleme aşamasında bu kontrolü gerçekleştirirsin:

Oyuncu bir kare ile temsil ediliyor ve ok tuşlarıyla hareket ettiriliyor. Ekran-daki kırmızı dikdörtgen ise engeli temsil ediyor. Her karede `collidirect()` çağrılarak çarpışma kontrol ediliyor ve çarpışma durumunda renk yeşile dönüyor.

21.2.3 contains(): İç İç Geçme Kontrolü

Bazen bir dikdörtgenin diğerinin tamamen içinde olup olmadığını kontrol etmek istersin. Örneğin bir güç artırıcının oyuncunun hitbox’ı içinde kalıp kalmadığını veya bir nesnenin güvenli bölge sınırları içinde olup olmadığını belirlemek isteyebilirsin.

Python – contains() Kullanımı

```

1 import pygame
2
3 buyuk_alan = pygame.Rect(50, 50, 200, 200)
4 kucuk_kutu = pygame.Rect(100, 100, 30, 30)
5 disardaki = pygame.Rect(300, 300, 30, 30)
6

```

```

7 print(buyuk_alan.contains(kucuk_kutu)) # True - tamamen
  ↳ içinde
8 print(buyuk_alan.contains(disardaki)) # False - disarida

```

`contains()` ile `collidirect()` arasındaki fark önemlidir: `collidirect()` kısmi örtüşmede bile `True` dönerken, `contains()` ancak bir dikdörtgen diğeri-
nin **tamamen** içindeyse `True` döndürür.

21.2.4 `collidelist()`: Birden Fazla Nesneyle Çarpışma

Bir oyunda genellikle tek bir düşman değil, düşman ordusu bulunur. Oyuncunun bu düşmanlardan herhangi biriyle çarpışıp çarpışmadığını kontrol etmek için `collidelist()` kullanabilirsin.

■ Tanım: `collidelist()` ve `collidelistall()`

`rect.collidelist(liste)`, verilen Rect listesindeki ilk çarpışan elemanın indeksini döndürür. Çarpışma yoksa `-1` döndürür. `rect.collidelistall(liste)` ise çarpışan tüm elemanların indeks listesini döndürür.

Python – `collidelist()` Kullanımı

```

1 import pygame
2
3 oyuncu = pygame.Rect(200, 200, 40, 40)
4
5 # Engel listesi
6 engeller = [
7     pygame.Rect(50, 50, 60, 60),
8     pygame.Rect(190, 190, 50, 50),    # Bu oyuncuyla
9     ↳ carpisiyor
10    pygame.Rect(400, 100, 70, 70),
11    pygame.Rect(210, 210, 30, 30),    # Bu da carpisiyor
12 ]
13
14 # İlk carpisan engelin indeksi
15 indeks = oyuncu.collidelist(engeller)
16 if indeks != -1:
17     print(f"Engel {indeks} ile carpisma!") # Engel 1 ile
18     ↳ carpisma!
19
20 # Carpisan tum engellerin indeksleri
21 tum_indeksler = oyuncu.collidelistall(engeller)
22 print(f"Carpisan engeller: {tum_indeksler}") # [1, 3]

```

`collidelist()` sadece ilk çarpışanı bulduğu için daha hızlıdır. Eğer hangi engellerle çarpışıldığı önemliyse — örneğin her engele farklı bir tepki vermek istiyorsan — `collidelistall()` kullan.

💡 İpucu: Performans İpucu

Çok sayıda engel varsa ve sadece “çarpışma var mı, yok mu?” sorusunu cevaplamak istiyorsan `collidelist()` yeterlidir. İlk çarpışmayı bulduğu anda durur ve listeyi sonuna kadar taramaz. Ancak tüm çarpışmaları bilmen gerekiyorsa `collidelistall()` zorunludur.

Bu konuda Rect sınıfının dikdörtgen çarpışma araçlarını öğrendin. Bir sonraki konuda, fare tıklamalarını algılamak ve UI butonları oluşturmak için `collidepoint()` metodunu keşfedeceksin.

21.3 Nokta Çarpışması

Önceki konuda iki dikdörtgenin çarpışmasını kontrol etmeyi öğrendin. Ancak oyunlarda sadece dikdörtgenler değil, **noktalar** da çarpışma kontrolüne dahil olur. En yaygın örneği zaten biliyorsun: fare imleci. Oyuncu ekranda bir butona tıkladığında, farenin (x, y) koordinatı butonun dikdörtgen alanı içinde mi? Bu soruyu `collidepoint()` ile cevaplayacaksın.

21.3.1 `collidepoint()`: Nokta Dikdörtgenin İçinde mi?

`collidepoint()`, verilen bir (x, y) noktasının Rect’in sınırları içinde olup olmadığını kontrol eder.

■ Tanım: `collidepoint()`

`rect.collidepoint(x, y)` veya `rect.collidepoint((x, y))` ifadesi, belirtilen noktanın dikdörtgenin içinde olup olmadığını kontrol eder. Nokta sınırlar dahilindeyse **True**, değilse **False** döndürür. Noktanın sol ve üst kenarlarda olması **True** dönerken, sağ ve alt kenarlarda olması **False** döndürür.

Python – `collidepoint()` Temel Kullanım

```
1 import pygame
2
3 buton = pygame.Rect(300, 200, 200, 50)
4
5 # Fare konumunu kontrol et
6 fare_x, fare_y = 350, 220
7 if buton.collidepoint(fare_x, fare_y):
8     print("Fare butonun üzerinde!")
9
10 # Tuple olarak da verilebilir
11 fare_pos = (350, 220)
12 if buton.collidepoint(fare_pos):
13     print("Fare butonun üzerinde!")
```

21.3.2 Fare Tıklama Algılama

Bölüm 15'te fare olaylarını nasıl yakalayacağını öğrenmiştin. Şimdi bu bilgiyle `collidepoint()` birleştirerek “oyuncu nereye tıkladı?” sorusunu cevaplayabilirsin.

Bu kodda bir dikdörtgen buton oluşturuluyor. Fare butona tıkladığında renk değişiyor ve bir mesaj yazdırılıyor. Dikkat edersen iki ayrı kontrol yapılıyor: önce `pygame.MOUSEBUTTONDOWN` olayı ile tıklama algılanıyor, sonra `collidepoint()` ile tıklamanın butonun üzerinde olup olmadığı kontrol ediliyor.

21.3.3 Hover Efekti: Fareyle Üzerine Gelme

Profesyonel bir oyun menüsünde butonlar sadece tıklandığında değil, fare üzerine geldiğinde de tepki verir. Buna *hover* efekti denir. Bu efekti de `collidepoint()` ile `pygame.mouse.get_pos()` birleştirerek elde edersin:

Python – Hover Efekti

```
1 # Oyun dongusu icinde (her karede)
2 fare_pos = pygame.mouse.get_pos()
3
4 if buton_rect.collidepoint(fare_pos):
5     # Fare butonun uzerinde - parlak renk
6     buton_renk = (100, 200, 100)
7 else:
8     # Fare disarida - normal renk
9     buton_renk = (50, 150, 50)
```

Burada `pygame.mouse.get_pos()` her karede farenin güncel konumunu verir. Bu konum butonun Rect'i içindeyse parlak renk, değilse normal renk kullanılır. Sonuç olarak fare butonun üzerine geldiğinde buton “parlar” ve kullanıcıya görsel geri bildirim sağlar.

21.3.4 Birden Fazla Buton Yönetimi

Gerçek bir oyun menüsünde genellikle birden fazla buton bulunur: “Başla”, “Ayarlar”, “Çıkış” gibi. Her buton için ayrı ayrı kontrol yazmak yerine, butonları bir listede tutabilir ve döngüyle kontrol edebilirsin:

Python – Çoklu Buton Yönetimi

```
1 # Buton verilerini sozluk listesi olarak tanimla
2 butonlar = [
3     {"rect": pygame.Rect(300, 150, 200, 50), "yazi":
4         ↪ "Basla",
5         "renk": (50, 150, 50)},
6     {"rect": pygame.Rect(300, 220, 200, 50), "yazi":
7         ↪ "Ayarlar",
8         "renk": (50, 50, 150)},
9     {"rect": pygame.Rect(300, 290, 200, 50), "yazi":
10        ↪ "Cikis",
```

```

8         "renk": (150, 50, 50)},
9     ]
10
11     # Tıklamada hangi butona basıldığını bul
12     for olay in pygame.event.get():
13         if olay.type == pygame.MOUSEBUTTONDOWN:
14             for buton in butonlar:
15                 if buton["rect"].collidepoint(olay.pos):
16                     print(f"{buton['yazi']} butonuna
                        ↳ basıldı!")

```

Bu yaklaşımda butonlar bir listede saklanıyor ve her tıklamada döngüyle kontrol ediliyor. Yeni buton eklemek istediğinde listeye bir sözlük daha eklemeniz yeterli — kontrol kodu değişmez. Bu, Bölüm 8’te öğrendiğin liste ve sözlük kavramlarının pratik bir uygulamasıdır.

● Bilgi: Oyun Menüsü Temelleri

Bu konuda öğrendiğin `collidepoint()` ve buton yönetimi teknikleri, Bölüm 35’te oluşturacağın tam özellikli oyun menü sisteminin temelini oluşturur. Şimdilik bu basit yapı, çarpışma algılama mantığını kavramak için yeterlidir.

Böylece noktasal çarpışma kontrolünün temellerini öğrendin. Fare tıklamaları ve hover efektleri ile basit ama işlevsel butonlar oluşturabilirsin. Bir sonraki konuda, öğrendiğin tüm çarpışma tekniklerini bir araya getirerek pratik bir çarpışma demosu oluşturacaksın.

21.4 Pratik Uygulama: Çarpışma Demosu

Artık Rect sınıfını, `collidect()` ile dikdörtgen çarpışmasını ve `collidepoint()` ile nokta çarpışmasını biliyorsun. Şimdi bu üç kavramı birleştirerek küçük ama eksiksiz bir çarpışma demosu oluşturacağız. Bu demo, ileride Bölüm 22’de öğreneceğin Sprite tabanlı çarpışma sistemine geçişte sağlam bir temel oluşturacak.

21.4.1 Demo Tasarımı

Oluşturacağımız demoda şu mekanikler var: bir oyuncu kutusu ok tuşlarıyla hareket ediyor, ekranda dağınık engeller (kırmızı kutular) bulunuyor ve oyuncu bir engele çarptığında renk değişerek görsel geri bildirim veriliyor. Ayrıca fare ile ekrana tıklandığında yeni engeller ekleniyor.

Bu demo üç temel beceriyi bir arada kullanır:

1. **Rect hareket:** Ok tuşlarıyla `move\_ip()` kullanımı
2. **collidect():** Oyuncu-engel çarpışma kontrolü
3. **collidepoint():** Fare tıklamasıyla engel ekleme

21.4.2 Adım 1: Temel Yapı

Önce oyun penceresini ve oyuncu kutusunu oluşturalım:

Python – Demo – Başlangıç Yapısı

```

1 import pygame
2 import random
3
4 # Sabitler
5 EKLAN_GENISLIK = 800
6 EKLAN_YUKSEKLIK = 600
7 FPS = 60
8 OYUNCU_BOYUT = 40
9 OYUNCU_HIZ = 5
10 ENGEL_BOYUT = 50
11
12 # Renkler
13 BEYAZ = (255, 255, 255)
14 MAVI = (50, 100, 200)
15 KIRMIZI = (200, 50, 50)
16 YESIL = (50, 200, 50)
17 GRI = (40, 40, 40)

```

Sabitlerde dikkat edilmesi gereken nokta, tüm “sihirli sayıların” burada tanımlanmasıdır. Oyuncu boyutunu değiştirmek istediğinde kodun her yerini aramak yerine sadece `OYUNCU_BOYUT` değerini güncelleme yeterli olur.

21.4.3 Adım 2: Engel Oluşturma

Engelleri rastgele konumlarda oluşturacağız. Her engel bir `pygame.Rect` nesnesi olarak listeye eklenir:

Python – Rastgele Engel Oluşturma

```

1 def engel_olustur(adet):
2     """Rastgele konumlarda engel Rect'leri olusturur."""
3     engeller = []
4     for _ in range(adet):
5         x = random.randint(0, EKLAN_GENISLIK -
6             ↪ ENGEL_BOYUT)
7         y = random.randint(0, EKLAN_YUKSEKLIK -
8             ↪ ENGEL_BOYUT)
9         engeller.append(pygame.Rect(x, y, ENGEL_BOYUT,
10             ↪ ENGEL_BOYUT))
11     return engeller

```

`random.randint()` ile rastgele koordinatlar üretiliyor. Üst sınır olarak `EKLAN_GENISLIK - ENGEL_BOYUT` kullanılıyor ki engel ekran dışına taşmasın.

Bu kodda dikkat edilmesi gereken birkaç önemli nokta var.

Birincisi, çarpışma kontrolü her karede oyun döngüsünün güncelleme aşamasında yapılıyor. `collidelistall()` kullanılarak oyuncunun aynı anda birden

fazla engelle çarpışması algılanabiliyor.

İkincisi, fare tıklamasıyla yeni engel eklemek için `collidepoint()` dolaylı olarak kullanılıyor: tıklanan noktaya bir engel `Rect`'i oluşturuluyor. Böylece oyuncu kendi engel parkurunu yaratabilir.

Üçüncüsü, sınır kontrolü `clamp_ip()` ile tek satırda yapılıyor. Bu, dört ayrı `if` koşulu yazmaktan çok daha temiz bir çözüm.

💡 İpucu: Demoyu Geliştir

Bu demoyu kendi başına geliştirmeyi dene. Örneğin engellere çarpınca skor artırabilir, belirli bir süre sonra engeller hareket edebilir veya oyuncu yerine bir Sprite görseli kullanabilirsin. Bir sonraki ünite tam olarak bunu yapacaksın: Sprite tabanlı çarpışma sistemi ile Uzay Kaçışı projesinde düşman ve mermi çarpışmalarını kodlayacaksın.

21.5 Sonuç

Bu ünite oyun geliştirmenin en kritik mekaniklerinden biri olan çarpışma algı-lamanın temellerini öğrendin. `pygame.Rect` sınıfının yapısını, konum özellik-lerini, `colliderect()` ile dikdörtgen çarpışmasını, `collidepoint()` ile fare tıklama algılamayı ve tüm bu teknikleri birleştiren bir pratik demo oluşturdun.

21.5.1 Anahtar Kavramlar

Tablo 21.2: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
<code>Rect</code>	Ekrandaki dikdörtgen bir alanı temsil eden sınıf (x, y, width, height)
<code>AABB</code>	Axis-Aligned Bounding Box; eksenlere paralel dikdörtgenlerle çarpışma algılama yöntemi
<code>colliderect()</code>	İki dikdörtgenin örtüşüp örtüşmediğini kontrol eder
<code>collidepoint()</code>	Bir noktanın dikdörtgenin içinde olup olmadığını kontrol eder
<code>contains()</code>	Bir dikdörtgenin diğerini tamamen kapsayıp kap-samadığını kontrol eder
<code>collidelist()</code>	Bir dikdörtgenin listedeki ilk çarpışan elemanın indeksini döndürür
<code>collidelistall()</code>	Bir dikdörtgenin listedeki tüm çarpışan eleman-ların indekslerini döndürür
<code>move_ip()</code>	<code>Rect</code> 'i yerinde (in-place) hareket ettirir
<code>clamp_ip()</code>	<code>Rect</code> 'i belirtilen sınırlar içinde tutar
<code>Hover</code>	Fare imlecinin bir nesnenin üzerine gelmesi du-rumu

21.5.2 Önemli Komutlar

Tablo 21.3: Ünite 1 – Temel PyGame komutları

Komut	Açıklama
<code>pygame.Rect(x, y, w, h)</code>	Yeni bir dikdörtgen oluşturur
<code>rect.collidect(other)</code>	İki dikdörtgenin çarpışmasını kontrol eder
<code>rect.collidepoint(x, y)</code>	Noktanın dikdörtgen içinde olup olmadığını kontrol eder
<code>rect.contains(other)</code>	Bir dikdörtgenin diğerini tamamen kapsayıp kapsamadığını kontrol eder
<code>rect.collidelist(liste)</code>	Listedeki ilk çarpışan elemanın indeksini döndürür
<code>rect.collidelistall(liste)</code>	Listedeki tüm çarpışan elemanların indekslerini döndürür
<code>rect.move(dx, dy)</code>	Yeni bir Rect döndürür (orijinal değişmez)
<code>rect.move_ip(dx, dy)</code>	Rect'i yerinde hareket ettirir
<code>rect.clamp_ip(other)</code>	Rect'i verilen sınırlar içinde tutar
<code>surface.get_rect(**kw)</code>	Surface'in boyutlarında Rect oluşturur

21.5.3 Sıradaki Adım

Bir sonraki üniteye PyGame'in Sprite tabanlı çarpışma fonksiyonlarını öğreneceksin. `pygame.sprite.spritecollide()` ve `pygame.sprite.groupcollide()` ile Sprite grupları arasında toplu çarpışma kontrolü yapabileceksin. Bu sayede Bölüm 20'te oluşturduğun Uzay Kaçışı projesine düşman çarpışması ve mermi isabet mekaniği ekleyeceksin. Artık “bu iki nesne birbirine değdi mi?” sorusunu güvenle cevaplayabilirsin.

💡 İpucu: Pratik Yap

Bu üniteye öğrendiklerini pekiştirmek için şunu dene: çarpışma demosuna bir skor sistemi ekle. Her fare tıklamasında engel eklenirken skor artırsın, oyuncu bir engeli çarptığında ise skor sıfırlansın. Skoru ekrana yazdırmak için `pygame.font.Font` kullan — bu konuyu henüz görmemiş olsan bile internetten hızlıca öğrenebilirsin.

Sprite Çarpışma Fonksiyonları

“Çarpışma algılama, oyun dünyasındaki nesnelerin birbirinden haberdar olmasını sağlar – onsuz her şey birbirinin içinden geçer.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.sprite.spritecollide()` ile tek bir Sprite'ın bir grupta çarpışmasını tespit etme
- `pygame.sprite.groupcollide()` ile iki Sprite grubu arasındaki çarpışmaları algılama
- `collide_mask`, `collide_circle` ve `collide_rect` ile piksel, daire ve dikdörtgen bazlı çarpışma yöntemlerini karşılaştırma
- Uzay Kaçışı v2 projesinde düşman gemileri, mermi sistemi ve çarpışma algılama uygulaması

22.1 `spritecollide()` – Tek Sprite ile Grup Çarpışması

Önceki ünite de `Rect` nesnelerinin `collidect()` ve `collidepoint()` gibi metotlarıyla iki dikdörtgen arasındaki çarpışmayı elle kontrol etmeyi öğrendin. Peki oyununda yüzlerce düşman varsa, oyuncu gemisinin hangi düşmana çarptığını tek tek kontrol mu edeceksin? Elbette hayır. PyGame'in Sprite modülü tam da bu iş için hazır fonksiyonlar sunar ve işe `pygame.sprite.spritecollide()` ile başlıyoruz.

■ Tanım: `spritecollide()`

`pygame.sprite.spritecollide(sprite, group, dokill)` fonksiyonu, tek bir Sprite ile bir gruptaki tüm Sprite'lar arasında çarpışma kontrolü yapar. Çarpışan Sprite'ların listesini döndürür. Eğer `dokill` parametresi `True` ise, çarpışan Sprite'lar gruplarından otomatik olarak silinir.

Bu fonksiyonu bir örnekle anlamak çok daha kolay. Uzay Kaçışı oyununu düşün: oyuncunun gemisi ekranda hareket ediyor ve yukarıdan düşman gemileri geliyor. Her karede “oyuncu gemisi, düşman grubundaki herhangi bir düşmana çarptı mı?” diye sormalısın. İşte `spritecollide()` tam olarak bu soruyu cevaplar.

22.1.1 Temel Kullanım

Fonksiyonun imzası şöyledir:

Python – `spritecollide` Imzası

```
1 carpisan_liste = pygame.sprite.spritecollide(
2     sprite,      # Kontrol edilecek tek Sprite
3     group,       # Kontrol edilecek grup
4     dokill       # True: carpisan Sprite'lari gruptan sil
5 )
```

Fonksiyon üç parametre alır. Birincisi kontrol edilecek **tek Sprite**'tır — genellikle oyuncu veya mermi gibi özel bir nesne. İkincisi, bu Sprite'ın çarpışıp çarpışmadığını kontrol edeceğin **gruptur**. Üçüncüsü ise çarpışan Sprite'ların gruptan silinip silinmeyeceğini belirleyen `dokill` parametresidir.

Şimdi somut bir örnek yazalım. Diyelim ki bir oyuncu gemisi ve bir düşman grubu var:

Python – `spritecollide` Orneği

```
1 # Her karede carpismayi kontrol et
2 carpisan_dusmanlar = pygame.sprite.spritecollide(
3     oyuncu, dusmanlar, True
4 )
5
6 # Carpisan dusmanlar varsa...
7 for dusman in carpisan_dusmanlar:
8     print(f"Dusmana carptin! Can: -{dusman.hasar}")
9     oyuncu.can -= dusman.hasar
```

Bu kodu satır satır inceleyelim:

1. `spritecollide(oyuncu, dusmanlar, True)` — Oyuncu Sprite'ı ile `dusmanlar` grubundaki her Sprite arasında çarpışma kontrolü yapar.
2. `True` parametresi sayesinde çarpışan düşmanlar otomatik olarak `dusmanlar` grubundan silinir (`kill()` çağrılır).

3. Fonksiyon, çarpışan düşmanların listesini döndürür. Bu liste boşsa çarpışma olmamıştır.
4. Döngü ile her çarpışan düşman için gerekli işlemi yaparız (hasar verme, puan ekleme, patlama efekti başlatma vb.).

22.1.2 `dokill` Parametresinin Etkisi

`dokill` parametresi çarpışma sonrası davranışı belirler ve doğru kullanımı oyun mekaniğine göre değişir.

`dokill=True` kullandığında, çarpışan Sprite'lar tüm gruplarından otomatik olarak çıkarılır — tıpkı Bölüm 20'da öğrendiğin `kill()` metodunun çağırılması gibi. Bu, mermi-düşman çarpışmasında idealdir: mermi düşmana isabet ettiğinde ikisinin de sahneden kalkması gerekir.

`dokill=False` kullandığında ise çarpışan Sprite'lar yerinde kalır. Bu, sürekli temas kontrolü gerektiren durumlarda işe yarar. Örneğin oyuncunun bir güç-artırım (*power-up*) nesnesine dokunup dokunmadığını kontrol ederken, güç-artırımı hemen silmek yerine önce efektini uygulamak isteyebilirsin.

Python – `dokill` Karsilastirmasi

```

1  # Senaryo 1: Mermi dusmana isabet etti -> ikisi de yok
   ↳ olsun
2  vurulanlar = pygame.sprite.spritecollide(mermi, dusmanlar,
   ↳ True)
3
4  # Senaryo 2: Oyuncu guc-artirimina dokundu -> nesneyi
   ↳ hemen silme
5  dokunan = pygame.sprite.spritecollide(oyuncu,
   ↳ guc_artirimlar, False)
6  for nesne in dokunan:
7      oyuncu.hiz += 2
8      nesne.kill() # Efekti uyguladiktan sonra elle sil
```

⚠ Dikkat: `dokill` Sadece Gruptaki Sprite'ları Etkiler

`dokill=True` yalnızca ikinci parametre olan **gruptaki** Sprite'ları siler. Birinci parametre olan Sprite'a (oyuncu gibi) dokunmaz. Oyuncuyu da silmek istiyorsan bunu ayrıca kendin yapmalısın.

22.1.3 Çarpışma Listesi ile Çalışmak

`spritecollide()` her zaman bir **liste** döndürür. Çarpışma yoksa boş liste (`[]`) döner. Bu özellik sayesinde hem çarpışma olup olmadığını hem de kaç nesneyle çarpışıldığını tek bir çağrıyla öğrenebilirsin:

Python – Carpisan Liste Kullanımı

```

1  # Carpisan dusmanlari al
2  vurulanlar = pygame.sprite.spritecollide(oyuncu,
   ↪   dusmanlar, False)
3
4  # Carpista var mi? (bos liste False, dolu liste True)
5  if vurulanlar:
6      print(f"{len(vurulanlar)} dusmana carptin!")
7
8  # Her carpisan dusman icin farkli islem yap
9  for dusman in vurulanlar:
10     if dusman.tur == "buyuk":
11         oyuncu.can -= 20
12     else:
13         oyuncu.can -= 5
14     dusman.kill()

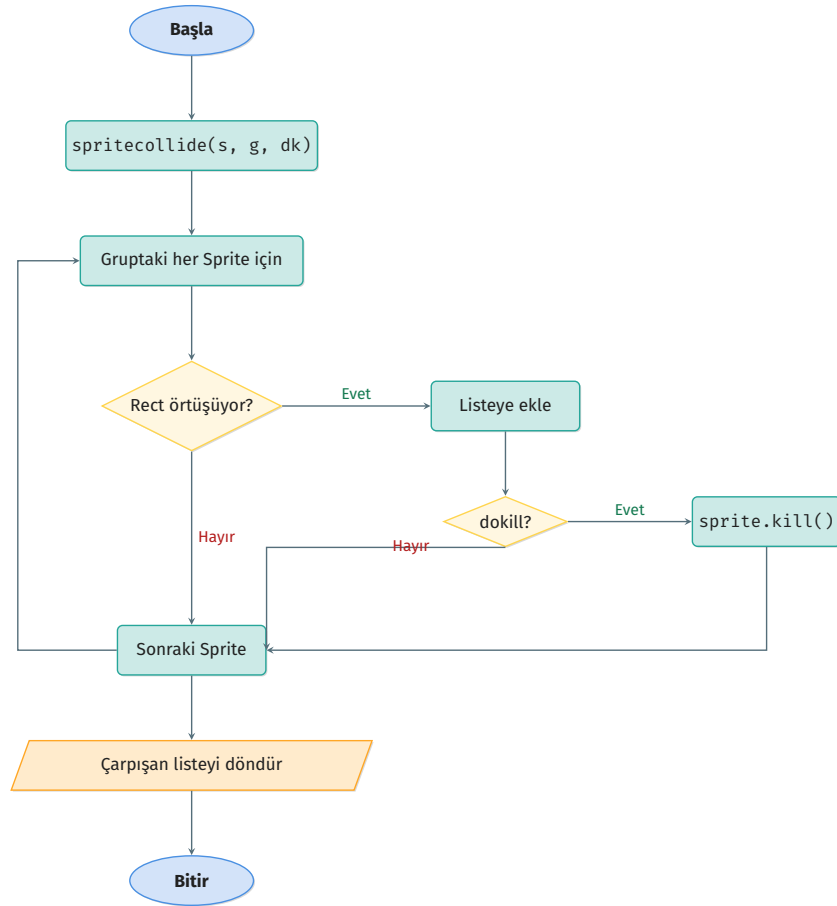
```

Python'da boş liste `False`, en az bir elemanlı liste `True` olarak değerlendirilir. Bu sayede `if vurulanlar:` kontrolü hem okunabilir hem de verimlidir.

💡 İpucu: Performans

`spritecollide()` gruptaki her Sprite ile tek tek çarpışma kontrolü yapar, dolayısıyla grubun boyutu arttıkça süre de artar. Yüzlerce Sprite'lık gruplarda bu fark hissedilir. İlerleyen konuda göreceğin `collide_mask` gibi yöntemler daha hassas ama daha yavaştır; `collide_rect` (varsayılan) ise en hızlısıdır.

Bu programda `spritecollide(oyuncu, yildizlar, True)` çağrısı her karede oyuncunun yıldız grubundaki herhangi bir yıldıza çarpıp çarpmadığını kontrol eder. `dokill=True` sayesinde toplanan yıldızlar otomatik olarak gruptan silinir ve ekrandan kaybolur. Skor her toplanan yıldız için 10 puan artar.

Şekil 22.1: `spritecollide()` fonksiyonunun çalışma akışı

● Bilgi: Arka Planda Neler Oluyor?

`spritecollide()` varsayılan olarak `Rect` tabanlı çarpışma kullanır — yani Konu 21.2’de öğrendiğin `collidect()` mantığının aynısı. Fark şu: sen tek tek döngü yazmak yerine, PyGame bunu senin için otomatik yapar ve sonucu liste olarak döndürür. Daha hassas çarpışma yöntemlerini Konu 22.3’de öğreneceksin.

22.2 `groupcollide()` – İki Grup Arası Çarpışma

Konu 22.1’de tek bir Sprite’ın bir grupta çarpışmasını öğrendin. Peki ya iki grubun **tamamı** birbirleriyle çarpışıyor olabiliyorsa? Örneğin, uzay oyununda onlarca mermi yukarı uçarken onlarca düşman aşağı iniyorsa, hangi merminin hangi düşmana isabet ettiğini nasıl tespit edersin?

Her mermi için ayrı ayrı `spritecollide()` çağırmak bir seçenek olabilir, ama PyGame çok daha zarif bir çözüm sunar: `pygame.sprite.groupcollide()`.

■ Tanım: `groupcollide()`

`pygame.sprite.groupcollide(group1, group2, dokill1, dokill2)` fonksiyonu, iki Sprite grubu arasındaki tüm çarpışmaları tespit eder. Sonucu bir *sözlük* (dictionary) olarak döndürür: anahtarlar `group1`'deki çarpışan Sprite'lar, değerler ise her birinin çarptığı `group2` Sprite'larının listesidir.

Bu fonksiyonu mermi-düşman çarpışması senaryosuyla düşünelim. Mermiler grubu ve düşmanlar grubu var. `groupcollide()` tek bir çağrıyla “hangi mermi hangi düşmana isabet etti?” sorusunun cevabını bir sözlük olarak verir.

22.2.1 Temel Kullanım

Fonksiyonun imzası şöyledir:

Python – `groupcollide` Imzası

```
1  carpisma_sozlugu = pygame.sprite.groupcollide(
2      group1,      # Birinci grup (sozlugin anahtarlari)
3      group2,      # Ikinci grup (sozlugin degerleri)
4      dokill1,     # True: group1'deki carpisan Sprite'lari
5          ↪ sil
6      dokill2      # True: group2'deki carpisan Sprite'lari
7          ↪ sil
8  )
```

Dört parametre alır: iki grup ve iki `dokill` bayrağı. `dokill1` birinci gruptaki çarpışan Sprite'ları silerken, `dokill2` ikinci gruptakileri siler. İkisini birbirinden bağımsız olarak `True` veya `False` yapabilirsin.

Şimdi klasik mermi-düşman senaryosunu yazalım:

Python – `groupcollide` Örneği

```
1  # Her karede mermi-dusman carpismasini kontrol et
2  carpismalar = pygame.sprite.groupcollide(
3      mermiler, dusmanlar, True, True
4  )
5
6  # Carpismalar sozlugunu isle
7  for mermi, vurulan_dusmanlar in carpismalar.items():
8      for dusman in vurulan_dusmanlar:
9          skor += dusman.puan
10         # Patlama efekti baslat (ileride ogreneceksin)
```

Bu kodu adım adım inceleyelim:

1. `groupcollide(mermiler, dusmanlar, True, True)` — Mermiler ve düşmanlar grupları arasındaki tüm çarpışmaları tespit eder. Her iki grup için de `dokill=True` olduğundan, çarpışan mermiler ve düşmanlar otomatik silinir.

2. Döndürülen sözlükte her anahtar bir **mermi**, her değer ise o merminin isabet ettiği **düşmanların listesi**dir.
3. `items()` ile sözlük üzerinde döngü kurarak her isabet için skor artırıyoruz.

22.2.2 Sözlük Yapısını Anlamak

`groupcollide()`'ın döndürdüğü sözlük yapısı ilk bakışta karmaşık gelebilir, ama aslında çok mantıklıdır. Bir örnek üzerinden görelim:

Python – Sozluk Yapisi Ornegi

```

1  # Diyelim ki:
2  # mermi_A -> dusman_1 ve dusman_2'ye isabet etti
3  # mermi_B -> dusman_3'e isabet etti
4  # mermi_C -> hicbir seye isabet etmedi
5
6  # groupcollide() su sozlugu dondurur:
7  # {
8  #     mermi_A: [dusman_1, dusman_2],
9  #     mermi_B: [dusman_3]
10 # }
11 # mermi_C sozlukte YOK (hicbir seye carpmadigindan)

```

Dikkat edersen, hiçbir şeye çarpmayan Sprite'lar sözlükte yer almaz. Bu sayede sözlük boşsa (`if not carpsmalar:`) hiç çarpışma olmadığını hemen anlarsın.

Bir merminin birden fazla düşmana aynı anda isabet edebileceğini de unutma. Eğer mermi yeterince büyükse veya iki düşman yan yana duruyorsa, tek mermi birden fazla düşmanı vurabilir. Sözlükteki liste yapısı tam da bu durumu modeller.

22.2.3 dokill Kombinasyonları

İki grup için iki ayrı `dokill` parametresi olduğundan, dört farklı kombinasyon mümkündür. Her biri farklı bir oyun mekaniğine uygun düşer:

Tablo 22.1: `groupcollide()` – `dokill` kombinasyonları ve kullanım senaryoları

dokill1	dokill2	Senaryo
True	True	Mermi düşmana çarptı: ikisi de yok olsun
True	False	Mermi kalkan vurdu: mermi yok olsun, kalkan kalsın
False	True	Lazer sürekli ateş: lazer kalsın, düşman yok olsun
False	False	Sadece temas kontrolü: hiçbir şeyi silme

Mermi-düşman senaryosunda genellikle (`True, True`) kullanırsın: mermi isabet edince hem mermi hem düşman yok olur. Ama diyelim ki oyununda “delici mermi” var ve birden fazla düşmana isabet edebiliyorsa, o zaman (`False, True`) kullanırsın — mermi devam eder, düşmanlar yok olur.

Python – Delici Mermi Ornegi

```

1  # Normal mermi: isabet edince yok olur
2  carpismalar = pygame.sprite.groupcollide(
3      normal_mermiler, dusmanlar, True, True
4  )
5
6  # Delici mermi: isabet etse bile devam eder
7  carpismalar = pygame.sprite.groupcollide(
8      delici_mermiler, dusmanlar, False, True
9  )

```

Bu programda `groupcollide(mermiler, hedefler, True, True)` her karede tüm mermi-hedef çarpışmalarını kontrol eder. Hem çarpışan mermi hem çarpışan hedef gruplarından silinir. Dikkat edersen, oyuncu Space tuşuna bastığında yeni bir `Mermi` Sprite'ı oluşturulup hem `tum_spritelar` hem de `mermiler` grubuna ekleniyor — Bölüm 20'da öğrendiğin çoklu grup yönetiminin tam da burada işe yaradığını görüyorsun.

⚠ Dikkat: groupcollide ile spritecollide Farkı

`spritecollide()` tek bir Sprite ile bir grup arasında çarpışma kontrol eder ve **liste** döndürür. `groupcollide()` ise iki grup arasında çarpışma kontrol eder ve **sözlük** döndürür. Yanlış fonksiyonu kullanırsan beklemediğin sonuçlar alabilirsin. Kural basit: oyuncuyla düşmanlar → `spritecollide()`, mermilerle düşmanlar → `groupcollide()`.

● Bilgi: Ne Zaman Hangisi?

Pratikte çoğu oyunda her ikisini birlikte kullanırsın. Uzay Kaçışı v2'de tam da bunu yapacağız: oyuncunun düşmana çarpmasını `spritecollide()` ile, mermilerin düşmanlara isabet etmesini `groupcollide()` ile kontrol edeceğiz. Bu ikili kullanım Sprite tabanlı oyunların standart kalıbıdır.

22.3 Çarpışma Yöntemleri: Rect, Circle ve Mask

Şimdiye kadar öğrendiğin tüm çarpışma kontrolleri **dikdörtgen tabanlıydı**. Py-Game varsayılan olarak her Sprite'ın `rect` alanını kullanarak çarpışma tespiti yapar. Bu yöntem hızlıdır ama her zaman doğru sonuç vermez. Neden mi?

Düşün: yuvarlak bir top Sprite'ı var ve `rect`'i karesel bir dikdörtgen. Topun köşeleri aslında boş (şeffaf) olmasına rağmen, dikdörtgen tabanlı çarpışma bu köşelere de “çarpmış” gibi davranır. Oyuncu “ama ben topa değmedim!” diye haklı olarak sızlanır. İşte bu sorunu çözmek için PyGame üç farklı çarpışma yöntemi sunar.

■ Tanım: Çarpışma Yöntemleri

PyGame'in Sprite çarpışma fonksiyonları (`spritecollide` ve `groupcollide`) son parametre olarak bir `collided` fonksiyonu alabilir. Bu fonksiyon çarpışmanın **nasıl** tespit edileceğini belirler:

1. `collide_rect` – Dikdörtgen tabanlı (varsayılan, en hızlı)
2. `collide_circle` – Daire tabanlı (yuvarlak nesneler için)
3. `collide_mask` – Piksel tabanlı (en hassas, en yavaş)

22.3.1 `collide_rect` – Dikdörtgen Çarpışma

Bu, şimdiye kadar kullandığın varsayılan yöntemdir. `spritecollide()` ve `groupcollide()` fonksiyonlarını son parametre olmadan çağırdığında PyGame otomatik olarak `collide_rect` kullanır.

Python – `collide_rect` Kullanımı

```
1 # Bu iki cagri ayni isi yapar:
2 carpisan = pygame.sprite.spritecollide(oyuncu, dusmanlar,
3     ↪ False)
4 carpisan = pygame.sprite.spritecollide(
5     oyuncu, dusmanlar, False, pygame.sprite.collide_rect
6 )
```

Dikdörtgen tabanlı çarpışma Konu 21.2'da öğrendiğin `collidect()` mantığının aynısıdır: iki `Rect`'in örtüşüp örtüşmediğini kontrol eder. Hesaplama çok hızlıdır çünkü sadece dört sayıyı (x, y, genişlik, yükseklik) karşılaştırır. Dikdörtgen şekilli nesneler (tuğlalar, platformlar, kutular) için idealdir.

Dezavantajı, şeffaf pikselleri hesaba katmamasıdır. Üçgen şeklinde bir uzay gemisinin `rect`'i dikdörtgendir ve köşelerdeki boş alanlar “çarpışma” olarak algılanır.

22.3.2 `collide_circle` – Daire Çarpışma

Toplar, mermiler, gezegenler gibi yuvarlak nesneler için daire tabanlı çarpışma çok daha doğru sonuç verir. Bu yöntem, iki Sprite'ın merkez noktaları arasındaki mesafeyi yarıçaplarının toplamıyla karşılaştırır.

Python – `collide_circle` Kullanımı

```
1 # Daire tabali carpisma
2 carpisan = pygame.sprite.spritecollide(
3     top, engeller, False, pygame.sprite.collide_circle
4 )
```

PyGame, Sprite'ın yarıçapını otomatik olarak `rect` boyutundan hesaplar. Ama kendi yarıçapını tanımlamak istersen `radius` özelliğini kullanabilirsin:

Python – Özel Yaricap Tanımlama

```

1 class Top(pygame.sprite.Sprite):
2     def __init__(self, x, y, yaricap):
3         super().__init__()
4         self.image = pygame.Surface(
5             (yaricap * 2, yaricap * 2), pygame.SRCALPHA
6         )
7         pygame.draw.circle(
8             self.image, (255, 0, 0),
9             (yaricap, yaricap), yaricap
10        )
11        self.rect = self.image.get_rect(center=(x, y))
12        # Özel yarıcap tanımla (collide_circle bunu
13        ↪ kullanır)
14        self.radius = yaricap

```

`collide_circle` yöntemi dikdörtgen yönteme göre biraz daha yavaştır çünkü mesafe hesabı yapar. Ancak yuvarlak nesnelerde çok daha doğru sonuç verir ve performans farkı çoğu oyunda hissedilmez.

22.3.3 `collide_mask` – Piksel Bazlı Çarpışma

En hassas çarpışma yöntemi `collide_mask`'tır. Bu yöntem, Sprite'ın görselindeki **her pikseli** kontrol eder ve sadece opak (şeffaf olmayan) pikseller örtüştüğünde çarpışma tespit eder.

Python – `collide_mask` Kullanımı

```

1 # Piksel bazli carpisma
2 carpisan = pygame.sprite.spritecollide(
3     oyuncu, dusmanlar, False, pygame.sprite.collide_mask
4 )

```

`collide_mask` kullanmak için Sprite'larının bir `mask` özelliğine sahip olması gerekir. Bu maskeyi `pygame.mask.from_surface()` fonksiyonuyla oluşturabilirsin:

Python – Mask Oluşturma

```

1 class Gemi(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.image = pygame.image.load(
5             "gemi.png"
6         ).convert_alpha()
7         self.rect = self.image.get_rect(center=(400, 500))
8         # Piksel bazli carpisma icin maske olustur
9         self.mask = pygame.mask.from_surface(self.image)

```

⚠ Dikkat: Mask Tanımlamayı Unutma

`collide_mask` kullanıyorsan, çarpışma kontrolüne dahil olan **her iki** Sprite'ın da `self.mask` özelliğine sahip olması gerekir. Birinde bile eksikse PyGame maskeyi çalışma zamanında otomatik oluşturmaya çalışır ama bu hem yavaş hem de beklenmedik sonuçlar doğurabilir. En iyisi `pygame.init()` içinde `self.mask = pygame.mask.from_surface(self.image)` satırını her zaman eklemektir.

22.3.4 Üç Yöntemin Karşılaştırması

Hangi durumda hangi yöntemi kullanacağını bilmek önemlidir:

Tablo 22.2: Çarpışma yöntemlerinin karşılaştırması

Yöntem	Hassasiyet	Hız	Uygun Nesneler
<code>collide_rect</code>	Düşük	En hızlı	Kutular, tuğlalar, platformlar
<code>collide_circle</code>	Orta	Hızlı	Toplar, mermiler, gezegenler
<code>collide_mask</code>	En yüksek	Yavaş	Düzensiz şekiller, detaylı görseller

Genel kural şudur: mümkün olduğunca basit yöntemi kullan. Çoğu oyunda `collide_rect` yeterlidir. Yuvarlak nesnelerde `collide_circle`'a geç. Sadece gerçekten piksel hassasiyeti gerektiğinde (örneğin karmaşık şekilli boss düşmanlar) `collide_mask` kullan.

💡 İpucu: Hibrit Yaklaşım

Bazı profesyonel oyunlarda “kaba-ince” (*broad-narrow phase*) çarpışma stratejisi uygulanır: önce hızlı `collide_rect` ile olası çarpışmaları filtrele, sonra sadece bu adaylar üzerinde yavaş `collide_mask` uygula. PyGame'in yapısı bunu doğrudan desteklemez ama kendin kodlayabilirsin.

Bu örneği çalıştırıp fare ile gemiyi hareket ettirdiğinde, dikdörtgen yönteminin köşelerde “yanlış alarm” verdiğini, daire yönteminin daha iyi ama tam doğru olmadığını ve mask yönteminin sadece gerçekten pikseller örtüştüğünde çarpışma tespit ettiğini göreceksin.

● Bilgi: Mask ve Döndürme

Eğer Sprite'ını `pygame.transform.rotate()` ile döndürüyorsan, **maski de güncellemelisin**. Döndürme yeni bir `image` oluşturduğundan, `self.mask` eski görsele ait kalır. `update()` içinde döndürme yaptıktan sonra `self.mask = pygame.mask.from_surface(self.image)` satırını eklemeyi unutma.

22.4 Mini Proje: Uzay Kaçışı v2 – Düşmanlar ve Mermiler

Bölüm 20’da Uzay Kaçışı v1 ile oyuncu gemisi ve kayan yıldız arka planını oluşturmuştun. Konu 20.4’deki yol haritasında belirttiğimiz gibi, şimdi v2 ile düşman gemileri, mermi sistemi ve çarpışma algılama eklemenin zamanı geldi. Bu ünite de öğrendiğin `spritecollide()` ve `groupcollide()` fonksiyonlarını gerçek bir projede uygulayacaksın.

22.4.1 v1’den v2’ye: Neler Değişiyor?

Uzay Kaçışı v1’de sadece oyuncu gemisi ve kayan yıldızlar vardı. v2’de şu yenilikler ekleniyor:

Tablo 22.3: Uzay Kaçışı v1 ve v2 karşılaştırması

Özellik	v1 (Bölüm 5)	v2 (Bölüm 6)
Oyuncu gemisi	✓	✓
Kayan yıldızlar	✓	✓
Düşman gemileri	–	✓
Mermi sistemi	–	✓
Çarpışma algılama	–	✓
Can sistemi	–	✓
Puan sistemi	Süre bazlı	Düşman vurma bazlı

22.4.2 Yeni Sprite Sınıfları

v2 için iki yeni Sprite sınıfı ekliyoruz: `Dusman` ve `Mermi`.

Dusman Sınıfı

Düşman gemileri ekranın üstünden rastgele konumlarda belirir ve aşağı doğru hareket eder. Ekranın altından çıkınca otomatik olarak yok olur:

Python – Dusman Sınıfı

```

1 class Dusman(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.image = gorsel_yukle("dusman.png", (40, 35))
5         self.rect = self.image.get_rect()
6         # Rastgele x konumunda, ekranın üstünde basla
7         self.rect.x = random.randint(0, GENISLIK -
8             ↪ self.rect.width)
9         self.rect.bottom = 0
10        self.hiz = random.uniform(2, 5)
11        self.puan = 10
12
13    def update(self):

```

```

13     self.rect.y += self.hiz
14     # Ekranın altından cikinca yok ol
15     if self.rect.top > YUKSEKLIK:
16         self.kill()

```

Burada önemli bir tasarım kararı var: düşman ekranın altından çıktığında `kill()` çağrılıyor. Bu, Bölüm 20’da öğrendiğin gibi Sprite’ı tüm gruplarından çıkarır. Böylece ekran dışına çıkan düşmanlar bellekte birikmez.

Her düşmanın rastgele bir hızı var (`random.uniform(2, 5)`), bu da oyuna çeşitlilik katar. `puan` özelliği ise bu düşman vurulduğunda kazanılacak puanı tutar — ileride farklı düşman tipleri ekleyerek farklı puan değerleri atayabilirsin.

Mermi Sınıfı

Mermi, oyuncunun Space tuşuna bastığında oluşur ve yukarı doğru hareket eder:

Python – Mermi Sınıfı

```

1 class Mermi(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         self.image = pygame.Surface((4, 12),
5             ↪ pygame.SRCALPHA)
6         pygame.draw.rect(self.image, SARI, (0, 0, 4, 12))
7         self.rect = self.image.get_rect(center=(x, y))
8         self.hiz = -8 # Yukarı doğru
9
10    def update(self):
11        self.rect.y += self.hiz
12        # Ekranın üstünden cikinca yok ol
13        if self.rect.bottom < 0:
14            self.kill()

```

Mermi sınıfında birkaç noktaya dikkat edelim. Birincisi, merminin konumu oyuncu gemisinden gelir — `__init__()` metoduna `x` ve `y` parametreleri geçiriyoruz. İkincisi, hız **negatif** çünkü mermi yukarı doğru hareket ediyor (PyGame’de `y` eksenini aşağı doğru artır, Bölüm 10’da öğrendiğin gibi). Üçüncüsü, ekranın üstünden çıkan mermiler de `kill()` ile temizleniyor.

22.4.3 Sprite Grupları

v2’de beş ayrı Sprite grubu kullanıyoruz:

Python – Grup Yapısı

```

1 tum_spritelar = pygame.sprite.Group() # Cizim icin
2 yildizlar = pygame.sprite.Group()     # Arka plan
3     ↪ yildizlari
4 dusmanlar = pygame.sprite.Group()     # Dusman gemileri
5 mermiler = pygame.sprite.Group()      # Oyuncu mermileri

```

```

5
6 # Oyuncuyu olustur
7 oyuncu = Oyuncu()
8 tum_spritelar.add(oyuncu)

```

Neden bu kadar çok grup? Çünkü her grubun ayrı bir görevi var. `tum_spritelar` çizim ve genel güncelleme için kullanılır. `dusmanlar` ve `mermiler` grupları ise çarpışma kontrolü için ayrı tutulur: `groupcollide(mermiler, dusmanlar, ...)` çağırısı bu iki gruba ihtiyaç duyar.

● Bilgi: Bir Sprite Birden Fazla Grupta Olabilir

Bölüm 20’da öğrendiğin gibi, bir Sprite aynı anda birden fazla gruba üye olabilir. Bir düşman hem `tum_spritelar` hem de `dusmanlar` grubundadır. `kill()` çağrıldığında tüm gruplarından çıkar. Bu sayede düşman vurulduğunda hem çizimden hem çarpışma kontrolünden otomatik olarak çıkarılır.

22.4.4 Düşman Üretme (Spawn) Sistemi

Düşmanları belirli aralıklarla üretmek için zamanlayıcı kullanıyoruz. Bu, v1’deki yıldız ekleme mantığına benzer:

Python – Dusman Spawn Sistemi

```

1 DUSMAN_OLAY = pygame.USEREVENT + 1
2 pygame.time.set_timer(DUSMAN_OLAY, 800) # 800 ms arayla
3
4 # Ana dongude:
5 for olay in pygame.event.get():
6     if olay.type == DUSMAN_OLAY:
7         dusman = Dusman()
8         tum_spritelar.add(dusman)
9         dusmanlar.add(dusman)

```

`pygame.time.set_timer()` fonksiyonu belirtilen süre aralığında tekrarlayan bir olay oluşturur. Her 800 milisaniyede bir `DUSMAN_OLAY` tetiklenir ve yeni bir düşman sahneye eklenir. Bu yöntem, ana döngüde zaman kontrolü yapmaktan daha temizdir ve PyGame’in olay sistemini kullanır.

22.4.5 Mermi Ateşleme

Oyuncu Space tuşuna bastığında mermi oluşturuyoruz:

Python – Mermi Ateşleme

```

1 for olay in pygame.event.get():
2     if olay.type == pygame.KEYDOWN:
3         if olay.key == pygame.K_SPACE:

```

```

4         mermi = Mermi(oyuncu.rect.centerx,
5                       oyuncu.rect.top)
6         tum_spritelar.add(mermi)
7         mermiler.add(mermi)

```

Merminin başlangıç konumu, oyuncu gemisinin üst orta noktasıdır (`rect.centerx` ve `rect.top`). Bu sayede mermi geminin burnundan fırlamış gibi görünür.

22.4.6 Çarpışma Kontrolü

İşte bu ünitenin asıl konusu. İki tür çarpışma kontrol ediyoruz:

Python – Carpisma Kontrolleri

```

1  # 1. Mermi-dusman carpismalari (groupcollide)
2  carpismalar = pygame.sprite.groupcollide(
3      mermiler, dusmanlar, True, True
4  )
5  for mermi, vurulan_dusmanlar in carpismalar.items():
6      for dusman in vurulan_dusmanlar:
7          skor += dusman.puan
8
9  # 2. Oyuncu-dusman carpismalari (spritecollide)
10 if oyuncu.alive():
11     vurulan = pygame.sprite.spritecollide(
12         oyuncu, dusmanlar, True
13     )
14     for dusman in vurulan:
15         oyuncu.can -= 25
16         if oyuncu.can <= 0:
17             oyuncu.kill()

```

Birinci kontrolde `groupcollide()` ile mermiler ve düşmanlar grupları arasındaki çarpışmaları tespit ediyoruz. `True, True` parametreleri sayesinde hem çarpışan mermi hem çarpışan düşman otomatik olarak tüm gruplarından silinir. Her vurulan düşman için puan ekleniyor.

İkinci kontrolde `spritecollide()` ile oyuncu gemisinin düşmanlara çarpıp çarpmadığını kontrol ediyoruz. Düşman çarparsa oyuncunun canı azalır. Can sıfırın altına düşerse oyuncu `kill()` ile yok edilir ve oyun biter.

⚠ Dikkat: `alive()` Kontrolü

Oyuncunun `kill()` ile yok edilmesinden sonra çarpışma kontrolü yapmaya devam edersen hata alabilirsin. Bu yüzden çarpışma kontrolünden önce `oyuncu.alive()` ile oyuncunun hala aktif olup olmadığını kontrol ediyoruz.

22.4.7 Can ve Skor Gösterimi (HUD)

Oyuncunun canını ve skoru ekranda göstermek için basit bir `hud_ciz()` fonksiyonu kullanıyoruz:

Python – HUD Fonksiyonu

```

1 def hud_ciz(ekran, font, skor, can):
2     """Skor ve can bilgisini ekrana ciz."""
3     # Skor (sol ust)
4     skor_yazi = font.render(f"Skor: {skor}", True, BEYAZ)
5     ekran.blit(skor_yazi, (15, 15))
6
7     # Can cubugu (sag ust)
8     cubuk_genislik = 200
9     cubuk_yukseklik = 20
10    doluluk = max(0, can / 100) * cubuk_genislik
11    # Arka plan (kirmizi)
12    pygame.draw.rect(ekran, KIRMIZI,
13                     (GENISLIK - 215, 15,
14                      cubuk_genislik, cubuk_yukseklik))
15    # Dolu kisim (yesil)
16    pygame.draw.rect(ekran, YESIL,
17                     (GENISLIK - 215, 15,
18                      doluluk, cubuk_yukseklik))

```

Can çubuğu oyuncunun `can` değerine göre doluluk oranını hesaplar. 100 can = tam dolu yeşil çubuk. Can azaldıkça yeşil kısım kısalır ve altındaki kırmızı görünür hale gelir. Bu basit ama etkili bir görsel geri bildirimdir.

Bu programı çalıştırdığında v1'deki kayan yıldız arka planına ek olarak şunları göreceksin: yukarıdan inen düşman gemileri, Space tuşuyla ateşlenen sarı mermiler, mermilerin düşmanlara isabet ettiğinde her ikisinin de yok olması, düşmanın oyuncuya çarpması durumunda can kaybı ve can sıfıra düşünce oyun bitmesi.

💡 İpucu: v2'yi Geliştir

Bu temel yapı üzerine şu eklentileri deneyebilirsin:

- Farklı düşman tipleri ekle (büyük = yavaş ama çok puanlı, küçük = hızlı ama az puanlı)
- Düşmanları da mermi ateşletir yap
- Belirli puanlarda güç-artırım (*power-up*) düşür
- Can sıfıra düşünce "Game Over" ekranı göster
- Zaman geçtikçe düşman sayısını ve hızını artır

📌 Bilgi: Sıradaki Versiyon

Bölüm 7'de Uzak Kaçışı v3 ile ses efektleri ve arka plan müziği ekleyeceğiz. Mermi ateşleme sesi, düşman patlama sesi ve epik bir uzay müziği ile oyun deneyimi çok daha sürükleyici olacak.

22.5 Sonuç

22.5.1 Ünite 2 Özeti

Bu ünite de PyGame'in Sprite tabanlı çarpışma fonksiyonlarını öğrendin. Konu 22.1'de `pygame.sprite.spritecollide()` ile tek bir Sprite'ın bir grupta çarpışmasını tespit etmeyi, `dokill` parametresinin etkisini ve döndürülen listeyle çalışmayı kavradın.

Konu 22.2'de `pygame.sprite.groupcollide()` ile iki grup arasındaki çarpışmaları sözlük olarak almayı öğrendin. Mermi-düşman senaryosunda hangi merminin hangi düşmana isabet ettiğini tek bir fonksiyon çağrısıyla tespit edebildin.

Konu 22.3'da üç farklı çarpışma yöntemini karşılaştırdın: `collide_rect` (hızlı, dikdörtgen), `collide_circle` (orta, daire) ve `collide_mask` (hassas, piksel). Her birinin avantajlarını, dezavantajlarını ve uygun kullanım senaryolarını gördün.

Son olarak Konu 22.4'da tüm bu bilgileri Uzay Kaçışı v2 projesinde bir araya getirdin: düşman gemileri, mermi sistemi, çarpışma algılama, can çubuğu ve puan sistemi ile gerçek bir oyun deneyimi oluşturdu.

22.5.2 Anahtar Kavramlar

Tablo 22.4: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
<code>spritecollide()</code>	Tek Sprite ile grup arası çarpışma (liste döndürür)
<code>groupcollide()</code>	İki grup arası çarpışma (sözlük döndürür)
<code>dokill</code>	Çarpışan Sprite'ları otomatik silme bayrağı
<code>collide_rect</code>	Dikdörtgen tabanlı çarpışma (varsayılan, hızlı)
<code>collide_circle</code>	Daire tabanlı çarpışma (yuvarlak nesneler)
<code>collide_mask</code>	Piksel tabanlı çarpışma (en hassas)
<code>Mask</code>	Sprite'ın opak piksellerinden oluşan ikili harita
<code>alive()</code>	Sprite'ın hala aktif olup olmadığını kontrol eder

22.5.3 Önemli Komutlar

Tablo 22.5: Ünite 2 – Önemli PyGame komutları

Komut	İşlev
<code>spritecollide(s, g, dk)</code>	Sprite-grup çarpışma kontrolü
<code>groupcollide(g1, g2, dk1, dk2)</code>	Grup-grup çarpışma kontrolü
<code>collide_rect</code>	Dikdörtgen çarpışma fonksiyonu
<code>collide_circle</code>	Daire çarpışma fonksiyonu
<code>collide_mask</code>	Piksel çarpışma fonksiyonu
<code>mask.from_surface(img)</code>	Görselden maske oluşturur
<code>sprite.alive()</code>	Sprite aktif mi kontrol eder
<code>time.set_timer(olay, ms)</code>	Tekrarlayan zamanlayıcı olay

22.5.4 Sıradaki Adım

Bir sonraki ünite de **Temel Fizik Simülasyonu** konusuna geçeceğiz. Yerçekimi, ivme, sürtünme ve yay fiziği gibi temel fizik kavramlarını PyGame ile simüle etmeyi öğreneceksin. Bu bilgiler özellikle platform oyunlarında zıplama mekaniği ve top fiziği simülasyonlarında çok işe yarayacak.

● Bilgi: Pratik Yap

Bu ünitenin kavramlarını pekiştirmek için Uzay Kaçışı v2'yi temel alarak şu eklentileri dene:

- Farklı renkli (kırmızı, mavi, yeşil) düşmanlar ekle ve her birine farklı puan değeri ver
- Belirli puanlara ulaşınca zorluk seviyesini artır (düşman hızı ve sayısı artsın)
- Oyuncunun mermi ateşleme hızına sınır koy (örneğin her 200 ms'de bir mermi)
- `collide_mask` yöntemini kullanarak daha hassas çarpışma algılama uygula
- Oyun bitince skor ve hayatta kalma süresini ekrana yazdır

Temel Fizik Simülasyonu

“Fizik, oyunların gerçekçi hissettiren gizli kahramanıdır. Oyuncunun farkında bile olmadan doğru hissettiren her hareket, arka planda çalışan fizik simülasyonunun eseridir.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Yerçekimi vektörünü simüle ederek nesnelerin düşmesini sağlama
- Hız ve momentum kavramlarını oyun fiziğine uygulama
- Sürtünme ve hava direnci ile gerçekçi hareket modelleme
- Tüm fizik kavramlarını birleştiren bir **PhysicsBody** sınıfı tasarlama

23.1 Yerçekimi Simülasyonu

Bir platformer oyunu düşün: karakterin zıplayıp havada asılı kalıyor ve sonra yere düşüyor. Ya da bir Angry Birds tarzı oyunda kuşu fırlatıyorsun ve kuş güzel bir yay çizerek hedefe iniyor. Bu hareketlerin hepsinin arkasında basit ama etkili bir kavram var: *yerçekimi simülasyonu*.

■ Tanım: Oyun Yerçekimi

Oyun yerçekimi, her karede bir nesnenin dikey hızına sabit bir değer ekleyerek aşağı doğru ivmelenmesini sağlayan simülasyondur. Gerçek fizikte yerçekimi ivmesi $g = 9.8 \text{ m/s}^2$ olsa da oyunlarda bu değer **oynanışa göre ayarlanır**; gerçekçilik değil, eğlencelilik ön plandadır.

Bunu günlük hayattaki bir deneyimle düşün: elindeki topu bıraktığında top başta yavaş düşer, sonra giderek hızlanır. İşte oyun fiziğinde de aynı prensip

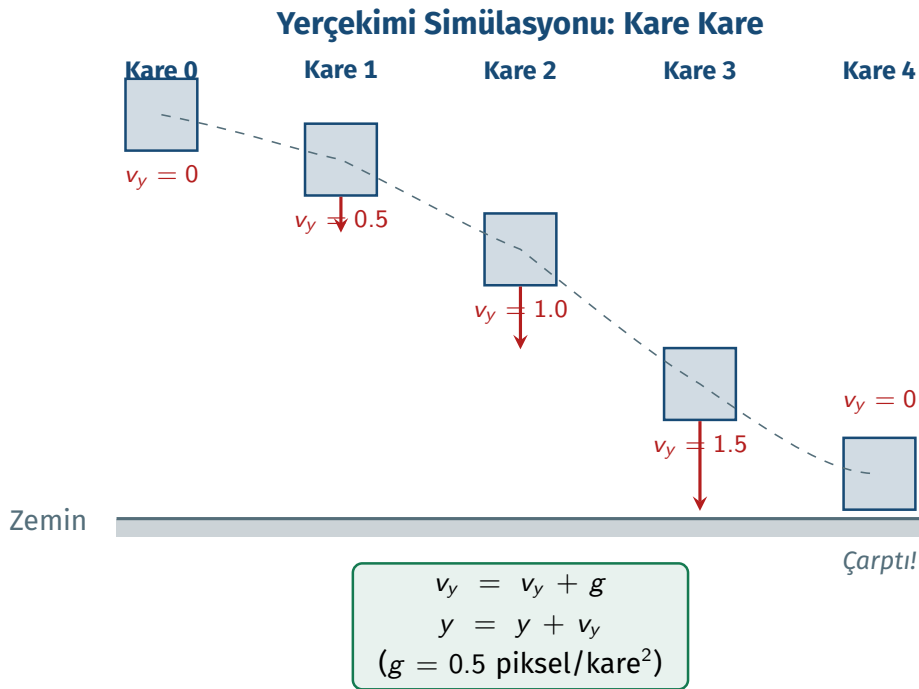
geçerli. Her karede nesnenin dikey hızına küçük bir miktar ekliyorsun ve nesne giderek daha hızlı düşüyor.

23.1.1 Yerçekimi Nasıl Çalışır?

Yerçekimi simülasyonunun matematiği son derece basittir. Her karede şu iki adım uygulanır:

1. Dikey hıza yerçekimi değerini ekle: $v_y = v_y + g$
2. Konumu hıza göre güncelle: $y = y + v_y$

Bu kadar! İki satırlık bir formülle gerçekçi bir düşme hareketi elde ediyorsun. Bu sürecin kare kare nasıl işlediğini Şekil 23.1’de görebilirsin.



Şekil 23.1: Yerçekimi simülasyonunun kare kare işleyişi: her karede hız artar ve nesne giderek daha hızlı düşer.

Şekil 23.1’de görüldüğü gibi nesne başlangıçta durağanken ($v_y = 0$), her karede hızı 0.5 piksel/kare artırıyor. Kareler arasındaki mesafe giderek büyüyor çünkü hız sürekli artıyor. Nesne zemine çarptığında hız sıfırlanıyor ve nesne duruyor.

Python – Temel Yerçekimi Formülü

```

1  # Yerçekimi sabiti (piksel/kare^2)
2  YERCEKIMI = 0.5
3
4  # Her karede calisacak guncelleme
5  hiz_y = 0 # Baslangicta nesne duruyor
6
7  # Oyun dongusu icinde:
8  hiz_y += YERCEKIMI # Hiz her karede artar

```

```
9 nesne_y += hiz_y # Konum hiza gore guncellenir
```

Dikkat edersen **YERCEKIMI** değeri 0.5 gibi küçük bir sayı. Gerçek dünyadaki 9.8 m/s^2 değerini kullanmıyoruz çünkü piksel tabanlı bir koordinat sisteminde çalışıyoruz. Bu değeri oyunun hissiyatına göre ayarlayacaksınız: düşük değerler ay yürüyüşü gibi yavaş düşüş, yüksek değerler ise hızlı ve sert bir düşüş hissi verir.

● Bilgi: Yerçekimi Değerini Ayarlama

Yerçekimi sabiti için kesin bir kural yoktur. Platform oyunlarında genellikle 0.3 ile 1.0 arasında bir değer iyi sonuç verir. Super Mario tarzı oyunlarda karakterin havada kalma süresi uzundur (düşük yerçekimi), Celeste gibi oyunlarda ise düşüş hızlı ve keskindir (yüksek yerçekimi). Oyununu test ederek en eğlenceli değeri bul.

23.1.2 Düşme Simülasyonu

Şimdi bu kavramı bir PyGame sprite'ına uygulayalım. Bölüm 19'te öğrendiğin `pygame.sprite.Sprite` sınıfından türeterek yerçekimine tabi bir nesne oluşturalım:

Python – Yerçekimine Tabi Sprite

```
1 class DusenNesne(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         self.image = pygame.Surface((30, 30))
5         self.image.fill((255, 100, 100))
6         self.rect = self.image.get_rect(topleft=(x, y))
7
8         # Fizik degiskenleri
9         self.hiz_y = 0
10        self.yercekimi = 0.5
11
12    def update(self):
13        # Yercekimi hizi etkiler
14        self.hiz_y += self.yercekimi
15
16        # Konum guncellenir
17        self.rect.y += self.hiz_y
18
19        # Zemin kontrolu (ekranin altı)
20        if self.rect.bottom >= 580:
21            self.rect.bottom = 580
22            self.hiz_y = 0
```

Bu kodda dikkat edilmesi gereken noktalar:

1. `self.hiz_y` değişkeni nesnenin dikey hızını tutar
2. Her `update()` çağrısında hıza yerçekimi eklenir

3. Konum, güncellenmiş hıza göre değişir
4. Zemin kontrolü ile nesnenin ekranın altından geçmesi engellenir

Zemin kontrolüne dikkat et: nesne zemine ulaştığında `hiz_y` sıfırlanıyor. Bu olmadan nesne ekranın dışına çıkardı. Ayrıca `self.rect.bottom` kullanarak nesnenin alt kenarını zemin seviyesine eşitliyoruz; böylece nesne yerin içine gömülüyor.

23.1.3 Zıplama Mekaniği

Yerçekimi simülasyonunun en güzel yanı, zıplama mekaniğinin bedavaya gelmesidir. Bir nesneye yukarı yönde bir hız verdiğinde, yerçekimi bu hızı her karede azaltır, nesne bir süre yükselir, durur ve sonra düşmeye başlar — tıpkı gerçek bir zıplama gibi.

Python – Zıplama Mekaniği

```

1  class ZiplayanKarakter(pygame.sprite.Sprite):
2      def __init__(self, x, y):
3          super().__init__()
4          self.image = pygame.Surface((40, 40))
5          self.image.fill((100, 200, 100))
6          self.rect = self.image.get_rect(topleft=(x, y))
7
8          self.hiz_y = 0
9          self.yercekimi = 0.5
10         self.ziplama_gucu = -12 # Negatif = yukari
11         self.yerde_mi = False
12
13     def zipla(self):
14         if self.yerde_mi:
15             self.hiz_y = self.ziplama_gucu
16             self.yerde_mi = False
17
18     def update(self):
19         # Yerçekimi uygula
20         self.hiz_y += self.yercekimi
21         self.rect.y += self.hiz_y
22
23         # Zemin kontrolu
24         if self.rect.bottom >= 580:
25             self.rect.bottom = 580
26             self.hiz_y = 0
27             self.yerde_mi = True

```

Burada `ziplama_gucu` değerinin **negatif** olduğuna dikkat et. PyGame’de y eksenini aşağı doğru arttığı için yukarı hareket negatif yöndedir. `-12` değeri karaktere güçlü bir yukarı yönlü hız verir. Yerçekimi bu hızı her karede 0.5 azaltır; 24 kare sonra hız sıfırlanır (tepe noktası) ve karakter düşmeye başlar.

⚠ Dikkat: Çift Zıplama Koruması

`zipla()` metodu `self.yerde_mi` kontrolü yapıyor. Bu olmadan oyuncu havadayken tekrar zıplayabilir ve sonsuza kadar yükselebilir. Platform oyunlarında bu kontrol çok önemlidir. Bazı oyunlarda “çift zıplama” özelliği kasıtlı olarak eklenir; o durumda bir `ziplama_sayaci` ile kaç kez zıplandığını takip edersin.

23.1.4 Terminal Hız

Gerçek dünyada düşen bir nesne sonsuza kadar hızlanmaz. Hava direnci bir noktada yerçekimi kuvvetini dengeler ve nesne sabit bir hızda düşmeye devam eder. Bu hız *terminal hız* denir.

Oyunlarda terminal hız uygulamak hem gerçekçilik sağlar hem de nesnelerin absürt hızlara ulaşmasını engeller:

Python – Terminal Hız Uygulaması

```
1  TERMINAL_HIZ = 15 # Maksimum dusme hizi
2
3  def update(self):
4      self.hiz_y += self.yercekimi
5
6      # Terminal hiz siniri
7      if self.hiz_y > TERMINAL_HIZ:
8          self.hiz_y = TERMINAL_HIZ
9
10     self.rect.y += self.hiz_y
```

Bu basit kontrol sayesinde nesnenin düşme hızı hiçbir zaman `TERMINAL_HIZ` değerini aşmaz. Özellikle yüksek platformlardan düşen karakterler için bu kontrol önemlidir; aksi takdirde karakter çok hızlı düşerek zemin çarpışma kontrolünü atlayabilir.

💡 İpucu: Profesyonel İpucu

Terminal hız değerini belirlerken zemin çarpışma kontrolünü göz önünde bulundur. Eğer karakterin düşme hızı zeminin kalınlığından büyükse, karakter bir karede zeminin altından geçebilir. Bu soruna “tünelleme” denir ve Bölüm 23’nun ilerideki konularında ele alacağız.

23.2 Hız ve Momentum

Bir araba oyununda gaza bastığında arabanın aniden tam hıza ulaşmasını istemezsin; yavaş yavaş hızlanmasını, frene bastığında da aniden durmasını değil, kademeli olarak yavaşlamasını beklersin. Ya da bir uzay oyununda geminin itici gücüyle yön değiştirmesini ama eski hızını tamamen kaybetmemesini istersin. Bu doğal hissiyatı sağlayan kavramlar *hız vektörleri*, *ivme* ve *momentum*’dur.

23.2.1 Hız Vektörleri

Bölüm 16’te nesneleri hareket ettirirken basit bir yaklaşım kullanmıştın: her karede x veya y koordinatına sabit bir değer eklemek. Şimdi bu yaklaşımı bir adım ileriye taşıyalım.

■ Tanım: Hız Vektörü

Hız vektörü, bir nesnenin hem **yönünü** hem de **hızını** temsil eden iki bileşenli bir büyüklüktür: (v_x, v_y) . Yatay bileşen v_x sağa-sola hareketi, dikey bileşen v_y yukarı-aşağı hareketi belirler.

Hız vektörünü bir ok gibi düşünebilirsin: okun yönü nesnenin hareket yönünü, okun uzunluğu ise nesnenin hızını gösterir. Her karede nesnenin konumuna bu vektörü eklersin ve nesne o yönde o kadar hareket eder.

Python – Hız Vektörü ile Hareket

```

1 class HareketliNesne(pygame.sprite.Sprite):
2     def __init__(self, x, y):
3         super().__init__()
4         self.image = pygame.Surface((20, 20))
5         self.image.fill((100, 150, 255))
6         self.rect = self.image.get_rect(center=(x, y))
7
8         # Hız vektörü
9         self.hiz_x = 3.0
10        self.hiz_y = -2.0
11
12    def update(self):
13        # Konum = Konum + Hız
14        self.rect.x += self.hiz_x
15        self.rect.y += self.hiz_y

```

Bu örnekte nesne her karede sağa 3, yukarı 2 piksel hareket ediyor. Sonuç olarak çapraz bir yol izliyor. Hız vektörünün bileşenlerini değiştirerek nesneyi istediğin yöne ve hızda hareket ettirebilirsin.

23.2.2 İvme: Hızın Değişimi

Gerçek hayatta nesneler aniden hız kazanmaz; *ivme* ile kademeli olarak hızlanır. Bir önceki konuda yerçekiminin dikey hıza her karede sabit bir değer eklediğini görmüştün. İvme de aynı mantıkla çalışır, sadece herhangi bir yönde uygulanabilir.

■ Tanım: İvme

İvme, hızın zamana göre değişim oranıdır. Her karede hız vektörüne eklenen sabit bir değer olarak uygulanır: $v = v + a$, burada a ivme değeridir.

İvmeyi oyunlara uyguladığında hareket çok daha doğal hisseder. Karakterin

aniden tam hıza ulaşması yerine kademeli olarak hızlanması, oyuncuya kontrol hissi verir:

Python – İvme ile Hızlanma

```

1  class IvmeliKarakter(pygame.sprite.Sprite):
2      def __init__(self, x, y):
3          super().__init__()
4          self.image = pygame.Surface((32, 32))
5          self.image.fill((200, 200, 50))
6          self.rect = self.image.get_rect(center=(x, y))
7
8          self.hiz_x = 0
9          self.hiz_y = 0
10         self.ivme = 0.3          # Hızlanma oranı
11         self.max_hiz = 6.0      # Hız sınırı
12         self.yercekimi = 0.5
13
14     def update(self):
15         tuslar = pygame.key.get_pressed()
16
17         # Yatay ivme uygula
18         if tuslar[pygame.K_RIGHT]:
19             self.hiz_x += self.ivme
20         elif tuslar[pygame.K_LEFT]:
21             self.hiz_x -= self.ivme
22
23         # Hız sınırı
24         if self.hiz_x > self.max_hiz:
25             self.hiz_x = self.max_hiz
26         elif self.hiz_x < -self.max_hiz:
27             self.hiz_x = -self.max_hiz
28
29         # Yercekimi
30         self.hiz_y += self.yercekimi
31
32         # Konum guncelle
33         self.rect.x += self.hiz_x
34         self.rect.y += self.hiz_y

```

Bu kodda birkaç önemli nokta var. Birincisi, tuşa basıldığında hıza doğrudan bir değer atanmıyor; bunun yerine `self.ivme` değeri ekleniyor. Bu sayede karakter kademeli olarak hızlanıyor. İkincisi, `self.max_hiz` ile bir üst sınır koyuyoruz; aksi takdirde karakter sonsuza kadar hızlanırdı.

Peki tuşu bıraktığında ne oluyor? Dikkat edersen tuş bırakıldığında hız sıfırlanmıyor, karakter aynı hızda kaymaya devam ediyor. Bu “kayma” etkisi bazı oyunlarda istenen bir özellik (buz platformları gibi), bazılarında ise istenmeyen bir davranış. Sürtünme konusunda bu sorunu çözeceğiz.

23.2.3 Momentum: Kütlenin Etkisi

Momentum, bir nesnenin hızı ile kütlesinin çarpımıdır: $p = m \times v$. Oyun fiziğinde momentum, farklı ağırlıktaki nesnelerin birbirleriyle etkileşimini belirler.

Bunu şöyle düşün: bir bilardo topuna vuran başka bir bilardo topu hızını aktarır. Ama aynı topla bir bowling topuna vurursan, bowling topu pek hareket etmez çünkü kütlesi çok daha büyüktür.

Python – Kütle ve Momentum

```

1 class FizikNesnesi(pygame.sprite.Sprite):
2     def __init__(self, x, y, kutle=1.0):
3         super().__init__()
4         # Kutleye göre boyut belirle
5         boyut = int(20 + kutle * 10)
6         self.image = pygame.Surface((boyut, boyut))
7         self.image.fill((150, 100, 200))
8         self.rect = self.image.get_rect(center=(x, y))
9
10        self.hiz_x = 0
11        self.hiz_y = 0
12        self.kutle = kutle
13
14        def kuvvet_uygula(self, kuvvet_x, kuvvet_y):
15            """Newton'un 2. yasasi:  $F = m * a$ , yani  $a = F / m$ """
16            self.hiz_x += kuvvet_x / self.kutle
17            self.hiz_y += kuvvet_y / self.kutle
18
19        def update(self):
20            self.rect.x += self.hiz_x
21            self.rect.y += self.hiz_y

```

Burada `kuvvet_uygula()` metodu Newton'un ikinci yasasını kullanıyor: $a = F/m$. Aynı kuvveti iki farklı kütle değerine sahip nesneye uyguladığında, hafif olan daha çok hızlanır, ağır olan daha az hızlanır. Bu basit formül sayesinde farklı ağırlıktaki nesneler farklı davranır.

Bilgi: Kütle Birimi

Oyun fiziğinde kütle birimi "kilogram" olmak zorunda değildir. Kendi birim sisteminizi oluşturabilirsiniz. Önemli olan nesneler arasındaki **orandır**: bir nesnenin kütlesi diğerinin iki katıysa, aynı kuvvet altında yarı kadar hızlanır.

23.2.4 Vektör Aritmetiği

Oyun fiziğinde hız, ivme ve kuvvet gibi büyüklüklerle sürekli çalışırsın. Bu büyüklüklerin hepsi vektördür ve vektörlerle bazı temel işlemler yapman gerekir.

Vektör toplama, nesnenin konumunu güncellerken kullanırsın: konum vektörüne hız vektörünü eklersin. Vektör çarpma (skaler ile), bir kuvveti artırıp

azaltmak için kullanırsın. Vektörün büyüklüğü (uzunluğu) ise nesnenin toplam hızını hesaplamak için gerekir:

$$|\vec{v}| = \sqrt{v_x^2 + v_y^2}$$

Bu formül, Pisagor teoreminin doğrudan uygulamasıdır. Yatay ve dikey hız bileşenlerinden toplam hızı hesaplıyorsın:

Python – Vektör Büyüklüğü Hesaplama

```
1 import math
2
3 def vektor_buyuklugu(vx, vy):
4     """Vektörün büyüklüğünü (uzunluğunu) hesaplar."""
5     return math.sqrt(vx ** 2 + vy ** 2)
6
7 # Örnek: yatay 3, dikey 4 hız
8 toplam_hiz = vektor_buyuklugu(3, 4)
9 # Sonuç: 5.0 (3-4-5 üçgeni)
```

Vektör büyüklüğünü özellikle çapraz hareket sınırlamasında kullanırsın. Bir karakter aynı anda sağa ve yukarı hareket ettiğinde, her ekseninde ayrı ayrı `max_hiz` kontrolü yaparsan çapraz hareket $\sqrt{2}$ kat daha hızlı olur. Bunun yerine toplam hız vektörünün büyüklüğünü kontrol etmek daha doğrudur:

Python – Çapraz Hareket Düzeltmesi

```
1 def hiz_sinirla(hiz_x, hiz_y, max_hiz):
2     """Toplam hizi sınırlar, yön oranını korur."""
3     buyukluk = math.sqrt(hiz_x ** 2 + hiz_y ** 2)
4     if buyukluk > max_hiz:
5         oran = max_hiz / buyukluk
6         hiz_x *= oran
7         hiz_y *= oran
8     return hiz_x, hiz_y
```

Bu fonksiyon, hız vektörünün büyüklüğü sınırı aştığında her iki bileşeni de orantılı olarak küçültür. Böylece yön korunur ama toplam hız sınırı aşmaz.

⚠ Dikkat: Çapraz Hareket Tuzağı

Yatay ve dikey hız sınırlarını ayrı ayrı kontrol etmek yaygın bir hatadır. Bu durumda karakter çapraz hareket ettiğinde yaklaşık %41 daha hızlı gider ($\sqrt{2} \approx 1.41$). Yukarıdaki `hiz_sinirla()` fonksiyonu bu sorunu çözer.

23.3 Sürtünme

Bir önceki konuda ivme ile hızlanan karakterin tuş bırakıldığında kaymaya devam ettiğini görmüştün. Gerçek dünyada bu olmaz: bisikleti çevirmeyi bıraktığında

yavaş yavaş durur, buz pistinde ise çok daha uzun süre kayarsın. Bu farkın sebebi *sürtünme* kuvvetidir.

23.3.1 Sürtünme Katsayısı

■ Tanım: Sürtünme

Oyun fiziğinde *sürtünme*, her karede hız vektörünü belirli bir oranda azaltan kuvvettir. Sürtünme katsayısı 0 ile 1 arasında bir değerdir: 0 sürtünme yok (sonsuz kayma), 1 ise anında durma anlamına gelir.

Sürtünmeyi uygulamanın en basit yolu, her karede hızı bir katsayı ile çarpmaktır. Bu katsayı 1'den küçük olduğu için hız her karede azalır:

Python – Basit Sürtünme

```

1  SURTUNME = 0.85 # Her karede hızın %85'i korunur
2
3  def update(self):
4      tuslar = pygame.key.get_pressed()
5
6      if tuslar[pygame.K_RIGHT]:
7          self.hiz_x += self.ivme
8      elif tuslar[pygame.K_LEFT]:
9          self.hiz_x -= self.ivme
10
11     # Surtunme uygula (tus basılmadığı zaman da çalışır)
12     self.hiz_x *= SURTUNME
13
14     # Çok küçük hızları sıfırla (titremeyi önle)
15     if abs(self.hiz_x) < 0.1:
16         self.hiz_x = 0
17
18     self.rect.x += self.hiz_x

```

Bu kodda her karede hız 0.85 ile çarpılıyor. Yani her karede hızın %15'i kayboluyor. Tuşa basıldığında ivme sürtünmeyi yeniyor ve karakter hızlanıyor; tuş bırakıldığında ise sürtünme hızı kademeli olarak sıfıra indiriyor.

Son satırdaki `abs(self.hiz_x) < 0.1` kontrolü önemli bir detaydır. Sürtünme hızı asla tam sıfıra indiremez (her karede bir öncekinin %85'i kalır). Bu yüzden çok küçük hız değerlerini manuel olarak sıfırlıyoruz; aksi takdirde karakter süresiz olarak mikro hareketler yapar.

23.3.2 Yüzey Sürtünmesi

Farklı yüzeyler farklı sürtünme katsayılarına sahiptir. Bir platform oyununda normal zemin, buz ve kum gibi yüzeyler karakterin hareket hissiyatını tamamen değiştirebilir:

Python – Yüzey Tabanlı Sürtünme

```

1  # Yüzey tipleri ve surtunme katsayilari
2  YUZEY_SURTUNME = {
3      "normal": 0.80,    # Standart zemin
4      "buz":     0.98,    # Neredeyse surtunme yok, kaygan
5      "kum":     0.60,    # Yuksek surtunme, yavas hareket
6      "cimen":   0.75,    # Orta surtunme
7  }
8
9  class Karakter(pygame.sprite.Sprite):
10     def __init__(self, x, y):
11         super().__init__()
12         self.image = pygame.Surface((32, 48))
13         self.image.fill((100, 200, 100))
14         self.rect = self.image.get_rect(midbottom=(x, y))
15
16         self.hiz_x = 0
17         self.ivme = 0.5
18         self.aktif_yuzey = "normal"
19
20     def update(self):
21         tuslar = pygame.key.get_pressed()
22
23         if tuslar[pygame.K_RIGHT]:
24             self.hiz_x += self.ivme
25         elif tuslar[pygame.K_LEFT]:
26             self.hiz_x -= self.ivme
27
28         # Aktif yuzeyin surtunmesini uygula
29         surtunme = YUZEY_SURTUNME[self.aktif_yuzey]
30         self.hiz_x *= surtunme
31
32         if abs(self.hiz_x) < 0.1:
33             self.hiz_x = 0
34
35         self.rect.x += self.hiz_x

```

Dikkat edersen buz yüzeyinde katsayı 0.98, yani her karede hızın sadece %2'si kayboluyor. Bu da karakterin çok uzun süre kayması anlamına geliyor. Kum yüzeyinde ise katsayı 0.60; her karede hızın %40'ı kayboluyor ve karakter hızla duruyor.

💡 İpucu: Profesyonel İpucu

Sürtünme katsayılarını oyun tasarımının bir parçası olarak düşün. Buz bölümleri zor ama eğlenceli mekanikler sunar, kum bölümleri ise yavaş ve stratejik bir tempo oluşturur. Farklı yüzeyler oyununa çeşitlilik katar.

23.3.3 Hava Direnci

Yüzey sürtünmesi sadece nesne bir zemine temas ettiğinde çalışır. Peki havadaki nesneler için ne yapacağız? Havadaki nesnelere *hava direnci* (drag) uygulanır.

Hava direnci, sürtünmeye benzer şekilde çalışır ama genellikle daha düşük bir katsayıya sahiptir. Havadaki bir nesne yerdekine göre çok daha yavaş yavaşlar:

Python – Hava Direnci

```

1  ZEMIN_SURTUNME = 0.80
2  HAVA_DIRENCI = 0.99 # Havada çok az yavaşlar
3
4  class PlatformKarakter(pygame.sprite.Sprite):
5      def __init__(self, x, y):
6          super().__init__()
7          self.image = pygame.Surface((32, 48))
8          self.image.fill((255, 180, 50))
9          self.rect = self.image.get_rect(midbottom=(x, y))
10
11         self.hiz_x = 0
12         self.hiz_y = 0
13         self.yercekimi = 0.5
14         self.yerde_mi = False
15
16     def update(self):
17         tuslar = pygame.key.get_pressed()
18
19         if tuslar[pygame.K_RIGHT]:
20             self.hiz_x += 0.5
21         elif tuslar[pygame.K_LEFT]:
22             self.hiz_x -= 0.5
23
24         # Duruma göre surtunme veya hava direnci
25         if self.yerde_mi:
26             self.hiz_x *= ZEMIN_SURTUNME
27         else:
28             self.hiz_x *= HAVA_DIRENCI
29
30         if abs(self.hiz_x) < 0.1:
31             self.hiz_x = 0
32
33         # Yercekimi
34         self.hiz_y += self.yercekimi
35
36         # Konum guncelle
37         self.rect.x += self.hiz_x
38         self.rect.y += self.hiz_y
39
40         # Zemin kontrolu
41         if self.rect.bottom >= 580:
42             self.rect.bottom = 580
43             self.hiz_y = 0

```

```

44         self.yerde_mi = True
45     else:
46         self.yerde_mi = False

```

Bu örnekte `self.yerde_mi` durumuna göre farklı bir katsayı uygulanıyor. Yerdeyken 0.80 ile hızlı yavaşlama, havadayken 0.99 ile çok yavaş bir yavaşlama. Bu fark oyuncuya “havada kontrol azalıyor” hissini verir ki bu, platform oyunlarının temel mekaniklerinden biridir.

● Bilgi: Havada Kontrol

Gerçek fizikte havadaki bir karakter yön değiştiremez (itme kuvveti yok). Ancak çoğu oyunda “havada kontrol” kasıtlı olarak bırakılır çünkü tam gerçekçilik eğlenceli değildir. Hava direnci katsayısını ayarlayarak havada kontrol seviyesini belirleyebilirsiniz: düşük katsayı = az kontrol, yüksek katsayı = çok kontrol.

23.3.4 Sürtünme Modelleri Karşılaştırması

Farklı sürtünme modellerinin davranışlarını anlamak için basit bir karşılaştırma yapalım. Başlangıç hızı 10 olan bir nesnenin 5 kare sonra hızına bakalım:

Tablo 23.1: Sürtünme Katsayılarının Etkisi (Başlangıç hızı: 10)

Yüzey	Katsayı	Kare 1	Kare 2	Kare 3	Kare 4	Kare 5
Kum	0.60	6.0	3.6	2.2	1.3	0.8
Normal	0.80	8.0	6.4	5.1	4.1	3.3
Buz	0.98	9.8	9.6	9.4	9.2	9.0
Hava	0.99	9.9	9.8	9.7	9.6	9.5

Tablo 23.1’de görüldüğü gibi kum yüzeyinde nesne 5 karede neredeyse duruyor, buz yüzeyinde ise hızını büyük ölçüde koruyor. Bu tablo, sürtünme katsayısını seçerken sana rehber olacaktır.

▲ Dikkat: FPS Bağımlılığı

Sürtünme modelimiz kare sayısına (FPS) bağımlıdır. 60 FPS’de çalışan bir oyun ile 30 FPS’de çalışan bir oyun farklı davranacaktır. İleri seviye fizik motorlarında `delta_time` kullanılarak bu sorun çözülür. Şimdilik `pygame.time.Clock` ile sabit FPS tutarak bu sorunu basitçe atlatabilirsin.

23.4 Basit Bir Fizik Motoru Tasarımı

Bu üniteye yerçekimi, ivme, momentum ve sürtünme kavramlarını ayrı ayrı öğrendin. Şimdi tüm bu parçaları bir araya getirip tekrar kullanılabilir bir `PhysicsBody`

sınıfı tasarlayacağız. Bu sınıf sayesinde herhangi bir Sprite'a fizik davranışı eklemek birkaç satırlık bir iş olacak.

23.4.1 PhysicsBody Sınıfı

PhysicsBody sınıfı, `pygame.sprite.Sprite` sınıfından türeyecek ve fizik ile ilgili tüm değişkenleri ve hesaplamaları kapsülleyecek. Böylece oyun nesnelerin bu sınıftan türetmek yeterli olacak:

Python – PhysicsBody Sınıfı

```

1  import pygame
2  import math
3
4  class PhysicsBody(pygame.sprite.Sprite):
5      """Fizik davranisi gosteren temel sprite sinifi."""
6
7      def __init__(self, x, y, genislik, yukseklik,
8          ↪ kutle=1.0):
9          super().__init__()
10         self.image = pygame.Surface((genislik, yukseklik))
11         self.rect = self.image.get_rect(topleft=(x, y))
12
13         # Hareket
14         self.hiz_x = 0.0
15         self.hiz_y = 0.0
16
17         # Fizik parametreleri
18         self.kutle = kutle
19         self.yercekimi = 0.5
20         self.surtunme = 0.80
21         self.hava_direnci = 0.99
22         self.max_hiz = 8.0
23         self.terminal_hiz = 15.0
24
25         # Durum
26         self.yerde_mi = False

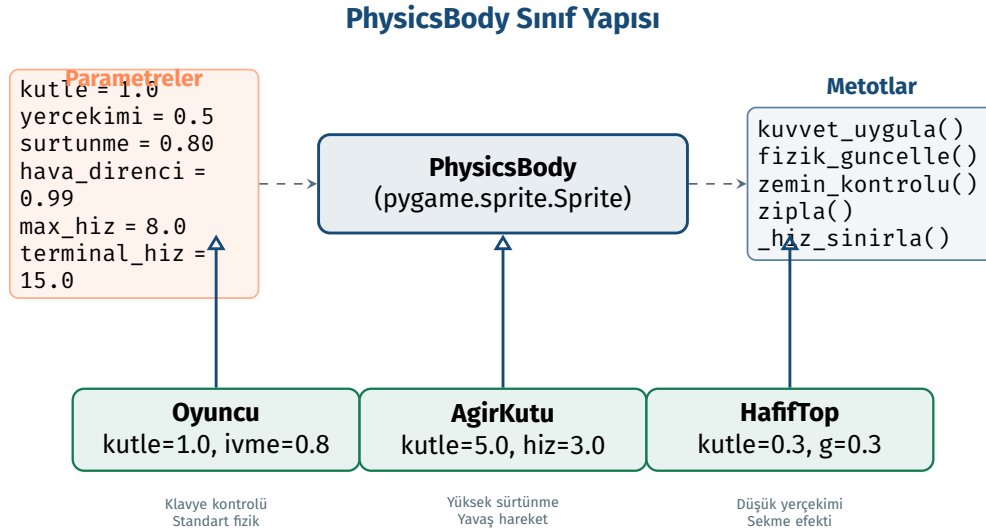
```

Şekil 23.2’de **PhysicsBody** sınıfının genel yapısını ve alt sınıflarla ilişkisini görebilirsin.

Bu yapıcı metot tüm fizik parametrelerini varsayılan değerlerle başlatıyor. Her parametre dışarıdan değiştirilebilir olduğu için farklı nesneler farklı fizik davranışları gösterebilir: ağır bir kutu düşük `max_hiz` ve yüksek `kutle` değerine, hafif bir tüy ise düşük `yercekimi` ve düşük `terminal_hiz` değerine sahip olabilir.

23.4.2 Kuvvet Uygulama ve Güncelleme

Fizik motorunun kalbi, kuvvet uygulama ve güncelleme metotlarıdır:



Şekil 23.2: PhysicsBody sınıf yapısı: parametreler, metotlar ve türetilmiş alt sınıflar.

Python – Kuvvet ve Güncelleme Metotları

```

1 def kuvvet_uygula(self, kuvvet_x, kuvvet_y):
2     """Nesneye kuvvet uygular (F = m * a -> a = F / m)."""
3     self.hiz_x += kuvvet_x / self.kutle
4     self.hiz_y += kuvvet_y / self.kutle
5
6 def fizik_guncelle(self):
7     """Her karede cagirilacak fizik hesaplamalari."""
8     # 1. Yercekimi
9     self.hiz_y += self.yercekimi
10
11     # 2. Surtunme / hava direnci
12     if self.yerde_mi:
13         self.hiz_x *= self.surtunme
14     else:
15         self.hiz_x *= self.hava_direnci
16
17     # 3. Cok kucuk hizlari sifirla
18     if abs(self.hiz_x) < 0.1:
19         self.hiz_x = 0
20
21     # 4. Hiz sinirlamalari
22     self._hiz_sinirla()
23
24     # 5. Konum guncelle
25     self.rect.x += int(self.hiz_x)
26     self.rect.y += int(self.hiz_y)
27
28 def _hiz_sinirla(self):
29     """Hiz vektorunun buyuklugunu sinirlar."""
30     # Yatay hiz siniri

```

```

31     buyukluk = math.sqrt(self.hiz_x ** 2 + self.hiz_y **
    ↪ 2)
32     if self.hiz_x > self.max_hiz:
33         self.hiz_x = self.max_hiz
34     elif self.hiz_x < -self.max_hiz:
35         self.hiz_x = -self.max_hiz
36
37     # Dikey hiz siniri (terminal hiz)
38     if self.hiz_y > self.terminal_hiz:
39         self.hiz_y = self.terminal_hiz

```

`fizik\guncelle()` metodu, fizik simülasyonunun tüm adımlarını doğru sırada uygular. Bu sıra önemlidir: önce yerçekimi hızı etkiler, sonra sürtünme hızı azaltır, ardından sınırlamalar kontrol edilir ve en son konum güncellenir.

`int()` dönüşümüne dikkat et: `Rect` sınıfı tam sayı koordinatlar kullanır, bu yüzden ondalıklı hız değerlerini tam sayıya çeviriyoruz. İleri seviye uygulamalarda ondalıklı koordinatlar için ayrı `x` ve `y` değişkenleri tutulur, ama şimdilik bu basit yaklaşım yeterlidir.

23.4.3 Zemin ve Çarpışma Kontrolü

`PhysicsBody` sınıfına zemin çarpışma kontrolü de ekleyelim:

Python – Zemin Kontrolü

```

1  def zemin_kontrolu(self, zemin_y):
2      """Nesnenin zemin altına düşmesini engeller."""
3      if self.rect.bottom >= zemin_y:
4          self.rect.bottom = zemin_y
5          self.hiz_y = 0
6          self.yerde_mi = True
7      else:
8          self.yerde_mi = False
9
10     def zipla(self, guc=-12):
11         """Nesneyi ziplatır (sadece yerdeyken)."""
12         if self.yerde_mi:
13             self.hiz_y = guc
14             self.yerde_mi = False

```

Bu metotlar sırasıyla zemin kontrolü ve zıplama işlevlerini yerine getiriyor. `zemin\kontrolu()` her karede çağrılarak nesnenin zemin seviyesinin altına düşmesi engellenir. `zipla()` ise yalnızca nesne yerdeyken çalışır ve yukarı doğru bir hız verir.

23.4.4 PhysicsBody Kullanımı

Şimdi bu sınıfı kullanarak farklı fizik davranışlarına sahip nesneler oluşturalım:

Python – PhysicsBody Kullanım Örneği

```

1  class Oyuncu(PhysicsBody):
2      """Oyuncu karakteri."""
3      def __init__(self, x, y):
4          super().__init__(x, y, 32, 48, kutle=1.0)
5          self.image.fill((100, 200, 100))
6          self.ivme_gucu = 0.8
7
8      def update(self):
9          tuslar = pygame.key.get_pressed()
10
11         if tuslar[pygame.K_RIGHT]:
12             self.kuvvet_uygula(self.ivme_gucu, 0)
13         if tuslar[pygame.K_LEFT]:
14             self.kuvvet_uygula(-self.ivme_gucu, 0)
15         if tuslar[pygame.K_SPACE]:
16             self.zipla()
17
18         self.fizik_guncelle()
19         self.zemin_kontrolu(580)
20
21 class AgirKutu(PhysicsBody):
22     """Agir, yavas hareket eden kutu."""
23     def __init__(self, x, y):
24         super().__init__(x, y, 50, 50, kutle=5.0)
25         self.image.fill((150, 100, 50))
26         self.max_hiz = 3.0          # Daha yavas
27         self.surtunme = 0.70       # Daha cok surtunme
28
29     def update(self):
30         self.fizik_guncelle()
31         self.zemin_kontrolu(580)
32
33 class HafifTop(PhysicsBody):
34     """Hafif, zipziplayan top."""
35     def __init__(self, x, y):
36         super().__init__(x, y, 20, 20, kutle=0.3)
37         self.image.fill((255, 200, 50))
38         self.yercekimi = 0.3       # Daha yavas duser
39         self.surtunme = 0.95       # Az surtunme, cok kayar
40
41     def update(self):
42         self.fizik_guncelle()
43         self.zemin_kontrolu(580)

```

Bu örnekte aynı **PhysicsBody** sınıfından türeyen üç farklı nesne var. **Oyuncu** standart fizik parametreleriyle çalışıyor ve klavye girdileriyle kontrol ediliyor. **AgirKutu** yüksek kütlesi sayesinde kuvvetlere karşı daha az tepki veriyor ve güçlü sürtünme ile hızla duruyor. **HafifTop** ise düşük yerçekimi ve düşük sürtünme ile uçarı bir davranış sergiliyor.

● Bilgi: Kalıtımın Gücü

Burada Bölüm 7’te öğrendiğin *kalıtım* kavramının gücünü görüyorsun. Fizik hesaplamaları tek bir yerde yazılıyor (**PhysicsBody**), her yeni nesne sadece parametreleri değiştirerek farklı davranışlar elde ediyor. Bir hata bulduğunda veya bir iyileştirme yaptığında sadece **PhysicsBody** sınıfını güncellersen tüm nesneler otomatik olarak güncellenir.

23.4.5 Tam Çalışan Örnek

Son olarak tüm parçaları bir araya getiren minimal bir örnek görelim:

Python – Fizik Motoru Demo

```

1  import pygame
2  import math
3
4  # --- PhysicsBody sinifi (yukarıda tanımlandığı gibi) ---
5
6  def main():
7      pygame.init()
8      ekran = pygame.display.set_mode((800, 600))
9      pygame.display.set_caption("Fizik Motoru Demo")
10     saat = pygame.time.Clock()
11
12     # Nesneleri oluşturun
13     oyuncu = Oyuncu(100, 400)
14     kutu = AgirKutu(400, 300)
15     top = HafifTop(600, 200)
16
17     # Kuvvet uygulayarak topu fırlat
18     top.kuvvet_uygula(5, -8)
19
20     tum_nesneler = pygame.sprite.Group(oyuncu, kutu, top)
21
22     calistir = True
23     while calistir:
24         for olay in pygame.event.get():
25             if olay.type == pygame.QUIT:
26                 calistir = False
27
28         tum_nesneler.update()
29
30         ekran.fill((30, 30, 50))
31
32         # Zemin çiz
33         pygame.draw.line(ekran, (100, 100, 100),
34                          (0, 580), (800, 580), 2)
35
36         tum_nesneler.draw(ekran)

```

```

37         pygame.display.flip()
38         saat.tick(60)
39
40     pygame.quit()
41
42     if __name__ == "__main__":
43         main()

```

Bu programı çalıştırdığında şunları gözlemleyeceksin: oyuncu ok tuşlarıyla hareket eder ve boşluk tuşuyla zıplar. Ağır kutu yerçekimiyle düşer ve hızla durur. Hafif top fırlatma kuvvetiyle yay çizerek hareket eder ve yere düştükten sonra uzun süre kayar. Üç nesne de aynı fizik motorunu kullanıyor ama parametreleri farklı olduğu için tamamen farklı davranıyorlar.

İpucu: Profesyonel İpucu

Bu fizik motorunu ileriki bölümlerde geliştirmeye devam edeceksin. Bölüm 23'nun bir sonraki ünitesinde çarpışma tepkileri ekleyecek, platform çarpışmaları ile entegre edeceksin. Şimdilik temel yapıyı iyi anla; geri kalanı bu temelin üzerine inşa edilecek.

23.5 Sonuç

Bu ünite de oyun fiziğinin temel yapı taşlarını öğrendin. Yerçekimi simülasyonundan başlayarak hız vektörleri, ivme, momentum, sürtünme ve hava direnci kavramlarını adım adım keşfettin. Son olarak tüm bu kavramları bir araya getiren **PhysicsBody** sınıfını tasarladın.

23.5.1 Anahtar Kavramlar

Tablo 23.2: Ünite 3 – Temel Fizik Simülasyonu: Anahtar Kavramlar

Kavram	Açıklama
Yerçekimi	Her karede dikey hıza sabit değer ekleyerek düşme simülasyonu: $v_y = v_y + g$
Hız Vektörü	Nesnenin yönünü ve hızını temsil eden iki bileşenli büyüklük: (v_x, v_y)
İvme	Hızın zamana göre değişim oranı; kademeli hızlanma sağlar
Momentum	Kütle ve hızın çarpımı: $p = m \times v$; farklı ağırlıkta nesnelerin etkileşimini belirler
Sürtünme	Hızı her karede azaltan kuvvet; katsayısı 0 ile 1 arasında
Terminal Hız	Düşme hızının ulaşabileceği maksimum değer
Hava Direnci	Havadaki nesnelere uygulanan düşük sürtünme
PhysicsBody	Tüm fizik kavramlarını kapsülleyen temel sınıf

23.5.2 Formül Özeti

Tablo 23.3: Ünite 3 – Temel Fizik Formülleri

İşlem	Formül	Açıklama
Yerçekimi	$v_y = v_y + g$	Dikey hıza sabit değer ekleme
Konum güncelleme	$pos = pos + v$	Konuma hızı ekleme
Kuvvet (ivme)	$a = F/m$	Newton'un ikinci yasası
Sürtünme	$v = v \times k$	Hızı katsayıyla çarpma ($0 < k < 1$)
Vektör büyüklüğü	$ v = \sqrt{v_x^2 + v_y^2}$	Toplam hız hesaplama

23.5.3 Sıradaki Adım

Artık nesnelerin hareket etmesini, düşmesini, zıplamasını ve yavaşlamasını sağlayabiliyorsun. Peki iki nesne birbirine çarptığında ne olacak? Bir sonraki ünite **Konu 23.4**'nda oluşturduğun **PhysicsBody** sınıfını çarpışma tepkileriyle genişleteceksin: geri sekme, hasar alma ve nesneleri yok etme gibi mekanikleri öğreneceksin. Uzay Kaçışı projesine düşmanlar ve mermiler ekleyerek gerçek bir oyun deneyimi oluşturmaya başlayacaksın.

💡 İpucu: Pratik Yap

Bu ünite **deki kavramları pekiştirmek için şu denemeleri yap: **PhysicsBody** sınıfında yerçekimi değerini değiştirerek ay yürüyüşü simülasyonu oluştur. Farklı sürtünme katsayılarıyla buz pisti ve kum plajı efektleri dene. Zıplama gücünü ayarlayarak hem kısa hem uzun zıplama mekanikleri geliştir.**

Çarpışma Tepkileri

“Bir top duvara çarptığında geri sekmesi, bir düşman vurulduğunda yok olması – oyun fiziği, çarpışmayı algılamakla değil, ona tepki vermekle tamamlanır.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

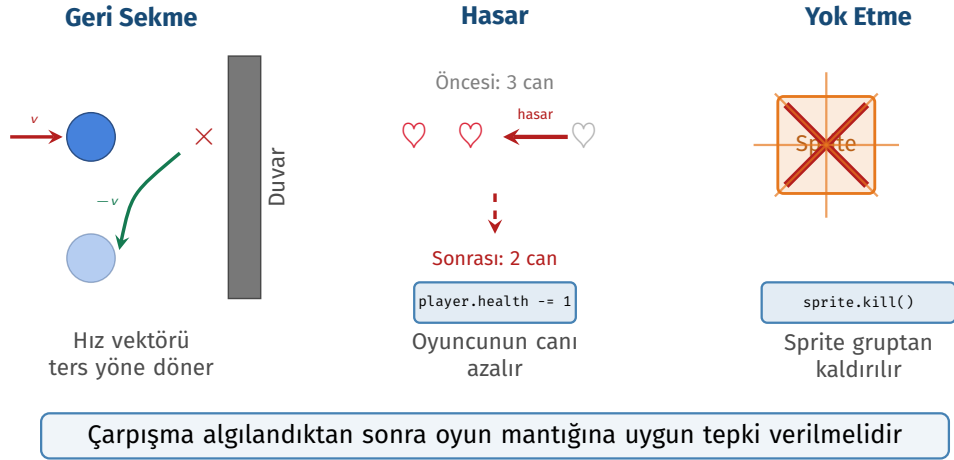
- Çarpışma sonrası **geri sekme** (yansıma) mekaniği ve hız vektörü değiştirme
- **Hasar ve can sistemi** (health/damage) ile invincibility frame kavramı
- Sprite **yok etme**, patlama efekti ve skor sistemi
- **Breakout (Tuğla Kırma)** yan projesi: tüm bölüm kavramlarını birleştiren mini oyun

24.1 Geri Sekme ve Yansıma Mekaniği

Bir bilardo topunu düşün: masanın kenarına çarptığında durmaz, gelen açıyla orantılı bir açıda geri sekip yoluna devam eder. Aynı mantık oyun geliştirmede de geçerli. Bölüm 21’de iki nesnenin çarpışıp çarpışmadığını algılamayı öğrendin. Peki çarpışma algılandıktan sonra ne olacak? Nesne duracak mı, geri mi sekecek, yoksa yok mu olacak? Bu ünite de tam olarak bu soruları cevaplayacağız.

Çarpışma *tepkisi* (collision response), çarpışma algılandıktan sonra oyun nesnelerinin nasıl davranacağını belirleyen mantıktır. En yaygın tepkilerden biri *geri sekme*dir: nesne çarptığı yüzeyden yansıyarak yoluna devam eder.

Çarpışma Tepki Türleri



Şekil 24.1: Üç temel çarpışma tepki türü: geri sekme, hasar ve yok etme

■ Tanım: Geri Sekme (Bounce)

Bir nesnenin bir yüzeye çarpması sonrasında hız vektörünün uygun ekseninde tersine çevrilmesiyle oluşan yansıma hareketidir. Yatay bir yüzeye çarpışta dikey hız, dikey bir yüzeye çarpışta yatay hız ters çevrilir.

24.1.1 Hız Vektörünü Tersine Çevirme

Bölüm 23’de hız vektörlerini (`hiz_x` ve `hiz_y`) kullanarak nesneleri hareket ettirmeyi öğrendin. Geri sekme mekaniğinin temeli çok basit: çarpışma yüzeyine dik olan eksenindeki hız bileşenini -1 ile çarp. Diğer bileşene dokunma.

Diyelim ki bir top sağa ve aşağıya doğru hareket ediyor: `hiz_x = 4`, `hiz_y = 3`. Bu top ekranın sağ duvarına çarptığında yatay yönde geri sekmeli, ama dikey hareketi aynı kalmalı. Yani `hiz_x = -4` olur, `hiz_y = 3` olarak kalır. Benzer şekilde alt duvara çarparsa `hiz_y = -3` olur, `hiz_x = 4` kalır.

Python – Basit Geri Sekme

```

1  # Topun hareketi
2  top_rect.x += hiz_x
3  top_rect.y += hiz_y
4
5  # Yatay duvar kontrolü (sol ve sag kenar)
6  if top_rect.left <= 0 or top_rect.right >= GENISLIK:
7      hiz_x = -hiz_x
8
9  # Dikey duvar kontrolü (ust ve alt kenar)
10 if top_rect.top <= 0 or top_rect.bottom >= YUKSEKLIK:
11     hiz_y = -hiz_y

```

Bu dört satırlık kod, birçok klasik oyunun temelini oluşturur: Pong’dan Breakout’a, Air Hockey’den Pinball’a kadar. Dikkat edersen burada Bölüm 21’deki

Rect özelliklerini (**left**, **right**, **top**, **bottom**) kullanıyoruz. Bu özellikler sayesinde topun hangi kenara çarptığını kolayca belirleyebiliyoruz.

24.1.2 Elastik ve İnelastik Çarpışma

Gerçek dünyada çarpışmalar iki kategoriye ayrılır. *Elastik çarpışma*da enerji korunur ve top aynı hızla geri seker. *İnelastik çarpışma*da ise enerji kaybı olur ve top her sekmede biraz yavaşlar.

Python – Enerji Kayıplı Geri Sekme

```
1  # Enerji kaybı katsayısı (0.0 - 1.0 arası)
2  # 1.0 = tam elastik, 0.8 = her sekmede %20 enerji kaybı
3  SEKME_KATSAYISI = 0.8
4
5  if top_rect.left <= 0 or top_rect.right >= GENISLIK:
6      hiz_x = -hiz_x * SEKME_KATSAYISI
7
8  if top_rect.top <= 0 or top_rect.bottom >= YUKSEKLIK:
9      hiz_y = -hiz_y * SEKME_KATSAYISI
```

SEKME_KATSAYISI 1.0 olduğunda tam elastik çarpışma elde edersin: top hiç enerji kaybetmeden sonsuza kadar sekmeye devam eder. Değeri 0.8 yaptığında her sekmede hızın %20'si kaybolur ve top giderek yavaşlar. Bu davranış özellikle zıplayan bir top veya düşen bir nesne simülasyonunda çok gerçekçi sonuçlar verir.

⚠ Dikkat: Sıkışma Problemi

Geri sekmede sık karşılaşılan bir sorun, nesnenin duvar içinde sıkışmasıdır. Top çok hızlıysa bir karede duvarın ötesine geçebilir ve sonraki karede tekrar ters çevrilip duvarın içinde salınmaya başlar. Bunu önlemek için çarpışma sonrasında topun konumunu düzeltmelisin: **top_rect.left = 0** (sol duvara yapıştır) veya **top_rect.right = GENISLIK** (sağ duvara yapıştır).

Python – Konum Düzeltmeli Geri Sekme

```
1  # Sol ve sag duvar
2  if top_rect.left <= 0:
3      top_rect.left = 0
4      hiz_x = abs(hiz_x) # Saga yonlendir
5  elif top_rect.right >= GENISLIK:
6      top_rect.right = GENISLIK
7      hiz_x = -abs(hiz_x) # Sola yonlendir
8
9  # Ust ve alt duvar
10 if top_rect.top <= 0:
11     top_rect.top = 0
12     hiz_y = abs(hiz_y) # Asagi yonlendir
13 elif top_rect.bottom >= YUKSEKLIK:
```

```

14 top_rect.bottom = YUKSEKLİK
15 hiz_y = -abs(hiz_y) # Yukari yonlendir

```

Burada `abs()` fonksiyonu işimize çok yarıyor. Mesela sol duvara çarptığında `hiz_x` zaten negatiftir (sola gidiyor). `abs(hiz_x)` ile mutlak değerini alıp pozitif yapıyoruz, yani top sağa yöneliyor. Bu yöntem, hız yönünden bağımsız çalıştığı için sıkışma problemini tamamen ortadan kaldırır.

24.1.3 İki Nesne Arası Geri Sekme

Duvarlar basit çünkü yatay ya da dikey yüzeyler. Peki iki hareketli nesne birbirine çarptığında ne olacak? Bu durum biraz daha karmaşık ama temel mantık aynı: çarpışma yönünü belirle ve hızları ters çevir.

Bölüm 22’de `pygame.sprite.spritecollide()` fonksiyonunu öğrenmiştin. Şimdi bu fonksiyonla çarpışma tespit ettikten sonra iki nesnenin birbirinden sekmesini sağlayalım.

Python – İki Nesne Arası Sekme

```

1 # İki top birbirine carptiyorsa
2 if pygame.sprite.collide_rect(top1, top2):
3     # Hizlari takas et (basit yaklasim)
4     top1.hiz_x, top2.hiz_x = top2.hiz_x, top1.hiz_x
5     top1.hiz_y, top2.hiz_y = top2.hiz_y, top1.hiz_y
6
7     # İç ice gecmeyi engelle: birbirinden uzaklastir
8     while pygame.sprite.collide_rect(top1, top2):
9         top1.rect.x += top1.hiz_x
10        top1.rect.y += top1.hiz_y
11        top2.rect.x += top2.hiz_x
12        top2.rect.y += top2.hiz_y

```

Bu basitleştirilmiş yaklaşımda iki topun hız vektörlerini takas ediyoruz. Gerçek fizikte momentum korunumu ve kütle farkları hesaplanır, ama birçok 2D oyun için bu basit takas yöntemi yeterince iyi sonuç verir.

İpucu: Profesyonel İpucu

Gerçekçi fizik istiyorsan `pymunk` gibi bir fizik motoru kullanabilirsin. Ancak bu kitapta amacımız fizik motorunun yaptığı işi anlamak olduğu için her şeyi sıfırdan yazıyoruz. İleride Bölüm 13’te gelişmiş fizik konusuna değineceğiz.

24.1.4 Breakout Bağlamında Geri Sekme

Bu ünitenin sonunda geliştireceğimiz Breakout (Tuğla Kırma) oyununda geri sekme mekaniği kritik öneme sahip. Top, duvarlardan, raketten ve tuğlalardan sekecek. Raketin neresine çarptığına bağlı olarak sekme açısı değişecek. Bu detayları Konu 24.4’ünde adım adım uygulayacağız. Şimdilik temel geri sekme mantığını kavradığından emin ol, çünkü tüm proje bu temelin üzerine inşa edilecek.

24.2 Hasar ve Can Sistemi

Bir RPG oyununda düşmana kılıçla vurduğunu düşün: düşman bir darbede yok olmuyor, can barı azalıyor ve birkaç vuruştan sonra yeniliyor. Ya da bir platform oyununda karakterin düşmana temas ettiğinde birkaç saniyelik yanıp sönme efektiyle geçici dokunulmazlık kazanıyor. Bu mekanikler oyunlara derinlik ve strateji katar. Önceki konuda çarpışma sonrası geri sekmeyi öğrendin. Şimdi başka bir çarpışma tepkisi türü olan *hasar sistemine* geçiyoruz.

■ Tanım: Hasar Sistemi (Damage System)

Oyun nesnelerinin bir can değeri (*health*) taşıması ve çarpışma sonucunda bu değerin azaltılması mekaniğidir. Can sıfıra düştüğünde nesne yok edilir veya oyun biter.

24.2.1 Health ve Damage Kavramları

Can sistemi iki temel değişkene dayanır: *can* (health) ve *hasar* (damage). Her oyun nesnesi bir *can* değerine sahiptir ve çarpışma gerçekleştiğinde karşı nesnenin *hasar* değeri kadar can kaybeder. Bu yaklaşım sayesinde farklı düşman türleri farklı dayanıklılıklara sahip olabilir: bir zayıf düşman tek mermiyle yok olurken, bir patron düşman onlarca vuruşa dayanabilir.

Python – Can Sistemi Temeli

```

1  class Dusman(pygame.sprite.Sprite):
2      def __init__(self, x, y, tur="normal"):
3          super().__init__()
4          self.image = pygame.Surface((40, 40))
5          self.image.fill(KIRMIZI)
6          self.rect = self.image.get_rect(center=(x, y))
7          self.hiz_y = 2
8
9          # Can sistemi
10         if tur == "normal":
11             self.can = 1      # Tek vurus yeter
12             self.puan = 10
13         elif tur == "guclu":
14             self.can = 3      # Uc vurus gerekir
15             self.puan = 30
16         elif tur == "patron":
17             self.can = 10     # On vurus gerekir
18             self.puan = 100
19
20     def hasar_al(self, miktar):
21         """Hasari uygula ve olup olmedini dondur."""
22         self.can -= miktar
23         if self.can <= 0:
24             self.kill()

```

```

25     return True    # Yok edildi
26     return False   # Hala hayatta

```

Dikkat et: `hasar\_al()` metodu bir boolean değer döndürüyor. Bu tasarım deseni çok kullanışlı çünkü çağıran kod, düşmanın yok olup olmadığını kontrol edebilir ve buna göre skor güncellemesi veya patlama efekti tetikleyebilir.

24.2.2 Çarpışma Sonrası Hasar Uygulama

Bölüm 22’da öğrendiğin `pygame.sprite.spritecollide()` fonksiyonunu şimdi hasar sistemiyle birleştirelim. Daha önce çarpışan Sprite’ları doğrudan yok ediyorduk (`dokill=True`). Artık hasar sistemimiz olduğuna göre, çarpışma sonrasında hemen yok etmek yerine hasar uygulayacağız.

Python – Mermi-Dusman Hasar Sistemi

```

1  # Her karede carpisma kontrolu
2  for mermi in mermiler:
3      # Bu mermiyle carpisan tum dusmanlari bul
4      carpisan_dusmanlar = pygame.sprite.spritecollide(
5          mermi, dusmanlar, False # dokill=False: dusmani
6          ↪ hemen silme
7      )
8
9      for dusman in carpisan_dusmanlar:
10         # Dusmana hasar ver
11         oldu = dusman.hasar_al(mermi.hasar)
12
13         if oldu:
14             # Dusman yok edildi: skor guncelle
15             skor += dusman.puan
16             # Patlama efekti olustur (Konu 3'te gorecegiz)
17
18             # Mermiyi yok et (bir dusmana isabet etti)
19             mermi.kill()
20             break # Bu mermi islendi, sonraki mermiye gec

```

Burada `dokill=False` kullanmamız kritik. Çünkü düşmanın gerçekten yok olup olmayacağına `hasar\_al()` metodu karar verecek. Eğer düşmanın canı henüz bitmemişse hayatta kalacak, sadece canı azalacak.

● Bilgi: Mermi Hasar Değeri

Farklı mermi türlerine farklı hasar değerleri atayabilirsin. Örneğin normal mermi 1 hasar, güçlendirilmiş mermi 3 hasar, roket ise 5 hasar verebilir. Bu, Bölüm 19’de öğrendiğin Sprite alt sınıfı kalıbıyla kolayca uygulanır.

24.2.3 Görsel Geri Bildirim: Can Bari

Oyuncunun düşmanın ne kadar canı kaldığını görmesi önemli. En yaygın yöntem, düşmanın üstüne küçük bir can bari çizmektir. Bu, Bölüm 21'daki **Rect** çizim bilgini kullanır.

Python – Dusman Can Bari

```

1  class Dusman(pygame.sprite.Sprite):
2      def __init__(self, x, y, max_can=3):
3          super().__init__()
4          self.max_can = max_can
5          self.can = max_can
6          # ... diger ozellikler
7
8      def hasar_al(self, miktar):
9          self.can -= miktar
10         if self.can <= 0:
11             self.kill()
12             return True
13         return False
14
15     def can_bari_ciz(self, yuzey):
16         """Sprite'in ustune can bari cizer."""
17         bar_genislik = self.rect.width
18         bar_yukseklik = 4
19         bar_x = self.rect.x
20         bar_y = self.rect.y - 8 # Sprite'in 8 piksel
21                               ↪ ustunde
22
23         # Arka plan (kirmizi)
24         pygame.draw.rect(yuzey, KIRMIZI,
25                           (bar_x, bar_y, bar_genislik,
26                            ↪ bar_yukseklik))
27
28         # Can orani (yesil)
29         oran = self.can / self.max_can
30         pygame.draw.rect(yuzey, YESIL,
31                           (bar_x, bar_y,
32                            int(bar_genislik * oran),
33                            ↪ bar_yukseklik))

```

Can bari iki dikdörtgenden oluşur: altta kırmızı bir arka plan (toplam can alanı) ve üstte yeşil bir dolgu (mevcut can oranı). **oran** değişkeni 0.0 ile 1.0 arasında bir değer tutar ve yeşil dikdörtgenin genişliğini belirler. Can azaldıkça yeşil alan daralır ve kırmızı arka plan görünür hale gelir.

▲ Dikkat: draw() ve can_bari_ciz() Sırası

`group.draw()` çağrısı Sprite'ların `image` özelliğini çizer. Can bari ise `image`'in parçası değil, ayrıca çizilir. Bu yüzden can barını `group.draw()`'dan sonra çizmelisin, aksi takdirde Sprite görseli can barının üstünü kapatır.

24.2.4 Invincibility Frames (Dokunulmazlık Kareleri)

Birçok oyunda karakter hasar aldıktan sonra kısa bir süre dokunulmaz olur. Bu mekanizma, oyuncunun art arda hasar alarak anında ölmesini engeller. Klasik platformerlerde karakterin yanıp sönmesi bu durumun görsel göstergesidir.

■ Tanım: Invincibility Frames (i-frames)

Bir nesnenin hasar aldıktan sonra belirli bir süre boyunca (genellikle 0.5–2 saniye) tekrar hasar almasını engelleyen zamanlayıcı mekanizmadır. Bu süre boyunca nesne genellikle yanıp söner.

Python – Dokunulmazlık Sistemi

```

1 class Oyuncu(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.can = 3
5         self.max_can = 3
6
7         # Dokunulmazlık sistemi
8         self.dokunulmaz = False
9         self.dokunulmazlik_suresi = 1500 # milisaniye
10        self.dokunulmazlik_baslangic = 0
11        self.gorunur = True
12        self.yanip_sonme_hizi = 100 # ms
13
14        def hasar_al(self, miktar):
15            """Dokunulmazlık kontrolü ile hasar al."""
16            if self.dokunulmaz:
17                return False # Dokunulmaz, hasar almaz
18
19            self.can -= miktar
20            if self.can <= 0:
21                self.kill()
22                return True # Oyuncu oldu
23
24            # Dokunulmazlık baslat
25            self.dokunulmaz = True
26            self.dokunulmazlik_baslangic =
27                ↪ pygame.time.get_ticks()
28            return False

```

```

28
29 def update(self):
30     # Dokunulmazlik zamanlayicisi
31     if self.dokunulmaz:
32         gecen = pygame.time.get_ticks() -
33             ↪ self.dokunulmazlik_baslangic
34         if gecen >= self.dokunulmazlik_suresi:
35             # Dokunulmazlik bitti
36             self.dokunulmaz = False
37             self.gorunur = True
38         else:
39             # Yanip sonme efekti
40             self.gorunur = (gecen //
41                 ↪ self.yanip_sonme_hizi) % 2 == 0
42
43     # ... hareket kodu

```

Bu kodda üç önemli nokta var. Birincisi, `hasar\_al()` metodu önce `self.dokunulmaz` bayrağını kontrol eder; eğer oyuncu dokunulmazsa hasar uygulanmaz. İkincisi, hasar alındığında `self.dokunulmaz = True` yapılır ve zamanlayıcı başlatılır. Üçüncüsü, `update()` içinde geçen süre kontrol edilir ve `self.gorunur` değişkeni yanıp sönme efekti için periyodik olarak değiştirilir.

Yanıp sönme efektini uygulamak için çizim aşamasında `self.gorunur` değerini kontrol etmen yeterli:

Python – Yanip Sonme Cizimi

```

1 # Ana dongude cizim
2 for sprite in tum_spritelar:
3     # Oyuncu icin gorunurluk kontrolu
4     if hasattr(sprite, 'gorunur') and not sprite.gorunur:
5         continue # Bu kareyi cizme (yanip sonme)
6     yuzey.blit(sprite.image, sprite.rect)

```

💡 İpucu: Profesyonel İpucu

Yanıp sönme yerine yarı saydam efekti de kullanabilirsin. `self.image.set_alpha(128)` ile Sprite'ı yarı saydam yapıp dokunulmazlık bitince `self.image.set_alpha(255)` ile tekrar opak yapabilirsin. Bu, daha modern bir görünüm sağlar.

24.2.5 Uzay Kaçışı Bağlamında

Bölüm 20'da başlattığımız Uzay Kaçışı projesinde hasar sistemi şu şekilde kullanılacak: oyuncu gemisi 3 canla başlar, düşman gemisiyle veya düşman mermisiyle temas ettiğinde 1 can kaybeder ve 1.5 saniyelik dokunulmazlık kazanır. Düşmanlar ise türüne göre 1–3 can arasında değişir. Normal düşmanlar tek mermiyle, güçlü düşmanlar üç mermiyle yok edilir. Bu sistemin tam implementasyonu Uzay Kaçışı v2 projesinde (Bölüm 6 sonunda) yer alacak.

24.3 Sprite Yok Etme ve Skor Sistemi

Bir düşmanın canı sıfıra düştüğünde ne olacak? Ekrandan aniden kaybolması oyuncuya tatmin edici bir his vermez. Bunun yerine küçük bir patlama efektiyle yok olması, skor sayacının artması ve belki bir ses efekti çalması gerekir. Bu konuda Sprite yok etme sürecini, basit patlama efektlerini ve skor sistemini ele alacağız.

24.3.1 Sprite Yaşam Döngüsü

Bölüm 20’da `kill()` metodunu öğrenmiştin: bir Sprite üzerinde `kill()` çağırıldığında o Sprite bağlı olduğu tüm gruplardan çıkarılır. Artık `group.draw()` çağırıldığında çizilmez ve `group.update()` çağırıldığında güncellenmez. Garbage collector da referansı kalmayan Sprite’ı bellekten temizler.

Ancak `kill()` çağırmadan önce yapmamız gereken işler olabilir: patlama animasyonu başlatmak, ses efekti çalmak, skor güncellemek veya güç-artırımı (power-up) düşürmek. Bu yüzden yok etme sürecini düzenli bir yapıda ele alalım.

Python – Duzgun Yok Etme Sureci

```

1 def dusman_yok_et(dusman, patlama_grubu, skor):
2     """Dusmani yok eder, patlama olusturur ve skoru
3     ↪ gunceller."""
4     # 1. Skor guncelle
5     skor += dusman.puan
6
7     # 2. Patlama efekti olustur (dusmanin konumunda)
8     patlama = Patlama(dusman.rect.center)
9     patlama_grubu.add(patlama)
10
11    # 3. Ses efekti cal (Bolum 7'de detaylandiracagiz)
12    # patlama_ses.play()
13
14    # 4. Dusmani tum gruplardan cikar
15    dusman.kill()
16
17    return skor

```

Bu fonksiyon dört adımı sıralı olarak gerçekleştirir. Dikkat et: `kill()` en son çağrılır çünkü ondan önce düşmanın konumuna ve puan değerine ihtiyacımız var. Kill çağırıldıktan sonra düşman grubundan çıkar ve artık erişemeyiz.

24.3.2 Basit Patlama Efekti

Patlama efekti, yok edilen nesnenin konumunda kısa süreliğine görünen bir animasyondur. Bölüm 8’de detaylı animasyon sistemi öğreneceksin, ama şimdilik basit bir yaklaşım kullanacağız: giderek büyüyen ve solan daireler.

Python – Basit Patlama Sınıfı

```

1 class Patlama(pygame.sprite.Sprite):
2     def __init__(self, merkez):
3         super().__init__()
4         self.merkez = merkez
5         self.yaricap = 5
6         self.max_yaricap = 30
7         self.alpha = 255
8         self.renk = TURUNCU
9
10        # Gorseli olustur
11        self._gorsel_guncelle()
12
13    def _gorsel_guncelle(self):
14        """Her karede gorseli yeniden olusturur."""
15        boyut = self.yaricap * 2
16        self.image = pygame.Surface((boyut, boyut),
17        ↪ pygame.SRCALPHA)
18        pygame.draw.circle(
19            self.image,
20            (*self.renk, self.alpha), # RGBA
21            (self.yaricap, self.yaricap),
22            self.yaricap
23        )
24        self.rect =
25        ↪ self.image.get_rect(center=self.merkez)
26
27    def update(self):
28        self.yaricap += 2 # Buyuyor
29        self.alpha -= 15 # Soluyor
30
31        if self.alpha <= 0:
32            self.kill() # Tamamen soldu, yok et
33            return
34
35        self._gorsel_guncelle()

```

Bu **Patlama** sınıfı her karede yarıçapını artırır ve alfa (saydamlık) değerini azaltır. Böylece patlama giderek büyür ve kaybolur. **SRCALPHA** bayrağı sayesinde çizdiğimiz dairenin alfa kanalı çalışır ve sönme efekti görünür. Alfa sıfıra düştüğünde **kill()** ile patlama kendini yok eder.

● Bilgi: Renk Geçişi

Daha gerçekçi bir patlama için rengi kademeli olarak değiştirebilirsin: başlangıçta beyaz (parlak merkez), sonra sarı, turuncu ve en son koyu kırmızı. Bu, ilerleme oranına göre renk interpolasyonu ile yapılır.

24.3.3 Çoklu Parçacık Patlaması

Tek bir daire yerine birden fazla küçük parçacık (particle) kullanan bir patlama çok daha etkileyici görünür. Her parçacık farklı yöne ve hızda fırlar.

Python – Parçacık Patlaması

```

1 class Parçacık(pygame.sprite.Sprite):
2     def __init__(self, merkez, renk=TURUNCU):
3         super().__init__()
4         self.image = pygame.Surface((4, 4),
5             ↳ pygame.SRCALPHA)
6         pygame.draw.circle(self.image, renk, (2, 2), 2)
7         self.rect = self.image.get_rect(center=merkez)
8
9         # Rastgele yön ve hız
10        aci = random.uniform(0, 2 * math.pi)
11        hiz = random.uniform(2, 6)
12        self.hiz_x = math.cos(aci) * hiz
13        self.hiz_y = math.sin(aci) * hiz
14
15        self.omur = random.randint(15, 30) # Kare sayısı
16
17    def update(self):
18        self.rect.x += self.hiz_x
19        self.rect.y += self.hiz_y
20        self.hiz_y += 0.1 # Hafif yerçekimi
21        self.omur -= 1
22        if self.omur <= 0:
23            self.kill()
24
25    def patlama_olustur(merkez, patlama_grubu,
26        ↳ parçacık_sayisi=15):
27        """Belirtilen konumda parçacık patlaması olusturur."""
28        renkler = [BEYAZ, SARI, TURUNCU, KIRMIZI]
29        for _ in range(parçacık_sayisi):
30            renk = random.choice(renkler)
31            parçacık = Parçacık(merkez, renk)
32            patlama_grubu.add(parçacık)

```

Her **Parçacık**, rastgele bir açıda ve hızda fırlar. `math.cos()` ve `math.sin()` ile açıyı x ve y hız bileşenlerine dönüştürüyoruz. Bölüm 23'deki trigonometri bilgimiz burada devreye giriyor. Ayrıca hafif bir yerçekimi (0.1) ekliyoruz, böylece parçacıklar düz gitmek yerine hafifçe aşağı doğru eğilir – bu, çok daha doğal görünür.

24.3.4 Skor Sistemi

Skor sistemi, oyuncuya ilerleme hissi veren temel bir oyun mekaniğidir. Bölüm 20'deki Uzay Kaçışı v1'de basit bir süre sayacı kullanmıştık. Şimdi çarpışma bazlı bir skor sistemi kuralım.

Python – Skor Yöneticisi

```

1  class SkorYoneticisi:
2      def __init__(self):
3          self.skor = 0
4          self.en_yuksekk = 0
5          self.combo = 0
6          self.combo_zamani = 0
7          self.combo_suresi = 2000 # ms - combo suresi
8
9      def puan_ekle(self, puan):
10         """Puan ekler ve combo kontrolu yapar."""
11         simdi = pygame.time.get_ticks()
12
13         # Combo kontrolu: 2 saniye icinde art arda vurur
14         if simdi - self.combo_zamani < self.combo_suresi:
15             self.combo += 1
16         else:
17             self.combo = 0
18
19         self.combo_zamani = simdi
20
21         # Combo carpani: 1x, 2x, 3x...
22         carpan = 1 + self.combo * 0.5
23         kazanilan = int(puan * carpan)
24         self.skor += kazanilan
25
26         # En yuksek skor guncelle
27         if self.skor > self.en_yuksekk:
28             self.en_yuksekk = self.skor
29
30         return kazanilan, self.combo + 1 # (kazanilan,
31         ↪ combo_seviye)
32
33     def sifirla(self):
34         """Yeni oyun icin skoru sifirlar."""
35         self.skor = 0
36         self.combo = 0

```

`SkorYoneticisi` sınıfı basit bir puan toplama işleminin ötesine geçer: *combo* sistemi sayesinde art arda hızlı vuruşlar daha fazla puan kazandırır. Eğer oyuncu 2 saniye içinde birden fazla düşman yok ederse combo sayacı artar ve puan çarpanı yükselir. Bu mekanik, oyuncuyu riskli ama ödüllendirici oyun stiline teşvik eder.

24.3.5 Yok Etme Akışının Tamamı

Şimdi tüm parçaları bir araya getirelim. Ana döngüdeki çarpışma kontrolü ve yok etme akışı şu şekilde çalışır:

Python – Tam Yok Etme Akisi

```

1  # Ana dongude her karede
2  carpismalar = pygame.sprite.groupcollide(
3      mermiler, dusmanlar, True, False
4  )
5
6  for mermi, dusman_listesi in carpismalar.items():
7      for dusman in dusman_listesi:
8          oldu = dusman.hasar_al(mermi.hasar)
9          if oldu:
10             # Patlama olustur
11             patlama_olustur(dusman.rect.center,
12                             ↪ patlamalar)
13             # Skor guncelle
14             kazanilan, combo =
15             ↪ skor_yon.puan_ekle(dusman.puan)

```

`pygame.sprite.groupcollide()` fonksiyonu iki grup arasındaki tüm çarpışmaları bir sözlük olarak döndürür. İlk parametre (**True**) mermileri otomatik yok eder, ikinci parametre (**False**) düşmanları yok etmez – çünkü düşmanların yok olup olmayacağına can sistemi karar verecek. Her çarpışma için `hasar\_al()` çağırılır ve düşman öldüyse patlama efekti tetiklenir.

⚠ Dikkat: Performans Notu

Her patlama 15 parçacık oluşturur ve her parçacık bir Sprite'tır. Ekranda aynı anda 10 patlama olursa 150 ek Sprite anlamına gelir. Bu genellikle sorun olmaz, ama çok sayıda eşzamanlı patlamada performans düşüşü yaşarsan parçacık sayısını azalt veya ömürlerini kısalt. Bölüm 13'te performans optimizasyonunu detaylı olarak inceleyeceğiz.

24.4 Yan Proje: Breakout (Tuğla Kırma)

Bu bölüm boyunca Rect çarpışma algılama, Sprite çarpışma fonksiyonları, temel fizik simülasyonu ve çarpışma tepkilerini öğrendin. Şimdi tüm bu bilgileri bir araya getirerek klasik bir oyun olan **Breakout (Tuğla Kırma)** oyununu geliştireceğiz. Bu proje, bölümün yan projesi olarak geri sekme, hasar ve yok etme kavramlarını tek bir oyunda birleştirir.

■ Tanım: Breakout (Tuğla Kırma)

1976'da Atari tarafından yayınlanan klasik arcade oyunudur. Oyuncu ekranın altındaki bir raketi kontrol eder, bir top raketten sekip yukarıdaki tuğlaları kırar. Tüm tuğlalar kırıldığında seviye tamamlanır, top ekranın altından düşerse oyuncu can kaybeder.

24.4.1 Proje Mimarisi

Breakout projesini dört ana sınıf üzerine inşa edeceğiz. **Raket** ekranın altında sağa-sola hareket eder. **Top** ekranda hareket eder ve yüzeylerden sekip tuğlaları kırar. **Tuğla** belirli bir düzende sıralanır ve vuruş olarak yok olur. Ana döngü ise tüm bu nesneleri Bölüm 20'daki grup sistemiyle yönetir.

Her sınıf, bu bölümde öğrendiğin bir kavramı doğrudan kullanır:

Tablo 24.1: Breakout sınıfları ve kullandıkları kavramlar

Sınıf	Kavram	Kaynak Ünite
Raket	Rect sınıfı, klavye hareketi	Bölüm 21
Top	Geri sekme, hız vektörü	Konu 24.1
Tuğla	Hasar/can sistemi	Konu 24.2
Yok etme	Patlama efekti, skor	Konu 24.3
Çarpışma	spritecollide()	Bölüm 22

24.4.2 Oyun Sabitleri

Önce oyunun temel sabitlerini tanımlayalım:

Python – Oyun Sabitleri

```

1  import pygame
2  import random
3  import math
4
5  # Ekran
6  GENISLIK = 800
7  YUKSEKLIK = 600
8  FPS = 60
9
10 # Renkler
11 SIYAH = (0, 0, 0)
12 BEYAZ = (255, 255, 255)
13 KIRMIZI = (231, 76, 60)
14 TURUNCU = (243, 156, 18)
15 SARI = (241, 196, 15)
16 YESIL = (46, 204, 113)
17 MAVI = (52, 152, 219)
18 MOR = (155, 89, 182)
19 GRI = (149, 165, 166)
20 KOYU_MAVI = (44, 62, 80)
21
22 # Raket
23 RAKET_GENISLIK = 100
24 RAKET_YUKSEKLIK = 12
25 RAKET_HIZ = 8
26 RAKET_RENK = MAVI
27
28 # Top

```

```

29 TOP_YARICAP = 8
30 TOP_HIZ = 5
31 TOP_RENK = BEYAZ
32
33 # Tugla
34 TUGLA_GENISLIK = 70
35 TUGLA_YUKSEKLIK = 25
36 TUGLA_BOSLUK = 4
37 TUGLA_SATIR = 6
38 TUGLA_SUTUN = 10
39 TUGLA_UST_BOSLUK = 60

```

24.4.3 Raket Sınıfı

Raket, Bölüm 21'deki `Rect` bilgini ve Bölüm 14'deki klavye girdisi bilgini birleştirir:

Python – Raket Sinifi

```

1 class Raket(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.image = pygame.Surface(
5             (RAKET_GENISLIK, RAKET_YUKSEKLIK)
6         )
7         self.image.fill(RAKET_RENK)
8         self.rect = self.image.get_rect()
9         self.rect.centerx = GENISLIK // 2
10        self.rect.bottom = YUKSEKLIK - 30
11        self.hiz = RAKET_HIZ
12
13    def update(self):
14        tuslar = pygame.key.get_pressed()
15        if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
16            self.rect.x -= self.hiz
17        if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
18            self.rect.x += self.hiz
19
20        # Ekran sinirlari
21        if self.rect.left < 0:
22            self.rect.left = 0
23        if self.rect.right > GENISLIK:
24            self.rect.right = GENISLIK

```

Raket sadece yatay ekseninde hareket eder. `clamp\_ip()` yerine manuel sınır kontrolü kullanıyoruz çünkü raketi sadece yatayda sınırlamak istiyoruz, dikey konumu sabit kalmalı.

24.4.4 Top Sınıfı

Top sınıfı, Konu 24.1'deki geri sekme mekaniğini ve Bölüm 23'deki hız vektörü kavramını kullanır:

Python – Top Sınıfı

```

1  class Top(pygame.sprite.Sprite):
2      def __init__(self, raket):
3          super().__init__()
4          self.image = pygame.Surface(
5              (TOP_YARICAP * 2, TOP_YARICAP * 2),
6              ↪ pygame.SRCALPHA
7          )
8          pygame.draw.circle(
9              self.image, TOP_RENK,
10             (TOP_YARICAP, TOP_YARICAP), TOP_YARICAP
11         )
12         self.rect = self.image.get_rect()
13         self.raket = raket
14
15         # Baslangic: raket ustunde, hareketsiz
16         self.rect.midbottom = self.raket.rect.midtop
17         self.hiz_x = 0
18         self.hiz_y = 0
19         self.aktif = False # Space ile firlatilacak
20
21     def firlat(self):
22         """Topu rastrele bir acida yukari firlatir."""
23         if not self.aktif:
24             self.aktif = True
25             aci = random.uniform(math.radians(220),
26                                 ↪ math.radians(320))
27             self.hiz_x = TOP_HIZ * math.cos(aci)
28             self.hiz_y = TOP_HIZ * math.sin(aci)
29
30     def update(self):
31         if not self.aktif:
32             # Raket ustunde bekle
33             self.rect.midbottom = self.raket.rect.midtop
34             return
35
36         # Hareketi uygula
37         self.rect.x += self.hiz_x
38         self.rect.y += self.hiz_y
39
40         # Duvar carpismalari (sol, sag, ust)
41         if self.rect.left <= 0:
42             self.rect.left = 0
43             self.hiz_x = abs(self.hiz_x)
44         elif self.rect.right >= GENISLIK:
45             self.rect.right = GENISLIK

```

```

44         self.hiz_x = -abs(self.hiz_x)
45
46     if self.rect.top <= 0:
47         self.rect.top = 0
48         self.hiz_y = abs(self.hiz_y)
49
50     # Alt kenar: top dustu (can kaybı)
51     if self.rect.top > YUKSEKLİK:
52         self.aktif = False
53         self.rect.midbottom = self.raket.rect.midtop
54         self.hiz_x = 0
55         self.hiz_y = 0

```

Top başlangıçta raketin üstünde hareketsiz bekler (`aktif = False`). Oyuncu Space tuşuna bastığında `firlat()` çağrılır ve top rastgele bir açıda yukarı doğru fırlar. Trigonometri kullanımına dikkat et: `math.cos()` ve `math.sin()` ile açıyı x ve y hız bileşenlerine dönüştürüyoruz. Açı aralığı 220–320 derece (yaklaşık yukarı-sol ile yukarı-sağ arası) olarak belirlenmiş, böylece top her zaman yukarıya doğru gider.

Duvar çarpışmalarında Konu 24.1'deki konum düzeltmeli geri sekme tekniğini kullanıyoruz. Alt kenar ise özel: top ekranın altından çıkarsa bu bir can kaybı anlamına gelir ve top tekrar raketin üstüne yerleşir.

24.4.5 Raket-Top Çarpışması: Açılı Sekme

Breakout'un en önemli mekaniği, topun raketin neresine çarptığına bağlı olarak farklı açılarda sekmesidir. Raketin sol ucuna çarpan top sola, sağ ucuna çarpan top sağa, ortasına çarpan top düz yukarı gider. Bu mekanik oyuncuya topun yönünü kontrol etme olanağı tanır.

Python – Raket-Top Çarpışması

```

1  def raket_top_carpismasi(top, raket):
2      """Topun raketten sekme acisini hesaplar."""
3      if not top.aktif:
4          return
5
6      if top.rect.colliderect(raket.rect) and top.hiz_y > 0:
7          # Carpisma noktasinin raket üzerindeki orani (-1.0
8          ↪ ile 1.0)
9          carpisma_noktasi = (
10              (top.rect.centerx - raket.rect.left) /
11              ↪ raket.rect.width
12          )
13          # 0.0 (sol uc) ile 1.0 (sağ uc) arasında normalize
14          ↪ et
15          # -1.0 ile 1.0 arasına çevir
16          oran = carpisma_noktasi * 2 - 1 # -1 sol, 0 orta,
17          ↪ 1 sağ

```



```

15     # Sekme acisi: 150 derece (sol) ile 30 derece
    ↪ (sag) arasi
16     # Ust yarikure: 150 ... 90 ... 30 derece
17     aci = math.radians(150 - oran * 60)
18
19     # Mevcut hizi koru, yonu degistir
20     hiz = math.sqrt(top.hiz_x**2 + top.hiz_y**2)
21     top.hiz_x = hiz * math.cos(acı)
22     top.hiz_y = -abs(hiz * math.sin(acı)) # Her zaman
    ↪ yukari
23
24     # Topu raketin ustune yerlestir (ic ice gecmeyi
    ↪ engelle)
25     top.rect.bottom = raket.rect.top

```

Bu fonksiyon şu adımları izler. Birincisi, çarpışma noktasının raketin sol ucundan ne kadar uzakta olduğunu hesaplar (0.0 ile 1.0 arası). İkincisi, bu oranı -1.0 ile 1.0 aralığına dönüştürür. Üçüncüsü, orana göre sekme açısını belirler: sol uç 150 derece (sola doğru), orta 90 derece (düz yukarı), sağ uç 30 derece (sağa doğru). Dördüncüsü, mevcut hız büyüklüğünü koruyarak yeni yönü uygular.

● Bilgi: Hız Artışı

Oyunu daha zorlu yapmak için her raket çarpışmasında topun hızını hafifçe artırabilirsin: `hiz *= 1.02`. Böylece oyun ilerledikçe top hızlanır ve zorluk kademeli olarak artar. Ancak bir maksimum hız limiti koymayı unutma, aksi takdirde top kontrol edilemez bir hıza ulaşır.

24.4.6 Tuğla Sınıfı

Tuğlalar Konu 24.2'deki can sistemiyle çalışır. Farklı renkler farklı dayanıklılıkları temsil eder:

Python – Tuğla Sınıfı

```

1  # Tuğla turleri: (renk, can)
2  TUGLA_TURLERI = {
3      1: (SARI, 1),          # Sari: 1 vurus
4      2: (TURUNCU, 1),      # Turuncu: 1 vurus
5      3: (KIRMIZI, 2),      # Kirmizi: 2 vurus
6      4: (MOR, 2),          # Mor: 2 vurus
7      5: (MAVI, 3),         # Mavi: 3 vurus
8      6: (GRI, -1),         # Gri: kirilamaz
9  }
10
11 class Tuğla(pygame.sprite.Sprite):
12     def __init__(self, x, y, tur=1):
13         super().__init__()

```

```

14     renk, self.can = TUGLA_TURLERI.get(tur, (BEYAZ,
15         ↪ 1))
16     self.max_can = self.can
17     self.renk = renk
18     self.puan = tur * 10
19
20     self.image = pygame.Surface(
21         (TUGLA_GENISLIK, TUGLA_YUKSEKLIK)
22     )
23     self._gorsel_guncelle()
24     self.rect = self.image.get_rect(topleft=(x, y))
25
26     def _gorsel_guncelle(self):
27         """Can durumuna gore gorseli gunceller."""
28         self.image.fill(self.renk)
29         # Kenarliklari ciz
30         pygame.draw.rect(
31             self.image, BEYAZ,
32             (0, 0, TUGLA_GENISLIK, TUGLA_YUKSEKLIK), 1
33         )
34         # Hasar almissa catlaklari goster
35         if self.max_can > 1 and self.can < self.max_can:
36             oran = self.can / self.max_can
37             if oran <= 0.5:
38                 # Agir hasar: capraz cizgi
39                 pygame.draw.line(self.image, SIYAH,
40                     (5, 5),
41                     (TUGLA_GENISLIK - 5,
42                     TUGLA_YUKSEKLIK - 5), 2)
43             # Hafif hasar: kucuk cizgi
44             pygame.draw.line(self.image, SIYAH,
45                 (TUGLA_GENISLIK // 3, 3),
46                 (TUGLA_GENISLIK // 2,
47                 TUGLA_YUKSEKLIK - 3), 1)
48
49     def hasar_al(self):
50         """Tuglaya hasar verir. Kirilip kirilmadigini
51         ↪ dondurur."""
52         if self.can == -1:
53             return False # Kirilamaz tugla
54
55         self.can -= 1
56         if self.can <= 0:
57             self.kill()
58             return True # Kirildi
59
60         self._gorsel_guncelle()
61         return False # Hala saglam

```

Her tuğla türü farklı renk, can ve puan değerine sahip. Gri tuğlalar (`can = -1`) kırılmaz ve engel görevi görür. Hasar alan ama henüz kırılmayan tuğlalar

üzerinde çatlak efekti görünür – bu, oyuncuya görsel geri bildirim sağlar.

24.4.7 Tuğla Düzeni Oluşturma

Tuğlaları bir grid (ızgara) düzeninde yerleştireceğiz. Satır numarası tuğla türünü belirler: üst satırlar daha dayanıklı ve daha değerli.

Python – Tuğla Düzeni

```

1 def tuglari_olustur(tugla_grubu, tum_spritelar):
2     """Tuğla gridini olusturur."""
3     # Tuğla alaninin baslangic x pozisyonu (ortalamak
4     ↪ için)
5     toplam_genislik = (
6         TUGLA_SUTUN * (TUGLA_GENISLIK + TUGLA_BOSLUK) -
7         ↪ TUGLA_BOSLUK
8     )
9     baslangic_x = (GENISLIK - toplam_genislik) // 2
10
11     for satir in range(TUGLA_SATIR):
12         for sutun in range(TUGLA_SUTUN):
13             x = baslangic_x + sutun * (TUGLA_GENISLIK +
14             ↪ TUGLA_BOSLUK)
15             y = TUGLA_UST_BOSLUK + satir * (
16                 TUGLA_YUKSEKLIK + TUGLA_BOSLUK
17             )
18
19             # Ust satirlar daha guclu
20             tur = TUGLA_SATIR - satir
21             tugla = Tuğla(x, y, tur)
22             tugla_grubu.add(tugla)
23             tum_spritelar.add(tugla)

```

24.4.8 Top-Tuğla Çarpışması

Top bir tuğlaya çarptığında iki şey olmalı: top sekmeli ve tuğla hasar almalı. Çarpışmanın hangi yönden geldiğini belirlemek önemli çünkü topun hangi hız bileşenini ters çevireceğimizi etkiler.

Python – Top-Tuğla Çarpışması

```

1 def top_tugla_carpismasi(top, tugla_grubu, patlama_grubu,
2     ↪ skor_yon):
3     """Top-tuğla carpismasini isler."""
4     if not top.aktif:
5         return
6
7     carpisan = pygame.sprite.spritecollide(top,
8     ↪ tugla_grubu, False)
9
10    for tugla in carpisan:

```

```

9      # Carpisma yonunu belirle
10     # Topun oncesi konumuna gore karar ver
11     dx = top.rect.centerx - tugla.rect.centerx
12     dy = top.rect.centery - tugla.rect.centery
13
14     # Yatay mi dikey mi carpisma?
15     if abs(dx) / tugla.rect.width > abs(dy) /
16         ↪ tugla.rect.height:
17         # Yatay carpisma (sol veya sag kenar)
18         top.hiz_x = -top.hiz_x
19     else:
20         # Dikey carpisma (ust veya alt kenar)
21         top.hiz_y = -top.hiz_y
22
23     # Tuglaya hasar ver
24     kirildi = tugla.hasar_al()
25     if kirildi:
26         # Patlama efekti
27         patlama_olustur(tugla.rect.center,
28             ↪ patlama_grubu, 8)
29         # Skor guncelle
30         skor_yon.puan_ekle(tugla.puan)
31
32     break # Bir karede tek tugla carpismasi isle

```

Çarpışma yönü tespiti biraz karmaşık görünebilir ama mantık basit: topun merkezinden tuğlanın merkezine olan mesafenin yatay ve dikey bileşenlerini karşılaştırıyoruz. Yatay mesafe oransal olarak daha büyükse top yandan çarpmıştır (`hiz_x` ters çevrilir), dikey mesafe daha büyükse üstten veya alttan çarpmıştır (`hiz_y` ters çevrilir).

24.4.9 Ana Oyun Döngüsü

Tüm parçaları bir araya getiren ana oyun döngüsü:

Python – Breakout Ana Dongusu

```

1  def main():
2      pygame.init()
3      ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
4      pygame.display.set_caption("Breakout - Tuğla Kirma")
5      saat = pygame.time.Clock()
6      font = pygame.font.Font(None, 36)
7
8      # Sprite gruplari
9      tum_spritelar = pygame.sprite.Group()
10     tugla_grubu = pygame.sprite.Group()
11     patlama_grubu = pygame.sprite.Group()
12
13     # Nesneler
14     raket = Raket()

```

```

15 top = Top(raket)
16 tum_spritelar.add(raket, top)
17
18 # Tuğlaları oluştur
19 tuğlaları_olustur(tuğla_grubu, tum_spritelar)
20
21 # Oyun durum değişkenleri
22 skor_yon = SkorYoneticisi()
23 canlar = 3
24 calistir = True
25
26 while calistir:
27     saat.tick(FPS)
28
29     # --- OLAYLAR ---
30     for olay in pygame.event.get():
31         if olay.type == pygame.QUIT:
32             calistir = False
33         elif olay.type == pygame.KEYDOWN:
34             if olay.key == pygame.K_SPACE:
35                 top.firlat()
36
37     # --- GÜNCELLEME ---
38     tum_spritelar.update()
39     patlama_grubu.update()
40
41     # Raket-top carpismasi
42     raket_top_carpismasi(top, raket)
43
44     # Top-tuğla carpismasi
45     top_tuğla_carpismasi(
46         top, tuğla_grubu, patlama_grubu, skor_yon
47     )
48
49     # Top dustu mu?
50     if not top.aktif and top.hiz_x == 0:
51         canlar -= 1
52         if canlar <= 0:
53             # Oyun bitti
54             calistir = False
55
56     # Tüm tuğlalar kırıldı mı?
57     kirilamaz = [t for t in tuğla_grubu if t.can ==
58                  ↪ -1]
59     if len(tuğla_grubu) <= len(kirilamaz):
60         # Seviye tamamlandı
61         calistir = False
62
63     # --- CIZIM ---
64     ekran.fill(KOYU_MAVI)
65     tum_spritelar.draw(ekran)

```

```

65     patlama_grubu.draw(ekran)
66
67     # HUD: Skor ve can
68     skor_yazi = font.render(
69         f"Skor: {skor_yon.skor}", True, BEYAZ
70     )
71     can_yazi = font.render(
72         f"Can: {canlar}", True, BEYAZ
73     )
74     ekran.blit(skor_yazi, (10, 10))
75     ekran.blit(can_yazi, (GENISLIK - 100, 10))
76
77     pygame.display.flip()
78
79     pygame.quit()

```

Bu döngüde her karede sırasıyla olaylar işlenir, tüm Sprite'lar güncellenir, çarpışmalar kontrol edilir ve ekran çizilir. Dikkat edersen Bölüm 11'de öğrendiğin klasik Input → Update → Render yapısını birebir takip ediyoruz.

Bu programı çalıştırdığında koyu mavi arka plan üzerinde renkli tuğlalar, bir raket ve bir top göreceksin. Space tuşuyla topu fırlat ve ok tuşlarıyla (veya A/D) raketi hareket ettirerek tuğlaları kır. Üst satırlardaki mavi tuğlalar 3 vuruşa, kırmızı ve mor tuğlalar 2 vuruşa, sarı ve turuncu tuğlalar ise tek vuruşa dayanır.

💡 İpucu: Projeyi Geliştir

Breakout projesini daha da geliştirebilirsin:

- Bazı tuğlalardan güç-artırım (power-up) düşür: geniş raket, çoklu top, yavaş top
- Seviye sistemi ekle: tüm tuğlalar kırılınca yeni ve daha zor bir düzen oluştur
- Tuğlaları CSV dosyasından yükleyerek özel seviyeler tasarla
- Topun izini gösteren bir kuyruk efekti ekle
- Bölüm 7'deki ses bilgisiyle tuğla kırma ve sekme ses efektleri ekle

24.5 Sonuç

24.5.1 Ünite 4 Özeti

Bu ünite de çarpışma algılamanın ötesine geçerek çarpışma *tepkilerini* öğrendin. Konu 24.1'de hız vektörünü ters çevirerek geri sekme mekaniğini, elastik ve inelastik çarpışma kavramlarını ve sıkışma problemini çözmeyi kavradın. Konu 24.2'nde can/hasar sistemiyle nesnelere dayanıklılık kazandırdın, can barı çizimini ve invincibility frame (dokunulmazlık kareleri) mekaniğini uyguladın. Konu 24.3'de Sprite yok etme sürecini düzene soktun, basit ve çoklu parçacık patlama efektleri oluşturduğun, combo mekanizmalı bir skor sistemi tasarladın. Son olarak Konu 24.4'nda tüm bu bilgileri Breakout (Tuğla Kırma) yan projesinde bir araya getirdin.

24.5.2 Bölüm 6 Genel Özeti

Bölüm 6'yı tamamladın, tebrikler! Bu bölüm boyunca çarpışma algılama ve fizik temellerinin tamamını kavradın. Şimdi geriye dönüp neler öğrendiğine bir bakalım.

Bölüm 21'da `pygame.Rect` sınıfının yapısını, `collidect()` ile iki dikdörtgenin çarpışma kontrolünü ve `collidepoint()` ile fare tıklama algılamayı öğrendin.

Bölüm 22'da `pygame.sprite.spritecollide()` ile tek bir Sprite'ın gruba karşı çarpışma kontrolünü, `pygame.sprite.groupcollide()` ile iki grup arasındaki toplu çarpışma tespitini ve özel çarpışma maskeleri kullanmayı kavradın.

Bölüm 23'de yerçekimi simülasyonu, hız ve ivme kavramları, sürtünme katsayısı ve temel trigonometri ile açılı hareket hesaplamalarını uyguladın.

Son olarak bu üniteye çarpışma tepkilerini (geri sekme, hasar, yok etme) öğrendin ve Breakout yan projesiyle tüm bölüm boyunca edindiğin bilgileri tek bir oyunda birleştirdin.

24.5.3 Anahtar Kavramlar

Tablo 24.2: Bölüm 6 – Anahtar kavramlar

Kavram	Açıklama
<code>collidect()</code>	İki Rect'in örtüşüp örtüşmediğini kontrol eder
<code>collidepoint()</code>	Bir noktanın Rect içinde olup olmadığını kontrol eder
<code>spritecollide()</code>	Bir Sprite'ın gruba karşı çarpışmasını tespit eder
<code>groupcollide()</code>	İki grup arasındaki çarpışmaları tespit eder
Yerçekimi	Her karede dikey hıza sabit bir değer eklenmesi
Sürtünme	Her karede hızın belirli bir oranda azaltılması
Geri sekme	Çarpışma yüzeyine dik hız bileşeninin ters çevrilmesi
Hasar sistemi	Nesnelerin can taşınması ve çarpışmada can kaybetmesi
i-frames	Hasar sonrası geçici dokunulmazlık süresi
Patlama efekti	Yok edilen nesne konumunda parçacık animasyonu
Combo	Art arda vuruşlarda artan puan çarpanı
Breakout	Raket, top ve tuğla mekaniğiyle klasik arcade oyunu

24.5.4 Önemli Komutlar

Tablo 24.3: Bölüm 6 – Önemli PyGame komutları

Komut	İşlev
<code>rect.colliderect(diger)</code>	İki dikdörtgenin çarpışmasını kontrol eder
<code>rect.collidepoint(x, y)</code>	Noktanın dikdörtgen içinde olup olmadığını
<code>sprite.spritecollide(s, g, dk)</code>	Sprite-grup çarpışma tespiti
<code>sprite.groupcollide(g1, g2, d1, d2)</code>	Grup-grup çarpışma tespiti
<code>sprite.collide_rect(s1, s2)</code>	İki Sprite'ın Rect çarpışması
<code>sprite.kill()</code>	Sprite'ı tüm gruplardan çıkarır
<code>math.cos(aci)</code>	Açının kosinüsü (x bileşeni)
<code>math.sin(aci)</code>	Açının sinüsü (y bileşeni)
<code>Surface(boyut, SRCALPHA)</code>	Alfa kanallı yüzey oluşturur
<code>time.get_ticks()</code>	Başlangıçtan beri geçen ms

24.5.5 Sıradaki Adım

Bölüm 7'de **Ses ve Müzik** konusuna geçeceğiz. Uzay Kaçışı projesine ses efektleri (mermi ateşleme, patlama, çarpışma) ve arka plan müziği ekleyeceğiz. `pygame.mixer` modülü ile ses dosyalarını yüklemeyi, çalmayı ve ses seviyesini kontrol etmeyi öğreneceksin. Breakout projesine de tuğla kırma ve sekme sesleri ekleyerek oyun deneyimini bir üst seviyeye taşıyabileceksin.

● Bilgi: Pratik Yap

Bölüm 6'da öğrendiklerini pekiştirmek için şu egzersizleri dene:

- Breakout'a güç-artırım sistemi ekle: tuğlalar kırıldığında rastgele düşen güç-artırımlar (geniş raket, çoklu top)
- Uzay Kaçışı v2'ye farklı düşman türleri ekle: hızlı ama zayıf, yavaş ama güçlü, zigzag hareket eden
- İki toplu bir bilardo simülasyonu yaz: toplar birbirine ve duvarlara çarparak seksin
- Can barını daha gelişmiş yap: renk geçişi (yeşil → sarı → kırmızı) ve sayısal gösterim ekle
- Patlama efektine duman parçacıkları ekle: patlamadan sonra yavaşça yükselen ve solan gri parçacıklar

“Ses olmadan bir oyun, ruhu olmayan bir bedene benzer.”

— ANONİM OYUN GELİSTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.mixer` modülünün başlatılması ve yapılandırılması
- Ses dosya formatlarının (WAV, OGG, MP3) farkları ve hangisini ne zaman kullanacağını
- `pygame.mixer.Sound` sınıfı ile ses yükleme ve temel ses işlemleri
- Ses kanalları kavramı ve aynı anda birden fazla ses çalma

25.1 Mixer Modülü

Şimdiye kadar oyunlarına hareket, çarpışma ve görsel efektler ekledin. Ama bir şey eksik kaldı: ses! Gözlerini kapat ve en sevdiğin oyunu düşün. Kılıcın çarpma sesi, arka plandaki epik müzik, düşman yok edildiğindeki patlama efekti... Ses olmadan o oyun aynı deneyimi verir miydi? Muhtemelen hayır. Bu ünite de oyunlarına hayat verecek ses sistemini öğreneceksin.

PyGame’de ses işlemleri `pygame.mixer` modülü tarafından yönetilir. Bu modül, ses dosyalarını yükleme, çalma, durdurma ve ses seviyesini ayarlama gibi tüm temel işlemleri sağlar. Tıpkı `pygame.display` modülünün ekranı yönetmesi gibi, `mixer` modülü de sesin tüm yönlerini kontrol eder.

■ Tanım: Mixer

`pygame.mixer` modülü, PyGame’in ses alt sistemidir. Ses dosyalarını yükler, dijital ses verilerini ses kartına gönderir ve birden fazla sesi aynı anda çalabilir. Çalışabilmesi için ilk olarak başlatılması (*initialize*) gerekir.

Mixer'ı bir müzik stüdyosundaki mikser masası olarak düşünebilirsin. Farklı enstrümanlardan (kanallardan) gelen sesleri alır, ses seviyelerini ayarlar ve hepsini birleştirerek hoparlöre gönderir. PyGame'deki mixer da tam olarak bunu yapar: farklı ses kaynaklarını yönetir ve ses kartına iletir.

25.1.1 Mixer'ı Başlatma

Bölüm ??'te `pygame.init` fonksiyonunun tüm PyGame alt sistemlerini başlattığını öğrenmiştin. Bu fonksiyon mixer'ı da otomatik olarak başlatır. Ancak ses sistemi için özel ayarlar yapmak istersen iki farklı yaklaşım kullanabilirsin.

Birinci yaklaşımda `pygame.mixer.init()` fonksiyonunu doğrudan çağırır-sın. Bu fonksiyon varsayılan ayarlarla ses sistemini başlatır:

Python – Mixer Başlatma – Temel Kullanım

```
1 import pygame
2
3 # Yontem 1: Sadece mixer'i baslat
4 pygame.mixer.init()
5
6 # Yontem 2: Tum PyGame'i baslat (mixer dahil)
7 pygame.init() # Bu da mixer'i otomatik baslatir
```

Peki varsayılan ayarlar nelerdir? Mixer başlatıldığında üç temel parametre belirlenir: *örnekleme hızı* (frequency), *bit derinliği* (size) ve *kanal sayısı* (channels). Bu kavramları kısaca açıklayalım.

Örnekleme hızı, sesin saniyede kaç kez örneklendiğini belirtir ve birimi Hertz'dir (Hz). Varsayılan değer 44100 Hz'dir ki bu CD kalitesine eşittir. *Bit derinliği* ise her örneğin kaç bit ile temsil edildiğini gösterir; varsayılan -16 değeri 16-bit işaretli ses anlamına gelir. *Kanal sayısı* mono (1) veya stereo (2) arasında seçim yapmanı sağlar; varsayılan olarak stereo (2) kullanılır.

● Bilgi: Varsayılan Mixer Ayarları

`pygame.mixer.init()` varsayılan olarak şu değerlerle başlar: örnekleme hızı 44100 Hz, bit derinliği -16 (16-bit işaretli) ve 2 kanal (stereo). Çoğu oyun için bu ayarlar gayet yeterlidir.

25.1.2 `pre_init()` ile Önceden Yapılandırma

Bazen varsayılan ayarları değiştirmek isteyebilirsin. Örneğin düşük donanımlı bir cihazda çalışacak bir oyun yapıyorsan örnekleme hızını düşürmek performansı artırabilir. Ya da ses dosyaların farklı bir formatta kaydedilmişse uyumlu ayarlar gerekebilir. Bu durumda `pygame.mixer.pre_init()` fonksiyonunu kullanırsın.

Python – pre_init() ile Mixer Yapılandırma

```

1  import pygame
2
3  # Mixer baslatılmadan ONCE ayarlari belirle
4  pygame.mixer.pre_init(
5      frequency=44100, # Ornekleme hizi (Hz)
6      size=-16,        # Bit derinligi (-16 = 16-bit
7                      ↪ isaretli)
8      channels=2,       # 1 = mono, 2 = stereo
9      buffer=512        # Tampon boyutu (kucuk = dusuk
10                      ↪ gecikme)
11 )
12
13 # Simdi PyGame'i baslat -- mixer bu ayarlarla baslar
14 pygame.init()

```

Burada dikkat edilmesi gereken nokta, `pre_init()` fonksiyonunun `pygame.init` veya `mixer.init()` çağrılmadan **önce** kullanılması gerektiğidir. Adındaki “pre” (ön) de buna işaret eder.

Tampon boyutu (buffer) özellikle önemli bir parametredir. Küçük bir tampon (örneğin 256 veya 512) ses gecikmesini azaltır ama CPU kullanımını artırır. Büyük bir tampon (2048 veya 4096) ise daha az CPU kullanır ama ses ile oyun olayları arasında fark edilir bir gecikme yaratabilir. Oyunlar için genellikle 512 veya 1024 iyi bir denge noktasıdır.

⚠ Dikkat: Buffer ve Gecikme

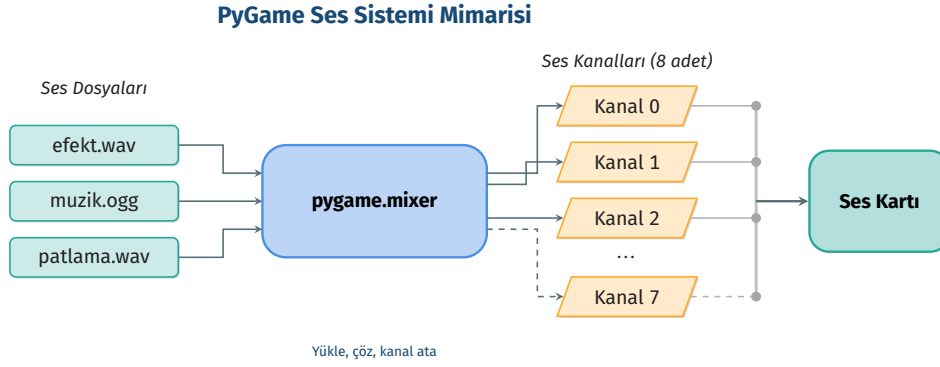
Tampon boyutunu çok büyük ayarlarsan (4096 ve üzeri), oyuncu bir tuşa bastığında ses efektinin gecikmeli çalındığını fark edebilir. Bu durum oyun deneyimini olumsuz etkiler. Özellikle aksiyon oyunlarında 512 veya 1024 değerini tercih et.

25.1.3 Ses Kanalları Kavramı

Bir oyunda aynı anda birden fazla ses çalması gerekir: arka plan müziği devam ederken oyuncu ateş eder, düşman patlama sesi çıkarır ve bir güç artırıcı toplanır. Tüm bu sesler aynı anda nasıl çalınır? İşte burada *ses kanalları* (channels) devreye girer.

Ses kanallarını bir orkestradaki enstrüman bölümleri gibi düşünebilirsin. Her kanal bağımsız olarak bir ses çalabilir ve tüm kanallar aynı anda aktif olabilir. PyGame varsayılan olarak 8 ses kanalı sağlar; bu da aynı anda 8 farklı ses efekti çalabileceğin anlamına gelir.

Bu kavramı görsel olarak incelediğimizde daha net anlayacağız.



Şekil 25.1: PyGame ses sistemi mimarisi: mixer, kanallar ve ses çıkışı

Şekil 25.1’de görüldüğü gibi, mixer modülü ses dosyalarını yükler ve her birini bir kanala atar. Her kanal bağımsız olarak ses çalabilir ve ses seviyesi ayarlanabilir. Tüm kanalların çıkışı birleştirilerek ses kartına gönderilir.

Kanal sayısını değiştirmek istersen `pygame.mixer.set_num_channels()` fonksiyonunu kullanabilirsin:

Python – Kanal Sayısını Ayarlama

```

1 import pygame
2 pygame.mixer.init()
3
4 # Kac kanal mevcut?
5 print(pygame.mixer.get_num_channels()) # Varsayilan: 8
6
7 # Kanal sayisini artir
8 pygame.mixer.set_num_channels(16) # Simdi 16 kanal var

```

Kanal sayısını artırmak, aynı anda daha fazla ses çalabilmeni sağlar. Ancak her kanalın bellek ve CPU kaynağı tükettiğini unutma. Çoğu 2D oyun için 8–16 kanal fazlasıyla yeterlidir.

💡 İpucu: Kanal Yönetimi

Kanal sayısını oyunun ihtiyacına göre ayarla. Basit bir oyunda 8 kanal yeterliyken, çok sayıda eş zamanlı ses efekti olan bir oyunda 16 veya daha fazla kanala ihtiyaç duyabilirsin. `pygame.mixer.find_channel()` fonksiyonu ile boş bir kanal bulabilirsin.

25.2 Ses Dosya Formatları

Ses sistemini başlattın, kanalların nasıl çalıştığını öğrendin. Peki ses dosyalarını nereden ve hangi formatta bulacaksın? Bu sorunun cevabı, oyunun kalitesini ve dosya boyutunu doğrudan etkiler.

Dijital ses dosyaları, ses dalgalarının sayısal olarak kaydedilmiş halidir. Farklı formatlar, bu verileri farklı şekillerde saklar. Kimisi sıkıştırma kullanmaz ve en

yüksek kaliteyi korur, kimisi ise dosya boyutunu küçültmek için kaliteden ödün verir. PyGame üç ana ses formatını destekler: WAV, OGG ve MP3.

25.2.1 WAV Formatı

WAV (Waveform Audio File Format), Microsoft ve IBM tarafından geliştirilmiş sıkıştırmasız bir ses formatıdır. Ses verisini olduğu gibi saklar, bu yüzden dosya boyutu büyüktür ama kalite kaybı yoktur.

WAV formatını bir dijital fotoğrafın RAW hali gibi düşünebilirsin. Hiçbir bilgi kaybı olmadan orijinal veriyi korur, ama dosya boyutu büyüktür. Bir saniyelik CD kalitesinde stereo WAV dosyası yaklaşık 172 KB yer kaplar.

Ne zaman WAV kullanmalısın? Kısa ses efektleri için WAV mükemmel bir tercihtir. Silah sesi, zıplama sesi, düğme tıklama sesi gibi birkaç saniyeyi geçmeyen efektlerde WAV kullan. Çünkü dosya boyutu küçük kalır, sıkıştırma çözme (decode) işlemi gerekmez ve CPU'ya ekstra yük bindirmez.

● Bilgi: WAV Dosyaları İçin Önerilen Ayarlar

Oyunlarda WAV dosyaları için 44100 Hz, 16-bit, mono ayarları genellikle yeterlidir. Kısa ses efektlerinde stereo yerine mono kullanmak dosya boyutunu yarıya indirir ve çoğu durumda fark edilmez.

25.2.2 OGG Formatı

OGG Vorbis, açık kaynaklı ve telif hakkı ücreti gerektirmeyen bir sıkıştırılmış ses formatıdır. Algısal sıkıştırma (perceptual compression) kullanır: insan kulağının fark edemeyeceği frekansları atarak dosya boyutunu küçültür. Tipik olarak WAV'a kıyasla dosya boyutunu 5–10 kat azaltır.

OGG formatının en büyük avantajı, iyi bir sıkıştırma oranıyla yüksek ses kalitesini korumasıdır. Ayrıca tamamen ücretsiz ve açık kaynaklıdır; herhangi bir lisans ücreti ödmeden kullanabilirsin. PyGame, OGG formatını çok iyi destekler ve tüm platformlarda sorunsuz çalışır.

Peki ne zaman OGG kullanmalısın? Arka plan müziği ve uzun ses efektleri için OGG en iyi tercihtir. Dakikalarca süren bir müzik parçasını WAV olarak saklamak yüzlerce megabayt yer kaplarken, OGG ile aynı parçayı birkaç megabayta sığdırabilirsin.

25.2.3 MP3 Formatı

MP3 (MPEG Audio Layer III), en yaygın bilinen sıkıştırılmış ses formatıdır. OGG'ye benzer şekilde algısal sıkıştırma kullanır. Ancak MP3'ün önemli bir dezavantajı vardır: geçmişte patent kısıtlamalarına tabi olmuştur. Patent süresi dolmuş olsa da PyGame'in MP3 desteği platforma göre değişkenlik gösterebilir.

PyGame belgesinde MP3 desteğinin "sınırlı" olduğu belirtilir. Bazı sistemlerde sorunsuz çalışırken, bazılarında beklenmedik hatalar verebilir. Bu nedenle oyun geliştirmede MP3 yerine OGG formatını tercih etmeni öneririm.

⚠ Dikkat: MP3 Uyumluluk Sorunu

PyGame'in MP3 desteği tüm platformlarda tutarlı değildir. Özellikle Linux sistemlerde sorun yaşanabilir. Oyununda platformlar arası uyumluluk istiyorsan **OGG formatını** tercih et.

25.2.4 Format Karşılaştırması

Bu üç formatı karşılaştırdığımızda her birinin güçlü ve zayıf yönlerini görebiliriz.

Ses Formatları Karşılaştırması



Şekil 25.2: WAV, OGG ve MP3 formatlarının karşılaştırması

Şekil 25.2'de üç formatın temel özelliklerini görebilirsin. Özetle, kısa ses efektleri için WAV, uzun müzik parçaları için OGG kullanmak en iyi stratejidir.

Tablo 25.1: Ses formatları karşılaştırma tablosu

Özellik	WAV	OGG	MP3	Not
Sıkıştırma	Yok	Var	Var	WAV en büyük dosya boyutu
Kalite	En yüksek	Yüksek	Yüksek	Fark çoğunlukla duyulmaz
Dosya boyutu	Büyük	Küçük	Küçük	OGG ≈ MP3 boyutu
CPU kullanımı	Düşük	Orta	Orta	WAV çözme gerektirmez
PyGame desteği	Tam	Tam	Sınırlı	OGG en güvenli seçim
Lisans	Ücretsiz	Ücretsiz	Ücretsiz	MP3 patenti süresi doldu

💡 İpucu: Altın Kural

Oyun geliştirmede şu basit kuralı uygula: kısa ses efektleri (< 3 saniye) için **WAV**, uzun müzik parçaları ve ambiyans sesleri için **OGG** kullan. Bu strateji hem dosya boyutunu optimize eder hem de tüm platformlarda sorunsuz çalışır.

25.2.5 Ücretsiz Ses Kaynakları

Oyunun için ses dosyalarını nereden bulacağını merak ediyorsan, birçok ücretsiz kaynak mevcut. **Freesound.org** binlerce ses efekti barındıran topluluk tabanlı bir platformdur. **OpenGameArt.org** ise özellikle oyun geliştirme için tasarlanmış görseller ve sesler sunar. **Kenney.nl** sitesi de yüksek kaliteli, telif ücretsiz ses paketleri sağlar. Bu kaynakları kullanırken lisans koşullarını kontrol etmeyi unutma; çoğu Creative Commons veya benzeri bir lisans altında sunulur.

25.3 Sound Sınıfı

Mixer'ı başlattın, ses formatlarını öğrendin. Şimdi sıra geldi sesin gerçek kahramanına: `pygame.mixer.Sound` sınıfına. Bu sınıf, ses dosyalarını belleğe yükler ve onları istediğin an çalmanı sağlar.

■ Tanım: Sound

`pygame.mixer.Sound`, bir ses dosyasını bellekte temsil eden sınıftır. Ses dosyasını okur, dijital ses verisine dönüştürür ve `play()`, `stop()`, `set_volume()` gibi metotlarla kontrol edilebilir hale getirir.

Sound nesnesini bir CD oynatıcıya takılmış CD gibi düşünebilirsin. CD'yi bir kez yüklersin (Sound nesnesi oluşturma), sonra istediğin zaman çal, durdur veya sesini ayarla. CD her zaman bellekte hazır bekler, böylece oyun sırasında dosyadan tekrar okuma gecikmesi yaşanmaz.

25.3.1 Ses Yükleme

Bir ses dosyasını yüklemek için `Sound` sınıfının yapıcı fonksiyonuna dosya yolunu verirsin:

Python – Sound Nesnesi Oluşturma

```
1 import pygame
2 pygame.mixer.init()
3
4 # Ses dosyasını yükle
5 ates_sesi = pygame.mixer.Sound("assets/sounds/ates.wav")
6 patlama_sesi =
   ↳ pygame.mixer.Sound("assets/sounds/patlama.ogg")
```

```

7  ziplama_sesi =
   ↪  pygame.mixer.Sound("assets/sounds/ziplama.wav")

```

Bu kod çalıştığında ses dosyaları diskten okunur ve tamamen belleğe yüklenir. Bundan sonra dosyaya tekrar erişmeye gerek kalmaz; ses verisi RAM’de hazır bekler. Yükleme işlemi sadece bir kez yapılır ve genellikle oyunun başlangıcında gerçekleştirilir.

Peki dosya bulunamazsa ne olur? PyGame bir `FileNotFoundError` hatası fırlatır. Bu durumu `try-except` bloğuyla yakalayarak oyunun çökmesini önleyebilirsiniz:

Python – Hata Yönetimi ile Ses Yükleme

```

1  import pygame
2  pygame.mixer.init()
3
4  try:
5      ates_sesi =
6      ↪  pygame.mixer.Sound("assets/sounds/ates.wav")
7      print("Ses dosyasi yuklendi!")
8  except FileNotFoundError:
9      print("Uyari: ates.wav bulunamadi!")
      ates_sesi = None

```

⚠ Dikkat: Bellek Kullanımı

Sound nesneleri ses verisini tamamen belleğe yükler. Uzun müzik dosyalarını (dakikalarca süren parçalar) Sound ile yüklemek çok fazla RAM tüketir. Arka plan müziği için bir sonraki ünite de öğreneceğin `pygame.mixer.music` modülünü kullan; bu modül müziği belleğe tamamen yüklemek yerine akış (stream) yöntemiyle çalar.

25.3.2 Ses Çalma ve Durdurma

Ses dosyasını yükledikten sonra `play()` metodu ile çalabilirsin. Bu kadar basit:

Python – Temel Ses Çalma

```

1  # Sesi bir kez cal
2  ates_sesi.play()
3
4  # Sesi 3 kez tekrarla (toplam 4 kez calar)
5  patlama_sesi.play(loops=3)
6
7  # Sesi sonsuz tekrarla (durdurana kadar)
8  ambiyans_sesi.play(loops=-1)

```

`play()` metodunun `loops` parametresi, sesin kaç kez **tekrar** edileceğini belirtir. Dikkat et: `loops=3` demek “3 kez tekrarla” anlamına gelir, yani ses toplamda

4 kez çalar (1 orijinal + 3 tekrar). Sonsuz döngü için `loops=-1` kullanılır — bu genellikle ambiyans sesleri veya döngüsel efektler için idealdir.

Çalan bir sesi durdurmak da aynı derecede kolaydır:

Python – Ses Durdurma

```
1 # Bu Sound nesnesinin tüm kopyalarını durdur
2 ates_sesi.stop()
3
4 # Tüm kanallardaki tüm sesleri durdur
5 pygame.mixer.stop()
```

`ates_sesi.stop()` sadece o Sound nesnesine ait sesleri durdurur. Diğer sesler çalmaya devam eder. `pygame.mixer.stop()` ise tüm kanallardaki tüm sesleri bir anda durdurur — bu genellikle sahne geçişlerinde veya oyun bitişinde kullanılır.

25.3.3 Ses Seviyesi Kontrolü

Bir oyunda tüm sesler aynı seviyede çalmamalıdır. Arka plan müziği daha kısık, patlama sesi daha yüksek olmalıdır. Sound sınıfının `set_volume()` metodu bu dengeyi kurmanı sağlar:

Python – Ses Seviyesi Ayarlama

```
1 # Ses seviyesi: 0.0 (sessiz) ile 1.0 (maksimum) arası
2 ates_sesi.set_volume(0.5) # Yarım ses
3 patlama_sesi.set_volume(0.8) # Yüksek ses
4 ambiyans_sesi.set_volume(0.2) # Düşük ses (arka plan için)
5
6 # Mevcut ses seviyesini öğren
7 seviye = ates_sesi.get_volume()
8 print(f"Ates sesi seviyesi: {seviye}")
```

Ses seviyesi 0.0 (tamamen sessiz) ile 1.0 (maksimum) arasında bir ondalıklı sayıdır. Varsayılan değer 1.0'dır. Oyunlarında sesler arasında denge kurmak için farklı seviyeler kullanman önemlidir; aksi takdirde tüm sesler aynı yükseklikte çalar ve kaotik bir ses ortamı oluşur.

25.3.4 Sound Nesnesinin Diğer Metotları

Sound sınıfı birkaç yararlı metot daha sunar. `get_length()` sesin süresini saniye cinsinden döndürür:

Python – Sound Metotları

```
1 import pygame
2 pygame.mixer.init()
3
4 ses = pygame.mixer.Sound("assets/sounds/patlama.ogg")
5
```

```

6  # Ses suresi (saniye)
7  sure = ses.get_length()
8  print(f"Ses suresi: {sure:.2f} saniye")
9
10 # Ses caliniyor mu?
11 kanal = ses.play()
12 if kanal is not None:
13     print(f"Kanal {kanal} üzerinde caliniyor")

```

`play()` metodunun bir `Channel` nesnesi döndürdüğünü fark ettin mi? Bu nesne, o sesi çalan kanalı temsil eder. Kanal nesnesi üzerinden sesin çalma durumunu sorgulayabilir, ses seviyesini kanal bazında ayarlayabilir veya sesi durdurabilirsin. Eğer boş kanal yoksa `play()` metodu `None` döndürür.

25.3.5 Uygulamalı Örnek: Temel Ses Sistemi

Şimdi öğrendiklerini birleştirerek küçük bir demo oluşturalım. Bu program mixer'ı başlatır, birden fazla ses oluşturur ve tuşlara basıldığında çalar. Adım adım inceleyelim.

Programatik Ses Oluşturma

Ses dosyası olmadan da test yapabilirsin. Python'un `array` ve `math` modüllerini kullanarak sinüs dalgası ile basit sesler üretebilirsin:

Python – Sinusoidal Ses Oluşturma

```

1  import pygame
2  import array
3  import math
4
5  def sinusoidal_ses(frekans=440, sure=0.3,
6     ↪ ornek_hizi=44100):
7     """Programatik olarak bir sinusoidal ses olusturur."""
8     ornek_sayisi = int(ornek_hizi * sure)
9     tampon = array.array("h")
10    genlik = 12000
11
12    for i in range(ornek_sayisi):
13        zaman = i / ornek_hizi
14        deger = int(genlik * math.sin(2.0 * math.pi *
15     ↪ frekans * zaman))
16        # Fade out efekti (son %20'de ses azalir)
17        if i > ornek_sayisi * 0.8:
18            oran = (ornek_sayisi - i) / (ornek_sayisi *
19     ↪ 0.2)
20            deger = int(deger * oran)
21        tampon.append(deger) # Sol kanal
22        tampon.append(deger) # Sag kanal

```

```
21 | return pygame.mixer.Sound(buffer=tampon)
```

Burada `Sound` sınıfının dosya yolu yerine `buffer` parametresi ile de oluşturulabildiğini görüyorsun. `array.array("h")` ile 16-bit işaretli tamsayı dizisi oluşturuyoruz; her örnek için hem sol hem sağ kanal verisi ekliyoruz (stereo). Son %20'lik kısımda ses seviyesini kademeli olarak azaltarak fade out efekti elde ediyoruz. Bu yöntem özellikle harici ses dosyası olmadan hızlı prototip geliştirmek için çok işe yarar.

Sesleri Hazırlama ve Tuş Eşlemesi

Oyun başlarken tüm sesleri bir kerede oluşturup bir sözlükte saklayalım. Ardından hangi tuşa basıldığında hangi sesin çalacağını belirleyelim:

Python – Ses Hazırlama ve Tuş Eşlemesi

```
1 def main():
2     """Ana oyun fonksiyonu."""
3     pygame.mixer.pre_init(frequency=44100, size=-16,
4         ↪ channels=2, buffer=512)
5     pygame.init()
6
7     # Sesleri olustur (programatik)
8     sesler = {
9         "ates": sinusoidal_ses(frekans=880, sure=0.15),
10        "patlama": sinusoidal_ses(frekans=150, sure=0.5),
11        "ziplama": sinusoidal_ses(frekans=520, sure=0.2),
12    }
13
14    # Ses seviyelerini ayarla
15    sesler["ates"].set_volume(0.6)
16    sesler["patlama"].set_volume(0.8)
17    sesler["ziplama"].set_volume(0.5)
18
19    # Tus-ses eslemesi
20    tus_haritasi = {
21        pygame.K_1: ("ates", "1: Ates sesi"),
22        pygame.K_2: ("patlama", "2: Patlama sesi"),
23        pygame.K_3: ("ziplama", "3: Ziplama sesi"),
24    }
25
26    ses_seviyesi = 0.5
```

Konu 25.1'de gördüğün `pre_init()` ile mixer'ı önceden yapılandırdığımıza dikkat et. Farklı frekanslarla farklı sesler elde ediyoruz: 880 Hz yüksek tınlı bir ateş sesi, 150 Hz ise düşük gümbürtülü bir patlama sesi verir. Sözlük yapısı sayesinde seslere isimle erişmek çok pratik. `tus_haritasi` ile tuş-ses eşlemesini de ayrı bir sözlükte tutarak kodu temiz tutuyoruz.

Oyun Döngüsünde Ses Çalma

Bölüm 11’te öğrendiğin oyun döngüsü yapısını burada ses çalma ile birleştiriyoruz. Bölüm 13’te detaylı gördüğün `pygame.KEYDOWN` olayları, hangi tuşa basıldığını kontrol etmemizi sağlıyor:

Python – Oyun Döngüsünde Ses Yönetimi

```

1  calistir = True
2  while calistir:
3      for olay in pygame.event.get():
4          if olay.type == pygame.QUIT:
5              calistir = False
6
7          elif olay.type == pygame.KEYDOWN:
8              # Ses calma tuslari
9              if olay.key in tus_haritasi:
10                 anahtar, aciklama = tus_haritasi[olay.key]
11                 sesler[anahtar].play()
12
13             # Ses seviyesi artir
14             elif olay.key == pygame.K_UP:
15                 ses_seviyesi = min(1.0, ses_seviyesi +
16                                     ↪ 0.1)
17                 for ses in sesler.values():
18                     ses.set_volume(ses_seviyesi)
19
20             # Ses seviyesi azalt
21             elif olay.key == pygame.K_DOWN:
22                 ses_seviyesi = max(0.0, ses_seviyesi -
23                                     ↪ 0.1)
24                 for ses in sesler.values():
25                     ses.set_volume(ses_seviyesi)
26
27             # Tum sesleri durdur
28             elif olay.key == pygame.K_s:
29                 pygame.mixer.stop()

```

Buradaki yapı oldukça düzenli: `tus_haritasi` sözlüğünde basılan tuş varsa ilgili sesi çalıştırıyoruz. Yukarı ve aşağı ok tuşlarıyla `set\_volume()` ile tüm seslerin seviyesini aynı anda değiştiriyoruz. `min()` ve `max()` ile ses seviyesinin 0.0–1.0 aralığının dışına çıkmasını engelliyoruz. `K_s` tuşu ise `pygame.mixer.stop()` ile tüm aktif sesleri bir anda susturuyor.

Görsel Geri Bildirim

Ses sistemi iyi çalışsa bile kullanıcıya görsel geri bildirim vermek önemlidir. Ses seviyesini bir çubukla gösterelim:

Python – Ses Seviyesi Göstergesi

```

1      # Cizim
2      ekran.fill((0, 0, 0))
3
4      # Ses seviyesi çubuğu
5      pygame.draw.rect(ekran, (100, 100, 100), (40, 355,
6      ↪ 200, 15))
7      pygame.draw.rect(
8      ↪ ekran, (50, 180, 100), (40, 355, int(200 *
9      ↪ ses_seviyesi), 15)
10     )
11     pygame.display.flip()
12     saat.tick(60)
13     pygame.quit()

```

Bölüm 12’te öğrendiğin `pygame.draw.rect()` ile iki dikdörtgen çiziyoruz: gri arka plan ve yeşil dolgu. Yeşil dikdörtgenin genişliğini `ses_seviyesi` ile çarparak seviyeye orantılı bir gösterge elde ediyoruz. Bu basit teknik, oyunlarında can çubuğu, enerji göstergesi veya yükleme çubuğu gibi pek çok yerde işine yarayacak.

Bu örnek basit görünse de oyunlardaki ses sisteminin temelini oluşturur. Bir sonraki üniteye sesleri `set\volume()` ile dengelerken, fade in/fade out efektleri eklerken ve ses kanallarını yönetirken hep bu temel yapı üzerine inşa edecek-

25.4 Sonuç

Bu üniteye oyunlarına ses eklemenin ilk adımlarını attın. `pygame.mixer` modülünün nasıl başlatılacağını, ses dosya formatlarının farklarını, `pygame.mixer.Sound` sınıfıyla ses yükleme ve çalma işlemlerini öğrendin. Artık oyunlarına ses efekti ekleyebilecek temel bilgiye sahipsin.

25.4.1 Anahtar Kavramlar

Tablo 25.2: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
Mixer	PyGame'in ses alt sistemi; ses dosyalarını yükler ve ses kartına gönderir
Örnekleme hızı	Sesin saniyede kaç kez örneklendiği (Hz); varsayılan 44100 Hz
Bit derinliği	Her ses örneğinin kaç bit ile temsil edildiği; varsayılan –16 (16-bit işaretli)
Ses kanalı	Bağımsız olarak ses çalabilen bir yol; varsayılan 8 kanal
WAV	Sıkıştırmasız ses formatı; yüksek kalite, büyük dosya boyutu
OGG Vorbis	Açık kaynaklı sıkıştırılmış ses formatı; iyi kalite, küçük dosya
MP3	Yaygın sıkıştırılmış format; PyGame'de sınırlı destek
Sound Buffer	Ses dosyasını bellekte temsil eden PyGame sınıfı Ses verisinin geçici olarak tutulduğu bellek alanı; boyutu gecikmeyi etkiler

25.4.2 Önemli Komutlar

Tablo 25.3: Ünite 1 – Temel PyGame ses komutları

Komut	Açıklama
<code>pygame.mixer.init()</code>	Ses sistemini varsayılan ayarlarla başlatır
<code>pygame.mixer.pre_init()</code>	Başlatmadan önce mixer ayarlarını belirler
<code>pygame.mixer.Sound(dosya)</code>	Ses dosyasını belleğe yükler
<code>ses.play(loops=0)</code>	Sesi çalar; loops tekrar sayısını belirtir
<code>ses.stop()</code>	Bu Sound nesnesinin tüm kopyalarını durdurur
<code>ses.set_volume(v)</code>	Ses seviyesini ayarlar (0.0 – 1.0)
<code>ses.get_volume()</code>	Mevcut ses seviyesini döndürür
<code>ses.get_length()</code>	Ses süresini saniye cinsinden döndürür
<code>mixer.set_num_channels(n)</code>	Ses kanalı sayısını ayarlar
<code>mixer.get_num_channels()</code>	Mevcut kanal sayısını döndürür
<code>mixer.stop()</code>	Tüm kanallardaki tüm sesleri durdurur

25.4.3 Sıradaki Adım

Bir sonraki ünite de ses efektlerinin ileri düzey kontrolünü öğreneceksin. Fade in ve fade out geçişleri, kanal bazlı ses yönetimi ve ses efektlerinin oyun olaylarıyla

senkronizasyonu gibi konulara dalacaksın. Ayrıca Bölüm 20’te başladığın Uzay Kaçışı projesine ateş, patlama ve güç artırıcı toplama sesleri ekleyeceksin.

💡 İpucu: Pratik Yap

Bu ünite de öğrendiklerini pekiştirmek için şunu dene: birkaç farklı WAV dosyası indir (Freesound.org’dan) ve her tuşa farklı bir ses ata. Bir basit piyano programı oluştur: A tuşu do, S tuşu re, D tuşu mi notasını çalsın. Bu egzersiz hem ses yüklemeyi hem de olay işlemeyi pekiştirir.

“Bir oyunda ses efektleri, oyuncuya yaptığı eylemin gerçekten bir etkisi olduğunu hissettirir.”

— CHRIS HECKER

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `Sound.play()` fonksiyonunun `loops`, `maxtime` ve `fade_ms` parametreleri ile detaylı kullanımı
- `set_volume()`, `get_volume()`, `fadeout()` ve `stop()` ile ses kontrolü
- `pygame.mixer.Channel` ile belirli kanalda ses çalma ve kanal yönetimi
- Uzay Kaçışı projesine mermi, patlama ve güç artırıcı toplama ses efektlerinin eklenmesi

26.1 `Sound.play()` – Ses Çalmanın Ayrıntıları

Önceki ünite de (Bölüm 25) PyGame’in ses sistemini başlatmayı ve `Sound` nesneleri oluşturmayı öğrendin. Bir ses dosyasını yükleyip `play()` çağırdığında sesin çaldığını gördün. Peki ya bir lazer sesinin art arda beş kez tekrarlanmasını istersen? Ya da bir patlama sesinin en fazla yarım saniye çalmasını? İşte bu ünite de `play()` fonksiyonunun tüm parametrelerini keşfedeceğiz ve ses efektlerini oyununa tam istediğin gibi entegre etmeyi öğreneceksin.

■ Tanım: `Sound.play()`

`Sound.play(loops=0, maxtime=0, fade_ms=0)` fonksiyonu, yüklenen bir ses efektini çalmaya başlar. Fonksiyon bir `Channel` nesnesi döndürür — bu nesne, sesin hangi kanalda çaldığını temsil eder. Eğer uygun bir kanal yoksa `None` döndürür.

26.1.1 Temel Kullanım

En basit haliyle `play()` parametresiz çağrılır:

Python – Basit Ses Calma

```
1 # Ses dosyasini yukle
2 lazer_ses = pygame.mixer.Sound("assets/sounds/lazer.wav")
3
4 # Sesi bir kez cal
5 lazer_ses.play()
```

Bu kod sesi tam olarak **bir kez** çalar. Ses bittiğinde kanal otomatik olarak serbest kalır ve başka sesler için kullanılabilir hale gelir. Peki ya sesin tekrar tekrar çalmasını istiyorsan?

26.1.2 **loops** Parametresi – Tekrar Sayısı

loops parametresi, sesin ilk çalınıştan sonra kaç kez daha tekrarlanacağını belirler. Dikkat: burada “tekrar” sayısı veriyorsun, toplam çalınma sayısı değil. Yani **loops=2** dediğinde ses toplamda **3 kez** çalar (1 orijinal + 2 tekrar).

Python – loops Parametresi

```
1 # Sesi 1 kez cal (varsayilan, loops=0)
2 lazer_ses.play()
3
4 # Sesi toplam 3 kez cal (1 + 2 tekrar)
5 lazer_ses.play(loops=2)
6
7 # Sesi sonsuz dongu ile cal (durdurana kadar)
8 lazer_ses.play(loops=-1)
```

Bu parametreyi günlük hayattan bir örnekle düşün: bir alarm sesi ayarlıyorsun. **loops=0** tek bir “bip” sesi, **loops=4** beş kez tekrarlayan bir alarm, **loops=-1** ise sen kapatana kadar öten çalar saat gibi çalışır.

⚠ Dikkat: Sonsuz Döngü

loops=-1 kullanırken sesi durdurmak senin sorumluluğundadır. `stop()` veya `fadeout()` çağırmaı unutursan ses sonsuza kadar çalmaya devam eder ve diğer ses efektlerinin çalmasını engelleyebilir.

26.1.3 **maxtime** Parametresi – Maksimum Süre

maxtime parametresi, sesin en fazla kaç **milisaniye** çalacağını belirler. Ses dosyası bu süreden daha uzunsa, belirtilen süre dolduğunda otomatik olarak kesilir. Varsayılan değeri **0**’dır ve bu “süre sınırı yok” anlamına gelir.

Python – maxtime Parametresi

```

1  # Patlama sesini en fazla 500ms cal
2  patlama_ses =
   ↳ pygame.mixer.Sound("assets/sounds/patlama.wav")
3  patlama_ses.play(maxtime=500)
4
5  # Arka plan efektini 2 saniye cal
6  ruzgar_ses =
   ↳ pygame.mixer.Sound("assets/sounds/ruzgar.wav")
7  ruzgar_ses.play(maxtime=2000)

```

Bu özellik özellikle uzun ses dosyalarından kısa efektler çıkarmak istediğinde kullanışlıdır. Örneğin, 5 saniyelik bir patlama ses dosyası varsa ama sadece ilk yarım saniyesini kullanmak istiyorsan `maxtime=500` işini görür.

26.1.4 fade_ms Parametresi – Yavaşça Başlama

`fade_ms` parametresi, sesin sıfır seviyesinden başlayıp belirtilen sürede tam ses seviyesine ulaşmasını sağlar. Bu, aniden başlayan sesleri daha yumuşak hale getirir — özellikle ortam sesleri ve döngüsel efektler için idealdir.

Python – fade_msParametresi

```

1  # Motor sesini 1 saniyede yükselterek baslat
2  motor_ses = pygame.mixer.Sound("assets/sounds/motor.wav")
3  motor_ses.play(loops=-1, fade_ms=1000)
4
5  # Ruzgar efektini 2 saniyede yükselterek 3 saniye cal
6  ruzgar_ses.play(maxtime=3000, fade_ms=2000)

```

İlk örnekte motor sesi sessizlikten başlar, 1 saniye içinde tam ses seviyesine ulaşır ve sonsuz döngüde çalmaya devam eder. İkinci örnekte ise rüzgar sesi 2 saniyede yükselir ama toplam 3 saniyeden fazla çalmaz.

26.1.5 Parametrelerin Birlikte Kullanımı

Bu üç parametre birlikte kullanıldığında çok esnek ses kontrolü elde edersin:

Python – Parametrelerin Birleşimi

```

1  # Lazer: 3 kez tekrar, her biri en fazla 200ms
2  lazer_ses.play(loops=2, maxtime=200)
3
4  # Ortam sesi: sonsuz dongu, 2sn fade-in, kanal nesnesi al
5  kanal = ruzgar_ses.play(loops=-1, fade_ms=2000)
6
7  # Uyari alarmI: 5 kez tekrar, 500ms fade-in
8  alarm_ses = pygame.mixer.Sound("assets/sounds/alarm.wav")
9  alarm_ses.play(loops=4, fade_ms=500)

```

İpucu: Channel Nesnesini Sakla

`play()` fonksiyonu bir `Channel` nesnesi döndürür. Bu nesneyi bir değişkende saklarsan, o sesin çaldığı kanal üzerinde sonradan kontrol sahibi olursun (duraklatma, ses seviyesi ayarlama, durdurma). Kanal yönetimini Konu 26.3’da detaylı öğreneceksin.

26.1.6 `play()` Dönüş Değeri

`play()` her zaman bir `Channel` nesnesi döndürür — ya da uygun kanal yoksa `None`. Bu dönüş değerini kontrol etmek iyi bir alışkanlıktır:

Python – Donus Degerini Kontrol Etme

```
1 kanal = lazer_ses.play()
2
3 if kanal is not None:
4     print("Ses caliniyor!")
5     print(f"Kanal numarası: {kanal}")
6 else:
7     print("Bos kanal yok, ses calinamadi!")
```

Eğer tüm kanallar meşgulse ve yeni bir ses çalmaya çalışırsan, PyGame varsayılan olarak en uzun süredir çalan kanalı kesip yeni sese verir. Ancak bu davranışı özelleştirmek istersen, kanal yönetimi konusunda (Konu 26.3) bunu detaylı göreceğiz.

Bilgi: Aynı Anda Birden Fazla Çalma

Aynı `Sound` nesnesini birden fazla kez `play()` ile çağırabilirsin. Her çağrı farklı bir kanalda çalar. Örneğin, hızlı ateş eden bir silah için her mermi atışında `lazer_ses.play()` dersen, aynı ses efekti birden fazla kanalda eş zamanlı çalabilir.

26.2 Ses Kontrolü – Seviye, Durdurma ve Geçişler

Bir ses efektini çalmak yalnızca başlangıçtır. Gerçek bir oyunda ses seviyesini oyun sırasında değiştirmen, sesleri yumuşak geçişlerle kapatman ve gerektiğinde anında durdurman gerekir. Düşün: karakterin bir mağaraya girdiğinde ortam sesinin kısılması, bir boss savaşında uyarı alarımının yükselmesi veya oyun duraklatıldığında tüm seslerin durması... Tüm bunlar ses kontrolü fonksiyonları ile mümkün.

26.2.1 `set_volume()` ve `get_volume()`

`Sound` nesnesinin ses seviyesini `set_volume()` ile ayarlar, `get_volume()` ile sorgularsın. Ses seviyesi `0.0` (sessiz) ile `1.0` (tam ses) arasında bir ondalıklı

sayıdır.

Python – Volume Ayarlama

```

1 patlama_ses =
  ↳ pygame.mixer.Sound("assets/sounds/patlama.wav")
2
3 # Ses seviyesini %50'ye ayarla
4 patlama_ses.set_volume(0.5)
5
6 # Mevcut seviyeyi sorgula
7 seviye = patlama_ses.get_volume()
8 print(f"Patlama ses seviyesi: {seviye}") # 0.5
9
10 # Sessiz yap
11 patlama_ses.set_volume(0.0)
12
13 # Tam ses
14 patlama_ses.set_volume(1.0)

```

`set_volume()` çağrısı, ilgili `Sound` nesnesinin tüm gelecekteki ve halihazırda çalan örneklerini etkiler. Yani bir ses zaten çalırken `set_volume(0.3)` dersen, o an çalan ses de dahil olmak üzere hepsi `0.3` seviyesine iner.

● Bilgi: Kanal ve Sound Seviyesi Farkı

PyGame’de iki farklı ses seviyesi kontrolü vardır: `Sound` nesnesinin kendi seviyesi ve `Channel`’ın (kanal) seviyesi. Son duyduğun ses, bu ikisinin çarpımıdır. Örneğin `Sound` seviyesi `0.5` ve `Channel` seviyesi `0.6` ise gerçek ses seviyesi $0.5 \times 0.6 = 0.3$ olur. Kanal seviyesini Konu 26.3’da öğreneceksin.

26.2.2 Dinamik Ses Seviyesi

Oyun sırasında ses seviyesini koşullara göre değiştirmek oyun deneyimini zenginleştirir. Örneğin, bir düşman yaklaştıkça tehlike sesinin yükselmesi gerçekçilik katar:

Python – Mesafeye Bağlı Volume

```

1 def mesafe_hesapla(a, b):
2     """İki nokta arası mesafe."""
3     dx = a[0] - b[0]
4     dy = a[1] - b[1]
5     return (dx**2 + dy**2) ** 0.5
6
7 # Her karede guncelle
8 mesafe = mesafe_hesapla(oyuncu.rect.center,
9     ↳ dusman.rect.center)
10 maks_mesafe = 500 # Bu mesafeden otesi sessiz

```

```

11 # Mesafe arttikca ses azalir (0.0 - 1.0 arasi)
12 seviye = max(0.0, 1.0 - (mesafe / maks_mesafe))
13 tehlike_ses.set_volume(seviye)

```

Bu kodda düşman yaklaştıkça `seviye` değeri `1.0`'a yaklaşır, uzaklaştıkça `0.0`'a iner. `max(0.0, ...)` ifadesi seviyenin negatife düşmesini engeller.

26.2.3 `stop()` – Anında Durdurma

`stop()` fonksiyonu, bir `Sound` nesnesinin çaldığı **tüm kanalları** anında durdurur. Ses kesilir, kaynaklar serbest kalır:

Python – `stop()` Kullanımı

```

1 motor_ses = pygame.mixer.Sound("assets/sounds/motor.wav")
2 motor_ses.play(loops=-1) # Sonsuz dongu
3
4 # ... oyun devam eder ...
5
6 # Motoru kapat
7 motor_ses.stop()

```

`stop()` geçişsizdir — ses anında kesilir. Eğer daha yumuşak bir kapanış istiyorsan `fadeout()` kullanmalısın.

⚠ Dikkat: `stop()` Kapsamı

`Sound.stop()`, o `Sound` nesnesinin çaldığı **tüm** kanalları durdurur. Eğer aynı ses efekti 3 farklı kanalda çalışıyorsa, üçü de durur. Sadece belirli bir kanalı durdurmak istersen `Channel.stop()` kullanmalısın.

26.2.4 `fadeout()` – Yumuşak Kapanış

`fadeout(ms)` fonksiyonu, sesi belirtilen milisaniye içinde kademeli olarak sıfıra indirip durdurur. Bu, ani kesintilerin kulağa hoş gelmediği durumlarda idealdir:

Python – `fadeout()` Kullanımı

```

1 # Motor sesini 2 saniyede sondur
2 motor_ses.play(loops=-1)
3
4 # ... bir sure sonra ...
5
6 # 2 saniyede yumusak kapanis
7 motor_ses.fadeout(2000)

```

`fadeout()` çağrıldığında ses hemen kesilmez. Belirtilen süre boyunca kademeli olarak azalır ve süre dolduğunda tamamen durur. Bu süreçte başka kodlar normal şekilde çalışmaya devam eder — `fadeout` arka planda gerçekleşir.

26.2.5 Sesin Durumunu Sorgulama

Bir sesin halihazırda çalıp çalmadığını `get_num_channels()` ile sorgulayabilirsin. Bu fonksiyon, o `Sound` nesnesinin kaç kanalda aktif olarak çaldığını döndürür:

Python – Ses Durumu Sorgulama

```
1 # Kac kanalda caliniyor?
2 aktif = lazer_ses.get_num_channels()
3 print(f"Lazer sesi {aktif} kanalda caliniyor")
4
5 # Sadece calmiyor ise yeni cal
6 if lazer_ses.get_num_channels() == 0:
7     lazer_ses.play()
```

Bu teknik, bir sesin birden fazla kez üst üste çalmasını engellemek istediğinde kullanışlıdır. Örneğin, bir kapı açılma sesi zaten çalırken tekrar başlatmak istemezsin.

💡 İpucu: Ses Uzunluğunu Öğrenme

`Sound.get_length()` fonksiyonu ses dosyasının süresini saniye cinsinden döndürür. Bu, `maxtime` veya `fadeout` değerlerini hesaplarken işine yarar:

```
sure = patlama_ses.get_length()
```

26.3 Kanal Yönetimi

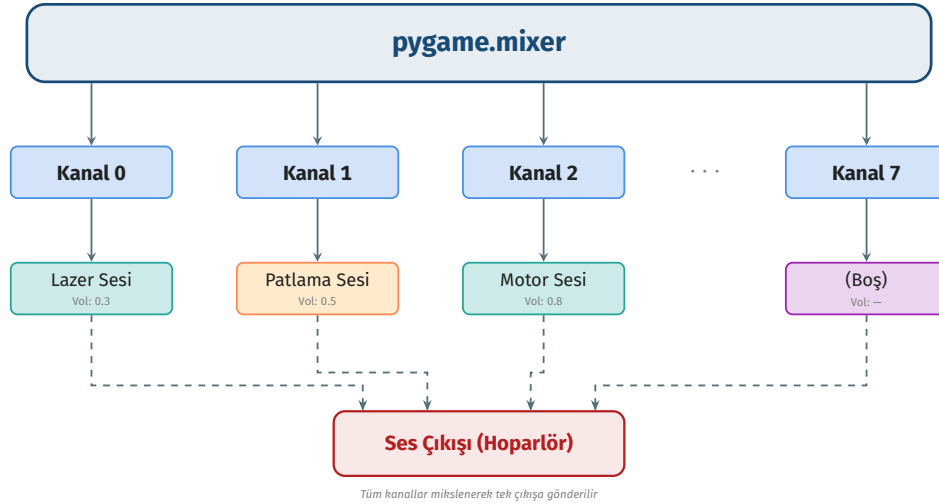
Şimdiye kadar sesleri çalıp durdurdun ama PyGame'in bu sesleri arka planda nasıl yönettiğini merak etmiş olabilirsin. Bir orkestrayı düşün: her müzisyen farklı bir enstrüman çalar ve şef kimin ne zaman çalacağına karar verir. PyGame'in ses sistemi de benzer çalışır — her ses bir *kanal*da (Channel) çalar ve sen bu kanalların şefi olabilirsin.

■ Tanım: Ses Kanalı (Channel)

`pygame.mixer.Channel`, mixer sistemindeki tek bir ses çalma yuvası temsil eder. Her kanal aynı anda yalnızca bir ses çalabilir. PyGame varsayılan olarak 8 kanal oluşturur ve bu sayı `pygame.mixer.set_num_channels()` ile değiştirilebilir.

Bu kavramı daha iyi anlamak için Şekil 26.1'ne bakalım.

Şekil 26.1'da görüldüğü gibi, mixer birden fazla kanalı paralel olarak yönetir. Kanal 0'da lazer sesi çalarken, Kanal 1'de patlama sesi ve Kanal 2'de motor sesi aynı anda çalabilir. Bu sayede oyununda birden fazla ses efekti eşzamanlı duyulur.



Şekil 26.1: PyGame mixer kanal sistemi: Her kanal aynı anda tek bir ses çalar, ancak farklı kanallar eşzamanlı olarak farklı sesler çalabilir.

26.3.1 Kanal Sayısını Ayarlama

Varsayılan 8 kanal çoğu basit oyun için yeterlidir, ancak yoğun aksiyon sahnelelerinde daha fazla kanal gerekebilir:

Python – Kanal Sayısı Ayarlama

```
1 import pygame
2
3 pygame.mixer.init()
4
5 # Mevcut kanal sayisini sorgula
6 print(f"Mevcut kanal sayisi:
   ↳ {pygame.mixer.get_num_channels()}") # 8
7
8 # 16 kanala cikar
9 pygame.mixer.set_num_channels(16)
10
11 # Dogrula
12 print(f"Yeni kanal sayisi:
   ↳ {pygame.mixer.get_num_channels()}") # 16
```

Ne kadar kanal ayarlamalısın? Genel kural olarak, oyununda aynı anda en fazla kaç farklı sesin çalabileceğini tahmin et ve birkaç tane fazladan ekle. Uzun Kaçış gibi bir oyun için 12–16 kanal yeterli olur.

26.3.2 Belirli Bir Kanalda Ses Çalma

`play()` fonksiyonu PyGame'in otomatik kanal seçimine bırakır. Ancak bazı durumlarda belirli bir kanalda ses çalmak istersin — örneğin, UI sesleri için ayrılmış bir kanal, silah sesleri için başka bir kanal kullanmak isteyebilirsin:

Python – Belirli Kanalda Calma

```

1  # Kanal nesnesi olustur (kanal numarası 0'dan baslar)
2  ui_kanal = pygame.mixer.Channel(0)
3  silah_kanal = pygame.mixer.Channel(1)
4  ortam_kanal = pygame.mixer.Channel(2)
5
6  # Sesleri yukle
7  tikla_ses = pygame.mixer.Sound("assets/sounds/tikla.wav")
8  lazer_ses = pygame.mixer.Sound("assets/sounds/lazer.wav")
9  ruzgar_ses =
   ↳ pygame.mixer.Sound("assets/sounds/ruzgar.wav")
10
11 # Her sesi kendi kanalında cal
12 ui_kanal.play(tikla_ses)
13 silah_kanal.play(lazer_ses)
14 ortam_kanal.play(ruzgar_ses, loops=-1)

```

`Channel.play()` fonksiyonu da `Sound.play()` gibi `loops`, `maxtime` ve `fade_ms` parametrelerini destekler. Aradaki fark, sesin hangi kanalda çalacağını senin belirlemendir.

26.3.3 Kanal Kontrolü

Her `Channel` nesnesinin kendi kontrol fonksiyonları vardır:

Python – Kanal Kontrolleri

```

1  kanal = pygame.mixer.Channel(1)
2  kanal.play(lazer_ses)
3
4  # Kanal ses seviyesini ayarla
5  kanal.set_volume(0.7)
6
7  # Kanalin calip calmadigini kontrol et
8  if kanal.get_busy():
9      print("Kanal aktif!")
10
11 # Kanali duraklat
12 kanal.pause()
13
14 # Kanali devam ettir
15 kanal.unpause()
16
17 # Kanali 1 saniyede sondur
18 kanal.fadeout(1000)
19
20 # Kanali anında durdur
21 kanal.stop()

```


● Bilgi: Duraklatma ve Devam Ettirme

`Channel.pause()` sesi olduğu noktada duraklatır — `Channel.unpause()` ile kaldığı yerden devam eder. `Sound.stop()` ise sesi tamamen sonlandırır ve başa sarmaz. Bu iki davranış farklıdır: duraklatma geçicidir, durdurma kalıcıdır.

26.3.4 Sıralı Ses Çalma – `queue()`

`Channel.queue()` fonksiyonu, mevcut ses bittiğinde otomatik olarak başlayacak bir sonraki sesi kuyruğa ekler:

Python – Kuyruk Sistemi

```
1 kanal = pygame.mixer.Channel(3)
2
3 giris_ses = pygame.mixer.Sound("assets/sounds/giris.wav")
4 dongu_ses = pygame.mixer.Sound("assets/sounds/dongu.wav")
5
6 # Once giris sesini cal, bitince dongu sesine gec
7 kanal.play(giris_ses)
8 kanal.queue(dongu_ses)
```

Bu özellik, giriş melodisi ve ardından döngüsel arka plan sesi gibi zincirleme ses efektleri oluşturmak için kullanışlıdır.

26.3.5 `set_endevent()` – Ses Bittiğinde Olay Tetikleme

`Channel.set_endevent()` ile bir kanaldaki ses bittiğinde olay kuyruğuna otomatik bir olay ekletebilirsin. Bu, ses bittiğinde bir şey yapman gereken durumlarda son derece kullanışlıdır — örneğin, geri sayım sesi bittikten sonra oyunu başlatmak gibi:

Python – `set_endevent` Kullanımı

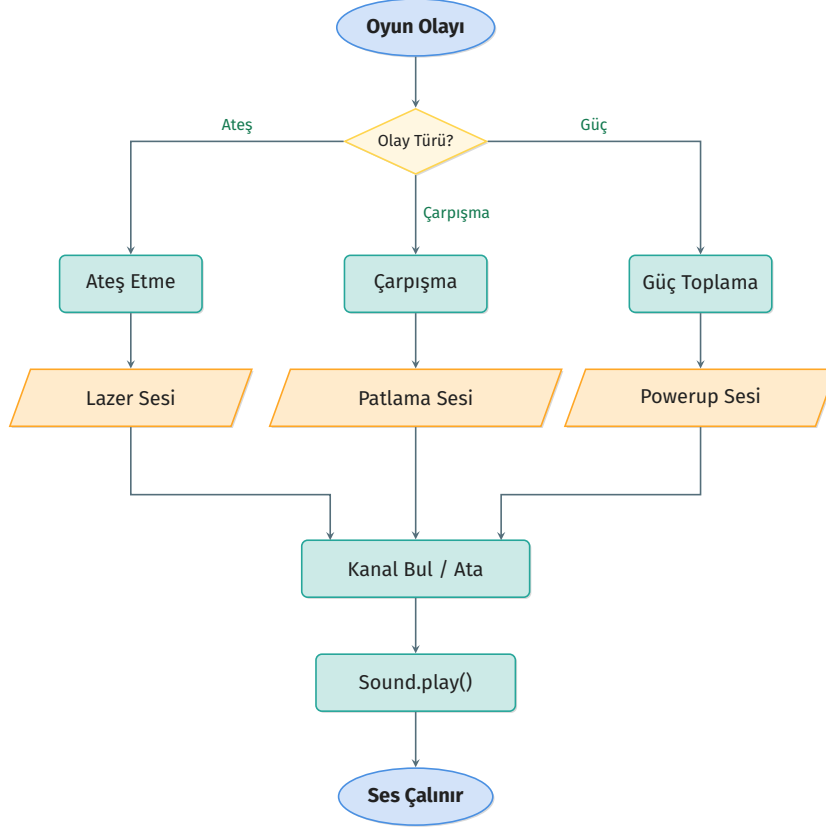
```
1 # Özel olay tani tanımla
2 SES_BITTI = pygame.USEREVENT + 1
3
4 kanal = pygame.mixer.Channel(0)
5 kanal.set_endevent(SES_BITTI)
6
7 # Sesi cal
8 geri_sayim =
9     ↪ pygame.mixer.Sound("assets/sounds/geri_sayim.wav")
10 kanal.play(geri_sayim)
11
12 # Oyun dongusunda olayi yakala
13 for olay in pygame.event.get():
14     if olay.type == SES_BITTI:
```

```

14 print("Geri sayım bitti! Oyun basliyor!")
15 oyun_baslat()

```

Şekil 26.2’nda oyun olayları ile ses efektleri arasındaki ilişkiyi görebilirsin.



Şekil 26.2: Oyun olayı → ses efekti tetikleme akış şeması: Her oyun olayı uygun ses efektini tetikler ve kanal yönetimi ile eşzamanlı çalma sağlanır.

Şekil 26.2’nda görüldüğü gibi, oyun döngüsünde bir olay meydana geldiğinde (çarpışma, ateş etme, güç toplama) ilgili ses efekti tetiklenir, uygun bir kanal bulunur ve ses çalınır. Kanal meşgulse ya otomatik atama yapılır ya da öncelikli bir kanal sistemi devreye girer.

İpucu: Kanal Havuzu Tasarımı

Büyük projelerde kanalları kategorilere ayırmak iyi bir pratiktir. Örneğin Kanal 0–3 silah sesleri, Kanal 4–5 UI sesleri, Kanal 6–7 ortam sesleri için ayrılabilir. Bu sayede bir kategorideki sesler diğerlerini engellemez.

26.4 Uzay Kaçışı – Ses Efektleri Entegrasyonu

Bölüm 22’da Uzay Kaçışı projesine düşman gemileri ve çarpışma algılama eklemiştin. Artık uzay gemini kontrol ederken lazer sesleri duyabilir, düşmanlar patladığında güçlü bir patlama efekti hissedebilir ve güç artırıcı topladığında

hoş bir toplama sesi duyabilirsin. Bu konuda öğrendiğin tüm ses efekti bilgilerini Uzay Kaçışı projesine entegre edeceğiz.

26.4.1 Ses Dosyalarını Hazırlama

Önce projemize ihtiyacımız olan ses dosyalarını tanımlayalım. Ücretsiz ses kaynakları olarak **freesound.org** ve **opengameart.org** sitelerini kullanabilirsin. Ses dosyalarını **assets/sounds/** dizinine koyacağız:

Python – Ses Dosyalarını Yükleme

```

1  # Ses dosyalarını yükle
2  class SesYoneticisi:
3      """Oyun ses efektlerini yöneten sınıf."""
4
5      def __init__(self):
6          """Ses efektlerini yükle ve yapılandır."""
7          self.lazer_ses =
8              ↳ pygame.mixer.Sound("assets/sounds/lazer.wav")
9          self.patlama_ses = pygame.mixer.Sound("assets/sou_
10              ↳ nds/patlama.wav")
11          self.guc_toplama_ses = pygame.mixer.Sound("assets_
12              ↳ /sounds/powerup.wav")
13          self.hasar_ses =
14              ↳ pygame.mixer.Sound("assets/sounds/hasar.wav")
15
16          # Ses seviyelerini ayarla
17          self.lazer_ses.set_volume(0.3)
18          self.patlama_ses.set_volume(0.5)
19          self.guc_toplama_ses.set_volume(0.4)
20          self.hasar_ses.set_volume(0.6)

```

Ses seviyelerini neden farklı ayarladığımıza dikkat et: lazer sesi sık tekrarlandığı için düşük tutulur, hasar sesi ise önemli bir geri bildirim olduğu için daha yüksek ayarlanır. Bu dengeleme işlemine ses tasarımında *mix* denir.

26.4.2 Mermi Ateşleme Sesi

Oyuncunun her mermi ateşlediğinde lazer sesi çalmalı. Bunu oyuncu sınıfının ateşleme metoduna ekleyeceğiz:

Python – Mermi Ateşleme Sesi

```

1  class Oyuncu(pygame.sprite.Sprite):
2      """Oyuncu gemi sınıfı."""
3
4      def __init__(self, ses_yoneticisi):
5          super().__init__()
6          # ... diğer özellikler ...
7          self.ses = ses_yoneticisi
8          self.son_ates = 0

```

```

9         self.ates_bekleme = 250 # ms
10
11     def ates_et(self, mermi_grubu):
12         """Mermi atesle ve ses cal."""
13         simdi = pygame.time.get_ticks()
14         if simdi - self.son_ates >= self.ates_bekleme:
15             mermi = Mermi(self.rect.centerx,
16                           ↪ self.rect.top)
17             mermi_grubu.add(mermi)
18             self.ses.lazer_ses.play()
19             self.son_ates = simdi

```

Burada `ates_bekleme` sayesinde mermi ateşleme hızını kontrol ediyoruz. Bu aynı zamanda lazer sesinin çok sık üst üste çalmasını da engeller. Dikkat edersen ses çalma işlemi tek bir satır: `self.ses.lazer_ses.play()`.

26.4.3 Patlama Sesi

Bir düşman yok edildiğinde patlama sesi çalmalı. Bu kontrolü oyun döngüsündeki çarpışma algılama kısmına ekleriz:

Python – Patlama Sesi

```

1 # Oyun dongusunda carpisma kontrolu
2 carpismalar = pygame.sprite.groupcollide(
3     mermiler, dusmanlar, True, True
4 )
5
6 for mermi, vurulan_dusmanlar in carpismalar.items():
7     for dusman in vurulan_dusmanlar:
8         # Patlama sesi cal
9         ses_yoneticisi.patlama_ses.play()
10        # Skor guncelle
11        skor += dusman.puan

```

Peki aynı karede 5 düşman birden patlarsa ne olur? Her biri için ayrı `play()` çağrılır ve PyGame her birini farklı bir kanalda çalar. Bu yüzden ses kanallarımızın yeterli olması önemli.

26.4.4 Güç Artırıcı Toplama Sesi

Oyuncu bir güç artırıcı (power-up) topladığında farklı bir ses efekti çalmak oyuncuya olumlu geri bildirim verir:

Python – Güç Artırıcı Sesi

```

1 # Güç artırıcı carpisma kontrolu
2 toplanan_gucler = pygame.sprite.spritecollide(
3     oyuncu, guc_gruplari, True
4 )

```

```

5
6 for guc in toplanan_gucler:
7     ses_yoneticisi.guc_toplama_ses.play()
8
9     # Guc turune gore etki uygula
10    if guc.tur == "hiz":
11        oyuncu.hiz_bonusu(5000) # 5 sn hiz bonusu
12    elif guc.tur == "kalkan":
13        oyuncu.kalkan_aktiflestir(3000) # 3 sn kalkan

```

26.4.5 Hasar ve Uyarı Sesleri

Oyuncu düşmanla çarpıştığında hasar sesi çalınmalı. Eğer can seviyesi kritik düşükse ek bir uyarı sesi de eklenebilir:

Python – Hasar Sesi

```

1 # Oyuncu-dusman carpisma kontrolu
2 dusman_carpisma = pygame.sprite.spritecollide(
3     oyuncu, dusmanlar, True
4 )
5
6 for dusman in dusman_carpisma:
7     oyuncu.can -= dusman.hasar
8     ses_yoneticisi.hasar_ses.play()
9
10    # Kritik can uyarisi
11    if oyuncu.can <= 20 and oyuncu.can > 0:
12        ses_yoneticisi.hasar_ses.play(loops=1) # Iki kez
        ↪ cal

```

26.4.6 Tüm Sesleri Bir Araya Getirme

Şimdi `SesYoneticisi` sınıfının tamamını ve oyun döngüsüne entegrasyonunu görelim:

Python – Tam Ses Entegrasyonu

```

1 class SesYoneticisi:
2     """Oyun ses efektlerini yoneten sinif."""
3
4     def __init__(self):
5         """Tum ses efektlerini yukle."""
6         self.sesler = {}
7         self._yukle()
8
9     def _yukle(self):
10        """Ses dosyalarini yukle ve seviyelerini
11        ↪ ayarla."""
12        ses_bilgileri = {

```

```

12         "lazer": ("assets/sounds/lazer.wav", 0.3),
13         "patlama": ("assets/sounds/patlama.wav", 0.5),
14         "powerup": ("assets/sounds/powerup.wav", 0.4),
15         "hasar": ("assets/sounds/hasar.wav", 0.6),
16     }
17     for anahtar, (dosya, seviye) in
18         ↪ ses_bilgileri.items():
19         ses = pygame.mixer.Sound(dosya)
20         ses.set_volume(seviye)
21         self.sesler[anahtar] = ses
22
23     def cal(self, ses_adi):
24         """Belirtilen ses efektini cal."""
25         if ses_adi in self.sesler:
26             self.sesler[ses_adi].play()
27
28     def tumu_durdur(self):
29         """Tum ses efektlerini durdur."""
30         for ses in self.sesler.values():
31             ses.stop()

```

Bu tasarımda sesleri bir sözlükte tutuyoruz. Bu sayede yeni ses eklemek için sadece `ses_bilgileri` sözlüğüne yeni bir satır eklememiz yeterli. `cal()` metodu ile ses adını vererek istediğimiz efekti çalabiliriz: `ses_yoneticisi.cal("lazer")`.

● Bilgi: Ses Dosyası Bulunamadığında

Ses dosyası bulunamazsa PyGame hata fırlatır ve oyun çöker. Üretim aşamadaki oyunlarda ses yüklemeyi bir `try/except` bloğuna almak ve dosya yoksa sessiz devam etmek iyi bir pratiktir.

26.5 Sonuç

Bu ünite de PyGame'in ses efekti sistemini derinlemesine öğrendin. Şimdi öğrendiğin kavramları iki tabloda özetleyelim.

Sound Fonksiyonları

Tablo 26.1: `pygame.mixer.Sound` temel fonksiyonları

Fonksiyon	Parametreler	Açıklama
<code>play()</code>	<code>loops, maxtime, fade_ms</code>	Sesi çal
<code>stop()</code>	—	Tüm kanalları durdur
<code>fadeout(ms)</code>	<code>ms</code>	Yumuşak kapanış
<code>set_volume(val)</code>	<code>0.0 - 1.0</code>	Ses seviyesi ayarla
<code>get_volume()</code>	—	Ses seviyesini sorgula
<code>get_length()</code>	—	Süreyi saniye olarak döndür
<code>get_num_channels()</code>	—	Aktif kanal sayısı

Channel Fonksiyonları

Tablo 26.2: `pygame.mixer.Channel` temel fonksiyonları

Fonksiyon	Parametreler	Açıklama
<code>play(sound)</code>	<code>loops, maxtime, fade_ms</code>	Belirli kanalda çal
<code>stop()</code>	—	Kanalı durdur
<code>pause()</code>	—	Kanalı duraklat
<code>unpause()</code>	—	Duraklamayı sürdür
<code>fadeout(ms)</code>	<code>ms</code>	Yumuşak kapanış
<code>set_volume(val)</code>	<code>0.0 - 1.0</code>	Kanal seviyesi
<code>get_busy()</code>	—	Kanal aktif mi?
<code>queue(sound)</code>	<code>Sound</code>	Sıradaki sesi ekle
<code>set_endevent(id)</code>	<code>event id</code>	Bitiş olayı tanımla

Temel İlkeler

Ses efektlerini oyununa eklerken şu ilkeleri aklında tut:

1. **Ses seviyesi dengesi:** Sık tekrarlanan sesler (lazer) düşük, önemli geri bildirimler (hasar) yüksek tutulmalı.
2. **Kanal planlama:** Ses kategorilerine göre kanal ayır — UI, silah, ortam sesleri birbirini engellemez.
3. **Hafıza yönetimi:** WAV dosyaları bellekte yer kaplar. Kısa ses efektleri için WAV, uzun sesler için OGG formatını tercih et.
4. **Yumuşak geçişler:** `fade_ms` ve `fadeout()` ile ani ses geçişlerinden kaçın.
5. **Hata yönetimi:** Ses dosyası bulunamazsa oyun çökmemeli — uygun hata yakalama mekanizması kullan.

Bir sonraki ünite (Bölüm 25'nün devamı olarak) arka plan müziği eklemeyi ve `pygame.mixer.music` modülünü öğreneceksin.

Arka Plan Müziği

“Müzik, oyuncunun duyduğu en güçlü duygu yönlendiricisidir.”

— NOBUO UEMATSU

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.mixer.music` modülü ile arka plan müziği yükleme ve çalma
- Müzik döngüsü, duraklatma, durdurma ve ses seviyesi kontrolleri
- Fade in/out efektleri ile akıcı müzik geçişleri
- Sahne bazlı müzik yönetimi ve `MusicManager` sınıfı tasarımı

27.1 Müzik Yükleme ve Çalma

Bölüm 25’te `pygame.mixer` modülünü başlatmayı ve Bölüm 26’nde `Sound` sınıfı ile kısa ses efektleri çalmayı öğrenmiştin. Peki bir oyundaki o unutulmaz arka plan melodileri nasıl çalıyor? Uzay Kaçışı oyununda menü ekranında huzurlu bir melodi, savaş anında heyecanlı bir ritim istesek ne yapacağız? İşte burada `pygame.mixer.music` modülü devreye giriyor.

■ Tanım: `pygame.mixer.music`

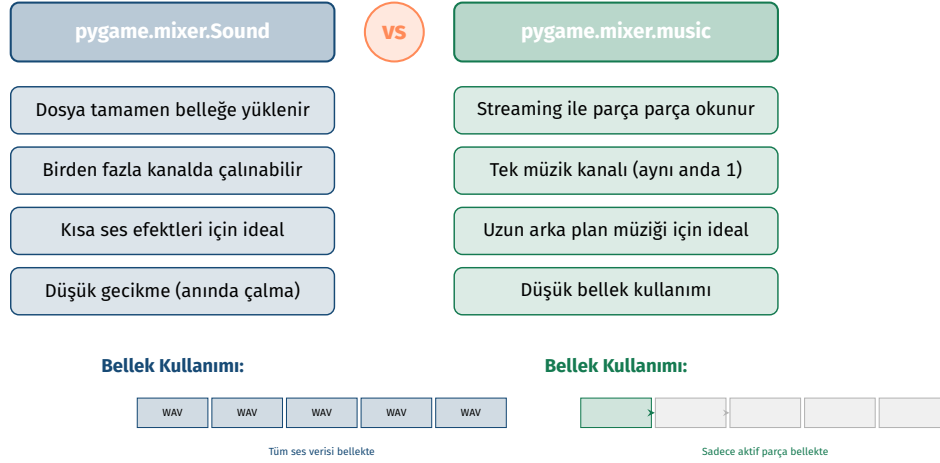
`pygame.mixer.music` modülü, arka plan müziği çalmak için tasarlanmış özel bir alt modüldür. `Sound` sınıfından farklı olarak müzik dosyasını **parça parça** (streaming) okur ve bellekte tamamını tutmaz. Bu sayede uzun müzik dosyalarını düşük bellek kullanımıyla çalabilirsin.

Bunu bir radyo yayını gibi düşünebilirsin. `Sound` sınıfı, tüm şarkıyı telefonuna indirip dinlemek gibidir — dosyanın tamamı belleğe yüklenir. `music` modülü ise

radio dinlemek gibidir — ses verisi anlık olarak işlenir, tamamını kaydetmeye gerek kalmaz.

27.1.1 Sound ve Music Farkı

Bu iki yaklaşım arasındaki farkı anlamak, doğru aracı doğru iş için kullanmanı sağlar.



Şekil 27.1: Sound ve Music modüllerinin karşılaştırması: bellekte tutma ve streaming yaklaşımları

Şekil 27.1’da gösterildiği gibi, **Sound** sınıfı dosyanın tamamını belleğe yükler ve birden fazla kanalda aynı anda çalınabilir. **music** modülü ise dosyayı parça parça okuyarak tek bir müzik kanalında çalar. Kısa ses efektleri (patlama, tıklama, zıplama) için **Sound**, uzun müzik parçaları için **music** kullanmalısın.

⚠ Dikkat: Tek Müzik Kanalı

pygame.mixer.music modülü aynı anda yalnızca **bir** müzik parçası çalabilir. Yeni bir müzik yüklediğinde ve çaldığında, o anda çalan müzik otomatik olarak durur. Aynı anda birden fazla müzik katmanı istiyorsan **Sound** sınıfını kullanman gerekir; ancak bu durumda bellek kullanımına dikkat etmelisin.

27.1.2 Müzik Dosyası Yükleme

Müzik yüklemek için **music.load()** fonksiyonunu kullanırız. Bu fonksiyon dosyayı belleğe tamamen yüklemeyi, sadece çalmaya hazırlar:

Python – Müzik Yükleme

```
1 import pygame
2
3 pygame.mixer.init()
4
```

```

5 # Muzik dosyasini yukle (streaming icin hazirla)
6 pygame.mixer.music.load("assets/sounds/menu_muzigi.ogg")
7
8 # Yuklenen muzigi cal
9 pygame.mixer.music.play()

```

Bu iki satır, arka plan müziği çalmanın en temel halidir. `load()` fonksiyonu dosyayı streaming için hazırlar, `play()` ise çalmayı başlatır.

● Bilgi: Desteklenen Formatlar

`music` modülü WAV, OGG ve MP3 formatlarını destekler. Ancak Bölüm 25'te öğrendiğin gibi, platform bağımsız çalışma ve sıkıştırma kalitesi açısından **OGG Vorbis** formatı en güvenli seçimdir. MP3 bazı sistemlerde sorun çıkarabilir.

27.1.3 Müzik Çalma Seçenekleri

`play()` fonksiyonu birkaç önemli parametre alır:

Python – `play()` Parametreleri

```

1 # Muzigi bir kez cal
2 pygame.mixer.music.play()
3
4 # Muzigi 3 kez tekrarla (toplam 4 kez calar)
5 pygame.mixer.music.play(loops=3)
6
7 # Muzigi sonsuz dongu ile cal (en yaygin kullanim)
8 pygame.mixer.music.play(loops=-1)
9
10 # Muzigi 5. saniyeden itibaren cal
11 pygame.mixer.music.play(start=5.0)
12
13 # Sonsuz dongu + 2 saniye fade-in ile basla
14 pygame.mixer.music.play(loops=-1, fade_ms=2000)

```

Burada en sık kullanacağın parametre `loops=-1` olacak. Bu, müziğin oyun boyunca sürekli arka planda çalmasını sağlar. `loops` parametresinin değeri, müziğin kaç **ek** kez tekrarlanacağını belirtir: `loops=0` bir kez çalar (varsayılan), `loops=3` toplamda dört kez çalar, `loops=-1` ise sonsuza kadar döner.

● İpucu: Profesyonel İpucu

Oyun müzikleri genellikle döngü halinde tasarlanır — başı ve sonu birbirine sorunsuz bağlanır. Bu tür müziklere *loopable* (döngülenebilir) müzik denir. Ücretsiz döngülenebilir müzik bulmak için **freemusicarchive.org**, **opengameart.org** veya **incompetech.com** sitelerini ziyaret edebilirsin.

Şimdi bu bilgileri kullanarak müzik çalan küçük bir uygulama yazalım. İlk olarak PyGame'i ve sabitlerimi hazırlayalım:

Python – Başlangıç Ayarları

```

1 import pygame
2
3 # Sabitler
4 GENISLIK = 640
5 YUKSEKLIK = 400
6 FPS = 60
7
8 # Renkler
9 SIYAH = (0, 0, 0)
10 BEYAZ = (255, 255, 255)
11 MAVI = (70, 130, 180)
12 YESIL = (50, 200, 100)
13 KIRMIZI = (200, 60, 60)

```

Burası sana tanıdık gelecektir — Bölüm 25'te pencere ve renk sabitlerini aynı şekilde tanımlamıştın. Şimdi asıl işin başladığı yere geçelim: müzik yükleme ve çalma.

Python – Müzik Yükleme ve Başlatma

```

1 def main():
2     """Ana fonksiyon."""
3     pygame.init()
4     pygame.mixer.init()
5
6     ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
7     pygame.display.set_caption("Müzik Yükleme Örneği")
8     saat = pygame.time.Clock()
9     font = pygame.font.SysFont("Arial", 20)
10    baslik_font = pygame.font.SysFont("Arial", 28,
11        ↪ bold=True)
12
13    # Müzik dosyasını yükle (streaming için hazırla)
14    pygame.mixer.music.load("assets/sounds/menu_muzigi.ogg")
15    ↪ g")
16
17    # Müziği sonsuz döngüde çal
18    pygame.mixer.music.play(loops=-1)

```

Burada önce `mixer.init()` ile ses sistemini başlatıyoruz — bunu Bölüm 25'te zaten öğrenmiştin. Ardından `music.load()` dosyayı streaming için hazırlıyor ama henüz çalmıyor. Çalmayı `play(loops=-1)` ile başlatıyoruz; `loops=-1` parametresi müziğin sonsuza kadar döngüde kalmasını sağlıyor.

Python – Oyun Dongusunda Muzik Durumu

```

1  calistir = True
2  while calistir:
3      for olay in pygame.event.get():
4          if olay.type == pygame.QUIT:
5              calistir = False
6          elif olay.type == pygame.KEYDOWN:
7              if olay.key == pygame.K_ESCAPE:
8                  calistir = False
9
10     ekran.fill(SIYAH)
11
12     # Baslik
13     baslik = baslik_font.render(
14         "Muzik Yukleme Ornegi", True, MAVI
15     )
16     ekran.blit(
17         baslik,
18         (GENISLIK // 2 - baslik.get_width() // 2, 30)
19     )
20
21     # Muzik durumunu kontrol et
22     caliyor = pygame.mixer.music.get_busy()
23     durum_renk = YESIL if caliyor else KIRMIZI
24     durum_metin = "Caliyor" if caliyor else "Durdu"
25     durum = font.render(
26         f"Muzik Durumu: {durum_metin}", True, durum_renk
27     )
28     ekran.blit(durum, (50, 100))

```

Oyun döngüsünün içinde `get\_busy()` metodunu kullanarak müziğin o anda çalıp çalmadığını sorguluyoruz. Bu metot `True` veya `False` döndürür. Duruma göre ekrana yeşil “Caliyor” veya kırmızı “Durdu” yazısı basıyoruz — böylece müziğin durumunu görsel olarak takip edebilirsiniz.

Python – Ek Bilgiler ve Ekran Guncelleme

```

1  # Ses seviyesi bilgisi
2  seviye = pygame.mixer.music.get_volume()
3  seviye_metin = font.render(
4      f"Ses Seviyesi: {seviye:.1f}", True, BEYAZ
5  )
6  ekran.blit(seviye_metin, (50, 140))
7
8  # Talimatlar
9  talimat = font.render(
10     "ESC: Cikis", True, (150, 150, 150)
11 )
12 ekran.blit(talimat, (50, YUKSEKLIK - 50))
13

```

```

14 pygame.display.flip()
15 saat.tick(FPS)

```

`get\_volume()` ile müziğin o anki ses seviyesini (0.0 ile 1.0 arasında) alıp ekrana yazdırıyoruz. Ses seviyesini nasıl değiştireceğini bir sonraki konuda öğreneceksin.

Python – Temizlik

```

1     # Muzigi durdur ve PyGame'i kapat
2     pygame.mixer.music.stop()
3     pygame.quit()
4
5 if __name__ == "__main__":
6     main()

```

Program kapanırken `music.stop()` ile müziği durdurmayı unutma. Bölüm 26’nde `Sound` nesneleri için yaptığın gibi, burada da düzgün bir temizlik işlemi önemlidir. Aksi halde ses sistemi arka planda açık kalabilir.

27.2 Müzik Kontrolleri

Bir oyunda müzik sadece çalmaz — oyuncu menüyü açtığında duraklar, kapattığında devam eder; ses ayarlarından seviyesi değişir; oyun bittiğinde durur. Tıpkı bir müzik çaların play, pause, stop düğmeleri gibi, `pygame.mixer.music` modülü de sana tam kontrol sunar.

27.2.1 Duraklatma ve Devam Ettirme

Oyuncunun oyunu duraklatması en yaygın senaryolardan biridir. Menü açıldığında müziğin aniden kesilmesi yerine duraklatılması çok daha profesyonel bir deneyim sağlar:

Python – Duraklatma ve Devam

```

1     # Muzigi duraklat (kaldigin yerden devam edebilirsiniz)
2     pygame.mixer.music.pause()
3
4     # Duraklatilan muzige kaldigin yerden devam et
5     pygame.mixer.music.unpause()

```

`pause()` ile `stop()` arasında önemli bir fark var. `pause()` müziği bulunduğu noktada dondurur ve `unpause()` ile kaldığı yerden devam eder. `stop()` ise müziği tamamen bitirir — tekrar çalmak için `play()` çağırman gerekir ve müzik baştan başlar.

27.2.2 Durdurma ve Yeniden Başlatma

Müziği tamamen durdurmak istediğinde `stop()` fonksiyonunu kullanırsın. Örneğin oyuncu ana menüye döndüğünde oyun müziğini durdurup menü müziğini başlatmak isteyebilirsin:

Python – Durdurma ve Yeniden Başlatma

```
1 # Müziği tamamen durdur
2 pygame.mixer.music.stop()
3
4 # Müziği bastan sarmak istersen (durdurduktan sonra)
5 pygame.mixer.music.rewind()
6
7 # Yeni bir müzik yükle ve çal
8 pygame.mixer.music.load("assets/sounds/menu_muzigi.ogg")
9 pygame.mixer.music.play(loops=-1)
```

`rewind()` fonksiyonu, çalan müziğin başına sarar. Bu, `stop()` ile birlikte kullanıldığında müziği durdurup baştan başlatmak için pratik bir yoldur. Ancak çoğu durumda doğrudan yeni bir `load()` ve `play()` çağrısı yapmak daha temiz bir yaklaşımdır.

27.2.3 Ses Seviyesi Kontrolü

Oyuncunun ses seviyesini ayarlaması neredeyse her oyunda bulunan bir özelliktir. `music` modülü 0.0 (sessiz) ile 1.0 (tam ses) arasında bir değer kabul eder:

Python – Ses Seviyesi

```
1 # Ses seviyesini ayarla (0.0 = sessiz, 1.0 = tam)
2 pygame.mixer.music.set_volume(0.5) # Yarı ses
3
4 # Mevcut ses seviyesini oku
5 seviye = pygame.mixer.music.get_volume()
6 print(f"Ses seviyesi: {seviye}")
```

● Bilgi: Ses Efektleri ve Müzik Sesi Ayrıdır

`music.set_volume()` yalnızca arka plan müziğinin sesini etkiler. Bölüm 26’nde öğrendiğin `Sound.set_volume()` ise ses efektlerinin sesini kontrol eder. Profesyonel oyunlarda genellikle Ana Ses, Müzik ve Efektler olmak üzere üç ayrı ses seviyesi kontrolü bulunur.

27.2.4 Müzik Durumunu Sorgulama

Oyun döngüsü içinde müziğin çalıp çalmadığını kontrol etmek isteyebilirsin. Örneğin müzik bittiğinde yeni bir parça başlatmak veya arayüzde “Çalıyor” / “Durdu” göstermek için:

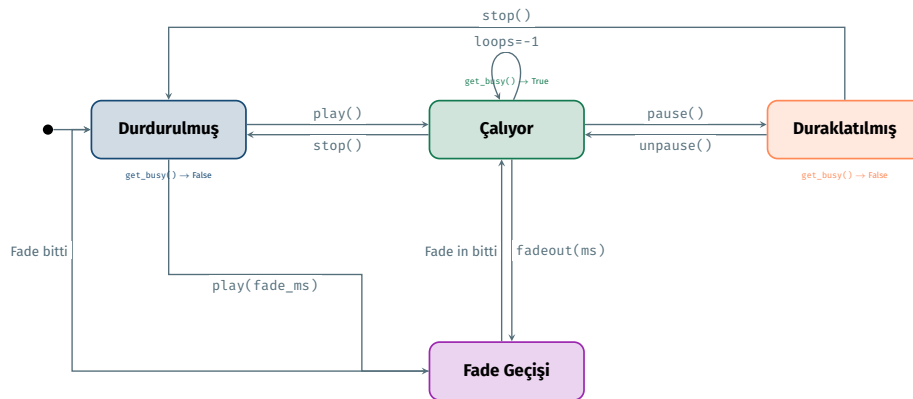
Python – Durum Sorgulama

```

1  # Muzik caliyor mu? (True/False)
2  caliyor = pygame.mixer.music.get_busy()
3
4  if caliyor:
5      print("Muzik caliyor")
6  else:
7      print("Muzik durdu")
8
9  # Muzik bitti mi olayini yakala
10 for olay in pygame.event.get():
11     if olay.type == pygame.USEREVENT:
12         # Muzik bitti, yeni parca baslatabilirsin
13         print("Muzik parcası bitti!")
14
15 # Muzik bittiginde ozel olay gonder
16 pygame.mixer.music.set_endevent(pygame.USEREVENT)

```

`get_busy()` fonksiyonu müzik aktif olarak çalışıyorsa `True`, duraklatılmış veya durdurulmuşsa `False` döner. `set_endevent()` ise müzik parçası doğal olarak bittiğinde belirtilen olay türünü tetikler — bu özellikle müzik kuyruğu ve otomatik parça değişimi için çok işe yarar.



Şekil 27.2: Müzik durumları arasındaki geçişler: play, pause, stop ve fade işlemleri

Şekil 27.2’de müziğin geçebileceği durumları ve bu durumlar arasındaki geçişleri görebilirsin. Müzik başlangıçta “Durdurulmuş” durumundadır. `play()` ile “Çalıyor” durumuna geçer, `pause()` ile “Duraklatılmış” olur ve `unpause()` ile tekrar çalmaya başlar. `stop()` veya `fadeout()` ise müziği tekrar “Durdurulmuş” durumuna döndürür.

Şimdi tüm bu kontrolleri bir arada kullanan interaktif bir müzik çalar oluşturalım. Adım adım ilerleyelim.

27.2.5 Uygulama: İnteraktif Müzik Çalar

Öncelikle programımızın temel yapısını kuralım. Bölüm 25’nde öğrendiğimiz gibi, `pygame.mixer` modülünü başlatmamız gerekiyor:

Python – Temel Yapı ve Sabitler

```

1  import pygame
2  import os
3  import math
4  import struct
5  import wave
6
7  # Sabitler
8  GENISLIK = 700
9  YUKSEKLIK = 480
10  BASLIK = "Muzik Kontrolleri"
11  FPS = 60
12
13  # Renkler
14  SIYAH = (0, 0, 0)
15  BEYAZ = (255, 255, 255)
16  MAVI = (70, 130, 180)
17  YESIL = (50, 200, 100)
18  KIRMIZI = (200, 60, 60)
19  SARI = (255, 200, 50)
20  GRI = (120, 120, 120)

```

Renk sabitlerimizi en başta tanımlıyoruz çünkü hem durum göstergeleri hem de arayüz elemanları için kullanacağız. **YESIL** müziğin çaldığını, **KIRMIZI** ise durduğunu gösterecek.

Gerçek bir oyunda müzik dosyalarını **assets/** klasöründen yüklersin. Ancak bu örnekte test kolaylığı için programatik olarak basit bir melodi oluşturacağız:

Python – Test Muzigi Olusturma

```

1  def test_muzik_olustur():
2      """Test için basit bir muzik dosyasi olusturur."""
3      dosya_yolu = "test_kontrol_muzik.wav"
4      if not os.path.exists(dosya_yolu):
5          sure = 10
6          ornekleme = 22050
7          with wave.open(dosya_yolu, "w") as wav:
8              wav.setnchannels(1)
9              wav.setsampwidth(2)
10             wav.setframerate(ornekleme)
11             for i in range(sure * ornekleme):
12                 frekans = 330 + 110 * math.sin(
13                     2 * math.pi * 0.5 * i / ornekleme
14                 )
15                 deger = int(
16                     12000 * math.sin(
17                         2 * math.pi * frekans * i /
18                             ↵ ornekleme
19                     )
20             )

```



```

20         wav.writeframes(struct.pack("<h", deger))
21     return dosya_yolu

```

Bu fonksiyon, frekansı zamanla değişen 10 saniyelik bir WAV dosyası üretir. Konu 25.2’nde gördüğümüz WAV formatını burada doğrudan `wave` modülü ile oluşturuyoruz. Dosya zaten varsa tekrar oluşturmuyoruz — bu gereksiz disk yazımını önler.

Ekranı ses seviyesini görsel olarak göstermek için bir çubuk çizim fonksiyonu tanımlayalım:

Python – Ses Çubuğu Çizimi

```

1 def ses_cubugu_ciz(ekran, x, y, genislik, yukseklik,
2     deger, renk):
3     """Ses seviyesi cubugu cizer."""
4     # Arka plan (bos cubuk)
5     pygame.draw.rect(ekran, GRI, (x, y, genislik,
6         yukseklik), 2)
7     # Dolu kisim
8     dolu_gen = int(genislik * deger)
9     if dolu_gen > 0:
10         pygame.draw.rect(ekran, renk, (x, y, dolu_gen,
11             yukseklik))

```

`deger` parametresi 0.0 ile 1.0 arasında bir sayıdır — doğrudan `get\volume()` dönüş değerini kullanabiliriz. Çubuğun genişliği bu değerle orantılı olarak dolar.

Şimdi `main()` fonksiyonumuzun başlangıç kısmını yazalım. Müziği yükleyip çalmaya başlıyoruz ve durum değişkenlerini tanımlıyoruz:

Python – Başlangıç ve Müzik Yükleme

```

1 def main():
2     """Ana fonksiyon."""
3     pygame.init()
4     pygame.mixer.init()
5
6     ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
7     pygame.display.set_caption(BASLIK)
8     saat = pygame.time.Clock()
9     font = pygame.font.SysFont("Arial", 18)
10    baslik_font = pygame.font.SysFont("Arial", 26,
11        bold=True)
12    tus_font = pygame.font.SysFont("Arial", 16, bold=True)
13
14    # Müzik yükle ve cal
15    muzik_dosyasi = test_muzik_olustur()
16    pygame.mixer.music.load(muzik_dosyasi)
17    pygame.mixer.music.set_volume(0.5)
18    pygame.mixer.music.play(loops=-1)
19
20    duraklatildi = False

```

```

20 ses_seviyesi = 0.5
21 durum_mesaji = "Muzik caliniyor"

```

`duraklatildi` bayrağı önemli bir detay: `get\_busy()` duraklatılmış müzik için `False` döner, bu yüzden duraklatma durumunu kendimiz takip etmeliyiz. `ses_seviyesi` değişkeni ise sessiz moda geçildiğinde önceki seviyeyi hatırlamak için kullanılır.

Oyun döngüsünün en kritik kısmı klavye olaylarını yakalamak. Her tuşa basıldığında ilgili müzik kontrolünü çağırıyoruz:

Python – Klavye ile Muzik Kontrolü

```

1  calistir = True
2  while calistir:
3      for olay in pygame.event.get():
4          if olay.type == pygame.QUIT:
5              calistir = False
6
7          elif olay.type == pygame.KEYDOWN:
8              if olay.key == pygame.K_ESCAPE:
9                  calistir = False
10
11             # BOSLUK: Duraklat / Devam
12             elif olay.key == pygame.K_SPACE:
13                 if duraklatildi:
14                     pygame.mixer.music.unpause()
15                     duraklatildi = False
16                     durum_mesaji = "Devam ediliyor"
17                 else:
18                     pygame.mixer.music.pause()
19                     duraklatildi = True
20                     durum_mesaji = "Duraklatildi"
21
22             # S: Durdur
23             elif olay.key == pygame.K_s:
24                 pygame.mixer.music.stop()
25                 duraklatildi = False
26                 durum_mesaji = "Durduruldu"
27
28             # R: Yeniden baslat
29             elif olay.key == pygame.K_r:
30                 pygame.mixer.music.play(loops=-1)
31                 duraklatildi = False
32                 durum_mesaji = "Bastan baslatildi"

```

Burada `K_SPACE` tuşu bir “toggle” mantığıyla çalışıyor: müzik çalışırsa `pause()`, duraklatılmışsa `unpause()` çağrılıyor. `K_s` ile `stop()` çağırduğumuzda `duraklatildi` bayrağını sıfırlıyoruz — çünkü müzik artık duraklatılmış değil, tamamen durdurulmuş durumda.

Ses seviyesi kontrolü için yukarı/aşağı ok tuşları ve sessiz mod için M tuşunu kullanıyoruz:

Python – Ses Seviyesi Kontrolleri

```

1  # YUKARI: Sesi artır
2  elif olay.key == pygame.K_UP:
3      ses_seviyesi = min(1.0, ses_seviyesi + 0.1)
4      pygame.mixer.music.set_volume(ses_seviyesi)
5      durum_mesaji = f"Ses: {ses_seviyesi:.1f}"
6
7  # ASAGI: Sesi azalt
8  elif olay.key == pygame.K_DOWN:
9      ses_seviyesi = max(0.0, ses_seviyesi - 0.1)
10     pygame.mixer.music.set_volume(ses_seviyesi)
11     durum_mesaji = f"Ses: {ses_seviyesi:.1f}"
12
13  # M: Sessize al / Sesi ac
14  elif olay.key == pygame.K_m:
15      if pygame.mixer.music.get_volume() > 0:
16          pygame.mixer.music.set_volume(0.0)
17          durum_mesaji = "Sessiz"
18      else:
19          pygame.mixer.music.set_volume(ses_seviyesi)
20          durum_mesaji = "Ses acildi"

```

`min()` ve `max()` fonksiyonları ses seviyesinin 0.0–1.0 aralığı dışına çıkmasını önler. M tuşu ile sessiz moda geçildiğinde ses 0.0 yapılır ama `ses_seviyesi` değişkeni önceki değeri korur — böylece tekrar M'ye basıldığında eski seviyeye dönlür.

Son olarak ekrana durum bilgilerini ve kontrol tuşlarını çizelim:

Python – Arayüz Çizimi ve Temizlik

```

1  # --- CIZIM ---
2  ekran.fill(SIYAH)
3
4  # Baslik
5  baslik = baslik_font.render("Muzik Kontrolleri", True,
6  ↪ MAVI)
7  ekran.blit(baslik, (GENISLIK // 2 - baslik.get_width()
8  ↪ // 2, 20))
9
10 # Durum bilgileri
11 y_pos = 80
12 caliyor = pygame.mixer.music.get_busy()
13 durum_renk = YESIL if caliyor else KIRMIZI
14 durum_text = "Caliyor" if caliyor and not duraklatildi
15 ↪ else (
16     "Duraklatildi" if duraklatildi else "Durdu"
17 )
18
19 ekran.blit(

```

```

17         font.render(f"Durum: {durum_text}", True,
18             ↪ durum_renk),
19         (50, y_pos),
20     )
21     y_pos += 30
22     # Ses seviyesi cubugu
23     ekran.blit(font.render("Ses Seviyesi:", True, BEYAZ),
24         ↪ (50, y_pos))
25     mevcut_ses = pygame.mixer.music.get_volume()
26     ses_cubugu_ciz(ekran, 200, y_pos + 2, 200, 20,
27         ↪ mevcut_ses, YESIL)
28     ekran.blit(
29         font.render(f"{mevcut_ses:.1f}", True, BEYAZ),
30         ↪ (410, y_pos)
31     )
32     pygame.display.flip()
33     saat.tick(FPS)
34     # Temizlik
35     pygame.mixer.music.stop()
36     pygame.quit()

```

Çizim kısmında `get\_busy()` ve `duraklatildi` bayrağını birlikte kullanarak doğru durum metnini belirliyoruz. `ses\_cubugu\_ciz()` fonksiyonu ile ses seviyesi görsel olarak gösteriliyor. Program kapanırken `stop()` ile müziği durduruyoruz ve `pygame.quit()` ile kaynakları serbest bırakıyoruz.

Bu programı çalıştırdığında klavye tuşlarıyla müziği kontrol edebilirsiniz: boşluk tuşu duraklatma/devam, yukarı/aşağı ok tuşları ses seviyesi, S tuşu durdurma, R tuşu yeniden başlatma ve M tuşu sessiz mod işlemlerini gerçekleştirir.

27.3 Fade Efektleri ve Müzik Kuyruğu

Bir korku oyununda karanlık bir koridora girdiğini düşün. Huzurlu müzik aniden kesilip gerilim müziği patlasa, oyuncu sıçrar ama deneyim yapay hissettirir. Bunun yerine huzurlu müzik yavaşça kısılır, kısa bir sessizlik olur ve gerilim müziği yavaşça yükselir — işte bu, profesyonel bir müzik geçişidir. PyGame’de bu etkiyi *fade* (solma) efektleriyle sağlarız.

27.3.1 Fade Out: Müziği Yavaşça Kısma

`fadeout()` fonksiyonu, çalan müziğin sesini belirtilen süre boyunca kademeli olarak azaltır ve süre sonunda tamamen durdurur:

Python – Fade Out

```

1 # Müziği 3 saniyede yavaşça kapat (milisaniye cinsinden)
2 pygame.mixer.music.fadeout(3000)

```

```

3
4 # Muzigi 500 milisaniyede (yarım saniye) kapat
5 pygame.mixer.music.fadeout(500)

```

Parametre milisaniye cinsindendir: 3000 = 3 saniye. Fade out sırasında müzik hala çalışıyor sayılır (`get_busy()` hala `True` döner), ancak ses seviyesi kademeli olarak sıfıra iner. Fade tamamlandığında müzik durur ve `set_endevent()` ile belirlediğin olay tetiklenir.

27.3.2 Fade In: Müziği Yavaşça Açma

Müziği kademeli olarak başlatmak için `play()` fonksiyonunun `fade_ms` parametresini kullanırız:

Python – Fade In

```

1 # Muzigi 2 saniye fade-in ile baslat (sonsuz dongu)
2 pygame.mixer.music.play(loops=-1, fade_ms=2000)
3
4 # Yeni muzik yukle ve fade-in ile cal
5 pygame.mixer.music.load("assets/sounds/gerilim.ogg")
6 pygame.mixer.music.play(loops=-1, fade_ms=3000)

```

`fade_ms=2000` ile müzik sessizden başlar ve 2 saniye içinde `set_volume()` ile ayarlanan hedef ses seviyesine ulaşır. Bu, özellikle sahne geçişlerinde veya oyun başladığında çok hoş bir etki yaratır.

27.3.3 Müzik Kuyruğu

Bir müzik parçası bittikten sonra otomatik olarak başka bir parçanın çalmasını isteyebilirsin. `queue()` fonksiyonu tam da bunu yapar:

Python – Müzik Kuyruğu

```

1 # İlk muzigi yukle ve cal
2 pygame.mixer.music.load("assets/sounds/giris_muzigi.ogg")
3 pygame.mixer.music.play()
4
5 # Siradaki muzigi kuyruğa ekle
6 pygame.mixer.music.queue("assets/sounds/ana_tema.ogg")

```

`queue()` ile eklenen müzik, mevcut müzik parçası bittiğinde otomatik olarak başlar. Dikkat: bu yalnızca müzik doğal olarak sona erdiğinde çalışır. `stop()` veya `fadeout()` ile durdurursan kuyruktaki müzik çalınmaz.

⚠ Dikkat: Kuyruk Sınırı

`queue()` fonksiyonu aynı anda yalnızca **bir** müzik parçasını kuyruğa alabilir. İkinci bir `queue()` çağrısı, öncekinin üzerine yazar. Birden fazla parçayı sırayla çalmak istiyorsan, bu işlemi kendi kodunla yönetmen gerekir — ileride `MusicManager` sınıfı ile bunu yapacağız.

27.3.4 Akıcı Müzik Geçişi

Bir sahneden diğerine geçerken en profesyonel yaklaşım, fade out ve fade in'i bir arada kullanmaktır. Ancak burada bir zamanlama sorunu var: fade out bitmeden yeni müziği yüklersen, eski müzik kesilir. Bu sorunu çözmek için `set_endevent()` ile müziğin bitmesini bekleyebilirsin:

Python – Akıcı Geçiş Stratejisi

```
1 import pygame
2
3 # Müzik bittiğinde özel olay gönderilsin
4 MUZIK_BITTI = pygame.USEREVENT + 1
5 pygame.mixer.music.set_endevent(MUZIK_BITTI)
6
7 # Gecis baslat: mevcut muzigi yavaşça kapat
8 pygame.mixer.music.fadeout(2000)
9
10 # Oyun dongusunda olaylari dinle
11 for olay in pygame.event.get():
12     if olay.type == MUZIK_BITTI:
13         # Eski muzik tamamen bitti, yenisini baslat
14         pygame.mixer.music.load("assets/sounds/yeni_muzik_
15                               ↪ .ogg")
16         pygame.mixer.music.play(loops=-1, fade_ms=2000)
```

Bu yaklaşımda önce mevcut müziği `fadeout()` ile kısarız. Müzik tamamen bittiğinde `MUZIK_BITTI` olayı tetiklenir. Bu olayı yakaladığımızda yeni müziği yükler ve fade in ile başlatırız. Sonuç: eski müzik yavaşça söner, yeni müzik yavaşça yükselir — tıpkı profesyonel bir oyundaki gibi.

Şimdi tüm bu kavramları bir araya getiren interaktif bir program yazalım. Program farklı tuşlarla fade efektlerini ve müzik kuyruğunu denememizi sağlayacak.

27.3.5 Programın Temel Yapısı

Öncelikle gerekli modülleri içe aktarıyoruz ve sabitlerle renkleri tanımlıyoruz. Test amacıyla kendi müzik dosyalarımızı oluşturacağımız için `wave`, `struct` ve `math` modüllerine de ihtiyacımız var:

Python – Sabitler ve Renkler

```

1  import pygame
2  import os
3  import math
4  import struct
5  import wave
6
7  # Sabitler
8  GENISLIK = 700
9  YUKSEKLIK = 500
10 BASLIK = "Fade Efektleri ve Muzik Kuyrugu"
11 FPS = 60
12
13 # Özel olay: muzik bitti
14 MUZIK_BITTI = pygame.USEREVENT + 1
15
16 # Renkler
17 SIYAH = (0, 0, 0)
18 BEYAZ = (255, 255, 255)
19 MAVI = (70, 130, 180)
20 YESIL = (50, 200, 100)
21 KIRMIZI = (200, 60, 60)
22 SARI = (255, 200, 50)
23 MOR = (150, 100, 200)
24 GRI = (120, 120, 120)

```

MUZIK_BITTI sabitini `pygame.USEREVENT + 1` olarak tanımlıyoruz. Bu özel olayı daha sonra `set\_endevent()` ile kullanarak müzik bittiğinde haberdar olacağız.

27.3.6 Test Müzik Dosyaları Oluşturma

Harici müzik dosyalarına ihtiyaç duymadan programı test edebilmek için farklı frekanslarda WAV dosyaları oluşturan bir yardımcı fonksiyon yazıyoruz:

Python – Test Müziği Oluşturma

```

1  def test_muzik_olustur(dosya_adi, frekans=440, sure=8):
2      """Farkli frekanslarda test muzik dosyalari
3      ↪ olusturur."""
4      if not os.path.exists(dosya_adi):
5          ornekleme = 22050
6          with wave.open(dosya_adi, "w") as wav:
7              wav.setnchannels(1)
8              wav.setsampwidth(2)
9              wav.setframerate(ornekleme)
10             for i in range(sure * ornekleme):
11                 # Hafif frekans modulasyonu ile daha
12                 ↪ ilginç ses
13                 mod_frekans = frekans + 50 * math.sin(

```

```

12         2 * math.pi * 0.3 * i / ornekleme
13     )
14     deger = int(
15         10000 * math.sin(
16             2 * math.pi * mod_frekans * i /
17             ↪ ornekleme
18         )
19     )
20     wav.writeframes(struct.pack("<h", deger))
    return dosya_adi

```

Bu fonksiyon, dosya zaten yoksa belirtilen frekansta bir sinüs dalgası üretir. Hafif frekans modülasyonu sayesinde monoton bir ses yerine daha ilginç bir ton elde ederiz. Her müzik parçası farklı bir frekansta olacağından hangisinin çaldığını kulağınla da ayırt edebilirsiniz.

27.3.7 Başlangıç Ayarları

`main()` fonksiyonunda PyGame'i başlatıyor, üç farklı test müziği oluşturuyor ve ilk müziği çalmaya başlıyoruz:

Python – Başlangıç Ayarları

```

1 def main():
2     """Ana fonksiyon."""
3     pygame.init()
4     pygame.mixer.init()
5
6     ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
7     pygame.display.set_caption(BASLIK)
8     saat = pygame.time.Clock()
9     font = pygame.font.SysFont("Arial", 18)
10    baslik_font = pygame.font.SysFont("Arial", 26,
11    ↪ bold=True)
12    tus_font = pygame.font.SysFont("Arial", 15, bold=True)
13
14    # Farkli muzik dosyalari olustur
15    muzik_a = test_muzik_olustur("test_muzik_a.wav",
16    ↪ frekans=330, sure=10)
17    muzik_b = test_muzik_olustur("test_muzik_b.wav",
18    ↪ frekans=440, sure=10)
19    muzik_c = test_muzik_olustur("test_muzik_c.wav",
20    ↪ frekans=520, sure=10)
21
22    # Muzik bitti olayini ayarla
23    pygame.mixer.music.set_endevent(MUZIK_BITTI)
24
25    # Ilk muzigi yukle ve cal
26    pygame.mixer.music.load(muzik_a)
27    pygame.mixer.music.set_volume(0.5)

```



```
24 | pygame.mixer.music.play(loops=-1)
```

Üç farklı frekansta (330, 440 ve 520 Hz) test müziği oluşturuyoruz. `set\__endevent()` ile müzik bittiğinde `MUZIK_BITTI` olayının tetiklenmesini sağlıyoruz. İlk müziği yüklüyor, ses seviyesini 0.5'e ayarlıyor ve sonsuz döngüde çalmaya başlıyoruz.

27.3.8 Durum Değişkenleri ve Log Sistemi

Programın ekranda durumu gösterebilmesi için birkaç değişken ve basit bir log fonksiyonu tanımlıyoruz:

Python – Durum Takibi

```
1 | mevcut_muzik = "Muzik A (330 Hz)"
2 | durum_mesaji = "Muzik A caliniyor"
3 | fade_durumu = ""
4 | log_mesajlari = []
5 | beklenen_muzik = None # Fade sonrasi yuklenecek muzik
6 |
7 | def log_ekle(mesaj):
8 |     """Log mesaj ekler (en fazla 6 satir tutar)."""
9 |     log_mesajlari.append(mesaj)
10 |    if len(log_mesajlari) > 6:
11 |        log_mesajlari.pop(0)
12 |
13 | log_ekle("[OK] Muzik A baslatildi")
```

`beklenen_muzik` değişkeni, akıcı geçiş sırasında fade out bittikten sonra hangi müziğin başlatılacağını tutar. `log_ekle()` fonksiyonu ise ekrandaki log alanında en fazla 6 mesaj gösterir; yeni mesaj geldiğinde en eskisini siler.

27.3.9 Olay Döngüsü: Müzik Bitti Olayı

Oyun döngüsünde ilk olarak pencere kapatma ve müzik bitti olaylarını işliyoruz. Fade sonrası beklenen müzik varsa burada başlatılır:

Python – Müzik Bitti Olayı

```
1 | calistir = True
2 | while calistir:
3 |     for olay in pygame.event.get():
4 |         if olay.type == pygame.QUIT:
5 |             calistir = False
6 |
7 |         elif olay.type == MUZIK_BITTI:
8 |             log_ekle("[!] Muzik parcasi bitti")
9 |             fade_durumu = ""
10 |            # Fade sonrasi beklenen muzik varsa baslat
11 |            if beklenen_muzik is not None:
12 |                dosya, isim = beklenen_muzik
```

```

13         pygame.mixer.music.load(dosya)
14         pygame.mixer.music.play(loops=-1,
15             ↪ fade_ms=2000)
16         mevcut_muzik = isim
16         durum_mesaji = f"{isim} fade-in ile
17             ↪ baslatildi"
17         log_ekle(f"[OK] {isim} fade-in ile
18             ↪ baslatildi (2s)")
18         beklenen_muzik = None

```

MUZIK_BITTI olayı yakalandığında, eğer `beklenen_muzik` değişkeni doluysa yeni müzik `fade_ms=2000` ile yüklenir ve fade in efektiyle başlatılır. Bu, akıcı geçişin ikinci yarısıdır.

27.3.10 Tuş Kontrolleri

Kullanıcının farklı fade efektlerini denemesi için tuş kontrolleri ekliyoruz:

Python – Fade ve Kuyruk Kontrolleri

```

1 elif olay.type == pygame.KEYDOWN:
2     if olay.key == pygame.K_ESCAPE:
3         calistir = False
4
5     # F: Fade out (3 saniye)
6     elif olay.key == pygame.K_f:
7         pygame.mixer.music.fadeout(3000)
8         fade_durumu = "Fade out (3s)..."
9         durum_mesaji = "Fade out baslatildi"
10        log_ekle("[->] Fade out baslatildi (3s)")
11
12    # G: Yeni muzik fade-in ile baslat
13    elif olay.key == pygame.K_g:
14        pygame.mixer.music.load(muzik_b)
15        pygame.mixer.music.play(loops=-1, fade_ms=3000)
16        mevcut_muzik = "Muzik B (440 Hz)"
17        fade_durumu = "Fade in (3s)..."
18        durum_mesaji = "Muzik B fade-in ile basladi"
19        log_ekle("[OK] Muzik B fade-in baslatildi (3s)")
20
21    # Q: Kuyruğa muzik ekle
22    elif olay.key == pygame.K_q:
23        # Once donguyu kapat ki muzik bitebilsin
24        pygame.mixer.music.play(loops=0)
25        pygame.mixer.music.queue(muzik_c)
26        durum_mesaji = "Muzik C kuyruğa eklendi"
27        log_ekle("[+] Muzik C kuyruğa eklendi")
28
29    # T: Gecis efekti (fade out + fade in)
30    elif olay.key == pygame.K_t:
31        beklenen_muzik = (muzik_a, "Muzik A (330 Hz)")

```

```

32     pygame.mixer.music.fadeout(2000)
33     fade_durumu = "Gecis: fade out..."
34     durum_mesaji = "Akici gecis baslatildi"
35     log_ekle("[->] Akici gecis: fade out (2s)")

```

F tuşu `fadeout(3000)` ile müziği 3 saniyede kısar. G tuşu yeni müziği `fade_ms=3000` parametresiyle başlatır. Q tuşu önce döngüyü kapatıp (`loops=0`) ardından `queue()` ile sıradaki müziği kuyruğa ekler. T tuşu ise akıcı geçişi başlatır: `beklenen_muzik` değişkenine yeni müziği kaydeder ve mevcut müziği `fade out` ile kapatır. Müzik bittiğinde `MUZIK_BITTI` olayı tetiklenir ve yeni müzik otomatik olarak `fade in` ile başlar.

Python – Doğrudan Müzik Değiştirme

```

1  # 1, 2, 3: Dogrudan muzik degistir
2  elif olay.key == pygame.K_1:
3      pygame.mixer.music.load(muzik_a)
4      pygame.mixer.music.play(loops=-1)
5      mevcut_muzik = "Muzik A (330 Hz)"
6      log_ekle("[OK] Muzik A baslatildi")
7
8  elif olay.key == pygame.K_2:
9      pygame.mixer.music.load(muzik_b)
10     pygame.mixer.music.play(loops=-1)
11     mevcut_muzik = "Muzik B (440 Hz)"
12     log_ekle("[OK] Muzik B baslatildi")
13
14  elif olay.key == pygame.K_3:
15     pygame.mixer.music.load(muzik_c)
16     pygame.mixer.music.play(loops=-1)
17     mevcut_muzik = "Muzik C (520 Hz)"
18     log_ekle("[OK] Muzik C baslatildi")

```

1, 2 ve 3 tuşları ile fade efekti olmadan doğrudan müzik değiştirebilirsin. Bu, fade efektli geçişlerle karşılaştırma yapmak için kullanışlıdır — aradaki farkı hem duyarak hem de ekrandaki log mesajlarından görerek anlayabilirsin.

27.3.11 Ekran Çizimi ve Temizlik

Son olarak ekrana durum bilgilerini, kontrol tuşlarını ve log mesajlarını çiziyoruz:

Python – Ekran Çizimi

```

1  # --- CIZIM ---
2  ekran.fill(SIYAH)
3
4  # Baslik
5  baslik = baslik_font.render(
6      "Fade Efektleri ve Muzik Kuyrugu", True, MAVI
7  )

```

```

8  ekran.blit(baslik, (GENISLIK // 2 - baslik.get_width() //
   ↪ 2, 15))
9
10 y_pos = 60
11
12 # Durum bilgileri
13 caliyor = pygame.mixer.music.get_busy()
14 durum_renk = YESIL if caliyor else KIRMIZI
15 ekran.blit(font.render(
16     f"Durum: {'Caliyor' if caliyor else 'Durdu'}",
17     True, durum_renk), (50, y_pos))
18 y_pos += 25
19 ekran.blit(font.render(
20     f"Mevcut: {mevcut_muzik}", True, BEYAZ), (50, y_pos))
21 y_pos += 25
22
23 ses = pygame.mixer.music.get_volume()
24 ekran.blit(font.render(
25     f"Ses Seviyesi: {ses:.2f}", True, BEYAZ), (50, y_pos))
26 y_pos += 25
27
28 if fade_durumu:
29     ekran.blit(font.render(
30         f"Fade: {fade_durumu}", True, MOR), (50, y_pos))
31 y_pos += 35

```

`get\_busy()` ile müziğin çalıp çalmadığını, `get\_volume()` ile anlık ses seviyesini sorguluyoruz. Fade durumu aktifse mor renkle ekranda gösteriliyor.

Python – Kontrol Listesi ve Log

```

1  # Kontroller
2  ekran.blit(baslik_font.render("Kontroller:", True, BEYAZ),
   ↪  (50, y_pos))
3  y_pos += 30
4
5  kontroller = [
6      ("F", "Fade out (3s)"),
7      ("G", "Muzik B fade-in ile baslat"),
8      ("Q", "Muzik C kuyruğa ekle"),
9      ("T", "Akici gecis (fade out -> fade in)"),
10     ("1/2/3", "Dogrudan muzik degistir (A/B/C)"),
11     ("ESC", "Cikis"),
12 ]
13
14 for tus, aciklama in kontroller:
15     tus_metin = tus_font.render(tus, True, SIYAH)
16     tus_gen = tus_metin.get_width() + 14
17     pygame.draw.rect(
18         ekran, BEYAZ, (60, y_pos, tus_gen, 22),
        ↪  border_radius=3

```

```

19     )
20     ekran.blit(tus_metin, (67, y_pos + 2))
21     ekran.blit(font.render(
22         aciklama, True, GRI), (60 + tus_gen + 12, y_pos +
23         ↪ 1))
24     y_pos += 27
25     # Log mesajlari
26     y_pos += 15
27     ekran.blit(baslik_font.render("Log:", True, SARI), (50,
28         ↪ y_pos))
29     y_pos += 28
30     for mesaj in log_mesajlari:
31         renk = YESIL if "[OK]" in mesaj else (
32             ↪ SARI if "[->]" in mesaj else GRI
33         )
34         ekran.blit(font.render(mesaj, True, renk), (60,
35             ↪ y_pos))
36         y_pos += 22
37     pygame.display.flip()
38     saat.tick(FPS)

```

Kontrol tuşları beyaz arka planlı küçük kutucuklar halinde çizilir. Log mesajları ise içeriğine göre renklendirilir: [OK] yeşil, [->] sarı, diğerleri gri renkte görünür.

Python – Temizlik

```

1     # Temizlik
2     pygame.mixer.music.stop()
3     pygame.quit()
4     for dosya in ["test_muzik_a.wav", "test_muzik_b.wav",
5         ↪ "test_muzik_c.wav"]:
6         if os.path.exists(dosya):
7             os.remove(dosya)
8
9     if __name__ == "__main__":
10         main()

```

Program kapanırken müziği durduruyoruz, PyGame'i kapatıyoruz ve oluşturduğumuz geçici test müzik dosyalarını siliyoruz. Böylece arkada gereksiz dosya bırakmamış oluruz.

27.4 Sahne Bazlı Müzik Yönetimi

Uzay Kaçışı oyunumuzu düşün: ana menüde sakın bir uzay melodisi, oyun sırasında tempolu bir aksiyon müziği, boss savaşında epik bir savaş teması ve oyun bittiğinde hüznü bir kapanış. Her sahnenin kendi müziği var ve bu müzikler

arasında akıcı geçişler yapılması gerekiyor. Tüm bu işlemleri oyun döngüsünün içine dağınık `load()` ve `play()` çağrılıyla yönetmek hem karmaşık hem de hata yapmaya açıktır.

Peki bunu nasıl düzenli bir şekilde yönetiriz? Cevap: bir `MusicManager` sınıfı tasarlayarak.

27.4.1 Neden Bir Yönetici Sınıfa İhtiyacımız Var?

Küçük bir oyunda müziği birkaç satırla kontrol edebilirsiniz. Ancak oyun büyüdükçe şu sorunlarla karşılaşsınız: aynı müzik zaten çalışırken tekrar yüklenmesi, fade geçişleri sırasında zamanlama hataları, ses seviyesinin farklı yerlerde tutarsız ayarlanması ve müzik dosya yollarının kodun her yerine dağılması.

Bir `MusicManager` sınıfı bu sorunların hepsini tek bir noktada çözer. Tüm müzik mantığı tek bir yerde toplanır, sahne geçişleri otomatik yönetilir ve ses ayarları tutarlı kalır.

27.4.2 MusicManager Sınıfı

Aşağıdaki tasarımda her sahne bir müzik dosyasıyla eşleştirilir ve geçişler otomatik olarak fade efektleriyle yapılır:

Python – MusicManager Sınıfı

```

1 class MusicManager:
2     """Sahne bazlı muzik yönetici sınıfı."""
3
4     def __init__(self):
5         """Yöneticiyi başlat."""
6         self.muzikler = {}
7         self.mevcut_sahne = None
8         self.ses_seviyesi = 0.5
9         pygame.mixer.music.set_volume(self.ses_seviyesi)
10
11     def muzik_ekle(self, sahne_adi, dosya_yolu):
12         """Bir sahneye muzik dosyasi ata."""
13         self.muzikler[sahne_adi] = dosya_yolu
14
15     def sahne_degistir(self, yeni_sahne,
16         ↪ fade_suresi=1000):
17         """Sahneyi degistir ve muzigi gecis yap."""
18         if yeni_sahne == self.mevcut_sahne:
19             return # Ayni sahne, degisiklik yok
20
21         if yeni_sahne not in self.muzikler:
22             return # Tanimlanmamis sahne
23
24         if self.mevcut_sahne is not None:
25             pygame.mixer.music.fadeout(fade_suresi)
26
27         self.mevcut_sahne = yeni_sahne

```

```

27
28     def guncelle(self, olay_listesi):
29         """Oyun dongusunda cagrilacak guncelleme."""
30         for olay in olay_listesi:
31             if olay.type == MUZIK_BITTI:
32                 if self.mevcut_sahne in self.muzikler:
33                     dosya =
34                         ↪ self.muzikler[self.mevcut_sahne]
35                     pygame.mixer.music.load(dosya)
36                     pygame.mixer.music.play(
37                         loops=-1, fade_ms=1000
38                 )

```

Bu sınıfın temel fikri basit: `muzik_ekle()` ile her sahneye bir müzik dosyası atarsın, `sahne_degistir()` ile geçiş yaparsın ve `guncelle()` fonksiyonunu her frame'de çağırarak fade geçişlerinin düzgün tamamlanmasını sağlarsın.

💡 İpucu: Profesyonel İpucu

Gerçek oyunlarda müzik yöneticisi genellikle “singleton” (tekil) deseniyle tasarlanır — yani oyunun her yerinden erişilebilen tek bir `MusicManager` örneği olur. Böylece hangi sahnede olursan ol, `muzik_yonetici.sahne_degistir("boss")` çağrısıyla müziği değiştirebilirsin.

27.4.3 Kullanım Örneği

`MusicManager` sınıfını kullanmak oldukça basit:

Python – MusicManager Kullanımı

```

1  # Yoneticiyi olustur
2  yonetici = MusicManager()
3
4  # Sahnelere muzik ata
5  yonetici.muzik_ekle("menu", "assets/sounds/menu.ogg")
6  yonetici.muzik_ekle("oyun", "assets/sounds/aksiyon.ogg")
7  yonetici.muzik_ekle("boss", "assets/sounds/boss.ogg")
8  yonetici.muzik_ekle("bitis",
9      ↪ "assets/sounds/game_over.ogg")
10
11 # Menu muzigini baslat
12 yonetici.sahne_degistir("menu")
13
14 # Oyun basladiginda
15 yonetici.sahne_degistir("oyun", fade_suresi=2000)
16
17 # Boss savasinda
18 yonetici.sahne_degistir("boss", fade_suresi=1500)

```

Dikkat edersen her yerde sadece sahne adıyla çalışıyoruz. Dosya yolları, fade süreleri ve durum kontrolleri **MusicManager** tarafından yönetiliyor. Bu yaklaşım kodunu hem okunabilir hem de bakımı kolay hale getirir.

27.4.4 Uzay Kaçışı Entegrasyonu

Uzay Kaçışı projemize ses entegrasyonu yapıyoruz. Önceki bölümlerde oyuncu gemisi, düşmanlar ve mermiler eklemiştik. Şimdi **MusicManager** sınıfının gelişmiş versiyonunu adım adım inceleyelim.

Sabitler ve Hazırlık

Önce programın ihtiyaç duyduğu sabitleri ve özel olay tanımını yapalım:

Python – Sabitler ve Özel Olay Tanımı

```

1  import pygame
2  import os
3  import math
4  import struct
5  import wave
6
7  # Sabitler
8  GENISLIK = 750
9  YUKSEKLIK = 550
10 BASLIK = "MusicManager - Sahne Bazlı Muzik Yonetimi"
11 FPS = 60
12
13 # Özel olaylar
14 MUZIK_BITTI = pygame.USEREVENT + 1
15
16 # Sahne renkleri
17 SAHNE_RENKLERI = {
18     "menu": (70, 130, 180),
19     "oyun": (50, 200, 100),
20     "boss": (200, 60, 60),
21     "bitis": (150, 100, 200),
22 }
```

Burada **MUZIK_BITTI** adında özel bir olay tanımlıyoruz. `pygame.USEREVENT` sabitine 1 ekleyerek kendi olay kodumuzu oluşturuyoruz. Bu olay, bir müzik parçası sona erdiğinde tetiklenecek ve fade geçişlerinin düzgün çalışmasını sağlayacak. **SAHNE_RENKLERI** sözlüğü ise her sahneyi görsel olarak ayırt etmek için kullanılacak.

Test Müzik Dosyası Oluşturma

Gerçek bir projede müzik dosyaların hazır olur, ancak bu örneği çalıştırabilmek için programatik olarak test müzikleri üretiyoruz:

Python – Test Muzik Oluşturucu

```

1 def test_muzik_olustur(dosya_adi, frekans=440, sure=15):
2     """Farkli frekanslarda test muzik dosyalari
3     ↪ olusturur."""
4     if not os.path.exists(dosya_adi):
5         ornekleme = 22050
6         with wave.open(dosya_adi, "w") as wav:
7             wav.setnchannels(1)
8             wav.setsampwidth(2)
9             wav.setframerate(ornekleme)
10            for i in range(sure * ornekleme):
11                t = i / ornekleme
12                mod = 30 * math.sin(2 * math.pi * 0.2 * t)
13                deger = int(
14                    10000 * math.sin(
15                        2 * math.pi * (frekans + mod) * t
16                    )
17                )
18                wav.writeframes(struct.pack("<h", deger))
19            return dosya_adi

```

Bu fonksiyon `wave` modülü ile WAV dosyaları oluşturur. Her sahne için farklı bir frekans kullanarak ayırt edilebilir sesler üretir. Frekans modülasyonu sayesinde düz bir ton yerine hafif dalgalanan, daha müzikal bir ses elde ederiz.

Gelişmiş MusicManager Sınıfı

Daha önceki basit tasarımı genişletelim. Bu sefer `_muzik_baslat()` yardımcı metodu, `hedef_sahne` takibi ve daha esnek parametreler ekliyoruz:

Python – Gelismis MusicManager – Kurucu

```

1 class MusicManager:
2     """Sahne bazli muzik yoneticisi sinifi."""
3
4     def __init__(self, varsayilan_ses=0.5,
5     ↪ varsayilan_fade=1500):
6         """Muzik yoneticisini baslat."""
7         self.muzikler = {}
8         self.mevcut_sahne = None
9         self.hedef_sahne = None
10        self.ses_seviyesi = varsayilan_ses
11        self.varsayilan_fade = varsayilan_fade
12        self.gecis_aktif = False
13
14        pygame.mixer.music.set_volume(self.ses_seviyesi)
15        pygame.mixer.music.set_endevent(MUZYIK_BITTI)

```

Kurucuda iki yeni alan dikkat çekiyor: `hedef_sahne` ve `gecis_aktif`. Fade-out sırasında mevcut müzik hala çalışıyor olduğundan, geçilecek sahneyi ayrı bir değişkende tutuyoruz. `set_endevent()` çağrısı ise müzik bittiğinde `MUZYIK_BITTI`

olayının tetiklenmesini sağlar.

Python – Sahne Degistirme ve Muzik Baslatma

```

1  def sahne_degistir(self, yeni_sahne, fade_suresi=None):
2      """Sahneyi degistir ve muzigi akici gecis yap."""
3      if yeni_sahne == self.mevcut_sahne and not
4          ↪ self.gecis_aktif:
5          return
6      if yeni_sahne not in self.muzikler:
7          return
8
9      if fade_suresi is None:
10         fade_suresi = self.varsayilan_fade
11
12     self.hedef_sahne = yeni_sahne
13
14     if pygame.mixer.music.get_busy():
15         pygame.mixer.music.fadeout(fade_suresi)
16         self.gecis_aktif = True
17     else:
18         self._muzik_baslat(yeni_sahne, fade_suresi)
19
20 def _muzik_baslat(self, sahne_adi, fade_suresi):
21     """Belirtilen sahnenin muzigini fade-in ile
22     ↪ baslatir."""
23     dosya = self.muzikler[sahne_adi]
24     pygame.mixer.music.load(dosya)
25     pygame.mixer.music.set_volume(self.ses_seviyesi)
26     pygame.mixer.music.play(loops=-1, fade_ms=fade_suresi)
27     self.mevcut_sahne = sahne_adi
28     self.hedef_sahne = None
29     self.gecis_aktif = False

```

`sahne\_degistir()` metodunda iki yollu bir mantık var: müzik halihazırda çalışıyorsa önce `fadeout()` ile mevcut müziği söndürüyor ve `gecis_aktif` bayrağını kaldırıyoruz. Müzik çalmıyorsa doğrudan `\_muzik\_baslat()` ile yeni sahneyi başlatıyoruz. `\_muzik\_baslat()` ise dosyayı yükler, ses seviyesini ayarlar ve sonsuz döngüde (`loops=-1`) fade-in ile çalmaya başlar.

Python – Guncelleme ve Ses Kontrolleri

```

1  def guncelle(self, olay_listesi):
2      """Oyun dongusunda her frame cagrilmali."""
3      for olay in olay_listesi:
4          if olay.type == MUZIK_BITTI:
5              if self.gecis_aktif and self.hedef_sahne:
6                  self._muzik_baslat(
7                      self.hedef_sahne, self.varsayilan_fade
8                  )
9

```

```

10 def ses_ayarla(self, seviye):
11     """Ses seviyesini ayarla (0.0-1.0)."""
12     self.ses_seviyesi = max(0.0, min(1.0, seviye))
13     pygame.mixer.music.set_volume(self.ses_seviyesi)
14
15 def ses_artir(self, miktar=0.1):
16     """Ses seviyesini artir."""
17     self.ses_ayarla(self.ses_seviyesi + miktar)
18
19 def ses_azalt(self, miktar=0.1):
20     """Ses seviyesini azalt."""
21     self.ses_ayarla(self.ses_seviyesi - miktar)

```

`guncelle()` metodu oyun döngüsünde her frame çağrılır. `MUZIK_BITTI` olayını yakaladığında, eğer bir geçiş aktifse hedef sahnenin müziğini başlatır. Böylece fade-out biter bitmez yeni müzik fade-in ile devreye girer. `ses\_ayarla()` ise `max()` ve `min()` ile değeri 0.0–1.0 aralığında tutar; `ses\_artir()` ve `ses\_azalt()` bu fonksiyonu kullanarak kolayca ses kontrolü sağlar.

Python – Duraklat, Devam ve Durum Bilgisi

```

1 def duraklat(self):
2     """Muzigi duraklat."""
3     pygame.mixer.music.pause()
4
5 def devam_et(self):
6     """Duraklatilmis muzige devam et."""
7     pygame.mixer.music.unpause()
8
9 @property
10 def caliyor_mu(self):
11     """Muzik caliyor mu?"""
12     return pygame.mixer.music.get_busy()
13
14 @property
15 def durum_bilgisi(self):
16     """Mevcut durum bilgisini sozluk olarak doner."""
17     return {
18         "sahne": self.mevcut_sahne or "Yok",
19         "hedef": self.hedef_sahne or "-",
20         "ses": self.ses_seviyesi,
21         "caliyor": self.caliyor_mu,
22         "gecis": self.gecis_aktif,
23     }

```

`duraklat()` ve `devam\_et()` metotları `pygame.mixer.music` modülünün `pause()` ve `unpause()` fonksiyonlarını sarmalar. `@property` dekoratörü ile tanımlanan `caliyor_mu` ve `durum_bilgisi` özellikleri, sınıfın durumunu sorgulamayı kolaylaştırır. `durum_bilgisi` özellikle hata ayıklama ve arayüzde bilgi gösterme için kullanışlıdır.

Ana Döngü ile Entegrasyon

Tüm bu parçaları bir araya getiren `main()` fonksiyonunda yöneticiyi oluşturuyoruz, müzikleri ekliyoruz ve tuş girdileriyle sahne geçişlerini kontrol ediyoruz:

Python – Ana Dongu – Baslangic ve Tus Kontrolleri

```

1  def main():
2      """Ana fonksiyon."""
3      pygame.init()
4      pygame.mixer.init()
5
6      ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
7      pygame.display.set_caption(BASLIK)
8      saat = pygame.time.Clock()
9
10     # Test muzik dosyalarini olustur
11     muzik_dosyalari = {
12         "menu": test_muzik_olustur("test_menu.wav",
13             ↪ frekans=262),
14         "oyun": test_muzik_olustur("test_oyun.wav",
15             ↪ frekans=392),
16         "boss": test_muzik_olustur("test_boss.wav",
17             ↪ frekans=523),
18         "bitis": test_muzik_olustur("test_bitis.wav",
19             ↪ frekans=196),
20     }
21
22     # MusicManager olustur ve muzikleri ekle
23     yoneticici = MusicManager(varsayilan_ses=0.5,
24         ↪ varsayilan_fade=1500)
25     for sahne, dosya in muzik_dosyalari.items():
26         yoneticici.muzik_ekle(sahne, dosya)
27
28     yoneticici.sahne_degistir("menu")

```

Her sahne için farklı bir frekans kullanıyoruz: menü 262 Hz (Do notası), oyun 392 Hz (Sol), boss 523 Hz (yüksek Do) ve bitiş 196 Hz (düşük Sol). Böylece sahne geçişlerini kulaklarınla da ayırt edebilirsin. `MusicManager` kurulduktan sonra tek bir `sahne\_degistir("menu")` çağrısıyla menü müziği başlıyor.

Python – Ana Dongu – Olay Isleme

```

1  calistir = True
2  while calistir:
3      olaylar = pygame.event.get()
4
5      for olay in olaylar:
6          if olay.type == pygame.QUIT:
7              calistir = False
8          elif olay.type == pygame.KEYDOWN:
9              if olay.key == pygame.K_1:

```

```

10     yonetici.sahne_degistir("menu")
11     elif olay.key == pygame.K_2:
12         yonetici.sahne_degistir("oyun")
13     elif olay.key == pygame.K_3:
14         yonetici.sahne_degistir("boss")
15     elif olay.key == pygame.K_4:
16         yonetici.sahne_degistir("bitis")
17     elif olay.key == pygame.K_UP:
18         yonetici.ses_artir()
19     elif olay.key == pygame.K_DOWN:
20         yonetici.ses_azalt()
21
22     # MusicManager guncelle (fade gecislerini yönet)
23     yonetici.guncelle(olaylar)

```

Olay döngüsünde 1–4 tuşları sahne geçişlerini, yukarı/aşağı ok tuşları ise ses seviyesini kontrol eder. Dikkat et: `guncelle()` çağrısını olay döngüsünün **dışında** ama `while` döngüsünün **içinde** yapıyoruz. Böylece tüm olaylar toplu olarak yöneticiye iletilir ve fade geçişleri doğru zamanda tetiklenir.

Python – Temizlik

```

1  # Temizlik
2  pygame.mixer.music.stop()
3  pygame.quit()
4  for dosya in muzik_dosyalari.values():
5      if os.path.exists(dosya):
6          os.remove(dosya)

```

Program kapanırken önce müziği durduruyor, ardından PyGame'i kapatıyor ve test için oluşturulan geçici WAV dosyalarını siliyor. Gerçek bir oyunda bu son adıma gerek olmaz çünkü müzik dosyaların zaten projenin parçasıdır.

27.5 Sonuç

Bu ünite de `pygame.mixer.music` modülünü derinlemesine inceledik ve arka plan müziğinin bir oyuna nasıl hayat verdiğini gördük. Öğrendiklerini kısaca özetleyelim.

27.5.1 Temel Fonksiyonlar

Aşağıdaki tablo, bu ünite de öğrendiğin tüm `music` modülü fonksiyonlarını bir arada gösterir:

Tablo 27.1: pygame.mixer.music fonksiyonları özet tablosu

Fonksiyon	Açıklama
<code>music.load(dosya)</code>	Müzik dosyasını streaming için hazırlar
<code>music.play(loops, fade_ms)</code>	Müziği çalar; <code>loops=-1</code> sonsuz döngü
<code>music.stop()</code>	Müziği tamamen durdurur
<code>music.pause()</code>	Müziği duraklatır (kaldığı yerden devam edilebilir)
<code>music.unpause()</code>	Duraklatılmış müziğe devam eder
<code>music.rewind()</code>	Müziği başa sarar
<code>music.fadeout(ms)</code>	Müziği belirtilen sürede kısarak durdurur
<code>music.queue(dosya)</code>	Parça bitince çalınacak sonraki müziği ayarlar
<code>music.set_volume(v)</code>	Ses seviyesini ayarlar (0.0 – 1.0)
<code>music.get_volume()</code>	Mevcut ses seviyesini döner
<code>music.get_busy()</code>	Müzik çalışıyorsa <code>True</code> döner
<code>music.set_endevent(e)</code>	Müzik bittiğinde tetiklenecek olayı ayarlar

27.5.2 Sound ve Music Karşılaştırması

Tablo 27.2: Sound sınıfı ile music modülünün karşılaştırması

Özellik	Sound	music
Bellek kullanımı	Tamamı belleğe yüklenir	Streaming (parça parça)
Aynı anda çalma	Birden fazla (çok kanallı)	Tek kanal
En uygun kullanım	Kısa ses efektleri	Uzun arka plan müziği
Fade desteği	<code>fadeout()</code>	<code>fadeout()</code> + <code>fade_ms</code>
Kuyruk desteği	Yok	<code>queue()</code>

27.5.3 En İyi Uygulamalar

Bu üniteyi tamamlarken akılda tutman gereken en önemli noktalar şunlardır. Birincisi, müzik formatı olarak **OGG Vorbis** kullan — platform bağımsızdır ve iyi sıkıştırma sağlar. İkincisi, sahne geçişlerinde her zaman **fade efektleri** kullan — ani müzik kesintileri profesyonel değildir. Üçüncüsü, müzik mantığını bir **MusicalManager** sınıfında topla — kodun bakımı ve genişletilmesi çok kolaylaşır. Son olarak, ses seviyesi kontrollerini oyuncuya bırak — herkesin ses tercihi farklıdır.

Bir sonraki ünite de ses tasarımı prensiplerini öğrenecek, ses efektleri ile müziği bir arada nasıl dengeli kullanacağını ve oyunun atmosferini sesle nasıl güçlendireceğini keşfedeceksin.

Ses Tasarımı Prensipleri

“İyi bir oyun sesi, oyuncunun fark etmediği ama yokluğunda hemen hissedeceği şeydir.”

— ROB BRIDGETT, OYUN SES TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Ses katmanları (SFX, müzik, UI, ambient) ve **ses miksajı** teknikleri
- **Ücretsiz ses kaynakları** (Freesound.org, OpenGameArt.org, Kenney.nl) ve lisanslama
- **Ses optimizasyonu**: bellek yönetimi, önyükleme ve ses havuzu sistemi
- Uzak Kaçışı projesinin **tam ses entegrasyonu**: SoundManager sınıfı ile SFX, müzik ve ses ayarları

28.1 Ses Miksajı ve Katmanlar

Bir filmin ses tasarımını düşün: diyaloglar, müzik, ortam sesleri ve efektler aynı anda çalınır ama birbirini boğmaz. Her ses kendi “katmanında” yaşar ve ses mühendisi bu katmanların dengesini ayarlar. Oyun geliştirmede de durum aynı. Bölüm 25’de `pygame.mixer` modülünü başlatmayı, Bölüm 26’nde ses efektlerini çalmayı ve Bölüm 27’nde arka plan müziği eklemeyi öğrendin. Şimdi tüm bu sesleri bir arada dengeli bir şekilde yönetmeyi öğreneceksin.

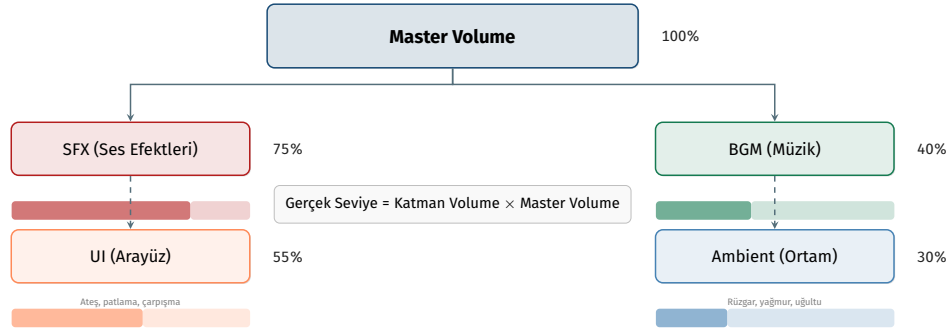
■ Tanım: Ses Miksajı (Audio Mixing)

Oyundaki farklı ses katmanlarının (müzik, efekt, UI, ortam) ses seviyelerinin birbirine göre dengelenmesi ve aynı anda uyumlu şekilde çalınması sürecidir. Amaç, hiçbir sesin diğerlerini boğmaması ve oyuncunun önemli sesleri net duyabilmesidir.

28.1.1 Ses Katmanları

Profesyonel oyunlarda sesler genellikle dört ana katmana ayrılır. Birincisi *SFX* (Sound Effects) katmanı: silah sesleri, patlama, zıplama, madeni para toplama gibi oyun içi efektlerdir. İkincisi *BGM* (Background Music) katmanı: arka planda sürekli çalan müzik. Üçüncüsü *UI* katmanı: menü tıklamaları, buton hover sesleri, bildirim sesleri gibi arayüz efektleri. Son olarak *Ambient* katmanı: rüzgar, yağmur, kalabalık uğultusu gibi ortam sesleri.

Bu katmanları görsel olarak incelediğimizde daha net anlayacağız.



Şekil 28.1: Ses katmanları hiyerarşisi ve volume dengesi

Şekil 28.1’da görüldüğü gibi, en üstte *Master Volume* bulunur ve tüm katmanları etkiler. Her katmanın kendi volume seviyesi vardır ve nihai ses seviyesi her katmanın kendi değeri ile master volume’ün çarpımıyla belirlenir.

28.1.2 Volume Balans Stratejisi

Peki hangi katman ne kadar yüksek olmalı? Genel kabul görmüş bir başlangıç noktası şöyle: SFX sesleri %70–80, müzik %30–50, UI sesleri %50–60 ve ortam sesleri %20–40 aralığında tutulur. Tabii ki bu değerler oyunun türüne göre değişir. Bir korku oyununda ortam sesleri çok daha yüksek olabilirken, bir aksiyon oyununda SFX sesleri ön plandadır.

Python – Katman Bazlı Volume Yonetimi

```

1  # Ses katmanlari ve varsayılan seviyeleri
2  ses_katmanlari = {
3      "master": 1.0,      # Tum sesleri etkiler
4      "sfx": 0.75,       # Ses efektleri
5      "bgm": 0.40,       # Arka plan muzigi
6      "ui": 0.55,        # Arayuz sesleri
7      "ambient": 0.30    # Ortam sesleri
8  }
9
10 def gercek_volume(katman_adi):
11     """Katmanin master volume ile carpilmis gercek
12     ↪ seviyesini dondurur."""
13     return ses_katmanlari[katman_adi] *
14     ↪ ses_katmanlari["master"]

```



```

14 # Ornek: SFX gercek seviyesi = 0.75 * 1.0 = 0.75
15 # Master %50'ye dusurulurse = 0.75 * 0.5 = 0.375

```

Bu kodda dikkat edilmesi gereken noktalar:

1. Her katmanın kendi volume değeri 0.0 ile 1.0 arasındadır
2. Master volume tüm katmanları oransal olarak etkiler
3. `gercek_volume()` fonksiyonu gerçek çıkış seviyesini hesaplar

28.1.3 Ses Önceliği

Aynı anda çok fazla ses çalmaya çalışırsan ne olur? PyGame'in kanal sayısı sınırlı (varsayılan 8). Eğer tüm kanallar doluyken yeni bir ses çalmak istersen, ya ses çalınmaz ya da önemli bir ses kesilir. Bu sorunu çözmek için *ses önceliği* sistemi kuralısın.

Temel yaklaşım şöyle: oyuncu ile doğrudan ilgili sesler (ateş etme, hasar alma) en yüksek önceliğe sahiptir. Düşman sesleri orta önceliktedir. Ortam sesleri ise en düşük önceliktedir ve kanal dolduğunda ilk kesilecek sesler bunlardır.

Python – Ses Önceliği Sistemi

```

1 import pygame
2
3 # Kanal sayisini artir
4 pygame.mixer.set_num_channels(16)
5
6 # Oncelikli kanallar ayir
7 KANAL_OYUNCU = pygame.mixer.Channel(0) # Yuksek oncelik
8 KANAL_SILAH = pygame.mixer.Channel(1) # Yuksek oncelik
9 KANAL_DUSMAN = pygame.mixer.Channel(2) # Orta oncelik
10 KANAL_UI = pygame.mixer.Channel(3) # Orta oncelik
11 # Kanallar 4-15: Genel kullanim (dusuk oncelik)
12
13 def oncelikli_cal(ses, kanal):
14     """Kanali kontrol edip sesi cal."""
15     if not kanal.get_busy():
16         kanal.play(ses)
17     # Kanal doluysa oyuncu sesi beklemesin
18     elif kanal == KANAL_OYUNCU:
19         kanal.stop()
20         kanal.play(ses)

```

Bu sistemde `set_num_channels(16)` ile toplam kanal sayısını 16'ya çıkarıyoruz. İlk dört kanalı özel amaçlar için ayırıyoruz. Geri kalan kanallar genel kullanım için serbest kalıyor. Oyuncu kanalı doluysa mevcut sesi durduruyoruz çünkü oyuncunun aksiyonlarına anında geri bildirim vermek en önemli önceliktir.

⚠ Dikkat: Dikkat

Kanal sayısını gereğinden fazla artırmak bellek tüketimini artırır. Çoğu 2D oyun için 16 kanal fazlasıyla yeterlidir. Kanal sayısını `mixer.init()` çağrısından **sonra** ayarla.

💡 İpucu: Profesyonel İpucu

Oyununun ses dengesini test ederken kulaklık yerine hoparlörle de dinle. Farklı cihazlarda ses dengesi değişebilir. Ayrıca oyunu sessiz modda da test et; önemli bilgilerin sadece sesle değil, görsel olarak da verildiğinden emin ol.

28.2 Ücretsiz Ses Kaynakları ve Lisanslama

Harika bir oyun mekaniği tasarladın, görselleri yerleştirdin, fizik motorunu kurdu ama sesleri nereden bulacaksın? Kendi ses efektlerini kaydetmek tabii ki bir seçenek, ama profesyonel kalitede patlama sesleri veya orkestra müziği üretmek kolay değil. Neyse ki internet üzerinde oyun geliştiriciler için özel olarak hazırlanmış ücretsiz ses kütüphaneleri var. Bu konuda en popüler üç kaynağı ve lisanslama kurallarını öğreneceksin.

28.2.1 Freesound.org

Freesound, dünyanın en büyük ücretsiz ses efekti kütüphanelerinden biridir. Yüz binlerce ses efekti içerir ve topluluk tarafından sürekli büyür. Patlama, silah, adım sesi, kapı gıcirtısı gibi neredeyse her türlü ses efektini burada bulabilirsin.

Freesound'da sesler farklı lisanslarla paylaşılır. En yaygın olanı *Creative Commons 0* (CC0): bu lisansla paylaşılan sesleri hiçbir kısıtlama olmadan kullanabilirsin, atıf bile gerekmez. *CC BY* lisansı ise sesi kullanabilirsin ama oluşturucunun adını belirtmen gerekir. *CC BY-NC* lisansı ise ticari olmayan projelerde kullanımı sınırlar.

💡 Bilgi: Freesound Kullanım İpucu

Freesound'da arama yaparken İngilizce terimler kullan: "explosion", "laser shot", "coin pickup", "menu click" gibi. Sonuçları lisansa göre filtrele ve CC0 olanları tercih et. İndirdikten sonra dosya formatını kontrol et; OGG veya WAV formatını tercih et.

28.2.2 OpenGameArt.org

OpenGameArt, özellikle oyun geliştiriciler için tasarlanmış bir platformdur. Hem görseller hem sesler bulabilirsin. Avantajı, tüm içeriklerin oyun kullanımı için uygun lisanslarla paylaşılmasıdır. Ses efektleri genellikle OGG formatında ve oyuna hazır kalitede sunulur.

OpenGameArt'ta ses paketleri halinde de içerik bulabilirsin. Örneğin “RPG Sound Pack” veya “Space Shooter SFX Bundle” gibi kategorilere ayrılmış paketler, birbirine uyumlu sesler içerir. Bu sayede oyunundaki tüm sesler tutarlı bir kalitede olur.

28.2.3 Kenney.nl

Kenney, oyun geliştiriciler arasında çok bilinen bir kaynak. Hem görseller hem sesler sunar ve tümü CC0 lisansı ile paylaşılır, yani tamamen özgürce kullanabilirsin. Kenney'nin ses paketleri özellikle iyi organize edilmiştir: UI sesleri, sci-fi efektleri, doğa sesleri gibi kategorilere ayrılmıştır.

Tablo 28.1: Ücretsiz ses kaynaklarının karşılaştırması

Kaynak	Lisans	Format	Avantaj
Freesound.org	CC0, CC BY, CC BY-NC	WAV, OGG, MP3	En geniş kütüphane
OpenGameArt.org	CC0, CC BY, GPL	OGG, WAV	Oyuna özel paketler
Kenney.nl	CC0	OGG, WAV	Tümü ücretsiz, kaliteli

28.2.4 Lisanslama ve Atıf

Oyunun için ses dosyası indirdiğinde lisans bilgisine mutlaka dikkat et. Lisans ihlali yasal sorunlara yol açabilir. Pratik bir yaklaşım olarak projenin kök dizininde **CREDITS.txt** veya **LISANS.txt** adında bir dosya oluştur ve kullandığın her ses dosyasının kaynağını, oluşturucusunu ve lisansını bu dosyaya yaz.

Python – credits.txt Örneği

```

1  # CREDITS.txt - Ses Dosyası Kaynakları
2  # =====
3  #
4  # patlama.ogg
5  #   Kaynak: Freesound.org
6  #   Oluşturucu: SoundUser42
7  #   Lisans: CC0 1.0
8  #   URL: https://freesound.org/s/12345
9  #
10 # menu_tikla.wav
11 #   Kaynak: Kenney.nl
12 #   Paket: UI Audio Pack
13 #   Lisans: CC0 1.0
14 #
15 # arkaplan_muzik.ogg
16 #   Kaynak: OpenGameArt.org
17 #   Oluşturucu: MusicArtist
18 #   Lisans: CC BY 3.0 (atıf gerekli)

```

28.2.5 Ses Düzenleme: Audacity

İndirdiğin ses dosyaları bazen doğrudan oyuna uygun olmayabilir. Ses çok uzun olabilir, başında veya sonunda sessizlik olabilir ya da volume seviyesi diğer seslerle uyumsuz olabilir. Bu durumda *Audacity* adlı ücretsiz ses düzenleme aracını kullanabilirsin.

Audacity ile yapabileceğin temel işlemler: sesi kırpma (trim), volume seviyesini normalleştirme (normalize), fade in/fade out efekti ekleme, dosya formatını dönüştürme (örneğin MP3'ten OGG'ye) ve gereksiz gürültüyü temizleme. Özellikle ses normalleştirme çok önemli: farklı kaynaklardan indirdiğin seslerin volume seviyeleri birbirinden farklı olabilir ve Audacity'nin "Normalize" efekti ile tüm seslerini aynı tepe seviyesine getirebilirsin.

İpucu: Profesyonel İpucu

Ses dosyalarını OGG formatında sakla. OGG, MP3'e göre daha küçük dosya boyutu sunar, patent sorunu yoktur ve PyGame ile mükemmel uyumludur. Audacity ile "Export as OGG" seçeneğini kullanarak kolayca dönüştürebilirsin.

28.3 Ses Optimizasyonu

Bir oyun düşün: her düşman yok edildiğinde patlama sesi çalınıyor, arka planda müzik dönüyor, ortam sesleri devam ediyor ve oyuncu ateş ediyor. Tüm bu sesler aynı anda yüklenmeli, çalınmalı ve bellekte tutulmalı. Eğer ses yönetimini doğru yapmazsan oyunun takılabilir, bellek tükenebilir veya sesler gecikmeli çalınabilir. Bu konuda ses dosyalarını nasıl optimize edeceğini ve performans sorunlarını nasıl önleyeceğini öğreneceksin.

28.3.1 Bellek Yönetimi

Bölüm 25'de `pygame.mixer.Sound` ile ses dosyası yüklemeyi öğrendin. Bir `Sound` nesnesi oluşturduğunda ses verisi tamamen belleğe yüklenir. Küçük ses efektleri için bu sorun değil ama büyük dosyalarda dikkatli olman gerekir.

Pratik bir kural: kısa ses efektlerini (ateş, patlama, tıklama gibi 1-2 saniyelik sesler) `Sound` nesnesi olarak belleğe yükle. Uzun ses dosyalarını (arka plan müziği gibi dakikalarca süren sesler) ise `pygame.mixer.music` modülü ile akış (streaming) olarak çal. Bu şekilde uzun müzik dosyaları belleği doldurmaz, dosyadan parça parça okunur.

Python – Bellek Bilinçli Ses Yükleme

```
1 import pygame
2
3 pygame.mixer.init(frequency=44100, size=-16, channels=2,
4   ↪   buffer=512)
5
6 # Kısa efektler -> Sound nesnesi (belleğe yükle)
```

```

6 ates_sesi = pygame.mixer.Sound("sesler/ates.ogg")      #
  ↳ ~50 KB
7 patlama_sesi = pygame.mixer.Sound("sesler/patlama.ogg") #
  ↳ ~80 KB
8 tikla_sesi = pygame.mixer.Sound("sesler/tikla.ogg")    #
  ↳ ~10 KB
9
10 # Uzun muzik -> music modulu (akis olarak cal)
11 pygame.mixer.music.load("sesler/arkaplan.ogg") # ~3 MB
  ↳ ama bellekte az yer kaplar
12 pygame.mixer.music.play(-1) # Sonsuz dongu

```

Bu ayrımı yapmak çok önemli. Eğer 3 MB'lık bir müzik dosyasını `Sound` nesnesi olarak yüklersen tamamı belleğe alınır. Aynı dosyayı `music.load()` ile yüklersen sadece küçük bir tampon bellek kullanılır ve müzik dosyadan okunarak çalınır.

⚠ Dikkat: Dikkat

`mixer.init()` içindeki `buffer` parametresi ses gecikmesini doğrudan etkiler. Küçük tampon (256–512) daha az gecikme sağlar ama CPU yükünü artırır. Büyük tampon (2048–4096) CPU'yu rahatlatır ama ses gecikmesi artabilir. Oyun türüne göre deneyerek en iyi değeri bul. Aksiyon oyunları için 512, strateji oyunları için 1024 iyi bir başlangıç noktasıdır.

28.3.2 Ses Önyükleme (Preloading)

Oyunun yükleme ekranında tüm sesleri önceden yüklemek (preloading) hem bellek kullanımını planlı hale getirir hem de oyun sırasında disk erişim gecikmesini önler. Bir ses dosyasını ilk kez çalmak istediğinde diskten okuma gecikmesi yaşanabilir. Önceden yüklenmiş seslerde ise çalma anında gecikme olmaz.

Python – Ses Onyukleme Sistemi

```

1 import os
2 import pygame
3
4 class SesYukleyici:
5     """Tum sesleri onceden yukleyen yardimci sinif."""
6
7     def __init__(self, ses_dizini):
8         self.ses_dizini = ses_dizini
9         self.sesler = {}
10
11     def hepsini_yukle(self):
12         """Dizindeki tum .ogg ve .wav dosyalarini
13         ↳ yukle."""
14         for dosya in os.listdir(self.ses_dizini):
15             if dosya.endswith((".ogg", ".wav")):

```

```

15         ad = os.path.splitext(dosya)[0] #
           ↳ Uzantiyi çıkar
16         yol = os.path.join(self.ses_dizini, dosya)
17         self.sesler[ad] = pygame.mixer.Sound(yol)
18         print(f" Yuklendi: {ad}")
19
20     def al(self, ad):
21         """Isimle ses nesnesini dondur."""
22         return self.sesler.get(ad)
23
24     # Kullanım
25     yukleyici = SesYukleyici("assets/sounds/sfx")
26     yukleyici.hepsini_yukle()
27
28     ates = yukleyici.al("ates")
29     if ates:
30         ates.play()

```

Bu sistemde `SesYukleyici` sınıfı belirtilen dizindeki tüm ses dosyalarını tarar ve belleğe yükler. Sonrasında isme göre erişim sağlarsın. Dosya adından uzantıyı çıkararak temiz bir anahtar elde ediyoruz: `patlama.ogg` dosyası "`patlama`" anahtarıyla erişilebilir.

28.3.3 Ses Havuzu (Sound Pool)

Aynı ses efektini çok hızlı art arda çalmak istediğinde ne olur? Örneğin oyuncu hızlıca ateş ediyorsa her mermi için aynı ses çalınmalı. Ama aynı `Sound` nesnesini art arda çalarsan önceki ses kesilip yenisi başlar. Bu durum seslerin kesik kesik duyulmasına neden olur.

Çözüm, aynı ses efektinin birden fazla kopyasını tutmak ve sırayla kullanmaktır. Buna *ses havuzu* (sound pool) denir.

Python – Ses Havuzu Sistemi

```

1  import pygame
2
3  class SesHavuzu:
4      """Aynı sesin birden fazla kopyasını yöneten havuz."""
5
6      def __init__(self, ses_dosyasi, kopya_sayisi=4):
7          self.sesler = []
8          self.siradaki = 0
9          for _ in range(kopya_sayisi):
10             self.sesler.append(pygame.mixer.Sound(ses_dos_
           ↳ yasi))
11
12     def cal(self, volume=1.0):
13         """Siradaki kopyayı cal ve bir sonrakine gec."""
14         ses = self.sesler[self.siradaki]
15         ses.set_volume(volume)

```

```

16     ses.play()
17     self.siradaki = (self.siradaki + 1) %
        ↪ len(self.sesler)
18
19     # Kullanım: Hızlı ates sesi
20     ates_havuzu = SesHavuzu("sesler/ates.ogg", kopya_sayisi=4)
21
22     # Her ates ettiğinde:
23     ates_havuzu.cal(volume=0.7)

```

Bu havuz sisteminde `siradaki` indeksi dairesel olarak ilerler. Dört kopya varsa sırayla 0, 1, 2, 3, 0, 1, 2, 3... şeklinde döner. Böylece aynı ses çok hızlı art arda çalınsa bile önceki kopyalar kesilmez ve sesler üst üste biner, daha doğal bir etki yaratır.

● Bilgi: Kopya Sayısı Ne Olmalı?

Çoğu oyun için 3–4 kopya yeterlidir. Çok hızlı ateşli silahlar için 6–8 kopya gerekebilir. Aynı anda hiçbir zaman çalınmayacak sesler (menü tıklaması gibi) için havuz gereksizdir, tek kopya yeterlidir.

28.3.4 Performans İpuçları

Ses optimizasyonunda birkaç pratik ipucu daha hayatını kolaylaştırır. Birincisi, ses dosyalarını mümkünse **mono** (tek kanal) olarak sakla. Stereo sesler iki kat bellek tüketir ama küçük efektler için stereonun fark edilebilir bir avantajı yoktur. İkincisi, örnekleme hızını (sample rate) ihtiyacına göre ayarla. Profesyonel müzik için 44100 Hz standart ama ses efektleri için 22050 Hz genellikle yeterlidir ve dosya boyutunu yarıya indirir.

Üçüncüsü, kullanılmayan sesleri bellekten kaldır. Bir seviyeye özel sesler varsa seviye değiştiğinde eski sesleri `del` ile sil ve yeni seviyenin seslerini yükle. Son olarak, ses dosyası boyutlarını kontrol altında tut: toplamda 10–20 MB ses dosyası küçük bir 2D oyun için fazlasıyla yeterlidir.

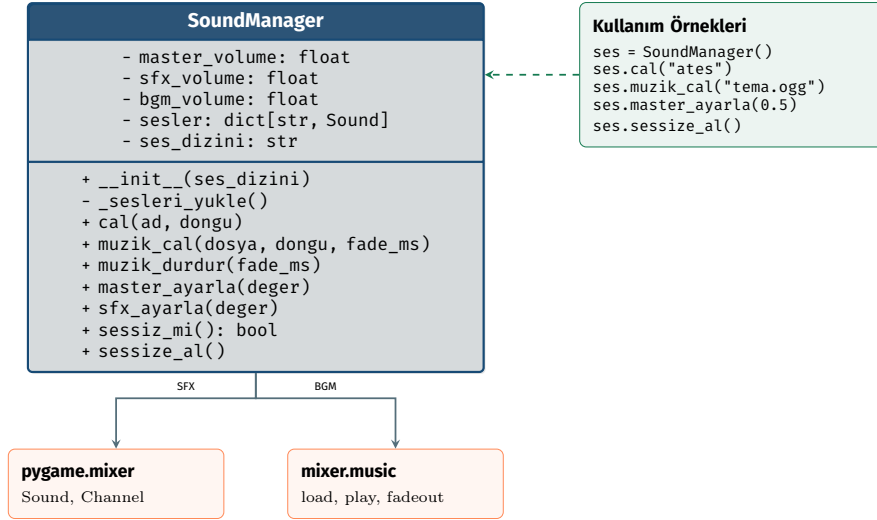
28.4 Uzay Kaçışı: Tam Ses Entegrasyonu

Bölüm 20’da Uzay Kaçışı projesinin ilk versiyonunu (sprite sistemi ve temel oynanış), Bölüm 22’de ikinci versiyonunu (düşman çarpışmaları ve mermi sistemi) oluşturdun. Bu bölüm boyunca ise ses sistemi temellerini, ses efektlerini ve arka plan müziği yönetimini öğrendin. Şimdi tüm bu bilgileri birleştirerek Uzay Kaçışı projesine tam bir ses entegrasyonu yapacağız. Merkezi bir `SoundManager` sınıfı tasarlayacak, SFX, müzik ve ses ayarlarını tek bir yapıda yöneteceğiz.

28.4.1 SoundManager Sınıfı

Ses yönetimini oyunun her yerinden tutarlı bir şekilde yapabilmek için merkezi bir yönetici sınıf tasarlamak en iyi yaklaşımdır. Bu sınıf ses yükleme, çalma,

durdurma ve volume ayarlama işlemlerini tek bir noktadan yönetir.



Şekil 28.2: SoundManager sınıfının yapısı ve metotları

Şekil 28.2’de SoundManager sınıfının genel yapısı görünüyor. Sınıf üç ana bölümden oluşur: özellikler (ses sözlükleri ve volume seviyeleri), yükleme metotları ve çalma/kontrol metotları. Şimdi bu sınıfı adım adım kodlayalım.

Python – SoundManager Sinifi - Baslangic

```
1 import os
2 import pygame
3
4 class SoundManager:
5     """Merkezi ses yonetim sinifi."""
6
7     def __init__(self, ses_dizini="assets/sounds"):
8         """Ses sistemini baslat ve sesleri yukle."""
9         # Volume seviyeleri (0.0 - 1.0)
10        self.master_volume = 1.0
11        self.sfx_volume = 0.75
12        self.bgm_volume = 0.40
13
14        # Ses sozlukleri
15        self.sesler = {} # SFX sesleri
16        self.ses_dizini = ses_dizini
17
18        # Sesleri yukle
19        self._sesleri_yukle()
```

Bu başlangıç yapısında `__init__` metodu volume seviyelerini varsayılan değerlerine ayarlıyor ve ses sözlüğünü oluşturuyor. Alt çizgi ile başlayan `_sesleri_yukle` metodu sınıfın iç kullanımı için tasarlanmış özel (private) bir metot.

Python – SoundManager – Ses Yükleme

```

1 def _sesleri_yukle(self):
2     """SFX seslerini belleğe yükle."""
3     sfx_dosyalari = {
4         "ates": "sfx/ates.ogg",
5         "patlama": "sfx/patlama.ogg",
6         "hasar": "sfx/hasar.ogg",
7         "bonus": "sfx/bonus.ogg",
8         "menu_tikla": "sfx/menu_tikla.ogg",
9         "oyun_bitti": "sfx/oyun_bitti.ogg",
10    }
11    for ad, dosya in sfx_dosyalari.items():
12        yol = os.path.join(self.ses_dizini, dosya)
13        if os.path.exists(yol):
14            self.sesler[ad] = pygame.mixer.Sound(yol)
15            # Baslangic volume ayarla
16            self.sesler[ad].set_volume(
17                self.sfx_volume * self.master_volume
18            )

```

Her ses dosyasını bir sözlükte tutuyoruz. Bu sayede oyunun herhangi bir yerinden `ses_yoneticisi.cal("ates")` gibi basit bir çağrıyla ses çalabilirsin. Dosya yolunu kontrol ederek eksik dosyalarda hata almayı önliyoruz.

Python – SoundManager – Calma ve Kontrol

```

1 def cal(self, ad, dongu=0):
2     """Isimle SFX sesi cal."""
3     ses = self.sesler.get(ad)
4     if ses:
5         ses.play(loops=dongu)
6
7 def muzik_cal(self, dosya, dongu=-1, fade_ms=1000):
8     """Arka plan muzigini cal."""
9     yol = os.path.join(self.ses_dizini, "muzik", dosya)
10    if os.path.exists(yol):
11        pygame.mixer.music.load(yol)
12        pygame.mixer.music.set_volume(
13            self.bgm_volume * self.master_volume
14        )
15        pygame.mixer.music.play(loops=dongu,
16                                ↪ fade_ms=fade_ms)
17
18 def muzik_durdur(self, fade_ms=500):
19     """Muzigi fade out ile durdur."""
20     pygame.mixer.music.fadeout(fade_ms)
21
22 def master_ayarla(self, deger):
23     """Master volume degistir ve tum sesleri guncelle."""
24     self.master_volume = max(0.0, min(1.0, deger))
25     # SFX seslerini guncelle

```

```

25     for ses in self.sesler.values():
26         ses.set_volume(self.sfx_volume *
27             ↪ self.master_volume)
28     # Muzik volume guncelle
29     pygame.mixer.music.set_volume(
30         self.bgm_volume * self.master_volume
31     )
32
33     def sfx_ayarla(self, deger):
34         """SFX volume degistir."""
35         self.sfx_volume = max(0.0, min(1.0, deger))
36         for ses in self.sesler.values():
37             ses.set_volume(self.sfx_volume *
38                 ↪ self.master_volume)
39
40     def sessiz_mi(self):
41         """Master volume sifir mi kontrol et."""
42         return self.master_volume == 0.0
43
44     def sessize_al(self):
45         """Tum sesleri kapat/ac (toggle)."""
46         if self.master_volume > 0:
47             self._onceki_volume = self.master_volume
48             self.master_ayarla(0.0)
49         else:
50             self.master_ayarla(
51                 getattr(self, "_onceki_volume", 1.0)
52             )

```

`master_ayarla` metodu kritik bir role sahip: master volume değiştiğinde tüm SFX seslerinin ve müziğin volume seviyesi otomatik olarak güncelleniyor. `max(0.0, min(1.0, deger))` ifadesi değerın 0–1 aralığında kalmasını garantiler. `sessize_al` metodu ise bir toggle görevi görür: sesli ise kapatır, sessiz ise önceki seviyeye geri döner.

28.4.2 Oyun İçi Entegrasyon

SoundManager sınıfını oyunun ana döngüsüne entegre edelim. Ana dosyada tek bir `SoundManager` örneği oluşturacak ve oyun olaylarına bağlayacağız.

Python – Uzay Kacisi - Ana Dongu Entegrasyonu

```

1  # Ana dosyada (uzay_kacisi.py)
2  ses = SoundManager("assets/sounds")
3  ses.muzik_cal("uzay_temasi.ogg")
4
5  # Oyun dongusunda:
6  # Oyuncu ates ettiginde
7  if ates_edildi:
8      ses.cal("ates")

```

```

9
10 # Dusman yok edildiğinde
11 if dusman_vuruldu:
12     ses.cal("patlama")
13
14 # Oyuncu hasar aldığında
15 if oyuncu_hasar:
16     ses.cal("hasar")
17
18 # Bonus aldığında
19 if bonus_toplandi:
20     ses.cal("bonus")
21
22 # Oyun bittğinde
23 if oyun_bitti:
24     ses.muzik_durdur(fade_ms=2000)
25     ses.cal("oyun_bitti")
26
27 # M tusuna basinca sessiz modu
28 if m_tusu_basildi:
29     ses.sessize_al()

```

Dikkat edersen tüm ses işlemleri `ses` nesnesi üzerinden yapılıyor. Oyunun farklı modüllerinde (düşman sınıfı, oyuncu sınıfı, menü sınıfı) aynı `SoundManager` örneğini paylaşarak tutarlı ses yönetimi sağlarsın.

● Bilgi: Tam Kod Örneği

Bu ünitedeki `SoundManager` sınıfının tam ve çalıştırılabilir halini `kod_ornekleri/bolum07/unite4/03_ses_yoneticisi.py` dosyasında bulabilirsin. Uzak Kaçışı projesinin ses entegrasyonlu tam versiyonu ise `04_uzak_kacisi_ses_tam.py` dosyasındadır.

💡 İpucu: Profesyonel İpucu

`SoundManager` sınıfına ileride kolayca yeni özellikler ekleyebilirsin: 3D konumsal ses (sol/sağ panning), ses efektleri sırası (queue), rastgele ses varyasyonları (aynı efektin farklı versiyonları) gibi. Sınıf yapısı bu genişlemelere açıktır.

28.5 Sonuç

28.5.1 Ünite 4 Özeti

Bu ünite de ses tasarımının prensiplerini ve pratik uygulamalarını öğrendin. Konu 28.1’nde ses katmanları (SFX, BGM, UI, Ambient) kavramını, volume balans stratejisini ve ses önceliği sistemini kavradın. Konu 28.2’nde ücretsiz ses kaynakları (Freesound.org, OpenGameArt.org, Kenney.nl), lisanslama kuralları

ve Audacity ile ses düzenlemeyi öğrendin. Konu 28.3’nda bellek yönetimi, ses ön-yükleme (preloading) ve ses havuzu (sound pool) sistemini uyguladın. Son olarak Konu 28.4’da tüm bu bilgileri birleştirerek Uzay Kaçışı projesine SoundManager sınıfı ile tam ses entegrasyonu yaptın.

28.5.2 Bölüm 7 Genel Özeti

Bölüm 7’yi tamamladın, tebrikler! Bu bölüm boyunca PyGame’in ses sistemini baştan sona kavradın. Şimdi geriye dönüp neler öğrendiğine bir bakalım.

Bölüm 25’de `pygame.mixer` modülünün başlatılmasını, ses dosya formatlarını (WAV, OGG, MP3) ve `Sound` sınıfı ile temel ses işlemlerini öğrendin.

Bölüm 26’nde ses efektlerini yüklemeyi, çalmayı, durdurmayı ve volume kontrolü yapmayı, ayrıca kanallar (Channel) ile aynı anda birden fazla ses çalmayı kavradın.

Bölüm 27’nde `pygame.mixer.music` modülü ile uzun müzik dosyalarını akış olarak çalmayı, sonsuz döngüde tekrarlamayı ve fade in/fade out geçişlerini uyguladın.

Son olarak bu üniteye ses katmanlarını dengelemeyi, ücretsiz ses kaynaklarını bulmayı, ses performansını optimize etmeyi ve tüm bunları merkezi bir SoundManager sınıfında birleştirmeyi öğrendin.

28.5.3 Anahtar Kavramlar

Tablo 28.2: Bölüm 7 – Anahtar kavramlar

Kavram	Açıklama
<code>pygame.mixer</code>	PyGame ses sistemi modülü
<code>mixer.Sound</code>	Ses dosyasını belleğe yükleyen sınıf
<code>mixer.music</code>	Arka plan müziği için akış modülü
<code>mixer.Channel</code>	Tek bir ses kanalını temsil eden nesne
Ses katmanları	SFX, BGM, UI, Ambient olarak seslerin sınıflandırılması
Master Volume	Tüm ses katmanlarını oransal etkileyen ana seviye
Ses önceliği	Kanal dolduğunda hangi sesin kesilip hangisinin çalacağı
Preloading	Tüm sesleri önceden belleğe yükleyerek gecikmeyi önleme
Sound Pool	Aynı sesin birden fazla kopyasıyla üst üste çalınabilmesi
CC0 lisans	Hiçbir kısıtlama olmayan ücretsiz kullanım lisansı
Normalize	Ses seviyelerini eşit tepe noktasına getirme işlemi
SoundManager	Tüm ses işlemlerini merkezi yöneten sınıf

28.5.4 Önemli Komutlar

Tablo 28.3: Bölüm 7 – Önemli PyGame komutları

Komut	İşlev
<code>mixer.init(freq, size, ch, buf)</code>	Ses sistemini başlatır
<code>mixer.Sound(dosya)</code>	Ses dosyasını belleğe yükler
<code>ses.play(loops, maxtime)</code>	Sesi çalar
<code>ses.stop()</code>	Sesi durdurur
<code>ses.set_volume(v)</code>	Volume seviyesini ayarlar
<code>mixer.Channel(n)</code>	Belirli kanala erişir
<code>mixer.set_num_channels(n)</code>	Toplam kanal sayısını ayarlar
<code>music.load(dosya)</code>	Müzik dosyasını yükler
<code>music.play(loops, fade_ms)</code>	Müziği çalar
<code>music.stop()</code>	Müziği durdurur
<code>music.fadeout(ms)</code>	Müziği kademeli durdurur
<code>music.set_volume(v)</code>	Müzik volume ayarlar

28.5.5 Sıradaki Adım

Bölüm 8’de **Animasyon ve Zamanlayıcılar** konusuna geçeceğiz. Sprite sheet’lerden frame animasyonları oluşturmayı, AnimatedSprite sınıfı tasarlamayı, `pygame.time.get_ticks()` ve `USEREVENT` ile zamanlayıcılar kurmayı ve parçacık/patlama gibi görsel efektler eklemeyi öğreneceksin. Uzay Kaçışı projesini animasyonlarla tamamlayarak oyunumuza son şeklini vereceğiz.

● Bilgi: Pratik Yap

Bölüm 7’de öğrendiklerini pekiştirmek için şu egzersizleri dene:

- SoundManager’a bir “rastgele varyasyon” özelliği ekle: aynı efektin 3 farklı versiyonunu yükle ve her seferinde rastgele birini çal (patlama1.ogg, patlama2.ogg, patlama3.ogg)
- Ses ayarlarını bir dosyaya kaydetmeyi dene: JSON formatında volume seviyelerini sakla ve oyun açıldığında geri yükle
- Breakout oyununa ses ekle: tuğla kırma, topun sekme sesi, raket vuruş sesi ve oyun bitti müziği
- Basit bir ses test aracı yaz: ekranda volume kaydırıcıları olan ve sesleri test etmeni sağlayan bir program
- Freesound.org’dan en az 5 farklı ses efekti indir, Audacity ile düzenle ve oyununa entegre et

Frame Animasyonu

“Animasyon, hareketsiz resimlere ruh vermektir.”

— WALT DISNEY

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Sprite sheet kavramı, grid yapısı ve ücretsiz sprite kaynakları
- `subsurface()` ile sprite sheet'ten frame kesme teknikleri
- Zamana bağlı frame animasyonu, loop ve one-shot modları
- 4 yönlü yürüyen karakter animasyonu uygulaması

29.1 Sprite Sheet Kavramı

Şimdiye kadar oyun karakterlerini tek bir görsel ile temsil ettik: sabit bir üçgen, bir daire veya bir PNG dosyası. Ama gerçek oyunlarda karakterler hareket eder, saldırır, zıplar ve ölür. Tüm bu hareketler nasıl gösterilir? Cevap basit: *animasyon*! Tıpkı bir çizgi filmde olduğu gibi, farklı pozlardaki görselleri hızlıca peş peşe göstererek hareket illüzyonu yaratırız.

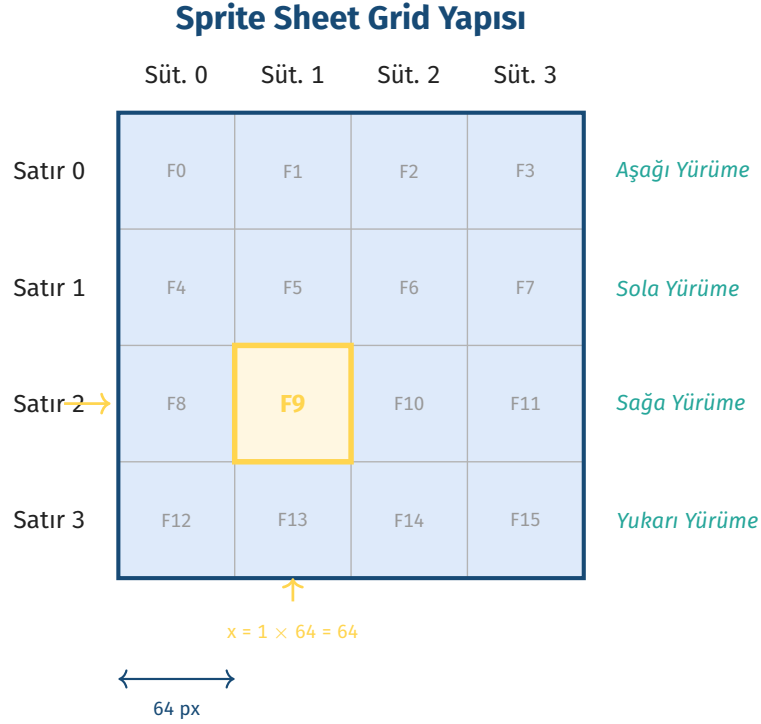
■ Tanım: Sprite Sheet

Bir *sprite sheet*, bir karakterin tüm animasyon karelerinin (*frame*) tek bir büyük görsel dosyasında düzenli bir şekilde sıralanmış halidir. Her frame, karakterin belirli bir andaki pozunu gösterir. Bu frameler hızlıca değiştirilerek animasyon oluşturulur.

Sprite sheet kullanmanın birçok avantajı vardır. Birincisi, **performans**: GPU tek bir büyük dokuyu (texture) bellekte tutmayı, yüzlerce küçük dosyayı yüklemeye tercih eder. İkincisi, **organizasyon**: bir karakterin tüm animasyonları tek bir dosyada toplandığı için yönetmesi kolaydır. Üçüncüsü, **bellek verimliliği**: tek bir *Surface* nesnesi üzerinden `subsurface()` ile frame'ler kesilir, her frame için ayrı bellek ayrılmaz.

29.1.1 Grid Yapısı

Sprite sheet'ler genellikle bir *grid* (ızgara) yapısında düzenlenir. Her hücre aynı boyuttadır ve bir frame'i temsil eder. Örneğin 4 sütun ve 4 satırdan oluşan bir sprite sheet, toplam 16 frame içerir.



Şekil 29.1: Sprite sheet grid yapısı: satır ve sütunlardan oluşan düzenli bir ızgara

Grid yapısında her frame'in konumunu bulmak için basit bir matematik kullanırız. Frame'in sol üst köşesinin *x* koordinatı $\text{sütun} * \text{frame_genişliği}$, *y* koordinatı ise $\text{satır} * \text{frame_yüksekliği}$ formülüyle hesaplanır.

Python – Sprite Sheet Grid Hesaplama

```

1  # Sprite sheet boyutları
2  sheet_genislik = 256 # piksel
3  sheet_yukseklik = 256
4
5  # Grid yapısı
6  sutun_sayisi = 4
7  satir_sayisi = 4
8
9  # Her frame'in boyutu
10 frame_gen = sheet_genislik // sutun_sayisi # 64 piksel
11 frame_yuk = sheet_yukseklik // satir_sayisi # 64 piksel
12
13 # Belirli bir frame'in konumu (satir=2, sutun=1)
14 x = 1 * frame_gen # 64
15 y = 2 * frame_yuk # 128

```


16 | # Frame (64, 128) konumundan baslar

29.1.2 Ücretsiz Sprite Kaynakları

Oyun geliştirirken kendi sprite sheet'lerini çizmek harika bir beceridir, ancak başlangıçta hazır kaynaklar kullanmak projeye hız kazandırır. İşte ücretsiz ve açık lisanslı sprite kaynakları:

Tablo 29.1: Ücretsiz sprite sheet kaynakları

Kaynak	Açıklama	Lisans
OpenGameArt.org Kenney.nl	Binlerce ücretsiz 2D/3D oyun varlığı Yüksek kaliteli sprite setleri, kullanıma hazır	CC0 / CC-BY CC0
itch.io/assets Liberated Pixel Cup	Bağımsız sanatçıların sprite paketleri RPG tarzı karakter ve ortam sprite'ları	Değişken CC-BY-SA

İpucu: Sprite Sheet Oluşturma

Kendi sprite sheet'ini oluşturmak istersen, **Aseprite** (ücretli ama açık kaynak derlenebilir), **Piskel** (ücretsiz, web tabanlı) veya **LibreSprite** (ücretsiz Aseprite fork'u) gibi piksel sanat editörlerini kullanabilirsin. Bu araçlar doğrudan sprite sheet formatında dışa aktarım yapabilir.

29.1.3 Animasyon Satırları

Tipik bir karakter sprite sheet'inde her **satır** farklı bir animasyonu, her **sütun** ise o animasyonun bir frame'ini temsil eder. Örneğin:

- **Satır 0:** Aşağı yürüme (4 frame)
- **Satır 1:** Sola yürüme (4 frame)
- **Satır 2:** Sağa yürüme (4 frame)
- **Satır 3:** Yukarı yürüme (4 frame)

Bu düzenleme sayesinde animasyon değiştirmek için sadece satır numarasını değiştirmemiz yeterlidir. Sütun numarasını ise zamana bağlı olarak artırarak frame'ler arasında geçiş yaparız.

Bilgi: Frame Boyutu

Yaygın sprite frame boyutları 16, 32, 48 ve 64'tür. Oyunun piksel sanat stiline göre seçim yapılır. Kenney.nl'deki sprite'lar genellikle 64 boyutundadır. Önemli olan sprite sheet'teki tüm frame'lerin aynı boyutta olmasıdır.

29.2 Frame Kesme

Sprite sheet kavramını öğrendin; şimdi bu büyük görselden tek tek frame'leri nasıl keseceğini görelim. PyGame'de bunun için iki temel yaklaşım vardır: `subsurface()` metodu ve `Surface.blit()` ile kopyalama. Her ikisini de inceleyeceğiz ama `subsurface()` yaklaşımı hem daha hızlı hem de daha bellek dostudur.

29.2.1 subsurface() ile Kesme

`Surface.subsurface(rect)` metodu, mevcut bir yüzeyin belirli bir dikdörtgen bölgesini referans olarak döndürür. Yeni bir kopya oluşturmaz; orijinal yüzeyin piksel verisini paylaşır. Bu sayede hem bellek tasarrufu sağlar hem de çok hızlı çalışır.

Python – subsurface() ile Frame Kesme

```

1 import pygame
2
3 pygame.init()
4
5 # Sprite sheet'i yükle
6 sheet = pygame.image.load("karakter.png").convert_alpha()
7
8 # Frame boyutlari
9 FRAME_GEN = 64
10 FRAME_YUK = 64
11
12 # Belirli bir frame'i kes (satir=0, sutun=2)
13 frame_rect = pygame.Rect(
14     2 * FRAME_GEN, # x = 128
15     0 * FRAME_YUK, # y = 0
16     FRAME_GEN,     # genislik = 64
17     FRAME_YUK,     # yukseklik = 64
18 )
19 frame = sheet.subsurface(frame_rect)

```

Burada dikkat edilmesi gereken nokta, `subsurface()` ile elde edilen yüzeyin orijinal sprite sheet ile aynı piksel verisini paylaşmasıdır. Bu, bellek açısından verimlidir ancak orijinal yüzey silinirse alt yüzey de geçersiz olur.

29.2.2 Tüm Frame'leri Toplama

Genellikle bir animasyonun tüm frame'lerini önceden keser ve bir listeye kaydederiz. Böylece animasyon sırasında sadece liste indeksini değiştirmemiz yeterli olur.

Python – Tüm Frame'leri Listele

```

1 def frameleri_kes(sheet, frame_gen, frame_yuk, satir):

```

```

2      """Sprite sheet'in belirli satirindaki tum frame'leri
      ↪ kes.
3
4      Args:
5          sheet: Sprite sheet yüzeyi.
6          frame_gen: Frame genisligi (piksel).
7          frame_yuk: Frame yuksekligi (piksel).
8          satir: Kesilecek satir numarası (0'dan baslar).
9
10     Returns:
11         list: Frame yuzeylerinin listesi.
12     """
13     frameler = []
14     sutun_sayisi = sheet.get_width() // frame_gen
15
16     for sutun in range(sutun_sayisi):
17         rect = pygame.Rect(
18             sutun * frame_gen,
19             satir * frame_yuk,
20             frame_gen,
21             frame_yuk
22         )
23         frame = sheet.subsurface(rect)
24         frameler.append(frame)
25
26     return frameler

```

Bu fonksiyonu kullanarak bir karakter sprite sheet'inden herhangi bir satırdaki animasyonu kolayca çıkarabiliriz:

Python – Fonksiyonu Kullanma

```

1  # Sprite sheet yukle
2  sheet = pygame.image.load("karakter.png").convert_alpha()
3
4  # Her satirdaki animasyonu kes
5  asagi_yuru = frameleri_kes(sheet, 64, 64, satir=0)
6  sola_yuru = frameleri_kes(sheet, 64, 64, satir=1)
7  saga_yuru = frameleri_kes(sheet, 64, 64, satir=2)
8  yukari_yuru = frameleri_kes(sheet, 64, 64, satir=3)
9
10 # asagi_yuru[0] = ilk frame
11 # asagi_yuru[1] = ikinci frame ...

```

29.2.3 convert_alpha() Önemi

Bölüm 17'te `convert_alpha()` fonksiyonunun PNG dosyalarındaki şeffaflık bilgisini koruyarak performansı artırdığını öğrenmiştin. Sprite sheet'lerde bu daha da önemlidir çünkü her frame'in arka planı şeffaf olmalıdır; aksi halde karakter çevresinde kare bir kutu görünür.

⚠ Dikkat: `convert_alpha()` Zamanlaması

`convert_alpha()` fonksiyonunu sprite sheet yüklendikten **hemen sonra** çağır, frame'leri kesmeden **önce**. Böylece tüm alt yüzeyler otomatik olarak doğru piksel formatına sahip olur. Her frame için ayrı ayrı çağırmaya gerek kalmaz.

29.2.4 Alternatif: `blit` ile Kopyalama

Bazı durumlarda `subsurface()` yerine yeni bir `Surface` oluşturup `blit()` ile kopyalamak isteyebilirsin. Bu yöntem orijinal sheet'ten bağımsız bir kopya oluşturur:

Python – `blit` ile Frame Kopyalama

```
1 def frame_kopyala(sheet, x, y, gen, yuk):
2     """Sprite sheet'ten bagimsiz bir frame kopyasi
   ↳ olustur."""
3     frame = pygame.Surface((gen, yuk), pygame.SRCALPHA)
4     frame.blit(sheet, (0, 0), (x, y, gen, yuk))
5     return frame
```

Bu yaklaşım daha fazla bellek kullanır ama orijinal sheet silinse bile frame'ler geçerli kalır. Çoğu durumda `subsurface()` tercih edilir; ancak sprite sheet'i dinamik olarak değiştirmen gerekiyorsa `blit()` kopyalama yöntemini kullan.

29.3 Animasyon Döngüsü

Frame'leri kesmeyi öğrendin; şimdi bu frame'leri sırayla göstererek gerçek bir animasyon oluşturacağız. Animasyonun temel fikri basittir: belirli zaman aralıklarıyla gösterilen frame'i değiştir. Ama bunu doğru yapmak, düşündüğünden biraz daha inceliklidir.

29.3.1 Frame İndeksi ve Zamanlama

Animasyonun en basit hali, bir sayaç (frame indeksi) tutmak ve her güncelleme çevriminde bu sayacı artırmaktır. Ancak bu yaklaşımda animasyon hızı FPS'e bağlı olur: 60'ta çok hızlı, 30'da yavaş oynar. Bunu istemeyiz.

Doğru yöntem, `pygame.time.get_ticks()` ile geçen zamanı ölçmek ve belirli bir süre geçtiğinde frame'i değiştirmektir:

Python – Zamana Bağlı Frame Değiştirme

```
1 import pygame
2
3 class BasitAnimasyon:
4     """Zamana bagli frame animasyonu."""
5
```

```

6   def __init__(self, frameler, frame_suresi=100):
7       """
8       Args:
9           frameler: Frame yuzeylerinin listesi.
10          frame_suresi: Her frame'in gorunme suresi
11                      ↪ (ms).
12          """
13          self.frameler = frameler
14          self.frame_suresi = frame_suresi
15          self.mevcut_frame = 0
16          self.son_guncelleme = pygame.time.get_ticks()
17
18      def guncelle(self):
19          """Zamana gore frame indeksini guncelle."""
20          simdi = pygame.time.get_ticks()
21          gecen = simdi - self.son_guncelleme
22
23          if gecen >= self.frame_suresi:
24              self.mevcut_frame += 1
25              self.mevcut_frame %= len(self.frameler)
26              self.son_guncelleme = simdi
27
28      def gorsel_al(self):
29          """Mevcut frame'in yuzeyini dondur."""
30          return self.frameler[self.mevcut_frame]

```

Bu kodda `frame_suresi` milisaniye cinsindendir. Değeri 100 ms olduğunda saniyede yaklaşık 10 frame gösterilir ki bu çoğu piksel sanat animasyonu için uygun bir hızdır. Dikkat edilmesi gereken nokta, `son_guncelleme` değerinin her frame geçişinde `simdi` ile güncellenmesidir — eğer `gecen` değeriyle artırılırsaydı, geçiş süreleri birikimli olurdu ve uzun vadede daha hassas bir zamanlama sağlardı. Ancak basit animasyonlar için mevcut yaklaşım yeterlidir ve anlaşılması daha kolaydır.

29.3.2 Loop (Döngüsel) Animasyon

Loop animasyon, son frame'e ulaşıldığında başa dönerek sürekli tekrarlanan animasyondur. Yürüme, koşma, bekleme (idle) gibi sürekli hareketler için kullanılır. Yukarıdaki kodda `self.mevcut_frame %= len(self.frameler)` satırı tam olarak bunu yapar: modülo operatörü frame indeksini sıfıra döndürür.

29.3.3 One-Shot (Tek Seferlik) Animasyon

One-shot animasyon ise yalnızca bir kez oynatılır ve son frame'de durur. Patlama, ölüm, saldırı gibi bir kerelik olaylar için idealdir.

Python – One-Shot Animasyon

```

1   class OneShotAnimasyon:
2       """Tek seferlik animasyon (patlama, olum vb.)."""

```

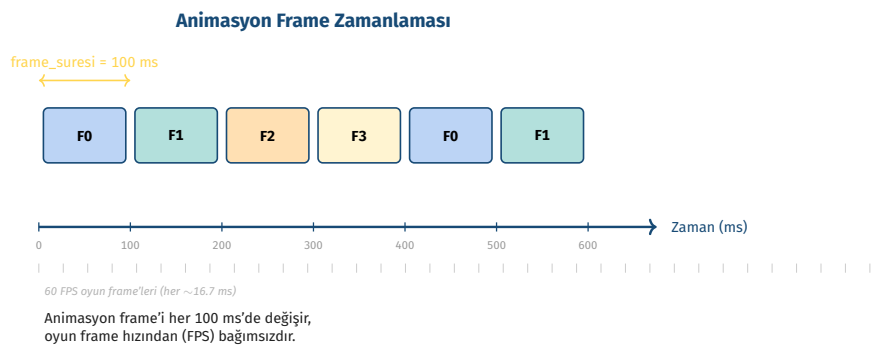
```

3
4 def __init__(self, frameler, frame_suresi=100):
5     self.frameler = frameler
6     self.frame_suresi = frame_suresi
7     self.mevcut_frame = 0
8     self.son_guncelleme = pygame.time.get_ticks()
9     self.bitti = False
10
11 def guncelle(self):
12     """Frame ilet. Son frame'e ulasinca dur."""
13     if self.bitti:
14         return
15
16     simdi = pygame.time.get_ticks()
17     gecen = simdi - self.son_guncelleme
18
19     if gecen >= self.frame_suresi:
20         self.mevcut_frame += 1
21         self.son_guncelleme = simdi
22
23         if self.mevcut_frame >= len(self.frameler):
24             self.mevcut_frame = len(self.frameler) - 1
25             self.bitti = True
26
27 def gorsel_al(self):
28     """Mevcut frame'i dondur."""
29     return self.frameler[self.mevcut_frame]
30
31 def sifirla(self):
32     """Animasyonu bastan baslat."""
33     self.mevcut_frame = 0
34     self.bitti = False
35     self.son_guncelleme = pygame.time.get_ticks()

```

29.3.4 Animasyon Hızı ve FPS İlişkisi

Animasyon hızı ile oyunun FPS değeri birbirinden bağımsız olmalıdır. `get_ticks()` kullanarak bunu sağladık ama kavramı biraz daha açalım.



Şekil 29.2: Animasyon frame zamanlaması: FPS'ten bağımsız frame geçişi

Animasyon hızını ayarlamak için `frame_suresi` parametresini değiştirmek yeterlidir:

- **50 ms:** Hızlı animasyon (saniyede 20 frame) – koşma, saldırı
- **100 ms:** Normal hız (saniyede 10 frame) – yürüme
- **200 ms:** Yavaş animasyon (saniyede 5 frame) – bekleme (idle)
- **500 ms:** Çok yavaş (saniyede 2 frame) – uyku, nefes alma

💡 İpucu: Animasyon Hızı İpucu

Piksel sanat animasyonlarında genellikle 80–150 ms aralığı doğal görünür. Anime/manga tarzı sprite'larda daha yavaş (150–250 ms), aksiyon oyunlarında daha hızlı (50–80 ms) değerler dene. En iyi yol, farklı değerleri deneyerek gözle kontrol etmektir.

29.4 Pratik Uygulama: 4 Yönlü Yürüyen Karakter

Bu konuda öğrendiğin tüm kavramları birleştirerek 4 yönlü yürüyen bir karakter animasyonu oluşturacaksın. Ok tuşlarıyla hareket eden ve yönüne göre doğru animasyonu oynatan bir karakter yapacağız.

29.4.1 Planlama

Karakterimiz 4 yönde yürüyebilecek: aşağı, sola, sağa ve yukarı. Her yön için bir animasyon satırı olacak. Karakter durduğunda ise son baktığı yönün ilk frame'i (idle) gösterilecek.

Kullanacağımız sprite sheet yapısı:

- **Satır 0:** Aşağı yürüme (4 frame)
- **Satır 1:** Sola yürüme (4 frame)
- **Satır 2:** Sağa yürüme (4 frame)
- **Satır 3:** Yukarı yürüme (4 frame)

29.4.2 Karakter Sınıfı

Sınıfımızı adım adım inşa edeceğiz. İlk olarak `__init__` metodu ile sprite sheet yüklemesi ve animasyon frame'lerinin hazırlanmasını görelim:

Python – YuruenKarakter – Kurucu Metot

```
1 class YuruenKarakter(pygame.sprite.Sprite):
2     """4 yonlu animasyonlu karakter."""
3
4     def __init__(self, x, y):
5         super().__init__()
6         # Sprite sheet yukle (veya fallback olustur)
7         self.sheet = self._sheet_yukle()
8         self.frame_gen = 64
9         self.frame_yuk = 64
```

```

10
11     # Her yon için animasyon frame'lerini kes
12     self.animasyonlar = {
13         "asagi": self._satirdan_kes(0),
14         "sol": self._satirdan_kes(1),
15         "sag": self._satirdan_kes(2),
16         "yukari": self._satirdan_kes(3),
17     }
18
19     # Animasyon durumu
20     self.yon = "asagi"
21     self.frame_indeks = 0
22     self.frame_suresi = 120 # ms
23     self.son_guncelleme = pygame.time.get_ticks()
24     self.hareket_ediyor = False
25
26     # Sprite ayarlari
27     self.image = self.animasyonlar[self.yon][0]
28     self.rect = self.image.get_rect(topleft=(x, y))
29     self.hiz = 3

```

Kurucu metotta dikkat edilecek en önemli nokta, `animasyonlar` sözlüğüdür. Her yön için bir liste tutuyoruz ve bu listeler `_satirdan_kes()` ile sprite sheet'ten kesilen frame yüzeylerinden oluşuyor. `frame_suresi` değerini 120 ms olarak ayarladık; bu, saniyede yaklaşık 8 frame gösterilmesi demektir ve piksel sanat yürüme animasyonları için doğal bir hız sağlar. Ayrıca `hareket_ediyor` bayrağı, karakterin durduğunda idle pozuna dönmesi için kullanılacaktır.

Şimdi sprite sheet yükleme ve dosya bulunamadığında kullanılacak fallback mekanizmasını görelim:

Python – Sprite Sheet Yükleme ve Fallback

```

1  def _sheet_yukle(self):
2      """Sprite sheet yukle, yoksa fallback olustur."""
3      try:
4          return pygame.image.load(
5              "assets/images/karakter.png"
6          ).convert_alpha()
7      except (pygame.error, FileNotFoundError):
8          return self._fallback_olustur()
9
10 def _fallback_olustur(self):
11     """Sprite sheet dosyasi yoksa renkli sekiller
12     ↪ olustur."""
13     boyut = (256, 256) # 4x4 grid, 64px frame
14     sheet = pygame.Surface(boyut, pygame.SRCALPHA)
15     renkler = [
16         (50, 150, 255), # asagi - mavi
17         (255, 150, 50), # sol - turuncu
18         (50, 255, 150), # sag - yesil
19         (255, 50, 150), # yukari - pembe

```



```

19 ]
20 for satir in range(4):
21     for sutun in range(4):
22         x = sutun * 64
23         y = satir * 64
24         renk = renkler[satir]
25         # Basit karakter sekli
26         pygame.draw.circle(
27             sheet, renk,
28             (x + 32, y + 24), 16
29         )
30         # Govde
31         pygame.draw.rect(
32             sheet, renk,
33             (x + 20, y + 38, 24, 20)
34         )
35         # Hareket gostergesi (sutuna gore ayak
36         ↪ pozisyonu)
37         offset = (sutun % 2) * 8 - 4
38         pygame.draw.circle(
39             sheet, renk,
40             (x + 26 + offset, y + 60), 5
41         )
42     return sheet

```

`_sheet_yukle()` metodu bir `try/except` bloğu ile dosyayı yüklemeye çalışır; bulamazsa `_fallback_olustur()` ile geometrik şekillerden oluşan yapay bir sprite sheet üretir. Bu tasarım, kodun herhangi bir görsel dosya olmadan da çalışmasını sağlar — öğrenci henüz kendi sprite sheet'ini hazırlamamış olsa bile programı test edebilir. Fallback'te her yön için farklı renk kullanıyoruz, böylece animasyonun doğru çalışıp çalışmadığını görsel olarak ayırt etmek kolaylaşır.

Frame kesme metodu, sprite sheet'ten belirli bir satırın tüm frame'lerini çıkarır:

Python – Sprite Sheet'ten Frame Kesme

```

1 def _satirdan_kes(self, satir):
2     """Belirli satirdan tum frame'leri kes."""
3     frameler = []
4     sutun_sayisi = self.sheet.get_width() //
5     ↪ self.frame_gen
6     for s in range(sutun_sayisi):
7         rect = pygame.Rect(
8             s * self.frame_gen,
9             satir * self.frame_yuk,
10            self.frame_gen,
11            self.frame_yuk
12        )
13        frameler.append(self.sheet.subsurface(rect))
14    return frameler

```

`subsurface()` metodu PyGame'in güçlü bir özelliğidir: orijinal yüzeyden

belirli bir dikdörtgen alanı paylaşımlı olarak keser. Bu sayede yeni bir piksel kopyası oluşturmak yerine, aynı bellek bölgesine referans verilir ve hem hız hem de bellek açısından verimli çalışılır. Sütun sayısını sheet genişliğinden otomatik hesapladığımız için farklı boyutlardaki sprite sheet'lerle de uyumlu çalışır.

Son olarak, her karede çağrılan `update()` metodu hem klavye girdisini işler hem de animasyon frame'ini günceller:

Python – Hareket ve Animasyon Güncelleme

```

1 def update(self):
2     """Klavye girdisi ve animasyon guncelleme."""
3     tuslar = pygame.key.get_pressed()
4     self.hareket_ediyor = False
5
6     if tuslar[pygame.K_DOWN]:
7         self.rect.y += self.hiz
8         self.yon = "asagi"
9         self.hareket_ediyor = True
10    elif tuslar[pygame.K_UP]:
11        self.rect.y -= self.hiz
12        self.yon = "yukari"
13        self.hareket_ediyor = True
14    elif tuslar[pygame.K_LEFT]:
15        self.rect.x -= self.hiz
16        self.yon = "sol"
17        self.hareket_ediyor = True
18    elif tuslar[pygame.K_RIGHT]:
19        self.rect.x += self.hiz
20        self.yon = "sag"
21        self.hareket_ediyor = True
22
23    # Animasyon guncelle
24    if self.hareket_ediyor:
25        simdi = pygame.time.get_ticks()
26        if simdi - self.son_guncelleme >=
27            ↪ self.frame_suresi:
28                self.frame_indeks += 1
29                self.frame_indeks %= len(
30                    self.animasyonlar[self.yon]
31                )
32                self.son_guncelleme = simdi
33    else:
34        self.frame_indeks = 0 # Duruyorsa ilk frame
35
36    self.image = self.animasyonlar[self.yon][
37        self.frame_indeks
38    ]

```

`update()` metodu iki aşamalı çalışır. İlk aşamada `pygame.key.get\_pressed()` ile anlık tuş durumu sorgulanır ve karakterin konumu ile yönü güncellenir. `elif` yapısı sayesinde aynı anda yalnızca bir yönde hareket edilir; bu, çarpaz hareketin önüne geçer. İkinci aşamada zamana dayalı animasyon güncel-

lenir: `pygame.time.get_ticks()` ile milisaniye cinsinden geçen süre ölçülerek FPS'ten bağımsız bir animasyon hızı sağlanır. Karakter durduğunda `frame_indeks` sıfıra döner ve son baktığı yönün ilk frame'i (idle pozu) gösterilir.

29.4.3 Ana Oyun Döngüsü

Karakter sınıfını kullanmak için basit bir oyun döngüsü yeterlidir:

Python – Yürüyen Karakter – Ana Döngü

```

1  def main():
2      pygame.init()
3      ekran = pygame.display.set_mode((640, 480))
4      pygame.display.set_caption("4 Yönlü Yurume
      ↪ Animasyonu")
5      saat = pygame.time.Clock()
6
7      # Karakter olustur
8      karakter = YuruenKarakter(288, 208)
9      tum_spritelar = pygame.sprite.Group(karakter)
10
11     calistir = True
12     while calistir:
13         saat.tick(60)
14         for olay in pygame.event.get():
15             if olay.type == pygame.QUIT:
16                 calistir = False
17
18         tum_spritelar.update()
19         ekran.fill((40, 40, 60))
20         tum_spritelar.draw(ekran)
21         pygame.display.flip()
22
23     pygame.quit()

```

Bu örnekte dikkat edilmesi gereken birkaç tasarım kararı var. Birincisi, animasyon mantığı `update()` metodunun içinde; böylece `tum_spritelar.update()` çağrıldığında karakter otomatik olarak güncellenir — ayrı bir fonksiyon çağırmaya gerek kalmaz. İkincisi, hareket etmediğinde `frame_indeks` sıfıra döner; bu, o yönün idle pozunu (ilk frame) gösterir ve karakter doğal bir şekilde durur. Üçüncüsü, yön değişikliği anında animasyon baştan başlamaz, mevcut frame'den devam eder; bu, yön geçişlerinde görsel bir sıçrama yaşanmasını önleyerek daha akıcı bir deneyim sunar.

● Bilgi: Bir Sonraki Adım

Bu ünite de frame animasyonunun temellerini öğrendin. Bir sonraki ünite de tüm bu mantığı genel amaçlı bir `AnimatedSprite` sınıfında kapsülleyecek, state machine ile birden fazla animasyon durumu yöneteceğiz.

29.5 Sonuç

Bu ünite de oyun animasyonunun temellerini öğrendin. Sprite sheet kavramından frame kesmeye, zamana bağlı animasyon döngüsünden 4 yönlü yürüyen karakter uygulamasına kadar birçok önemli konuyu ele aldık.

29.5.1 Anahtar Kavramlar

Tablo 29.2: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
Sprite Sheet	Tüm animasyon karelerinin tek bir görselde düzenli şekilde sıralanmış hali
Frame	Animasyonun tek bir karesi; karakterin belirli andaki pozunu
Grid	Sprite sheet'teki satır-sütun düzeni
subsurface()	Mevcut yüzeyden bellek paylaşımı alt bölge çıkaran metot
Loop Animasyon	Son frame'den başa dönerek sürekli tekrarlanan animasyon
One-Shot Animasyon	Yalnızca bir kez oynatılıp duran animasyon
Frame Süresi	Her frame'in ekranda kalma süresi (milisaniye)
convert_alpha()	PNG şeffaflığını koruyarak performans artıran dönüşüm

29.5.2 Önemli Komutlar

Tablo 29.3: Ünite 1 – Temel komutlar

Komut	Açıklama
<code>image.load(dosya)</code>	Görsel dosyasını yükler
<code>convert_alpha()</code>	Şeffaflıklı dönüşüm uygular
<code>Surface.subsurface(rect)</code>	Alt bölge yüzeyi döndürür
<code>Surface.blit(kaynak, konum, alan)</code>	Kaynak yüzeyden belirli alanı kopyalar
<code>pygame.time.get_ticks()</code>	Başlangıçtan bu yana geçen ms
<code>Rect(x, y, gen, yuk)</code>	Dikdörtgen alan tanımlar

29.5.3 Sıradaki Adım

Bir sonraki ünite de bu temel yapıyı bir adım öteye taşıyacağız. Genel amaçlı bir `AnimatedSprite` sınıfı tasarlayacak, state machine ile birden fazla animasyon durumunu (idle, walk, attack, death) yönetecek ve hız kontrolü, ping-pong modu gibi gelişmiş özellikler ekleyeceğiz. Ardından Uzay Kaçışı projesine ilk animasyonları ekleyeceğiz.

Animasyon Sınıfı Tasarımı

“İyi bir tasarım, karmaşıklığı basitlik arkasına gizler.”

— JONY IVE

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `AnimatedSprite` sınıfı tasarımı (`Sprite` alt sınıfı olarak)
- State machine ile animasyon durumları: idle, walk, attack, death
- Hız kontrolü, ping-pong modu, pause/resume özellikleri
- Uzay Kaçışı v4'e temel animasyonların eklenmesi

30.1 AnimatedSprite Sınıfı

Önceki ünite de frame animasyonunun temellerini öğrendin ve `YuruenKarakter` sınıfında animasyon mantığını doğrudan sprite'ın içine gömdük. Bu yaklaşım çalışır ama tekrar kullanılabilir değildir: her yeni karakter için aynı kodu tekrar yazmak gerekir. Şimdi daha iyi bir tasarım yapacağız: `AnimatedSprite` adında genel amaçlı bir sınıf oluşturacağız.

30.1.1 Tasarım Hedefleri

`AnimatedSprite` sınıfı şu özelliklere sahip olmalıdır:

1. `pygame.sprite.Sprite` alt sınıfı olmalı (grup uyumluluğu)
2. Birden fazla animasyon durumunu desteklemeli (idle, walk, attack...)
3. Animasyon hızı ayarlanabilmeli
4. Loop ve one-shot modlarını desteklemeli
5. Mevcut sprite sistemiyle uyumlu çalışmalı

30.1.2 Sınıf Yapısı

Sınıfın kurucu metoduyla başlayalım. Burada animasyon verilerini saklayacak yapılar ve PyGame sprite sistemiyle uyumlu çalışması için gerekli `image/rect` nitelikleri hazırlanıyor:

Python – AnimatedSprite – Kurucu Metot

```

1  class AnimatedSprite(pygame.sprite.Sprite):
2      """Genel amacli animasyonlu sprite sinifi.
3
4      Birden fazla animasyon durumunu (state) yönetir.
5      pygame.sprite.Sprite alt sinifi oldugu için
6      sprite gruplariyle uyumlu calisir.
7      """
8
9      def __init__(self, x, y):
10         super().__init__()
11         # Animasyon verileri
12         # {"durum_adi": [frame1, frame2, ...]}
13         self.animasyonlar = {}
14         self.mevcut_durum = None
15         self.frame_indeks = 0
16         self.frame_suresi = 100 # ms
17         self.son_guncelleme = pygame.time.get_ticks()
18
19         # Animasyon modu
20         self.dongu = True # True = loop, False =
21         ↪ one-shot
22         self.bitti = False # One-shot bitti mi?
23
24         # Sprite gereksinimleri
25         self.image = pygame.Surface((1, 1))
26         self.rect = self.image.get_rect(topleft=(x, y))

```

`animasyonlar` sözlüğü, sınıfın omurgasıdır: her durum adı (“idle”, “walk” vb.) bir frame listesi ve o duruma özel süre bilgisiyle eşlenir. Başlangıçta 1x1 piksellik geçici bir yüzey atanıyor çünkü `pygame.sprite.Sprite` alt sınıfları `image` ve `rect` niteliklerine sahip olmak zorundadır. İlk animasyon eklendiğinde bu geçici yüzey gerçek frame ile değiştirilecektir.

Şimdi animasyon ekleme ve durum değiştirme metotlarını görelim. Bu ikili, sınıfın dış dünyaya sunduğu temel arayüzü oluşturur:

Python – Animasyon Ekleme ve Durum Değiştirme

```

1  def animasyon_ekle(self, ad, frameler, frame_suresi=None):
2      """Yeni bir animasyon durumu ekle.
3
4      Args:
5          ad: Durum adi ("idle", "walk" vb.).
6          frameler: Frame yuzeylerinin listesi.
7          frame_suresi: Bu durum için özel süre (ms).

```

```

8                                     None ise varsayılan kullanılır.
9                                     """
10 self.animasyonlar[ad] = {
11     "frame": frame,
12     "sure": frame_suresi or self.frame_suresi,
13 }
14 # İlk eklenen animasyonu aktif yap
15 if self.mevcut_durum is None:
16     self.durum_degistir(ad)
17
18 def durum_degistir(self, yeni_durum, dongu=True):
19     """Animasyon durumunu degistir.
20
21     Args:
22         yeni_durum: Hedef durum adi.
23         dongu: True = loop, False = one-shot.
24     """
25     if yeni_durum not in self.animasyonlar:
26         return
27
28     if yeni_durum != self.mevcut_durum:
29         self.mevcut_durum = yeni_durum
30         self.frame_indeks = 0
31         self.bitti = False
32         self.dongu = dongu
33         self.son_guncelleme = pygame.time.get_ticks()
34         self._gorsel_guncelle()

```

`animasyon_ekle()` metodu her duruma özel bir frame süresi tanımlamaya olanak verir: idle animasyonu 200 ms ile yavaş, saldırı animasyonu 80 ms ile hızlı oynatılabilir. İlk eklenen animasyon otomatik olarak aktif yapılır, böylece sınıf oluşturulduktan sonra hemen kullanılabilir duruma gelir. `durum_degistir()` metodundaki `yeni_durum != self.mevcut_durum` kontrolü kritiktir: bu kontrol olmasa, her frame'de aynı durum tekrar atanır ve animasyon sürekli baştan başlar — görsel olarak ilk frame'de takılıp kalma problemi oluşur.

Son olarak, her frame'de çağrılan `update()` ve yardımcı `_gorsel_guncelle()` metotları:

Python – Animasyon Güncelleme

```

1 def _gorsel_guncelle(self):
2     """image ve rect'i mevcut frame'e gore guncelle."""
3     anim = self.animasyonlar[self.mevcut_durum]
4     eski_merkez = self.rect.center
5     self.image = anim["frame"][self.frame_indeks]
6     self.rect = self.image.get_rect(center=eski_merkez)
7
8 def update(self):
9     """Her frame'de cagrilan guncelleme metodu."""
10    if self.mevcut_durum is None or self.bitti:

```

```

11         return
12
13     anim = self.animasyonlar[self.mevcut_durum]
14     simdi = pygame.time.get_ticks()
15     gecen = simdi - self.son_guncelleme
16
17     if gecen >= anim["sure"]:
18         self.frame_indeks += 1
19         self.son_guncelleme = simdi
20
21     toplam = len(anim["frameler"])
22     if self.frame_indeks >= toplam:
23         if self.dongu:
24             self.frame_indeks = 0
25         else:
26             self.frame_indeks = toplam - 1
27             self.bitti = True
28
29     self._gorsel_guncelle()

```

`update()` metodu her karede çağrılır ve iki kritik işi yapar: zamanlayıcıyı kontrol ederek sıradaki kareye geçişi belirler ve mevcut animasyon karesini `self.image`'a atar. `pygame.time.get_ticks()` ile milisaniye cinsinden geçen süreyi ölçerek FPS'ten bağımsız bir animasyon hızı sağlıyoruz. Loop modunda son frame'e ulaşıldığında indeks sıfıra döner; one-shot modunda ise son frame'de kalınır ve `self.bitti` bayrağı `True` yapılır. `_gorsel_guncelle()` metodunda `eski_merkez` kullanarak, farklı boyutlardaki frame'ler arasında geçişte sprite'ın merkez konumunu koruyoruz.

Şimdi bu sınıfı bir oyun döngüsünde nasıl kullanacağımızı görelim:

Python – AnimatedSprite Kullanımı

```

1  # Sprite sheet yukle ve frame'leri kes
2  sheet = pygame.image.load("karakter.png").convert_alpha()
3
4  # Frame kesme fonksiyonu (Unite 1'den)
5  idle_frameler = frameleri_kes(sheet, 64, 64, satir=0)
6  yuru_frameler = frameleri_kes(sheet, 64, 64, satir=1)
7  saldir_frameler = frameleri_kes(sheet, 64, 64, satir=2)
8
9  # AnimatedSprite olustur
10 karakter = AnimatedSprite(100, 200)
11 karakter.animasyon_ekle("idle", idle_frameler, 200)
12 karakter.animasyon_ekle("walk", yuru_frameler, 100)
13 karakter.animasyon_ekle("attack", saldir_frameler, 80)
14
15 # Durum degistir
16 karakter.durum_degistir("walk") # Yurume animasyonu basla
17 karakter.durum_degistir("attack", dongu=False) # Tek
    ↪ seferlik

```


Bu tasarımda dikkat edilmesi gereken önemli bir detay var: `durum_degistir()` metodu aynı duruma geçişi engeller (`yeni_durum != self.mevcut_durum` kontrolü). Bu, animasyonun her frame’de baştan başlamasını önler. Eğer bir animasyonu zorla yeniden başlatmak istersen, önce farklı bir duruma geçip sonra geri dönebilir ya da `frame_indeks`’i elle sıfırlayabilirsin. Ayrıca `animasyon_ekle()` ile her duruma farklı `frame_suresi` atanabildiğine dikkat et: idle için 200 ms gibi yavaş bir değer doğal görünürken, saldırı için 80 ms gibi hızlı bir değer daha dinamik bir hissiyat yaratır.

İpucu: Merkez Korunması

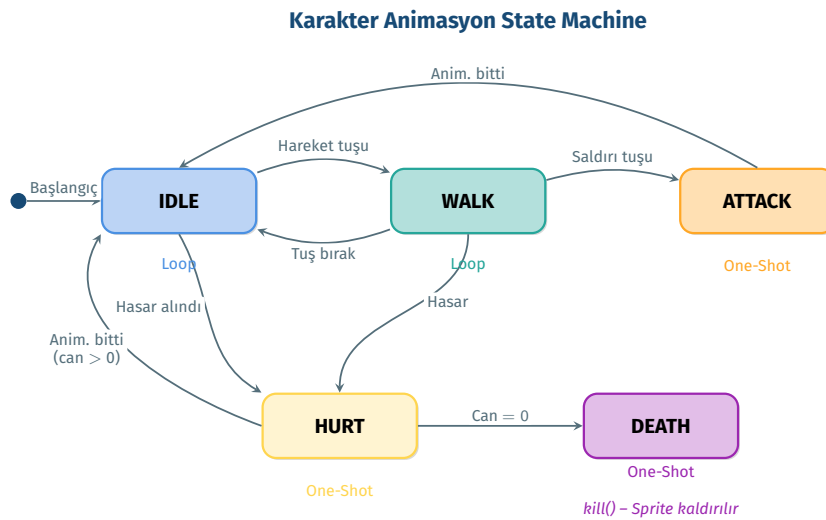
`_gorsel_guncelle()` metodunda `eski_merkez` kullanarak frame boyutu değişse bile sprite’ın merkez konumunu koruyoruz. Bu, farklı boyutlardaki frame’ler arasında geçişte sprite’ın zıplamasını önler.

30.2 Animasyon Durumları (State Machine)

`AnimatedSprite` sınıfı birden fazla animasyon durumunu saklayabilir. Peki bu durumlar arasındaki geçişleri nasıl yönetiriz? Cevap, yazılım mühendisliğinde sıkça kullanılan *state machine* (durum makinesi) desenidir.

30.2.1 State Machine Nedir?

Bir *state machine*, bir nesnenin olabileceği durumları ve bu durumlar arasındaki geçiş kurallarını tanımlayan bir modeldir. Oyun animasyonlarında karakter her an belirli bir durumda bulunur (idle, walk, attack, death) ve belirli olaylar durumlar arası geçişi tetikler.



Şekil 30.1: Karakter animasyon state machine diyagramı

30.2.2 Oyun Karakteri State Machine

Tipik bir oyun karakterinin animasyon durumları şunlardır:

Tablo 30.1: Animasyon durumları ve geçiş koşulları

Durum	Mod	Geçiş Koşulu
idle	Loop	Hareket tuşuna basılınca → walk
walk	Loop	Tuş bırakılınca → idle, saldırı → attack
attack	One-Shot	Animasyon bitince → idle
hurt	One-Shot	Animasyon bitince → idle (can varsa)
death	One-Shot	Animasyon bitince → sprite kaldırılır

30.2.3 State Machine Uygulaması

`AnimatedSprite` sınıfını genişleterek state machine mantığı ekleyelim. İlk olarak kurucu metot ve animasyonların yüklenme mantığını görelim:

Python – Karakter – Kurucu ve Animasyon Yükleme

```

1  class Karakter(AnimatedSprite):
2      """State machine ile animasyonlu karakter."""
3
4      def __init__(self, x, y):
5          super().__init__(x, y)
6          self.hiz = 3
7          self._animasyonlari_yukle()
8
9      def _animasyonlari_yukle(self):
10         """Tum animasyonlari yukle ve ekle."""
11         sheet = self._sheet_yukle_veya_olustur()
12
13         self.animasyon_ekle(
14             "idle", frameleri_kes(sheet, 64, 64, 0), 200
15         )
16         self.animasyon_ekle(
17             "walk", frameleri_kes(sheet, 64, 64, 1), 100
18         )
19         self.animasyon_ekle(
20             "attack", frameleri_kes(sheet, 64, 64, 2), 80
21         )
22         self.animasyon_ekle(
23             "death", frameleri_kes(sheet, 64, 64, 3), 150
24         )

```

`Karakter` sınıfı, `AnimatedSprite`'in alt sınıfı olarak animasyon altyapısını miras alır ve üzerine state machine mantığı ekler. Her animasyon durumuna

farklı bir `frame_suresi` atandığına dikkat et: idle yavaş (200 ms), saldırı hızlı (80 ms), ölüm orta hızda (150 ms). Bu farklar, her durumun doğal ritmini yansıtır. `frameleri_kes()` fonksiyonu daha önceki üniteden bildiğimiz sprite sheet kesme işlemini yapar.

Sınıfın kalbi olan `update()` metodu ve durum geçiş mantığını ayrı bir blokta inceleyelim:

Python – State Machine Güncelleme Mantığı

```

1 def update(self):
2     """Durum makinesini isle, sonra animasyonu
3     ↪ guncelle."""
4     self._durum_guncelle()
5     super().update() # Animasyon frame ilerlemesi
6
7     # One-shot animasyon bittiyse gecis yap
8     if self.bitti:
9         self._gecis_kontrol()
10
11 def _durum_guncelle(self):
12     """Girdi ve oyun durumuna gore state gecisi."""
13     tuslar = pygame.key.get_pressed()
14     hareket = (
15         tuslar[pygame.K_LEFT] or tuslar[pygame.K_RIGHT]
16         or tuslar[pygame.K_UP] or tuslar[pygame.K_DOWN]
17     )
18
19     # Olum durumundaysa baska duruma gecilemez
20     if self.mevcut_durum == "death":
21         return
22
23     # Saldiri sirasinda hareket engellenir
24     if self.mevcut_durum == "attack" and not self.bitti:
25         return
26
27     if hareket:
28         self.durum_degistir("walk")
29         self._hareket_et(tuslar)
30     else:
31         self.durum_degistir("idle")

```

`update()` metodu üç adımlı bir düzen izler: önce durum geçişini kontrol et, sonra animasyon frame'ini ilerlet (`super().update()`), en son one-shot animasyon bittiyse otomatik geçiş yap. Bu sıralama kritiktir — durum geçişi animasyon güncellemesinden önce yapılmazsa, bir frame gecikmeyle yanlış animasyon gösterilebilir. `_durum_guncelle()` metodundaki erken dönüş kontrolleri, state machine'in öncelik hiyerarşisini uygular: ölüm durumunda hiçbir şey yapılmaz, saldırı sırasında hareket engellenir.

Hareket, geçiş kontrolleri ve dışarıdan tetiklenecek eylem metotlarını da görelim:

Python – Hareket ve Durum Geçiş Eylemleri

```

1 def _hareket_et(self, tuslar):
2     """Tus girdisine gore hareket."""
3     if tuslar[pygame.K_LEFT]:
4         self.rect.x -= self.hiz
5     if tuslar[pygame.K_RIGHT]:
6         self.rect.x += self.hiz
7     if tuslar[pygame.K_UP]:
8         self.rect.y -= self.hiz
9     if tuslar[pygame.K_DOWN]:
10        self.rect.y += self.hiz
11
12 def _gecis_kontrol(self):
13     """One-shot animasyon bittikten sonra gecis."""
14     if self.mevcut_durum == "attack":
15         self.durum_degistir("idle")
16     elif self.mevcut_durum == "death":
17         self.kill() # Sprite'i gruptan kaldır
18
19 def saldiri(self):
20     """Saldiri animasyonunu baslat."""
21     if self.mevcut_durum != "death":
22         self.durum_degistir("attack", dongu=False)
23
24 def hasar_al(self, miktar):
25     """Hasar al, can sifirsa olum animasyonu baslat."""
26     self.can -= miktar
27     if self.can <= 0:
28         self.durum_degistir("death", dongu=False)

```

`_hareket_et()` metodunda `elif` yerine `if` kullanıldığına dikkat et: bu sayede çapraz hareket mümkün olur (aynı anda yukarı ve sağa gidilebilir). `_gecis_kontrol()` metodu one-shot animasyonlar tamamlandığında otomatik geçişi sağlar: saldırı bitince `idle`'a döner, ölüm bitince `kill()` ile sprite gruptan kaldırılır. `saldiri()` ve `hasar_al()` metotları ise dışarıdan tetiklenecek eylemlerdir; `dongu=False` parametresi ile one-shot modda çalışmalarını sağlarız.

30.2.4 Geçiş Kurallarının Önemi

State machine'in en kritik kısmı geçiş kurallarıdır. Yanlış kurallar garip davranışlara yol açar:

⚠ Dikkat: Yaygın Hatalar

- Ölüm durumundayken hareket edebilmek (kontrol eksikliği)
- Saldırı animasyonu tamamlanmadan `idle`'a geçmek (kesinti)
- Her frame'de `durum_degistir` çağırarak animasyonu sürekli baştan başlatmak (aynı durum kontrolü şart)

İpucu: Öncelik Sırası

Durumları öncelik sırasına koy: death > hurt > attack > walk > idle. Yüksek öncelikli bir durumdayken düşük öncelikli duruma geçişi engelle.

30.3 Hız Kontrolü ve Gelişmiş Modlar

`AnimatedSprite` sınıfımız temel loop ve one-shot modlarını destekliyor. Şimdi daha gelişmiş özellikler ekleyeceğiz: animasyon hızını dinamik olarak değiştirme, ping-pong modu ve pause/resume işlevleri.

30.3.1 Hız Parametresi

Animasyon hızını çalışma zamanında değiştirmek, oyun mekaniğiyle animasyonu senkronize etmek için önemlidir. Örneğin karakter hızlı koşarken animasyon da hızlanmalı, yavaşlarken yavaşlamalıdır.

Python – Dinamik Hız Ayarlama

```

1 def hiz_ayarla(self, carpan):
2     """Animasyon hizini carpanla degistir.
3
4     Args:
5         carpan: Hiz carpani. 1.0 = normal, 2.0 = 2x hizli,
6             0.5 = yarim hiz.
7     """
8     if self.mevcut_durum is None:
9         return
10
11     anim = self.animasyonlar[self.mevcut_durum]
12     orijinal = anim.get("orijinal_sure", anim["sure"])
13     anim["orijinal_sure"] = orijinal
14     anim["sure"] = max(10, int(orijinal / carpan))

```

`hiz_ayarla()` metodundaki `orijinal_sure` kaydı önemlidir: `carpan` her değiştiğinde orijinal değerden hesaplama yapılır, böylece tekrarlanan çağrılarda hız yuvarlama hatalarıyla giderek sapma olmaz. `max(10, ...)` kontrolü ise frame süresinin 10 ms'nin altına düşmesini engeller; aksi halde saniyede 100'den fazla frame gösterilir ki bu hem görsel olarak anlamsız hem de performans açısından pahalı olur.

Bu yöntemi kullanarak karakter hızıyla animasyon hızını eşleyebilirsiniz:

Python – Hız-Animasyon Senkronizasyonu

```

1 # Karakterin hareket hizina gore animasyon hizi
2 hareket_hizi = 5 # piksel/frame
3 maks_hiz = 8
4
5 # Hiz oranini hesapla (0.5 - 2.0 arasi)

```

```

6 oran = hareket_hizi / maks_hiz * 2.0
7 oran = max(0.5, min(2.0, oran))
8
9 karakter.hiz_ayarla(oran)

```

30.3.2 Ping-Pong Modu

Ping-pong animasyonda frame'ler önce ileri, sonra geri sayılarak döngü oluşturur. Bu mod, nefes alma, sallanma veya ışık parıldaması gibi yumuşak, ritmik hareketler için idealdir.

Normal loop: 0, 1, 2, 3, 0, 1, 2, 3, ...

Ping-pong: 0, 1, 2, 3, 2, 1, 0, 1, 2, 3, ...

Python – Ping-Pong Desteği

```

1 class AnimatedSpritePP(AnimatedSprite):
2     """Ping-pong modunu destekleyen AnimatedSprite."""
3
4     def __init__(self, x, y):
5         super().__init__(x, y)
6         self.ping_pong = False
7         self._ileri = True # Yon: True=ileri, False=geri
8
9     def durum_degistir(self, yeni_durum, dongu=True,
10                        ping_pong=False):
11         """Ping-pong destekli durum degisimi."""
12         super().durum_degistir(yeni_durum, dongu)
13         self.ping_pong = ping_pong
14         self._ileri = True
15
16     def update(self):
17         """Ping-pong destekli guncelleme."""
18         if self.mevcut_durum is None or self.bitti:
19             return
20
21         anim = self.animasyonlar[self.mevcut_durum]
22         simdi = pygame.time.get_ticks()
23         gecen = simdi - self.son_guncelleme
24
25         if gecen >= anim["sure"]:
26             self.son_guncelleme = simdi
27             toplam = len(anim["frameler"])
28
29             if self.ping_pong:
30                 if self._ileri:
31                     self.frame_indeks += 1
32                     if self.frame_indeks >= toplam - 1:
33                         self._ileri = False
34                 else:
35                     self.frame_indeks -= 1

```

```

36         if self.frame_indeks <= 0:
37             self._ileri = True
38     else:
39         self.frame_indeks += 1
40         if self.frame_indeks >= toplam:
41             if self.dongu:
42                 self.frame_indeks = 0
43             else:
44                 self.frame_indeks = toplam - 1
45                 self.bitti = True
46
47     self._gorssel_guncelle()

```

30.3.3 Pause ve Resume

Animasyonu duraklatıp devam ettirmek, menü açıldığında veya oyun duraklatıldığında kullanışlıdır.

Python – Pause/Resume Özellikleri

```

1  def duraklat(self):
2      """Animasyonu duraklat."""
3      self._duraklatildi = True
4
5  def devam_et(self):
6      """Duraklatılmış animasyonu devam ettir."""
7      self._duraklatildi = False
8      self.son_guncelleme = pygame.time.get_ticks()
9
10 def update(self):
11     """Duraklama destekli guncelleme."""
12     if getattr(self, '_duraklatildi', False):
13         return
14     # Normal guncelleme devam eder...

```

`devam_et()` metodunda `son_guncelleme`'yi `pygame.time.get_ticks()` ile sıfırlamamız önemlidir. Aksi halde duraklatma süresinde biriken zaman farkı nedeniyle animasyon birkaç frame birden atlayabilir — örneğin 5 saniyelik bir duraklatmadan sonra 50 frame'lik bir atlama gerçekleşir ki bu, animasyonun birden son frame'e sıçramasına neden olur.

● Bilgi: Oyun Duraklatma

Animasyonu duraklatmak ile oyunu duraklatmak farklı şeylerdir. Oyun duraklatıldığında tüm sprite'ların `update()` metodu çağrılmaz. Ama belirli bir sprite'ın animasyonunu duraklatıp diğerlerini çalıştırmak istersen (örneğin donmuş bir düşman), sprite düzeyinde duraklama gerekir.

30.4 Uzay Kaçışı v4 – Temel Animasyonlar

Bölüm 28’nda Uzay Kaçışı v3’te ses entegrasyonunu tamamlamıştık. Şimdi v4’e geçiş yaparak `AnimatedSprite` sınıfımızı projeye entegre edeceğiz. Bu konuda temel animasyonları ekleyeceğiz; görsel efektler (parçacık, patlama, screen shake) ise Ünite 4’te tamamlanacak.

30.4.1 v3’ten v4’e Geçiş

v3’te tüm sprite’lar sabit görsellerle çalışıyordu. v4’te şu animasyonları ekleyeceğiz:

- **Oyuncu gemisi:** Motor ateşi titreşim animasyonu (idle)
- **Düşman gemileri:** Uçuş sırasında hafif salınım
- **Mermi:** Parıldama (blink) efekti
- **Bonus:** Dönerek düşme animasyonu

30.4.2 Animasyonlu Oyuncu

Oyuncu gemisine basit bir motor ateşi animasyonu ekleyelim. Geometrik şekillerle (fallback) 3 frame’lik bir titreşim animasyonu oluşturacağız:

Python – Animasyonlu Oyuncu Gemisi

```

1  class OyuncuV4(AnimatedSprite):
2      """Animasyonlu oyuncu gemisi (v4)."""
3
4      def __init__(self):
5          super().__init__(GENISLIK // 2, YUKSEKLIK - 60)
6          self.hiz = OYUNCU_HIZ
7          self.can = OYUNCU_CAN
8          self.sinir = pygame.Rect(0, 0, GENISLIK,
9              ↪ YUKSEKLIK)
9          self.son_ates_zamani = 0
10
11         # Motor animasyonu olustur (3 frame)
12         frameler = self._motor_frameleri()
13         self.animasyon_ekle("idle", frameler, 120)
14
15         def _motor_frameleri(self):
16             """Motor atesi ile 3 frame olustur."""
17             frameler = []
18             alev_yukseklikleri = [8, 14, 10]
19
20             for alev_y in alev_yukseklikleri:
21                 yuzey = pygame.Surface(
22                     (50, 50), pygame.SRCALPHA
23                 )
24                 # Gemi govdesi (ucgen)
25                 govde = [(25, 2), (46, 38), (4, 38)]
26                 pygame.draw.polygon(yuzey, ACIK_MAVI, govde)

```



```

27         pygame.draw.polygon(yuzey, BEYAZ, govde, 2)
28         # Kokpit
29         pygame.draw.circle(
30             yuzey, SARI, (25, 20), 4
31         )
32         # Motor atesi (titreyen)
33         pygame.draw.polygon(
34             yuzey, TURUNCU,
35             [(15, 40), (25, 40 + alev_y), (35, 40)]
36         )
37         framekler.append(yuzey)
38
39     return framekler
40
41     def update(self):
42         """Hareket + animasyon guncelleme."""
43         tuslar = pygame.key.get_pressed()
44         if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
45             self.rect.x -= self.hiz
46         if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
47             self.rect.x += self.hiz
48         if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
49             self.rect.y -= self.hiz
50         if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
51             self.rect.y += self.hiz
52         self.rect.clamp_ip(self.sinir)
53
54         # Animasyonu guncelle
55         super().update()
56
57     def ates_edebilir_mi(self):
58         """Mermi bekleme suresi kontrolu."""
59         simdi = pygame.time.get_ticks()
60         if simdi - self.son_ates_zamani > MERMI_BEKLEME:
61             self.son_ates_zamani = simdi
62             return True
63         return False

```

30.4.3 Animasyonlu Bonus

Bonus nesnesine dönerek düşme animasyonu ekleyelim:

Python – Animasyonlu Bonus

```

1 class BonusV4(AnimatedSprite):
2     """Donerek dusen bonus nesnesi (v4)."""
3
4     def __init__(self, x, y):
5         super().__init__(x, y)
6         self.hiz_y = 2
7

```

```

8         # 4 frame'lik donus animasyonu
9         frameler = self._donus_frameleri()
10        self.animasyon_ekle("spin", frameler, 150)
11
12    def _donus_frameleri(self):
13        """4 acidan bonus gorunumu."""
14        frameler = []
15        genislikler = [20, 14, 20, 14] # Perspektif
16        ↪ etkisi
17
18        for gen in genislikler:
19            yuzey = pygame.Surface(
20                (20, 20), pygame.SRCALPHA
21            )
22            offset = (20 - gen) // 2
23            pygame.draw.ellipse(
24                yuzey, YESIL,
25                (offset, 2, gen, 16)
26            )
27            pygame.draw.ellipse(
28                yuzey, BEYAZ,
29                (offset + 3, 5, gen - 6, 10)
30            )
31            frameler.append(yuzey)
32
33        return frameler
34
35    def update(self):
36        """Asagi hareket + donus animasyonu."""
37        self.rect.y += self.hiz_y
38        if self.rect.top > YUKSEKLIK:
39            self.kill()
40        super().update()

```

30.4.4 Entegrasyon Notları

v4'te yapılan değişiklikler v3 koduyla geriye dönük uyumludur çünkü `AnimatedSprite`, `pygame.sprite.Sprite`'ın tüm özelliklerini miras alır. Sprite grupları, çarpışma algılama ve çizim işlemleri aynen çalışmaya devam eder.

● Bilgi: v4 Tamamlanması

Bu konuda temel animasyonları ekledik. Uzay Kaçışı v4'ün tam versiyonu (parçacık efektleri, patlama animasyonları, screen shake) Ünite 4'te tamamlanacak. Orada tüm görsel efektleri birleştirerek projenin animasyon aşamasını bitireceksin.

30.5 Sonuç

Bu ünite de animasyon sistemini profesyonel düzeye taşıdın. Genel amaçlı `AnimatedSprite` sınıfı tasarladın, state machine ile birden fazla animasyon durumunu yönettin, ping-pong ve pause/resume gibi gelişmiş modları uyguladın. Son olarak Uzak Kaçışı v4'e temel animasyonları ekledin.

30.5.1 Anahtar Kavramlar

Tablo 30.2: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
AnimatedSprite	Genel amaçlı animasyonlu sprite sınıfı
State Machine	Durumlar ve geçiş kurallarından oluşan model
Durum (State)	Karakterin anlık animasyon modu (idle, walk, attack, death)
Geçiş (Transition)	Bir durumdan diğerine koşullu geçiş
Ping-Pong	Frame'lerin ileri-geri sırayla oynatılması
Pause/Resume	Animasyonu duraklatma ve devam ettirme
Hız çarpanı	Animasyon hızını dinamik olarak ayarlayan oran

30.5.2 Önemli Komutlar

Tablo 30.3: Ünite 2 – AnimatedSprite arayüzü

Metot	Açıklama
<code>animasyon_ekle(ad, frame, sure)</code>	Yeni animasyon durumu ekler
<code>durum_degistir(ad, dongu)</code>	Aktif animasyonu değiştirir
<code>hiz_ayarla(carpan)</code>	Animasyon hızını çarpanla değiştirir
<code>duraklat()</code>	Animasyonu duraklatır
<code>devam_et()</code>	Duraklatılmış animasyonu devam ettirir
<code>update()</code>	Frame ilerlemesini gerçekleştirir

30.5.3 Sıradaki Adım

Bir sonraki ünite de zamanla ilgili kavramlara dalacağız. `pygame.time.get_ticks()` ile hassas zaman ölçümü, `USEREVENT` ile kendi özel olaylarını tanımlama, `set_timer()` ile periyodik görevler oluşturma ve cooldown sistemi tasarımı konularını öğreneceksin. Bu araçlar hem animasyonların hem de oyun mekaniğinin temelini oluşturur.

*“Zamanlama, bir oyunda her şeyin temelidir.
Doğru anda doğru şeyi yapmak farkı yaratır.”*

— JOHN CARMACK

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `pygame.time.get_ticks()` ile zaman ölçümü ve delta time hesaplama
- `USEREVENT` ve `set_timer()` ile periyodik olaylar oluşturma
- Cooldown sınıfı tasarımı: mermi ve yetenek cooldown sistemi
- Dalga sistemi, Spawner sınıfı ve artan zorluk mekaniği

31.1 Zaman Fonksiyonları

Önceki ünitelerde animasyon zamanlaması için `get_ticks()` fonksiyonunu kullandık. Bu ünite de PyGame’in zaman fonksiyonlarını daha geniş bir perspektiften inceleyeceğiz: zamanlayıcılar, cooldown sistemleri ve periyodik olaylar. Zamanlama, animasyondan düşman oluşturmaya, zorluk artışına kadar oyunun her alanında kritik öneme sahiptir.

31.1.1 `pygame.time.get_ticks()`

`pygame.time.get_ticks()` fonksiyonu, `pygame.init()` çağrıldığından bu yana geçen süreyi milisaniye cinsinden döndürür. Bu değer sürekli artar ve asla sıfırlanmaz (oyun yeniden başlatılmadığı sürece).

Python – `get_ticks()` Kullanımı

```
1 import pygame
2 pygame.init()
```

```

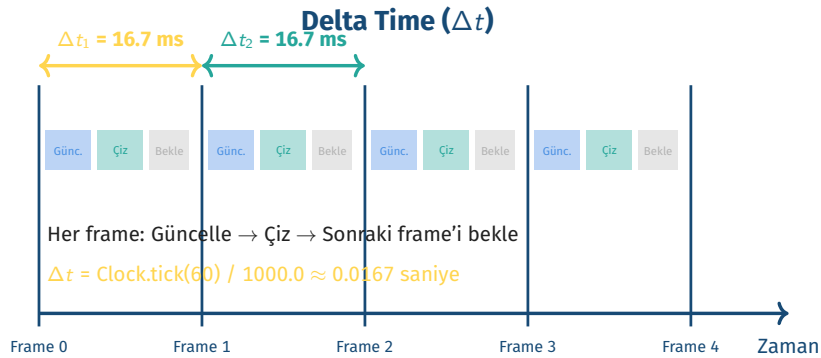
3
4 # Oyun basladigindan beri gecen sure
5 simdi = pygame.time.get_ticks()
6 print(f"Gecen sure: {simdi} ms")
7
8 # Zaman farki olcumu
9 baslangic = pygame.time.get_ticks()
10 # ... bir islem yap ...
11 bitis = pygame.time.get_ticks()
12 gecen = bitis - baslangic
13 print(f"Islem suresi: {gecen} ms")

```

Önceki ünite de bu fonksiyonu animasyon frame zamanlaması için kullanmış-tık. Bu ünite de aynı fonksiyonu cooldown sistemleri, periyodik spawn ve efekt zamanlaması için de kullanacağız.

31.1.2 Delta Time Hatırlatması

Konu 11.3'te delta time kavramını, Konu ??'de ise delta time ile FPS'ten bağımsız hareketi öğrenmiştik. Kısaca hatırlayalım: *delta time* (Δt) iki ardışık frame arasında geçen süredir ve tüm hareket/animasyon hesaplamalarını FPS'ten bağımsız hale getirir.



Şekil 31.1: Delta time: iki frame arasında geçen süre

Bu bölümde delta time'ı animasyon ve zamanlayıcı sistemlerinde de kullanacağız. Hatırlatma olarak iki hesaplama yöntemi:

Python – Delta Time Hesaplama – Hatırlatma

```

1 # Yontem 1: Clock.tick() donusu (tercih edilen)
2 saat = pygame.time.Clock()
3 dt = saat.tick(60) / 1000.0 # FPS siniri + saniye
  ↳ cinsinden dt
4
5 # Yontem 2: get_ticks() ile elle hesaplama
6 onceki_zaman = pygame.time.get_ticks()
7 # ... oyun dongusu ...
8 simdi = pygame.time.get_ticks()

```

```

9 dt = (simdi - oncesi_zaman) / 1000.0
10 oncesi_zaman = simdi

```

● Bilgi: Neden Tekrar?

Delta time'ı daha önce hareket için kullandık. Bu ünite de aynı prensibi cooldown süreleri, spawn aralıkları ve efekt ömürleri için de uygulayacağız. Temel kural aynı: süreye bağlı her şeyi **dt** ile çarp.

31.1.3 pygame.time.wait() ve delay()

Bu iki fonksiyon programı belirli süre bekletir. Ancak oyun döngüsünde **kullanılmamalıdır** çünkü tüm işleyişi durdurur:

Python – wait ve delay – Kullanma

```

1 # [UYARI] Oyun dongusunda kullanma!
2 pygame.time.wait(1000) # 1 saniye bekle (CPU'yu bırakır)
3 pygame.time.delay(1000) # 1 saniye bekle (CPU'yu mesgul
  ↳ tutar)

```

Bu fonksiyonlar yalnızca yükleme ekranı veya splash screen gibi oyun döngüsü dışındaki durumlarda kullanılabilir. Oyun içi zamanlama için `get_ticks()` veya `set_timer()` kullan.

💡 İpucu: Clock.tick() Yeterlidir

Çoğu oyun için `saat.tick(60)` ile elde edilen milisaniye dönüşü yeterlidir. Delta time kullanarak tüm hareket ve animasyon hesaplamalarını FPS'ten bağımsız hale getirebilirsin. Bu kitapta basitlik için `get_ticks()` yaklaşımını tercih ediyoruz ama profesyonel projelerde delta time standart yöntemdir.

31.2 Kullanıcı Olayları ve Zamanlayıcılar

Bölüm 13'de PyGame'in olay sistemini öğrenmiştin: `pygame.QUIT`, `pygame.KEYDOWN`, `pygame.MOUSEBUTTONDOWN` gibi hazır olaylar. Ama ya kendi olaylarını tanımlamak istersen? Belirli aralıklarla düşman oluşturmak, periyodik olarak bonus göndermek veya zamanlı bir tehlike tetiklemek istersen ne yaparsın? İşte burada `USEREVENT` ve `set_timer()` devreye girer.

31.2.1 USEREVENT ile Özel Olaylar

PyGame, kullanıcıların kendi olaylarını tanımlayabilmesi için `pygame.USEREVENT` sabitini sağlar. Bu sabitten itibaren 32'ye kadar özel olay tanımlayabilirsin:

Python – USEREVENT Tanımlama

```

1  # Özel olaylar tanımla
2  DUSMAN_SPAWN = pygame.USEREVENT + 1
3  BONUS_SPAWN = pygame.USEREVENT + 2
4  ZORLUK_ARTISI = pygame.USEREVENT + 3
5  DALGA_BASLAT = pygame.USEREVENT + 4
6
7  # USEREVENT + 0 da kullanılabilir ama
8  # genellikle +1'den başlanır (acilik için)

```

⚠ Dikkat: Olay Numarası Sınırı

PyGame'de en fazla 32 özel olay tanımlayabilirsin (`USEREVENT` ile `USEREVENT + 31` arası). Daha fazla zamanlayıcı gerekiyorsa tek bir olayı birden fazla amaç için kullan veya `get_ticks()` tabanlı zamanlama tercih et.

31.2.2 set_timer() ile Periyodik Olaylar

`pygame.time.set_timer(olay, milisaniye)` fonksiyonu, belirli aralıklarla otomatik olarak olay kuyruğuna olay ekler. Bu, oyun döngüsünde ayrı bir sayaç tutmaktan çok daha temiz bir çözümdür.

Python – set_timer() Kullanımı

```

1  # Her 800 ms'de bir dusman olayi tetikle
2  DUSMAN_SPAWN = pygame.USEREVENT + 1
3  pygame.time.set_timer(DUSMAN_SPAWN, 800)
4
5  # Her 5000 ms'de bir bonus olayi tetikle
6  BONUS_SPAWN = pygame.USEREVENT + 2
7  pygame.time.set_timer(BONUS_SPAWN, 5000)
8
9  # Zamanlayiciyi durdurmak icin sure = 0 ver
10 pygame.time.set_timer(DUSMAN_SPAWN, 0) # Durdur

```

31.2.3 Olayları İşleme

Tanımlanan özel olaylar, standart olaylarla aynı şekilde olay döngüsünde yakalanır:

Python – Özel Olayları İşleme

```

1  for olay in pygame.event.get():
2      if olay.type == pygame.QUIT:
3          calistir = False
4
5      elif olay.type == DUSMAN_SPAWN:

```

```

6         # Yeni dusman olustur
7         dusman = Dusman()
8         tum_spritelar.add(dusman)
9         dusmanlar.add(dusman)
10
11     elif olay.type == BONUS_SPAWN:
12         # Yeni bonus olustur
13         bonus = Bonus(
14             random.randint(20, GENISLIK - 20), -20
15         )
16         tum_spritelar.add(bonus)
17         bonuslar.add(bonus)

```

31.2.4 Tek Seferlik Zamanlayıcı

`set_timer()`'ın `loops` parametresi ile tek seferlik zamanlayıcı oluşturabilirsin (PyGame 2.0.0.dev3 ve sonrası):

Python – Tek Seferlik Zamanlayıcı

```

1     # 3 saniye sonra tek seferlik olay
2     BOSS_UYARI = pygame.USEREVENT + 5
3     pygame.time.set_timer(BOSS_UYARI, 3000, loops=1)
4
5     # Olay islendiginde zamanlayici otomatik durur

```

● Bilgi: loops Parametresi

`loops=0` varsayılandır ve sonsuz tekrar anlamına gelir. `loops=1` tek seferlik tetikleme sağlar. Bu parametre PyGame'in yeni sürümlerinde eklenmiştir; eski sürümlerde olay işlendiğinde `set_timer(olay, 0)` ile elle durdurman gerekir.

31.2.5 Uzay Kaçışı'nda Zamanlayıcılar

Uzay Kaçışı v3'te zaten `USEREVENT` tabanlı zamanlayıcılar kullanıyorduk. Hatırlayalım:

Python – v3'teki Zamanlayıcılar (Tekrar)

```

1     # v3'te kullanılan zamanlayicilar
2     DUSMAN_OLAY = pygame.USEREVENT + 1
3     pygame.time.set_timer(DUSMAN_OLAY, 800) # 800 ms
4
5     BONUS_OLAY = pygame.USEREVENT + 2
6     pygame.time.set_timer(BONUS_OLAY, 5000) # 5 saniye

```

Bu yapı çalışır ama zorluk artışı, dalga sistemi ve cooldown gibi daha karmaşık zamanlama gereksinimleri için yeterli değildir. Sonraki konularda bu ihtiyaçları

karşılacak Cooldown ve Spawner sınıflarını tasarlayacağız.

31.3 Cooldown Sistemi

Bir oyuncunun saniyede 60 mermi atması mantıklı mı? Ya da bir büyünün hiç bekleme olmadan sürekli kullanılması? Tabii ki hayır. İşte *cooldown* (bekleme süresi) sistemi tam da bunu düzenler: bir eylemin tekrar kullanılabilmesi için geçmesi gereken minimum süreyi kontrol eder.

31.3.1 Basit Cooldown

Uzay Kaçışı v3'te zaten basit bir cooldown kullanıyorduk:

Python – Basit Cooldown (v3 Yaklaşımı)

```

1 class Oyuncu(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.son_ates_zamani = 0
5
6     def ates_edebilir_mi(self):
7         simdi = pygame.time.get_ticks()
8         if simdi - self.son_ates_zamani > 200: # 200 ms
9             self.son_ates_zamani = simdi
10            return True
11            return False

```

Bu yöntem çalışır ama birden fazla cooldown yönetmek gerektiğinde kod karmaşıklaşır. Her yetenek için ayrı *son_zaman* değişkeni ve ayrı kontrol fonksiyonu yazmak gerekir.

31.3.2 Cooldown Sınıfı

Daha iyi bir çözüm, cooldown mantığını ayrı bir sınıfta kapsüllemektir. Bu sınıf herhangi bir yeteneğe bağlı değildir; mermi, roket, kalkan veya başka herhangi bir eylem için kullanılabilir:

Python – Cooldown – Temel Metotlar

```

1 class Cooldown:
2     """Tekrar kullanılabilir cooldown zamanlayicisi.
3
4     Herhangi bir eylem için bekleme suresi yönetir.
5     """
6
7     def __init__(self, sure_ms):
8         """
9         Args:
10            sure_ms: Cooldown suresi (milisaniye).
11            """

```

```

12         self.sure = sure_ms
13         self.son_kullanim = 0
14
15     def hazir_mi(self):
16         """Cooldown suresi gecti mi?
17
18         Returns:
19             bool: True ise eylem kullanılabilir.
20         """
21         simdi = pygame.time.get_ticks()
22         return (simdi - self.son_kullanim) >= self.sure
23
24     def kullan(self):
25         """Cooldown'u baslat (eylemi kullandıktan sonra
26         ↪ cagir)."""
27         self.son_kullanim = pygame.time.get_ticks()
28
29     def dene(self):
30         """hazir_mi() + kullan() birlesimi.
31
32         Returns:
33             bool: True ise eylem kullanildi ve cooldown
34             ↪ basladi.
35         """
36         if self.hazir_mi():
37             self.kullan()
38             return True
39         return False

```

`son_kullanim` başlangıçta 0 olarak ayarlanır; bu, oyun başladığında cooldown'un hemen hazır olmasını sağlar (çünkü `get_ticks()` her zaman 0'dan büyük bir değer döndürür). `dene()` metodu en sık kullanılan yardımcıdır: tek çağrıda hem kontrol hem kullanım yapar, böylece `hazir_mi()` ve `kullan()` çağrılarını ayrı ayrı yazmak gerekmez. Bu, özellikle oyun döngüsünde tuş kontrolü sırasında kodu sadeleştirir.

Cooldown'un görsel gösterge için gereken bilgi metotları ve sıfırlama:

Python – Cooldown – Bilgi ve Sıfırlama Metotları

```

1     def kalan_sure(self):
2         """Cooldown'un bitmesine kalan sure (ms).
3
4         Returns:
5             int: Kalan milisaniye (0 = hazir).
6         """
7         gecen = pygame.time.get_ticks() - self.son_kullanim
8         kalan = self.sure - gecen
9         return max(0, kalan)
10
11     def ilerleme(self):
12         """Cooldown ilerleme yuzdesi (0.0 - 1.0).

```

```

13
14     Returns:
15         float: 1.0 = hazır, 0.0 = yeni baslatildi.
16         """
17     if self.sure == 0:
18         return 1.0
19     gecen = pygame.time.get_ticks() - self.son_kullanim
20     return min(1.0, gecen / self.sure)
21
22 def sifirla(self):
23     """Cooldown'u sifirla (hemen hazır yap)."""
24     self.son_kullanim = 0

```

`ilerleme()` metodu 0.0'dan 1.0'a doğru artan bir değer döndürür ve HUD'daki cooldown çubuğunu doldurmak için idealdir. `kalan_sure()` ise milisaniye cinsinden kalan süreyi verir; bu, "2.3 saniye kaldı" gibi metin göstergeleri için kullanılabilir. `sifirla()` metodu cooldown'u anında hazır hale getirir — bonus toplama gibi ödül mekanizmalarında işe yarar.

31.3.3 Cooldown Kullanım Örnekleri

Python – Cooldown ile Oyuncu Yetenekleri

```

1 class Oyuncu(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         # Farkli eylemler icin farkli cooldown'lar
5         self.cd_mermi = Cooldown(200)      # 200 ms
6         self.cd_roket = Cooldown(2000)     # 2 saniye
7         self.cd_kalkan = Cooldown(5000)    # 5 saniye
8
9     def update(self):
10        tuslar = pygame.key.get_pressed()
11
12        # Mermi atesi (Space)
13        if tuslar[pygame.K_SPACE]:
14            if self.cd_mermi.dene():
15                self._mermi_at()
16
17        # Roket atesi (R)
18        if tuslar[pygame.K_r]:
19            if self.cd_roket.dene():
20                self._roket_at()
21
22        # Kalkan aktifle (Q)
23        if tuslar[pygame.K_q]:
24            if self.cd_kalkan.dene():
25                self._kalkan_ac()

```

31.3.4 Cooldown Göstergesi

Oyuncuya cooldown durumunu göstermek, oyun deneyimini iyileştirir. `ilerleme()` metodu ile basit bir görsel gösterge çizebilirsin:

Python – Cooldown HUD Göstergesi

```

1 def cooldown_goster(ekran, x, y, cooldown, renk, ad):
2     """Cooldown ilerleme cubugu ciz."""
3     gen = 60
4     yuk = 8
5     ilerleme = cooldown.ilerleme()
6
7     # Arka plan
8     pygame.draw.rect(ekran, (60, 60, 60), (x, y, gen,
9         ↪ yuk))
10    # Doluluk
11    dolu = int(gen * ilerleme)
12    pygame.draw.rect(ekran, renk, (x, y, dolu, yuk))
13    # Cerceve
14    pygame.draw.rect(ekran, (180, 180, 180), (x, y, gen,
15        ↪ yuk), 1)

```

💡 İpucu: Cooldown Tasarım İpuçları

- Mermi cooldown'u kısa tut (100–300 ms), yoksa oyun hissiyatı yavaşlar
- Güçlü yeteneklere uzun cooldown ver (3–10 saniye)
- Cooldown göstergesini mutlaka HUD'da göster
- Cooldown sıfırlama bonusu eklemek oyunu zenginleştirir

31.4 Dalga Sistemi ve Spawner

Uzay Kaçışı v3'te düşmanlar sabit bir aralıkla (800 ms) sürekli oluşuyordu. Bu basit ama monotondur: oyun hiç zorlaşmaz. Gerçek oyunlarda düşmanlar dalgalar halinde gelir, her dalga öncekinden daha zor olur ve dalgalar arası kısa bir nefes molası verilir. Bu konuda böyle bir sistemi tasarlayacağız.

31.4.1 Spawner Sınıfı

`Spawner` sınıfı, belirli aralıklarla nesne oluşturan genel amaçlı bir zamanlayıcıdır. `set_timer()` kullanmak yerine `get_ticks()` tabanlı çalışır; böylece dinamik aralık değişikliği yapılabilir.

Python – Spawner – Kurucu ve Güncelleme

```

1 class Spawner:
2     """Periyodik nesne olusturucu.
3
4     Belirli araliklarla callback fonksiyonunu cagirarak

```

```

5      yeni nesneler olusturur. Aralik dinamik
        ↳ degistirilebilir.
6      """
7
8      def __init__(self, aralik_ms, olusturucu, aktif=True):
9          """
10         Args:
11             aralik_ms: Olusturma araligi (milisaniye).
12             olusturucu: Cagirilacak fonksiyon (nesne
13                 ↳ dondurur).
14             aktif: Baslangicta aktif mi?
15         """
16         self.aralik = aralik_ms
17         self.olusturucu = olusturucu
18         self.aktif = aktif
19         self.son_spawn = pygame.time.get_ticks()
20         self.toplam_spawn = 0
21
22     def guncelle(self):
23         """Zamani kontrol et, gerekirse nesne olustur.
24
25         Returns:
26             object: Olusturulan nesne veya None.
27         """
28         if not self.aktif:
29             return None
30
31         simdi = pygame.time.get_ticks()
32         if simdi - self.son_spawn >= self.aralik:
33             self.son_spawn = simdi
34             self.toplam_spawn += 1
35             return self.olusturucu()
36
37         return None

```

Spawner'ın kurucu metodu bir `olusturucu` callback fonksiyonu alır. Bu fonksiyon her spawn anında çağrılarak yeni nesne döndürür — böylece Spawner, ne tür nesne oluşturduğunu bilmek zorunda kalmaz ve düşman, bonus veya parçacık gibi farklı nesneler için tekrar kullanılabilir. `pygame.time.get_ticks()` tabanlı zamanlama, `set_timer()` kullanmaya göre bir avantaj sunar: aralık değeri çalışma zamanında dinamik olarak değiştirilebilir.

Aralık değiştirme ve başlat/durdur kontrol metotları:

Python – Spawner – Kontrol Metotları

```

1      def aralik_degistir(self, yeni_aralik):
2          """Spawn araligini degistir (zorluk artisti icin)."""
3          self.aralik = max(100, yeni_aralik) # En az 100 ms
4
5      def durdur(self):
6          """Spawner'i durdur."""

```

```

7     self.aktif = False
8
9     def baslat(self):
10         """Spawner'i baslat."""
11         self.aktif = True
12         self.son_spawn = pygame.time.get_ticks()

```

`aralik_degistir()` metodundaki `max(100, ...)` kontrolü önemli bir güvenlik önlemidir: aralık 100 ms'nin altına düşerse saniyede 10'dan fazla nesne oluşur ki bu hem performansı düşürür hem de oyunu oynanamaz hale getirir. `baslat()` metodunda `son_spawn` değerini sıfırlamak, yeniden başlatıldığında hemen bir nesne üretmesini önler ve temiz bir zamanlama başlangıcı sağlar.

31.4.2 Dalga Sistemi

Dalga sistemi, Spawner'ı kullanarak düşmanları gruplar halinde gönderir ve dalgalar arası zorluk artışı uygular. Kurucu metotta dalga parametreleri ve Spawner entegrasyonu hazırlanır:

Python – DalgaYoneticisi – Kurucu Metot

```

1  class DalgaYoneticisi:
2      """Dalga bazli dusman yonetimi.
3
4      Her dalga belirli sayida dusman icerir.
5      Dalga tamamlaninca kısa mola verilir ve
6      bir sonraki dalga daha zor olur.
7      """
8
9      def __init__(self):
10         self.dalga_no = 1
11         self.dalga_dusman_sayisi = 5
12         self.kalan_dusman = self.dalga_dusman_sayisi
13         self.spawn_aralik = 800 # ms
14         self.mola_suresi = 3000 # 3 saniye mola
15         self.mola_baslangic = 0
16         self.mola_aktif = False
17         self.dalga_aktif = True
18
19         # Dusman spawner
20         self.spawner = Spawner(
21             self.spawn_aralik, self._dusman_olustur
22         )
23
24     def _dusman_olustur(self):
25         """Yeni dusman nesnesi olustur."""
26         return Dusman()

```

`DalgaYoneticisi`, içinde bir `Spawner` barındırarak bileşim (composition) desenini kullanır: düşman oluşturma zamanlamasını `Spawner`'a devreder, kendisi ise dalga mantığına odaklanır. `mola_suresi` 3000 ms (3 saniye) olarak ayarlan-

mıştır; bu süre oyuncuya nefes alma fırsatı tanır. İlk dalga 5 düşman içerir ve 800 ms aralıklarla spawn edilir.

Her frame’de çağrılan `guncelle()` metodu dalga durumuna göre farklı davranır — mola sırasında bekler, dalga aktifken düşman üretir:

Python – DalgaYoneticisi – Güncelleme Mantığı

```

1 def guncelle(self):
2     """Dalga durumunu guncelle.
3
4     Returns:
5         object: Olusturulan dusman veya None.
6     """
7     # Mola sirasinda bekle
8     if self.mola_aktif:
9         simdi = pygame.time.get_ticks()
10        if simdi - self.mola_baslangic >=
11            ↪ self.mola_suresi:
12            self.mola_aktif = False
13            self._yeni_dalga()
14        return None
15
16    # Dalga aktifse dusman olustur
17    if self.dalga_aktif and self.kalan_dusman > 0:
18        sonuc = self.spawner.guncelle()
19        if sonuc is not None:
20            self.kalan_dusman -= 1
21            if self.kalan_dusman <= 0:
22                self._dalga_bitti()
23            return sonuc
24
25    return None

```

`guncelle()` metodu aslında basit bir iki durumlu state machine olarak çalışır: ya mola modundadır (zamanlayıcıyı bekler) ya da dalga modundadır (Spawner’dan düşman ister). Döndürülen değer ya bir düşman nesnesidir ya da `None`; bu sayede çağırın kod sadece dönen değeri kontrol ederek yeni düşmanı sprite grubuna ekleyebilir.

Dalga tamamlanma ve yeni dalga başlatma ile birlikte HUD bilgisi döndüren metotları görelim:

Python – DalgaYoneticisi – Dalga Geçişleri ve HUD

```

1 def _dalga_bitti(self):
2     """Mevcut dalga tamamlandı."""
3     self.dalga_aktif = False
4     self.mola_aktif = True
5     self.mola_baslangic = pygame.time.get_ticks()
6     self.spawner.durdur()
7
8 def _yeni_dalga(self):

```

```

9      """Yeni dalga baslat (zorluk artir)."""
10     self.dalga_no += 1
11     # Her dalga 2 daha fazla dusman
12     self.dalga_dusman_sayisi += 2
13     self.kalan_dusman = self.dalga_dusman_sayisi
14     # Spawn araligi azalir (minimum 300 ms)
15     self.spawn_aralik = max(300, self.spawn_aralik - 50)
16     self.spawner.aralik_degistir(self.spawn_aralik)
17     self.spawner.baslat()
18     self.dalga_aktif = True
19
20     def dalga_bilgisi(self):
21         """Dalga durumu metni."""
22         if self.mola_aktif:
23             kalan = self.mola_suresi - (
24                 pygame.time.get_ticks() - self.mola_baslangic
25             )
26             return f"Dalga {self.dalga_no + 1} - {kalan //
27                 ↪ 1000 + 1}s"
28         return f"Dalga {self.dalga_no} - {self.kalan_dusman}
29             ↪ kaldı"

```

`_yeni_dalga()` metodu zorluk artışının iki eksenini uygular: düşman sayısı her dalga +2 artar (5, 7, 9, ...) ve spawn aralığı her dalga 50 ms azalır (800, 750, 700, ... minimum 300 ms). Bu kademeli artış, oyuncunun ilk birkaç dalgada mekaniği öğrenmesini, sonra giderek artan baskıyla mücadele etmesini sağlar. `dalga_bilgisi()` metodu ise HUD'da gösterilecek durum metnini döndürür: mola sırasında geri sayım, dalga sırasında kalan düşman sayısı.

31.4.3 Zorluk Artışı

Dalga sisteminde zorluk artışı birden fazla eksenle yapılabilir:

Tablo 31.1: Zorluk artışı parametreleri

Parametre	Başlangıç	Artış
Düşman sayısı	5	Her dalga +2
Spawn aralığı	800 ms	Her dalga -50 ms (min 300)
Düşman hızı	2-5	Her 3 dalgada +0.5
Düşman canı	1	Her 5 dalgada +1

● Bilgi: Dengeli Zorluk

Zorluk artışı kademeli olmalıdır. Çok hızlı artan zorluk oyuncuyu sinirlendirir, çok yavaş artan zorluk sıkar. Genellikle ilk 5 dalga kolay, 6-10 orta, 11+ zor olacak şekilde ayarlanır. Oyunculardan geri bildirim alarak dengeleme (balancing) yapmak en iyi yoldur.

31.5 Sonuç

Bu ünite de zamanlama ve periyodik olayların oyun geliştirmedeki kritik rolünü öğrendin. `get_ticks()` ile hassas zaman ölçümü, `USEREVENT` ve `set_timer()` ile özel zamanlayıcılar, Cooldown sınıfıyla eylem bekleme süreleri ve DalgaYonecisi ile dinamik zorluk artışını kavradın.

31.5.1 Anahtar Kavramlar

Tablo 31.2: Ünite 3 – Anahtar kavramlar

Kavram	Açıklama
<code>get_ticks()</code>	PyGame başlangıcından bu yana geçen milisaniye
Delta Time	İki ardışık frame arasında geçen süre
<code>USEREVENT</code>	Kullanıcı tanımlı özel olay türleri için başlangıç sabiti
<code>set_timer()</code>	Belirli aralıklarla otomatik olay tetikleyen zamanlayıcı
Cooldown	Bir eylemin tekrar kullanılması için gereken minimum bekleme süresi
Spawner	Periyodik olarak yeni nesne oluşturan zamanlayıcı sınıfı
Dalga Sistemi	Düşmanları gruplar halinde gönderen ve dalgalar arası zorluk artıran sistem
Zorluk Artışı	Oyunun ilerlemesiyle parametrelerin (hız, sayı, aralık) değişmesi

31.5.2 Önemli Komutlar

Tablo 31.3: Ünite 3 – Zamanlama komutları

Komut	Açıklama
<code>pygame.time.get_ticks()</code>	Geçen süreyi ms cinsinden döndürür
<code>saat.tick(fps)</code>	FPS sınırlama ve delta time elde etme
<code>pygame.time.set_timer(olay, ms)</code>	Periyodik olay zamanlayıcısı
<code>pygame.time.set_timer(olay, 0)</code>	Zamanlayıcıyı durdurur
<code>pygame.USEREVENT + n</code>	Özel olay numarası (0–31 arası)

31.5.3 Sıradaki Adım

Son ünite de görsel efektlere geçeceğiz: parçacık (particle) sistemleri, patlama animasyonları, fade/blink efektleri ve screen shake. Bu efektler oyuna profesyonel bir dokunuş katar ve oyuncunun geri bildirim almasını sağlar. Ardından Uzay Kaçışı v4'ün final versiyonunda tüm bu sistemleri birleştireceğiz.

“Detaylar mükemmelliği yaratır, ama mükemmellik bir detay değildir.”

— LEONARDO DA VINCI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- **Particle** ve **ParticleEmitter** sınıfları ile parçacık sistemi
- Sprite ve parçacık tabanlı patlama efektleri (**Explosion** sınıfı)
- Fade, blink, screen shake ve **EffectManager** sınıfı
- Uzak Kaçışı v4 FINAL: tüm efektlerin entegrasyonu

32.1 Parçacık Sistemi

Bir düşman yok edildiğinde sadece ekrandan kaybolması sıkıcıdır. Ama etrafa küçük parçacıklar saçılırsa, bir ateş topu efekti oluşursa veya kıvılcımlar uçuşursa oyuncu “harika bir şey yaptığını” hisseder. İşte *parçacık sistemi* (particle system) tam da bu duyguyu yaratmak için kullanılır.

32.1.1 Parçacık Nedir?

Bir *parçacık* (particle), kısa ömürlü, küçük bir görsel öğedir. Tek başına önemsiz görünür ama yüzlerce parçacık bir arada çalışınca etkileyici efektler oluşturur: ateş, duman, kar, yağmur, kıvılcım, toz bulutu...

■ Tanım: Parçacık Sistemi

Bir *parçacık sistemi*, çok sayıda küçük parçacığı oluşturan, güncelleyen ve yok eden bir yönetici yapıdır. Her parçacığın konumu, hızı, rengi, boyutu ve ömrü vardır. Parçacıklar oluşturulur, belirli bir süre yaşar ve sonra kaybolur.

32.1.2 Particle Sınıfı

Her parçacık şu özelliklere sahiptir: konum, hız, renk, boyut ve ömür. Basit bir `Particle` sınıfı tasarlayalım:

Python – Particle – Kurucu Metot

```

1  class Particle(pygame.sprite.Sprite):
2      """Tek bir parçacık.
3
4      Belirli bir konumdan rastgele yone doğru hareket eder,
5      zamanla kuculerek kaybolur.
6      """
7
8      def __init__(self, x, y, renk, hiz_x, hiz_y,
9                  boyut=4, omur=500):
10         """
11         Args:
12             x, y: Başlangıç konumu.
13             renk: RGB renk değeri.
14             hiz_x, hiz_y: Piksel/frame cinsinden hiz.
15             boyut: Başlangıç boyutu (piksel).
16             omur: Yaşam süresi (milisaniye).
17         """
18         super().__init__()
19         self.pos_x = float(x)
20         self.pos_y = float(y)
21         self.hiz_x = hiz_x
22         self.hiz_y = hiz_y
23         self.renk = renk
24         self.boyut = boyut
25         self.omur = omur
26         self.dogum = pygame.time.get_ticks()
27
28         # Gorsel olustur
29         self.image = pygame.Surface(
30             (boyut * 2, boyut * 2), pygame.SRCALPHA
31         )
32         pygame.draw.circle(
33             self.image, renk, (boyut, boyut), boyut
34         )
35         self.rect = self.image.get_rect(center=(x, y))

```

Her parçacık kendi konumunu `float` olarak saklar (`pos_x`, `pos_y`). Bu, piksel tabanlı tam sayı konumlarına göre çok daha yumuşak bir hareket sağlar; çünkü hız değerleri 0.5 gibi kesirli olabilir. `pygame.SRCALPHA` bayrağıyla oluşturulan yüzey, alfa kanalı destekler ve parçacığın zamanla soluklaşmasına olanak tanır. `dogum` zamanı, parçacığın ne kadar yaşadığını hesaplamak için kullanılır.

`update()` metodu her karede parçacığın konumunu günceller, basit bir yerçekimi efekti uygular ve ömrü dolduğunda parçacığı yok eder:

Python – Particle – Güncelleme ve Yaşam Döngüsü

```

1  def update(self):
2      """Konumu guncelle, omru dolduysa yok et."""
3      # Hareket
4      self.pos_x += self.hiz_x
5      self.pos_y += self.hiz_y
6
7      # Yerçekimi etkisi (opsiyonel)
8      self.hiz_y += 0.1
9
10     self.rect.center = (int(self.pos_x), int(self.pos_y))
11
12     # Omur kontrolu
13     gecen = pygame.time.get_ticks() - self.dogum
14     kalan_oran = 1.0 - (gecen / self.omur)
15
16     if kalan_oran <= 0:
17         self.kill()
18         return
19
20     # Zamanla kucul ve soluklas
21     yeni_boyut = max(1, int(self.boyut * kalan_oran))
22     self.image = pygame.Surface(
23         (yeni_boyut * 2, yeni_boyut * 2), pygame.SRCALPHA
24     )
25     alfa = int(255 * kalan_oran)
26     renk_alfa = (*self.renk[:3], alfa)
27     pygame.draw.circle(
28         self.image, renk_alfa,
29         (yeni_boyut, yeni_boyut), yeni_boyut
30     )
31     self.rect = self.image.get_rect(
32         center=(int(self.pos_x), int(self.pos_y))
33     )

```

Yerçekimi etkisi sadece `self.hiz_y += 0.1` satırıyla elde edilir: dikey hız her karede biraz artar ve parçacık aşağı doğru eğri bir yol izler. `kalan_oran` değeri (1.0'dan 0.0'a doğru azalır) hem boyut küçülmesi hem de alfa şeffaflığı için kullanılır, böylece parçacık ömrünün sonuna doğru hem küçülür hem de soluklaşarak kaybolur. Her karede yeni bir `Surface` oluşturmak performans açısından pahalı görünebilir ama parçacıklar zaten kısa ömürlüdür ve toplam sayı kontrol altında tutulduğunda sorun yaratmaz.

32.1.3 ParticleEmitter Sınıfı

`ParticleEmitter` belirli bir konumdan birden fazla parçacık üreten kaynaktır. Kurucu metodunda parçacıkların görsel ve fiziksel özelliklerini belirleyen aralık parametreleri tanımlanır:

Python – ParticleEmitter – Kurucu Metot

```

1  import random
2
3  class ParticleEmitter:
4      """Parçacık üretici.
5
6      Belirli bir konumdan rastgele yonlere parçacık
7      ↪ fırlatır.
8      Tek seferlik (patlama) veya sürekli (ates, duman)
9      ↪ modda
10     çalışabilir.
11     """
12
13     def __init__(self, x, y, renkler, grup,
14                 parçacık_sayisi=20, hiz_aralik=(1, 5),
15                 boyut_aralik=(2, 6), omur_aralik=(300,
16                 ↪ 800)):
17
18         """
19         Args:
20         x, y: Emisyon merkezi.
21         renkler: Renk listesi (rastgele secilir).
22         grup: Parçacıkların ekleneceği sprite grubu.
23         parçacık_sayisi: Üretilen parçacık sayısı.
24         hiz_aralik: (min, max) hız değeri.
25         boyut_aralik: (min, max) boyut değeri.
26         omur_aralik: (min, max) omur süresi (ms).
27         """
28
29         self.x = x
30         self.y = y
31         self.renkler = renkler
32         self.grup = grup
33         self.parçacık_sayisi = parçacık_sayisi
34         self.hiz_aralik = hiz_aralik
35         self.boyut_aralik = boyut_aralik
36         self.omur_aralik = omur_aralik

```

Emitter'ın tüm parametreleri aralık olarak tanımlanır (*hiz_aralik*, *boyut_aralik*, *omur_aralik*). Bu tasarım, her parçacığın biraz farklı olmasını sağlayarak doğal ve organik bir görüntü yaratır. Eğer tüm parçacıklar aynı hız ve boyutta olsaydı, efekt yapay ve mekanik görünürdü. *renkler* listesinden rastgele seçim yapılması da renk çeşitliliği katarak efekti zenginleştirir.

patla() metodu, tüm parçacıkları aynı anda oluşturarak patlama efekti yaratır. Her parçacık rastgele bir açıda fırlatılır:

Python – ParticleEmitter – Patlama Efekti

```

1  def patla(self):
2      """Tüm parçacıkları aynı anda uret (patlama
3      ↪ efekti)."""
4      for _ in range(self.parçacık_sayisi):

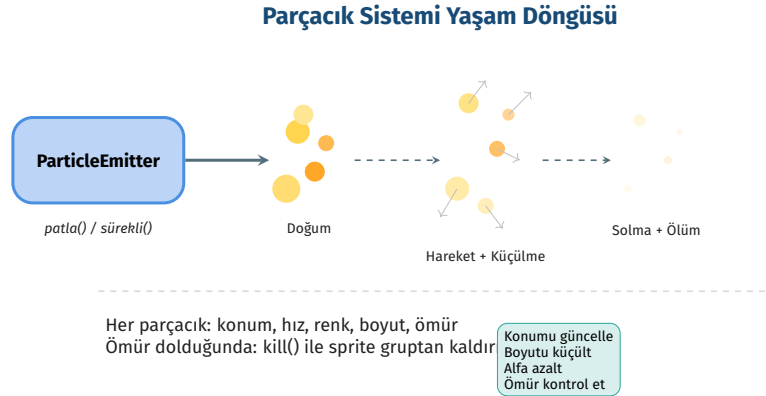
```

```

4 renk = random.choice(self.renkler)
5 aci = random.uniform(0, 6.2832) # 0 - 2*pi
6 hiz = random.uniform(*self.hiz_aralik)
7
8 import math
9 hx = math.cos(aci) * hiz
10 hy = math.sin(aci) * hiz
11 boyut = random.randint(*self.boyut_aralik)
12 omur = random.randint(*self.omur_aralik)
13
14 p = Particle(
15     self.x, self.y, renk, hx, hy, boyut, omur
16 )
17 self.grup.add(p)

```

Patlama efektinin sırrı `math.cos(aci)` ve `math.sin(aci)` hesaplamaların-
dadır: 0 ile 2π arasında rastgele bir açı seçilir ve bu açıdaki birim vektör, rastgele
bir hız büyüklüğüyle çarpılarak parçacığın x ve y hız bileşenleri elde edilir. So-
nuçta parçacıklar merkez noktadan her yöne eşit olasılıkla dağılır ve dairesel bir
patlama deseni oluşturur.



Şekil 32.1: Parçacık sistemi: EmitterSınıfı'ndan yayılan parçacıkların yaşam döngüsü

İpucu: Performans

Çok sayıda parçacık performansı düşürebilir. Toplam aktif parçacık sayısını 200–500 arasında tutmak iyi bir hedeftir. Her patlama için 15–30 parçacık genellikle yeterli görsel etkiyi yaratır.

32.2 Patlama Efektleri

Parçacık sistemi harika efektler yaratır ama tek başına yeterli olmayabilir. Bir düşman yok edildiğinde hem sprite tabanlı bir patlama animasyonu hem de parçacık etkisi birlikte kullanıldığında çok daha etkileyici bir sonuç elde edilir. Bu konuda her iki yaklaşımı birleştireceğiz.

32.2.1 Sprite Tabanlı Patlama

Patlama animasyonu için bir sprite sheet kullanılabilir veya geometrik şekillerle basit bir patlama simüle edilebilir:

Python – Explosion – Kurucu ve Güncelleme

```

1 class Explosion(AnimatedSprite):
2     """Sprite tabanlı patlama animasyonu.
3
4     One-shot modda çalışır: animasyon bitince
5     sprite otomatik olarak gruptan kaldırılır.
6     """
7
8     def __init__(self, x, y, boyut=64):
9         super().__init__(x, y)
10        frameler = self._patlama_frameleri(boyut)
11        self.animasyon_ekle("patlama", frameler, 60)
12        self.durum_degistir("patlama", dongu=False)
13
14    def update(self):
15        """Animasyonu guncelle, bittiyse sprite'i
16        ↪ kaldır."""
17        super().update()
18        if self.bitti:
19            self.kill()

```

`Explosion` sınıfı `AnimatedSprite`'in alt sınıfıdır ve tüm animasyon altyapısını miras alır. Kurucu metotta `dongu=False` parametresiyle one-shot modda başlatılır; böylece patlama yalnızca bir kez oynatılır. `update()` metodunda `self.bitti` kontrolü yapılarak animasyon bittiğinde `kill()` ile sprite gruptan otomatik olarak kaldırılır — bellek sızıntısı önlenir ve efekt temiz bir şekilde sonlanır. Frame süresi 60 ms olarak ayarlanmıştır; 6 frame'lik bir patlama yaklaşık 360 ms sürer ki bu, hızlı ama fark edilebilir bir etki yaratır.

Patlama frame'lerini geometrik şekillerle oluşturan yardımcı metot, gerçek bir sprite sheet olmadan etkileyici bir görüntü üretir:

Python – Explosion – Patlama Frame'leri Oluşturma

```

1 def _patlama_frameleri(self, boyut):
2     """6 frame'lik patlama animasyonu olustur."""
3     frameler = []
4     renkler = [
5         (255, 255, 200), # Beyaz-sari (parlak)
6         (255, 220, 100), # Sari
7         (255, 150, 50),  # Turuncu
8         (255, 80, 30),   # Koyu turuncu
9         (200, 50, 20),   # Kirmizi
10        (100, 30, 10),    # Koyu kirmizi (soluyor)
11    ]
12

```

```

13 for i, renk in enumerate(renkler):
14     yuzey = pygame.Surface(
15         (boyut, boyut), pygame.SRCALPHA
16     )
17     # Disariya dogru buyuyen daireler
18     yaricap = int(boyut * 0.2 + (boyut * 0.3) * (i /
19         ↪ 5))
20     alfa = 255 - int(i * 40)
21
22     # Dis halka
23     dis_renk = (*renk, max(0, alfa - 50))
24     pygame.draw.circle(
25         yuzey, dis_renk,
26         (boyut // 2, boyut // 2),
27         yaricap
28     )
29
30     # Ic parlak cekirdek
31     if i < 4:
32         ic_renk = (255, 255, 220, alfa)
33         pygame.draw.circle(
34             yuzey, ic_renk,
35             (boyut // 2, boyut // 2),
36             max(2, yaricap // 3)
37         )
38
39     frameler.append(yuzey)
40
41 return frameler

```

Frame üretim mantığı iki katmanlı bir daire çizimi kullanır: dış halka renk geçişiyle (beyaz-sarıdan koyu kırmızıya) patlamanın ilerlemesini gösterirken, iç çekirdek parlak bir nokta olarak ilk 4 frame’de görünür ve patlamanın merkezindeki yoğun ışığı simüle eder. **yaricap** formülü her frame’de biraz daha büyük bir daire çizer, **alfa** değeri ise kademeli olarak azalarak soluklaşma efekti yaratır. Bu basit geometrik yaklaşım, sprite sheet hazırlamadan makul kalitede bir patlama animasyonu elde etmek için etkili bir yöntemdir.

32.2.2 Kombine Patlama: Sprite + Parçacık

En etkileyici sonuç, sprite animasyonu ile parçacık efektini birleştirmektir:

Python – Kombine Patlama Oluşturma

```

1 def patlama_olustur(x, y, tum_spritelar, efekt_grubu):
2     """Bir noktada kombine patlama efekti olustur.
3
4     Hem sprite animasyonu hem de parcacik efekti uretir.
5
6     Args:

```



```

7         x, y: Patlama merkezi.
8         tum_spritelar: Ana sprite grubu (cizim icin).
9         efekt_grubu: Efekt sprite grubu.
10    """
11    # 1. Sprite patlama animasyonu
12    patlama = Explosion(x, y, boyut=64)
13    tum_spritelar.add(patlama)
14    efekt_grubu.add(patlama)
15
16    # 2. Parcacik efekti
17    renkler = [
18        (255, 220, 100), # Sari
19        (255, 150, 50), # Turuncu
20        (255, 80, 30),  # Kirmizimsi
21        (200, 200, 200), # Gri (duman)
22    ]
23    emitter = ParticleEmitter(
24        x, y, renkler, tum_spritelar,
25        parcacik_sayisi=15,
26        hiz_aralik=(1, 4),
27        boyut_aralik=(2, 5),
28        omur_aralik=(200, 600)
29    )
30    emitter.patla()

```

32.2.3 Oyun Döngüsüne Entegrasyon

Çarpışma algılama sırasında patlama efektini tetiklemek:

Python – Çarpışma + Patlama

```

1  # Mermi-dusman carpismalari
2  carpismalar = pygame.sprite.groupcollide(
3      mermiler, dusmanlar, True, True
4  )
5  for mermi, vurulanlar in carpismalar.items():
6      for dusman in vurulanlar:
7          skor += dusman.puan
8          ses.cal("patlama")
9          # Patlama efekti olustur
10         patlama_olustur(
11             dusman.rect.centerx,
12             dusman.rect.centery,
13             tum_spritelar, efektler
14         )

```

● Bilgi: Efekt Grubu

Efektleri ayrı bir sprite grubunda tutmak iyi bir pratiktir. Böylece efektlerin çarpışma algılamaya dahil edilmesini önler ve gerektiğinde tüm efektleri kolayca temizleyebilirsiniz.

32.2.4 Patlama Boyutu ve Çeşitlilik

Farklı olaylar için farklı patlama boyutları kullanmak oyunu zenginleştirir:

- **Küçük patlama** (32): Mermi çarpması, küçük düşman yok edilmesi
- **Orta patlama** (64): Normal düşman yok edilmesi, bonus toplama
- **Büyük patlama** (128): Boss yok edilmesi, özel silah etkisi

💡 İpucu: Görsel Zenginlik

Aynı olaya birden fazla efekt katmanı eklemek (sprite animasyonu + parçacıklar + ekran sarsılması) oyuncuya güçlü bir geri bildirim verir. "Juice" olarak adlandırılan bu kavram, oyunun hissiyatını büyük ölçüde iyileştirir.

32.3 Görsel Geçişler ve Efektler

Patlama efektleri dışında oyunlarda kullanılan birçok görsel geçiş ve efekt vardır. Bu konuda en yaygın olanları inceleyeceğiz: fade (solma), blink (yanıp sönme), screen shake (ekran sarsılması) ve bunları yöneten bir `EffectManager` sınıfı.

32.3.1 Fade (Solma) Efekt

Fade efekti, bir yüzeyin şeffaflığını kademeli olarak değiştirerek görünmesini veya kaybolmasını sağlar. Sahne geçişlerinde, hasar alındığında veya öğelerin belirmesinde kullanılır.

Python – FadeEfekti – Kurucu Metot

```

1 class FadeEfekti:
2     """Ekran solma efekti (fade in / fade out).
3
4     Tam ekran siyah overlay ile çalışır.
5     """
6
7     def __init__(self, sure_ms=1000, yon="out"):
8         """
9         Args:
10            sure_ms: Efekt suresi (milisaniye).
11            yon: "in" = siyahtan açık, "out" = açıktan
12                ↪ siyah.
13            """
14        self.sure = sure_ms

```

```

14     self.yon = yon
15     self.baslangic = pygame.time.get_ticks()
16     self.bitti = False
17     self.overlay = pygame.Surface(
18         (GENISLIK, YUKSEKLIK)
19     )
20     self.overlay.fill((0, 0, 0))

```

FadeEfekti, tam ekran boyutunda siyah bir overlay yüzeyi oluşturur ve bu yüzeyin alfa (şeffaflık) değerini zamanla değiştirerek solma etkisi yaratır. **yon** parametresi geçiş yönünü belirler: “out” sahneyi karartır (oyun sonu, sahne geçişi), “in” ise karanlıktan aydınlığa geçiş yapar (yeni sahne başlangıcı). **pygame.Surface.set_alpha()** metodu tüm yüzeyin şeffaflığını tek seferde ayarlar; piksel bazlı alfa yerine daha performanslıdır.

Güncelleme ve çizim metotları:

Python – FadeEfekti – Güncelleme ve Çizim

```

1  def guncelle(self):
2      """Efekt durumunu guncelle."""
3      gecen = pygame.time.get_ticks() - self.baslangic
4      ilerleme = min(1.0, gecen / self.sure)
5
6      if self.yon == "out":
7          alfa = int(255 * ilerleme)
8      else: # "in"
9          alfa = int(255 * (1.0 - ilerleme))
10
11     self.overlay.set_alpha(alfa)
12
13     if ilerleme >= 1.0:
14         self.bitti = True
15
16     def ciz(self, ekran):
17         """Overlay'i ekrana ciz."""
18         ekran.blit(self.overlay, (0, 0))

```

ilerleme değeri 0.0'dan 1.0'a doğru zamanla artar. Fade out modunda alfa doğrudan bu değerle orantılıdır (0 → 255, şeffaftan opağa); fade in modunda ise tersi uygulanır (255 → 0). **min(1.0, ...)** kontrolü ilerlemenin 1.0'ı aşmasını engeller. **ciz()** metodu, oyun sahnesinin *üzerine* overlay'i çizerek çalışır; bu nedenle çizim sırasında en son çağrılmalıdır.

32.3.2 Blink (Yanıp Sönme) Efekti

Hasar alındığında karakterin yanıp sönmesi, dokunulmazlık süresini görsel olarak gösterir:

Python – BlinkEfekti – Kurucu Metot

```

1 class BlinkEfekti:
2     """Sprite yanıp sonme efekti.
3
4     Hasar sonrasi dokunulmazlik gostergesi olarak
5     ↪ kullanilir.
6     """
7
8     def __init__(self, sprite, sure_ms=1000, hiz_ms=100):
9         """
10        Args:
11        sprite: Yanıp sonecek sprite nesnesi.
12        sure_ms: Toplam yanıp sonme suresi (ms).
13        hiz_ms: Yanıp sonme hizi (ms).
14        """
15        self.sprite = sprite
16        self.sure = sure_ms
17        self.hiz = hiz_ms
18        self.baslangic = pygame.time.get_ticks()
19        self.bitti = False
20        self.gorunur = True
21        self.son_degisim = self.baslangic
22        self._orijinal_image = sprite.image.copy()

```

BlinkEfekti, doğrudan sprite nesnesine referans alarak çalışır ve sprite'ın **image** niteliğini değiştirerek yanıp sönme efekti yaratır. **_orijinal_image** ile sprite'ın mevcut görselinin bir kopyası saklanır; bu kritiktir çünkü efekt bittiğinde orijinal görsel geri yüklenmelidir. **hiz_ms** parametresi yanıp sönme frekansını belirler: 100 ms ile saniyede 5 kez yanıp söner ki bu, hasar dokunulmazlığı için tipik bir hızdır.

Güncelleme metodu zamanı takip ederek sprite'ın görünürlüğünü periyodik olarak değiştirir:

Python – BlinkEfekti – Güncelleme

```

1 def guncelle(self):
2     """Gorunurlugu degistir."""
3     simdi = pygame.time.get_ticks()
4
5     # Sure doldu mu?
6     if simdi - self.baslangic >= self.sure:
7         self.bitti = True
8         self.sprite.image = self._orijinal_image
9         return
10
11    # Yanıp sonme
12    if simdi - self.son_degisim >= self.hiz:
13        self.gorunur = not self.gorunur
14        self.son_degisim = simdi
15

```

```

16         if self.gorunur:
17             self.sprite.image = self._orijinal_image
18         else:
19             # Seffaf gorsel (gorunmez)
20             self.sprite.image = pygame.Surface(
21                 self._orijinal_image.get_size(),
22                 pygame.SRCALPHA
23             )

```

Görünmezlik, sprite'ın `image` niteliğine tamamen şeffaf bir yüzey atanarak sağlanır. `pygame.SRCALPHA` bayrağıyla oluşturulan boş bir `Surface` varsayılan olarak tamamen şeffaftır, böylece sprite grubu çizim yaptığında bu frame'de hiçbir şey görünmez. Efekt süresi dolduğunda orijinal görsel geri yüklenir ve `self.bitti = True` ile efekt yöneticisine tamamlandığı bildirilir.

32.3.3 Screen Shake (Ekran Sarsılması)

Güçlü bir çarpışma veya patlama olduğunda ekranın sarsılması, etkiyi dramatik olarak artırır:

Python – ScreenShake – Kurucu Metot

```

1  class ScreenShake:
2      """Ekran sarsilma efekti.
3
4      Cizim konumuna rastgele ofset ekleyerek
5      titresim etkisi yaratir.
6      """
7
8      def __init__(self, siddet=10, sure_ms=300):
9          """
10         Args:
11             siddet: Maksimum piksel ofseti.
12             sure_ms: Sarsilma suresi (ms).
13         """
14         self.siddet = siddet
15         self.sure = sure_ms
16         self.baslangic = pygame.time.get_ticks()
17         self.bitti = False
18         self.offset_x = 0
19         self.offset_y = 0

```

`ScreenShake`, sprite'ları doğrudan hareket ettirmek yerine tüm oyun sahnesinin çizim konumunu kaydırarak sarsılma etkisi yaratır. Bu yaklaşım çok daha basit ve verimlidir: tek bir ofset değeriyle ekrandaki her şey birlikte sarsılır. `siddet` parametresi maksimum piksel kaymasını belirler; 10 piksel orta şiddetli bir sarsılma, 5 piksel hafif bir titreme, 20 piksel ise dramatik bir deprem efekti yaratır.

Güncelleme ve uygulama metotları:

Python – ScreenShake – Güncelleme ve Uygulama

```

1 def guncelle(self):
2     """Offset degerlerini guncelle."""
3     gecen = pygame.time.get_ticks() - self.baslangic
4     ilerleme = gecen / self.sure
5
6     if ilerleme >= 1.0:
7         self.bitti = True
8         self.offset_x = 0
9         self.offset_y = 0
10        return
11
12    # Azalan siddet
13    mevcut_siddet = self.siddet * (1.0 - ilerleme)
14    self.offset_x = random.randint(
15        int(-mevcut_siddet), int(mevcut_siddet)
16    )
17    self.offset_y = random.randint(
18        int(-mevcut_siddet), int(mevcut_siddet)
19    )
20
21 def uygula(self, ekran, sahne):
22     """Sahneyi ofsetle ekrana ciz.
23
24     Args:
25         ekran: Hedef yuzey (display).
26         sahne: Tum oyun iceriginin cizdirildigi yuzey.
27     """
28     ekran.blit(sahne, (self.offset_x, self.offset_y))

```

Sarsılma şiddetinin zamanla azalması ($1.0 - \text{ilerleme}$ çarpanı) doğal bir sönümlenme efekti yaratır: başlangıçta güçlü titreşim, zamanla sakinleşme. Eğer şiddet sabit kalsa, sarsılma ani bir şekilde dururdu ve yapay görünürdü. `uygula()` metodu oyun sahnesini bir ara yüzeye çizip, bu yüzeyi ofsetli şekilde ekrana aktarır; böylece sprite konumları değişmez, sadece görüntü kayar.

32.3.4 EffectManager Sınıfı

Birden fazla efekti yönetmek için merkezi bir yönetici sınıfı kullanılır. Bu sınıf, oyun döngüsünde tek bir `guncelle()` çağrısıyla tüm aktif efektleri işler ve tamamlananları otomatik olarak temizler:

Python – EffectManager – Kurucu ve Ekleme

```

1 class EffectManager:
2     """Tum gorsel efektleri merkezi yoneten sinif."""
3
4     def __init__(self):
5         self.efektler = []
6         self.screen_shake = None

```

```

7
8     def ekle(self, efekt):
9         """Yeni efekt ekle."""
10        self.efektler.append(efekt)
11
12    def sarsilma_baslat(self, siddet=10, sure_ms=300):
13        """Screen shake efekti baslat."""
14        self.screen_shake = ScreenShake(siddet, sure_ms)

```

`screen_shake` ayrı tutulur çünkü ekran sarsılması diğer efektlerden farklı bir mekanizma kullanır: sprite'ların görselini değiştirmez, çizim ofsetini etkiler. `sarsilma_baslat()` her çağrıldığında önceki sarsılmayı yeni olanla değiştirir; böylece aynı anda birden fazla sarsılma üst üste binmez.

Güncelleme ve ofset sorgulama metotları:

Python – EffectManager – Güncelleme ve Ofset

```

1     def guncelle(self):
2         """Tum efektleri guncelle, bitenleri temizle."""
3         for efekt in self.efektler[:]:
4             efekt.guncelle()
5             if efekt.bitti:
6                 self.efektler.remove(efekt)
7
8         if self.screen_shake:
9             self.screen_shake.guncelle()
10            if self.screen_shake.bitti:
11                self.screen_shake = None
12
13    def shake_ofseti(self):
14        """Mevcut screen shake ofseti.
15
16        Returns:
17            tuple: (ofset_x, ofset_y) veya (0, 0).
18        """
19        if self.screen_shake:
20            return (self.screen_shake.ofset_x,
21                    self.screen_shake.ofset_y)
22        return (0, 0)

```

`guncelle()` metodunda `self.efektler[:]` ile listenin bir kopyası üzerinde döngü yapıyoruz; bu, döngü sırasında orijinal listeden eleman silmeyi güvenli hale getirir. Orijinal liste üzerinde silme yapılsaydı indeks kayması nedeniyle bazı efektler atlanabilirdi. `shake_ofseti()` metodu, oyun döngüsünün çizim aşamasında çağrılarak sahne yüzeyinin ne kadar kaydırılacağını bildirir.

İpucu: Juice Kuralı

Oyun tasarımcıları “oyuna juice ekle” derler. Bu, her oyuncu eylemine görsel ve işitsel geri bildirim vermektir: düşman vurulduğunda patlama + parçacık + screen shake + ses efekti. Fazla kaçırmadan dengeli kullanmak oyunu profesyonel hissettirir.

32.4 Uzay Kaçışı v4 FINAL

Bölüm 8 boyunca öğrendiğin tüm kavramları birleştirme zamanı! Bu konuda Uzay Kaçışı v4’ün final versiyonunu tamamlayacağız. v3’teki ses entegrasyonunun üzerine animasyonlar, zamanlayıcılar, parçacık efektleri, patlama animasyonları ve screen shake ekleyeceğiz.

32.4.1 v4’te Eklenen Özellikler

v3’ten v4’e eklenen tüm özellikler:

1. **AnimatedSprite alt yapısı:** Tüm sprite’lar **AnimatedSprite** tabanlı
2. **Motor animasyonu:** Oyuncu gemisinde titreyen motor ateşi
3. **Dönen bonus:** Bonus nesneleri dönerek düşer
4. **Cooldown sınıfı:** Mermi ateşleme cooldown’u
5. **Dalga sistemi:** Düşmanlar dalgalar halinde gelir, zorluk artar
6. **Patlama efekti:** Düşman yok edilince sprite + parçacık patlaması
7. **Screen shake:** Oyuncu hasar alınca ekran sarsılması
8. **Blink efekti:** Hasar sonrası dokunulmazlık yanıp sönmesi
9. **Dalga bilgisi HUD:** Mevcut dalga numarası ve durum göstergesi

32.4.2 Ana Döngü Değişiklikleri

v4’teki ana döngüde önceki versiyonlara göre üç yeni yönetici sınıf devreye girer. İlk olarak başlatma ve yapılandırma kısmını görelim:

Python – v4 Başlatma ve Yapılandırma

```

1 def main():
2     """Uzay Kacisi v4 -- FINAL."""
3     pygame.init()
4     pygame.mixer.init(
5         frequency=44100, size=-16, channels=2, buffer=512
6     )
7     ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
8     pygame.display.set_caption("Uzay Kacisi v4 - FINAL")
9     saat = pygame.time.Clock()
10
11     # Sahne yüzeyi (screen shake için)
12     sahne = pygame.Surface((GENISLIK, YUKSEKLIK))
13
14     # Yoneticiler

```



```

15     ses = SoundManager("assets/sounds")
16     efekt_yoneticisi = EffectManager()
17     dalga_yoneticisi = DalgaYoneticisi()

```

v3'ten farklı olarak burada bir **sahne** yüzeyi oluşturuyoruz. Tüm oyun içeriği doğrudan **ekran**'a değil, bu ara yüzeye çizilir; ardından **EffectManager**'dan alınan ofset ile ekrana aktarılır. Bu iki katmanlı çizim mimarisi, screen shake efektini mümkün kılar. **DalgaYoneticisi**, v3'teki sabit **set_timer()** tabanlı düşman oluşturmayı tamamen devralmıştır.

Sprite grupları ve oyun nesnelerinin oluşturulması:

Python – v4 Sprite Grupları ve Oyun Döngüsü

```

1  # Sprite gruplari
2  tum_spritelar = pygame.sprite.Group()
3  dusmanlar = pygame.sprite.Group()
4  mermiler = pygame.sprite.Group()
5  bonuslar = pygame.sprite.Group()
6  efektler = pygame.sprite.Group()
7
8  # Oyuncu (animasyonlu)
9  oyuncu = OyuncuV4()
10 tum_spritelar.add(oyuncu)
11
12 # Yildizlar
13 yildizlar = pygame.sprite.Group()
14 for _ in range(YILDIZ_SAYISI):
15     y = Yildiz(ustten_baslat=False)
16     tum_spritelar.add(y)
17     yildizlar.add(y)
18
19 skor = 0
20 oyun_aktif = True
21
22 # Ana dongu
23 calistir = True
24 while calistir:
25     saat.tick(FPS)
26     # ... olay isleme ...
27     # ... guncelleme ...
28     # ... cizim ...

```

v4'te yeni eklenen **efektler** grubu, patlama ve parçacık sprite'larını ayrı tutar. Bu ayrım önemlidir: efektler çarpışma algılamaya dahil edilmemeli, sadece görsel amaçlı kullanılmalıdır. **OyuncuV4** sınıfı **AnimatedSprite** tabanlıdır ve motor ateşi animasyonu gibi görsel iyileştirmeler içerir.

32.4.3 Çarpışma ve Efekt Entegrasyonu

Güncelleme aşamasında dalga sistemi ve çarpışma algılama birlikte çalışır. İlk olarak güncelleme ve düşman oluşturma mantığını görelim:

Python – v4 Güncelleme ve Dalga Sistemi

```

1  if oyun_aktif:
2      tum_spritelar.update()
3      efekt_yoneticisi.guncelle()
4
5      # Dalga sistemi ile dusman olustur
6      yeni_dusman = dalga_yoneticisi.guncelle()
7      if yeni_dusman:
8          tum_spritelar.add(yeni_dusman)
9          dusmanlar.add(yeni_dusman)

```

v3'te düşman oluşturma `USEREVENT` ile tetiklenirken, v4'te `DalgaYoneticisi` bu görevi üstlenir. Her frame'de `dalga_yoneticisi.guncelle()` çağrılır; döndürülen değer `None` değilse yeni bir düşman oluşturulmuş demektir ve ilgili sprite gruplarına eklenir. `efekt_yoneticisi.guncelle()` ise aktif tüm efektleri (blink, fade vb.) günceller ve bitenleri temizler.

Çarpışma algılama ve efekt tetikleme kısmı, v3'teki çarpışma mantığının üzerine görsel geri bildirimler ekler:

Python – v4 Çarpışma Algılama ve Efektler

```

1  # Mermi-dusman carpismalari
2  carpismalar = pygame.sprite.groupcollide(
3      mermiler, dusmanlar, True, True
4  )
5  for mermi, vurulanlar in carpismalar.items():
6      for dusman in vurulanlar:
7          skor += dusman.puan
8          ses.cal("patlama")
9          # Patlama efekti
10         patlama_olustur(
11             dusman.rect.centerx,
12             dusman.rect.centery,
13             tum_spritelar, efektler
14         )
15         # Kucuk screen shake
16         efekt_yoneticisi.sarsilma_baslat(5, 150)
17
18 # Oyuncu-dusman carpismalari
19 if oyuncu.alive():
20     vurulanlar = pygame.sprite.spritecollide(
21         oyuncu, dusmanlar, True
22     )
23     for dusman in vurulanlar:
24         oyuncu.can -= dusman.hasar
25         ses.cal("hasar")
26         # Buyuk screen shake
27         efekt_yoneticisi.sarsilma_baslat(12, 300)
28         # Blink efekti
29         blink = BlinkEfekti(oyuncu, 1000, 80)

```

```

30     efekt_yoneticisi.ekle(blink)
31
32     if oyuncu.can <= 0:
33         oyuncu.can = 0
34         oyuncu.kill()
35         oyun_aktif = False
36         # Büyük patlama
37         patlama_olustur(
38             oyuncu.rect.centerx,
39             oyuncu.rect.centery,
40             tum_spritelar, efektler
41         )
42         ses.muzik_durdur(fade_ms=2000)

```

Her çarpışma türü farklı şiddette geri bildirim üretir: düşman yok edildiğinde hafif sarsılma (5 piksel, 150 ms) ve patlama efekti, oyuncuya çarpışmada ise güçlü sarsılma (12 piksel, 300 ms), yanıp sönme efekti ve ses. Bu “juice” katmanları, aynı mekanik olan çarpışma algılamayı çok daha tatmin edici bir deneyime dönüştürür. Oyuncu öldüğünde büyük bir patlama efekti oluşturulur ve müzik 2 saniyelik fade-out ile söner — dramatik bir oyun sonu anı yaratılır.

32.4.4 Screen Shake ile Çizim

Screen shake efektini uygulamak için tüm oyun sahnesini bir ara yüzeye çizip, ofsete göre ekrana aktarıyoruz. Bu iki katmanlı çizim yaklaşımı v4’ün en önemli mimari farkıdır:

Python – v4 Çizim (Screen Shake Destekli)

```

1  # Sahne yuzeyine ciz
2  sahne.fill(KOYU_MAVI)
3  tum_spritelar.draw(sahne)
4  hud_ciz(sahne, font, skor, oyuncu.can,
5          len(dusmanlar), ses.master_volume)
6
7  # Dalga bilgisi
8  dalga_metin = dalga_yoneticisi.dalga_bilgisi()
9  dalga_yazi = font.render(dalga_metin, True, SARI)
10 sahne.blit(dalga_yazi, (15, 75))
11
12 # Screen shake ofseti ile ekrana aktar
13 ofset = efekt_yoneticisi.shake_ofseti()
14 ekran.fill(SIYAH) # Kenarlar için
15 ekran.blit(sahne, ofset)
16
17 pygame.display.flip()

```

32.4.5 Proje Yolculuğu

Uzay Kaçışı projesinin gelişimini gözden geçirelim:

Tablo 32.1: Uzay Kaçışı versiyonları

Versiyon	Bölüm	Eklenen Özellikler
v1	Bölüm 5	Sprite temelleri, oyuncu, düşman, yıldızlar
v2	Bölüm 6	Çarpışma, mermi, can, skor, bonus
v3	Bölüm 7	SoundManager, SFX, müzik, sessiz mod
v4	Bölüm 8	Animasyon, dalga sistemi, efektler, screen shake

● Bilgi: Tam Kaynak Kod

Uzay Kaçışı v4'ün tam kaynak kodu [kod_ornekleri/projeler/uzay_kacisi/v4_animasyon](#) dosyasında bulunmaktadır. Kodu kendi bilgisayarında çalıştırarak tüm efektleri deneyebilirsiniz.

32.5 Sonuç

32.5.1 Ünite 4 Özeti

Bu üniteye görsel efektlerin gücünü keşfettin. Konu 32.1'nde `Particle` ve `ParticleEmitter` sınıflarıyla parçacık sistemi tasarladın. Konu 32.2'nde sprite animasyonu ve parçacık efektini birleştirerek etkileyici patlama efektleri oluşturdun. Konu 32.3'de fade, blink, screen shake gibi geçiş efektlerini ve bunları yöneten `EffectManager` sınıfını uyguladın. Son olarak Konu 32.4'da tüm bu sistemleri Uzay Kaçışı v4 FINAL'de bir araya getirdin.

32.5.2 Bölüm 8 Genel Özeti

Bölüm 8'i tamamladın, tebrikler! Bu bölüm, oyunlarına görsel canlılık ve profesyonel his katan dört temel alanı kapsadı.

Bölüm 29'nda sprite sheet kavramını, `subsurface()` ile frame kesmeyi ve zamana bağlı animasyon döngüsünü öğrendin.

Bölüm 30'nda genel amaçlı `AnimatedSprite` sınıfı tasarladın, state machine ile animasyon durumlarını yönettin ve ping-pong, pause/resume gibi gelişmiş modları uyguladın.

Bölüm 31'da `get_ticks()`, `USEREVENT`, `set_timer()` ile zamanlayıcılar, `Cooldown` sınıfı ile eylem bekleme süreleri ve `DalgaYoneticisi` ile dinamik zorluk artışı sistemini kavradın.

Son olarak bu üniteye parçacık sistemi, patlama efektleri, fade, blink, screen shake ve `EffectManager` ile görsel efekt yönetimini öğrendin. Uzay Kaçışı v4 FINAL ile tüm bu sistemleri birleştirdin.

32.5.3 Anahtar Kavramlar

Tablo 32.2: Bölüm 8 – Anahtar kavramlar

Kavram	Açıklama
Sprite Sheet	Animasyon karelerinin tek görselde düzenlenmesi
AnimatedSprite	Genel amaçlı animasyonlu sprite sınıfı
State Machine	Animasyon durumları ve geçiş kuralları modeli
Delta Time	İki frame arasında geçen süre
Cooldown	Eylem tekrar kullanımı için bekleme süresi
Dalga Sistemi	Düşmanları gruplar halinde gönderen zorluk modeli
Particle	Kısa ömürlü küçük görsel parçacık
ParticleEmitter	Parçacık üreten kaynak sınıfı
Explosion	Sprite tabanlı patlama animasyonu
Screen Shake	Ekranın kısa süreli rastgele titremesi
Blink	Sprite'ın yanıp sönmesi (dokunulmazlık göstergesi)
EffectManager	Tüm görsel efektleri merkezi yöneten sınıf

32.5.4 Önemli Komutlar

Tablo 32.3: Bölüm 8 – Önemli komutlar ve sınıflar

Komut / Sınıf	İşlev
<code>subsurface(rect)</code>	Sprite sheet'ten frame keser
<code>get_ticks()</code>	Geçen süreyi ms cinsinden döndürür
<code>set_timer(olay, ms)</code>	Periyodik zamanlayıcı kurar
<code>USEREVENT + n</code>	Özel olay numarası
AnimatedSprite	Animasyonlu sprite temel sınıfı
Cooldown	Eylem bekleme süresi yönetimi
DalgaYoneticisi	Dalga bazlı düşman oluşturma
Particle / ParticleEmitter	Parçacık sistemi
Explosion	Patlama animasyonu
ScreenShake	Ekran sarsılma efekti
EffectManager	Efekt yönetim merkezi

32.5.5 Sıradaki Adım

Bölüm 9'da **Metin, UI ve Menü Sistemleri** konusuna geçeceğiz. Oyuna yazı eklemeyi (font sistemi), puan tablosu ve can çubuğu gibi HUD (Heads-Up Display) bileşenlerini, buton ve menü sistemi tasarımı ve sahne yönetimini (ana menü, oyun, duraklatma, oyun sonu) öğreneceksin. Bu bölüm ile oyunun kullanıcı arayüzü profesyonel düzeye gelecek.

● Bilgi: Pratik Yap

Bölüm 8’de öğrendiklerini pekiştirmek için şu egzersizleri dene:

- Uzay Kaçışı’na yeni bir düşman türü ekle: zigzag hareket eden ve farklı patlama efekti olan bir düşman
- Parçacık sistemini genişlet: ateş ve duman efektleri oluştur (turuncu-kırmızı parçacıklar yukarı, gri parçacıklar yavaşça yükselsin)
- Breakout oyununa patlama efekti ekle: tuğla kırılınca küçük parçacıklar saçılsın
- DalgaYöneticisi’ne boss dalgası ekle: her 5. dalgada büyük ve dayanıklı bir boss düşman gelsin
- Cooldown göstergesini HUD’a entegre et: mermi ve özel silah cooldown’larını görsel çubuklarla göster

Metin Render Etme

“Bir oyunda metin olmadan oyuncuya hiçbir şey anlatamazsın. Skor, can, mesaj — hepsi metinle başlar.”

— ANONİM OYUN GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- PyGame’in font sistemi ve `pygame.font` modülünün yapısı
- `Font()` ve `SysFont()` ile yazı tipi oluşturma
- `render()` ile metin yüzeyine dönüştürme ve `blit()` ile ekrana çizme
- Çok satırlı metin, metin hizalama ve dinamik metin güncelleme teknikleri

33.1 Font Sistemi

Oyun ekranında bir skor göstermeyi, “GAME OVER” yazmayı ya da oyuncuya bir mesaj iletmeyi düşün. Tüm bunlar için ekrana *metin* çizmen gerekiyor. PyGame’de metin çizimi, görselleri ekrana çizmekten biraz farklı çalışır: önce bir *font* (yazı tipi) nesnesi oluşturursun, sonra bu nesneyle metni bir *Surface*’e dönüştürürsün ve ardından bu Surface’i ekrana `blit()` ile çizersin.

■ Tanım: Font (Yazı Tipi)

Font, harflerin ve karakterlerin görünüşünü belirleyen bir tasarım şablonudur. Her font ailesi (Arial, Times New Roman, Courier vb.) harfleri farklı şekillerde çizer. PyGame’de font nesnesi, metni piksel tabanlı bir yüzeye dönüştürmek için kullanılır.

PyGame’in font sistemi iki temel sınıf sunar: `pygame.font.Font` ve `pygame.font.SysFont`. İkisi arasındaki fark, yazı tipinin nereden yüklendiğidir.

33.1.1 pygame.font Modülü

Font sistemi `pygame.font` modülü altında yer alır. Bu modülü kullanabilmen için PyGame'in başlatılmış olması gerekir — `pygame.init()` çağrısı zaten font modülünü de başlatır.

Python – Font Modülünü Kontrol Etme

```
1 import pygame
2
3 pygame.init()
4
5 # Font modulu baslatildi mi?
6 print(pygame.font.get_init()) # True
7
8 # Sistemdeki font listesi
9 fontlar = pygame.font.get_fonts()
10 print(f"Sistemde {len(fontlar)} font var")
11 print(fontlar[:5]) # İlk 5 fontu goster
```

`get_fonts()` fonksiyonu, işletim sisteminde yüklü olan tüm fontların isimlerini küçük harfle döndürür. Bu liste her bilgisayarda farklı olabilir — bu yüzden oyunlarda genellikle kendi font dosyasını projeye birlikte taşımak en güvenli yaklaşımdır.

33.1.2 Font() ile Dosyadan Yükleme

`pygame.font.Font()` sınıfı, bir font dosyasını (.ttf veya .otf) doğrudan yükler. Bu yöntem, oyununla birlikte taşıdığın bir font dosyasını kullanmak istediğinde idealdir.

Python – Font Dosyasından Yükleme

```
1 # Kendi font dosyanla (proje dizininde)
2 baslik_fontu =
3     pygame.font.Font("assets/fonts/PressStart2P.ttf", 36)
4
5 # PyGame'in varsayılan fontu (None = freesansbold)
6 varsayilan = pygame.font.Font(None, 28)
```

İlk parametre font dosyasının yoludur. `None` geçersen PyGame'in yerleşik varsayılan fontu (freesansbold) kullanılır. İkinci parametre ise *punto* (point size) — yazı boyutudur. Punto değeri ne kadar büyükse harfler o kadar büyük çizilir.

⚠ Dikkat: Font Dosyası Bulunamadı Hatası

Eğer belirttiğin dosya yolu yanlışsa veya dosya yoksa PyGame bir `FileNotFoundError` hatası fırlatır. Font dosyalarını projenin `assets/fonts/` dizininde tutmayı ve yolları doğru vermeyi unutma.

33.1.3 SysFont() ile Sistem Fontu

`pygame.font.SysFont()` ise işletim sisteminde yüklü olan bir fontu ismiyle yükler. Hızlı prototipleme ve test için kullanışlıdır ancak her bilgisayarda aynı fontun bulunacağı garanti değildir.

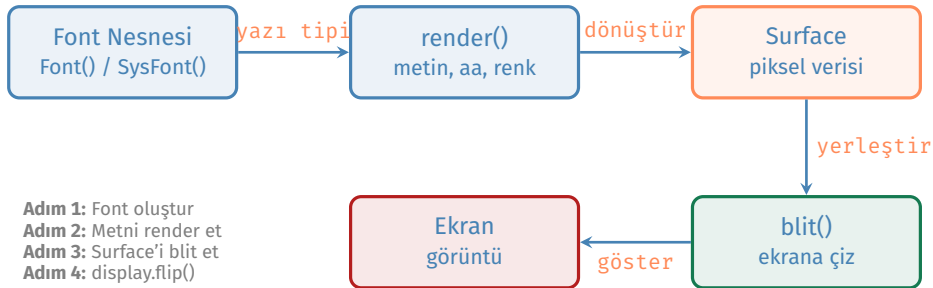
Python – Sistem Fontu Kullanımı

```
1 # Sistem fontunu isimle yükle
2 arial = pygame.font.SysFont("arial", 24)
3
4 # Kalın ve italik seçenekleri
5 kalin = pygame.font.SysFont("arial", 24, bold=True)
6 italik = pygame.font.SysFont("arial", 24, italic=True)
7 kalin_italik = pygame.font.SysFont("arial", 24, bold=True,
  ↳ italic=True)
```

● Bilgi: Font() vs SysFont()

`Font()` kendi dosyanı yükler, her bilgisayarda aynı sonucu verir. `SysFont()` sisteme bağlıdır, farklı bilgisayarlarda farklı görünebilir. Dağıtacağın oyunlarda `Font()` ile kendi font dosyanı kullan; geliştirme sırasında hızlı test için `SysFont()` veya `Font(None, boyut)` kullanabilirsin.

PyGame Metin Render Akışı



Şekil 33.1: PyGame’de metin render etme akışı: Font → render() → Surface → blit()

33.2 Metin Oluşturma ve Ekrana Çizme

Bir font nesnesi oluşturduktan sonra sıra metni ekrana çizmeye geldi. PyGame’de metin doğrudan ekrana yazılmaz — önce `render()` ile bir `Surface`’e dönüştürülür, sonra bu `Surface` ekrana `blit()` ile yerleştirilir. Bu iki adımlı süreç ilk başta garip gelebilir ama aslında çok mantıklıdır: metin yüzeyini bir kere oluşturup istediğin yere, istediğin kadar çizebilirsin.

33.2.1 render() Metodu

`render()` metodu, bir metin dizgesini piksel tabanlı bir Surface nesnesine dönüştürür.

Python – render() Kullanımı

```
1 font = pygame.font.Font(None, 36)
2
3 # render(metin, antialias, renk, arka_plan=None)
4 metin_yuzeyi = font.render("Merhaba PyGame!", True, (255,
    ↪ 255, 255))
```

Parametreleri inceleyelim:

1. **metin**: Ekrana yazılacak metin dizgesi
2. **antialias**: `True` ise kenarlar yumuşatılır (daha düzgün görünüm), `False` ise piksel piksel çizilir
3. **renk**: Metnin rengi — RGB tuple
4. **arka_plan** (opsiyonel): Metin arka plan rengi; verilmezse şeffaf olur

💡 İpucu: Antialias Seçimi

Çoğu durumda `antialias=True` kullan — metin çok daha okunaklı görünür. Ancak piksel sanatı tarzı oyunlarda `False` tercih edilebilir çünkü kenar yumuşatma piksel estetiğini bozar.

33.2.2 blit() ile Ekrana Yerleştirme

`render()` bir Surface döndürür. Bu Surface'i ekrana çizmek için `blit()` kullanırsın — tıpkı Bölüm 17'te görselleri ekrana çizdiğin gibi.

Python – Metin Çizme Döngüsü

```
1 import pygame
2
3 pygame.init()
4 ekran = pygame.display.set_mode((800, 600))
5 saat = pygame.time.Clock()
6 font = pygame.font.Font(None, 48)
7
8 calistir = True
9 while calistir:
10     for olay in pygame.event.get():
11         if olay.type == pygame.QUIT:
12             calistir = False
13
14     ekran.fill((30, 30, 50))
15
16     # Metni Surface'e donustur
```

```

17     metin = font.render("Merhaba PyGame!", True, (255,
    ↪     255, 0))
18     # Surface'i ekrana çiz
19     ekran.blit(metin, (250, 280))
20
21     pygame.display.flip()
22     saat.tick(60)
23
24     pygame.quit()

```

Bu kodda `font.render()` her karede çağrılıyor. Metin değişmiyorsa bunu döngü dışına taşıyarak performans kazanabilirsin — Surface'i bir kere oluşturup her karede sadece `blit()` çağırırsın.

33.2.3 Arka Plan Renkli Metin

`render()` fonksiyonunun dördüncü parametresi arka plan rengini belirler. Bu özellik, metnin arkasına renkli bir kutu çizmek istediğinde kullanışlıdır.

Python – Arka Plan Renkli Metin

```

1  # Seffaf arka plan (varsayılan)
2  metin1 = font.render("Seffaf Arka Plan", True, (255, 255,
    ↪  255))
3
4  # Renkli arka plan
5  metin2 = font.render("Mavi Arka Plan", True, (255, 255,
    ↪  255), (0, 0, 128))

```

● Bilgi: Surface Boyutu

`render()` ile oluşturulan Surface'in boyutu, metnin uzunluğuna ve font boyutuna göre otomatik belirlenir. Bu boyutu `metin_yuzeyi.get_size()` ile öğrenebilirsin. Metin hizalamasında bu bilgi çok işe yarayacak.

33.2.4 Metin Boyutu Hesaplama

Font nesnesinin `size()` metodu, bir metni render etmeden boyutlarını öğrenmeni sağlar. Bu, metin yerleşimi planlarken çok kullanışlıdır.

Python – Metin Boyutu Hesaplama

```

1  font = pygame.font.Font(None, 36)
2
3  # Metnin piksel boyutlarını öğren
4  genislik, yukseklik = font.size("Skor: 1000")
5  print(f"Metin boyutu: {genislik}x{yukseklik} piksel")
6

```

```

7  # Font yukseklik bilgileri
8  print(f"Satir yuksekligi: {font.get_linesize()} piksel")
9  print(f"Ascent: {font.get_ascent()} piksel")
10 print(f"Descent: {font.get_descent()} piksel")

```

`get\_linesize()` fontu kullanarak çok satırlı metin yazacağın zaman satırlar arası boşluğu belirlemek için kullanılır. `get\_ascent()` harflerin taban çizgisinin üstündeki yüksekliği, `get\_descent()` ise altındaki derinliği verir.

33.3 Metin Hizalama ve Çok Satırlı Metin

Metni ekranın tam ortasına yerleştirmek, sağa yaslamak veya birden fazla satır halinde göstermek istediğinde basit bir `blit()` çağrısı yetmez. Metnin Surface boyutunu hesaplayarak konumunu ayarlamak gerekir. Bu bölümde metin hizalama tekniklerini ve çok satırlı metin çizimini öğreneceksin.

33.3.1 Yatay ve Dikey Hizalama

Bir metni ekranın ortasına yerleştirmek için metin Surface'inin `Rect` nesnesini kullanabilirsin. Bu, Bölüm 21'te öğrendiğin `get\_rect()` yöntemine benzer.

Python – Metni Ortaya Hizalama

```

1  font = pygame.font.Font(None, 48)
2  metin = font.render("Ortaya Hizali", True, (255, 255,
   ↪ 255))
3
4  # Metin Rect'ini al ve merkezini ekran merkezine ayarla
5  metin_rect = metin.get_rect(center=(400, 300))
6  ekran.blit(metin, metin_rect)

```

Bu teknik çok güçlüdür çünkü metnin uzunluğu ne olursa olsun otomatik olarak ortaya hizalanır. `get_rect()` fonksiyonuna farklı konum parametreleri vererek farklı hizalamalar yapabilirsin:

Tablo 33.1: Metin hizalama seçenekleri

Parametre	Açıklama
<code>center=(x, y)</code>	Merkezden hizalama (tam ortaya)
<code>toleft=(x, y)</code>	Sol üst köşeden (varsayılan)
<code>topright=(x, y)</code>	Sağ üst köşeden (sağa yaslama)
<code>midtop=(x, y)</code>	Üst kenarın ortasından
<code>midbottom=(x, y)</code>	Alt kenarın ortasından
<code>midleft=(x, y)</code>	Sol kenarın ortasından
<code>midright=(x, y)</code>	Sağ kenarın ortasından

33.3.2 Çok Satırlı Metin

`render()` metodu tek satırlık metin üretir — satır atlama karakteri (`\n`) desteklemez. Çok satırlı metin göstermek için metni satırlara bölüp her satırı ayrı ayrı render etmen gerekir.

Python – Çok Satırlı Metin Fonksiyonu

```
1 def cok_satirli_metin(ekran, metin, font, renk, x, y):
2     """Cok satirli metni ekrana cizer."""
3     satirlar = metin.split("\n")
4     satir_yuksekligi = font.get_linesize()
5
6     for i, satir in enumerate(satirlar):
7         metin_yuzeyi = font.render(satir, True, renk)
8         ekran.blit(metin_yuzeyi, (x, y + i *
9             ↪ satir_yuksekligi))
```

Bu fonksiyon, verilen metni `\n` karakterinden böler ve her satırı bir öncekinin altına çizer. `get_linesize()` satırlar arası uygun boşluğu otomatik belirler.

33.3.3 Metin Gölgesi

Metin gölgesi, metnin arka plandan daha iyi ayrışmasını sağlar. Basit bir gölge efekti için aynı metni iki kez çizersin: önce koyu renkte birkaç piksel kaydırarak, sonra asıl renkte.

Python – Metin Gölge Efekti

```
1 def golge_metin(ekran, metin, font, renk, x, y,
2     ↪ golge_renk=(0,0,0)):
3     """Golgeli metin cizer."""
4     # Once golgeyi ciz (2 piksel kaydirmali)
5     golge = font.render(metin, True, golge_renk)
6     ekran.blit(golge, (x + 2, y + 2))
7
8     # Sonra asil metni ciz
9     asil = font.render(metin, True, renk)
10    ekran.blit(asil, (x, y))
```

💡 İpucu: Performans İpucu

Değişmeyen metinleri (başlık, etiket gibi) oyun döngüsünün dışında bir kere render et ve Surface değişkeninde sakla. Her karede sadece `blit()` çağır. Sürekli değişen metinler (skor, zamanlayıcı) için ise her karede yeniden render etmek kaçınılmazdır.

33.4 Dinamik Metin Güncelleme

Bir oyunda birçok metin sürekli değişir: skor artar, süre azalır, seviye yükselir. Bu tür metinleri her karede yeniden render etmen gerekir. Bu bölümde dinamik

metin güncelleme tekniklerini ve performans optimizasyonlarını öğreneceksin.

33.4.1 f-string ile Dinamik Metin

Python'un **f-string** özelliği, değişken değerlerini metin içine yerleştirmenin en pratik yoludur. Bu özelliği **render()** ile birlikte kullanarak dinamik metinler oluşturabilirsin.

Python – Dinamik Skor Gösterimi

```

1  skor = 0
2  can = 3
3  seviye = 1
4
5  # Her karede guncellenen metin
6  while calistir:
7      # ... olay isleme ...
8
9      ekran.fill((30, 30, 50))
10
11     # Dinamik metin olustur
12     skor_metin = font.render(f"Skor: {skor}", True, (255,
13         ↪ 255, 0))
14     can_metin = font.render(f"Can: {can}", True, (255,
15         ↪ 100, 100))
16     seviye_metin = font.render(f"Seviye: {seviye}", True,
17         ↪ (100, 255, 100))
18
19     ekran.blit(skor_metin, (10, 10))
20     ekran.blit(can_metin, (10, 50))
21     ekran.blit(seviye_metin, (650, 10))
22
23     pygame.display.flip()
```

Bu yaklaşımda her karede üç metin yeniden render ediliyor. Basit bir oyun için bu sorun oluşturmaz, ancak çok sayıda metin varsa performansı düşürebilir.

33.4.2 Koşullu Render Optimizasyonu

Metni sadece değiştiğinde yeniden render etmek, performansı önemli ölçüde artırır. Bunun için önceki değeri saklayıp karşılaştırma yapabilirsin.

Python – Koşullu Render

```

1  onceki_skor = -1
2  skor_yuzeyi = None
3
4  while calistir:
5      # ... oyun mantigi ...
6
7      # Sadece skor degistiyse yeniden render et
```

```

8     if skor != onceki_skor:
9         skor_yuzeyi = font.render(f"Skor: {skor}", True,
10             ↪ (255, 255, 0))
11         onceki_skor = skor
12
13     ekran.blit(skor_yuzeyi, (10, 10))

```

33.4.3 FPS Göstergesi

Oyun geliştirme sırasında ekranda FPS (kare hızı) göstermek çok yararlıdır. `Clock` nesnesinin `get_fps()` metodu anlık kare hızını döndürür.

Python – FPS Göstergesi

```

1  saat = pygame.time.Clock()
2  fps_font = pygame.font.Font(None, 24)
3
4  while calistir:
5      # ... oyun mantigi ...
6
7      # FPS bilgisini goster
8      fps = saat.get_fps()
9      fps_metin = fps_font.render(f"FPS: {fps:.0f}", True,
10          ↪ (0, 255, 0))
11      ekran.blit(fps_metin, (730, 10))
12
13      pygame.display.flip()
14      saat.tick(60)

```

`:.0f` format belirteci, ondalık kısmı atarak tam sayı olarak gösterir. FPS değeri saniyede çizilen kare sayısını temsil eder — 60'a yakın değerler akıcı bir oyun deneyimi anlamına gelir.

33.4.4 Zamanlayıcı Metin

Geri sayım veya geçen süreyi göstermek için `pygame.time.get_ticks()` fonksiyonunu kullanabilirsin. Bu fonksiyon, PyGame başlatıldığından beri geçen süreyi milisaniye cinsinden döndürür.

Python – Geri Sayım Zamanlayıcısı

```

1  baslangic = pygame.time.get_ticks()
2  sure_limiti = 60 # 60 saniye
3
4  while calistir:
5      # ... olay isleme ...
6
7      gecen_ms = pygame.time.get_ticks() - baslangic
8      gecen_saniye = gecen_ms // 1000
9      kalan = max(0, sure_limiti - gecen_saniye)

```

```

10
11 dakika = kalan // 60
12 saniye = kalan % 60
13 sure_metin = font.render(f"{dakika:02d}:{saniye:02d}",
14     ↪ True, (255, 255, 255))
    ekran.blit(sure_metin, (370, 10))

```

:02d format belirteci, sayıyı en az iki basamaklı olarak gösterir ve eksik basamağı sıfırla doldurur. Böylece süre “01:05” gibi düzgün formatta görünür.

⚠ Dikkat: get_ticks() Sıfırlanamaz

pygame.time.get_ticks() her zaman PyGame’in başlangıcından itibaren artar, sıfırlanamaz. Geri sayım veya bölüm süresi için başlangıç anını bir değişkende sakla ve farkı hesapla.

33.5 Sonuç

Bu ünite de PyGame’in font sistemini ve metin render etme sürecini öğrendin. Font() ve SysFont() ile yazı tipleri oluşturmayı, render() ile metni Surface’e dönüştürmeyi, farklı hizalama tekniklerini ve dinamik metin güncellemeyi uyguladın.

33.5.1 Anahtar Kavramlar

Tablo 33.2: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
Font	Harflerin görünüşünü belirleyen yazı tipi nesnesi
render()	Metin dizgesini piksel tabanlı Surface nesnesine dönüştürür
Antialias	Kenar yumuşatma; True ise metin daha düzgün görünür
blit()	Surface’i hedef yüzeye çizer
get_linesize()	Satırlar arası uygun boşluğu piksel cinsinden döndürür
f-string	Python’da değişken değerlerini metin içine yerleştirme yöntemi
get_ticks()	PyGame başlangıcından beri geçen süreyi milisaniye olarak döndürür

33.5.2 Önemli Komutlar

Tablo 33.3: Ünite 1 – Temel PyGame komutları

Komut	Açıklama
<code>pygame.font.Font(dosya, boyut)</code>	Font dosyasından yazı tipi yükler
<code>pygame.font.SysFont(isim, boyut)</code>	Sistem fontunu isimle yükler
<code>font.render(metin, aa, renk)</code>	Metni Surface'e dönüştürür
<code>ekran.blit(surface, konum)</code>	Surface'i ekrana çizer
<code>font.size(metin)</code>	Metnin piksel boyutlarını döndürür
<code>font.get_linesize()</code>	Satır yüksekliğini döndürür
<code>surface.get_rect(**kw)</code>	Surface'in Rect'ini döndürür
<code>pygame.font.get_fonts()</code>	Sistem fontlarının listesini döndürür

33.5.3 Sıradaki Adım

Bir sonraki ünite de metin bilgini kullanarak oyun içi UI elemanları tasarlayacaksınız. Skor göstergesi, sağlık barı ve HUD (Heads-Up Display) gibi bileşenler oluşturacaksınız. Artık ekrana metin yazabiliyorsun — şimdi bu metni anlamlı bilgi panellerine dönüştürme zamanı.

“Oyuncuya bilgiyi göstermek, bilgiyi hesaplamak kadar önemlidir. İyi bir HUD, oyuncuyu oyunun içinde tutar.”

— ANONİM UI TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Skor göstergesi tasarlama ve ekrana yerleştirme
- Sağlık barı (health bar) çizme ve güncelleme
- HUD (Heads-Up Display) sistemi oluşturma
- Birden fazla UI elemanını koordineli çalıştırma

34.1 Skor Göstergesi

Oyunlarda skor, oyuncunun performansını ölçen en temel göstergedir. Bölüm 33’de öğrendiğin metin render tekniklerini şimdi bir skor göstergesi bileşenine dönüştürelim. Bu bileşen, skoru takip edecek, güncelleyecek ve ekrana çizecek.

■ Tanım: Skor Göstergesi

Skor göstergesi, oyuncunun mevcut puanını ekranın belirli bir köşesinde sürekli gösteren bir UI (kullanıcı arayüzü) bileşenidir. Skor değiştiğinde metin otomatik olarak güncellenir.

34.1.1 Basit Skor Göstergesi

En basit haliyle skor göstergesi, bir değişkenin değerini ekrana yazdırmaktır. Ancak bunu bir sınıf olarak tasarlamak, kodu düzenli tutar ve tekrar kullanılabilirliği artırır.

Python – SkorGostergesi Sınıfı

```

1  class SkorGostergesi:
2      """Ekranda skor gosteren UI bileseni."""
3
4      def __init__(self, x, y, font_boyutu=32):
5          self.x = x
6          self.y = y
7          self.skor = 0
8          self.font = pygame.font.Font(None, font_boyutu)
9          self._guncelle_yuzeyi()
10
11     def _guncelle_yuzeyi(self):
12         """Metin yuzeyini yeniden olusturur."""
13         self.yuzeyi = self.font.render(
14             f"Skor: {self.skor}", True, (255, 255, 0)
15         )
16
17     def arttir(self, miktar=1):
18         """Skoru belirtilen miktar kadar arttir."""
19         self.skor += miktar
20         self._guncelle_yuzeyi()
21
22     def ciz(self, ekran):
23         """Skor gostergesini ekrana cizer."""
24         ekran.blit(self.yuzeyi, (self.x, self.y))

```

Bu sınıfta `_guncelle_yuzeyi()` metodu, metin Surface'ini sadece skor değiştiğinde yeniden oluşturur. Her karede render çağırmak yerine önbellege alınmış Surface'i blit etmek çok daha verimlidir.

34.1.2 Gelişmiş Skor Göstergesi

Skor göstergesine ikon, arka plan ve animasyon efektleri ekleyerek daha profesyonel bir görünüm elde edebilirsiniz.

Python – Gelişmiş Skor Göstergesi

```

1  class GelismisGostergesi:
2      """Arka planli ve ikonlu skor gostergesi."""
3
4      def __init__(self, x, y):
5          self.x = x
6          self.y = y
7          self.skor = 0
8          self.font = pygame.font.Font(None, 28)
9          self.arkaplan_renk = (0, 0, 0, 150) # Yari seffaf
10             ↪ siyah
11          self.metin_renk = (255, 215, 0) # Altin rengi
12          self._guncelle_yuzeyi()

```

```

13 def _guncelle_yuzeyi(self):
14     self.metin = self.font.render(
15         f"Skor: {self.skor:06d}", True,
16         self.metin_renk
17     )
18     # Arka plan dikdortgeni
19     self.arkaplan = pygame.Rect(
20         self.x - 5, self.y - 3,
21         self.metin.get_width() + 10,
22         self.metin.get_height() + 6
23     )
24
25 def ciz(self, ekran):
26     pygame.draw.rect(ekran, (0, 0, 0), self.arkaplan)
27     pygame.draw.rect(ekran, self.metin_renk,
28         self.arkaplan, 1)
29     ekran.blit(self.metin, (self.x, self.y))

```

:06d format belirteci, skoru altı basamaklı gösterir ve eksik basamakları sıfırla doldurur. Böylece skor “000150” gibi düzgün bir formatta görünür — arcade oyunlarının klasik tarzı.

34.2 Sağlık Barı

Sağlık barı (health bar), oyuncunun kalan canını görsel olarak gösteren en yaygın UI elemanıdır. Bir dikdörtgenin dolu kısmı mevcut canı, boş kısmı kaybedilen canı temsil eder. Bu bölümde `pygame.draw.rect()` ile sağlık barı çizmeyi öğreneceksin.

■ Tanım: Sağlık Barı (Health Bar)

Sağlık barı, bir değerin maksimum değerine oranını yatay bir çubuk olarak gösteren UI elemanıdır. Genellikle yeşilden kırmızıya değişen renk gradyanı ile canın durumunu sezgisel şekilde iletir.

34.2.1 Temel Sağlık Barı

Sağlık barı çizmek için iki dikdörtgen kullanırsın: biri arka plan (maksimum can), diğeri ön plan (mevcut can).

Python – Temel Sağlık Barı

```

1 def saglik_bari_ciz(ekran, x, y, genislik, yukseklik, can,
2   max_can):
3     """Sağlık barını ekrana çizer."""
4     oran = can / max_can
5     doluluk = int(genislik * oran)

```

```

6      # Arka plan (kirmizi - kaybedilen can)
7      pygame.draw.rect(ekran, (200, 0, 0), (x, y, genislik,
8      ↪ yukseklik))
9      # Ön plan (yesil - mevcut can)
10     pygame.draw.rect(ekran, (0, 200, 0), (x, y, doluluk,
11     ↪ yukseklik))
12     # Cerceve
13     pygame.draw.rect(ekran, (255, 255, 255), (x, y,
14     ↪ genislik, yukseklik), 2)

```

`oran` değişkeni 0.0 ile 1.0 arasında bir değerdir. Bu oranı barın genişliğiyle çarparak dolu kısmın piksel genişliğini hesaplarsın. Önce kırmızı arka planı, sonra üzerine yeşil ön planı çizersin — böylece can azaldıkça kırmızı kısım görünür hale gelir.

34.2.2 Renk Geçişli Sağlık Barı

Daha profesyonel bir görünüm için canın durumuna göre renk değiştiren bir sağlık barı tasarlayabilirsin.

Python – Renk Geçişli Sağlık Barı

```

1  def renkli_saglik_bari(ekran, x, y, genislik, yukseklik,
2  ↪ can, max_can):
3      """Can durumuna gore renk degistiren saglik bari."""
4      oran = max(0, can / max_can)
5      doluluk = int(genislik * oran)
6
7      # Orana gore renk hesapla
8      if oran > 0.6:
9          renk = (0, 200, 0)          # Yesil - iyi durumda
10     elif oran > 0.3:
11         renk = (255, 200, 0)        # Sari - dikkat
12     else:
13         renk = (200, 0, 0)          # Kirmizi - tehlike
14
15     # Arka plan
16     pygame.draw.rect(ekran, (60, 60, 60), (x, y, genislik,
17     ↪ yukseklik))
18     # Dolu kisim
19     if doluluk > 0:
20         pygame.draw.rect(ekran, renk, (x, y, doluluk,
21         ↪ yukseklik))
22     # Cerceve
23     pygame.draw.rect(ekran, (200, 200, 200), (x, y,
24     ↪ genislik, yukseklik), 2)

```

Can yüzde 60'ın üzerindeyse yeşil, yüzde 30 ile 60 arasındaysa sarı, yüzde 30'un altındaysa kırmızı renkte gösterilir. Bu renk geçişi, oyuncuya canının durumunu bir bakışta anlamasını sağlar.

34.2.3 SağlıkBari Sınıfı

Sağlık barını bir sınıf olarak tasarlayarak daha düzenli ve yeniden kullanılabilir hale getirelim.

Python – SağlıkBari Sınıfı

```

1  class SağlıkBari:
2      """Yeniden kullanılabilir sağlık bari bileşeni."""
3
4      def __init__(self, x, y, genişlik, yükseklik,
5          ↪ max_can):
6          self.rect = pygame.Rect(x, y, genişlik, yükseklik)
7          self.max_can = max_can
8          self.can = max_can
9
10     def hasar_al(self, miktar):
11         """Can değerini azaltır."""
12         self.can = max(0, self.can - miktar)
13
14     def iyiles(self, miktar):
15         """Can değerini arttırır."""
16         self.can = min(self.max_can, self.can + miktar)
17
18     def ciz(self, ekran):
19         """Sağlık barını ekrana çizer."""
20         oran = self.can / self.max_can
21         doluluk = int(self.rect.width * oran)
22
23         if oran > 0.6:
24             renk = (0, 200, 0)
25         elif oran > 0.3:
26             renk = (255, 200, 0)
27         else:
28             renk = (200, 0, 0)
29
30         pygame.draw.rect(ekran, (60, 60, 60), self.rect)
31         if doluluk > 0:
32             dolu = pygame.Rect(
33                 self.rect.x, self.rect.y,
34                 doluluk, self.rect.height
35             )
36             pygame.draw.rect(ekran, renk, dolu)
37             pygame.draw.rect(ekran, (200, 200, 200),
38                 ↪ self.rect, 2)

```

💡 İpucu: Sprite Üstünde Sağlık Bari

Düşman sprite'larının üzerinde küçük sağlık barları göstermek yaygın bir tekniktir. Bunun için barın *x* ve *y* değerlerini sprite'ın *rect* konumuna bağla ve her karede güncelle.

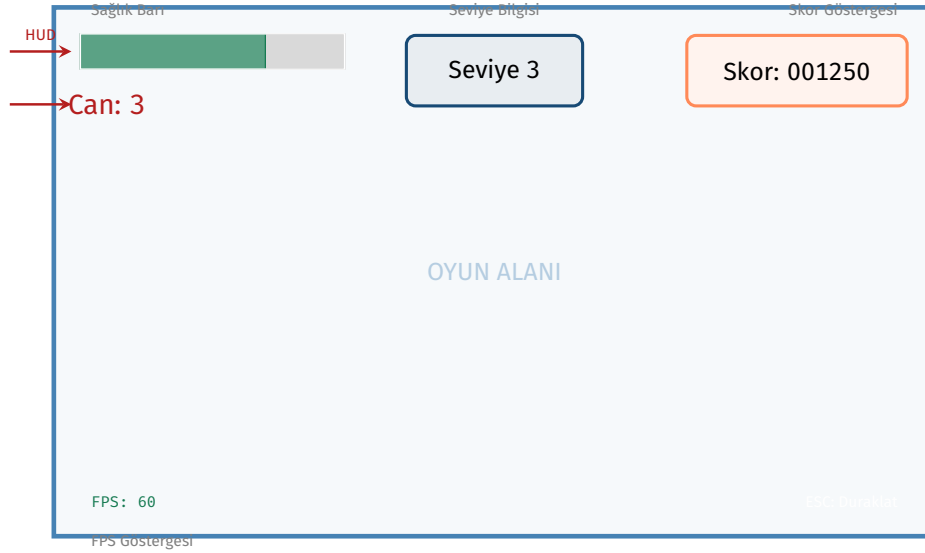
34.3 HUD Tasarımı

HUD (Heds-Up Display), oyuncuya oyun durumu hakkında anlık bilgi veren arayüz katmanıdır. Skor göstergesi, sağlık barı, seviye bilgisi, mini harita — tüm bunlar HUD'un parçalarıdır. Bu bölümde birden fazla UI elemanını bir araya getirerek tutarlı bir HUD sistemi tasarlayacaksınız.

■ Tanım: HUD (Heds-Up Display)

HUD, oyun ekranının üzerine yerleştirilen bilgi katmanıdır. Adını savaş pilotlarının gördüğü şeffaf göstergelerden alır. Oyun dünyasının bir parçası değildir — oyuncuya bilgi vermek için ekranın üstüne çizilir.

Tipik Oyun HUD Yapısı



Şekil 34.1: Tipik bir oyun HUD yapısı: skor, can barı, seviye, FPS bilgileri

34.3.1 HUD Sınıfı

Tüm UI elemanlarını bir HUD sınıfı altında toplayarak yönetimi kolaylaştırabiliriz.

Python – HUD Sınıfı

```
1 class HUD:
2     """Oyun içi bilgi gosterge paneli."""
3
4     def __init__(self, ekran_genislik, ekran_yukseklik):
5         self.font = pygame.font.Font(None, 28)
6         self.kucuk_font = pygame.font.Font(None, 20)
```

```

7         self.genislik = ekran_genislik
8         self.yukseklik = ekran_yukseklik
9
10        # UI bileşenleri
11        self.skor = 0
12        self.seviye = 1
13        self.saglik = SaglikBari(10, 10, 200, 20, 100)
14
15    def guncelle(self, skor, can, seviye):
16        """HUD verilerini gunceller."""
17        self.skor = skor
18        self.seviye = seviye
19        self.saglik.can = can
20
21    def ciz(self, ekran):
22        """Tüm HUD elemanlarını cizer."""
23        # Saglik bari (sol ust)
24        self.saglik.ciz(ekran)
25
26        # Skor (sag ust)
27        skor_metin = self.font.render(
28            f"Skor: {self.skor:06d}", True, (255, 215, 0)
29        )
30        skor_rect =
31            ↪ skor_metin.get_rect(topright=(self.genislik -
32            ↪ 10, 10))
33        ekran.blit(skor_metin, skor_rect)
34
35        # Seviye (orta ust)
36        seviye_metin = self.font.render(
37            f"Seviye {self.seviye}", True, (255, 255, 255)
38        )
39        seviye_rect =
40            ↪ seviye_metin.get_rect(midtop=(self.genislik //
41            ↪ 2, 10))
42        ekran.blit(seviye_metin, seviye_rect)

```

HUD sınıfı, tüm UI elemanlarını tek bir `ciz()` çağrısıyla ekrana yerleştirir. Oyun döngüsünde sadece `hud.guncelle()` ve `hud.ciz(ekran)` çağırman yeterlidir.

34.3.2 HUD Arka Planı

HUD elemanlarının arka planına yarı şeffaf bir panel eklemek, okuma kolaylığını artırır ve profesyonel bir görünüm sağlar.

Python – Yarı Şeffaf HUD Paneli

```

1 def hud_paneli_ciz(ekran, x, y, genişlik, yükseklik,
    ↪ alfa=150):

```



```

2     """Yarı seffaf HUD paneli cizer."""
3     panel = pygame.Surface((genislik, yukseklik))
4     panel.set_alpha(alfa)
5     panel.fill((0, 0, 0))
6     ekran.blit(panel, (x, y))

```

`set_alpha()` metodu ile `Surface`'in saydamlık değerini ayarlarsın. 0 tamamen şeffaf, 255 tamamen opak anlamına gelir. 150 civarı bir değer, arka planın görünmesini engellemeden metnin okunaklılığını artırır.

⚠ Dikkat: Alpha ve `convert_alpha()`

`set_alpha()` ile `per-pixel alpha` (`convert_alpha` ile oluşturulan `Surface`'lerdeki piksel bazlı şeffaflık) aynı anda kullanılamaz. HUD paneli için `Surface((g, y))` ile oluşturulan basit bir yüzey ve `set_alpha()` kullan.

34.4 UI Uygulaması

Bu bölümde önceki konularda öğrendiğin UI bileşenlerini bir araya getirerek tam bir HUD sistemli demo uygulama oluşturacaksın. Oyuncu hasar alacak, iyileşecek, skor kazanacak ve tüm bu değişiklikler ekranda anlık olarak gösterilecek.

34.4.1 Demo Uygulama Tasarımı

Demomuzda bir oyuncu kutusunu ok tuşlarıyla hareket ettireceksin. Ekranda rastgele beliren kırmızı engeller hasar verecek, yeşil kutular ise iyileşme sağlayacak. HUD, tüm bilgileri anlık gösterecek.

Python – HUD Demo Yapısı

```

1  # Oyun nesneleri
2  oyuncu = pygame.Rect(400, 500, 40, 40)
3  saglik_bari = SaglikBari(10, 10, 200, 20, 100)
4  skor = 0
5  font = pygame.font.Font(None, 28)
6
7  engeller = [] # Hasar veren nesneler
8  iyilesmeler = [] # Can veren nesneler

```

34.4.2 HUD Entegrasyonu

HUD elemanlarını oyun döngüsüne entegre etmek, oyun mantığı ile görsel sunum arasında köprü kurmaktır.

Python – Oyun Döngüsünde HUD

```

1 while calistir:
2     # ... olay isleme ve hareket ...
3
4     # Carpisma kontrolleri
5     for engel in engeller[:]:
6         if oyuncu.collidect(engel):
7             saglik_bari.hasar_al(10)
8             engeller.remove(engel)
9             if saglik_bari.can <= 0:
10                 oyun_durumu = "bitti"
11
12     for iyilesme in iyilesmeler[:]:
13         if oyuncu.collidect(iyilesme):
14             saglik_bari.iyiles(20)
15             iyilesmeler.remove(iyilesme)
16             skor += 50
17
18     # Cizim
19     ekran.fill((30, 30, 50))
20     # ... oyun nesnelerini ciz ...
21
22     # HUD cizimi (en son, her sey in ustune)
23     saglik_bari.ciz(ekran)
24     skor_metin = font.render(f"Skor: {skor}", True, (255,
25         ↪ 215, 0))
26     ekran.blit(skor_metin, (ekran.get_width() - 150, 10))

```

Bilgi: HUD Çizim Sırası

HUD elemanlarını her zaman **en son** çiz. Oyun nesneleri (oyuncu, düşmanlar, arka plan) önce çizilir, HUD bunların üzerine yerleştirilir. Bu sayede HUD her zaman görünür kalır.

34.4.3 Can Barı Animasyonu

Can barının ani geçişler yerine yumuşak şekilde azalması, daha profesyonel bir görünüm sağlar. Bunu mevcut değer ile hedef değer arasında kademeli geçiş yaparak elde edebilirsiniz.

Python – Animasyonlu Can Barı

```

1 class AnimasyonluSaglikBari(SaglikBari):
2     """Yumusak gecisli saglik bari."""
3
4     def __init__(self, x, y, genislik, yukseklik,
5         ↪ max_can):
6         super().__init__(x, y, genislik, yukseklik,
7             ↪ max_can)

```

```

6         self.gosterilen_can = float(max_can)
7
8     def guncelle(self):
9         """Gosterilen degeri hedefe dogru kaydirir."""
10        fark = self.can - self.gosterilen_can
11        self.gosterilen_can += fark * 0.1 # Yumusak gecis
12
13    def ciz(self, ekran):
14        oran = self.gosterilen_can / self.max_can
15        doluluk = int(self.rect.width * max(0, oran))
16        # ... geri kalan cizim kodu ayni ...

```

`fark * 0.1` ifadesi, gösterilen değeri her karede hedef değere yüzde 10 oranında yaklaştırır. Bu teknik *lerp* (linear interpolation) olarak bilinir ve oyun geliştirmede çok sık kullanılır.

34.5 Sonuç

Bu ünite de oyun içi UI elemanları tasarlamayı öğrendin. Skor göstergesi sınıfı, renk geçişli sağlık barı, HUD sistemi ve yarı şeffaf paneller oluşturmayı uyguladın. Tüm bu bileşenleri bir demo uygulamada bir araya getirdin.

34.5.1 Anahtar Kavramlar

Tablo 34.1: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
UI	Kullanıcı Arayüzü; oyuncu ile oyun arasındaki görsel iletişim katmanı
HUD	Heads-Up Display; oyun ekranının üzerine yerleştirilen bilgi göstergeleri
Skor göstergesi	Oyuncunun puanını gösteren UI bileşeni
Sağlık barı	Kalan canı orantılı bir çubuk olarak gösteren bileşen
Lerp	Linear interpolation; iki değer arasında kademeli geçiş tekniği
set_alpha()	Surface'in saydamlık değerini ayarlayan metot

34.5.2 Önemli Komutlar

Tablo 34.2: Ünite 2 – Temel komutlar

Komut	Açıklama
<code>pygame.draw.rect(s, renk, rect)</code>	Dolu dikdörtgen çizer
<code>pygame.draw.rect(s, renk, rect, w)</code>	Çerçeve çizer (w piksel kalınlıkta)
<code>surface.set_alpha(deger)</code>	Yüzeyin saydamlığını ayarlar (0–255)
<code>surface.get_width()</code>	Yüzeyin genişliğini döndürür
<code>surface.get_height()</code>	Yüzeyin yüksekliğini döndürür
<code>rect.collidect(other)</code>	İki dikdörtgenin çarpışmasını kontrol eder

34.5.3 Sıradaki Adım

Bir sonraki ünite de menü sistemlerinin temel yapı taşı olan buton sınıfını tasarlayacaksın. Hover efekti, tıklama algılama ve callback mekanizması ile etkileşimli butonlar oluşturacaksın. Bu butonlar, menü ekranlarının temelini oluşturacak.

“Bir buton, oyuncu ile oyun arasındaki en temel köprüdür. Basit görünür ama arkasında güçlü bir tasarım yatar.”

— ANONİM ARAYÜZ GELİŞTİRİCİ

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Temel bir buton sınıfı tasarlama ve çizme
- Hover (fareyle üzerine gelme) efekti ekleme
- Click (tıklama) algılama ve callback fonksiyonları
- Buton gruplama ve yeniden kullanılabilir buton bileşeni oluşturma

35.1 Temel Buton Tasarımı

Menü ekranlarının, ayar panellerinin ve oyun içi diyalogların ortak noktası nedir? Hepsinde *butonlar* vardır. “Oyuna Başla”, “Ayarlar”, “Çıkış” — tüm bu etkileşimler buton sınıfı ile gerçekleştirilir. Bu bölümde sıfırdan bir buton sınıfı tasarlayacağız.

■ Tanım: Buton (Button)

Buton, ekranda belirli bir alanı kaplayan, metin veya ikon içeren ve kullanıcı tıkladığında bir eylem tetikleyen etkileşimli UI elemanıdır. Oyun arayüzlerinin en temel yapı taşıdır.

35.1.1 Buton Sınıfının Yapısı

Bir butonun temel özellikleri şunlardır: konum, boyut, metin, renkler ve bir dikdörtgen (çarpışma alanı). Hatırla, Bölüm 21’te öğrendiğin *collidepoint()* metodu fare tıklamasını algılamak için mükemmel çalışır.

Python – Buton Kullanımı

```

1  # Buton olustur
2  basla_buton = Buton(300, 250, 200, 50, "Oyuna Basla")
3  cikis_buton = Buton(300, 320, 200, 50, "Cikis", renk=(180,
    ↪ 50, 50))
4
5  # Olay dongusunda kontrol et
6  for olay in pygame.event.get():
7      if basla_buton.tiklanildi_mi(olay):
8          print("Oyun basliyor!")
9      if cikis_buton.tiklanildi_mi(olay):
10         calistir = False

```

⚠ Dikkat: MOUSEBUTTONDOWN ve MOUSEBUTTONUP

Tıklama algılamada `MOUSEBUTTONDOWN` kullan, `MOUSEBUTTONUP` değil. Kullanıcı butona basıp fareyi butonun dışına sürükleyerek bırakabilir — bu durumda tıklama sayılmamalıdır. Ancak daha sağlam bir çözüm için her iki olayı da izlemen gerekir; bunu bir sonraki konuda göreceksin.

35.2 Hover Efekti

Profesyonel bir butona fareyi üzerine getirdiğinde görsel bir değişiklik olur — renk açılır, kenarlık belirir veya boyut büyür. Bu *hover* (üzerine gelme) efekti, kullanıcıya butonun etkileşimli bir eleman olduğunu bildirir.

35.2.1 Fare Konum Kontrolü

Hover efekti için her karede fare konumunun butonun `Rect`'i içinde olup olmadığını kontrol edersin.

Python – Hover Algılama

```

1  class Buton:
2      def __init__(self, x, y, genislik, yukseklik, metin,
3          renk=(70, 130, 180), hover_renk=(100,
    ↪ 160, 210)):
4          self.rect = pygame.Rect(x, y, genislik, yukseklik)
5          self.metin = metin
6          self.renk = renk
7          self.hover_renk = hover_renk
8          self.font = pygame.font.Font(None, 28)
9          self.uzerinde = False # Hover durumu
10
11     def guncelle(self):
12         """Her karede hover durumunu gunceller."""
13         fare_konumu = pygame.mouse.get_pos()

```

```

14         self.uzerinde =
15             ↪ self.rect.collidepoint(fare_konumu)
16
17     def ciz(self, ekran):
18         """Hover durumuna gore buton cizer."""
19         # Hover durumuna gore renk sec
20         aktif_renk = self.hover_renk if self.uzerinde else
21             ↪ self.renk
22
23         pygame.draw.rect(ekran, aktif_renk, self.rect)
24         pygame.draw.rect(ekran, (255, 255, 255),
25             ↪ self.rect, 2)
26
27         metin_yuzeyi = self.font.render(self.metin, True,
28             ↪ (255, 255, 255))
29         metin_rect =
30             ↪ metin_yuzeyi.get_rect(center=self.rect.center)
31         ekran.blit(metin_yuzeyi, metin_rect)

```

`guncelle()` metodu her karede çağrılmalıdır. `pygame.mouse.get_pos()` fare imlecinin anlık konumunu döndürür ve `collidepoint()` ile butonun sınırları içinde olup olmadığını kontrol ederiz.

35.2.2 Gelişmiş Hover Efektleri

Renk değişiminin yanı sıra kenarlık kalınlığı, metin rengi veya gölge efekti gibi ek görsel geri bildirimler ekleyebilirsiniz.

Python – Gelişmiş Hover

```

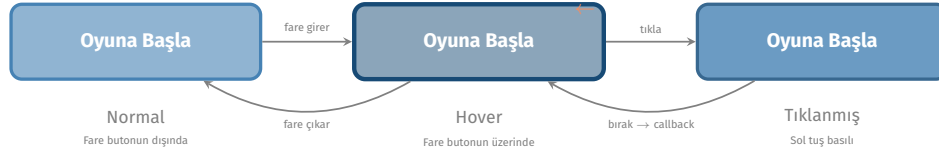
1     def ciz(self, ekran):
2         """Gelisimis hover efektli cizim."""
3         if self.uzerinde:
4             # Hover: parlak renk + kalin cerceve
5             pygame.draw.rect(ekran, self.hover_renk,
6                 ↪ self.rect)
7             pygame.draw.rect(ekran, (255, 255, 100),
8                 ↪ self.rect, 3)
9             metin_renk = (255, 255, 200)
10        else:
11            # Normal: standart gorunum
12            pygame.draw.rect(ekran, self.renk, self.rect)
13            pygame.draw.rect(ekran, (200, 200, 200),
14                ↪ self.rect, 1)
15            metin_renk = (255, 255, 255)
16
17        metin_yuzeyi = self.font.render(self.metin, True,
18            ↪ metin_renk)
19        metin_rect =
20            ↪ metin_yuzeyi.get_rect(center=self.rect.center)

```


16

ekran.blit(metin_yuzeyi, metin_rect)

Buton Durumları



Şekil 35.1: Buton durumları: normal, hover ve tıklanmış

35.2.3 İmleç Değiştirme

Fare butonun üzerindeyken imleci “el” simgesine çevirmek, kullanıcı deneyimini iyileştirir.

Python – İmleç Değiştirme

```

1 def guncelle(self):
2     fare_konumu = pygame.mouse.get_pos()
3     self.uzerinde = self.rect.collidepoint(fare_konumu)
4
5     if self.uzerinde:
6         pygame.mouse.set_cursor(pygame.SYSTEM_CURSOR_HAND)
7     else:
8         pygame.mouse.set_cursor(pygame.SYSTEM_CURSOR_ARROW)

```

⚠ Dikkat: Birden Fazla Buton ve İmleç

Birden fazla buton varsa, her butonun `guncelle()` metodu imleci ayarlar ve sonuncusu kazanır. Daha iyi bir yaklaşım, butonların hover durumlarını kontrol eden merkezi bir yönetici kullanmaktır — bunu bir sonraki konuda göreceksin.

💡 İpucu: Ses Efektini ile Hover

Bölüm 26’te öğrendiğin ses efektlerini buton etkileşimlerine de ekleyebilirsin. Fare butonun üzerine geldiğinde kısa bir “tık” sesi çalması, oyun deneyimini zenginleştirir.

35.3 Click ve Callback Mekanizması

Bir buton tıklandığında ne olacağını belirlemek için *callback* (geri çağırma) fonksiyonları kullanılır. Callback, butona “tıklandığında şu fonksiyonu çağır” demektir.

Bu yaklaşım, buton sınıfını esnek ve yeniden kullanılabilir kılar.

■ Tanım: Callback (Geri Çağırma)

Callback, bir olaya yanıt olarak çağrılmak üzere bir nesneye iletilen fonksiyondur. Buton bağlamında, tıklama olayı gerçekleştiğinde otomatik olarak çağrılacak fonksiyonu ifade eder.

35.3.1 Callback Parametresi

Buton sınıfına bir callback fonksiyonu parametre olarak vererek her butonun farklı bir eylem tetiklemesini sağlayabilirsin.

Python – Callback'li Buton

```

1  class Buton:
2      """Callback destekli buton sinifi."""
3
4      def __init__(self, x, y, genislik, yukseklik, metin,
5                  callback=None, renk=(70, 130, 180),
6                  hover_renk=(100, 160, 210)):
7          self.rect = pygame.Rect(x, y, genislik, yukseklik)
8          self.metin = metin
9          self.callback = callback
10         self.renk = renk
11         self.hover_renk = hover_renk
12         self.font = pygame.font.Font(None, 28)
13         self.uzerinde = False
14
15     def olay_isle(self, olay):
16         """Olayi isler; tiklanirsa callback cagrilir."""
17         if olay.type == pygame.MOUSEBUTTONDOWN and
18             ↪ olay.button == 1:
19             if self.rect.collidepoint(olay.pos):
20                 if self.callback:
21                     self.callback()
22                 return True
23         return False

```

35.3.2 Callback Fonksiyonları Tanımlama

Callback olarak herhangi bir fonksiyon kullanılabilir. Lambda ifadeleri de kısa eylemler için idealdir.

Python – Callback Kullanımı

```

1  # Callback fonksiyonlari
2  def oyunu_baslat():
3      print("Oyun basliyor!")
4

```

```

5 def ayarlari_ac():
6     print("Ayarlar aciliyor!")
7
8 def oyundan_cik():
9     pygame.quit()
10    sys.exit()
11
12    # Butonlari olustur
13    basla = Buton(300, 200, 200, 50, "Oyuna Basla",
14        ↳ callback=oyunu_baslat)
15    ayarlar = Buton(300, 270, 200, 50, "Ayarlar",
16        ↳ callback=ayarlari_ac)
17    cikis = Buton(300, 340, 200, 50, "Cikis",
18        ↳ callback=oyundan_cik,
19        renk=(180, 50, 50), hover_renk=(220, 80,
20        ↳ 80))

```

35.3.3 Buton Yöneticisi

Birden fazla butonu yönetmek için bir yönetici sınıfı oluşturmak kodu temiz tutar ve imleç yönetimi sorununu çözer.

Python – Buton Yöneticisi

```

1 class ButonYoneticisi:
2     """Birden fazla butonu yoneten sinif."""
3
4     def __init__(self):
5         self.butonlar = []
6
7     def ekle(self, buton):
8         """Yeni buton ekler."""
9         self.butonlar.append(buton)
10
11    def olay_isle(self, olay):
12        """Tum butonlara olayi iletir."""
13        for buton in self.butonlar:
14            buton.olay_isle(olay)
15
16    def guncelle(self):
17        """Tum butonlarin hover durumunu gunceller."""
18        fare_konumu = pygame.mouse.get_pos()
19        herhangi_hover = False
20
21        for buton in self.butonlar:
22            buton.uzerinde =
23            ↳ buton.rect.collidepoint(fare_konumu)
24            if buton.uzerinde:
25                herhangi_hover = True

```

```

26         # Imlecu merkezi olarak yonet
27         if herhangi_hover:
28             pygame.mouse.set_cursor(pygame.SYSTEM_CURSOR_
                ↳ HAND)
29         else:
30             pygame.mouse.set_cursor(pygame.SYSTEM_CURSOR_
                ↳ ARROW)
31
32     def ciz(self, ekran):
33         """Tum butonlari cizer."""
34         for buton in self.butonlar:
35             buton.ciz(ekran)

```

ButonYoneticisi sınıfı, tüm butonları tek bir listede tutar ve tek bir **guncelle()** çağrısıyla hepsinin hover durumunu, tek bir **ciz()** çağrısıyla hepsini ekrana çizer. İmleç yönetimi de merkezi olarak yapılır.

● Bilgi: Python'da Fonksiyonlar Birinci Sınıf Nesnedir

Python'da fonksiyonlar değişkenlere atanabilir, parametre olarak gönderilebilir ve listede saklanabilir. Bu özellik, callback mekanizmasını mümkün kılar. **callback=oyunu_baslat** derken fonksiyonu çağırıyorsun (parantez yok), fonksiyonun kendisini butona iletiyorsun.

35.4 Buton Uygulaması

Bu bölümde önceki konularda geliştirdiğin buton sınıfını ve buton yöneticisini kullanarak tam bir etkileşimli menü demosu oluşturacaksın. Butonlar hover efektine, tıklama geri bildirimine ve callback fonksiyonlarına sahip olacak.

35.4.1 Tam Buton Sınıfı

Önceki konularda parça parça geliştirdiğimiz buton sınıfının tamamını bir araya getirelim.

Python – Tam Buton Sınıfı

```

1  class Buton:
2      """Hover, click ve callback destekli buton."""
3
4      def __init__(self, x, y, genislik, yukseklik, metin,
5                  callback=None, renk=(70, 130, 180),
6                  hover_renk=(100, 160, 210)):
7          self.rect = pygame.Rect(x, y, genislik, yukseklik)
8          self.metin = metin
9          self.callback = callback
10         self.renk = renk
11         self.hover_renk = hover_renk

```

```

12     self.font = pygame.font.Font(None, 28)
13     self.uzerinde = False
14     self.basildi = False
15
16     def olay_isle(self, olay):
17         if olay.type == pygame.MOUSEBUTTONDOWN and
18             ↪ olay.button == 1:
19             if self.rect.collidepoint(olay.pos):
20                 self.basildi = True
21         elif olay.type == pygame.MOUSEBUTTONUP and
22             ↪ olay.button == 1:
23             if self.basildi and
24                 ↪ self.rect.collidepoint(olay.pos):
25                 if self.callback:
26                     self.callback()
27                 self.basildi = False
28
29     def ciz(self, ekran):
30         if self.basildi and self.uzerinde:
31             aktif_renk = tuple(max(0, c - 30) for c in
32                 ↪ self.renk)
33         elif self.uzerinde:
34             aktif_renk = self.hover_renk
35         else:
36             aktif_renk = self.renk
37
38         pygame.draw.rect(ekran, aktif_renk, self.rect,
39             ↪ border_radius=5)
40         pygame.draw.rect(ekran, (255, 255, 255),
41             ↪ self.rect, 2,
42                 border_radius=5)
43
44         metin_yuzeyi = self.font.render(self.metin, True,
45             ↪ (255, 255, 255))
46         metin_rect =
47             ↪ metin_yuzeyi.get_rect(center=self.rect.center)
48         ekran.blit(metin_yuzeyi, metin_rect)

```

Bu tam versiyonda üç görsel durum vardır:

1. **Normal:** Buton standart renginde
2. **Hover:** Fare üzerindeyken açık renk
3. **Basılı:** Tıklanmış ve fare hala üzerindeyken koyu renk

`MOUSEBUTTONDOWN` ile basılı durumu başlatıp `MOUSEBUTTONUP` ile callback'i çağırma tekniği, kullanıcının yanlışlıkla tıklamalarını önler. Kullanıcı butona bastıktan sonra fareyi dışarı sürükleyip bırakırsa tıklama iptal edilir.

35.4.2 Demo Menü Ekranı

Buton yöneticisini kullanarak basit bir menü ekranı oluşturalım.

Python – Menü Demo

```

1  # Durum degiskeni
2  oyun_durumu = "menu"
3
4  def baslat():
5      global oyun_durumu
6      oyun_durumu = "oyun"
7
8  # Buton yoneticisi ve butonlar
9  yonetici = ButonYoneticisi()
10 yonetici.ekle(Buton(300, 200, 200, 50, "Oyuna Basla",
    ↪ callback=baslat))
11 yonetici.ekle(Buton(300, 270, 200, 50, "Ayarlar"))
12 yonetici.ekle(Buton(300, 340, 200, 50, "Cikis",
13                 callback=lambda: sys.exit(),
14                 renk=(180, 50, 50), hover_renk=(220,
    ↪ 80, 80)))
15
16 # Ana dongu
17 while calistir:
18     for olay in pygame.event.get():
19         if olay.type == pygame.QUIT:
20             calistir = False
21         if oyun_durumu == "menu":
22             yonetici.olay_isle(olay)
23
24     if oyun_durumu == "menu":
25         yonetici.guncelle()
26         ekran.fill((20, 20, 40))
27         baslik = baslik_font.render("Oyunum", True, (255,
    ↪ 255, 255))
28         ekran.blit(baslik, baslik.get_rect(center=(400,
    ↪ 100)))
29         yonetici.ciz(ekran)
30
31     pygame.display.flip()
32     saat.tick(60)

```

💡 İpucu: **border_radius** Parametresi

`pygame.draw.rect()` fonksiyonunun `border_radius` parametresi, dik-dörtgenin köşelerini yuvarlar. Bu, butonlara modern ve zarif bir görünüm kazandırır. PyGame 2.0 ve üzeri sürümlerde desteklenir.

35.5 Sonuç

Bu ünite de oyun menülerinin temel yapı taşı olan buton sınıfını sıfırdan tasarladın. Temel buton çiziminden başlayarak hover efekti, tıklama algılama, callback mekanizması ve buton yöneticisi sınıfına kadar ilerledin. Bu bileşenlerle etkileşimli bir

menü demosu oluşturdu.

35.5.1 Anahtar Kavramlar

Tablo 35.1: Ünite 3 – Anahtar kavramlar

Kavram	Açıklama
Buton	Tıklanabilir, metin içeren etkileşimli UI elemanı
Hover	Fare imlecinin butonun üzerine gelmesi durumu
Click	Fare ile butona tıklama eylemi
Callback	Olay gerçekleştiğinde çağrılacak fonksiyon referansı
ButonYoneticisi	Birden fazla butonu merkezi olarak yöneten sınıf
border_radius	Dikdörtgen köşelerini yuvarlama parametresi

35.5.2 Önemli Komutlar

Tablo 35.2: Ünite 3 – Temel komutlar

Komut	Açıklama
<code>pygame.mouse.get_pos()</code>	Fare konumunu (x, y) olarak döndürür
<code>rect.collidepoint(pos)</code>	Noktanın dikdörtgen içinde olup olmadığını kontrol eder
<code>pygame.MOUSEBUTTONDOWN</code>	Fare düğmesi basılma olayı
<code>pygame.MOUSEBUTTONUP</code>	Fare düğmesi bırakılma olayı
<code>pygame.mouse.set_cursor()</code>	Fare imlecinin görünümünü değiştirir
<code>SYSTEM_CURSOR_HAND</code>	El şeklinde imleç sabiti
<code>SYSTEM_CURSOR_ARROW</code>	Normal ok şeklinde imleç sabiti

35.5.3 Sıradaki Adım

Bir sonraki ünite de buton sınıfını kullanarak tam menü sistemleri tasarlayacaksınız. Ana menü, pause menüsü ve game over ekranı oluşturacaksınız. Oyun durumu yönetimi ile farklı ekranlar arasında geçiş yapacak ve tüm bileşenleri bir mini projede bir araya getireceksin.

“Bir oyunun ilk izlenimi menüsüdür. Profesyonel bir menü, profesyonel bir oyunun işaretidir.”

— ANONİM OYUN TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Ana menü tasarlama ve oyun akışına entegre etme
- Pause (duraklatma) menüsü oluşturma
- Game Over ekranı ve yeniden başlatma mekaniği
- Tam bir menü sistemi ile mini projeyi tamamlama

36.1 Ana Menü

Ana menü, oyuncunun oyunla ilk karşılaştığı ekrandır. “Oyuna Başla”, “Ayarlar”, “Çıkış” gibi temel seçenekleri sunar. Bu bölümde Bölüm 35’te tasarladığın buton sınıfını kullanarak profesyonel bir ana menü oluşturacaksın.

36.1.1 Oyun Durumu Yönetimi

Menü sistemi, oyunun farklı *durumlarını* (state) yönetmekle başlar. Oyun herhangi bir anda şu durumlardan birinde olabilir: menü, oyun, duraklatma veya bitti.

■ Tanım: Oyun Durumu (Game State)

Oyun durumu, oyunun o anki modunu belirleyen bir etiket veya nesnedir. Her durumun kendi güncelleme mantığı, çizim kodu ve olay işleme davranışı vardır. Durumlar arası geçiş, oyun akışını kontrol eder.

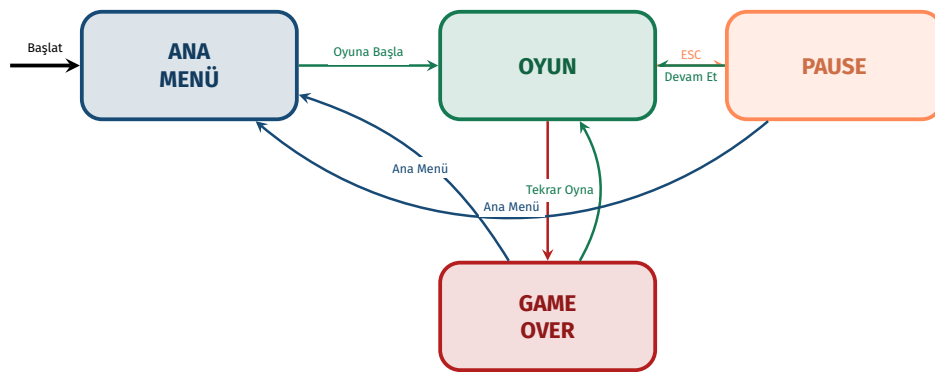
Python – Durum Yönetimi

```

1  # Oyun durumları
2  MENU = "menu"
3  OYUN = "oyun"
4  PAUSE = "pause"
5  GAME_OVER = "game_over"
6
7  oyun_durumu = MENU

```

Oyun Durumları Arası Geçişler



Şekil 36.1: Oyun durumları arası geçişler

36.1.2 Ana Menü Sınıfı

Ana menüyü bir sınıf olarak tasarlamak, kodu modüler ve yönetilebilir kılar.

Python – AnaMenu Sınıfı

```

1  class AnaMenu:
2      """Oyunun ana menu ekrani."""
3
4      def __init__(self, ekran_boyut):
5          self.genislik, self.yukseklik = ekran_boyut
6          self.baslik_font = pygame.font.Font(None, 72)
7          self.yonetici = ButonYoneticisi()
8          self.sonuc = None # Menu sonucu
9
10         # Butonlari olustur
11         orta_x = self.genislik // 2 - 100
12         self.yonetici.ekle(
13             Buton(orta_x, 250, 200, 50, "Oyuna Basla",
14                 callback=self._baslat)
15         )
16         self.yonetici.ekle(
17             Buton(orta_x, 320, 200, 50, "Cikis",

```

```

18         callback=self._cikis,
19         renk=(180, 50, 50), hover_renk=(220, 80,
20         ↪ 80))
21     )
22     def _baslat(self):
23         self.sonuc = OYUN
24
25     def _cikis(self):
26         self.sonuc = "cikis"
27
28     def olay_isle(self, olay):
29         self.yoneticisi.olay_isle(olay)
30
31     def guncelle(self):
32         self.yoneticisi.guncelle()
33
34     def ciz(self, ekran):
35         ekran.fill((20, 20, 40))
36         # Baslik
37         baslik = self.baslik_font.render("Uzay Kacisi",
38         ↪ True, (255, 255, 255))
39         baslik_rect =
40         ↪ baslik.get_rect(center=(self.genislik // 2,
41         ↪ 120))
42         ekran.blit(baslik, baslik_rect)
43         # Butonlar
44         self.yoneticisi.ciz(ekran)

```

`self.sonuc` değişkeni, menüde bir seçim yapıldığında hangi duruma geçileceğini belirtir. Ana oyun döngüsü bu değişkeni kontrol ederek durumu değiştirir.

36.1.3 Menü ile Oyun Arasında Geçiş

Ana döngüde oyun durumuna göre farklı kod blokları çalıştırılır.

Python – Durum Tabanlı Ana Döngü

```

1 ana_menu = AnaMenu((800, 600))
2
3 while calistir:
4     for olay in pygame.event.get():
5         if olay.type == pygame.QUIT:
6             calistir = False
7
8         if oyun_durumu == MENU:
9             ana_menu.olay_isle(olay)
10
11     if oyun_durumu == MENU:
12         ana_menu.guncelle()
13         ana_menu.ciz(ekran)

```

```

14
15     if ana_menu.sonuc == OYUN:
16         oyun_durumu = OYUN
17         # Oyunu baslat veya sifirla
18     elif ana_menu.sonuc == "cikis":
19         calistir = False
20
21     elif oyun_durumu == OYUN:
22         # Oyun mantigi ve cizimi
23         pass
24
25     pygame.display.flip()
26     saat.tick(60)

```

Bilgi: Durum Makinesi Yaklaşımı

Bu yapı, bilgisayar bilimlerinde *durum makinesi* (state machine) olarak bilinir. Her durum kendi olay işleme, güncelleme ve çizim mantığına sahiptir. Durumlar arası geçişler belirli koşullarla tetiklenir. Oyun geliştirmede bu desen çok yaygındır.

36.2 Pause Menüü

Oyuncu oyun sırasında ESC tuşuna bastığında oyun duraklar ve bir pause menüsü görünür. Bu menü “Devam Et”, “Ana Menü” ve “Çıkış” seçenekleri sunar. Pause menüsünün özel yanı, arka planda oyun ekranının donmuş halinin görünmesidir.

36.2.1 Pause Durumuna Geçiş

ESC tuşu ile pause durumuna geçiş yapmak için olay döngüsüne bir kontrol eklenir.

Python – Pause Geçişi

```

1  for olay in pygame.event.get():
2      if olay.type == pygame.QUIT:
3          calistir = False
4      elif olay.type == pygame.KEYDOWN:
5          if olay.key == pygame.K_ESCAPE:
6              if oyun_durumu == OYUN:
7                  oyun_durumu = PAUSE
8                  # Oyun ekraninin bir kopyasini al
9                  pause_arka_plan = ekran.copy()
10             elif oyun_durumu == PAUSE:
11                 oyun_durumu = OYUN

```

`ekran.copy()` ile oyun ekranının o anki halini bir Surface’e kopyalarsın. Bu kopyayı pause menüsünün arka planı olarak kullanırsın.

36.2.2 PauseMenu Sınıfı

Python – PauseMenu Sınıfı

```

1  class PauseMenu:
2      """Oyun duraklatma menusu."""
3
4      def __init__(self, ekran_boyut):
5          self.genislik, self.yukseklik = ekran_boyut
6          self.font = pygame.font.Font(None, 64)
7          self.yonetici = ButonYoneticisi()
8          self.sonuc = None
9
10         # Yari seffaf karartma katmani
11         self.karartma = pygame.Surface(ekran_boyut)
12         self.karartma.set_alpha(150)
13         self.karartma.fill((0, 0, 0))
14
15         orta_x = self.genislik // 2 - 100
16         self.yonetici.ekle(
17             Buton(orta_x, 250, 200, 50, "Devam Et",
18                 callback=self._devam)
19         )
20         self.yonetici.ekle(
21             Buton(orta_x, 320, 200, 50, "Ana Menu",
22                 callback=self._ana_menu)
23         )
24         self.yonetici.ekle(
25             Buton(orta_x, 390, 200, 50, "Cikis",
26                 callback=self._cikis,
27                 renk=(180, 50, 50), hover_renk=(220, 80,
28                     ↪ 80))
29         )
30
31     def _devam(self):
32         self.sonuc = OYUN
33
34     def _ana_menu(self):
35         self.sonuc = MENU
36
37     def _cikis(self):
38         self.sonuc = "cikis"
39
40     def sifirla(self):
41         """Menu sonucunu sifirlar."""
42         self.sonuc = None
43
44     def olay_isle(self, olay):
45         self.yonetici.olay_isle(olay)
46
47     def guncelle(self):
48         self.yonetici.guncelle()

```

```

48
49 def ciz(self, ekran, arka_plan):
50     """Donmuş oyun ekranı üzerine pause menüsü
51     ↪ çizilir."""
52     ekran.blit(arka_plan, (0, 0))
53     ekran.blit(self.karartma, (0, 0))
54
55     baslik = self.font.render("DURAKLATILDI", True,
56     ↪ (255, 255, 255))
57     baslik_rect =
58     ↪ baslik.get_rect(center=(self.genislik // 2,
59     ↪ 150))
60     ekran.blit(baslik, baslik_rect)
61
62     self.yonetici.ciz(ekran)

```

Pause menüsünün `ciz()` metodu iki önemli katman çizer: önce donmuş oyun ekranını (`arka_plan`), sonra yarı şeffaf siyah karartmayı (`self.karartma`). Bu, oyuncuya “oyun hala burada ama duraklatıldı” mesajını görsel olarak iletir.

36.2.3 Pause Entegrasyonu

Python – Pause Menüü Entegrasyonu

```

1  pause_menu = PauseMenu((800, 600))
2  pause_arka_plan = None
3
4  # Ana dongude:
5  if oyun_durumu == PAUSE:
6      for olay in pygame.event.get():
7          if olay.type == pygame.KEYDOWN and olay.key ==
8          ↪ pygame.K_ESCAPE:
9              oyun_durumu = OYUN
10             pause_menu.oyay_isle(olay)
11
12     pause_menu.guncelle()
13     pause_menu.ciz(ekran, pause_arka_plan)
14
15     if pause_menu.sonuc:
16         oyun_durumu = pause_menu.sonuc
17         pause_menu.sifirla()

```

💡 İpucu: Pause Sırasında Zamanlayıcılar

Oyun duraklatıldığında Bölüm 31’te öğrendiğin zamanlayıcılar da duraklatılmalıdır. Aksi halde pause sırasında olaylar birikir ve devam ettiğinde aniden tetiklenir. Pause durumuna geçerken zamanlayıcıları durdur, devam ederken yeniden başlat.

36.3 Game Over Ekranı

Oyuncunun canı bittiğinde veya belirli bir koşul sağlandığında oyun sona erer ve bir “Game Over” ekranı gösterilir. Bu ekran, oyuncunun skorunu gösterir ve yeniden başlama veya ana menüye dönme seçenekleri sunar.

36.3.1 GameOver Sınıfı

Python – GameOver Sınıfı

```

1  class GameOverEkranı:
2      """Oyun bitis ekranı."""
3
4      def __init__(self, ekran_boyut):
5          self.genislik, self.yukseklık = ekran_boyut
6          self.baslik_font = pygame.font.Font(None, 72)
7          self.skor_font = pygame.font.Font(None, 48)
8          self.yonetici = ButonYoneticisi()
9          self.sonuc = None
10         self.skor = 0
11
12         orta_x = self.genislik // 2 - 100
13         self.yonetici.ekle(
14             Buton(orta_x, 350, 200, 50, "Tekrar Oyna",
15                 callback=self._tekrar,
16                 renk=(50, 150, 50), hover_renk=(80, 180,
17                     ↪ 80))
18         )
19         self.yonetici.ekle(
20             Buton(orta_x, 420, 200, 50, "Ana Menu",
21                 callback=self._ana_menu)
22         )
23
24     def _tekrar(self):
25         self.sonuc = OYUN
26
27     def _ana_menu(self):
28         self.sonuc = MENU
29
30     def skor_ayarla(self, skor):
31         """Final skorunu ayarlar."""
32         self.skor = skor
33         self.sonuc = None
34
35     def olay_isle(self, olay):
36         self.yonetici.olay_isle(olay)
37
38     def guncelle(self):
39         self.yonetici.guncelle()
40
41     def ciz(self, ekran):

```

```

41     ekran.fill((40, 10, 10))
42
43     # GAME OVER basligi
44     baslik = self.baslik_font.render("GAME OVER",
45     ↪ True, (220, 50, 50))
46     baslik_rect =
47     ↪ baslik.get_rect(center=(self.genislik // 2,
48     ↪ 120))
49     ekran.blit(baslik, baslik_rect)
50
51     # Skor bilgisi
52     skor_metin = self.skor_font.render(
53     ↪ f"Skor: {self.skor}", True, (255, 215, 0)
54     ↪ )
55     skor_rect =
56     ↪ skor_metin.get_rect(center=(self.genislik //
57     ↪ 2, 230))
58     ekran.blit(skor_metin, skor_rect)
59
60     self.yoneticici.ciz(ekran)

```

36.3.2 Game Over Geçişi

Oyun sırasında oyuncunun canı sıfıra düştüğünde game over durumuna geçilir.

Python – Game Over Geçişi

```

1  # Oyun dongusunda:
2  if oyun_durumu == OYUN:
3      # ... oyun mantigi ...
4
5      # Can kontrolu
6      if saglik_bari.can <= 0:
7          oyun_durumu = GAME_OVER
8          game_over.skor_ayarla(skor)
9
10 elif oyun_durumu == GAME_OVER:
11     for olay in pygame.event.get():
12         game_over.olay_isle(olay)
13
14     game_over.guncelle()
15     game_over.ciz(ekran)
16
17     if game_over.sonuc == OYUN:
18         # Oyunu sifirla ve yeniden baslat
19         skor = 0
20         saglik_bari.can = saglik_bari.max_can
21         oyun_durumu = OYUN
22         game_over.sonuc = None
23 elif game_over.sonuc == MENU:

```

```

24     oyun_durumu = MENU
25     ana_menu.sonuc = None
26     game_over.sonuc = None

```

36.3.3 Yüksek Skor Tablosu

Game Over ekranına yüksek skor tablosu eklemek, oyuncuyu tekrar oynamaya motive eder.

Python – Yüksek Skor Yönetimi

```

1  class YuksekSkor:
2      """Yuksek skorlari yoneten sinif."""
3
4      def __init__(self, dosya="yuksek_skor.txt",
5          ↪ max_kayit=5):
6          self.dosya = dosya
7          self.max_kayit = max_kayit
8          self.skorlar = self._yukle()
9
10     def _yukle(self):
11         """Dosyadan skorlari yukler."""
12         try:
13             with open(self.dosya, "r") as f:
14                 ↪ return [int(s.strip()) for s in f.readlines()]
15         except FileNotFoundError:
16             return []
17
18     def kaydet(self, skor):
19         """Yeni skor ekler ve siralar."""
20         self.skorlar.append(skor)
21         self.skorlar.sort(reverse=True)
22         self.skorlar = self.skorlar[:self.max_kayit]
23         with open(self.dosya, "w") as f:
24             for s in self.skorlar:
25                 f.write(f"{s}\n")
26
27     def en_yuksek(self):
28         """En yuksek skoru dondurur."""
29         ↪ return self.skorlar[0] if self.skorlar else 0

```

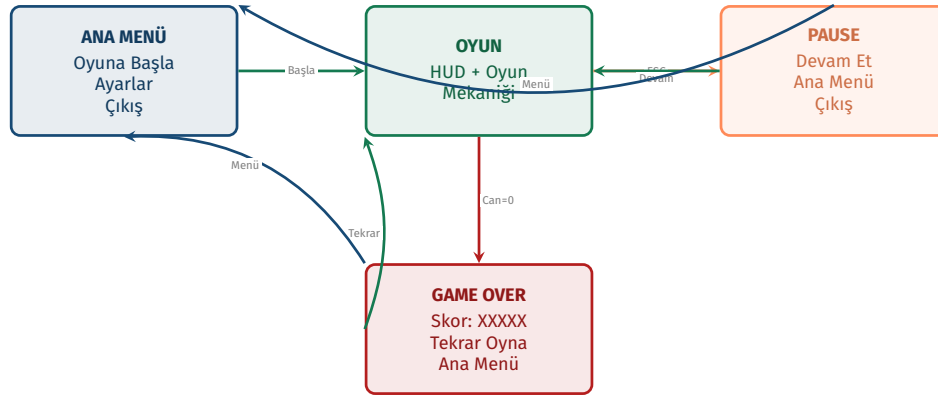
⚠ Dikkat: Dosya İşlemleri ve Hata Yönetimi

Dosya okuma/yazma işlemlerinde her zaman `try/except` kullan. Dosya bulunamayabilir, izin sorunu olabilir veya disk dolu olabilir. Oyunun dosya hatasından dolayı çökmemesi gerekir.

36.4 Mini Proje: Tam Menü Sistemi

Bu bölümde tüm ünite de öğrendiklerini bir araya getirerek tam bir menü sistemi oluşturacaksın. Ana menü, oyun ekranı, pause menüsü ve game over ekranı arasında sorunsuz geçişler yapan profesyonel bir yapı kuracaksın.

Tam Menü Sistemi Akış Diyagramı



Şekil 36.2: Tam menü sistemi akış diyagramı

36.4.1 Proje Yapısı

Tam menü sistemi projesi şu dosyalardan oluşur:

Tablo 36.1: Mini proje dosya yapısı

Dosya	Açıklama
<code>main.py</code>	Ana oyun döngüsü ve durum yönetimi
<code>buton.py</code>	Buton ve ButonYoneticisi sınıfları
<code>menu.py</code>	AnaMenu, PauseMenu ve GameOverEkranı sınıfları
<code>hud.py</code>	SaglikBari, SkorGostergesi ve HUD sınıfları
<code>oyun.py</code>	Basit oyun mekaniği (demo amaçlı)

36.4.2 Merkezi Durum Yöneticisi

Tüm ekranları ve geçişleri yöneten merkezi bir sınıf, kodu düzenli tutar.

Python – OyunYoneticisi Sınıfı

```

1 class OyunYoneticisi:
2     """Oyun durumları arası geçişleri yöneten sınıf."""
3
4     def __init__(self):
5         self.ekran = pygame.display.set_mode((800, 600))
6         self.saat = pygame.time.Clock()
```

```

7         self.calistir = True
8         self.durum = MENU
9
10        # Ekranlar
11        self.ana_menu = AnaMenu((800, 600))
12        self.pause_menu = PauseMenu((800, 600))
13        self.game_over = GameOverEkrani((800, 600))
14        self.pause_arka_plan = None
15
16        # Oyun verileri
17        self.skor = 0
18        self.hud = HUD(800, 600)
19
20    def calistir_oyun(self):
21        """Ana oyun dongusunu baslatir."""
22        while self.calistir:
23            olaylar = pygame.event.get()
24            for olay in olaylar:
25                if olay.type == pygame.QUIT:
26                    self.calistir = False
27
28                if self.durum == MENU:
29                    self._menu_isle(olaylar)
30                elif self.durum == OYUN:
31                    self._oyun_isle(olaylar)
32                elif self.durum == PAUSE:
33                    self._pause_isle(olaylar)
34                elif self.durum == GAME_OVER:
35                    self._game_over_isle(olaylar)
36
37            pygame.display.flip()
38            self.saat.tick(60)
39
40        pygame.quit()

```

36.4.3 Durum İşleme Metotları

Her durum için ayrı bir işleme metodu oluşturmak, kodu okunaklı kılar.

Python – Durum İşleme

```

1    def _menu_isle(self, olaylar):
2        for olay in olaylar:
3            self.ana_menu.olay_isle(olay)
4            self.ana_menu.guncelle()
5            self.ana_menu.ciz(self.ekran)
6
7        if self.ana_menu.sonuc == OYUN:
8            self.durum = OYUN
9            self._oyunu_sifirla()

```

```

10         self.ana_menu.sonuc = None
11     elif self.ana_menu.sonuc == "cikis":
12         self.calistir = False
13
14     def _oyunu_sifirla(self):
15         """Oyun verilerini sifirlar."""
16         self.skor = 0
17         self.hud.saglik.can = self.hud.saglik.max_can
18
19     def _pause_isle(self, olaylar):
20         for olay in olaylar:
21             if olay.type == pygame.KEYDOWN and olay.key ==
22                 ↪ pygame.K_ESCAPE:
23                 self.durum = OYUN
24                 return
25             self.pause_menu.olay_isle(olay)
26         self.pause_menu.guncelle()
27         self.pause_menu.ciz(self.ekran, self.pause_arka_plan)
28
29     if self.pause_menu.sonuc:
30         self.durum = self.pause_menu.sonuc
31         self.pause_menu.sifirla()

```

36.4.4 Projeyi Çalıştırma

Tüm bileşenleri bir araya getirerek projeyi çalıştırmak çok basittir:

Python – Projeyi Çalıştırma

```

1 import pygame
2 import sys
3
4 pygame.init()
5 yoneticisi = OyunYoneticisi()
6 yoneticisi.calistir_oyun()

```

Bu mini proje, bu bölümde öğrendiğin tüm kavramları bir arada gösterir: metin render etme, UI elemanları, buton sınıfı ve durumlar arası geçiş. Bu yapıyı kendi oyun projelerine temel olarak kullanabilirsin.

💡 İpucu: Genişletme Fikirleri

Bu menü sistemini şu özelliklerle genişletebilirsin:

- Ayarlar menüsü (ses seviyesi, çözünürlük)
- Geçiş animasyonları (fade in/out)
- Yüksek skor tablosu
- Karakter seçim ekranı

36.5 Sonuç

Bu ünite de oyun menü sistemlerini tasarlamayı öğrendin. Ana menü, pause menüsü ve game over ekranı oluşturdu. Durum makinesi yaklaşımıyla farklı ekranlar arasında geçiş mekanizması kurdun ve tüm bileşenleri bir mini projede bir araya getirdin.

36.5.1 Anahtar Kavramlar

Tablo 36.2: Ünite 4 – Anahtar kavramlar

Kavram	Açıklama
Game State	Oyunun o anki modunu belirleyen durum (menü, oyun, pause, game over)
State Machine	Durumlar ve geçişlerle oyun akışını kontrol eden tasarım deseni
Ana Menü	Oyunun başlangıç ekranı; temel seçenekleri sunar
Pause Menüsü	Oyun sırasında duraklatma ekranı
Game Over	Oyun sona erdiğinde gösterilen ekran
Yüksek Skor	Oyuncuların en iyi performanslarını kaydeden sistem

36.5.2 Önemli Komutlar

Tablo 36.3: Ünite 4 – Temel komutlar

Komut	Açıklama
<code>ekran.copy()</code>	Ekranın o anki halinin bir kopyasını oluşturur
<code>surface.set_alpha(val)</code>	Surface'in saydamlığını ayarlar (0–255)
<code>surface.fill(renk)</code>	Surface'i verilen renkle doldurur
<code>pygame.K_ESCAPE</code>	ESC tuşu sabiti
<code>open(dosya, "r"/"w")</code>	Dosya okuma/yazma işlemi
<code>list.sort(reverse=True)</code>	Listeyi büyükten küçüğe sıralar

36.5.3 Bölüm Değerlendirmesi

Bu bölümde PyGame'in font sistemiyle başlayarak tam bir oyun arayüzü oluşturmayı öğrendin:

1. **Metin Render:** Font oluşturma, `render()` ve `blit()` ile metin çizme
2. **UI Elemanları:** Skor göstergesi, sağlık barı ve HUD tasarımı
3. **Buton Sınıfı:** Hover, click ve callback mekanizması
4. **Menü Sistemleri:** Durum makinesi ile ekranlar arası geçiş

Bu bilgiler, Uzay Kaçışı projesine profesyonel bir menü sistemi ve HUD eklemek için ihtiyacın olan her şeyi sağlar. Bir sonraki bölümde tile-based oyun dünyaları oluşturacak ve oyun haritaları tasarlayacaksın.

Tile Sistemi Temelleri

*“Büyük dünyalar küçük karelerden doğar.
Her tile, dev bir maceranın yapı taşıdır.”*

— ANONİM OYUN TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Tile kavramı ve grid tabanlı dünyaların neden popüler olduğu
- Tileset’ten `subsurface()` ile tek tek tile kesme
- Piksel ve grid koordinatları arasında dönüşüm yapma
- 2B liste ile elle tanımlanmış ilk haritanı ekrana çizme

37.1 Tile Kavramı ve Grid Tabanlı Dünyalar

Oyun geliştirmeye başladığın ilk günden beri karakterleri ekranda piksel piksel yerleştiriyor, çarpışmaları tek tek `Rect`’lerle kontrol ediyordun. Peki kocaman bir platform haritası, bir RPG dünyası ya da bir bulmaca oyunu için her duvarı, her çimi, her suyu elle yerleştirmek ister misin? Tabii ki hayır. İşte bu yüzden *tile* (karo) sistemi var.

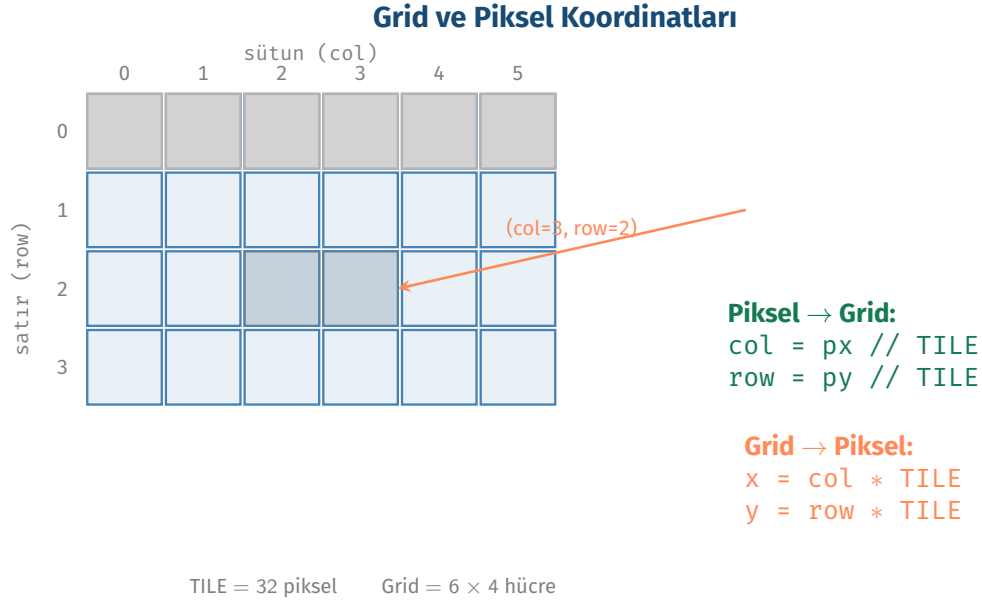
Bir tile, haritanın en küçük yapı taşıdır: sabit genişlikte ve yükseklikte (tipik olarak 32×32 ya da 64×64 piksel) bir karedir. Büyük dünyalar, bu küçük kareleri bir ızgara (grid) üzerinde yan yana dizerek üretilir. Ekrana tek bir büyük resim çizmek yerine, onlarca küçük resmi organize biçimde yerleştirirsin.

■ Tanım: Tile (Karo)

Tile, harita üzerindeki sabit boyutlu bir hücreyi temsil eden görsel birimdir. Her tile bir *tile ID*’sine sahiptir; bu ID, tile’in ne olduğunu (çim, taş, su, duvar, vb.) belirler. Harita veriyapısı, bu ID’lerden oluşan 2 boyutlu bir tablodur.

Peki neden tile sistemi kullanıyoruz? Birkaç nedeni hemen sayabilirim:

1. **Bellek verimliliği:** Her hücre yeni bir görsel değil, tile ID'sinden oluşan bir sayıdır. 100×100 bir harita sadece 10.000 tamsayıdır.
2. **Kolay düzenleme:** Harita bir 2B liste olduğundan, Tiled gibi editörlerle görsel olarak çizilebilir.
3. **Hızlı çarpışma:** Oyuncunun hangi tile üzerinde olduğunu tek bir bölme işlemiyle (`px // TILE`) bulursun.
4. **Tekrar kullanılabilir:** Aynı çim tile'ı yüzlerce hücrede bellekte tek kopya olarak durur.



Şekil 37.1: Grid tabanlı bir haritada satır/sütun ve piksel koordinatları arasındaki ilişki.

37.1.1 Harita = 2B Liste

Python'da bir tile haritasını en basit şekilde 2 boyutlu listeyle gösterirsin. Her iç liste bir satırdır, her eleman bir tile ID'sidir.

Python – Basit 2B Harita

```

1  # 0 = boş, 1 = çim, 2 = taş duvar
2  HARITA = [
3      [2, 2, 2, 2, 2, 2],
4      [2, 1, 1, 1, 1, 2],
5      [2, 1, 0, 0, 1, 2],
6      [2, 1, 1, 1, 1, 2],
7      [2, 2, 2, 2, 2, 2],
8  ]
9
10 print(f"Satır: {len(HARITA)}, Sütun: {len(HARITA[0])}")
11 print(f"(row=2, col=3) = {HARITA[2][3]}") # 0 (boş)

```

Bu haritayı okurken `HARITA[row][col]` sırasına dikkat et — önce satır, sonra sütun. Ekranı çizirken ise genellikle `(col, row)` düzeni daha doğaldır, çünkü

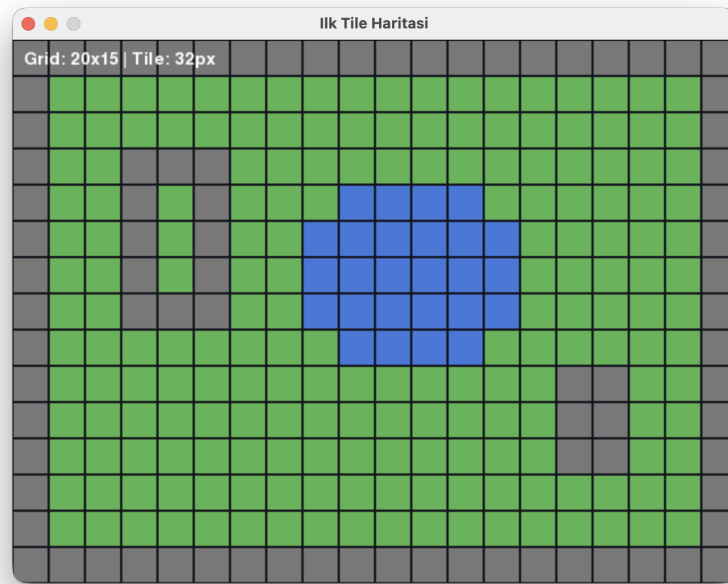
piksel koordinatları (x, y) yani (*sütun yönü*, *satır yönü*) olarak ilerler.

37.1.2 Tile Boyutu ve Ekran Çözünürlüğü

Tile boyutunu seçerken ekran çözünürlüğünü ve oyun tarzını düşün. İşte yaygın kombinasyonlar:

Tablo 37.1: Yaygın tile boyutları ve harita büyüklükleri

Tile boyutu	Ekran (grid)	Kullanım
16×16	50×37 (800×600)	Retro / NES tarzı
32×32	25×18 (800×600)	Klasik platformer
48×48	20×15 (960×720)	Modern 2B RPG
64×64	15×11 (960×720)	Büyük sprite'lı oyun



Şekil 37.2: Tile tabanlı bir haritanın önizlemesi — ilerleyen konularda adım adım oluşturacağız.

⚠ Dikkat: Satır/Sütun Karışıklığı

En sık yapılan hata `HARITA[x][y]` yazmaktır. Bu yanlıştır! Python'da 2B listede önce dış liste (satır = y / row), sonra iç liste (sütun = x / col) gelir. Yani `HARITA[row][col]` doğru sıralamadır.

💡 İpucu: Sabitleri Kullan

`TILE`, `GRID_GEN`, `GRID_YUK` gibi sabitleri modülün en üstünde tanımla. Tile boyutunu değiştirmek istediğinde tek bir yeri değiştirmen yeterli olur.

🕒 Bilgi: Pratik Alıştırma

Aşağıdaki 2B listeyi zihninde canlandır: Her 1 değeri yeşil, her 2 değeri gri olacak şekilde hangi şekil oluşur?

```
[[2,2,2,2],[2,1,1,2],[2,1,1,2],[2,2,2,2]]
```

Cevabı bir kağıda çiz; sonuç taş duvarla çevrili 2×2 çimen bir odadır. İlk haritanı böyle tasarlayabilirsin!

37.2 Tileset ve subsurface() ile Frame Kesme

Bir tile haritası için 50 farklı tile türü olsaydın, 50 ayrı PNG dosyası ister miydin? Kesinlikle hayır! Profesyonel oyunlarda tüm tile'lar tek bir büyük resimde – *tileset* (karo seti) – bir araya toplanır. Bu tileset'i bir kere yükler, sonra ihtiyacın olan tile'ı `subsurface()` ile kesip alırsın.

■ Tanım: Tileset

Tileset, birden fazla tile'ı yan yana içeren tek bir görsel dosyadır. Tile'lar sabit bir *sütun/satır* düzeninde yerleştirilmiştir, her tile aynı boyuttadır (ör. 32×32). Oyun motoru bu görseli yükler ve her tile'ı küçük bir alt yüzey (subsurface) olarak kullanır.

Tileset'i yüklemek `Surface` yükleyen her PyGame programı gibi basittir:

Python – Tileset Yükleme

```
1 import pygame
2
3 pygame.init()
4 ekran = pygame.display.set_mode((640, 480))
5
6 # Diskten yükle, şeffaflığı koru
7 tileset = pygame.image.load("assets/tileset.png").convert_
   ↳ _alpha()
8
9 print(f"Tileset boyutu: {tileset.get_size()}")
```

`convert\_alpha()` çağrısı tileset'in şeffaflık kanalını korur ve çizim hızını önemli ölçüde artırır. Tileset yüklemeden önce `display.set_mode()` çağrılmış olmalı, aksi halde `convert\_alpha()` başarısız olur.

37.2.1 subsurface() ile Tek Tek Tile Kesme

`subsurface(rect)` bir Surface'in belirli bir bölümüne *referans veren* yeni bir Surface döndürür. Bu yeni Surface bağımsız bir kopya değildir — orijinal Surface'in aynı piksellerine bakar. Bu nedenle çok hızlıdır ve bellek dostudur.

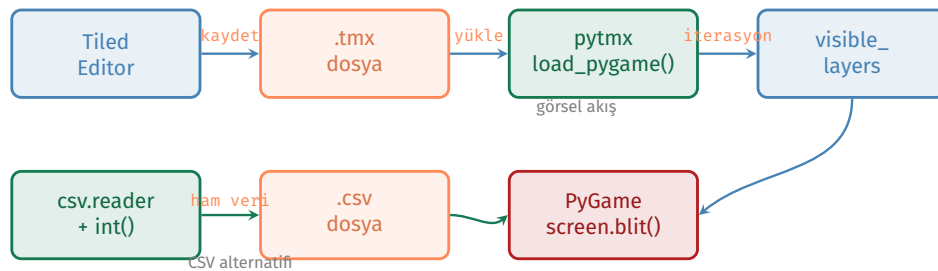
Python – Tileset'ten Tile Kesme

```

1  TILE = 32
2
3  # Tileset 4 sütun, 2 satır
4  def tileset_parcala(tileset, sutun, satir, tile_boyut):
5      tiles = []
6      for row in range(satir):
7          for col in range(sutun):
8              rect = pygame.Rect(col * tile_boyut,
9                                  row * tile_boyut,
10                                 tile_boyut, tile_boyut)
11              tiles.append(tileset.subsurface(rect))
12      return tiles
13
14  tiles = tileset_parcala(tileset, 4, 2, TILE)
15  print(f"{len(tiles)} tile kesildi")
16
17  # Her tile Surface bağımsız blit edilebilir
18  ekran.blit(tiles[3], (100, 100)) # 4. tile'ı (id=3) çiz

```

Harita Pipeline: Tiled → pytmx → PyGame



Şekil 37.3: Tileset bir kere yüklenir, sonra tüm tile'lar subsurface ile erişilir.

37.2.2 Tile ID'den Surface'e Eşleme

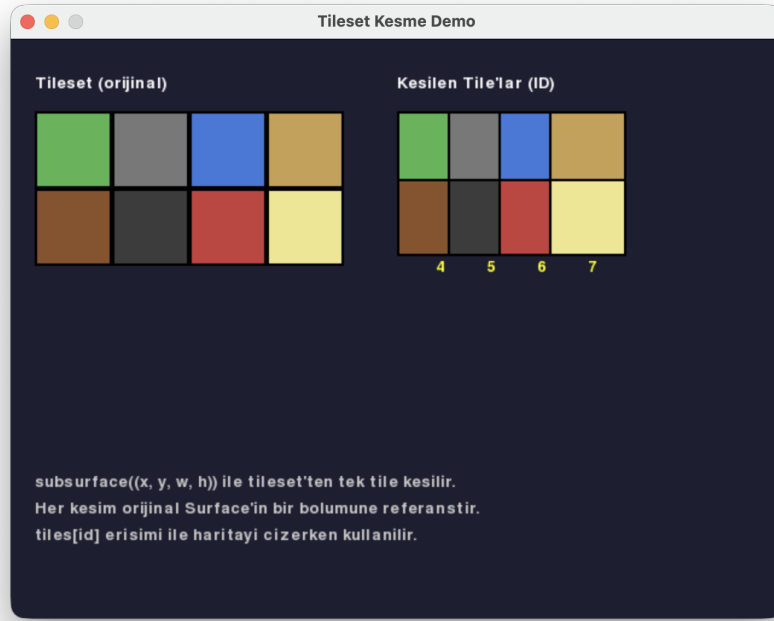
Haritanı çizerken `HARITA[row][col]` sana bir tile ID verir. Sen de bu ID'yi kullanarak doğru Surface'i bulursun:

Python – ID -> Surface Eşlemesi

```

1  # Harita verisi
2  harita = [

```



Şekil 37.4: subsurface() ile kesilmiş tile'ların ID'leriyle listelenmiş görünümü.

```

3     [0, 1, 1, 1, 2],
4     [0, 1, 0, 1, 2],
5     [0, 1, 1, 1, 2],
6 ]
7
8 # Çiz
9 for r, satir in enumerate(harita):
10     for c, tile_id in enumerate(satir):
11         ekran.blit(tiles[tile_id], (c * TILE, r * TILE))

```

Bu kısa döngü, haritanın tamamını çizmek için yeterlidir. Her hücre için doğru Surface'i bulmak sadece bir liste indeksidir — çok hızlı bir işlemdir.

⚠ Dikkat: Sınır Dışı subsurface()

Eğer `subsurface()`'e tileset'in dışına taşan bir Rect verirsen PyGame `ValueError: subsurface rectangle outside surface area` hatası fırlatır. Tileset'in satır ve sütun sayısını doğru hesapladığından emin ol: `sutun = tileset.get_width() // TILE`.

İpucu: `convert_alpha()` ile Hız

Diskten yüklenen Surface'ler yavaştır. `convert_alpha()` çağrısı Surface'i ekranın piksel formatına çevirir ve `blit()` hızını 3 – 10 kat artırır. Tileset yüklerken ya da sprite yüklerken her zaman çağır.

Bilgi: Pratik Alıştırma

`kod_ornekleri/bolum10/unite1/02_tileset_kesme.py` dosyasını çalıştır. Tileset'i 4 sütun yerine 2 sütun olarak yeniden düzenlemek için kaç tane parametreyi değiştirmen gerektiğini say. İpucu: Sadece `TILESET_SUTUN` ve `TILESET_SATIR` sabitleri yeterli olmalıdır.

37.3 Piksel ve Grid Koordinat Dönüşümleri

Tile sistemiyle çalışırken sürekli iki farklı dünya arasında gidip geleceksin: *piksel koordinatları* ((x, y) olarak PyGame'in ekranı düşünme biçimi) ve *grid koordinatları* ((col, row) olarak haritanın 2B liste indeksleri). Oyuncunun ekran üzerindeki konumunu aldın — peki bu hangi tile'a denk geliyor? Kullanıcı haritaya tıkladı — hangi hücreyi seçti? Bu soruları iki basit bölme işlemiyle çözeceğiz.

37.3.1 Piksel → Grid

Bir piksel koordinatından tile'ın ait olduğu grid hücreisini bulmak için *tam sayı bölme* (`//`) yeterlidir:

Tanım: Piksel → Grid Formülü

$$\text{col} = \left\lfloor \frac{px}{\text{TILE}} \right\rfloor \quad \text{row} = \left\lfloor \frac{py}{\text{TILE}} \right\rfloor$$

Python – Piksel'den Grid'e

```
1 TILE = 32
2
3 def piksel_to_grid(px, py):
4     return px // TILE, py // TILE
5
6 # Fare (150, 75) konumunda tıkladı
7 col, row = piksel_to_grid(150, 75)
8 print(f"Grid: ({col}, {row})") # (4, 2)
9
10 # HARITA[row][col] ile o hücredeki tile'ı bul
11 tile_id = HARITA[row][col]
```

Dikkat et: `//` operatörü tam sayı bölmedir ve negatif değerlerde aşağı yuvarlar. `-5 // 32 == -1`'dir, 0 değil. Bu yüzden oyuncu haritanın dışına çıktığında

negatif indeksler oluşabilir — bu indeksler Python’da listenin sonundan sayar ve beklenmedik sonuçlar verir.

37.3.2 Grid → Piksel

Bir tile’ı ekrana çizmek için grid indeksinden piksel konumuna dönmen gerekir. Bu daha da basittir — sadece çarpma:

Python – Grid’den Piksel’e

```
1 def grid_to_piksel(col, row):
2     return col * TILE, row * TILE
3
4 # (4, 2) grid hücresi ekranda neresidir?
5 x, y = grid_to_piksel(4, 2)
6 print(f"Sol üst: ({x}, {y})") # (128, 64)
7
8 # Tile'ın merkezi
9 cx, cy = x + TILE // 2, y + TILE // 2
```

Bu iki fonksiyon oyun geliştirirken çok sık kullanılır: fare tıklaması, oyuncunun üzerindeki hücreyi bulma, ışın taraması (raycasting), yol bulma (pathfinding)... Hepsi için grid/piksel dönüşümü gereklidir.

37.3.3 Sınır Kontrolü

Formüller basittir ama haritanın dışına düşen koordinatlar tehlikelidir. Her zaman sınır kontrolü ekle:

Python – Güvenli Tile Erişimi

```
1 def guvenli_tile(harita, col, row, varsayilan=0):
2     """Grid dışında kalırsa varsayılan değer döner."""
3     yuk = len(harita)
4     gen = len(harita[0]) if yuk > 0 else 0
5     if 0 <= row < yuk and 0 <= col < gen:
6         return harita[row][col]
7     return varsayilan
8
9 # Haritanın tamamen dışında bir piksel
10 col, row = piksel_to_grid(-10, 5000)
11 tile = guvenli_tile(harita, col, row) # 0 (varsayılan)
```

⚠ Dikkat: Negatif Koordinatlar

Oyuncu (0, 0) hücresinin sol üst köşesindeyken bir piksel bile sola giderse `-1 // 32 == -1` olur. Python bu negatif indeksi listenin sonuna sarar! Her zaman `0 <= row < len(harita)` kontrolü yap.

İpucu: Float Koordinatları İnt'e Çevir

Fizik hesaplamaları genellikle `float` değerler üretir ($x = 12.7$). Grid erişiminden önce mutlaka `int()` çağırılmalıdır: `col = int(x // TILE)`. PyGame `Rect` kendisi zaten `int` kullanır, ama kendi vektörleriyle çalışırken dikkatli ol.

Bilgi: Pratik Alıştırma

Bir oyuncunun piksel konumu (157, 103). Tile boyutu 32 piksel. Oyuncu hangi grid hücresindedir? Hücrenin sol üst köşesinin piksel konumu nedir? Oyuncu hücreden ne kadar içeridedir?

Cevaplar: `col = 4`, `row = 3`. Hücre sol üst = (128, 96). Hücre içi offset = (29, 7).

37.4 İlk Haritanı Çiz: 2B Listedeki Ekrana

Artık tile'ın ne olduğunu, tileset'ten nasıl kesildiğini ve grid dönüşümlerinin nasıl çalıştığını biliyorsun. Şimdi tüm bunları birleştirip ilk tile haritanı ekrana çizelim. Başlangıçta gerçek bir tileset görseli yerine basit renkli kareler kullanacağız — bu şekilde odak noktamız çizim mantığı olur, asset yönetimi değil.

37.4.1 Program İskeletini Kuralım

İlk olarak sabitleri, renk paletini ve harita verisini tanımla:

Python – Sabitler ve Harita

```
1 import pygame
2
3 TILE = 32
4 GRID_GEN = 20
5 GRID_YUK = 15
6 GENISLIK = TILE * GRID_GEN # 640
7 YUKSEKLIK = TILE * GRID_YUK # 480
8
9 # tile_id -> renk
10 RENKLER = {
11     0: (30, 30, 50), # boş
12     1: (80, 180, 80), # çim
13     2: (120, 120, 120), # taş
14     3: (60, 120, 220), # su
15 }
16
17 HARITA = [
18     [2]*20,
19     [2,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,2],
20     # ... (diğer satırlar)
21     [2]*20,
```

22 |]

Pencere boyutunu tile boyutu $\times$ grid boyutuna göre hesapladık — bu sayede pencere tam olarak haritayı kaplayacak, kenarlarda boşluk kalmayacak.

37.4.2 Çizim Fonksiyonu

Harita çizimi, iç içe iki `for` döngüsünden ibarettir. Dış döngü satırları, iç döngü hücreleri tarar:

Python – Harita Çizim Fonksiyonu

```
1 def harita_ciz(ekran, harita):
2     for r, satir in enumerate(harita):
3         for c, tile_id in enumerate(satir):
4             renk = RENKLER.get(tile_id, (0, 0, 0))
5             x = c * TILE
6             y = r * TILE
7             pygame.draw.rect(ekran, renk,
8                             (x, y, TILE, TILE))
```

`enumerate()` kullanmak çok önemli — indeksi ve değeri aynı anda verir, kod çok daha okunaklı olur.

⚠ Dikkat: `for r in range(len(harita))` Kullanma

Birçok başlangıç yazarı `for r in range(len(harita))` yazar. Bu doğrudur ama Pythonic değildir. `enumerate()` hem daha okunaklı hem de her satıra tekrar erişme gereği bırakmaz.

37.4.3 Ana Döngüyü Birleştirme

Şimdi her şeyi ana oyun döngüsüne oturt:

Python – Ana Döngü

```
1 pygame.init()
2 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
3 pygame.display.set_caption("İlk Tile Haritası")
4 saat = pygame.time.Clock()
5
6 calistir = True
7 while calistir:
8     for olay in pygame.event.get():
9         if olay.type == pygame.QUIT:
10             calistir = False
11
12     ekran.fill((0, 0, 0))
13     harita_ciz(ekran, HARITA) # her karede çiz
```

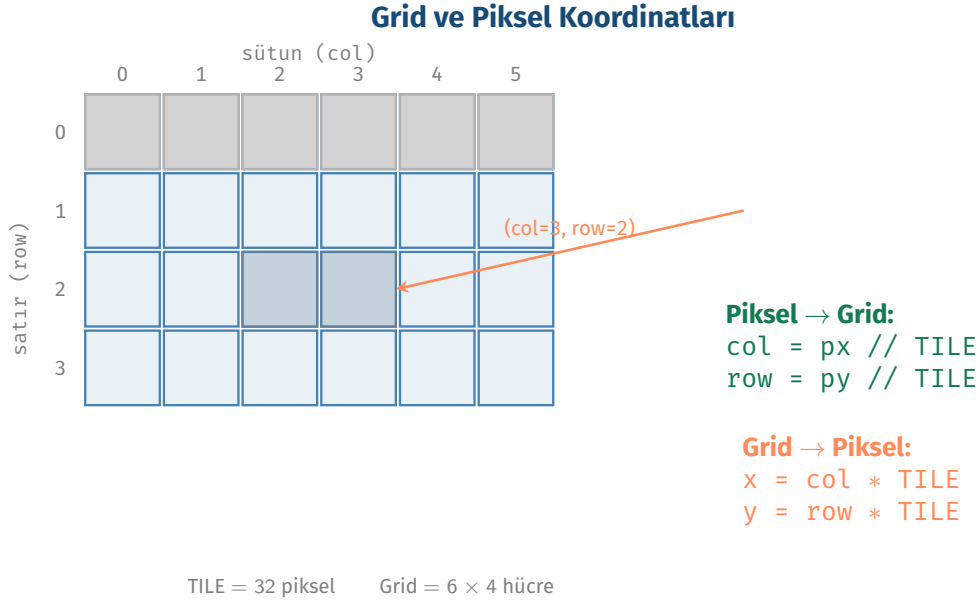


```

14     pygame.display.flip()
15     saat.tick(60)
16
17     pygame.quit()

```

Bu kadar! Çalıştırdığında 640×480 piksellik bir pencerede 20×15 grid bir harita görürsün. Taş duvarlarla çevrili, içinde çim, su ve boş alanların olduğu bir manzara...



Şekil 37.5: Grid üzerinde bir tile'ın satır/sütun ve piksel konumunun ilişkisi.

37.4.4 Performans Notu: Sadece Gerekeni Çiz

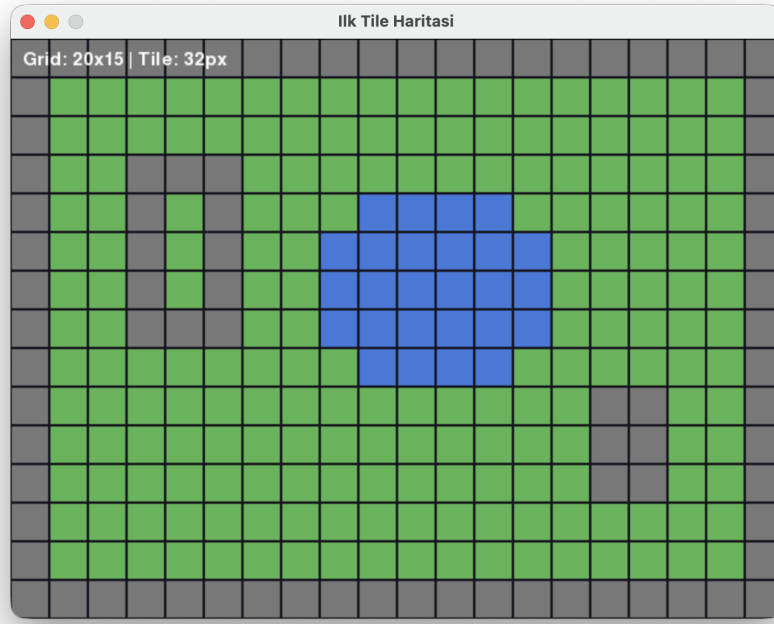
Küçük haritalar için her karede tüm tile'ları çizmek sorun değildir. Ama 200×200 'lük dev bir haritada bu yavaşlar! Çözüm: sadece *ekranda görünen* tile'ları çizmek. Bunu Ünite 3'te kamera konusunu işlerken detaylıca göreceğiz.

💡 İpucu: Statik Haritayı Önceden Render Et

Eğer haritan oyun boyunca değişmiyorsa, bir kere büyük bir Surface'e çizip her karede bu Surface'i tek `blit()`'le ekrana kopyalamak çok daha hızlıdır. Buna *prerender* (önceden render) denir.

📌 Bilgi: Pratik Alıştırma

`kod_ornekleri/bolum10/unite1/01_tile_render.py` dosyasını çalıştır. Haritayı düzenleyerek ortada bir "X" şeklinde taş yol oluşturun. İpucu: Tile ID 2 (taş) değerini uygun hücrelere elle yaz. Sonuçta diagonal olarak geçen iki çizgi görünmeli.



Şekil 37.6: Elle tanımlanmış 2B listeden render edilen ilk tile haritası.

37.5 Sonuç

Bu ünite de tile tabanlı oyun dünyasının temellerini attın. Tile'ın ne olduğunu, bir tileset'ten tile'ları `subsurface()` ile nasıl kesebileceğini, piksel ile grid koordinatları arasında nasıl dönüşeceğini ve elle tanımlanmış bir 2B listeden ilk haritanı nasıl ekrana çizeceğini öğrendin. Artık küçük bir tile haritası tasarlayıp çizen bir PyGame programı yazabilirsin.

37.5.1 Anahtar Kavramlar

Tablo 37.2: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
Tile	Haritanın en küçük yapı birimi, sabit boyutlu kare
Tileset	Birden fazla tile'ı içeren tek bir büyük görsel dosya
Tile ID	Bir tile'ı tanımlayan tam sayı kimliği
Grid	Satır ve sütunlardan oluşan 2B hücre ızgarası
<code>subsurface()</code>	Bir Surface'in bir bölümüne referans veren alt Surface
Grid koordinat	(col, row) biçiminde hücre adresi
Piksel koordinat	(px, py) biçiminde ekran konumu

37.5.2 Önemli Komutlar

Tablo 37.3: Ünite 1 – Temel PyGame ve Python komutları

Komut	Açıklama
<code>pygame.image.load(yol)</code>	Bir görseli Surface olarak yükler
<code>surface.convert_alpha()</code>	Surface'i hızlı blit için formatlar
<code>surface.subsurface(rect)</code>	Bir bölgeye referans veren alt Surface
<code>px // TILE</code>	Piksel → grid sütun/satırı
<code>col * TILE</code>	Grid → piksel koordinatı
<code>pygame.draw.rect(...)</code>	Ekrana renkli dikdörtgen çizer
<code>enumerate(liste)</code>	(indeks, değer) çifti üretir

37.5.3 Sıradaki Adım

Bu ünite de harita verisini kodun içine elle yazdın. Bu küçük demolar için uygundur ama ciddi bir oyun geliştirirken haritayı kodun dışında saklamak istersin — böylece haritayı değiştirdiğinde Python dosyana dokunmazsın. Bir sonraki ünite de haritayı *CSV dosyalarından* ve *Tiled Editor ile üretilen .tmx dosyalarından* nasıl yükleyeceğini öğreneceksin. Harita artık kendi dosyasında yaşayacak, sen de görsel bir editörle tasarlayabileceksin.

“Bir seviyeyi kodla yazmak mümkündür, ama bir seviyeyi çizmek çok daha eğlencelidir.”

— TILED EDITÖR KULLANICI KILAVUZU

Bu Ünitenin Özeti

Bu ünite de řu konuları öğreneceksin:

- Python 2B listeleriyle harita temsil etme
- `csv` modülü ile dışarıdan harita dosyası okuma
- Tiled Editor ile görsel olarak seviye çizme ve `.tmx` kaydetme
- `pytmx.load_pygame()` ile çok katmanlı haritaları yükleme

38.1 Python 2B Listeleriyle Harita Temsil Etme

İlk ünite de harita verisini kodun içine doğrudan yazdın. Bu küçük örneklerde işe yarar ama büyük bir seviye için 300 satırlık bir Python listesi hayal et — okunmaz hale gelir. Bu ünite de önce 2B liste yaklaşımını sağlamlařtıracak, sonra adım adım harita verisini kodun dışına çıkaracağız.

38.1.1 2B Liste Nedir?

■ Tanım: 2B Liste

2B (iki boyutlu) liste, elemanları da liste olan bir Python listesidir. Dış liste satırları, iç listeler ise o satırdaki hücreleri temsil eder. `harita[row][col]` ifadesiyle herhangi bir hücreye erişilir.

Python’da bir 2B listeyi birkaç farklı yolla oluşturabilirsin:

Python – 2B Liste Oluşturma Yolları

```

1  # Yol 1: Literal (elle yazılmış)
2  harita_a = [
3      [2, 2, 2, 2],
4      [2, 1, 1, 2],
5      [2, 2, 2, 2],
6  ]
7
8  # Yol 2: List comprehension ile boş harita
9  GEN, YUK = 10, 8
10 harita_b = [[0 for _ in range(GEN)] for _ in range(YUK)]
11
12 # Yol 3: Sabit değer ile doldur
13 harita_c = [[1] * GEN for _ in range(YUK)]

```

⚠ Dikkat: Paylaşımlı Referans Hatası

`harita = [[0] * GEN] * YUK` **yanlıştır!** Bu, aynı iç listeye *YUK* kere referans verir. `harita[0][0] = 5` yazdığında *tüm satırlarda* aynı değişiklik görülür. Her zaman `[[0 for _ in range(GEN)] for _ in range(YUK)]` veya `[[0] * GEN for _ in range(YUK)]` kullan — ikinci `for` her satır için yeni liste yaratır.

38.1.2 Hücre Erişimi ve Güncelleme

Bir 2B listedeki belirli bir hücreye erişmek ve değiştirmek kolaydır, ama sırayı unutma: önce satır, sonra sütun.

Python – Hücre Okuma ve Yazma

```

1  # Oku
2  tile_id = harita_a[1][2]    # row=1, col=2
3
4  # Yaz
5  harita_a[1][2] = 3
6
7  # Boş hücre kontrolü
8  if harita_a[row][col] == 0:
9      print("Boş hücre")

```

Küçük bir ipucu: `len(harita)` satır sayısını, `len(harita[0])` sütun sayısını verir. Haritanın kaç hücre geniş/uzun olduğunu öğrenmek için bunları kullan.

38.1.3 2B Listeyi Doldurma Örüntüleri

Bazı durumlarda sabit bir harita değil, programatik olarak üretilmiş bir harita istersin. İşte yaygın desenler:

Python – Duvarlı Harita

```

1 def duvarli_harita(gen, yuk):
2     """Kenarlarında taş duvarı olan, içi boş bir
   ↳ harita."""
3     harita = []
4     for r in range(yuk):
5         satir = []
6         for c in range(gen):
7             if r == 0 or r == yuk - 1 or c == 0 or c ==
   ↳ gen - 1:
8                 satir.append(2) # taş (duvar)
9             else:
10                satir.append(1) # çim (iç)
11        harita.append(satir)
12    return harita
13
14 harita = duvarli_harita(20, 15)

```

Bu desen çoğu platformer ve bulmaca oyunu için ideal bir başlangıç iskeletidir. Daha sonra iç kısma elle veya programatik olarak engel eklersin.

38.1.4 2B Listenin Sınırları

2B liste yaklaşımı kodu basit tutar ama bazı sorunları vardır:

1. **Boyut büyüdükçe okunmaz:** 100 satırlık bir liste insanca okunmaz.
2. **Tek katmanlıdır:** Her hücre tek bir tile ID tutar. Arkaplan + ön plan istersen çoklu liste gerekir.
3. **Dosyadan yüklemek zor:** Python dosyasını her değiřtirdiğinde kodu yeniden çalıştırmalısın.

Bir sonraki konuda harita verisini CSV dosyasına taşıyacak ve bu sorunlardan birini çözeceğiz.

💡 İpucu: Python Listesi = Hızlı Prototip

2B listelerle harita tanımlamayı küçümseme. Yeni bir oyun mekaniğini test etmek istediğinde hiçbir şey elle yazılmış 10×10'luk bir haritadan daha hızlı değildir. Sonra gerçek harita formatına geçersin.

● Bilgi: Pratik Alıştırma

15 × 10 büyüklüğünde, kenarlarda taş duvar, ortada 5 × 4 boyutunda boş bir "oda" içeren bir harita üreten fonksiyon yaz. Haritayı konsolda ASCII karakterlerle göster (0 → ., 2 → #).

38.2 CSV Dosyasından Harita Yükleme

Haritayı kodun içine yazmak artık yeterli değil. Büyüyen bir oyun için haritayı kendi dosyasında, metin editörüyle düzenleyebileceğin bir formatta tutmak istersin. En basit ve en yaygın format: **CSV** (Comma-Separated Values, virgülle ayrılmış değerler).

38.2.1 CSV Formatı

■ Tanım: CSV Dosyası

CSV, her satırın bir metin satırı olduğu ve her değerın virgülle ayrıldığı düz metin dosyasıdır. Tile haritası için her satır bir grid satırını, her değer bir tile ID'sini temsil eder.

Bir `harita.csv` dosyası şöyle görünür:

● Bilgi: Örnek CSV İçeriği

```
2,2,2,2,2,2,2,2,2,2,2
2,1,1,1,1,1,1,1,1,1,2
2,1,1,0,0,1,1,1,1,1,2
2,1,1,0,4,1,1,1,1,1,2
2,1,1,1,1,1,1,1,1,1,2
2,2,2,2,2,2,2,2,2,2,2
```

Bu formatın güzelliği, herhangi bir metin editöründe açıp elle düzenleyebilmendir. Excel veya LibreOffice gibi hesap tabloları da CSV'yi doğal olarak açar — böylece haritayı tablo gibi görebilir, kolayca kopyala-yapıştır yapabilirsin.

38.2.2 csv Modülü ile Okuma

Python'ın yerleşik `csv` modülü CSV dosyalarını satır satır okur. Her satır otomatik olarak bir liste halinde gelir, sen de sadece `int`'e çevirirsin.

Python – CSV Harita Yükleme

```
1 import csv
2 from pathlib import Path
3
4 def harita_yukle(yol):
5     """CSV'den 2B int listesi üretir."""
6     harita = []
7     with open(yol, "r", encoding="utf-8") as f:
8         okuyucu = csv.reader(f)
9         for satir in okuyucu:
10             if not satir:
11                 continue # boş satırları atla
```

```

12         harita.append([int(h) for h in satir])
13     return harita
14
15     # Script dizininden çözümle
16     yol = Path(__file__).parent / "harita.csv"
17     harita = harita_yukle(yol)
18     print(f"Boyut: {len(harita[0])}x{len(harita)}")

```

⚠ Dikkat: Dosya Yolu Tuzağı

`open("harita.csv")` yazarsan program nereden çalıştırıldığına göre farklı dizinlerde arar — bu beklenmedik hatalara yol açar. `Path(__file__).parent / "harita.csv"` ile script'in kendi dizinini kullanmak her zaman güvenlidir.

38.2.3 CSV'den Ekrana

Yüklenen harita verisini Ünite 1'de öğrendiğin çizim fonksiyonuyla aynen gösterebilirsin. Fark yok — veri nereden geldi önemli değil, 2B liste aynı mantıkla çalışır:

Python – Yüklenen Haritayı Çizme

```

1  TILE = 32
2  RENKLER = {0: (140,80,40), 1: (80,180,80),
3             2: (120,120,120), 3: (60,120,220),
4             4: (240,230,140)}
5
6  harita = harita_yukle("harita.csv")
7  gen = len(harita[0]) * TILE
8  yuk = len(harita) * TILE
9  ekran = pygame.display.set_mode((gen, yuk))
10
11  for r, satir in enumerate(harita):
12      for c, tile_id in enumerate(satir):
13          pygame.draw.rect(ekran, RENKLER[tile_id],
14                          (c*TILE, r*TILE, TILE, TILE))

```

Artık haritayı değiştirmek için kodu açmana gerek yok — sadece `harita.csv` dosyasını düzenle ve programı tekrar çalıştır.

38.2.4 Harita Doğrulama

CSV dosyaları metin olduğu için kullanıcı hataları olabilir: bir satırda eksik değer, bir harf karakteri, vs. Haritayı yükledikten sonra doğrulamak iyi bir alışkanlıktır:



Şekil 38.1: harita.csv dosyasından yüklenmiş 20×15 tile haritası.

Python – Basit Doğrulama

```

1 def dogrula(harita):
2     if not harita:
3         return False, "Harita boş"
4     beklenen = len(harita[0])
5     for i, satir in enumerate(harita):
6         if len(satir) != beklenen:
7             return False, f"Satır {i}: uzunluk
8                 ↳ {len(satir)}, beklenen {beklenen}"
9     return True, f"{len(harita)}x{beklenen}"
10
11 ok, mesaj = dogrula(harita)
12 print(f"[OK] {mesaj}" if ok else f"[X] {mesaj}")

```

İpucu: newline=" Parametresi

`open(yol, newline="")` yazmak `csv` modülünün platformlar arası satır sonlarıyla düzgün çalışmasını sağlar. Windows'da üretilen CSV'yi macOS'te açarken fark yaratabilir.

● Bilgi: Pratik Alıřtırma

`harita.csv` dosyasını bir metin editöründe aç. Ortaya bir taş sütun ekle (örneğin tek bir hücreye `2` yaz). Programı tekrar çalıştır ve değışikliğin görüldüğünü doğrula. Kodu hiç değıştirmeden haritayı değıştirebildiğin için tebrikler — bu, veri odaklı tasarımın temelidir.

38.3 Tiled Editor ile Görsel Seviye Tasarımı

CSV harita formatı güzel ama 50×30 büyüklüğünde bir harita yazdığını düşün — 1500 sayıyı elle girmek saatler alır ve hatalara çok açıktır. Profesyonel oyun geliştiricilerin çözümü basittir: harita bir *tile editörü*yle çizilir. En popüler ücretsiz seçenek **Tiled**'dir.

■ Tanım: Tiled

Tiled, açık kaynaklı bir tile editörüdür. Bir tileset yükler, tile'ları fareyle boyar gibi haritaya yerleřtirirsin. Birden fazla katman (arkaplan, platform, obje) destekler ve sonucu `.tmx` (XML tabanlı) ya da CSV formatında kaydeder.

38.3.1 Kurulum

Tiled'i <https://www.mapeditor.org> adresinden indir. Windows, macOS ve Linux için ücretsizdir. Kurulumdan sonra yeni bir harita oluşturmak için:

1. **File** → **New** → **New Map...** menüsünü aç.
2. **Orientation**: Orthogonal (kare grid)
3. **Tile layer format**: CSV veya Base64 (zlib)
4. **Map size**: 30×15 (veya istediğin boyut)
5. **Tile size**: 32×32 piksel

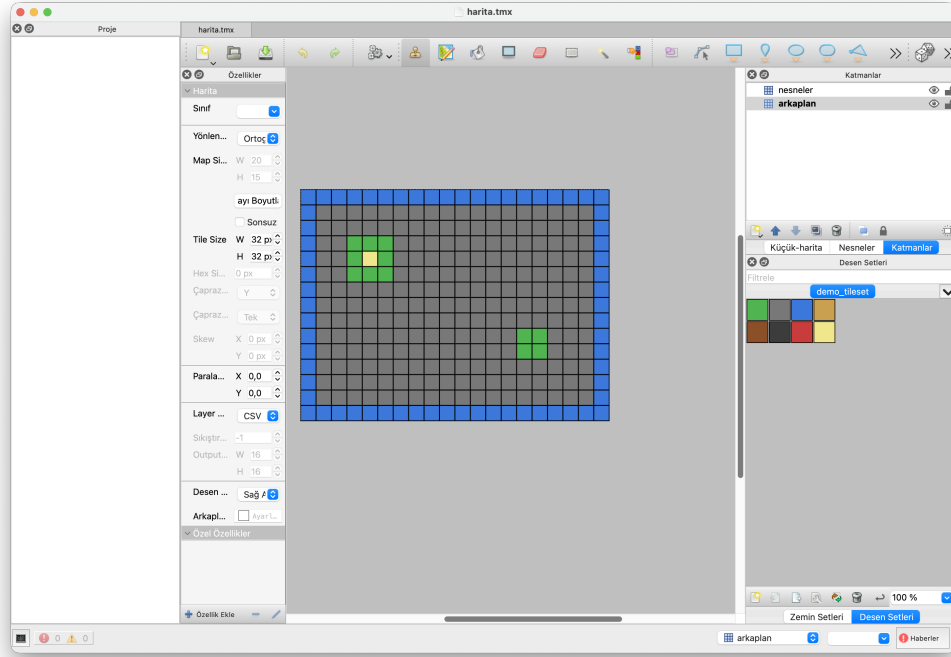
Yeni haritayı aç, sonra bir tileset ekle: **Map** → **New Tileset...** menüsünden kendi PNG tileset'ini yükle.

38.3.2 Katmanlar

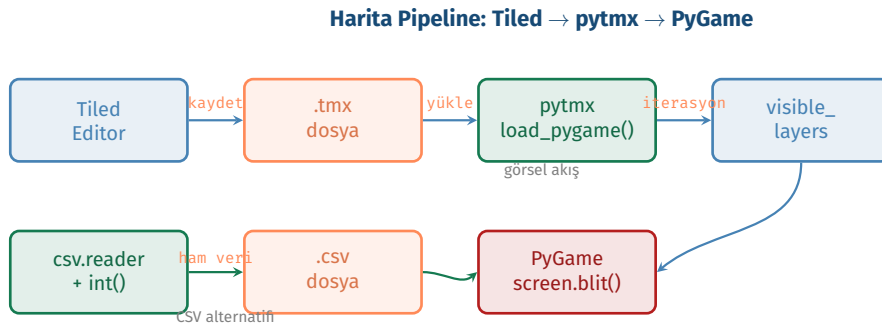
Tiled'in en güçlü özelliklerinden biri *katman* (layer) sistemidir. Bir harita birden fazla katmandan oluşabilir — tıpkı bir PhotoShop projesi gibi. Bir platformer için tipik katman düzeni:

1. **arkaplan**: Gökyüzü, uzak tepeler, ağaclar (çarpışma yok)
2. **platform**: Oyuncunun üzerinde duracağı solid tile'lar
3. **esyalar**: Altın, anahtar, diken gibi obje tile'ları
4. **onplan**: Oyuncunun önüne gelen dekoratif tile'lar

Sağ paneldeki katman listesinden yeni katman ekler, her katmanı ayrı olarak düzenlersin. Katmanları gizleyip gösterebilir, sıralarını değıştirebilirsin.



Şekil 38.2: Tiled Editor ana penceresi: sol panelde tileset, orta panelde harita, sağda katmanlar.



Şekil 38.3: Tiled ile başlayan harita pipeline: editor → .tmx → pytmx → PyGame.

38.3.3 .tmx Kaydetme

Harita çiziminin sonunda **File → Save** ile **.tmx** formatında kaydet. Bu dosya XML tabanlıdır ve insan tarafından okunabilir:

● Bilgi: .tmx Dosyasının İeriđi

```
<map version="1.10" width="30" height="15"
  tilewidth="32" tileheight="32" ...>
  <tileset firstgid="1" source="tileset.tsx"/>
  <layer id="1" name="platform" ...>
    <data encoding="csv">
      2,2,2,2,...
      2,1,1,1,...
      ...
    </data>
  </layer>
</map>
```

Her katman kendi **data** blođuna sahiptir ve CSV gibi virg lle ayrılmıř ID'ler ierir. Tile ID'leri *gid* (global ID) olarak adlandırılır ve tileset'e g re 1'den bařlar (  boş h cre anlamına gelir).

▲ Dikkat: CSV vs .tmx GID Farkı

Tiled'in  rettiđi CSV'lerde ID'ler 1'den bařlar (  boş h cre). Kendi elle yazdıđın CSV'lerde ise genellikle  'dan bařlarsın. Bu y zden Tiled ıktısını okurken tile ID'den 1 ıkarmak gerekir: `tile_id = gid - 1`.

💡 İpucu:  nce K  k Haritalarla Dene

Tiled'i ilk  đrenirken 10 × 10 gibi k  k bir harita yap.  nce yerleřtirme, kaydetme ve y klemeyi utan ua deneyimle. Sonra gerek seviye tasarımına ge.

● Bilgi: Pratik Alıřtırma

Tiled Editor'  kur ve 20 × 12 boyutunda bir platform haritası iz. Bir arkaplan (im) katmanı ve bir platform (tař) katmanı ekle. Haritayı `ilk_harita.tmx` olarak kaydet. Sonraki konuda bu haritayı pytmx ile y kleyeceksin.

38.4 pytmx ile .tmx Harita Y kleme

Tiled'in  rettiđi **.tmx** dosyasını elle XML olarak parse etmek m mk nd r ama zaten bizim iin bunu yapan harika bir k t phane var: **pytmx**. pytmx, .tmx dosyalarını okuyup PyGame'in anlayabileceđi Surface nesnelerini otomatik olarak hazırlar.

■ Tanım: pytmx

pytmx, Tiled tarafından üretilen `.tmx` harita dosyalarını Python'a yükleyen açık kaynak kütüphanedir. PyGame Surface'leri destekleyen `load_pygame()` yardımcısıyla kullanılır. Tüm katmanlar, tileset'ler ve obje grupları otomatik işlenir.

38.4.1 Kurulum

pytmx'i `uv` ile projeye ekle:

● Bilgi: pytmx Kurulum

```
uv add pytmx
```

Kurulduktan sonra içe aktar:

Python – pytmx İçe Aktarma

```
1 import pygame
2 import pytmx
3 from pytmx.util_pygame import load_pygame
```

38.4.2 Harita Yükleme

`load\_pygame(yol)` fonksiyonu sana bir `TiledMap` nesnesi döndürür. Bu nesne, haritanın tüm verisine (boyut, tile boyutu, katmanlar, objeler) yüksek seviyeli bir arayüz sunar.

Python – .tmx Dosyası Yükleme

```
1 # Harita dosyasını yükle
2 tmx_data = load_pygame("harita.tmx")
3
4 print(f"Boyut: {tmx_data.width}x{tmx_data.height}")
5 print(f"Tile: {tmx_data.tilewidth}x{tmx_data.tileheight}")
6 print(f"Katman sayısı: {len(tmx_data.layers)}")
7
8 # Dünya piksel boyutu
9 dünya_gen = tmx_data.width * tmx_data.tilewidth
10 dünya_yuk = tmx_data.height * tmx_data.tileheight
```

`load\_pygame()` Surface nesnelerini otomatik olarak `convert\_alpha()` ile çevirir, yani hızlı `blit` almak için ekstra iş yapman gerekmez.

38.4.3 Katmanları Çizme

Her katmanın `tiles()` metodu üç değerli tuple'lar üretir: (col, row, image). İlk iki değer grid koordinatıdır, üçüncü değer ise o hücrede çizilecek Surface'tir:

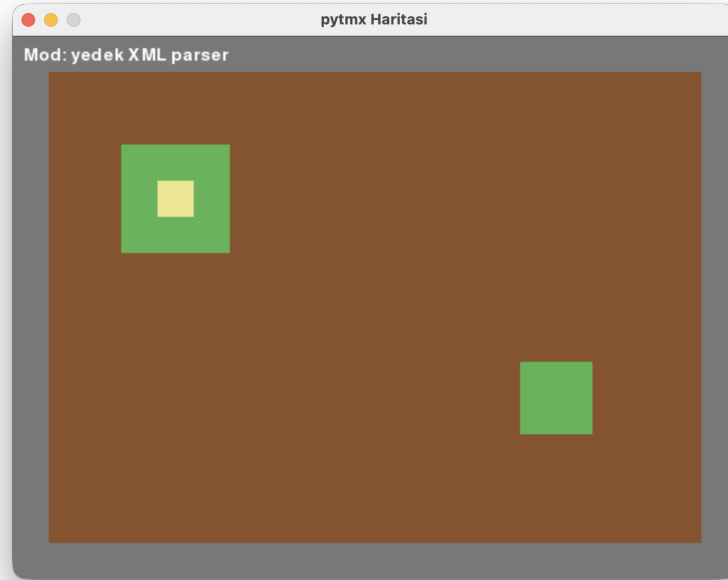
Python – Tm Katmanları Çizme

```

1 def tmx_ciz(ekran, tmx_data, kamera_offset=(0, 0)):
2     tg = tmx_data.tilewidth
3     for layer in tmx_data.visible_layers:
4         if hasattr(layer, "tiles"):
5             for x, y, image in layer.tiles():
6                 if image is None:
7                     continue # boş hücre
8                 cizim_x = x * tg - kamera_offset[0]
9                 cizim_y = y * tg - kamera_offset[1]
10                ekran.blit(image, (cizim_x, cizim_y))

```

`visible_layers` yalnızca Tiled’de görünr yapılan katmanları döner; gizli katmanları atlar. Bu sayede Tiled’deki katman görünrlk ayarları otomatik olarak oyununa yansır.



řekil 38.4: pytmx ile klenmiř ok katmanlı Tiled haritasi.

38.4.4 Katman zelliklerine Gre Davranıř

Tiled’de her katmana isim verebilirsin (“platform”, “esyalar”, ...). pytmx ile bu isimleri okuyup farklı davranıřlar uygulayabilirsin:

Python – Katman Adına Gre arpıřma Grubu

```

1 katilar = [] # solid platformlar
2 esyalar = [] # toplanabilir
3
4 for layer in tmx_data.visible_layers:

```

```

5     if layer.name == "platform":
6         for x, y, image in layer.tiles():
7             rect = pygame.Rect(x * TILE, y * TILE, TILE,
8                               ↪ TILE)
9             katilar.append((rect, image))
10    elif layer.name == "esyalar":
11        for x, y, image in layer.tiles():
12            rect = pygame.Rect(x * TILE, y * TILE, TILE,
13                              ↪ TILE)
14            esyalar.append((rect, image))

```

Böylece Tiled'de istediğin gibi çizdiğin haritayı, katman isimlerine göre çarpışma ve etkileşim gruplarına ayırabilirsin.

⚠ Dikkat: Tileset Yolu Bağlı

pytmx `.tmx` dosyasının içindeki tileset yolunu `.tmx` dosyasına göre *bağlı* çözümler. Eğer `.tmx` dosyasını başka bir dizine taşırsan tileset görseli bulunmaz hatası alırsın. Haritayı ve tileset'i aynı dizinde tutmak en güvenli yaklaşımdır.

💡 İpucu: pytmx Yoksa Yedek

Eğer pytmx kurulu değilse standart kütüphane `xml.etree.ElementTree` ile de `.tmx` dosyasını parse edebilirsin. Kod örneklerindeki `03_pytmx_yukleme.py` bu yedek yaklaşımı gösterir — pytmx yoksa otomatik olarak XML parser'a düşer.

● Bilgi: Pratik Alıştırma

Önceki konudaki alıştırmada oluşturduğun `ilk_harita.tmx` dosyasını pytmx ile yükle. Terminalde her katmanın adını ve tile sayısını yazdır. İpucu: `for layer in tmx_data.visible_layers: print(layer.name)`.

38.5 Sonuç

Bu ünite de harita verisini kodun içinden dışarı taşıdın. Python 2B listelerinin temelini pekiştirdin, sonra CSV dosyalarıyla ham metin tabanlı harita formatını kullanmayı öğrendin. Ardından Tiled Editor'ü keşfettin ve profesyonel `.tmx` formatını pytmx kütüphanesiyle yüklemeyi pratiği yaptın. Artık harita verisinin nerede yaşayacağına ve oyununun ihtiyaçlarına göre hangi yaklaşımın daha uygun olduğuna karar verebilirsin.

38.5.1 CSV vs pytmx Karřılařtırması

Tablo 38.1: CSV ve pytmx/Tiled yaklařımlarının karřılařtırması

Özellik	CSV	Tiled + pytmx
Kurulum	Standart kütüphane	<code>uv add pytmx</code>
Çoklu katman	Yok (kendin yaz)	Var (doğrudan)
Görsel editor	Yok	Tiled
Dosya formatı	Düz metin	XML (.tmx)
Öğrenme kolaylığı	Çok kolay	Orta
Prototip hızı	En hızlı	Orta
Büyük harita	Elle zor	Çok kolay
Obje grupları	Yok	Var

Ne zaman hangisi? Küçük, prototip bir oyun için CSV fazlasıyla yeterlidir. Büyük, çok katmanlı, profesyonel görünümlü bir seviye tasarımı için Tiled + pytmx tercih edilmelidir. Bu kitapta Zıplayan Macera projesi için her iki formatı da örneklerimizde tutacağız.

38.5.2 Anahtar Kavramlar

Tablo 38.2: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
CSV	Virgülle ayrılmış değerler formatı
.tmx	Tiled'in XML tabanlı harita dosya biçimi
Katman (layer)	Bir harita üstüne eklenen ayrı çizim yüzeyi
GID	Tiled'de global tile kimliği, 1'den başlar
Tileset	Tile'ları içeren görsel atlas
pytmx	.tmx dosyasını Python'a yükleyen kütüphane

38.5.3 Önemli Komutlar

Tablo 38.3: Ünite 2 – Temel komutlar

Komut	Açıklama
<code>csv.reader(dosya)</code>	CSV dosyasını satır satır okur
<code>open(yol, newline="")</code>	CSV için önerilen dosya açma
<code>Path(__file__).parent</code>	Script'in kendi dizini
<code>load_pygame(yol)</code>	.tmx dosyasını PyGame için yükler
<code>tmx_data.visible_layers</code>	Görünür katmanlar üzerinde iterasyon
<code>layer.tiles()</code>	(col, row, image) tuple'ları üretir
<code>tmx_data.width / height</code>	Grid boyutu (hücre cinsinden)

38.5.4 Sıradaki Adım

Haritayı yüklemeyi öğrendin ama haritan bir ekrandan daha büyük olduğunda ne yapacaksın? Oyuncunun etrafını göstermek için haritayı “kaydırman” gerekir. Bir sonraki ünite de *kamera* kavramını tanıtıyoruz: dünya-ekran koordinat ayrımı, basit bir Camera sınıfı, oyuncuyu takip eden yumuşak (lerp) geçişler ve kamerayı harita sınırları içinde tutmak için *clamp* tekniği.

Kamera ve Scrolling

“Dünya sabit durur; değişen, ona bakan kameradır.”

— OYUN MÜHENDİSLİĞİ DERS NOTLARI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Dünya koordinatları (world-space) ve ekran koordinatları (screen-space) ayrımı
- Basit bir `Camera` sınıfı tasarlamak ve `apply()` ile sprite'lara uygulamak
- Kamerayı oyuncuya kilitleyip yumuşak takip (lerp) eklemek
- Kamerayı harita sınırları içinde tutmak için `clamp` tekniği

39.1 World-Space ve Screen-Space

Haritayı ekrana sığdırabildiğin sürece kamera konusu biraz lüks gibi görünebilir. Ama oyun dünyası ekrandan büyük olduğu an — ki ciddi bir seviye için hep öyle olur — “hangi kısmı göstereceğiz?” sorusu ortaya çıkar. İşte bu nedenle iki farklı koordinat sistemine ihtiyacımız var: *world-space* (dünya koordinatları) ve *screen-space* (ekran koordinatları).

39.1.1 İki Farklı Koordinat

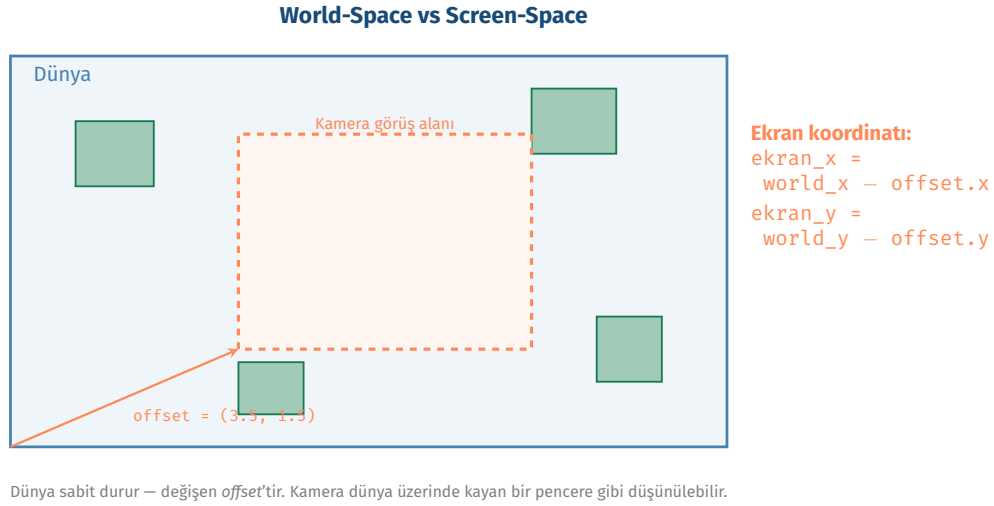
■ Tanım: World-Space

Dünya koordinatı, bir nesnenin haritadaki *sabit* konumudur. Oyuncu haritanın ortasındaysa world-space konumu (1500, 800) olabilir. Bu değer kamera nereye bakarsa baksın değişmez.

■ Tanım: Screen-Space

Ekran koordinatı, bir nesnenin o anki *görünen* konumudur ve ekran sol üst köşesine göre dir. Oyuncu ekran ortasındaysa screen-space konumu (400, 300) olur — kamera kayarsa bu değer de kayar.

Bu ikisi arasında basit bir ilişki vardır: $\text{screen} = \text{world} - \text{offset}$. Offset, kamera-nın dünyadaki sol üst köşesinin nerede olduğunu söyler.



Şekil 39.1: Dünya koordinatları ile ekran koordinatları arasındaki ilişki: offset kamera penceresinin dünya üzerindeki kayma miktarıdır.

39.1.2 Offset Uygulaması

Bir nesneyi çizmek istediğinde dünya konumundan offset'i çıkararak ekran konumuna geç:

Python – Offset Uygulaması

```

1  # Dünyadaki sabit nesne
2  nesne_world = pygame.Rect(1200, 800, 32, 48)
3
4  # Kamera dünyanın (1000, 500) noktasına bakıyor
5  kamera_offset = pygame.Vector2(1000, 500)
6
7  # Ekran koordinatına çevir
8  ekran_x = nesne_world.x - kamera_offset.x
9  ekran_y = nesne_world.y - kamera_offset.y
10
11  ekran.blit(sprite, (ekran_x, ekran_y))
12  # ya da Rect.move ile:
13  cizim_rect = nesne_world.move(-kamera_offset.x,
    ↪ -kamera_offset.y)

```

`Rect.move(dx, dy)` yeni bir Rect döndürür; orijinali değiştirmez. Bu, dünya üzerindeki gerçek `Rect`'i değiştirmeden sadece çizim için kaydırılmış bir kopya almanı sağlar.

39.1.3 Dünya Sabit, Kamera Kayan

Bu yaklaşımın en önemli zihinsel modeli: *dünya sabit durur, değişen kameradır*. Oyuncu dünya üzerinde (1500, 800) konumundadır; kamera onu takip etmek için offset'ini günceller. Dünya üzerindeki diğer nesneler de aynı offset'e göre yeniden konumlandırılır.

⚠ Dikkat: Tek Yerden Offset Uygula

Tüm çizim çağrılarında offset'i unutmamak önemlidir. Eğer bazı nesneleri offset ile, bazılarını offset'siz çizersen oyun kırılır. Bir sonraki konuda bir `Camera` sınıfı yazacağız — her zaman `apply()` metoduyla çizim yaparsan hiçbir şeyi atlamazsın.

39.1.4 HUD Farklı: Screen-Space'te Kalır

Dikkat: Sağlık barı, skor göstergesi ve menüler *kameradan etkilenmez*. Bunlar doğrudan ekran koordinatlarına çizilir — kamera nereye giderse gitsin HUD sabit kalır. İşte pratik örnek:

Python – Oyun Nesneleri vs HUD

```
1 # Oyun nesneleri (kameralı)
2 for n in nesneler:
3     ekran.blit(n.sprite, kamera.apply(n.rect))
4
5 # HUD (kamera uygulanmaz)
6 ekran.blit(saglik_bari, (10, 10))
7 ekran.blit(skor_metni, (700, 10))
```

💡 İpucu: Vector2 Kullan

Offset için `(x, y)` tuple yerine `pygame.Vector2` kullanmak çok daha esnek. Vektörleri doğrudan toplayabilir, çıkarabilir, ölçekleyebilirsin. Lerp (yumuşak geçiş) de doğrudan desteklenir.

● Bilgi: Pratik Alıştırma

`01_kamera_temel.py` dosyasını çalıştır ve ok tuşlarıyla kamerayı hareket ettir. Ekranın ortasındaki sarı artı işaretinin neden kamera ile birlikte hareket etmediğini düşün. Cevap: Artı screen-space'te sabit çizilir, dünya nesneleri ise world-space'te.

39.2 Camera Sınıfı Tasarımı

Offset kavramını öğrendin. Şimdi bu offset'i yönetmek için düzenli bir yapı kuralım — bir **Camera** sınıfı. Bu sınıf offset'i tutacak, nesnelere uygulayacak ve güncellenmesini sağlayacak. Küçük bir sınıftır ama projede tekrar tekrar kullanacağın güçlü bir araçtır.

39.2.1 Minimum Camera Sınıfı

Başlangıç olarak en basit hali:

Python – Temel Camera Sınıfı

```

1 class Camera:
2     def __init__(self, gen, yuk):
3         self.offset = pygame.Vector2(0, 0)
4         self.gen = gen
5         self.yuk = yuk
6
7     def apply(self, rect):
8         """Dünya rect'ini ekran rect'ine çevirir."""
9         return rect.move(-int(self.offset.x),
10                          -int(self.offset.y))
11
12     def apply_point(self, x, y):
13         """Bir noktayı ekran koordinatına çevirir."""
14         return (x - int(self.offset.x),
15                y - int(self.offset.y))

```

`apply()`, bir **Rect**'i alıp yeni, kaydırılmış bir **Rect** döndürür — orijinali değiştirmez. `apply_point()` bir (x, y) çiftini kaydırır ve tuple döndürür. Bu iki yardımcıyla tüm çizim çağrılarını kameradan geçirebilirsin.

39.2.2 Nesneleri Çizme

Her nesnenin **rect**'i dünya koordinatlarındadır. Çizerken sadece `apply()` çağırırısın:

Python – Kamera ile Çizim

```

1 kamera = Camera(GENISLIK, YUKSEKLIK)
2
3 # Ana döngüde
4 for nesne in nesneler:
5     ekran.blit(nesne.sprite, kamera.apply(nesne.rect))
6
7 # Oyuncu da aynı şekilde
8 ekran.blit(oyuncu.sprite, kamera.apply(oyuncu.rect))

```

Tüm dünya nesneleri için aynı `apply()` çağrısını kullandığından hiçbir şeyi atlama riski yok. Eğer bir nesneyi *kameradan etkilenmez* yapmak istiyorsan (HUD

gibi) `apply()` çağırma, doğrudan ekran konumunu ver.

39.2.3 Kamera Gruplarına Doğru

PyGame'in `sprite.Group` sınıfını genişleterek kendi `KameraGrup` sınıfını yazabilirsin. Bu, Ünite 5'te (sprite tabanlı çizim) göreceğin bir desene çok benzer:

Python – KameraGrup

```
1 class KameraGrup(pygame.sprite.Group):
2     def __init__(self, kamera):
3         super().__init__()
4         self.kamera = kamera
5
6     def ciz(self, ekran):
7         for sprite in self.sprites():
8             ekran.blit(sprite.image,
9                         self.kamera.apply(sprite.rect))
```

Bu grup, eklediğin tüm sprite'ları otomatik olarak kamera offset'iyle çizer. Tek yapman gereken normalde `draw(ekran)` çağırдың yerde `ciz(ekran)` çağırmaktır.

⚠ Dikkat: `int()`'e Çevirmeyi Unutma

Offset'i `Vector2` olarak tuttuğumuz için değerleri float'tur. `Rect.move()` float alır ama içinde `int`'e yuvarlar. Eğer kendi `blit`'lerini yaparken `self.offset.x`'i doğrudan kullanırsan pikselin ortasına çizim yapmaya çalışırsın — görünüm bulanıklaşır. Her zaman `int(self.offset.x)` kullan.

39.2.4 `update()` Metodu İçin Hazırlık

Şu an `apply()` ve `apply\_point()` dışında sınıfımızda hiç davranış yok — offset'i elle güncelliyoruz. Bir sonraki konuda `update(hedef)` metodu ekleyeceğiz: bu metod bir hedef `Rect` alıp kamera offset'ini otomatik olarak günceller, hedefi ekran ortasına çekip yumuşak takip sağlar.

💡 İpucu: Sprite'larla Kamera = Çok Güzel Eşleşme

Eğer Bölüm 6'da gördüğün gibi `pygame.sprite.Sprite` sınıfını kullanıyorsan, her sprite'ın zaten `rect` özelliği vardır. Kamera grubunun çizim döngüsü bu rect'leri otomatik kaydırır — sprite'ın kendisi hiç değişmez.

● Bilgi: Pratik Alıştırma

`Camera` sınıfına bir `reset()` metodu ekle: offset'i (0, 0)'a geri döndürsün. Ayrıca `set\_offset(x, y)` metodu da ekle: elle mutlak offset atayabilmek için. Bu iki metod debug ve seviye geçişleri için çok kullanışlıdır.

39.3 Oyuncu Takibi ve Lerp

Kameranın en önemli işi oyuncuyu takip etmektir. Eğer kamera oyuncuya anlık yapışırsa çalışır ama pürüzsüz değildir — oyuncunun her küçük hareketi kamerada hissedilir. Profesyonel oyunlarda kamera hedefe *yumuşak* biçimde geçer. Bunun matematiksel adı *lerp*'dir (linear interpolation, doğrusal ara değer).

39.3.1 Hedef Offset

Oyuncunun kameranın merkezinde olması için kameranın sol üst köşesinin nerede olması gerektiğini hesapla:

Python – İstenen Offset

```
1 istenen = pygame.Vector2(
2     oyuncu.rect.centerx - ekran_gen // 2,
3     oyuncu.rect.centery - ekran_yuk // 2,
4 )
```

Bu değeri doğrudan `self.offset` olarak atarsan kamera oyuncuya aniden yapışır — çalışır ama pürüzlü bir his verir. Lerp ile bu atamayı “yumuşatabilirsin”.

39.3.2 Lerp Formülü

■ Tanım: Linear Interpolation (Lerp)

Lerp, iki değer arasında belirli bir oranda ara değer hesaplar:

$$\text{yeni} = \text{eski} + (\text{hedef} - \text{eski}) \cdot t$$

t değeri 0 ile 1 arasındadır. $t = 0.1$ ise her karede hedefe kalan mesafenin %10'u kadar yaklaşır.

Python – Lerp ile Kamera Güncelleme

```
1 class Camera:
2     def __init__(self, gen, yuk, lerp=0.1):
3         self.offset = pygame.Vector2(0, 0)
4         self.gen = gen
5         self.yuk = yuk
6         self.lerp = lerp
```

```

7
8     def update(self, hedef_rect):
9         istenen = pygame.Vector2(
10             hedef_rect.centerx - self.gen // 2,
11             hedef_rect.centery - self.yuk // 2,
12         )
13         self.offset += (istenen - self.offset) * self.lerp

```

Vektör aritmetiği sayesinde formül tek satıra sığar: “mevcut offset’e, hedeften olan farkın lerp katını ekle.”

⚠ Dikkat: Lerp Değeri Çok Büyük

`lerp=1.0` verirsen anlık takip olur (yumuşatma yok). `lerp>1` verirsen overshoot yaşarsın — hedefi geçer, geri döner, titrer. $lerp \in [0.05, 0.2]$ aralığı genellikle iyi sonuç verir.

39.3.3 Kameranın Çalışması

Kameraya hedef ver ve her karede güncelle:

Python – Ana Döngüde Güncelleme

```

1 kamera = Camera(GENISLIK, YUKSEKLIK, lerp=0.08)
2
3 while calistir:
4     # Girdi ve fizik
5     oyuncu.guncelle(tuslar, katilar)
6
7     # Kamera hedefi takip etsin
8     kamera.update(oyuncu.rect)
9
10    # Çizim
11    ekran.fill((30, 30, 50))
12    for n in nesneler:
13        ekran.blit(n.sprite, kamera.apply(n.rect))
14    ekran.blit(oyuncu.sprite, kamera.apply(oyuncu.rect))
15    pygame.display.flip()

```

Bu basit değişiklikle oyun çok daha “cool” hissettirir. Oyuncu koştuğunda kamera bir tık geride kalır, dururken yavaşça yerine oturur — göze çok hoş gelir.

39.3.4 Lerp’in Matematiğini Kısaca Anla

Lerp her karede “kalan mesafenin belli bir kısmını kapat” der. İlk karede bu büyük bir değişimdir, sonraki karelerde küçülür. Bu yüzden hareket üstel bir eğri çizer: hızlı başlar, sonra yavaşlar — insan gözüne “akıcı” hissi veren şey budur.



Şekil 39.2: Kamera lerp ile oyuncuyu yumuşak biçimde takip eder.

💡 İpucu: Framerate Bağımsız Lerp

Yukarıdaki formül 60 FPS’de güzel çalışır ama 120 FPS’de kamera iki kat hızlı yakalar. Gerçek zaman bazlı lerp için: $faktor = 1 - \exp(-hiz * dt)$ formülü kullanılır. Bu kitabın başlangıç seviyesinde sabit FPS varsayıyoruz, ama akılda tut.

📘 Bilgi: Pratik Alıştırma

`02_kamera_takip.py` dosyasını çalıştır. `Camera` sınıfının `lerp=0.08` parametresini sırayla `0.02`, `0.15` ve `0.5` yap. Her değerde kameranın davranışının nasıl değiştiğini kendi gözlerinle gör. Hangisi en doğal hisseder?

39.4 Kamerayı Harita Sınırlarıyla Kısıtla: Clamp

Kamera oyuncuyu takip ederken harika bir sorun yaşarsın: oyuncu haritanın köşesine yaklaştığında kamera haritanın *dışını* göstermeye başlar! Bu bölgede hiçbir şey olmadığı için ekranda siyah boşluklar çıkar. Çözüm: kamera offset’ini harita sınırlarıyla *clamp* (sıkıştır) et.

■ Tanım: Clamp

Clamp, bir değeri belirlenen alt ve üst sınır arasına zorlama işlemidir. Matematiksel ifadesi:

$$\text{clamp}(x, \text{min}, \text{max}) = \max(\text{min}, \min(x, \text{max}))$$

39.4.1 Sınır Hesaplaması

Kamera offset'inin geçerli aralığını düşün: offset (0,0) olduğunda kamera haritanın sol üst köşesine yapışmıştır. Offset (dünya_gen - ekran_gen, dünya_yuk - ekran_yuk) olduğunda sağ alt köşeye yapışır. Bu ikisinin dışına çıkamamalı:

Python – Offset Clamp

```
1 def _clamp(self):
2     max_x = max(0, self.dunya_gen - self.ekran_gen)
3     max_y = max(0, self.dunya_yuk - self.ekran_yuk)
4     if self.offset.x < 0:         self.offset.x = 0
5     if self.offset.y < 0:         self.offset.y = 0
6     if self.offset.x > max_x:     self.offset.x = max_x
7     if self.offset.y > max_y:     self.offset.y = max_y
```

`max(0, ...)` önemlidir! Eğer dünya boyutu ekran boyutundan küçükse (mesela bir bulmaca oyununda) `dunya_gen - ekran_gen` negatif olur. Bu durumda offset 0'da kalmalı — harita ekran ortasında sabit dursun.

39.4.2 Camera Sınıfına Entegre Etme

Clamp'i `update()`'in sonunda çağır. Bu şekilde lerp ile güncellenmiş offset harita sınırlarını aşamayacak:

Python – Clamp'li Camera

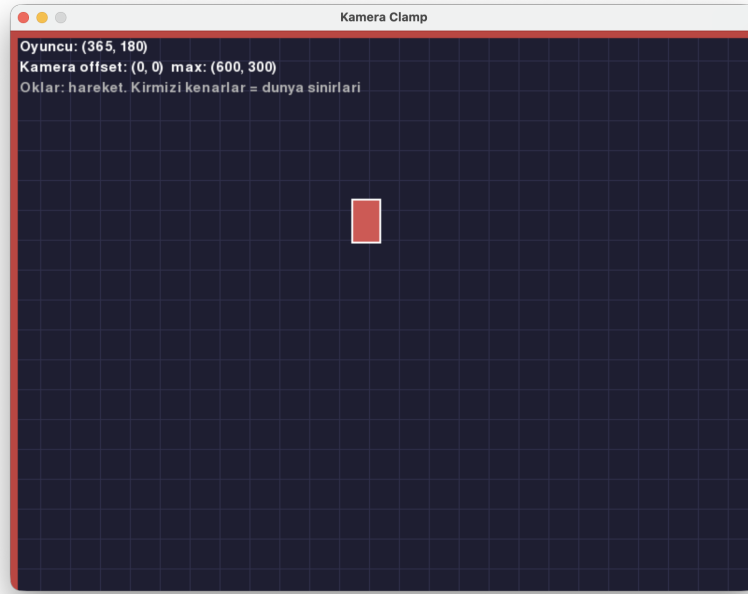
```
1 class Camera:
2     def __init__(self, ekran_gen, ekran_yuk,
3                 dunya_gen, dunya_yuk, lerp=0.1):
4         self.offset = pygame.Vector2(0, 0)
5         self.ekran_gen = ekran_gen
6         self.ekran_yuk = ekran_yuk
7         self.dunya_gen = dunya_gen
8         self.dunya_yuk = dunya_yuk
9         self.lerp = lerp
10
11     def update(self, hedef):
12         istenen = pygame.Vector2(
13             hedef.centerx - self.ekran_gen // 2,
14             hedef.centery - self.ekran_yuk // 2,
15         )
16         self.offset += (istenen - self.offset) * self.lerp
```

```

17     self._clamp()
18
19     def apply(self, rect):
20         return rect.move(-int(self.offset.x),
21                           -int(self.offset.y))

```

Artık oyuncu köşeye gitse bile kamera harita sınırlarını aşmaz. Köşede oyuncu ekranın ortasında değil, köşe tarafında görünür — bu hem gerçekçi hem de doğru davranıştır.



Şekil 39.3: Oyuncu haritanın sağ alt köşesinde iken clamp kamerayı sınıra yapıştırır.

39.4.3 Harita Boyutunu Nereden Alırız?

Bu değerler tile haritasının gerçek piksel boyutudur:

Python – Harita Piksel Boyutu

```

1  # CSV'den yüklenmiş harita
2  dünya_gen = len(harita[0]) * TILE
3  dünya_yuk = len(harita) * TILE
4
5  # pytmx ile yüklenmiş harita
6  dünya_gen = tmx_data.width * tmx_data.tilewidth
7  dünya_yuk = tmx_data.height * tmx_data.tileheight
8
9  kamera = Camera(GENISLIK, YUKSEKLIK, dünya_gen, dünya_yuk)

```

Her iki yaklaşımda da harita boyutunu tek yerde hesapla ve `Camera` constructor'ına ver. Kamera yaşamı boyunca bu değerleri kullanır.

39.4.4 Clamp Sonrası Oyuncu Hareketi

Önemli bir detay: clamp yalnızca kamerayı sınırlandırır, oyuncuyu değil. Oyuncu harita dışına çıkabilir! Oyuncu için ayrı bir `clamp_ip` çağırısı yapmalısın:

Python – Oyuncu İçin Ayrı Clamp

```
1 # Hareket sonrası oyuncuyu harita içinde tut
2 oyuncu.rect.clamp_ip(
3     pygame.Rect(0, 0, dünya_gen, dünya_yuk))
```

`clamp_ip()` (in-place) Rect'i verilen sınır içinde zorlar. Kameraya bir ek, oyuncuya bir ek — ikisini birlikte kullanmak gerekir.

⚠ Dikkat: Clamp Sırası Önemlidir

Önce `self.offset += ... lerp uygula`, sonra `_clamp()` çağır. Tersini yaparsan lerp bir sonraki karede clamp'i atlar. Sıra: güncelle → sıkıştır.

💡 İpucu: Küçük Bir Margin Ekle

Bazı oyunlarda kamerayı tam sınıra yapıştırmak yerine küçük bir margin bırakırsın (örneğin 16 piksel). Bu, “oyuncu sınırın hemen kenarında” his-sini verir ama biraz nefes alanı sağlar. Sadece `max_x`'e `+margin`, `min_x`'e `-margin` eklemek yeterlidir.

● Bilgi: Pratik Alıştırma

`03_kamera_clamp.py` dosyasını çalıştır. Oyuncuyu haritanın 4 köşesine de götür ve kameranın nasıl davrandığını gözlemle. Sonra `DUNYA_GEN` de-ğerini `600`'e düşür (ekrandan küçük!) ve programı tekrar çalıştır — harita artık tüm ekranı kaplamaz ama kamera titremez, sabit durur.

39.5 Sonuç

Bu ünite de ekranın sınırlarını aşan büyük dünyaları yönetmek için kamera sis-temi kurduk. World-space ve screen-space arasındaki ayrımı öğrendin, `Camera` sınıfı ile offset'i kapsülledin, lerp yumuşak takibi ekledin ve clamp ile kamerayı harita sınırlarıyla sıkıştırdın. Artık ekrandan çok daha büyük haritalarda özgürce oyuncuyu takip edebilecek bir sistemin var.

39.5.1 Anahtar Kavramlar

Tablo 39.1: Ünite 3 – Anahtar kavramlar

Kavram	Açıklama
World-space	Dünyadaki sabit koordinat sistemi
Screen-space	Ekran sol üst köşesine göre koordinat
Offset	Kameranın dünya üzerindeki kayma miktarı
Camera	Offset'i yöneten ve uygulayan sınıf
apply()	Bir Rect'i dünya'dan ekrana kaydıran metod
Lerp	İki değer arasında doğrusal ara değer hesaplama
Clamp	Bir değeri belirli bir aralığa zorlama
HUD	Kameradan etkilenmeyen ekran katmanı

39.5.2 Önemli Komutlar

Tablo 39.2: Ünite 3 – Temel komutlar

Komut	Açıklama
<code>pygame.Vector2(x, y)</code>	2B vektör oluşturur (matematik dostu)
<code>rect.move(dx, dy)</code>	Rect'i kaydıran yeni Rect döndürür
<code>rect.clamp_ip(sinir)</code>	Rect'i sınır içinde in-place zorlar
<code>max(0, min(x, m))</code>	Bir değeri 0–m aralığına sıkıştırır
<code>offset += (hedef - offset) * t</code>	Lerp güncelleme

39.5.3 Sıradaki Adım

Artık büyük haritaların üstünde gezinebilen akıllı bir kamera var. Son ünite de bu sistemi bir gerçek seviye inşasına uygulayacağız. Platformları, tehlikeleri (lava, diken) ve toplanabilirleri (altın, anahtar) tile gruplarına ayıracak, “Zıplayan Macera” projesinin ilk oynanabilir seviyesini inşa edeceğiz. Harita, kamera ve oyuncu fiziği bir araya gelip bir tam seviye oluşturacak.

*“İyi bir seviye oyuncuya hem meydan
okur hem de ona öğretmenlik eder.”*

— MARK BROWN, GAME MAKER'S TOOLKIT

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Tile türlerine göre katı (solid), tehlike ve toplanabilir gruplar oluşturma
- Platform çarpışmalarını tile grid'i üzerinden yönetme
- Lava, diken gibi tehlike tile'larına zarar mantığı bağlama
- “Zıplayan Macera” projesinin ilk oynanabilir seviyesini harita entegrasyonu ile inşa etme

40.1 Platformlar ve Solid Tile Çarpışması

Bir tile haritasını ekrana çizebilmek güzel ama oyuncu o haritayla *etkileşime* giremezse öylece donmuş bir resim kalır. Bu konuda solid tile'ların (taş, ağaç, duvar gibi geçilmez hücrelerin) üzerinde oyuncunun nasıl duracağını öğreneceğiz. Tile tabanlı fizik şaşırtıcı derecede basittir — sadece doğru sırayla yapmak gerekir.

40.1.1 Solid Tile'ları Gruplama

İlk adım: harita verisinden yalnızca solid olan tile'ları çıkarıp **Rect** listesi halinde saklamak. Oyuncu hareket ederken bu listeye karşı çarpışma kontrolü yaparız.

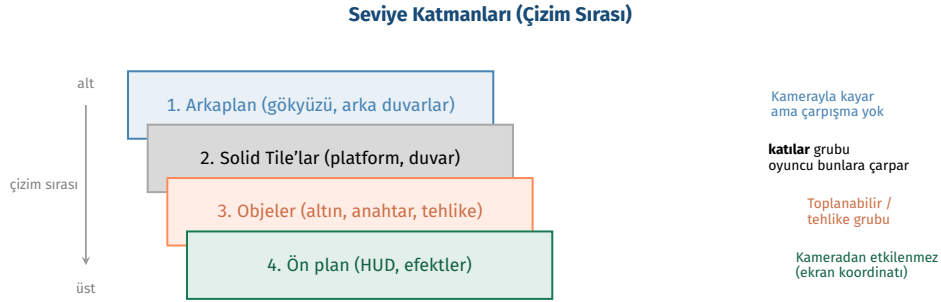
Python – Solid Tile Çıkarma

```
1 def katilari_cikar(harita, solid_id=2):
2     """Haritadan solid hücreler için Rect listesi
   ↳ üretir."""
3     katilar = []
```

```

4   for r, satir in enumerate(harita):
5       for c, tile_id in enumerate(satir):
6           if tile_id == solid_id:
7               rect = pygame.Rect(c * TILE, r * TILE,
8                                   TILE, TILE)
9               katilar.append(rect)
10          return katilar

```



Şekil 40.1: Bir seviyenin tile katmanları: arkaplan, solid, obje, ön plan.

40.1.2 Yatay ve Dikey Çarpışmayı Ayır

Tile çarpışmasında en önemli kural: *yatay ve dikey hareketi ayrı işle*. Eğer tek seferde ikisini birden çözmeye çalışırsan “köşede takılma” bug’ı yaşarsın. Doğru algoritma şöyledir:

1. Sadece yatay hareketi uygula (`rect.x += vx`).
2. Yatay çarpışmaları çöz (sola veya sağa it).
3. Sadece dikey hareketi uygula (`rect.y += vy`).
4. Dikey çarpışmaları çöz (yukarı veya aşağı it).

Python – Yatay Çarpışma

```

1   def _yatay_carpisma(self, katilar):
2       for k in katilar:
3           if self.rect.colliderect(k):
4               if self.vx > 0: # sağa giderken
5                   self.rect.right = k.left
6               elif self.vx < 0: # sola giderken
7                   self.rect.left = k.right

```

Python – Dikey Çarpışma

```

1   def _dikey_carpisma(self, katilar):
2       for k in katilar:
3           if self.rect.colliderect(k):
4               if self.vy > 0: # düşüşte
5                   self.rect.bottom = k.top
6                   self.vy = 0
7                   self.yerde = True

```

```

8         elif self.vy < 0: # zıplarken
9             self.rect.top = k.bottom
10            self.vy = 0

```

40.1.3 Yerçekimi ve Zıplama

Platform oyunlarında yerçekimi bir sabittir: oyuncunun dikey hızına her karede küçük bir değer eklersin. Yerde olunca `space` ile zıpla; yerden ayrılınca yerçekimi yine aşağı çeker.

Python – Platformer Fizik Döngüsü

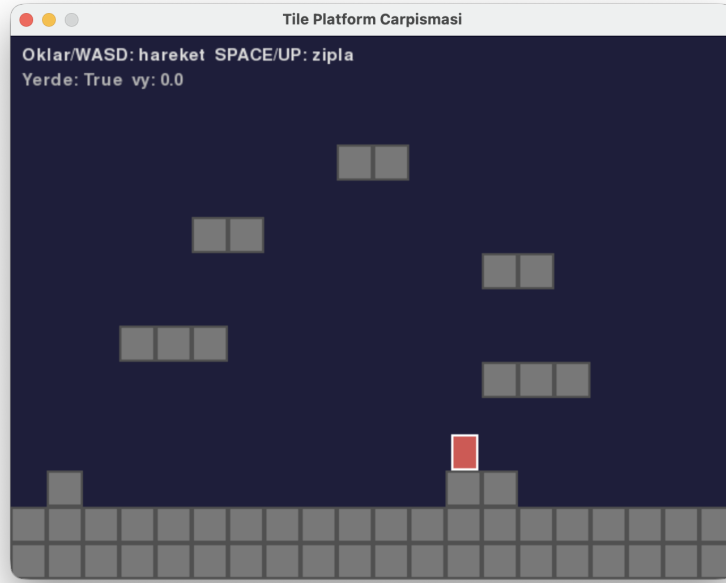
```

1 class Oyuncu:
2     def __init__(self, x, y):
3         self.rect = pygame.Rect(x, y, 24, 32)
4         self.vx = 0
5         self.vy = 0
6         self.yerde = False
7
8     def guncelle(self, tuslar, katilar):
9         # Yatay
10        self.vx = 0
11        if tuslar[pygame.K_LEFT]:
12            self.vx = -3
13        if tuslar[pygame.K_RIGHT]:
14            self.vx = 3
15        self.rect.x += self.vx
16        self._yatay_carpisma(katilar)
17
18        # Dikey (yerçekimi + çarpışma)
19        self.vy += 0.5 # yerçekimi
20        if self.vy > 10: # terminal hız
21            self.vy = 10
22        self.rect.y += int(self.vy)
23        self.yerde = False
24        self._dikey_carpisma(katilar)
25
26    def zipla(self):
27        if self.yerde:
28            self.vy = -10

```

40.1.4 Neden Tüm Solidlerle Kontrol?

Yukarıdaki çarpışma döngüsü her karede *tüm* solid tile'larla kontrol yapar. Küçük haritalarda bu hızlıdır ama 100 × 100 gibi büyük haritalarda yavaş olur. Optimizasyon: sadece oyuncunun bulunduğu grid hücresinin komşularına bak:



Şekil 40.2: Tile tabanlı platform çarpışması: oyuncu solid tile'ların üzerinde durur.

Python – Yalnızca Komşu Tile'larla Kontrol

```

1 def yakin_katilar(harita, rect):
2     """Oyuncunun çevresindeki katı tile'ları döndürür."""
3     col = rect.centerx // TILE
4     row = rect.centery // TILE
5     katilar = []
6     for dr in range(-1, 2):
7         for dc in range(-1, 2):
8             r, c = row + dr, col + dc
9             if 0 <= r < len(harita) and 0 <= c <
               ↪ len(harita[0]):
10                if harita[r][c] == 2:
11                    katilar.append(
12                        pygame.Rect(c*TILE, r*TILE, TILE,
13                                   ↪ TILE))
14
15     return katilar

```

Bu sayede her karede sadece 9 tile ile kontrol yaparsın, 10000 değil. Bunu Ünite 3'teki `collidirect` döngüsünün girdisi olarak kullan.

⚠ Dikkat: Büyük Hızlarda Tunnelling

Eğer oyuncu bir karede 32 pikselden fazla hareket ederse tile'lardan “geçip” gidebilir (tunnelling bug). Tile boyutundan büyük hız kullanmaktan kaçın; gerekirse hareketi birkaç küçük adımda uygula.

💡 İpucu: yerde Kontrolü

`self.yerde` flag'ini *her karede* `False` yap ve dikey çarpışma sırasında `True` yap. Bu şekilde oyuncu yerde olup olmadığını her zaman doğru bilir — zıplama mantığı `if self.yerde` ile güvenle çalışır.

● Bilgi: Pratik Alıştırma

`01_platform_carpisma.py` dosyasını çalıştır. SPACE ile zıpla ve oklarla hareket et. Oyuncunun platformların kenarlarında nasıl davrandığını gözlemle. Sonra `gravity` değerini (`0.5`) `0.2` yap — oyuncu ay yerçekimi altındaymış gibi hafifler.

40.2 Tehlike Tile'ları: Lava ve Diken

Bir seviyenin sadece platformlardan ibaret olması yeterli değildir. Oyuncuya meydan okumak için *tehlike* eklemek gerekir. Bu konuda lava ve diken gibi zarar veren tile'ları seviyene nasıl ekleyeceğini öğreneceksin. Güzel tarafı: Platformlar için yazdığın çarpışma altyapısı burada da olduğu gibi çalışır — sadece *davranış* değişir.

40.2.1 Tehlike Grubu

Solid tile'ları nasıl ayrı bir listeye çıkardıysan, tehlike tile'larını da ayrı bir listeye çıkar. Böylece bir tile'a dokunmak “üzerinde dur” yerine “zarar al” anlamına gelecek.

Python – Tile Gruplarına Ayırıştırma

```
1 def tile_gruplari(harita):
2     katilar, tehlikeler = [], []
3     for r, satir in enumerate(harita):
4         for c, tid in enumerate(satir):
5             rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
6             if tid == 2: # taş (solid)
7                 katilar.append(rect)
8             elif tid in (5, 6): # 5=lava, 6=diken
9                 tehlikeler.append((tid, rect))
10    return katilar, tehlikeler
```

Tuple olarak (tip, rect) sakladığımıza dikkat et — böylece her tehlikenin türünü ayırt edebiliriz (lava ile diken farklı görsellere sahip).

40.2.2 Çarpışma ve Zarar

Ana döngüde, oyuncu hareketi bitince tehlikelerin üstünde olup olmadığını kontrol et. Değilse devam et; dokunmuşsa can kaybet ve spawn noktasına dön.

Python – Tehlike Kontrolü

```

1  for tip, rect in tehlikeler:
2      if oyuncu.rect.colliderect(rect):
3          oyuncu.can -= 1
4          oyuncu.yeniden_dogur()
5          if oyuncu.can <= 0:
6              print("[GAME OVER]")
7              calistir = False
8              break # tek karede birden fazla zarar alma

```

`break` çok önemlidir: bir tehlikeye dokunduğunda hemen döngüden çık, yoksa aynı karede birkaç tile ile çarpışıp can çift/üçlü kaybedebilirsiniz.

40.2.3 Spawn Noktasına Dönüş

`yeniden_dogur()` metodu oyuncunun konumunu ve hızını sıfırlar:

Python – Spawn Dönüşü

```

1  class Oyuncu:
2      def __init__(self, x, y):
3          self.spawn = (x, y)
4          self.rect = pygame.Rect(x, y, 24, 32)
5          ...
6
7      def yeniden_dogur(self):
8          self.rect.topleft = self.spawn
9          self.vx = 0
10         self.vy = 0

```

Daha ileri seviyede *checkpoint* sistemi eklersin: oyuncu belirli noktalara geldiğinde spawn konumunu günceller. Bu kitapta tek bir başlangıç noktası kullanıyoruz — çok daha sonraki bölümlerde checkpoint'e döneceğiz.

40.2.4 Tehlike Görselleri

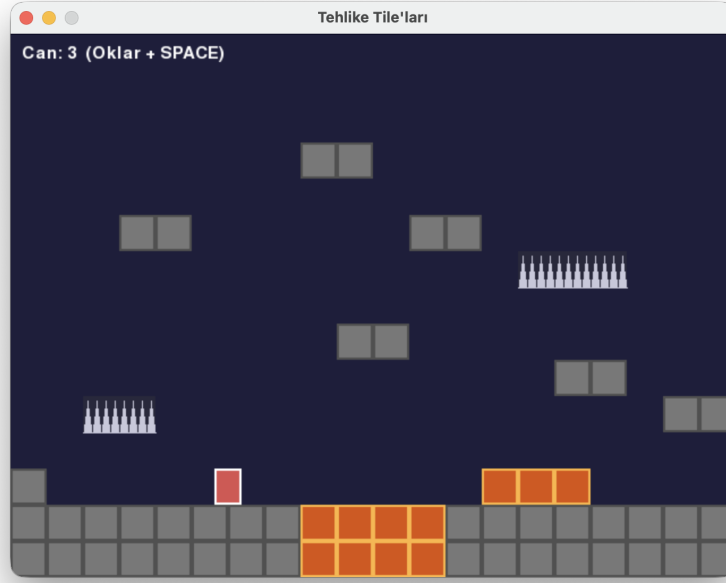
Lava ve dikenini basit `pygame.draw` çağrılarıyla çizebilirsin. Gerçek bir oyunda tileset'ten tile kullanırsın ama demo için şöyle yapılır:

Python – Tehlike Çizimi

```

1  def tehlike_ciz(ekran, tehlikeler):
2      for tip, r in tehlikeler:
3          if tip == 5: # lava
4              pygame.draw.rect(ekran, (220, 80, 0), r)
5              pygame.draw.rect(ekran, (255, 180, 60), r, 2)
6          elif tip == 6: # diken
7              pygame.draw.rect(ekran, (40, 40, 60), r)
8              for i in range(4):
9                  x = r.x + i * 8

```



Şekil 40.3: Lava ve diken tile'larına dokunan oyuncu can kaybeder.

```

10     pygame.draw.polygon(
11         ekran, (200, 200, 220),
12         [(x, r.bottom),
13          (x+4, r.top+4),
14          (x+8, r.bottom)])

```

Dikenin sivri uçlarını küçük üçgenlerle çiziyoruz — basit ama etkili. Lava ise turuncu dolgu bir dikdörtgendir.

⚠ Dikkat: Dokunulmazlık Süresi

Oyuncu bir tehlikeye dokunup spawn'a döndükten sonra hemen hareket edemiyorsa veya yeniden tehlikede olursa iki kare içinde iki kez can kaybedebilir. Profesyonel oyunlarda "dokunulmazlık süresi" vardır (2 saniye boyunca tehlikelerden etkilenme). Bu kitapta basit tutuyoruz ama akılda bulunsun.

💡 İpucu: Tile ID Sözlüğü

Birden fazla tehlike türü ekledikçe `if/elif` zincirleri uzar. Sözlük kullan:

```
TEHLIKE_TURU = {5: "lava", 6: "diken", 7: "asit"}
```

Sonra sadece `if tid in TEHLIKE_TURU` kontrolü yeter.

● Bilgi: Pratik Alıştırma

`02_tehlike_tile.py` dosyasına bir tehlike daha ekle: “asit gölü” tile’ı (`ID=7`). Kendi renkli çizimini yap (yeşil + sarı) ve harita stringinde en az iki hücreye yerleştir. Oyuncu buna dokunursa lava gibi spawn’a dönsün.

40.3 Toplanabilirler: Altın ve Anahtar

Her iyi platformer’ın üç temel bileşeni vardır: platformlar (zorluk), tehlikeler (risk) ve *toplanabilirler* (ödül). Bu konuda haritaya altın ve anahtar gibi toplanabilir tile’ları ekleyecek, oyuncu dokunduğunda bunları listeden çıkarıp skora ya da envantere ekleyeceğiz.

40.3.1 Toplanabilir Grubu

Tehlike tile’ları gibi toplanabilirleri de ayrı bir listeye çıkar. Ancak farklı olarak, bunları *mutable* (değiştirilebilir) bir liste olarak tut — çünkü toplandıklarında listeden silinecekler.

Python – Toplanabilir Listeleri

```
1 def tile_gruplari(harita):
2     katilar, altinlar, anahtarlar = [], [], []
3     for r, satir in enumerate(harita):
4         for c, tid in enumerate(satir):
5             rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
6             if tid == 2:
7                 katilar.append(rect)
8             elif tid == 4: # altın
9                 altinlar.append(rect)
10            elif tid == 8: # anahtar
11                anahtarlar.append(rect)
12    return katilar, altinlar, anahtarlar
```

40.3.2 Liste Üzerinde Değişiklik

Oyuncu bir altına dokunduğunda onu listeden sil. Ama “çalışırken üzerinden geçtiğin listeyi değiştirme” kuralı Python’da da geçerlidir. Bu yüzden listenin *kopyası* üzerinde dolaş: `altinlar[:]`.

Python – Güvenli Liste Silme

```
1 for a in altinlar[:]: # kopyası üzerinde dolaşıyoruz
2     if oyuncu.rect.colliderect(a):
3         altinlar.remove(a) # asıl listeden sil
4     skor += 10
```

`altinlar[:]` ifadesi listenin sıg bir kopyasını yapar. Dolaşım bu kopyada yapılır, silme ise orijinalde. Böylece iterasyon sırasında herhangi bir hata oluşmaz.

40.3.3 Skor ve Envanter

Toplanabilir türüne göre ne değişir? Altın: skora eklenir. Anahtar: envantere eklenir, sonradan kapı açmak için kullanılır. İşte basit bir envanter yönetimi:

Python – Envanter ve Skor

```

1  # Oyun durumu
2  skor = 0
3  envanter = {"anahtar": 0, "bilet": 0}
4
5  # Çarpışma kontrolleri
6  for a in altinlar[:]:
7      if oyuncu.rect.colliderect(a):
8          altinlar.remove(a)
9          skor += 10
10
11  for a in anahtarlar[:]:
12      if oyuncu.rect.colliderect(a):
13          anahtarlar.remove(a)
14          envanter["anahtar"] += 1

```

40.3.4 HUD'a Yansıtma

Skor ve envanter bilgisi her karede HUD'a çizilir. Bölüm 9'da öğrendiğin font render akışını hatırla:

Python – HUD Çizimi

```

1  font = pygame.font.Font(None, 28)
2
3  def hud_ciz(ekran):
4      skor_metin = font.render(
5          f"[ALTIN] {skor}", True, (255, 215, 0))
6      ekran.blit(skor_metin, (10, 10))
7
8      anahtar_metin = font.render(
9          f"Anahtar: {envanter['anahtar']}",
10         True, (200, 200, 220))
11     ekran.blit(anahtar_metin, (10, 40))

```

HUD *screen-space*'te kalır — kameradan etkilenmez. Bölüm 9'daki tüm HUD bilgilerini burada tazeleyebilirsiniz.

40.3.5 Altın Animasyonu

Basit bir görsel tatlılık: altınları yumuşak salınım yaparken çiz. Pozisyonlarını değiştirme, sadece yükseklik offset'i olarak bir `sin` dalgası ekle:

Python – Altın Salınım Efekti

```

1 import math
2
3 def altin_ciz(ekran, altinlar, zaman):
4     for a in altinlar:
5         dy = int(math.sin(zaman * 3 + a.x * 0.02) * 4)
6         merkez = (a.centerx, a.centery + dy)
7         pygame.draw.circle(ekran, (240, 200, 60), merkez,
            ↪ 10)

```

Her altın kendi x konumuna göre farklı fazlarda salınır — güzel bir dalgalanma efekti verir.

⚠ Dikkat: Liste Modifikasyonu Kural

`for a in altinlar: altinlar.remove(a)` bu kod **yanlıştır**. Python iterasyon sırasında indeksleri kaydırır ve eleman atlar. Her zaman `altinlar[:]` (kopyası) üzerinde dolaş, asıl listeyi düzenle.

💡 İpucu: Ses Efekti Zamanı

Bir altın toplandığında küçük bir “ding” sesi çalarsan oyunun tatmin duygusu büyük ölçüde artar. Bölüm 7’de öğrendiğin `pygame.mixer.Sound`’u kullanarak bunu denemeyi unutma.

● Bilgi: Pratik Alıştırma

`o3_toplanabilir.py` programına bir üçüncü toplanabilir ekle: “bilet” (`ID = 9`). Bir yeşil üçgen olarak çiz, toplandığında `envanter["bilet"]`’i artırsın. Sadece “Anahtar: N” satırının altına “Bilet: N” satırı eklemen yeterli.

40.4 Mini Proje: Zıplayan Macera – Seviye 1

Bu ünite de öğrendiğin her şeyi bir araya getirme zamanı. Artık “Zıplayan Macera” isimli platform oyunumuzun ilk oynanabilir seviyesini inşa edeceğiz. Harita bir CSV dosyasından yüklenecek, kamera oyuncuyu takip edecek, platform çarpışması çalışacak ve altın toplayıp dikenlerden kaçarak seviyeyi bitirebileceksin.

40.4.1 Proje Yapısı

Proje tek bir `main.py` dosyası ve bir `harita/` dizininden oluşur. Harita dosyasını ayrı tutmak, oynanış koduyla veri arasında net bir ayrım yapar.

Tablo 40.1: Proje dosya yapısı

Dosya	Açıklama
main.py	Oyun giriş noktası, ana döngü
harita/seviye1.csv	Seviye 1 tile verisi
harita/seviye1.tmx	Tiled alternatifi (isteğe bağlı)

40.4.2 Seviye Sınıfı

Harita yükleme ve tile gruplarına ayrıştırmayı tek bir **Seviye** sınıfında topluyoruz. Bu sayede **main.py** temiz kalır ve ileride ikinci, üçüncü seviyeler eklemek kolaylaşır.

Python – Seviye Sınıfı

```

1 class Seviye:
2     def __init__(self, csv_yol):
3         self.harita = self._yukle(csv_yol)
4         self.yuk = len(self.harita)
5         self.gen = len(self.harita[0])
6         self.dunya_gen = self.gen * TILE
7         self.dunya_yuk = self.yuk * TILE
8         self.katilar, self.tehlikeler, self.altinlar = [],
          ↳ [], []
9         self._grupla()
10
11     def _yukle(self, yol):
12         with open(yol, newline="") as f:
13             return [[int(x) for x in row] for row in
                  ↳ csv.reader(f) if row]
14
15     def _grupla(self):
16         for r, satir in enumerate(self.harita):
17             for c, t in enumerate(satir):
18                 rect = pygame.Rect(c*TILE, r*TILE, TILE,
                  ↳ TILE)
19                 if t == 2:
20                     self.katilar.append(rect)
21                 elif t == 4:
22                     self.altinlar.append(rect)
23                 elif t == 5:
24                     self.tehlikeler.append(rect)

```

40.4.3 Ana Döngü

Tüm parçaları bir araya getiren ana döngü. Önceki ünitelerden gelen kamera, oyuncu fiziği ve tile çarpışması burada bir araya geliyor.

Python – main() Fonksiyonu

```

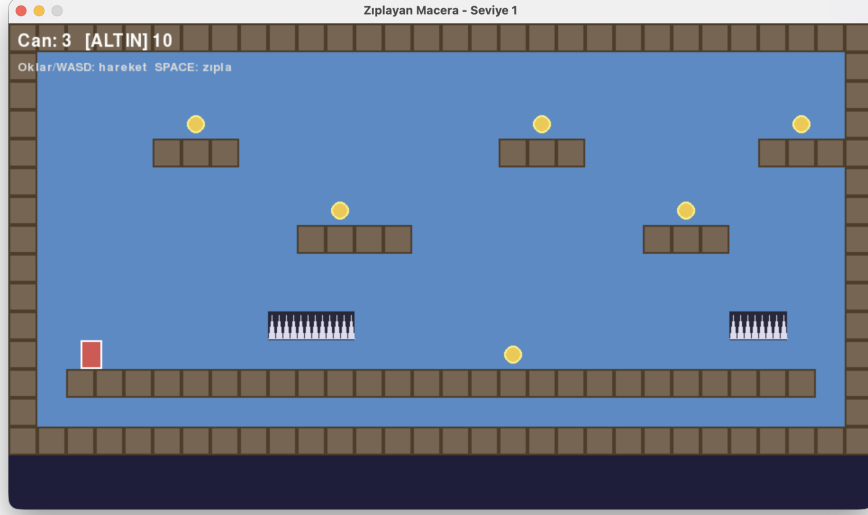
1 def main():
2     pygame.init()
3     ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
4     pygame.display.set_caption("Zıplayan Macera - Seviye
5         ↪ 1")
6     saat = pygame.time.Clock()
7
8     seviye = Seviye("harita/seviye1.csv")
9     kamera = Camera(GENISLIK, YUKSEKLIK,
10         seviye.dunya_gen, seviye.dunya_yuk)
11     oyuncu = Oyuncu(80, seviye.dunya_yuk - 120)
12     skor, can = 0, 3
13
14     calistir = True
15     while calistir:
16         for olay in pygame.event.get():
17             if olay.type == pygame.QUIT:
18                 calistir = False
19             elif olay.type == pygame.KEYDOWN:
20                 if olay.key in (pygame.K_SPACE,
21                     ↪ pygame.K_UP):
22                     oyuncu.zipla()
23
24             oyuncu.guncelle(pygame.key.get_pressed(),
25                 ↪ seviye.katilar)
26
27             # Tehlike ve toplanabilir
28             for t in seviye.tehlikeler:
29                 if oyuncu.rect.colliderect(t):
30                     can -= 1
31                     oyuncu.yeniden_dogur()
32                     break
33             for a in seviye.altinlar[:]:
34                 if oyuncu.rect.colliderect(a):
35                     seviye.altinlar.remove(a)
36                     skor += 10
37
38             kamera.update(oyuncu.rect)
39
40             ekran.fill((30, 30, 60))
41             seviye.ciz(ekran, kamera)
42             pygame.draw.rect(ekran, (220, 80, 80),
43                 kamera.apply(oyuncu.rect))
44
45             # HUD
46             # ... (skor, can yaz)
47
48             pygame.display.flip()
49             saat.tick(60)
50

```

47

`pygame.quit()`

Dikkat: Bu kod parçası tüm dosyanın özetidir. Çalıştırmak için [kod_ornekleri/bolum10/pr](#) dosyasını kullan — orada tam ayrıntılı sürümü bulacaksın.



Şekil 40.4: Zıplayan Macera seviye 1: harita, kamera takibi, altın ve diken entegrasyonu.

40.4.4 Kazanma ve Kaybetme

Oyun bittiğinde ne olacak? İki durum var:

1. **Kaybetme:** `can <= 0` olunca ana döngüyü kır, “GAME OVER” yaz.
2. **Kazanma:** Tüm altınlar toplandığında “Seviye tamamlandı!” yaz.

Python – Son Durum Kontrolü

```

1 if can <= 0:
2     metin = font.render("[GAME OVER]", True, (255, 80,
3         ↪ 80))
4     ekran.blit(metin, metin.get_rect(
5         center=(GENISLIK // 2, YUKSEKLIK // 2)))
6
7 if not seviye.altinlar:
8     metin = font.render("Seviye tamamlandı!",
9         True, (120, 255, 120))
10    ekran.blit(metin, metin.get_rect(
11        center=(GENISLIK // 2, 80)))

```

40.4.5 Seviye Tasarım İpuçları

`harita/seviye1.csv` dosyasını düzenlerken şu rehbera uy:

1. Başlangıç noktası güvenli olsun (etrafında tehlike olmasın).

2. İlk platform atlaması kolay olmalı; oyuncu mekaniği öğrensin.
3. Dikenler kritik ama geçilebilir yerlere yerleştirilsin.
4. Altınlar riskin karşılığı olmalı — kolay değil ama ulaşılabilir.
5. En az bir “açık” ana yol olsun; ikinci yol isteğe bağlı bonus.

▲ Dikkat: Çok Yakın Platform = Sinir Bozucu

Zıplamalar ardı ardına çok yakınsa oyuncu sürekli ekrana yapışır. İki platform arasında en az 2 – 3 tile yatay boşluk bırak ki oyuncu hareketi hissetsin.

💡 İpucu: Yatay Seviyeyi Daha Uzun Yap

Seviye kamerayı hak etmek için ekrandan geniş olmalı. Seviye 1’imiz 30×15 tile yani 960×480 piksel — genişliği neredeyse iki ekran uzunluğunda. Bu sayede kameranın takibi hissedilir.

● Bilgi: Pratik Alıştırma

`projeler/zıplayan_macera_harita/main.py` dosyasını çalıştır ve seviyeyi baştan sona oyna. Sonra `harita/seviye1.csv` dosyasını bir metin editöründe aç. Kendi bir platform grubunu ekle, bir altın yerleştir, belki bir diken öbeği. Programı tekrar çalıştır ve kendi yaptığın seviye ile oyna. Kendi seviyeni tasarladın — tebrikler!

40.5 Sonuç

Bu bölümün son ünitesinde tile sistemini gerçek bir oynanışa dönüştürdün. Solid platformlarla oyuncuya bir yer kazandırdın, tehlike tile’larıyla risk ekledin, toplanabilirlerle ödül kurdun ve hepsini Zıplayan Macera’nın ilk seviyesinde birleştirdin. Artık bir harita dosyasından oynanabilir bir seviye inşa edecek tüm parçalara sahipsin.

40.5.1 Anahtar Kavramlar

Tablo 40.2: Ünite 4 – Anahtar kavramlar

Kavram	Açıklama
Solid tile	Oyuncunun çarpıp üzerinde durduğu tile
Tehlike tile	Dokununca zarar veren tile (lava, diken)
Toplanabilir	Oyuncu dokununca listeden silinen tile
Spawn	Oyuncunun ölünce döndüğü başlangıç konumu
Yatay/dikey çarpışma ayırma	Tile fiziğinin temel prensibi
clamp_ip	Bir Rect’i sınır içinde in-place zorlar
Liste kopyası üzerinde iterasyon	Güvenli silme deseni

40.5.2 Önemli Komutlar

Tablo 40.3: Ünite 4 – Temel komutlar

Komut	Açıklama
<code>rect.colliderect(diger)</code>	İki Rect'in çakışıp çakışmadığı
<code>rect.bottom = k.top</code>	Dikey çarpışmada oyuncuyu yerleştir
<code>rect.right = k.left</code>	Yatay çarpışmada oyuncuyu yerleştir
<code>liste[:]</code>	Listenin sıg kopyası
<code>liste.remove(eleman)</code>	Belirli elemanı listeden çıkarır
<code>math.sin(t)</code>	Salınım efektleri için sinüs dalgası

40.5.3 Bölüm 10'un Özeti

Üç önemli alanda güçlü bir temel kurduk:

1. **Tile Sistemi:** 2B liste → CSV → Tiled+pytmx ile haritayı kodun dışına çıkardık.
2. **Kamera:** World-space ayrımı, lerp takibi ve clamp sınırlandırmasıyla büyük haritaları yönetebilir olduk.
3. **Seviye Mekanikleri:** Solid, tehlike ve toplanabilir tile gruplarıyla bir oynanış katmanı oluşturduk.

Zıplayan Macera projemizin ilk seviyesi hazır — sadece “oynanır” değil, aynı zamanda genişlemeye hazır bir çerçeve.

40.5.4 Sıradaki Adım

Bölüm 11'de bu seviyeyi daha da zenginleştireceğiz: paralaks arkaplan katmanları (arka dağlar, yakın ağaçlar farklı hızlarda kayar), animasyonlu tile'lar (akan su, titreşen lava), kamera shake (sarsıntı) efektleri ve daha fazlası. Tile haritasının üzerine görsel-işitsel polish eklemeye başlayacağız.

Ayrıca seviyeyi bitirince bir sonraki seviyeye geçiş mantığı da gelecek: sıra halinde seviye 2, seviye 3 ve boss dövüşü. Bölüm 10'da attığın temel, bu bölümlerde genişleyecek yapıyı taşıyor.

Şimdilik: Oturup `main.py`'yi çalıştır, altınları topla, dikenlerden kaç ve kendi seviyelerini tasarlamamanın tadını çıkar. Bu artık senin platformer'ın.

Gelişmiş Karakter Hareketi

“İyi bir platformer her hareketin kendini doğru hissettirmesi üzerine kuruludur. Oyuncu kazandığını hak etmeli, kaybettiğinde suçu kendinde bulmalı.”

— MARK BROWN, GAME MAKER'S TOOLKIT

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Platform oyununda “hissiyat” (game feel) kavramı: ivme, sürtünme ve maksimum hız
- Coyote time ile platformdan az önce düşmüş oyuncuya zıplama toleransı tanıma
- Jump buffer ile zıplama tuşu erken basılmış olsa bile yere değer değmez kaydı kullanma
- Variable jump ile tuşun ne kadar tutulduğuna göre farklı yüksekliklere zıplama
- Double jump mekaniği ile havada ikinci bir zıplama hakkı tanıma

41.1 Platform Hissiyatı: İvme, Sürtünme ve Maksimum Hız

Bölüm 10’da yazdığın **Oyuncu** sınıfı işleyen ama ham bir fiziğe sahip: tuş basılır basılmaz hız +3 veya -3 olur, tuş bırakılınca hız anında sıfıra döner. Bu çalışır, evet — ama eline bir Super Mario veya Celeste kumandası alıp da aynı kodu tekrar denersen ne kadar kuru hissettiğini fark edersin. Asıl iyi platformerlar hareketin kendini bir ödül gibi hissettirirler. İşte bu konuda o hissi üretmeyi öğreniyoruz.

■ Tanım: Game Feel

Game feel, bir oyunun kumanda tepkisinin nasıl algılandığını anlatan terimdir. Steve Swink'e göre oyun hissi "kumanda tepkisi", "sanal uzayda uzantı" ve "doğgun etkileşim" olarak üç temel sütunda oturur. Platform oyunlarında bunun üç görünümü vardır: *kontrolün sıklığı*, *hareketin ağırlığı* ve *hatanın affedilmesi*.

Bu ünitenin dört konusu da aslında aynı amaca hizmet eder: hareketin iyi hissetmesine. Bu ilk konuda ivme ve sürtünmeyi; sonraki konularda coyote time ve jump buffer gibi "affedici" mekanikleri; son olarak da variable ve double jump gibi "güçlendirici" mekanikleri inşa edeceğiz.

41.1.1 Ham Hız Yerine İvme

Bölüm 10'daki `Oyuncuguncelle` şöyleydi:

Python – Ham Hız - Bölüm 10

```
1 self.vx = 0
2 if tuslar[pygame.K_LEFT]:
3     self.vx = -3
4 if tuslar[pygame.K_RIGHT]:
5     self.vx = 3
```

Burada hız ya tam maksimumdadır ya tam sıfırdadır. Aradaki geçiş yok. Oysa gerçek bir karakter hareket etmeye başladığında önce yavaş başlar, sonra hızlanır; tuş bırakıldığında da anında değil, yavaşça durur.

Python – İvme ile Yumuşak Hareket

```
1 IVME = 0.5          # her karede hıza eklenecek miktar
2 MAX_HIZ = 5.0       # yatay maksimum hız
3 SURTUNME = 0.3      # tuş bırakılınca her karede hızdan
4                     ↪ düşülecek
5
6 if tuslar[pygame.K_LEFT]:
7     self.vx -= IVME
8 elif tuslar[pygame.K_RIGHT]:
9     self.vx += IVME
10 else:
11     # Sürtünme: hızı yavaşça sıfıra yaklaştır
12     if self.vx > 0:
13         self.vx = max(0, self.vx - SURTUNME)
14     elif self.vx < 0:
15         self.vx = min(0, self.vx + SURTUNME)
16
17 # Maksimum hız sınırı
18 self.vx = max(-MAX_HIZ, min(MAX_HIZ, self.vx))
```

Şimdi hareket "ağırlık" kazanır. Tuşa bastığın anda karakter bir anda maksimi-

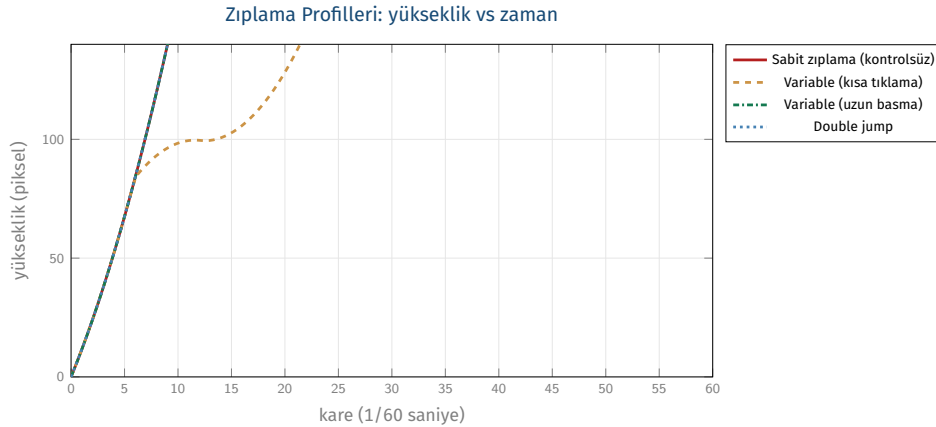
mum hızda koşmaya başlamaz; birkaç karede hızlanır. Bu farkı hissetmek için `01_ham_ziplama.py` ile yeni kodu yan yana çalıştır; gözle değil, *parmakla* hissedeceksin.

41.1.2 İvme, Maksimum Hız ve Sürtünmenin Roller

Her parametrenin rolü farklıdır. Hızlıca tanıyalım:

Tablo 41.1: Platform hareketi parametreleri ve etkileri

Parametre	Etkisi
IVME	Karakterin hızlanma oranı. Yüksek değer “fırla” hissi, düşük değer “sürünerek başla” hissi verir.
MAX_HIZ	Ulaşabileceği azami hız. Oyunun “tempo”sunu belirler.
SURTUNME	Tuş bırakıldığında ne kadar hızlı durduđu. Yüksek değer hassas kontrol, düşük değer buzda kayma hissi.
YERCEKIMI	Dikey hızın her karede artış miktarı. Yüksek değer ağır düşüş, düşük değer ay yürüyüşü.
ZIP_HIZI	Zıplama anında atanan başlangıç dikey hızı. Yercekimiyle birlikte zıplama yüksekliğini belirler.



Şekil 41.1: Farklı `IVME`, `MAX_HIZ` ve `ZIP_HIZI` kombinasyonlarının zıplama profili üzerindeki etkisi.

41.1.3 “Hissiyatı Ayarlamak” Ne Demek?

Platform oyununu büyük yapan şey genellikle bu sabitlerin *tutkuyla* ayarlanmış olmasıdır. Celeste’in yaratıcısı Matt Thorson, oyunun zıplama fiziğini haftalarca ayarladığını söylemişti. Sen de aynı yolu izleyebilirsin:

1. Varsayılan değerlerle oyna ve ne hissettiğine dikkat et.
2. Sadece *bir* parametreyi değiştir; ikisini birden değil.
3. Değişikliği test et; hissin ne kadar farklılaştığını yaz.

4. Tersini dene: değeri iki katına çıkar, yarıya indir. Uç değerler fikir verir.
5. En sevdiğin değeri sabitle ve bir sonrakine geç.

⚠ Dikkat: Tek Seferde Her Şeyi Değiştirme

Eğer hem `IVME`, hem `SURTUNME`, hem de `MAX_HIZ` birlikte değişirse hangi parametrenin ne etki ettiğini anlayamazsın. Her seferinde tek bir parametreyle çalış.

💡 İpucu: Canlı Ayar Panosu

Ayar yaparken sürekli kodu düzenleyip çalıştırmak yorucudur. Yan tuşlarla sabitleri oyun içinde değiştirebilen küçük bir debug paneli yazarsan saatler kazanırsın. Bkz. `hissiyat_karsilastirma.py` örneği.

● Bilgi: Pratik Alıştırma

`01_ham_ziplama.py` dosyasını çalıştır. Önce varsayılan değerlerle sağa sola koşarak bir tur at. Sonra `IVME`'yi 0.5'ten 0.1'e düşür. Karakterin neredeyse hiç hareket etmediğini, tuşu beklediğini göreceksin. Tekrar 1.5 yap; bu sefer sert, anlık bir his aldığını göreceksin. Sonra `SURTUNME`'yi değiştir ve tuş bırakıldığında karakterin nasıl durduğunu karşılaştı.

41.2 Coyote Time: Hatanın Affedilmesi

Bir platform oyununda tutarsız bir şekilde “hata” yaptığını düşündüğün durumları hatırla: Platformun kenarından düştün, zıplamaya bastın ama *tam bir kare geç kaldığın için* karakter zaten uçurumdaydı. Suçu kendinde aramaya başlama-dan önce bil ki bu gerçekten senin hatan değildi. Bu tür “ince hatalar” için iyi platformerlar oyuncuya bir affediş tanır: *coyote time*. Tam da Looney Tunes'daki Wile E. Coyote'un uçurumdan sonra birkaç saniye havada asılı kalmasına atfen adlandırılmış.

■ Tanım: Coyote Time

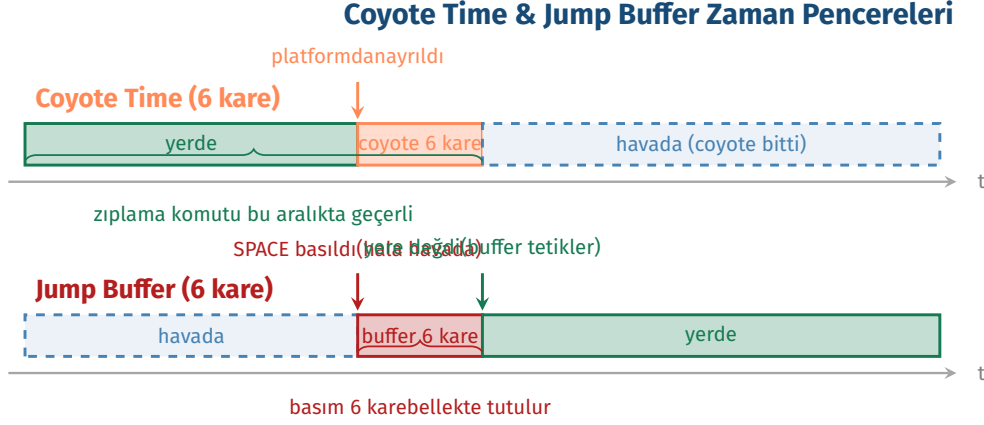
Coyote time, karakterin platformdan ayrıldıktan *hemen sonra* da bir süre zıplama yapabildiği toleransa verilen addır. Tipik değer 3–8 kare arasındır (60 FPS'de 50–130 ms). Oyuncu için görünmez bir “yumuşatıcı”dır; hissiyatı ciddi biçimde değiştirir.

41.2.1 Coyote Time Neden İşe Yarar?

Platform oyununda oyuncunun ve karakterin yaptığı arasında küçük bir “algı gecikmesi” vardır. Oyuncu gözüyle platformun kenarını görür, parmağına zıplama komutu gönderir; bu arada karakter bir-iki kare yürümeye devam etmiş olur. Eğer

katı bir “sadece yerdeyken zıplayabilirsin” kuralı uygularsak bu mikro-gecikme oyuncuyu sürekli cezalandırır.

Coyote time şunu der: “Platformdan yeni ayrıldıysan, senin niyetin hala oradayken zıplamaktı; sana bu fırsatı vereceğim.”



Şekil 41.2: Coyote time ve jump buffer zaman pencereleri: yeşil bölgelerde zıplama komutu geçerlidir.

41.2.2 Nasıl Uygulanır?

Fikri basittir: `self.yerde` bayrağı `False` olduğu anda bir sayaç başlatırız. Bu sayaç her karede azalır; sayaç 0’dan büyükse oyuncu hâlâ zıplayabilir. Sayaç sıfırlanınca gerçek “havadayım” durumu başlar.

Python – Coyote Time Sayacı

```

1 COYOTE_SURE = 6 # kare cinsinden (60 FPS'de ~100 ms)
2
3 class Oyuncu:
4     def __init__(self, x, y):
5         self.rect = pygame.Rect(x, y, 24, 32)
6         self.vx = 0
7         self.vy = 0
8         self.yerde = False
9         self.coyote_sayac = 0
10
11     def guncelle(self, tuslar, katilar):
12         eski_yerde = self.yerde
13         # ... yatay ve dikey hareket (Bölüm 10'daki gibi)
14         ↪ ...
15
16         # Coyote: yere basmayı BIRAKTIGI anda sayacı
17         ↪ başlat
18         if eski_yerde and not self.yerde:
19             self.coyote_sayac = COYOTE_SURE

```

```

18         elif self.coyote_sayac > 0:
19             self.coyote_sayac -= 1
20
21     def zıpla(self):
22         if self.yerde or self.coyote_sayac > 0:
23             self.vy = -11
24             self.coyote_sayac = 0 # tüketildi
25             self.yerde = False

```

Dikkat: Sayacı zıpladıktan hemen sonra sıfırlıyoruz. Aksi halde aynı coyote penceresi içinde oyuncu iki kez zıplayabilir. Bu ünitenin son konusunda *kasıtlı* olarak ikinci zıplamayı getireceğiz (double jump), ama o zaman ayrı bir bayrakla kontrol edeceğiz.

41.2.3 Coyote Time Değerini Nasıl Seçerim?

Çok kısa bir coyote time hiç coyote time olmamasına yakındır; çok uzun olan ise “sihir” gibi hissedilir ve oyuncu karakterin uçtuğunu düşünür. Aşağıdaki aralık iyi bir başlangıç noktasıdır:

Tablo 41.2: Coyote time değer aralıkları (60 FPS varsayımı)

Değer	ms cinsinden	His
0 kare	0 ms	Katı; her kare önemli
3 kare	≈ 50 ms	Hafif tolerans, hissedilmez
6 kare	≈ 100 ms	Standart, doğal (önerilen)
10 kare	≈ 165 ms	Bağışlayıcı, kolay
20+ kare	> 330 ms	Sihirli, gerçek dışı

Celeste 6 kare, Super Mario Odyssey 5 kare, Hollow Knight 8 kare civarı kullanır. 6 varsayılan değer olarak mantıklıdır.

41.2.4 Coyote + Çifte Kontrol

`self.yerde` tek başına yeterli değil; “yerdeydim, az önce bırakmıştım” durumunu da kapsaması gerekiyor. Zıplama kontrolünü şöyle yeniden yazarız:

Python – Zıplama Kontrol Fonksiyonu

```

1 def zıpla(self):
2     # yerde ya da coyote penceresi açıksa zıpla
3     if self.yerde or self.coyote_sayac > 0:
4         self.vy = -11
5         self.coyote_sayac = 0
6         self.yerde = False

```

⚠ Dikkat: Coyote Sayacını Doğru Yerde Başlat

Sayacı sadece **yerde** → **havada** geçişinde başlatmalısın, her **not yerde** karede değil. Aksi takdirde oyuncu uzun süre havada düşerken coyote hiç bitmez ve sorun yaratır.

💡 İpucu: Test İçin Yavaşlatma

Coyote time 6 karede biter; gözünle görmek zor. Test ederken `saat.tick(10)` ile oyunu 10 FPS'e düşür; artık pencereyi saniyelerce görebilirsin. Doğruluğunu test edip sonra 60'a geri al.

● Bilgi: Pratik Alıştırma

`02_coyote_time.py` dosyasını çalıştır. Üst panelde **coyote: N** sayacı her karede güncelleniyor. Platformun kenarından koşarak düş ve hemen SPACE'e bas. **COYOTE_SURE** değerini 0 yap; “hep kaçırıyorum” hissini yaşa. Tekrar 20 yap; karakterin havada bir gömü kazıyormuş gibi zıpladığını göreceksin. Sonra 6'ya dön — işte o sihirli tatlı nokta.

41.3 Jump Buffer: Erken Basılan Tuşu Kaydetmek

Coyote time platformdan *ayrıldıktan sonra* zıplamaya izin veren toleranstır. Bunun aynadaki karşılığı olan bir başka affediş daha var: *jump buffer*, yani oyuncunun yere değmeden birkaç kare *önce* zıplama tuşuna basması durumunda o komutu hatırlayıp yer temasıyla birlikte gerçekleştirmek.

■ Tanım: Jump Buffer

Jump buffer, zıplama tuşuna basıldığı anı kısa bir süre (genelde 3–8 kare) bellekte tutar. Eğer bu süre içinde karakter yere değerse tuş *o anda* basılmış gibi işlenir. Kısa süre içinde yer temasıyla eşleşmezse tuş kaydı silinir.

41.3.1 Neden Jump Buffer?

Bir oyuncu hızlı hareket halinde, platforma inmeden önce SPACE'e basar — bu çok yaygın bir alışkanlıktır. “Hava-zıplama” yani yere değmeden zıplama komutu vermek. Eğer biz bu erken tuş basmayı “yok” sayarsak, oyuncuya “önce yere bas, sonra tekrar bas” demek zorunda kalırız. Bu ritmi bozar ve oyunu “katı” hissettirir.

Jump buffer şöyle der: “Sen tuşa basmışsın, ben seni duydum. Sadece birkaç kare bekle — ayağın yere değer değmez zıplatırım.”

Ayrıca Şekil 41.2 coyote time ile jump buffer pencerelerini yan yana gösterir.

41.3.2 Uygulama

Jump buffer mantığı coyote time'a çok benzer, sadece ters yönde çalışır: coyote "yerdeydim, hâlâ sayılsın" der; buffer ise "tuşa bastım, yer temasında işlesin" der.

Python – Jump Buffer Sayacı

```

1  BUFFER_SURE = 6
2
3  class Oyuncu:
4      def __init__(self, x, y):
5          # ... önceki init ...
6          self.buffer_sayac = 0
7
8      def zipla_istegi(self):
9          """KEYDOWN olayından çağrılır; buffer'ı
10             ↪ doldurur."""
11          self.buffer_sayac = BUFFER_SURE
12
13      def guncelle(self, tuslar, katilar):
14          # ... hareket, çarpışma ...
15
16          # Buffer sayacı her karede azalır
17          if self.buffer_sayac > 0:
18              self.buffer_sayac -= 1
19
20          # Buffer doluyken yerde isek zipla
21          if self.buffer_sayac > 0 and (self.yerde or
22             ↪ self.coyote_sayac > 0):
23              self.vy = -11
24              self.yerde = False
25              self.buffer_sayac = 0
26              self.coyote_sayac = 0

```

Artık oyuncunun SPACE basma anıyla yere değme anı farklı olabilir; sistem onları birleştirir.

41.3.3 Ana Döngüde Olay Akışı

Jump buffer özelliği anlık bir tuş basımı (pygame.KEYDOWN) ile başlar. Ana döngüde şu şekilde yakalanır:

Python – KEYDOWN Yakalama

```

1  for olay in pygame.event.get():
2      if olay.type == pygame.QUIT:
3          calistir = False
4      elif olay.type == pygame.KEYDOWN:
5          if olay.key in (pygame.K_SPACE, pygame.K_UP,
6             ↪ pygame.K_w):
7              oyuncu.zipla_istegi() # buffer'ı doldur

```

Önemli: `KEYDOWN` kullan, `key.get_pressed()` değil. `get_pressed()` tuşun hâlâ basılı olup olmadığını söyler ama hangi kareden *başladığını* söylemez. Buffer için bize anlık bir olay gerekiyor.

41.3.4 Coyote + Jump Buffer = Tam Affediş

Coyote ve buffer birlikte kullanıldığında dört farklı kombinasyon olasılığı ortaya çıkar:

Tablo 41.3: Coyote time + Jump buffer kombinasyonları

Yer teması	Tuş basımı	Sonuç
Yerde, önce	Yerde, sonra	Klasik zıplama
Yerdeydi, yeni düştü	Sonra basıldı (< 6 kare)	Coyote devreye girer
Havada, önce bastı	Sonra yere değdi (< 6 kare)	Buffer devreye girer
Havada, önce bastı	Yere hiç değmedi	Tuş silinir

Üç farklı durumda “zıplama çalıştı” hissi oluşur; hiçbirinde oyuncu “hata yaptım” diye düşünmez. İşte bu, gerçek hissiyat.

41.3.5 Fark Görmek İçin Yan Yana Karşılaştır

Bu ünitenin örneklerinde `01_ham_zıplama.py` (hiç affediş yok), `02_coyote_time.py` (sadece coyote) ve `03_jump_buffer.py` (hem coyote hem buffer) dosyaları bile-rek aynı haritayı paylaşır. Aynı ritimde oyna; farkı parmağın anlayacak.

⚠ Dikkat: Buffer Süresini Coyote’tan Büyük Yapma

Eğer `BUFFER_SURE > COYOTE_SURE` yaparsan ve oyuncu tuşa çok önceden basarsa, karakter yere değer değmez zıplayıp gider — oyuncu “yere bastığımı bile fark etmedim” der. 6 kareye her ikisi için de iyi bir değerdir.

💡 İpucu: Debug Metni

Oyuncu karakterinin üzerinde `buffer: 4 coyote: 2` gibi bir etiket göster. Oyuncu hissetmez, ama sen parametreleri ayarlarken tam olarak ne zaman hangi pencerenin aktif olduğunu görürsün. Son sürümde kaldırırsın.

● Bilgi: Pratik Alıştırma

`03_jump_buffer.py` dosyasını çalıştır. Havada iken arka arkaya üç kez `SPACE`’e bas. Sadece `son` tuş basımı 6 kare içinde yere değerse zıplama olur. Şimdi `BUFFER_SURE = 15` yap (yani 250 ms). Eski tuş basımları hâlâ “hatırlanıyor”; oyuncu yerden şey yapmamış gibi, sihirli bir şekilde zıplıyor gibi hissedilir. Daha sonra 0 yap: tüm erken tuş basımları kayboluyor, oyun katılıyor.

41.4 Variable Jump ve Double Jump

İyi bir platformer yalnızca “affedici” olmakla kalmaz, aynı zamanda *ifade edicidir*. Oyuncu zıplamanın ne kadar yüksek olacağını kendi kontrol edebilmeli; hatta bazen havadayken bir ek güç hakkı da isteyebilir. Bu konuda iki klasik mekanik ekliyoruz: *variable jump* (tuş tutma süresine göre yükseklik) ve *double jump* (havada ikinci zıplama).

41.4.1 Variable Jump: Kısa Tıkla, Uzun Zıpla

Oyuncu SPACE’e hızlıca basıp bırakırsa kısa bir zıplama olmalı; tuşu basılı tutarsa tam yüksek bir zıplama olmalı. Bu “yüksekliği ben ayarlarım” kontrolü bir platform oyununda inanılmaz büyük bir fark yaratır: Küçük bir engelin üzerinden atlamak ile yüksek bir platforma çıkmak artık *aynı tuşun* iki farklı kullanımıdır.

Nasıl çalışır? Fikir basit: zıplama anında tam hız veriyoruz, ama eğer oyuncu tuşu erken bırakırsa yukarı doğru olan hızı yarıya keseriz.

Python – Variable Jump Mantığı

```

1  class Oyuncu:
2      def zipla(self):
3          if self.yerde or self.coyote_sayac > 0:
4              self.vy = -11          # tam zıplama gücü
5              self.yerde = False
6              self.zipliyor = True    # bayrak: yukarı
7                                      ↪ hareketteyiz
8
9      def tus_birakildi_ziplama(self):
10         """KEYUP olayında çağrılır. Tuş erken bırakıldıysa
11         yukarı hızı kısalt."""
12         if self.zipliyor and self.vy < 0:
13             self.vy *= 0.4          # yukarı hızı %60 azalt
14             self.zipliyor = False

```

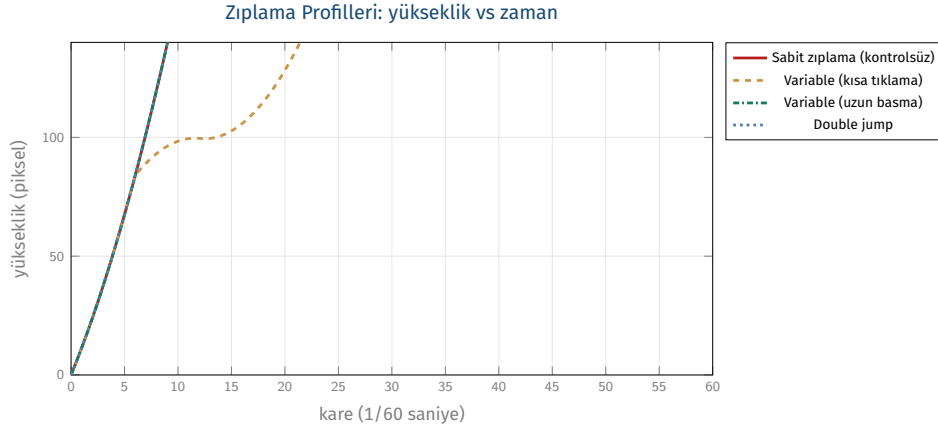
Ana döngüdeki olay yakalama:

Python – KEYUP ile Kesme

```

1  for olay in pygame.event.get():
2      if olay.type == pygame.KEYDOWN:
3          if olay.key in (pygame.K_SPACE, pygame.K_UP):
4              oyuncu.zipla_istegi()    # jump buffer doldur
5      elif olay.type == pygame.KEYUP:
6          if olay.key in (pygame.K_SPACE, pygame.K_UP):
7              oyuncu.tus_birakildi_ziplama()

```



Şekil 41.3: Zıplama profilleri: sabit (kırmızı), variable kısa (sarı), variable uzun (yeşil), double jump (mavi).

41.4.2 Variable Jump Parametresi

Yukarıdaki kodda `self.vy *= 0.4` satırı kritik. Bu çarpan variable jump hissini belirler:

Tablo 41.4: Variable jump kesme çarpanı

Çarpan	His
0.0	Tuğu bırakır bırakmaz yukarı hız sıfır. Düşmeye başlar.
0.3–0.5	Doğal, reaktif (önerilen)
0.7	Hafif etki; oyuncu farkı zor hisseder
1.0	Variable jump yok; sabit yükseklik

Dikkat: Kesme sadece `vy < 0` (yani yukarı hareket) sırasında yapılır. Düşüşte bir etkisi olmaz. Bu sayede oyuncu zirveye ulaştıktan sonra tuğu bırakmasa bile fark yaratmaz.

41.4.3 Double Jump: Havada İkinci Güç

Double jump, karakterin yerdeyken bir, havadayken bir daha olmak üzere iki zıplama hakkı kullanmasına izin veren mekaniktir. Oyuncunun platformlar arası manevra alanını ciddi biçimde artırır; seviye tasarımını da daha dikey ve “zıplama bulmacası” yapmaya açar.

Python – Double Jump

```

1 MAX_ZIPLAMA = 2 # toplam zıplama hakkı (double jump için
  ↳ 2)
2
3 class Oyuncu:
4     def __init__(self, x, y):
5         # ... önceki init ...

```

```

6         self.kalan_ziplama = MAX_ZIPLAMA
7
8     def guncelle(self, tuslar, katilar):
9         # ... hareket ve çarpışma ...
10
11         # Yere değince ziplama haklarını yenile
12         if self.yerde:
13             self.kalan_ziplama = MAX_ZIPLAMA
14
15     def zipla(self):
16         if self.kalan_ziplama > 0:
17             self.vy = -11
18             self.kalan_ziplama -= 1
19             self.yerde = False
20             self.zipliyor = True

```

Artık oyuncu yerdeyken bir zıplar (hak: $2 \rightarrow 1$), havada iken bir daha zıplar (hak: $1 \rightarrow 0$). Yere değmez değmez hak tekrar 2'ye yükselir.

⚠ Dikkat: Coyote Time + Double Jump

Coyote time “sanki yerdeyim” toleransı sağlar; double jump ise “havada ek hak” verir. İki mekanik beraber çalışmalı. İlk zıplama coyote penceresini kullanabilir; ikinci zıplama sadece `kalan_ziplama`'ya bakar.

41.4.4 Hepsi Bir Arada: Güncellenmiş Zıplama

Şimdi coyote time, jump buffer, variable jump ve double jump'ı tek bir `zipla` fonksiyonunda birleştirelim:

Python – Tam Zıplama Mantığı

```

1     def zipla(self):
2         if self.yerde or self.coyote_sayac > 0:
3             # İlk zıplama (yerdeyken veya coyote penceresinde)
4             self.vy = -11
5             self.kalan_ziplama = MAX_ZIPLAMA - 1
6             self.coyote_sayac = 0
7             self.yerde = False
8             self.zipliyor = True
9         elif self.kalan_ziplama > 0:
10            # Double jump (havadayken ek hak)
11            self.vy = -9 # biraz daha zayıf
12            self.kalan_ziplama -= 1
13            self.zipliyor = True

```

Bazı oyunlar double jump'ı ilk zıplamadan daha az güçlü yapar (yukarıda -9 vs -11). Bu oyuncunun “iki zıplamayı birleştirip tek zıplamaya denk yapmamasını” sağlar; her zıplamanın kendine özgü bir rolü olur.

41.4.5 Test Desenleri

Tüm mekanikler doğru mu diye kontrol etmek için şu test desenlerini dene:

1. **Kısa tıklama:** SPACE'e kısa bas, karakter alçak zıplasın.
2. **Uzun basma:** SPACE'i uzun tut, tam yüksek zıplasın.
3. **Coyote:** Platformdan düş, hemen SPACE bas — zıplamalı.
4. **Buffer:** Havadayken SPACE bas; yere değince zıplamalı.
5. **Double jump:** Zıpla; havada tekrar bas; ikinci zıplama olsun.
6. **Üçüncü zıplama?** Tekrar bas; *olmamalı*.
7. **Yenilenme:** Yere değ; tekrar iki zıplama hakkın olmalı.

💡 İpucu: Animasyon Bağlantısı

Double jump genelde ayırt edici bir animasyonla desteklenir: “takla” veya “kanat çırpma”. Bölüm 8'deki animasyon framework'üyle `if self.zipliyor and self.kalan_ziplama < MAX_ZIPLAMA-1:` kontrolü ile ayrı bir animasyon durumu tetikleyebilirsiniz.

🔵 Bilgi: Pratik Alıştırma

`04_variable_double_jump.py` dosyasını çalıştır. Önce kısa tıklama, sonra uzun basma ile zıplama yüksekliğinin farkına bak. Ardından havada ikinci kez SPACE bas — double jump devrede. `MAX_ZIPLAMA` değerini 3 yap: artık triple jump! Oyun dengene bu mekaniği ekleme vakti geldi.

41.5 Sonuç

Bu ünite de karakterin hareketini “çalışan” olmaktan “harika hissettiren” olmaya çıkardık. Ham hızı ivmeyle değiştirdik, coyote time ile düşüş hatalarını affettik, jump buffer ile erken tuş basımlarını kurtardık ve variable + double jump ile zıplamaya ifade yeteneği kazandırdık. Artık elinde bir platform oyunu için gerekli “hareket hissi” temeli var.

41.5.1 Anahtar Kavramlar

Tablo 41.5: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
Game feel	Kumanda tepkisinin nasıl hissedildiği; sıkılık, ağırlık, affediş
İvme	Her karede hıza eklenen miktar (yumuşak başlangıç)
Sürtünme	Tuş bırakıldığında hızı sıfıra yaklaştıran azaltma
Coyote time	Platformdan ayrıldıktan sonra kısa bir süre hâlâ zıplayabilme toleransı
Jump buffer	Yere değmeden önce basılmış SPACE tuşunu tutarak yer temasında işlemek
Variable jump	Tuş bırakıldığında yukarı hızı kesmek; basma süresi = zıplama yüksekliği
Double jump	Havada ikinci bir zıplama hakkı
Zıplama sayacı	<code>kalan_zıplama</code> ile toplam zıplama hakkını kontrol etme

41.5.2 Önemli Komutlar

Tablo 41.6: Ünite 1 – Temel komutlar

Komut	Açıklama
<code>pygame.KEYDOWN</code>	Tuşa basıldığı anki tek seferlik olay
<code>pygame.KEYUP</code>	Tuşun bırakıldığı anki olay (variable jump için)
<code>self.vy *= 0.4</code>	Yukarı hızı kesme (variable jump)
<code>max(-MAX, min(MAX, vx))</code>	Hızı bir aralığa sıkıştırma
<code>self.coyote_sayac</code>	Coyote penceresi için kare sayacı
<code>self.buffer_sayac</code>	Jump buffer penceresi için kare sayacı
<code>self.kalan_zıplama</code>	Havada kalan zıplama hakkı sayacı

41.5.3 Bir Sonraki Ünite: Platform Çarpışmaları

Karakterimiz artık harika hissettiriyor. Sıradaki hedef: kendisinin üzerine durduğu *platformları* daha zengin yapmak. Ünite 2’de:

- Çarpışmanın hangi yönden geldiğini tespit edeceğiz
- Tek yönlü (one-way) platformlar yapacağız — alttan geçmek ama üsten durmak
- Hareketli platformlar ekleyeceğiz — oyuncuyu üstüne alıp taşırlar
- Dağılan ve zıplatan özel platformlar yapacağız

Hareket hissedilir oldu; şimdi hareketin *içinde olduğu dünyayı* aynı özenle tasarlama zamanı.

💡 İpucu: Hissiyat Ayarlaması Bitmez

Profesyonel platformerlar fiziği üzerinde ayları harcar. Celeste'in tasarımcısı oyunun fiziğini 40+ iterasyonda yeniden ayarladığını söylemişti. Sen de bu üniteye yazdığın `MAX_HIZ`, `IVME`, `ZIP_HIZI`, `COYOTE_SURE` ve `BUFFER_SURE` değerleriyle oyna — oyunun karakteri bu sayıların içinde gizli.

Platform Çarpışmaları

“Her platform çeşidi, oyuncuya yeni bir soru sorar: “Bunu nasıl aşarsın?””

— ANONİM PLATFORMER TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Çarpışmanın hangi yönden (üst, alt, sol, sağ) geldiğini tespit etme
- Tek yönlü (one-way) platform: sadece üstten çarpışma ve aşağı inme (drop-through)
- Hareketli platform: oyuncuyu delta transferiyle taşıma
- Dağılan (crumbling) platform: ayak basınca zamanla kaybolan yüzey
- Zıplatan (bouncy) platform: oyuncuya yüksek bir zıplama hızı aşıl原因an yaylı yüzey

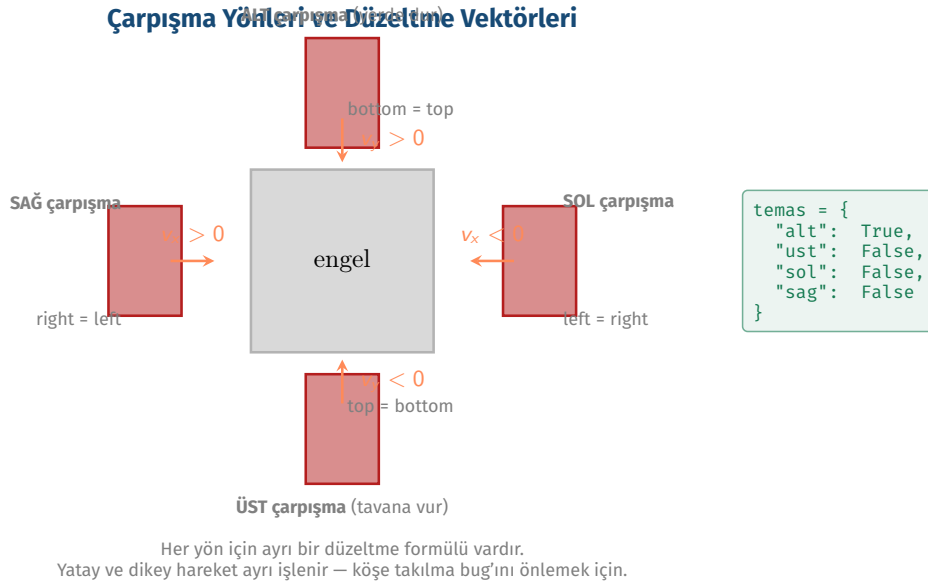
42.1 Çarpışma Yönü Tespiti

Bölüm 10’da öğrendiğin yatay/dikey ayırma deseniyle çarpışma yönünü aslında “zaten” biliyoruz: yatay blokta $v_x > 0$ ise sağdan, $v_x < 0$ ise soldan çarpıştık; dikey blokta $v_y > 0$ ise üstten indik, $v_y < 0$ ise alttan vurduk. Ama bu bilgiyi isimlendirip yapılandırmak bize Ünite 2 boyunca çok işe yarayacak.

■ Tanım: Çarpışma Yönü

İki Rect’in çarpışmasında oyuncu ile engel arasındaki temasın hangi yönden olduğuna *çarpışma yönü* denir. Dört temel yön vardır: **üst, alt, sol, sağ**. Çözüm (oyuncuyu nereye itelim) her yön için farklıdır.

42.1.1 Dört Yön ve Çözümleri



Şekil 42.1: Oyuncu ile engel arasındaki dört temel çarpışma yönü ve her biri için konum düzeltmesi.

Her yön için tek satırlık düzeltme formülü aşağıda:

Tablo 42.1: Çarpışma yönleri ve düzeltme

Yön	Koşul	Düzeltilme
Alt (yerde durma)	$v_y > 0$	<code>rect.bottom = blok.top,</code> <code>yerde = True</code>
Üst (tavana vurma)	$v_y < 0$	<code>rect.top = blok.bottom,</code> <code>vy = 0</code>
Sağ (sağa çarpma)	$v_x > 0$	<code>rect.right = blok.left</code>
Sol (sola çarpma)	$v_x < 0$	<code>rect.left = blok.right</code>

42.1.2 Bilgi Döndüren Çarpışma Fonksiyonu

Yön bilgisini ihtiyaç duyanlara iletmek için `guncelle` fonksiyonunu güncelleyelim. Artık “hangi yönlerden temas oldu” bilgisini sözlük halinde döndüreceğiz:

Python – Çarpışma Yönü Sözlüğü

```

1 def _yatay_carpisma(self, katilar):
2     temas = {"sol": False, "sag": False}
3     for k in katilar:
4         if self.rect.colliderect(k):
5             if self.vx > 0:
6                 self.rect.right = k.left
7                 temas["sag"] = True

```

```

8         elif self.vx < 0:
9             self.rect.left = k.right
10            temas["sol"] = True
11        return temas
12
13    def _dikey_carpisma(self, katilar):
14        temas = {"ust": False, "alt": False}
15        for k in katilar:
16            if self.rect.colliderect(k):
17                if self.vy > 0:
18                    self.rect.bottom = k.top
19                    self.vy = 0
20                    temas["alt"] = True # yerde
21                elif self.vy < 0:
22                    self.rect.top = k.bottom
23                    self.vy = 0
24                    temas["ust"] = True # tavana vurdu
25        return temas

```

Artık dışarıdan birileri “bu karede tavana vurdu mu?” sorusunu cevaplayabilir. Bu, Ünite 2’nin sonraki konularında (tek yönlü platformlar, hareketli platformlar) doğrudan işimize yarayacak.

42.1.3 Neden Önemli? Somut Örnekler

Çarpışma yönünü bilmenin pratik değerini birkaç gerçek durumda görelim:

1. **Tek yönlü platform:** Sadece `temas["alt"]` aktif olursa durdurulur; oyuncu alttan zıplarken `temas["ust"]` olmaz — geçer.
2. **Düşman stomp:** Eğer oyuncu düşman üzerine `temas["alt"]` ile iniyorsa düşman ölür, oyuncu zıplar. Yan temas ise oyuncu hasar alır.
3. **Duvar zıplama (wall jump):** `temas["sol"]` veya `temas["sag"]` aktif ve oyuncu havadayken duvardan itebilir.
4. **Crumbling platform tetiği:** `temas["alt"]` bir kez aktif olursa platform zamanlayıcısı başlar.

Yönü bilmek, birçok farklı platform davranışını tek bir altyapıyla destekler.

42.1.4 Sweep (Genişletilmiş) Yön Tespiti

Bazen oyuncu çok hızlı hareket ederse tek bir karede hem yatay hem dikey aynı engele çarpabilir. Klasik ayır-işle yaklaşımı bunu çözer ama bazı durumlarda “köşe pikseli” kararsız olabilir. İleri bir teknik olarak *sweep* testi kullanılır:

Python – Hareket Öncesi Geçmiş Konum

```

1    def _yon_tespit(self, eski_rect, blok):
2        """Çarpışma yönünü önceki konuma göre belirler."""
3        if eski_rect.right <= blok.left:
4            return "sag"
5        elif eski_rect.left >= blok.right:

```

```

6         return "sol"
7     elif eski_rect.bottom <= blok.top:
8         return "alt"    # yukarıdan iniyordu
9     elif eski_rect.top >= blok.bottom:
10        return "ust"    # aşağıdan çarpıyordu
11    return None

```

Bu yöntem oyuncunun *önceki* konumu ile çarpıştığı bloğun durumunu karşılaştırır. Böylece “çarpışmaya hangi yönden yaklaştım?” sorusunu net olarak cevaplar.

⚠ Dikkat: vx = 0 Durumu

`self.vx = 0` iken çarpışma olursa (başlangıç konumu bir blok içinde ise) yön tespiti yapılmaz. Böyle durumlarda varsayılan bir yön (örneğin “hiç iterek çıkar”) belirle.

💡 İpucu: Debug ile Yönleri Gör

Çarpışma yönlerini görselleştirmek için oyuncu kutusunun dört kenarına ufak dikdörtgenler çiz; `temas["alt"]` doğruysa alt kenarı yeşil, değilse gri yap. Dört kenarın yandığı karelerde kod doğru çalışıyor demektir.

● Bilgi: Pratik Alıştırma

`01_carpisma_yonu.py` dosyasını çalıştır. Oyuncunun etrafında dört renkli şerit göreceksin: üst (mavi), alt (yeşil), sol (kırmızı), sağ (turuncu). Temas olduğunda ilgili şerit yanar. Platformun kenarına, altına ve sağına çarparak hangi yönün hangi durumda yandığını gözlemler.

42.2 Tek Yönlü Platform ve Drop-Through

Platform oyunlarının en ikonik mekaniklerinden biri *tek yönlü platform* türüdür: Alttan zıplayıp yukarı geçersin ama üstten bastığında durursun. Ayrıca **drop-through** denen hareketle aşağı tuşunu basılı tutup zıplayınca platformun altına geri geçebilirsin.

■ Tanım: Tek Yönlü Platform

Sadece bir yönden (genellikle üstten) çarpışma veren platform türüdür. Oyuncu alttan geçebilir, kenarlardan serbestçe girebilir ama üzerine indiğinde durur. Mario’nun bulutları, Super Smash Bros’un platformları, Celeste’in sarı tahtaları bu türe örnektir.

42.2.1 İki Rect Listesi

Tek yönlü platformları solid tile'lardan ayrı bir listede tutarız. Solid platformlarla aynı listeye koyup “yön kontrolü” yapmayı deneyebilirsin ama iki farklı liste daha okunaklıdır.

Python – İki Grup: Solid + One-Way

```

1 class Seviye:
2     def _grupla(self):
3         for r, satir in enumerate(self.harita):
4             for c, t in enumerate(satir):
5                 rect = pygame.Rect(c*TILE, r*TILE, TILE,
6                                     ↪ TILE)
7                 if t == 2:
8                     self.katilar.append(rect)
9                 elif t == 6:      # YENİ: tek yönlü
10                    ↪ platform
11                    self.tek_yonluler.append(rect)
12                 elif t == 4:
13                     self.altinlar.append(rect)
14                 elif t == 5:
15                     self.tehlikeler.append(rect)

```

Bölüm 10’da kullanılan tile ID şeması (1=boş, 2=solid, 4=altın, 5=diken) korunur; 6 numaralı yeni bir ID ekledik: tek yönlü platform.

42.2.2 Çarpışma Mantığı

Tek yönlü çarpışma kuralı basittir: sadece oyuncu *yukarıdan aşağıya inerken* ve *platformun üst kenarını henüz geçmemişken* çarpışma olur. Bu iki koşul birlikte sağlanırsa karakter durur; aksi halde yokmuş gibi davranılır.

Python – Tek Yönlü Çarpışma

```

1 def _dikey_carpisma_tek_yonlu(self, tek_yonluler,
2     ↪ eski_alt):
3     """Sadece üstten iniyorken platformu durdur."""
4     for p in tek_yonluler:
5         if self.rect.colliderect(p):
6             # Düşüyoruz VE önceki karede platformun
7             ↪ üstündeydik
8             if self.vy > 0 and eski_alt <= p.top + 1:
9                 self.rect.bottom = p.top
10                self.vy = 0
11                self.yerde = True

```

`eski_alt` değişkeni oyuncu hareket etmeden önceki `rect.bottom` değeridir. Bu iki kontrol sayesinde:

- Alttan zıplayan oyuncu (`vy < 0`) engellenmez: geçer.
- Yanlardan giren oyuncu: “önceki alt” platform üstünde değildi, geçer.
- Üstten inen oyuncu (`vy > 0`, `eski_alt` platform üstü): durur.

42.2.3 Drop-Through: Aşağı + Zıplama

Bazen oyuncu bilerek platformdan aşağıya geçmek ister. Bu hareket için standart desen: *aşağı tuşunu basılı tut ve zıplama tuşuna bas*. Karakter bir süre (örneğin 10 kare) o platformlarla çarpışmayı tamamen yok sayar.

Python – Drop-Through Zamanlayıcı

```

1 DROP_SURE = 10 # kare
2
3 class Oyuncu:
4     def __init__(self, x, y):
5         # ... önceki init ...
6         self.drop_sayac = 0 # > 0 iken tek yönlüler yok
7         ↪ sayılır
8
9     def drop_through(self, tuslar):
10        """Aşağı + SPACE birlikte basılıysa drop
11        ↪ tetikle."""
12        asagi = tuslar[pygame.K_DOWN] or
13        ↪ tuslar[pygame.K_s]
14        if asagi and self.yerde:
15            self.drop_sayac = DROP_SURE
16            self.yerde = False # platform yokmuş gibi
17            ↪ davran

```

Çarpışma mantığını da güncelleriz: Eğer `drop_sayac > 0` ise tek yönlü platformlar yok sayılır.

Python – Drop Sırasında Yok Sayma

```

1 def _dikey_carpisma_tek_yonlu(self, tek_yonluler,
2     ↪ eski_alt):
3     if self.drop_sayac > 0:
4         self.drop_sayac -= 1
5         return # bu kare platformları yok say
6     for p in tek_yonluler:
7         # ... yukarıdaki normal kontrol ...

```

⚠ Dikkat: Drop Süresi Çok Uzunsa

`DROP_SURE` değeri çok büyükse oyuncu drop komutuyla platformların altına düşmüş olur ve tekrar yukarı zıplayamaz. 10–15 kare (yaklaşık 170–250 ms) genellikle yeter.

42.2.4 Görselleştirme

Tek yönlü platform genelde ince bir çizgi veya sarı renkli bir kalas ile gösterilir. Oyuncu üst kenarın yukarıda olduğunu anında anlamalı:

Python – One-Way Platform Çizimi

```

1 for p in seviye.tek_yonlular:
2     c = kamera.apply(p)
3     # Sadece üst kısmı çiz (ince bir çizgi)
4     pygame.draw.rect(ekran, (230, 200, 60),
5                       (c.x, c.y, c.width, 8))
6     pygame.draw.rect(ekran, (160, 140, 40),
7                       (c.x, c.y, c.width, 8), 2)

```

Sarı-siyah ince şerit oyuncuya “buradan zıplayıp geçebilirsin” hissini verir.

💡 İpucu: Seviye Tasarımında Kullanım

Tek yönlü platformları dikey seviyelerde dizin. Oyuncu alttan zıplayarak yukarı çıkar, ama ilk yanlış hareketinde aşağı düşer. Celeste’in “Mt. Celeste” tırmanışı bu deseni sıkça kullanır. Drop-through ise “geri dönüş” tasarımı için harikadır.

🔹 Bilgi: Pratik Alıştırma

`02_tek_yonlu_platform.py` dosyasını çalıştır. Altında üç sarı tek yönlü platform göreceksin. Altından zıplayarak en üste çık. Sonra aşağı + SPACE ile bir alt platforma geç. Bu iki mekaniğin tek bir oyun için ne kadar zengin bir dikey hareket alanı sunduğunu göreceksin.

42.3 Hareketli Platform ve Delta Transferi

Hareketli platform (moving platform) kendi başına yatay veya dikey olarak ileri-geri gidiyor. Oyuncu üzerine atladığında bir problem beliriyor: Platform sağa kayarken oyuncu aynı hızla onunla beraber kaymazsa hemen platformdan düşer. Çözüm: platformun hareket ettiği miktarı (*delta*) oyuncuya da uygulamak.

■ Tanım: Delta Transferi

Delta transferi, hareketli bir platformun o kare içinde yaptığı konum değişimini ($\text{delta} = \text{yeni_konum} - \text{eski_konum}$) üzerindeki oyuncuya aynen aktarmaktır. Bu sayede platform kaydığında oyuncu üzerinde “yapışık” kalır.

42.3.1 HareketliPlatform Sınıfı

Önce platformu yapalım. Düz bir `Rect` yerine artık kendi hızı olan bir sınıf kullanıyoruz:

Python – HareketliPlatform Sınıfı

```

1 class HareketliPlatform:
2     def __init__(self, x, y, w, h, hedef_x, hedef_y,
3         ↪ hiz=2):
4         self.rect = pygame.Rect(x, y, w, h)
5         self.baslangic = pygame.Vector2(x, y)
6         self.hedef = pygame.Vector2(hedef_x, hedef_y)
7         self.hiz = hiz
8         self.yon = 1 # 1 = hedefe, -1 = başlangıca
9         self.delta = pygame.Vector2(0, 0) # bu karede ne
10        ↪ kadar hareket etti
11
12     def guncelle(self):
13         eski = pygame.Vector2(self.rect.topleft)
14
15         if self.yon == 1:
16             # Hedefe doğru git
17             istikamet = self.hedef -
18             ↪ pygame.Vector2(self.rect.topleft)
19         else:
20             istikamet = self.baslangic -
21             ↪ pygame.Vector2(self.rect.topleft)
22
23         mesafe = istikamet.length()
24         if mesafe < self.hiz:
25             # Hedefe vardık: yön değiştir
26             if self.yon == 1:
27                 self.rect.topleft = self.hedef
28             else:
29                 self.rect.topleft = self.baslangic
30             self.yon *= -1
31         else:
32             adım = istikamet.normalize() * self.hiz
33             self.rect.x += int(adım.x)
34             self.rect.y += int(adım.y)
35
36         # Bu karedeki delta
37         self.delta = pygame.Vector2(self.rect.topleft) -
38         ↪ eski

```

Artık platform kendisi hareket ediyor ve her karede **delta** özniteliğinde “bu karede ne kadar hareket ettim” bilgisini tutuyor.

42.3.2 Oyuncuya Delta Uygulama

Oyuncu hareketli platform üzerine indiğinde ona “yapıştırmak” için üzerindeki platformun deltasını oyuncunun **rect**'ine uyguluyoruz.

Python – Oyuncu Güncelleme

```

1 class Oyuncu:
2     def __init__(self, x, y):
3         # ... önceki init ...
4         self.uzerindeki_platform = None
5
6     def guncelle(self, tuslar, katilar, hareketliler):
7         # 1. Platforma binmişsek DELTA'yı ekle (oyuncu
8         ↪ konumuna)
9         if self.uzerindeki_platform:
10             self.rect.x +=
11                 ↪ int(self.uzerindeki_platform.delta.x)
12             self.rect.y +=
13                 ↪ int(self.uzerindeki_platform.delta.y)
14
15         # 2. Normal yatay hareket + çarpışma
16         # ... eski kod ...
17
18         # 3. Dikey hareket + çarpışma (hareketli platformu
19         ↪ da solid say)
20         tum_katilar = katilar + [h.rect for h in
21             ↪ hareketliler]
22         # ... dikey çarpışma ...

```

Delta'yı oyuncu tuşları işlemekten önce uygularız. Aksi takdirde hem platform hem oyuncu hareket ederken konum çakışmasında görsel “zıplama” oluşur.

42.3.3 Hangi Platforma Binmiş?

Oyuncunun hangi hareketli platforma “binmiş” olduğunu bilmek için dikey çarpışma sırasında kaydederiz:

Python – Bindiği Platformu Kaydet

```

1 def _dikey_carpisma(self, katilar, hareketliler):
2     self.uzerindeki_platform = None
3     self.yerde = False
4
5     # Önce solid (sabit) platformlar
6     for k in katilar:
7         if self.rect.colliderect(k):
8             if self.vy > 0:
9                 self.rect.bottom = k.top
10                self.yerde = True
11            elif self.vy < 0:
12                self.rect.top = k.bottom
13                self.vy = 0
14
15     # Sonra hareketli platformlar (delta transferi için
16     ↪ referans tut)

```

```

16     for h in hareketliler:
17         if self.rect.colliderect(h.rect):
18             if self.vy > 0:
19                 self.rect.bottom = h.rect.top
20                 self.yerde = True
21                 self.uzerindeki_platform = h # BİNDİK!
22             elif self.vy < 0:
23                 self.rect.top = h.rect.bottom
24                 self.vy = 0

```

42.3.4 Yatay Hareketli Platformda Sıkışma

Eğer platform sağa doğru hareket ederken oyuncunun arkasında bir duvar varsa platform oyuncuyu duvara doğru iter ve sıkıştırır. Bunu önlemek için delta uygulandıktan sonra yatay çarpışma kontrolü yaparız; duvara çarpılırsa oyuncu duvarın önünde durur, platform altından kayıp gider:

Python – Delta Sonrası Yatay Çarpışma

```

1  # Delta uygulandı
2  if self.uzerindeki_platform:
3      self.rect.x += int(self.uzerindeki_platform.delta.x)
4      # Duvara çarpma kontrolü
5      for k in katilar:
6          if self.rect.colliderect(k):
7              dx = int(self.uzerindeki_platform.delta.x)
8              if dx > 0:
9                  self.rect.right = k.left
10             elif dx < 0:
11                 self.rect.left = k.right

```

⚠ Dikkat: Çift Sayım Tuzağı

Delta transferini hem `rect.x`'e hem de `self.vx`'e ekleme; sadece `rect.x`'e doğrudan uygula. `vx` oyuncunun *kendi* yatay hızıdır ve klavye tarafından kontrol edilir. İkisini karıştıрма!

42.3.5 Platform Çizimi

Hareketli platform görsel olarak sabit olanlardan ayırt edilebilir olmalı. Mavi bir renk ve kenarlarında “kayar” efekti vermek için ince çizgiler iyi bir seçimdir.

Python – Hareketli Platform Çizimi

```

1  for h in hareketliler:
2      c = kamera.apply(h.rect)
3      pygame.draw.rect(ekran, (80, 140, 220), c)
4      pygame.draw.rect(ekran, (40, 80, 160), c, 2)
5      # Yön göstergesi: küçük üçgenler

```

```

6     if h.yon == 1 and h.hedef.x > h.baslangic.x:
7         pygame.draw.polygon(ekran, (255, 255, 255),
8             [(c.right - 10, c.centery),
9              (c.right - 16, c.centery - 4),
10             (c.right - 16, c.centery + 4)])

```

İpucu: Platform Çeşitleri

Yatay hareket en yaygındır (sağa-sola). Dikey hareket (yukarı-aşağı) “asansör” etkisi verir. Çapraz hareket (diagonal) daha nadirdir ama bulmaca platformları için güzeldir. `HareketliPlatform` sınıfı üçünü de destekler — sadece `hedef_x` ve `hedef_y` parametrelerini değiştir.

Bilgi: Pratik Alıştırma

`03_hareketli_platform.py` dosyasını çalıştır. Bir yatay bir de dikey hareketli platform göreceksin. Her birine atla; üzerinde “yapışık” kal. Şimdi `HareketliPlatform` sınıfında delta transferini yorum satırı yap: oyuncu platform kaydığında hemen geride kalacak ve düşecek. Delta transferinin ne kadar kritik olduğunu böyle fark edeceksin.

42.4 Dağılan ve Zıplatan Platformlar

Standart ve tek yönlü platformların ötesinde, oyuncuya özel meydan okumalar sunan iki popüler tür daha var: *crumbling* (dağılan) platform ayak basılınca bir süre sonra kaybolur; *bouncy* (zıplatan) platform oyuncuya ani bir yüksek zıplama aşılır.

42.4.1 Crumbling Platform: Zamanlı Kaybolma

Crumbling platform oyuncuyu hızlı karar vermeye zorlar: “Üzerinde kalamayacağımı biliyorum, bir an önce atlamalıyım.” Tasarım olarak enerji katar.

Üç durum ile bir state machine gibi yönetiriz:

1. **SAGLAM**: Normal, sabit platform.
2. **SARSILİYOR**: Basıldı, 30 kare (yarım saniye) titreşiyor.
3. **YOK**: Kayboldu, çarpışma yok. Belli bir süre sonra tekrar **SAGLAM** olur.

Python – CrumblingPlatform Sınıfı

```

1  SARSILMA_SURE = 30    # kare
2  YOK_SURE = 120       # kare (geri gelme süresi)
3
4  class CrumblingPlatform:
5      def __init__(self, x, y, w, h):
6          self.rect = pygame.Rect(x, y, w, h)
7          self.durum = "saglam"

```

```

8         self.sayac = 0
9         self.oriğin = (x, y)
10
11     def tetikle(self):
12         """Oyuncu basınca çağrılır."""
13         if self.durum == "sağlam":
14             self.durum = "sarsiliyor"
15             self.sayac = SARSILMA_SURE
16
17     def guncelle(self):
18         if self.durum == "sarsiliyor":
19             self.sayac -= 1
20             # Hafif titreşim (±2 piksel)
21             self.rect.x = self.oriğin[0] +
22                 ↳ pygame.math.Vector2(
23                     1, 0).rotate(self.sayac * 30).x * 2
24             if self.sayac <= 0:
25                 self.durum = "yok"
26                 self.sayac = YOK_SURE
27                 self.rect.x = self.oriğin[0] # titreşimi
28                 ↳ sıfırla
29             elif self.durum == "yok":
30                 self.sayac -= 1
31                 if self.sayac <= 0:
32                     self.durum = "sağlam"
33
34     @property
35     def cikarir(self):
36         """Çarpışmaya katılıyor mu?"""
37         return self.durum in ("sağlam", "sarsiliyor")

```

Çizimde durum göstergesi:

Python – CrumblingPlatform Çizimi

```

1 for cp in crumbling_list:
2     if cp.durum == "yok":
3         continue # görünmez
4     c = kamera.apply(cp.rect)
5     renk = (180, 140, 80) if cp.durum == "sağlam" else
6         ↳ (220, 100, 60)
7     pygame.draw.rect(ekran, renk, c)
8     pygame.draw.rect(ekran, (80, 60, 40), c, 2)

```

Oyuncu `temas["alt"] = True` iken bastığı crumbling platformu bulup `tetikle()` çağırır:

Python – Basınca Tetikleme

```

1 # Dikey çarpışma sonrası
2 if temas["alt"]:

```

```

3   for cp in crumbling_list:
4       if cp.cikarir and self.rect.colliderect(cp.rect):
5           cp.tetikle()

```

42.4.2 Bouncy Platform: Yaylı Zıplatma

Bouncy platform üstüne indiğinde oyuncuya normalden daha yüksek bir zıplama hızı verir — tuşa basmadan! Bu sayede oyuncu normalde erişemeyeceği yüksekliklere çıkar.

Python – BouncyPlatform Sınıfı

```

1  ZIPLAMA_GUCU = -18 # normal -11, buradan fırlarken çok
   ↳ daha fazla
2
3  class BouncyPlatform:
4      def __init__(self, x, y, w, h):
5          self.rect = pygame.Rect(x, y, w, h)
6          self.anim_sayac = 0 # sıkışma animasyonu için
7
8      def tetikle(self):
9          self.anim_sayac = 12 # sıkışma efekti 12 kare
10
11     def guncelle(self):
12         if self.anim_sayac > 0:
13             self.anim_sayac -= 1

```

Oyuncu üzerine düştüğünde normal “yerde dur” davranışı yerine “yukarı fırla” yaparız:

Python – Bouncy Çarpışma

```

1  for bp in bouncy_list:
2      if self.rect.colliderect(bp.rect):
3          if self.vy > 0 and eski_alt <= bp.rect.top + 1:
4              self.rect.bottom = bp.rect.top
5              self.vy = ZIPLAMA_GUCU # yaylı zıplama!
6              self.kalan_ziplama = MAX_ZIPLAMA # hakları
   ↳ yenile
7              bp.tetikle()

```

`kalan_ziplama` da yenilenir — artık oyuncu bouncy platformdan ileri atlayarak ek zıplama haklarını kullanabilir.

Çizimde sıkışma efekti için yüksekliği kısaltırız:

Python – BouncyPlatform Çizimi

```

1  for bp in bouncy_list:
2      h_azalma = bp.anim_sayac * 2 # sıkışınca kısalır
3      c = kamera.apply(bp.rect)
4      c.y += h_azalma

```



```

5   c.height -= h_azalma
6   pygame.draw.rect(ekran, (80, 220, 120), c)
7   pygame.draw.rect(ekran, (40, 120, 60), c, 2)
8   # Yay görünümü: üst kenarda zigzag
9   for i in range(4):
10      x0 = c.x + i * 10 + 4
11      pygame.draw.line(ekran, (220, 255, 220),
12                      (x0, c.y), (x0 + 4, c.y + 4))

```

42.4.3 Platform Türlerini Tek Listede Yönetme

Seviye büyüdükçe farklı platform türlerinin tek bir yerde düzenlenmesi faydalıdır. Bunun için her platform türünü ayrı tile ID'sine atarız:

Tablo 42.2: Bölüm 11 tile ID şeması (genişletilmiş)

ID	Tür	Ünite
1	Boş (arkaplan)	Bölüm 10
2	Solid taş	Bölüm 10
4	Altın (toplanabilir)	Bölüm 10
5	Diken (tehlike)	Bölüm 10
6	Tek yönlü platform	Ünite 2
7	Hareketli platform	Ünite 2
8	Crumbling platform	Ünite 2
9	Bouncy platform	Ünite 2

Python – Genişletilmiş Seviye Grubu

```

1   def _grupla(self):
2       for r, satir in enumerate(self.harita):
3           for c, t in enumerate(satir):
4               rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
5               if t == 2:
6                   self.katilar.append(rect)
7               elif t == 4:
8                   self.altinlar.append(rect)
9               elif t == 5:
10                  self.tehlikeler.append(rect)
11              elif t == 6:
12                  self.tek_yonlular.append(rect)
13              elif t == 7:
14                  self.hareketliler.append(
15                      HareketliPlatform(c*TILE, r*TILE,
16                                          TILE*3, TILE,
17                                          c*TILE + TILE*6,
18                                          r*TILE))
19              elif t == 8:
20                  self.crumblings.append(

```

```

19         CrumblingPlatform(c*TILE, r*TILE,
20                             ↳ TILE, TILE))
21     elif t == 9:
22         self.bouncylar.append(
            BouncyPlatform(c*TILE, r*TILE, TILE,
                           ↳ TILE))

```

⚠ Dikkat: Hareketli Platformun Hedefi

`HareketliPlatform` ID 7 ile çağrıldığında hedef konumu CSV’de otomatik belirlenemiyor; burada örnek olarak “sağa 6 tile kay” dedik. Gerçek oyunda farklı ID’ler kullanıp farklı uzunluklar tanımlayabilir veya `Tiled obj` katmanında özel obje olarak tutabilirsin.

💡 İpucu: Özel Platformların Bir Arada Kullanımı

Bu dört platform türü birlikte kullanıldığında çok yönlü bulmaca senaryoları ortaya çıkar: “Crumbling platformdan zıpla, hareketli platforma bin, oradan bouncy’e düş, uçarken tek yönlü platforma konum al.” Bu zinciri seviye 3 veya 4’e sakla — ustalaşmış oyuncu için ödül.

🕒 Bilgi: Pratik Alıştırma

`04_crumbling_bouncy.py` dosyasını çalıştır. Bir crumbling platforma atla ve 30 karede (yarım saniye) dağıldığını gör; üzerinde oyalanırsan düşersin. Sonra yeşil yaylı platforma in — normal zıplamadan çok daha yüksek çıkacaksın. `ZIPLAMA_GUCU` değerini -25 yap: artık ekranın üstünden çıkar!

42.5 Sonuç

Bu ünite statik, tek tip platformlardan çıkıp *farklı karakterde* platformlara geçtik. Çarpışma yönünü tespit edip bu bilgiyi kullanmak; tek yönlü platformlar ile dikey hareket alanlarını genişletmek; hareketli platformlarla oyuncuyu taşımak; crumbling ve bouncy ile oynanışa ritim ve sürpriz eklemek artık seni bir tool kit’iyle donattı. Bu dört platform türünü birleştirerek çok daha yaratıcı seviyeler inşa edebilirsin.

42.5.1 Anahtar Kavramlar

Tablo 42.3: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
Çarpışma yönü	Temas “üst, alt, sol, sağ” yönlerinden hangisinden geldi?
Temas sözlüğü	<code>{"alt": True, ...}</code> yapısıyla yönleri bildirme
Tek yönlü platform	Sadece üstten çarpışan platform (one-way)
Drop-through	Aşağı + SPACE ile tek yönlü platformun altına geçme
Hareketli platform	Kendi hızıyla ileri-geri veya yukarı-aşağı hareket eden platform
Delta transferi	Platformun kare deltasını üstündeki oyuncuya aktarma
Crumbling	Basılınca zamanla kaybolan platform (FSM ile)
Bouncy	Üstüne düşeni yukarı fırlatan yaylı platform

42.5.2 Önemli Komutlar

Tablo 42.4: Ünite 2 – Temel komutlar

Komut	Açıklama
<code>eski_alt <= p.top + 1</code>	Oyuncu platformun üstünde miydi kontrolü
<code>if self.vy > 0</code>	Düşüş yönünü belirleme
<code>self.delta = yeni - eski</code>	Platformun kare deltasını hesaplama
<code>rect.x += int(delta.x)</code>	Delta’yı oyuncu konumuna aktarma
<code>durum == "sağlam"</code>	State machine durum kontrolü
<code>istikamet.normalize() * hiz</code>	Yön vektörünü normalize edip hızla çarpma
<code>self.vy = ZIPLAMA_GUCU</code>	Bouncy’de ani yüksek zıplama

42.5.3 Bir Sonraki Ünite: Düşman Davranışları

Dünya artık hareket eder; ama henüz bir *tehdit* içermiyor. Bölüm 10’daki dikenler pasif: onlara çarparsan zarar alırsın, ama onlar gelip seni kovalamazlar. Ünite 3’te bunu değiştireceğiz:

- `Dusman(Sprite)` temel sınıfı ve sonlu durum makineleri
- Patrol AI: iki nokta arasında ileri-geri gezinen düşman
- Chase AI: belirli mesafeye girince oyuncuyu takip eden düşman
- Ranged AI: mermi fırlatan uzak mesafe düşmanı
- Stomping: üstten zıplayıp düşmanı ezme

Platformer’ı gerçek bir oyuna dönüştüren şey, zeki tehditlerdir.

💡 İpucu: Seviye Tasarımında Platform Zinciri

Ünite 2’de öğrendiğin dört platform türünü bir senaryo halinde birleştirebilirsin: “Zıplatan yaylı platformla yükseğe fırla, crumbling platformdan hızla geç, tek yönlü platforma in, hareketli platformla üstü örtülü sonraki bölüme taşın.” Bu tip zincirler seviye tasarımının estetik tepesidir.

Düşman Davranışları

“Bir oyundaki düşmanlar oyuncuya ne yapabileceğini, ne yapamayacağını ve neyi deneyebileceğini öğretir.”

— JONATHAN BLOW, BRAID

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- **Dusman(Sprite)** temel sınıfı ve sonlu durum makinesi (FSM) deseni
- Patrol AI: iki nokta arasında ileri-geri gezinen düşman ve uçurum kenarı tespiti
- Chase AI: oyuncuyu belirli bir mesafeye girince takip etmeye başlayan düşman
- Ranged AI: mermi fırlatan düşman ve basit **Mermi(Sprite)** sınıfı
- Stomping: oyuncunun üstten düşmanın üzerine zıplayarak onu yok etmesi

43.1 Düşman Temel Sınıfı ve FSM Deseni

Platform dünyamız hareket etti ama hâlâ pasif: dikenler yerinde duruyor, oyuncu onlara çarpmadıkça tehlike yok. Şimdi *aktif* bir tehdit ekleyelim: düşmanlar. Bu konuda tüm düşman çeşitleri için ortak bir üst sınıf ve onların davranışını yöneten bir *sonlu durum makinesi* (FSM, Finite State Machine) deseni kuracağız.

■ Tanım: Finite State Machine (FSM)

Bir nesnenin belirli “durumlar”dan (state) birinde olduğu ve olaylara göre bir durumdan diğerine *geçtiği* (transition) bir modeldir. Örneğin bir düşman “gezinti, takip, saldırı, öldü” durumlarının birindedir. **idle**, **patrol**, **chase**, **attack** gibi ayrı dallar halinde programlanması çok okunaklıdır.

43.1.1 Neden Sprite Altında?

Bölüm 6'dan beri `pygame.sprite.Sprite` sınıfı tanıdık. Onun sunduğu otomatik grüplama, çizim ve `kill` mekanizmaları düşmanlar için harika. Tüm düşman türleri `Sprite`'tan türer.

Python – Dusman Temel Sınıfı

```

1  class Dusman(pygame.sprite.Sprite):
2      """Tüm düşmanların ortak atası."""
3
4      def __init__(self, x, y, w, h):
5          super().__init__()
6          self.image = pygame.Surface((w, h))
7          self.image.fill((200, 60, 60))
8          self.rect = self.image.get_rect(topleft=(x, y))
9          self.vx = 0
10         self.vy = 0
11         self.can = 1
12         self.durum = "idle" # FSM başlangıç durumu
13         self.öldürülebilir = True # stomping ile
14         ↪ öldürülebilir mi?
15
16     def guncelle(self, oyuncu, katilar):
17         """Alt sınıflar override eder."""
18         self._fizik(katilar)
19
20     def _fizik(self, katilar):
21         """Yerçekimi + yatay ve dikey çarpışma."""
22         # Yatay
23         self.rect.x += int(self.vx)
24         for k in katilar:
25             if self.rect.colliderect(k):
26                 if self.vx > 0:
27                     self.rect.right = k.left
28                     self.vx = 0
29                 elif self.vx < 0:
30                     self.rect.left = k.right
31                     self.vx = 0
32         # Yerçekimi
33         self.vy = min(self.vy + 0.5, 10)
34         self.rect.y += int(self.vy)
35         for k in katilar:
36             if self.rect.colliderect(k):
37                 if self.vy > 0:
38                     self.rect.bottom = k.top
39                     self.vy = 0
40                 elif self.vy < 0:
41                     self.rect.top = k.bottom
42                     self.vy = 0
43
44     def hasar_al(self, miktar=1):

```

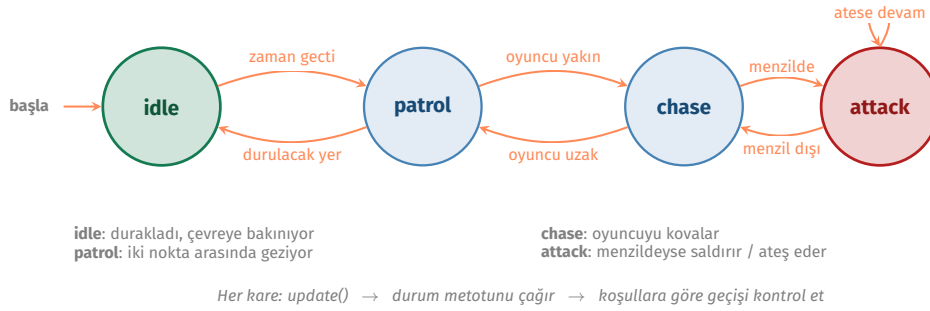
```

44 self.can -= miktar
45 if self.can <= 0:
46     self.kill()

```

Her düşman alt sınıfı bu temeli kullanacak; sadece kendi **guncelle** metodunu özelleştirecek ve FSM geçişlerini yönetecek.

Düşman Sonlu Durum Makinesi (FSM)



Şekil 43.1: Düşmanların genelleşmiş FSM diyagramı: Idle → Patrol → Chase → Attack geçişleri.

43.1.2 FSM İskeletini Kurma

Bir FSM'yi Python'da en basit şekilde bir **if/elif** zinciriyle veya durumu metoda eşleyen bir sözlükle yazabilirsin. İkincisi daha zarif:

Python – FSM Durum Tablosu

```

1 class Dusman(pygame.sprite.Sprite):
2     def guncelle(self, oyuncu, katilar):
3         # Durum tablosuna göre ilgili metodu çağır
4         durum_metotlari = {
5             "idle": self._idle,
6             "patrol": self._patrol,
7             "chase": self._chase,
8             "attack": self._attack,
9         }
10        metot = durum_metotlari.get(self.durum)
11        if metot:
12            metot(oyuncu)
13        self._fizik(katilar)
14
15    def _idle(self, oyuncu):
16        pass # alt sınıflar doldurur
17
18    def _patrol(self, oyuncu):
19        pass
20
21    def _chase(self, oyuncu):

```

```

22         pass
23
24     def _attack(self, oyuncu):
25         pass
26
27     def _durum_degistir(self, yeni):
28         self.durum = yeni

```

`_durum_degistir()` metoduna “duruma giriş” olayları da ekleyebilirsiniz (örneğin `chase`’e girerken hızlanmak).

43.1.3 Stomping ile Düşman Öldürme

Platform oyunlarının klasiği: oyuncu düşmanın üstüne zıplayarak iner ve onu ezer. Bu mekaniği uygulamak için çarpışma yönünü (Ünite 2) kullanırız.

Python – Stomping Mantığı

```

1  def oyuncu_dusman_carpismasi(oyuncu, dusmanlar):
2      for d in dusmanlar:
3          if oyuncu.rect.colliderect(d.rect):
4              # Oyuncu üstten mi iniyor?
5              if oyuncu.vy > 0 and oyuncu.rect.bottom <
6                  ↪ d.rect.top + 16:
7                  # STOMP: düşman öldü, oyuncu zıplar
8                  if d.oldurulebilir:
9                      d.hasar_al()
10                     oyuncu.vy = -9 # küçük zıplama
11             else:
12                 # Yandan temas: oyuncu hasar alır
13                 oyuncu.hasar_al()

```

`oyuncu.rect.bottom < d.rect.top + 16` kontrolü şunu der: “oyuncunun alt kenarı düşmanın üst kenarından en fazla 16 piksel aşağıda ise üstten ezdin”. Bu tolerans kenar temaslarını “üstten” saymayı kolaylaştırır.

43.1.4 Görsel Temsil

Şimdilik düşmanları basit renkli dikdörtgenler olarak gösteriyoruz. Bölüm 8’de animasyonu zaten öğrendin; her düşman türü için farklı animasyon sözlüğü tanımlanabilir:

Python – Durum Bazlı Animasyon

```

1  class Dusman(pygame.sprite.Sprite):
2      def __init__(self, ...):
3          # ...
4          self.animasyonlar = {
5              "idle": [frame1_sur, frame2_sur],
6              "patrol": [walk1, walk2, walk3, walk4],

```



```

7         "chase": [run1, run2, run3, run4],
8     }
9     self.frame_sayac = 0
10
11     def _animasyonu_ilerlet(self):
12         frames = self.animasyonlar.get(self.durum,
13             ↳ [self.image])
14         self.frame_sayac = (self.frame_sayac + 1) %
15             ↳ (len(frames) * 8)
16         self.image = frames[self.frame_sayac // 8]

```

⚠ Dikkat: Sprite Grubu vs Liste

`pygame.sprite.Group` kullandığında `group.update()` tüm üyeleri otomatik çağırır; ama `update` imzası “ek parametre yok” varsayar. Ek parametreler (oyuncu, katılar) iletmek için `group.update(oyuncu, katilar)` şeklinde çağırılmalı — bizim `guncelle` zaten bunu alıyor.

💡 İpucu: State Enum Yerine String

Büyük projelerde `Enum` ile durumları tip güvenli yapabilirsin (`DusmanDurum.PATROL`). Ancak bu kitap için string ("`patrol`") yeterli ve okunaklıdır.

● Bilgi: Pratik Alıştırma

`01_dusman_temel.py` dosyasını çalıştır. Ekranda kırmızı bir kutu (düşman) duruyor. Üstüne zıpla: öldü. Yandan dokun: oyuncu canı azaldı. Düşmanı FSM ile karmaşıklaştırmadan önce bu temelin sağlam olduğunu test et.

43.2 Patrol AI ve Uçurum Tespiti

En basit düşman davranışı “gezinme”dir: düşman bir noktadan diğerine yürür, oraya varınca geri döner; oyuncuya dokunursa zarar verir. Bu davranış tek başına çok görsel olarak etkileyici olmasa da oyunculara düşman davranışlarını *tahmin etme* fırsatı verir — iyi platformer tasarımının temelidir.

■ Tanım: Patrol AI

İki sınır noktası (sol-sağ) veya bir zamanlayıcıyla tanımlanmış bir mesafe boyunca ileri-geri yürüyen düşman davranışı. Tahmin edilebilir hareket oyuncuya “okunabilir” bir tehdit sunar.

43.2.1 Temel Patrol Sınıfı

Patrol düşmanımız iki sınır (**sol_x**, **sag_x**) arasında gidip gelir. Sol sınıra varınca sağa döner, sağ sınıra varınca sola.

Python – Patrol Düşmanı

```

1 class PatrolDusman(Dusman):
2     def __init__(self, x, y, sol_x, sag_x, hiz=1.5):
3         super().__init__(x, y, 28, 28)
4         self.image.fill((220, 60, 60))
5         self.sol_x = sol_x
6         self.sag_x = sag_x
7         self.hiz = hiz
8         self.vx = hiz      # başlangıçta sağa yürür
9         self.durum = "patrol"
10
11     def _patrol(self, oyuncu):
12         # Sınırlara geldiysek dön
13         if self.rect.left <= self.sol_x:
14             self.vx = self.hiz
15         elif self.rect.right >= self.sag_x:
16             self.vx = -self.hiz

```

43.2.2 Uçurum Tespiti (Ledge Check)

Düşman eğer bir platformun kenarına geldiğinde dönmezse aşağı düşer ve oyunu dolandırır. Bunu önlemek için her karede “bir sonraki adımda ayağımın altı boş mu?” diye sorarız.

Yöntem basittir: düşmanın hareket yönünde, ayağının altında küçük bir test **Rect** oluşturup katılarla çarpıştırırız.

Python – Uçurum Tespiti

```

1 def _ucurum_var_mi(self, katilar):
2     """Hareket yönünde bir sonraki tile'da solid var
3     mı?"""
4     yon = 1 if self.vx > 0 else -1
5     test_x = self.rect.right + 2 if yon > 0 else
6         self.rect.left - 6
7     test = pygame.Rect(test_x, self.rect.bottom + 2, 4, 6)
8     for k in katilar:
9         if test.colliderect(k):
10             return False    # ayak altı dolu, devam
11     return True             # ayak altı boş, uçurum

```

Patrol fonksiyonuna bu kontrolü ekleriz:

Python – Uçurum + Sınır Kontrol

```

1 def _patrol(self, oyuncu, katilar):
2     if self._ucurum_var_mi(katilar):
3         self.vx *= -1 # uçurum görünce dön
4     elif self.rect.left <= self.sol_x:
5         self.vx = self.hiz
6     elif self.rect.right >= self.sag_x:
7         self.vx = -self.hiz

```

Uçurum tespiti için her karede ayak altı küçük bir test dikdörtgeniyle kontrol edilir. Test dikdörtgeni solid tile ile kesişmiyorsa “uçurum” algılanır ve yön tersine çevrilir.

Artık düşman platformun dışına düşmüyor.

43.2.3 Dönüşte Duraksama

Bazı oyunlar düşmanı dönmeden önce kısa bir süre durdurur. Bu ufak detay oyuncuya zaman ölçmede yardımcı olur ve düşmanı daha “canlı” hissettirir.

Python – Duraksamalı Patrol

```

1 DURAKSAMA = 20 # kare
2
3 class PatrolDusman(Dusman):
4     def __init__(self, ...):
5         super().__init__(...)
6         self.duraklama_sayac = 0
7
8     def _patrol(self, oyuncu, katilar):
9         if self.duraklama_sayac > 0:
10             self.duraklama_sayac -= 1
11             self.vx = 0 # dur
12             return
13         if self._ucurum_var_mi(katilar) or \
14            self.rect.left <= self.sol_x or \
15            self.rect.right >= self.sag_x:
16             self.vx *= -1 if self.vx != 0 else 1
17             self.duraklama_sayac = DURAKSAMA

```

43.2.4 Görsel Yön

Patrol düşmanın hangi yöne baktığını göstermek için sprite'ını hareket yönüne göre flipleriz. Bölüm 8'deki `pygame.transform.flip` yöntemini hatırla.

Python – Görsel Yön Takibi

```

1 def _animasyonu_ilerlet(self):
2     # Baz resim

```

```

3 frames = self.animasyonlar["patrol"]
4 frame = frames[self.frame_sayac // 8 % len(frames)]
5 # Sağa bakıyorsa normal, sola bakıyorsa flip
6 if self.vx < 0:
7     frame = pygame.transform.flip(frame, True, False)
8 self.image = frame

```

43.2.5 Seviye Tasarımı İçin Düşünceler

Patrol düşmanlar iyi yerleştirilince oyuna ritim katar. Genel kurallar:

- **Kısa patrol mesafeleri** baskı yaratır (“sıkış-zıpla” hissi).
- **Uzun patrol mesafeleri** zamanlama bulmacası oluşturur.
- Altını bir patrol’un arkasına koy; oyuncunun yüreği hoplatsın.
- Farklı hızlarda iki patrol aynı anda = senkron bulmacası.

⚠ Dikkat: Uçurum Testi Çok Dar Olmasın

Eğer test Rect’ini 2×2 gibi çok küçük yaparsan tile kenarlarında yanlış okuma olabilir. 4×6 civarı genellikle güvenli sonuç verir. Çok geniş yaparsan düşman küçük boşluklara girmeyi reddeder.

💡 İpucu: Patrol Sınırlarını Kodda Belirtme

CSV tile ID 3 ile “düşman spawn + patrol sınırı” işaretlemesi yapabilirsin: solda ve sağda iki özel işaretli tile arasında düşman gezinir. Bu kitapta basitlik için kodda elle belirtiyoruz, ama Tiled editoründe obje katmanına “waypoint” koymak da yaygın bir yaklaşım.

● Bilgi: Pratik Alıştırma

`02_patrol_ai.py` dosyasını çalıştır. Platformlarda ileri-geri gezen üç kırmızı düşman göreceksin. Birinin üzerine zıpla: ezildi. Yandan dokun: oyuncu hasar aldı. `_ucurum_var_mi` fonksiyonunu `return False` yapıp devre dışı bırak: düşmanlar uçurumdan düşerek kaybolacak. İşte bu yüzden uçurum tespiti var.

43.3 Chase AI: Oyuncuyu Takip Eden Düşman

Patrol düşmanları tahmin edilebilirdir; oyuncunun gözü onları okuduktan sonra kolayca geçer. İşte bu yüzden platformer bestiaryları *aktif* düşmanlar da içerir: belirli bir mesafeye girdiğinde oyuncuyu takip etmeye başlayan *chase* düşmanları. Bunlar oyuncuya sürekli “hareket et, dur durma” baskısı yapar.

■ Tanım: Chase AI

Bir oyuncuyu “fark etme” (detection) mesafesi tanımlanmış olan, bu mesafeye girince takip etmeye başlayan, takip kaybolunca geri dönen düşman davranışdır. Genellikle patrol ile birleşir: patrol → chase → kaybedince patrol’a geri dön.

43.3.1 Fark Etme Mesafesi ve Çember Testi

Chase düşmanının gözleri vardır: çevresinde belirli bir yarıçapta bir çember oluşturup oyuncu bu çember içine girdiğinde takip modunu açar.

Python – Basit Mesafe Testi

```
1 FARK_MESAFE = 200 # piksel
2
3 def _oyuncu_yakin_mi(self, oyuncu):
4     merkez = pygame.Vector2(self.rect.center)
5     oyuncu_merkez = pygame.Vector2(oyuncu.rect.center)
6     return merkez.distance_to(oyuncu_merkez) <=
       ↪ FARK_MESAFE
```

`distance_to` iki vektör arasında Öklid mesafesi verir; bu sayede düşman yatay ve dikey olarak “çevresinde” kontrol yapar.

43.3.2 FSM ile Durum Geçişleri

Chase düşmanı en az üç durumda yaşar: patrol, chase, geri-dönüş. Her karede durumu güncelleriz:

Python – ChaseDusman FSM

```
1 class ChaseDusman(Dusman):
2     def __init__(self, x, y, sol_x, sag_x, hiz=1.5,
       ↪ chase_hiz=3.0):
3         super().__init__(x, y, 28, 28)
4         self.image.fill((200, 100, 60))
5         self.sol_x, self.sag_x = sol_x, sag_x
6         self.hiz = hiz
7         self.chase_hiz = chase_hiz
8         self.vx = hiz
9         self.durum = "patrol"
10
11     def guncelle(self, oyuncu, katilar):
12         if self.durum == "patrol":
13             self._patrol(oyuncu, katilar)
14             if self._oyuncu_yakin_mi(oyuncu):
15                 self.durum = "chase"
16         elif self.durum == "chase":
17             self._chase(oyuncu)
```

```

18         if not self._oyuncu_yakin_mi(oyuncu):
19             self.durum = "patrol"
20             self.vx = self.hiz
21         self._fizik(katilar)
22
23     def _chase(self, oyuncu):
24         if oyuncu.rect.centerx < self.rect.centerx:
25             self.vx = -self.chase_hiz
26         else:
27             self.vx = self.chase_hiz
28
29     def _patrol(self, oyuncu, katilar):
30         if self.rect.left <= self.sol_x:
31             self.vx = self.hiz
32         elif self.rect.right >= self.sag_x:
33             self.vx = -self.hiz

```

Chase durumunda düşman oyuncunun `centerx`'ine göre yön seçer; patrol durumunda eski iki-sınır arası gidip gelmeye döner.

`patrol` durumunda düşman standart rotasında gezinir. Oyuncu `FARK_MESAFE` içine girince `chase`'e geçer ve hızı 2x olur. Oyuncu çemberden çıkınca `patrol`'a geri döner.

43.3.3 Duvarlara Karşı Chase Tuzağı

Chase düşmanı oyuncunun ardından giderken bir duvara çarpabilir ve oraya sıkışabilir. Aynı `_fizik` yatay çarpışma rutini hızı sıfırlar; eğer durumu `chase`'te kalmaya devam ederse düşman “duvara yapışıp bekler”. Bunu çözmenin iki yolu var:

1. Duvara çarpınca `attack`'a geç (yerine saldır, eğer saldırı hareketi varsa).
2. Bir zaman sayacıyla “belirli süre sıkışık kaldım, patrol'a dön”.

Python – Chase'ten Çıkış

```

1 CHASE_SIKISMA_SURE = 60 # 1 saniye
2
3 class ChaseDusman(Dusman):
4     def __init__(self, ...):
5         super().__init__(...)
6         self.sikisma_sayac = 0
7
8     def _chase(self, oyuncu):
9         eski_x = self.rect.x
10        if oyuncu.rect.centerx < self.rect.centerx:
11            self.vx = -self.chase_hiz
12        else:
13            self.vx = self.chase_hiz
14        # Önceki konumdan farksız mı?
15        if self.rect.x == eski_x:

```

```

16         self.sikisma_sayac += 1
17         if self.sikisma_sayac > CHASE_SIKISMA_SURE:
18             self.durum = "patrol"
19             self.sikisma_sayac = 0
20     else:
21         self.sikisma_sayac = 0

```

43.3.4 Dikey Chase ve Zıplama

Bazı chase düşmanları oyuncuyu yukarıdayken yakalamak için zıplarlar. Bunu eklemek için düşmanın yere basması gerekir (fizikten gelen *yerde* bilgisi) ve oyuncunun *y*'si düşmandan yukarıdaysa zıplama tetikle:

Python – Zıplayan Chase

```

1 def _chase(self, oyuncu):
2     # ... yatay kısım ...
3     if self.yerde and oyuncu.rect.bottom < self.rect.top -
4         ↪ 20:
5         self.vy = -9 # üsten oyuncuyu yakalamak için
6         ↪ zıpla

```

Bu ekleme düşmanı çok daha “tehlikeli” hissettirir; oyuncu artık “yüksek platformdayım, güvendeyim” diyemez.

43.3.5 Görsel İpucu: Ünlem

Chase moduna geçildiğinde oyuncuya fark etmesi için bir görsel ipucu göstermek sinema-oyun klasiği bir uygulamadır:

Python – Ünlem Efekt

```

1 def ciz(self, ekran, kamera):
2     ekran.blit(self.image, kamera.apply(self.rect))
3     if self.durum == "chase":
4         # Kafası üstünde kırmızı bir "!"
5         font = pygame.font.Font(None, 24)
6         metin = font.render("!", True, (255, 60, 60))
7         ekran.blit(metin, (self.rect.centerx - 4,
8                             self.rect.top - 20))

```

⚠ Dikkat: Çok Hızlı Chase Adaletsiz

chase_hiz oyuncunun *MAX_HIZ*'ini geçerse oyuncu artık kaçamaz. Bu bazı oyunlarda “panic” hissi istense de çoğu zaman adaletsizliktir. Düşman hızını oyuncunun %80–%90'ı kadar tut.

💡 İpucu: Görüş Hattı (Line of Sight)

İleri bir teknik olarak chase moduna geçmek için sadece mesafe değil, aynı zamanda “aramızda solid bir tile var mı?” de kontrol edebilirsin. Eğer duvar varsa düşman oyuncuyu görmez – “kör”dür. Bu da seviye tasarımı için fazladan bir araç olur (“düşmana görünmeden yaklaş”).

● Bilgi: Pratik Alıştırma

`03_chase_ai.py` dosyasını çalıştır. Turuncu bir düşman platformda gezinirken ona 200 piksel içinde yaklaşmayı dene: aniden üstüne doğru koşmaya başlayacak. Geri kaç: tekrar patrol’a dönecek. Şimdi `FARK_MESAFE`’yi 400 yap ve denemeyi tekrarla; düşman neredeyse haritanın her yerinden seni görür.

43.4 Ranged AI: Mermi Fırlatan Düşman

Patrol düşmanları yerde dolaşır; chase düşmanları oyuncuyu kovalar. Şimdi üçüncü bir arketip ekleyelim: *ranged* (uzak mesafe) düşmanı. Bu düşman yerinden kıpırdamaz ama belli aralıklarla oyuncunun yönüne mermi atar. Platform oyunlarında “ölümcül göz”, “ok atan iskelet” veya “bomba atan domuz” bu arketipin örnekleridir.

■ Tanım: Ranged AI

Sabit veya hafifçe hareketli, periyodik olarak mermi (projectile) fırlatan düşman. Mermi bir `Sprite`’tir; kendi hareket vektörü ve ömrü vardır. Oyuncu mermiye dokunursa hasar alır.

43.4.1 Mermi Sınıfı

Önce mermiyi yapalım. Kendi `vx`, `vy` hızı ve `omur` değeri olan bir sprite:

Python – Mermi Sınıfı

```
1 class Mermi(pygame.sprite.Sprite):
2     def __init__(self, x, y, vx, vy, omur=120):
3         super().__init__()
4         self.image = pygame.Surface((8, 8))
5         self.image.fill((255, 240, 80))
6         self.rect = self.image.get_rect(center=(x, y))
7         self.vx = vx
8         self.vy = vy
9         self.omur = omur
10
11     def update(self, katilar):
12         self.rect.x += int(self.vx)
```



```

13     self.rect.y += int(self.vy)
14     # Duvara çarpınca yok ol
15     for k in katilar:
16         if self.rect.colliderect(k):
17             self.kill()
18             return
19     # Ömür
20     self.omur -= 1
21     if self.omur <= 0:
22         self.kill()

```

Mermiler `pygame.sprite.Group` içinde saklanır ve ana döngüde `mermiler.update(katilar)` ile güncellenir.

43.4.2 RangedDusman Sınıfı

Şimdi asıl düşmanı yazalım. Her N karede bir mermi ateşler, oyuncuya doğru yönelir.

Python – Ranged Düşmanı

```

1  ATES_ARALIK = 90      # kare (1.5 saniye 60 FPS'de)
2  MERMI_HIZ = 4
3
4  class RangedDusman(Dusman):
5      def __init__(self, x, y, mermi_grubu):
6          super().__init__(x, y, 32, 32)
7          self.image.fill((140, 60, 160))
8          self.ates_sayac = 0
9          self.mermi_grubu = mermi_grubu      # kendi
          ↪ mermilerini ekleyecek
10         self.durum = "ates"
11
12     def guncelle(self, oyuncu, katilar):
13         self.ates_sayac += 1
14         if self.ates_sayac >= ATES_ARALIK:
15             self._ates_et(oyuncu)
16             self.ates_sayac = 0
17         self._fizik(katilar) # yerçekimi + zeminde dur
18
19     def _ates_et(self, oyuncu):
20         """Oyuncuya doğru bir mermi fırlat."""
21         baslangic = pygame.Vector2(self.rect.center)
22         hedef = pygame.Vector2(oyuncu.rect.center)
23         yon = (hedef - baslangic).normalize()
24         mermi = Mermi(baslangic.x, baslangic.y,
25                       yon.x * MERMI_HIZ, yon.y *
26                       ↪ MERMI_HIZ)
27         self.mermi_grubu.add(mermi)

```

Düşman, ateş etme kararını aldığı anda oyuncunun konumuna göre mermiyi

yönlendirir. `normalize()` yön vektörünü uzunluk 1'e getirir, sonra `MERMI_HIZ` ile çarparak istediğimiz hızı elde ederiz.

43.4.3 Mermi-Oyuncu Çarpışması

Ana döngüde her karede mermileri oyuncuyla kontrol ederiz. PyGame'in `spritecollide` metodu bu iş için biçilmiş kaftan:

Python – Mermi Çarpışması

```
1 # Mermi ile oyuncu
2 for m in pygame.sprite.spritecollide(oyuncu, mermiler,
   ↳ True):
3     # True = çarpışan mermileri mermiler grubundan sil
4     oyuncu.hasar_al()
```

`dokill=True` parametresi çarpışan mermileri otomatik öldürür (grup'tan çıkarır). Oyuncu sadece bir hasar alır.

43.4.4 Ateş Sırasında Görsel İpuçları

Ateş etmeden önce oyuncuya bir “hazırlık” ipucu vermek çok önemlidir. Mermi anlık fırlarsa oyuncu reaksiyon gösteremez. İki yaygın seçenek:

1. **Renk değiştirme:** Ateşten 15 kare önce düşman parlansın (`self.image.fill((255, 255, 255))`).
2. **Şarj çizgisi:** Düşmandan oyuncuya doğru ince bir çizgi çiz (“nişan alıyor”).

Python – Nişan Alma Görsel İpucu

```
1 def ciz_nisan(self, ekran, kamera, oyuncu):
2     # Son 15 karede nişan alma çizgisi
3     if ATES_ARALIK - self.ates_sayac <= 15:
4         start = kamera.apply_pos(self.rect.center)
5         end = kamera.apply_pos(oyuncu.rect.center)
6         pygame.draw.line(ekran, (255, 100, 100, 120),
   ↳ start, end, 1)
```

Ranged düşman ateşten 15 kare önce oyuncuya ince bir kırmızı çizgi gönderir. Oyuncu bu çizgiyi görüp kaçabilir; çizgi bittiğinde mermi fırlar.

43.4.5 Mermi Türleri

Tek tür mermi yeter mi? Çoğu zaman hayır. Farklı mermi türleri düşmanı daha ilginç yapar:

Tablo 43.1: Mermi çeşitleri

Mermi türü	Davranış
Düz mermi	Sabit yönde gider, duvara çarpınca kaybolur
Yerçekimli mermi	Kavisli düşer (bomba gibi)
Takip eden mermi	Her karede oyuncuya doğru yeniden yönelir
Patlayıcı mermi	Hedefe değince alan etkisi

Yerçekimli mermi için `Mermi`'yi genişletmek yeterli:

Python – Yerçekimli Mermi

```

1 class YercekimliMermi(Mermi):
2     def update(self, katilar):
3         self.vy += 0.3 # yavaş yerçekimi
4         super().update(katilar)

```

43.4.6 Stomping ve Ranged

Çoğu ranged düşmanı üstten ezilmez (ok atan iskelet hâlâ oktur). Temel sınıftaki `oldurulebilir` bayrağını kullanarak bunu belirtebiliriz:

Python – Ezilmez Ranged

```

1 class RangedDusman(Dusman):
2     def __init__(self, ...):
3         super().__init__(...)
4         self.oldurulebilir = False # stomp bu düşmanı
           ↪ öldürmez

```

Stomping kontrolü temel sınıfta `if d.oldurulebilir` kontrolünü yaptığı için: oyuncu üstüne zıplasa da düşman ölmez, ama hala yan temas hasarı alır — oyuncu kaçmalı.

⚠ Dikkat: Mermi Grubunu Düşmana Geçirme

`RangedDusman` üretirken ana mermi grubunu parametre olarak geçiriyoruz. Tüm ranged düşmanlar aynı grubu paylaşır, ana döngü tek bir grup üzerinden `update()` ve `spritecollide()` çağırır. Her düşmana ayrı grup verme — karmaşıklık artar.

💡 İpucu: Ateş Aralığını Rastgeleleştirme

Sabit bir `ATES_ARALIK` öngörülebilir. Küçük bir rastgelelik eklemek (örneğin ± 15 kare) oyunculara sabit ritim yakalamayı zorlaştırır: `self.ates_sayac = random.randint(-15, 15)`

● Bilgi: Pratik Alıştırma

`04_ranged_ai_mermi.py` dosyasını çalıştır. İki mor düşman belli aralıklarla sarı mermiler atar. Ateş etmeden önce kırmızı nişan çizgisi görünür — 15 karede kaçabilirsen güvendesin. Mermi çarparsa canın azalır. `MERMI_HIZ`'yi 2'ye düşür: mermiyi kolayca dodge edersin. 8 yap: artık neredeyse imkansız.

43.5 Sonuç

Bu üniteye pasif dikenlerden aktif, *düşünen* düşmanlara geçtik. Temel bir `Dusman(Sprite)` sınıfı ile üç klasik düşman arketipini (patrol, chase, ranged) inşa ettik. Sonlu durum makinesiyle her düşmanın farklı bağlamlara tepki verebilmesini sağladık. Stomping ile oyuncunun düşmanla oynayabildiği bir ilişki kurduk.

43.5.1 Anahtar Kavramlar

Tablo 43.2: Ünite 3 – Anahtar kavramlar

Kavram	Açıklama
FSM (Finite State Machine)	Durum bazlı davranış yönetim deseni
Durum tablosu	String anahtarı → metot eşlemesi
Patrol AI	İki sınır arasında gidip gelen düşman
Uçurum tespiti	Ayak altı kontrolü ile platformdan düşmeyi engelleme
Chase AI	Oyuncuyu belirli mesafede tespit edip takip etme
Fark etme mesafesi	<code>distance_to</code> ile Öklid mesafesi kontrolü
Ranged AI	Periyodik mermi fırlatan düşman
Mermi (projectile)	Kendi hareketi ve ömrü olan bağımsız sprite
Stomping	Oyuncunun düşmana üstten zıplayıp ezmesi
Nişan alma ipucu	Oyuncuya mermiden önce görsel uyarı

43.5.2 Önemli Komutlar

Tablo 43.3: Ünite 3 – Temel komutlar

Komut	Açıklama
<code>pygame.sprite.Sprite</code>	Temel sprite sınıfı (Bölüm 5–6)
<code>pygame.sprite.Group</code>	Sprite grup konteyneri
<code>pygame.sprite.spritecollide</code>	Grup ile tek sprite çarpışması
<code>Vector2.distance_to(o)</code>	İki vektör arası Öklid mesafesi
<code>Vector2.normalize()</code>	Yönü koruyarak uzunluğu 1 yapar
<code>self.kill()</code>	Sprite'ı tüm gruplardan siler
<code>self.image = frames[i]</code>	Animasyon frame değiştirme
<code>pygame.transform.flip(img, True, False)</code>	Yatay flip

43.5.3 Bir Sonraki Ünite: Oyun Sistemleri

Karakterimiz harika hissettiriyor, dünya zengin, düşmanlar tehditkâr — ama oyuncunun bu dünyadan *dönebilmesi* için daha ne gerekiyor? Sonraki ünite bu kuracağız:

- `PowerUp(Sprite)` sınıfı ile hız artırma, ekstra zıplama, kalkan gibi geçici etkiler
- Can yenileme (kalp toplamak veya pasif iyileşme)
- Invincibility frames (i-frames): hasar aldıktan sonra kısa süre dokunulmazlık
- Flicker efekti: i-frames süresince parlayıp sönen sprite
- Tüm bu mekanikleri birleştirerek Zıplayan Macera v3'ü inşa etme

Oyun sistemleri eklendiğinde oyun sadece “çalışan bir karakter” değil, kendi içinde duygusal ritmi olan bir deneyim haline gelir.

💡 İpucu: Düşman Karışımı

Tek tip düşman tek düzeliktir. Hafiften başla: seviye 1'de sadece patrol, seviye 2'de patrol + ranged, seviye 3'te hepsi bir arada. Oyuncuyu tedricen zorla; kendisinin nasıl geliştiğini hissetsin.

“Oyun sistemleri, oyuncunun bir oyunla konuştuğu gizli bir dildir. Power-up bir ödüldür; i-frame bir nefestir; can yenileme ise affedilmedir.”

— ANONİM OYUN TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- **PowerUp(Sprite)** sınıfı ile zaman sınırlı geçici etkiler (hız, zıplama, kalkan) tasarlama
- Ölçekli hasar ve knockback: düşman türüne göre farklı hasar ve geri itme
- Can yenileme: kalp toplanabilirleri ve zamana bağlı pasif yenileme
- Invincibility frames (i-frames) ve flicker efektiyle görsel geri bildirim
- Zıplayan Macera v3: tüm mekanikleri güncellenmiş harita üzerinde entegre etme

44.1 Power-Up: Zaman Sınırlı Etkiler

Oyuncunun yaşlı bir Zelda veya Mario için ne heyecan verici bir şey varsa o da “power-up” bulmaktır. Gücün açılır, karakter önceden yapamadığı şeyi yapabilir, sonra güç kaybolur ve oyuncu “bir dakika kadardı ama harikaydı!” diye oyunun devamını heyecanla bekler. Bu konuda **PowerUp(Sprite)** sınıfıyla bunu nasıl kuracağımızı göreceğiz.

■ Tanım: Power-Up

Oyuncunun belirli bir süre boyunca farklı yetenekler kazandığı veya parametrelerinin değiştiği geçici efektte denir. Örnek: çift hız (15 saniye), ekstra zıplama hakkı (10 saniye), kalkan (bir vuruş soğurur). Süre bitince oyuncu normal haline döner.

44.1.1 PowerUp Sınıfı

Temel yapı: toplanabilir bir sprite, oyuncu dokununca grup'tan çıkar ve oyuncuya kendine özgü etkisini uygular.

Python – PowerUp Sınıfı

```

1 class PowerUp(pygame.sprite.Sprite):
2     def __init__(self, x, y, tur="hiz"):
3         super().__init__()
4         self.tur = tur
5         renkler = {
6             "hiz": (80, 220, 240),    # mavi: hız
7             "zipla": (240, 180, 60),  # sarı: zıplama
8             "kalkan": (180, 80, 220), # mor: kalkan
9         }
10        self.image = pygame.Surface((24, 24))
11        self.image.fill(renkler.get(tur, (200, 200, 200)))
12        self.rect = self.image.get_rect(topleft=(x, y))
13        self.anim_faz = 0    # hafif salınım için
14
15    def update(self):
16        # Küçük yukarı-aşağı salınım (bob)
17        import math
18        self.anim_faz += 0.1
19        self.rect.y += int(math.sin(self.anim_faz) * 0.5)

```

Üç power-up türü tanımladık: hız, ekstra zıplama, kalkan. Her biri oyuncuda farklı bir etki yaratır.

44.1.2 Oyuncuda Etki Yönetimi

Oyuncunun birden fazla power-up'ı aynı anda aktif olabilir. Her birinin bitiş süresi farklıdır. Temiz bir yol: oyuncuda bir sözlük tut.

Python – Oyuncu Etki Yönetimi

```

1 class Oyuncu:
2     def __init__(self, x, y):
3         # ... önceki init ...
4         self.efektler = {}    # {"hiz": 600, "kalkan":
5                               ↪ 900, ...}
6
7     def power_up_al(self, tur):
8         sureler = {"hiz": 600, "zipla": 600, "kalkan":
9                     ↪ 900}
10        self.efektler[tur] = sureler.get(tur, 300)
11
12    def guncelle_efektler(self):
13        # Tüm efektleri kare bazında azalt; biten sil
14        for tur in list(self.efektler.keys()):
15            self.efektler[tur] -= 1

```

```

14         if self.efektler[tur] <= 0:
15             del self.efektler[tur]
16
17     @property
18     def hiz_aktif(self):
19         return "hiz" in self.efektler
20
21     @property
22     def zipla_aktif(self):
23         return "zipla" in self.efektler
24
25     @property
26     def kalkan_aktif(self):
27         return "kalkan" in self.efektler

```

Süreyi kare cinsinden tutuyoruz — 60 FPS’de 600 kare = 10 saniye. Sona erdiğinde efekt silinir.

44.1.3 Hareket ve Zıplamayı Etkiyle Ayarla

`guncelle` içindeki hareket parametrelerini efekte göre ayarlarız:

Python – Efektli Parametreler

```

1 def guncelle(self, tuslar, katilar):
2     # Hız efekti varsa MAX_HIZ iki katı
3     max_hiz = MAX_HIZ * (2.0 if self.hiz_aktif else 1.0)
4
5     if tuslar[pygame.K_LEFT]:
6         self.vx -= IVME
7     elif tuslar[pygame.K_RIGHT]:
8         self.vx += IVME
9     # ...
10    self.vx = max(-max_hiz, min(max_hiz, self.vx))
11
12    def zipla(self):
13        # Zıplama efekti varsa MAX_ZIPLAMA = 3 (triple jump)
14        max_zip = 3 if self.zipla_aktif else MAX_ZIPLAMA
15        if self.kalan_ziplama < max_zip:
16            # ... normal zıplama kodu ...

```

44.1.4 Kalkan: Özel Durum

Kalkan efekti “bir sonraki vuruşu soğur”. Yani can kaybı olunca önce kalkanı silip hasarı iptal eder:

Python – Kalkan Hasarı İptal

```

1 def hasar_al(self, miktar=1):
2     if self.kalkan_aktif:

```



```

3         # Kalkanı kaldır ama hasarı iptal et
4         del self.efektler["kalkan"]
5         return
6     self.can -= miktar
7     if self.can <= 0:
8         self._olum()

```

Kalkan bir vuruşluk değil zaman bazlıysa bu kadar basittir. Bazı oyunlarda kalkan “yenilenebilir” — her birkaç saniyede bir kullanım hakkı. Tasarım tercihinine bağlı.

44.1.5 Power-Up Toplama

Ana döngüde power-up grup ile oyuncu arasında spritecollide:

Python – Power-Up Toplama

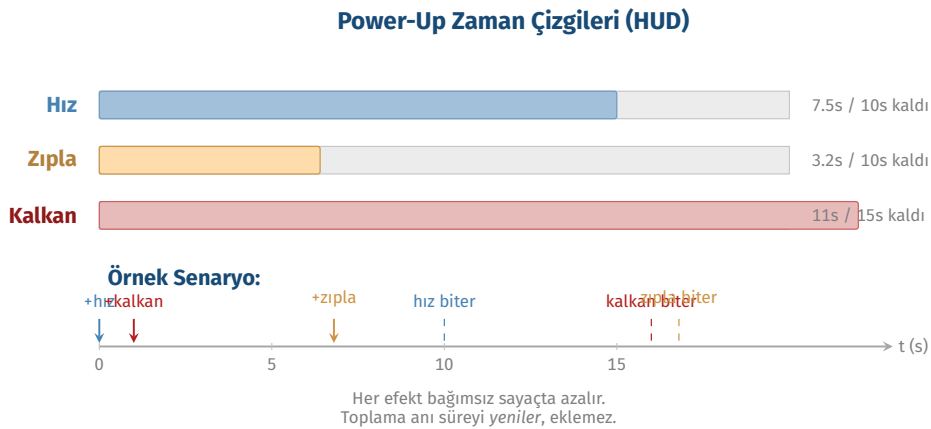
```

1     # Her karede
2     for pu in pygame.sprite.spritecollide(oyuncu,
3         ↳ powerup_grubu, True):
4         oyuncu.power_up_al(pu.tur)
5         # Opsiyonel: ses efekti
6         # ses_pickup.play()

```

44.1.6 HUD’da Aktif Efektleri Göster

Oyuncu hangi efektin ne kadar süre kaldığını görmelidir. Basit bir HUD ikon + süre çubuğu:



Şekil 44.1: Aktif power-up’ların HUD’da zaman çizgisi gösterimi. Süre bittiğinde çubuk sıfırlanır ve efekt silinir.

Python – HUD Efekt Çubuğu

```

1     def ciz_efekt_hud(ekran, oyuncu):
2         x = 10
3         y = 60

```

```

4   renkler = {"hiz": (80, 220, 240),
5             "zipla": (240, 180, 60),
6             "kalkan": (180, 80, 220)}
7   maks_sureler = {"hiz": 600, "zipla": 600, "kalkan":
8   ↪ 900}
9   for tur, sure in oyuncu.efektler.items():
10      oran = sure / maks_sureler.get(tur, 600)
11      # Arkaplan çubuğu
12      pygame.draw.rect(ekran, (60, 60, 60), (x, y, 80,
13      ↪ 10))
14      # Aktif kısım
15      pygame.draw.rect(ekran, renkler[tur], (x, y,
16      ↪ int(80*oran), 10))
17      y += 15

```

⚠ Dikkat: İki Aynı Efekt Süresi Yenilenir

Eğer oyuncu zaten aktif bir efektte sahipken aynı türü tekrar toplarsa `self.efektler[tur] = sure` satırı süreyi yeniler, toplamaz. Daha karmaşık oyunlarda `min(mevcut + sure, maks)` gibi bir ekleme mantığı kullanılabilir.

💡 İpucu: Renk Kodlaması

Power-up'ların renkleri HUD ile eşleşmeli. Mavi hız, sarı zıplama, mor kalkan gibi kalıcı bir renk kodu seç; oyuncu bilinçsizce bu rengi o yetenekle özdeşleştirir.

● Bilgi: Pratik Alıştırma

`01_powerup_temel.py` dosyasını çalıştır. Haritada üç farklı renkte küp göreceksin: mavi (hız), sarı (zıplama), mor (kalkan). Bir tanesini topla; HUD'da süre çubuğu belirir. Mavi aldıysan koşu hızın iki katı; sarı ile triple jump; mor ile bir sonraki vuruşu emer. Süreler bitince her şey normale döner.

44.2 Can Sistemi: Yenileme, Ölçekli Hasar, Knockback

Bölüm 10'da basit bir can sayacımız vardı: `can = 3`, bir dikene değince `can -= 1`. Şimdi bunu daha zengin bir sisteme çevireceğiz:

- Oyuncu canını *yenileyebilsin* (kalp toplayarak veya pasif)
- Farklı düşmanların farklı miktarda hasar vermesi (ölçekli)
- Hasar alındığında *knockback* (geri itme) — oyuncu az da olsa fırlar

44.2.1 Can Değişkenleri

Tek bir sayaç yerine oyuncuda iki değer tutarız: mevcut can (`can`) ve maksimum can (`max_can`). Bu ayırım can yenilemeyi kolaylaştırır.

Python – Genişletilmiş Can

```
1 class Oyuncu:
2     def __init__(self, x, y):
3         # ... önceki init ...
4         self.max_can = 3
5         self.can = 3
```

Maksimumu geçmek yasak; toplanan fazla kalplerin etkisi olmaz.

44.2.2 Kalp Toplama ile Yenileme

En klasik yenileme yöntemi: haritaya serpilmiş kalpler. Bunlar `Kalp` adında bir sprite sınıfıyla temsil edilir.

Python – Kalp Sprite

```
1 class Kalp(pygame.sprite.Sprite):
2     def __init__(self, x, y, miktar=1):
3         super().__init__()
4         self.image = pygame.Surface((16, 16))
5         self.image.fill((240, 80, 100))
6         self.rect = self.image.get_rect(center=(x + 8, y +
7             ↪ 8))
8         self.miktar = miktar # kaç can ekleyecek
```

Oyuncuda ise:

Python – Can Yenileme

```
1 def can_yenile(self, miktar):
2     self.can = min(self.max_can, self.can + miktar)
```

Ana döngüde kalp çarpışması:

Python – Kalp Çarpışması

```
1 for kalp in pygame.sprite.spritecollide(oyuncu,
2     ↪ kalp_grubu, True):
3     oyuncu.can_yenile(kalp.miktar)
```

44.2.3 Pasif (Zamanla) Yenileme

Bazı oyunlar oyuncu hasar almadığı sürece zamanla yavaşça can yenilemesine izin verir. Klasik örneği Hollow Knight'tır (*Focus* ile hızlı yenileme). Basit hali:

Python – Pasif Yenileme

```

1 PASIF_YENILEME_SURE = 300 # 5 saniye (60 FPS)
2
3 class Oyuncu:
4     def __init__(self, x, y):
5         # ...
6         self.pasif_yenileme_sayac = 0
7
8     def guncelle(self, ...):
9         # ...
10        # Pasif yenileme: her 5 saniyede 1 can
11        self.pasif_yenileme_sayac += 1
12        if self.pasif_yenileme_sayac >=
13            ↪ PASIF_YENILEME_SURE:
14            self.pasif_yenileme_sayac = 0
15            self.can_yenile(1)
16
17    def hasar_al(self, miktar=1):
18        # Hasar alınca sayacı sıfırla (yenileme geciksin)
19        self.pasif_yenileme_sayac = 0
20        # ...

```

Sakin kalbi olan oyuncular ödüllendirilir. Aktif çatışma sırasında yenileme gecikir.

44.2.4 Ölçekli Hasar

Bütün düşmanlar aynı hasarı verirse oyuncu düşman türlerini “hissetme”sinin bir yolu kalmaz. Patrol bir canlı birim, ama ranged bir mermi daha dehşet verici olmalı. Çözüm: `hasar\_al` fonksiyonunu parametrelili yapmak.

Python – Ölçekli Hasar Tanımı

```

1 DUSMAN_HASAR = {
2     "patrol": 1,
3     "chase": 1,
4     "ranged": 1, # mermi
5     "boss": 2,
6     "diken": 1,
7     "lava": 2,
8 }
9
10 class Oyuncu:
11     def hasar_al(self, miktar=1):
12         if self.kalkan_aktif:
13             del self.efektler["kalkan"]
14             return
15         self.can -= miktar
16         if self.can <= 0:
17             self._olum()

```

Ana döngüde:

Python – Hasar Kaynağına Göre Miktar

```

1 for d in pygame.sprite.spritecollide(oyuncu, dusmanlar,
   ↪ False):
2     # üstten değilse yan temas:
3     if not (oyuncu.vy > 0 and oyuncu.rect.bottom <
   ↪ d.rect.top + 16):
4         miktar = DUSMAN_HASAR.get(type(d).__name__.lower(
   ↪ ).replace("dusman", ""), 1)
5         oyuncu.hasar_al(miktar)
6
7 for m in pygame.sprite.spritecollide(oyuncu, mermiler,
   ↪ True):
8     oyuncu.hasar_al(DUSMAN_HASAR["ranged"])

```

44.2.5 Knockback: Geri İtme

Hasar aldığı anda oyuncu bir an fırlar — bu hem görsel olarak etkileyici hem de oyuncuya “geri çekil” mesajı verir. Yöntem kolay: hasar yönüne göre yatay hız at.

Python – Knockback Uygulaması

```

1 KNOCKBACK_HIZ = 6
2 KNOCKBACK_SURE = 12
3
4 class Oyuncu:
5     def __init__(self, x, y):
6         # ...
7         self.knockback_sayac = 0
8
9     def hasar_al(self, miktar=1, kaynak_rect=None):
10        # ... kalkan kontrolü, can azaltma ...
11        if kaynak_rect is not None:
12            yon = 1 if self.rect.centerx >
   ↪ kaynak_rect.centerx else -1
13            self.vx = yon * KNOCKBACK_HIZ
14            self.vy = -5 # küçük yukarı atım
15            self.knockback_sayac = KNOCKBACK_SURE
16
17    def guncelle(self, tuslar, katilar):
18        # Knockback sırasında klavye girişi iptal
19        if self.knockback_sayac > 0:
20            self.knockback_sayac -= 1
21        else:
22            # Normal klavye kontrolü
23            # ...

```

Knockback süresince oyuncu klavye kontrolünü kaybeder; fırladığı yere savrulur. 12 kare (200 ms) çoğu oyun için uygun süredir.

⚠ Dikkat: Knockback + I-Frames

Knockback olduğu anda oyuncunun hemen aynı düşmandan tekrar hasar almaması için bir sonraki konuda göreceğimiz *i-frames* (dokunulmazlık) devreye girmeli. İkisi birlikte çalışmalı.

💡 İpucu: Can Barı Stilleri

Sayısal can yerine kalp ikonu dizisi (♥♥♥) veya uzun bir bar kullanabilirsin. Sayı küçük değerler için iyidir (3–5 can); bar büyük değerler için (100/100).

🕒 Bilgi: Pratik Alıştırma

`02_can_sistemi.py` dosyasını çalıştır. 3 can ile başla. Haritada kırmızı kalplere yürü ve can yenile. Düşmana ya da dikene değ: hasar alırken knockback ile geri fırlayacaksın. 5 saniye düşmansız kal: pasif yenileme bir can eklesin. Boss tile'ına değ: 2 hasar. Bu sistem oyununun derinliğini görünür kılar.

44.3 I-Frames ve Flicker Efekt

Hasar aldıktan sonra karakter bir an için “dokunulmaz” olur; aynı düşmana değmekle canını üst üste kaybetmez. Bu görünmez ama kritik bir lütuftur. Oyun geliştirmede kısaca *i-frames* (invincibility frames) deriz. Ayrıca bu süre boyunca karakter yarı saydam veya parlayıp sönen (flickering) bir halde gösterilerek oyuncuya “şu an güvendesin ama uzun sürmeyecek” mesajı verilir.

■ Tanım: Invincibility Frames (I-Frames)

Hasar aldıktan hemen sonra oyuncunun belirli bir süre (genelde 30–60 kare) tekrar hasar almasını engelleyen dokunulmazlık penceresidir. Oyuncuya düşman gruplarından kaçmak, knockback'ten toparlanmak için zaman tanır. Görsel olarak *flicker* (titrek saydamlık) ile desteklenir.

44.3.1 I-Frames Sayacı

Kod basit. Hasar alındığı anda bir sayaç başlat; sayaç > 0 iken hasar yok sayılır.

Python – I-Frames Sayacı

```
1 I_FRAME_SURE = 60 # 1 saniye (60 FPS)
2
3 class Oyuncu:
4     def __init__(self, x, y):
5         # ...
6         self.iframe_sayac = 0
```

```

7
8     def guncelle(self, ...):
9         # ...
10        if self.iframe_sayac > 0:
11            self.iframe_sayac -= 1
12
13    def hasar_al(self, miktar=1, kaynak_rect=None):
14        if self.iframe_sayac > 0:
15            return # hala dokunulmaz
16        if self.kalkan_aktif:
17            del self.efektler["kalkan"]
18            self.iframe_sayac = I_FRAME_SURE # kalkan
19            ↪ yine iframe tetikler
20            return
21
22        self.can -= miktar
23        self.iframe_sayac = I_FRAME_SURE # dokunulmazlık
24        ↪ penceresi aç
25
26        if kaynak_rect is not None:
27            # knockback...
28            yon = 1 if self.rect.centerx >
29                ↪ kaynak_rect.centerx else -1
30            self.vx = yon * KNOCKBACK_HIZ
31            self.vy = -5
32            self.knockback_sayac = KNOCKBACK_SURE
33
34        if self.can <= 0:
35            self._olum()

```

Artık oyuncu bir düşmanla yan yana duruyor bile olsa, ilk temasta bir hasar alır ve 60 kare boyunca (1 saniye) aynı düşmanla tekrar hasar almaz. Bu sırada knockback ile zaten geriye fırladığı için çoğu zaman uzaklaşmış olur.

44.3.2 Flicker: Parlayıp Sönen Görsel

I-frames süresince oyuncu titremeli ki oyuncu “hasar aldım ama şu an korunuyorum” bilsin. Bunu yapmanın en kolay yolu: her iki-üç karede bir sprite’ı çizmemek.

Python – Flicker Çizimi

```

1     def ciz(self, ekran, kamera):
2         # I-frames boyunca her 4 karede 1 kareyi çizme
3         if self.iframe_sayac > 0 and (self.iframe_sayac // 4)
4             ↪ % 2 == 0:
5             return # bu kare çizme (flicker)
6         ekran.blit(self.image, kamera.apply(self.rect))

```

$(sayac // 4) \% 2 == 0$ ifadesi, 4 kare boyunca “çizme”, sonraki 4 kare “çiz” döngüsüdür. Sonuç: saniyede yaklaşık 7.5 yanıp sönme. Hız düşük: 8 kare

yap — daha yavaş yanıp sönme, daha telaşlı his.

Flicker efekti: yatay ekseninde zaman (kare), dikey ekseninde sprite'ın görünür olup olmadığı. İki durum arasında eşit bölünür; oyuncu sprite'ın titrediğini görür.

44.3.3 Alternatif: Saydamlık (Alpha)

Tamamen çizmek-çizmemek sert bir efektdir. Daha yumuşak bir his için alfa değerini oynatabilirsin:

Python – Alfa ile Flicker

```
1 def ciz(self, ekran, kamera):
2     kopya = self.image.copy()
3     if self.iframe_sayac > 0:
4         import math
5         # 0'dan 255'e salınım
6         alpha = int(128 + 127 * math.sin(self.iframe_sayac
7             ↪ * 0.5))
8         kopya.set_alpha(alpha)
9         ekran.blit(kopya, kamera.apply(self.rect))
```

Her frame alfayı `sin` dalgasıyla salındır; oyuncu “hayaletleşmiş” gibi görünür. Retro oyunlarda “on/off” flicker daha yaygın, modern oyunlarda alfa salınımı daha estetik.

44.3.4 Düşmandan Gelen I-Frames mi, Toplam mı?

İki yaklaşım var:

1. **Toplam i-frames:** Oyuncu bir kez hasar aldı, 60 kare herhangi bir kaynaktan hasar almaz.
2. **Düşman bazlı i-frames:** Oyuncu A düşmanından hasar aldı, ama aynı anda B düşmanı ile temas geçerse yine hasar alabilir.

Birinci yaklaşım hem kolay hem standarttır; bu kitap onu kullanıyor. İkincisi daha “hardcore” oyunlarda tercih edilir (Dark Souls gibi).

44.3.5 I-Frames Süresi Ayarı

Doğru süre oyununun karakteristiğine bağlıdır:

Tablo 44.1: I-frames süresi rehberi

Süre (kare)	Saniye	Karakter
20	0.33	Hardcore, cezalandırıcı
45	0.75	Celeste/Hollow Knight tipi
60	1.0	Mario/Sonic tipi (önerilen)
120	2.0	Kazander, düşman yoğunluğu çok yüksek oyunlar

Oyunun zorluk eğrisine göre ayarla. Oyuncu “çok hızlı ardı ardına hasar alıyorum” diye şikâyet ederse süreyi artır.

⚠ Dikkat: Knockback Süresi I-Frames’ten Kısa Olmalı

Knockback (`KNOCKBACK_SURE = 12`) i-frames (`I_FRAME_SURE = 60`) içinde bitmeli. Aksi halde oyuncu klavye kontrolünü uzun süre kaybeder. İyi değerler: knockback 10–15 kare, i-frames 45–75 kare.

💡 İpucu: Ses Efekti İle Destekle

Hasar alındığı anda bir “vuruş” sesi ve i-frames süresince devam eden hafif bir tin-tin çingirak sesi oyuncuya “koruma altındayım” hissini güçlendirir. Bölüm 7’deki ses kütüphanesini kullanarak ekle.

● Bilgi: Pratik Alıştırma

`03_iframes.py` dosyasını çalıştır. Bir düşmana yürü: hasar aldığın anda karakter titremeye başlar ve 1 saniye boyunca dokunulmazdır. Düşmana üst üste dokunmayı dene: üç-dört kez değsen de bir tane can kaybedersin. Şimdi `I_FRAME_SURE` değerini 0 yap: bir düşmanla temas 60 FPS’de saniyede 60 kez hasar verir — iki saniyede ölürsün. Bu nedenle i-frames hayatı bir “adalet” mekanizmasıdır.

44.4 Mini Proje: Zıplayan Macera v3 – Mekanik Entegrasyonu

Bölüm 10’da tek bir `main.py` dosyası ile Zıplayan Macera’nın ilk oynanabilir seviyesini çıkarmıştık: tile haritası, kamera, oyuncu fiziği, altın ve diken. Bölüm 11’de bu temele bu ünite de öğrendiğimiz her şeyi ekliyoruz: coyote time, jump buffer, variable + double jump, yeni platform türleri, üç düşman arketipi, power-up, genişletilmiş can sistemi ve i-frames.

Bu konuda asıl yaptığımız şey *kodu modüllere ayırmak*. Tek bir dev dosya yerine görev bazlı bölünmüş üç dosya: `main.py`, `entities.py`, `platforms.py`.

44.4.1 Proje Yapısı

Tablo 44.2: Ziplayan Macera v3 dosya yapısı

Dosya	İçerik
<code>main.py</code>	Oyun giriş noktası, ana döngü, <code>Seviye</code> yükleme, HUD
<code>entities.py</code>	<code>Oyuncu</code> , <code>Dusman</code> ve alt sınıfları, <code>PowerUp</code> , <code>Kalp</code> , <code>Mermi</code>
<code>platforms.py</code>	<code>HareketliPlatform</code> , <code>CrumblingPlatform</code> , <code>BouncyPlatform</code>
<code>harita/seviye1.csv</code>	Genişletilmiş tile haritası (ID 1–9)

Bu ayırım kod tabanını ölçeklenebilir yapar: yarın yeni bir düşman türü eklemek için sadece `entities.py`'yi düzenlersin; `main.py` aynı kalır.

44.4.2 Güncellenmiş Tile ID Şeması

Python – Tile ID Şeması

```

1 # ID haritası (Bölüm 10 + Bölüm 11)
2 # 1 = boş (görsel arkaplan)
3 # 2 = solid taş (Bölüm 10)
4 # 3 = oyuncu spawn noktası
5 # 4 = altın (Bölüm 10)
6 # 5 = diken / tehlike (Bölüm 10)
7 # 6 = tek yönlü platform (Ünite 2)
8 # 7 = hareketli platform (Ünite 2)
9 # 8 = crumbling platform (Ünite 2)
10 # 9 = bouncy platform (Ünite 2)
11 # düşmanlar ve power-up'lar CSV'nin dışında,
12 # main.py içinde listeler olarak tanımlanır (basitlik
   ↪ için)
```

Daha gelişmiş yaklaşım: Tiled editor'ün “obje katmanı” ile düşmanları ve power-up'ları konumlandırmak; ama bu kitap için düz CSV + Python listesi yeterli.

44.4.3 `entities.py` İçeriği

Bu modül oyuncu, düşmanlar ve pickup sprite'larını barındırır:

Python – `entities.py` İskeleti

```

1 """entities.py - Oyuncu, Düşmanlar, Pickups"""
2
```

```

3 import pygame
4
5 # --- Sabitler ---
6 IVME = 0.7
7 MAX_HIZ = 5.0
8 SURTUNME = 0.4
9 ZIP_HIZI = -11
10 MAX_ZIPLAMA = 2
11 COYOTE_SURE = 6
12 BUFFER_SURE = 6
13 I_FRAME_SURE = 60
14 KNOCKBACK_HIZ = 6
15 KNOCKBACK_SURE = 12
16
17
18 class Oyuncu(pygame.sprite.Sprite):
19     """Hareket hissiyatı + can sistemi + efektler."""
20     # Ünite 1--4'te tasarlanan tüm özellikler burada
21
22 class Dusman(pygame.sprite.Sprite):
23     """Tüm düşmanların atası. FSM iskeleti."""
24
25 class PatrolDusman(Dusman):
26     """Ünite 3 - Konu 2"""
27
28 class ChaseDusman(Dusman):
29     """Ünite 3 - Konu 3"""
30
31 class RangedDusman(Dusman):
32     """Ünite 3 - Konu 4"""
33
34 class Mermi(pygame.sprite.Sprite):
35     pass
36
37 class PowerUp(pygame.sprite.Sprite):
38     pass
39
40 class Kalp(pygame.sprite.Sprite):
41     pass

```

Gerçek kod [kod_ornekleri/bolum11/projeler/ziplayan_macera_mekanik/entit](#) içinde — her sınıfın tam uygulaması orada.

44.4.4 platforms.py İçeriği

Bu modül özel platform türlerini barındırır:

Python – platforms.py İskeleti

```

1 """platforms.py - Hareketli, Crumbling, Bouncy
   ↳ platformlar"""

```

```

2
3 import pygame
4
5 TILE = 32
6
7
8 class HareketliPlatform:
9     """Ünite 2 - Konu 3"""
10
11 class CrumblingPlatform:
12     """Ünite 2 - Konu 4"""
13
14 class BouncyPlatform:
15     """Ünite 2 - Konu 4"""

```

Bu üç sınıf `pygame.sprite.Sprite`'tan türemez; çünkü kendi hareketlerini ve çizimlerini yönetirler. Basit Python sınıfları olarak kalırlar.

44.4.5 main.py: Tutkallar

`main.py` içinde `Seviye` sınıfı CSV'yi yükler, entities ve platforms'tan sınıfları import eder, ana döngüyü sürdürür:

Python – main.py Çekirdeği

```

1 import csv
2 from pathlib import Path
3 import pygame
4
5 from entities import Oyuncu, PatrolDusman, ChaseDusman, \
6     RangedDusman, PowerUp, Kalp
7 from platforms import HareketliPlatform, \
8     CrumblingPlatform, BouncyPlatform
9
10 TILE = 32
11 GENISLIK = 960
12 YUKSEKLIK = 540
13
14
15 class Seviye:
16     def __init__(self, csv_yol):
17         self.harita = self._yukle(csv_yol)
18         self.gen = len(self.harita[0])
19         self.yuk = len(self.harita)
20         self.dunya_gen = self.gen * TILE
21         self.dunya_yuk = self.yuk * TILE
22         self.spawn = (80, self.dunya_yuk - 120)
23         self.katilar = []
24         self.altinlar = []
25         self.tehlikeler = []
26         self.tek_yonluler = []

```

```

27     self.hareketliler = []
28     self.crumblings = []
29     self.bouncylar = []
30     self._grupla()
31
32     # ... _yukle, _grupla, ciz metotları ...
33
34
35 def main():
36     pygame.init()
37     ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
38     pygame.display.set_caption("Zıplayan Macera v3")
39     saat = pygame.time.Clock()
40
41     seviye = Seviye(Path(__file__).parent / "harita" /
42     ↪ "seviye1.csv")
43     oyuncu = Oyuncu(*seviye.spawn)
44
45     # Düşman ve power-up'lar elle yerleştirilir
46     dusmanlar = pygame.sprite.Group(
47         PatrolDusman(300, 300, 280, 450),
48         ChaseDusman(700, 300, 650, 900),
49         RangedDusman(1200, 200, mermi_grubu :=
50         ↪ pygame.sprite.Group())
51     )
52     mermiler = pygame.sprite.Group()
53     powerups = pygame.sprite.Group(
54         PowerUp(500, 350, "hiz"),
55         PowerUp(900, 200, "kalkan"),
56     )
57     kalpler = pygame.sprite.Group(Kalp(800, 400))
58
59     # ... ana döngü ...

```

44.4.6 Oyun Tasarım Kararları

Zıplayan Macera v3'te şu tasarım tercihlerini kullanıyoruz:

1. **Can başlangıcı:** 3 (zorlayıcı ama adil)
2. **Coyote/Buffer:** 6 kare (standart)
3. **Max jump:** 2 (double jump var)
4. **Patrol düşmanı:** 2 adet, sırada
5. **Chase düşmanı:** 1 adet, altın yakınında
6. **Ranged düşmanı:** 1 adet, düz koridor ortasında
7. **Power-up'lar:** 3 farklı türde, seviye boyunca dağılmış
8. **I-frames:** 60 kare (1 saniye)

Bu değerler `entities.py` üstünde sabit olarak durur; oynanışı tamamen yeniden ayarlamak için tek dosya düzenlemek yeter.

44.4.7 Çalıştırma ve Yaratıcı Deney

Proje `kod_ornekleri/bolum11/projeler/ziplayan_macera_mekanik/` dizinindedir. Çalıştırmak için:

Python – Çalıştırma

```
1 cd kod_ornekleri/bolum11/projeler/ziplayan_macera_mekanik
2 uv run python main.py
```

⚠ Dikkat: Paket Yolu

Proje dizinini kök olarak çalıştır; aksi halde `entities` ve `platforms` modülleri bulunamaz. Python çalıştırırken `-m` bayrağı değil, doğrudan `python main.py` kullan.

💡 İpucu: Deney Saatleri

Oyunu çalıştırıp ilk saati “şu güzel oldu, şu değil” diye yazmaya ayır. Sonra parametreleri oynat. `MAX_HIZ`, `IVME`, `I_FRAME_SURE`, `FARK_MESAFE`, power-up süreleri, düşman sayıları — hepsi `entities.py`’nin en üstünde, açıkça sabit olarak. Kendi “Zıplayan Macera”nı şekillendir.

● Bilgi: Pratik Alıştırma

Projeyi çalıştır ve seviyeyi baştan sona oynat. Tüm mekanikleri dene: coyote time (uçurumdan düşerken SPACE), jump buffer (havada bir kere bas, yere değince zıpla), double jump (havada ikinci kez bas), tek yönlü platform (alttan zıpla, üstten dur), hareketli platform (bin, taşı), crumbling (dağılmadan zıpla), bouncy (fırla), power-up (topla), i-frames (düşmandan sonra dokunulmaz kal). Sonra kendi mini eklemeni yap: `harita/seviye1.csv`’e yeni bir bölge ekle veya yeni bir düşman spawn et.

44.5 Sonuç

Bu ünite de oyuncunun dünyayla kurduğu duygusal ilişkiyi derinleştirdik. Power-up’lar sürprizler getirdi; can yenileme affedişi kurdu; i-frames adaleti sağladı. Son olarak tüm Bölüm 11 öğrendiklerini modüler bir mimari (`main.py`, `entities.py`, `platforms.py`) ile Zıplayan Macera v3’te birleştirdik.

44.5.1 Anahtar Kavramlar

Tablo 44.3: Ünite 4 – Anahtar kavramlar

Kavram	Açıklama
Power-up Efekt sözlüğü	Zaman sınırlı geçici etki (hız, zıplama, kalkan) Aktif power-up'ları {"hiz": 300} şeklinde tutma
Can yenileme	Kalp toplama veya pasif (zamana bağlı) iyileşme
Ölçekli hasar	Düşman türüne göre farklı hasar değeri
Knockback	Hasar alındığında oyuncunun geri savrulması
I-frames	Hasar sonrası kısa dokunulmazlık penceresi
Flicker	I-frames süresince sprite'ın yanıp sönmesi
Modüler mimari	Kodu görev bazlı dosyalara bölme (main/entities/platforms)

44.5.2 Önemli Komutlar

Tablo 44.4: Ünite 4 – Temel komutlar

Komut	Açıklama
<code>self.efektler[tur] = sure</code>	Power-up etkisini aktive et
<code>del self.efektler[tur]</code>	Etki bitti, sil
<code>self.can = min(self.max_can, ...)</code>	Can taşmasını önle
<code>self.iframe_sayac > 0</code>	Dokunulmazlık kontrolü
<code>self.image.set_alpha(n)</code>	Saydamlık (flicker için)
<code>from entities import ...</code>	Modüler import
<code>pygame.sprite.spritecollide(s, g, True)</code>	Grup ile çarpışma, dokill=True

44.5.3 Bölüm 11'in Özeti

Bu bölüm boyunca platform oyunu mekaniğini dört ana sütunda inşa ettik:

1. **Hareket Hissiyatı** (Ünite 1): İvme, sürtünme, coyote time, jump buffer, variable jump, double jump
2. **Platform Çeşitliliği** (Ünite 2): Çarpışma yönü, tek yönlü, hareketli, crumbling, bouncy platformlar
3. **Düşman Davranışları** (Ünite 3): FSM desen, patrol, chase, ranged AI, stomping
4. **Oyun Sistemleri** (Ünite 4): Power-up, can yenileme, ölçekli hasar, knockback, i-frames, flicker

Zıplayan Macera v3 bu dört sütunu tek bir modüler projede birleştirir. Oyunun temeli artık sağlam.

44.5.4 Sıradaki Adım: Bölüm 12

Projemiz çalışıyor, ama her oyun oturumu baştan başlıyor — önceki ilerlemen, toplanmış altınlar, geçilen seviyeler hatırlanmıyor. Bölüm 12’de bunu değiştireceğiz:

- **State Machine Tasarımı:** Oyun durumlarını (menu, oyun, pause, game-over, kayıt ekranı) güçlü bir FSM ile yönetme
- **Dosya İşlemleri:** JSON ve pickle ile Python veri yapılarını diske yazma
- **Save/Load Sistemi:** Kayıt formatı, slot sistemi, otomatik kayıt
- **Ayarlar ve Konfigürasyon:** Sesler, kontroller, görsel seçenekler için kalıcı **Settings** sınıfı

Karakterin iyi hissettiği, dünyanın çeşitli olduğu, düşmanların tehditli olduğu, sistemlerin ödül verdiği bir oyun yaptık. Şimdi o oyunu “kaydedebilen” bir oyun yapacağız.

💡 İpucu: Bölüm 11’i Süsleme

Bu kitap fonksiyonel olana odaklandı. Kendi eklemelerini yapmak için şu alanlar verimli:

- Parçacık efektleri (toz, yaprak, kıvılcım) hareket hissini artırır
- Ekran sallanması (camera shake) yüksek hasar veya zıplatan platform efektleri için güzel
- Sprite yerine gerçek sanat: Bölüm 8’in animasyon sistemi buraya ağırlanmayı bekliyor
- Ses: her zıplama, hasar, power-up toplama — ses hissiyatı iki katına çıkarır (Bölüm 7)

State Machine Tasarımı

“Bir oyun, aslında durumlar arasında geçiş yapan büyük bir makinedir. Menüye basarsın, oyuna geçer; ESC’e basarsın, durur; ölürsün, “game over” ekranı gelir. Bu durumların her biri kendi mantığını ve çizimini taşır.”

— ROBERT NYSTROM, GAME PROGRAMMING PATTERNS

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Oyununun büyüdükçe neden bir *state machine*’e ihtiyaç duyduğunu
- Soyut bir `State` sınıfının `handle\_events`, `update`, `draw` arayüzü
- Menü, Play, Pause ve GameOver durumlarını ayrı sınıflar olarak modelleme
- `StateManager` ile yığın veya tekil değişim mantığı kullanarak durumlar arası geçiş
- Bölüm 11’in Zıplayan Macera v3 oyununu state machine mimarisine taşıma

45.1 State Machine Nedir? Neden Lazım?

Bölüm 11’de Zıplayan Macera v3 oyununu tamamladın. Açtığında karakter hemen sahnenin ortasına düşüyor, oynamaya başlıyorsun. Peki ya bir *ana menü* eklemek istesen? Ya `ESC` tuşuyla oyunu duraklatıp tekrar başlatmak istesen? Ya öldüğünde *game over* ekranı çıkıp “yeniden dene” dese? Bütün bunları tek bir `main` döngüsünün içinde `if menu: ... elif oyun: ... elif pause: ...` şeklinde yazabilirsin — ama bir hafta sonra o döngüye baktığında ağlarsın. İşte tam bu noktada *state machine* (durum makinesi) deseni devreye girer.

■ Tanım: State Machine

Bir *state machine*, sistemin aynı anda yalnızca bir durumda bulunduğu ve her durumun kendi davranışını tanımladığı bir desendir. Belirli olaylar (oyuncu girişi, zaman, çarpışma) bir durumdan diğerine *geçiş* (transition) tetikler. Her durum kendi *giriş*, *güncelleme* ve *çıkış* mantığını taşır.

Oyunlarda iki tür state machine ayırımı yaparız: bir karakter veya düşmanın iç davranışını yöneten *karakter FSM'leri* (Bölüm 11 Ünite 3'teki Patrol/Chase/Idle gibi), bir de tüm oyun ekranlarını yöneten *oyun durumu makineleri*. Bu ünite de ikincisini ele alıyoruz.

45.1.1 Devasa **if-elif** Bloğunun Sorunu

Önce şu kötü örneğe bir bak. State machine olmadan bir menü-oyun- pause akışı şöyle yazılır:

Python – State Machine'siz Karmaşa

```

1  durum = "menu"
2
3  while calistir:
4      for olay in pygame.event.get():
5          if olay.type == pygame.QUIT:
6              calistir = False
7          if durum == "menu":
8              if olay.type == pygame.KEYDOWN and olay.key ==
                 ↳ pygame.K_RETURN:
9                  durum = "oyun"
10             elif durum == "oyun":
11                 if olay.type == pygame.KEYDOWN and olay.key ==
                     ↳ pygame.K_ESCAPE:
12                     durum = "pause"
13             elif durum == "pause":
14                 if olay.type == pygame.KEYDOWN and olay.key ==
                     ↳ pygame.K_ESCAPE:
15                     durum = "oyun"
16
17             if durum == "menu":
18                 ekran.fill((30, 30, 60))
19                 ekran.blit(menu_basligi, (200, 100))
20                 # ... menü çizimleri ...
21             elif durum == "oyun":
22                 oyuncu.guncelle(tuslar, katilar)
23                 # ... düşman, harita, kamera ...
24                 ekran.fill((20, 20, 30))
25                 # ... oyun çizimleri ...
26             elif durum == "pause":
27                 # Önce oyun karesini çiz, üstüne yarı-saydam panel
28                 # ... 50 satır daha ...

```

29
30

```
pygame.display.flip()
```

Burada üç durum birbirine girmiş. Bir tanesini değiştirmek istesen, diğerlerine zarar vermeden bunu yapabileceğine emin değilsin. `durum` değişkeninin geçerli değerlerini hatırlamak senin sorumluluğun — IDE sana yardım etmiyor. Yeni bir *game over* durumu eklemek için iki yerde (olay döngüsü ve çizim döngüsü) yeni `elif` eklemen gerekiyor.

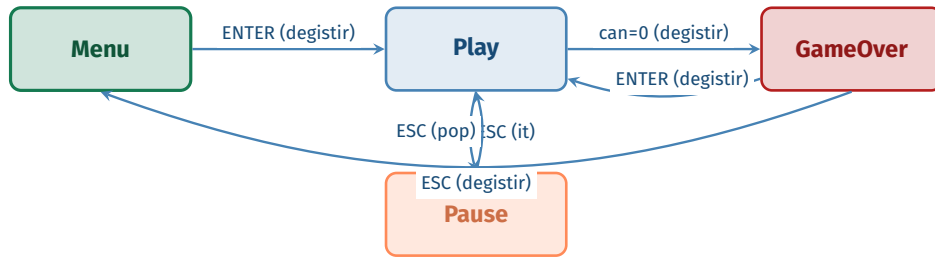
⚠ Dikkat: Bu Kod Büyürse...

Üç durum 80 satır oldu. Beş durum 200 satır olur. Yedi durum 400 satır. Sekizinci durumu eklerken *yedinci* durumu bozarsın. Bu, klasik bir “God main loop” anti-paterninin başlangıcıdır.

45.1.2 State Machine’in Kazandırdıkları

State machine deseninde her durum *kendi sınıfıdır*. Sınıfın `handle\_events`, `update` ve `draw` metotları yalnızca *o duruma* ait kodu içerir.

Oyun Durum Geçiş Diyagramı



degistir: ust state kapanir, yenisi gelir (Menu → Play)
it (push): ust state bekler, yenisi yigina eklenir (Play → Pause)
pop: ust state kapanir, alttaki tekrar aktif olur (Pause → Play)

Şekil 45.1: Bir oyunun tipik durum geçiş diyagramı. Her ok bir geçiş, her düğüm bir durum sınıfıdır.

Avantajları:

1. **Tek sorumluluk:** Bir `MenuState` dosyası yalnızca menü mantığını içerir. Oyun mantığını okumak için o dosyayı açmana gerek yok.
2. **Temiz geçişler:** “Şu durumdan şuna geç” tek bir çağrıyla olur: `manager.degistir(PlaySt`
3. **Yeniden kullanım:** `PauseState` hangi oyun sahnesinin üstünde kullanıldığına bakmaz. Stack tabanlı state machine ile her sahnenin üstünde aynı kodla çalışır.
4. **Test edilebilirlik:** Bir durumu izole olarak çalıştırabilirsin — dev test menüsü, fizik test sahnesi gibi.

45.1.3 İki Yaklaşım: Tekil Değişim ve Yığın (Stack)

State machine'ler iki farklı şekilde organize edilir:

Tekil değişim (single-state): Aynı anda yalnızca bir durum aktiftir. Yeni duruma geçince eski durum tamamen unutulur. Menü → Oyun → Game Over → Menü gibi doğrusal akışlarda idealdir.

Yığın (stack-based): Durumlar bir yığına itilir. Üstteki durum aktiftir; ESC ile pop edildiğinde altındaki durum geri döner. Bu, *pause* için doğal bir desendir: PlayState'in *üstüne* PauseState itilir, pop edildiğinde oyun aynı yerde devam eder.

İpucu: Hangisi Daha İyi?

Cevap: *ikisi birden*. Profesyonel oyunlarda altta bir tekil değişim makinesi (Menü, Oyun, Skor), üstünde bir yığın (Pause, Inventory, Dialog) bulunur. Bu üniteye yığın tabanlı bir tasarım inşa edeceğiz; ihtiyaç olunca sadece tek üyeli bir yığın gibi çalışır.

45.1.4 Bu Üniteden Çıkacağımız Yer

Bu ünitenin sonunda elinde şunlar olacak:

- Soyut bir **State** taban sınıfı (`state.py`)
- Yığın tabanlı **StateManager** (`state_manager.py`)
- Dört somut durum: **MenuState**, **PlayState**, **PauseState**, **GameOverState**
- Bölüm 11'in oyununu kırarak şekilde değil, modüler bir şekilde yeni mimariye taşıyan bir `main.py`

Bilgi: Hazırlık

Dosya işlemleri Ünite 2'ye kadar kullanılmayacak. Bu üniteyi tamamen state akışına odakla; kayıt-yükleme bir sonraki üniteye "zaten doğru durumdayken hangi veriyi yazıyoruz?" sorusuyla gelecek.

45.2 Soyut State Sınıfı ve StateManager

Bir önceki konuda neden state machine istediğimizi gördük. Şimdi hangi tuğlaları yerleştireceğimizi tasarlıyoruz. Hedefimiz şu: her oyun durumu (menü, oyun, pause, game over) ortak bir arayüze sahip olsun. Bu arayüz üç metoda dayansın: olayları işle, mantığı güncelle, ekrana çiz. Üzerine bir de **StateManager** koyalım ki durumlar arası geçiş tek bir merkezi noktadan yapılsın.

45.2.1 Soyut State Sınıfı

Python'da soyut sınıflar için `abc` modülünü kullanırız. Ama oyun geliştirmede genellikle daha hafif bir yaklaşım yeterlidir: sadece metotları tanımla, alt sınıfların `override` etmesini bekle.

Python – state.py – Soyut Taban Sınıfı

```

1 class State:
2     """Tüm oyun durumlarının türetildiği soyut taban
   ↪ sınıf."""
3
4     def __init__(self, manager):
5         # Geri yönlü bağlantı: durum, manager'a "kendini
   ↪ kapat" diyebilir
6         self.manager = manager
7
8     def gir(self):
9         """Bu duruma yeni girildiğinde çağrılır
   ↪ (opsiyonel)."""
10        pass
11
12    def cik(self):
13        """Bu durumdan çıkılırken çağrılır (opsiyonel)."""
14        pass
15
16    def olaylari_isle(self, olaylar):
17        """pygame olay listesini bu durum işler."""
18        pass
19
20    def guncelle(self, dt):
21        """dt: bu kare ile önceki kare arası saniye."""
22        pass
23
24    def ciz(self, ekran):
25        """Ekrana çizim yapar."""
26        pass

```

Beş metoda dikkat. **gir** ve **cik** yaşam döngüsü kancalarıdır: durum belleğe geldiğinde hazırlanma (örneğin müzik başlat), bellekten çıkarken temizleme (müzik durdur). **olaylari_isle**, **guncelle** ve **ciz** ise klasik oyun döngüsü üçlemesidir.

■ Tanım: Soyut Sınıf

Soyut sınıf, doğrudan örneği oluşturulamayan, yalnızca türetilmek için var olan sınıftır. Türetilmiş sınıfların hangi metotlara sahip olması gerektiğini tarif eder. Python'da **abc.ABC**'den türetebilirsin, ama küçük oyun projelerinde sadece boş metotlar bırakmak çoğu zaman yeterlidir.

45.2.2 Manager'a Geri Yönlü Referans

self.manager'ı dikkat et. Bir durum, kendi içinde "geçiş zamanı geldi" kararını verdiğinde manager'a haber vermeli. Örneğin **MenuState** içindeki "Başla" butonuna basıldığında:

Python – Geçiş Çağrısı Örneği

```

1  # MenuState içinde:
2  def olaylari_isle(self, olaylar):
3      for olay in olaylar:
4          if olay.type == pygame.KEYDOWN and olay.key ==
           ↳ pygame.K_RETURN:
5              from play_state import PlayState
6              self.manager.degistir(PlayState(self.manager))

```

⚠ Dikkat: Döngüsel Import

Üstteki `from play_state import PlayState` satırı *fonksiyon içinde*. Eğer dosyanın başında importlasaydık ve `play_state.py` de `menu_state.py`'yi importlasaydı, döngüsel import (circular import) hatası alırdık. Geç import (lazy import) ile bu sorunu basitçe geçeriz.

45.2.3 Yığın Tabanlı StateManager

Manager'ın görevleri:

- Aktif durumu bilmek
- Yeni durumu *itmek* (push) veya *pop* etmek
- Mevcut durumu başkasıyla değiştirmek (replace)
- Olayları, güncellemeyi ve çizimi en üstteki duruma yönlendirmek

Python – state_manager.py - Yığın Yönetimi

```

1  class StateManager:
2      """Yığın tabanlı oyun durumu yöneticisi."""
3
4      def __init__(self):
5          self._yigin = [] # Aktif durumlar yığını
6
7      @property
8      def aktif(self):
9          """Yığının en üstündeki durum (veya None)."""
10         return self._yigin[-1] if self._yigin else None
11
12     def it(self, yeni_state):
13         """Mevcut durumu duraklat, yeni durumu üstüne
           ↳ ekle."""
14         self._yigin.append(yeni_state)
15         yeni_state.gir()
16
17     def pop(self):
18         """Üstteki durumu çıkar; altındaki tekrar aktif
           ↳ olur."""
19         if self._yigin:
20             ust = self._yigin.pop()

```

```

21         ust.cik()
22
23     def degistir(self, yeni_state):
24         """Üstteki durumu çıkarıp yerine yenisini koy."""
25         if self._yigin:
26             self.pop()
27         self.it(yeni_state)

```

Üç temel işlem: `it`, `pop`, `degistir`. `it` pause için doğal: oyun durumu yığında kalır, üzerine pause eklenir. `pop` pause'dan oyuna döndürür. `degistir` ise “oyundan menüye dön” gibi tek yönlü geçişler için.

45.2.4 Manager'ı Ana Döngüye Bağlamak

Şimdi `main.py` ne kadar sade olabilir, ona bak:

Python – main.py - State Machine ile

```

1  import pygame
2  from state_manager import StateManager
3  from menu_state import MenuState
4
5  pygame.init()
6  ekran = pygame.display.set_mode((800, 600))
7  saat = pygame.time.Clock()
8
9  manager = StateManager()
10 manager.it(MenuState(manager)) # İlk durum: menü
11
12 calistir = True
13 while calistir and manager.aktif:
14     dt = saat.tick(60) / 1000.0 # Saniye cinsinden delta
15     olaylar = pygame.event.get()
16     for olay in olaylar:
17         if olay.type == pygame.QUIT:
18             calistir = False
19
20     manager.aktif.olaylari_isle(olaylar)
21     manager.aktif.guncelle(dt)
22     manager.aktif.ciz(ekran)
23     pygame.display.flip()
24
25 pygame.quit()

```

Tamam — 18 satır. Bu döngüye yeni bir durum eklemek için *hiçbir satırını değiştirmen gerekmiyor*. Tüm değişiklikler kendi durum sınıflarının içinde olacak.

💡 İpucu: Yığın Boşaldığında Çık

`while calistir and manager.aktif` koşuluna dikkat. Yığın boşalırsa (son state pop edilirse) oyun otomatik kapanır. Bu, “Çıkış” butonu için `manager.pop()` çağrısı bile yeterli demektir; ekstra bayrak yönetmek gerekmez.

45.2.5 Ortak Aşağı Akış: `dt` Parametresi

`dt` (delta time) bu üniteden itibaren her güncelleme çağrısının parametresi olacak. Bölüm 11’de oyun mantığını “her kare 1/60 sn” varsayarak yazıyorduk; oyunun donduğu bir karede karakter daha az hareket ediyordu. `dt` kullanırsan bu sorun ortadan kalkar:

Python – `dt` ile Frame-Independent Hareket

```
1 # Sabit oran (eski yol):
2 self.x += 4      # her kare 4 piksel; FPS düşerse oyun
   ↳ yavaşlar
3
4 # dt ile (yeni yol):
5 HIZ = 240        # piksel/saniye
6 self.x += HIZ * dt  # 60 FPS'de yine 4, 30 FPS'de 8
   ↳ piksel
```

⚠ Dikkat: Tutarlı Birim

`dt`’yi saniye olarak kullanırsan tüm hızları *birim/saniye* olarak yazman gerekir. `ZIP_HIZI = -550` saniyede 550 piksel yukarı demektir; karelik birim değil. Yarı yolda ölçü değiştirme!

Sıradaki konuda dört somut durumu (`MenuState`, `PlayState`, `PauseState`, `GameOverState`) inşa edeceğiz.

45.3 Somut Oyun Durumları: Menu, Play, Pause, GameOver

İskelet hazır. Şimdi yığına itilecek somut oyun durumlarını yazıyoruz. Her birini ayrı bir Python dosyasında tutacağız: `menu_state.py`, `play_state.py`, `pause_state.py`, `game_over_state.py`. Bu, dosya başına yaklaşık 60–90 satır kod demek; her biri tek sorumluluğu olan, okunmaya değer küçük modüller.

45.3.1 MenuState: Oyunun Yüzü

Python – menu_state.py

[illegible]

Üç dikkat noktası: **gir** font ve seçenek listesini bu durumda inşa eder (her menüye girildiğinde tazelenir); **olaylari_isle** sadece menü tuşlarını dinler; **_secimi_uygula** geçişleri tek noktadan yönetir.

● Bilgi: Cycle Mod ile Seçim

`(self.secili + 1) % len(self.secenekler)` ifadesi en son seçenekeyken aşağı oka basınca ilk seçeneğe döner. Bu, dairesel menü davranışdır ve oyunlarda standarttır.

45.3.2 PlayState: Oyunun Eti

Python – play_state.py - Iskelet

```

1  import pygame
2  from state import State
3
4  class PlayState(State):
5      def gir(self):
6          # Bölüm 11'in oyuncu/harita/düşman kurulumu burada
7          from oyun_dunyasi import OyunDunyasi
8          self.dunya = OyunDunyasi()
9
10     def olaylari_isle(self, olaylar):
11         for olay in olaylar:
12             if olay.type == pygame.KEYDOWN:
13                 if olay.key == pygame.K_ESCAPE:
14                     from pause_state import PauseState
15                     self.manager.it(PauseState(self.manager,
16                                             snapshot=self))
17                 elif olay.key == pygame.K_SPACE:
18                     self.dunya.oyuncu.zipla_istegi()
19
20     def guncelle(self, dt):
21         tuslar = pygame.key.get_pressed()
22         self.dunya.guncelle(dt, tuslar)
23         if self.dunya.oyuncu.can <= 0:
24             from game_over_state import GameOverState
25             self.manager.degistir(GameOverState(self.manager,
26                                                 ger,
27                                                 skor=self
28                                                 .dunya
29                                                 .a.skor))
26
27     def ciz(self, ekran):
28         self.dunya.ciz(ekran)

```

PlayState, oyun mantığını **OyunDunyasi** adlı bir yardımcı sınıfa devreder (Bölüm 11'in mevcut oyuncu, düşman, harita, kamera kodu oraya taşınır). State

sınıfı kendisi sadece durum geçişlerini yönetir.

İki kritik geçiş:

- **ESC**: `it` ile pause yığına eklenir — oyun durumu *kaybedilmez*.
- **can <= 0**: `degistir` ile game over durumuyla yer değiştirir — artık geri dönemezsin.

💡 İpucu: Snapshot Geçirmek

`PauseState`'i oluştururken `snapshot=self` parametresi veriyoruz. Bu sayede pause ekranı altındaki oyun karesini bir kez çizip dondurabilir; sürekli güncellemek yerine. Detayı bir sonraki alt bölümde.

45.3.3 PauseState: Yığının Gücü

Python – pause_state.py

```

1  import pygame
2  from state import State
3
4  class PauseState(State):
5      def __init__(self, manager, snapshot=None):
6          super().__init__(manager)
7          self.snapshot = snapshot    # Altındaki PlayState
8
9      def gir(self):
10         self.font = pygame.font.Font(None, 48)
11         # Altındaki oyun karesini bir kez yakala
12         if self.snapshot is not None:
13             self.altyuz = pygame.Surface((800, 600))
14             self.snapshot.ciz(self.altyuz)
15             # Karartma efekti (yarı saydam siyah)
16             kararti = pygame.Surface((800, 600))
17             kararti.set_alpha(140)
18             kararti.fill((0, 0, 0))
19             self.altyuz.blit(kararti, (0, 0))
20
21         def olaylari_isle(self, olaylar):
22             for olay in olaylar:
23                 if olay.type == pygame.KEYDOWN and olay.key ==
24                     ↪ pygame.K_ESCAPE:
25                     self.manager.pop()    # Pause'dan çık, oyun
26                     ↪ devam eder
27                 elif olay.type == pygame.KEYDOWN and olay.key
28                     ↪ == pygame.K_q:
29                     from menu_state import MenuState
30                     self.manager._yigin.clear()    # Yığının
31                     ↪ boşalt
32                     self.manager.it(MenuState(self.manager))
33
34         def guncelle(self, dt):

```



```

18         self.manager.degistir(PlayState(self._
19             ↪ manager))
20     elif olay.key == pygame.K_ESCAPE:
21         from menu_state import MenuState
22         self.manager.degistir(MenuState(self._
23             ↪ manager))
24
25     def guncelle(self, dt):
26         pass
27
28     def ciz(self, ekran):
29         ekran.fill((30, 0, 0))
30         baslik = self.font_b.render("OYUN BITTI", True,
31             ↪ (255, 80, 80))
32         ekran.blit(baslik, baslik.get_rect(center=(400,
33             ↪ 200)))
34         skor_metin = self.font_o.render(
35             ↪ f"Skor: {self.skor}", True, (255, 215, 0))
36         ekran.blit(skor_metin,
37             ↪ skor_metin.get_rect(center=(400, 300)))
38         ipucu = self.font_o.render(
39             ↪ "ENTER: Tekrar   ESC: Menu", True, (200, 200,
40             ↪ 200))
41         ekran.blit(ipucu, ipucu.get_rect(center=(400,
42             ↪ 400)))

```

Game over basit: bir başlık, skor bilgisi, iki tuş seçeneği. Burada `degistir` kullanıyoruz çünkü game over'dan oyuna geri dönüş yok — *yeni* bir oyun başlatıyoruz.

● Bilgi: Skoru Geçirme

`GameOverState` kurucusunda `skor` parametresi var. `PlayState`, oyuncusu öldüğünde `GameOverState(self.manager, skor=self.dunya.skor)` ile skoru aktarır. Bu, durumlar arası veri akışının tipik desendir: *constructor parametreleri*.

45.3.5 Toparlama

Beş dosya yazdık: `state.py`, `state_manager.py`, `menu_state.py`, `play_state.py`, `pause_state.py`, `game_over_state.py` + ana `main.py`. Toplam yaklaşık 250 satır. Bunun karşısında `if-elif` versiyonu en az 600 satır olurdu — ve okumaya çalışsaydın bir saatte gözünü kaybederdin.

Sıradaki konuda durumlar arası geçişlerin daha karmaşık varyantlarını göreceğiz: dialog ve inventory state'leri, transition efektleri, durum makinesinin durum makinesi.

45.4 State Geçişleri: Push, Pop, Replace ve Transition Efektleri

Dört durumumuz hazır. Şimdi geçişlerin inceliklerini öğreniyoruz: hangi geçiş hangi metodu kullanır, üst üste yığılmış birden fazla durum nasıl çizilir (transparent overlay), bir durumdan diğerine geçerken ekrana *fade-in/out* gibi geçiş efektleri eklemek.

45.4.1 Üç Geçiş Türünün Hatırlanması

Tablo 45.1: Üç state geçiş türünün karşılaştırması

İşlem	Etki	Tipik Kullanım
<code>it(yeni)</code>	Üstte yeni durum, alttaki bekler	Pause, dialog, inventory
<code>pop()</code>	Üstteki kapanır, alttaki tekrar aktif	Pause'dan oyuna dönüş
<code>degistir(yeni)</code>	Üstteki kapanır, yenisi gelir	Menü → Oyun, Game Over → Menü

Doğru aracı seçmek hem mantık hem de bellek açısından önemli. Yanlış araç seçersen ya state'ler birikmeye başlar (yığın şişer) ya da geri döndüğünde devam etmek istediğin yerden devam edemezsin.

⚠ Dikkat: Pause İçin **degistir** Kullanma

`manager.degistir(PauseState(self.manager))` yazarsan altındaki oyun durumu *yok olur*. ESC ile pause'dan çıktığında oyun yeniden başlamak zorunda kalır. Pause için her zaman `it` (push) kullanılmalı.

45.4.2 Şeffaf Üstüne Çizim (Transparent Overlay)

Yığında birden fazla durum varken — üstteki olay alır ama altındaki de *görünebilir* olmalı. Pause ekranı bunun klasik örneği: arkadaki donmuş oyun karesi görünüyor.

İki yaklaşım vardır:

- 1. Snapshot yaklaşımı:** Üstteki state `gir` anında altındaki state'i bir surface'e çizer ve oradan kullanır. Konu 3'te `PauseState` bunu yapıyordu.
- 2. Şeffaf çizim yaklaşımı:** Manager her karede yığının *tümünü* alttan üste sırayla çizer. Üst state'lerin `ciz` metodu yarı saydam yüzeylere çizim yapar.

Python – Manager'a Şeffaf Çizim Eklentisi

```
1 class StateManager:
2     # ... mevcut kod ...
3
```

```

4     def ciz_hepsini(self, ekran):
5         """Yığındaki tüm state'leri alttan üste çizer."""
6         for state in self._yigin:
7             state.ciz(ekran)

```

💡 İpucu: Hangi Yaklaşım?

Snapshot yaklaşımı daha verimli (alttaki kez değil her kare çizilmez) ama gerçek zamanlı animasyonu pause altında durdurur. Şeffaf çizim yaklaşımı esnek ama maliyetlidir. Pause için snapshot, dialog sistemi (oyun arkada devam ederken konuşma) için şeffaf çizim tercih edilir.

45.4.3 Fade Geçiş Efektli

Profesyonel oyunlarda durumlar arası geçişler ani olmaz. Menüden oyuna geçerken ekran kararır, sonra oyun belirir. Bu, küçük bir `FadeTransitionState` ile sağlanır — bir sahneden diğerine köprü olur.

Python – fade_state.py - Fade Geçişli

```

1  import pygame
2  from state import State
3
4  class FadeTransitionState(State):
5      """Bir state'ten diğerine fade-out + fade-in
6         ↳ geçişi."""
7
8      def __init__(self, manager, hedef_state, sure=0.5):
9          super().__init__(manager)
10         self.hedef = hedef_state
11         self.sure = sure          # Toplam saniye
12         self.zaman = 0
13         self.faz = "out"         # "out" sonra "in"
14
15     def gir(self):
16         self.altyuz = pygame.display.get_surface().copy()
17
18     def olaylari_isle(self, olaylar):
19         pass
20
21     def guncelle(self, dt):
22         self.zaman += dt
23         if self.faz == "out" and self.zaman >= self.sure /
24             ↳ 2:
25             self.faz = "in"
26             self.manager.degistir(self.hedef)    # Bu,
27             ↳ hedef state'i aktif eder
28             # Ama biz hala üstteyiz --- hayır,
29             ↳ değiştirildik. Geçici fix:

```

```

26
27     def ciz(self, ekran):
28         ekran.blit(self.altyuz, (0, 0))
29         ilerleme = self.zaman / (self.sure / 2)
30         alpha = int(255 * min(1.0, ilerleme))
31         kararti = pygame.Surface(ekran.get_size())
32         kararti.set_alpha(alpha)
33         kararti.fill((0, 0, 0))
34         ekran.blit(kararti, (0, 0))

```

● Bilgi: Tasarım İpucu

Yukarıdaki taslak fade-out kısmını gösterir. Tam fade-in/out implementasyonu için manager'a iki aşamalı geçiş yapan ayrı bir mekanizma eklemek (veya hedef state içine fade-in mantığını gömmek) gerekir. Lab 12'deki Çalışma 1'de bunun pratik bir versiyonunu yazacaksın.

45.4.4 Dialog ve Inventory için Yığın Kullanımı

Pause ekranı yığın deseninin yalnızca bir uygulamasıdır. Daha çok örnek:

- **Dialog kutusu:** Oyuncu bir NPC'yle konuşunca üstüne bir `DialogState` itilir. Oyun devam etmez, konuşma biter, `pop` edilir.
- **Envanter:** I tuşuyla `InventoryState` itilir. Oyuncu eşyaya bakar, eşya kullanır veya ESC ile çıkar.
- **Cutscene:** Belirli olaylar tetiklenince `CutsceneState` itilir; sinematik bittiğinde otomatik `pop`.
- **Onay penceresi:** "Çıkmak istediğine emin misin?" sorusu için `ConfirmDialogState`.

Hepsi tek bir desene çözülür: `it` ile aç, `pop` ile kapat. Manager'ın yığın deseni, bu kullanım çeşitliliğini hiçbir ek kod olmadan destekler.

45.4.5 Olay Tüketimi (Event Consumption)

Yığında birden fazla state olduğunda, olayları sadece üstteki durumun alması çoğu zaman doğrudur. Ama her zaman değil. Örneğin pause ekranındayken **F11** (tam ekran) tuşuna basılınca tüm uygulamanın bunu yakalaması gerekir, sadece üstteki state'in değil.

Python – Olay Tüketimi - Gelişmiş Manager

```

1  def olaylari_dagit(self, olaylar):
2      """Olayları üstten alta dağıtır; durdurulanlar alta
   ↪ gitmez."""
3      for state in reversed(self._yigin):
4          kalan = state.olaylari_isle(olaylar)
5          if kalan is None:
6              return # Bu state olayları tüketti
7          olaylar = kalan

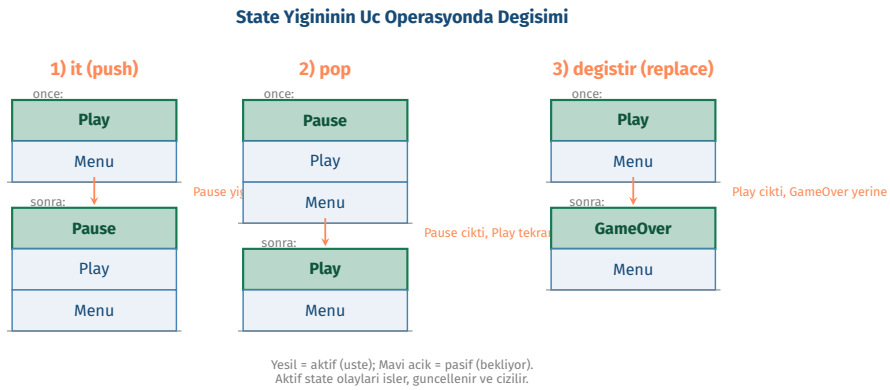
```


State'ler artık `olaylari\_isle`'dan ya `None` ("hepsini yedim") ya da kullanmadığı olayları ("alta geçirelim") döndürür.

⚠ Dikkat: Karmaşıklığı Erken Ekleme

Olay tüketimi mantığı küçük oyunlarda gereksiz karmaşıklık. İhtiyaç olmadan eklemeyin — "üstteki tüm olayları alır" kuralı çoğu oyunda yeterlidir. Önce basit kuralla başla, gerekirse karmaşıklığı sonra ekle.

45.4.6 Mimari Özet



Şekil 45.2: State yığının üç farklı operasyonda nasıl değiştiği. Üstteki katman daima aktif olandır.

Sıradaki üniteye geçmeden önce şu üç önemli noktayı zihninde tut:

1. Her state *kendi sınıfıdır*; bir dosyada tüm durumlarınca paylaşılan kod *olmamalı*.
2. Manager bir *yığındır*; üstteki aktif, alttakiler bekliyor.
3. Geçiş aracı seçimi mantığa bağlıdır: kalıcı dönüş için **it** + **pop**, tek yönlü değişim için **degistir**.

45.5 Sonuç

Bu ünite de oyun mimarisinin omurgasını değiştirdik. Tek bir devasa `main` döngüsü yerine, her ekranı kendi sınıfı olan ve aralarında temiz geçişlerle bağlanan bir state machine inşa ettik. Bölüm 11'den gelen Zıplayan Macera v3 oyunu artık bir `PlayState`'in içinde; etrafında `MenuState`, `PauseState` ve `GameOverState` ona kucak açıyor.

45.5.1 Anahtar Kavramlar

Tablo 45.2: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
State machine	Sistemin aynı anda tek bir durumda olduğu ve durumlar arası geçişlerin tetiklendiği desen
Soyut State	Tüm somut durumların türetildiği taban sınıf; gir , cik , olaylari_isle , guncelle , ciz arayüzü
StateManager	Yığın tabanlı durum yöneticisi; it , pop , degistir işlemleri
gir / cik	Yaşam döngüsü kancaları; durum aktif olunca / pasif olunca tetiklenir (müzik, font, vb.)
dt (delta time)	Bu kareyle önceki kare arası saniye; frame-bağımsız hareket için zorunlu
Push / Pop	Yığına yeni durum ekleme / üstteki durumu çıkarma
Replace	Üstteki durumu yenisiyle değiştirme
Snapshot	Pause sırasında alttaki ekranı dondurmak için surface yakalama
Olay tüketimi	Bir state'in olayı işleyip "alta geçirmem" diyebilmesi (gelişmiş)

45.5.2 Önemli Komutlar

Tablo 45.3: Ünite 1 – Temel komutlar

Komut	Açıklama
<code>class State:</code>	Soyut taban sınıf; alt sınıflar override eder
<code>manager.it(state)</code>	State yığına eklenir, gir() çağrılır
<code>manager.pop()</code>	Üstteki state çıkar, cik() çağrılır
<code>manager.degistir(state)</code>	pop + it ; eski yok olur
<code>manager.aktif</code>	Yığının en üstündeki durum (veya None)
<code>from x import Y</code>	Geç import (lazy); döngüsel import'tan kaçınma
<code>saat.tick(60) / 1000.0</code>	dt 'yi saniye cinsinden hesaplama
<code>pygame.display.get_surface()</code>	Aktif ekran yüzeyini al
<code>kararti.set_alpha(140)</code>	Yarı saydam karartma efekti

45.5.3 Bir Sonraki Ünite: Dosya İşlemleri

State machine, oyunun *anlık* durumunu yönetir — aktif ekran, oyuncunun konumu, mevcut skor. Ama oyunu kapattığında bu bilgilerin hepsi uçuyor. Sıradaki ünite, Python'da dosya yazma ve okumayı öğrenerek bu bilgileri *kalıcı* hale getirme yolculuğuna çıkıyoruz:

- Metin ve binary dosya işlemlerinin temelleri
- JSON formatı: ne zaman, neden ve nasıl
- `pickle` ile karmaşık Python nesnelerini diske yazma
- Hangi koşulda hangi formatın seçileceği

Bu bilgiler Ünite 3'te `SaveManager` sınıfının inşası için gerekli temel olacak. Ardından Ünite 4'te aynı bilgileri ayar sistemine uygulayacağız.

💡 İpucu: State Machine'i Genişletme

Bu kitap dört durumla yetiniyor (`Menu`, `Play`, `Pause`, `GameOver`). Sen kendi oyununa şu durumları da ekleyebilirsin:

- `LoadState`: Slot listesi, kayıt seçme ekranı
- `InventoryState`: Eşya yönetimi
- `SettingsState`: Ses, kontrol, ekran ayarları (Ünite 4'te yapacağız)
- `CreditsState`: Yapımcılar listesi

Hepsi aynı `State` arayüzünü uygular; manager hiçbir değişiklik yapmadan onları yığına itip pop edebilir. İşte mimarinin gücü budur.

“Bir oyunun belleği yoksa, oyuncunun içindeki “ben başardım” hissi bir saatten uzun yaşayamaz. Diske yazılan tek bir satır bile, yarın tekrar oyuna dönmenin sebebi olabilir.”

— ANONİM OYUN TASARIMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Python’da dosya açma, okuma ve yazmanın temelleri (`open`, `with`)
- Metin dosyaları ile binary dosyalar arasındaki fark
- JSON formatı: insan okur, makine okur; `json.load` ve `json.loads`
- JSON’a Python sözlüğü ve listesi yazma; `json.dump` ve `json.dumps`
- `pickle` ile karmaşık Python nesnelerini diske serileştirme ve geri okuma
- Hangi durumda JSON, hangi durumda pickle tercih etmen gerektiği

46.1 Python’da Dosya Açma, Okuma, Yazma

State machine sayesinde oyunu duraklatabiliyor, menüye dönebiliyor, yeniden başlayabiliyorsun. Ama oyunu *kapatıp* açtığında her şey sıfırdan başlıyor — skor, level, toplanmış altın, geçilmiş bölümler... Hepsi RAM’de yaşıyordu, oyun kapanınca uçtu. Bu üniteden itibaren diske yazıyoruz.

46.1.1 `open` Fonksiyonu ve Mod Karakteri

Python’da bir dosya açmanın temel yolu `open`’dır:

Python – Temel Dosya Açma

```

1 dosya = open("kayit.txt", "w")      # yazma için aç
2 dosya.write("Merhaba diske!\n")
3 dosya.close()                      # KAPAT --- unutursan
   ↳ tehlike

```

İkinci parametre *mod* karakteridir. En çok kullanılan altı tane:

Tablo 46.1: `open()` mod karakterleri

Mod	Anlamı	Kullanım
"r"	Okuma (varsayılan); dosya yoksa hata	Kayıt yükleme
"w"	Yazma; dosya varsa SİLER, yoksa oluşturur	Yeni kayıt yazma
"a"	Sona ekleme (append); dosya yoksa oluşturur	Log dosyası
"x"	Yalnızca yeni dosya oluşturma; varsa hata	Çakışma korumalı yazma
"rb" / "wb"	Binary modda okuma / yazma	Pickle, ses, görsel
"r+" / "w+"	Hem okuma hem yazma (üzerine)	Nadir kullanılır

⚠ Dikkat: "w" Modu Mevcut Dosyayı SİLER

`open("kayit.txt", "w")` eğer `kayit.txt` zaten varsa *anında ve geri alınamaz şekilde* içeriğini boşaltır. “Aslında okumak istemiştım” diyen ve W harfine yanlış basan çok sayıda kişi vardır. Test sırasında "x" (yeni dosya) kullanmak güvenli alışkanlıktır.

46.1.2 with Bloğu ile Otomatik Kapatma

`open`'ı her seferinde `close` ile eşleştirmek can sıkıcı ve unutulabilir. Ortada bir hata olursa `close` çağrılmaz, dosya açık kalır, veriler diske aktarılmaz. Çözüm: **with** bloğu.

Python – with: Otomatik Kapatma

```

1 with open("kayit.txt", "w") as dosya:
2     dosya.write("Bu satır diske yazılır.\n")
3     dosya.write("Bu da yazılır.\n")
4 # Buraya geldiğimizde dosya KAPATILMIŞTIR --- istisna olsa
   ↳ bile.

```

with bloğundan çıktığında (normal veya istisna ile) `dosya.__exit__` otomatik çağrılır ve `close` işlenir. Bu, Pythonic dosya işleminin standart yoludur.

■ Tanım: Context Manager

`with` ifadesinin birlikte çalıştığı nesnelere *context manager* denir. Bu nesneler `__enter__` ve `__exit__` metodlarıyla “açılış-kapanış” ritüellerini otomatikleştirir. Dosyalar, kilitler, veritabanı bağlantıları, ağ soketleri — hepsi context manager’dır.

46.1.3 Okuma Yöntemleri

Bir dosyayı okumanın üç yolu vardır:

Python – Üç Okuma Yöntemi

```

1  # 1) Tek seferde tümünü oku (küçük dosyalar için ideal)
2  with open("kayit.txt", "r") as f:
3      icerik = f.read()          # str
4  print(icerik)
5
6  # 2) Satır satır (büyük dosyalar için)
7  with open("loglar.txt", "r") as f:
8      for satir in f:            # f bir iteratör --- bellek
9          dostu
10             print(satir.strip())
11
12 # 3) Tüm satırları liste olarak al
13 with open("isimler.txt", "r") as f:
14     isimler = f.readlines()    # ["Ali\n", "Ayse\n", ...]
```

💡 İpucu: Hangi Yöntem?

Oyun kayıt dosyaları nadiren 100 KB’tan büyük olur; `read()` güvenle kullanılabilir. Log dosyaları binlerce satır olabileceği için `for satir in f` yöntemi tercih edilir. `readlines()` liste döndürür — bellek dostu değildir, ama tüm satırlara `index` ile erişmek istediğinde elverişlidir.

46.1.4 Yazma Yöntemleri

Python – İki Yazma Yöntemi

```

1  # 1) write() --- tek str
2  with open("kayit.txt", "w") as f:
3      f.write("Skor: 1500\n")
4      f.write("Bolum: 3\n")
5
6  # 2) writelines() --- liste/iteratör; otomatik \n EKLEMEZ
7  satirlar = ["Skor: 1500\n", "Bolum: 3\n", "Can: 5\n"]
8  with open("kayit.txt", "w") as f:
9      f.writelines(satirlar)
```

⚠ Dikkat: `writelines` Yeni Satır Eklemez

`writelines(["a", "b", "c"])` tek satıra "abc" yazar. Liste elemanlarına `\n` eklemek senin sorumluluğun. İsim çağrıştırıcı: `writelines` = "çoklu `write`" demektir, "çoklu satır yazma" değil.

46.1.5 Metin vs Binary Dosyalar

Mod karakterindeki ekstra "b" harfi büyük fark yaratır:

- "r" / "w" (**metin modu**): Veriler `str` olarak okunur/yazılır. Sistem işletim sistemine göre satır sonu karakteri otomatik dönüştürülür (Windows'ta `\r\n`, Linux/macOS'ta `\n`). UTF-8 kodlama varsayılır.
- "rb" / "wb" (**binary modu**): Veriler `bytes` olarak okunur/yazılır. Hiçbir dönüşüm yapılmaz. Görseller, ses dosyaları, pickle çıktıları binary modda işlenir.

Python – Binary Mod Örneği

```
1 # pygame ile bir görseli binary olarak aç:
2 with open("oyuncu.png", "rb") as f:
3     bytes_icerik = f.read()
4     print(f"Dosya boyutu: {len(bytes_icerik)} bayt")
5     print(f"İlk 8 bayt: {bytes_icerik[:8]}")
6 # PNG dosyaları her zaman b'\x89PNG\r\n\x1a\n' ile başlar
```

● Bilgi: Encoding Belirleme

Türkçe karakter içeren metin dosyalarında `open(..., "r", encoding="utf-8")` açıkça belirtmek iyi alışkanlıktır. Windows'ta varsayılan kodlama `cp1254` olabilir ve Türkçe karakterler bozulabilir.

46.1.6 Yol (Path) Yönetimi: `pathlib`

String olarak yol birleştirme ("`klasör`" + "/" + "`dosya.txt`") işletim sistemleri arasında taşınabilir değil. Modern Python'da `pathlib.Path` kullanılır:

Python – `pathlib` Kullanımı

```
1 from pathlib import Path
2
3 # Kullanıcının ev dizini altında oyun klasörü
4 oyun_dizini = Path.home() / ".ziplayan_macera"
5 oyun_dizini.mkdir(exist_ok=True) # Yoksa oluştur
6
7 kayit_yolu = oyun_dizini / "save_slot1.json"
8 print(kayit_yolu)
```

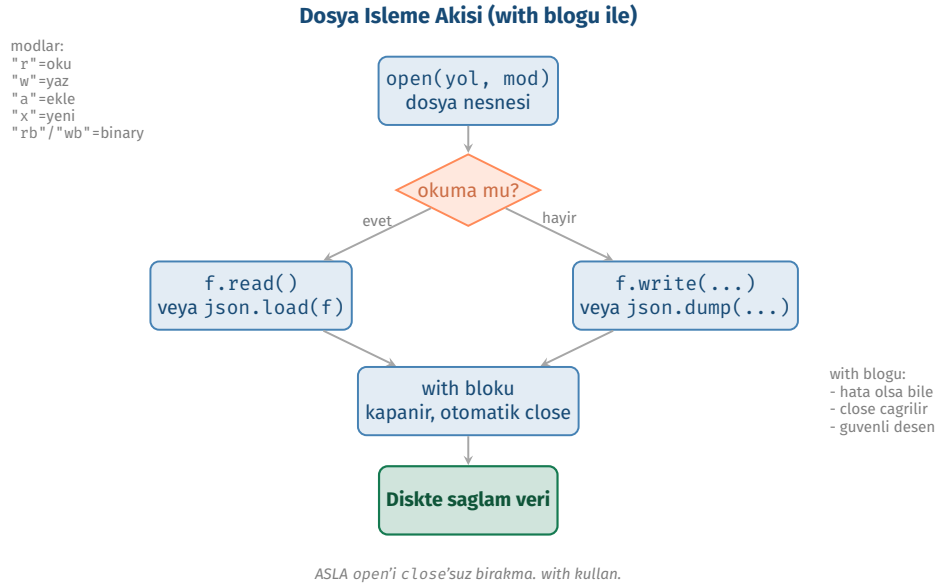
```

9  # Linux/macOS:
   ↪ /home/kullanici/.ziplayan_macera/save_slot1.json
10 # Windows:
   ↪ C:\Users\kullanici\.ziplayan_macera\save_slot1.json
11
12 # Dosya var mı?
13 if kayit_yolu.exists():
14     print("Kayıt bulundu!")
15
16 # Dosyayı oku
17 metin = kayit_yolu.read_text(encoding="utf-8")

```

💡 İpucu: Path Daima!

`pathlib` 2014'ten beri standart kütüphanede. `os.path` ile string manipülasyonu yerine her yeni projede `Path` kullan. Daha kısa, daha okunabilir, hatalara daha dirençli.



Şekil 46.1: Bir dosyanın açılma, okuma/yazma ve kapatma akışı. `with` bloğu kapanış adımını otomatize eder.

46.1.7 Toparlama

Bu konuda dosya I/O'nun temellerini ele aldık. Sıradaki konu metin yerine *yapılandırılmış* veri saklamak için JSON formatına odaklanacak — oyun kayıtları, ayarlar, lider tablosu gibi yapıli verileri JSON ile çok daha kolay yönetiriz.

▲ Dikkat: Dosya İşlemi Yavaştır

Her oyun karesinde dosya açıp kapatma yapma. Disk işlemleri RAM'den yüzlerce kat yavaştır. Yalnızca *olay* anlarında yaz: oyuncu kaydetme tuşuna bastığında, level değiştiğinde, oyun kapanırken. Sürekli yazma istiyorsan *async* (arka plan thread) gerekir; bu kitabın kapsamı dışında.

46.2 JSON Formatı ve Diskten Okuma

Düz metin dosyalar bilgi tutar, ama *yapı* tutmazlar. “Skor: 1500” satırını okuyup tekrar 1500 sayısına çevirmek için `int(satir.split(":")[1].strip())` yazman gerekir — ve bu sadece bir alan için. Üç oyuncu, beş seviye, on envanter eşyası tutmak istesen kendi metin formatını yazmaya başlarsın — ki bu yol kötü biter. *JSON* (JavaScript Object Notation) bunun standart çözümüdür.

46.2.1 JSON Nedir?

■ Tanım: JSON

JSON (JavaScript Object Notation), insan tarafından okunabilir ve makineler tarafından kolayca işlenebilen bir veri değişim formatıdır. JavaScript nesne sözdizimine dayanır ama dilden bağımsızdır. Python, C++, Rust, Go, JavaScript — hepsi JSON okuyup yazabilir.

JSON yalnızca *altı* türden veri saklayabilir:

Tablo 46.2: JSON ile Python türleri arasındaki eşleşme

JSON	Python	Örnek
object	dict	<code>{"isim": "Ayse"}</code>
array	list	<code>[1, 2, 3]</code>
string	str	<code>"merhaba"</code>
number	int veya float	<code>42, 3.14</code>
boolean	bool	<code>true, false</code>
null	None	<code>null</code>

▲ Dikkat: JSON'ın Tutamadıkları

JSON şu türleri *doğrudan* desteklemez: *tuple* (liste olarak yazılır, tuple olarak değil), *set*, *datetime*, *bytes*, kendi tanımladığın sınıflar. Bunları JSON'a dönüştürmek için manuel olarak *dict/list/str*'e çevirmek gerekir. Pickle bu sınırlamayı aşar (sıradaki konuda).

46.2.2 Bir Oyun Kaydı JSON'da Nasıl Görünür?

Tipik bir kayıt dosyası:

Python – Örnek Save JSON Dosyası

```

1  # kayıt.json içeriği:
2  """
3  {
4      "version": 1,
5      "oyuncu": {
6          "isim": "Ayse",
7          "x": 320,
8          "y": 480,
9          "can": 5,
10         "max_can": 5,
11         "altin": 1250
12     },
13     "seviye": "orman_2",
14     "kesfedilen_bolumler": ["orman_1", "orman_2"],
15     "envanter": [
16         {"ad": "Anahtar", "miktar": 1},
17         {"ad": "Iksir", "miktar": 3}
18     ],
19     "oyun_suresi": 1820.5,
20     "ayarlar_aktif": true
21 }
22 """

```

Bu dosyayı bir not defterinde aç — olduğu gibi okuyabilir, elinle düzenleyebilirsiniz. Bu, JSON'ın baş özelliğidir: hem program hem insan dostu.

46.2.3 json Modülü ile Okuma

Python'da JSON için standart modül `json`'dır. Dört temel fonksiyonu vardır; ikisi okuma, ikisi yazma:

Tablo 46.3: `json` modülü temel fonksiyonları

Fonksiyon	İşlevi
<code>json.load(f)</code>	Açık dosya nesnesinden okur, Python objesi döndürür
<code>json.loads(s)</code>	String'den okur (l-o-a-d-S, "string")
<code>json.dump(o, f)</code>	Python objesini dosyaya yazar
<code>json.dumps(o)</code>	Python objesini string'e dönüştürür

İlk okuma örneği:

Python – JSON Dosyasından Okuma

```

1  import json
2
3  with open("kayit.json", "r", encoding="utf-8") as f:
4      veri = json.load(f)

```

```

5
6 # veri artık bir dict
7 print(veri["oyuncu"]["isim"])      # Ayse
8 print(veri["oyuncu"]["altin"])     # 1250
9 print(veri["envanter"][0]["ad"])   # Anahtar
10 print(len(veri["envanter"]))       # 2

```

İki nokta:

- `json.load(f)` dosya nesnesini alır, yolu değil. Yolu doğrudan vermek istersen `json.loads(Path("kayit.json").read_text(encoding="utf-8"))` yazabilirsin.
- Türkçe karakter okurken `encoding="utf-8"` ZORUNLU. Aksi halde Windows'ta hatalar yaşarsın.

46.2.4 String'den Okuma: `json.loads`

Bazı durumlarda JSON içeriği bir string'in içindedir — ağ yanıtı, gömülü konfigürasyon, test verisi. `json.loads` (load-from-string) bu durumda kullanılır:

Python – String'den JSON Okuma

```

1 import json
2
3 ham = '{"isim": "Mehmet", "puan": 850}'
4 veri = json.loads(ham)
5 print(veri["isim"])      # Mehmet
6 print(veri["puan"])      # 850
7 print(type(veri))        # <class 'dict'>

```

`loads`'taki son S harfi *string* demektir; kafa karışıklığı yapan tek nokta budur. Hatırlama yolu: “`loads` = load string sonu *s*”.

46.2.5 Hata Yakalama: Bozuk JSON

Diskteki dosya bozulmuş olabilir — elle düzenlenirken bir virgül unutulmuştur, veya yarısı yazılırken işletim sistemi çökmüştür. `json.load` bu durumda `json.JSONDecodeError` fırlatır:

Python – Güvenli Okuma

```

1 import json
2 from pathlib import Path
3
4 def kayit_yukle(yol):
5     """Hata güvenli kayıt yükleyici."""
6     yol = Path(yol)
7     if not yol.exists():
8         return None      # Kayıt yok --- yeni oyun
9     try:
10        with open(yol, "r", encoding="utf-8") as f:

```

```

11         return json.load(f)
12     except json.JSONDecodeError as e:
13         print(f"[UYARI] Kayıt dosyası bozuk: {e}")
14         # Yedek dosya varsa onu dene
15         yedek = yol.with_suffix(".json.bak")
16         if yedek.exists():
17             with open(yedek, "r", encoding="utf-8") as f:
18                 return json.load(f)
19     return None

```

💡 İpucu: Yedek Dosya

Her kaydetmeden önce mevcut kayıt dosyasını **.bak** uzantısıyla yedeklemek profesyonel bir alışkanlıktır. Diske yazma yarıda kalırsa yedek hala sağlamdır. Ünite 3'te **SaveManager**'a bunu ekleyeceğiz.

46.2.6 Türleri Anlama: Sözlüklerden Sınıflara

`json.load` sana *sözlük* verir, *sınıf* değil. Yani:

Python – Sözlük vs Sınıf

```

1 import json
2
3 with open("kayit.json", "r") as f:
4     veri = json.load(f)
5
6 # Bu çalışır:
7 print(veri["oyuncu"]["x"])
8
9 # Bu ÇALIŞMAZ --- attribute erişimi yok:
10 # print(veri.oyuncu.x) # AttributeError!

```

Eğer sözlük yerine **Oyuncu** sınıfı nesnesi istersen, sözlüğü manuel olarak sınıfa dönüştürmen gerekir:

Python – Sözlükten Sınıfa Dönüşüm

```

1 class Oyuncu:
2     def __init__(self, x, y, can, altin):
3         self.x = x
4         self.y = y
5         self.can = can
6         self.altin = altin
7
8     @classmethod
9     def from_dict(cls, d):
10         return cls(x=d["x"], y=d["y"],
11                    can=d["can"], altin=d["altin"])

```

```

12
13 # Kullanım:
14 veri = json.load(open("kayit.json"))
15 oyuncu = Oyuncu.from_dict(veri["oyuncu"])
16 print(oyuncu.altin) # Artık attribute erişimi çalışır

```

Bu `from\_dict` deseni Ünite 3'ün omurgası olacak. Sınıfla sözlük arasındaki köprüyü iki yönlü kuracağız: `to\_dict` ile sınıfı kaydetmek, `from\_dict` ile geri yüklemek.

Sıradaki konu: aynı modülün diğer iki fonksiyonu — `json.dump` ve `json.dumps` ile diske yazma.

46.3 JSON ile Diske Yazma

Okumayı öğrendik. Şimdi yazmaya. `json.dump` ve `json.dumps` ile Python sözlüğünü diske veya string'e dönüştüreceğiz, formatlama seçeneklerini öğreneceğiz, atomik yazma ile veri kaybına karşı korunacağız.

46.3.1 İlk Yazma

Python – Diske JSON Yazma

```

1 import json
2
3 veri = {
4     "version": 1,
5     "oyuncu": {"isim": "Ayse", "altin": 1250, "can": 5},
6     "seviye": "orman_2"
7 }
8
9 with open("kayit.json", "w", encoding="utf-8") as f:
10     json.dump(veri, f)

```

Ortaya çıkan dosya tek satırlık: `{"version": 1, "oyuncu": {...}, "seviye": "orman_2"}`. Programlar için yeterli, ama bir insan açıp inceleyemez.

46.3.2 Güzel Formatlama: `indent`

`indent` parametresi ile JSON'a girinti ekleyebilirsin. İnsanların okuyabilmesi için bunu *her zaman* yap:

Python – Girintili JSON

```

1 import json
2
3 veri = {"oyuncu": {"isim": "Ayse", "altin": 1250},
4         "seviye": "orman_2"}
5

```

```

6 with open("kayit.json", "w", encoding="utf-8") as f:
7     json.dump(veri, f, indent=2)

```

Sonuç:

```

{
  "oyuncu": {
    "isim": "Ayse",
    "altin": 1250
  },
  "seviye": "orman_2"
}

```

💡 İpucu: Standart Girinti: 2 veya 4

2 ya da 4 boşluk en yaygın seçimdir. Tablar (`indent="\t"`) da çalışır ama editör ayarlarına duyarlıdır. Saf 2 boşlukla kal.

46.3.3 Türkçe Karakterler: `ensure_ascii`

Varsayılan davranış: Türkçe karakterleri `\u` escape'ine çevirir. Yani "İ" yerine `"\u0130"` yazar. İnsana okunabilirlik açısından korkunç.

Python – Türkçe Karakter Koruma

```

1 import json
2
3 veri = {"isim": "Şükrü", "sehir": "İzmir"}
4
5 # Varsayılan (kötü):
6 json.dumps(veri)
7 # {"isim": "\u015e\u00fckr\u00fc", "sehir": "\u0130zmir"}
8
9 # ensure_ascii=False ile (iyi):
10 json.dumps(veri, ensure_ascii=False)
11 # {"isim": "Şükrü", "sehir": "İzmir"}

```

⚠ Dikkat: Her Yerde `ensure_ascii=False`

Türkçe içerik yazıyorsan `ensure_ascii=False` olmazsa olmazdır. Bunu unutursan kayıt dosyaları okunamaz hale gelir; oyuncu adında "Şükrü" yerine `\u015e\u00fckr\u00fc` gören oyuncu sinirlenecektir.

46.3.4 Tam Doğru Yazma Şablonu

Buradan itibaren her JSON yazma çağrın bu şablonu izlemeli:

Python – Standart JSON Yazma Deseni

```

1 import json
2 from pathlib import Path
3
4 def json_yaz(veri, yol):
5     """Standart, güvenli JSON yazımı."""
6     yol = Path(yol)
7     yol.parent.mkdir(parents=True, exist_ok=True)
8     with open(yol, "w", encoding="utf-8") as f:
9         json.dump(
10             veri,
11             f,
12             indent=2,           # İnsan okur
13             ensure_ascii=False, # Türkçe korunur
14             sort_keys=True,     # Anahtarlar alfabetik
15                                 ↪ (diff dostu)
16         )

```

`sort_keys=True` dosyaları *deterministik* yapar — aynı veri her seferinde aynı sırada yazılır. Bu, git diff'lerini okumayı ve bakım yapmayı kolaylaştırır.

46.3.5 Atomik Yazma: Veri Kaybına Karşı

Bir kaydetme işlemi yarıda kesilirse (oyun çökerse, elektrik giderse) eski kayıt yarısı yeni, yarısı eski olabilir — yani ne işe yarayan bir kayıt, ne de hiç. Çözüm: önce *geçici* dosyaya yaz, sonra atomik olarak rename et.

Python – Atomik JSON Yazma

```

1 import json
2 import os
3 from pathlib import Path
4
5 def atomik_json_yaz(veri, yol):
6     """Yarıda kesintiye karşı dirençli yazma."""
7     yol = Path(yol)
8     yol.parent.mkdir(parents=True, exist_ok=True)
9     tmp = yol.with_suffix(yol.suffix + ".tmp")
10
11     # Önce geçici dosyaya yaz
12     with open(tmp, "w", encoding="utf-8") as f:
13         json.dump(veri, f, indent=2, ensure_ascii=False)
14         f.flush()
15         os.fsync(f.fileno()) # OS önbelleğini diske
16                               ↪ zorla
17
18     # Sonra atomik rename --- ya tamamen başarılı, ya hiç
19     ↪ olmamış
20     tmp.replace(yol)

```

■ Tanım: Atomik İşlem

Atomik işlem, kesintiye uğrayamayacak bir bütündür: ya tamamen tamamlanır, ya hiç başlamamış sayılır. Modern dosya sistemlerinde `rename` (`Path.replace`) işlemi atomiktir; ya yeni isim eski içeriğe işaret eder, ya eski isimde değişmemiş orijinal içerik bulunur. Yarıda kalmış bir dosya yoktur.

💡 İpucu: `fsync` Niye?

`f.flush()` Python tarafındaki tampon belleği boşaltır; `os.fsync(f.fileno())` işletim sisteminin disk önbelleğini fiziksel diske aktarmasını zorlar. Bu, elektrik kesintisine karşı ekstra güvence verir. Performans bedeli vardır; kayıt sıklığı düşük olduğu için sorun değildir.

46.3.6 Birlikte: Tam Save Akışı

Bu üç parçayı (girinti + UTF-8 + atomik) birleştiren tam bir `kaydet` fonksiyonu:

Python – Üretim Kalitesinde Save

```

1  import json
2  import os
3  from pathlib import Path
4
5  def oyun_kaydet(oyun_durumu, slot=1):
6      yol = Path.home() / ".ziplayan_macera" /
7          ↪ f"slot{slot}.json"
8      yol.parent.mkdir(parents=True, exist_ok=True)
9
10     # 1) Eski kaydı yedekle
11     if yol.exists():
12         yedek = yol.with_suffix(".json.bak")
13         yedek.write_bytes(yol.read_bytes())
14
15     # 2) Geçici dosyaya yaz
16     tmp = yol.with_suffix(".json.tmp")
17     with open(tmp, "w", encoding="utf-8") as f:
18         json.dump(oyun_durumu, f, indent=2,
19                 ensure_ascii=False, sort_keys=True)
20         f.flush()
21         os.fsync(f.fileno())
22
23     # 3) Atomik rename
24     tmp.replace(yol)
25     return yol

```

Bu fonksiyon Ünite 3'teki `SaveManager`'ın `kaydet` metodunun temelini oluşturacak.

46.3.7 Yaygın Hata: **set** ve **tuple**

JSON'a **set** veya **tuple** yazmaya çalışırsan **TypeError** alırsın:

Python – Olmayan Türler

```

1 import json
2
3 # tuple --- otomatik liste olur (sessiz dönüşüm)
4 veri = {"konum": (10, 20)}
5 print(json.dumps(veri)) # {"konum": [10, 20]}
6 # Yüklerken liste olarak gelir; tuple oluşturmak
  ↳ istiyorsan
7 # kendin tuple(...) yapacaksın.
8
9 # set --- TypeError!
10 # json.dumps({"oyuncular": {"Ali", "Ayse"}}) # HATA
11
12 # Çözüm: liste yap
13 veri = {"oyuncular": list({"Ali", "Ayse"})}
14 json.dumps(veri)

```

⚠ Dikkat: Tuple Sessiz Dönüşüm

Python tuple'ı yazarken Python sessizce listeye çevirir; sen JSON spec'ine göre çalışıyorsun. Geri yüklediğinde liste alırsın. (x, y) koordinatlarını sözlük olarak $\{"x": x, "y": y\}$ saklamak daha güvenli olabilir.

46.3.8 Toparlama

JSON ile yazma: `indent=2, ensure_ascii=False, sort_keys=True` olmazsa olmaz; ciddi yazma için atomik desen ve yedek dosya. Sıradaki konuda JSON'ın yetersiz kaldığı durumlar için *pickle* modülüne bakacağız — hangi koşulda ne kullanmalı sorusuna net cevap vereceğiz.

46.4 *pickle* ile Karmaşık Nesneleri Serileştirme

JSON güzel çalışıyor — ama yalnızca altı temel veri türü ile. Bir *Oyuncu* nesnesini, içinde *pygame.Surface*, dict ile düşman listesi, set ile aktif power-up'lar olan karmaşık bir yapıyı doğrudan diske yazmak istesen JSON yetmez. Burada *pickle* modülü devreye girer.

46.4.1 pickle Nedir?

■ Tanım: Pickle / Serializasyon

Pickle, Python'a özgü bir *serializasyon* formatıdır. Hemen hemen her Python nesnesini bayt akışına çevirebilir (**pickle**) ve geri okuyabilir (**unpickle**). "Serializasyon" = bir nesneyi diske yazılabilir veya ağ üzerinden gönderilebilir bir bayt dizisine dönüştürme.

JSON dilden bağımsız iken, pickle yalnızca Python'a özeldir. JSON metin tabanlı iken, pickle binary'dir.

46.4.2 Temel Pickle Kullanımı

Python – Sınıfı Pickle ile Diske Yazma

```

1  import pickle
2
3  class Oyuncu:
4      def __init__(self, isim, x, y, can):
5          self.isim = isim
6          self.x = x
7          self.y = y
8          self.can = can
9          self.envanter = []
10
11  oyuncu = Oyuncu("Ayse", 320, 480, 5)
12  oyuncu.envanter = [{"ad": "Anahtar", "miktar": 1},
13                      {"ad": "Iksir", "miktar": 3}]
14
15  # Diske yazma --- DİKKAT: "wb" (binary)
16  with open("oyuncu.pkl", "wb") as f:
17      pickle.dump(oyuncu, f)
18
19  # Diskten okuma --- DİKKAT: "rb" (binary)
20  with open("oyuncu.pkl", "rb") as f:
21      geri_oyuncu = pickle.load(f)
22
23  print(geri_oyuncu.isim)           # Ayse
24  print(geri_oyuncu.envanter)      # Aynı liste
25  print(type(geri_oyuncu))         # <class 'Oyuncu'>

```

İlk farkı görüyorsun: Pickle'dan dönen nesne *gerçek bir Oyuncu*'dur, **dict** değil. JSON'da **from_dict** yazmak zorundaydık; pickle bunu hediye eder.

46.4.3 API: *json*'a Çok Benzer

Tablo 46.4: *pickle* ile *json* API karşılaştırması

İşlem	<i>json</i>	<i>pickle</i>
Dosyaya yaz	<code>json.dump(o, f)</code>	<code>pickle.dump(o, f)</code>
String/bytes'a yaz	<code>json.dumps(o)</code>	<code>pickle.dumps(o)</code> → bytes
Dosyadan oku	<code>json.load(f)</code>	<code>pickle.load(f)</code>
String/bytes'tan oku	<code>json.loads(s)</code>	<code>pickle.loads(b)</code>
Mod	"r", "w"	"rb", "wb"

API neredeyse aynı; iki kritik fark: *pickle* daima binary moddur, ve string yerine bytes ile çalışır.

46.4.4 Hangi Türler Pickle Edilebilir?

Pickle JSON'dan çok daha cömerttir:

- Tüm temel türler: int, float, str, bool, None
- Container türler: list, tuple, dict, set, frozenset
- Kendi sınıfların (*Oyuncu*, *Dusman*, ...)
- İç içe nesneler ve referanslar
- Pickle başvurusu `__reduce__` ile özelleştirilebilir

Yine de *pickle* edilemeyen türler var:

- *Açık dosya nesneleri*: kapatılmış olmaları gerekir
- *Soketler, threadler, kilitler*: işletim sistemi kaynakları
- *Lambda fonksiyonları*: isimsiz, geri yüklenemez
- *İç içe yerel sınıflar* (function içinde tanımlanan)
- *pygame.Surface*: tipik olarak hayır (genelde *pickle* edemezsin — yeniden yüklenmesi gerekir)

⚠ Dikkat: *pygame.Surface*'i Pickle Etme

Bir *Oyuncu* nesnesinin içinde `self.image = pygame.image.load("oyuncu.png")` varsa *pickle* hata verir veya beklenmeyen davranışlar gösterir. Doğrusu: *pickle* öncesi `self.image._yolu = "oyuncu.png"` olarak *yolu* sakla; unpickle sonrası tekrar yükle.

46.4.5 Pickle'in Karanlık Yüzü: GÜVENLİK

Pickle'ı asla başkasının verdiği bir dosyayı açmak için kullanma. Pickle dosyası kod çalıştırabilir.

Python – Pickle Güvenlik Açığı (Demonstrasyon)

```
1 import pickle
2
```

```

3 class Kotucu:
4     def __reduce__(self):
5         # Bu unpickle sırasında ÇALIŞIR
6         import os
7         return (os.system, ("rm -rf
8             ↪ /tmp/onemli_dosyalar",))
9
10 # Saldırgan bunu üretir, sana gönderir:
11 # zararli_bytes = pickle.dumps(Kotucu())
12
13 # Sen masum şekilde yüklersen --- komut çalışır
14 # pickle.loads(zararli_bytes) # Sistem zarar görür

```

⚠ Dikkat: KRİTİK: Pickle = Kod Yürütme

Pickle dosyası açmak, içindeki kodu çalıştırmaya eşittir. Sadece kendi ürettiğin pickle dosyalarını oku. İnternette indirdiğin, e-postayla aldığın, kullanıcının yüklediği pickle dosyalarına asla `pickle.load` uygulama — saldırgan makinene komut çalıştırabilir. JSON bu açıdan tamamen güvenlidir çünkü saf veri taşır, kod taşımaz.

46.4.6 JSON vs Pickle: Karşılaştırma Matrisi

Tablo 46.5: JSON ve Pickle karşılaştırması

Kriter	JSON	Pickle
İnsan okur	Evet	Hayır (binary)
Dilden bağımsız	Evet	Hayır (Python'a özel)
Güvenli (untrusted veri)	Evet	HAYIR (kod yürütür)
Sınıf instance'ı	Hayır (manuel dönüşüm)	Evet (otomatik)
<code>set, tuple</code>	Liste olur	Korunur
Dosya boyutu	Daha büyük (metin)	Daha küçük (binary)
Hız (büyük veri)	Daha yavaş	Daha hızlı
Sürüm geçişi	Kolay (versioning ekle)	Zor (sınıf değişirse karar)

💡 İpucu: Hangi Format?

Genel kural: **JSON kullan**. Yalnızca şu durumlarda *pickle*'a geç:

- Performans kritik (binlerce nesne, milisaniye önemli)
- Veri *yalnızca* senin yazdığın kod tarafından okunacak (tek oyunculu oyun kayıt dosyası)
- Veri yapısı çok karmaşık ve manuel dönüşüm yorucu

Bu kitabın save sistemi JSON kullanacak — güvenli, taşınabilir, debug edilebilir.

46.4.7 Hibrit Yaklaşım: JSON İçinde Versiyonlama

Kayıt formatın gelecekte değişebilir. Bugün *can* alanı varken yarın *max_can* eklemek isteyebilirsin. Eski kayıtları kırmamak için *version* alanı koy:

Python – Versiyon Yönetimi

```

1 import json
2
3 KAYIT_VERSION = 2
4
5 def kayit_yukle(yol):
6     with open(yol, "r", encoding="utf-8") as f:
7         veri = json.load(f)
8
9     # Eski sürümleri yeni sürüme migre et
10    if veri.get("version", 1) == 1:
11        # Sürüm 1'de max_can yoktu --- can ile aynı yap
12        veri["oyuncu"]["max_can"] = veri["oyuncu"]["can"]
13        veri["version"] = 2
14
15    return veri

```

📌 Bilgi: Versiyon Disiplini

İlk kayıt formatında *version: 1* koy. Her yeni alan veya yapı değişiminde sürümü artır ve göç (migration) kodu yaz. Bu, oyuncuların 1 yıl önceki kayıtlarını bugünün oyununda açabilmesini garanti eder. *Pickle* bu açıdan zayıftır — sınıf adı değişirse eski *pickle* dosyaları kırılır.

46.4.8 Toparlama

pickle hızlı ve esnek; ama güvenlik ve sürüm geçişi açısından risklidir. Bu kitap save sisteminde JSON kullanacak — ve Ünite 3'te bu seçimin sonuçlarını yaşayacağız: *to_dict* ve *from_dict* desenleri sınıf-sözlük köprüsünü inşa edecek.

46.5 Sonuç

Bu ünite de oyunun belleğini diske bağlamayı öğrendik. `open` ile dosya açtık, `with` ile güvenli kapattık, `json` ile yapılandırılmış veri yazıp okuduk, `pickle` ile karmaşık nesneleri serileştirdik. Hangi durumda neyin tercih edileceğini gördük: JSON varsayılan, pickle yalnızca özel durumlarda.

46.5.1 Anahtar Kavramlar

Tablo 46.6: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
<code>open</code>	Dosyayı açar; mod karakteriyle (r/w/a/x, b ekiyle binary)
Context manager	<code>with</code> ile çalışan, otomatik açılış-kapanış yapan nesne
Metin vs binary mod	<code>"r"</code> str döner, <code>"rb"</code> bytes döner
<code>pathlib.Path</code>	İşletim sistemi bağımsız yol yönetimi
JSON	İnsan okur, dilden bağımsız, güvenli veri formatı
<code>json.dump/load</code>	Dosyaya yazma / okuma
<code>json.dumps/loads</code>	String'e yazma / string'den okuma
<code>ensure_ascii=False</code>	Türkçe karakterleri korur
<code>indent=2</code>	İnsan okunabilir girintili JSON
Pickle	Python'a özgü, binary, sınıf instance'ı serileştirebilen format
Atomik yazma	<code>.tmp</code> 'ye yaz, sonra <code>rename</code> — yarım dosyayı önler
Versiyonlama	Kayıt formatına <code>version</code> alanı eklemek

46.5.2 Önemli Komutlar

Tablo 46.7: Ünite 2 – Temel komutlar

Komut	Açıklama
<code>with open(yol, "r", encoding="utf-8") as f:</code>	Standart metin okuma deseni
<code>json.load(f)</code>	Dosyadan JSON okur, dict döndürür
<code>json.dump(d, f, indent=2, ensure_ascii=False)</code>	Diske güzel JSON yazar
<code>json.JSONDecodeError</code>	Bozuk JSON için istisna türü
<code>Path.home() / ".oyun"</code>	Kullanıcı ev dizininde gizli klasör
<code>yol.with_suffix(".bak")</code>	Aynı isim, farklı uzantı (yedek isim)
<code>tmp.replace(hedef)</code>	Atomik rename — veri kaybına uğramaz
<code>pickle.dump(o, f)</code>	Nesneyi binary diske yazar
<code>pickle.load(f)</code>	Binary dosyadan nesne okur (telik)
<code>os.fsync(f.fileno())</code>	İşletim sistemine “diske gerçekte yazılmasını” bildirir

46.5.3 JSON vs Pickle: Bu Kitabın Tercihi

Bu kitap tüm *save ve settings işlemlerinde JSON kullanacak*. Sebepleri:

1. **Güvenli:** Oyuncunun manipüle ettiği bir kayıt dosyası kod çalıştıramaz.
2. **Debug edilebilir:** Bozuk bir kayıt dosyasını not defterinde açıp sorunu görebilirsin.
3. **Modder dostu:** Oyuncular kayıtlarını paylaşıp inceleyebilir.
4. **Sürüm yönetimi kolay:** Eski kayıtları yeni sürüme göç ettirmek doğal.

Pickle bu kitapta gösterildi ama *kullanılmayacak*. Production oyununda da çoğu zaman tercih edilmeyecek.

46.5.4 Bir Sonraki Ünite: Save/Load Sistemi

Artık temel araçlar elimizde. Sıradaki üniteye bu araçları **SaveManager** adlı tek bir sınıfta birleştireceğiz:

- Bir kayıt formatı tasarlayacağız (oyuncu, seviye, süre, envanter)
- `to_dict` / `from_dict` deseniyle sınıflar arası köprüyü kuracağız
- Birden fazla kayıt slotu (Slot 1/2/3) yönetimini yazacağız
- Otomatik kayıt sistemini ekleyeceğiz
- Kayıt klasörünün doğru yerde tutulmasını sağlayacağız

İpucu: Pratik Alıştırma

Bir sonraki üniteye geçmeden önce, kendi makinende `Path.home() / ".ziplayan_macera"` dizinini elinle oluştur. İçine `deneme.json` adında 5–10 satırlık bir JSON dosyası yaz, Python'da aç ve oku. Kullanıcı dizininin nerede olduğunu ve gizli klasörlerin (`.` ile başlayan) sistemde nasıl görüldüğünü deneyimle.

“Kayıt sistemi, oyunun oyuncuya verdiği bir sözdür: “Buraya kadar geldin, ben unutmayacağım.””

— JONATHAN BLOW, BRAID YARATICISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Bir kayıt formatının ne içermesi gerektiği (oyuncu konumu, can, altın, seviye, süre)
- `to\_dict` / `from\_dict` deseniyle oyun durumunu serileştirilebilir hale getirme
- `SaveManager` sınıfı ile diskten okuyup yazma; sürüm (version) alanı ile uyumluluk
- Birden fazla kayıt slotu (Slot 1, Slot 2, Slot 3) ve slot yönetim ekranı
- Otomatik kayıt (autosave): ne zaman, hangi sıklıkta, ne kadar yer kaplar
- Kayıt klasörünün doğru yerde tutulması: `Path.home() / ".ziplayan_macera"`

47.1 Bir Kayıt Formatı Tasarlamak

Araçlarımız hazır: state machine ile akışı, JSON ile veri formatını biliyoruz. Şimdi soru: *neyi* kaydedeceğiz? Bir oyunun kayıt dosyasında ne olur, ne olmaz? Bu konuda bunun cevabını planlıyoruz.

47.1.1 Kayıt Edilmesi Gerekenler

Genel olarak ayırım: *kalıcı oyun durumu* kaydedilir, *geçici çalıştırma durumu* kaydedilmez.

Kayıt edilenler:

- Oyuncunun konumu, hızı, can durumu
- Skor, altın, deneyim puanı

- Mevcut seviye / harita
- Envanter (sahip olunan eşyalar)
- Toplanmış altınlar, açılmış kapılar
- Geçilmiş bölümler, kazanılmış başarımlar
- Toplam oyun süresi, kayıt zamanı
- Aktif ana görev / yan görevler

Kayıt edilmeyenler:

- Düşmanların anlık konumları (oyun yüklendiğinde yeniden doğarlar)
- Mermi, parçacık efekti, animasyon karesi
- Kamera hesaplamaları, FPS sayacı
- `pygame.Surface` nesneleri (ID veya yol kaydedilir)
- Geçici buffer'lar (jump buffer, coyote sayacı, i-frames)

💡 İpucu: Yeniden Kurulabilir Olanları Atla

Kural: kayıttan oyun yüklendiğinde *yeniden hesaplanabilen* hiçbir şeyi kaydetme. Düşmanlar harita verisinden yeniden doğar, mermiler temizlenir, animasyon karesi 0'dan başlar. Ne kadar az kayıt edersen o kadar küçük dosya, o kadar az hata yüzeyi.

47.1.2 Tipik Bir Kayıt Şeması

Zıplayan Macera için tasarladığımız format:

Python – Kayıt Şeması v1

```

1  {
2      "version": 1,                # Format sürümü ---
3      ↪ ZORUNLU
4      "kayit_zamani": "2026-05-04 14:30:22",
5      "oyuncu_adi": "Ayse",
6      "toplam_sure": 1820.5,       # Saniye
7
8      "oyuncu": {
9          "x": 320,
10         "y": 480,
11         "can": 4,
12         "max_can": 5,
13         "altin": 1250,
14         "deneyim": 340,
15         "envanter": [
16             {"ad": "Anahtar", "miktar": 1},
17             {"ad": "Iksir", "miktar": 3}
18         ]
19     },
20
21     "seviye": {
22         "ad": "orman_2",
23         "kesfedilen_alanlar": ["A1", "A2", "B1"],

```

```

23         "toplanan_altinlar": [12, 15, 18, 23],
24         "acilan_kapilar": ["kapi_3"]
25     },
26
27     "ilerleme": {
28         "tamamlanan_seviyeler": ["orman_1"],
29         "basarimlar": ["ilk_zipla", "10_dusman_yendi"],
30         "toplam_olum": 7
31     }
32 }

```

Beş üst seviye alan: **version** (formatın sürümü), **kayit_zamani** (insan dostu metadata), **oyuncu_adi**, **oyuncu** (kişiye ait durum), **seviye** (mevcut sahnenin durumu), **ilerleme** (global başarımlar).

47.1.3 version Alanının Önemi

⚠ Dikkat: Versiyon Olmadan Format Yönetilemez

Kayıt dosyasının ilk alanı **version** olmalı. Bugün **version: 1**. Yarın yeni alan eklediğinde **version: 2** yapacaksın. Eski kayıtları kırmadan yeni özellik ekleyebilmenin biricik yolu budur.

Versiyon olmasaydı, ilk kayıttan üç ay sonra “**max_can** alanını da ekleyelim” dediğinde eski kayıtlar yüklenmez olur — çünkü kod `veri["oyuncu"]["max_can"]` bekler ama eski dosyalarda yoktur. Versiyon ile şu yazılır:

Python – Sürüm Geçişi (Migration)

```

1 def migrate(veri):
2     """Eski sürüm kayıtları yeni sürüme yükselt."""
3     if veri["version"] == 1:
4         # v1 -> v2: max_can eklendi
5         veri["oyuncu"]["max_can"] = veri["oyuncu"]["can"]
6         veri["version"] = 2
7
8     if veri["version"] == 2:
9         # v2 -> v3: deneyim alanı eklendi
10        veri["oyuncu"].setdefault("deneyim", 0)
11        veri["version"] = 3
12
13    return veri

```

Her sürüm geçişi *eski sürümden bir sonraki sürüme* yükseltir. Çoklu sürüm geçişi otomatik kademeli olur: v1 dosyası önce v2’ye, sonra v3’e gider.

47.1.4 to_dict / from_dict Deseni

Sınıfların kendi serileştirme bilgisini taşıması en sürdürülebilir yaklaşımdır:

Python – Oyuncu Sınıfı – to_dict / from_dict

```

1 class Oyuncu:
2     def __init__(self, x, y, can=5, altin=0):
3         self.x = x
4         self.y = y
5         self.can = can
6         self.max_can = can
7         self.altin = altin
8         self.envanter = []
9
10    def to_dict(self):
11        """Bu oyuncunun JSON'a yazılabilir gösterimini
12        ↪ üret."""
13        return {
14            "x": self.x,
15            "y": self.y,
16            "can": self.can,
17            "max_can": self.max_can,
18            "altin": self.altin,
19            "envanter": list(self.envanter), # Kopya
20        }
21
22    @classmethod
23    def from_dict(cls, d):
24        """Sözlükten Oyuncu inşa et."""
25        oyuncu = cls(x=d["x"], y=d["y"], can=d["can"],
26                     altin=d.get("altin", 0))
27        oyuncu.max_can = d.get("max_can", d["can"])
28        oyuncu.envanter = list(d.get("envanter", []))
29        return oyuncu

```

`from_dict`'in `@classmethod` olduğuna dikkat. Bu, alt sınıfların (`HiziliOyuncu(Oyuncu)` gibi) doğru sınıfı oluşturmasını sağlar.

💡 **İpucu:** `d.get(...)` Kullan

`d["max_can"]` alan yoksa `KeyError` verir; eski sürüm kayıtları kırılır. `d.get("max_can", d["can"])` varsayılan değer ile güvenli erişim yapar. Eski sürüm uyumluluğu böyle korunur.

47.1.5 Boyut ve Gizlilik

Kayıt dosyasının büyüklüğü hızla artabilir. Eğer envanterde 1000 eşya olabiliyorsa, her eşyayı tek tek yazmak yerine ID listesi tutmak daha verimli:

Python – Verimli Saklama

```

1 # Kötü: tam nesne tekrar tekrar yazılır
2 "envanter": [

```

```

3      {"ad": "Anahtar", "agirlik": 0.5, "sprite": "key.png",
4      ↪ ...},
5      {"ad": "Anahtar", "agirlik": 0.5, "sprite": "key.png",
6      ↪ ...},
7  ]
8
9  # İyi: ID + miktar yazılır, tanım veritabanından okunur
10 "envanter": [
11     {"id": "anahtar", "miktar": 2}
12 ]

```

⚠ Dikkat: Hassas Bilgi Saklama

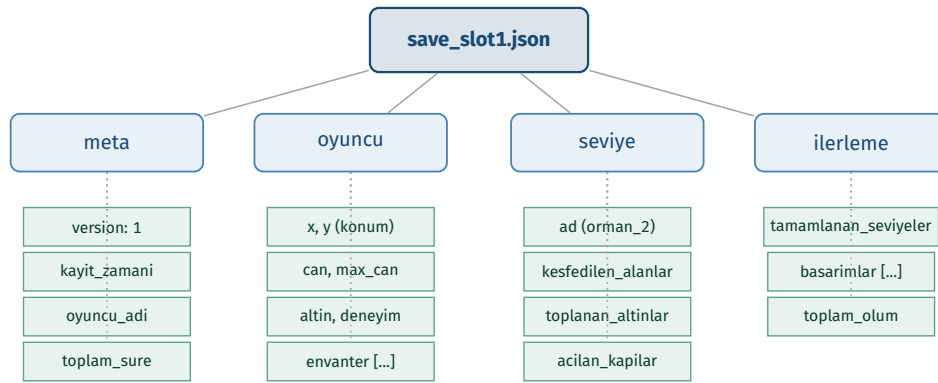
Kayıt dosyası açıktır — oyuncu üzerinde değişiklik yapabilir. *Asla* oyuncunun şifresini, e-postasını veya hiledirenmeyi zorlaştıracak değerleri (lisans anahtarı, premium aktivasyon) oraya yazma. Bu kitabın oyunu tek oyunculu/offline olduğu için bu sorunu yok; ama prensibi bil.

47.1.6 Toparlama

İyi bir kayıt formatı:

1. **version** alanı içerir (her zaman)
2. Yeniden hesaplanabilen şeyleri *kaydetmez*
3. **to_dict** / **from_dict** deseniyle sınıflara bağlanır
4. **d.get("alan", varsayilan)** ile geriye uyumlu okunur
5. Veri yoğun listeleri ID + miktar olarak saklar

save_slot1.json – Kayıt Dosyasının Yapısı



version her zaman ilk alan; eski kayıtları migrate etmek için şart.

Şekil 47.1: Kayıt dosyasının üst seviye yapısı: meta, oyuncu, seviye, ilerleme.

Sıradaki konuda **SaveManager** sınıfını yazıp bu formatı diske yazma kısmını yapacağız.

47.2 SaveManager: Kaydetme Tarafı

Format hazır. Şimdi onu diske yazan sınıfı inşa ediyoruz. `SaveManager`, oyunun her yerinden tek bir nokta üzerinden çağrılabilen bir araç olacak: `save_mgr.kaydet(slot=1, ...)`.

47.2.1 Sınıfın İskeleti

Python – save_manager.py - İskelet

```

1 import json
2 import os
3 from datetime import datetime
4 from pathlib import Path
5
6 KAYIT_VERSION = 1
7
8 class SaveManager:
9     """Oyun kayıtlarını disk üzerinde yöneten merkezi
10     ↪ sınıf."""
11
12     def __init__(self, kayıt_dizini=None):
13         if kayıt_dizini is None:
14             kayıt_dizini = Path.home() /
15             ↪ ".ziplayan_macera"
16         self.dizin = Path(kayıt_dizini)
17         self.dizin.mkdir(parents=True, exist_ok=True)
18
19     def slot_yolu(self, slot):
20         """Verilen slot için tam dosya yolunu döndür."""
21         return self.dizin / f"slot{slot}.json"

```

İki tasarım kararı:

- Kayıtlar `~/.ziplayan_macera/` altında tutulur. Bu, oyunun kurulu olduğu klasörden bağımsızdır; oyun silinse bile kayıtlar kalır.
- Slot bazlı dosya isimlendirme: `slot1.json`, `slot2.json`, `slot3.json`. Slot sistemi Konu 4'te detayıyla ele alınacak.

47.2.2 Kayıt Yazma Metodu

Python – kaydet Metodu

```

1 def kaydet(self, slot, oyuncu, seviye_durumu, ilerleme,
2 toplam_sure=0):
3     """Tüm oyun durumunu slot'a yaz."""
4     veri = {
5         "version": KAYIT_VERSION,
6         "kayıt_zamani":
7             ↪ datetime.now().isoformat(timespec="seconds"),

```

```

7         "oyuncu_adi": getattr(oyuncu, "isim", "Oyuncu"),
8         "toplam_sure": toplam_sure,
9         "oyuncu": oyuncu.to_dict(),
10        "seviye": seviye_durumu,
11        "ilerleme": ilerleme,
12    }
13    return self._atomik_yaz(self.slot_yolu(slot), veri)
14
15    def _atomik_yaz(self, yol, veri):
16        """Atomik yazma: önce .tmp, sonra rename."""
17        tmp = yol.with_suffix(yol.suffix + ".tmp")
18        with open(tmp, "w", encoding="utf-8") as f:
19            json.dump(veri, f, indent=2, ensure_ascii=False,
20                      sort_keys=True)
21            f.flush()
22            os.fsync(f.fileno())
23        tmp.replace(yol)
24        return yol

```

Atomik yazma deseni Ünite 2'den geliyor: `.tmp`'ye yaz, `flush + fsync` ile diske aktar, sonra `replace` ile adını gerçek isimle değiştir. Yarıda kesilirse eski dosya sağlam kalır.

47.2.3 Yedek (Backup) Sistemi

Yarıda kesintiye karşı atomik yazma korur. Ama bir de *başarılı* yazımda dosya bozulmuş olabilir (disk hatası, virüs, kullanıcı elinin değmesi). Bunun için her başarılı kayıttan önce eski dosyayı `.bak` olarak yedekle:

Python – Yedekli Kayıt

```

1    def kaydet(self, slot, oyuncu, seviye_durumu, ilerleme,
2              toplam_sure=0):
3        yol = self.slot_yolu(slot)
4
5        # Önceki kayıt varsa yedekle
6        if yol.exists():
7            yedek = yol.with_suffix(".json.bak")
8            yedek.write_bytes(yol.read_bytes())
9
10       veri = {
11           "version": KAYIT_VERSION,
12           "kayit_zamani":
13               ↳ datetime.now().isoformat(timespec="seconds"),
14           "oyuncu_adi": getattr(oyuncu, "isim", "Oyuncu"),
15           "toplam_sure": toplam_sure,
16           "oyuncu": oyuncu.to_dict(),
17           "seviye": seviye_durumu,
18           "ilerleme": ilerleme,
19       }
20       return self._atomik_yaz(yol, veri)

```

Artık her slot için iki dosya tutuluyor: `slot1.json` (en güncel) ve `slot1.json.bak` (bir önceki kayıt). Konu 3'teki yükleme tarafı bozuk dosya bulduğunda `.bak`'a düşer.

47.2.4 Otomatik Kayıt (Autosave)

Oyuncu manuel kaydetmeyi unutabilir. Otomatik kayıt sistemiyle oyun belirli olaylarda veya zamanda kendiliğinden kaydeder:

Python – Otomatik Kayıt

```

1 class SaveManager:
2     AUTOSAVE_SLOT = 0 # Slot 0 = otomatik kayıt rezervi
3
4     def __init__(self, ...):
5         # ... mevcut kod ...
6         self._son_autosave_zamani = 0
7         self.autosave_aralik = 60.0 # Saniye
8
9     def autosave_kontrol(self, oyun_zamani, oyuncu,
10         ↪ seviye, ilerleme,
11         ↪ toplam_sure):
12         """Belirli aralıkla autosave yap."""
13         if oyun_zamani - self._son_autosave_zamani >=
14         ↪ self.autosave_aralik:
15             self.kaydet(self.AUTOSAVE_SLOT, oyuncu,
16             ↪ seviye, ilerleme,
17             ↪ toplam_sure)
18             self._son_autosave_zamani = oyun_zamani
19
20     def olay_autosave(self, oyuncu, seviye, ilerleme,
21         ↪ toplam_sure):
22         """Önemli bir olay sonrası autosave (level geçişi,
23         ↪ vs.)."""
24         self.kaydet(self.AUTOSAVE_SLOT, oyuncu, seviye,
25         ↪ ilerleme,
26         ↪ toplam_sure)

```

İki tip autosave: *zaman tabanlı* (her 60 saniyede bir) ve *olay tabanlı* (level geçişi, boss yenilgisi). Her ikisi de ayrı bir slot'a (slot 0) yazar ki manuel slot'ların üzerine yazmasın.

⚠ Dikkat: Autosave Görünür Olmalı

Otomatik kayıt sırasında oyuncuya bir *ipucu* göster: ekranın köşesinde küçük bir disk ikonu, "Kaydediliyor..." yazısı. Aksi halde oyuncu kaydetmediğini düşünüp manuel kayıt arar. Görünmeyen sistem, var olmayan sistemdir.

47.2.5 Bilgi Sızdırmayan Tasarım

Oyuncunun arayüzünde sadece bir `kaydet` çağrısı kalmalı:

Python – Oyun Tarafından Çağrı

```

1  # Oyun içinden:
2  save_mgr = SaveManager()
3
4  # Manuel kayıt:
5  save_mgr.kaydet(
6      slot=1,
7      oyuncu=self.oyuncu,
8      seviye_durumu=self.seviye.snapshot(),
9      ilerleme=self.ilerleme.to_dict(),
10     toplam_sure=self.toplam_sure,
11 )
12
13 # Autosave (her karede çağır, içinde aralık kontrolü var):
14 save_mgr.autosave_kontrol(
15     oyun_zamani=pygame.time.get_ticks() / 1000.0,
16     oyuncu=self.oyuncu,
17     seviye=self.seviye.snapshot(),
18     ilerleme=self.ilerleme.to_dict(),
19     toplam_sure=self.toplam_sure,
20 )

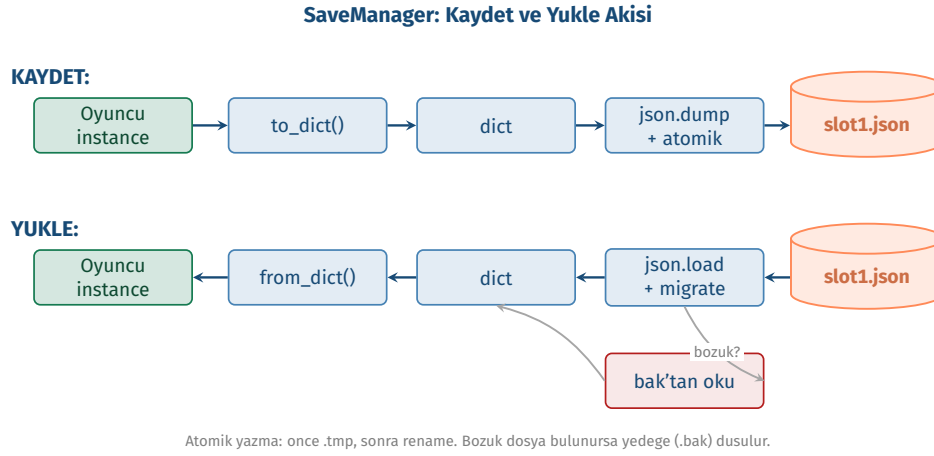
```

`SaveManager` dışarı yalnızca üç metot sunar: `kaydet`, `autosave\_kontrol`, `olay\_autosave`. İçerideki atomik yazma, yedekleme, formatlama detayları kapsüllenmiştir — oyunun başka hiçbir yerinden `json.dump` çağrılmamalı.

💡 İpucu: Kapsülleme

Bir sistem ne kadar az dış arayüz sunarsa o kadar değiştirilebilir olur. Bugün JSON kullanıyorsun; yarın pickle'a veya SQLite'a geçmek istesen, sadece `SaveManager`'ın iç yapısını değiştirirsin — oyunun geri kalanı haberdar bile olmaz.

47.2.6 Kayıt Akış Diyagramı



Şekil 47.2: SaveManager kaydetme ve yükleme akışı: oyun durumu → to_dict → JSON → atomik yazma; ve geri.

47.2.7 Toparlama

SaveManager'ın yazma kısmı şu özelliklere sahip:

- Atomik yazma (yarıda kesintiden korunma)
- Yedek dosya (önceki kaydın korunması)
- Versiyon alanı (sürüm geçişi)
- İki autosave türü (zaman ve olay)
- Tek nokta arayüzü (kapsülleme)

Sıradaki konuda yükleme tarafını yazıp eksik bir parçayı tamamlayacağız.

47.3 SaveManager: Yükleme Tarafı

Kaydedilmiş veriyi geri okumak, yazmaktan daha hassas bir iştir. Dosya bozuk olabilir, silinmiş olabilir, eski sürüme ait olabilir, elle değiştirilmiş olabilir. SaveManager'ın yükleme metodu tüm bu durumları zarafetle ele almalı.

47.3.1 Temel Yükleme Metodu

Python – yükle Metodu - Temel

```

1 class SaveManager:
2     # ... mevcut kod (kaydet, vb.) ...
3
4     def yukle(self, slot):
5         """Slot'tan oyun durumunu yükle. Yoksa veya
6         ↪ bozuksa None döner."""
7         yol = self.slot_yolu(slot)
8         if not yol.exists():
9             return None # Boş slot

```

```

9
10     try:
11         with open(yol, "r", encoding="utf-8") as f:
12             veri = json.load(f)
13     except json.JSONDecodeError:
14         # Bozuk dosya --- yedeğe dön
15         return self._yedekten_yukle(slot)
16
17     # Sürüm migrasyonu uygula
18     veri = self._migrate(veri)
19     return veri
20
21 def _yedekten_yukle(self, slot):
22     """Bozuk ana dosya varsa .bak'tan yükle."""
23     yol = self.slot_yolu(slot)
24     yedek = yol.with_suffix(".json.bak")
25     if not yedek.exists():
26         return None
27     try:
28         with open(yedek, "r", encoding="utf-8") as f:
29             veri = json.load(f)
30             return self._migrate(veri)
31     except json.JSONDecodeError:
32         return None # Yedek de bozuksa pes

```

Üç durumun farkı net olmalı:

- `return None`: “Bu slot boş — yeni oyun başlat.”
- `return veri`: “Yüklendi, oyun bu durumdan devam.”
- İstisna fırlatma: *yapma* — oyunu çökertir.

⚠ Dikkat: İstisna Yutma vs Bilgi

Bozuk dosyayı sessizce `None` döndürmek oyuncunun şikayetlerini çözmesini imkansız kılar. Mutlaka log yaz: `print(f"[YUKLE] Slot {slot} bozuk: {e}")`. İdeal olarak ekrana “kayıt bozuk, yedek kullanıldı” bildirimi de göster.

47.3.2 Sürüm Migrasyonu

Konu 1’de tanıttığımız `migrate` mantığını burada kapsülliyoruz:

Python – Migrate Metodu

```

1 def _migrate(self, veri):
2     """Eski sürüm verisini en güncel sürüme yükselt."""
3     v = veri.get("version", 1)
4
5     if v < 2:
6         # v1 -> v2: max_can eklendi

```

```

7         if "oyuncu" in veri:
8             veri["oyuncu"].setdefault(
9                 "max_can",
10                veri["oyuncu"].get("can", 5),
11            )
12            v = 2
13
14        if v < 3:
15            # v2 -> v3: ilerleme alanı eklendi (eski
16            ↪ kayıtlarda yok)
17            veri.setdefault("ilerleme", {
18                "tamamlanan_seviyeler": [],
19                "basarimlar": [],
20                "toplam_olum": 0,
21            })
22            v = 3
23
24        veri["version"] = KAYIT_VERSION
25        return veri

```

Her sürüm geçişi *kademelidir*. Bir oyuncunun çok eski v1 kaydı önce v2'ye, sonra v3'e yükseltilir. Her geçişte yalnızca *o sürümde değişen alanlar* dokunulur.

💡 İpucu: Migrate Test Etmeyi Unutma

Eski sürüm kayıt dosyalarını test data olarak proje klasöründe sakla. `tests/saves/v1_basit.json`, `tests/saves/v2_envanter.json`. Her yeni sürüm migrasyonu yazdıktan sonra hepsini yükleyip beklenen sonucu doğrula. Bu, “yeni özellik eski kayıtları kırdı” felaketinden kaçınmanın yoludur.

47.3.3 Sözlükten Sınıfa: `from_dict`

`yukle` sana `dict` verir. Oyunda *sınıf instance'ları* istiyorsan dönüşüm yapman gerekir:

Python – Yükleme Sonrası Sınıf Oluşturma

```

1  # main.py veya PlayState içinde:
2  save_mgr = SaveManager()
3  veri = save_mgr.yukle(slot=1)
4
5  if veri is None:
6      # Yeni oyun başlat
7      oyuncu = Oyuncu(x=100, y=400)
8      seviye = Seviye("orman_1")
9  else:
10     # Kayıttan oyun başlat
11     oyuncu = Oyuncu.from_dict(veri["oyuncu"])

```

```

12 seviye = Seviye.from_dict(veri["seviye"])
13 toplam_sure = veri.get("toplam_sure", 0)

```

`Oyuncu.from\_dict` ve `Seviye.from\_dict` sınıfların kendi içinde tanımlı. Bu, sınıfların *kendi serileştirme detaylarını taşıması* prensibine uygun.

47.3.4 Slot Listesini Çıkarma

Yükleme ekranında “Slot 1: Ayse, Bölüm 3, 30 dakika” gibi özet bilgi göstereceksin. Tüm slotlardaki bilgileri tek seferde çekmek için bir yardımcı:

Python – Slot Özetleri

```

1 def slot_ozetleri(self, slot_sayisi=3):
2     """Tüm slotlar için özet bilgi listesi döndür."""
3     ozetler = []
4     for i in range(1, slot_sayisi + 1):
5         yol = self.slot_yolu(i)
6         if not yol.exists():
7             ozetler.append({"slot": i, "bos": True})
8             continue
9         try:
10             with open(yol, "r", encoding="utf-8") as f:
11                 veri = json.load(f)
12                 ozetler.append({
13                     "slot": i,
14                     "bos": False,
15                     "oyuncu_adi": veri.get("oyuncu_adi", "?"),
16                     "seviye": veri.get("seviye", {}).get("ad",
17                                     ↪ "?"),
18                     "sure": veri.get("toplam_sure", 0),
19                     "zaman": veri.get("kayit_zamani", "?"),
20                     "altin": veri.get("oyuncu",
21                                     ↪ {}).get("altin", 0),
22                 })
23             except json.JSONDecodeError:
24                 ozetler.append({"slot": i, "bos": False,
25                                 ↪ "bozuk": True})
26     return ozetler

```

Bu özet listesi yükleme menüsünün ihtiyacı olan her şeyi içerir: slot numarası, doluluk durumu, oyuncu bilgisi, son kayıt zamanı.

47.3.5 Silme: sil

Oyuncu bir slotu silmek isteyebilir:

Python – Slot Silme

```

1 def sil(self, slot):
2     """Slot dosyasını ve yedeğini sil."""
3     yol = self.slot_yolu(slot)
4     yedek = yol.with_suffix(".json.bak")
5     if yol.exists():
6         yol.unlink()
7     if yedek.exists():
8         yedek.unlink()

```

⚠ Dikkat: Silme Onayı Şart

`sil` çağrılmadan önce kullanıcıya “Slot {N} silinecek, emin misin?” diye sor. Yanlışlıkla silinen kayıt geri gelmez. Onay penceresi için Ünite 1’de tartıştığımız `ConfirmDialogState` deseni kullanılabilir.

47.3.6 Tam SaveManager: Bütün Parçaları Birleştir

Bu noktada `SaveManager`’ın tam arayüzü:

Tablo 47.1: `SaveManager` dış arayüzü

Metot	İşlevi
<code>slot_yolu(slot)</code>	Verilen slot için tam dosya yolu
<code>kaydet(slot, ...)</code>	Atomik + yedekli kayıt yazımı
<code>yukle(slot)</code>	Slot’tan veri okur (None = boş)
<code>slot_ozetleri(n=3)</code>	Tüm slotlar için özet bilgi
<code>sil(slot)</code>	Slot dosyasını ve yedeğini siler
<code>autosave_kontrol(...)</code>	Aralıklı autosave
<code>olay_autosave(...)</code>	Olay sonrası autosave

Yedi metot, birkaç yardımcı. Yaklaşık 150 satır kod, ama oyunun en önemli özelliklerinden birini sağlam şekilde sunuyor.

47.3.7 Toparlama

Yükleme tarafı yazmadan daha karmaşık çünkü “hata var” senaryolarına karşı dirençli olmalı. Üç katman koruma:

1. Dosya yoksa → `None` (yeni oyun)
2. Dosya bozuksa → `.bak`’tan yükle
3. Yedek de bozuksa → `None` ve log

Bir de sürüm migrasyonu, slot özetleri, silme metotları... Sıradaki konuda bunları çok slotlu bir yükleme menüsünde nasıl göstereceğini öğreneceğiz.

47.4 Çoklu Slot Sistemi ve Yükleme Menüsü

Tek bir kayıt yetmez. Oyuncu farklı denemeler yapmak isteyebilir, çocuğuyla aynı oyunu paylaşabilir, yedek tutmak isteyebilir. Çoğu oyun 3, 5 veya daha fazla *slot* sunar. Bu konuda hem mantığı (SaveManager üzerinden), hem arayüzü (LoadState) inşa ediyoruz.

47.4.1 Slot Felsefesi

■ Tanım: Save Slot

Bir *save slot*, oyunun farklı bir oyun oturumunu / hattını kaydedebileceği ayrı bir kayıt yuvasıdır. 3 slotlu sistemde oyuncu üç paralel oyun yürütebilir; her birinin kendi karakteri, seviyesi, envanteri olur.

Tipik slot organizasyonu:

- Slot 0: Otomatik kayıt (autosave)
- Slot 1, 2, 3: Manuel oyuncu slotları
- İsteğe bağlı: “Quick Save” (F5) için ayrı bir slot

Bizim sistemde slot dosyaları `slot0.json` ile `slot3.json` arasında. Slot 0 autosave için rezerv.

47.4.2 Yükleme Ekranı: LoadState

Ünite 1’de yazdığımız state machine üzerine `LoadState` ekleyeceğiz:

Python – load_state.py

```

1  import pygame
2  from state import State
3
4  class LoadState(State):
5      def __init__(self, manager, save_mgr):
6          super().__init__(manager)
7          self.save_mgr = save_mgr
8          self.secili = 0
9
10     def gir(self):
11         self.font_b = pygame.font.Font(None, 48)
12         self.font_o = pygame.font.Font(None, 28)
13         self.font_k = pygame.font.Font(None, 20)
14         # Her gir'de slot bilgilerini tazele
15         self.slotlar =
16             ↳ self.save_mgr.slot_ozetleri(slot_sayisi=3)
17
18     def olaylari_isle(self, olaylar):
19         for olay in olaylar:
20             if olay.type != pygame.KEYDOWN:
21                 continue

```

```

21         if olay.key == pygame.K_DOWN:
22             self.secili = (self.secili + 1) %
                ↳ len(self.slotlar)
23         elif olay.key == pygame.K_UP:
24             self.secili = (self.secili - 1) %
                ↳ len(self.slotlar)
25         elif olay.key == pygame.K_RETURN:
26             self._sloti_yukle()
27         elif olay.key == pygame.K_DELETE:
28             self._sloti_sil()
29         elif olay.key == pygame.K_ESCAPE:
30             self.manager.pop()      # Menüye dön
31
32     def _sloti_yukle(self):
33         slot = self.secili + 1
34         veri = self.save_mgr.yukle(slot)
35         if veri is None:
36             return      # Boş slot --- ses çal, animasyon,
                ↳ vs.
37         from play_state import PlayState
38         ps = PlayState(self.manager,
                ↳ baslangic_verisi=veri)
39         self.manager.degistir(ps)
40
41     def _sloti_sil(self):
42         slot = self.secili + 1
43         self.save_mgr.sil(slot)
44         self.slotlar =
                ↳ self.save_mgr.slot_ozetleri(slot_sayisi=3)
45
46     def guncelle(self, dt):
47         pass
48
49     def ciz(self, ekran):
50         ekran.fill((20, 20, 40))
51         baslik = self.font_b.render("KAYIT YUKLE", True,
                ↳ (255, 215, 0))
52         ekran.blit(baslik, baslik.get_rect(center=(400,
                ↳ 80)))
53
54         for i, ozet in enumerate(self.slotlar):
55             y = 180 + i * 100
56             renk = (255, 255, 255) if i == self.secili
                ↳ else (160, 160, 180)
57             self._slot_kutusu_ciz(ekran, ozet, y, renk)
58
59         ipucu = self.font_k.render(
60             "ENTER: Yukle DEL: Sil ESC: Geri", True,
                ↳ (200, 200, 200))
61         ekran.blit(ipucu, ipucu.get_rect(center=(400,
                ↳ 560)))

```

```

62
63 def _slot_kutusu_ciz(self, ekran, ozet, y, renk):
64     cerceve = pygame.Rect(150, y, 500, 80)
65     pygame.draw.rect(ekran, renk, cerceve, 2)
66     if ozet.get("bos"):
67         metin = self.font_o.render(
68             f"Slot {ozet['slot']}: -- BOS --", True,
69             renk)
70         ekran.blit(metin,
71             metin.get_rect(center=cerceve.center))
72         return
73     if ozet.get("bozuk"):
74         metin = self.font_o.render(
75             f"Slot {ozet['slot']}: BOZUK", True, (255,
76             80, 80))
77         ekran.blit(metin,
78             metin.get_rect(center=cerceve.center))
79         return
80     # Normal slot
81     ust = self.font_o.render(
82         f"Slot {ozet['slot']}: {ozet['oyuncu_adi']}",
83         True, renk)
84     alt = self.font_k.render(
85         f"{ozet['seviye']} | {int(ozet['sure'])}sn |
86         {ozet['altin']} altin",
87         True, renk)
88     zaman = self.font_k.render(ozet['zaman'], True,
89         (150, 150, 170))
90     ekran.blit(ust, (cerceve.x + 15, cerceve.y + 10))
91     ekran.blit(alt, (cerceve.x + 15, cerceve.y + 38))
92     ekran.blit(zaman, (cerceve.x + 15, cerceve.y +
93         58))

```

İpucu: Slot Bilgisini Tazele

`gir` her çağrıldığında `self.slotlar`'ı yeniliyoruz. Çünkü oyuncu `LoadState`'ten çıkıp menüye dönebilir, bir oyunu oynayıp kaydedebilir, sonra tekrar `LoadState`'e gelebilir — slot listesi güncel olmalı. `cik` sırasında yenilemek de işe yarar; `gir`'de tutarsız olamayacak şekilde tazelemek en güvenli.

47.4.3 Kayıt Ekranı: `SaveState`

Yükleme menüsünün simetrisi: `SaveState`. Aynı slot listesi, ama ENTER ile yüklemek yerine “oyunun şu anki durumunu bu slota yaz” anlamına gelir.

Python – save_state.py – Kayıt Menüsü

```

1  import pygame
2  from state import State
3  from confirm_dialog import ConfirmDialogState
4
5  class SaveState(State):
6      def __init__(self, manager, save_mgr, oyun_durumu):
7          super().__init__(manager)
8          self.save_mgr = save_mgr
9          self.oyun_durumu = oyun_durumu  # {oyuncu, seviye,
10             ↪ ilerleme, sure}
11          self.secili = 0
12
13  def gir(self):
14      self.font_b = pygame.font.Font(None, 48)
15      self.font_o = pygame.font.Font(None, 28)
16      self.slotlar =
17             ↪ self.save_mgr.slot_ozetleri(slot_sayisi=3)
18
19  def olaylari_isle(self, olaylar):
20      for olay in olaylar:
21          if olay.type != pygame.KEYDOWN:
22              continue
23          if olay.key == pygame.K_DOWN:
24              self.secili = (self.secili + 1) %
25                 ↪ len(self.slotlar)
26          elif olay.key == pygame.K_UP:
27              self.secili = (self.secili - 1) %
28                 ↪ len(self.slotlar)
29          elif olay.key == pygame.K_RETURN:
30              self._kaydet_secili_slot()
31          elif olay.key == pygame.K_ESCAPE:
32              self.manager.pop()
33
34  def _kaydet_secili_slot(self):
35      slot = self.secili + 1
36      if not self.slotlar[self.secili].get("bos"):
37          # Üzerine yazma onayı iste
38          self.manager.it(ConfirmDialogState(
39              self.manager,
40              soru=f"Slot {slot}'un üzerine yaz?",
41              evet_callback=lambda:
42                 ↪ self._gerçekten_kaydet(slot),
43          ))
44      else:
45          self._gerçekten_kaydet(slot)
46
47  def _gerçekten_kaydet(self, slot):
48      self.save_mgr.kaydet(slot, **self.oyun_durumu)
49      # Slotları tazele

```

```

45         self.slotlar =
           ↳ self.save_mgr.slot_ozetleri(slot_sayisi=3)
46
47         # ... ciz metodu LoadState ile çok benzer ...

```

ConfirmDialogState: dolu slot üzerine yazmadan önce ekstra soru. Yığın deseninin gücünün bir uygulaması — onay penceresi pop edildiğinde **SaveState**'e döner.

47.4.4 Otomatik Kayıt Bildirimi

Autosave gerçekleştiğinde oyuncuya hissettirmen gerek. Basit bir yaklaşım:

Python – Autosave Görsel Bildirim

```

1  class HudCizici:
2      def __init__(self):
3          self.font = pygame.font.Font(None, 22)
4          self.autosave_zaman = 0      # ms cinsinden, son
           ↳ autosave anı
5
6      def autosave_basladi(self):
7          """SaveManager.autosave_kontrol başarıyla yazınca
           ↳ çağrılır."""
8          self.autosave_zaman = pygame.time.get_ticks()
9
10     def ciz(self, ekran):
11         # Son autosave 2 saniye içindeyse göster
12         gecen = pygame.time.get_ticks() -
           ↳ self.autosave_zaman
13         if gecen < 2000:
14             alpha = 255 - int(255 * (gecen / 2000))
15             yuzey = self.font.render(
16                 "Otomatik kayıt yapıldı", True, (255, 215,
           ↳ 0))
17             yuzey.set_alpha(alpha)
18             ekran.blit(yuzey, (ekran.get_width() - 240,
           ↳ 20))

```

İkonlu versiyonu daha iyi olur (sprite + metin), ama prensibi aynı: *görünür, geçici* bir bildirim. Kalıcı bildirim oyuncuyu rahatsız eder.

47.4.5 Slot Sayısını Yapılandırma

Bazı oyuncular 3 slot ister, bazıları 10. **SaveManager** ve **LoadState** her ikisine de uyumlu olmalı:

Python – Slot Sayısı Sabit

```

1 # config.py veya settings.json'dan:
2 MAX_SLOT = 3
3
4 # LoadState içinde:
5 def gir(self):
6     self.slotlar =
7         self.save_mgr.slot_ozetleri(slot_sayisi=MAX_SLOT)

```

Slot sayısını arttırmak yalnızca bu sabit değiştirir; başka hiçbir yerde değişiklik yapmana gerek yok.

İpucu: Slot Sayısı Üst Limiti

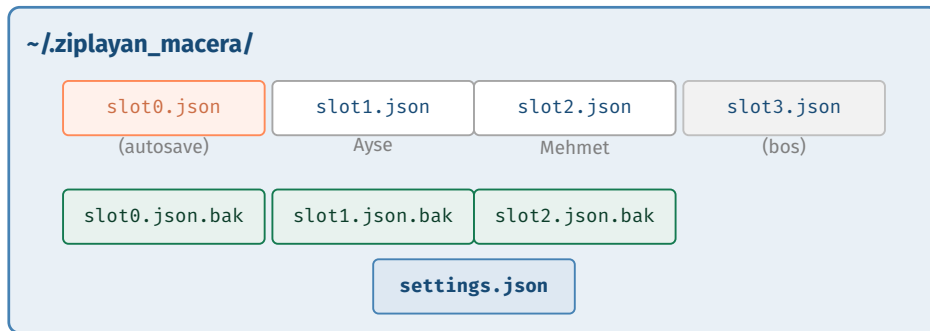
3–5 slot çoğu oyun için yeterli. 10+ slot olunca yükleme menüsünde liste sığmaz, scroll gerekir, kullanıcı kafası karışır. Aşırıya kaçma — “oyuncunun ihtiyacı kadar” kuralına uy.

47.4.6 Toparlama

Bu konuda slot ekosistemi tamamlandı:

- **LoadState**: yükleme menüsü, slot seçimi, silme
- **SaveState**: kayıt menüsü, üzerine yazma onayı
- **ConfirmDialogState**: tehlikeli aksiyon öncesi soru
- Autosave bildirimi: ekranda kısa süreli görünüm
- Slot sayısı tek bir sabit ile yapılandırılır

Slot Sisteminin Disk Organizasyonu



Slot 0 = otomatik kayıt. Slot 1-3 = manuel oyuncu kayıtları. Her .json için .bak yedek dosyası tutulur. settings.json ayrıca.

Şekil 47.3: Üç manuel slot ve bir autosave slotunun disk üzerindeki organizasyonu.

Sıradaki ünite konusu farklı: aynı disk yazma altyapısını oyun *kayıtları* için değil, oyun *ayarları* (ses, kontrol, ekran) için kullanmak.

47.5 Sonuç

Bu ünite de oyununun *belleğini* kazandırdık. Bir kayıt formatı tasarladık, `SaveManager` sınıfıyla yazma ve okuma tarafını inşa ettik, çoklu slot sistemini ve yükleme/kayıt ekranlarını ekledik. Artık Zıplayan Macera oyunu kapanıp açıldığında oyuncunun ilerlemesini hatırlıyor.

47.5.1 Anahtar Kavramlar

Tablo 47.2: Ünite 3 – Anahtar kavramlar

Kavram	Açıklama
Kayıt formatı	JSON şeması: version, oyuncu, seviye, ilerleme, metadata
<code>version</code> alanı	Sürüm geçişi (migration) için zorunlu üst seviye alan
<code>to_dict</code>	Sınıf instance'ından JSON serileştirilebilir sözlük üretme
<code>from_dict</code> (@classmethod)	Sözlükten sınıf instance'ı inşa etme
<code>SaveManager</code>	Kayıt işlemlerinin merkezi arayüz sınıfı
Slot	Bağımsız kayıt yuvası (slot1.json, slot2.json, ...)
Autosave	Belirli aralık veya olay sonrası otomatik kayıt (slot 0)
Atomik yazma	.tmp + replace ile yarım dosya korunması
Yedek (.bak)	Önceki kaydın yedeği; bozuk dosyaya karşı koruma
Sürüm migrasyonu	Eski kayıtları kademeli olarak yeni sürüme yükseltme
<code>LoadState</code>	Yükleme menüsü — slot listesi, seçim, silme
<code>SaveState</code>	Kayıt menüsü — üzerine yazma onayı
<code>ConfirmDialogState</code>	Tehlikeli aksiyon öncesi onay penceresi

47.5.2 Önemli Komutlar

Tablo 47.3: Ünite 3 – Temel komutlar

Komut	Açıklama
<code>Path.home() / ".oyun"</code>	Kullanıcı dizininde gizli klasör
<code>datetime.now().isoformat(timespec="seconds")</code>	İnsan okunabilir zaman damgası
<code>getattr(obj, "alan", "varsayılan")</code>	Güvenli attribute erişimi
<code>d.get("alan", varsayılan)</code>	Güvenli sözlük erişimi (eski kayıtlı)
<code>d.setdefault("alan", varsayılan)</code>	Yoksa ekle, varsa dokunma
<code>tmp.replace(yol)</code>	Atomik dosya rename
<code>yol.unlink()</code>	Dosya silme (<code>Path</code> metodu)
<code>yol.write_bytes(...)</code>	Binary yazma (yedekleme için kısım)
<code>@classmethod from_dict(cls, d)</code>	Sınıf metoduyla instance oluşturma
<code>json.JSONDecodeError</code>	Bozuk JSON için istisna türü
<code>lambda: eylem(arg)</code>	Onay diyalogu için callback geçiri

47.5.3 Bir Sonraki Ünite: Ayarlar ve Konfigürasyon

Save sistemi tamamlandı — oyuncu ilerlemesi kalıcı. Ama bir başka “hatırlanması gereken” kategori daha var: oyuncunun *tercihleri*. Ses seviyesi, müzik açık/kapalı, hangi tuş hangi eyleme bağlı, tam ekran mı pencere mi... Bunlar bir oyundan diğerine değil, *oturumdan oturuma* hatırlanır. Sıradaki ünite:

- Ne tür ayarlar olur, varsayılan değerleri ne olmalı
- `Settings` sınıfı ile tek noktadan ayar erişimi
- Ayarların `settings.json`'da kalıcı tutulması
- Oyun içi ayar menüsü: slider, toggle, key-binding
- Sürüm uyumluluğu: yeni ayar eklendiğinde eski dosyayı kırmamak

İpucu: Bölüm 12'nin Yarıları Tamamlandı

İlk üç ünite oyuna kalıcılık kazandırdı: state akışı, dosya I/O, save/load. Son ünite ise oyuncuya kişiselleştirme imkanı veriyor. Bu dördü birlikte oyununu *ciddi bir uygulama*'ya yaklaştırır. Bölüm sonunda ZıplayanMacera v4 elinde olacak: state machine tabanlı, kalıcı kayıtlı, kişiselleştirilebilir bir oyun.

Ayarlar ve Konfigürasyon

“İyi bir oyun, oyuncusunun zevkine uyum sağlar. Ses yüksekliğinden tuş atamalarına kadar her ayarın hatırlanması, “bu oyun benim” duygusunu güçlendirir.”

— JANE MCGONIGAL, REALITY IS BROKEN

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Bir oyunun ne tür ayarları olur: ses, müzik, ekran, kontroller
- Varsayılan ayarlar (**DEFAULTS**) ile kullanıcı ayarlarını birleştirme
- **Settings** sınıfı ile tek noktadan ayar erişimi
- Ayarların **settings.json** dosyasında kalıcı tutulması
- Oyun içi ayar menüsü: slider, toggle, key-binding satırları
- Sürüm uyumluluğu: yeni ayar eklendiğinde eski ayar dosyasını kırma-dan geçiş

48.1 Bir Oyunun Ayar İhtiyaçları

Save sistemi tamamlandı — karakterin nerede olduğu hatırlanıyor. Şimdi başka bir tür “hatırlama” ihtiyacı: oyuncunun *tercihleri*. Müzik sesini kapatmıştın; oyun yeniden açılınca müzik tekrar açık başlamamalı. Sol-sağ tuşlarını W-D yapmıştın; her seferinde yeniden ayarlamak istemezsin. Ayarlar da kayıt sistemi gibi diske yazılır, ama farklı bir yapıda.

48.1.1 Ayar Kategorileri

Tipik bir oyunda ayarlar dört ana kategoride toplanır:

Tablo 48.1: Tipik oyun ayar kategorileri

Kategori	Tipik Ayarlar
Ses	Genel ses, müzik sesi, efekt sesi, sessiz mod
Görsel	Çözünürlük, tam ekran, V-Sync, parçacık efektleri, ekran sallanması
Kontroller	Tuş atamaları (jump, sol, sağ, ateş), mouse hassasiyeti
Oyun	Zorluk seviyesi, dil, otomatik kayıt aralığı, HUD göster

Bizim Zıplayan Macera için pratik bir başlangıç:

- Ses ayarları (3 değer)
- Tam ekran toggle
- 4 tuş binding (sol, sağ, zıpla, pause)
- Zorluk (kolay/orta/zor)
- Dil seçimi

11 ayar — yönetilebilir bir başlangıç.

48.1.2 Varsayılan Değerlerin Önemi

Her ayarın bir *varsayılan değeri* olmalı. İlk kurulumda, ayar dosyası bozulduğunda veya yeni bir ayar eklendiğinde varsayılan kullanılır.

Python – DEFAULTS Sözlüğü

```

1 import pygame
2
3 DEFAULTS = {
4     "version": 1,
5
6     # Ses (0.0 -- 1.0)
7     "ses_genel": 1.0,
8     "ses_muzik": 0.7,
9     "ses_efekt": 1.0,
10    "sessiz": False,
11
12    # Görsel
13    "tam_ekran": False,
14    "cozunurluk": [800, 600],
15    "vsync": True,
16    "ekran_sallanma": True,
17
18    # Kontroller --- pygame tuş sabitlerinin değerleri
19    "tus_sol": pygame.K_LEFT,
20    "tus_sag": pygame.K_RIGHT,
21    "tus_zipla": pygame.K_SPACE,
22    "tus_pause": pygame.K_ESCAPE,
23

```

```

24     # Oyun
25     "zorluk": "orta",           # "kolay", "orta", "zor"
26     "dil": "tr",               # "tr", "en"
27     "autosave_aralik": 60,     # saniye
28     "hud_goster": True,
29 }

```

⚠ Dikkat: Varsayılanlar JSON'da Sınırlı

`pygame.K_SPACE` aslında bir `int` değer (32). JSON'a yazınca sayı olarak gider, okuyunca sayı olarak gelir; sen kendin `pygame` sabitiyle eşleştirirsin. "Tuş tanımı" tasarımı önemlidir (Konu 4'te detaylanacak).

48.1.3 Save vs Settings: Farklar

Kayıt dosyası ile ayar dosyası birbirine benzer (ikisi de JSON, ikisi de `Path.home()` altında), ama bazı önemli farklar var:

Tablo 48.2: Kayıt vs Ayar dosyası karşılaştırması

Özellik	Save	Settings
Sayı	1–N (slotlar)	Bir tane
Sıklık	Yavaş (oyuncu istediğinde)	Hemen (her değişim)
Yedek	Kritik	İsteğe bağlı
Kapsam	Oyun durumu	Kullanıcı tercihi
Kayıp tolerans	Düşük (oyuncu üzülür)	Orta (varsayıllana döner)
İçerik	Karmaşık (dict, list, sınıf)	Basit (anahtar-değer)

Bu farklar nedeniyle `Settings` sınıfı `SaveManager`'dan biraz farklı tasarlanır — daha basit, daha sık yazma yapan bir yapı.

48.1.4 Tek Bir Yer: `settings.json`

Tüm ayarlar tek bir dosyada toplanır:

Python – `settings.json` İçeriği

```

1 {
2     "version": 1,
3     "ses_genel": 1.0,
4     "ses_muzik": 0.7,
5     "ses_efekt": 1.0,
6     "sessiz": false,
7     "tam_ekran": false,

```



```

8     "cozunurluk": [800, 600],
9     "vsync": true,
10    "ekran_sallanma": true,
11    "tus_sol": 1073741904,
12    "tus_sag": 1073741903,
13    "tus_zipla": 32,
14    "tus_pause": 27,
15    "zorluk": "orta",
16    "dil": "tr",
17    "autosave_aralik": 60,
18    "hud_goster": true
19 }

```

💡 İpucu: Konum: `config.json` mu, `settings.json` mu?

İsim seçimi konvansiyondur. `config.json`, `settings.json`, `prefs.json` tümü aynı işi görür. Bu kitap `settings.json` kullanacak — “ayarlar” Türkçe ile bire bir eşleşiyor.

48.1.5 Birleştirme (Merging)

Diskten okunan ayarlar varsayılanlarla *birleştirilir*. Eski sürüm dosyada bazı yeni ayarlar yoktur; varsayılan kullanılır.

Python – Ayarları Varsayılanla Birleştir

```

1 def birlestir(varsayilan, kullanıcı):
2     """varsayilan'dan kopyala, kullanıcı'nin değerleriyle
   ↳ override et."""
3     sonuc = dict(varsayilan)           # Sığ kopya
4     for k, v in kullanıcı.items():
5         if k in varsayilan:
6             sonuc[k] = v               # Tanınan ayarı al
7             # Tanınmayan ayarları görmezden gel (eski sürüm
   ↳ temizliği)
8     return sonuc

```

İki nokta:

- *Tanınmayan ayarları yutmak*: Eski sürümde olup yeni sürümde silinmiş bir ayar kullanıcı dosyasından görmezden gel. Aksi halde “çöp birikmesi” olur.
- *Tanınan ayarın yokluğu*: Yeni eklenen bir ayar kullanıcı dosyasında yoktur; varsayılan kullanılır — otomatik.

⚠ Dikkat: İç İç Sözlükler için Derin Birleştirme

Üstteki `birlestir` sığdır — iç içe sözlükler için doğru çalışmaz. Bu kitabın ayar yapısı düz olduğu için yeterli; daha karmaşık yapılar için `copy.deepcopy` ve özyinelemeli birleştirme gerekir.

48.1.6 Yapılandırma vs Sabit

Bir değer ne zaman ayar, ne zaman sabit olmalı?

Sabit (kod içinde `ZIP_HIZI = -11`): Oyunun mekanik bütünlüğüne ait; oyuncu değiştirmemeli. Karakterin zıplama yüksekliği, yerçekimi, düşman canı.

Ayar (`settings["ses_muzik"]`): Oyuncunun rahatlığını etkiler; tercih meselesi. Ses, kontroller, görsel kalite.

İpucu: Şüpheli mi? Sabit Tut

Karar veremiyorsan sabit yap. Sabit ayar yapmak kolay (DEFAULTS'a ekle), ayarı sabit yapmak zordur (kullanıcı kayıt dosyaları kırılır). Önce *aşağı seviye sabit*, ihtiyaç olunca *ayar*'a yükselt.

48.1.7 Toparlama

Bu konuda ayar sisteminin felsefesini ve genel haritasını çıkardık:

- Dört ayar kategorisi: ses, görsel, kontrol, oyun
- Her ayarın varsayılanı zorunlu
- Tek dosya: `settings.json`
- Yüklemede varsayılanla birleştirme
- “Şüpheli mi? Sabit tut” kuralı



Hepsi tek bir dosyada toplanır: `settings.json`

Şekil 48.1: Tipik bir oyunun dört ayar kategorisi ve içerdikleri tipik ayarlar.

Sıradaki konuda `Settings` sınıfını yazıyoruz — diskten okuyup yazan, oyunun her yerinden çağrılabilen bir merkezi yapı.

48.2 Settings Sınıfı: Tek Noktadan Erişim

Kategoriler ve varsayılan değerler hazır. Şimdi bunları yöneten sınıfı yazıyoruz.

Hedefimiz: oyunun her yerinden `settings.ses_muzik` veya `settings["ses_muzik"]` diye erişilebilir, değişiklikler otomatik diske yazılan, sade bir API.

48.2.1 Sınıfın İskeleti

Python – settings.py – İskelet

```

1 import json
2 import pygame
3 from pathlib import Path
4
5 DEFAULTS = {
6     "version": 1,
7     "ses_genel": 1.0,
8     "ses_muzik": 0.7,
9     "ses_efekt": 1.0,
10    "sessiz": False,
11    "tam_ekran": False,
12    "cozunurluk": [800, 600],
13    "tus_sol": pygame.K_LEFT,
14    "tus_sag": pygame.K_RIGHT,
15    "tus_zipla": pygame.K_SPACE,
16    "tus_pause": pygame.K_ESCAPE,
17    "zorluk": "orta",
18    "dil": "tr",
19    "autosave_aralik": 60,
20    "hud_goster": True,
21 }
22
23 class Settings:
24     """Oyun ayarlarını yöneten merkezi sınıf."""
25
26     def __init__(self, dosya_yolu=None):
27         if dosya_yolu is None:
28             dosya_yolu = Path.home() / ".ziplayan_macera"
29             ↪ / "settings.json"
30         self.yol = Path(dosya_yolu)
31         self.yol.parent.mkdir(parents=True, exist_ok=True)
32         self._veri = dict(DEFAULTS)
33         self.yukle()

```

`__init__` dört iş yapar: yolu belirle, klasörü oluştur, varsayılanları kopyala, var olan dosyayı yükle.

💡 İpucu: Erken Yükleme

`Settings()` oluşturulduğu anda diskten yükleme yapar. Oyunun `main` dosyasının en başında “`settings = Settings()`” çağırırsan, daha pencere bile açılmadan tam ekran ayarı bilinir ve doğru çözünürlükte pencere açılır.

48.2.2 Yükleme ve Kaydetme

Python – Yukle ve Kaydet

```

1  class Settings:
2      # ... __init__ ...
3
4      def yukle(self):
5          """Diskten ayarları oku, varsayılanlarla
6             ↪ birleştir."""
7          if not self.yol.exists():
8              return # İlk çalıştırma --- varsayılanlar
9                  ↪ zaten yüklü
10         try:
11             with open(self.yol, "r", encoding="utf-8") as
12                 ↪ f:
13                 kullanıcı = json.load(f)
14         except json.JSONDecodeError:
15             print("[Settings] Bozuk dosya, varsayılanlara
16                 ↪ dondu")
17             return
18             # Tanınan ayarları al, tanınmayanları yut
19         for k, v in kullanıcı.items():
20             if k in DEFAULTS:
21                 self._veri[k] = v
22
23     def kaydet(self):
24         """Mevcut ayarları diske yaz."""
25         with open(self.yol, "w", encoding="utf-8") as f:
26             json.dump(self._veri, f, indent=2,
27                 ↪ ensure_ascii=False,
28                     sort_keys=True)

```

⚠ Dikkat: Atomik Yazma Burada da Önemli

Üstteki `kaydet` kısa tutuldu — gerçek üretimde Ünite 2'deki `.tmp + replace` desenini buraya da uygulamalısın. Ayar dosyası küçüktür ama bozulması da rahatsız edicidir.

48.2.3 Erişim API'si: Üç Seçenek

Ayara üç farklı yolla erişim destekleyebiliriz:

Python – Üç Erişim Stili

```

1  class Settings:
2      # ... mevcut kod ...
3
4      # 1) Sözlük stili: settings["ses_muzik"]
5      def __getitem__(self, anahtar):

```

```

6         return self._veri[anahtar]
7
8     def __setitem__(self, anahtar, deger):
9         if anahtar not in DEFAULTS:
10             raise KeyError(f"Bilinmeyen ayar: {anahtar}")
11         self._veri[anahtar] = deger
12         self.kaydet()      # Anında diske yaz
13
14     # 2) Get/Set metot stili: settings.get("ses_muzik")
15     def get(self, anahtar, varsayilan=None):
16         return self._veri.get(anahtar, varsayilan)
17
18     def set(self, anahtar, deger):
19         self[anahtar] = deger      # __setitem__ üzerinden
20
21     # 3) Attribute stili: settings.ses_muzik
22     def __getattr__(self, anahtar):
23         if anahtar.startswith("_"):
24             raise AttributeError(anahtar)
25         if anahtar in self._veri:
26             return self._veri[anahtar]
27         raise AttributeError(f"Bilinmeyen ayar:
    ↪ {anahtar}")

```

Üç stil de aynı veriye erişir. Hangisini seçersen kullanım tercihi: bazı geliştiriciler köşeli parantez sever (`settings["x"]`), bazıları nokta (`settings.x`). Her ikisini de desteklemek esneklik kazandırır.

⚠ Dikkat: `__setattr__` Tehlikeli

Yukarıda yalnızca `__getattr__` tanımladık, `__setattr__` tanımlamadık. `settings.ses_muzik = 0.5` bu nedenle bir attribute oluşturur ama dosyaya yazmaz. Eğer `__setattr__` eklersen, Python'ın iç attribute yönetimi (`self._veri = ...` gibi) ile çakışır. Set için `set` veya `settings["x"] = y` kullan.

48.2.4 Anında Kaydetme: Tetikli Davranış

`__setitem__` her değişimde `kaydet` çağırıyor. Bu, ayar menüsünden bir kaydırıcıyı sürüklerken *her hareket* diske yazma anlamına gelir — saniyede 60 yazma. Çoğu ayar için sorun olmaz, ama bilinçli olunmalı.

İki strateji:

1. **Anında yazma (üstteki gibi):** Basit, oyuncunun değişikliği asla kaybolmaz, ama sık disk yazma yapar.

2. **Toplu yazma (deferred):** Ayar menüsü kapanırken topluca `kaydet` çağrılır. Daha verimli ama oyun çökerse değişiklik kaybolur.

Python – Toplu Yazma Stili

```

1 class Settings:
2     def __init__(self, ...):
3         # ...
4         self._kirli = False      # Yazılmamış değişiklik var
5                                   ↪ mı?
6
7     def __setitem__(self, anahtar, deger):
8         if anahtar not in DEFAULTS:
9             raise KeyError(f"Bilinmeyen ayar: {anahtar}")
10        if self._veri.get(anahtar) != deger:
11            self._veri[anahtar] = deger
12            self._kirli = True
13
14    def yazsin(self):
15        """Ayar menüsü kapanırken çağrılır."""
16        if self._kirli:
17            self.kaydet()
18            self._kirli = False

```

💡 İpucu: Hibrit Yaklaşım

Hassas ayarlar (sessiz, tam ekran toggle) hemen yazılır; slider (ses seviyesi) toplu yazılır. Anlamlı bir denge sağlar.

48.2.5 Kullanım Örneği

Python – Settings Kullanım Örnekleri

```

1 settings = Settings()
2
3 # Okuma --- üç farklı stil
4 ses = settings["ses_muzik"]      # 0.7
5 zorluk = settings.get("zorluk", "orta")
6 zorluk = settings.zorluk         # __getattr__ ile
7
8 # Yazma --- iki stil
9 settings["ses_muzik"] = 0.5      # __setitem__ + auto-save
10 settings.set("zorluk", "zor")    # set metoduyla
11
12 # pygame'e uygulama
13 pygame.mixer.music.set_volume(settings.ses_muzik)
14
15 # Tuş kontrolü
16 tuslar = pygame.key.get_pressed()
17 if tuslar[settings.tus_sol]:
18     oyuncu.sola_git()

```

`settings.tus_sol` ifadesi varsayılan olarak `pygame.K_LEFT` (yani 1073741904

sayısı). Oyuncu A tuşuna ayarlarsa `pygame.K_a` (97) olur. Tuşlar Konu 4'te detayıyla işlenecek.

48.2.6 Pygame Subsystem'leri ile Senkronizasyon

Bazı ayarlar değiştiğinde *anında* pygame ayarına da yansımaları gerek:

Python – Senkronizasyon Yardımcıları

```

1 class Settings:
2     # ... mevcut kod ...
3
4     def pygame_uygula(self):
5         """Mevcut ayarları pygame subsystem'lerine
6         ↪ uygula."""
7         # Ses
8         if self.sessiz:
9             pygame.mixer.music.set_volume(0)
10        else:
11            pygame.mixer.music.set_volume(self.ses_muzik *
12            ↪ self.ses_genel)
13
14        # Pencere
15        if self.tam_ekran:
16            pygame.display.set_mode(self.cozunurluk,
17            ↪ pygame.FULLSCREEN)
18        else:
19            pygame.display.set_mode(self.cozunurluk)

```

`pygame._uygula` oyunun başlangıcında bir kez ve ayar menüsünden çıkışta tekrar çağrılır.

⚠ Dikkat: Çözünürlük Değişimi Pahalı

`pygame.display.set_mode` her çağrıldığında pencere yeniden oluşturulur. OpenGL surface'leri kaybolur, font cache boşalır. Sadece *gerektiğinde* çağır — her ayar değişiminde değil.

48.2.7 Toparlama

`Settings` sınıfının özellikleri:

- Varsayılan değerler kod içinde tanımlı
- Üç erişim stili: dict, get/set, attribute
- Anında veya toplu disk yazma seçenekleri
- Pygame subsystem'lerine uygulama metodu
- Yükleme sırasında tanınmayan ayarları temizleme

Sıradaki konu disk yazımının ince noktaları, kullanıcı klasörünün işletim sistemine göre yerleşimi ve sürüm uyumluluğu üzerine olacak.

48.3 Kalıcı Ayarlar: Konum, Sürüm, Hata Yönetimi

`Settings` sınıfı çalışıyor — ama gerçek dünya kaba. Kullanıcının diski dolu olabilir, dosyaları silinmiş olabilir, yeni sürümde başka ayarlar gelmiş olabilir, başka bir bilgisayara ayar dosyasını taşıyor olabilir. Bu konuda bu kenar durumları kapatıyoruz.

48.3.1 İşletim Sistemine Göre Klasör

`Path.home() / ".ziplayan_macera"` basit bir çözüm ama her işletim sistemi için en uygun yer farklıdır:

Tablo 48.3: İşletim sistemine göre uygulama veri klasörü

Sistem	Önerilen Yer
Linux	<code>~/.config/ziplayan_macera/</code> (XDG standardı)
macOS	<code>~/Library/Application Support/ZiplayanMacera/</code>
Windows	<code>%APPDATA%\ZiplayanMacera\</code> veya <code>%LOCALAPPDATA%</code>

Tüm bunları manuel halletmek külfet. `platformdirs` kütüphanesi bu işi devralır:

Python – platformdirs Kullanımı

```

1  # pip install platformdirs (veya: uv add platformdirs)
2  import platformdirs
3
4  uygulama_dir = Path(platformdirs.user_data_dir(
5      appname="ZiplayanMacera",
6      appauthor="KTU",
7  ))
8  print(uygulama_dir)
9  # Linux:   /home/kullanici/.local/share/ZiplayanMacera/
10 # macOS:   /Users/kullanici/Library/Application
11           ↳ Support/ZiplayanMacera/
12 # Windows:
13           ↳ C:\Users\kullanici\AppData\Roaming\KTU\ZiplayanMacera\

```

💡 İpucu: Bu Kitabın Tercihi

Bu kitap, eğitim amaçlı basitliği koruyup `Path.home() / ".ziplayan_macera"` kullanacak — üç işletim sisteminde de çalışır, görmesi kolay. Üretim oyununa `platformdirs` eklemek bir satırlık değişikliktir.

48.3.2 Versiyonlu Settings

Save sistemi gibi, settings için de **version** alanı şart. İlk başta basit kalsın diye yazmadık; şimdi ekliyoruz:

Python – Versiyonlu Settings

```

1  SETTINGS_VERSION = 2
2
3  class Settings:
4      def yukle(self):
5          if not self.yol.exists():
6              return
7          try:
8              with open(self.yol, "r", encoding="utf-8") as
                 ↪ f:
9                  kullanıcı = json.load(f)
10         except json.JSONDecodeError:
11             return
12
13         # Sürüm kontrol ve migrate
14         v = kullanıcı.get("version", 1)
15         if v < 2:
16             # v1 -> v2: hud_goster eklendi (varsayılanı
                 ↪ al)
17             kullanıcı["hud_goster"] = True
18             v = 2
19             kullanıcı["version"] = SETTINGS_VERSION
20
21         # Geri kalanı varsayılanla birleştir
22         for k, val in kullanıcı.items():
23             if k in DEFAULTS:
24                 self._veri[k] = val

```

Settings migrasyonu kayıt migrasyonundan kolaydır — ayar yapısı düz olduğu için iç içe geçişler nadirdir.

48.3.3 Hata Toleransı: Geçersiz Değerler

JSON'da **ses_muzik: -3.5** olabilir — ya kullanıcı elle düzenlemiştir, ya bug oluşmuştur. Settings yüklerken *validation* yap:

Python – Değer Validasyonu

```

1  class Settings:
2      VALIDATORS = {
3          "ses_genel": lambda v: 0.0 <= v <= 1.0,
4          "ses_muzik": lambda v: 0.0 <= v <= 1.0,
5          "ses_efekt": lambda v: 0.0 <= v <= 1.0,
6          "zorluk":    lambda v: v in ("kolay", "orta",
                 ↪ "zor"),
7          "dil":       lambda v: v in ("tr", "en"),

```

```

8         "autosave_aralik": lambda v: isinstance(v, int)
          ↪ and v >= 10,
9     }
10
11     def yukle(self):
12         # ... mevcut yükleme kodu ...
13         for k, val in kullanıcı.items():
14             if k not in DEFAULTS:
15                 continue
16             if k in self.VALIDATORS and not
          ↪ self.VALIDATORS[k](val):
17                 # Geçersiz değer --- varsayılanlara dön
18                 print(f"[Settings] Geçersiz {k}={val},
          ↪ varsayılanlara donduruldu")
19                 continue # _veri zaten DEFAULTS'tan
          ↪ dolu
20                 self._veri[k] = val

```

Validasyon sayesinde:

- Negatif veya 1'den büyük ses değerleri rastgele dönmez
- Bilinmeyen zorluk seviyesi ("efsane") reddedilir
- Negatif autosave aralığı reddedilir

⚠ Dikkat: Sessiz Düzeltme vs Bildirme

`print` ile kullanıcıya bildiriyoruz ama oyuncu bunu görmez (konsol açık değil). Üretim oyununda bu durumlar bir log dosyasına da yazılmalı (örneğin `settings.log`). Daha ileri seviyede, ayar menüsünde “ayarlarınız sıfırlandı” bildirimi göstermek mümkün.

48.3.4 Reset to Defaults

Oyuncuya “hepsini sıfırla” butonu vermek standart:

Python – Sıfırlama Metodu

```

1 class Settings:
2     def sıfırla(self):
3         """Tüm ayarları varsayılan değerlere döndür ve
          ↪ kaydet."""
4         self._veri = dict(DEFAULTS)
5         self.kaydet()
6
7     def kategori_sıfırla(self, kategori):
8         """Belirli bir kategoriye sıfırla (ses, kontrol,
          ↪ vb.)."""
9         kategoriler = {
10             "ses": ["ses_genel", "ses_muzik", "ses_efekt",
          ↪ "sessiz"],

```

```

11         "kontrol": ["tus_sol", "tus_sag", "tus_zipla",
12             ↪ "tus_pause"],
13         "gorse": ["tam_ekran", "cozunurluk",
14             ↪ "vsync"],
15     }
16     if kategori not in kategoriler:
17         return
18     for k in kategoriler[kategori]:
19         self._veri[k] = DEFAULTS[k]
20     self.kaydet()

```

💡 İpucu: Sıfırlama Onayı

`sifirla` çağrılmadan önce kullanıcıya “Tüm ayarlar varsayılanına dönecek, emin misin?” diye sor. Bu, `ConfirmDialogState` ile yapılır — save silme-deki gibi.

48.3.5 Çoklu Profil (Bonus)

Bir bilgisayarı paylaşan iki kişi farklı ayar profili isteyebilir. Çözüm: ayar dosyasını profile bağla:

Python – Profil Bazlı Settings

```

1 class Settings:
2     def __init__(self, profil="varsayilan",
3         ↪ dosya_yolu=None):
4         if dosya_yolu is None:
5             dosya_yolu = (Path.home() / ".ziplayan_macera"
6                 ↪ /
7                     "profiller" / f"{profil}.json")
8             # ... geri kalan __init__ aynı ...
9
10    # Kullanım:
11    ali_ayarlari = Settings(profil="ali")
12    ayse_ayarlari = Settings(profil="ayse")

```

Profil seçimi ana menüye eklenir. Her profilin kendi save slot’ları da olabilir — save dosyaları profil klasörüne taşınır.

📌 Bilgi: Bu Kitap Tek Profile Yetiniyor

Çoklu profil çoğu eğitim projesi için aşırı. Bu kitap tek profile çalışacak; gerçek üretimde gerek olursa bu deseni uygula.

48.3.6 Ayarların Test Edilmesi

Settings sınıfı, gerçek diske dokunmaz halde test edilebilmeli — aksi halde her test çalıştırışında oyuncunun ayar dosyası kirlenir. İzole etmek için kurucuya yol parametresini ekledik (*dosya_yolu*):

Python – Test Senaryosu

```

1 import tempfile
2 from pathlib import Path
3
4 def test_ayar_kaydetme():
5     with tempfile.TemporaryDirectory() as tmp:
6         s = Settings(dosya_yolu=Path(tmp) /
7             ↪ "test_settings.json")
8         assert s.ses_muzik == 0.7 # varsayılan
9         s["ses_muzik"] = 0.3
10        # Yeniden oku
11        s2 = Settings(dosya_yolu=Path(tmp) /
12            ↪ "test_settings.json")
13        assert s2.ses_muzik == 0.3 # kalıcı
14
15 test_ayar_kaydetme()

```

💡 İpucu: Test İçin Geçici Dizin

`tempfile.TemporaryDirectory()` otomatik silinen bir dizin verir. Test sonrası temizlik kendiliğinden olur. Üretim kodunda *asla* `tempfile` kullanma — gerçek kullanıcı veri klasörü ev dizinindedir.

48.3.7 Toparlama

Bu konuda ayar sistemine üretim sertliği kazandırdık:

- İşletim sistemi farklılıkları (`platformdirs`)
- Sürüm migrasyonu (settings için de `version`)
- Değer validasyonu (kötü değerleri reddet)
- Sıfırlama (full ve kategori bazlı)
- Profil desteği (opsiyonel)
- Test edilebilirlik (yol parametresi)

Sıradaki konu son: tüm bu mantığa bir oyuncu arayüzü (`SettingsState`) ekleyeceğiz.

48.4 Oyun İçi Ayar Menüsü: `SettingsState`

Settings sınıfı arka planda harika çalışıyor. Şimdi oyuncunun ayarları görüp değiştirebileceği bir ekran inşa ediyoruz. Üç farklı UI ögesi gerekiyor: *slider* (ses seviyesi), *toggle* (tam ekran açık/kapalı), *key-binding* satırı (tuş atama). Hepsisi tek bir `SettingsState` altında toplanacak.

48.4.1 Ayar Satırı Kavramı

Bir ayar satırının üç parçası vardır:

- Etiket: “Müzik Sesi”
- Mevcut değer gösterimi: “70%”
- Etkileşim widget’ı: kaydırıcı, on/off göstergesi, atanmış tuş adı

Tüm satırların ortak bir taban sınıfı olabilir, ama eğitim için basit bir veri yapısıyla başlayalım:

Python – Ayar Satır Tanımı

```

1 AYAR_SATIRLARI = [
2     {"tip": "slider", "ad": "ses_genel", "etiket": "Genel
    ↪ Ses",
3     "min": 0.0, "max": 1.0, "step": 0.05},
4     {"tip": "slider", "ad": "ses_muzik", "etiket": "Muzik
    ↪ Sesi",
5     "min": 0.0, "max": 1.0, "step": 0.05},
6     {"tip": "slider", "ad": "ses_efekt", "etiket": "Efekt
    ↪ Sesi",
7     "min": 0.0, "max": 1.0, "step": 0.05},
8     {"tip": "toggle", "ad": "sessiz", "etiket":
    ↪ "Sessiz Mod"},
9     {"tip": "toggle", "ad": "tam_ekran", "etiket": "Tam
    ↪ Ekran"},
10    {"tip": "secim", "ad": "zorluk", "etiket":
    ↪ "Zorluk",
11    "secenekler": ["kolay", "orta", "zor"]},
12    {"tip": "tus", "ad": "tus_sol", "etiket":
    ↪ "Sol"},
13    {"tip": "tus", "ad": "tus_sag", "etiket":
    ↪ "Sag"},
14    {"tip": "tus", "ad": "tus_zipla", "etiket":
    ↪ "Zipla"},
15 ]

```

Veri tabanlı yaklaşım: yeni bir ayar eklemek için sadece bu listeye bir satır eklenir. UI kodu listeyi tarayıp her satırı uygun widget’a render eder.

48.4.2 *SettingsState* İskeleti

Python – settings_state.py

```

1 import pygame
2 from state import State
3
4 class SettingsState(State):
5     def __init__(self, manager, settings):
6         super().__init__(manager)
7         self.settings = settings

```

```

8         self.satirlar = AYAR_SATIRLARI
9         self.secili = 0
10        self.tus_atama_modu = False # Tuş atanırken aktif
11
12    def gir(self):
13        self.font_b = pygame.font.Font(None, 40)
14        self.font_o = pygame.font.Font(None, 26)
15
16    def olaylari_isle(self, olaylar):
17        for olay in olaylar:
18            if olay.type != pygame.KEYDOWN:
19                continue
20            if self.tus_atama_modu:
21                self._tus_atandi(olay.key)
22            else:
23                self._normal_olay(olay.key)
24
25    def _normal_olay(self, tus):
26        satir = self.satirlar[self.secili]
27        if tus == pygame.K_ESCAPE:
28            self.settings.kaydet()
29            self.manager.pop()
30        elif tus == pygame.K_DOWN:
31            self.secili = (self.secili + 1) %
32                        ↪ len(self.satirlar)
33        elif tus == pygame.K_UP:
34            self.secili = (self.secili - 1) %
35                        ↪ len(self.satirlar)
36        elif tus in (pygame.K_LEFT, pygame.K_RIGHT):
37            self._deger_degistir(satir, +1 if tus ==
38                                ↪ pygame.K_RIGHT else -1)
39        elif tus == pygame.K_RETURN:
40            if satir["tip"] == "tus":
41                self.tus_atama_modu = True
42            elif satir["tip"] == "toggle":
43                self.settings[satir["ad"]] = not
44                ↪ self.settings[satir["ad"]]

```

LEFT/RIGHT okları slider veya seçim ayarlarını değiştirir; ENTER tuş atama modunu açar.

48.4.3 Değer Değiştirme: Slider ve Seçim

Python – Değer Değiştirme

```

1    def _deger_degistir(self, satir, yon):
2        ad = satir["ad"]
3        tip = satir["tip"]
4
5        if tip == "slider":

```

```

6         mevcut = self.settings[ad]
7         yeni = mevcut + satir["step"] * yon
8         yeni = max(satir["min"], min(satir["max"], yeni))
9         self.settings[ad] = round(yeni, 2)
10        # Slider değişiminde sesi anında uygula
11        if ad in ("ses_genel", "ses_muzik", "ses_efekt"):
12            self.settings.pygame_uygula()
13
14        elif tip == "secim":
15            secenekler = satir["secenekler"]
16            mevcut = self.settings[ad]
17            try:
18                idx = secenekler.index(mevcut)
19            except ValueError:
20                idx = 0
21            yeni_idx = (idx + yon) % len(secenekler)
22            self.settings[ad] = secenekler[yeni_idx]

```

💡 İpucu: Anında Uygulama

Ses ayarı değiştiğinde `pygame._uygula` çağırarak değişikliği *anlık* hissettiriyoruz. Oyuncu kaydırıcıyı sürüklerken sesi duyar; kaydırma bitince ekrana dönmesine gerek yok.

48.4.4 Tuş Atama Modu

Tuş atamada özel bir akış var: oyuncu ENTER'a basar, sonra atamak istediği tuşa basar. Ekranda "Tuşa basın..." yazısı çıkar:

Python – Tuş Atama

```

1 def _tus_atandi(self, tus):
2     if tus == pygame.K_ESCAPE:
3         self.tus_atama_modu = False
4         return # İptal
5     # Çakışma kontrolü
6     for satir in self.satirlar:
7         if satir["tip"] != "tus":
8             continue
9         if (satir["ad"] !=
10             ↪ self.satirlar[self.secili]["ad"]
11             and self.settings[satir["ad"]] == tus):
12             # Çakışma: bu tuş zaten başka bir eyleme
13             ↪ atanmış
14             print(f"[SettingsState] Tus zaten atanmis:
15             ↪ {satir['ad']}")
16             self.tus_atama_modu = False
17             return
18
19     # Ata

```

```

16 self.settings[self.satirlar[self.secili]["ad"]] = tus
17 self.tus_atama_modu = False

```

▲ Dikkat: Tuş Çakışmasını Önle

İki eyleme aynı tuş atanırsa kaos olur. Yukarıdaki kontrol bunu engeller. Daha gelişmiş: çakışma varsa “X tuşu zaten Y’ye atanmış, yer değiştir mi?” diye sor.

48.4.5 Çizim: Üç Widget Türü

Python – Çizim Metodu

```

1 def ciz(self, ekran):
2     ekran.fill((25, 25, 50))
3     baslik = self.font_b.render("AYARLAR", True, (255,
4         ↪ 215, 0))
5     ekran.blit(baslik, baslik.get_rect(center=(400, 50)))
6
7     for i, satir in enumerate(self.satirlar):
8         y = 110 + i * 45
9         renk = (255, 255, 255) if i == self.secili else
10            ↪ (160, 160, 180)
11         self._satir_ciz(ekran, satir, y, renk)
12
13     if self.tus_atama_modu:
14         self._tus_atama_overlay(ekran)
15
16     ipucu = pygame.font.Font(None, 22).render(
17         "OK: Sec YON: Degistir ENTER: Tus ESC: Kaydet
18         ↪ ve Cik",
19         True, (180, 180, 200))
20     ekran.blit(ipucu, ipucu.get_rect(center=(400, 580)))
21
22 def _satir_ciz(self, ekran, satir, y, renk):
23     etiket = self.font_o.render(satir["etiket"], True,
24         ↪ renk)
25     ekran.blit(etiket, (150, y))
26
27     ad = satir["ad"]
28     deger = self.settings[ad]
29     tip = satir["tip"]
30
31     if tip == "slider":
32         # 200 piksel uzunluğunda bar
33         bar = pygame.Rect(450, y + 10, 200, 8)
34         pygame.draw.rect(ekran, (60, 60, 80), bar)
35         doluluk = int(200 * (deger - satir["min"]) /
36             (satir["max"] - satir["min"]))

```



```

33     dolu = pygame.Rect(450, y + 10, doluluk, 8)
34     pygame.draw.rect(ekran, renk, dolu)
35     yuzde = self.font_o.render(f"{int(deger * 100)}%",
36     ↪ True, renk)
37     ekran.blit(yuzde, (660, y))
38
39     elif tip == "toggle":
40         metin = "ACIK" if deger else "KAPALI"
41         yuzey = self.font_o.render(metin, True, renk)
42         ekran.blit(yuzey, (500, y))
43
44     elif tip == "secim":
45         yuzey = self.font_o.render(f"< {deger} >", True,
46         ↪ renk)
47         ekran.blit(yuzey, (500, y))
48
49     elif tip == "tus":
50         tus_adi = pygame.key.name(deger)
51         yuzey = self.font_o.render(tus_adi.upper(), True,
52         ↪ renk)
53         ekran.blit(yuzey, (500, y))
54
55 def _tus_atama_overlay(self, ekran):
56     kararti = pygame.Surface(ekran.get_size())
57     kararti.set_alpha(180)
58     kararti.fill((0, 0, 0))
59     ekran.blit(kararti, (0, 0))
60     yazi = self.font_b.render("TUSA BASIN...", True, (255,
61     ↪ 215, 0))
62     ekran.blit(yazi, yazi.get_rect(center=(400, 280)))
63     ipucu = self.font_o.render("ESC: Iptal", True, (200,
64     ↪ 200, 200))
65     ekran.blit(ipucu, ipucu.get_rect(center=(400, 320)))

```

💡 İpucu: `pygame.key.name`

`pygame.key.name(K_SPACE)` → "space"; `pygame.key.name(K_a)` → "a". Bu, ekrana kullanıcı dostu tuş ismi yazmak için ideal. `.upper()` ile büyük harfe çevirip "SPACE", "A" gibi göster.

48.4.6 Menüye Bağlama

`MenuState`'in seçenekler listesine "Ayarlar" eklendiyse:

Python – Menüden Bağlantı

```

1 # MenuState içinde:
2 def _secimi_uygula(self):
3     secim = self.secenekler[self.secili]

```

```

4   if secim == "Ayarlar":
5       from settings_state import SettingsState
6       self.manager.it(SettingsState(self.manager,
7                                   ↪ self.settings))
8   # ...

```

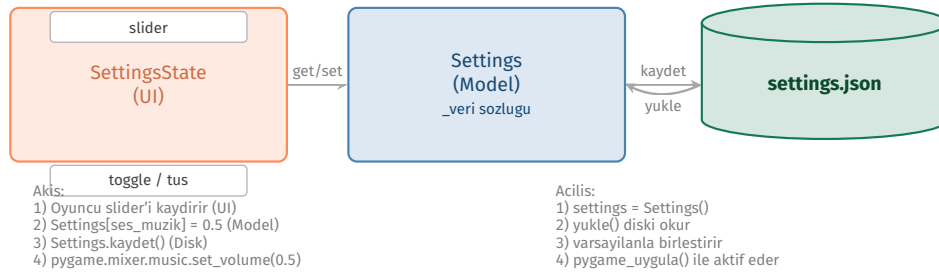
Ayar menüsünden çıkıldığında `pop` ile menüye dönülür — yığın deseninin doğal kullanımı.

48.4.7 Toparlama

`SettingsState` ile ayar sistemi tamamlandı:

- Veri tabanlı satır tanımı (`AYAR_SATIRLARI`)
- Ortak `olaylari\_isle` ve `ciz` desenleri
- Üç widget türü: slider, toggle, seçim, tuş
- Tuş atama modu (overlay ile)
- Çakışma kontrolü
- Anında uygulama (ses) veya çıkışta uygulama (çözünürlük)

Settings Menu Mimarisi: Uc Katman



Şekil 48.2: Ayar menüsünün üç widget türü ve veri akışı: `SettingsState` (UI) → `Settings` (model) → `settings.json` (disk).

Bölüm 12 burada bitiyor. Sonuç bölümünde tüm bu parçaları nasıl birleştireceğimizi göreceğiz: state machine + save sistemi + ayarlar + Bölüm 11'in oyunu = Zıplayan Macera v4.

48.5 Sonuç

Bu üniteye oyuncuya *kişiselleştirme* hakkı verdik. Bir `Settings` sınıfı yazdık, kalıcı ayar dosyasını yönettik, oyun içi ayar menüsüyle slider, toggle ve key-binding sunduk. Artık oyuncu müzik sesini kapatınca yeniden açıldığında müzik kapalı gelmeyi bekliyor — ve geliyor.

48.5.1 Anahtar Kavramlar

Tablo 48.4: Ünite 4 – Anahtar kavramlar

Kavram	Açıklama
Ayar kategorileri	Ses, görsel, kontroller, oyun mantığı
<code>DEFAULTS</code> sözlüğü	Tüm ayarların varsayılan değerleri
<code>Settings</code>	Tek noktadan ayar erişimi sınıfı
<code>platformdirs</code>	İşletim sistemine uygun veri klasörü
Birleştirme (merge)	Diskten okunan ayarları varsayılanla birleştirme
Validasyon	Geçersiz değerleri reddedip varsayılanla dönme
Anında vs toplu yazma	Her değişimde diske yazma vs menü çıkışında yazma
<code>pygame_uygula</code>	Ayarları pygame subsystem'lerine yansıtma
Tuş atama modu	Oyuncunun sıradaki bastığı tuşu yakalama
Tuş çakışması	Aynı tuşu iki eyleme atamayı engelleme
<code>pygame.key.name</code>	Tuş kodundan kullanıcı dostu isim
Reset to defaults	Tüm ayarları sıfırlama
Profil	Aynı bilgisayarda farklı kullanıcılar için ayar setleri

48.5.2 Önemli Komutlar

Tablo 48.5: Ünite 4 – Temel komutlar

Komut	Açıklama
<code>settings["ses_muzik"]</code>	Sözlük stili erişim
<code>settings.ses_muzik</code>	Attribute stili erişim (<code>__getattr__</code>)
<code>settings.set("x", v)</code>	Get/set metot stili
<code>settings.kaydet()</code>	Mevcut ayarları diske yaz
<code>settings.yukle()</code>	Diskten ayarları oku
<code>settings.sifirla()</code>	Hepsini varsayılanla döndür
<code>pygame.key.name(tus_kodu)</code>	Tuş ID'sinden okunabilir isim
<code>pygame.mixer.music.set_volume(v)</code>	Müzik sesi (0.0 – 1.0)
<code>pygame.display.set_mode(boyut, FULLSCREEN)</code>	Tam ekran moda geç
<code>tempfile.TemporaryDirectory()</code>	Test için geçici dizin

48.5.3 Bölüm 12'nin Özeti: Zıplayan Macera v4

Bu bölüm boyunca oyununu *ciddi bir uygulama*'ya dönüştüren dört temel yetenek ekledik:

1. **State Machine** (Ünite 1): Menü, oyun, pause, game over arasında temiz geçişler. Her ekran kendi sınıfı, her geçiş yığın tabanlı.
2. **Dosya İşlemleri** (Ünite 2): JSON ile yapılandırılmış veri okuma/yazma; pickle ile karmaşık nesne serileştirme. Hangi durumda hangisinin tercih edileceği.

3. **Save/Load Sistemi** (Ünite 3): **SaveManager** ile çoklu slot yönetimi, atomik yazma, yedek dosyalar, autosave, sürüm migrasyonu, tam yükleme/kayıt menüsü.
4. **Ayarlar** (Ünite 4): **Settings** sınıfı, kalıcı ayar dosyası, üç widget türlü oyun içi ayar menüsü, tuş binding.

Bu dördü Bölüm 11'in oyununun üstüne eklenince ortaya *Zıplayan Macera v4* çıkıyor:

- Açılır, ana menü görünür
- Oyun başlat veya kayıt yükle seçilir
- Oyun oynanır; ESC ile pause, kayıt menüsü
- Ölüncü game over, tekrar dene veya menüye dön
- Ayarlar her zaman erişilebilir; değişiklikler kaydedilir
- Otomatik kayıt arka planda devam eder

48.5.4 Bir Sonraki Bölüm: Gelişmiş Konular ve Optimizasyon

Oyunun çekirdeği tamam, sağlam temelli, kalıcı, kişiselleştirilebilir. Bölüm 13'te performans, profileme, veri-odaklı tasarım, ileri animasyon teknikleri gibi konularla bu temelin üstüne çıkacağız.

💡 İpucu: Bölüm 12'yi Tamamlama

Şimdi pratiğe geç. [out/LabExamples/Lab12_Oyun_Durumu_Veri/](#) altındaki başlangıç kodları seni dört çalışma boyunca yönlendirecek:

1. Çalışma 1: Mini state machine (Menu – Play – Pause)
2. Çalışma 2: JSON ile basit save/load
3. Çalışma 3: Çoklu slot sistemi + autosave
4. Çalışma 4: **Settings** sınıfı + ayar menüsü

Bunları tamamlayıp birleştirdiğinde *Zıplayan Macera v4*'ün çekirdeği elinde olacak.

📌 Bilgi: Bölüm Sonu Pratiği

Bölüm 11'in mevcut Python kodunu ([kod_ornekleri/bolum11/projeler/ziplayan_macera](#)) açıp, manuel olarak `state.py`, `state_manager.py`, `play_state.py` dosyalarını ekleyerek mimariyi ZM v4'e yükseltmeye çalış. Önce Çalışma 1'i bitir; sonra 2, 3, 4. Bitirdikten sonra v4 kodunu okudun mu, anladın mı, kendi başına yeniden yazabilir misin — gerçek anlama buradan geliyor.

Performans Optimizasyonu

*“Erken optimizasyon tüm kötülüklerin anasıdır
— ama bir oyun 60 FPS yerine 30 FPS’te çalışıyorsa, optimizasyon artık “erken” değildir.”*

— DONALD KNUTH (UYARLANMIŞ)

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- `time.perf_counter` ve `cProfile` ile kodun nerede yavaşladığını ölçme
- Hot path ve darboğaz kavramları; 60 için `16.67` ms kare bütçesi
- Culling ile ekran dışındaki sprite’lar için hem çizimi hem güncellemeyi atlamak
- Object pooling ile mermi ve parçacık nesnelerini sürekli yaratıp yok etmek yerine yeniden kullanmak
- Uzay Kaçışı oyununu profilelemek, darboğazları tespit edip ölçülebilir bir kazanca dönüştürmek

49.1 Profiling: Yavaşlığın Kaynağını Ölçmek

Bir önceki bölümde Zıplayan Macera v3’ü tamamladın. Şimdi Uzay Kaçışı’nı yeniden açıyorsun, ekrana yüzlerce düşman ve mermi saçyorsun — ve fark ediyorsun ki kare hızı 60’tan 35’e düşüyor. Hareket sarsılıyor, mermi atışları geç tepki veriyor. Peki kod tam olarak *nerede* yavaşlıyor? Çizimde mi? Çarpışma kontrolünde mi? Müzik çözümlemede mi?

Bu sorunun cevabını “tahmin ederek” aramak en yaygın hatadır. Profesyonel oyun geliştirmede tek bir kural vardır: önce *ölç*, sonra optimize et. Bu konuda bir oyun karesinin zaman bütçesini, `time.perf_counter` ile manuel ölçümü ve `cProfile` ile otomatik profil çıkarmayı öğreneceksin.

■ Tanım: Profiling

Profiling, bir programın hangi fonksiyonlarda ne kadar zaman harcadığını ölçmek anlamına gelir. Bir profiler her fonksiyon çağrısının süresini, kaç kez çağrıldığını ve toplam süreye oranını raporlar. Programın *darboğazlarını* (hot path) görmenin tek doğru yolu profil çıkarmaktır.

49.1.1 Kare Bütçesi: 16.67 ms

60 demek saniyede 60 kare demek. O hâlde her kareye düşen toplam süre:

$$\frac{1 \text{ saniye}}{60 \text{ kare}} = \frac{1000 \text{ ms}}{60} \approx 16.67 \text{ ms}$$

Bu 16.67 ms içine *her şey* sığmak zorunda: olay işleme, oyuncu güncellemesi, düşman güncellemesi, mermi güncellemesi, çarpışma kontrolü, çizim, sonra `display.flip()`. Eğer kareyi 20 ms’de tamamlıyorsan, oyunun gerçekte $1000/20 = 50$ FPS’te çalışıyor demektir.

Tablo 49.1: Hedef FPS değerleri ve kare bütçeleri

FPS	Kare Bütçesi	Kullanım
30	33.33 ms	Mobil, eski cihazlar, bazı konsol oyunları
60	16.67 ms	Standart PC ve modern konsol hedefi
120	8.33 ms	Yüksek hızlı monitörler, kompetitif oyunlar
144	6.94 ms	Üst düzey gaming monitörleri

▲ Dikkat: Bütçeyi Aşmak “Donma” Demektir

Tek bir karede bütçeyi aşarsan, `pygame.time.Clock.tick` bir sonraki karede telafi etmeye çalışır ama bunu yapamaz; kare hızı düşer ve oyuncu *takıldığını* hisseder. Bütçe aşımı görsel olarak “stutter” (kekemelik) şeklinde algılanır.

49.1.2 Manuel Ölçüm: `time.perf_counter`

Bir kod bloğunun ne kadar sürdüğünü en hızlı şekilde ölçmenin yolu `time.perf_counter` kullanmaktır. Bu fonksiyon *tek tonlu* (monotonic) bir saatten çok hassas zaman damgası döndürür — saniyenin milyonda biri seviyesinde.

Python – Bir Bloğun Süresini Ölçme

```

1 import time
2
3 baslangic = time.perf_counter()
4
5 # Ölçmek istediğin kod buraya:
6 for sprite in tum_sprite_grubu:
7     sprite.update(dt)
```

```

8
9 bitis = time.perf_counter()
10 gecen_ms = (bitis - baslangic) * 1000
11 print(f"Update süresi: {gecen_ms:.2f} ms")

```

Bu yöntemle oyun döngüsünün her aşamasını ayrı ayrı ölçebilirsiniz:

Python – Kare Aşamalarını Bölmek

```

1 while calistir:
2     to = time.perf_counter()
3     # 1) Olaylar
4     for olay in pygame.event.get():
5         ...
6     t1 = time.perf_counter()
7     # 2) Güncelleme
8     tum_gruplar.update(dt)
9     t2 = time.perf_counter()
10    # 3) Çarpışma
11    pygame.sprite.groupcollide(mermiler, dusmanlar, True,
12    ↪ True)
13    t3 = time.perf_counter()
14    # 4) Çizim
15    ekran.fill((0, 0, 0))
16    tum_gruplar.draw(ekran)
17    pygame.display.flip()
18    t4 = time.perf_counter()
19
20    print(f"olay={1000*(t1-to):.1f}
21    ↪ update={1000*(t2-t1):.1f} "
22    ↪ f"carpisma={1000*(t3-t2):.1f}
23    ↪ cizim={1000*(t4-t3):.1f}")

```

Birkaç saniye oyunu çalıştır, terminalden çıkan sayıları izle. Hangi aşama en büyük sayıya sahipse darboğaz orada saklı.

💡 İpucu: Ortalamayı Al, Tek Kareye Bakma

Tek bir karenin süresi tesadüfen yüksek olabilir (işletim sistemi arada başka iş yapıyor olabilir). Saniyede 60 tane veriyi tek tek yazdırmak yerine 60 karenin ortalamasını al ve saniyede bir kez yazdır. Daha okunaklıdır.

49.1.3 Otomatik Profil: cProfile

Manuel ölçüm *nerede* yavaş olduğunu söyler ama *hangi fonksiyonda* yavaş olduğunu söylemez. Bunun için Python'un yerleşik **cProfile** modülünü kullanırız. **cProfile** programdaki *her* fonksiyon çağrısını sayar ve süresini ölçer.

Python – cProfile ile Oyun Döngüsünü Sarmak

```

1 import cProfile
2 import pstats
3
4 profiler = cProfile.Profile()
5 profiler.enable()      # ölçüm başlasın
6
7 main()                # tüm oyun döngüsü burada koşar
8
9 profiler.disable()    # ölçüm bitsin
10 istatistikler = pstats.Stats(profiler)
11 istatistikler.sort_stats('cumulative')
12 istatistikler.print_stats(20)  # ilk 20 sırayı göster

```

Çıktıyı şöyle bir tablo gibi düşün:

Python – Tipik cProfile Çıktısı

```

1 ncalls  tottime  percall  cumtime  percall
  ↳ filename:lineno(function)
2   3600    2.450    0.001    5.120    0.001
  ↳ sprite.py:215(groupcollide)
3  216000    1.820    0.000    1.820    0.000
  ↳ sprite.py:298(rect)
4   3600    0.940    0.000    0.940    0.000
  ↳ surface.py:411(blit)
5    60    0.380    0.006    0.380    0.006
  ↳ mixer.py:120(decode)

```

- **ncalls**: Fonksiyonun toplam çağrı sayısı
- **tottime**: O fonksiyonda *kendi içinde* geçen süre (alt çağrılar hariç)
- **cumtime**: O fonksiyon *ve* altındaki tüm çağrılarda geçen toplam süre
- **percall**: Çağrı başına ortalama süre

`sort_stats('cumulative')` bize en çok zaman harcanan fonksiyonu en üstte gösterir. `'tottime'` ile sıralarsak alt çağrılar olmayan en yavaş çekirdek fonksiyon en üstte gelir. `'ncalls'` ile sıralayınca da en sık çağrılan fonksiyonu görürüz; gereksiz tekrarlı çağrılar bu sıralamada öne çıkar.

49.1.4 %80–%20 Kuralı ve Hot Path

Profil çıktısının en kritik gözlemi *Pareto ilkesi*dir: neredeyse her programda toplam zamanın %80'i *birkaç fonksiyonda* harcanır. Bunlara *hot path* denir.

■ Tanım: Hot Path

Bir programın çalışma zamanının büyük kısmını tek başına tüketen kod yoluna *hot path* (sıcak yol) denir. Optimizasyon çabasını burada yoğunlaştırmak, çoğu zaman tüm kodu eşit oranda iyileştirmekten çok daha verimlidir.

Bu yüzden cProfile çıktısının *ilk üç satırı* dışındaki satırları *önce göz ardı et*. `groupcollide`'da 5 s geçen bir programda `set_caption`'ın 0.001 s'sini kısaltmanın anlamı yoktur.

▲ Dikkat: Optimizasyondan Önce Profil

Donald Knuth'un meşhur sözü tam olarak şöyledir: “*Erken optimizasyon tüm kötülüklerin anasıdır.*” Yani profil çıkarmadan “bu döngü yavaştır galiba” deyip kodu karmaşılaştırma. Ölçü yoksa kazanç da yoktur.

49.1.5 Bu Üniteden Çıkacağımız Yer

Sıradaki üç konuda profil sonuçlarına dayanarak iki yaygın darboğazı çözeceğiz: ekran dışı çizim (culling) ve sürekli nesne yaratma maliyeti (object pooling). Sonra Konu 4'te tüm bunları Uzay Kaçışı'nın gerçek profilinde uygulayıp önce-sonra sayılarını ölçeceğiz.

● Bilgi: Pratik Alıştırma

`02_basit_profil.py` dosyasını çalıştır. Oyun döngüsü `cProfile` ile sarılmıştır; oyunu birkaç saniye oynayıp kapat. Terminalde çıkan ilk 5 satırı bir kenara not al — sonraki konularda bu satırların değişimini izleyeceksin.

49.2 Culling: Görmediğini Çizme, Hesaplama

Geniş bir tile haritası düşün: Bölüm 10'da Bölüm 39 ile yaptığın gibi 200 ekran genişliğinde uzun bir seviye. Haritada 5,000 taş duvar sprite'ı var; ama kameranda aynı anda en fazla 50'si görünüyor. Eğer her karede 5,000 sprite'ın hepsini güncelleyip ekrana gönderiyorsan, 4,950 sprite'ı çizmek için boşuna efor harcıyorsun — üstelik onlar hiçbir piksele dönüşmeyecekler. İşte tam burada *culling* devreye girer.

■ Tanım: Culling

Culling (eleme/budama), her karede sadece görünür ya da yakın zamanda görünür olabilecek nesneleri işlemek anlamına gelir. 2D oyunlarda en yaygın türü *viewport culling*'dir: kamera dikdörtgeninin dışındaki sprite'lar ne çizilir ne de güncellenir. 3D oyunlarda buna *frustum culling* denir.

Konu 1'de profil çıkardığımız Uzay Kaçışı çıktısında `Surface.blit` ve `Group.draw` satırlarının çoğu zaman ilk üçte yer aldığını göreceksin. Çizim çağrıları pahalıdır; her `blit` ekran piksellerine bayt taşır. Gerçekten görünmeyen bir nesneyi çizmek tam bir israftır.

49.2.1 Viewport: Kameranın Dünyaya Açılan Penceresi

Önce kameranın dünya koordinatlarındaki dikdörtgenini hatırla. `kamera.x` ve `kamera.y` dünyada hangi pikselin sol-üst köşesinde olduğunu, ekranın boyutu da görünür alanın genişlik ve yüksekliğini verir.

Python – Viewport Dikdörtgeni

```

1 import pygame
2
3 class Kamera:
4     def __init__(self, genislik, yukseklik):
5         self.x = 0
6         self.y = 0
7         self.genislik = genislik
8         self.yukseklik = yukseklik
9
10    @property
11    def viewport(self):
12        """Dünya koordinatlarında görünür alan."""
13        return pygame.Rect(self.x, self.y,
14                            self.genislik, self.yukseklik)

```

`viewport` bir `pygame.Rect` olduğu için `collidect` metodu vardır. Bu metot iki dikdörtgenin kesişip kesişmediğini $O(1)$ zamanda söyler.

49.2.2 Çizimi Filtrelemek

En basit culling, `Group.draw`'u atlayıp sadece kesişen sprite'ları manuel `blit` etmektir.

Python – Culling'siz Çizim - Tüm Sprite'lar

```

1 def cizim(ekran, grup, kamera):
2     for sprite in grup:
3         ekran.blit(sprite.image,
4                     (sprite.rect.x - kamera.x,
5                      sprite.rect.y - kamera.y))

```

Python – Culling ile Çizim - Sadece Görünür

```

1 def cizim(ekran, grup, kamera):
2     viewport = kamera.viewport
3     cizilen = 0
4     for sprite in grup:
5         if sprite.rect.collidect(viewport):
6             ekran.blit(sprite.image,
7                         (sprite.rect.x - kamera.x,
7                          sprite.rect.y - kamera.y))
8             cizilen += 1
9     return cizilen
10

```

İkinci sürümde her karede *sadece* görünür sprite'lar çizilir. **cizilen** sayacı debug için faydalıdır: 5,000 sprite'lı haritada bu değer çoğu zaman 30–80 arasında kalacaktır.

⚠ Dikkat: Sprite Boyutuna Dikkat

Eğer sprite gerçekten ekrana *taşıyorsa* (yarısı viewport'un dışında, yarısı içinde), **collidirect True** döner ve sprite çizilir. Bu doğru davranıştır. Sadece tamamen dışta kalan sprite'lar atlanır.

49.2.3 Update'i de Atlamak

Culling'in en güçlü hâli *çizimi* değil, *güncellemeyi* de atlamaktır. Ekrandan 30 ekran uzakta olan bir düşmanı her karede güncellenmenin anlamı var mı? Çoğu durumda yok.

Python – Culling ile Güncelleme

```
1 def gorunur_guncelle(grup, kamera, dt, pay=200):
2     """Sadece kameraya yakın sprite'ları güncelle."""
3     aktif_alan = kamera.viewport.inflate(pay * 2, pay * 2)
4     for sprite in grup:
5         if sprite.rect.collidirect(aktif_alan):
6             sprite.update(dt)
```

Burada **inflate** ile viewport'u her yönde **pay** piksel genişlettik. Neden? Çünkü kamera hareket ettiğinde tam ekran kenarındaki düşman *aniden* güncellenmeye başlamalı, “kasılarak” canlanmamalı. **pay** kadar bir bant tutmak, kameranın hareketinden hemen önce nesneleri ısıtır.

💡 İpucu: Pay'ı Hareket Hızına Göre Seç

Kamera saniyede 200 px hareket ediyorsa, 1 saniyelik bir pay 200 px etmelidir. Daha agresif olmak için 400 px de seçebilirsiniz. Çok küçük pay görsel hatalara, çok büyük pay performans kaybına yol açar.

49.2.4 Hangi Nesneler Asla Cull Edilmemeli

Bazı nesnelerin culling'den *muaf* tutulması gerekir:

- **Oyuncu sprite'ı:** Kamera onu takip eder; her zaman güncellenmelidir.
- **Aktif takip eden düşmanlar:** Patrolde olan bir düşman ekran dışına çıkıp tekrar girince yerinden devam etmelidir. (Bölüm 11 Ünite 3 Patrol AI)
- **Mermiler:** Hızlı hareket ettikleri için pay bandını anında geçerler.
- **Skoru ve canı etkileyen sprite'lar:** Ekran dışındayken oyunun durumuna etki etmemeli.

Pratikte bunlar için ayrı bir grup (örn. **her_karede_grup**) tutmak ve sadece dekoratif/statik sprite'lara culling uygulamak en güvenli yoldur.

49.2.5 Mini Karşılaştırma: Kazanç Ne Kadar?

`03_culling_test.py` dosyasında 5,000 rastgele tile sprite'ından oluşan bir sahne hazırladık. Aynı sahne hem culling ile hem de culling'siz koşulduğunda tipik sonuç şöyledir:

Tablo 49.2: Culling'in 5,000 sprite'lı sahnedeki etkisi

Yöntem	Kare süresi	FPS
Culling yok	42 ms	24
Çizim culling	11 ms	90
Çizim + Update culling	7 ms	140

Görüldüğü gibi tek bir `collidect` kontrolü tüm karenin süresini 42 msaniyeden 7 msaniyeye indirebilir. Üstelik kod karmaşıklığı çok az artmıştır.

● Bilgi: Pratik Alıştırma

`03_culling_test.py` dosyasını çalıştır. F1 tuşuyla culling'i aç-kapa yapabilirsin. Ekrana bastırılan *cizilen* ve *toplam* sayılarını izle. Kamerayı haritanın ortasına götür, sonra köşelere; sayıların nasıl değiştiğini gör.

49.3 Object Pooling: Mermiyi Yok Etme, Yeniden Kullan

Bölüm 24 bölümünde mermilere çarpışma tepkisi yazarken `kill` çağrısını kullanmıştın: çarpışan mermi gruptan çıkarılıp *yok* ediliyordu. Ardından oyuncu yeni mermi atınca `Mermi(...)` çağrısıyla yepyeni bir nesne yaratılıyordu. Bu, kavramsal olarak temiz bir yaklaşım ama performans açısından gizli bir sorun barındırır: *her saniye yüzlerce nesne yaratılıp yok ediliyor*.

Konu 1'de profil çıkardığımız Uzay Kaçışı'nda büyük bir saldırı sahnesinde `Sprite.__init__` fonksiyonunun ilk on satırda yer aldığını görürsün. Sebebi tam olarak budur: Python her `Mermi()` çağrısında bellek ayırır, `convert\_alpha` çalıştırır, gruba kayıt yapar. Bunların hepsi mikroskobik ama toplamda kareyi yiyen bir maliyettir.

■ Tanım: Object Pool

Bir *object pool*, sabit sayıda önceden yaratılmış nesneyi saklayan ve ihtiyaç oldukça *ödünç verip* sonra *geri alan* bir veri yapısıdır. Yeni nesne yaratmak yerine havuzdan "boştaki" bir nesneyi alır, kullanırsın, işin bittiğinde havuza geri verirsın. Yaratma ve çöp toplama maliyetleri böylece 0'a iner.

49.3.1 Niye Pool Kullanmalı?

Üç ana sebep:

1. **Yaratma maliyeti:** `__init__`, `convert\_alpha`, `Surface` kopyalama, gruba ekleme — bunların hepsi mikrosaniyeler tüketir. 200 nesne için $200 \times \mu s$ büyür.
2. **Çöp toplama (GC) duraklamaları:** Python'un GC'si arada bir tetiklenip ölü nesneleri toplar. Sürekli ölen yüzlerce nesne GC'yi sık tetikler ve bu duraklamalar oyunu *sarsar*.
3. **Bellek parçalanması:** Sürekli alma-bırakma bellekte boşluklar bırakır; tahsis hızı düşer.

⚠ Dikkat: Pool Her Zaman Doğru Çözüm Değildir

Saniyede üç tane menü düğmesi yaratıyorsan pool'a ihtiyacın yok. Pool, nesnenin *çok ve sık* yaratılıp yok edildiği durumlar için anlamlıdır: mermiler, parçacıklar, hasar sayıları, kısa ömürlü patlama efektleri.

49.3.2 En Basit Pool: **acquire** ve **release**

Önce kavramı saf Python'da görelim:

Python – Generic Object Pool

```

1 class Pool:
2     def __init__(self, sinif, kapasite):
3         self.kullanilabilir = [sinif() for _ in
4             ↪ range(kapasite)]
5         self.aktif = []
6
7     def acquire(self):
8         if not self.kullanilabilir:
9             return None # havuz boş
10        nesne = self.kullanilabilir.pop()
11        self.aktif.append(nesne)
12        return nesne
13
14    def release(self, nesne):
15        if nesne in self.aktif:
16            self.aktif.remove(nesne)
17            self.kullanilabilir.append(nesne)

```

- **acquire:** “Boştaki” bir nesne ver, “aktif” listesine taşı.
- **release:** Nesneyi aktiften al, boştalara geri koy.

`kullanilabilir` ve `aktif` ayrı listeler tutmamızın nedeni iterasyonda hızdır: aktif olan nesneleri her karede çizmek ya da güncellemek için tek liste yeterli.

49.3.3 Pygame Sprite ile Pool: **kill** Yerine **active**

`pygame.sprite.Sprite` doğal olarak pool'a uyumlu değildir çünkü `kill` sprite'ı tüm gruplardan siler. Bunun yerine sprite'a bir `active` bayrağı koyarız:

Python – Pool'a Uyumlu Mermi

```

1 class Mermi(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.image = pygame.Surface((6, 12),
5             ↪ pygame.SRCALPHA)
6         pygame.draw.rect(self.image, (255, 220, 100),
7             self.image.get_rect())
8         self.rect = self.image.get_rect()
9         self.vy = 0
10        self.active = False
11
12    def firlat(self, x, y, vy=-9):
13        """Pool'dan alındığında çağrılır."""
14        self.rect.center = (x, y)
15        self.vy = vy
16        self.active = True
17
18    def update(self, dt):
19        if not self.active:
20            return
21        self.rect.y += self.vy
22        # Ekran dışına çıkınca havuza dön
23        if self.rect.bottom < 0:
24            self.active = False

```

`active = False` olan mermi ne çizilir ne güncellenir; ama nesne hâlâ bel-
lektedir, bir sonraki ateşte yeniden kullanılır.

Python – Mermi Pool'unu Kullanmak

```

1 mermi_pool = Pool(Mermi, kapasite=64)
2 mermi_grubu = pygame.sprite.Group()
3 for m in mermi_pool.kullanilabilir:
4     mermi_grubu.add(m)
5 for m in mermi_pool.aktif:
6     mermi_grubu.add(m)
7
8 # Ateş etme:
9 def ates_et(x, y):
10     mermi = mermi_pool.acquire()
11     if mermi is None:
12         return # havuz dolu, yeni mermiye yer yok
13     mermi.firlat(x, y)

```

`ates_et` yeni nesne yaratmıyor; sadece mevcut bir mermiyi uyandırıp konum-
landırıyor. Maliyet: bir liste pop'u ve bir kaç atama — nano saniyeler.

49.3.4 Pool Boşluğunu İzlemek

Mermi havuzundaki kapasite oyun tasarımının bir parçasıdır. Aynı anda ekrandaki maksimum mermi sayısını biliyorsan kapasiteyi buna göre belirleyebilirsin: 64, 128, 256. Eğer `acquire None` dönmeye başlamışsa kapasite yetmiyor demektir. Bu durumda iki seçenek var: ya kapasiteyi artır ya da en eski mermiyi zorla geri al ve onu kullan (rotational reuse).

İpucu: Debug Sayaç

Aktif/toplam oranını ekrana basmak çok yardımcıdır:
`f"{len(pool.aktif)}/{len(pool.kullanilabilir)+len(pool.aktif)}".`
 Pool'un doluluk oranı oyunun zorluk eğrisini de gösterir.

49.3.5 Parçacık Havuzu: Patlama Efekti

Mermiyle aynı mantık parçacıklar (particle) için de geçerlidir — aslında parçacıklar pool için çok daha kritiktir, çünkü tek bir patlamada 50 parçacık doğar. Patlama efekti şöyle çalışır:

Python – Parçacık Pool ile Patlama

```

1  import random
2
3  class Parçacık(pygame.sprite.Sprite):
4      def __init__(self):
5          super().__init__()
6          self.image = pygame.Surface((4, 4))
7          self.image.fill((255, 180, 60))
8          self.rect = self.image.get_rect()
9          self.vx = 0; self.vy = 0
10         self.omur = 0
11         self.active = False
12
13     def firlat(self, x, y):
14         self.rect.center = (x, y)
15         aci = random.uniform(0, 6.283)
16         hiz = random.uniform(2, 6)
17         self.vx = hiz * pygame.math.Vector2(1,
18             ↪ 0).rotate_rad(aci).x
19         self.vy = hiz * pygame.math.Vector2(1,
20             ↪ 0).rotate_rad(aci).y
21         self.omur = 30 # 30 kare
22         self.active = True
23
24     def update(self, dt):
25         if not self.active:
26             return
27         self.rect.x += int(self.vx)
28         self.rect.y += int(self.vy)

```

```

27         self.omur -= 1
28         if self.omur <= 0:
29             self.active = False
30
31     parcacik_pool = Pool(Parcacik, kapasite=300)
32
33     def patlama(x, y, sayi=40):
34         for _ in range(sayi):
35             p = parcacik_pool.acquire()
36             if p is None:
37                 break
38             p.firlat(x, y)

```

`patlama` fonksiyonu bir karede 40 parçacık aktive eder ama *tek bir* `Parcacik()` çağırısı yapmaz. Tüm parçacıklar oyun başında bir kez yaratılmıştır.

● Bilgi: Pratik Alıştırma

`04_mermi_havuz.py` dosyasını incele. Önce pool'suz sürümü çalıştır ve cProfile ile profilini al. Sonra pool'lu sürüme geç ve aynı profil komutunu yeniden koştur. `Sprite.__init__`, `Surface` ve `convert_alpha` satırlarının cumtime sıralamasından düştüğünü göreceksin.

49.4 Uygulama: Uzay Kaçışı'nın Frame Profili

Şimdiye kadar profili öğrendin, culling'i öğrendin, pool'u öğrendin — ama bunları gerçek bir oyunda *birlikte* uygulamadın. Bu konuda Bölüm 8 sonunda tamamladığın Uzay Kaçışı v3 oyununu açacağız, profil çıkaracağız, darboğazları tespit edeceğiz, optimize edeceğiz ve önce-sonra rakamlarını karşılaştıracacağız. Asıl ders bu adımların *sırasındadır*: ölç → değiştir → tekrar ölç.

▲ Dikkat: Sıralamayı Bozma

Optimizasyon yaparken en sık yapılan hata önce kodu değiştirip sonra ölçmeye çalışmaktır. Doğru sıra: *Önce ölç, kanıtla ki yavaş; değiştir; tekrar ölç ve değişimi kanıtla*. Bu sıra korunmazsa kodu boşuna karmaşıktırır, hatta bazen yavaşlatırsın.

49.4.1 Adım 1: Başlangıç Ölçümü

Uzay Kaçışı v3'ün ana dosyasını şu kalıpla sar:

Python – Profil Sarmalı

```

1 import cProfile, pstats
2
3 def run_game():

```



```

4      # ... oyunun olağan main fonksiyonu ...
5      main()
6
7      if __name__ == "__main__":
8          profiler = cProfile.Profile()
9          profiler.enable()
10         try:
11             run_game()
12         finally:
13             profiler.disable()
14             st = pstats.Stats(profiler).sort_stats('cumulative')
15             st.print_stats(15)

```

Şimdi oyunu *tipik* bir oturum gibi oyna: 30 saniye boyunca düşmanlara ateş et, parçacıkları tetikle, biraz hareket et. Sonra pencereyi kapat. Terminalde şuna benzer bir çıktı göreceksin (rakamlar makineye göre değişir):

Python – Optimizasyon Öncesi Profil

```

1  ncalls  tottime  cumtime  filename:lineno(function)
2    1800    3.120    7.450
   ↳ uzay_kacisi.py:140(main_loop_iteration)
3  180000    1.870    1.870  sprite.py:298(rect_property)
4    1800    1.450    2.310  sprite.py:215(groupcollide)
5  45000    0.940    1.540  uzay_kacisi.py:62(Mermi.__init__)
6    1800    0.730    0.730  surface.py:411(blit)
7     60    0.510    0.510  mixer.py:120(decode)
8    1800    0.380    0.380
   ↳ uzay_kacisi.py:88(yildiz_arka_plani)

```

Toplam 30 saniyelik oturumun ~ 7.45 s'sini sadece bir kare iterasyonunda harcamışız. Kare başına ~ 4.1 ms değil ama kare başına ~ 4 ms *tek başına* **groupcollide** ve ~ 0.85 ms **Mermi__init__**. 1800 kare için bu, oturumun yaklaşık %65'i demek. İşte darboğazlarımız.

Tablo 49.3: Optimizasyon öncesi: tipik FPS değerleri

Sahne	FPS
Sakin (5 düşman, 0 mermi)	60
Orta (15 düşman, 30 mermi)	48
Yoğun (30 düşman, 80 mermi)	35
Patlama (40 düşman, 200 parçacık)	22

Hedef 60'tır. Yoğun ve patlama sahneleri bütçeyi aşıyor. Şimdi neden olduğunu profil bize söylediğine göre çözüme geçiyoruz.

49.4.2 Adım 2: Mermi ve Parçacık için Pool

`Mermi.__init__` ve aynı sırada görünen `Surface.convert_alpha` bize Konu 49.3 konusunun çözümünü işaret eder. Mermi sınıfını pool'a uyumlu hâle getirelim ve oyun başında 128 kapasiteli bir mermi havuzu, 400 kapasiteli bir parçacık havuzu kuralım.

Python – Optimize Edilmiş Ateş Etme

```
1 # Eski:
2 # yeni_mermi = Mermi(oyuncu.rect.centerx, oyuncu.rect.top)
3 # mermi_grubu.add(yeni_mermi)
4
5 # Yeni:
6 mermi = mermi_pool.acquire()
7 if mermi is not None:
8     mermi.firlat(oyuncu.rect.centerx, oyuncu.rect.top)
```

`kill` çağrılarını `self.active = False` ile değiştir. Çarpışma sonrası eski yapıdaki:

Python – Çarpışma Sonrası: Eski

```
1 pygame.sprite.groupcollide(mermi_grubu, dusman_grubu,
    ↪ True, True)
```

şu hâle gelir:

Python – Çarpışma Sonrası: Yeni

```
1 carpan = pygame.sprite.groupcollide(mermi_grubu,
    ↪ dusman_grubu,
2                                     False, False)
3 for mermi, dusmanlar in carpan.items():
4     mermi.active = False
5     for d in dusmanlar:
6         patlama(d.rect.centerx, d.rect.centery)
7         d.active = False
```

Bu değişiklik tek başına `Mermi.__init__` ve `Surface.convert_alpha` çağrılarını profilin tamamen dışına atar.

49.4.3 Adım 3: Sprite `convert` ile Görselleri Hızlandır

Profilin alt sıralarında `Surface.blit`'in 0.73 s'sini gördün. `Surface`'lar ekrana çizilirken otomatik olarak piksel formatı dönüşümü yapılır — *eğer* format ekranınkiyle uyuşmazsa. `convert` ya da `convert_alpha` bu dönüşümü yükleme zamanında bir kez yapar.

Python – Görselleri Hızlı Yapmak

```

1 # YANLIS: blit her seferinde format çevirir
2 mermi_img = pygame.image.load("mermi.png")
3
4 # DOGRU: yüklemede bir kez dönüştür
5 mermi_img = pygame.image.load("mermi.png").convert_alpha()

```

Tüm sprite görselleri için bunu yap. Tek başına bu değişiklik `blit` sürelerini %30–%50 düşürebilir.

49.4.4 Adım 4: Yıldız Arka Planını Önceden Hesapla

Profilin altında `yildiz_arka_plani` fonksiyonu görünüyor. Bu fonksiyon her karede 200 yıldız tek tek `set_at` ile çiziyor olabilir. Ama yıldızlar hareket etmiyorsa — niye her kare yeniden çiziyoruz?

Python – Yıldız Arka Planını Önbelleğe Al

```

1 # Bir kez oluştur:
2 def yildiz_yuzeyi_olustur(genislik, yukseklik, sayi=200):
3     yuzey = pygame.Surface((genislik, yukseklik))
4     yuzey.fill((5, 5, 25))
5     for _ in range(sayi):
6         x = random.randint(0, genislik - 1)
7         y = random.randint(0, yukseklik - 1)
8         parlaklik = random.randint(120, 255)
9         yuzey.set_at((x, y), (parlaklik,)*3)
10    return yuzey.convert()
11
12 YILDIZ_YUZEYI = yildiz_yuzeyi_olustur(800, 600)
13
14 # Her karede sadece blit:
15 ekran.blit(YILDIZ_YUZEYI, (0, 0))

```

200 adet `set_at` çağrısı (yavaş) yerine 1 adet `blit` çağrısı (hızlı). Kazanç dramatiktir.

💡 İpucu: Bu Bir Genel Desendir

“Her karede aynı şeyi tekrar hesaplama; bir kez hesapla, `blit`’le.” Bu desen 2D oyunlarda her yerde işine yarar: HUD arka planı, seviye haritası, durağan dekorlar, gradyan arka planlar.

49.4.5 Adım 5: Çarpışmada Erken Çıkış

`groupcollide` her merminin her düşmanla `Rect` karşılaştırmasını yapar. Yoğun sahnelerde $80 \times 30 = 2400$ karşılaştırma demek. Şu an bunu *spatial hashing* ile optimize etmiyoruz — o, Ünite 2’nin konusu — ama şimdilik düşman sayısını bant’a göre düşürebiliriz.

Ekran dışındaki düşmanları çarpışma kontrolünden çıkar:

Python – Çarpışma için Filtreli Grup

```

1 def gorunur_dusmanlar(dusman_grubu, ekran_rect):
2     return [d for d in dusman_grubu
3             if d.active and
4                d.rect.colliderect(ekran_rect)]
5
6 # Çarpışmada bu daraltılmış listeyi kullan
7 hedefler = gorunur_dusmanlar(dusman_grubu,
8                               ekran.get_rect())
9 gecici_grup = pygame.sprite.Group(hedefler)
10 pygame.sprite.groupcollide(mermi_grubu, gecici_grup, ...)

```

Tile haritalı bir oyunda culling (Konu 49.2) zaten ekran dışındaki nesneleri güncellemediği için, çarpışma listesinin de doğal olarak küçüldüğünü unutma.

49.4.6 Adım 6: Yeniden Ölçme

Aynı 30 saniyelik oturumu, aynı oynayış stiliyle yeniden çek. Yeni profil çıktısı şuna benzeyecek:

Python – Optimizasyon Sonrası Profil

```

1 ncalls   tottime   cumtime   filename:lineno(function)
2 1800     1.420     3.610
3   ↳ uzay_kacisi.py:140(main_loop_iteration)
4 1800     0.620     0.910   sprite.py:215(groupcollide)
5 1800     0.310     0.310   surface.py:411(blit)
6 1800     0.140     0.140   uzay_kacisi.py:88(arka_plan_blit)
7 60       0.085     0.085   mixer.py:120(decode)

```

`Mermi.__init__` satırı kayboldu. `groupcollide` yarıdan fazla küçüldü. `yildiz_arka_plani` basitleşmiş bir `arka_plan_blit`'e dönüştü. Toplam kare iterasyonu ~ 7.45 s'den ~ 3.61 s'ye düştü. Şimdi FPS tablosunu yeniden çekelim:

Tablo 49.4: Optimizasyon öncesi-sonrası FPS karşılaştırması

Sahne	Önce	Sonra	Kazanç
Sakin	60	60	— (zaten cap)
Orta	48	60	+25%
Yoğun	35	60	+71%
Patlama	22	58	+164%

Tipik bir oyuncu için fark gece ile gündüz arasındır. Patlama sahnesi artık hedef bütçenin sadece 0.5 ms altında, kabul edilebilir.

49.4.7 Sonuç: Sıralı Bir Disiplin

Bu süreçte yaptıklarımızı geri gözden geçir:

1. Profil aldık, en pahalı satırları belirledik.
2. En büyük kazanç vereni seçtik — mermi pool'u.
3. Tek bir değişiklik yaptık, yeniden ölçtük.
4. Sonraki en pahalıya geçtik — yıldız önbellegi.
5. İşlem tekrar etti.

Bu disiplin Konu 1'de söz verdiğimiz %80–%20 kuralının gerçek hayattaki uygulamasıdır. Beş optimizasyondan üçü tek başına büyük kazanç verdi; ikisi marjinaldi. Önemli olan hangisinin hangi kategoride olduğunu *tahmin değil ölçümle* bilmektir.

● Bilgi: Pratik Alıştırma

Kendi Uzay Kaçışı kopyana yukarıdaki adımları sırayla uygula. Her adımdan sonra cProfile çıktısını ve FPS sayısını bir kenara kaydet. Sonunda altı satırlık bir tablo elde edeceksin: oyununun optimizasyon hikayesi.

49.5 Sonuç

Bu ünite de oyununun performansını “çalışıyor ama sarsılıyor” seviyesinden “akıcı 60 FPS” seviyesine çıkardık. Yolculuk şuydu: önce *ölçtük* (`time.perf_counter` ve `cProfile`), sonra *görmediğimizi atladık* (culling), sonra *nesneleri yeniden kullandık* (object pooling) ve sonunda Uzay Kaçışı'nın gerçek profilinde bütün bunları üst üste koyup ölçülebilir bir kazanca dönüştürdük. Önemli olan teknikler değil — önemli olan ölç-değiştir-tekrar ölç disiplini.

49.5.1 Anahtar Kavramlar

Tablo 49.5: Ünite 1 – Anahtar kavramlar

Kavram	Açıklama
Kare bütçesi	60 için 16.67 ms; 30 için 33.33 ms
<code>time.perf_counter</code> <code>cProfile</code>	Mikrosaniye hassasiyetli manuel ölçüm Tüm fonksiyon çağrılarını otomatik profilleyen modül
<code>totttime</code> / <code>cumtime</code>	Sırasıyla kendinde geçen / alt çağrılarla birlikte geçen süre
Hot path	Toplam zamanın büyük kısmının harcandığı kod yolu
%80–%20 kuralı	Kazancın çoğu birkaç satırda saklı; oraya odaklan
Viewport culling	Kamera dikdörtgeni dışındaki sprite'ları çizmemek/güncellememek
Aktif bant (pay)	Viewport'u <code>inflate</code> ile büyütüp ısınma alanı tanımlamak
Object pool	Nesneleri yok edip yeniden yaratmak yerine ödünç verip geri almak
<code>active</code> bayrağı	<code>kill</code> yerine sprite'ı pasifleştirip pool'a iade etmek
<code>convert_alpha</code>	Yükleme zamanında piksel formatını ekrana uydurmak
Önbelleğe alma	Karede değişmeyen yüzeyleri önceden hesaplayıp <code>blit</code> lemek

49.5.2 Önemli Komutlar

Tablo 49.6: Ünite 1 – Temel komutlar

Komut	Açıklama
<code>time.perf_counter()</code>	Yüksek hassasiyetli zaman damgası (saniye)
<code>cProfile.Profile()</code>	Profiler nesnesi yarat
<code>profiler.enable()</code> / <code>disable()</code>	Ölçümü başlat / durdur
<code>pstats.Stats(profiler)</code>	Sonuçları analiz için sar
<code>.sort_stats('cumulative')</code>	Toplam süreye göre sırala
<code>rect.colliderect(viewport)</code>	Viewport ile kesişim testi
<code>rect.inflate(dx, dy)</code>	Dikdörtgeni şişirip pay tanımla
<code>pygame.image.load(p).convert_alpha()</code>	Yükleme + format dönüşümü
<code>sprite.active = False</code>	<code>kill</code> yerine pool'a iade

49.5.3 Bir Sonraki Ünite: Gelişmiş Çarpışma

Profil çıktımızda `groupcollide` hâlâ üst sıralardaydı. Mermi sayısını ve düşman sayısını ne kadar düşürsek de, çarpışma algoritmasının kendisi $\mathcal{O}(n \cdot m)$ karmaşıklıkta: n mermiyle m düşmanı her karede tek tek karşılaştırıyoruz. 1000 mermi ve 1000 düşman olduğunda bu, kareyi tek başına yer.

Ünite 2’de bu darboğaza özel teknikler öğreneceğiz:

- *Geniş faz ve dar faz ayrımı*: önce kabaca yakın olanları bul, sonra kesin testi sadece o aday çiftler için yap.
- *Spatial hashing*: dünyayı sabit boyutlu hücrelere böl; her sprite hangi hücrelerdeyse sadece o hücrelerdekililerle karşılaştır.
- *Quad tree*: yoğunluğa göre uyarlanan bir uzamsal veri yapısı.

Bu ünite de ölçtük, gördüğümüzü atladık, yarattığımızı sakladık. Sıradakinde aramayı zekileştireceğiz.

💡 İpucu: Optimizasyon Bir Disiplin, Bir Hedef Değil

“Daha hızlı yapayım” düşüncesi seni karmaşık koda götürür. “60 FPS’in altına düştüğümde profil çekip darboğazı bulayım” düşüncesi seni başarıya götürür. Hedef hız değil — hedef bütçenin içinde kalmaktır. Bütçe karşılandığında bırak, başka bir özelliğe geç.

“Algoritmanı seç, ölçeği gelsin: 100 nesne için her şey çalışır, 10 000 nesne için sadece doğru veri yapısı çalışır.”

— ANONİM OYUN PROGRAMCISI

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Geniş faz (broad-phase) ve dar faz (narrow-phase) çarpışma ayrımı; AABB kavramı
- Spatial hashing: dünyayı sabit hücrelere bölüp `dict[(cx, cy)] -> list` ile sorgu hızlandırma
- Quad tree: yoğunluğa göre uyarlanan özyinelemeli dört yönlü alt bölme
- Üç yaklaşımı (brute force, spatial hash, quad tree) farklı sprite sayılarında kıyaslama
- Hangi senaryoda hangi algoritmanın seçileceğine dair pratik tavsiyeler

50.1 Geniş Faz ve Dar Faz

Bölüm 21’de iki dikdörtgeni `colliderect()` ile karşılaştırmayı, Bölüm 22’de ise bir sprite’ı koca bir grupta `pygame.sprite.spritecollide()` aracılığıyla test etmeyi öğrendin. Şimdi sahneye yüzlerce, hatta binlerce nesne koy. Her oyuncu mermisi her düşmanla, her parçacık her platformla, her düşman her diğer düşmanla karşılaştırılırsa kare bütçen anında erir. Bu konuda problemin neden *kuadratik* büyüdüğünü, ve “önce ele, sonra incele” fikriyle nasıl iki aşamalı bir çarpışma tasarımı kuracağımızı göreceğiz.

50.1.1 Saf $O(n^2)$ Yaklaşımının Problemi

Sahne de n nesne var ve her nesneyi her diğer nesneyle test etmek istiyoruz. Yapılan kontrol sayısı $\binom{n}{2} \approx n^2/2$ olur. Aşağıdaki tablo bu büyümenin neden hızla

kontrolden çıktığını gösteriyor.

Tablo 50.1: Saf çift karşılaştırmanın kuadratik büyümesi

Nesne sayısı n	Çift sayısı $\approx n^2/2$	60 FPS bütçesinde uygun mu?
50	1 250	Rahatlıkla
100	5 000	Sorunsuz
500	125 000	Sınırdadır
1 000	500 000	Genelde geç kalır
5 000	12 500 000	Saniyede 1–2 kare bile zor

60 FPS oyununda elinde sadece 16.67 ms var. Saf $O(n^2)$ yaklaşımı, 1000 nesne-den sonra bu bütçeyi tek başına yer. Ama dikkat edersen *çoğu çift zaten birbirine çok uzakta*: ekranın sol üstündeki bir mermi, sağ alttaki düşmanla hiçbir karede çarpışmayacak. Onu test etmek tamamen israf.

▲ Dikkat: Erken Optimizasyona Karşı

Bölüm 49’de gördüğümüz gibi optimizasyona *ölçtükten sonra* başlanır. Eğer oyununda 30 düşman varsa, brute force çarpışma kodu mükemmel iş görür ve okumayı kolaylaştırır. Bu ünite, profilde `groupcollide` satırını ekranın üstünde gördüğünde okuyacağın bölümdür.

50.1.2 İki Aşamalı Çarpışma Fikri

Profesyonel motorlar (Box2D, Bullet, Unity Physics) çarpışmayı her zaman *iki fazda* yapar. Fikir basit: kesin geometri testi pahalı, ama kabaca eleme ucuz. Önce ucuzu yap, sonra geriye kalan adaylara pahalı testi uygula.

■ Tanım: Geniş Faz / Dar Faz

Geniş faz (broad-phase), sahnedeki nesneler arasından “muhtemelen çarpışabilir” olan çiftleri hızlıca bulan eleme adımıdır. Çıktı: aday çiftler listesi. *Dar faz* (narrow-phase) ise bu adaylar üzerinde *kesin* geometri testini uygular: piksel-mükemmel, poligon-poligon veya hassas `collidirect`.

İki faz birlikte şu boru hattını oluşturur:

1. **Geniş faz** — n nesneden, gerçekten yakın olan k aday çift bulunur. ($k \ll n^2$, ideal durumda $k \sim n$.)
2. **Dar faz** — yalnızca bu k çift için kesin test uygulanır.

Toplam maliyet: $O(\text{geniş faz}) + O(k)$. Geniş fazı doğru kurarsan, brute force’taki n^2 patlamasını n ’e veya $n \log n$ ’e indirirsin.

50.1.3 AABB ve Neden pygame.Rect Yeter

Geniş fazın temel taşı, her nesneyi onu *çevreleyen* basit bir kutu ile temsil etmektir.

■ Tanım: AABB — Axis-Aligned Bounding Box

Eksenlere paralel sınır kutusu. Yani kutunun kenarları x ve y eksenlerine paralel olan, dönmeyen bir dikdörtgen. İki AABB'nin kesişim testi sadece dört karşılaştırma alır: `a.right > b.left` ve `a.left < b.right` ve `a.bottom > b.top` ve `a.top < b.bottom`.

Burada güzel haber: `pygame.Rect` zaten bir AABB'dir. `collidect` tam olarak yukarıdaki dört karşılaştırmayı yapar. Yani PyGame'de geniş faz için ekstra bir veri yapısı kurmana gerek yok — nesnelerin `rect` özelliği geniş faz için hazır.

Python – AABB Kesişim – pygame.Rect

```
1  # İki AABB'nin kesisme testi: 4 karsilastirma
2  def aabb_kesisir(a, b):
3      return (a.right > b.left and a.left < b.right
4              and a.bottom > b.top and a.top < b.bottom)
5
6  # pygame.Rect zaten bunu yapar:
7  if mermi.rect.collidect(dusman.rect):
8      ...
```



Şekil 50.1: Geniş faz AABB ile aday çiftleri seçer; dar faz seçilen çiftler için piksel veya poligon testini uygular.

50.1.4 Dar Faz: Ne Zaman ve Ne Kadar Hassas?

Geniş faz “çarpmıyor olabilir” dediğinde, dar fazda istediğin hassasiyette test yaparsın. Üç yaygın seviye:

1. **Sıkı AABB:** Çoğu arcade oyun için yeterli. `pygame.sprite.collide_rect` kullanır. Geniş ve dar faz aynı olduğunda zaten geniş fazın sonucunu kabul edersin.
2. **Daire / mesafe testi:** Yuvarlak nesneler için `pygame.sprite.collide_circle`. Tek bir karekök ve karşılaştırma.
3. **Pixel-perfect mask:** `pygame.sprite.collide_mask`. Şeffaf piksellere izin vermez. Pahalı: iki sprite’ın çakışan bölgesindeki tüm pikselleri tarar. Önce AABB ile elemek *şart*.
4. **Poligon-poligon (SAT):** Dönen nesneler için Separating Axis Theorem. Py-Game yerleşigi yok; harici kütüphane veya elle yazılır.

Dar fazın pahalı oluşu, geniş fazın elemesini değerli kılar. 1000 sprite arasından geniş faz 50 aday çift bulduysa, mask testini 500 000 kez değil 50 kez çalıştırırsın.

Python – İki Fazlı Çarpışma – İskelet

```

1 def carpisma_kontrol(adaylar, dar_faz_test):
2     """adaylar: [(a, b), ...] --- geniş fazdan gelen
   ↪ çiftler.
3     dar_faz_test: kesin testi yapan fonksiyon."""
4     sonuclar = []
5     for a, b in adaylar:
6         if dar_faz_test(a, b):
7             sonuclar.append((a, b))
8     return sonuclar

```

`adaylar` listesini nasıl üreteceğimiz, sonraki iki konunun *tek* sorusu: Konu 50.2’de hücre tabanlı hash, Konu 50.3’de quad tree.

50.1.5 Bu Konunun Çıkarımı

Çarpışmayı tek bir adımda düşünmeyi bırak. “Aday bul, sonra test et” zihniyetiyle bak. Geniş faz iyi kurulmuşsa, dar faz hangi olursa olsun sürdürülebilir kalır. Sıradaki iki konuda iki farklı geniş faz veri yapısı kuracağız.

💡 İpucu: Önce Profil, Sonra Veri Yapısı

Bir oyunun profilinde `groupcollide` %5’in altındaysa, hiçbir şey yapma. Brute force yeterlidir. Geniş faz veri yapıları, profil sana “çarpışma kareyi yiyor” dediği gün anlamlı olur.

📌 Bilgi: Sahnede Statik Çoğunluk Var Mı?

Çoğu platformer’da *platformlar* hareket etmez. Statik nesneler için geniş faz veri yapısını oyun başlarken *bir kez* kurarsın ve hiç güncellemezsin. Sadece dinamik nesneler (oyuncu, düşman, mermi) için yapı her karede yeniden inşa edilir. Bu, gerçek dünyada en büyük kazançlardan biridir.

50.2 Spatial Hashing — Hücre Tabanlı Geniş Faz

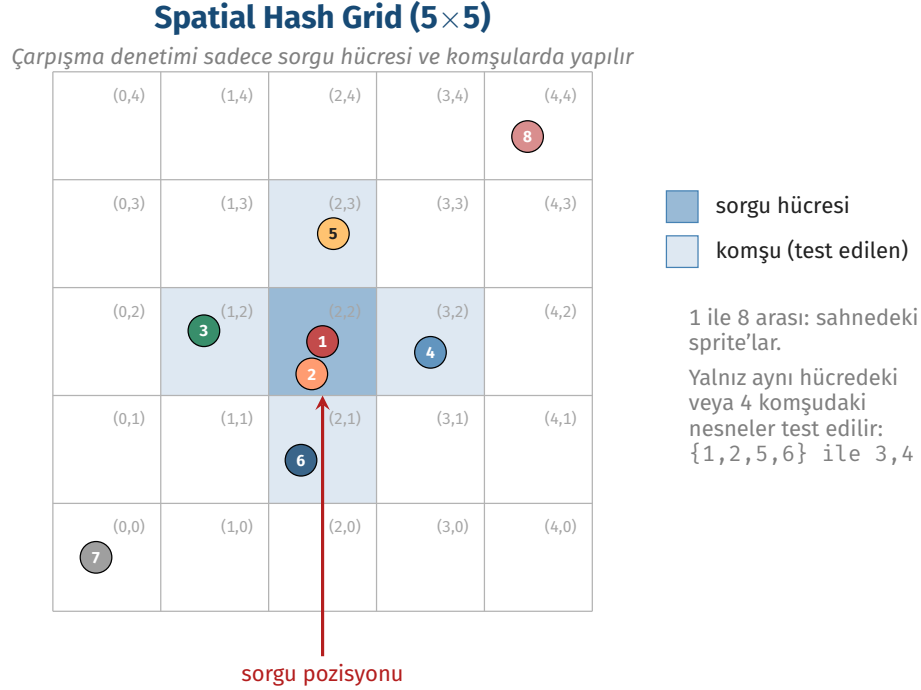
Konu 50.1’de “aday çiftleri ucuza bul” fikrini koyduk ortaya. Şimdi bu fikri ilk somut veri yapısıyla, *spatial hash* ile gerçekleştireceğiz. Spatial hash’in tek satırlık özeti şudur: dünyayı sabit boyutlu bir ızgaraya böl, her sprite’ı içinde bulunduğu hücrelerin listesine yaz; sonra çarpışma sorularını sadece o hücrelere sor.

50.2.1 Temel Fikir: Dünyayı Hücrelere Böl

Bir kâğıdı düşün. Üzerine 64×64 piksel kareli bir ızgara çizdin. Her sprite, üzerinde durduğu kare(ler)in adıyla etiketlenmiştir. Şimdi sana “oyuncu hangi düşmanlarla çarpışabilir?” diye sorduğumda, oyuncunun bulunduğu hücreye ve etrafındaki sekiz komşu hücreye bakıp, sadece oradaki düşmanları test ederim.

■ Tanım: Spatial Hash

Dünyayı sabit boyutlu (örn. 64×64 piksel) hücelere bölen, her hücre koordinatından oradaki sprite listesine eşleyen veri yapısı. Eşleme genelde Python sözlüğüyle yapılır: `dict[(cx, cy)] -> list[Sprite]`.



Şekil 50.2: Sahne 64×64 hücelere bölünmüş. Oyuncunun (sarı) test edeceği hücreler kendi hücresi ve sekiz komşusudur; uzaktaki düşmanlara hiç bakılmaz.

50.2.2 Hücre İndekslemesi

Bir sprite'ın `rect`'ini hangi hücre(ler)e koyacağını bulmak, basit bir tam sayı bölmesidir:

Python – Hücre İndeksi

```

1 HUCRE = 64 # piksel
2
3 def hucreler(rect):
4     """rect'in girdigi tum hucre koordinatlarini
5     ↪ uretir."""
6     cx0 = rect.left // HUCRE
7     cx1 = rect.right // HUCRE
8     cy0 = rect.top // HUCRE
9     cy1 = rect.bottom // HUCRE
10    for cx in range(cx0, cx1 + 1):
11        for cy in range(cy0, cy1 + 1):
12            yield (cx, cy)

```

Burada kritik nokta: bir sprite tek bir hücreye değil, *birden fazla hücreye* girebilir. Geniş bir patron sprite'ı dört hücreyi birden kapsayabilir; minik bir mermi bir hücredeyken bir sonraki karede iki hücreyi de aşabilir. Geniş sprite'lar için her girdiği hücreye eklenir.

⚠ Dikkat: Negatif Koordinatlar

// operatörü Python'da negatif sayılar için "aşağıya yuvarlar" ($-1 // 64 = -1$, $0 // 64 = 0$). Bu sezgisel ve doğrudur. Ama bazı dillerde (C, JavaScript) sıfıra doğru yuvarlar; Python kodunu oraya taşıırken dikkat et.

50.2.3 Yapıyı Kuran Sınıf

Tüm fikri tek bir sınıfta toplayalım:

Python – SpatialHash sınıfı

```

1  from collections import defaultdict
2
3  class SpatialHash:
4      def __init__(self, hucre_boyutu=64):
5          self.h = hucre_boyutu
6          self.kova = defaultdict(list)
7
8      def temizle(self):
9          self.kova.clear()
10
11     def ekle(self, sprite):
12         r = sprite.rect
13         cx0, cx1 = r.left // self.h, r.right // self.h
14         cy0, cy1 = r.top // self.h, r.bottom // self.h
15         for cx in range(cx0, cx1 + 1):
16             for cy in range(cy0, cy1 + 1):
17                 self.kova[(cx, cy)].append(sprite)
18
19     def yakindakiler(self, rect):
20         """rect'i kapsayan hucre + komsulardaki
21         ↪ sprite'leri verir."""
22         sonuc = set()
23         cx0, cx1 = rect.left // self.h, rect.right //
24         ↪ self.h
25         cy0, cy1 = rect.top // self.h, rect.bottom //
26         ↪ self.h
27         for cx in range(cx0, cx1 + 1):
28             for cy in range(cy0, cy1 + 1):
29                 sonuc.update(self.kova.get((cx, cy), ()))
30         return sonuc

```

Üç temel işlem var: **temizle** (her karenin başında), **ekle** (her sprite için bir kez), **yakindakiler** (her sorgu için). **set** kullanmamızın sebebi, çok-hücreli sprite'ın aynı sonuçta iki kez gözükmesini engellemek.

50.2.4 Her Karede Yeniden İnşa

Dinamik nesnelerle çalışırken en pratik yol, hash'i *her karede* sıfırlayıp yeniden doldurmaktır. Sprite'lar hareket ettikçe hangi hücreye girdikleri değişir; takip etmeye çalışmak kodu kirletir.

Python – Karede Yeniden İnşa

```

1 def kare_basinda(hash_, dinamikler):
2     hash_.temizle()
3     for s in dinamikler:
4         hash_.ekle(s)
5
6 # Sorgu:
7 def carpisanlari_bul(hash_, oyuncu):
8     adaylar = hash_.yakindakiler(oyuncu.rect)
9     adaylar.discard(oyuncu) # kendisini cikar
10    return [a for a in adaylar if
        ↳ oyuncu.rect.colliderect(a.rect)]

```

İlk satır geniş faz (aday topla), son satır dar faz (kesin AABB testi).

İpucu: Statik Engelleri Bir Kez Doldur

Sahnedeki platformlar/duvarlar hareket etmiyorsa, ayrı bir `statik_hash` kur ve oyun başında doldurup hiç temizleme. Sadece dinamik nesneler için `dinamik_hash`'i her karede sıfırla. İki yapıya da sırayla sorgu yaparsın. Pratik kazanç dramatik olabilir.

50.2.5 Hücre Boyutu Seçimi

Hücre boyutu, hash'in en önemli ayarıdır. Üç senaryo var:

- **Çok küçük hücre** (örn. 8 px): Sprite'lar onlarca hücreye yazılır. Ekleme pahalı, sorgu da pahalı (çok hücreyi tarar). Bellek de şişer.
- **Çok büyük hücre** (örn. 1024 px): Hemen hemen tüm sahne tek hücreye düşer. Yapı brute force'a dejenere olur.
- **İdeal** (ortalama sprite boyutu civarı): Her sprite bir veya iki hücrede yer alır. Sorgu küçük komşulukla yetinir.

Pratik kural: *en yaygın çarpışan nesnenin (örn. düşmanın) çapı kadar* bir hücre seç. 32 × 32 sprite oyununda 64 piksel iyi bir başlangıçtır.

Tablo 50.2: Spatial hash karmaşıklığı (n: nesne sayısı, k: bir hücredeki ortalama nesne)

İşlem	Ortalama	En kötü
<code>ekle</code> (n nesne)	$O(n)$	$O(n \cdot \text{kapsanan hücre})$
<code>yakindakiler</code> (rect)	$O(k)$	$O(n)$ (her şey aynı hücrede)
Tüm çiftler taraması	$O(n \cdot k)$	$O(n^2)$

Homojen dağılımda k küçük bir sabit kalır; toplam çift taraması $O(n)$ gibi davranır. Bu, brute force'un $O(n^2)$ 'sinden niteliksel olarak farklı bir sınıftır.

50.2.6 Spatial Hash'in Sınırları

Spatial hash basit ve hızlıdır, ama her senaryo için ideal değildir:

1. **Dengesiz dağılım:** Bütün düşmanlar aynı odadaysa, o hücre patlar; sorgu pratikte $O(n)$ 'ye düşer.
2. **Çok büyük dünyalar:** Hücre boyutu sabit olduğundan, boş alan da kapsanır. Hash boş hücreleri tutmaz, ama *aralık sorgusu* büyük dünyada hâlâ pahalı olabilir.
3. **Çok farklı sprite boyutları:** Bir patron 512 px, mermiler 4 px ise, tek hücre boyutu ikisini birden iyi temsil edemez. Çözüm: birden fazla seviyeli hash veya quad tree.

Bu sınırları aşmak için bir sonraki konuda *quad tree* göreceğiz: yoğunluğa göre kendini uyarlayan ağaç yapısı.

● Bilgi: Pratik Uygulama

`02_spatial_hash_demo.py` dosyasında 1000 sprite'ın brute force ve spatial hash ile çarpışma testi süreleri yan yana ölçülür. Hücre boyutunu değiştirip (16, 32, 64, 128, 256) hangi değer en iyi sonucu verdiğini gözlemle. Çoğu zaman ortalama sprite boyutunun iki katı kazanan olur.

50.3 Quad Tree — Yoğunluğa Uyarlanan Geniş Faz

Konu 50.2'in sonunda dengesiz dağılım sorununu gördük: tüm düşmanlar tek köşedeysen, oradaki hücre patlar. *Quad tree* bu probleme verilen klasik cevaptır. Sabit ızgara yerine, *yoğun yerlerde derin, seyrek yerlerde sık* olan bir ağaç kurar. Patron odası 128 piksellik kareye bölünür, boş gökyüzü tek bir kare olarak kalır.

50.3.1 Özyinelemeli Dört Yönlü Alt Bölme

■ Tanım: Quad Tree

Bir dikdörtgen alanı temsil eden ağaç yapısı. Her düğümün ya 0 alt düğümü vardır (yaprak) ya da *tam* 4 alt düğümü: kuzey-batı (NW), kuzey-doğu (NE), güney-batı (SW), güney-doğu (SE). Bir yaprak çok dolduğunda *alt bölme* (subdivide) yaparak dört çocuk oluşturur ve nesnelerini onlara dağıtır.

images/bolum13/unite2/quad_tree_subdivide.pdf

Şekil 50.3: Quad tree alt bölme: yoğun bölgeler özyinelemeli olarak dörde ayrılır, seyrek bölgeler tek bir yaprak kalır.

İki ana parametre vardır:

- `max_objects` — Bir yaprakta tutulabilen en fazla nesne sayısı. Bu sayı aşılmıca yaprak alt bölünür. Tipik değer: 4–10.
- `max_depth` — En çok kaç seviye derine inilebileceği. Sonsuz alt bölmeyi engeller (üst üste gelen nesnelerde gerekli). Tipik değer: 5–8.

50.3.2 Düğüm Sınıfı

Python – QuadTree dugumu

```

1 class QuadNode:
2     def __init__(self, sinir, derinlik=0,
3                 max_objects=8, max_depth=6):
4         self.sinir = sinir          # pygame.Rect
5         self.derinlik = derinlik
6         self.max_objects = max_objects
7         self.max_depth = max_depth
8         self.nesneler = []          # bu yapragin
          ↪ nesneleri

```

```

9         self.cocuklar = None           # None ise yaprak;
        ↪     değilse 4 dugum
10
11     def yaprak_mi(self):
12         return self.cocuklar is None

```

sinir bu düğümün kapsadığı alan. **nesneler** sadece yaprak düğümlerde dolu olur. Alt bölme olduğunda nesneler çocuklara aktarılır.

50.3.3 Alt Bölme: Subdivide

Bir yaprak **max_objects**'i aştığında dörde ayrılır:

Python – Subdivide

```

1  def _alt_bol(self):
2      x, y, w, h = self.sinir
3      yw, yh = w // 2, h // 2
4      d = self.derinlik + 1
5      kw = dict(max_objects=self.max_objects,
6                max_depth=self.max_depth)
7      self.cocuklar = [
8          QuadNode(pygame.Rect(x,          y,          yw, yh),
9                  ↪     d, **kw), # NW
10         QuadNode(pygame.Rect(x + yw,     y,          yw, yh),
11                 ↪     d, **kw), # NE
12         QuadNode(pygame.Rect(x,          y + yh,     yw, yh),
13                 ↪     d, **kw), # SW
14         QuadNode(pygame.Rect(x + yw,     y + yh,     yw, yh),
15                 ↪     d, **kw), # SE
16     ]
17     # Mevcut nesneleri çocuklara aktar
18     eskiler, self.nesneler = self.nesneler, []
19     for n in eskiler:
20         self._cocuklara_ekle(n)
21
22     def _cocuklara_ekle(self, nesne):
23         for c in self.cocuklar:
24             if c.sinir.collidirect(nesne.rect):
25                 c.insert(nesne)

```

Dikkat: bir nesne, sınırını birden fazla çocukla paylaşıyorsa *her çocuğa ayrı ayrı eklenir*. Bu, sorguda aday çiftlerin doğru toplanmasını sağlar.

50.3.4 Insert ve Query

Python – Insert

```

1 def insert(self, nesne):
2     if not self.yaprak_mi():
3         self._cocuklara_ekle(nesne)
4         return
5
6     self.nesneler.append(nesne)
7
8     # Yaprak kapasiteyi astıysa ve daha derine
9     ↪ inilebiliyorsa, alt bol
10    if (len(self.nesneler) > self.max_objects
11        and self.derinlik < self.max_depth):
12        self._alt_bol()

```

Python – Query

```

1 def query(self, alan, sonuc=None):
2     """alan ile kesisen nesneleri sonuc set'ine ekler."""
3     if sonuc is None:
4         sonuc = set()
5     if not self.sinir.collidirect(alan):
6         return sonuc
7
8     if self.yaprak_mi():
9         for n in self.nesneler:
10            if n.rect.collidirect(alan):
11                sonuc.add(n)
12    else:
13        for c in self.cocuklar:
14            c.query(alan, sonuc)
15    return sonuc

```

`query` özyinelemeli inerken, sınırı sorgu alanıyla kesişmeyen alt ağaçları tamamen atlar. Bu, ağacın dengesiz oluşunu *avantaja* çevirir: boş bölgeler tek hamlede elenir.

50.3.5 Her Karede Yeniden İnşa

Spatial hash gibi, dinamik dünyada en pratik yol her karede ağacı sıfırlayıp yeniden kurmaktır:

Python – Karede Yeniden İnşa

```

1 def kare_basinda(dunya_sinir, dinamikler):
2     agac = QuadNode(dunya_sinir)
3     for s in dinamikler:
4         agac.insert(s)
5     return agac

```

```

6
7 # Sorgu:
8 adaylar = agac.query(oyuncu.rect)
9 adaylar.discard(oyuncu)
10 carpisanlar = [a for a in adaylar
11                if oyuncu.rect.colliderect(a.rect)]

```

Statik nesneler için (platformlar, duvarlar) ağaç oyun başlarken *bir kez* kurulur ve hiç güncellenmez — spatial hash'teki mantığın aynısı.

⚠ Dikkat: Üst Üste Yığın

Eğer 20 nesne tam aynı koordinatta ise (örn. tek bir noktada üst üste mermiler), `max_depth`'e ulaşılan kadar alt bölme devam eder ve son yaprakta 20 nesnenin hepsi kalır. `max_depth` sınırı bu durumda devreye girer; aksi halde sonsuz alt bölme oluşur.

50.3.6 Görselleştirme: Hangi Bölge Ne Kadar Bölünmüş?

Quad tree'yi anlamamanın en güzel yolu çizdirmektir. Her düğümün sınırını farklı bir derinlik renginde çizersen, sahnede yoğunluğun nerede olduğunu gözlerinle görürsün:

Python – Ağacı çiz

```

1 def cizdir(dugum, ekran):
2     renk = (50, 50 + 30 * dugum.derinlik, 200)
3     pygame.draw.rect(ekran, renk, dugum.sinir, 1)
4     if not dugum.yaprak_mi():
5         for c in dugum.cocuklar:
6             cizdir(c, ekran)

```

Patron sahnesinde patron etrafının çok fazla alt bölündüğünü, boş gökyüzünün ise tek büyük dikdörtgen kaldığını göreceksin. Bu, quad tree'nin spatial hash'e karşı asıl üstünlüğünün resmidir.

50.3.7 Quad Tree Ne Zaman Spatial Hash'ten İyi?

Üç tipik senaryo:

1. **Patron + sürü:** Patron etrafında 200 minion, harita geri kalanı boş. Spatial hash'te boss hücresi 200'lük listeye döner; quad tree o bölgeyi 5–6 seviye böler ve sorguyu küçük yapraklara indirir.
2. **Çok farklı boyutlar:** 4 px mermi ve 512 px patron aynı sahnede. Quad tree büyük nesneyi ata düğümlerde, küçükleri yapraklarda tutar. Spatial hash'te tek bir hücre boyutu ikisi için de iyi olamaz.
3. **Açık dünya:** Kameranın görmediği uzak bölgeler. Quad tree o yaprakları hiç ziyaret etmez; spatial hash boş hücreleri zaten tutmuyordur ama aralık taraması yine de pahalıdır.

İpucu: İlk Tercih Hâlâ Spatial Hash

Quad tree daha güçlüdür ama daha karmaşıktır ve özyinelemeli kurulum maliyeti spatial hash'in bir hash table doldurmasından yüksektir. Sahnenin homojense ve sprite boyutları yakınsa, spatial hash genelde *daha hızlı çalışır*. Quad tree, dengesiz veya çok ölçekli dünyalarda kazanır.

Bilgi: Pratik Uygulama

`03_quad_tree_demo.py` dosyasında bir patron etrafında 300 düşman ve uzak köşelerde 20 düşman olan bir sahne yer alıyor. Brute force, spatial hash ve quad tree süreleri yan yana ölçülür. Sahnedeki quad tree sınırlarını da renklendirerek çiz; alt bölme yoğunluğunun sahne yoğunluğunu izlediğini gözlemle.

50.4 Karşılaştırma — Hangisi, Ne Zaman?

Önceki üç konuda sırayla brute force, spatial hash ve quad tree'yi gördük. Şimdi üçünü tek bir tezgâhta yan yana koyup, gerçek sayılarla “hangisi ne zaman kazanır?” sorusunu cevaplayacağız. Ölçüm yapmadan seçim yapma — bu konunun kalbi de bu cümle.

50.4.1 Karmaşıklık Tablosu

Önce kuramsal beklentileri özetleyelim. Bölüm 49'de gördüğümüz gibi karmaşıklık *eğilimdir*, sabit faktörler oyun boyutuna göre belirleyici olabilir.

Tablo 50.3: Üç yaklaşımın karmaşıklığı (n: nesne sayısı)

Yaklaşım	Kurulum	Sorgu (tek)	Tüm çiftler
Brute force	—	$O(n)$	$O(n^2)$
Spatial hash (ortalama)	$O(n)$	$O(1) - O(k)$	$O(n)$
Spatial hash (en kötü)	$O(n)$	$O(n)$	$O(n^2)$
Quad tree (ortalama)	$O(n \log n)$	$O(\log n)$	$O(n \log n)$
Quad tree (en kötü)	$O(n^2)$	$O(n)$	$O(n^2)$

Eğer dağılım homojen ve hücre boyutu doğruysa, spatial hash en iyi ortalama davranışı verir. Quad tree ortalama olarak biraz daha pahalı ama dengesiz dağılımlarda çok daha iyidir.

50.4.2 Test Senaryosu

Adil kıyas için tüm yaklaşımlar aynı sahneyi alır: $w = 1600$, $h = 900$ piksellik bir dünyada, sprite'ların `rect` boyutu 32×32 piksel, hızları rastgele.

Python – Test iskeleti

```

1 import time
2 import pygame
3 import random
4
5 GENISLIK, YUKSEKLIK = 1600, 900
6
7 def sprite_uret(n):
8     return [pygame.Rect(
9         random.randint(0, GENISLIK - 32),
10        random.randint(0, YUKSEKLIK - 32),
11        32, 32
12    ) for _ in range(n)]
13
14 def brute(rects):
15     ciftler = 0
16     for i, a in enumerate(rects):
17         for b in rects[i + 1:]:
18             if a.colliderect(b):
19                 ciftler += 1
20     return ciftler

```

Üç yaklaşım için `brute` fonksiyonunun karşılığını yazıp çalışma sürelerini ölçeriz:

Python – Ölçüm

```

1 def olc(fn, rects, tekrar=5):
2     sureler = []
3     for _ in range(tekrar):
4         to = time.perf_counter()
5         fn(rects)
6         sureler.append(time.perf_counter() - to)
7     return min(sureler) * 1000 # ms

```

Tek karenin değil, kararlı en iyi süreyi alıyoruz: çöp toplayıcı veya işletim sistemi gürültüsünü minimize eder.

50.4.3 Beklenen Sonuçlar (Tipik Değerler)

Aşağıdaki tablo, yukarıdaki test ortamında bir orta seviye masaüstü makinada (ortalama Python 3.11 + PyGame) elde edilen tipik büyüklükleri gösterir. Senin makinanda mutlak değerler farklı çıkacaktır; ama *oranlar* tutarlıdır.

Tablo 50.4: Tüm-çiftler çarpışma testi süresi (homojen dağılım, ms)

n	Brute force	Spatial hash	Quad tree
100	0.6	0.4	0.7
500	14	1.8	3.5
2000	230	7.2	18
5000	1450	19	52

Birkaç gözlem:

- 100 sprite'a kadar üç yaklaşım da kabul edilebilir. Brute force'un sade kodu, küçük sahnelerde hâlâ en iyi seçim.
- 500'de brute force 14 ms ile kare bütçesinin önemli kısmını yer; spatial hash $\sim 8\times$ daha hızlıdır.
- 2000'de fark uçurumdur: brute force 230 ms (yani saniyede sadece ~ 4 kare!), spatial hash hâlâ kare bütçesinin içinde.
- 5000'de bile spatial hash 19 ms ile kare bütçesinin biraz dışına çıkar; quad tree 52 ms ile bu sahnede kazançlı değil çünkü dağılım homojendir.

⚠ Dikkat: Tablodaki Değerler Sabit Değil

Aynı kod farklı CPU'larda, farklı PyGame sürümlerinde, farklı sprite boyutlarında farklı sonuç verir. *Mutlak* ms değerini kopyalama; görelî oranlara bak.

50.4.4 Dengesiz Dağılım Testi

Aynı testi farklı bir dağılımla tekrar yapalım: sprite'ların %80'i sahnenin küçük bir köşesinde (patron odası), %20'si dağınık.

Tablo 50.5: Tüm-çiftler çarpışma testi süresi (dengesiz dağılım, ms)

n	Brute force	Spatial hash	Quad tree
500	14	9.5	4.1
2000	230	140	22
5000	1450	920	68

Burada quad tree tahta liderini gösterir: spatial hash'in tek hücresi 4000 sprite ile patladığı için sorgu $O(n)$ 'ye düşer. Quad tree o yoğun bölgeyi alt bölme ile ufalar ve sorgu mantıklı kalır.

50.4.5 Hangisini Ne Zaman Seç?

Tüm bunları tek bir karar tablosuna sığdırıyoruz:

Tablo 50.6: Geniş faz seçim rehberi

Senaryo	Önerilen	Neden
<50 nesne, basit oyun	Brute force	En sade kod, profil zaten gösteriyor ki sorun yok
Çok sayıda küçük nesne, homojen dağılım (mermi, parçacık)	Spatial hash	Sabit hücre boyutu ortalama sprite ile uyumlu, kurulum çok ucuz
Patron + sürü, çok farklı boyut	Quad tree	Yoğun bölgeyi otomatik bölme, boş bölgeleri atlama
Tilemap tabanlı platformer	Spatial hash (statik)	Karoların kendisi zaten ızgara; hash bedava
Açık dünya, dağınık nesneler	Quad tree (statik)	Boş bölgeler tek yaprak kalır, sorgu o bölgeyi atlar

İpucu: Karma Yaklaşım

Profesyonel motorlar genelde *ikisini birden* kullanır: statik dünya için quad tree (oyun başında bir kez kurulur), dinamik nesneler için spatial hash (her karede yeniden kurulur). Sorgu önce statik ağaca, sonra dinamik hash'e gider. İki yapı da kendi güçlü yanını kullanır.

50.4.6 Pratik Tavsiye — Profil Et, Sonra Seç

Bu ünitenin en önemli cümlesi şudur: *önce ölç, sonra seç*.

1. Bölüm 49'deki `cProfile` yöntemiyle çarpışma kodunun gerçekten kareyi yiyip yemediğini doğrula. Eğer çarpışma toplam karenin %5'inden azını alıyorsa, hiç dokunma.
2. Çarpışma gerçekten darboğazsa, sahnenin *dağılımına* bak: homojen mi, dengersiz mi?
3. Homojense önce spatial hash dene. Hücre boyutunu ortalama sprite boyutunun 1.5 – 2 katı seç.
4. Dengesiz dağılımda quad tree dene. `max_objects` ve `max_depth`'i 4 – 8 aralığında ayarla.
5. Her değişiklikten sonra aynı profil sahnesini tekrar çalıştır. *Ölçülebilir kazanç* yoksa eski koda geri dön.

Bilgi: Karşılaştırma Demosu

`04_carpisma_benchmark.py` üç yaklaşımı yan yana ölçer; çıktısı yukarıdaki tablodaki sütunları doldurur. Hücre boyutunu, `max_depth` ve `max_objects` parametrelerini değiştirip senin makinanda en iyi sonucun hangisi olduğunu bul. Unutma: “en iyi” her zaman “en karmaşık” değildir.

50.5 Sonuç

Bu ünite, Ünite 1’in sonunda profile gördüğümüz `groupcollide` darboğazını gerçekten yok ettik. Önce çarpışmayı iki faza ayırdık: *geniş faz* ile aday bulup *dar faz* ile kesin testi yaparak n^2 patlamasını n ’in lineer ya da $n \log n$ büyümesine indirdik. Ardından iki farklı geniş faz veri yapısı kurduk: sabit ızgaraya dayalı *spatial hash* ve yoğunluğa uyarlanan *quad tree*. Sonunda üçünü ölçtük ve hangisinin ne zaman kazandığını net bir karar tablosuna oturttuk.

50.5.1 Anahtar Kavramlar

Tablo 50.7: Ünite 2 – Anahtar kavramlar

Kavram	Açıklama
Geniş faz (broad-phase)	“Muhtemelen çarpışıyor” aday çiftleri ucuza bulan adım
Dar faz (narrow-phase)	Aday çiftler için kesin geometri testi
AABB	Eksenlere paralel sınır kutusu; <code>pygame.Rect</code> zaten budur
Aday çift	Geniş fazın ürettiği, dar faza giden çift listesi
Spatial hash	<code>dict[(cx, cy)] -> list</code> ile hücre tabanlı eşleme
Hücre boyutu	Ortalama sprite boyutu civarı; çok küçük/büyük ikisi de zarar
Quad tree	Özyinelemeli dört yönlü alt bölme ağacı
<code>max_objects</code>	Yaprak alt bölme eşiği (tipik 4–10)
<code>max_depth</code>	En çok inilebilen seviye (tipik 5–8)
Subdivide	Yaprağı dört çocuğa ayırıp nesneleri dağıtma işlemi
Statik vs dinamik yapı	Hareketsiz nesneler bir kez, hareketliler her karede

50.5.2 Önemli Komutlar ve Şablonlar

Tablo 50.8: Ünite 2 – Temel kod şablonları

Şablon	Açıklama
<code>a.collidirect(b)</code>	İki AABB için kesişim testi (geniş faz)
<code>collide_mask(a, b)</code>	Pixel-mükemmel dar faz testi
<code>rect.left // hücre</code>	Hücre koordinatı hesabı
<code>defaultdict(list)</code>	Spatial hash için boş kova ile başlangıç
<code>kova[(cx, cy)].append(s)</code>	Sprite'ı hücre listesine ekle
<code>hash.yakindakiler(rect)</code>	Geniş faz aday seti
<code>QuadNode(sinir)</code>	Quad tree kök düğümü
<code>node.insert(s) / node.query(rect)</code>	Quad tree ekleme ve sorgu
<code>cProfile</code>	Hangi yaklaşımın gerçekten kazandığını ölçer

50.5.3 Bir Sonraki Adım

Çarpışmayı ölçeklendirmeyi öğrendik. Sıradaki ünite, motor düzeyinde performans ve görsel kalite arasındaki dengeyi ele alacağız: hareket bulanıklığı, parçacık sistemleri ve sahne mimarisi. Çarpışma artık darboğaz değil; sırada *görsel* darboğazlar var.

💡 İpucu: Doğru Sıra

Hatırla: ölç → darboğazı bul → uygun veri yapısını seç → tekrar ölç. Brute force'tan spatial hash'e geçmek bile küçük sahnede yavaşlatabilir; ölçmeden geçme. Bu ünitenin en kalıcı dersi, üç algoritma değil, bu döngüdür.

● Bilgi: Genel Çıkarım

Geniş faz iyi tasarlandığında dar fazda istediğin kadar pahalı bir test yapabilirsin — çünkü 1000 kez değil, 20 kez çalışacaktır. Tüm motor mimarileri aynı fikir üzerine kuruludur: *önce eler, sonra kesin teste götürür*. Bu, sadece çarpışma için değil, render sıralaması, AI hedef seçimi ve ses uzaysallaştırması için de geçerli bir desendir.

Temel AI Teknikleri

“İyi yapay zeka oyuncuya akıllı görünür; harika yapay zeka oyuncunun kendini akıllı hissetmesini sağlar.”

— SID MEIER

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Sonlu durum makinesi (FSM) deseniyle düşman davranışlarını **Idle**, **Patrol**, **Chase**, **Attack**, **Flee** gibi ayrı durumlara ayırmak
- Tile tabanlı haritada A* yol bulma algoritması: grid graph, heuristic fonksiyonları, açık/kapalı liste ve $f = g + h$ kuralı
- Steering behaviors yaklaşımıyla *seek*, *flee*, *arrival* ve *wander* davranışlarını vektör tabanlı yazmak
- Bu üç tekniği birleştirerek Zıplayan Macera oyununa görüş hatlı, A* ile takip eden ve yumuşak hareket eden gelişmiş bir düşman eklemek
- Bölüm 11’in basit AI’sından (patrol, chase, ranged) Bölüm 12’nin oyun-state makinesinden farklı, *karakter içi* state makinesine geçmek

51.1 Sonlu Durum Makinesi (FSM) Temeli

Bölüm 43’nde **PatrolDusman**, **ChaseDusman** ve **RangedDusman** sınıflarını yazarken “durum” kavramını sezgisel olarak kullandın: düşman bazen geziniyordu, bazen oyuncuyu kovalıyordu. Şimdi bu sezgiyi formal bir desene dönüştürelim: *sonlu durum makinesi* (Finite State Machine, FSM). Bu desen sayesinde düşmanın davranışını okunabilir, genişletilebilir ve hatasız tutmak çok daha kolay olacak.

■ Tanım: Sonlu Durum Makinesi (FSM)

Bir nesnenin belirli sayıda *durumdan* (state) yalnızca birinde bulunabildiği ve önceden tanımlı *geçiş* (transition) koşullarıyla bir durumdan diğerine atladığı bir hesaplama modelidir. Durumlar düğüm, geçişler kenar olarak çizilen bir yönlü grafa karşılık gelir.

Bir FSM üç şeyle tanımlanır: durumlar kümesi S , geçiş koşulları ve başlangıç durumu. Düşmanlarımız için tipik bir küme şudur:

$$S = \{\text{Idle, Patrol, Chase, Attack, Flee}\}$$

51.1.1 Bölüm 12'deki State Machine'den Farkı

Burada bir kavram karışıklığını önceden çözmek lazım. Bölüm 45'de `MenuState`, `PlayState`, `PauseState` sınıflarıyla *tüm oyunun* ekranlarını yöneten bir state makinesi yazdın. O makine *oyun seviyesinde* çalışıyordu: bir anda yalnızca bir sahne gösteriliyor, ESC menüye geri dönüyordu.

Bu üitedeki FSM ise *karakter seviyesinde* çalışır. Sahne hâlâ `PlayState`; ama o sahnedeki her düşmanın *kendi içinde* küçük bir durum makinesi var. Birinci düşman “patrol” durumunda gezerken, ikinci düşman “chase” durumunda oyuncuyu kovalıyor olabilir.

▲ Dikkat: İki Farklı Düzey

Oyun-state makinesi (sahne) ile karakter-state makinesi (AI) aynı desendir ama *farklı kapsamda* çalışırlar. `PlayState` çalışırken içindeki onlarca düşmanın her biri kendi FSM'sini koşturuyor olabilir.

51.1.2 Geçiş Koşulları

Bir FSM'yi etkileyen kararlar koşullara dayanır. Düşman AI için tipik koşullar şunlardır:

- **Mesafe:** Oyuncu 200 pikselden yakındaysa Patrol → Chase
- **Görüş hattı (line of sight):** Aramızda solid tile yoksa fark et
- **Can düzeyi:** Can ≤ 1 ise Chase → Flee
- **Zaman:** Saldırı yaptıktan sonra 30 kare *cooldown*
- **Saldırı menzili:** Oyuncuya 40 piksel uzaklıkta ise Chase → Attack

Geçiş diyagramını metinsel olarak yazalım:

```
Idle  --(oyuncu yakin)-->  Patrol
Patrol --(mesafe<200 ve gorus var)--> Chase
Chase  --(mesafe<40)-->    Attack
Attack --(cooldown bitti)--> Chase
Chase  --(can<=1)-->      Flee
Flee   --(mesafe>400)-->  Idle
```

Her geçiş tek yönlüdür. “Attack” durumundan “Patrol”a doğrudan geçilmez; mantıksal akışın tek kabul edilen yolu vardır ve bu da hata bulmayı kolaylaştırır.

51.1.3 En Basit FSM: Enum/String + if-elif

Bölüm 11’de yaptığımız gibi, durumu bir string’de tutup her karede `if-elif` ile dallanabiliriz:

Python – String Tabanlı FSM

```

1  class AkilliDusman(Dusman):
2      def __init__(self, x, y):
3          super().__init__(x, y, 28, 28)
4          self.durum = "patrol"
5          self.cooldown = 0
6
7      def guncelle(self, oyuncu, katilar):
8          mesafe = self._mesafe(oyuncu)
9
10         if self.durum == "patrol":
11             self._patrol_davranis(katilar)
12             if mesafe < 200 and self._gorus_var(oyuncu,
13                 katilar):
14                 self.durum = "chase"
15
16         elif self.durum == "chase":
17             self._chase_davranis(oyuncu)
18             if mesafe < 40:
19                 self.durum = "attack"
20             elif mesafe > 400:
21                 self.durum = "patrol"
22
23         elif self.durum == "attack":
24             if self.cooldown == 0:
25                 oyuncu.hasar_al()
26                 self.cooldown = 30
27             self.cooldown -= 1
28             if self.cooldown <= 0:
29                 self.durum = "chase"
30
31         self._fizik(katilar)

```

Bu yaklaşım küçük FSM’ler için yeterlidir. Ama beş-altı durum olduğunda `guncelle` metodu yüz satırı geçer ve okunması zorlaşır.

51.1.4 Sınıf Tabanlı FSM

Daha temiz bir tasarımda her durum kendi sınıfıdır. `State` soyut sınıfı `enter`, `update` ve `exit` arayüzünü tanımlar:

Python – Sınıf Tabanlı FSM

```

1  class State:
2      def __init__(self, dusman):
3          self.dusman = dusman
4
5      def enter(self): pass
6      def update(self, oyuncu, katilar): pass
7      def exit(self): pass
8
9
10 class PatrolState(State):
11     def enter(self):
12         self.dusman.vx = self.dusman.hiz
13
14     def update(self, oyuncu, katilar):
15         # Sınır + uçurum mantığı
16         if self.dusman._mesafe(oyuncu) < 200:
17             self.dusman.degistir(ChaseState(self.dusman))
18
19
20 class ChaseState(State):
21     def enter(self):
22         self.dusman.hiz = 3.0 # hızlan
23
24     def update(self, oyuncu, katilar):
25         yon = 1 if oyuncu.rect.x > self.dusman.rect.x else
26             ↪ -1
27         self.dusman.vx = yon * self.dusman.hiz
28         if self.dusman._mesafe(oyuncu) < 40:
29             self.dusman.degistir(AttackState(self.dusman))
30
31     def exit(self):
32         self.dusman.hiz = 1.5 # eski hıza dön

```

Dusman sınıfı yalnızca aktif durumu tutar ve ona iletir:

Python – Düşmanın Durum Yöneticisi

```

1  class AkilliDusman(Dusman):
2      def __init__(self, x, y):
3          super().__init__(x, y, 28, 28)
4          self.aktif = PatrolState(self)
5          self.aktif.enter()
6
7      def degistir(self, yeni_durum):
8          self.aktif.exit()
9          self.aktif = yeni_durum
10         self.aktif.enter()
11
12     def guncelle(self, oyuncu, katilar):
13         self.aktif.update(oyuncu, katilar)

```

14

`self._fizik(katilar)`

`enter` ve `exit` sayesinde duruma girerken (hızlanma, animasyon değişimi, ses çalma) ve çıkarken (cooldown sıfırlama) yapılacak işler tek yerde toplanır.

51.1.5 Görüş Hattı (Line of Sight) Yardımcı

FSM geçişlerinin kalbinde sıkça *görüş hattı* kontrolü vardır: “düşman ile oyuncu arasında engel var mı?”. Bunu tile bazlı haritada düşmandan oyuncuya doğru çizdiğin sanal bir çizgi üzerinde eşit aralıklarla noktalar üretip her birinin solid tile’da olup olmadığına bakarak ölçeriz.

Python – Basit Görüş Hattı

```

1 def _gorus_var(self, oyuncu, katilar):
2     x0, y0 = self.rect.center
3     x1, y1 = oyuncu.rect.center
4     adim = 20
5     for i in range(1, adim):
6         t = i / adim
7         x = x0 + (x1 - x0) * t
8         y = y0 + (y1 - y0) * t
9         nokta = pygame.Rect(int(x), int(y), 2, 2)
10        for k in katilar:
11            if nokta.colliderect(k):
12                return False
13    return True

```

💡 İpucu: FSM’i Kâğıda Çiz

Karmaşık bir FSM yazmadan önce kâğıda durumları yuvarlak içinde, geçişleri ok olarak çiz. “Bu durumdan hangi koşulla nereye gidebilirim?” sorunu her durum için tek tek cevapla. Yazılım son adım, tasarım ilk adım.

🕒 Bilgi: Pratik Alıştırma

`01_fsm_temel.py` dosyasını çalıştır. Bir düşman ekranda gezerken sen yaklaşırsan kovalamaya başlar; çok yakına gidersen saldırır; uzaklaşırsan tekrar gezintiye döner. Konsola hangi duruma geçtiğini yazdırır. Geçiş eşiklerini değiştir ve düşmanın “kişiliğini” kendin belirle.

51.2 A* Yol Bulma Algoritması

Bölüm 43’nde `ChaseDusman` oyuncuya doğru basitçe yatay yön belirliyordu. Bu, açık bir alanda işe yarar, ama harita karmaşılaşınca düşman duvarlara çarpıp takılır kalır. Düşmanın *plan yapması* gerekiyor: “A noktasından B noktasına nasıl

giderim?”. Bu sorunun adı *pathfinding* (yol bulma) ve standart cevabı A^* algoritmasıdır.

■ Tanım: Pathfinding

Bir gratta belirli bir başlangıç düğümünden hedef düğüme *en kısa maliyetli* yolu bulma problemi. Oyunlarda graf genellikle Bölüm ??’nde gördüğün tile haritadan türetilir; her tile bir düğüm, komşu tile’lar ise birer kenardır.

51.2.1 Grid Graph: Haritayı Grafa Çevirmek

Tile haritamızı doğrudan A^* ’a verebiliriz. Her hücre bir düğümdür; solid (engel) tile’lar düğüm değildir veya *geçilmez* kabul edilir. Komşuluklar iki şekilde tanımlanabilir:

- **4 yön** (Manhattan): Yukarı, aşağı, sol, sağ.
- **8 yön** (Chebyshev/Octile): Yukarıdaki dördüne ek olarak çapraz dört yön.

Platformer için 4-yön (yatay-düşey) genellikle yeterlidir; top-down oyunlarda 8-yön daha doğal hareket verir.

51.2.2 Heuristic Fonksiyonu

A^* , körü körüne tüm yolları taramaz; *tahmin* kullanır. “Bu hücreden hedefe ne kadar uzakta olabilirim?” tahminini yapan fonksiyona *heuristic* (kestirim) denir.

Python – Üç Heuristic

```
1 def manhattan(a, b):
2     return abs(a[0] - b[0]) + abs(a[1] - b[1])
3
4 def euclidean(a, b):
5     return ((a[0] - b[0])**2 + (a[1] - b[1])**2) ** 0.5
6
7 def chebyshev(a, b):
8     return max(abs(a[0] - b[0]), abs(a[1] - b[1]))
```

Hangisini ne zaman kullanırsın?

- **Manhattan:** 4-yön hareket için *kesin doğru* mesafedir.
- **Euclidean:** 8-yön ya da serbest hareket için kuş bakışı uzaklık.
- **Chebyshev:** 8-yön hareket için *kesin doğru* (çapraz adımı 1 sayar).

▲ Dikkat: Heuristic’i Şişirme

Heuristic değeri *gerçek* maliyetten büyükse A^* artık en kısa yolu garantilemez. Hatalı bir heuristic “çalışıyor gibi görünür” ama düşman bazen tuhaf yollar seçer. Manhattan’ı dikey-yatay hareketle, Chebyshev’i 8-yönle eşleştir.

51.2.3 Açık Liste, Kapalı Liste ve $f = g + h$

A* her düğüm için üç sayı tutar:

- g : Başlangıçtan bu düğüme buraya kadar gelen *gerçek* yol maliyeti
- h : Bu düğümünden hedefe *tahmini* (heuristic) maliyet
- $f = g + h$: Toplam tahmini yol maliyeti

■ Tanım: Açık ve Kapalı Liste

Açık liste (open set): Henüz incelenmemiş ama keşfedilmiş düğümler. Her zaman *en küçük f değerli* düğüm seçilir; bu yüzden öncelikli kuyruk (**heapq**) kullanılır. **Kapalı liste** (closed set): Zaten incelenmiş düğümler — tekrar açmayız.

51.2.4 A* Sözde Kodu

Algoritmanın özü kısa ve simetriktir:

```
acik = priority_queue() ; basla'yi (f=h) ile koy
geldi = {basla: None}    ; her dugumun atasi
g_skor = {basla: 0}
```

```
while acik bos degil:
    mevcut = en_kucuk_f_olani_cikar(acik)
    if mevcut == hedef: yol_kur(geldi, hedef)

    for komsu in komsular(mevcut):
        if engel(komsu): devam
        yeni_g = g_skor[mevcut] + 1
        if yeni_g < g_skor.get(komsu, sonsuz):
            geldi[komsu] = mevcut
            g_skor[komsu] = yeni_g
            f = yeni_g + heuristic(komsu, hedef)
            acik'a (f, komsu) ekle
```

Hedefe ulaşıncaya **geldi** sözlüğünü hedeften başa doğru takip ederek yolu inşa ederiz: *parent zinciri*.

51.2.5 Python Uygulaması

Python – A* (Manhattan, 4-yön)

```
1 import heapq
2
3 def a_star(harita, basla, hedef):
4     """harita: 2B liste, 0=bos, 1=engel
5     basla, hedef: (col, row) tuple
6     Donus: hucre listesi (basla'dan hedefe)."""
7     H = len(harita)
```

```

8   W = len(harita[0])
9
10  def gecerli(p):
11      c, r = p
12      return 0 <= c < W and 0 <= r < H and harita[r][c]
           ↪ == 0
13
14  def komsular(p):
15      c, r = p
16      for dc, dr in [(1,0),(-1,0),(0,1),(0,-1)]:
17          yeni = (c+dc, r+dr)
18          if gecerli(yeni):
19              yield yeni
20
21  def heuristic(a, b):
22      return abs(a[0]-b[0]) + abs(a[1]-b[1])
23
24  acik = []
25  heapq.heappush(acik, (0, basla))
26  geldi = {basla: None}
27  g_skor = {basla: 0}
28
29  while acik:
30      _, mevcut = heapq.heappop(acik)
31      if mevcut == hedef:
32          break
33      for komsu in komsular(mevcut):
34          yeni_g = g_skor[mevcut] + 1
35          if yeni_g < g_skor.get(komsu, float("inf")):
36              g_skor[komsu] = yeni_g
37              f = yeni_g + heuristic(komsu, hedef)
38              heapq.heappush(acik, (f, komsu))
39              geldi[komsu] = mevcut
40
41      # Yol kur
42      if hedef not in geldi:
43          return []
44      yol = []
45      p = hedef
46      while p is not None:
47          yol.append(p)
48          p = geldi[p]
49      yol.reverse()
50      return yol

```

51.2.6 Tile Engellerini Hesaba Katma

Yukarıdaki `harita[r][c] == 0` satırı kritiktir. Bölüm 38’da CSV ile yüklediğin haritada `1` solid, `0` boş tile demektir. Aynı kolaylık burada da bize yol veriyor: solid tile’a komşu olarak bile yazmıyoruz — A* kendiliğinden duvarları dolandır.

51.2.7 Görselleştirme

A*’ı debug ederken üç hücre türünü farklı renkte çizmek paha biçilmezdir:

- **Yeşil** — nihai yol hücreleri
- **Mavi** — açık liste (henüz incelenmedi)
- **Sarı** — kapalı liste (incelendi, atlandı)

Bu sayede heuristic’inin *neden* bazı koridorları seçtiğini gözünle görürsün.

Tile haritada A* görselleştirme: yeşil hücreler nihai yolu, sarı hücreler kapalı listeyi, mavi hücreler açık listeyi gösterir. Engel (gri) tile’lar yolun dolanmasına neden olur.

51.2.8 Performans: Her Karede Çalıştırma

A* ucuz değildir; 20×20 haritada milisaniye düzeyinde, 200×200 haritada onlarca milisaniye sürebilir. 60 demek 16,67 ms bütçen var demektir, yani her karede A* çağırırsan oyun durur.

💡 İpucu: Yolu Cache’le

A*’ı sadece şu durumlarda yeniden çalıştır:

- Düşman yolun sonuna geldi
- Oyuncu son hesaplamadan beri “yeterince” (örneğin 1 tile) hareket etti
- Yol üzerindeki bir tile değişti

Aksi halde mevcut yolu kullan. Bu tek başına FPS’yi 5 kattan fazla artırabilir.

⚠ Dikkat: Tile Koordinatı ↔ Piksel

A* sana tile koordinatları döner: (`col`, `row`). Düşmanı oraya yönlendirmek için piksele çevirmen gerekir: `px = col * TILE_BOY + TILE_BOY//2`. Bu çeviriyi yardımcı fonksiyona koyman okunabilirliği artırır.

🔵 Bilgi: Pratik Alıştırma

`02_astar.py` dosyasını çalıştır. Tile harita üzerinde fareyle tıkladığın yere düşman A* ile yol bulup yürüyor; sarı kareler kapalı listeyi, yeşil kareler son yolu gösteriyor. Heuristic fonksiyonunu `return 0` yap (Dijkstra’ya çevir): A*’ın *çok daha* fazla hücre incelediğini görürsün.

51.3 Steering Behaviors: Yumuşak Hareket

A* mükemmel bir yol planlar; ama bu yolu izleyen düşman tile kenarlarında robot gibi keskin dönüşler yaparsa oyun yapay durur. İşte burada *steering behaviors* (yönlendirme davranışları) devreye girer. Bu yaklaşım yapay zeka karakterlerini birer *vektör mekaniği* öznesi olarak modeller: konum, hız ve ivme vektörleri ile.

Yöntemin atası, 1999'da Craig Reynolds'un yayımladığı klasik "Steering Behaviors For Autonomous Characters" makalesidir. Reynolds o makalede tek bir denklemle düzinelerce davranışı açıklamıştır.

51.3.1 Temel Denklem

Her steering davranışının özünde aynı üç adım vardır:

1. **İstenen hız** (`desired_velocity`): Karakterin *gitmek istediği* yönü ve hızı veren vektör
2. **Yönlendirme** (`steering = desired - current`): Mevcut hızı istenen hıza dönüştürmek için gerekli düzeltme
3. **Uygulama**: `velocity += steering`, `velocity`'yi `max_speed` ile sınırla, sonra `position += velocity`

`steering`'i sınırlayarak (`max_force`) karakterin ne kadar keskin dönebileceğini ayarlarsın — ağır bir tank yavaş döner, çevik bir kılıç ustası anında döner.

51.3.2 pygame.math.Vector2 Kullanımı

PyGame yerleşik `Vector2` sınıfı işimizi kolaylaştırır:

Python – Vector2 Temel Kullanım

```

1  from pygame.math import Vector2
2
3  konum = Vector2(100, 200)
4  hiz = Vector2(0, 0)
5  hedef = Vector2(400, 300)
6
7  # Yön bulma
8  yon = (hedef - konum)           # vektör farkı
9  mesafe = yon.length()          # büyüklük
10 yon = yon.normalize()          # birim vektör
11
12 # Skalerle ölçekleme
13 istenen = yon * MAX_HIZ

```

⚠ Dikkat: Sıfır Vektörü Normalize Etme

Hedef ile konum eşitse (`hedef - konum`) sıfır vektörüdür ve `normalize` hata fırlatır. Önce `length_squared` ile kontrol etmek alışkanlık edinilmelidir.

51.3.3 Seek: Hedefe Doğru Git

En temel davranış: belirli bir noktaya doğru tam hızda gitmek.

Python – Seek

```

1 def seek(self, hedef):
2     desired = (hedef - self.konum).normalize() * MAX_HIZ
3     steering = desired - self.hiz
4     if steering.length() > MAX_FORCE:
5         steering.scale_to_length(MAX_FORCE)
6     self.hiz += steering
7     if self.hiz.length() > MAX_HIZ:
8         self.hiz.scale_to_length(MAX_HIZ)
9     self.konum += self.hiz

```

Karakterin hedefe yaklaştıkça *yavaşlamadığına* dikkat et: hep tam hızdadır ve hedefin etrafında salınır. Bu sorunu *Arrival* çözer.

51.3.4 Flee: Hedeften Kaç

Seek'in tam tersidir: *desired* ters yöne bakar.

Python – Flee

```

1 def flee(self, tehlike):
2     desired = (self.konum - tehlike).normalize() * MAX_HIZ
3     steering = desired - self.hiz
4     if steering.length() > MAX_FORCE:
5         steering.scale_to_length(MAX_FORCE)
6     self.hiz += steering

```

Düşmanın canı azaldığında flee davranışına geçmesi (FSM'de Flee durumu) klasik bir desendir.

51.3.5 Arrival: Yumuşak Yaklaşma

Arrival, Seek'in olgun versiyonudur: hedefe yaklaştıkça hızı kontrollü şekilde düşürür. *slowing_radius* (yavaşlama yarıçapı) içinde girince hızı orantılı olarak azaltırız.

Python – Arrival

```

1 SLOWING_RADIUS = 80 # piksel
2
3 def arrive(self, hedef):
4     yon = hedef - self.konum
5     uzaklik = yon.length()
6     if uzaklik == 0:
7         self.hiz *= 0
8         return
9
10    if uzaklik < SLOWING_RADIUS:
11        # Hedef yaklaştıkça yavaşla
12        hedef_hiz = MAX_HIZ * (uzaklik / SLOWING_RADIUS)

```

```

13     else:
14         hedef_hiz = MAX_HIZ
15
16         desired = yon.normalize() * hedef_hiz
17         steering = desired - self.hiz
18         if steering.length() > MAX_FORCE:
19             steering.scale_to_length(MAX_FORCE)
20         self.hiz += steering

```

Karakter artık hedefe *yumuşakça* iner, etrafında salınmaz.

51.3.6 Wander: Rastgele Dolaşma

Patrol gibi sabit bir gidip-gelme değil, daha doğal görünen rastgele gezinme. Reynolds'un klasik formülasyonu şöyledir:

1. Karakterin önünde belirli bir mesafede bir *çevre noktası* hayal et (`circle_center`)
2. Bu noktanın etrafında belirli yarıçaplı bir çember düşün
3. Çember üzerinde rastgele bir hedef nokta seç (`target_offset`)
4. Hedef noktanın açısını her karede ufak bir açı sapmasıyla güncelle (yumuşak değişim)
5. Bu hedefe doğru Seek uygula

Python – Wander

```

1  import math, random
2
3  class Wanderer:
4      def __init__(self, x, y):
5          self.konum = Vector2(x, y)
6          self.hiz = Vector2(1, 0)
7          self.wander_aci = 0 # radyan
8
9      def wander(self):
10         ileri = self.hiz.normalize() if self.hiz.length()
11         ↪ > 0 else Vector2(1, 0)
12         merkez = self.konum + ileri * 50 # önümüzdeki
13         ↪ nokta
14         self.wander_aci += random.uniform(-0.3, 0.3) #
15         ↪ ufak sapma
16         offset = Vector2(
17             math.cos(self.wander_aci) * 25,
18             math.sin(self.wander_aci) * 25
19         )
20         hedef = merkez + offset
21         self.seek(hedef)

```

`random.uniform(-0.3, 0.3)` sayesinde açı her karede çok az değişir; karakter rotasını ani değiştirmez ama yine de tahmin edilemez bir şekilde dolaşır.

51.3.7 A* ile Steering'in Farkı ve Birleşimi

Bu noktada iki teknik elimizde:

- **A*** *plan yapar*: “oradan oraya nasıl giderim?” sorusunu yanıtlar; uzun mesafe karar.
- **Steering** *anlık manevradır*: “şu an hangi yöne döneyim?” sorusunu yanıtlar; kısa mesafe icra.

İkisini birleştirmek hem akıllı hem doğal hareket sağlar:

1. A* ile oyuncuya tile listesi (waypoint dizisi) hesapla
2. Düşmanın bulunduğu konuma en yakın waypoint'i *anlık hedef* say
3. Bu hedefe *Arrival* davranışıyla yumuşakça yaklaş
4. Hedefe yaklaşınca bir sonraki waypoint'e geç
5. Yol biterse ya tekrar A* çağır ya da *Wander*'a düş

Python – Yol İzleyici (Path Follower)

```

1 class YolIzleyici:
2     def __init__(self):
3         self.yol = []          # [(px, py), ...]
4         self.indeks = 0
5
6     def yol_at(self, yol_pikseller):
7         self.yol = yol_pikseller
8         self.indeks = 0
9
10    def hedef_al(self):
11        if self.indeks >= len(self.yol):
12            return None
13        return Vector2(*self.yol[self.indeks])
14
15    def guncelle(self):
16        if self.indeks < len(self.yol):
17            hedef = self.hedef_al()
18            self.arrive(hedef)
19            if (hedef - self.konum).length() < 8:
20                self.indeks += 1

```

İpucu: Birden Fazla Davranışı Karıştırma

Bir karakter aynı anda birden fazla davranıştan etkilenebilir: “oyuncuyu *seek* et + duvarlardan *flee* et”. Bu durumda her davranışın **steering**'ini ayrı hesaplar, ağırlıklı toplam alır ve sonra **max_force** ile sınırlandırır. Buna *steering combination* denir.

⚠ Dikkat: Birim Karıştırmaması

A* tile koordinatlarıyla, steering pikselle çalışır. İki sistemi birleştirirken her zaman pikseli baz al; yol bulma sonucu mutlaka $(col * TILE, row * TILE)$ ile piksele çevrilmelidir.

● Bilgi: Pratik Alıştırma

`03_steering.py` dosyasını çalıştır. Üç noktayı fareyle sürükley: ekrandaki ajan onlardan birine *seek*, diğerinden *flee* yapar; üçüncüsünün etrafında *wander* eder. `MAX_FORCE` sabitini büyüt: ajan keskinleşir; küçült: tank gibi hantal döner.

51.4 AI Entegrasyon: Zıplayan Macera’da Akıllı Düşman

Üç teknik artık elimizde: FSM kararı verir, A* yolu çıkarır, steering yumuşak hareketi sağlar. Şimdi hepsini birleştirip Bölüm 43’ndeki düşmanı bir üst seviyeye taşıyoruz. Hedefimiz şu: *Zıplayan Macera*’ya, oyuncuyu gördüğünde A*’la peşine düşen, görüşten çıkınca arama yapan ve yumuşak hareket eden gelişmiş bir düşman ekleyeceğiz.

51.4.1 Hedef Davranış: FSM Tasarımı

Akıllı düşmanın FSM’si:

```
Patrol  --(gorus var ve mesafe<400)--> Detect
Detect  --(0.5 sn doldu)--> Chase
Detect  --(gorus kayboldu)--> Patrol
Chase   --(mesafe<40)--> Attack
Chase   --(gorus 2 sn yok)--> Search
Attack  --(cooldown bitti)--> Chase
Search  --(son konuma vardi ve gorus yok)--> Patrol
Search  --(gorus geri geldi)--> Chase
```

Bu altı durumlu makine düşmana “insan gibi” bir his verir. “Detect” durumu özellikle önemli: oyuncu ışıklı bir koridora çıktığında düşman *anında saldırmaz*, önce yarım saniye boyunca “baktım, gördüm, emin oldum” der. Bu detay oyunculara kaçma fırsatı verir ve adil hisseder.

51.4.2 Sınıf İskeleti

Bölüm 44’nde `Dusman` tabanını zaten kurmuştuk. Onu kalıtım yoluyla genişletiyoruz:

Python – AkilliDusman İskeleti

```

1 from pygame.math import Vector2
2
3 class AkilliDusman(Dusman):
4     def __init__(self, x, y, harita, tile_boy=32):
5         super().__init__(x, y, 28, 28)
6         self.image.fill((180, 40, 200))
7         self.harita = harita          # 2B int listesi
8         self.tile = tile_boy
9         self.konum = Vector2(x, y)
10        self.hiz = Vector2(0, 0)
11
12        # AI alt parçaları
13        self.aktif = PatrolState(self)
14        self.aktif.enter()
15        self.yol = []                  # A* sonucu (piksel)
16        self.yol_indeks = 0
17        self.son_oyuncu_tile = None   # Search için
18        self.detect_sayac = 0
19        self.attack_cooldown = 0

```

51.4.3 Görüş Hattı: Tile Mask ile

Konu 1'de basit görüş hattı yazmıştık. Tile bazlı haritada bunu daha verimli yapabiliriz: çizgi üzerindeki noktaların düştüğü tile'ları kontrol et, herhangi biri solid ise görüş yok.

Python – Tile Tabanlı Görüş

```

1 def gorus_var(self, oyuncu):
2     x0, y0 = self.rect.center
3     x1, y1 = oyuncu.rect.center
4     adim = max(abs(x1 - x0), abs(y1 - y0)) // self.tile +
5     ↪ 1
6     for i in range(1, adim + 1):
7         t = i / adim
8         x = x0 + (x1 - x0) * t
9         y = y0 + (y1 - y0) * t
10        col = int(x) // self.tile
11        row = int(y) // self.tile
12        if 0 <= row < len(self.harita) and 0 <= col <
13        ↪ len(self.harita[0]):
14            if self.harita[row][col] == 1:
15                return False
16
17    return True

```

Adım sayısını piksel mesafe yerine *tile mesafe*'ye bağlamak performansı belirgin biçimde iyileştirir.

51.4.4 Detect Durumu: Yarım Saniye Düşün

Detect durumu *aldatıcı bir oyuncu deneyimi* sağlar: düşman oyuncuyu tam olarak görmüş gibi yapmadan önce gözüyle takip ediyormuş hissi verir.

Python – DetectState

```

1 class DetectState(State):
2     def enter(self):
3         self.dusman.hiz = Vector2(0, 0)
4         self.dusman.detect_sayac = 30 # 60 FPS'de 0.5 sn
5
6     def update(self, oyuncu, katilar):
7         d = self.dusman
8         if not d.gorus_var(oyuncu):
9             d.degistir(PatrolState(d))
10            return
11        d.detect_sayac -= 1
12        if d.detect_sayac <= 0:
13            d.degistir(ChaseState(d))

```

51.4.5 Chase Durumu: A* + Steering

Chase, üç tekniğin birleştiği noktadır:

Python – ChaseState

```

1 class ChaseState(State):
2     def enter(self):
3         self.yol_yenile_sayac = 0
4
5     def update(self, oyuncu, katilar):
6         d = self.dusman
7
8         # Her 15 karede bir A* yenile
9         if self.yol_yenile_sayac <= 0:
10            d._yolu_yenile(oyuncu)
11            self.yol_yenile_sayac = 15
12            self.yol_yenile_sayac -= 1
13
14        # Yolu izle (Arrival ile yumuşak)
15        d._yolu_izle()
16
17        # Geçişler
18        mesafe = d.konum.distance_to(Vector2(*oyuncu.rect
19            ↪ .center))
20        if mesafe < 40:
21            d.degistir(AttackState(d))
22        elif not d.gorus_var(oyuncu):
23            # Son bilinen konumu kaydet
24            d.son_oyuncu_tile =
25            ↪ d._piksel_tile(oyuncu.rect.center)

```

24

```
d.degistir(SearchState(d))
```

`_yolu_yenile` A* çağırır ve sonucu piksel listesine çevirir; `_yolu_izle` Konu 3'teki `YolIzleyici`'nin `guncelle` metodunu çalıştırır.

51.4.6 A* Yenileme ve Steering Birleşimi

Python – Yol Yenileme

```

1 def _piksel_tile(self, pos):
2     return (pos[0] // self.tile, pos[1] // self.tile)
3
4 def _yolu_yenile(self, oyuncu):
5     basla = self._piksel_tile(self.rect.center)
6     hedef = self._piksel_tile(oyuncu.rect.center)
7     tile_yolu = a_star(self.harita, basla, hedef)
8     # Tile -> piksel (merkez)
9     self.yol = [
10         ((c + 0.5) * self.tile, (r + 0.5) * self.tile)
11         for c, r in tile_yolu
12     ]
13     self.yol_indeks = 1 # 0 zaten kendi konumum
14
15 def _yolu_izle(self):
16     if self.yol_indeks >= len(self.yol):
17         return
18     hedef = Vector2(*self.yol[self.yol_indeks])
19     yon = hedef - self.konum
20     uzaklik = yon.length()
21     if uzaklik < 6:
22         self.yol_indeks += 1
23         return
24     # Arrival davranışı
25     SLOW = 30
26     hedef_hiz = MAX_HIZ * (uzaklik / SLOW) if uzaklik <
27         ↳ SLOW else MAX_HIZ
28     desired = yon.normalize() * hedef_hiz
29     steering = desired - self.hiz
30     if steering.length() > MAX_FORCE:
31         steering.scale_to_length(MAX_FORCE)
32     self.hiz += steering
33     if self.hiz.length() > MAX_HIZ:
34         self.hiz.scale_to_length(MAX_HIZ)
35     self.konum += self.hiz
36     self.rect.center = (int(self.konum.x),
37         ↳ int(self.konum.y))

```

51.4.7 Search Durumu: Son Konuma Git

Görüş kaybolunca düşman “oyuncu nereye kaçtı?” diye son bilinen yerine A* ile gider. Orada da bulamazsa Patrol’a döner.

Python – SearchState

```

1 class SearchState(State):
2     def enter(self):
3         d = self.dusman
4         if d.son_oyuncu_tile is not None:
5             tile_yolu = a_star(d.harita,
6                               d._piksel_tile(d.rect.cent
7                               er),
8                               d.son_oyuncu_tile)
9             d.yol = [((c+0.5)*d.tile, (r+0.5)*d.tile)
10                     for c, r in tile_yolu]
11             d.yol_indeks = 1
12
13     def update(self, oyuncu, katilar):
14         d = self.dusman
15         d._yolu_izle()
16         if d.gorus_var(oyuncu):
17             d.degistir(ChaseState(d))
18         elif d.yol_indeks >= len(d.yol):
19             d.degistir(PatrolState(d))

```

51.4.8 Performans ve Karelerin Bütçesi

60 hedefiyle her karenin 16,67 ms bütçesi var. Akıllı düşman tek başına sıkıntı vermez, ama haritada 20 akıllı düşman varsa A* 20 kere çağrılırsa kare patlar. İki kural:

- **Yol yenileme aralığı:** Her düşman 15-30 karede bir A* çağırırsın, her karede değil.
- **Yenileme dağıtma:** Düşmanların yenileme zamanlarını kaydır; hepsi aynı karede A* çağırırsa lag yaşanır. `self.yol_yenile_sayac = random.randint(0, 30)` başlangıçta dağıtım yapar.

▲ Dikkat: Yapay Zeka Çok “Akıllı” Olmasın

Akıllı düşman oyunu zorlaştırır ama *adil* olmayabilir. A* ile seni saniyede 20 piksel hassasiyetle takip eden düşmanı yenmek mümkün değildir. `MAX_HIZ`’ı oyuncudan biraz *düşük* tut; oyuncu kaçabilsin diye bilinçli olarak “mantıklı bir kayıp” ekle.

İpucu: Test Görselleştirmesi

Geliştirme sırasında düşmanın hesapladığı yolu (sarı çizgi), aktif durumunu (üstüne yazı) ve görüş hattını (sebze rengi çizgi) ekrana çiz. Bu üç *debug overlay* olmadan AI'yı düzeltmek çok zorlaşır. Yayında tek satırla kapatabilen bir `DEBUG_AI = False` bayrağı bırak.

Bilgi: Pratik Alıştırma

`04_akilli_dusman.py` dosyasını çalıştır. Zıplayan Macera seviyesine bir akıllı düşman koyduk. Saklan: düşman seni göremez, patrol eder. Aniden çık: durur, 0,5 sn bakar, sonra A*'la peşine düşer. Bir köşeye saklan: gelir, son gördüğü yere kadar arar, sonra patrol'a döner. `DEBUG_AI = True` ile yolu ve durum etiketini ekranda gör.

51.5 Sonuç

Bu ünite de oyun yapay zekasının üç temel taşını öğrendin ve hepsini tek bir akıllı düşmanda birleştirdin. Bölüm 43'nin basit kovalama mantığından çok daha ileri bir noktadasın artık.

51.5.1 Üç Tekniğin Roller

Üç tekniğin birbirinin yerine değil, *yan yana* çalıştığını göstermek bu ünitenin asıl mesajıydı:

- **FSM:** “Şu an ne yapıyorum?” sorusunun cevabıdır. Karar mekanizmasıdır. Patrol mı, Chase mi, Attack mı, Search mi?
- **A*:** “Oraya nasıl giderim?” sorusunun cevabıdır. Plancıdır. Tile haritada engelleri dolaşan en kısa yolu çıkarır.
- **Steering:** “Şu an hangi yöne döneyim?” sorusunun cevabıdır. İcra mekanizmasıdır. Vektör mekaniğiyle yumuşak hareket sağlar.

51.5.2 Temel Çıkarımlar

1. Sonlu durum makinesi karmaşık `if-elif` bloklarını okunabilir, genişletilebilir bir desene dönüştürür. Sınıf tabanlı yaklaşım büyük FSM'ler için string tabanlıdan üstündür.
2. A^* , $f = g + h$ kuralıyla heuristic kullanarak Dijkstra'dan çok daha hızlıdır; tek koşul heuristic'in gerçek mesafeyi aşmamasıdır. Manhattan 4-yön, Chebyshev 8-yön ile uyumludur.
3. Steering behaviors vektör tabanlı yumuşak hareket sağlar; Seek, Flee, Arrival ve Wander dört temel davranıştır.
4. A^* uzun mesafe planı yapar, steering anlık manevradır; ikisini birleştirmek hem akıllı hem doğal hareket verir.
5. Akıllı düşman performansını korumak için A^* 'ı her karede değil, 15-30 karede bir veya oyuncu hareket ettiğinde çağır.

51.5.3 Bölüm İçi Bağlantılar

Bu ünite Bölüm 49'ın ölçme alışkanlıklarını ve Bölüm 50'nin broad-phase fikirlerini kullanır: 20 akıllı düşman içeren bir sahnede önce profille, darboğazları gör, sonra yol yenileme aralığını ayarla. AI ile optimizasyon birlikte düşünülmesi gereken iki konudur.

51.5.4 Yapay Zekanın Sınırları

Bu ünitedeki teknikler “klasik AI” olarak bilinir; gerçek oyunlarda da hâlâ %95 oranında bu yöntemler kullanılır. Daha gelişmiş yaklaşımlar (behavior trees, GOAP, utility AI, machine learning) vardır ama hepsinin temelinde yine durum, plan ve hareket bileşenleri vardır. Bu üçlüyü iyi anladıysan ileri konulara hızlı geçersin.

51.5.5 Bir Sonraki Ünite

Bir sonraki üniteye oyununu *paketlemeyi* öğreneceksin: PyInstaller ile bağımsız .exe ve .app dosyaları üretmek, varlık yollarını doğru çözmek ve oyunu itch.io üzerinden dağıtmak. Yıllık çalışman artık başkalarının bilgisayarında çalışacak.

● Bilgi: Hatırlatma

Akıllı düşmanın yan etkisi: oyunun zorluk eğrisi değişir. Bölüm 11'deki düşmanları doğrudan akıllı düşmanla değiştirmeden önce seviye tasarımını gözden geçir. “Adil zorluk”, “haksız zorluk”tan çok daha zevkli bir oyun deneyimi yaratır.

“Bir oyunu bitirmek başka şey, oyunu birinin oynayabileceği bir kutuya koymak başka şey. İkincisini yapmadıkça oyununu kimse oynamaz.”

— JONATHAN BLOW

Bu Ünitenin Özeti

Bu ünite de şu konuları öğreneceksin:

- Niye dağıtım şart: `python` kurulu olmayan bir bilgisayara oyununu nasıl götürürsün?
- `PyInstaller` ile `.py` dosyalarını tek bir çalıştırılabilir pakete dönüştürmek; `-onefile` ve `-windowed` bayrakları
- `sys._MEIPASS` ile paketlenmiş ortamda görsel ve ses dosyalarına ulaşmak; `resource_path` yardımcı fonksiyonu ve `.spec` dosyasını düzenlemek
- Oyununa simge eklemek, dosya boyutunu küçültmek ve dağıtım klasörünü ZIP olarak hazırlamak
- `itch.io` platformunda oyun sayfası açmak, ZIP yüklemek, `butler` CLI ile sürüm güncellemek; topluluk geri bildirimi

52.1 PyInstaller ile İlk Paketleme

Önceki üç ünite de oyunun *performansı*, *çarpışma mantığı* ve *yapay zekâsı*yla uğraştın. Artık oyun akıcı çalışıyor, düşmanlar akıllı, FPS düşmüyor. Şimdi başka bir soru: bu oyunu bir arkadaşına nasıl göndereceksin? `game.py` dosyasını e-postayla yollasan bile, arkadaşın `python`’u kuracak, `pip install pygame` diyecek, hata alırsa sürüm uyumsuzluğuyla uğraşacak. Çoğu oyuncunun zamanı yok; daha ko-mutu yazmadan vazgeçer.

Çözüm *dağıtım*: oyunun tüm bağımlılıklarını (Python yorumlayıcısı, `pygame`, görseller, sesler) tek bir çalıştırılabilir dosyaya gömmek. Hedef bilgisayarda kurulum gerekmez — çift tıkla ve oyna.

■ Tanım: Dağıtım (Distribution)

Dağıtım, geliştirici makinesinde çalışan kaynak kodun son kullanıcının çalıştırabileceği *kendi kendine yeten* bir pakete dönüştürülmesidir. Python kurulu olmayan bir bilgisayarda da çift tıklayıp oynanabilen ZIP veya kurulum dosyası bu yolla üretilir.

52.1.1 Niye .py Dosyası Yetmez?

Kaynak kodu doğrudan paylaşmanın üç problemi vardır:

1. **Yorumlayıcı yok:** Hedef makinede `python` kurulu olmayabilir.
2. **Bağımlılık çakışması:** Olsa bile sürüm farklı olabilir; `pygame 2.5` ile `python 3.8` her zaman iyi anlaşmaz.
3. **Kullanıcı deneyimi:** Terminal açıp komut yazmak, çoğu oyuncunun yapmak isteyeceği son şeydir.

Bir oyun mağazasında çift tıklanıp açılan ne varsa, arkasında *donmuş* (frozen) bir Python yorumlayıcısı vardır.

52.1.2 PyInstaller Nedir?

`PyInstaller`, Python projelerini tek bir çalıştırılabilir pakete dönüştüren yardımcı bir araçtır. Yaptığı iş özetle şudur:

- Senin `.py` dosyalarını analiz eder, hangi modülleri `import` ettiğini bulur.
- Python yorumlayıcısının kendisini ve gerekli `.dll`, `.so` dosyalarını paket içine kopyalar.
- Hedefe özel bir başlatıcı (`bootloader`) ekler.
- Tümünü bir `dist/` klasörüne veya tek bir `.exe/.app/ELF` dosyasına yazar.

Tablo 52.1: Python paketleme araçları

Araç	Tipik Kullanım
<code>PyInstaller</code>	En yaygın, çapraz platform, küçük orta projeler
<code>cx_Freeze</code>	Daha düşük seviyeli, manuel yapılandırma ister
<code>py2exe</code>	Sadece Windows; eski projelerde sıkça görülür
<code>Nuitka</code>	Python'u C'ye çevirip derler; küçük ve hızlı çıktı
<code>Briefcase</code>	Mobil ve masaüstü hedefli (BeeWare projesinden)

Bu kitap `PyInstaller` kullanır — `pygame` ile en iyi belgelenmiş ve en basit çözümdür.

52.1.3 Kurulum

PyInstaller'ı diğer paketler gibi proje ortamına ekle. Eğer `uv` kullanıyorsan:

Python – uv ile Kurulum

```
1 | uv add pyinstaller
```

Klasik `pip` kullanıyorsan:

Python – pip ile Kurulum

```
1 | pip install pyinstaller
```

Kurulduğunu doğrula:

Python – Sürüm Kontrolü

```
1 | pyinstaller --version
2 | # Tipik çıktı: 6.5.0
```

⚠ Dikkat: Sanal Ortamı Unutma

PyInstaller'ı oyunun *kendi sanal ortamına* kur. Aksi halde sistemdeki başka paketler de pakete sızar ve `.exe` gereksiz büyür. `uv` bunu otomatik yapar; `venv` kullanıyorsan paketlemeden önce ortamı aktive etmeyi unutma.

52.1.4 İlk Paketleme

Diyelim oyun dosyamızın adı `oyun.py`. En basit komut:

Python – İlk PyInstaller Komutu

```
1 | pyinstaller oyun.py
```

Komut çalıştığında `PyInstaller` üç şey üretir:

Tablo 52.2: PyInstaller'ın ürettikleri

Yol	İçerik
<code>build/</code>	Ara dosyalar, geçici çıktı; silebilirsin
<code>dist/oyun/</code>	Dağıtılabilir paket: <code>oyun.exe</code> + bağımlılıklar
<code>oyun.spec</code>	Yapılandırma dosyası; ileride elle düzenlenir

`dist/oyun/oyun` (Linux/Mac) veya `dist/oyun/oyun.exe` (Windows) dosyasına çift tıklayarak oyunu çalıştırabilirsin. Henüz görsel/ses dosyaları paket içine girmedi — onu Konu 2'de halledeceğiz.

52.1.5 Faydalı Bayraklar: `--onefile` ve `--windowed`

Varsayılan paket bir *klasördür*: yanında onlarca `.dll` ve `.pyd` dosyası vardır. Kullanıcıya tek bir dosya vermek istersen `--onefile` bayrağı kullan:

Python – Tek Dosya Paketleme

```
1 | pyinstaller --onefile oyun.py
```

Bu mod paketi sıkıştırır ve çalışırken geçici bir klasöre açar. Kolaylığı dağıtım açısından büyük; bedeli açılış süresinin biraz uzaması (yaklaşık 1–3 saniye).

İkinci önemli bayrak `--windowed`: konsol penceresi açmadan başlatır.

Python – Konsolsuz Pencereci Paketleme

```
1 | pyinstaller --onefile --windowed oyun.py
```

⚠ Dikkat: Oyunlar İçin `--windowed` Şart

`--windowed` olmadan paketlenen oyun başlatıldığında arkasında siyah bir terminal penceresi açar. Geliştirici modunda yararlı (`print` çıktıları görünür) ama son kullanıcıya bu çirkin görünür. Oyun dağıtımında daima `--windowed` kullan.

52.1.6 Hızlı Test

Paketin çalışıp çalışmadığını anlamak için `dist/` altına gir ve oyununu çalıştır:

Python – Linux/Mac'te Test

```
1 | ./dist/oyun/oyun
2 | # veya --onefile ile:
3 | ./dist/oyun
```

Python – Windows'ta Test

```
1 | dist\oyun\oyun.exe
2 | # veya --onefile ile:
3 | dist\oyun.exe
```

Eğer ekrana siyah bir pencere geliyor ama görseller görünmüyorsa, sorun varlık yönetimindedir — bir sonraki konuda `sys._MEIPASS` ile bunu çözeceğiz.

💡 İpucu: Önce Geliştirme Modunda Çalıştığını Garanti Et

Paketleme oyunu *daha kötü* çalıştırmaz, sadece çalışmasını zorlaştırır. `python oyun.py` ile temiz çalışmayan bir oyunu paketleyince problemler büyür. Önce kaynak kod sorunsuz koşsun, sonra pakete.

52.1.7 Toparlama

Bu konuda dağıtımın *niye* ve *nasıl*'ının ilk katmanını gördük:

- `.py` kaynaklı dağıtım son kullanıcı için pratik değildir
- `PyInstaller` en yaygın çözümdür; alternatifleri `cx_Freeze`, `Nuitka`
- `pyinstaller oyun.py` komutu `build/`, `dist/` ve `oyun.spec` üretir
- `-onefile` tek dosya, `-windowed` konsolsuz
- Oyun dağıtımında daima `-windowed` kullan

Sıradaki konuda paketin neden henüz görselleri bulamadığını ve `resource_path` fonksiyonu ile bunu nasıl düzelteceğimizi göreceğiz.

52.2 Paketlenmiş Ortamda Varlık Yönetimi

`PyInstaller` ile oyununu paketledin, çalıştırdın, ekran açıldı — ama oyuncu sprite'ı yerine kırmızı bir ünlem işareti çıktı, ya da `FileNotFoundError: images/oyuncu.png` hatasıyla program kapandı. Sorun klasik: kaynak kodda `pygame.image.load("images/oyuncu.png")` dediğinde, geliştirme modunda `images/` klasörü zaten yanında. Paketlenmiş ortamda ise dosyalar farklı bir geçici klasöre açılıyor; klasik göreceli yol artık geçerli değil.

Bu konuda problemi anlayacak, `sys._MEIPASS` özniteliğini, `resource_path` yardımcı fonksiyonunu ve `.spec` dosyasında `datas` alanını öğreneceğiz.

52.2.1 Problemin Kaynağı: `sys._MEIPASS`

`PyInstaller -onefile` modunda çalışırken paketi işletim sisteminin geçici klasörüne açar (Windows'ta `%TEMP%\_MEIxxxxx`, Linux'ta `/tmp/_MEIxxxxx`). Oyun bu klasörden çalışır ama *çalışma dizini* (`os.getcwd()`) yine de kullanıcının `.exe`'yi tıkladığı yerdir.

Donmuş bir paket içinde Python'un `sys` modülüne `_MEIPASS` adlı yeni bir öznitelik eklenir: bu, geçici açılma klasörünün yoludur.

■ Tanım: `sys._MEIPASS`

`PyInstaller` ile donmuş bir programda, paketin açıldığı geçici klasörün yolunu tutan `sys` modül özniteliği. Geliştirme modunda (kaynak koddan çalışırken) *tanımlı değildir* — bu fark, paketlenmiş ortamı ayırt etmek için kullanılır.

Yani basit bir kontrolle “Şu an paketlenmiş ortamda mıyım?” sorusunu cevaplayabilirsin:

Python – Donmuş Ortam Tespiti

```
1 import sys
2
3 if hasattr(sys, "_MEIPASS"):
4     print("Donmuş paket içinde çalışıyorum")
5 else:
```

```
6 | print("Geliştirme modunda çalışıyorum")
```

52.2.2 resource_path Yardımcı Fonksiyonu

İki ortamda da çalışan tek bir yol fonksiyonu yazalım. Hem geliştirme modunda dosyaları proje klasöründen, hem paketlenmiş modda `_MEIPASS` altından bulsun:

Python – resource_path Yardımcısı

```
1 import os
2 import sys
3 from pathlib import Path
4
5 def resource_path(rel_path: str) -> str:
6     """Hem geliştirme hem donmuş ortamda doğru yolu
7     ↪ döner."""
8     if hasattr(sys, "_MEIPASS"):
9         # PyInstaller donmuş paketinde
10        kok = Path(sys._MEIPASS)
11    else:
12        # Normal Python çalışırken: bu dosyanın bulunduğu
13        ↪ dizin
14        kok = Path(__file__).resolve().parent
15    return str(kok / rel_path)
```

Artık varlık yüklerken doğrudan göreceli yol vermek yerine bu fonksiyondan geç:

Python – Oyun Kodunda Kullanım

```
1 oyuncu_yolu = resource_path("images/oyuncu.png")
2 self.image =
3     ↪ pygame.image.load(oyuncu_yolu).convert_alpha()
4
5 muzik_yolu = resource_path("sounds/tema.ogg")
6 pygame.mixer.music.load(muzik_yolu)
```

💡 İpucu: Tek Yerden Yükle

`resource_path` kullanmaya en başta başla — oyun küçükken de. Sonradan yüzlerce `load` çağrısının başına eklemek sıkıcı bir refactor olur. Ünite 1 Konu 3'te gördüğümüz *önbellekleme* yardımcısının yanına yerleştir; tek varlık yükleyici fonksiyonun olsun.

52.2.3 Varlıkları Pakete Dahil Etmek: `-add-data`

`resource_path` doğru yolu *üretiyor* ama varlıkların gerçekten paket içine girmesi gerekir. Bunun için `-add-data` bayrağı kullanılır:

Python – Varlık Eklemenin Söz Dizimi

```

1  # Linux ve Mac (':' ayraç):
2  pyinstaller --onefile --windowed \
3      --add-data "images:images" \
4      --add-data "sounds:sounds" \
5      oyun.py
6
7  # Windows (';' ayraç):
8  pyinstaller --onefile --windowed ^
9      --add-data "images;images" ^
10     --add-data "sounds;sounds" ^
11     oyun.py

```

Söz dizimi **KAYNAK<ayrac>HEDEF**. Sol taraftaki yol senin makinendeki kaynak klasör, sağ taraftaki yol paket içindeki hedef. **"images:images"** der ki “proje kökünde **images/** klasörü var; pakete **images/** adıyla ekle”.

⚠ Dikkat: Platform Ayraç Tuzağı

Linux/Mac iki nokta üst üste (:), Windows noktalı virgül (;) kullanır. Yanlış ayraç kullanırsan **PyInstaller** sessizce dosyayı atlar; pakete giren bir şey olmaz. Hatadan değil eksiklikten anlarsın — klasörü gözle kontrol et.

52.2.4 .spec Dosyası ile Yapılandırma

Komut satırında **--add-data** yığını uzayınca okunamaz hâle gelir. Daha temiz yol: **PyInstaller**’ın ürettiği **oyun.spec** dosyasını düzenlemek. **.spec** aslında bir Python dosyasıdır.

Python – Tipik oyun.spec Dosyası

```

1  # oyun.spec
2  a = Analysis(
3      ['oyun.py'],
4      pathex=[],
5      binaries=[],
6      datas=[
7          ('images', 'images'),
8          ('sounds', 'sounds'),
9          ('fonts', 'fonts'),
10     ],
11     hiddenimports=['pygame.mixer'],
12     hookspath=[],
13     runtime_hooks=[],
14     excludes=[],
15     noarchive=False,
16 )
17
18 pyz = PYZ(a.pure)

```

```

19
20 exe = EXE(
21     pyz, a.scripts, a.binaries, a.datas, [],
22     name='oyun',
23     debug=False,
24     strip=False,
25     upx=True,
26     console=False,          # --windowed ile aynı
27     icon='oyun.ico',
28 )

```

`datas` listesi (kaynak, hedef) demetlerini alır. `hiddenimports` de önemli: `PyInstaller` bazen dinamik olarak `import` edilen modülleri keşfedemez ve pakete almaz. `pygame.mixer` klasik örnektir; başka `pygame` alt modüllerini de buraya eklemen gerekebilir.

`.spec` dosyası elindeyse paketleme komutu sadeleşir:

Python – .spec ile Yeniden Paketleme

```
1 | pyinstaller oyun.spec
```

`PyInstaller` artık `.py` yerine `.spec`'i girdi olarak kabul eder, içindeki yapılandırmayı kullanır.

İpucu: .spec Dosyasını Versiyon Kontrolüne Al

`oyun.spec` bir kez doğru yapılandırıldıktan sonra projenin parçasıdır. `git`'e ekle. `build/` ve `dist/` klasörlerini ise `.gitignore`'a koy — bunlar her paketlemede yeniden üretilir.

52.2.5 Test: Temiz Konuma Kopyala

Paketin gerçekten kendi kendine yetip yetmediğini en garanti şekilde test etmenin yolu, paketi *başka bir bilgisayara* ya da en azından temiz bir klasöre kopyalayıp çalıştırmaktır:

1. `dist/oyun/` klasörünü (veya `oyun.exe`'yi) masaüstüne ya da `/tmp/test/` altına kopyala
2. Oradan çalıştır
3. `images/` ve `sounds/` görüldüğünden emin ol; bir düşmanı vur, ses geliyor mu kontrol et

Eğer dosyalar görünmüyorsa, kontrol noktaları:

- `resource_path` fonksiyonu kullanılıyor mu?
- `.spec`'in `datas` listesi doğru mu?
- `--add-data` ayaracı platforma uygun mu?
- `oyun.exe`'yi başka bir klasöre kopyaladığında yanına `images/` klasörünü de götürdün mü? (Klasör modunda)

▲ Dikkat: **–onefile** Hatası: Çalışma Dizini

–onefile modunda `os.getcwd()` kullanıcının `.exe`'yi tıkladığı yerdir, paket içindeki geçici klasör değildir. Eğer bir kayıt dosyası yazıyorsan `getcwd` mantıklı; ama varlık *yüklerken* yine `_MEIPASS`'a git. `resource_path` bu ayrımı doğru yapar.

52.2.6 Toparlama

Bu konuda paketin görselleri ve sesleri bulması için gereken köprüyü kurduk:

- `sys._MEIPASS` donmuş ortamı tanır
- `resource_path` iki ortamda da çalışan tek arayüzdür
- `–add-data` kaynakları pakete kopyalar; araç platforma göre değişir
- `.spec` dosyasındaki `datas` ve `hiddenimports` listeleri bu yapılandırmayı kalıcı tutar
- Temiz bir konumda test etmek paketin gerçekten çalıştığının tek garantisidir

Sıradaki konuda paketi son kullanıcıya *satılabilir* hâle getireceğiz: simge, dosya boyutu, platform farkları, ZIP arşivi.

52.3 Dağıtıma Hazır Paket: Simge, Boyut ve Platform

Konu 2'nin sonunda paket çalışıyor: dosyalar yerinde, sesler geliyor. Ama paket hâlâ *geliştirici çıktısı* gibi görünüyor: `.exe`'nin simgesi soluk Python logosu, dosya boyutu 150 MB civarında, klasörde `README` yok. Bu konuda paketi son kullanıcıya gönderebilir hâle getireceğiz.

52.3.1 Simge Ekleme

Bir oyunun `.exe`'sinde generic Python simgesi görmek amatör izlenimi verir. `–icon` bayrağı veya `.spec` dosyasında `icon=` alanı bunu çözer.

Tablo 52.3: Platforma göre simge dosya formatları

Platform	Format	Tipik Boyutlar
Windows	<code>.ico</code>	16, 32, 48, 256 px (çoklu)
Mac	<code>.icns</code>	16 – 1024 px (çoklu)
Linux	<code>.png</code>	Genelde <code>.desktop</code> dosyasıyla

Python – Komut Satırından Simge

```

1 # Windows:
2 pyinstaller --onefile --windowed --icon=oyun.ico oyun.py
3
4 # Mac:
5 pyinstaller --onefile --windowed --icon=oyun.icns oyun.py

```

Eğer elinde sadece `.png` varsa, `Pillow` ile `.ico` üretebilirsin:

Python – PNG’den ICO Üretmek

```
1 from PIL import Image
2
3 img = Image.open("oyun.png")
4 img.save("oyun.ico", sizes=[(16,16), (32,32), (48,48),
   ↪ (256,256)])
```

💡 İpucu: Simge 256 × 256’dan Küçük Olmasın

Modern Windows masaüstü yüksek DPI’de `.exe` simgesini büyük gösterir. Sadece 32 px versiyon koyarsan büyütünce bulanık görünür. `.ico` dosyasında en az 256 px versiyon bulunsun.

52.3.2 Konsolu Gizlemek

Konu 1’de `-windowed` bayrağına değindik. `.spec` dosyasında bu, EXE bloğundaki `console=False` satırına karşılık gelir:

Python – Spec’te Konsolu Kapatmak

```
1 exe = EXE(
2     pyz, a.scripts, a.binaries, a.datas, [],
3     name='oyun',
4     console=False,          # konsol penceresi açma
5     icon='oyun.ico',
6     ...
7 )
```

⚠ Dikkat: Geliştirme Sırasında Konsol Açık Tut

`print` ve `traceback` çıktılarını ancak konsolda görürsün. Geliştirme sürecinde `console=True` bırak; sadece son sürümde `False` yap. İki ayrı `.spec` dosyası tutmak (`oyun-debug.spec`, `oyun-release.spec`) yaygın bir tekniktir.

52.3.3 Dosya Boyutu Sorunu

Tipik bir `pygame` oyununun paketlenmiş hâli 50 – 150 MB arasında çıkar. Çünkü paket içine:

- Python yorumlayıcısı (5 MB)
- `pygame` kütüphaneleri (SDL, ses kodekleri, görüntü kodekleri — 30 MB)
- Standart kütüphane (`numpy` bile gelmiş olabilir)
- Senin `images/`, `sounds/` klasörlerin

İlk paketleme sonrası `dist/` altındaki boyuta bakıp şaşırma — normaldir. Ama biraz küçültebiliriz.

52.3.4 Gereksiz Modülleri Çıkarmak: `--exclude-module`

`PyInstaller` bazen kullanılmayan büyük modülleri gereksiz yere pakete dahil eder. `tkinter`, `numpy`, `matplotlib` gibi modüller `pygame` oyununda nadiren gereklidir.

Python – Modül Hariç Tutma

```
1 pyinstaller --onefile --windowed \
2   --exclude-module tkinter \
3   --exclude-module numpy \
4   --exclude-module matplotlib \
5   --exclude-module unittest \
6   oyun.py
```

`.spec`'te `Analysis` bloğundaki `excludes` listesi de aynı işi görür:

Python – Spec'te Hariç Tutma

```
1 a = Analysis(
2     ['oyun.py'],
3     excludes=['tkinter', 'numpy', 'matplotlib',
4               ↪ 'unittest'],
5     ...
6 )
```

⚠ Dikkat: Bağımlılığı Kestiğine Emin Ol

`numpy`'yi hariç tutmadan önce gerçekten kullanmadığından emin ol. `pygame.surfarray` bazı durumlarda `numpy` ister; çıkarınca oyun çalışırken patlar. `Excludes` ile her çıkardığından sonra paketi test et.

52.3.5 UPX Sıkıştırma

`UPX` (Ultimate Packer for eXecutables) çalıştırılabilir dosyaları sıkıştıran ücretsiz bir araçtır. `PyInstaller` `--upx-dir` bayrağıyla otomatik kullanır.

Python – UPX ile Paketleme

```
1 # UPX'i sisteme kur, sonra:
2 pyinstaller --onefile --windowed --upx-dir=/usr/local/bin
3   ↪ oyun.py
```

`UPX` paketi tipik olarak %30 – %50 küçültür ama açılış süresine 0.5 – 1 saniye ekler. Küçük dağıtımlar için iyi takas.

💡 İpucu: Antivirüs Yanlış Pozitifleri

UPX ile sıkıştırılmış `.exe`'ler bazen Windows Defender'da *şüpheli* işaretlenir. Çünkü kötü amaçlı yazılımlar da UPX kullanır. Çoğu zaman zararsızdır ama sıkıştırma yapmak istemiyorsan `-noup` bayrağı vardır.

52.3.6 Platform Farklılıkları

`PyInstaller` çapraz derleme yapmaz. Yani Linux'ta çalıştırdığın `pyinstaller`, Windows için `.exe` üretmez — sadece o platformun çıktısını verir.

Tablo 52.4: Platforma göre çıktı türleri

Platform	Çıktı	Açıklama
Windows	<code>oyun.exe</code>	PE32+ taşınabilir çalıştırılabilir
Mac	<code>oyun.app</code>	Bundle dizin yapısı; tek dosya gibi açılır
Linux	<code>oyun</code>	ELF (Executable and Linkable Format)

Üç platform için de paket istiyorsan, üç ayrı makine (ya da sanal makine, ya da CI/CD) gerekir. [GitHub Actions](#) “matrix” iş akışları bu işi en kolay yapan çözümdür — `ubuntu-latest`, `windows-latest`, `macos-latest` runner'larında ayrı paketleme yaptırırın.

⚠️ Dikkat: Mac `.app` İmzalama

Mac paketleri Apple Developer hesabıyla imzalanmazsa modern macOS “geliştirici doğrulanamadı” diye açılışı reddeder. Ücretsiz dağıtım için kullanıcıya “Sağ tık → Aç” yönergesini vermek yaygın çözüm; profesyonel için yıllık 99 USD'lık geliştirici aboneliği gerekir.

52.3.7 Dağıtım Klasörü Düzeni

Paketi tek başına ZIP'lemek yerine, etrafına yardımcı dosyalar ekleyip “oyuncuya hazır” bir paket düzenle:

Python – Tipik Dağıtım Klasörü

```

1 ZiplayanMacera_v1.0_Windows/
2 | ZiplayanMacera.exe           (veya klasör modunda altklasör)
3 | README.txt                  # Kontroller, sistem
   ↳ gereksinimleri
4 | LISANS.txt                  # Telif ve lisans bilgisi
5 | KREDILER.txt                # Müzik, ses, font kaynakları
6 | DEGISIKLIKLER.txt           # Sürüm notları
```

`README.txt` oyuncuya ilk gösterilecek dosyadır:

Python – README.txt İskeleti

```

1  ZIPLAYAN MACERA v1.0
2  =====
3
4  OYNAMAK
5  -----
6  ZiplayanMacera.exe dosyasına çift tıkla.
7
8  KONTROLLER
9  -----
10 Sol/Sağ ok : Hareket
11 Boşluk      : Zıpla
12 ESC         : Pause / Menü
13
14 SİSTEM GEREKSİNİMLERİ
15 -----
16 - Windows 10 veya üstü
17 - 200 MB disk alanı
18 - 4 GB RAM

```

52.3.8 ZIP ve Hash

Son adım: klasörü tek bir arşive sıkıştırmak ve dağıtım sırasında bütünlüğü doğrulanabilsin diye hash hesaplamak.

Python – ZIP ve SHA256

```

1  # Linux/Mac:
2  zip -r ZiplayanMacera_v1.0_Windows.zip
   ↪ ZiplayanMacera_v1.0_Windows/
3  shasum -a 256 ZiplayanMacera_v1.0_Windows.zip
4
5  # Windows PowerShell:
6  Compress-Archive -Path ZiplayanMacera_v1.0_Windows ^
7   -DestinationPath ZiplayanMacera_v1.0_Windows.zip
8  Get-FileHash ZiplayanMacera_v1.0_Windows.zip -Algorithm
   ↪ SHA256

```

Hash'i indirme sayfasına yaz: kullanıcı indirdikten sonra aynı hash'i kendi hesaplayıp dosyanın bozulmadan ulaştığını doğrulayabilir.

💡 İpucu: Sürüm Numarası ve Platform Eki Şart

Dosya adına **v1.0** ve **Windows** (Mac, Linux) ekle. Üç ay sonra elinde **ZiplayanMacera.zip** adında üç dosya olduğunda, hangisinin hangi sürüm ve platform olduğunu unutmuş olursun. Adlandırma disiplini emek istemez, hata önler.

52.3.9 Toparlama

Bu konuda paketi *dağıtılabılır* yaptık:

- `-icon` ile özel simge
- `-windowed` / `console=False` ile konsolsuz başlatma
- `-exclude-module` ve UPX ile dosya boyutu küçültme
- Platform farkları: `.exe`, `.app`, ELF; çapraz derleme yok
- README, LİSANS, KREDİLER eklenmiş dağıtım klasörü düzeni
- ZIP arşivleme ve SHA256 hash

Sıradaki konuda paketi internete koyacağız: itch.io platformu, oyun sayfası, [butler](https://github.com/itchio/butler) CLI ve toplulukla buluşma.

52.4 itch.io ile Oyununu Yayınlamak

ZIP elinde, hash hesaplanmış. Şimdi oyununu nereye koyup oyuncularla buluşturacağız? Çoğu profesyonel oyun mağazasının yayınlama süreci karmaşık ve ücretlidir: Steam yayıncı hesabı 100 USD'dır, App Store ve Google Play yıllık ücretli, sıkı inceleme gerektirir. Bağımsız (indie) geliştiricilerin ilk oyunlarını koyduğu yer ise neredeyse tartışmasız aynı: itch.io.

Bu konuda itch.io hesabı açacak, oyun sayfası oluşturacak, ZIP yükleyecek, [butler](https://github.com/itchio/butler) CLI aracıyla güncelleme akışı kuracak ve oyununun topluluk geri bildiriminden nasıl yararlanacağını göreceksin.

52.4.1 itch.io Nedir?

itch.io bağımsız geliştiriciler için kurulmuş açık bir oyun pazarıdır. Ücretsiz hesap aç, oyununu yükle, parasıyla, “ne kadar verirsen” modeliyle veya tamamen ücretsiz dağıt. Site 2013’ten beri faaliyette ve indie sahnesinde *de facto* buluşma yeridir.

Tablo 52.5: itch.io’nun başlıca özellikleri

Özellik	Açıklama
Hesap	Ücretsiz; hem oyuncu hem geliştirici aynı hesap
Komisyon	Geliştirici belirler (0% – 30% arası tipik)
Format	Windows, Mac, Linux, Android, HTML5
Topluluk	Devlog, yorum, oyun jam’leri, anket
Ödeme	Stripe veya PayPal; doğrudan geliştiriciye
Analitik	İndirme sayısı, ülke, kaynak sayfası

itch.io’nun [Steam](https://steamcommunity.com)’den temel farkı: editöryel süreç *yok*. Yüklediğin oyun aynı dakikada yayında olur. Bu özgürlüğün karşılığı: keşfedilebilirlik daha düşük; oyunu duyurmak senin sorumluluğun.

52.4.2 Hesap ve Yeni Proje Açma

Adımlar:

1. itch.io adresine git, *Register* tıkla.
2. Kullanıcı adı (**username**), e-posta, parola gir. Kullanıcı adı oyunlarının URL'sine girer: kullaniciadi.itch.io/oyun-adi.
3. Profil sayfaında sağ üstten *Upload new project* seç.

Yeni proje formunda şu alanlar var:

- **Title:** Oyunun adı ([Zıplayan Macera](#)).
- **Project URL:** Otomatik üretilir ([ziplayan-macera](#)); değiştirebilirsin.
- **Short description:** Bir cümlelik özet ([Cesur bir karakterin uçan adalarda yeşil yatağı arayışı](#)).
- **Classification:** [Games](#).
- **Kind of project:** [Downloadable](#) (klasik) veya [HTML](#), [Android](#) vb.
- **Pricing:** [No payments](#) (ücretsiz), [Paid](#) (sabit fiyat), [No \\$ - or pay what you want](#) (oyuncu seçer).

52.4.3 Etiketler ve Tür

Sayfanın aşağısında [Tags](#) alanı var. Buradaki etiketler oyunun [itch.io](#) arama ve gezinti sayfalarında görünmesini sağlar. Anahtar etiketler:

- Tür: [platformer](#), [shooter](#), [puzzle](#), [rpg](#)
- Sanat: [pixel-art](#), [2d](#), [retro](#)
- Teknoloji: [pygame](#), [python](#)
- Tema: [adventure](#), [difficult](#), [family-friendly](#)

[itch.io](#) en fazla 10 etikete izin verir; en doğru olanları seç, alakasız [popular-tag](#) eklemeye çalışma (spam algılanır).

İpucu: [pygame](#) Etiketi Topluluk Çağırır

[pygame](#) etiketiyle yüklenen oyunlar Python ve PyGame topluluklarında daha çok dikkat çeker. Reddit [r/pygame](#), [Pygame Community](#) [Discord](#) ve [Python Discord](#) sayfalarında [itch.io](#) bağlantın paylaşılsa erken geri bildirim alabilirsin.

52.4.4 Kapak Görseli ve Önizlemeler

Görsel pazarlama [itch.io](#)'da neredeyse içeriğin yarısı.

Tablo 52.6: itch.io görsel boyutları

Görsel	Boyut (px)	Format
Cover image	630 × 500	PNG/JPG
Screenshot	En az 315 × 250	PNG/JPG (3 – 5 adet)
Animated preview	320 × 240	GIF (5 – 10 sn döngü)

- **Cover:** Oyun adı + ana karakter + arka plan. Listeleme sayfasında ilk göze çarpan budur.
- **Ekran görüntüleri:** Oyunun en güzel karelerini seç. *Menü ekranı koyma;* hareketli oyun sahneleri koy.
- **GIF:** Bir-iki saniyelik oynanış kesiti. [ffmpeg](#) ile videodan üretebilirsin:

Python – Video Kaydından GIF Üretmek

```
1 ffmpeg -i oynanıs.mp4 -vf "fps=12,scale=320:-1" -loop 0
   ↳ onizleme.gif
```

⚠ Dikkat: Cover Boyutuna Saygı Göster

[itch.io](#) 630×500 dışı boyutları kabul eder ama listeleme sayfasında otomatik kırpır. Sıradan kareler seçilirse oyun adı kapaktan çıkar, profesyonellik kaybolur. Önceden kırpılmış görsel yükle.

52.4.5 Açıklama Metni

Sayfa açıklaması (*description*) markdown destekler. İskelet:

Python – Açıklama Metni İskeleti

```
1 ## Hikaye
2
3 Cesur kahramanın uçan adalarda kaybolmuş yeşil yatağı
4 aramaya başlar. Düşmanları atlat, kayıp parçaları topla,
5 sonunda yatağına ulaş.
6
7 ## Özellikler
8
9 - 12 el yapımı seviye
10 - Yumuşak fizik tabanlı zıplama
11 - 3 farklı düşman türü
12 - Otomatik kayıt sistemi
13 - Tam ekran ve özelleştirilebilir tuşlar
14
15 ## Kontroller
16
17 | Tuş | Eylem |
18 | --- | ---- |
19 | Sol/Sağ ok | Hareket |
20 | Boşluk | Zıpla |
21 | ESC | Pause / Menü |
22
23 ## Krediler
24
25 Tasarım ve programlama: [Senin adın]
26 Müzik: [Sanatçı] (CC-BY 4.0 lisanslı)
27 Sprite'lar: [Sanatçı] (kendi yapımı veya CC0)
```

```

28
29 ## Sistem Gereksinimleri
30
31 Windows 10+, Mac 10.15+, Ubuntu 20.04+

```

💡 İpucu: Krediler Bölümünü Atlama

Müzik veya sprite başkasından alındıysa kredi vermek *lisans gereği* zorunludur. CC-BY “atfı yap” demek; yapmadığın takdirde teknik olarak telif ihlali ediyorsun. [KREDILER.txt](#) dosyasını da pakete ekle.

52.4.6 ZIP Yükleme ve “Executable” İşaretleme

Sayfanın *Uploads* bölümünden **Upload files** butonuyla ZIP’i yükle.

1. ZIP yüklendikten sonra yanına platform etiketi seç: *Windows, macOS, Linux*.
2. *This file will be played in the browser* işaretsiz kalsın (HTML5 değil).
3. Eğer ZIP içinde tek bir **.exe** varsa *itch.io app* (uygulamasını) bu dosyayı çift tıklayıp çalıştırabilir; bunun için yanındaki *Executable* onay kutusunu işaretle.

⚠ Dikkat: Üç Platform Üç ZIP

Tek bir ZIP’e Windows, Mac, Linux paketlerini hep birlikte koyma — oyuncu indireceği dosya kendi platformunkinden büyük olur. Üç ayrı ZIP yükle ve her birini ilgili platform ile etiketle.

52.4.7 butler CLI: Sürüm Güncellemeyi Otomatikleştir

butler, [itch.io](#)’nun resmi komut satırı aracıdır. Web arayüzünden ZIP yüklemek pratiktir ama sürüm güncelleme sıklığı arttıkça **butler** hayat kurtarır.

Kurulum:

Python – butler Kurulumu

```

1 # https://itch.io/docs/butler/installing.html adresinden
2 # platforma uygun ikiliği indir, PATH'e ekle.
3
4 butler --version
5 butler login          # tarayıcıda kimlik doğrula

```

İlk yayın ve sonraki güncellemeler:

Python – butler ile Yayın

```

1 # Format: butler push KAYNAK kullanıcı/oyun:kanal
2
3 butler push ZiplayanMacera_v1.0_Windows.zip ^

```

```

4      benimadim/ziplayan-macera:windows
5
6  butler push ZiplayanMacera_v1.0_Mac.zip      ^
7      benimadim/ziplayan-macera:mac
8
9  butler push ZiplayanMacera_v1.0_Linux.zip    ^
10     benimadim/ziplayan-macera:linux

```

butler yükleme sırasında *ikili fark* (binary diff) kullanır: sadece değişen baytları yükler. 100 MB'lık bir oyunda küçük bir değişiklik için saatlerce upload yerine saniyeler içinde tamamlanır.

Python – Sürüm Etiketi Ekleme

```

1  butler push ZiplayanMacera_v1.1_Windows.zip      ^
2      benimadim/ziplayan-macera:windows --userversion 1.1

```

💡 İpucu: CI/CD ile butler

GitHub Actions'da **butler push** komutunu son adım yaparsan, **git tag v1.1** her itildiğinde paket otomatik oluşur ve **itch.io**'ya gider. Manuel adım kalmaz, hata azalır.

52.4.8 HTML5 ve pygbag: Tarayıcıda Oyna

itch.io'da bir oyunun *tarayıcıda anında oynanır* olması keşfedilebilirliğini büyük ölçüde artırır. PyGame için bunun yolu **pygbag**: **pygame** kodunu WebAssembly'a derleyen bir araçtır.

Python – pygbag ile HTML5 Çıktısı

```

1  pip install pygbag
2  pygbag oyun.py          # build/web/ altında HTML çıktısı
   ↪ oluştur

```

build/web/ klasörünü ZIP'leyip **itch.io**'ya yükle, *This file will be played in the browser* kutusunu *işaretle* ve viewport boyutunu (800×600) ayarla. Oyuncu siteye girer girmez oynar; indirme yok.

⚠ Dikkat: pygbag Sınırları

pygbag her **pygame** özelliğini desteklemez: çoklu iş parçacığı, dosya yazma (oyuncunun diskine), bazı ses formatları sınırlıdır. Karmaşık bir oyunu tarayıcıya çıkarmak öncesinde **pygbag** kısıtlarını oku; basit projeler ve demolar için harika bir vitrin.

52.4.9 Topluluk Geri Bildirimi ve Devlogs

Sayfayı yayınladıktan sonra iş bitmedi; başlıyor. itch.io geliştiricilere üç pazarlama aracı sunar:

- **Devlog:** Sayfanın altına “geliştirme günlüğü” yazıları eklersin. Her devlog yeni özellikleri duyurur, takipçilere bildirim gönderir.
- **Yorumlar:** Oyuncular sayfa altından yorum yazabilir. Tutarlı, kibar cevaplar topluluk inşa eder.
- **Game Jams:** itch.io’nun en güçlü keşfedilebilirlik kanalı. [Ludum Dare](https://ludumdare.com), [GMTK Game Jam](https://gmj.org), [Pygame Community Jam](https://pygamecommunityjam.com) gibi etkinliklerde oyununu yarışmaya kat.

İpucu: İlk Sürümün Mükemmel Olmasını Bekleme

itch.io erken sürüm (early access) için ideal. Oyununu **v0.5** (alfa) etiketiyle yükle, geri bildirimle **v1.0**’a getir. Mükemmel olana kadar bekleyenler genellikle hiç yayınlamadan vazgeçer; erken yayınlayanlar bitirir.

52.4.10 Toparlama

Bu konuda paketin son kilometresini koştuk:

- itch.io indie pazarın açık kapısı; ücretsiz yayın
- Hesap aç, *Upload new project* ile sayfa kur
- 630 × 500 kapak, 315 × 250 ekran görüntüsü, GIF önizleme
- Markdown destekli açıklama metni: hikaye, özellikler, kontroller, krediler
- ZIP yükle ve *Executable* işaretle; her platforma ayrı ZIP
- [butler push](https://butler.itch.io) ile sürüm güncelleme; ikili fark sayesinde hızlı
- [pygbag](https://pygbag.itch.io) ile HTML5 çıktısı; tarayıcıda oynanır
- Devlog’lar, yorumlar, oyun jam’leri ile topluluk inşası

Sıradaki konuda hem Bölüm 13’ün hem de tüm kursun toplu özetine bakacağız ve final projesinin kapısını aralayacağız.

52.5 Sonuç

Bu ünite de oyununu **.py** kaynak dosyasından itch.io sayfasında çalıştırılabilir bir ürüne taşıdık. [PyInstaller](https://pyinstaller.com) ile çift tıklanabilir paket ürettik, sys._MEIPASS ile varlıkları doğru yerden yükledik, simge ve sıkıştırma ile paketi cilaladık, [butler](https://butler.itch.io) ile sürüm güncellemeyi otomatikleştirdik. Artık bir arkadaşına “oyna” deyip ZIP gönderdiğinde gerçekten *oynayabilecek*.

52.5.1 Anahtar Kavramlar

Tablo 52.7: Ünite 4 – Anahtar kavramlar

Kavram	Açıklama
Dağıtım	Kaynaktan kendine yeten paket üretme süreci
PyInstaller	Python projeleri için en yaygın paketleyici
–onefile	Tek dosya çıktısı; geçici klasöre açılır
–windowed	Konsol penceresi olmadan başlatma
sys._MEIPASS	Donmuş paketin geçici açılış klasörü
resource_path	Geliştirme + paket için tek yol arayüzü
.spec dosyası	Paketleme yapılandırmasının kalıcı hâli
datas / hiddenimports	Spec’te varlıklar ve gizli bağımlılıklar
UPX	Çalıştırılabilir dosya sıkıştırıcı
Çapraz derleme yokluğu	Her platform kendi makinesinde paketlenir
itch.io	Bağımsız oyunlar için açık dağıtım pazarı
butler CLI	İkili fark ile hızlı sürüm güncelleme aracı
pygbag	PyGame’i WebAssembly’a derleyen tarayıcı çık-tısı

52.5.2 Önemli Komutlar

Tablo 52.8: Ünite 4 – Temel komutlar

Komut	Açıklama
uv add pyinstaller	PyInstaller’ı projeye ekle
pyinstaller oyun.py	İlk paketleme
pyinstaller –onefile –windowed oyun.py	Tek dosya, konsolsuz
pyinstaller oyun.spec	Spec dosyasıyla paketleme
–add-data "images:images"	Linux/Mac varlık ekleme
–add-data "images;images"	Windows varlık ekleme
–icon=oyun.ico	Özel simge ata
–exclude-module tkinter	Gereksiz modülü çıkar
shasum -a 256 dosya.zip	SHA256 hash hesapla
butler login	itch.io kimlik doğrula
butler push DOSYA kullanıcı/oyun:windows	Sürüm yayınla
pygbag oyun.py	HTML5 / WASM çıktısı üret

52.5.3 Bölüm 13’ün Özeti: Profesyonelleşme

Bölüm 13 oyununu *öğrenci projesi* seviyesinden *tamamlanmış ürün* seviyesine taşıdı. Dört ünite boyunca şunları ekledik:

1. **Performans Optimizasyonu** (Ünite 1): **cProfile** ile darboğaz tespit, viewport culling, object pooling. Uzay Kaçışı’nın kare bütçesini 33 ms’den 14 ms’ye indirdik (Konu 49.5).

2. **Gelişmiş Çarpışma** (Ünite 2): Geniş/dar faz ayrımı, spatial hashing ile $O(n^2)$ 'den $O(n)$ 'ye düşüş, quad tree ile yoğun-seyrekle karışık sahneler.
3. **Temel AI Teknikleri** (Ünite 3): FSM, A* pathfinding, steering behaviors. Düşmanlar “zombi yürüyüş” yerine durum bilinciyle hareket ediyor.
4. **Oyun Dağıtımı** (Ünite 4): [PyInstaller](#), varlık yönetimi, simge ve boyut, [itch.io](#), [butler](#).

52.5.4 Genel Kurs Özeti: 1'den 13'e Yolculuk

Bu noktaya kadar gelmek 13 haftalık bir yolculuktu. Geriye dönüp bakalım:

Tablo 52.9: Bölümlerin ana kazanımları

Bölüm	Konu	Ana Kazanım
1	Python Temelleri	Değişken, koşul, döngü, fonksiyon
2	Veri Yapıları	Liste, sözlük, set, küme
3	PyGame'e Giriş	Pencere, ana döngü, olay sistemi
4	Şekil ve Resim	Surface, blit, çizim primitifleri
5	Sprite Mimarisi	Sprite , Group , çarpışma
6	Klavye ve Fare	Girdi yönetimi, tuş durumu
7	Uzay Kaçışı v1	İlk tam oyun: skor, can, ses
8	Animasyon ve Ses	Spritesheet, kare bazlı animasyon, mixer
9	Fizik ve Hareket	Yerçekimi, zıplama, sürtünme
10	Tile Tabanlı Dünya	Harita, kamera, scrolling
11	Zıplayan Macera v3	AI, parçacık, HUD, seviye akışı
12	Oyun Durumu	State machine, save/load, ayarlar
13	Optimizasyon ve Dağıtım	Profil, AI, çarpışma, paket

Hangi tekniği hangi oyun türünde kritik olduğuna dair pratik rehber:

- **Shoot-em-up / bullet hell:** Object pooling (Konu [49.5](#)), spatial hashing, FPS culling
- **Platform oyunu:** Tile harita (Bölüm 10), fizik (Bölüm 9), state machine (Bölüm 12)
- **RPG / macera:** Save/load (Bölüm 12), A* pathfinding, ayarlar
- **Puzzle:** Sprite mimarisi (Bölüm 5), girdi (Bölüm 6), state machine (Bölüm 12)
- **Tüm türler:** Profilleme, dağıtım, oyun döngüsü disiplini

52.5.5 Bir Sonraki Bölüm: Final Projesi

Bölüm 14 öğrenilenlerin uygulanacağı yer. Tek başına ya da küçük bir grupla *kendi tasarladığın* bir oyunu sıfırdan geliştirip yayınlayacaksın. Bölüm 14'ün iskeleti şudur:

- Oyun tasarım dokümanı (GDD): konsept, mekanikler, seviye planı
- En az üç farklı mekanik (zıplama + tile + skor değil — daha karakterli)

- Menü sistemi, ses, görsel efekt
- itch.io sayfasında yayın

💡 İpucu: Final Projesi Seçimi

“Çok büyük olsun” tuzağına düşme. Bu kursun kapsamında kazandığın araçlarla *bitirilebilir* bir oyun seç. Tek ekran bir bulmaca, 5 seviyeli bir runner, küçük bir tower defense — her biri Bölüm 14 için yeterli ölçek. *Bitmiş küçük oyun, bitmemiş büyük oyun*’dan her zaman daha değerlidir.

52.5.6 Sonraki Adımlar: Kursun Ötesinde

Bu kursla sahip olduğun temel sağlam ama dünya geniş. Eğer oyun geliştirmeye devam etmek istersen birkaç yön:

- **Godot Engine:** Açık kaynak, ücretsiz, GDScript Python’a benzer. Daha büyük 2D ve 3D projeler için doğal sıçrama.
- **Unity / Unreal:** AAA seviyeli endüstri standartları. Daha dik bir öğrenme eğrisi ama iş ilanlarında en çok aranan.
- **Networking ve çok oyunculu:** [socket](#) modülü, [asyncio](#), [websockets](#); client-server mimari, gecikme telafisi.
- **Shader’lar:** GLSL, post-processing, görsel efektler. [modernGL](#) ile [pygame](#)’a entegre edilebilir.
- **Oyun jam’leri:** [Ludum Dare](#), [GMTK Jam](#), [js13kGames](#). 48 saatte bir oyun bitirmek hızlı öğrenmenin en iyi yoludur.
- **Topluluk:** [Pygame Community Discord](#), [r/gamedev](#), yerel [Game Dev İstanbul](#) buluşmaları. Yalnız öğrenmek yerine birlikte öğrenmek daha hızlı.

🕒 Bilgi: Bölüm Sonu Pratiği

Bu hafta için iki görev: (1) Zıplayan Macera v4’ü [PyInstaller](#) ile paketle, kendi makinende ZIP olarak çalıştığını doğrula. (2) Kişisel bir [itch.io](#) hesabı aç ve oyununu *taslak* (draft) olarak yükle — *Restricted (only those given the link can view)* modunda kalsın, hocaya ya da arkadaşlara bağlantı gönderip ilk geri bildirimi al. Final projende bu adımı tekrar edeceksin; şimdi prova zamanı.

💡 İpucu: Bitirmenin Hissi

Bir oyunu sıfırdan paketleyip dünyaya açmak — bu noktaya kadar gelen herkes bunun ne kadar uzun bir yol olduğunu bilir. Tebrikler. Şimdi Bölüm 14’te kendi oyununu seçeceksin: o oyunun ilk satırlarını yazarken bu kursun ilk haftasında “acaba yaparım mı?” dediğin günü hatırla. Yapıyorsun.